Verification Continuum™

VC Formal User Guide

Version X-2025.06-SP1, September 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Feedback

Contents

1 About VC Formal 31

1.1 Introduction to Formal Verification 31

1.1.1 What is Formal verification? 32

1.1.2 Requirements for Formal verification 33

1.1.3 Formal Verification Results 33

1.1.4 Scalability - Design size, Complexity and Performance Considerations 34

1.2 About VC Formal 35

1.3 VC Formal Components 37

1.4 VC Formal Applications 38

1.4.1 Automatically Extracted Properties 38

1.4.2 AIP (Assertion IPs) 39

1.4.3 Connectivity Checking 39

1.4.4 Coverage Analysis 40

1.4.5 Data Path Validation 41

1.4.6 Formal Low Power Application 41

1.4.7 Formal Property Verification 41

1.4.8 Formal Register Verification 42

1.4.9 Functional Safety 42

1.4.10 Formal Security Verification 42

1.4.11 Formal Testbench Analyzer 43

1.4.12 Formal X-propagation 43

1.4.13 Navigator 43

1.4.14 Sequential Equivalence Checking 44

1.5 Using VC Formal for Convergence and SignOff 44

1.6 Interactive Verdi based Unified Debug 45

1.7 VC Formal Documentation 46

1.7.1 Viewing HTML Documentation From GUI 47

1.7.2 Smart Search in VC Formal 49

1.8 Next-gen Commands in VC Formal 51

2 Setting up VC Formal Design Environment 53

Feedback

2.1 Supported Design Languages and Properties 53
2.2 Setting The VC_STATIC_HOME Environment Variable 54
2.2.1 vcf Shell Command Line Options 54
2.2.2 Changing the VC Static Session Name and Location 60
2.2.3 Updating the vcf Setup File 60
2.3 Using Standalone VC Formal Installation 62
2.3.1 Setting The VC_FORMAL_HOME Environment Variable 63
2.4 Updating VC Static Platform and VC Formal Application Variable Settings 63
2.5 Setting the Application modes 64

3 Managing Constraints and Properties for Formal Analysis 66
3.1 About Constraints 66
3.2 Types of Constraints 68
3.2.1 Depth Based Constraints 69
3.2.2 Environment Global Constraints 70
3.2.3 Constant Constraints 70
3.3 Constraint Examples 71
3.3.1 Random but Constant Constraint 71
3.3.2 Simple Combinational Constraints 72
3.3.3 Legal Value Selector 72
3.3.4 Handshaking 72
3.3.5 Packet Generation 73
3.3.6 Accessing Signals Inside SV Interface 73
3.4 Stabilizing Constraints in Multi-Clock Designs 74
3.4.1 Reevaluating Results When Constraints are Modified 75
3.4.2 Interactive Behavior of set_change_at 76
3.4.3 Signal Timing 76
3.5 About Properties 77
3.5.1 Class Attribute 78
3.5.2 Type Attribute 79
3.5.3 name Attribute 84
3.5.4 usage Attribute 85
3.5.5 Status Attribute 85
3.5.6 Other Property Attributes 86
3.6 Property Configuration Commands 87
3.6.1 Script Property Expression Syntax 87

Feedback

3.7 Using SVA for Formal Analysis 89
3.7.1 Using SVAs from the OVL and VCS sva_cg Library 90
3.7.2 Using SVAs from the SVA Checker Library 91
3.8 Adding SVAs in the Design 96
3.8.1 Instantiating the SVA 96
3.8.2 Binding the SVA 96
3.8.3 Adding SVAs Inline 98
3.8.4 Adding SVAs Incrementally from the Shell 98
3.9 Supported SVA Syntax 99
3.9.1 Supporting Immediate/Deferred Asserts in Tasks and Functions 100
3.9.2 Supported Forms of User-defined Properties 100
3.9.3 Limitations of SVA Support 104
3.10 Updating Properties for Formal Analysis 106
3.10.1 Changing Property Usage 106
3.10.2 Extending Cover Properties to End-of-Test Conditions 108
3.11 Reviewing Property Usage 110
3.11.1 Special Considerations 113

4 Compiling Designs and Verifying Formal Setup 115
4.1 Working with VC Formal Quick Start Mode 115
4.1.1 Enabling VC Formal Quick Start Mode 116
4.2 Compiling Designs 118
4.2.1 Examples for Reading Design and Properties in Different Languages 119
4.2.2 Support for Multiple-Edge-Assignment and the Multiple-Process-Assignment 120
4.2.3 Preserving RTL Signal Connectivity 121
4.3 Compiling Designs Incrementally 122
4.3.1 Enabling Incremental Compile 123
4.3.2 Use Model: Incremental Compile Tcl Scipt 123
4.4 Changing Severity of VC Static Error Messages 124
4.4.1 Use Model 125
4.5 Suppressing Printing of Warning or Info Messages 126
4.6 Reviewing Properties in Design 127
4.7 Integrating DesignWare Components 128
4.8 Black Boxing Modules in a Design 128
4.8.1 Identifying Empty modules, Black boxes, Undefined Modules 130

Feedback

4.9 Setting the stop_on_synth_error Application Variable 131
4.10 Setting Up Profiling Information 131
4.11 Setting Up User-Defined Clocks 132
4.11.1 Specifying User-defined Clocks 132
4.11.2 Specifying Reference Clock 133
4.11.3 The Concept of System Clock 134
4.11.4 Specifying a Single Clock 134
4.11.5 Specifying Multiple Clocks 135
4.11.6 Specifying Virtual Clocks 135
4.11.7 Specifying Asynchronous Clocks 136
4.11.8 Getting a Report of User-defined Clocks 136
4.11.9 Checking for Potential Clocking Issues in Formal Model 137
4.11.10 Limitation 138
4.12 Setting Up User-defined Resets 138
4.12.1 Specifying Single Resets 139
4.12.2 Specifying Multiple Resets 140
4.12.3 Getting a Report of User-defined Resets 140
4.13 Specifying the Initial State 141
4.13.1 Specific Value Assignment 141
4.13.2 Specifying Reset State Using a Text File 148
4.13.3 Simulation Based Reset State 149
4.13.4 Reset State Extraction from FSDB file 153
4.13.5 Limitation 154
4.13.6 Loading Reset State from Property trace 155
4.14 Debugging the Reset State 156
4.14.1 Getting the Reset State Values 156
4.15 Verifying VC Formal Designs 157
4.15.1 Configuring Design Checks 158
4.15.2 Running Formal Design Checks 159
4.15.3 Debugging Formal Design Checks Results 160
4.15.4 Example to Run and Configure Combinational Loop Checks 163
4.15.5 Configuring Formal Design Checks in the GUI 164
4.15.6 Running Design Checks in GUI 165
4.15.7 VC Formal Design Checks 165
4.16 Support for Trace Checks 167
4.16.1 Configuring Trace Checks 168
4.16.2 Performing Trace Checks 169

4.16.3 Reporting Trace Checks 170

5 VC Formal Regression Mode Accelerator 171

5.1 About VC Formal RMA 171

5.2 VC Formal Applications Supporting RMA 173

5.3 Enabling RMA Through fvlearn_config 174

5.4 Viewing RMA Messages and Gain Summary 176

5.5 Viewing RMA Engines and Identifying Resolved Goals 177

5.6 Switching RMA On/Off in VC Formal Runs 179

5.7 RMA Usage Flows 179

5.8 Special Considerations 181

5.9 FAQs 182

6 Performing and Configuring VC Formal Checks 183

6.1 Performing VC Formal Checks 184

6.1.1 Stopping the Run When Constraints are Bad 187

6.2 Running on Compute Farm 190

6.2.1 Configuring Grids 190

6.2.2 Getting a Report of the Grid Configuration 194

6.2.3 Controlling Grids 194

6.2.4 Prefetching Workers in a Grid 194

6.2.5 Debugging the Grid Setup 195

6.2.6 Viewing Workers Status Grid Report in GUI 195

6.2.7 Debugging Grid Setup Using Scripts 196

6.2.8 Diagnosing the Run Directory For Grid Configuration Issues 197

6.3 Running VC Formal on Scale-Out Computing on AWS (SOCA) 198

6.3.1 Use Model 198

6.4 Synopsys Grid Reservation System 199

6.4.1 The snps_grs utility 199

6.4.2 Starting the Grid Reservation Server 200

6.4.3 Submitting Jobs 202

6.4.4 Shutting Down 202

6.5 Controlling check_fv Runs 202

6.5.1 Stopping/Resuming check_fv Command 203

6.5.2 Setting Limits for the check_fv Run 203

6.5.3 Managing Time Limits with `fml_max_check_time` 204
6.5.4 Ensuring Executing Set of Commands Post check_fv with -run_finish Option 205
6.5.5 Specifying Maximum Progress Time Limit 205
6.5.6 Restoring Session from Abnormal Termination of check_fv Run 206
6.5.7 Improving Performance of Subsequent check_fv runs 206
6.5.8 Resuming check_fv from the Last Known Maximum Bounded Depth 207
6.5.9 Using Proven Assertions for Converging on Subsequent Assertions 207
6.6 Viewing Progress of check_fv Runs 208
6.7 Viewing Details for VC Formal Verification Goals 208
6.7.1 The report_fml_engines Command 209
6.7.2 The report_fml_jobs Command 212
6.7.3 Reporting Peak Engine Memory 215
6.7.4 Bound Statistics for a Goal 216
6.8 Viewing the Task Progress Report 217
6.8.1 Viewing the Job Summary Report 217
6.8.2 Viewing the Engine Details 218
6.8.3 Viewing the Goals Depths Distribution 219
6.8.4 Viewing the Grid Workers Summary 220
6.9 Orchestration in VC Formal 221
6.9.1 Configuring Orchestration in VC Formal 221
6.9.2 Configuring Engines for VC Formal Orchestration 223
6.10 Viewing the Solve/Engine Time for Proven/Falsified Goals 225
6.10.1 Limitations 226
6.11 Viewing Bad Code 226
6.11.1 Use Model 227
6.12 Viewing ProofTree (Orchestration) Details 228
6.13 Viewing Bit Level Model Statistics in VC Formal 231
6.14 Viewing the Engine Summary in VC Formal GUI 232
6.15 Defining User Orchestration for VC Formal Runs 233
6.15.1 Defining User Orchestration in GUI 235
6.15.2 Getting a Report of the Orchestration 239
6.16 Applying User-defined Orchestration on Top of Another Effort Level 240
6.17 Supporting the Bug Hunting Feature 240
6.17.1 The bug_hunting_config Command 240
6.17.2 The report_fml_bug_hunting command 241
6.17.3 Providing User-Defined Property Order to Improve Bug Hunting 241

Feedback

6.18 Generating Cover Properties for Bug Hunting 243
6.18.1 The gen_covers_for_range command 243
6.18.2 The gen_covers_for_coi_counters command 244
6.18.3 The gen_covers_from_cov_targets command 245

7 Debugging Results of Formal Verification 247
7.1 Property Status Description 247
7.1.1 Property Status 248
7.1.2 Vacuity Goal Status 249
7.1.3 Witness Goal Status 251
7.1.3.1 Vacuity Definition for Uncoverable Witness 251
7.1.4 Conditional Proven Status 253
7.2 Viewing Results of check_fv Runs 254
7.3 Generating a report_fv Output in XML Format 258
7.4 Debugging VC Formal Results in VC Formal GUI 262
7.4.1 Using the VC Formal Toolbar 264
7.4.2 Managing and Analyzing Properties Using the GoalList 266
7.4.3 Customizing the View Settings in VC Formal 273
7.4.4 Viewing Witness Traces 282
7.4.5 Viewing Abstract Counters in VC Formal GUI 284
7.4.6 Debugging False Proofs 285
7.4.7 Checking Vacuity Goals 285
7.4.8 Preserving Multiple GUI Layouts 286
7.5 Support for Unified Debug in Formal 287
7.5.1 Opening the Unified Debug Window 288
7.5.2 Other Uses for Unified Debug Window 289
7.6 Using VC Formal Console in GUI 291
7.6.1 Searching with Strings 291
7.6.2 Locating a Specified Line Number 292
7.6.3 Executing Commands from the VC Formal Console 292
7.6.4 Opening Smart Log 293
7.7 Using OneSearch for Design Information 293
7.8 Debugging Conflicting Constraints 295
7.8.1 About Conflicting Constraints 295
7.8.2 Performing Conflicting Constraints Checks Up to a Specified Depth 297
7.8.3 Viewing the Reports of Conflicting Constraints and Dead-ends 298

7.8.4 Dead-ends due to Constraints 300
7.8.5 Debugging Dead-end Engines 302
7.8.6 Command Dedicated For Checking Constraints 302
7.9 Identifying Reduced Constraints for Proofs 302
7.9.1 The compute_reduced_constraints Command 303
7.9.2 The report_reduced_constraints Command 303
7.9.3 The get_reduced_constraints Command 304
7.10 Exporting FSDB for Property Traces 304
7.10.1 Saving FSDB Traces in VC Formal 305
7.10.2 Reading FSDB in Verdi 306
7.10.3 Limitations 306
7.11 Saving and Replaying Traces 306
7.12 Evaluating External FSDB 309
7.12.1 Use Model 310
7.13 Signal Groups for Debugging 310

8 Managing Formal Work Environment 313
8.1 Creating Verification Tasks 313
8.1.1 Advantages of Using Verification Tasks 314
8.1.2 Limitations of Verification Tasks 314
8.1.3 Adding Verification Tasks 315
8.1.4 Getting a list of Verification tasks 317
8.1.5 Getting Report of the Verification Tasks 317
8.1.6 Verification Task Use Model 318
8.1.7 Using Verification Tasks in the VC Formal GUI 318
8.1.8 Running Parallel check_fv Commands for Different Tasks 321
8.1.9 Controlling Task Specific Variables and Attributes 322
8.2 Saving and Restoring Sessions 324
8.2.1 Saving and Restoring Sessions in VC Formal GUI 325
8.2.2 Reports Saved and Restored in VC Formal GUI 328
8.3 Support for Synopsys Testcase Packager (STP) 329
8.4 VC Formal Setup Wizard 330
8.4.1 Opening VC Formal Setup Wizard 331
8.4.2 Defining Application and Formal Variables 333
8.4.3 Importing Design and Properties 335
8.4.4 Setting Up Clocks, Resets and Constants 338

Feedback

8.4.5 Setting Up Proofs . . . 341
8.4.6 Tcl Project Summary . . . 343

9 Exploring Design Using Navigator . . . 345
9.1 About Navigator . . . 345
9.2 Opening Navigator . . . 346
9.3 Opening Multiple Navigator Sessions . . . 348
9.4 Exploring Properties/Design Using Navigator . . . 348
9.4.1 Add/Edit/Delete Properties in Navigator . . . 348
9.4.2 Edit Waveform Mode . . . 353
9.4.3 Extend Cycles . . . 354
9.4.4 Plotting the Waveform . . . 355
9.4.5 Snapshot of a Waveform . . . 357
9.4.6 Reloading RTL in Navigator Sessions . . . 357
9.5 Saving/Exporting Navigator Properties . . . 358
9.6 Saving/Exporting Navigator Tcl . . . 359
9.7 Closing Navigator . . . 360

10 Automatically Extracted Properties Application . . . 362
10.1 About AEP Application . . . 362
10.1.1 Supported AEPs and Switches to Extract . . . 363
10.1.2 Support for VHDL AEPs . . . 370
10.2 Methodology for AEP Application . . . 371
10.2.1 Setting up and Running AEP . . . 371
10.2.2 Tcl Setup for AEP . . . 371
10.2.3 Enabling AEP Properties Present in the COI of FPV Goal . . . 373
10.2.4 Enabling or Disabling AEP Checks . . . 375
10.2.5 Enabling fsm_deadlock, fsm_livelock, fsm_unionlock Type of Checks . . . 375
10.2.6 Configuring AEP Instrumentation . . . 376
10.2.7 Generating FSM Fairness Constraints . . . 377
10.2.8 Recommendations for Using AEPs . . . 380
10.3 Standard Use Model for AEPs . . . 380
10.3.1 Running AEP in Tcl mode . . . 380
10.3.2 Run AEP in GUI mode . . . 381
10.3.3 Running AEP in Regression or Batch Mode . . . 382
10.3.4 Running AEP on compute farm . . . 382

Feedback

10.3.5 Setting up AEP for Multiple Tasks 382
10.3.6 Saving and Restoring 383
10.3.7 Viewing Progress of Runs 383
10.3.8 Reviewing Results and Report Generation 383
10.3.9 Debugging AEP Failures 383
10.3.10 Waiver Support for AEP 383
10.4 Performance and Convergence 384
10.5 AEP Specific Application Variables 384

11 Formal Coverage Analyzer Application 386
11.1 About Formal Coverage Analyzer Application 386
11.1.1 Supported Coverage Metrics 387
11.1.2 X Semantics Support 391
11.2 Unreachability Analysis to Accelerate Coverage Closure in Simulation (FCA UNR) 392
11.2.1 VC Formal FCA UNR Methodology 393
11.2.2 VC Formal FCA UNR Setup Tcl File 395
11.2.3 Running VC Formal FCA UNR 400
11.2.4 Saving the Exclusions Using Tcl 401
11.2.5 Saving the Coverage Database Using TCL 404
11.2.6 Review the Coverage from Verdi GUI 405
11.2.7 Saving and Restoring FCA UNR Sessions 406
11.2.8 Viewing Progress of FCA Run 406
11.2.9 Reviewing Results of VC Formal FCA Run 406
11.2.10 Viewing Coverage Report Using URG 408
11.2.11 Debugging VC Formal FCA Results 408
11.2.12 FCA UNR In Project Regression Phase: Ignore Accepted Exclusions 410
11.2.13 FCA UNR in Late Verification Phase: Validate Exclusions 411
11.3 FPV Signoff Coverage Features in FCA 412
11.4 Intentional Uncoverable Analysis 413
11.4.1 Intentional Uncoverable Analysis 413
11.4.2 Intention Based Analysis for Else/Default Branch Line of FSM/x-assign 415
11.4.3 Intentional Unreachability Analysis for Ternary Operators and Condition Coverage Metric 416
11.4.4 Saving Intentional Uncoverable Lines in an Exclusion File 418
11.4.5 Reporting and Filtering 418
11.4.6 Limitations of Intent Based Coverage Analysis 419
11.5 Coverage Goal Waiving Mechanisms 423

Feedback

11.5.1 Waive and Unwaive of Coverage Goals Using Tcl Commands 424
11.5.2 Waiving and Unwaiving of Coverage Goals Using VC Formal GUI 425
11.6 Performance and Convergence 428
11.6.1 Partition Runs by Hierarchy Trees Using -cm_hier option in VCS 428
11.6.2 Excluding Bind Hierarchies From Coverage 431
11.6.3 Blackboxing Modules Where Coverage Analysis Not Required 431
11.6.4 Enabling Automatic Abstraction 431
11.6.5 Controlling Trace Generation 432
11.6.6 Controlling Compute Resources 432
11.6.7 Running Selective Coverage Metrices 433
11.7 AutoScale Flow 434
11.7.1 Configuring Auto-Scale in VC Formal FCA 435
11.7.2 Configuring Auto Scale in VCS UNR 438
11.7.3 Example Tcl Outputs 439
11.7.4 Viewing Autoscale Results in GUI 440
11.8 Generating VCS UNR Setup from FCA 441
11.9 Current Limitations of FCA 442
11.10 Variables Related to FCA 442

12 Formal X-Propagation Application 444
12.1 About FXP Application 444
12.2 VC Formal FXP: Methodology 445
12.2.1 Inputs 446
12.3 Standard Use - Configuring FXP 446
12.3.1 The fxp_generate Command 447
12.3.2 The fxp_assume -injectx Command 451
12.3.3 The fxp_assume -nox Command 452
12.3.4 The fxp_assert Command 453
12.4 Performing FXP Analysis in VC Formal GUI 453
12.4.1 Reviewing Results and Report Generation 453
12.4.2 Debugging Failures 454
12.4.3 Debugging Failures with Temporal Flow View 455
12.4.4 Computing and Reporting Root Cause to Debug FXP Failure 456

13 Connectivity Checking Application 458
13.1 About Connectivity Checks 458

Feedback

13.2 VC Formal CC: Methodology 460
13.2.1 Specifying Connectivity Properties 460
13.2.2 Specifying CCs in CSV Format 461
13.2.3 The load_cc CSV Fomat 463
13.2.4 Specifying CCs in Table Format 467
13.2.5 The report_load_cc Command 468
13.2.6 Specifying CCs in Tcl Format 469
13.2.7 Adding Connections With Latency 473
13.2.8 Using Alias in the Specification 479
13.3 Specifying Initial State 480
13.3.1 Specifying Clocks and Resets 480
13.4 Specifying Constraints 480
13.5 Structural Checks 481
13.5.1 The fml_allow_reverse_cc_path Formal Variable 482
13.6 Automatic Black-boxing 483
13.6.1 Aggressive Automatic Black-boxing 483
13.6.2 The fml_cc_autobbox Application Variable 484
13.6.3 Snip Modeling in Connectivity Checks Application 485
13.6.4 Enable Dumping of Abstracted Model After autobbox 486
13.7 Enabling/Disabling Reset Verification on Sequential Elements 487
13.8 Standard Use Model 487
13.9 Proving Connectivity Properties 488
13.9.1 Viewing Connectivity Checking Results 488
13.10 Debugging Connectivity Checks 490
13.10.1 Determining the Root-cause for Failing Connectivity Checks 493
13.10.2 Interpreting list_path Results 497
13.10.3 Determining the Root-cause for a Subset of Checks 497
13.10.4 Getting a Reports of the Root-cause for Failing Connectivity Checks 498
13.10.5 Debugging Connectivity Checks in GUI 498
13.11 Extracting Connections 502
13.11.1 The generate_cc Command 502
13.11.2 Examples 504
13.11.3 Connectivity Extraction and Constraints 505
13.11.4 Controlling Enable Expression Signals 506
13.11.5 Limitations 506
13.12 Performing Coverage Analysis for Connectivity Checks 507

Feedback

13.12.1 Example . . . 507
13.12.2 Supported Connectivity Coverage . . . 509
13.12.3 Commands for Computing Connectivity Coverage . . . 510
13.12.4 Computing Connectivity Coverage in Tcl Flow . . . 512
13.12.5 Computing Connectivity Coverage in VC Formal GUI . . . 513

14 Sequential Equivalence Checking Application . . . 517
14.1 About VC SEQ . . . 517
14.1.1 Equivalence Checking . . . 518
14.1.2 Video Links . . . 518
14.2 Methodology . . . 519
14.2.1 VC Formal SEQ Flow . . . 519
14.2.2 Designs . . . 520
14.2.3 Same Design Compile . . . 521
14.3 Setting up the Equivalence Problem . . . 522
14.3.1 Mapping the Designs . . . 522
14.3.2 Creating SEQ Properties on the Fly . . . 538
14.3.3 SEQ Hierarchical Verification Flow . . . 543
14.3.4 Setting Up Clocks and Resets . . . 544
14.3.5 Setting Up Initial State . . . 545
14.3.6 Getting a Report of Mappings . . . 545
14.4 Running SEQ . . . 562
14.4.1 Viewing the Progress of Mappings . . . 564
14.5 Viewing SEQ Results in Tcl . . . 566
14.6 Running SEQ in a Grid Environment . . . 568
14.6.1 Specify two host files . . . 568
14.7 Resuming Previous Run . . . 568
14.8 Advanced Convergence Techniques . . . 569
14.8.1 Snip Points . . . 569
14.8.2 Black-boxing . . . 572
14.8.3 SEQ Abstractions . . . 573
14.8.4 Support for Creating Properties Around Abstracted Memory . . . 574
14.8.5 Increasing Bounded Depth in SEQ . . . 576
14.8.6 Decomposition Using Partial Word Equivalence . . . 577
14.8.7 Clock-Gating Verification Convergence . . . 578
14.9 Debugging SEQ Failures . . . 580

14.9.1 Running SEQ in VC Formal GUI 581
14.9.2 Debugging And Reporting SEQ Failures 586
14.9.3 Debugging SEQ Failures using Temporal Flow View 590
14.10 Viewing Advanced SEQ Reports 591
14.10.1 Proof Tree 591
14.10.2 Detailed Results for Each Proof 593
14.10.3 Viewing the Proof Tree in Verdi 597
14.10.4 Viewing the status of the Internal Equivalence Points in the Verdi Source Browser 600
14.10.5 Viewing the Designer View 601
14.11 Finding Root Cause of a Failure in SEQ 602
14.12 Performing Coverage Analysis for SEQ Mode 603
14.12.1 Opening Verdi Coverage GUI 604
14.12.2 Types of Coverage Metrics in the SEQ Mode 604
14.12.3 Viewing Coverage Metrics in the SEQ Mode 606
14.12.4 Viewing the Source for COV Properties 608
14.13 Performing Formal Core in SEQ Mode 609
14.14 VC SEQ Known Limitations 610
14.15 SEQ Clock Gating Covers 610

15 Supported AIPs 612
15.1 About AIPs 612
15.2 Methodology 613
15.2.1 Inputs to this AIP 613
15.2.2 What is look for and what to avoid 614
15.2.3 The aip_load Command 614
15.3 Supported Protocols and Verification Features 616
15.4 Performance and Convergence 618
15.5 Formal Variables Relevant for AIP 618

16 Formal Register Verification Application 619
16.1 About Formal Register Verification 619
16.1.1 Video Links 620
16.2 Methodology 620
16.3 Performing FRV on your Design 621

Feedback

16.3.1 FRV Example 622
16.4 Commands for Performing Register Checks 622
16.4.1 The frv_load Command 622
16.4.2 The frv_config Command 630
16.4.3 The frv_report Command 632
16.5 VC Formal FRV GUI and Debug 634
16.5.1 Running Register Checks in the Formal GUI 634
16.5.2 Review Results and Report Generation 635
16.5.3 Configuring Registers Tab 636
16.5.4 Using RMB Menu Options in the Registers Tab 637
16.5.5 Filtering the Registers Tab Based on the Register Attributes 640
16.5.6 Viewing the Properties of Register 640
16.5.7 Debug a Falsified Property of a Register Check 641
16.5.8 Searching for Registers, Assertions, Constraints in the GoalList Table 643
16.6 CSV Format for Register Specification 644
16.7 Extensions Supported in the IP-XACT File 648
16.7.1 Write and Read Mask Capability 648
16.7.2 HW Update Capability 650
16.7.3 Aliased Registers Capability 652
16.7.4 Latency Update Capability 655
16.7.5 Disable Checks Capability 656
16.7.6 alternateRegisters Capability 657
16.7.7 Variable Reset Capability 659
16.7.8 Array Support Capability 660
16.7.9 Individual Reset for Field Capability 660
16.7.10 Out of Address range Capability 662
16.7.11 Support for registerFile in IP-XACT 662
16.8 Extensions Supported in the RALF File 663
16.8.1 Write and Read Mask Capability 663
16.8.2 Hardware Update Capability 665
16.8.3 Latency Update Capability 665
16.8.4 Disable Checks Capability 666
16.8.5 Variable Reset Capability 667
16.8.6 Individual Reset for Field Capability 667
16.8.7 Support for registerFile in IP-XACT 668
16.8.8 Latency Setting for Read Action 669
16.8.9 Support for regfile in RALF 670

Feedback

16.9 Register/Field Access Types 671
16.9.1 RISC-V CSR Specific Access Types 673
16.10 IPXACT Attributes Support 674
16.10.1 DTD Support for IPXACT Attributes 674
16.11 Checkers parameters 675
16.11.1 Support for AIP FRV 677
16.12 Considerations While Specifying Write Data Transaction for FRV APB Bus Wrapper . . 679
16.13 Formal Interrupt Verification 681

17 Formal Property Verification Application 686
17.1 About FPV Application 687
17.1.1 Formal Property Verification 687
17.1.2 Video Links 687
17.2 Methodology 688
17.2.1 Inputs for FPV 688
17.2.2 Creating FPV Testbench 689
17.2.3 Executing FPV Testbench 690
17.2.4 Analyzing Results 693
17.2.5 Analyzing Assertion Results 696
17.2.6 Analyzing Cover Property Results 703
17.2.7 Analyzing Assumption Results 706
17.3 Property Control 709
17.3.1 Adding Script Property 709
17.3.2 Disabling and Enabling Property 709
17.3.3 Changing Property Attributes 709
17.4 Formal Runtime Control 709
17.4.1 Controlling Formal Run 709
17.4.2 Controlling Stop 709
17.4.3 Controlling Resume 710
17.4.4 Controlling Engine Effort 710
17.5 Proof Acceleration 711
17.5.1 Controlling Resources for a Run 711
17.5.2 Using Machine Learning 712
17.5.3 Using Proven Assertion as Assumption 712
17.5.4 Turning off Trace Generation 713
17.6 Convergence Improvement 713

Feedback

18 Considerations for Convergence Improvement 714
18.1 Convergence Challenges When Applying Formal Verification 714
18.1.1 Re-write Assertions 715
18.1.2 Tune Formal Engines 715
18.1.3 Abstraction 717
18.1.4 Utilize Invariants 719
18.2 Performing Complexity Analysis in Tcl 719
18.2.1 The report_fv_complexity Command 720
18.2.2 The get_fv_complexity Command 727
18.2.3 The compare_coi Command 731
18.2.4 The is_in_fanin Command 733
18.2.5 Limitations 734
18.3 Viewing Complexity Report (COI) in VC Formal GUI 734
18.3.1 Generating COI Report 735
18.3.2 Viewing Source, Waveform, and Reset waveform of an Element 740
18.3.3 Adding Snips, Setting Constants, Setting State Of an Element, and Setting Abstractions 740
18.3.4 Filtering the COI Report 742
18.3.5 Setting State By Inline Editing 742
18.3.6 Viewing Snip Icon in nSchema 743
18.4 Using Automatic Abstraction To Reduce Complexity 744
18.4.1 The set_abstractions Command 745
18.4.2 The report_abstractions Command 748
18.4.3 The get_abstractions Command 750
18.5 Reviewing the Formal Core of Property 750
18.5.1 Formal Core for Convergence 751
18.6 Support for Proof Assist 751
18.6.1 Use Model for Computing Proof Assist 752
18.6.2 Running Proof Assist for the First Time 753
18.6.3 Computing Proof Assist 753
18.6.4 Viewing Proof Assist Report 754
18.6.5 Proof Assist Notes 769
18.6.6 Computing and Viewing Proof Assist Reports in VC Formal GUI 771
18.7 Iterative Convergence Methodology 780
18.7.1 ICM Debug Flow 781
18.7.2 Configuring ICM View Settings 781

18.8 Using Proof Architect for Faster Convergence 787

19 Compositional Verification Using Proof Architect 788

19.1 Advantages of Proof Architect 790

19.2 Proof Architect Usage Scenarios 793

19.3 Types of Proof Strategies 794

19.3.1 Copy 794

19.3.2 Abstraction 795

19.3.3 Over-Constraint 796

19.3.4 Assume-Guarantee 797

19.3.5 Case-Split 800

19.4 Proof Architect: Use Model 802

19.4.1 Example Use Model for Compositional Verification Using Proof Architect 803

19.5 Enabling Proof Architect Verification 805

19.6 Tcl: Creating and Reporting Proof Strategies and Corresponding Tasks 805

19.6.1 Tcl Examples - Creating Copy Proof Strategy 807

19.6.2 Tcl Examples - Creating assume_guarantee Proof Strategy 808

19.6.3 Tcl Examples - Creating Abstraction Proof Strategy 809

19.6.4 Tcl Examples - Creating Over-Constraint Proof Strategy 810

19.6.5 Tcl Examples - Creating Case-Split Proof Strategy 810

19.6.6 Reporting Proof Strategies 810

19.7 GUI: Creating Proof Strategies and Corresponding Tasks 812

19.7.1 Creating Partitions for Properties 812

19.7.2 Creating Copy Proof Strategies 815

19.7.3 Creating Abstraction Proof Strategies 818

19.7.4 Creating Over-Constraint Proof Strategies 821

19.7.5 Creating Assume Guarantee Proof Strategies 823

19.7.6 Creating Case-Split Proof Strategies 825

19.8 Verifying Modeling Consistency and Soundness 827

19.9 Speculative Tasks Marking 827

19.9.1 Tcl: Enabling Speculative Task Marking 828

19.9.2 GUI: Enabling Speculative Task Marking 830

19.10 Auditing Proof Strategies and Tasks 831

19.11 Performing Proof Architect Verification 832

19.12 Enabling RMA With Proof Architect 834

19.13 Proof Architect: Known Limitations 835

20 Obtaining Formal Signoff For a Design 836
20.1 Introduction to Signoff 836
20.2 Methodology 836
20.2.1 Inputs 837
20.3 Standard Use 838
20.3.1 Signoff Setup 838
20.4 COI (Property Density) Analysis 839
20.4.1 Viewing the Property Density Report 840
20.4.2 Property Density Toolbar 848
20.4.3 Running Property Density (COI) in Command Mode 852
20.4.4 Running Deadcode Checks in COI Analysis 853
20.4.5 Running Bit-Level Checks in COI Analysis 854
20.4.6 Reviewing COI Results in Verdi Coverage 854
20.5 Over-constraint (OC) Analysis 855
20.5.1 Running Over-constraint Analysis in Command Mode 857
20.5.2 Reviewing OC Results in Verdi Coverage 858
20.6 Formal Core (FC) Analysis 860
20.6.1 Use Model for Computing Formal Core 861
20.6.2 Computing Formal Core in Tcl 862
20.6.3 Reporting Formal Core in Tcl 864
20.6.4 Getting Formal Core in Tcl 869
20.6.5 Generating Formal Core On-the-fly 870
20.6.6 Formal Core Known Issues and Limitations 871
20.6.7 Computing and Reporting Formal Core in GUI 872
20.6.8 Formal Core Report 876
20.6.9 Viewing Formal Core Coverage Report in Signoff Mode 884
20.6.10 Supported Coverage Metrics and Formal Core Coverage 885
20.7 Formal Testbench Analyzer (FTA) 888
20.8 Handling Waivers in Coverage Flow 889
20.8.1 Waiving Versus Excluding 889
20.8.2 Waiving Out-of-Scope Instances 891
20.8.3 Applying the Elfiles in Signoff Coverage 891
20.8.4 Apply Waivers in FTA 895
20.9 Using the VC Formal Signoff Dashboard 896

Feedback

20.9.1 Opening Signoff Dashboard . . . 897
20.9.2 Configuring Signoff Dashboard . . . 898
20.9.3 Reporting Signoff Results using Tcl Command . . . 900
20.9.4 Performing Low Effort Signoff . . . 901
20.9.5 Performing Medium Effort Signoff . . . 905
20.9.6 Performing High Effort Signoff . . . 907
20.9.7 Signoff Dashboard Taskbar . . . 909
20.9.8 Understanding OCFT Annotation . . . 913

21 Formal Testbench Analyzer App . . . 914
21.1 About Formal Testbench Analyzer . . . 914
21.1.1 Fault Injection . . . 915
21.1.2 Fault Classes . . . 916
21.1.3 Fault Qualification Analysis Process . . . 918
21.1.4 Fault Qualification Analysis with FTA . . . 919
21.1.5 Video Links . . . 920
21.2 Methodology . . . 920
21.2.1 Inputs to Setup and Run FTA . . . 922
21.3 Using GUI to Configure Properties and Fault Classes . . . 933
21.3.1 To perform FTA based on the statuses (proven or inconclusive): . . . 933
21.3.2 To perform clustering of FTA Faults: . . . 934
21.3.3 To perform FTA based on the fault types . . . 934
21.4 What to Look for and What to Avoid . . . 936
21.5 Testbench Quality Assurance Fault Coverage Database . . . 938
21.5.1 Leverage Simulation Testbench Quality Assurance Fault Database . . . 938
21.5.2 Saving FTA Results Into Testbench Quality Assurance Database . . . 939
21.5.3 View FTA Results in Testbench Quality Assurance Database . . . 939
21.5.4 Merge FTA Database with Testbench Quality Assurance Database . . . 940
21.6 Mapping FTA Results to Coverage Database . . . 940
21.7 Standard Use . . . 941
21.7.1 Running FTA using Tcl command . . . 941
21.7.2 Running FTA using GUI . . . 941
21.7.3 Running FTA in regression or batch mode . . . 942
21.7.4 Running FTA on compute farm . . . 942
21.7.5 Setting up FTA for Multiple Tasks . . . 942
21.7.6 Save and Restore . . . 942
21.7.7 Progress of run . . . 942

Feedback

21.7.8 Review Results and Report Generation 942
21.7.9 Debug in FTA GUI 947
21.8 Performance and Convergence 953
21.8.1 Fast Activation Analysis 953
21.8.2 Partition the Runs 953
21.8.3 Improve Fault Check Performance 953
21.8.4 Viewing Convergence Report 953
21.9 Advanced Techniques 955
21.9.1 Cluster Non-detected Faults to Reduce Debug Time 955
21.9.2 Automated Debugging of Non-detected Faults 955
21.9.3 Checking for Non-detected Fault Propagation 958
21.10 Use Cases 959
21.11 Application Variables and Formal Variables for FTA 959
21.12 Backward Compatibility 960

22 VC Formal Security Verification Application 961
22.1 Introduction FSV Application 961
22.1.1 Video Links 963
22.2 FSV Security Verification Methodology 963
22.2.1 Reading Design 964
22.2.2 Setting up the Verification Environment 965
22.2.3 Validating Setup 965
22.2.4 Create Security Properties 965
22.2.5 Proof Setup 965
22.2.6 Proving Properties 966
22.2.7 Performance Considerations 966
22.2.8 Debugging 966
22.2.9 Signoff Review 966
22.3 Specifying Security Properties 966
22.3.1 Excluding legal paths from consideration in Security Properties 967
22.3.2 Specifying Source and Destination Conditions 968
22.3.3 Source and Destination Condition Vacuity Checks 971
22.4 Property Specification Use Case Examples 972
22.4.1 Specifying Multiple Sources and Destinations 972
22.4.2 Creating One Assertion for Each Source-Destination Pair 972
22.4.3 Viewing and Reporting Security Properties 973

Feedback

22.5 Black Boxing and Security Verification 977
22.5.1 The fsv_blackbox Command 979
22.5.2 Customizing Secure Blackboxes 981
22.5.3 Reporting Secure Blackboxes 981
22.6 Abstractions 983
22.7 Proving Security Properties 984
22.7.1 Configuring the Customized View Settings for FSV 985
22.8 Grid Usage 986
22.9 Viewing the Status of Security Properties 987
22.9.1 Relation Between Security and Property Tables 988
22.9.2 Status of Security Properties 989
22.9.3 Filtering and Viewing the Columns in Security Table 990
22.10 Debugging the Failing Security Properties 991
22.10.1 Enabling the Auto Invoke TFV for FSV 993
22.10.2 Finding Root Cause Signal for Failing Assertions 995
22.11 Property Convergence 998
22.11.1 Are Formal Jobs Running as Expected? 998
22.11.2 Optimizing Security Properties for Performance 1000
22.11.3 Are Proofs Making Progress 1000
22.11.4 Reducing Complexity 1002
22.12 Signoff Review 1004
22.13 Advanced Techniques 1005
22.13.1 Security Assert and Assume Properties 1005
22.13.2 Grouping Properties Using Zone Name 1007
22.14 Application Variables Specific to FSV 1008

23 Functional Safety Application 1009
23.1 Introduction 1010
23.1.1 Functional Safety Analysis 1011
23.2 Running FuSa with VC Formal 1014
23.2.1 Methodology 1015
23.3 Running FuSA Tcl command mode 1027
23.4 Performing FuSa in VC Formal GUI 1029
23.4.1 Viewing Observation Points and Performing Complexity Analysis 1035
23.4.2 Performing RMB Menu Operations 1036

23.5 Running FuSA Analysis with VC FCM 1037

24 VC Formal Low Power Application 1040

24.1 About VC Formal Low Power Application 1040

24.1.1 Supported Methodologies in VC Formal FLP 1041

24.2 VC Formal Low Power Connectivity Checking Application 1041

24.2.1 FLP CC Methodology 1042

24.2.2 FLP CC Setup Example 1044

24.2.3 Specifying Connectivity Properties 1047

24.2.4 Enabling Vacuity and Witness in FLP CC 1047

24.2.5 Generating Constraints Using Power State Table (PST) 1049

24.2.6 Supported LPA_TYPES 1050

24.2.7 Printing Combinational Logic for LPA_BASIC Property Path 1056

24.2.8 UPF/IEEE 1801 Modeling: Isolation/Retention/Power Supply Network 1056

24.2.9 Debugging Failures in FLP CC 1059

25 Data Path Validation Application 1066

25.1 About DPV Application Mode 1066

25.1.1 Video Links 1067

25.2 Writing a DPV Wrapper for C/C++ Model 1067

25.3 Compiling a design 1068

25.3.1 Analyzing C++ Files 1068

25.3.2 Analyzing Verilog and VHDL files 1068

25.3.3 Compiling C/C++ Specification Design 1069

25.3.4 Compiling RTL Implementation Design 1069

25.3.5 Mapping the Designs 1069

25.3.6 Running the proofs 1070

25.3.7 Starting DPV App in VC Formal 1070

25.4 Performing DPV Analysis in VC Formal GUI 1071

25.5 Status of Lemmas in VC Formal DPV 1077

25.6 Debugging Failures 1078

25.6.1 Correcting Errors 1079

26 Appendix A - Design Query Commands 1081

26.1 Terminology Used in the VC Static Tools 1081

Feedback

26.1.1 Designs 1081
26.1.2 Design Objects 1082
26.2 Design Query Commands 1083
26.2.1 Regular Expression Behavior 1084
26.2.2 Filter Expression Behavior 1085
26.2.3 Object Name Filtering 1085
26.2.4 Getting Default or Current Values for Every Argument of the Command 1085
26.3 Using Collections 1087
26.3.1 Creating Collections 1087
26.3.2 Saving Collections 1089
26.3.3 Querying Objects in a Collection 1090
26.3.4 Using Wildcard Characters 1091
26.3.5 Filtering Collections 1092
26.3.6 Iterating Over the Elements of a Collection 1095
26.3.7 Removing From a Collection and Adding to a Collection 1096
26.3.8 Sorting Collections 1097
26.3.9 Using Collection Utility Commands 1098
26.3.10 Comparing Collections 1098
26.3.11 Copying Collections 1099
26.3.12 Indexing Collections 1099
26.4 Using Attributes 1100
26.4.1 Attribute-Related Commands 1100
26.4.2 Predefined Application Attributes 1101
26.4.3 The object_class Attribute 1101
26.4.4 Cell Attributes 1101
26.4.5 Pin Attributes 1103
26.4.6 lib_pin Attributes 1104
26.4.7 Port Attributes 1105
26.4.8 Net Attributes 1106
26.4.9 Design Attributes 1106
26.4.10 lib_cell Attributes 1107

27 Appendix B - SVA Coding Guidelines 1109

28 Appendix C - Formal Examples and Use Models 1115
28.1 Running VC Formal Examples 1115
28.2 One-design Flow in VC SEQ 1115

Feedback

29 Appendix D - Support for Formal Scoreboard and RAM/FIFO Verification Library 1118

30 Appendix E - Application Variables 1119

31 Appendix F - Formal Variables 1127

32 Appendix G - Engine Controls for User Controlled Orchestration 1141

32.1 Engine Labeling 1142
32.2 Engine b1/b4/b6/b7/b10/b11/b13/b14/b15/b16/b17 1143
32.3 Engine b3 1144
32.4 Engines b5 1144
32.5 Engines b8/b9 1145
32.6 Engine b20 1146
32.7 Engine e1/e18/e23 1146
32.8 Engine e2 1149
32.9 Engine e3 1149
32.10 Engine e4 1149
32.11 Engine e7 1150
32.12 Engine e19 1150
32.13 Engines e8/e9/e15/e16/e24 1150
32.14 Engines e13/e14 1152
32.15 Engine e17 1153
32.16 Engine e25 1153
32.17 Engine s1 1153
32.18 Engine s7/s12 1154
32.19 Engine e12 1157
32.20 Engine l1/l11/l16/l13/l14/l18 1158
32.21 Engine l4 1158
32.22 Engine l5 1159
32.23 Engine l6 1160
32.24 Engine l8 1161
32.25 Engine l9 1162

Feedback

32.26 Engine I10/I12/I17 1163
32.27 Engine I22 1164
32.28 Engine Usage 1165

33 Verdi Debugging Tips for VC Formal 1166
33.1 Automatic Tracing of Value With Verdi's Temporal Flow View 1166
33.2 Automatic Tracing of X's With Verdi's Temporal Flow View 1167
33.3 Active Tracing of RTL 1167
33.4 Function Debugging 1167
33.5 Macro Debugging 1167
33.6 Count the Transitions of a Clock or Register 1168
33.7 Modify Existing Logical Operation Signals 1168
33.8 Collapse Source Code 1168

1

About VC Formal

The section contains that following topics:

- Introduction to Formal Verification
- About VC Formal
- VC Formal Components
- VC Formal Applications
- Using VC Formal for Convergence and SignOff
- Interactive Verdi based Unified Debug
- VC Formal Documentation
- Next-gen Commands in VC Formal

1.1 Introduction to Formal Verification

As design size and complexity continues to grow, verification approach needs to evolve over time. IP reuse is an integral part of design methodology today; it helps to shorten the design cycle, but it also brings new verification challenges. Verification methodology needs to adopt to these changes to ensure design is bug free and time to market goals are predictably met. Simulation based verification has been very successful for last few decades, and continues to be effective at both block and SoC level, but it does have observability and controllability challenges. Simulation is very efficient for verifying deep sequential behavior, but it is not exhaustive. For mission critical blocks and some very specific applications, exhaustive analysis is must. Property based formal analysis provides both observability and controllability, and it also provides exhaustive analysis to ensure signoff quality verification. Selecting the right block for formal verification is very important, it works best on control path intensive blocks where many concurrent events must be verified. Formal can also be used for specific data path verification problems wherein math heavy logic is written in RTL, and it needs to be functionally compared against reference C/C++ model.

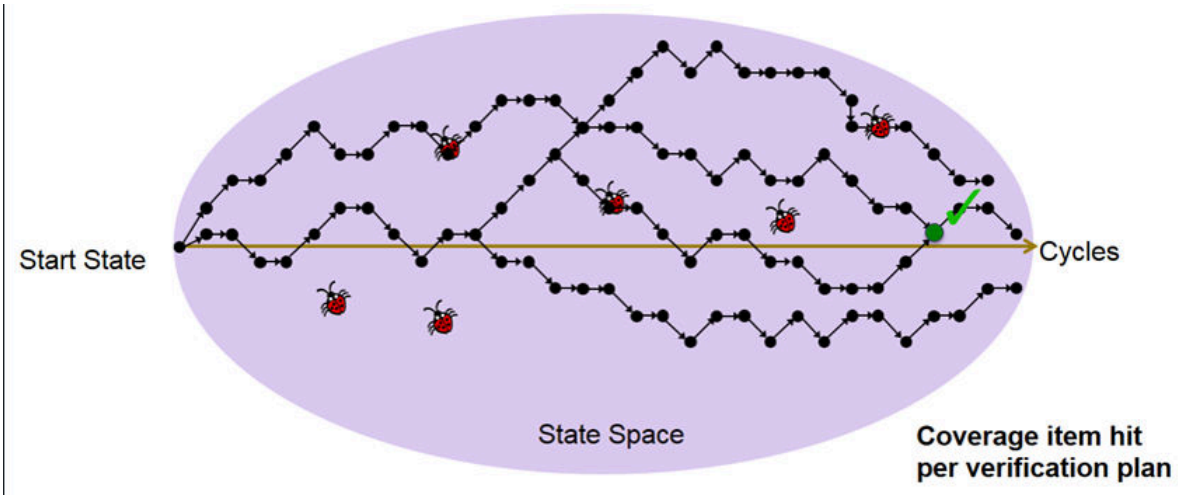
Formal verification has two main broad applications. One is model checking and second is equivalence checking. This document mostly refers to model checking. However, two

VC Formal applications where equivalence checking is employed to address specific verification problems are also described in this document.

1.1.1 What is Formal verification?

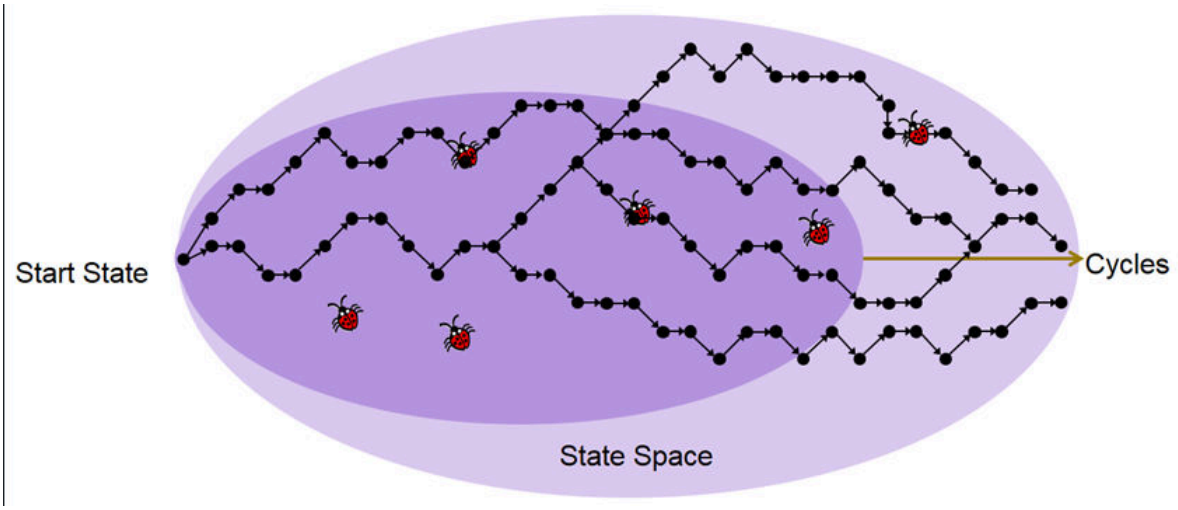
Formal verification leverages mathematical analysis to exhaustively explore design state space to verify the correctness of the design. It is complimentary to simulation-based verification. Simulation is very productive to explore deep sequential behavior and extensive coverage metrics are used to track the verification progress (Figure 1).

Figure 1: Coverage Metrics Driven Simulation-based Verification



Formal based verification requires user defined or auto generated properties to check the correctness of the design. Formal starts from known reset state and exhaustively explores the design state space (Figure 2). Formal problem continues to get more complex with the increase in the sequential depth of the analysis because it has to keep track of more and more register states.

Figure 2: Property Based Formal Based Verification



1.1.2 Requirements for Formal verification

A property is used to describe a design behavior/function. Property can be user written, auto extracted from the design or auto generated using application dependent specification. This property can be used as:

- Assertion/Cover: Property used to describe what is expected design behavior, target for formal verification.
- Constraint - Property used to describe inputs behavior that Formal needs to adhere.
- Synthesizable RTL - Written in HDL (Verilog, VHDL or SystemVerilog).

Formal verification uses exhaustive analysis to prove correctness of the assertions. These assertions can be written in SVA (System Verilog Assertion) or these assertions can be added using predefined assertion libraries (OVL - Accellera assertion library). These assertions can be added in the RTL or in a separate file and then connected to design using language specific methods/constructs.

1.1.3 Formal Verification Results

Formal verification provides three answers; Proof, Falsification, Bounded Proof.

- Proof - If design adheres to the behavior described by the assertion, assertion is proven. This means that assertion would never fail irrespective of how much you simulate it. Function described by the assertion is guaranteed to always pass.
- Falsification: If design does not adhere to the behavior described by the assertion, assertion is falsified. A falsified waveform (also known as counter example waveform) is provided to aid further debug.
- Bounded Proof: If assertion is not proved or falsified, assertion is considered to be bounded proven, this means that assertion is proven up to the sequential depth to which Formal analysis is performed. Assertion cannot fail within the reported sequential depth but it can possibly fail or get proven if analysis is performed for larger sequential depth. Next step is to run Formal analysis for higher depth with higher effort level or user can choose to simplify the Formal verification problem using abstraction techniques.

As part of Formal verification process, a mathematical model is created for design under test and every verification problem is converted into a state bit driven by collection of registers. For an assertion, if this state bit can be asserted, the status is updated to *falsified* and a waveform is provided to visualize the chain of events that led to setting this bit. If it is conclusively determined that this state bit cannot be asserted, then the status is updated to *proven*. If formal engines fail to provide a conclusive answer then the status is updated to *inconclusive* and sequential depth to which exhaustive analysis is performed, is reported as bounded (safe) depth.

On the other hand, if state bit for a cover property is asserted, the status is updated to *covered* and a waveform is provided, if the state bit cannot be asserted, then the status is updated to *uncoverable*. If formal engines fail to provide a conclusive answer, then status is updated to *inconclusive* and sequential depth to which exhaustive analysis is performed is reported as bounded depth.

1.1.4 Scalability - Design size, Complexity and Performance Considerations

For all verification tools and methodologies, scalability is one of the most important consideration. Design of any size and complexity can be simulated as long as required hardware resources are available but the same is not true for Formal. Efficacy and performance of Formal verification is dependent on the size of state space that needs to be taken into account. Size of the state space continues to increase with the increasing sequential depth of Formal analysis. Hence, it is recommended to deploy Formal verification at the block level. Note that for some specific application (like Connectivity checking application), Formal verification can be used at SoC level but some built-in and user-defined design size reduction techniques are used to manage the verification problem.

What is the recommended size? There is no definite answer for this because it is dependent on variety of things:

- Size of the RTL design
- Union of design logic required to verify all properties
- Total Numbers and kind of properties (function that these properties are checking)
- Complexity of constraints

It is recommended to have accurate estimation of size and complexity involved. It helps to achieve better convergence through the Formal verification process. Some of it can be static in nature which can be identified before Formal verification starts and rest of it can be dynamic in nature which can be seen and reviewed when Formal engines are in the process of solving verification problems. If verification problem is large and complex, having access to large machines (large number of cores and memory) accelerates the formal verification process and reduces the turnaround time.

1.2 About VC Formal

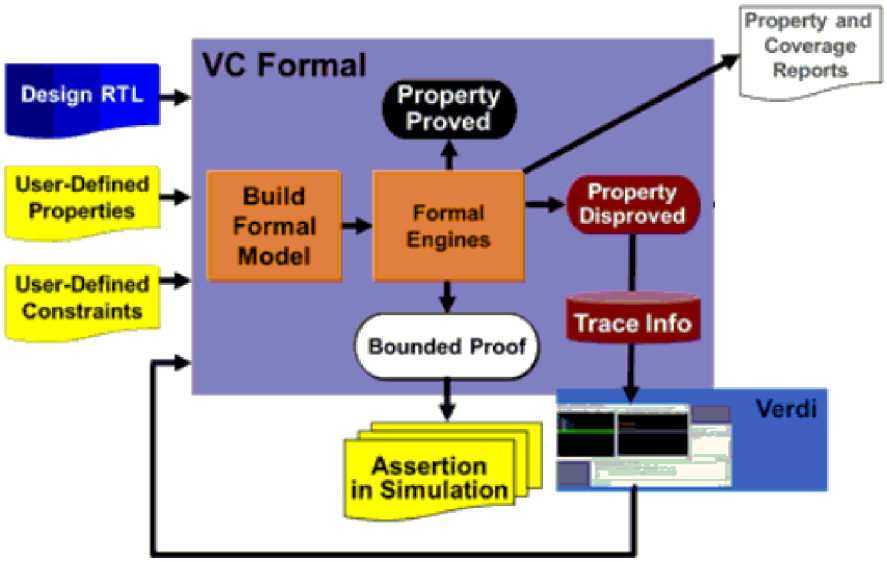
VC Formal is a comprehensive high performance and high capacity solution for functional verification. VC Formal performs property checking that consists of mathematical techniques used to test properties or assertions to ensure the correct functionality of RTL designs.

Formal verification verifies the correctness of a property using formal or model checking methods. Formal or model checking methods can verify assertions or properties written in Verilog or SVA against RTL design. Formal assertion based testbenches, built along RTL development can detect bugs closest to the sources, resulting in easier diagnosis, faster bug fixes and turnaround. Formal Verification also provides an added advantage of letting you add new checkers and necessary constraints to exhaustively verify a feature that is being developed.

Figure 3 shows the design flow for VC Formal.

Feedback

Figure 3: Design Flow for VC Formal

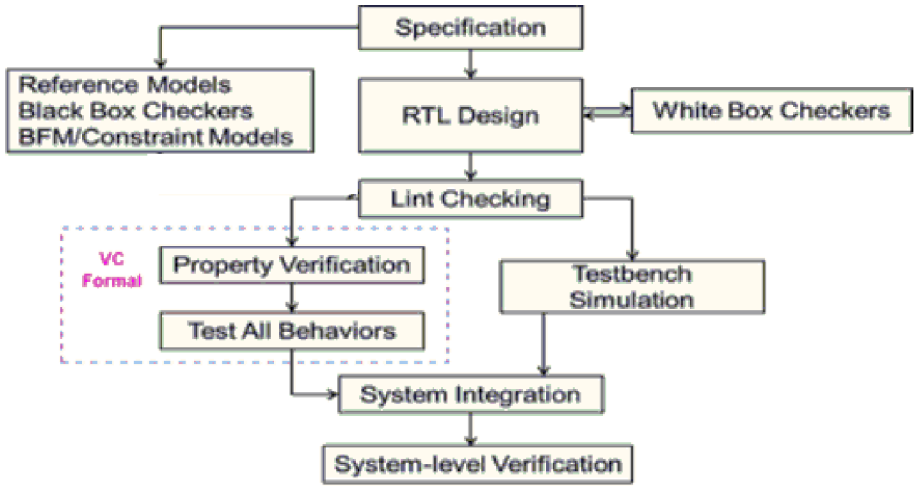


VC Formal in the Verification Flow

VC Formal can be used throughout the design and verification process from specification to validation, using white box and black box property checking, for pre and post silicon debugging.

Figure 4 shows VC Formal in the verification flow.

Figure 4: VC Formal in the Verification Flow



VC Formal GenFV®

The GenFV capability in VC Formal leverages Generative AI along with formal verification and analysis techniques to improve the efficiency of users. One such capability is the

automatic generation of SystemVerilog Assertions (SVA) in the interactive mode or batch mode processing.

For more information on how to enable and use VC Formal GenFV®, see the *VC Formal GenFV Quick Start Guide*.

Note:
VC Formal GenFV® is a Limited Customer Availability (LCA) feature. Contact Synopsys support for more information.

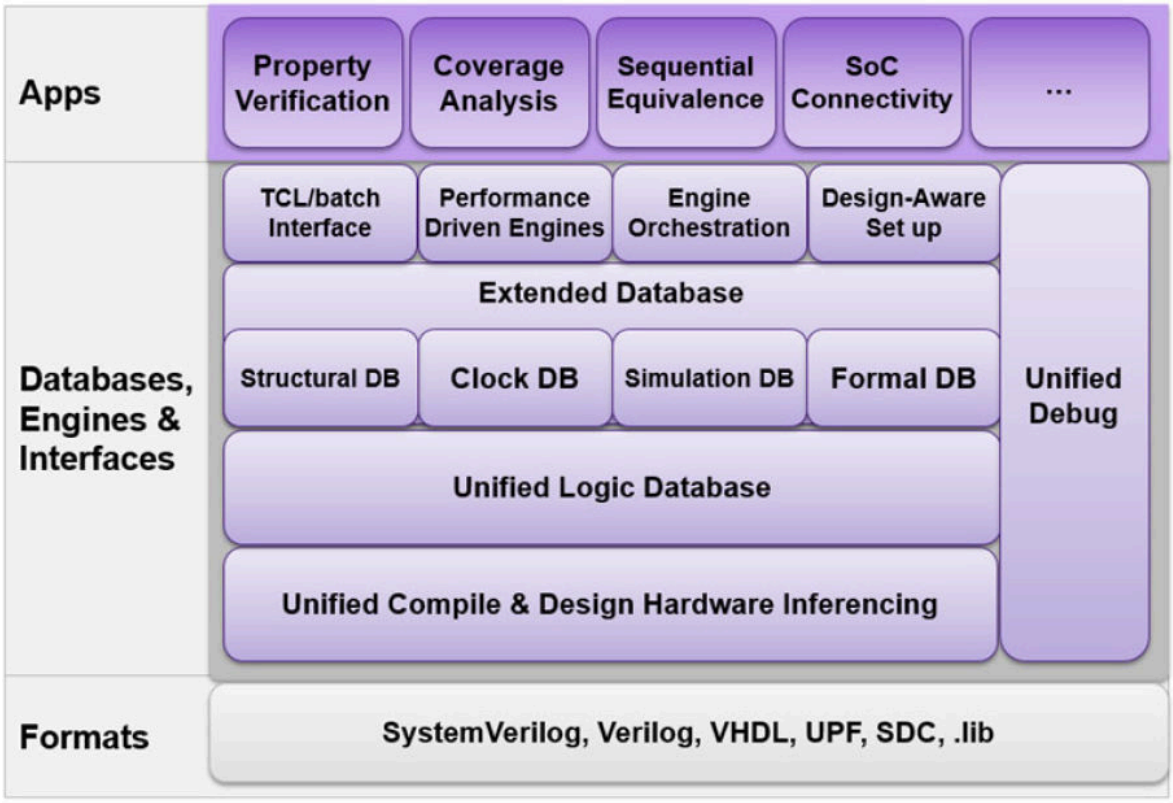
1.3 VC Formal Components

The following are the major components in VC Formal:

- Front-end parser to build a netlist
- Built-in simulator to set up the initial state
- Formal model builder, formal engines and engine orchestration
- Interactive Verdi based Unified Debug
- Tcl interface to open the Verdi GUI, trace, schematic when the tool is run in non-GUI mode
- Result database from where the GUI or Tcl command interface accesses the results

Figure 5 shows the various VC Formal components.

Figure 5: VC Formal Components



1.4 VC Formal Applications

VC Formal offers various applications as part of its capabilities. These applications enable you to setup the right configuration for the verification task. After the design is initially read into the VC Formal, properties (example: SVA properties, coverage properties, AEP properties etc.) are loaded into the set application modes. By default, all tasks are in the FPV application mode, if no application mode is specified.

Note:

All the VC Formal applications require specific license keys. For more information on the license keys required for each of the VC Formal applications, see the *VC Static Platform Installation Notes*.

1.4.1 Automatically Extracted Properties

The Automatically Extracted Properties (AEP) application mode can be used to automatically extract properties from the design and verify them. AEPs are properties

generated by the tool to verify the functionality independent conditions in RTL which can lead to incorrect behavior.

The AEP supports the following types of checks:

- Array Bounds Check
- Arithmetic Overflow
- X-Assignments Check
- Simultaneous Set/Reset Check
- Full-Case / Parallel-Case
- Multi driver, Conflict driver checks
- Floating Bus Checks
- FSM Starvation Checks

For more details on the AEP application mode, see section Automatically Extracted Properties Application.

1.4.2 AIP (Assertion IPs)

Assertion IP have pre-built assertions and constraints that help verify and signoff the usage of IP's in System on Chip (SoC) designs. AIPs help significantly to reduce the verification effort and to improve the design quality by simply monitoring a target design without modifying its RTL and without simulation test-benches. In addition, AIPs help to quickly setup Formal environment and sanity test various configurations like 1-manager multiple-subordinate scenarios easily.

For more details on the supported AIPs, see section Supported AIPs.

1.4.3 Connectivity Checking

The Connectivity Checking (CC) application mode can be used for checking connectivity between various nodes in the design. VC CC application also provides additional features like structural checking and root cause analysis. The CCs can be specified either as CSV format or TCL commands.

- The CC application can be used for the checking the following:
- SoC I/O Connectivity
- Block pin muxing/demuxing
- Connectivity and constant checking of macros

- Scan mode connectivity and constant checking
- Reset and global signal connectivity
- Registers to debug bus
- Configurable interconnect verification

For more details on the CC application mode, see section Connectivity Checking Application.

1.4.4 Coverage Analysis

Coverage is a form of metrics used by project teams with the goal of providing a clear, quantitative, objective measure for assessing their overall verification process, as well as assessing the project's progress towards completeness.

As part of the signoff review process, code coverage needs to meet a certain target. When that target is not met at the beginning phase of coverage convergence, many project managers require that the verification team present their final coverage results and assess the risk for not hitting a particular coverage item.

Verification teams want to minimize the effort of reporting and justifying uncovered coverage items during the review process, especially if the uncovered item is really unreachable.

The Formal Coverage Analyzer (FCA) application mode can be used to run unreachability analysis and find coverage properties in your design which cannot be covered with any stimulus. This helps to find exceptions that can be used to achieve coverage closure much faster. But, before using these exceptions for closing coverage, they need to be analyzed carefully to ensure that they are not masking any potential bugs in the design. The FCA mode can perform coverage analysis for the following metrics:

- Line
- Condition
- Branch
- Toggle
- Finite State Machine (FSM) state
- FSM transition
- SV covergroups

For more details on the FCA application mode, see section Formal Coverage Analyzer Application.

1.4.5 Data Path Validation

Data Path Validation (DPV) application uses HECTOR technology to verify data transformation blocks. Example of these data path blocks are floating point/integer adder, multiplier, divider, multiplication accumulate and so on. These design structures are very common in CPU, GPU, and AI/ML designs. VC Formal DPV leverages transactional equivalence to compare two versions of the design, one version representing the design functionality at architecture level (mostly untimed C or C++ model) and second version representing the pipeline implementation (mostly RTL).

For more information on the DPV application mode, see section Data Path Validation Application.

1.4.6 Formal Low Power Application

VC Formal Low Power (FLP) application allows complete verification of power controller, power aware connectivity checking with VC Formal CC application.

Low power formal verification is one of the final stages in the verification cycle. Validating a complex power critical design in simulation (with more than 100 power islands) can be time consuming and tedious, as it requires developing additional power sequencing tests in the testbench to test all the power modes. Therefore, VC Formal FLP becomes extremely useful and productive to left shift the verification cycle and catch the bugs early in the flow.

For more details on the FLP application mode, see section VC Formal Low Power Application.

1.4.7 Formal Property Verification

The Formal Property Verification (FPV) application mode can be used to verify user defined properties in the design. These properties are written by the designer/verification engineers to describe the design intent and legal behavior of primary inputs of the design. These properties are configured as:

- Assertions to check design behavior
- Assumptions to constrain the formal verification environment
- Coverage property to monitor interesting/expected events

For more details on the FPV application mode, see section Formal Property Verification Application.

1.4.8 Formal Register Verification

The Formal Register Verification (FRV) application mode enables you to automatically create assertions for various access modes of registers in the design. You can provide the register specification information in the IP-XACT, RALF or CSV format where all the registers and/or reg-fields are defined. VC Formal performs various checks on the registers based on the descriptions provided and reports the results of the checks.

For more details on the FRV application mode, see section Formal Register Verification Application.

1.4.9 Functional Safety

For automotive designs, functional safety is considered mission critical. As part of ISO 26262 certification, faults analysis helps to determine if there are enough safety mechanisms implemented in the design to ensure the robust functionality of the design. Diagnostic coverage is one of the metrics and it provides information on proportion of the hardware element failure rate that is detected or controlled by the implemented safety mechanisms.

Faults simulators like Synopsys ZOIX calculate diagnostic coverage but like functional simulation, fault simulation is also dependent on available test vectors. VC Formal Functional Safety (FuSa) application is part of the unified functional safety verification flow, and works on same set of faults and helps to close diagnostic coverage faster.

For more information on VC Formal FuSa application, see section Functional Safety Application.

1.4.10 Formal Security Verification

FSV application ensures that unexpected data propagation does not happen from secure to non secure areas and vice-versa in a design. These two scenarios are called data leakage and data integrity.

- Data Leakage: secure data must not reach or influence the non secure areas of the design
- Data Integrity: non secure data must not reach or influence the secure areas of the design

FSV performs the data propagation analysis between a source and a destination. The source and destinations can be inputs, outputs, nets, registers, memories, and instances.

For more information on the FSV application mode, see section VC Formal Security Verification Application.

1.4.11 Formal Testbench Analyzer

The key for obtaining a formal signoff of a design block is to have a formal testbench environment that ensures:

- Enough assertions are written to cover the complete design functionality.
- The set of properties are correct and complete.

If there is a bug in the design, can the formal testbench catch it? If not, corner case bugs could escape to silicon. Combining VC Formal with Certitude Fault Injection through RTL mutation, the FTA can be used to check if the injected faults can be detected by the formal testbench. This is an essential step in achieving signoff.

The Formal Testbench Analyzer (FTA) application mode can be used after you think that the formal testbench has a comprehensive set of properties that are proven or inconclusive on the original RTL in the FPV mode. You can use the same constraints and target properties in the FTA application on a fault injected RTL model to qualify whether an injected fault can be detected.

For more details on the FTA application mode, see section Formal Testbench Analyzer App.

1.4.12 Formal X-propagation

The FXP application in VC Formal allows you to check that unknown signal values cannot propagate through a design. There are the following types of points defined in the design:

- X-injection: Point at which an unknown value is introduced into a design.
- X-observation: Point at which an unknown value is observed.

You can configure the X-injection and X-observation points from a list of predefined types and auto generate the required properties for checking.

If a property fails, then FXP debug environment allows you to trace the failure to the source of X through both waveform and schematic views.

For more information on the FXP application mode, see section Formal X-Propagation Application.

1.4.13 Navigator

As part of the verification flow, you must may want to refine constraints, understand design behavior, or debug an assertion failure. In these scenarios, the explore and plot capabilities enable you to do a what-if analysis and observe design behavior such as

graphically defining new constraints, covers, asserts and observing the effects of these new constraints, asserts, covers and repeat the process.

It is difficult and tedious to explore the intended design behavior and corner case scenarios where a design might not work as intended in a simulation environment. Navigator makes it easy as it does not require a simulation test bench environment. It provides an interactive capability to explore a design and develop formal properties. It provides the flexibility to explore the design step by step. You can modify the waveform for one scenario (or add a new condition in the form of constraints) and then incrementally explore the next set of scenarios.

For more details on Navigator, see section Exploring Design Using Navigator.

1.4.14 Sequential Equivalence Checking

The Sequential Equivalence Checking (SEQ) application mode can be used to compare two RTL designs written in synthesizable SystemVerilog or VHDL. It compares two RTL designs and verify that they are equivalent on a cycle-by-cycle basis.

One of the designs is designated as the specification or spec, and the other as the implementation or impl. Typically, the spec design is known to be correct, and the VC Formal SEQ application is used to verify that the post-modification, unknown design (impl) matches the behavior of the spec design.

A typical use of the VC Formal SEQ application is when you modify a known-good design to add some transformation such as clock-gating or re-timing and want to ensure that the RTL that results from the transformation still exhibits the same correct behavior as the RTL before the transformation.

For more details on the SEQ application mode, see section Sequential Equivalence Checking Application.

1.5 Using VC Formal for Convergence and SignOff

VC Formal Convergence Techniques

For complex properties, out of the box analysis may take longer to get a conclusive answer or you may not get a conclusive answer at all. There are various recommended techniques and flows using which you can simplify the verification problem or make better use of available resources which can help you get answers relatively quickly.

For more information the convergence techniques, see section Considerations for Convergence Improvement.

Obtaining Formal Signoff

Signoff criteria for simulation-based verification environment is very well-established, and it includes extensive coverage metrics, for example, code coverage, functional coverage, protocol coverage and so on. These coverage metrics help verification teams track verification progress, and determine the completion of verification cycle for a given design.

For decades, users have been looking for similar coverage metrics for Formal verification. For specific application like connectivity, VC Formal has notion of connectivity coverage using which user can determine if list of connections in the connectivity specification is complete or not. Specifically, for formal property checking (FPV), as part of signoff process, user needs to find out the answer for following questions:

- Do I have enough properties?
- How do I ensure that I am not getting any false positives?
- If I am unable to conclusively prove a property, what does this mean from verification point of view?
- If I have managed to prove a property, what part of the design logic was verified by this property?
- Quality of the properties? If I have any bug in my design, then do I have a property to catch that bug?

As part of VC Formal, there are features/technology available to answer all these questions. For more information, see section Obtaining Formal Signoff For a Design.

1.6 Interactive Verdi based Unified Debug

Verdi based interactive VC Formal GUI provides the capability to debug formal properties. VC Formal GUI supports all standard Verdi based debug features using waveform, source browser, assertion analyzer and so, it also includes:

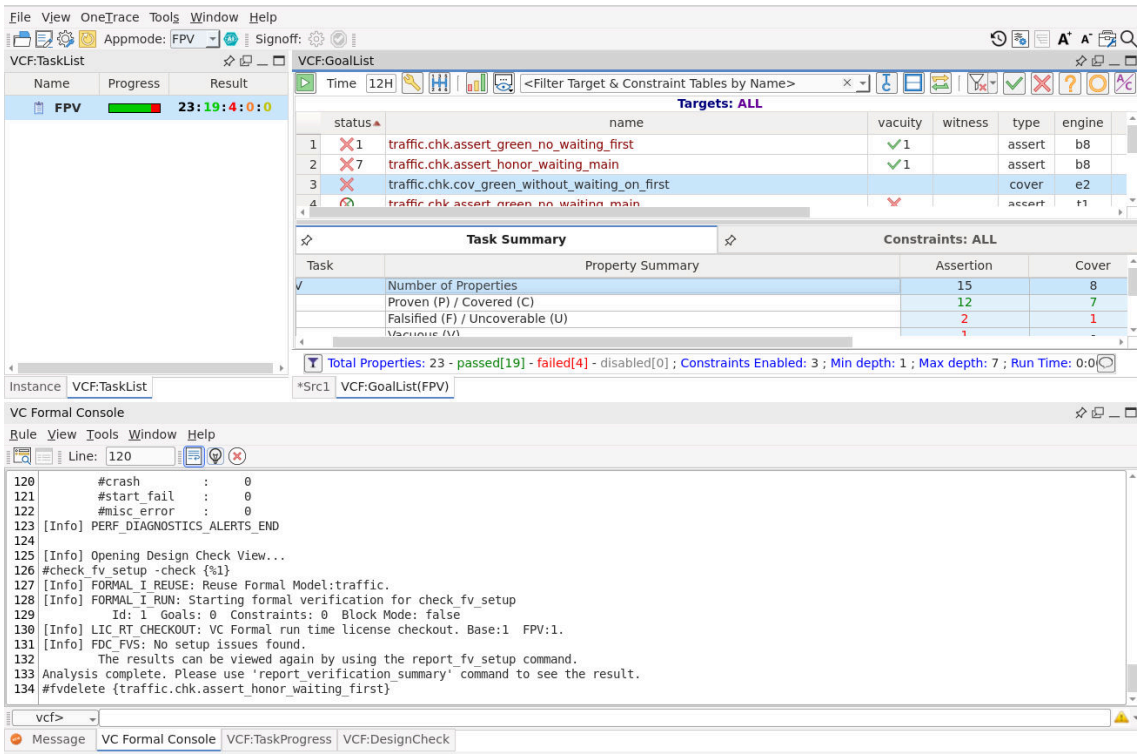
- Restarting and editing scripts from the Verdi top-level toolbar
- Setting application modes
- Filtering properties from the *GoalList* toolbar
- Viewing targets and constraints in the GoalList
- Viewing progress and complexity reports in the Verdi shell and message view
- Running Formal coverage analysis (see section “Formal Coverage Analyzer Application” for more details)

Feedback

The VC Formal GUI provides a layout as shown in Figure 6, which contains the following panes:

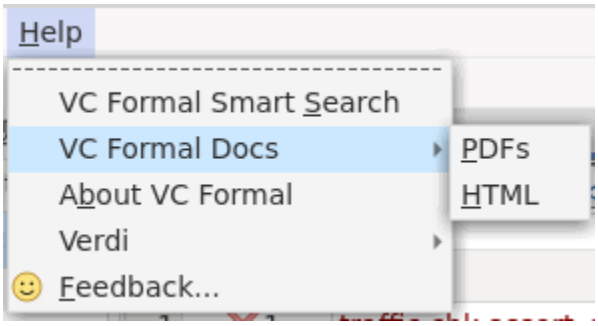
- VC Static and Formal toolbar (see section “Using VC Static and Formal Toolbar” for more details)
- *TaskList* (see section Using the TaskList for more details)
- *GoalList* (see section Using the GoalList for more details)
- Interactive Tcl Shell

Figure 6: VC Formal GUI



1.7 VC Formal Documentation

On the VC Formal toolbar in the Verdi GUI, use the **Help** menu to access the VC Formal Documentation.

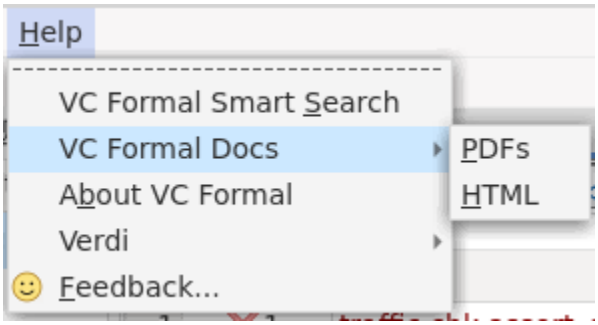


Note:
The preferred PDF reader to open all PDFs in the installation is Evince (GNOME Document Viewer). The PDFs are tested with Evince - Version 3.28.2 on the Linux CentOS, ARM CentOS, and Rocky Linux machines.

Note:
The preferred browser to open the HTML help is Firefox. The HTML help is tested with Firefox on the ARM CentOS, and Rocky Linux machines.

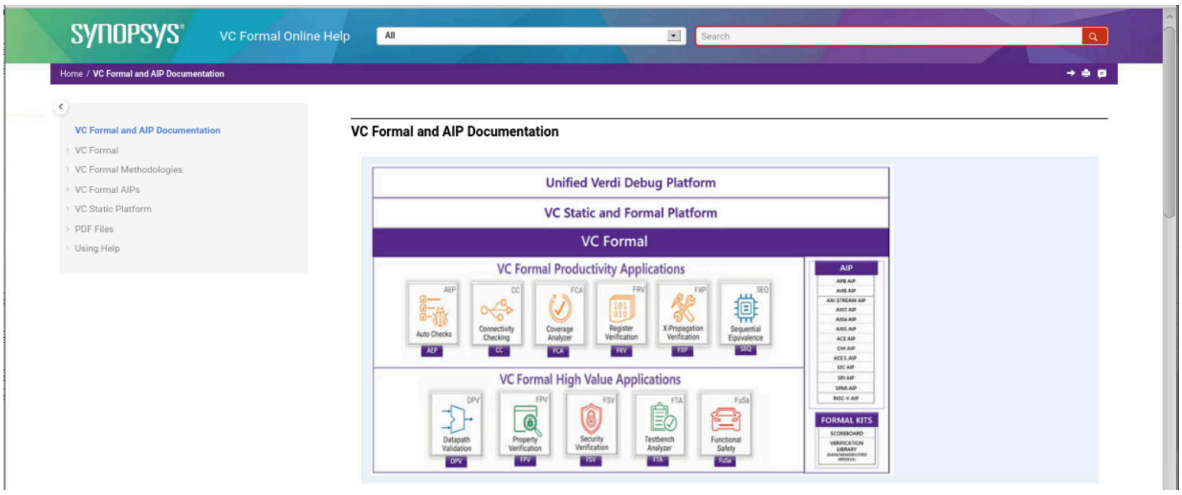
1.7.1 Viewing HTML Documentation From GUI

You can also view the documentation in the HTML format from the **Help > VC Fomal Docs > HTML** menu as shown in the following figure:



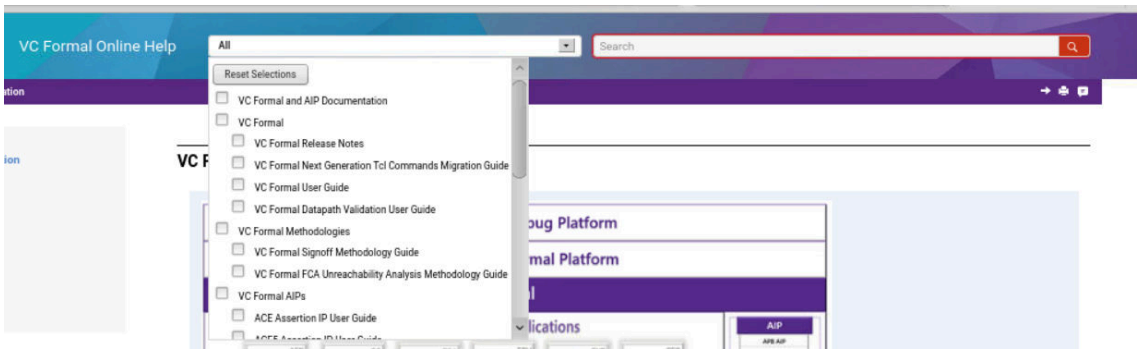
Feedback

The **VC Formal Online Help** window appears as illustrated in the following figure:

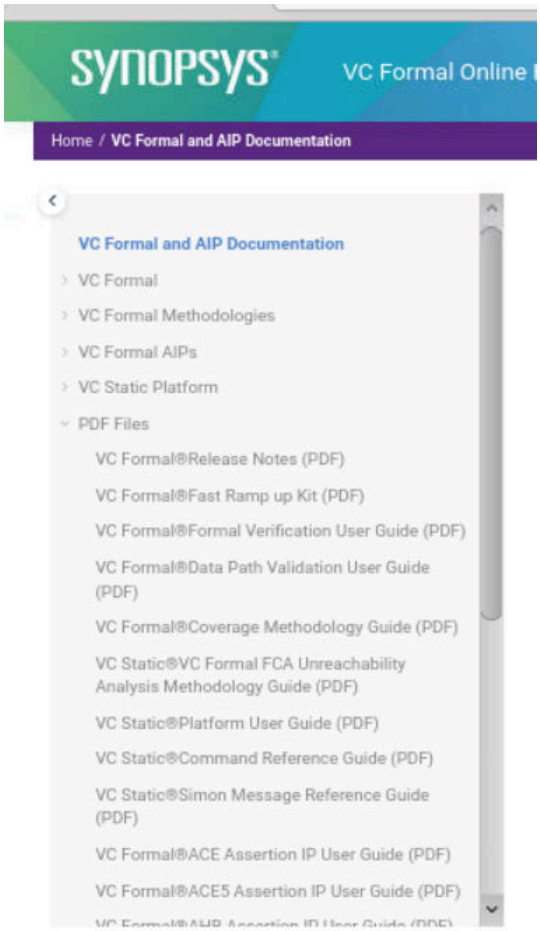


The **VC Formal Online Help** enables you to:

- Search within a selected or subset of documents.



- Download the PDFs of all documents from the browsers.



1.7.2 Smart Search in VC Formal

VC Formal provides the capability to efficiently search for a specific title or topic in the VC Formal documentation using Smart search.

Smart Search provides the following capabilities:

- Robust search techniques, which enable you to find out the required information or topics quickly, even with the partial knowledge of the product.
- Handles queries in natural language.
- Learns from user interaction and maintains history of searches, which can show improved results in future iterations.

Feedback

Accessing Smart Search

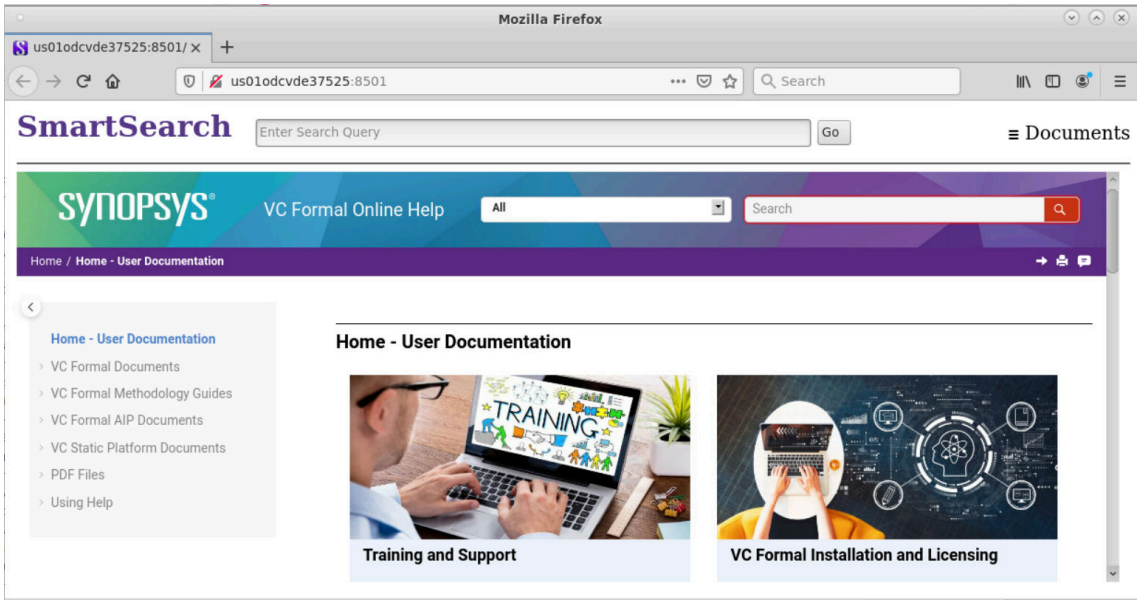
To access Smart Search:

- From the menu, select **File > VC Formal Smart Search**.

  Alternatively, you can use the `smartsearch -tool formal` command to open Smart Search in Tcl.

  The *Smart Search* window appears, as shown in Figure 7. You can also navigate to previous and next topics and toggle between the home and contents page using the icons on the Smart Search toolbar.

Figure 7: Smart Search

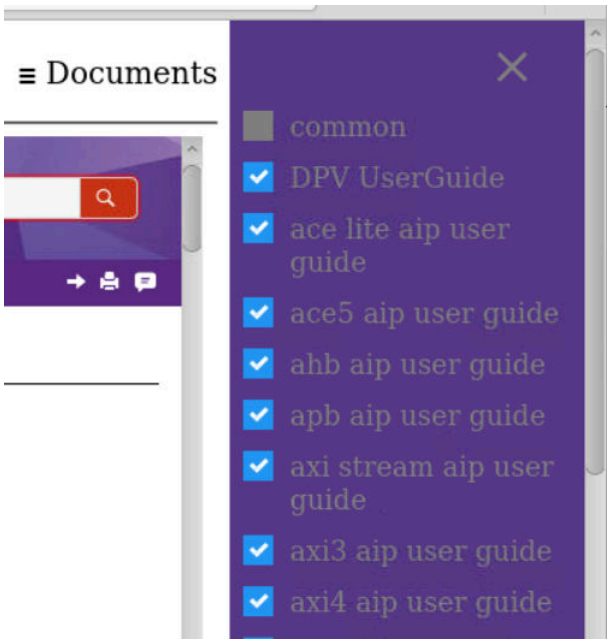


Feedback

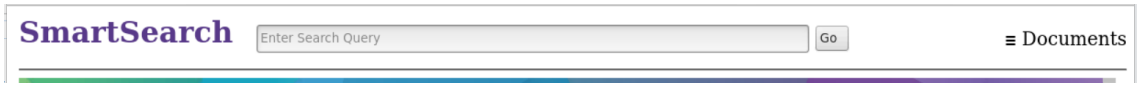
Searching for Specific Topics Using Smart Search

To search for information or topics using the Smart Search:

1. Choose *Documents* > *Select/Deselect All*,*VC Formal User Guide* or *VC Static Platform Command Reference* within which you want to search for information.



2. Enter the topic or query in the *Type query and press enter* box. The search results are listed on the requested topic.



3. Rate the topics using the following icons:

1.8 Next-gen Commands in VC Formal

Starting with the V-2023.12-SP1 release, VC Formal is enhanced with a new set of easy-to-use Tcl commands and interfaces (available as Early Access (Beta)). These set of Next-gen commands provide a consistent manner to interact with the tool and a simplified set of configuration options across all the Apps. The benefits are especially applicable to new users as they provide an intuitive and user-friendly way to interact with VC Formal.

Note:

In the V-2023.12-SP1 release and until further notice, there are no changes to the existing commands and interfaces. All existing commands and Tcl scripts will work as is.

For more information on the Next-gen commands supported in VC Formal, contact the *Synopsys VC Formal Support team*.

2

Setting up VC Formal Design Environment

This section discusses the prerequisites and the steps to setup the VC Formal design environment.

The sections contains that following topics:

- Supported Design Languages and Properties
- Setting The VC_STATIC_HOME Environment Variable
- Using Standalone VC Formal Installation
- Updating VC Static Platform and VC Formal Application Variable Settings
- Setting the Application modes

2.1 Supported Design Languages and Properties

VC Formal supports the following design languages for synthesizable RTL:

- Verilog
- VHDL

  VHDL is supported for the following features:

  - Formal Property Verification (FPV)
  - Formal Coverage Analyzer (Line, Condition)
  - Automatically Extracted Properties (Bus Checks).

    The following are the limitations for VHDL support:

  - Counter examples cannot be viewed in the Verdi waveform when the VHDL configurations are elaborated in VC Formal.
  - The `hdl_xmr` procedure in VHDL is not supported.
  - Record types cannot be viewed in Verdi Waveform.

- MX
- SystemVerilog Design

VC Formal supports properties in the following languages or format:

- Automatically extracted properties generated by the tool.
- Coverage properties including line, condition, toggle, fsm state, or fsm state transition coverage analysis properties generated by the tool.
- Script properties using Tcl commands.
- Source properties in SVA or OVL.
- Connectivity properties using Tcl commands or in CSV format.
- SystemVerilog Assertions (SVAs).

  For more details on supported SVA format, see section “Using SVA for Formal Analysis”.
- The register verification information in the IP-XACT format.

2.2 Setting The VC_STATIC_HOME Environment Variable

VC Formal uses the pivotal environment variable: VC_STATIC_HOME. This variable must be set to point to the installation directory as shown in the following code snippet. In the installation directory, you can find the bin, lib, doc and other directories.

```
% setenv VC_STATIC_HOME /tools/synopsys/vcst
```

You can add `$VC_STATIC_HOME/bin` to your `$PATH`.

To start the VC Formal tool, execute the following command:

```
% $VC_STATIC_HOME/bin/vcf
```

This command starts a VC Formal shell session and you see the following prompt:

```
vcf>
```

2.2.1 vcf Shell Command Line Options

The following command line options are available for VC Formal shell. The options may be abbreviated by leaving out the text in parenthesis; for example, either `-f` or `-file` can be used to give the name of a script file to execute.

Feedback

Note:

A separate directory is recommended for creating the Tcl command file and running VC Formal.

Syntax

```
[-batch]                              Start tool in batch mode
 (non-interactive).
[-out_dir <dir_name>]                 Name of output directory.
[-cmd_log_file <log_name>]            Name of command log file in current
 directory.
[-f(ile) <file_name>]                 Script file to exec after setup.
[-fmode <FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV>] Set fml mode.

[-gui]                                Start the tool in GUI mode. By default,
                                      a Verdi Elite license is checked out.
                                      Add -verdi_base or -verdi_apex to
                                      check out a Verdi Base or Verdi Apex
                                      license respectively.
[-verdi]                              Start the tool in GUI mode using
                                      Verdi Elite license mode.
[-verdi_base]                         Start the tool in GUI mode using
                                      Verdi Base license mode.
[-verdi_apex]                         Start the tool in GUI mode using
                                      Verdi Apex license mode.
[-h(elp)]                             Print this help message.
[-id | -ID]                           Give more information about application
 build/env.
[-lic_wait <minutes>]                 Wait for license for #minutes.
[-no_init]                            Don't load .synopsys_vcst.setup files.
[-no_restore]                         Remove previous session and start a new
 one.
[-no_ui]                              Starts the tool without the GUI/UI
 process.
[-output_log_file <log_name>]         Capture console output in given log
 file.
[-read_only]                          Restore a previous session in read-only
 mode.
[-reset]                              Clean up the old data of the work
 session.
[-restore]                            Restore a previous session.
[-session <session_name>]             Use the <session_name> directory for
 the runtime database.
[-use_ipv4]                           Use IPv4 protocol for inter process
 communications.
[-use_ipv6]                           Use IPv6 protocol for inter process
 communications.
[-x <config_tcl_cmd>]                 Execute configuration TCL command
[-y <post_config_tcl_cmd>]            Execute post configuration TCL command
[-doc]                                Open tool documentation in SmartSearch
[-fml_base]                           Enable VC Formal Base Licensing Mode
 (fml_base)
```

Feedback

```
[-fml_ultra]                        Enable VC Formal Ultra Licensing Mode
 (fml_ultra)
[-fml_elite]                        Enable VC Formal Elite Licensing Mode
 (fml_elite)
[-fml_token]                        Enable VC Formal Token Licensing Mode
 (fml_token)
[-fml_legacy]                       VC Formal Legacy Licensing Mode (bypass
 fml_base/fml_elite/fml_token licensing lookup)
[-fml_no_legacy]                    VC Formal Non-Legacy Licensing Mode (No
 default to legacy licensing)
[-fml_zoix]                         VC Formal Tiered FuSa Licensing Mode
[-fml_z01x]                         VC Formal Tiered FuSa Licensing Mode
[-fml_auto_restore_level <none|job|engine|host|all>] Set restore level of
 auto saved formal session.
[-scl_span]                         Enable grabbing licenses across
 multiple servers.
[-rma_off]                          Do not enable RMA automatically in
 Elite, Token, and Ultra licensing modes
[-container]                        Start tool within a container
 environment.
[-noecho]                           Disable reporting commands in the
 screen output and session log.
[-fml_nxg <value>]                  Use the values 0(default), 1, 2 to
 enable the required VCF command consistency mode.

debug options:
[-echo]                             Echo the environment but do not invoke
 the executable.
```

vcf shell Command Line Examples

- To start VC Formal with an interactive shell:

  ```
  vcf
  ```

- To get help information on the interactive shell:

  ```
  vcf -help
  ```

- To start VC Formal in batch mode:

  ```
  vcf -f vcst.tcl -batch -o vcst_screen.log
  ```

- To save the output in a log file:

  ```
  vcf -output_log_file filename
  ```

- To start VC Formal with a command script file:

  ```
  vcf -f ip_filename.tcl
  ```

- To start VC Formal in the GUI:

  ```
  vcf -verdi
  ```

Or

```
vcf -verdi -f ip_filename.tcl
```

- To set the application mode in the `vcf` shell:

  ```
  vcf -fmode <value>
  ```

  where `value` can be one of `{FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV}`. If no value is specified, the FPV application mode is set.

  This setting is same as the setting achieved by the `set_fml_appmode <FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV|FTA>` command in VC Formal.

  Note:

  An error message is reported when you start VC Formal with the FTA application mode:

  ```
  % vcf -fmode FTA
  ```

  Error: Value of -fmode should be one of FPV, COV, SEQ, CC, AEP, FRV, FXP, FSV, FUSA, DPV

  You cannot start the tool with the FTA application mode because we usually start in FPV application mode before switching to FTA. You can use the FTA application mode with the `set_fml_appmode` command.

  Note:

  To see the application mode currently set, use the `get_fml_appmode` command. This command returns the application name as set with `set_fml_appmode` command.

- The IP networking protocol has been going through a transition (from IPv4 to IPv6) to support the continually increasing IP addresses required globally, as well as to provide improvements in efficiency, security, and ease of management. All VC Formal applications rely on IP addressing (either directly or indirectly) for tasks such as socket-based communication and job management in a distributed processing application.

  By default, VC Formal should be able to automatically detect if the network protocol is IPv4 or IPv6.

  However, because there are many variations of the network setups, an option -use_ipv6 or -use ipv4 can be used at the invocation of VC Formal if you encounter issues when launching server jobs. For example,

  ```
  %> vcf -use_ipv6
  ```

- With the S-2021.09 or later release, the following licensing modes are enabled:
  - When the license startup options (`[-fml_base] [-fml_ultra] [-fml_elite] [-fml_legacy]`) are not specified, VC Formal first looks up for the base license keys. If the base license keys are not available, VC Formal looks for the Elite license keys. If the Elite license is not available, VC Formal looks for the Legacy license keys.
  - When the `-fml_base` startup option is specified, VC Formal looks for the base license keys. If the base license keys are not available, VC Formal looks for the legacy license keys.
  - When `-fml_legacy, -fml_ultra` start up option is specified, VC Formal does not look for the Elite license keys, it will directly look for the Ultra license keys.
  - When the `-fml_ultra` or `-fml_elite` start up option is specified, VC Formal first tries to check out Elite license keys. If the Elite license keys are not found, VC Formal uses the legacy licensing keys, where it tries to pick up an Ultra license key.
  - When the `-fml_zoix` or `-fml_z01x` start up option is specified, VC Formal first tries to check out Elite license keys. If the Elite license keys are not found, VC Formal uses the legacy licensing keys, where it tries to pick up an Ultra license key.

    Once a specific license mode has been determined: elite, base, legacy ultra, or legacy base (VC-STATIC-FV), the associated base, elite, legacy ultra, legacy base run time licenses are used during a check. Once in a session, the license mode cannot be changed, and there is no mix/match of licenses. On a save/restore, you can change the license mode.

    Note:

    When you use the -batch option, VC Formal automatically quits the shell even when -quit is not explicitly specified in the vcst.tcl file or when an unexpected error occurs and the full run is not complete. This is useful for running regressions.

Controlling Verdi License Modes When Running VC Formal

Verdi supports the following three license modes:

- Base (license key: VERDI-SX)
- Elite (license key: Verdi)
- Apex (license key: VERDI-ULTRA)

Depending on how the VC Formal GUI is opened, the Verdi license mode can be configured either at runtime or before starting the VC Formal GUI:

- Using `% vcf -gui` starts the VC Formal GUI in the Elite Verdi license mode.

  Add `-verdi_base` or `-verdi_apex` to `% vcf -gui` to set the Verdi license mode to Base or Apex respectively, as these options override the default `-gui` setting.

  ```
  % vcf -gui -verdi_base
  % vcf -gui -verdi_apex
  ```

  Alternatively, set the `SNPS_VCST_VERDI_LAUNCH_MODE` application variable to `verdi_base` or `verdi_apex` before launching VC Formal to specify the required Verdi license mode.

- Using `% vcf -verdi` or `start_gui/view_activity/start_verdi -verdi` starts VC Formal in GUI mode using the Elite Verdi license mode.
- Using `% vcf -verdi_base` or `start_gui/view_activity/start_verdi -verdi_base` starts VC Formal in GUI mode using the Base Verdi license mode.
- Using `% vcf -verdi_apex` or `start_gui/view_activity/start_verdi -verdi_apex` starts VC Formal in GUI mode using the Apex Verdi license mode.
- By default, using the `start_gui/view_activity/start_verdi` command opens the GUI for a VC Formal session currently running in command-line mode using the Elite Verdi license mode.

  Alternatively, set the `SNPS_VCST_VERDI_LAUNCH_MODE` application variable to `verdi_base` or `verdi_apex` before launching VC Formal to set the Verdi license mode to Base or Apex respectively.

  Example

  To use the Apex Verdi license mode when using command `start_gui`:

  ```
  % setenv SNPS_VCST_VERDI_LAUNCH_MODE verdi-apex
  % vcf
  vcf> start_gui
  ```

Note:

If an Elite license is not available, the tool automatically falls back to the Apex license mode. However, there is no fallback mechanism from the Base or Apex license modes. If either of these licenses is unavailable, a license checkout error is reported.

Feedback

2.2.2 Changing the VC Static Session Name and Location

Once the `vc_static_shell` is run in any user work directory, VC Static creates a default session (work database directory) in the current working area called *vcst_rtdb* (VC Static Run Time Data Base) along with default log files. The default session name is *vcst*.

All the environment variables and their values which are set at the tool startup is saved in the *vcst_rtdb/logs/env.txt* file.

You can change the name of the session and the location of the session at the time of invoking `vcf`.

```
vcf -session my_path/my_session  <other commands>
```

This command creates a database directory *my_session_rtdb* inside the directory ./ my_path

In addition, if you want to fork a new session from the existing session and save the new session, use the `save_session` command.

```
vcf> save_session -session <session_name>
```

However, forking (taking a snapshot) a new session is allowed only after design is loaded successfully into the current session.

2.2.3 Updating the vcf Setup File

VC Formal uses the *.synopsys_vcst.setup* file to configure its environment for the design files for the VC Formal run. You can create your own custom setup file with all the required settings (application variables, settings, custom scripts) that must be loaded each time you invoke the `vcf`. The key components of the *.synopsys_vcst.setup* file are the name mappings in the design libraries and the variable assignments.

When you invoke VC Static, VC Formal looks for the `.synopsys_vcst.setup` files in the following three directories in the following order:

1. Primary Setup Directory

   The `.synopsys_vcst.setup` file in the *$VC_STATIC_HOME/admin/setup* directory contains default settings for the entire installation. If this file exists, VC Formal reads it first.

2. User Home Directory

   VC Formal reads the setup file in your home directory second, if present. The settings in this file take precedence over the conflicting settings in the `.synopsys_vcst.setup` file in the primary setup directory, and carry over the rest.

3. User Run Directory

VC Formal reads the setup file in your design directory last. The settings in this file take precedence over the conflicting settings in the `.synopsys_vcst.setup` file in the primary setup directory, and the `.synopsys_vcst.setup` file in your home directory, and will carry over the rest. You can use this file to customize the environment for a particular design.

Table 1: The .synopsys_vcst.setup Files

File	Location	Function
.synopsys_vcst.setup	Primary setup directory ($VC_STATIC_HOME/admin/setup)	Contains general VC Formal setup information.
.synopsys_vcst.setup	User home directory	Contains your preferences for the VC Formal working environment.
.synopsys_vcst.setup	User run directory	Contains design-specific VC Formal setup information.

Note:

If you want to prevent VC Formal from reading setup files, you can use the `-no_init` command line switch.

Enabling and Configuring Startup Settings Using Environment Variables

Use the `SNPS_VC_STARTUP_SETTINGS_LOCATION` variable to specify the location from which the tool should load the *.synopsys_vcst.setup* file.

Note:

When the SNPS_VC_STARTUP_SETTINGS_LOCATION environment variable is not specified, the default behavior is to load file *.synopsys_vcst.setup* from the tool installation, then the user home directory, and then the current project working directory.

Syntax

```
setenv SNPS_VC_STARTUP_SETTINGS_LOCATION <admin | project | user | <path
 to the settings file>
```

- `admin`: Loads the settings from the .synopsys_vcst.setup file in the tool installation area.
- `user`: Loads the settings from the .synopsys_vcst.setup file present in the user's home area.

- `project`: Loads the settings from the .synopsys_vcst.setup file present in the working directory.
- `<path to the settings file>`: Specify the name of the relative or absolute path to a file (any name is okay) that is to be loaded when starting the tool. It is NOT a directory location where to find the file, but points to the file itself.

Example

```
setenv SNPS_VC_STARTUP_SETTINGS_LOCATION project
```

When the settings file cannot be found under the specified location, a warning message is displayed. For example, *[WARN]:startup settings file: ./.synopsys_vcst.setup doesn't exist.*

2.3 Using Standalone VC Formal Installation

In all the previous releases, all the VC Static products including VC Formal, VC SpyGlass CDC/Lint/RDC, and VC LP were available strictly in the same installation package on SolvNetPlus (known as vc_static). VC Formal was not available as a standalone installation package.

Starting with this release, a separate installer for VC Formal is available on SolvNetPlus (called vc_formal). You can install VC Formal as a standalone installation using the separate installer to distinguish between the different VC Static products which may need different versions. After installation, you can access the standalone VC Formal installation using the `module load/avail` or by setting the `$VC_FORMAL_HOME` environment variable pointing to the new installer. See section Setting The VC_FORMAL_HOME Environment Variable for more details.

You may want to install the standalone VC Formal installer and use its binary (by setting appropriate environment variable) if you are maintaining multiple VC Static products with different versions. For example, you may want to use VC Static for the VC LP product, but VC Formal for only the VC Formal product.

Note:

There are no changes to the existing VC Static installation packages available on SolvNetPlus. You can continue to use VC Formal with the VC_STATIC_HOME environment variable if you are installing the VC Static installation package.

Note:

There are no functional differences between the different installers and binaries (VC Static or VC Formal). The packaging is strictly to help customers manage different product versions.

2.3.1 Setting The VC_FORMAL_HOME Environment Variable

VC Formal uses the pivotal environment variable: VC_FORMAL_HOME. This variable must be set to point to the installation directory as shown in the following code snippet. In the installation directory, you can find the bin, lib, doc and other directories.

```
% setenv VC_FORMAL_HOME /tools/synopsys/vcst
```

You can add `$VC_FORMAL_HOME/bin` to your `$PATH`.

To start the VC Formal tool, execute the following command:

```
% $VC_FORMAL_HOME/bin/vcf
```

This command starts a VC Formal shell session and you see the following prompt:

```
vcf>
```

2.4 Updating VC Static Platform and VC Formal Application Variable Settings

VC Static Platform offers a list of application variables that can be used as per your requirements. You can change the default behavior of VC Static Platform by changing the default settings of the application variables.

Note:

The formal specific variables hold true for all the formal applications (FPV, SEQ, COV and so on). You should control these variables carefully when running different applications in one console.

To see the list of all the VC Static Platform available application variables and their current settings in the `vcf`, use the following command:

```
vcf> printvar (or)
vcf> report_app_var
```

The `printvar` command reports all variables including user defined variables, while the `report_app_var` command reports only the VC Static platform application variable settings.

Example 1

```
vcf> printvar fml_*
fml_auto_save        = "default"
fml_cc_comb          = "false"
fml_cc_coverage_analyzer = "false"
fml_cc_overabstract  = "false"
fml_composite_trace  = "true"
```

Feedback

```
...
....
....
.....
```

Example 2

```
vcf> report_app_var fml_*
Variable                 Value     Type    Default    Constraints
------------------------ --------- ------- ----------
 ------------------------
fml_auto_save            default   string  default
fml_cc_comb              false     bool    false
fml_cc_coverage_analyzer false     bool    false
fml_cc_overabstract      false     bool    false
fml_composite_trace      true      bool    true
fml_cov_enable_int_unr   false     bool    false
...
....
....
.....
```

Example 3

```
vcf> report_app_var fml_composite_trace -verbose
vcf> report_app_var fml_composite_trace -verbose
Variable                 Value     Type    Default    Constraints
------------------------ --------- ------- ----------
 ------------------------
fml_composite_trace      true      bool    true
   # Turns on composite trace generation.
```

You can use the `set_app_var` command to change the setting of an application variable. The following example shows how to set a variable to true:

```
vcf>set_app_var fml_composite_trace true
```

2.5 Setting the Application modes

You must set the required application mode before you run the verification tasks. By default, all tasks are started in the FPV application mode. It is recommended that you start the program in the specified application mode that you want to use. This enables the Verdi GUI to present the appropriate view.

Use the following command to set the required application mode:

```
vcf -fmode <value>
```

where `value` can be one of `{FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV}`. If no value is specified, the FPV application mode is set.

This setting is same as the setting achieved by the `set_fml_appmode <FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV|FTA>` command in VC Formal.

To see the application mode currently set, use the `get_fml_appmode` command. This command returns the application name as set with `set_fml_appmode` command.

Note:

An error message is reported when you start VC Formal with the FTA application mode:

```
% vcf -fmode FTA
```

Error: Value of -fmode should be one of FPV, COV, SEQ, CC, AEP, FRV, FXP, FSV, FUSA, DPV

You cannot start the tool with the FTA application mode because we usually start in FPV application mode before switching to FTA. You can use the FTA application mode with the `set_fml_appmode` command.

3

Managing Constraints and Properties for Formal Analysis

The section describes the different classes of properties and constraints, and how to use them in VC Formal.

This section contains the following topics:

- About Constraints
- Types of Constraints
- Constraint Examples
- Stabilizing Constraints in Multi-Clock Designs
- About Properties
- Property Configuration Commands
- Using SVA for Formal Analysis
- Adding SVAs in the Design
- Supported SVA Syntax
- Updating Properties for Formal Analysis
- Reviewing Property Usage

3.1 About Constraints

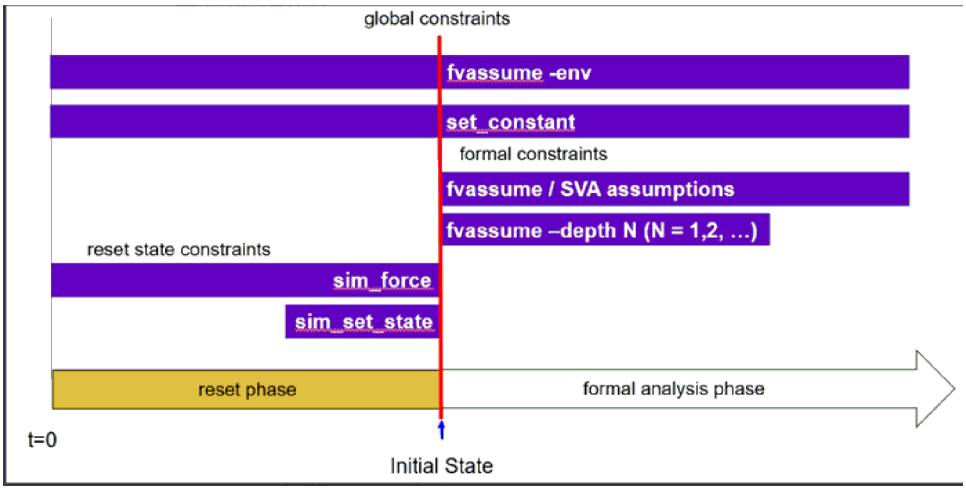
Constraints are properties that must be true for the design to operate correctly.

You can add constraints for reset state verification, for formal analysis, or for both reset state verification and formal analysis.

Figure 8 shows the sets of commands to apply constraints and how they are associated in different phases.

Feedback

Figure 8: Categories of Constraints



For more details on how to add constraints for reset verification, see section Specifying the Initial State.

Constraints for formal analysis can be added as properties in the RTL source file, checker files, or script properties using the `fvassume` command.

Syntax

```
vcf> fvassume -help
Usage: fvassume    # Creates a constraint or sets the usage attribute to
 assume and enabled
        [-class <list-of-class-attributes>]
                                (property class selection:
                                 Values: aep, connection, coverage,
                                 no_class, register, script, security,
                                 seq, source, xprop)
        [-type <list-of-type-attributes>]
                                (property type selection:
                                 Values: arith_oflow, assert, assume,
                                 boundconstraint, bounds_check,
                                 condition, conflict_driver,
                                 constconstraint, cover, covergroup,
                                 envconstraint, exclude, fault_check,
                                 floating_bus, fsm_livelock, fsm_state,
                                 fsm_transition, full_case, injectx,
                                 injectx_oba, injectx_reset,
                                 injectx_undef, injectx_xassign,
                                 internal, line, mark, multi_driver,
                                 no_type, nox, nox_reset, parallel_case,
                                 resetconstraint, set_reset,
                                 stableconstraint, structural, toggle,
                                 , x_assign)
        [-usage <list-of-usage-attributes>]
                                (property type selection:
```

Feedback

```
                                Values: assert, assume, cover)
       [-filter <class>]       (Filter expression to apply)
       [-regexp]               (Use regular-expression instead of glob
filtering)
       [-exact]                (No pattern matching performed)
       [-clock string]         (Optional clock for property. Checks
property on posedge of specified clock.)
       [-expr string]          (Expression to be checked.)
       [-global]               (Apply to all current tasks)
       [-scope instance_path] (Allow specification of signal names in
expression relative to instance scope.)
       [-col]                  (Outputs collection of modified or created
properties)
       [-env]                  (Constraint applies to both reset sequence
computation and formal analysis. The property expression is limited to a
constant, equality, or simple combinational implication)
       [-stable]               (Specifies that the signal in -expr should
be a random stable value. Expression is limited to primary input,
 undriven net, or blackbox output.)
       [-depth <integer>]      (Constraint only applies in the first <n>
cycles of the formal analysis.)
       [<list-of-names-ids-or-collections-of-properties>]
                               ( (List of property names, name patterns,
or property collections))
```

Example

You can add stable constraints in formal analysis, that is, you can constrain a signal so that it takes a random value from the beginning and remains stable throughout formal analysis.

The signal must be a primary input, undriven net, snipped net or black box output. The signal must start as a random logic 1 or logic 0, but then it must be stable after the selected initial value.

Use the following command for the input `pipe_id`, that is, `pipe_id` is either 0 or 1 in cycle 1, and remains constant thereafter.

```
fvassume const_pipeid -stable -expr {pipe_id}
```

For more information on the usage of properties as formal constraints, see sections Class Attribute and Using SVA for Formal Analysis.

You can delete constraints using the `fvdelete` command. For more information on the `fvdelete` command, see the man page of the `fvdelete` command.

3.2 Types of Constraints

You can add different types of constraints for reset and formal analysis.

3.2.1 Depth Based Constraints

The depth based constraints can be applied up to the specified depth after the reset.

Use the `-depth <n>` switch in the `fvassume` command to specify constraints that must be applied for <*n*> cycles after reset, where<*n*>can be a positive number greater than zero.

Syntax

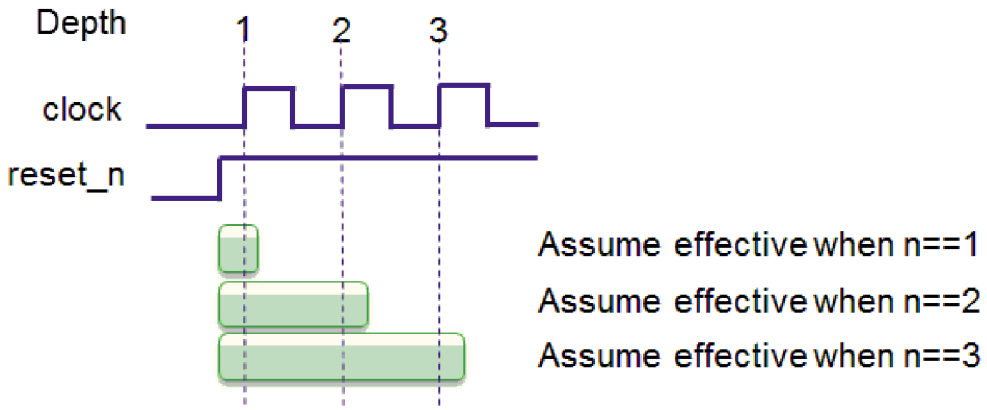
```
fvassume -expr <expr> -depth <n>
```

Example

To add a constraint state to 2'b0 for the first three cycles after reset, use the following command:

```
fvassume -expr { state == 2'b0 } -depth 3
```

Figure 9: Clock Periods that Assume is Effective



By default, the `-depth` option counts cycles with respect to the posedges of the reference clock. If you want to use a different clock, that clock can be specified using the `-clock` switch of the `fvassume` command.

The `-depth` switch requires `-expr`. The expression specified using `-expr`must be a purely combinational expression. No SVA or temporal operators are allowed and an error message is reported if the expression is not combinational.

The`-depth` switch is also mutually exclusive with the `-env, -stable`, and `-from_zero`switches.

3.2.2 Environment Global Constraints

Environment constraints always apply during the reset sequence simulation as well as during formal analysis. The property expression is limited to only primary inputs, undriven nets, snip points or black box outputs. To avoid over-constraining during reset simulation, not all constraint expressions can be used during simulation.

Note:

The `fvassume -env` command must be issued before the `sim_run` command for the constraint to take effect during reset. Otherwise, the constraint will hold only during formal analysis.

The following are some examples of environment constraints:

```
fvassume -env -expr {vld_status == 1 }
fvassume -env -expr {s3_vld == s4_vld }
fvassume -env -expr {s5_vld == vld_status }
```

Similar to `fvassume -depth` type of constraints, only certain expressions are supported.

3.2.3 Constant Constraints

Use the `set_constant` command to apply constant value for formal analysis as well as for reset state computation.

The `set_constant` command enables:

- Constant propagation to all fanout logic allowing for a simplified COI, thus improves run-time performance and capacity.
- Does not imply a property, therefore cannot be changed to an assertion.

Example

The `scan_en` should be disabled during functional verification, that is, should be 0. The following command can be used to accomplish this:

```
set_constant scan_en -value 0
```

Constraints or properties set to assume usage by `fvassume` may assign constant values to pins or nets in the design. In this respect, they appear to do the same thing as setting a constant on a pin or net using the `set_constant` command. However, there are some caveats:

- Assume properties may have their usage changed to assert.
- Assume properties may not hold true during reset state.

In the case where a signal is to remain constant during formal analysis, but a different state during reset, it is recommended to use `set_constant` combined with `sim_force` commands. If used together on the same signal, the value set by `set_constant` can be overridden by `sim_force` during the reset phase.

For example, signal SCAN_EN should be set to 1 during reset and changed to 0 during formal analysis as the following code snippet shows:

```
set_constant SCAN_EN -apply 0
sim_force SCAN_EN -apply 1
…
sim_run
sim_save_reset
```

There are some exceptions where this may not be the best thing to do. For example, if you have a primary input SSE directly driving a latch and you wish to constrain SSE to be opposite values, use `sim_force` combined with the `fvassume` command.

For example, signal SSE should be set to 1 during reset and changed to 0 during formal analysis:

```
sim_force SSE -apply 1
…
sim_run
sim_save_reset
fvassume sse0 -expr {SSE == 0}
```

Otherwise, for reset and formal analysis, you may run into falsifications caused by the mismatch of the initial state.

3.3 Constraint Examples

The following are some examples for adding constraints for formal analysis.

3.3.1 Random but Constant Constraint

Input A must be random in the first cycle and remain constant thereafter. This can be done using a user written property in SVA or a script property.

Example in SVA

```
a_input_stable : assume property (@(posedge clk) ##1 $stable(A)) ;
```

Example in script property

```
fvassume -stable -expr {A}
```

3.3.2 Simple Combinational Constraints

In the case where the input constraint must always hold, an immediate assertion must be used rather than a concurrent assertion:

Input A must not be asserted at the same time as input B.

Example in SVA

```
always @(*)
 a_inputa_mutex : assume (! (A&B)) ;
end
```

Example in script property

```
fvassume -expr { !(A & B) }
```

3.3.3 Legal Value Selector

Input must be one of a set of legal values.

Example in SVA

```
a_legal_op : assume property (@(posedge clk)
OP==`OPCODE_ADD || OP==`OPCODE_SUB
|| OP==`OPCODE_MUL || OP==`OPCODE_DIV);
```

Example in script property

```
set OPCODE_ADD 0
set OPCODE_SUB 1
set OPCODE_MUL 2
set OPCODE_DIV 3
fvassume -expr { @(posedge clk) OP==$OPCODE_ADD || OP==$OPCODE_SUB
|| OP==$OPCODE_MUL || OP==$OPCODE_DIV }
```

or

```
fvassume -expr { OP==$OPCODE_ADD || OP==$OPCODE_SUB
|| OP==$OPCODE_MUL || OP==$OPCODE_DIV } -clock clk
```

3.3.4 Handshaking

Asserts the `data_valid` signal until the design acknowledges the data.

Example in SVA

```
a_hand_shake : assume property (@(posedge clk) disable iff (reset)
(data_valid && !data_ack) |=> (data_valid && $stable(data)));
```

3.3.5 Packet Generation

A random length sequence of bytes with sop (start of packet) and EOP (end of packet) signals. The packet length is between eight and 255 bytes.

Example in SVA

```
logic[7:0] counter;
always @(posedge clk)
counter <= reset? 0 : (sop==1 ? (data-1) : (counter!=0 ? (counter-1) :
 0));
a_packet_length : assert property (@(posedge clk) sop |-> (data > 8 &&
 data < 255));
a_assert_eop : assert property (@(posedge clk)
((counter==0 && $past(counter)!=0) |-> eop));
a_dessert_eop : assert property (@(posedge clk)
(!(counter==0 && $past(counter)!=0)) |-> !eop);
a_sop : assert property (@(posedge clk) (counter!=0 || eop) |-> !sop);
```

3.3.6 Accessing Signals Inside SV Interface

The following example shows the command to list out the signals of the foo_i instance of an interface:

```
vcf > get_nets -word foo_i.*
{"foo_i.waiting", "foo_i.t1", "foo_i.t4"}
```

The following example illustrates how to add a constraint on all the signals in the interface, like

```
vcf > foreach_in_collection sig [get_nets -word foo_i.*]
{ fvassume -stable -env -expr [get_attribute $sig name] }
```

Reproducer:

```
typedef struct packed
{      logic first;      logic dummy;       }
first_t;
typedef struct packed
{       logic main;       logic mummy;       }
main_t;
typedef struct packed
{         main_t main;         first_t first;         }
waiting_t;
interface foo ();
waiting_t waiting;
   logic t1;
   logic [3:0] t4;
endinterface
module …
```

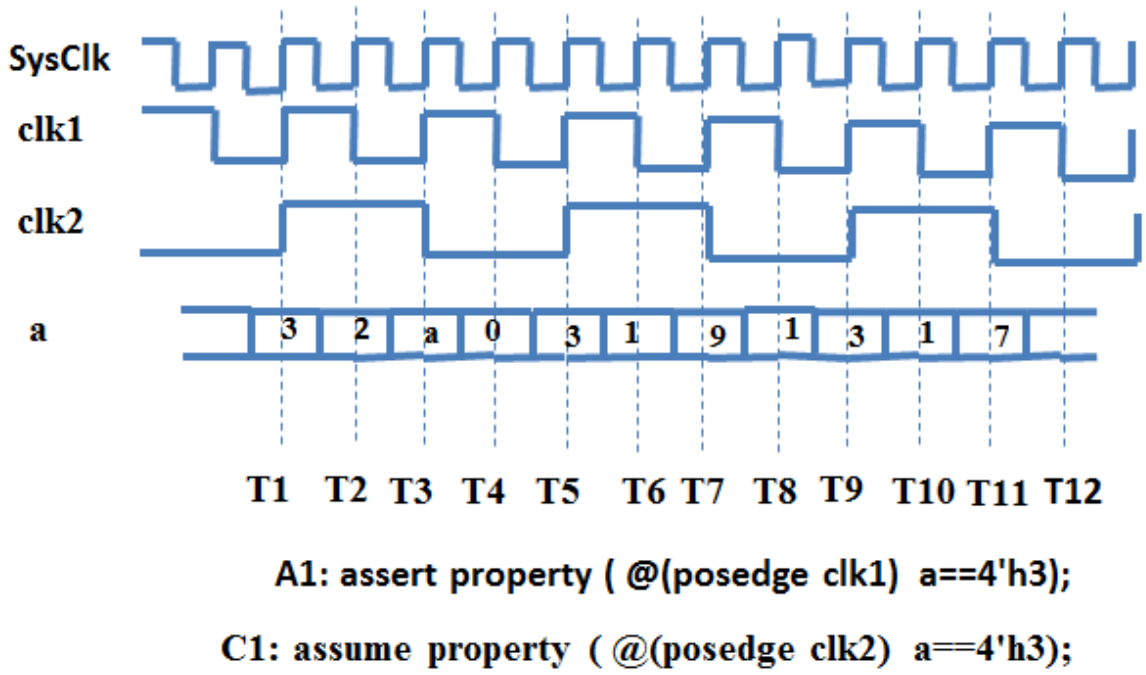
```
foo foo_i();
endmodule
```

3.4 Stabilizing Constraints in Multi-Clock Designs

In some cases, particularly in multi-clock designs with different frequencies, using SVA assert/assumes on these different clock events, VC Static Formal processes constraints in ways that are difficult to understand.

Figure 10 illustrates a simple example of the problem, using a design with two clocks, including an assume using the slow clock, and an assert using the fast clock.

Figure 10: Need for Stability Constraint Between Edges of Active Clocking



As illustrated in Figure 10, signal a has been specified to have the value 'h3 at every posedge of clock clk2 by the constraint C1. Further, the assertion A1 verifies that a has the value 'h3 on every posedge of clk1. Two user clocks have been defined, with clk1 being twice the frequency of clk2.

In Figure 10, the formal model changes state at T1, T2, T3, etc. At each of these state-change points, the formal engines first determine legal input changes given the current state, then compute the next state. When representing this waveform in Formal, the input changes are shown to change value well before the active system clock edge. In fact, they are seen to change on the negative system clock edges.

In SVA, the assume is required to be true in the prepone region of the clocking expression. So, you can see that signal a always has value ‘h3 prior to posedge clk2. Intermediate state-change points are unconstrained between posedge clk2 points. Therefore, signal a has value ‘h3 at times T1, T5, and T9, and at all other times, it has different values. More specifically, at active clock events for the assert statement, at times T3, T7, and T11, the assertion is violated.

You can use the following commands to specify and report on certain signals that should be “stable” in between certain clocking events. This lets you define clocking events at which the signal is allowed to change.

- `set_change_at`
- `report_change_at`
- `get_change_at`

Example

```
asm_2 : assume property (@(posedge clk2)
disable iff (!rst_x)
b!=4'hf |=> b==$past(b)+1'b1);
ast_2_0 : assert property (@(posedge clk1)
disable iff (!rst_x)
##1 !$stable(b) |=> $stable(b)[*1]);
```

With clocks defined as follows, the assertion fails because b can change between posedge clk2 when it is checked by the assertion.

```
create_clock clk1 -period 100
create_clock clk2 -period 200
```

The assertion passes when changes are restricted on b to only happen at posedge clk2 edges:

```
set_change_at -clock clk2 b
```

Note:

It is important that you exercise caution while using these stabilization mechanisms as they can over-constrain the design and result in false proofs or even false negatives at times.

3.4.1 Reevaluating Results When Constraints are Modified

When constraints are added or removed, the preexisting property results are selectively invalidated. Signals involved in the `set_change_at` command are seen as either adding or removing a constraint, and data are invalidated accordingly. When adding a constraint, falsifications are invalidated. When removing a constraint, proofs are invalidated.

3.4.2 Interactive Behavior of set_change_at

The following four types of signals can be stabilized:

- Primary inputs
- Black box outputs
- Snipped signals
- Undriven nets

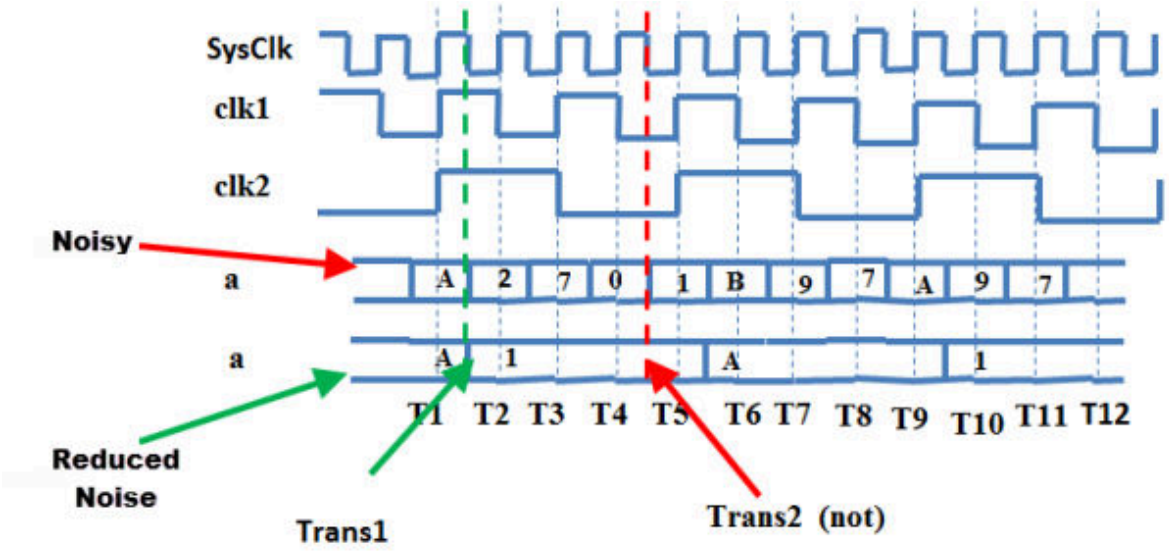
Data associated with `set_change_at` might be changed interactively, if a signal changes such that it is no longer valid for the `set_change_at` command, an information message is issued and the `set_change_at` command is marked as invalid in the `report_change_at` output. An example might be if a snip point removed.

In the cases where previous `set_change_at` command signals are no longer valid, VC Formal attempts to re-validate the constraint at each `check_fv` command startup. If you do not wish to not see this revalidation attempt, the constraint can be removed using the `set_change_at -remove` command.

3.4.3 Signal Timing

Figure 11 illustrates the original waveform for signal a that is very noisy. Correct values according to the constraint C1 occur, but many other transitions of the signal a are unconstrained, leading to violations of the assertion A1. A new value for signal a is chosen on every system clock cycle, and shown aligned with the negedge of the system clock.

Figure 11: Corrected Waveform with Timing




```
C1: assume property (@(posedge clk2) (a == 4'hA) || (a == 4'h1));
A1: assert property ( @(posedge clk1)  ((a == 4'hA) || (a == 4'h1)));
 change_at –clock clk2 –signals a
```

The second waveform for signal a shows a much less noisy waveform where the assertion is now met at all times. The transition of the aligned signal with respect to the clock (signal a, clock clk2) is chosen in the first system clock cycle after each clocking event. In effect a very long setup time is provided. The alternative is to choose the transition in the system clock period immediately prior to the clocking event.

3.5 About Properties

Properties are essential for formal analysis. Properties are statements about the expected behavior of the design under test or the environment around it. They describe temporal or boolean relationships of signals or groups of signals.

There are three kinds of properties verifying different aspects of a design:

- Properties that verify the behavior of the DUT are referred to as asserts, assertions, checker properties.
- Properties that describe the legal behavior of the inputs to the DUT, that is, the environment, are referred to as assumes, assumptions, or constraints.
- Properties that monitor specific DUT behaviors are referred to as cover properties. Each property object has attributes which designate static information such as class or type and dynamic information such as status or usage.

Each property object has attributes which designate certain static information (eg “class” or “type”) as well as certain information which is dynamic (“status” or “usage”). These attributes may all be accessed with the `list_attribute` command, and may be used in filter syntax with a variety of commands which access property objects.

This section describes the key property attributes and values. For a complete list of property attributes use: `list_attribute -application -class property`.

- Class Attribute
- Type Attribute
- name Attribute
- usage Attribute
- Status Attribute
- Other Property Attributes

3.5.1 Class Attribute

Properties are categorized into different classes based on how they are created. VC Formal supports the following classes of properties:

- *AEPs* - Automatically extracted properties generated by the tool.
- *Coverage properties* – Line, condition, toggle, fsm state, or fsm state transition coverage analysis properties generated by the tool.
- *Script properties* – Simple user written properties using Tcl commands.
- *Source properties*– User written properties in SVA or OVL in input design files.
- *Connectivity properties* – User created properties from the `add_cc` or `load_cc` commands. For more information, see section Connectivity Checking Application.

3.5.2 Type Attribute

The type attribute differentiates between properties within a class. This attribute indicates information at the property creation time and cannot be changed after creation. The following type attribute values are used:

- *assert, assume, cover*: These property types are associated with the source class. The type indicates if the property was declared using the assert, assume or cover language keyword. These properties are created at design compile time, when the `-sva` option is used with either the `read_file` or `elaborate` commands.
- *assert, assume, cover, envconstraint, boundconstraint, stableconstraint*: These property types are associated with the script class. These properties are created using `fvassert, fvassume` or `fvcover` commands. The type indicates with which command the property was created. In the case of the `fvassume` command, use of the `-env, -depth`or `-stable` options causes `envconstraint, boundconstraint` and `stableconstraint` respectively.
- *connection:* A structural type property is associated with the connection class. A structural connectivity property is created with the `add_cc` or `load_cc` commands. For more information, see section Connectivity Checking Application.
- *arith_oflow, bounds_check, conflict_driver, floating_bus, full_case, multi_driver, parallel_case, set_reset, x_assign*: These property types are associated with the AEP class. They are created when the`-aep` switch is used with the `read_file` or `elaborate` commands.
- *condition, fsm, line, tgl, fsm, fsm_state, fsm_transition:*These property types are associated with the coverage class. They are created at design compile time, with the `-cov` switch for the `read_file` and `elaborate` commands.

Automatically Extracted Properties

AEPs are properties generated by the tool after analyzing the design. VC Formal extracts AEPs from the design and creates specially instrumented RTL signals that can be asserted or de-asserted in the formal models.

The AEP properties are loaded only in the AEP application mode. You must set the application mode to AEP to access the AEP properties. Use the `read_file` or `elaborate`command to enable AEPs.

Example

- To enable both arithmetic overflow and `x_assign` AEP checks, use the following code snippet:

```
read_file -aep arith_oflow+x_assign -sva -top $top -format verilog
 -vcs "-sverilog -f demo.files"
```

- To enable all of the AEP checks, use the following code snippet:

```
read_file -aep all -sva -top $top -format verilog -vcs "-sverilog -f
 demo.files"
```

Coverage Properties

Coverage properties are generated by instrumenting goals from line, condition, toggle, fsm state and fsm state transition metrics in the same form as from the vdb coverage database. VC Formal performs coverage analysis to determine whether these goals are reachable or unreachable.

The coverage properties are loaded only in the COV application mode. You must set the application mode to COV to access the coverage properties. Use the `read_file` or `elaborate`command to enable coverage properties.

Example

- To enable both line and condition coverage checks, use the following code snippet:

```
read_file -cov line+cond -sva -top $top -format verilog -vcs
 "-sverilog -f demo.files"
```

- To enable all of the coverage checks, use the following code snippet:

```
read_file -cov all -sva -top $top -format verilog -vcs "-sverilog -f
 demo.files"
```

For more information on the types of coverage checks, see section Formal Coverage Analyzer Application.

Source Properties

User written source properties are located in the same modules as the design or in separate files and bound to the design. The source properties are declared in the assert, assume, or cover property types in the source file definition. However, the source properties that are assume properties in the RTL are automatically copied as constraints to all the other application modes.

The source properties are loaded only in the FPV application mode. You must set the application mode to FPV to access the source properties.

Use the following commands to specify the usage of the source properties: `fvassume, fvassert, fvcover, fvenable` and `fvdisable`.

- `fvassume:` This command sets the usage attribute to assume, that is, it identifies and/or creates properties that should be used for assumes (constraints) during formal analysis.
- `fvassert:` This command sets the usage attribute to assert, that is, it identifies and/or creates properties that should be used for assertions during formal analysis.
- `fvcover:` This command sets the usage attribute to cover, that is, it identifies and/or creates properties that should be used for cover targets during formal analysis.
- `fvenable:` This command is used to enable properties or property subtype goals, without modification of the usage attribute. This command selects a collection of properties or property subtype goals, based on user directives, and sets the enabled attribute to true. This command returns 1 for success, 0 for failure; optionally, a collection of selected properties is returned.
- `fvdisable:` This command is used to disable properties or property subtype goals, without modification of the usage attribute. This command returns 1 for success, 0 for failure; optionally, a collection of selected properties are returned.

Script Properties

Script properties offer the advantage and convenience to debug and explore the behavior of the design without having to recompile. Use the `fvassert, fvassume` and `fvcover` commands to create script properties using simple expressions from the Tcl command line.

Script properties are loaded only into the current application mode, unless you specify the `-global` option when the property is created.

Examples

- You can add assert, assume and cover script properties interactively or in a command file:

```
fvassert property_name -expr <expression>
fvassume property_name -expr <"expression">
fvcover property_name -expr <"expression">
```

  The `property_name` switch is optional. If used, it must be unique within the scope of all properties. If the `property_name` is not specified, an internally generated name `script_prop_N` is used. N is the number assigned to that script property.

- Script properties support built-in functions. The following example states gnt3, gnt2, gnt1 and gnt0 must be 0 one hot:

```
fvassert gnts_oh0 -expr {$onehot0\(\{gnt3,gnt2,gnt1,gnt0\}\)}
```

- Expression syntax supports simple Verilog operators, vector or scalar signals and certain SystemVerilog constructs and system functions. For example, an RTL contains the following declaration:

```
wire reset_error = !rst_n && (last_gnt != 2'b00);
```

- To create an assertion called `rst_err` to check that `reset_error` never occurs:

```
fvassert rst_err -expr "reset_error==0"
```

- If the `reset_error` signal does not exist in the RTL, the simple expression given above can be replaced with the following command:

```
fvassert rst_err_exp -expr "(!rst_n && (last_gnt != 2'b00)) ==0"
```

- Two newly created properties can be manipulated in the same way as properties written in SVA. For example, they are retrieved by the `get_props` command and they have the type attribute set to assert since they were created using the `fvassert` command:

```
vcf> get_props -type assert
{"assert_no_gnt_if_stall", "rst_err", "rst_err_exp"}
```

  The only difference is in the class attribute as shown in the following code snippet:

```
vcf> get_attribute [get_props rst_err] class
script
vcf> get_attribute [get_props assert_no_gnt_if_stall] class
source
```

- The script properties can have their usage attribute changed. For example, they can be used as assumptions.

```
vcf> fvassume rst_err
```

- The script property can also be created from activity view in the GUI.

  Note:

  The use of parameters and certain sequential operators and functions are not supported yet. In each case, the clock to be used can be specified when the expression is defined. By default, the reference clock is used. User-defined macros are not recognized by script properties.

Vacuity Properties

When properties use overlapping (|->) and non-overlapping (|=>) implication operators, additional properties called vacuity properties or vacuity checks are automatically generated to check that the antecedent (or precondition) expression is actually coverable. These properties are equivalent to a cover property checking that the antecedent expression can be covered.

In the case when the antecedent expression is uncoverable, the main property is considered vacuous and is logically proven since no falsification trace can be found. In VC Formal, when vacuity properties are enabled, the status of the main property becomes vacuous instead of proven if the associated vacuity check is uncoverable.

Examples

Consider an assert property written for the following condition:

VALID must remain asserted until READY

```
assert property (@(posedge CLK) (VALID && !READY) |=> VALID);
```

The antecedent expression for this property is “(VALID && !READY)” and the vacuity property will check if that expression is coverable similar to the following cover property:

```
cover property (@(posedge CLK) (VALID && !READY));
```

Witness Properties

In addition to vacuity properties, another way to check for potential issues (tautology, inconsistencies, setup issues, and so on) with target properties is to enable witness properties or witness checks. These properties, are equivalent to a cover property, which checks that the main property expression (antecedent and consequent) can be covered. In other words, witness properties check if there exists a good scenario where the main property does not fail, thus allowing for such trace to be displayed.

In the case where the expression is uncoverable, there is no scenario where the target property is passing, thus indicating that there is potential issue that should be investigated and addressed.

Example

Consider an assert property written for the following condition:

VALID must remain asserted until READY

```
assert property (@(posedge CLK) (VALID && !READY) |=> VALID);
```

The witness property will check if the property expression is coverable, similar to the following cover property:

```
cover property (@(posedge CLK) (VALID && !READY) ##1 VALID);
```

Connectivity Properties

For information on the setup, usage and running of connectivity properties, see section Connectivity Checking Application.

Initial Safety Assertions

According to the LRM, the initial assertions execute only a single evaluation attempt starting at the first clock tick. In simulation, it is the first tick after or at time 0. In VC Formal, the first tick of the leading clock of the assertion is after the reset process is completed. Typically, the usefulness of initial assertions is as assumptions, for example, to constrain signals for several initial cycles:

initial Asm: `assume property(@(posedge clk) a[*3] #=# always !a);`

Initially a is constrained to be 1 for 3 cycles and thereafter 0 forever.

Script properties do allow specifying -fromZero qualifier, which is not available for assertions that are part of the design and compiled by SVA Compiler.

VC Formal supports initial assertions for both liveness and safety properties, and allows you to specify such assumptions.

You can write initial assertions in the design or as script properties for both liveness and safety forms. This applies to assume, assert, cover and vacuity checks. For example, a liveness initial assert property may have a safety witness and vacuity. A safety initial `Asrt: assert property (@(posedge clk) a [*3] #=# always !a)` has as liveness cover or witness because the success trace has an infinite suffix showing that `!a` is always true. There is no vacuity check in this case.

Initial assertions are supported for all supported forms of properties, including multi-clock, complex clock and local variables.

Examples

- `initial A1: assert property(@(posedge clk) a |=> s_eventually !a);` // if a is 1 at the first clock tick, then in the future it will become 0 at least once.
- `initial A2: assert property(@(edge clk) a ##1 a);` // at the first and 2nd clock edges *a* is required to be 1, it can take any value thereafter.

3.5.3 name Attribute

All properties have names which are unique within the scope of all properties. The name is static and set at property creation time. In the case of unnamed SVA properties, VCS naming conventions are used.

3.5.4 usage Attribute

The usage attribute is dynamic, and indicates how the property will be used in subsequent verification. Values of the usage attribute are:

- `assert`: The property will be used as an assertion or checker.
- `assume`: The property will be used as an assume or constraint.
- `cover`: The property will be used as a coverage goal.

The usage attribute may be changed, usually with the commands fvassert, fvassume, fvcover, .

By default at creation time, all properties are enabled as one or the other of the above usages. SVA properties are enabled based on usage in the source files. Script properties are enabled based on which command creates it: fvassume, fvassert, fvcover. Properties created with add_cc or load_cc are set to use as an assertion.

3.5.5 Status Attribute

For properties with usage either as assertion or cover, the status attribute indicates current verification status. The following status attribute values will be seen:

- `no_status`: This is any property to be used as a constraint or unused.
- `not_checked`: This is for a property used as an assertion or coverage, before any analysis task has processed the property.
- `paused or stopped`: This is for an assertion which was being checked at the time the check_fv –pause command was used. When the check is resumed, the verification will continue.
- `bounded`: This status is for an assertion with bounded proof data.
- `falsified`: This is for a property used as an assertion, after the assertion has been falsified. A counter-example trace exists.
- `proven`: This is for a property used as an assertion, after the property has been proven true.
- `timed-out`: This is for a property used as assertion or cover, after resource limits have been exhausted. VSI will no longer work on this property.
- `covered`: This is for a property used as a cover after a witness trace has been discovered.
- `uncoverable`: This is for a property used as a cover after it has been proven that the property cannot be covered.

3.5.6 Other Property Attributes

There is a set of less complicated property attributes of potential interest to script writers. Those attributes are listed here.

- `bounded_proof_depth`: This attribute indicates bounded proof depth of a property with assertion usage. Depth is -1 before checking has started and if status is proven.
- `debug_signals`: This attribute is a space delimited list of signal names in the support of the property.
- `elapsed_time`: A string indicated elapsed wall time since the property checking has started.
- `expression`: A string attribute for the property expression.
- `finish_time`: Wall clock time indicating what final status was achieved for the property.
- `instance`: Owning instance name for the property.
- `language`: This attribute indicates language of the property. Typically SVA, PSL, or script.
- `signals`: Similar to debug_signals attribute.
- `snip_input` : A boolean attribute indicating if all inputs to the property were snipped. Values are true or false.
- `start_time`: Wall clock time when property checking started.
- `trace`: This is a string indicating internal pathname to the trace file. Only valid if the property was falsified or covered.
- `vacuity`: If vacuity checks are enabled, the attribute indicates status of the vacuity check for the property. Values are checking, no status, not_checked, paused, reached, stopped, timed_out, unreachable.
- `vacuity_safe_depth`: If vacuity checks are enabled, this attribute indicates the depth of the safety property.
- `vacuity_trace` : This is a string indicating internal pathname to the vacuity trace file. Only valid if the vacuity is reached.
- `witness`: If witness traces are enabled, this string attribute indicates status of the witness trace for a source type assertion. Values seen will be: checking, not_checked, paused, reached, stopped, timed_out, unreachable.
- `witness_safe_depth`: If witness checks are enabled, this attribute indicates the depth of the safety property.

- `witness_trace` : This is a string indicating internal pathname to the witness trace file. Only valid if the witness is reached.
- `converge_time`: The time taken by the leaf engine that leads to the convergence of the property.

3.6 Property Configuration Commands

By default, when a property is created, its usage attribute is set so that verification can proceed without further property setup. By default:

- All SVA properties are enabled based on their declaration in the source code.
- All connectivity checks, added with add_cc or load_cc are enabled by default.
- All script properties, created with fvassert, fvassume, or fvcover are enabled.

Further customization of properties – perhaps using as a constraint a SVA assert property – requires use of one of the following commands.

When script properties are created with the following commands, clocking semantics will use the reference clock. Refer to the discussion on create_clock for more information on reference clocks.

Clocking Behavior

VC Formal parses the potential clocking event in the expression in the following ways:

- If the property has a clocking event, the property's clock is used.

```
fvassert -expr {@(posedge clk) (a |-> b)}
```

- If you specify a clock using the`-clock` switch, then a warning message is printed, and the clocking event in the expression syntax is used.

```
fvassert -expr {@(posedge clk) (a |-> b)} -clock clk2
```

- If no clocking expression is specified, then thereference clock is used.

```
create_clock clk -period 100
fvassert -expr {a |-> b}
```

3.6.1 Script Property Expression Syntax

The fvassert, fvassume, and fvcover commands can be used to create simple properties from the tcl command line. Each of these properties supports a syntax for these "Script" type properties. These commands are described later in this document, including how this syntax is used.

Certain sequential operators and functions are supported. In each case, the clock to be used can be specified when the expression is defined; by default, the reference clock is used.

Expression syntax supports simple Verilog operators, vector or scalar signals, and certain SystemVerilog constructs and system functions. Supported simple operators are:

- `{}`: Concat operator
- `!, &&, ||, ==, !=`: Logical operators
- `~, &, |, ^, ~^` : bitwise operators
- `<, >, <=, >=` : relational operators
- `+, -, *, /` : Simple mathematical operators (integer, unsigned arithmetic)
- `|->` : Overlapping implication
- `|=>` : Non-overlapping implication. In this case, the reference clock is used for sampling semantics.

Operator precedence is left-to-right. Parentheses are supported for more control of precedence.

Builtin functions include:

- `$onehot(<expr>)`: True if one bit of the expression is logic 1, and all others are logic 0.
- `$onehot0(<expr>)`: True if zero or one bit of the expression is logic 1, and all others are logic 0.
- `$countbits(<expr>)`: Returns count of the total number of bits of the expression.
- `$countones(<expr>)`: Returns count of the total number of bits of the expression which are logic 1.
- `$isunknown(<expr>)`: Boolean function that returns true if any part of the expression is logic X. This function returns false always in model checking usages.
- `$signed(<expr>)`: Treat the expression as a signed value.
- `$unsigned(<expr>)`: Treat the expression as an unsigned value.
- `$stable(<stableExpr>)`: Stability function. True if all bits of <stableExpr> do not change.
- `$rose(<expr>)`: Boolean function which returns true if expression is seen to rise in two consecutive clock cycles.

- `$fell(<expr>)`: Boolean function which returns true if expression is seen to fall in two consecutive clock cycles.
- `$past(<expr>[, <#cycles>)`: The $past function returns the past value of <expr>, <#cycles> in the past. By default, <#cycles> is 1. The 2nd parameter is optional.

All sampled-value functions are sampled on the user's reference clock, or on a specific clock (-clock option) specified in the command. This includes functions: $stable, $rose, $fell, and $past. Also note that these functions are more limited in features in some cases than the corresponding SVA functions (for example $past does not allow a gating expression).

3.7 Using SVA for Formal Analysis

You can use SVAs as a declarative method to develop properties that you can be used as checkers or constraints in VC Formal. This section explains how to use SVAs.

VC Formal by default supports IEEE SV LRM 2009. There is one fundamental difference between SV LRM 2009 and SV LRM 2005. In SV LRM 2005, `##[1:$]` is strong for property and weak for cover, but LRM was changed at SV LRM 2009 with opposite. Therefore, weak properties that could fail in SV LRM 2005 will now become always passing because a weak property cannot fail in SV LRM 2009. In this case, the property becomes a tautology.

- If you are unsure whether you have properties written in SV LRM 2005, you should look for messages like this:

  Warning-[SVAC-TAUT-PROPERTY] Property is a tautology . Property is a tautology if it is used as an assertion or an assumption. Please correct it and try again. file: ./assertion/traffic.sva, line: 85, property: assume_waiting_might_happen_first

  In cases like this, you should correct the property indicated in the message to be compliant with SV LRM 2009 by adding `strong` or `s_eventually` key word.

  For example:

```
assume_waiting_will_happen_first: assume property ( @(posedge clk)
disable iff (rst) ~waiting_first |=> ##[0:$] (waiting_first));
```

  should be changed to:

```
assume_waiting_will_happen_first: assume property ( @(posedge clk)
disable iff (rst) ~waiting_first |=> s_eventually(waiting_first));
```

  or

```
assume_waiting_will_happen_first: assume property ( @(posedge clk)
disable iff (rst) ~waiting_first |=> strong ##[0:$] (waiting_first));
```

- During SVA compile, if you encounter the following message:

  Warning- [SVAC-TAUT-PROPERTY]] Property is a tautology about parameter check assertion.

  The message indicates that the property cannot fail because of parameter setting. For example, consider that an SVA property is coded as follows:

  ```
  property p_snps_axi3_param_wr_max_bursts @(posedge clk) disable iff
  (rst) (WR_MAX_BUTSTS >= 1);
  ```

  In this case, the WR_MAX_BURSTS is a parameter. The parameter value has been set to 0 in the bind, therefore, the property cannot fail. You should check for the parameter used in the property to verify if the wrong parameter value is used.

3.7.1 Using SVAs from the OVL and VCS sva_cg Library

VC Formal supports the OVL and VCS sva_cg library. The OVL and VCS sva_cg library is a collection of modules/interfaces that can used for RTL verification, including simulation and formal verification.

The following are the characteristics of the OVL and VCS sva_cg library:

- Based on IEEE SV LRM 2005
- Each library element is encapsulated in a checker, and the checkers are wrapped in a module/interface.
- The module must be instantiated or bound as such.
- Most of the checkers in the library are clocked.
- The modules are parameterized by argument type and size.

To use predefined assertions from the OVL and VCS sva_cg library, add the following VCS compile options in `read_file`.

Example

```
set SVA_LIB [getenv VC_STATIC_HOME]
read_file -format verilog -sva -vcs "-y $SVA_LIB/vcs-mx/packages/sva_cg
 +libext+.sv +incdir+$SVA_LIB/vcs-mx/packages/sva_cg +define+ASSERT_ON "
```

Note:

All checkers are suitable for formal analysis except the `assert_driven`, `assert_proposition` and `assert_memory_async` checkers. These checkers are not compatible with formal analysis because unsynthesizable constructs are used and a formal model cannot be built.

3.7.2 Using SVAs from the SVA Checker Library

VC Formal supports the SVA checker library. The SVA checker library supplies a collection of checker entities that can be used for RTL verification, including simulation and formal verification.

The following are the characteristics of the SVA checker library.

- It is based on the IEEE SV LRM 2012.
- Each library element is encapsulated in a checker entity, and the checker entity is wrapped in a macro.
- The library contains both clocked and unclocked checker entities.
- The checker entities can be instantiated within or out of the procedural code.
- The checker entity instantiation is simple and concise, and default values for arguments can be applied.
- The checker entity arguments have uniform syntax and similar organization.
- The checker entities can be extended by passing sequences and properties.

To use predefined entities from the SVA checker library, add the following VCS compile options in the `read_file` command.

Example

```
read_file -sva -top top -vcs "$testDir/test9.v -sverilog -assert
svaext+checker +incdir+$VC_STATIC_HOME/packages/SVACheckerLib +define
+SVA_GLOBAL_CLOCK
```

Note:

The +define+SVA_GLOBAL_CLOCK is required for checkers with global clocks. You must define global clocks somewhere in the design and map its clocking event to a clock in the Tcl file.

Types of Macros

The following types of macros can be added for RTL verification:

- Combinational: These are library macros which are unclocked, and they check boolean properties.
- Sequential: These are library macros which are clocked, and they check either boolean or temporal properties.

Defining Macros for Different Properties

The following are the naming conventions of the macros for different properties that must be followed:

- *Assertions macros*: These macros check if the design fulfills specified behavior.

  The assertion macro names must start with `ASSERTS_*` for concurrent (or sequential), and `ASSERTC_*`for boolean.

  For example, `ASSERTC_MUTEX, ASSERTS_MUTEX`

- *Assumption macros*: These macros specify the behavior of the design environment.

  The assumption macro names start with `ASSUMES_*` for concurrent and `ASSUMEC_*`for boolean.

  For example, `ASSUMES_MUTEX`and `ASSUMEC_MUTEX`

- *Positive coverage macros*: These macros check if the specified behavior is exhibited during design testing.

  The cover macro names start with `COVERS_*` and `COVERC_*` for positive sequential and positive boolean covers.

  For example, `COVERC_MUTEX` and `COVERS_MUTEX`

- *Negative coverage macros*: These macros check if the negation of the specified behavior is exhibited during design testing.

  The cover macro names start with `NOT_<name>_COVERS` and `NOT_<name>_COVERC`for negative boolean covers (cover failures of assertion).

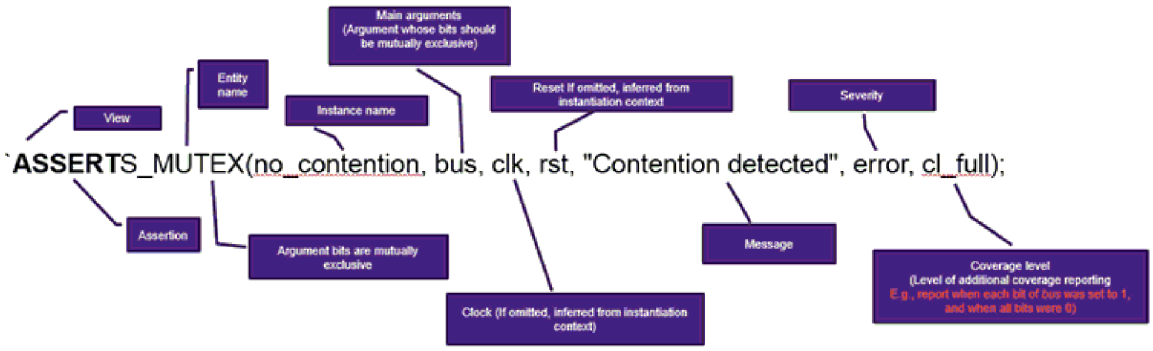
  For example, `NOT_MUTEX_COVERC` and `NOT_MUTEX_COVERS`

Syntax of Macro

Figure 12 illustrates the syntax of an SVA library macro.

Feedback

Figure 12: Syntax of a Macro



The following code shows an example where the *ASSERTS_MUTEX*macro is instantiated.

```
`include "sva_lib.h"
module m (logic clk, rst, …);
default disable iff rst;
logic read, write;
//…
always @(posedge clk) begin
read <= …;
write <= …;
`ASSERTS_MUTEX(rw_conflict, {read, write});
end
endmodule : m
```

Note:

For all the macros, the disable if the condition is inferred from the default disable statement, and the clock is inferred from the always procedure.

The following are examples of few other macros.

- `` `ASSERTS_EVENTUALLY_HOLDS(ASRTSevHAllArgs, a ##1 b, b, posedge clk, rst, "FAILED", error, cl_none); ``
- `` `ASSERTS_GRAY_CODE(ASRTSgCAllARgs, vect, posedge clk, rst, "FAILED", error, cl_none); ``

Entities Available in the SVA Library

The following entities are available with the SVA Library.

Note:

If you have global clocks defined in the design, and you want to define macros that use these global clocks, then you must prefix (G) in the name of the macro.

Table 2: Boolean Entities

Name	Arguments	Description
AT_MOST_BITS_HIGH	sig, n	At most n bits of sig are high
AT_MOST_BITS_LOW	sig, n	At most n bits of sig are low
BITS_HIGH	sig, n	Exactly n bits of sig are high
BITS_LOW	sig, n	Exactly n bits of sig are low
FIRE		Fire error message
FORBIDDEN	cond	Condition cond never occurs
KNOWN_AND_DRIVEN	sig	No bit of sig has a value X or Z
MAX	val, max_val	val <= max_val
MIN	val, min_val	val >= min_val
MUST	cond	The value of cond is always true
MUTEX	sig	At most one bit of sig is high
ONE_HOT	sig	Exactly one bit of sig is high
RANGE	sig, low, high	low <= sig <= high
SAME	siga, sigb	siga = sigb
SAME_BITS	sig	All bits of sig hold the same value

Table 3: General Temporal Entities

Name	Arguments	Description
BEFORE_EVENT	en, first, second	After match of en, first happens before second
BETWEEN	start_ev, end_ev, cond	cond holds between start_ev and end_ev
BETWEEN_TIME	trig, start_time, end_time, cond	After trig, cond holds between start_time and end_time
CLOCK_TICKING	clock	clock is ticking
CONT_REQ_GRANTED	req, gnt	Request remains pending until granted

Table 3: General Temporal Entities (Continued)

Name	Arguments	Description
DATA_TRANSFER	start_ev, start_data, end_ev, end_data	Data is transferred correctly
DELAYED_TRIGGER	trig, delay, prop	prop satisfied delay clock ticks after trig
EVENTUALLY_HOLDS	en, prop	Once en happens, prop eventually holds
GRAY_CODE	sig	Two successive values of sig differ in only one bit
NEVER	scenario	Scenario never occurs
NEXT_EVENT	en, ev, prop	Starting from en, the next time ev holds, prop holds as well
RECUR_TRIGGERS	trig, n, cond	cond holds after at most n repetitions of trig
REQ_GRANTED	req, gnt	Request is eventually granted
REQ_GRANTED_WITHIN	req, min, max. gnt	Request granted within min and max
TRIGGER	trig, prop	prop is satisfied at the match of trig
UNTIL_STRONG	start_ev, cond, end_ev	cond should hold from start_ev until next_ev (not including)
VERIFY	prop	prop is true

Table 4: Stability Entities

Name	Arguments	Description
GSTABLE_BETWEEN_TICKS	sig	sig is stable between clock ticks
(G)REMAIN_HIGH	sig, n	sig remains high during n clock ticks
(G)REMAIN_HIGH	sig, n	sig remains low during n clock ticks
(G)STABLE	sig, start_ev, end_ev	sig is stable between start_ev and end_ev
(G)STABLE_AFTER	sample, sig, clks_after	sig keeps same value after sample during clks_after clock ticks
(G)STABLE_FOR	sig, n	sig keeps same value during n clock ticks

Table 4: Stability Entities (Continued)

Name	Arguments	Description
(G)STABLE_WINDOW	sample, sig, clks_before, clks_after	sig keeps same value during clks_before clock ticks before sample and clks_after after it

3.8 Adding SVAs in the Design

You can use the following three ways to add the SVA library checkers or user-written assertions in your design:

- Instantiate the SVA
- Bind the SVA
- Add the SVA Inline

3.8.1 Instantiating the SVA

You can instantiate an SVA as shown in the following example.

```
sva_checkers sva_checkers_inst ( clk, reset, input_ready, EN_PASS_1);
```

3.8.2 Binding the SVA

SVA bind statements can be inside a module and specified along with your other design files in the `read_file` command. However, you cannot put a bind statement in the module you are binding (no self-binding). In that case, move the bind statement outside of the module definition as shown in the following example:

```
module bindings;
bind dc_encode sva_checkers sva_checkers_inst
 (clk,reset,input_ready,EN_PASS_1);
endmodule
```

Or

```
bind dc_encode sva_checkers sva_checkers_inst
(clk,reset,input_ready,EN_PASS_1);
```

For pure VHDL cases, a separate module must contain bind files as shown in the first example.

Adding Parameters On-the-fly in the Bind File

You can add more SVA properties with a property and bind files on the fly after the initial design build. Use the add_sva_property to add parameters on-the-fly in the bind file.

Syntax

```
add_sva_property -props <property file name> -bind <bind file name> [-vcs
 <options>]
```

Example

The following code shows the conventional method of adding SVAs in the Tcl file:

```
set_app_var fml_mode_on true
set_fml_var fml_witness_on true

analyze -verbose -f sverilog -vcs "-f filelist +incdir+../rtl/verilog/"
analyze -f sverilog prop.sv
analyze -f sverilog binder.sv
elaborate -sva MAC_top -vcs "binder"

create_clock Clock -period 100
create_reset Reset -sense high
sim_run 5
sim_save_reset
check_fv -block
report_fv
exit
```

The following code shows how to add SVA with new incremental bind properties:

```
set_app_var fml_mode_on true
set_fml_var fml_witness_on true

analyze -verbose -f sverilog -vcs "-f filelist +incdir+../rtl/verilog/"
elaborate MAC_top
add_sva_property -props prop.sv -bind binder.sv

create_clock Clock -period 100
create_reset Reset -sense high
sim_run 5
sim_save_reset
check_fv -block
report_fv
exit
```

The following code shows how to add an SVA property module that is parameterized:

```
module prop_gen_top(clk,rst_x,x,pstate);
input logic clk,rst_x;
input logic [7:0] x,pstate;
parameter red = 2'b00;
```

```
parameter blue = 2'b01;
parameter green = 2'b10;
parameter yellow = 2'b11;

  a_p1: assert property (@(posedge clk) disable iff (!rst_x) ((x == 0) &&
 (pstate == blue)) |=> pstate == green);  c_p1: cover property (@(posedge
 clk) disable iff (!rst_x) ((x == 0) && (pstate == blue)) |=> pstate ==
 green);
endmodule
```

You can then instantiate with parameters in the bind file as:

```
module bind_inst;
bind test prop_gen_top #(.blue(8))
 inst_prop_gen_top(.clk(clk),.rst_x(rst_x),.x(x),.pstate(pstate));
endmodule
```

3.8.3 Adding SVAs Inline

You can add inline SVAs in RTL code as shown below:

```
sva_encoder_ready: assert property(@( posedge clk)
disable iff(~reset) input_ready |=> EN_PASS_1);
```

3.8.4 Adding SVAs Incrementally from the Shell

You can also add properties using the `fvassert`, `fvassume` and `fvcover`commandsdirectly from the shell.

Use the`-expr` switch in these commands to specify the property's SVA expression. The`-expr`switch allows a limited subset of SVA including combinational Verilog, some sampled value functions, and both forms of implication operator (overlapping and non-overlapping). The current expression syntax allows many of the common SVA functions ($rose, $fell, and so on).

In some cases, the sequential operators are inferred. In these cases, the initial value of these operators is unknown. If you want to have known values for these sequential operators, then you must create the properties before simulating the reset sequence, or set up new simulation sequence to establish a new correct reset state.

To use `-expr` switch:

- The temporal properties must have a clock (see section)
- The SVA expression must be a string. When you use a Tcl variable in the string, you must enclose the expression with quotes, “”, instead of curly brackets, {}. Otherwise, the Tcl variable is used as a system function call. For example:

  ```
  set net_name [get attribute [get_nets a.b.x] name]
  ```

`fvassert -expr "$net_name == 1 "`// OK

`fvassert -expr {$net_name == 1} //` Syntax error: illegal system function $net_name

Incremental SVA Limitations

The Incremental SVA feature has the following limitations:

- Clock edge may only be posedge
- The following syntax is not supported in the expression
  - Parameters
  - Enums
  - Use of multiclock properties
  - ##N and ## [N:M] with non-constant expressions
  - Local variables
  - Properties declared in Source Code
  - Sequences and sequence methods declared in Source Code
  - Gated Clock [{@(posedge (clk && en)) q == en}]
  - Action Block
  - 2009 constructs
  - Functions declared in source code
  - Macros
  - Use of “\” for continuation to split the code on multiple lines
  - System verilog constructs (like logic, bit, and so on) in script properties

3.9 Supported SVA Syntax

The Incremental SVA feature has the following limitations:

- Clock edge may only be posedge
- The following syntax is not supported in the expression
  - Parameters
  - Enums

- Use of multiclock properties
- ##N and ## [N:M] with non-constant expressions
- Local variables
- Properties declared in Source Code
- Sequences and sequence methods declared in Source Code
- Gated Clock [{@(posedge (clk && en)) q == en}]
- Action Block
- 2009 constructs
- Functions declared in source code
- Macros
- Use of “\” for continuation to split the code on multiple lines
- System verilog constructs (like logic, bit, and so on) in script properties

3.9.1 Supporting Immediate/Deferred Asserts in Tasks and Functions

VC Formal supports all the legal usage of immediate or deferred asserts that cover tasks and functions based on LRM 2012. It reports failures for such asserts and supports debug using the CEX FSDB trace.

VC Formal supports assert and cover statements to be defined:

- In function body
- In for-loops, case, if statements within the function body
- In nested functions with the child function(s) containing assertions
- In functions and tasks with assertions defined in packages
- In functions and tasks with assertions defined in $unit
- In function calls in processes within generate statements

3.9.2 Supported Forms of User-defined Properties

SVAC compiler accepts all previously supported forms of local variables: a single assignment to a local variable in the antecedent of implication or in cover on sequences and any usage in linear properties.

The following forms of properties are also supported:

- In properties involving |->, |=> #-#, #=# on sequences:
  - In an assert property, an operator involving a disjunction (the *OR* operator) can be used only in the antecedent sequence before an assignment, and an operator involving a conjunction (the *and* operator) can be used in the consequent of implications before an assignment.

    The following assignments are supported:

    - Any assignment in the antecedent of |->, |=> if not preceded by a sequence containing *and*.
    - Any assignment in an assert on a sequence which may include #=# or #-# on sequences, but a sequence containing *or*.
    - Any assignment in the consequent sequence of |->. |=> if not preceded by [=], or a range in ##[M:N] or [*M:N], or a sequence or property containing *or*.
- In a cover property, an operator involving a disjunction can be used in the consequent of implication before an assignment, and an operator involving a conjunction before an assignment can be used in the antecedent.

  The following assignments are supported:

  - Any assignment in the consequent sequence of |->, |=>, but not after a sequence containing and.
  - Any assignment in the antecedent and consequent sequence of #-#, #=#, but not after a sequence containing and.
  - Any assignment in the antecedent of |->, |=> as long as it is not preceded by [=], or a range in ##[M:N], [*M:N], or a sequence containing or.

If property operators are used, then any form involving liveness is not supported even if the above rules are satisfied.

- Assignments in assert property can be made in the sequence operand of *always*, but cannot be made in cover property because it becomes a liveness property.
- No assignment is supported in any operand of *until, until_with, s_until, s_until_with*.
- No assignment is supported in any operand of *implies* and *iff*.
- Assignments are supported in the operand of *nexttime* and *s_nexttime,*provided all the other rules are satisfied.

  Note:

  If a top-level *not* operator is used, then the rules for assert property and cover property are interchanged.

Example 1

In the following example for property p2lv(), if |=> is changed to #=#, then cover property is supported on p2lv, but not assert property.

```
 property p2lv();
     logic [3:0] lv;
     bit     xv=1'b1;
     @(posedge clk) disable iff (rst)
       (a && xv, lv = 0 , $display("AA")) ##0 ((b && c && (lv[1:0] <
int'(2))), lv=lv+c, xv ^= 1)
             ##[1:2] (c && (lv==int'(1)) && !xv, xv = !xv)
         |=> e && xv ##[1:2] f;
  endproperty
 A2lv: assert property(p2lv); // assert ok, no witness, $display is
ignored
```

Example 2

In the following example for property p22lv():

- If d[->1] is changed to d[=1], then neither assert nor cover is supported; assert is not supported because of [=1] in the consequent, cover is not supported because of ##[1:2] in the antecedent.
- If d[->1] is changed to ##[1:2], then neither assert nor cover is supported; assert is not supported because of ## range in the consequent and cover is not supported because of ## range in the antecedent.

```
property p22lv();
      bit     xv=1'b1;
      @(posedge clk) disable iff (rst)
        (a && xv) ##0 ((b ), xv ^= 1)  ##[1:2] (c, xv = !xv)
          |=> d[->1] ##1 (e, xv = xv & f) ##1 xv;
   endproperty
   A22lv: assert property(p22lv); // assert ok, no witness
```

Support of Complex Clocks

The System Verilog Assertion Compiler (SVAC) is enhanced for VC Formal to use complex clocks (composite clocks and both edge triggered clocks) in both single clocked as well as multiclock SV assertions.

For example, the following assertions use complex clocks:

```
A1: assert property (@(clk) a ##1 b);
A2: assert property (@(clk or posedge clk1) a ##1 @(clk2) b);
A3: assert property (@(posedge clk or negedge clk1) disable iff(reset)
 (up_down && out[0]) |=> out[0]==1'b0);
```

Complex clock can be used with all supported constructs of the System Verilog Assertion (SVA) like assert, assume and cover, SVA sequences and also with all supported SVA functions like $fell, $changed, $onehot, $stable, $unknown, $past, $rose and so on.

Limitations

The following are the restrictions on the use of complex clocks in assertions:

- Events and sequence instances used as clocks are not supported, the assertion is skipped.
- If a complex clock involves a multi-bit expression, the assertion is skipped.
- If complex clocks are present in sampled value functions (like $rose, $past and so on) in procedural code that is outside of SVA, an error message is issued.

Support of SVA sequence .matched

VC FV supports the detection of registered endpoint of a sequence in a target sequence or property using the `sequence.matched`sequence property. Consider the following example:

```
sequence seq;
@(posedge clk1) a ##1 a;
endsequence
A: assert property(@(posedge clk2) b |=> seq.matched);
```

In this case, whenever `seq`has a match it is memorized until the next tick of `posedge clk2`. if assertion A evaluation reached the consequent and a tick of `posedge clk2` is detected, if the memorized match is observed, the assertion evaluation succeeds, otherwise it fails. If at the time of a tick of `posedge clk2` another tick of `posedge clk1`occurs and the sequence matches, this match is memorized till the strictly next tick of `posedge clk2`.

Support for Multi-clock SVA Properties

VC Formal supports user written multi-clock properties. These properties involve more than one active sampling clock. For example, the following assertion uses a simple multi-clock property:

Such assertions often arise on design interfaces at clock domain crossing boundaries.

```
A: assert property(@(posedge clk1) a |=> @(posedge clk2) b);
```

Support for $isunknown() System Task

The *$isunknown(param)* is a system task that evaluates if a X or Z value appears in the value of the *param.*VC Formal supports the usage of *$isunknown* in SVA properties.

Example

```
ast_w_not_unknown : assert property ( @(posedge clk)
                                        disable iff (!rst_x)
```

Feedback

```
                                                    not
$isunknown(signal_name) )
```

VC Formal supports sub-expressions of the form *$isunknown(signal_name)* in SVA assertions.

By default, VC Formal does not recognize x's under the *$isunknown* task. You must set the following formal application variable to enable this support:

```
set_fml_var fml_x_detect_mode 0/1/2
```

The `fml_x_detect_mode` application variable can take any of the three values: `0`, `1`, or `2`.

When this application variable is set to:

- `0`: VC Formal does not detect x propagation.
- `1`: VC Formal detects x's from hanging nets, multi-driven nets, x-initialized registers, out of bound access for arrays and memories, explicit x assignments and so on.
- `2`: VC Formal detects x's from input ports (all inputs are treated as potential source of x).

The default value of this application variable is 0.

Note:

Avoid using the `fml_x_detect_mode` formal variable in the VC Formal FCA application mode (`vcf -fmode COV`) as this can have an impact on the overall performance.

3.9.3 Limitations of SVA Support

The following are the current SVA supported features and limitations:

- Immediate assertions in initial blocks are not supported.
- No user-defined function calls in concurrent assertions. User functions can be used in immediate assertions.
- No recursive properties.
- No nested bind statements. No support binding to a module that is itself bound.
- The `$inset` and `$insetz` assertion system functions are not supported.
- No subroutine calls on sequence matches.
- No expect statement.
- SystemVerilog data types in SVA are supported except for unpacked unions as they are not synthesizable.

- Simple immediate, deferred observed and deferred final assertions, covers and assumptions are supported in always procedures and deferred assertions as stand-alone, but they cannot be used in functions or tasks.
- Use of action blocks in SVAs is supported and code inside these action blocks can read internal design signals. When using SVA action blocks with VC Formal, observe the following limitations:
  - The code must be synthesizable.
  - No SVA system tasks (for example, $past, $rose, etc.).
  - No user-defined tasks or functions.
  - Writing to internal signals is not supported in action blocks.
- Local variables are supported with the following restrictions:
  - Only a single assignment to any variable is allowed, that includes assignments in declaration and assignments under repetition operators ([* M:N]…), property containing “or” operator.
  - In assert property statements, any assignment must be in the antecedent of a top level implication |=> or |->
  - Linear properties with local variables can be used as assumptions, only if the property contains no boolean property operators, temporal property operators or sequence operators. However, the use of delays (##n), repetitions ([*n]) and suffix operators (|->, |=>, #-#, #=#) are permitted. For more information on ‘Linear properties with local variables’ refer to Appendix B - SVA Coding Guidelines.
  - In cover property statements, assignments in the sequence can be anywhere, in addition to the form allowed for assert property statements.
  - Sequence methods `.ended/.matched/first_match` cannot have local variables.
- The `first_match` operator is supported anywhere if the operand sequence is linear, that is, it has at most one matching point. It can also be used in any consequent of a top-level implication if the consequent is a sequence and the first match is applied to the sequence. This is because in that case the `first_match` operator is redundant since the consequent property has an implicit first match.
- Liveness properties are supported. Unqualified sequence properties (those without a weak or a strong operator) are interpreted according to the 2012 LRM as strong in covers and weak in assertions and assumptions.

- Multi-clock assertions and the `.matched` method are supported with the following limitations:
  - Only certain property operators or some operators are restricted to Boolean operands.
  - Complex clocks are restricted to `posedge, negedge clk`, with `iff` condition.
  - No cross-module references to `seq.matched`and `seq.ended.`
  - No local variable support in the autonomous sequences (the top-level assertion can use local variables, but not the sequences used with `.matched` or `.ended`).
  - Local variables cannot flow in and flow out of sequences used with `.matched` or `.ended`.

3.10 Updating Properties for Formal Analysis

Assertions are properties the user selects for formal verification. They are checked with all the constraints or assumptions applied. All the properties except cover properties can be used as assertions or constraints regardless of what type they are declared as in the source code. Only the usage of assert and assume properties can be changed. Cover properties cannot be used as constraints or assertions. Assert and cover type of properties are checked by formal engines. Assume properties are used to constrain the analysis space.

All properties have attributes to describe how they are declared and how they are used.

By default, VC Formal automatically treats each type as the same as its usage.

```
fvassume -type assume *
fvassert -type assert *
fvcover -type cover *
```

3.10.1 Changing Property Usage

When a property is created either in the rtl as an SVA property or as a script property, its type: assert, assume, or cover determines its default usage in formal analysis. When the formal model is created, all forms of the property are created (see Special Considerations below). This means that a property created as an assert can be used as a cover or assume, a cover can be used as an assert or assume, and so on.

To change the default usage or current set usage of properties, use the commands `fvenable, fvdisable, fvassert, fvassume` and `fvcover` along with options such as `-usage, -type` and `-class` to enable, disable or change usage of properties.

- The `usage` attribute takes on only `cover, assert, assume` values.
- Data invalidations occur only when constraints are added or removed from a session. Marking a property disabled (enabled attribute value = false) does not remove status information from cover or assert properties.

```
fvcover <property>: usage attribute= "cover". enabled attribute=
 "true".
fvassert <property>: usage attribute= "assert". enabled attribute=
 "true".
fvassume <property>: usage attribute= "assume". enabled attribute=
 "true".
fvdisable <property>: usage attribute unchanged.
```

- The `check_fv` commands operate only on enabled properties (enabled `attribute = true`).
- All disabled properties are reported in the `Unused` category with usage attribute and status information preserved in the output of the `report_fv` command.
- A `-filter {enabled==<true|false>}` switch is available with the commands to help with property selection.
- The current `-enabled` and `-not_enabled` switches for `report_fv`(in CC mode) are renamed as `-reached_enable` and `-unreachable_enable`.
- You can also review and change of property usage in the VC Formal GUI. For more details on how to manage properties in the GUI, see section Using the GoalList.

Examples

- To disable all properties for checking or constraints:

  ```
  fvdisable *
  ```

- To disable all current assertions:

  ```
  fvdisable -usage assert *
  ```

- To disable all current constraints:

  ```
  fvdisable -usage assume *
  ```

- To disable all current covers:

  ```
  fvdisable -usage cover *
  ```

- To change one assume type property to an assert:

  ```
  fvassert assume_prop_name
  ```

  Or

```
fvassert -type assume assume_prop_name
```

- To change one assert type property to an assume:

  ```
  fvassume assert_prop_name
  ```

  Or

  ```
  fvassume -type assert assert_prop_name
  ```

- To enable only AEP class properties with constraints and disable all other checks:

  ```
  fvdisable -usage {assert cover}
  fvassert -class aep *
  ```

3.10.2 Extending Cover Properties to End-of-Test Conditions

Extending coverage properties ensures that, after a coverage point is hit, the design continues to function correctly until it reaches an idle or "end of test" state. This feature is essential because simply hitting a coverage point does not guarantee that the design remains error-free in subsequent cycles. By extending coverage properties to the end-of-test condition, you can verify that the design maintains its integrity and does not fail any assertions after coverage is achieved.

Enabling the cover property extension in VC Formal provides several benefits:

- Confirms that the design reaches a clean idle state after coverage is hit.
- Ensures all injected transactions are processed and no pending operations remain.
- Detects assertion failures that may occur after coverage points are reached.
- Improves confidence in the completeness and robustness of the verification process.

Example

Consider the following two properties:

Property name	Expression	Class	Proof depth
Cover1	(Count[3:0] == 15) ##1 (Count == 0)	Script	200
Cover2	Sig2 == 1	Source	100

By extending the cover properties to End-of-Test Conditions, the tool creates new cover properties with the EOT condition appended. The following two additional properties are created with an appended expression.

Property name	Expression	Class	Proof depth
Cover1	(Count[3:0] == 15) ##1 (Count == 0)	Script	200
Cover2	Sig2 == 1	Source	100
Cover1_eot	(Count[3:0] == 15) ##1 (Count == 0) ## [0:$] end_of_count == 1	Script	200+X
Cover2_eot	Sig2 == 1 ##[0:$] end_of_count == 1	Script	100+Y

Enabling Cover Property Extension

Use the `fvcover -extend_cond <end_of_test_expr> [-suffix <suffix>]` command to extend cover properties with an end-of-test condition.

- Use the `-extend_cond` switch to specify the end-of-test condition.
- Use the `-suffix` switch to define a custom suffix for the new cover property name (optional).

Note:

- The `-expr` and `-extend_cond` switches for `fvcover` cannot be used together.
- The `-extend_cond` switch supports only boolean expressions.

Syntax

```
fvcover -extend_cond <end_of_test_expr> [-suffix <suffix>]
 <base_cover_property>
```

Examples

- `fvcover -extend_cond {end_of_count} -suffix "eot" prop1`
  - If `prop1` has the cover expression `expr1`, this command creates a new cover property `prop1_eot` with the expression `expr1 ##[0:$] end_of_count`.
  - If the `-suffix` switch is not specified, the new cover property will be named by appending `_extended` to the base property name.
- To extend all cover properties:

  ```
  fvcover -usage cover -extend_cond {expr} -suffix eot
  ```

- To extend cover groups:

  ```
  fvcover -type covergroup -extend_cond {expr} -suffix eot_covergroup
  ```

- To extend SVA-sourced cover properties:

```
fvcover -class source -extend_cond {expr} -suffix eot_source
```

- To extend script cover properties:

```
fvcover -class script -extend_cond {expr} -suffix eot_script
```

- To select base covers using a regular expression and extend them:

```
fvcover -regexp <regular_expression> -extend_cond {expr} -suffix
 eot_regexp
```

3.11 Reviewing Property Usage

Use the `get_props` command to review the properties in the design.

The results of the properties can be obtained using the `report_fv` command. For more information on the `report_fv` command, see section Viewing Results of check_fv Runs.

Syntax

```
vcf> get_props -help
Usage: get_props    #
        [-class <list-of-class-attributes>]
                               (property class selection:
                                Values: aep, connection, coverage, fusa,
                                no_class, register, script, security,
                                seq, source, xprop)
        [-type <list-of-type-attributes>]
                               (property type selection:
                                Values: arith_oflow, assert, assume,
                                boundconstraint, bounds_check, branch,
                                condition, conflict_driver,
                                constconstraint, control, cover,
                                covergroup, detect, envconstraint,
                                envstableconstraint, exclude,
                                fault_check, floating_bus, fsm_deadlock,
                                fsm_livelock, fsm_state, fsm_transition,
                                fsm_unionlock, full_case, injectx,
                                injectx_oba, injectx_reset,
                                injectx_undef, injectx_xassign,
                                internal, line, mark, multi_driver,
                                no_type, nox, nox_reset, observe,
                                one_hot, parallel_case, resetconstraint,
                                restrict, seq_gcc, set_reset,
                                stableconstraint, structural, toggle,
                                , x_assign)
        [-usage <list-of-usage-attributes>]
                               (property type selection:
                                Values: assert, assume, cover)
        [-status <list-of-status-attributes>]
```

Feedback

```
                                (property status selection:
                                 Values: "", checking, cond_proven,
                                 connected, covered, detected, falsified,
                                 inconclusive, non_activated,
                                 non_detected, non_formalcore,
                                 non_propagated, not_run, proven,
                                 unconnected, uncoverable, vacuous)
       [-vacuity <list-of-vacuity-attributes>] (property vacuity
selection:
                                 Values: "", checking, cond_non_vacuous,
                                 inconclusive, non_vacuous, not_run,
                                 vacuous)
       [-filter <filterExpr>] (Filter expression to apply)
       [-regexp]              (Use regular-expression instead of glob
filtering)
       [-exact]               (No pattern matching performed)
       [-selected]            (Returns selected properties, can be used
with -usage <assert/cover/assume>)
       [<list-of-names-ids-or-collections-of-properties>] (List of
property names, name patterns, or property collections)
```

Example

Consider a design with the following three properties:

```
assume_req0_until_gnt: assume property (@(posedge clk)
req0 && !gnt0 |=> req0);
assert_no_gnt_if_stall: assert property (@(posedge clk)
stall_i |=> (!gnt0 && !gnt1 && !gnt2 && !gnt3));
cover_all_req_asserted: cover property (@(posedge clk)
(req_cover == 4'b1111));
```

- The `get_props` command without any option returns a collection of all properties.

  ```
  vcf> get_props
  {"assert_no_gnt_if_stall", "assume_req0_until_gnt",
   "cover_all_req_asserted"}
  ```

- When the `-filter` option is used with the `get_props` command, the `assert_no_gnt_if_stall` property is declared and used as an assertion.

  ```
  vcf> get_props -filter {type==assert}
  {"assert_no_gnt_if_stall"}
  vcf> get_props -usage assert
  {"assert_no_gnt_if_stall"}
  ```

- To get the type and usage information print the attribute for a specific property using the `get_attribute` command.

  ```
  vcf> get_attribute [get_props assert_no_gnt_if_stall] typeassert
  vcf> get_attribute [get_props assert_no_gnt_if_stall] usage
  assert
  vcf> get_attribute [get_props assert_no_gnt_if_stall] name
  ```

Feedback

```
assert_no_gnt_if_stall
vcf> get_attribute [get_props assert_no_gnt_if_stall] class
source
```

- To change the usage of the assume property `assume_req0_until_gnt` to assert, use the type attribute:

```
vcf> fvassert -type assume assume_req0_until_gnt
{"assume_req0_until_gnt"}
```

- Or simply specify usage by name directly, regardless of the type:

```
vcf> fvassert assume_req0_until_gnt
vcf> get_props -usage assert
{"assert_no_gnt_if_stall", "assume_req0_until_gnt"}
```

- If you look at the attributes of the property, you can see that the usage and type attributes are different:

```
vcf> get_attribute [get_props
 priority_arb.my_priority_arb_assert.assume_req0_until_gnt] type
assume
vcf> get_attribute [get_props assume_req0_until_gnt] usage
assert
```

- The `fvassert, fvassume, fvcover, fvdisable`and `fvenable` commands can also operate on all properties of a specific type.

  For example, to exclude all properties declared as cover, use the following command:

  ```
  vcf> fvdisable -type cover
  ```

  The usage attributes for all cover properties is set to none and the `get_props` command returns an empty list:

  ```
  vcf> get_props -usage cover
  ```

  As expected, the property attribute for usage is none:

  ```
  get_attribute [get_props cover_all_req_asserted] usage none
  ```

- To get a collection of selected properties, use the `-selected` option

```
vcf>get_props -selected -usage assert
{"traffic.chk.assert_signal_sequencing_on_main"}

vcf>get_props -selected -usage {assert assume}
{"traffic.chk.assert_signal_sequencing_on_main",
 "traffic.chk.assume_continuous_waiting_first"}

vcf>get_props -selected
```

```
{"traffic.chk.assert_signal_sequencing_on_main",
 "traffic.chk.cov_both_green",
 "traffic.chk.assume_continuous_waiting_first"}
```

Note:

See section Special Considerations for cases where conversion from cover to assert/assume is not allowed.

- Properties that are declared in the SVA code, but are not used in an assert, assume, or cover statement, are not usable in the VC Formal tool. For example, the named property `p_req_until_gnt` is ignored because it is never referenced in a verification directive:

```
property p_req_until_gnt;
@(posedge clk) req3 && !gnt3 |=> req3;
endproperty
```

It is not shown by the `get_props` command and the following command returns no result:

```
get_props p_req_until_gnt
```

3.11.1 Special Considerations

There may be cases where it is not possible to create all forms of the automata required to handle all usages. The following are known exceptions:

- Cover properties on temporal sequences cannot be converted to assert/assume. The is because the property is bound to fail as assert and cause conflicts as assume.

  `C: cover property(@(posedge clk) a ##[1:3] b);`// cannot be used as assert or assume

- Assert/cover properties with local variables can be converted to assumes only in limited cases. For cases where such a conversion is not supported, a message is displayed.

  [Warning] PROP_W_SET_LOCALVARASSUME_PROP: Assert property 'test.AST_LOC_VAR' with local variables cannot be used as a constraint. This may be because the local variables are used in a non-constant or unbounded time window Command ignored on this property.

- Automata Construction aborted

  Example: If an automaton construction uses more than a maximum number of states, the construction is aborted and a warning message is reported that the associated usage (assert, cover, assume, vacuity) cannot be activated.

Warning-[SVAC-TMS] Automaton has too many states ../sva/St_AhbBus_mag.sva, 686 St_AhbBus

The property has too many states if used as an assert or an assume. That role of the property is thus not available. You can consider modifying the property, avoiding large ranges on sequence and property operators.

4

Compiling Designs and Verifying Formal Setup

This section discusses the steps to create and verify a Formal setup.

- Working with VC Formal Quick Start Mode
- Compiling Designs
- Compiling Designs Incrementally
- Changing Severity of VC Static Error Messages
- Suppressing Printing of Warning or Info Messages
- Reviewing Properties in Design
- Integrating DesignWare Components
- Black Boxing Modules in a Design
- Setting the stop_on_synth_error Application Variable
- Setting Up Profiling Information
- Setting Up User-Defined Clocks
- Setting Up User-defined Resets
- Specifying the Initial State
- Debugging the Reset State
- Verifying VC Formal Designs
- Support for Trace Checks

4.1 Working with VC Formal Quick Start Mode

The VC Formal quick start mode applies default configurations such as automatic abstractions that can potentially reduce design complexity to achieve higher convergence, thereby improving the out-of-the-box user experience.

Feedback

The VC Formal quick start mode determines the following settings automatically:

- Inferencing setup
- Black box defaults
- Abstraction defaults
- Clock edge defaults
- Compile option defaults
- Flexibility to switch on/off features
- Efficient incremental compilation for RTL and SVA

It is recommended to use the quick start mode with the initial porting of a new design to VC Formal for verification. Based on the initial results, the quick start mode can be disabled or configurations can be adjusted to get the best overall verification results. The quick start mode is also a great option when convergence is not improving as a way to target remaining properties after applying the default configurations such as automatic abstractions.

Note:
VC Formal Quick Start Mode support in VC Formal is a Limited Customer Availability (LCA) feature. Contact Synopsys support for more information.

4.1.1 Enabling VC Formal Quick Start Mode

Use the `fv_quickstart` command to enable the quick start mode for VC Formal.

Syntax

```
vcf>fv_quickstart -help
Usage: fv_quickstart    # Starts the quick start mode for VC Formal
        [-enable <value-list>] (Enables specified fv_quickstart settings:
                                Values: abstraction, aep_unique_name,
                                aep_vhdl, aep_vhdl_bus_check,
                                auto_sva_compile, build_time_out,
                                change_at_posedge, coi_fsm_complexity,
                                coi_reduction, compile_abstraction,
                                cov_fsm_state_extraction, cov_vhdl,
                                defer_property_compile, dpv_cov_map,
                                dpv_cpp_ufd, dpv_do, dpv_do_bh,
                                dpv_do_effort,
                                dpv_design_based_complexity,
                                dpv_do_version,
                                dpv_orch_node_inteq_limit,
                                elaborate_j_value, fxp_auto_rootcause,
                                search_include_file_path,
                                seq_direct_env_equality,
```

Feedback

```
                                seq_map_uninit_x, synthesis_semantic,
                                default, all)
      [-disable <value-list>]
                               (Disables specified fv_quickstart
                                settings:
                                Values: abstraction, aep_unique_name,
                                aep_vhdl, aep_vhdl_bus_check,
                                auto_sva_compile, build_time_out,
                                change_at_posedge, coi_fsm_complexity,
                                coi_reduction, compile_abstraction,
                                cov_fsm_state_extraction, cov_vhdl,
                                defer_property_compile, dpv_cov_map,
                                dpv_cpp_ufd, dpv_do, dpv_do_bh,
                                dpv_do_effort,
                                dpv_design_based_complexity,
                                dpv_do_version,
                                dpv_orch_node_inteq_limit,
                                elaborate_j_value, fxp_auto_rootcause,
                                search_include_file_path,
                                seq_direct_env_equality,
                                seq_map_uninit_x, synthesis_semantic,
                                default, all)
```

- To configure quick start settings, use the `fv_quickstart` command with either the `-enable` or `-disable` option.
- By default, `fv_quickstart` is equivalent to `fv_quickstart -enable default` that enables some of the settings. The settings include: `aep_unique_name, aep_vhdl, aep_vhdl_bus_check, auto_sva_compile, build_time_out, coi_fsm_complexity, cov_fsm_state_extraction, cov_vhdl, defer_property_compile, search_include_file_path and change_at_posedge`
- To enable selected quick start settings, use:

  ```
  fv_quickstart -enable <value-list>
  ```

- To disable selected quick start settings, use:

  ```
  fv_quickstart -disable <value-list>
  ```

- The `fv_quickstart` command shows the set of configurations to be applied during design read. You can change those configurations between the `fv_quickstart` and `read_file/elaborate` commands.

Special Considerations

The order of invocation determines which quick start settings are applied.

- Invoking `fv_quickstart` before `set_fml_appmode` applies FPV settings.

  ```
  fv_quickstart
  set_fml_appmode SEQ
  ```

- Invoking `fv_quickstart` after `set_fml_appmode SEQ` applies both FPV and SEQ settings.

```
set_fml_appmode SEQ
fv_quickstart
```

Note:

The quick start mode applies default configurations such as automatic abstractions which can be too strong and lead to falsified assertions and covered cover properties. It is recommended to carefully review proof results when using the quick start mode. While results for proven assertions and uncoverable cover properties are safe, check if abstractions have caused an incorrect failure for an assertion or scenario for a cover property. If so, re-run the proof for those properties where abstractions are removed or the quick start mode is disabled.

Note:

Auto abstraction is disabled for the FSV and FUSA applications in the quick start mode. For the SEQ and FXP applications, only MULT, DIV, MOD and POW constructs are enabled in the quick start mode.

Example for FPV Application

```
Example for FPV Application:
vcf> set_fml_appmode FPV
vcf> fv_quickstart
[Info] QUICK_START: Enabled SVA Processing.
 To override: fv_quickstart -disable auto_sva_compile.
[Info] QUICK_START: Compile timeout per module set to 60 seconds.
 Previous value was 3000 seconds.
To override: fv_quickstart -disable build_time_out
[Info] QUICK_START: set_change_at -posedge -clock <reference_clock> will
 be applied at formal model build if no user set_change_at commands are
 found.
 To override: Add any set_change_at or remove with 'set_change_at
 -default -remove' after read_file/elaborate.
[Info] QUICK_START: Enabled Deferred Property Compile flow.
 To override: fv_quickstart -disable defer_property_compile
[Info] QUICK_START: -search_include_file_path added to compile options.
 To override: fv_quickstart -disable search_include_file_path

Default options are enabled for FPV with respective commands on how to
 disable these commands
```

4.2 Compiling Designs

VC Formal provides the following commands for reading in design and properties.

- `read_file`
- `analyze`
- `elaborate`

For detailed information on these commands, see the *VC Static Platform Reference Guide*.

Note:

VC Formal honors the initial values before running the elaborate and read_file commands, if you set the following application variable to true:
`vcf>set_fml_var fml_enable_initial_blocks true`

For the `-netlist`mode and `.db` library support, libcells are unfolded and takes more build-time memory.

Note:

By default, bind files that are not in the same analyze command of top module are elaborated together. To change default behavior, set the following application variable to false `set_app_var apply_bind_in_all_units false`

This is a video tutorial on how to compile a VHDL design in a single step, using a UUM model, and for AEP, FPV, and COV applications.

To compile a VHDL design in a single step, use the `read_file` command with the `-format vhdl` option. Additional options like `-top` for specifying the top module and `-vcs` for file lists remain consistent with Verilog compilation. Execute the tool using `vcf -f`. For UUM model compilation, use the `analyze` command to compile and elaborate to process the design, again specifying `-format vhdl`. To run AEP, set the app mode to AEP, enable AEP checks with `set_app_var`, and use the `-aep all` option. All AEP checks are supported for VHDL. For coverage analysis, switch to COV mode, enable coverage with `-cov all`, and run the design without errors.

4.2.1 Examples for Reading Design and Properties in Different Languages

- *Verilog + SVA*

```
read_file -format sverilog -sva -top arb -vcs "arb.v arb.sva
arb_bind.v"
```

or

```
analyze -format sverilog -vcs "arb.v arb.sva arb_bind.v"
elaborate arb -sva
```

- VHDL + SVA or MX (VHDL top) + SVA

Feedback

```
analyze -format vhdl {arbiter.vhd }
analyze -format sverilog "assertion.sv bind.sv "
elaborate ARBITER -sva -vcs "-lca -sva_bind_enable bind_file"
```

- MX (Verilog top) + SVA

```
analyze -format vhdl {arbiter.vhd }
analyze -format sverilog "arbiter_top.v assertion.sv bind.sv "
elaborate arbiter_top -sva -vcs "-lca"
```

- VHDL/MX + SVA

```
analyze -formal vhdl arb.vhdl
elaborate arb -sva
```

- SystemVerilog+ SVA

```
read_file -format sverilog -sva -top arb -vcs "arb.sv arb.sva
arb_bind.v"
```

or

```
analyze -format sverilog -vcs "arb.v arb.sva arb_bind.v"
elaborate arb -sva
```

Watch this video to understand the process of compiling Verilog and SystemVerilog designs in VC Formal using single-step and two-step flows.

4.2.2 Support for Multiple-Edge-Assignment and the Multiple-Process-Assignment

The Multiple-Edge-Assignment and the Multiple-Process-Assignment are not synthesizable as they bring non-deterministic behaviors. Therefore, by default, VC Formal does not support Multiple-Edge-Assignment and the Multiple-Process-Assignments.

To enable support for Multiple-Edge-Assignment and the Multiple-Process-Assignment in VC Formal, you must set the following application variable before the `read_file`, `elaborate`, or `elaborate_seq` commands.

```
set_fml_var fml_enable_ndmerge true
```

Example for Multiple Process Assignment

```
always @(posedge clk1)
     begin
         if (en1)
          mem[waddr1] = wdata1;
```

Feedback

```
     end

always @(posedge clk2)
     begin
        if (en2)
          mem[waddr2] = wdata2;
     end
```

Example for Multiple Edge Assignment

```
always @ (posedge clk or negedge clk or negedge rst) begin
   if (!rst) begin
      a <= 1;
      b <= 0;
   end
   else
     begin
        b <= jack[0];
        a <= ~jack[0];
     end
end
```

4.2.3 Preserving RTL Signal Connectivity

Snipping internal signals may lead to the following message at formal model build time:

[Warning] RAW_SIGNALS_FOUND: 1 signals used in snip_driver/set_constant commands are read after write signals used in blocking assignments.

This indicates that the signal was read after being assigned in a combinational always block using blocking assignments. This may lead to incorrect results during formal analysis.

Use the following command to identify such signals:

```
get_attribute [get_fvtask] raw_signals
```

Example

Consider you have added a snip on an internal signal and noticed that the signal being driven by the snip had a different value than expected.

Consider the following module:

```
module xfsgmod(…);
logic [1:0] id;
always_comb begin
  id = shm ? addr[3:2] : addr[1:0];
  valid_mask = ((id == 2'b1) && mask);
  bank_id = id;
end
endmodule
```

The resulting logic after synthesis identifies `id` as a read after write (RAW) signal, and the first usage of id is replaced with the expression that was assigned in the write statement:

```
logic [1:0] id;
always_comb begin
  id = shm ? addr[3:2] : addr[1:0];
  valid_mask = (((shm ? addr[3:2] : addr[1:0]) == 2'b1) && mask);
  bank_id = id;
end
```

Functionally this is correct. However, if the signal, *id*, is snipped in the synthesized design, the snip value only impacts the `bank_id` in the above example, and `valid_mask` is not affected by the snip. This can potentially lead to false proofs.

Therefore, VC Formal has introduced the `preserve_raw_signal_fanout` application variable to enable the user to preserve the connectivity of all signals in synthesized models. By default, the `preserve_raw_signal_fanout` application variable is set to false for all VC Formal applications, except FuSA.

When the `preserve_raw_signal_fanout` application variable is set to true, VC Formal does not perform checks on all the RAW signals even if it is snipped, and does not report the RAW_SIGNALS_FOUND warning message.

If you snip a RAW signal, and do not set the `preserve_raw_signal_fanout` application variable, the RAW_SIGNALS_FOUND warning message is reported.

Note:
Only in the VC Formal FuSA application mode, by default, the `preserve_raw_signal_fanout` application variable is set to true.

4.3 Compiling Designs Incrementally

Incremental compilation allows you to optimize the compilation process when the design and property files remain unchanged. This flow significantly reduces runtime and CPU usage for large designs. When enabled, the tool skips recompilation under the following conditions:

- No changes are detected in the design and property files during analysis.
- Design elaboration options remain unchanged.
- The compile data directory (including its timestamp) has not been modified since the initial compilation.

If any of these conditions are not met, the tool reports messages indicating that module declarations are being overwritten.

Note:
Compiling Designs Incrementally in VC Formal is a Limited Customer Availability (LCA) feature. Contact Synopsys support for more information.

Note:
Incremental compile does not work with VHDL designs.

4.3.1 Enabling Incremental Compile

To enable incremental compilation, set the following application variable to true before executing the `analyze`/`elaborate` or `read_file` commands:

```
set_app_var enable_incremental_compile true
```

Use the `incr_compile_data_path` application variable to specify the directory containing incremental compile data. This is particularly useful if the *vcst_rtdb* from a previous session has been moved to a different location and needs to be reused in the current session:

```
set_app_var incr_compile_data_path <path of the vcst_rtdb>
```

4.3.2 Use Model: Incremental Compile Tcl Scipt

Single-Step Flow

To enable incremental compilation in a single-step flow, include the following commands in your Tcl script:

```
set_app_var enable_incremental_compile true
set_app_var incr_compile_data_path <path_of_vcst_rtdb>
read_file -top traffic -format sverilog -sva -vcs {...}
```

In a single-step flow, if the design remains unchanged, the tool will display the following message and skip the recompilation and re-elaboration process:

The design has not changed and does not require recompilation. To force recompilation, delete the file `vcst_rtdb/.internal/design/simv.daidir/.vcs.timestamp` and rerun VCS.

Two-Step Flow

To enable incremental compile in a two-step flow, set the following commands in the Tcl script:

```
set_app_var enable_incremental_compile true
set_app_var incr_compile_data_path <path_of_vcst_rtdb>
analyze -format sverilog -f file_list
elaborate top_module
```

Feedback

The following example demonstrates a successful incremental compilation message:

```
analyze -format sverilog -vcs "+define+FPV_ON -assert svaext -f test.scr"
Lint-[VCF_IA_CHECKPASS] Incr-Analysis check passes
  The design and all other inputs have not changed, so reanalysis is not
  necessary.
  To force analyzer to re-analyze the design, please set app var
  'enable_incremental_compile' to false.
Note-[VCF_NONEED_ELABORATE] Re-elaboration checking success
  There is no need to re-elaborate the design.
  If you want to re-elaborate the design, please set app var
  'enable_incremental_compile' to false.
```

Handling Front-End Incompatibility

If there is a front-end incompatibility between VC Formal versions, an error is reported. In such cases, you must disable the incremental compilation flow and compile the design at least once using the new version. Alternatively, delete the compile data directory to force recompilation with the updated version.

```
Error-[IVWU] Incompatible vlogan
Incompatible vlogan was used
Verilog portion of the design in directory
'./try/vcst_rtdb/.internal/design/worklib/AN.DB' was analyzed using
 incompatible version of vlogan(U-2023.03-SP2 2023-08-26 20:42:52).
Please clean the pre-analyzed contents from
 './try/vcst_rtdb/.internal/design/worklib/AN.DB' and re-analyze the
 source code in logical library 'DEFAULT' with the current version and
 elaborate/compile the design again.
```

4.4 Changing Severity of VC Static Error Messages

You can change the severity of the VC Static messages. You can:

- Upgrade all *Warning* messages to Errors
- Upgrade selected *Error* messages to Warnings
- Configure the number of error messages that must be displayed before the run stops
- Configure whether an error message must result in stopping the run, or continuing the run
- Suppress messages of a specified severity

  Note:

  This error control mechanism is only applicable for VC Static messages. Any VCS side messages must be controlled using the already existing error control mechanism of VCS.

Feedback

4.4.1 Use Model

- To specify the severity for a message id, use the `set_message_severity` command.

Syntax

```
vcf> set_message_severity -names message_names <error | warning | info>
```

- Where,
  - `-names message_names`: Specify the list of error message names whose severity must be changed. For example, DB_COPT003.
  - `<error | warning | info>`: Set the required severity for the specified message. Possible values: error, warning, info.

    Example

    ```
    vcf> set_message_severity -name CC_UID017 error
    ```

- To configure whether an error message must result in continuing with the run or stopping the run, use the `set_message_error_action` command. This command can also be used to specify the number of such errors that must reported before the run stops. This helps in fixing many similar issues in one go.

  ```
  vcf> set_message_error_action [-stop_at_error_count error_count] <stop
   | continue>
  ```

  Where,

  - `[-stop_at_error_count error_count]`: Specify the number of the error message to be displayed in the run.
  - `<stop | continue>`: Specify if the run should be stopped or continued when the error message is reported. Values: stop, continue

    Example

    ```
    vcf> set_message_error_action stop
    ```

- Use the `msg_print_threshold` application variable to set the severity level of the messages which must be reported. The levels can be *Info*, *Warning*, *Error,*or *Fatal*.

  The priority of the severity level is *info* < *warning* < *error* < *fatal*.

  By default, the severity value is set to *Info* which means by default all messages are reported.

  If it is set to *warning*, then all messages of severity *warning* or greater than that, like *error*/*fatal* are reported. The info messages are suppressed.

  Example

The following example suppresses all *Info* messages. Only Warning, Error and Fatal messages are reported.

```
vcf> set_app_var msg_print_threshold warning
```

Example

In the following example, the severity of the CC_UID017 message ID is set to *Error,*and when this error message is reported, the run is terminated.

`vcf> set_message_severity -name CC_UID017 error` ##severity of CC_UID017 should be error

`vcf> set_message_error_action stop` ##stop on CC_UID017 error

4.5 Suppressing Printing of Warning or Info Messages

The `suppress_message` command provides a mechanism to disable the printing of messages. You can suppress only warning and info messages. The result of `suppress_message` is always the empty string. A given message can be suppressed more than once. So, a message must be unsuppressed (using `unsuppress_message`) as many times as it was suppressed in order for it to be enabled. The `print_suppressed_messages` command displays the currently suppressed messages.

Syntax

```
vcf> suppress_message -help
Usage: suppress_message    # Suppress printing of messages
        messages           (List of message codes to suppress)
```

```
vcf> unsuppress_message -help
Usage: unsuppress_message    # Re-establish printing of messages
        message_list           (List of message codes to unsuppress)
```

```
vcf> print_suppressed_messages -help
Usage: print_suppressed_messages    # Display the message ids which are
 currently suppressed
```

Examples

When the argument to the `unalias` command does not match any existing aliases, the CMD-029 warning message displays. This example shows how to suppress the CMD-029 message:

```
prompt> unalias q*
   Warning: no aliases matched 'q*' (CMD-029)
   prompt> suppress_message CMD-029
```

Feedback

```
prompt> unalias q*
prompt>
```

4.6 Reviewing Properties in Design

After the design is read in, all the properties included in the design setup must be reviewed. You can review properties in the VC Formal GUI or using Tcl. Use the `report_fv` command to review the properties that are available for further configuration. Figure 13 shows an example of the report output.

For more information on how to review properties in the VC Formal GUI, see section Using the GoalList.

For information on how to manage properties, see section Managing Constraints and Properties for Formal Analysis.

Figure 13: Report of Properties After Design Is Read

```
report_fv –list
 Summary Results
   Property Summary:
   ----------------
   > Assertion
     - # found       : 18
     - # not_checked  : 18
   > Vacuity
     - # found       : 8
     - # not_checked  : 8
   > Constraint
     - # found       : 2
 -------------
   > Assertion
     # Assertion: 18
.....
     [  4] not_checked          (not_checked) - traffic.chk.assert_green_no_waiting_first
     [  5] not_checked          (not_checked) - traffic.chk.assert_green_no_waiting_main
......
   > Cover
     # Cover: 9
     [ 18] not_checked                  - traffic.chk.cov_both_green
     [ 19] not_checked                  - traffic.chk.cov_both_red
     [ 20] not_checked                  - traffic.chk.cov_both_yellow
     [ 21] not_checked                  - traffic.chk.cov_green_with_waiting_on_first
     [ 22] not_checked                  - traffic.chk.cov_green_with_waiting_on_main
.....
   > Constraint
     # Constraint: 2
     [ 16] constrained                - traffic.chk.assume_continuous_waiting_first
     [ 17] constrained                - traffic.chk.assume_continuous_waiting_main
```

4.7 Integrating DesignWare Components

The VC Static Platform supports designs that include the Synopsys DesignWare (DW) components. The DW source libraries shipped by Synopsys contain two versions of the DW parts. Therefore, additional steps are needed to ensure the correct version is used.

There are two versions of behavioral models written in Verilog and is used for simulation.

- If used in a VC Static application, the behavioral code is surrounded by Synopsys translate off/on pragmas that result in an empty model or black box.

  When the `analyze_skip_translate_body` application variable is set to true, VC Static honors pragmas in user HDL files (synopsys_translate_off and synopsys_translate_on) and skips the code in between the pragmas from its analysis. If set to false, VC Static ignores the pragmas and thereby tries to compile and elaborate the code inside the pragmas.

  If you want to analyze some files with honoring the pragmas and some other files by ignoring pragmas, you can set the Tcl variable accordingly before each file (or a set of files) that must to be compiled.

  By default, the `analyze_skip_translate_body` application variable is set to true.

- The encrypted source files written in Verilog/VHDL. These are synthesizable and can be used within Synopsys applications. These parts may be hierarchical and use other encrypted DW subparts. To use these parts in the VC Static flow, all of the parts referenced by a DW part must be analyzed prior to elaboration.

Supporting DW components in the VC Static platform is a two step approach.

- Pre-analyze the DW library as part of the VC Static installation. This is done once per customer site.
- Compile designs with DW using the pre-analyzed library.

For more details on how to integrate DesignWare components in VC Formal, refer to the VC Static Platform User Guide.

4.8 Black Boxing Modules in a Design

Black boxing certain design modules impact how Formal checks are applied. There may be many reasons why you may want to black box certain design modules. For example, black boxing helps remove modules from design that are not needed for proving formal checks.

You can define black boxes using the `set_blackbox` command. This command must be set before `read_file` or `elaborate` commands.

Syntax

```
vcf>set_blackbox -help
Usage: set_blackbox    # Marks black-box in the design
        [-level <N>]           (Set design hierarchy at level N and below
 as black box:
                                Value >= 1)
        [-inst_level <N>]      (Set instance hierarchy at level N as
 black box:
                                Value >= 1)
        [-exclude <design>]    (Set modules not specified in exclude list
 as blackbox)
        [-cells <cell>]        (List of cell to be set as black box)
        [-designs <design>]    (List of design to be set as black box)
        [-ignore_abstract_model] (Ignore linking to abstract models, if
 exist)
        [-verbose]             (Display additional information)
        [-exclude_cells <exclude cell list>](List of cells to be excluded
 from blackbox list which are specified by -designs,-cells or -inst_level
 options)
        [-directories <directory list>] (Designs which reside in these
 directories are blackboxed)
```

Note:

Use the `-designs` option to black box *modules*. Use the `-cells` option to black box instances in the hierarchical path. The `set_blackbox -cells` is applied in the post-synthesis stages.

Example 1

To black box *moduleA* module, use the following commands in the Tcl file:

```
set_blackbox -designs {moduleA}
read_file -format verilog -top top top.v
```

When a module is black boxed, all the ports are treated as unconstrained (as inputs only) in the design. When the design is read in, the following message is printed for each black-boxed module instance:

Note-[SM_BB_SKIP] Skipping blackboxed Module/Entity

Note:

If the definition of the black boxed modules are not available, Verdi treats the black boxed module instances the same as the normal undefined module instances.

Example 2

You can provide a numerical range support for the `set_blackbox -designs` option.

```
set_blackbox -designs A{1..2}
```

This is equivalent to:

```
set_blackbox -designs A1
set_blackbox -designs A2
```

Example 3

Black box all cells but exclude instance *a1.b1.lf1*using its full pathname and the instances up to the top module:

```
set_blackbox -cells * -exclude_cells {a1.b1.lf1}
```

This is equivalent to specifying all instances between top module and instance *a1.b1.lf1*:

```
set_blackbox -cells * -exclude_cells {a1 a1.b1 a1.b1.lf1}
```

Example 4

Black box all cells but exclude instances with a pathname containing**b*lf1* and the instances up to the top module

```
set_blackbox -cells * -exclude_cells {*b*lf1}
```

Example 5

To black-box all instances at a given instance level 2 and below:

```
set_blackbox -inst_level 2
```

To black-box all instances in level 2 and below, and exclude instances of Module M:

```
set_blackbox -inst_level 2 -exclude M
```

4.8.1 Identifying Empty modules, Black boxes, Undefined Modules

You can identify empty modules, black boxes, undefined modules, and modules do not qualify for the synthesis stage using the `report_link`Tcl command.

Syntax

```
vcf> report_link -help
Usage: report_link    # Report status of the design build using Simon/VNR
        [-sim_match]  (Includes library cells where a simulation model is
 given)
```

For more details on the `report_link` command, see the *VC Static Platform User Guide*.

By default, undefined modules are automatically black-boxed in all VC Formal applications (except for the DPV application) when the design is read in VC Static. However, if you

want VC Static to report an error message when undefined modules are present in the design, set the following application variable to false:

```
vcf>set_app_var autobb_unresolved_modules false
```

To perform automatic black boxing of undefined modules in the DPV application mode, use the `-Xpartialdesign` option.

Note:

If the definition of the black boxed modules are not available, Verdi will treat the black boxed module instances the same as the normal undefined module instances.

4.9 Setting the stop_on_synth_error Application Variable

By default, VC Formal does not stop the run when there is a synthesis error. You can set the `stop_on_synth_error` application variable to true, if you want the tool to stop when there is any error during synthesis. When the `stop_on_synth_error`application variable is set to true, VC Formal stops the run if there is any error during synthesis.

By default, the `stop_on_synth_error`application variable is set to false, and VC Formal gracefully continues when there is a synthesis error by ignoring, bypassing, and black boxing such error scenarios.

Note:

Ignoring, bypassing, black boxing synthesis issues might mask some serious issue. You can set the `stop_on_synth_error`application variable to true, if you want to examine the errors before proceeding.

4.10 Setting Up Profiling Information

Use the `fml_report_profile_data` formal variable to get the profiling information such as the CPU time, wall time, and peak memory at specific steps like reset simulation, formal model build, and so on.

Syntax

```
set_fml_var fml_report_profile_data <bool> [-global]
```

Where,

- `false`: Does not report the profiling information. By default, the `fml_report_profile_data`is set to false.
- `true`: Reports the profiling information.
- `-global`: Applies the specified value to all tasks.

You can get a warning message when memory usage in local machine exceeds the percentage threshold specified in the `fml_session_memory_warn_pct` formal variable. By default, `fml_session_memory_warn_pct`is set to 0.

Syntax

```
set_fml_var fml_session_memory_warn_pct <0 <= value <= 100>
```

4.11 Setting Up User-Defined Clocks

You can also specify your own clocks in the setup. Before defining clocks in a design, it is important to understand the concept of reference clocks, system clocks, and the timing relationships of clock edges and data.

Consider a design consisting of an AND gate and an assertion to check that the output is always 0.

```
ANDCHK: assert property ( @(posedge clk) disable iff(rst) out == 0);
```

One would expect the assertion ANDCHK to pass since clk is 0 at the assertion sampling point, so, out should be 0 regardless of A. It passes in simulation but the assertion fails in formal analysis. There is no difference between sampled value and actual value in the formal representation. When the assertion is triggered at posedge clk, clk is 1 and the value of out is determined by the value of A and can be both 0 and 1, so, the assertion fails.

Note:

Using clock as data in an assertion may lead to unexpected results in formal analysis.

4.11.1 Specifying User-defined Clocks

To specify clocks in the design, use the `create_clock` command.

Feedback

Syntax

```
vcf> create_clock -help
Usage: create_clock    # Defines a clock
        -period <period_value> (Specifies clock period value)
        [-name <clock_name>]   (Specifies clock name)
        [-waveform <edge_list>] (Specifies edge list)
        [-add]                 (Add new options to the original clock)
        [-comment <string>]    (Specifies comment)
        [-refclk]              (Use clock as reference clock [formal])
        [-initial <value>]     (Specify clock initial value (0/1 -
 default 1) at time 0 [formal])
        [source_objects]       (nets, ports and pins on which this clock
 is defined)
```

Note:

As the initial value is set to 1 by default, this change may affect the results and/or performance of the existing proofs. To revert to the previous behavior, Set the initial value as 0 in `create_clock` commands or Set this hidden formal `set_fml_var fml_start_clocks_on_posedge` variable to false prior to any `create_clock` commands.

Use different names while creating different clock signals. Using the same clock name overwrites the previous clock and an informational message is reported. For example,

```
vcf> create_clock -name myclk -period 100 clkA
vcf> create_clock -name myclk -period 100 clkB
```

[Info] FML_CLOCK_REMOVED: The following formal clock(s) were removed: clkA This may be due to a 'remove_clock' or a 'create_clock' redefinition.

4.11.2 Specifying Reference Clock

The reference clock is the default clock for script properties, reset sequence simulation, reported depths in property falsification and bounded proof reports.

For designs with a single clock, user-defined clock is the reference clock. For designs with multiple clocks, the default reference clock is the fastest user-defined clock. You can override the default settings of a reference clock.

Example

To specify the clock that must be used as the reference clock, use the following command:

```
create_clock clk400 -period 400 -refclk
```

Feedback

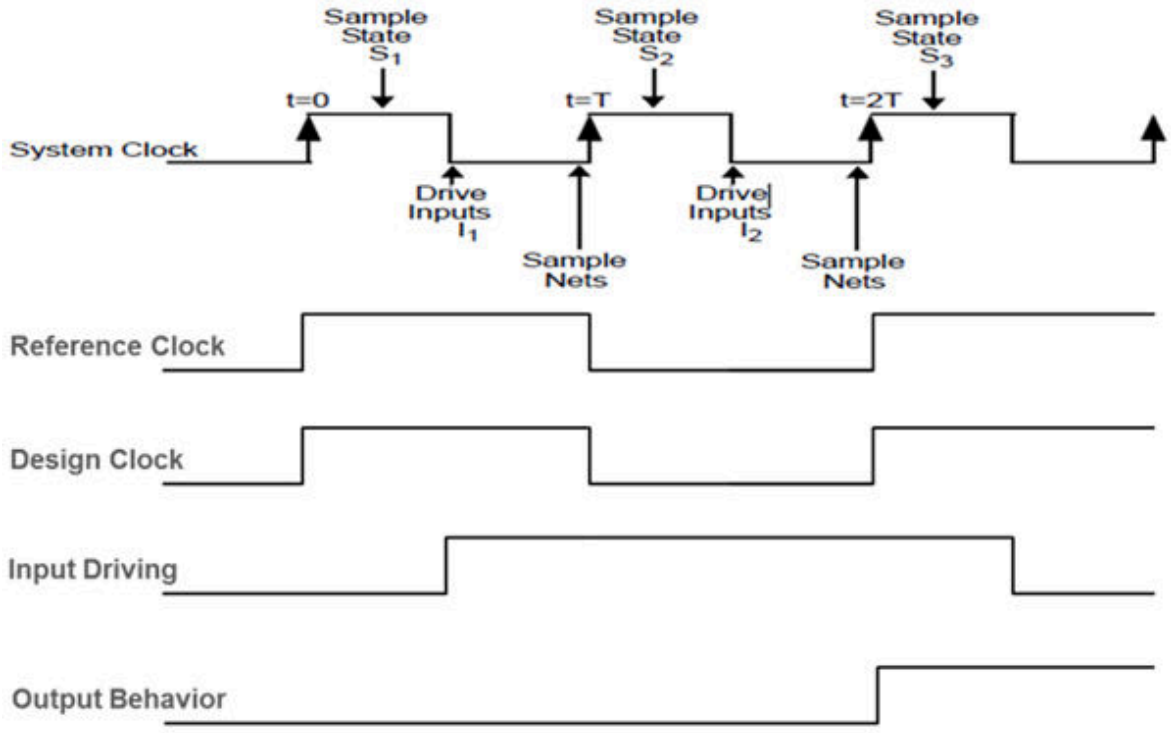
4.11.3 The Concept of System Clock

The formal representation of a design is always a single clock model, regardless of how many clocks are present in the design. System clock is an internally generated clock, which is at least twice as fast as the fastest user defined clock. This allows state changes on both positive and negative edges of the fastest user defined clock. If the user defined clocks are not multiples of one another, the system clock is created using the common denominator of all user defined clocks.

The timing relationship of the system clock, data driving and sampling state are shown in Figure 14.

Given that the system clock period is T, at T=0, the system clock rises. At T/4, the circuit state is sampled. At T/2, stimulus is applied to the primary inputs of the design. At one tick before T, the net and goal values are sampled.

Figure 14: Timing Relationship of Clocks, Data Driving and Sampling State



4.11.4 Specifying a Single Clock

You can create a single clock using the `create_clock`Tcl command.

Example

```
vcf> create_clock clk -period 100
```

Here, by default, the clock starts with an initial value of 1. The clock period must be a positive integer greater than 0.

To specify the waveform, specify the rising edge and falling edge, along with the period:

```
create_clock clk -waveform {10 90} -period 100
```

4.11.5 Specifying Multiple Clocks

You can define multiple clocks with different clock periods using multiple `create_clock`Tcl commands.

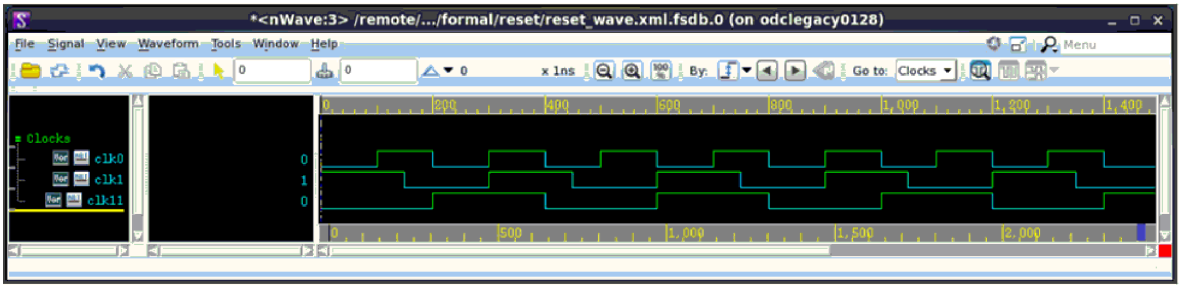
VC Formal analyzes design behavior in a cycle-based manner, so, the absolute number is usually not critical. However, large value differences in multiple clock periods may slightly impact performance and capacity with increased runtime and reduced capacity. It is recommended that you create clocks with large period differences only when necessary.

Example

```
vcf> create_clock clk0 -period 200 -initial 0
vcf> create_clock clk1 -period 300 -initial 1
vcf> create_clock clk11 -period 400 -initial 0
```

The wave form in Figure 15 shows the three clocks defined by the three commands. In this case, the reference clock is the same as clk0.

Figure 15: Multiple Clocks Defined By the User



4.11.6 Specifying Virtual Clocks

You can specify properties in the design related to a virtual clock/s. A virtual clock does not have related clock port/net in the design and may have a frequency different from the clocks in the design.

Example

To specify a virtual clock using the following command:

```
vcf> create_clock -name vclk -period 400
```

Note:

The `-name` option must be used in the `create_clock` command for specifying virtual clocks. To specify system clock names, use the identifier without the`-name` option.

Limitations

- Virtual clocks have the same limitation as user clocks relative to where they can be defined in the formal flow.
- Virtual clocks apply to formal script commands that have the `-clock`option

  `add_cc, fvassert, fvassume, fvcover, get_change_at, set_change_at, report_change_at`
- Virtual clock names cannot have the same names as the existing clock names in the design.

4.11.7 Specifying Asynchronous Clocks

If the clocks in the design are asynchronous, specify only one of the clocks. The other clocks behave as data inputs, that is, randomized and uncorrelated.

4.11.8 Getting a Report of User-defined Clocks

To get a list of all the user-defined clocks, use the `get_clocks` command.

Syntax

```
vcf> get_clocks -help
Usage: get_clocks    # Returns clock objects, whose name matches with the
 specified pattern
        [-filter <filterexp>]  (Filter in those objects whose name
 matches with <filterexp>)
        [-regexp]              (Indicates filter expression type to be
 regular expression)
        [-quiet]               (Does not print any information)
        [-nocase]              (Reads in pattern in case insensitive)
        [-exact]               (Specifies exactly matching)
        [-of_objects obj_list] (Object list <pin, port, net, clock_root,
 clock_domain>)
        [patterns]             (Specifies name patterns)
```

Example

- The following example shows the output of the clock report for the multiple clocks in the example in section Specifying Multiple Clocks:

```
vcf> get_clocks
{"clk0", "clk1", "clk11"}
vcf> get_clocks clk1*
{"clk1", "clk11"}
vcf> get_clocks -filter {name==clk1}
{"clk1"}
```

- To get a list of virtual clocks in the design, use the following command:

```
vcf> get_clocks -filter is_virtual
```

4.11.9 Checking for Potential Clocking Issues in Formal Model

When properties use a gated clock such as `gated_clock = clock & clk_en` (where `clock` and `clk_en` are inputs), under simulation semantics used for formal analysis, clocks and inputs can change at different times leading to an unexpected property behavior. Additionally, if properties use a clock directly connected to an input but not defined as clock, then the sampled values might be incorrect as it will trigger at an unexpected time relative to the inputs.

To detect such potential clocking issues, set the `fv_property_check_clock` formal variable to true. When the `fv_property_check_clock` variable is set to true, VC Formal enables a static check during the model build that will check if the primary clocks of properties are not set up correctly.

By default, the `fv_property_check_clock` formal variable is set to false.

For example, when the clocks are tied to non-clock inputs, the following warning message is reported:

```
[Warning] FV_NO_CLOCK: Found 4 properties with clocks that are tied to
 non-clock free inputs.
     Please register the clocking signals below with the command
 create_clock.
     Property fvtop.n2 is clocked by [c]
     Property fvtop.p2 is clocked by [c]
     Property fvtop.p4 is clocked by [w]
     Property fvtop.px is clocked by [cx]
```

When the clocks are gated or complex and involve non-clock inputs, the following warning message is reported:

```
[Warning] FV_COMPLEX_CLOCK: Found 2 properties with complex clocking
 expressions that have signals that may be influenced by non-clock free
 inputs.
```

```
    Please check these inputs and register as clocks as necessary.
Non-clock                    control signals may also impact results and
may need to be constrained.
    Property fvtop.e2 is influenced by [c]
    Property fvtop.p3 is influenced by [c, i]
```

4.11.10 Limitation

Once the design is loaded, all access is relative to the wrapper and not the dut, since the wrapper introduces another layer of hierarchy.

```
vcf> create_clock dut.clk -period 100
vcf> create_clock ifx_a0.clk -period 100
```

Ensure that all references to clock, reset, and other constrained signals are relative to the wrapper.

4.12 Setting Up User-defined Resets

The formal analysis is run from an initial state. The initial state of the formal model, also known as the reset state, represents the value held in all state elements in the design. To specify reset state in a design, use the `create_reset` command.

Syntax

```
vcf> create_reset -help
Usage: create_reset    # Creates a reset
        [-sync]                (If signal drives synchronous reset. If
 this option is not provided then reset is taken as async by default.)
        [-async]               (If signal drives asynchronous reset.)
        [-both]                (If signal drives either asynchronous
 reset or synchronous reset or both. If this option is not provided then
 reset is taken as async by default.)
        [-type reset|set|both] (specifies the type of reset signal,
 Default: reset)
        [-name <reset_name>]   (specifies the name of reset signal)
        [-sense <low|high|any>]
                               (Specifies active value for set/reset,
 low means the reset is asserted with value 0, high indicates the reset
 is asserted with value 1. Default: high. When app-var enable_rdc or
 enable_cdc or enable_lint is set, then Default is used as low, otherwise
 high. This option is not recommended and the new option -value is to be
 used in place of -sense.:
                                Values: any, high, low)
        [-value <low|high>]    (Specifies active value for set/reset,
 low means the reset is asserted with value 0, high indicates the reset
 is asserted with value 1. Default: high. When app-var enable_rdc or
 enable_cdc or enable_lint is set, then Default is used as low, otherwise
 high.:
                                Values: high, low)
```

Feedback

```
        [-add]                  (Add new options to the original reset)
        [-tdr]                  (If signal is test and/or debug reset)
        [-disable_assertions_db]
                                (Disable AssertionDB generation for a
constraint)
        [-untimed]              (Specifies that reset signal is not a
clocked output of a sequential element)
        [source_objects]        (List of nets, ports and pins on which
this reset is defined)
```

This command specifies the reset and its active phase. This command implicitly does both `sim_force reset_active_value` and`set_constant reset_inactive_value`.

For VC Formal, the`-type`is not used. The only thing that VC Formal uses in the `create_reset` command is the -sense (low/high). This drives the asserted value of the reset signal. When a `create_reset` is encountered, the formal application gets a callback. If the sense is low we apply a force of 0 during simulation, and a constant 1 during formal analysis, and vice-versa if the sense is high.

Reset low, `create_reset rst -sense low` is the same as:

```
sim_force rst -apply 1'b0
set_constant rst -apply 1'b1
```

Reset high, `create_reset rst -sense high` is the same as:

```
sim_force rst -apply 1'b1
set_constant rst -apply 1'b0
```

Note:

VC Formal does not support virtual reset, and the usage of `create_reset -name rst`is not equivalent to `create_reset rst`.

4.12.1 Specifying Single Resets

You can specify a single reset state in the design using the `create_reset` Tcl command.

Example

To specify a single reset rst active high in the design, use the following command:

```
vcf>create_reset rst -sense high
```

Here, rst is set to 1 during reset phase and held constant as 0 during formal analysis.

Although random resets are allowed if reset pins are not defined by the `create_reset` or `set_constant` commands, it also implies that the reset itself is synthesizable and most assertions fail without `disable iff (reset)`. This may not be as intended.

Feedback

4.12.2 Specifying Multiple Resets

You can specify multiple resets in the design using multiple `create_reset` Tcl commands.

Example

In the following example, three reset signals are defined with different active values.

```
vcf>create_reset rst0 -sense high
vcf> create_reset rst1_n -sense low
vcf> create_reset rst2 -sense high
```

Here, rst0 and rst2 is 1 during reset phase and held constant 0 during formal analysis, while rst1 is 0 during reset phase and held constant 1 during formal analysis.

4.12.3 Getting a Report of User-defined Resets

To get a list of all the user-defined resets, use the `get_resets` command.

Syntax

```
vcf> get_resets -help
Usage: get_resets    # Returns reset objects, whose name matches with the
 specified pattern
        [-filter <filterexp>]  (Filter in those objects whose name
 matches with <filterexp>)
        [-regexp]              (Indicates filter expression type to be
 regular expression)
        [-quiet]               (Does not print any information)
        [-nocase]              (Reads pattern in case insensitive)
        [-exact]               (Specifies exactly matching)
        [-of_objects obj_list] (Object list <pin, port, net, reset_root>)
        [patterns]             (Specifies name patterns)
```

Example

- The following example shows the output of the reset report for the multiple resets in the example in section Specifying Multiple Resets.

  ```
  vcf> get_resets
  {"rst0", "rst1_n", "rst2"}
  vcf> get_resets rst1*
  {"rst1_n"}
  vcf> get_resets -filter {name==rst2}
  {"rst2"}
  ```

4.13 Specifying the Initial State

Formal analysis is run from an initial state. The initial state values in the formal model can be 0, 1, x, or z. If no initial state is specified, the default is to start with an initial state where all the state holding elements are of value x. This means that all possible values are considered for the state element when proving assertions. This may produce false failures because the formal analysis was based on an initial state that can never occur in the real design.

Most sequential designs do not operate properly without a reset procedure which establishes a valid starting state. You must apply the same sort of initialization when running formal analysis to get valid results.

Regardless of the method used to set the simulation model state, it needs to be loaded into the formal model using the `sim_save_reset` command before proving any assertions. Reset constraints may be used to define relationships between initial values of state elements without setting them to specific 0, 1, x and z values.

For example, the initial value of a register may be random but it has to be one-hot. These reset constraints are only applied during the reset phase as opposed to other constraints which are only applied during normal operation after reset.

Therefore, simply specifying only the reset ports as mentioned in section Specifying User-defined Clocksmay not be enough. Sometimes, some constraints on the initial state are required to produce best results.

There are several methods for specifying the initial state:

- Specific value assignment for all or certain state elements
- Specify reset state using a text file
- Simulation based reset state
- Reset state extraction from FSDB file
- Specify constraints on initial state values

4.13.1 Specific Value Assignment

Typically, the initial state represents a set of potential starting values (X, Z, 0 or 1) for all registers (sequentials including flip-flops and latches) in the design. An initial value of X denotes an uninitialized register where both 0 and 1 values are possible. Use the `sim_force`, `sim_load_state`, `sim_set_state`, and `set_constant` commands to set the initial state (or reset sate) for the entire design or specific registers.

Feedback

- `sim_set_state`

  The `sim_set_state` command is used to specify the initial value for registers explicitly or to replace the value assigned during reset simulation. These will be the register values used as initial state for formal analysis. Setting the initial state with the `sim_set_state` command is used as a convergence technique known as initial state abstraction. Initial state abstraction is particularly useful for counter abstraction, FSM abstraction, and other registers where the initial value is set to X as a way to reach deeper states faster.

  Use the `sim_set_state` command to set the initial value (X, Z, 0 or 1) directly, from an FSDB file, or readmemb/readmemh files for memories.

```
vcf> sim_set_state -help
Usage: sim_set_state    # Sets sequential state values to one of the
 four values: 0,1,x,z
       [-fsdb <<fName> <instName> <timeVal>>]
                                (Specifies the fsdb file for the reset
 state of sequentials)
       [-uninit_script <file_name>]
                                (Specifies the file name to be
 generated for those uninitialized sequentials absent in the fsdb
 file)
       [-apply <logicVal>]    (Specifies the logic value for the
 sequentials specified)
       [-exact]               (The list-of-seq-patterns should be
 used as sequential names without any wild-card expansion)
       [-regexp]              (The list-of-seq-patterns should use
 regular-expression expansion rather than the default glob-style
 expansion)
       [-all]                 (Sets given state value for all
 sequentials)
       [-uninitialized]       (Initialize the uninitialized
 sequentials in the design)
       [-readmemb <file memory_name [start_addr [end_addr]]>]
                                (Initialize the memory from the
 readmemb file)
       [-readmemh <file memory_name [start_addr [end_addr]]>]
                                (Initialize the memory from the
 readmemh file)
       [-exclude_package]     (Not to load signals from VC Formal
 packages)
       [-top <instance_name>] (The instance name under which the
 sequential values will be loaded)
       [-xml <file_name>]     (Specifies the reset xml file for the
 reset state of sequentials)
       [<list-of-seq-patterns-or-collection-of-sequentials>]
                                (Specifies a list of name patterns
 to match sequentials or a collection of sequentials in the design
 (default))
```

- VC Formal supports two methods for setting the initial state in a verification task. You can assign register values directly using the `sim_set_state` command, or you can load values from a specific time point in an FSDB file with `sim_set_state -fsdb`. After setting the initial state, use the `sim_save_reset` command to save this state and generate a task-specific reset.xml file. This process enables each verification task to maintain a unique reset state and its corresponding reset.xml file.

  To load an initial state from a previously saved reset.xml file, use the following command:

  ```
  sim_set_state -xml <reset.xml>
  ```

  This command imports the reset state from an existing reset.xml file into the current verification task. After running `sim_set_state -xml <reset.xml>`, execute `sim_save_reset` to create a new reset.xml for the current task, reflecting the imported state. This feature streamlines the sharing and reuse of reset states across tasks or runs, improving efficiency and consistency in the verification process.

- The `sim_set_state` command does not take effect if used before the `sim_run` command that runs reset simulation. Since VC Formal runs reset simulation automatically before formal proofs when the formal setup has changed, it is recommended to use `sim_set_state` and `sim_save_reset` just before proof commands and avoid changes that impact the formal setup in between.

- Setting formal variable `fml_reset_auto_generation` to `false` with `set_fml_var fml_reset_auto_generation false` disables automatic reset simulation when the formal setup is changed. Check section Automatically Establish Initial State for more details on the automatic reset simulation behavior.

- You cannot generate a composite trace FSDB file when register values are set with the `sim_set_state` command. Generate the FSDB file without composite trace and define an appropriate time stamp in the FSDB file to load correct value of the registers.

  Generate the composite trace using the following commands:

  ```
  % setenv SNPS_VCST_DISABLE_VF 1
  <Launch VC Formal session>
  vcf>fvtrace -composite <property_name>
  ```

- `sim_load_state`

  Use the `sim_load_state` command to take a snapshot of a property trace at the specified time, the values of the sequential signals at that moment will be used as the

Feedback

initial values in the next `check_fv`. In addition, the command truncates the property trace.

The first half of the trace will be used as the reset part for the next composite trace.

Syntax

```
vcf> sim_load_state    #Takes a snapshot of the property trace at the
 specified time
        -property <property_name> The name of the property for the trace)
        [-time <snapshot_time>]  (The time to take the snapshot of the
 trace)
        [-witness]              (Indicate the witness trace)
        [-vacuity]              (Indicate the vacuity trace)
        [-include_reset]        (Time is from a composite trace including
 reset)
        [-task <task_name>]     (The name of the task)
```

- `sim_force`

  Use the `sim_force`command is used to force a value onto one or more signals (nets). These forces override any previous forces and any design drivers. Values are the same format described for the `sim_set_state` command. The force is applied until it is explicitly released using the `sim_release` command, or until a `sim_reset -clear_forces`. A `sim_force` value overrides constants specified using `set_constant` during simulation to compute reset states. The `sim_force` values are not seen by formal algorithms (use `set_constant`).

Syntax

```
vcf> sim_force -help
Usage: sim_force    # Assigns a logic value to a signal in the design
 until explicitly released
        [-delay <delay_value>] (Specifies a delay in terms of number of
 System Clock cycles for the force to be applied)
        [-exact]               (The list-of-signal-patterns should be
 used as signal names without any wild-card expansion)
        [-regexp]              (The list-of-signal-patterns should
 use regular-expression expansion rather than the default glob-style
 expansion)
        [-apply <logic_value>] (Specifies a 4 value logic vector, e.g.
 4'b0z1x, 16'ha)
        [-apply_to_driver]     (Identifies the driver for the given net
 and applies the sim_force on the driver)
        [-signals <collection-of-signals>] (Specifies a subset of nets to
 be set by the command (default))
        [{list-of-signal-patterns}] (Specifies a list of name patterns to
 match signals in the design)
```

Feedback

- `sim_config`

  Use this command to specify certain configuration parameters for subsequent simulation. For example, use the `sim_config` command to configure the reset simulation. The `sim_config -rst_wave OFF` command turns off the waveform generation for the reset simulation.

Syntax

```
vcf> sim_config -help
Usage: sim_config    # Specifies certain configuration parameters for
 subsequent simulation
        [-refClock <clock_name>]
                                (Specifies the clock to use as the default
 clock)
        [-rst_wave <ON|OFF>]    (Turns waveform (FSDB) generation on and
 off)
        [-format <FSDB|VPD|ALL>]
                                (Specifies waveform format (default is
 ALL))
        [-file <file_name>]     (Specifies file to use for monitored
 signal values)
        [-units <ps|ns|ms>]     (Time units to use for monitored signal
 values)
        [-default_input <0|1|x|R>]
                                (Set default input value for reset
 generation)
        [-default_state <0|1|x|R>]
                                (Set default state value for trace replay)
        [-osc_limit <num_iteration>]
                                (Set the maximum iteration of
 oscillations)
        [-user_def_only <ON|OFF>]
                                (Put only user defined objects in the
 waveform)
        [-replay_all_wave <ON|OFF>]
                                (Generate wave forms for the entire design
 during replay)
        [-stable_limit <num_cycles>]
                                (Set the maximum number of reference clock
 cycles to reach stable states)
        [-fsdb_exclude_ports <ON|OFF>]
                                (Not to save the traces of the ports)
        [-check_pending_forces <ON|OFF>]
                                (Check the uncommitted sim_force commands
 before check_fv)
        [-fix_sim_clock <ON|OFF>]
                                (Fix some of the clock issues, such as the
 edge alignment and the period truncation)
        [-no_event_trigger_in_setup <ON|OFF>]
                                (No sequential event triggering in sim_run
 setup)
```

Feedback

```
        [-xprop <ON|OFF>]       (Simulation for XPROP)
        [-enable_xz_edge_control <ON|OFF>] (Enable treating x and
z transitions to/from 0/1 as posedge and negedge transitions for
sequentials in simulation. If this is set to false, only transitions
from 1 to 0 and 0 to 1 will be considered as negedge and posedge
transitions respectively (default is ON))

        [-put_x_for_unstable_register <ON|OFF>]
                                (Put X for unstable registers)
        [-useNewLevelize <ON|OFF>]
                                (Use new levelize algorithm)
        [-use_bitwise_operator <ON|OFF>]
                                (Use bit precise arithmetic operations
(default is ON))
        [-report_uninit <ON|OFF>]
                                (Report Uninitialized registers after
sim_run)
        [-uninit_count <num_uninitialized_registers>]
                                (Specify the number of uninitialized
registers to be reported)
        [-uninit_file <file_name>]
                                (List the uninitialized registers to a
file)
        [-apply_force_to_driver]
                                (Identifies the driver for the given net
and applies the sim_force on the driver)
        [-uninitialized <user_only|all>]
                                (Specifies the set of uninitialized
sequentials to be used for sim_set_state -uninitialized:
                                 Values: user_only, all)
```

- `set_constant`

  Use the `set_constant`command is used to assign constant values to nets in the design; this constant value overrides any other potential drivers of the signal.

Syntax

```
vcf> set_constant -help
Usage: set_constant    # Set net to constant value
        -value <constant>       (specifies the constant value (verilog
 number/integer))
        [-quiet]                (Does not print any information)
        [-nocase]               (Case insensitive)
        [-hierarchical]         (Lookup using paths relative to current
 design scope)
        [-regexp]               (Indicates filter expression type to be
 regular expression)
        [-exact]                (Specifies exactly matching)
        [-global]               (Apply to all formal apps)
        [-filter <expr>]        (filters the collection with the value of
 the expression)
        [-of_objects object_list]
```

Feedback

```
                                (Object list)
       [-name <constraint_name>]
                                (Specify the name of this constraint)
       [-undriven]              (Apply the value to all undriven bits)
       [net name patterns]      (Pattern names)
```

- **Note:**

  The reset should be set to active value during reset phase and automatically held constant in inactive state during formal analysis. This way, properties are not falsified by reset being asserted randomly during formal analysis.

Examples

- Set all the state elements to value 0:

```
sim_run -stable
sim_set_state -all -apply 0
sim_save_reset
```

- Set all uninitialized state elements to value 0:

```
sim_run -stable
sim_set_state -uninitialized -apply 0
sim_save_reset
```

- Set some of the state elements to value 1:

```
sim_run -stable
sim_set_state *seqName* -apply 1
sim_save_reset
```

- Set 4-bit counter timer initial value to X for initial state abstraction:

```
sim_run -stable
sim_set_state traffic.first.timer -apply 4'bx
sim_save_reset
```

- Start formal analysis from an uninitialized state by skipping the `sim_run` command:

```
#sim_run -stable # Commented out to skip reset simulation
sim_save_reset
check_fv
```

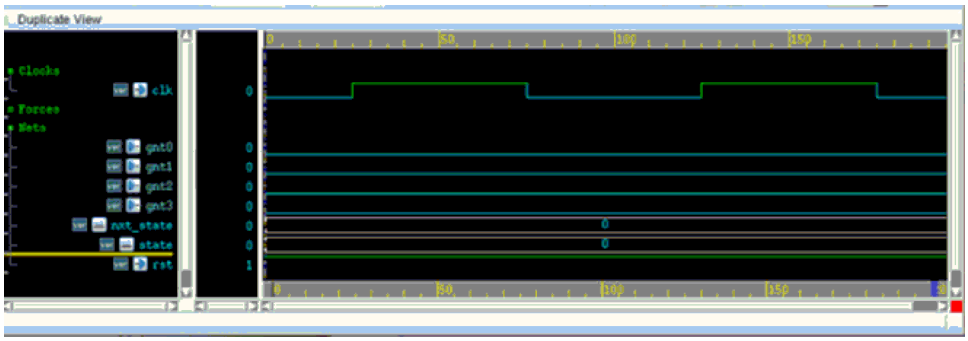
- Disable automatic reset simulation with formal variable `fml_reset_auto_generation`:

```
sim_run -stable
sim_set_state -traffic.first.time -apply 4'bx
sim_save_reset
set_fml_var fml_reset_auto_generation false
<Commands that affect formal setup potentially causing automatic reset
 simulation>
check_fv
```

Feedback

- Consider the example design shown in Figure 16. The registers is set to 0 when rst is asserted in simulation. However, if no proper steps are taken to setup the initial state, gnt* does not automatically get reset to 0. Their values remains as X as the initial condition in the formal model.

Figure 16: Reset Example in RTL



Suppose an assertion is written to check one of the properties of gnt*:

```
assert_gnt_oh0 : assert property (@(posedge clk)
$onehot0({gnt3,gnt2,gnt1,gnt0}));
```

If the design is not initialized for formal analysis, the property fails the onehot0 because the gnt registers are uninitialized:

```
vcf> sim_get {gnt3 gnt2 gnt1 gnt0}
{x x x x }
```

When all the gnt registers initialized to 0, the assertion passes. To initialize the gnt registers to 0:

```
vcf> sim_set_state -apply 1'b0 {gnt0 gnt1 gnt2 gnt3}
```

You can also use the `get_sequentials` command with the `sim_set_state` command to initialize the gnt registers to 0:

```
vcf> sim_set_state -apply 1'b0 [get_sequentials gnt*]
{"gnt0", "gnt1", "gnt2", "gnt3"}
```

Note:

Over constraining the reset state could potentially mask a real design bug.

4.13.2 Specifying Reset State Using a Text File

You can provide a reset sequence in a text file format along with the *sim_run* command:

```
vcf> sim_run -file reset_seq.txt
```

The reset sequence file should consist of a list of signals, and their corresponding values in reset, followed by the reset duration. The reset simulation run cycles are specified by the line starting with the number, which can be followed by the clock name. If no clock is provided in the file, the *sim_run*command takes the default clock for driving simulation.

Example

If you have a file named *reset_seq.txt* with the following contents:

```
<top>.<a0>.<b0>.<c0> 1'b1
<top>.<a0>.<b0>.<c1> 2'b01
<top>.<a1>.<b1>.<c2> 1'b0
.
.
100 clk
<top>.<a0>.<b0>.<c0> 1'b0
<top>.<a0>.<b0>.<c1> 2'b10
.
.
100 clk
```

The *sim_run* command reads the hierarchical path for each of these signals and sets it to the corresponding value for the reset duration.

You can specify the reference clock in the file or it can also be specified with the *sim_run -clk*option. The clock given with *-clk*option of *sim_run* command takes precedence when the clock is specified using the both the options.

Note:

You can define multiple value change in the same text file.

4.13.3 Simulation Based Reset State

In most cases, the reset state of the design can be determined by asserting the reset pin and clocking the design with some number of clock cycles by running the internal netlist simulator. The state of all sequential elements, after the simulation, is then loaded into the formal model. Any subsequent analysis uses the saved reset state. Sometimes, a more specific sequence may be applied to put the design in a legal reset state. A built-in netlist simulator can be used to perform this task.

Initializing in the simulation-based reset is two-step process:

1. Set up the desired reset values using `sim_config, sim_force` and `sim_set_state` type of commands, followed by or inter-weaved with `sim_run` command.
2. The state of the simulation model is loaded into the formal model using the `sim_save_reset` command.

Feedback

By default, all inputs without a constant constraint, or signals in `create_reset`, or `create_clock` is driven as X during the simulation. This is usually fine since the sequential elements that can be reset automatically initializes to the correct state during the reset phase.

Example 1

Sometimes, specific values need to be applied to all the inputs for the design to be in a legal state. To drive a constant 0 or constant 1 to all inputs during reset:

- The clock signal *clk* is defined with period 100.

  vcf>

- The reset signal *rst* is set to its active value high for the `sim_*` commands. It is then held to its inactive value low during the formal analysis phase.

  ```
  vcf> create_reset rst -sense high
  ```

- Set default input value during simulation to 0.

  ```
  vcf> sim_config -default_input 0
  ```

- Run simulation for 10 clock cycles.

  ```
  vcf> sim_run 10
  ```

- Load the current simulation state value of all storage elements into the formal model. This state will be used as the starting state for formal analysis.

  ```
  vcf> sim_save_reset
  ```

Example 2

To apply certain values to the specified inputs or any signals during reset.

- The clock signal *clk* is defined with period 100.

  ```
  vcf> create_clock clk -period 100
  ```

- The reset signal *rst* is set to its active value high for the `sim_*` commands. It is then held to its inactive value low during the formal analysis phase.

  ```
  vcf> create_reset rst -sense high
  ```

- Apply 0 to input pins `req*` during simulation.

  ```
  vcf> sim_force req* -apply 0
  ```

- Apply 0 to signal `arb.req_queue` during simulation.

  ```
  vcf> sim_force arb.req_queue -apply 0
  ```

- Run the simulation until design is in stable state using the `sim_run -stable` command. This command runs the simulation until the values of all sequential logic in the design are stable (that is, the values do not change regardless how many more cycles are run). If it is not possible to reach a stable state, VC Formal issues an error reporting the oscillation situation in design.

  ```
  vcf> sim_run -stable
  ```

- Load the current simulation state value of all storage elements into the formal model. This state will be used as the starting state for formal analysis.

  ```
  vcf> sim_save_reset
  ```

It is common to have uninitialized state elements after the initialization process is complete. For example, data registers, delay registers or memories do not need to be initialized to a specific value. Uninitialized control registers may cause false assertion failures and require additional debugging.

Automatically Establish Initial State

If you have specified the clock and reset definition, but do not provide complete initialization information before running the `check_fv` command, VC Formal automatically establishes the initial state based on the usage of the `sim_run` and `sim_save_reset` commands.

The following table describes how the initial states are automatically saved based on the different scenarios of `sim_run` and `sim_save_reset` command usage.

Note:

If no clock is specified, automatically run will not occur, but `sim_save_reset` will always be issued in any case, whether user issued the command or not.

	sim_run	sim_save_reset	Tool Behaviour
Case 1 Standard Usage `check_fv`	No	No	Auto Save
Case 2 Both sim_run -stable and sim_save_reset are applied `sim_run -stable sim_save_reset check_fv`	Yes	Yes	golden
Case 3 Only when sim_run -stable is used `sim_run -stable check_fv`	Yes	No	Auto Save
Case 4 When only sim_save_reset are applied `sim_save_reset check_fv`	No	Yes	Auto Save

	sim_run	sim_save_reset	Tool Behaviour
Case 5 `sim_run 1 -clk clk sim_force in1 -apply 'b0 sim_run 1 -clk clk check_fv`	User specified sim_run behavior	No	No additional sim_run occur when user specify any sim_run command. The sim_save_reset is applied.

The automatic reset generation in `check_fv` can overwrite previous reset state. This can impact proof results, particularly for scenarios where a complex reset sequence was previously applied.

When using `check_fv`, automatic reset generation can overwrite the existing reset state.

A complex reset sequence is characterized by any of the following:

- Applying `sim_force` or `sim_release` after `sim_run`.

```
sim_run
sim_force sig1 -apply 0 // or sim_release sig1
sim_save_reset
```

- Using `sim_set_state` or `sim_load_state` commands.

```
sim_run
sim_set_state -uninitialized -apply 0
sim_save_reset
```

- Executing multiple `sim_run` N commands, where N is the number of reference clock cycles.

```
sim_run 5
sim_save_reset
sim_run 10
sim_save_reset
```

The tool can now identify if a complex reset sequence was used before `check_fv`. In such cases, the following warning is displayed:

```
check_fv -block
```

[Warning] COMPLEX_SIM_DETECTED: A model change has introduced uninitialized registers. Automatic reset may not accurately reproduce the intended reset state due to the detected complex reset sequence.

To ensure correct results, it is recommended to reset the simulation model with 'sim_reset' and repeat the exact simulation command sequence before running check_fv again.

Feedback

Monitor Simulation Behavior

You can monitor the behavior of signals during simulation using the `sim_add_monitor`command:

Example

```
vcf> sim_add_monitor [get_sequentials *gnt*]
[Info] SIM_I_CREATE: Create Simulation Model.
{"gnt0", "gnt1", "gnt2", "gnt3"}

vcf> sim_run -stable
0 gnt0=1'bx gnt1=1'bx gnt2=1'bx gnt3=1'bx
50 gnt0=1'bx gnt1=1'bx gnt2=1'bx gnt3=1'bx
100 gnt0=1'bx gnt1=1'bx gnt2=1'bx gnt3=1'bx
150 gnt0=1'bx gnt1=1'bx gnt2=1'bx gnt3=1'bx
200 gnt0=1'bx gnt1=1'bx gnt2=1'bx gnt3=1'bx

vcf> sim_add_monitor [get_sequentials -word -hierarchical]
{"first.green", "first.red", "first.state_out", "first.timer",
 "first.yellow", "main.green", "main.red", "main.state_out",
 "main.timer", "main.yellow", "s_reg1", "s_reg2", "s_reg3"}

vcf> sim_run 1
150 first.green=1'b0 first.red=1'b1 first.state_out=5'b00001
 first.timer=4'b0001 first.yellow=1'b0 main.green=1'b0 main.red=1'b1
 main.state_out=5'b00001 main.timer=4'b0001 main.yellow=1'b0
 s_reg1=6'b000000 s_reg2=6'b000000 s_reg3=6'b000000
200 first.green=1'b0 first.red=1'b1 first.state_out=5'b00001
 first.timer=4'b0001 first.yellow=1'b0 main.green=1'b0 main.red=1'b1
 main.state_out=5'b00001 main.timer=4'b0001 main.yellow=1'b0
 s_reg1=6'b000000 s_reg2=6'b000000 s_reg3=6'b000000
1
```

4.13.4 Reset State Extraction from FSDB file

The reset state for the design can also be extracted from an FSDB file from testbench simulation. The hierarchy is usually different in the saved FSDB file, since the top level in the testbench is not the same as the design top level module used in formal verification. The instance path name in the simulation environment to the top level module in the formal model needs to be specified so that the signal names match between the design and the FSDB file.

The FSDB file is read in using the `sim_set_state` command.

Example

```
vcf> sim_set_state -fsdb {arb_test.fsdb testbench_arb.arb_top.arb
1704971ps}
```

The three arguments to the `-fsdb`option are:

- Name of the FSDB file
- Hierarchical path down to instance matching the top level module used for VC Formal
- The time in the file to load values

In this example, the values of all the state elements in design arb from FSDB file `arb_test.fsdb` at simulation time 1704971ps are extracted and loaded into the formal model as the initial state to start formal analysis from.

Sometimes, not all sequential elements in the setup for formal verification are present in the FSDB file. They may not be initialized to the proper states.

The following command generates a file with all the sequential elements not present in the FSDB, examines the file, sets it to the proper value and then sources in the Tcl command.

```
vcf> sim_set_state -fsdb {test.fsdb tb.arb_top.arb 1704971ps}
-uninit_script filename
```

Where, *filename* is the generated file that lists all the sequentials not present in *test.fsdb*. You can modify the sequential values in the file and source it again.

Currently, only the FSDB file format is supported for the reset state extraction from simulation. VPD and VCD file formats need to be converted to FSDB files first. Two separate programs can be used for this purpose:

- `vpd2vcd`
- `vcd2fsdb`

4.13.5 Limitation

Once the design is loaded, all access is relative to the wrapper and not the dut since the wrapper introduces another layer of hierarchy.

```
vcf> create_reset dut.resetn -sense low
vcf> create_reset ifx_a0.resetn -sense low
```

Ensure that all references to clock, reset and other constrained signals are relative to the wrapper.

Feedback

4.13.6 Loading Reset State from Property trace

Without the `view_trace -composite p1` issued before, the behavior of `sim_load_state` is as follows:

1. `sim_load_state -property p1`

   The snapshot time is automatically set to the ending time of the original trace. The reset trace will not be inserted to the new reset trace.

2. `sim_load_state -property p1 -time N`

   The user specified snapshot time N should be less than or equal to the ending time of the original trace. The reset trace will not be inserted to the new reset trace.

3. `sim_load_state -property p1 -time N -include_reset`

   The user specified snapshot time N should be greater than the length of reset trace, and less than or equal to the length of the supposed composite trace, which is the concatenation of the reset trace length and the l original trace. The reset trace will be inserted to the new reset trace.

4. `sim_load_state -property p1 -include_reset`

   Same as 3 with the snapshot time equals to the length of the supposed composite trace.

When `view_trace -composite p1` issued before, the behavior of `sim_load_state` is as follows:

1. `sim_load_state -property p1`

   The snapshot time is automatically set to the ending time of the composite trace.The composite trace of p1 will be the new reset trace.

2. `sim_load_state -property p1 -time N`

   The user specified snapshot time N should be greater than the reset trace, less than or equal to the length of the composite trace. The trimmed composite trace with length N will be the new reset trace.

3. `sim_load_state -property p1 -time N -include reset`

   Same as 2.

4. `sim_load_state -property p1 -include_reset`

   Same as 2 with the snapshot time automatically set to the length of the composite trace.

Feedback

4.14 Debugging the Reset State

If assertions fail unexpectedly, the reset state may be incorrect. You can debug the reset state generating a reset waveform trace or reporting the value of the sequential elements of interest.

By default, the generation of a reset trace is enabled by default. The reset waveform generation can be resource intensive. It should only be enabled when debugging reset state. Once the reset state is verified, disable the reset waveform generation by setting the `-rst_wave` option to OFF:

```
vcf> sim_config -rst_wave OFF
```

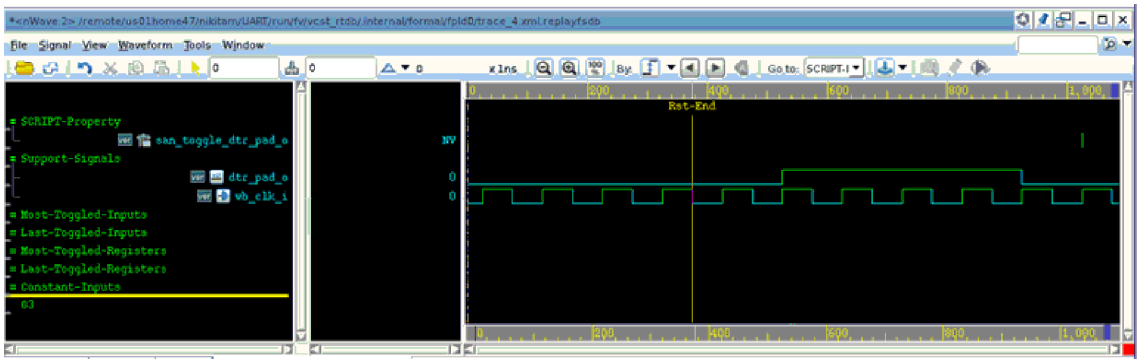
If the `sim_config -rst_wave ON` command is set before `sim_run`, the reset waveform is generated upon `sim_save_reset`.

To view the reset waveform in the GUI:

- Right-click a property in the GoalList, and select *View Trace* > *Reset Only*.
- Click *View Trace* > *Property +Reset*to bring up the reset trace in the waveform viewer.

  Figure 17 shows the start reset debug trace using the GUI.

Figure 17: Start Reset Debug Trace From GUI



Alternatively, you can use the `view_trace -reset` command to view the reset trace.

4.14.1 Getting the Reset State Values

You can also get the reset state values using the `get_sequentials` and `sim_get` commands.

Example

- ```
  vcf> get_sequentials
  "nxt_state[2]" "nxt_state[1]" "nxt_state[0]" "gnt3" "gnt0" "gnt2"
   "gnt1" "state[2]" "state[1]" "state[0]"
  ```

- ```
  vcf> sim_get gnt0
  { gnt0=0 }
  ```

  The `sim_get` command returns the value of a net in the simulator at the current time. If used right after the reset simulation, it returns the value in the reset state.

- By combining the commands, you can get the reset value of all state elements in the design:

  ```
  vcf> sim_get [get_sequentials *gnt*]
  { gnt0=0 gnt1=0 gnt2=0 gnt3=0 }
  ```

4.15 Verifying VC Formal Designs

Even simple mistakes, such as forgetting to add a `create_clock` command has significant impact on the formal modeling and can lead to incorrect results that may take a lot of effort to track down and analyze the root cause. VC Formal provides a way to check for such possible potential oversights or design issues as early as possible, before formal engines start checking on the correctness of properties. Therefore, you must verify the formal design before performing checks on the properties.

Investigating false failures due to issues in the design is very time consuming and in many cases be avoided if static checking of specific design issues can be done prior to the formal checks. VC formal supports checking for combinational loops in the design before running formal checks. It also performs multiple drivers check where combinational loop is mentioned.

The combinational loop checks can be run at any point or at specific check points after the design is loaded. All the snips specified prior to the check are considered for the combinational loop check. By default, the combinational loop checks is automatically run right after the design is loaded.

The POTENTIONAL_SETUP_ISSUE message is reported when `check_fv` is run before `check_fv_setup`. Even though `check_fv_setup` has not been run, when the formal model is being, the internally generated model checks are run where potential issues are discovered.

However, these checks are only built but not run. VC Formal reports the POTENTIONAL_SETUP_ISSUE message to suggest to run `check_fv_setup` and `report_fv_setup`. This way, these internal generated checks are run and more accurate information is available. You can get to know whether the “potential” issue is a real setup issue based on the `check_fv_setup` results.

Feedback

4.15.1 Configuring Design Checks

You can explicitly run all or some of the checks. You can also disable certain checks depending on which point in the flow that the `fv_setup_config` command is used.

Syntax

```
vcf>fv_setup_config [-check <list-of-check-names>] [-category <category>]
 [-severity <severity>] [-autorun <checkpoint>] [-disable] [-enable]
 [-list] [-verbosity <message_level>]
```

Where,

- `[-check <list-of-check-names>]:` Specify the list of checks to be performed. If no name is specified, it defaults to all checks: Values: *clock, comb_loop, glitch, multi_driver, osc_loop, osc_seq, reset*)
- `[-severity <severity>]:`Specify the severity to be used when running the check. By default, all the checks have a severity of warning. Values: error, info, warning. When set to *error*, VC Formal errors out if any combinational loops are detected.
- `[-category <category>]`: Specifies the category of the check to run. Values: `all, general, modport`.

  `-category` for any category other than `all` requires exactly one `-check` to be specified.

  - `all`: Specifies all categories for the check.
  - `general`: Applies to all checks.
  - `modport`: modport applies to `-check multi_driver`. Category 'modport' collects multidriven nets where there is atmost one generic driver, and the rest are driven by interface output modports that are not assigned a value other than z's.

    For example, interface I has modport M that assigns port P as an output - if three instances all use modport M, then P is driven by all three instances. However, if there is only an explicit assignment in one instance, the implicit 'z' assignments on P from the other instances generally is not a real problem, so they are separated from generic multidriver situations. The output modports does not feed values backwards into the child instances.
- `[-autorun <checkpoint>]`: Automatically run this check at the specified checkpoint: Values: `design_load`.

  Note:

  In this release, the combinational loop check can only be run at design load checkpoint.

- `[-disable]`: Specify this option to disable the specified check.
- `[-enable]`: Specify this option to enable the specified check.
- `[-list]`: Specify this option to list the configuration.
- `[-verbosity <message_level>]`: Specify the verbosity level of messages for the specific check while performing the checks. The default verbosity is summary: Values: none, summary, verbose.

Example

- The following command configures the check severity to error, which makes the tool to error out if any combinational loops are detected.

```
fv_setup_config -check comb_loop -severity error
```

- The following command disables the combination loop checks

```
fv_setup_config -disable
```

4.15.2 Running Formal Design Checks

To perform formal design checks, use the following command:

```
vcf> check_fv_setup
```

Syntax

```
vcf> check_fv_setup [-block] [-run_finish <cmds>] [-stop] [-break
 <number-of-falsifications>] [-check <list-of-checks>] [-category
 <category>] [-verbose]
```

- `[-block]:`: When this option is used with `check_fv_setup`, the shell is blocked until command execution is complete. If used without the `-block` option, `check_fv_setup` does not block the shell and you can query the current status using the `report_fv_setup` command.
- `[-run_finish <cmds>]`: Specifies the set of commands to run when this command finishes in non-blocking mode.
- `[-stop]`: Specify this option to stop execution of the command.
- `[-break <number-of-falsifications>]`: Specify this option to stop execution after the specified number of falsifications.
- `[-check <list-of-checks>]`: Specify the list of checks to be performed. If no name is specified, it defaults to all checks: Values:*clock, comb_loop, glitch, multi_driver, osc_loop, osc_seq, reset*)

- `[-category <category>]`: Specifies the category of the check to run. Values: `all, general, modport`.

  `-category` for any category other than `all` requires exactly one `-check` to be specified.

  - `all`: Specifies all categories for the check.
  - `general`: Applies to all checks.
  - `modport`: modport applies to `-check multi_driver`. Category 'modport' collects multidriven nets where there is atmost one generic driver, and the rest are driven by interface output modports that are not assigned a value other than z's.

    For example, interface I has modport M that assigns port P as an output - if three instances all use modport M, then P is driven by all three instances. However, if there is only an explicit assignment in one instance, the implicit 'z' assignments on P from the other instances generally is not a real problem, so they are separated from generic multidriver situations. The output modports does not feed values backwards into the child instances.

- `[-verbose]`: Report violations item by item.

By default, the `check_fv_setup` command is run in non-blocking mode. You can interact with the vcf while it is running in the background. If you call the `check_fv_setup` command with the `-block` option, the shell is blocked until the `check_fv_setup` command is executed.

The `check_fv_setup` command also checks for power logic anomalies through the power switches. For the power logic anomalies checks, UPF must be read (`read_upf`) prior to the command.

4.15.3 Debugging Formal Design Checks Results

Use the `report_fv_setup`command to get the results of the `check_fv_setup` command. The `report_fv_setup` command prints a summary of the violations that are found in the formal setup.

The `report_fv_setup` command can be executed after the `check_fv_setup`command or when the `check_fv_setup` is running (in non-blocking mode). In the latter case, the `report_fv_setup` command prints only the current status.

Syntax

```
vcf> report_fv_setup -help
Usage: report_fv_setup    # Reports the results of the check_fv_setup
 command
        [-list]                 (Reports violations item by item.)
        [-limit <int>]          (Limits number of printed nets in -list)
```

Feedback

```
       [-check <list-of-check-attributes>]
                                (check type selection (default is all
checks):
                                 Values: clock, comb_loop, glitch,
                                 multi_driver, osc_loop, osc_seq, reset)
       [-category <category>] (Specifies the category of the check
to report. Requires -check with a single check. 'all' specifies all
categories for the check. 'general' applies to all checks. 'modport'
applies to check 'multi_driver'.
                                 Values: all, general, modport)
```

Example

- The following is an example output for the `report_fv_setup` command:

```
Design/Setup analysis results
  ----------------------------
  Reset violations                 :    2
  Clock violations                 :    0
  Glitch violations                :    0
  Oscillating pure comb. loops     :    0
  Oscillating latch/ff comb. loops :    0
> Model is incorrect due to design/setup issues
    Please fix issues before continuing.
```

- If the `report_fv_setup` command is executed during a non-blocking call to the `check_fv_setup` command, it prints the current status, that is, how many violations were checked and how many were detected:

```
Design/Setup analysis results
  ----------------------------
  Reset violations                 :    0     running 3(3)
  Clock violations                 :    0     done
  Glitch violations                :    0     done
  Oscillating pure comb. loops     :    0     running 25(25), 25(25)
 checks
  Oscillating latch/ff comb. loops :    0     running 15(15), 825(825)
 checks
  > Checking in progress ...
```

  The numbers in parentheses are the total number of checks, for example, `running 3(3)` means that currently 3 checks out of total 3 are still running.

- Use the `-list`option to get a more detailed description of the results:

```
Design/Setup analysis results
  ----------------------------
> Reset violations (grouped by reset expressions)
      Reset of these flip-flop groups are not always inactive during
 formal:
[   0]    signal_stretcher
      [   1]    jdi_assert.queue.state
                jdi_assert.queue.head
```

Feedback

```
                jdi_assert.queue.tail
Summary
  -------
  Reset violations                 :    2
  Clock violations                 :    0
  Glitch violations                :    0
  Oscillating pure comb. loops     :    0
  Oscillating latch/ff comb. loops :    0
  > Model is incorrect due to design/setup issues
    Please fix issues before continuing.
```

The violations are grouped such that each group has the same cause. In the example above, `jdi_assert.queue.state, jdi_assert_queue.head` and `jdi_assert.queue.tail` are flip-flops that have the same reset input. This reset-input is not always inactive during the formal phase and thus causes the violation to be reported.

- The following is an example output of the `report_fv_setup` command:

```
report_fv_setup -check comb_loop -list

  Design/Setup analysis results
  -----------------------------

  > Combinational loops

      Loops comprise these groups of net:

      [   0]    t3
                o1
                t1
                t2

  Summary
  -------
 Combinational loops              :    1
```

- The following example shows how power logic loops are reported. It reports a loop where the power switch is controlled by a signal which is present in the power domain controlled by the same power switch therefore resulting in the anomaly.

```
> report_fv_setup -check comb_loop -list

  Design/Setup analysis results
  -----------------------------
> Combinational loops
      Loops comprise these groups of net:
      [   0]    V1SW (Supply Net)
                Associated Internal Wire for V1SW (Supply Net)
                my_pipeline1[1].out1
                ctrl
                Power Switch: V1_header_switch (Control Net)
```

Feedback

```
                  V1SW (Supply Net)
Summary
-------
Combinational loops
```

4.15.4 Example to Run and Configure Combinational Loop Checks

The following are examples that show how to run and configure combinational loop checks.

- To run the check in default configuration and get a report the results:

```
vcf> read_file -format verilog -top top dut.v
vcf> report_fv_setup -check comb_loop -list
```

- To configure and run the combinational loop check after the design is loaded:

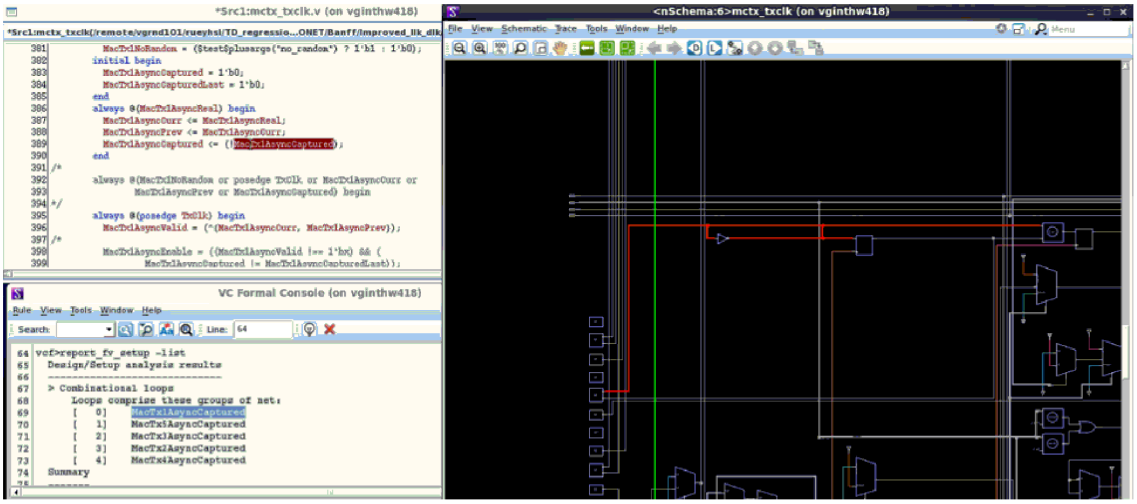
```
vcf> fv_setup_config -check comb_loop -severity error
vcf> check_fv_setup -check comb_loop
vcf> report_fv_setup -check comb_loop -list
```

- To debug the combinational loops in GUI:

```
vcf> read_file -format verilog -top top dut.v
vcf> start_gui
```

Figure 18 shows debugging the loop in GUI mode.

Figure 18: Debugging Combinational Loops Checks



Feedback

Watch this video to understand how to debug combinational loops (combo loops) in VC Formal. Combinational loops are logic loops without registers between nodes, which can complicate timing analysis and reduce fault coverage. Oscillating combinational loops are particularly problematic as they produce unpredictable outputs.

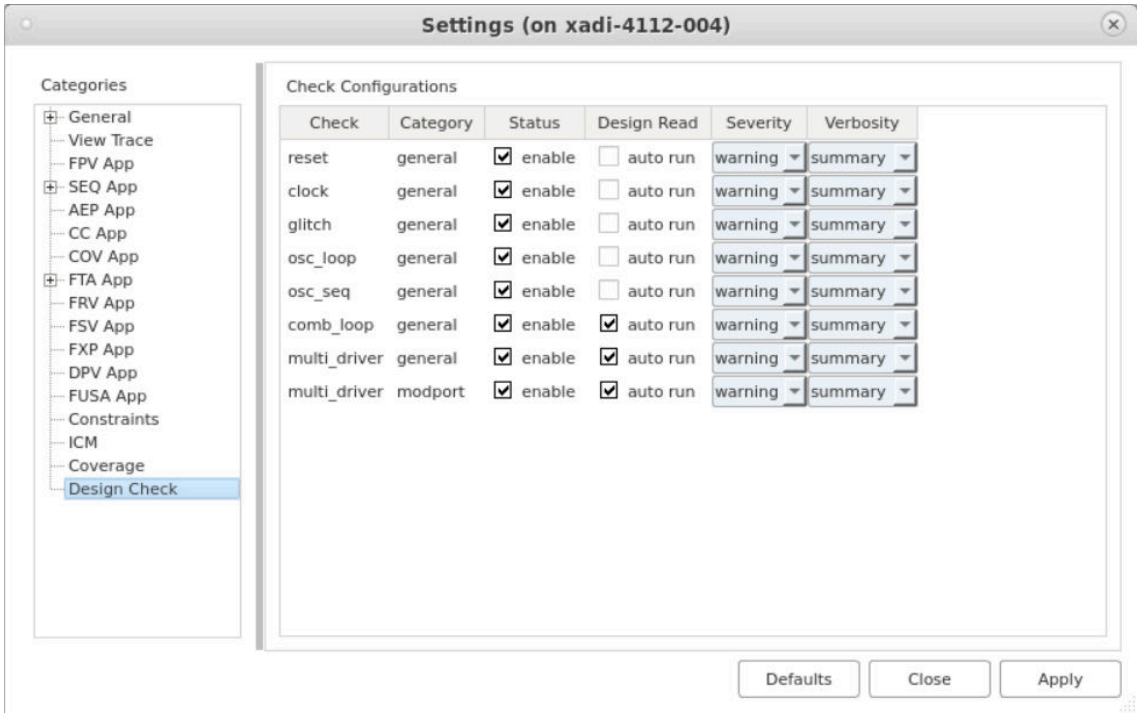
4.15.5 Configuring Formal Design Checks in the GUI

You can configure formal design checks in the GUI using the *Customize View Settings* dialog box.

1. Click in the VC Formal toolbar to open the *Customize View Settings* dialog box.

   The configurations related to the `fv_setup_config` command appears when you select *Design Check* in the categories.

Figure 19: Customize View Settings

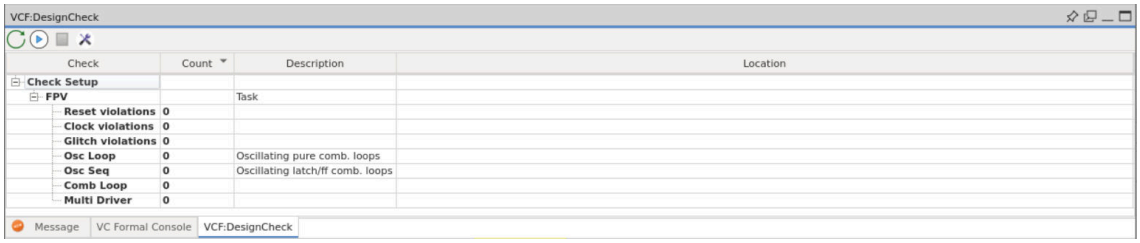


2. Select the required checks to be run and the other settings. Click ***Apply***.

4.15.6 Running Design Checks in GUI

1. To run Design checks, click and select *Verification Setup Reports*.

   The *DesignCheck* table appears. Each signal names in the table are grouped according to their check type (osc_loop, osc_seq, glitch, and so on).



2. Click to run the design checks.
3. Right-click on the signal and select *Show Schematic* and *Show Source* to view schematic or source of the signals.

   Alternatively, select a signal, and click to open the schematic view or the source view.

   - *Show Schematic* opens the schematic view and displays the signal in the view.
   - *Show Source* opens the source view and highlights the signal name in source code.
4. Click to refresh the design check reports.
5. Click to stop the design checks.

4.15.7 VC Formal Design Checks

The following structural checks and AEPs are checked in the `check_fv_setup` command.

Feedback Loop AEPs

Formal modeling automatically cuts the loop and set the cut to a symbolic value and then unrolls combinational loops n times (where n is configurable). If the loop is a good combinational loop that is converging, the symbolic value at the cut does not influence the values of the nets in the loop.

For oscillating combinational loops, the symbolic value will influence the values of the nets and then when you get a counter-example, you typically see that the cut and the inputs to the cut have diverging values. For example, assume we cut at the output of an and-gate. If the loop is not converging, we can have the situation that after n times unrolling the inputs to the and-gate are 0, 0 but the output is 1.

Sequential feedback loops are most often introduced in latch-based designs where all transparent latches in a feedback loop are open at the same time. This is usually a setup problem. Latch-based designs usually have primary/secondary latches which operate on different phases of the clock. Most likely, the clocks are setup incorrectly so that both the primary as well as secondary latch are open at the same time.

You can use the `report_fv_setup -list` command to print all the nets that are involved in the combinational cycle.

Glitch Detectors

If a signal switches multiple times in a time-step, a glitch occurs and the design behavior is unpredictable. Glitches are a sign of bad setup and design, and they must be removed. Glitches are often introduced in designs with faulty clock-gating logic or designs with races. The `check_fv_setup`command only tests clock-inputs of flip-flops for glitches because these are the points where they cause the most harm. They cause the flip-flop to switch even though the clock input then goes back to its original value. The spike does not appear in the waveform and the reason why the flip-flop changed its state is not known.

You can use the `report_fv_setup -list` command to get the names of the flip-flops whose clock-input has a glitch. You can then use the source-viewer or schematic-viewer to trace back the clock-input to find the faulty clock-gating cell.

The `report_fv_setup` command groups the glitches into equivalence classes. The cause of a glitch is the same for every flip-flop in an equivalence class, therefore if you debug one of them, you can easily find the cause for all the others in the same class. Equivalence classes are numbered with square brackets *[n]*.

Clock Violations: Flip-flops Changing when Inputs Change

In most designs, flip-flops are allowed to change with a clock. If they change with an input-change, then there is an issue. Most likely, you have forgotten to specify a `create_clock`command. Even for clock-gating designs where the flip-flop clock input depends on clocks and inputs, the flip-flop should only change with the clock and not with any of the inputs.

You can use the `report_fv_setup -list`command to print the names of all flip-flops whose clock inputs switch at input changes. Again, the flip-flops are grouped into equivalence classes.

Reset Violations: Flip-flops with Non-constant Reset Expressions

In most designs, flip-flop resets remain asserted during the formal phase. Asynchronous resets that are active during formal proofs are typically considered setup issues, such as when the `create_reset` command is missing. However, a de-asserted reset is not always an error. In some cases, the verification intent is to leave asynchronous resets unspecified. In these situations, reset violations are reported as warnings, and it is the responsibility of

the verification engineer to determine whether to add a `create_reset` command to the setup.

You can use the `report_fv_setup -list` command to print the names of the flip-flops with non-de-asserted reset. The flip-flops are grouped according to the functions on their reset-pins.

If a register's reset is controlled by a user-defined sequential element, such as a pipelined reset, it is not considered a violation if the reset remains active immediately after reset. A violation is only reported if the reset re-activates for a register after it has been deactivated.

4.16 Support for Trace Checks

Once a property is falsified, a check on the trace may yield information that helps in debugging the failure. Once a trace is generated, the following information about the trace may help debug the failure:

- Undriven signals/snip points
- Uninitialized registers
- Fail nodes
- Loop cutpoints

These conditions inject unconstrained behavior into the design. When unrecognized, you may waste time looking for root causes of the arbitrary injected values, or conclude that the trace provided by VCF is inconsistent with their design logic. By making these X-injection conditions more apparent, VC Formal helps in improving the efficiency and self-sufficiency for debugging traces.

With this support, you can

- Run the check at any point in the flow after the trace is generated
- Run the check automatically on trace generation
- Get a report of the trace results at any point after the check

When VC Formal runs the checks, the anomalous traces are indicated as such in the property table in the VC Formal GUI before users begin debugging. When the relevant signal waveforms are added to the display, the signals are annotated to indicate their special status.

Feedback

4.16.1 Configuring Trace Checks

Use the `fvtrace_config` command to specify the type of trace checks must be performed. The following types of trace checks can be configured:

Types for Trace Checks

Table 5: Types of Trace Checks

Check	Type	Comment
initialstate	design	State elements from the design, uninitialized by the reset sequence
	sampled	Uninitialized state elements created for SVA constructs such as $past(net,4)
undriven	design	Undriven nets in the design
	usersnip	Undriven due to snip_driver
	userabstraction	Undriven due to set_abstraction
	userblackbox	Undriven due to a user-specified blackbox
	autoblackbox	Black-boxed automatically by compilation or FV model creation
	internal	Undriven due to compiler limitations or toolchain issues
aepnode	bound	Undefined due to array bounds violation
	explicitx	Undefined due to explicit X assignment
	divbyzero	Undefined due to division by zero
	multidriven	Undefined due to multiple drivers
	ztox	Undefined due to a Z-to-X conversion
	other	Other undefined assignments
loop_cutpoint	internal	Undriven nets terminating unrolled loops. Usually indicates a non-converging combinational loop.

Syntax

Feedback

```
vcf> fvtrace_config [-check <list-of-checks>] [-type <list-of-types>]
 [-autorun <checkpoint>] [-severity <severity>] [-verbosity <verbosity>]
 [-enable] [-disable]
```

Where,

- `[-check <list-of-check-names>]`: Specify the list of trace checks to be performed. If nothing is specified, all types of trace checks are performed.
- `[-type <list-of-check-types>]`: Specify the check type or list of types to be performed. If no type is specified, trace checks will be performed for all check types.
- `[-autorun [<checkpoint>] ]`: Specify this option to automatically run the trace checks when the `view_trace` command is specified. The automatic checking is disabled, if this option is set to `none`. Currently, the `view_trace` command is the only supported checkpoint.
- `[-severity <severity>]`: Set the severity of the check.
- `[-verbosity <message_level>]`: Set the verbosity of the check to "summary" or "verbose".
- `[-enable]`: Enable this check.
- `[-disable]`: Disable this check.

Examples

```
# Uninitialized state: Enabled, but low severity
vcf> fvtrace_config -check initialstate -autorun viewtrace -severity low
# AEPNodes: Indicate immediately upon detection as "high" severity
vcf> fvtrace_config -check aepnode -autorun viewtrace -severity high
```

4.16.2 Performing Trace Checks

Use the `fvtrace_check` command to perform the trace checks. When the trace check is performed, the property status icon and waveform signals are displayed in the *Tasklist* in the VC Formal Debug mode.

Syntax

```
vcf> fvtrace_check <property_name> [-check <list-of-check-names>] [-type
 <list-of-type-names>] [-vacuity] [-witness] [-clear] [-quiet]
```

Where,

- `<property_name>`: Name of the property trace being checked. Trace must exist.
- `[-check <list-of-check-names>]`: Optional check name or list of check names. If no name is specified, it defaults to all enabled checks.

- `[-type <list-of-type-names>]`: Optional type name or list of type names. If no name is specified, it defaults to all enabled types.
- `[-vacuity]`: Check the vacuity trace of the specified property.
- `[-witness]`: Check the witness trace of the specified property.
- `[-clear]`: Clear the trace analysis results from this property trace.
- `[-quiet]`: Do not print out results of check during the run.

4.16.3 Reporting Trace Checks

After a trace checks is completed, you can view the summary or detailed information for the trace check using the *fvtrace_report*command.

Syntax

```
vcf> fvtrace_report [<propname>] [-vacuity] [-witness] [-check
 <list-of-checks>] [-type <list-of-types>][-noSummary] [-list] [-limit
 <int>] [xml] [-config] [-no_summary]
```

Where

- `<property-name>`: Specify the property name of the the generated trace.
- `[-vacuity]`: Check the vacuity trace of the specified property.
- `[-witness]`: Check the witness trace of the specified property.
- `[-check <list-of-check-names>]:` Specify the list of the check names. If not specified, all checks are reported.
- `[-type <check_type>]` : Specify the type of the checks to be reported. If not specified, all types are reported.
- `[-list]`: Specify this option to get a detailed report on the line items of checks.
- `[-limit <# design objects for verbose report>]`: Specify this option to limit the number of printed design objects in -list.
- `[-xml]`: Specify this option to get a xml version of the report.
- `[-config]`: Specify this option to get a report on the trace analysis configuration, instead of the analysis results.
- `[-no_summary]`: Specify this option if you do not want the summary to be printed in the summary information.

5

VC Formal Regression Mode Accelerator

This section describes the benefits and standard usage of VC Formal Regression Mode Accelerator (RMA).

- About VC Formal RMA
- VC Formal Applications Supporting RMA
- Enabling RMA Through fvlearn_config
- Viewing RMA Messages and Gain Summary
- Viewing RMA Engines and Identifying Resolved Goals
- Switching RMA On/Off in VC Formal Runs
- RMA Usage Flows
- Special Considerations
- FAQs

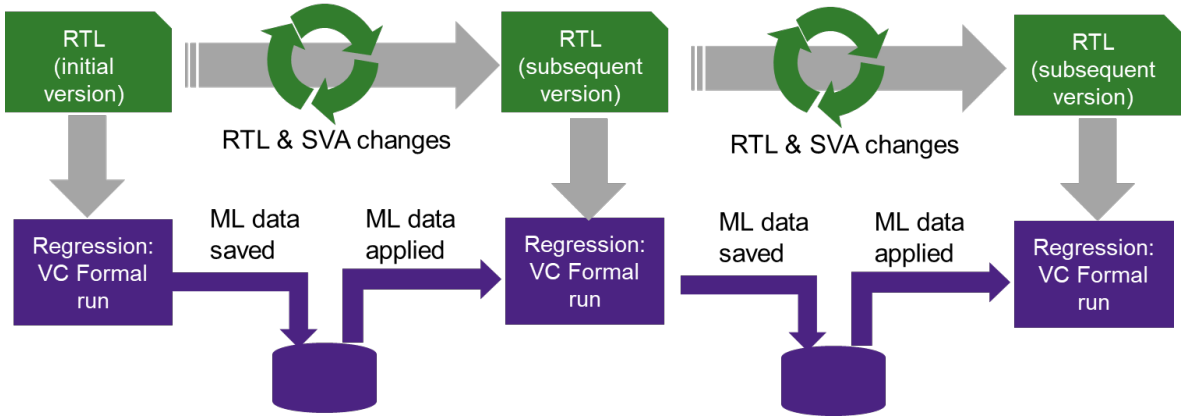
5.1 About VC Formal RMA

VC Formal Regression Mode Accelerator (RMA) is a powerful component based on ML (Machine Learning) techniques to boost performance when you are re-running formal proofs while the target DUT (Design Under Test) is being implemented and verified. RMA offers great benefits when enabled in regression runs where RTL and SVA changes happen incrementally and on a limited scale.

With every formal proof, RMA accumulates learnings for the current Verification Problem into a database. With every subsequent run, the learnings are applied to the parts of the Verification Problem that have not changed while accumulating new learnings for the parts that have changed.

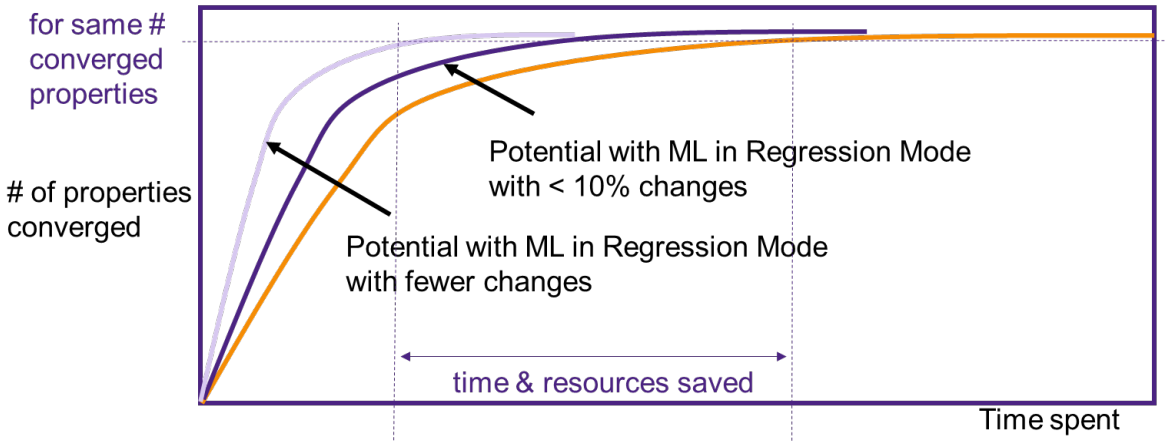
Feedback

Figure 20: RMA Flow



Applying learnings from the RMA database saves time and compute resources as formal proofs results are quickly re-established for the parts of the Verification Problem that are the same. Additionally, applied learnings can help with convergence of the remaining properties. Since formal proofs target only the remaining properties with the same time and compute resources, convergence is better for the remaining properties.

Figure 21: RMA Convergence Improvements Graph



The following table illustrates how applying RMA learnings from the first run allow significant runtime speedups and additional convergence in most cases.

Figure 22: RMA Convergence Improvements Numbers

Design name	# of properties solved	First Run (minutes)	Follow-up run (minutes)	Runtime Speedup	Extra Properties Solved
D1	131	546	9	61x	14
D2	138	546	11	50x	2
D3	248	588	34	17x	3
D4	381	646	42	15x	0
D5	103	186	10	18x	0
D6	319	715	32	22x	4

Recommended Flows Benefiting From RMA Capabilities

The following flows best leverage RMA capabilities:

- Daily regression runs where RTL and SVA do not change or have limited changes
- Daily runs for designers coding and verifying RTL blocks
  - Proof of all properties as RTL and SVA change
  - Interactive proofs of specific properties with different abstractions, proof settings, and so on.
- Re-establishing previous proof results in minutes instead of hours or days as an alternative to saving/restoring sessions or saving FSDB traces for debugging.

5.2 VC Formal Applications Supporting RMA

RMA accumulates learnings for the VC Formal Apps listed in the following table.

Table 6: RMA Runtime

VC Formal App	Accumulated Learnings
FPV	`check_fv`

Feedback

Table 6: RMA Runtime (Continued)

VC Formal App	Accumulated Learnings
FCA	`check_fv / compute_cov / signoff_check` **Note:** When RMA is enabled in the COV application mode, it may impact the performance if the number of goals is huge. Saving the database can take time. Hence, use it cautiously while considering the overall complexity and total number of goals.
SEQ	`check_fv`
AEP	`check_fv`
CC	`check_fv`
FRV	`check_fv`
FXP	`check_fv`
DPV	`check_fv / solveNB / run_solver` **Note:** If RMA learnings are not applied because of changes to the designs, the last set of successful solve scripts are applied to the proof.

Note:

RMA learnings accumulation does not apply to some common steps in the flow such as setup checks with command `check_fv_setup` and Formal Core computation with command `compute_formal_core`.

5.3 Enabling RMA Through fvlearn_config

You can enable RMA by adding `fvlearn_config` before the formal proof commands (the `check_fv` command) where learnings are to be accumulated and/or applied.

You can configure RMA using the `fvlearn_config` command and use it in different use cases as described in section RMA Usage Flows.

Syntax

```
vcf> fvlearn_config [-local_dir <dir_path>] [-enable <0/1>]
 [-local_dir_learn <localDirNameForLearn>] [-local_dir_apply
```

Feedback

```
<localDirNameForApply>] [-learn_only] [-apply_only] [-global] [-verbose]
[-status]
```

Where,

- `-local_dir <localDirName>`: Specify this option to set the path where the learnings of the `check_fv` command must be saved or reused. In this mode, all the data and learnings are saved in the local directory specified by the user.
- `-enable <0/1>`: Specify this option to enable or disable RMA.
- `-local_dir_learn <localDirNameForLearn>`: Specify this option if you want the current learning data to be stored in a local directory.
- `-local_dir_apply <localDirNameForApply>`: Specify this option when you want to apply the learnings saved in the local directory in the `check_fv` run.
- `-learn_only`: Specify this option when you do not want to apply the learnings of the previous `check_fv` runs, but you want to save the learnings of the current `check_fv` run. This option overwrites the existing learnings with the new learnings of the current `check_fv` run.
- `-apply_only`: Specify this option when you want to apply the learnings in the `check_fv` run, and you do not want to overwrite the existing learnings. This prevents corruption of the already learned data, however, no new learnings from the current session is recorded.
- `-global`: Specify this option if you want to apply the command to all the current tasks.
- `-verbose`: Specify this option to get a report of the RMA benefits on the fly.
- `-status`: Specify this option to get the RMA ON/OFF status and the RMA database path as a Tcl dictionary.

Examples

- Enabling RMA with *RMA_learn* as target database without changing Tcl script

  ```
  vcf -f run.tcl -x "fvlearn_config -local_dir RMA_learn"
  ```

- Enabling RMA with *RMA_learn* as target database in Tcl script or interactively

  ```
  fvlearn_config -local_dir RMA_learn
  check_fv
  ```

- Enabling RMA to apply learnings but disabling accumulation

  ```
  fvlearn_config -local_dir RMA_learn -apply_only
  check_fv
  ```

- Enabling RMA to accumulate learnings but disabling application

```
fvlearn_config -local_dir RMA_learn -learn_only
check_fv
```

- Enabling RMA to apply learnings from RMA_in and accumulate learnings to RMA_out

```
fvlearn_config -local_dir_apply RMA_in -local_dir_learn RMA_out
check_fv
```

- Enabling RMA then turning it off and on during the session

```
fvlearn_config -local_dir RMA_learn
check_fv
…
fvlearn_config -enable 0
check_fv -property assert_A
…
fvlearn_config -enable 1
check_fv -property assert_B
```

- Enabling RMA and activate verbose logging for RMA gains

```
fvlearn_config -local_dir RMA_learn -verbose
check_fv
```

- The following table provides a summary of the various scenarios for the `-status` option and its corresponding outputs:

Scenario	**Sample Outputs from `fvlearn_config -status` (Tcl format)**
`fvlearn_config -local_dir LEARN_DIR` `fvlearn_config -status`	**rma_on** true **dir** full-path-to-LEARN_DIR
`fvlearn_config -enable 0` `fvlearn_config -status`	**rma_on** false **dir** ""
`fvlearn_config -status -local_dir LEARN_DIR`	[Info] OPTIONS_IGNORED: Any switch provided together with '-status' is ignored **rma_on** true **dir** full-path-to-LEARN_DIR

5.4 Viewing RMA Messages and Gain Summary

The first time RMA learnings are accumulated, the following message shows the target database:

[Warning] LEARNED_DATA_NOT_FOUND: No existing learned data found in 'RMA_learn'

Feedback

When RMA learnings are applied from the specified database, the following message shows indicating which Verification Problem they are applied to:

[Info] RETRIEVE_LEARN_DATA: Retrieved learned data from (local directory) RMA_learn

When RMA learnings are applied with the `-verbose` switch, detailed messages show the gains achieved on-the-fly during the proof as well as a summary such as the following:

```
======== Regression Mode Accelerator Summary ==================
Total Num of Goals (current run/prev run):    50/50
Num of Goals converged (current run/prev run):    34/34
Num of Goals benefited  :    29
Max gain     :    3.00x
Avg gain     :    2.17x
Min gain     :    2.00x
Num of Goals reached same depth faster:    15
```

5.5 Viewing RMA Engines and Identifying Resolved Goals

RMA leverages dedicated engines *rq1*, *rf1*, and *rp1* to apply and accumulate learnings:

- *rq1*: Target proven/uncoverable and falsified/covered properties
- *rf1*: Target falsified/covered properties
- *rp1*: Target proven/uncoverable properties

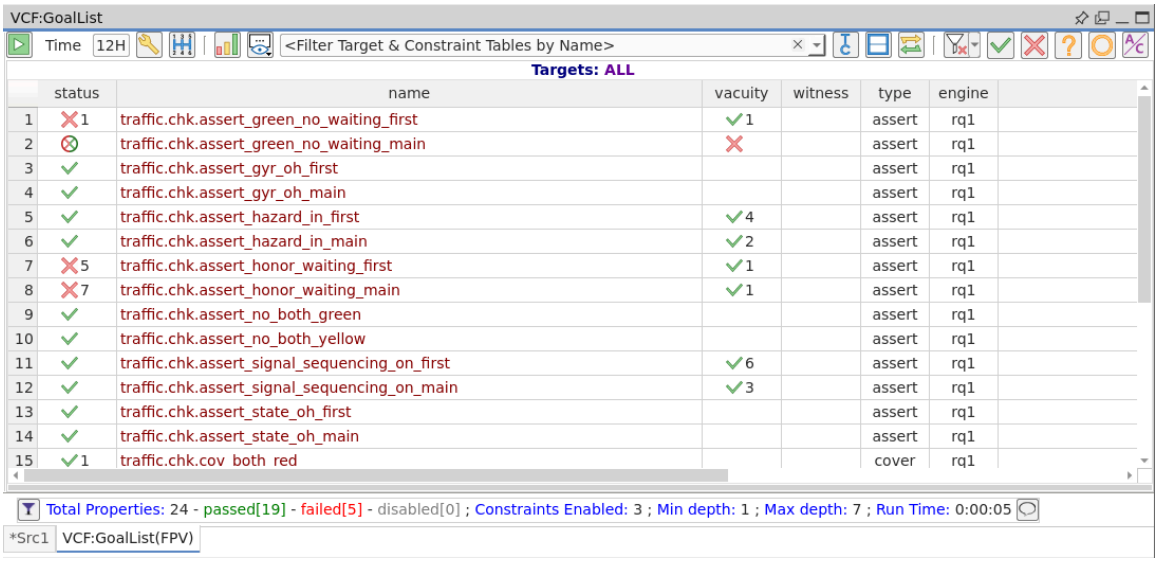
When applying learnings is successful, command `report_fv -verbose` and the GUI will report one of the RMA engines as the proof engine for the property.

Feedback

Figure 23: RMA Engines in Verbose Report

```
vcf>report_fv -verbose -no_summary
  Verbose Results
   Property Verbose List:
   ----------------------
   > Assertion
     # Assertion: 16
     > ID: [0] falsified (depth=1)
      - name          : traffic.chk.assert_green_no_waiting_first
      - vacuity       : non_vacuous
      - language      : SVA
      - type          : assert
      - usage         : assert
      - update_time   : 18:15:27 - Aug 18
      - elapsed_time  : 00:00:01
      - converge_time : 00:00:00
      - location      : ../sva/traffic.sva:35
      - engine        : rq1
      - safe_depth    : 0
      - trace_depth   : 1
      - vacuity_engine: rq1
      - vacuity_safe_depth: 0
      - vacuity_depth : 1
```

Figure 24: RMA Engines in GUI



When proof results are re-established with RMA engines *rq1*, *rf1*, or *rp1*, they indicate parts of the Verification Problem (or parts of the RTL and SVA) that are unchanged compared to the run when RMA learnings were accumulated. On the other hand, properties converging with a different engine (see note below regarding t* engines) or remaining inconclusive probably include modified RTL and/or SVA in their COI (Cone of Influence), or the formal proofs setup has changed.

In the absence of formal proofs setup changes, disabling all engines except the RMA engines r* (*rq1*, *rf1*, and *rp1*) and running a proof provides good information about the parts of the RTL and SVA that have changed.

For example, to enable RMA and disable all engines but the RMA ones

```
fvlearn_config -local_dir RMA_learn
set_engine -off *
set_engine -on r*
check_fv
```

Note:
Note that transformation engines t* (t1, t2, …) are run before RMA engines and cannot be turned off. Properties converging with a transformation engine do not necessarily indicate that the Verification Problem associated with those properties has changed but it could also be the case.

5.6 Switching RMA On/Off in VC Formal Runs

When VC Formal is using Elite or Token license modes, RMA is enabled by default. If the Tcl script does not specify the location of the RMA database explicitly with the `fvlearn_config` command, the RMA database is saved in the current working directory with the name `<top_module>_learn_dir`. RMA can be disabled explicitly if needed.

- Example 1: Elite license mode enabling RMA by default

  ```
  vcf -f run.tcl -fml_elite
  ```

- Example 2: Token license mode enabling RMA by default

  ```
  vcf -f run.tcl -fml_token
  ```

- Example 3: Disabling RMA explicitly with the `-rma_off` switch

  ```
  vcf -f run.tcl -fml_elite -rma_off
  ```

- Example 4: Disabling RMA explicitly with the `fvlearn_config` command

  ```
  vcf -f run.tcl -fml_elite
  ```

  In run.tcl:

  ```
  fvlearn_config -enable 0
  ```

5.7 RMA Usage Flows

The following sections describes the different flows where RMA can be leveraged.

Local RMA database with apply and learn done locally

Target usage flows:

- Daily regression runs where RTL and SVA do not change or with limited changes
- Daily runs for designers coding and verifying RTL blocks
  - Proof of all properties as RTL and SVA change
  - Interactive proofs of specific properties with different abstractions, proof settings, and so on.

The RMA database is used to apply and accumulate learnings continuously with a single user (regression user or designer/verification lead). There is minimal risk of interference between sessions and users trying to access the same database.

The RTL and SVA can be copied from a centralized location along with the RMA database to be modified locally. Note that it is important that the RMA database does not get saved back to the centralized location if the new learnings can impact the original database.

The `fvlearn_config` command can be used when learnings are applied from and accumulated to the local RMA database:

```
fvlearn_config -local_dir RMA_learn
```

Centralized RMA database with apply only and learn saved locally

Target usage flows:

- Re-establishing previous proof results in minutes instead of hours or days as an alternative to saving/restoring sessions or saving FSDB traces for debugging

The RMA database in a centralized or shared location is used to apply learnings only as multiple users can leverage the data while working on different parts of the RTL and/or SVA properties.

Learnings are applied from an RMA database in a centralized or shared location while accumulation is disabled, or a different RMA database is specified. Note that it is important that the RMA database in the centralized or shared location is not updated by mistake. Since RMA does not provide versioning control itself, the RMA database itself should be backed up to prevent any unexpected modification. It is safer to copy the original RMA database locally to avoid any potential issue.

The `fvlearn_config` command can be used prevent learnings accumulation to the centralized RMA database:

```
fvlearn_config -local_dir RMA_central -apply_only
```

or

```
fvlearn_config -local_dir_apply <path_to_shared_area>/RMA_central
-local_dir_learn RMA_local
```

Centralized RMA database with apply and learn

This usage model is not recommended when multiple users can both read (apply learnings) and write (accumulate learnings) at the same time in the same location. This could result in the RMA database being corrupted or learnings from incompatible Verification Problems overriding the original one. Here again, since RMA does not provide versioning control itself, the RMA database itself should be backed up to prevent any unexpected modification.

5.8 Special Considerations

Backward Compatibility

RMA learnings accumulated with a previous version (based on date) can be applied when running with a newer version of the tool. However, RMA learnings accumulated with a newer version (based on date) cannot be applied when running with a previous version of the tool.

In other words, if the RMA learnings accumulation happened with the VC Formal version from February 2024, an error message will show when applying those learnings with the VC Formal version from December 2023 as follows:

```
[Info] RETRIEVE_LEARN_DATA: Retrieved learned data from...
[Error] NEW_RMA_DB_VERSION: Tool version is older than RMA learn database
 version. RMA is disabled.
```

Note that RMA learnings accumulation still happens using the previous version of the tool thus effectively overwriting the original RMA database.

RMA and Version Control

RMA does not include a versioning control system. Different versions of the RMA database based on proof commands and Verification Problems are not available and the RMA database will reflect learnings accumulated up to the last proof command only.

It is recommended to save/copy the original RMA database (or use apply from one database and save learnings to another database) before running `check_fv` when there is a concern about impacting existing learnings.

Conditions Impacting RMA Learnings Accumulation

Accumulation and application of learnings is expected to be 100% when the Verification Problem is the same between the original run and when RMA learnings are applied. On the other hand, with modified RTL and/or SVA properties, the Verification Problem can change and only a subset of the RMA learnings might be applied.

The following situations that can lead to less than expected RMA learnings being applied with a modification to the Verification Problem:

- RTL and/or SVA changes
  - This is an expected scenario where RMA still provides big performance boost. It is recommended to run RMA regularly when changes are localized and roughly less than 10% of the code.
- Frontend synthesis and modeling differences between VC Formal versions
- Instrumenting coverage goals at elaboration time
- Injecting faults at elaboration time
- Different reset states between proof commands
- Applying abstractions between proof commands
- Enabling COI reduction when properties are checked in separate commands

RMA and Task Management

RMA learnings accumulation is related to the current task where the formal proof is run. Even if the task is copied with all its properties, RMA learnings accumulated for the original task will not be applied to the new one.

5.9 FAQs

- *Question*: Do learnings from one design work on another similar design (different mode of operation)?

  Answer: The percentage of learnings applied successfully might probably be low, but RMA does not cause any issue even if the RMA database was generated for a different design.

- *Question*: Do learnings from a proof done with the top targeting a sub-instance work when the top is now higher in the hierarchy?

  Answer: The percentage of learnings applied successfully might probably be low but RMA does not cause any issue even if the RMA database was generated for a module instantiated under the current top module.

6

Performing and Configuring VC Formal Checks

The section describes how to perform, control, and configure VC Formal checks.

This section contains the following topics:

- Performing VC Formal Checks
- Running on Compute Farm
- Running VC Formal on Scale-Out Computing on AWS (SOCA)
- Synopsys Grid Reservation System
- Controlling check_fv Runs
- Viewing Progress of check_fv Runs
- Viewing Details for VC Formal Verification Goals
- Viewing the Task Progress Report
- Orchestration in VC Formal
- Viewing the Solve/Engine Time for Proven/Falsified Goals
- Viewing Bad Code
- Viewing ProofTree (Orchestration) Details
- Viewing Bit Level Model Statistics in VC Formal
- Viewing the Engine Summary in VC Formal GUI
- Defining User Orchestration for VC Formal Runs
- Applying User-defined Orchestration on Top of Another Effort Level
- Supporting the Bug Hunting Feature
- Generating Cover Properties for Bug Hunting

6.1 Performing VC Formal Checks

After you have verified the formal setup and reviewed the properties in the design, you must perform formal verification checks for the properties. Use the `check_fv` command to perform Formal verification checks for the different types of properties in their respective application modes.

The `check_fv` command:

- Verifies all properties with usage attribute set to assert.
- Verifies reachability of all properties with usage attribute set to cover.
- Obeys all constraints specified by properties with usage set to assume.
- Any nets held at a constant value through the `set_constant` command.
- Takes into account snipped nets and black boxes.

By default, the `check_fv` command starts the run process. A main server dispatches auxiliary workers in parallel. The main server is a process that controls all the auxiliary processes. Each auxiliary server worker is a separate process that runs in parallel.

Syntax

```
vcf> check_fv -help
Usage: check_fv    # Check the properties
        [-block]                (Makes command input block while this
 command is active.)
        [-run_finish <cmds>]    (Set of commands to run when this command
 finishes in non-                                blocking mode.)
        [-stop]                 (Stops execution)
        [-break <#of falsification(s)>]
                                (#of falsification(s) to stop execution)
        [-property <list-or-collection-of-properties>]
                                (List or collection of properties)
        [-subtype <list-of-subtypes>]
                                (goal subtype selection:
                                 Values: property, vacuity, witness)
        [-assume]               (Use proven assertions and uncoverable
 covers as                                       constraints)
```

Examples

- Use the `-block` option to run `check_fv` command in block mode. When you specify this option you cannot interact with the vcf shell when the `check_fv` is in progress. A typical set of run commands used to run in block mode are:

  ```
  check_fv -block
  report_fv -list > results.txt
  ```

Feedback

You can use Ctrl-C can be used to interrupt the block mode run.

- Use the `-run_finish` option to execute commands when the run is finished in non-block mode. You work with the shell in an interactive mode while the `check_fv` run is in progress, and also pre-program tasks to be done when the run is completed.

  For example, setting the vcf shell to run in interactive mode before programming it to get a summary when the run is completed and then exiting the vcf shell, the following command sequence can be used:

  ```
  check_fv -run_finish {
    report_fv -list -status falsified > falsification_list.txt
    report_fv -list -status proven > proven_list.txt
    report_fv -list -status unknown > unknown_list.txt
    quit
  ```

  }

- To check all property goals

  ```
  check_fv -property $myprop
  ```

- To check specific property goal

  ```
  check_fv -property $myprop -subtype property
  ```

- To check only the property's vacuity goal

  ```
  check_fv -property $myprop -subtype vacuity
  ```

- To check only the property's witness goal

  ```
  check_fv -property $myprop -subtype witness
  ```

- To check the property's vacuity and witness goals; exclude the goal for the property itself

  ```
  check_fv -property $myprop -subtype {vacuity witness}
  ```

- To check all unresolved property and vacuity goals; exclude witnesses:

  ```
  check_fv -subtype {property vacuity}
  ```

- The `check_fv` command by default runs in the non-blocking mode and the main run server dispatches four auxiliary workers. This means that the proof processes are running in the background and you have access to the Tcl interpreter after the check is initiated. Activities such as querying results, debug and analysis can be done while the checks are running. However, editing, setup or check commands that would affect the running command are not allowed to execute concurrently.

Feedback

Other configurations (such as vacuity check is by default enabled and witness check is by default disabled) are predefined through a number of application variables. These application variables can be viewed using the `report_app_var` command. They may be changed using the `set_app_var` command.

Example

Consider the following example that has some properties:

```
vcf> get_props -usage assume
{"assume_req0_until_gnt", "assume_req1_until_gnt",
 "assume_req2_until_gnt", "assume_req3_until_gnt"}
vcf> get_props -usage assert
{"assert_gnt_one_hot", "assert_no_gnt0_if_no_req",
 "assert_no_gnt1_if_no_req", "assert_no_gnt2_if_no_req",
 "assert_no_gnt3_if_no_req", "assert_no_gnt_if_stall"}
vcf> get_props -usage cover
{"cover_all_gnt_asserted", "cover_all_req_asserted"}
```

When you run the `check_fv` in either blocking or non-blocking mode and the following information about the run is displayed on the screen:

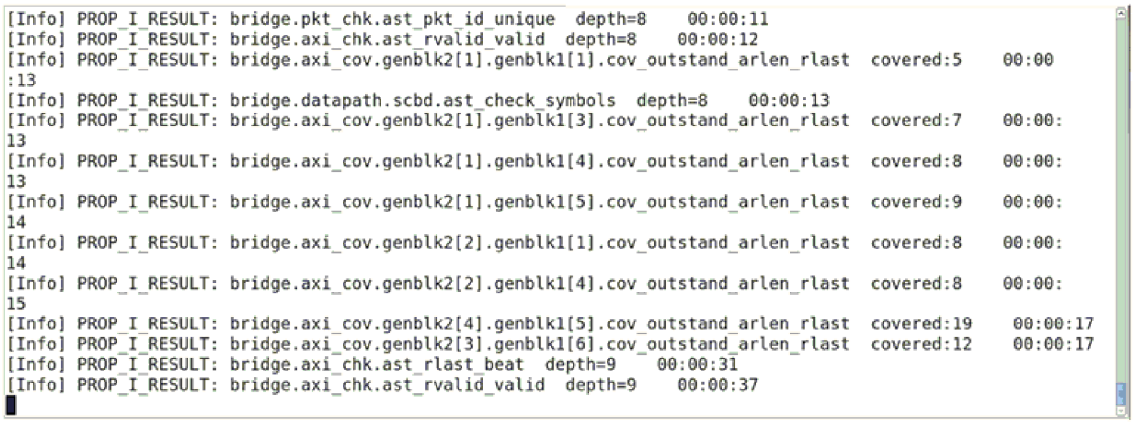
```
vcf> check_fv
[Info] FORMAL_I_CREATE: Create Formal Model:priority_arb.
[Info] FORMAL_I_RUN: Starting formal verification for check_assertions
     Id: 1 Goals: 9 Constraints: 4 Block Mode: false
1
vcf>
```

Here, the goal number is the total number of properties sent to the formal engines. It includes asserts, cover properties, connection properties, vacuity checks and witness checks, if enabled.

As the formal engines start to work the goals, information messages about the status of each assertion continue to appear on the screen as the verification progresses. The proof status such as checking, bounded depth, final result and the time it used for each property is shown in Figure 25.

Feedback

Figure 25: Live Run Status Update



When the run is in progress and messages are continuously streaming on the screen, hitting RETURN returns the `vcf` prompt for interactive command entry.

The following command resumes screen output.

```
vcf> traceon
```

The GUI (as illustrated in Figure 25) is the best way to get live run status update.

6.1.1 Stopping the Run When Constraints are Bad

The `check_config` command enables you to specify the stop condition in the `check_fv` command if any of the constraints has vacuous or unreachable witness.

Syntax

```
vcf> check_config [-stop_on <stop_condition>] [-clear][-formal_core
 <status>] [-perf_diagnostics_alerts <0/1>]
```

Where,

- `[-stop_on <stop_condition>]`: Specify the condition at which you want to stop the run. You can specify the stop condition value as `unreachable_witness` and `vacuous_constraint`.
- `[-clear]`: Clears all the existing stop conditions.
- `[-worker_stat]`: Print worker allocation messages.
- `[-formal_core <status>]`: Values can be *off, proven, inconclusive,*or *both*. If it is set to *proven* (*inconclusive*) before executing the `check_fv` command, then orchestration attempts to compute formal core for a property that has *proven* (*bounded*) status. If it is set to *both*, then formal core will be computed for both the statuses. If it is set to *off*,

then no formal core is generated during the `check_fv` command. For more information on the on-the-fly formal core generation, see the Generating Formal Core On-the-fly section.

- `[-perf_diagnostics_alerts <0/1>]`: Prints performance related diagnostics messages during the run. Use the values 0 to disable and 1(default) to enable it.

Examples

- To stop the run, when there is a vacuous constraint:

```
vcf> check_config -stop_on vacuous_constraint
vcf> check_fv
```

- To enable performance related diagnostics alerts:

```
vcf> check_config -perf_diagnostics_alerts 1
```

When the performance related diagnostics alerts are enabled, the following information is printed in the console:

  - At the start of the run, the configuration details such as *Task name, Command name, Orchestration/Effort, Run time limit, Memory limit, Number of workers requested, FPV default/bug_hunting flow version, Orc tactic (if any), Grid related settings* are seen:

```
[Info] PERF_DIAGNOSTICS_ALERTS_START
Run configuration:
[Info] FV_RUN_CONFIG_INFO:  Task : FPV, cmd : check_fv
[Info] Configuration Details
[Info] Orchestration/Effort : default
[Info] Run time limit : 12H
[Info] Memory limit : 8GB
[Info] Number of workers requested : 4
[Info] FPV flow ver : v7
[Info] PERF_DIAGNOSTICS_ALERTS_END
```

  - During the run, information related to the workers allotted are seen:

```
[Warning] WORKER_ALLOCATION_WARN: Task : FPV, No new workers have
 been allocated in the past 5s.
 Please verify your set_grid_usage configuration. Run
 report_fml_hosts for more information.

[Warning] WORKER_ALLOCATION_WARN: Task : FPV, No new workers have
 been allocated in the past 10s.
 Please verify your set_grid_usage configuration. Run
 report_fml_hosts for more information.

[Info] WORKER_STAT: [10s] Worker Allocated/Requested : 1/4
[Info] WORKER_STAT: [10s] Worker Allocated/Requested : 2/4
[Info] WORKER_STAT: [10s] Worker Allocated/Requested : 3/4
```

Feedback

```
[Info] WORKER_STAT: [10s] Worker Allocated/Requested : 4/4

[Info] ALL_WORKERS_ALLOCATED: Task : FPV, All requested workers have
 been successfully allocated.
 4/4 workers allocated in 10s.
```

- At the end of the run, the output of the `report_fv` and `report_fml_jobs` are seen.

```
[Info] PERF_DIAGNOSTICS_ALERTS_START

  Summary Results
   Property Summary: FPV
   ----------------
   > Assertion
     - # found        : 22
     - # proven       : 19
     - # falsified    : 3

   > Cover
     - # found        : 10
     - # covered      : 6
     - # uncoverable  : 4

   > Constraint
     - # found        : 3

   AEP Summary: FPV
   ------------
   > Assertion
     - # found        : 16
     - # proven       : 16

Solver jobs summary for verification task FPV (19 solver jobs)
    #scheduled     :     0
    #running       :     0
    #completed     :    19
        #normal         :    15
        #timeout        :     0
        #memout         :     0
        #orc_killed     :     4
        #sig_killed     :     0
        #crash          :     0
        #start_fail     :     0
        #misc_error     :     0
[Info] PERF_DIAGNOSTICS_ALERTS_END
```

- To disable performance related diagnostics alerts:

```
vcf> check_config -perf_diagnostics_alerts 0
```

Feedback

6.2 Running on Compute Farm

VC Formal can use LSF, Sun Grid Engine (SGE), Runtime Design Automation (RTDA), PBS (Portable Batch System), and SLURM (Simple Linux Utility for Resource Management) compute farms to complete jobs faster by running many jobs in parallel. By default, the `check_fv` command consists of one main process and four auxiliary worker processes. You can configure the grid to add many auxiliary worker processes to the main process. These auxiliary worker processes can all run on one machine or each of them on different machines.

You can view the grid jobs, debug errors in the grid submission, and terminate grid jobs. You can also configure, view, and control the grid configuration from the VC Static shell and the GUI.

6.2.1 Configuring Grids

You can use the `set_grid_usage`commands to configure a `check_fv` run to launch workers (executing solver tasks) inside server farms, and then retrieve the status of grid submissions.

Syntax

```
vcf> set_grid_usage -help
Usage: set_grid_usage    # set grid configuration for VC-Static Formal
        [-type <string>]       (the of grid and the number of worker,
 [sge|lsf|rsh|ssh|rtda]=<num_of_worker>)
        [-control <string>]    (resource control string for grid job
 submission, eg. { qsub -P bnormal -V arch=glinux })
        [-dir <string>]        (temp directory for the workers)
        [-file <string>]       (configuration file for grid job
 submission)
        [-memory <string>]     (memory in GB associated with this grid
 when using multiple grids with -level)
        [-global]              (apply to all current tasks)
        [-default]             (reset grid usage to default settings)
        [-job_array]           (enable job array)
        [-prefetch_pct <percent>]
                               (percentage of requested workers to
 pre-fetch:
                                Range: 0 to 100)
        [-level <integer>]     (order of this grid when using multiple
 grids with -memory, where orders must be consecutive starting at 0)
        [-navigator]           (apply to (all) navigator tasks, rather
 than the current task)
```

Example

Feedback

You can configure a [LSF|SGE|RSH|RTDA|PBS|SLRUM] run in the shell using the following commands:

```
set_grid_usage -type [LSF|SGE|RTDA]=<#_of_worker> -control
{<submission_cmd/opt>}
```

or

```
set_grid_usage -type [SGE]=<#_of_worker>
```

or

```
set_grid_usage -type [RTDA]=<#_of_worker>
```

or

```
set_grid_usage -type PBS=10 -control { qsub -q alwayson -l
instance_type=r5.large,select=1:ncpus=1 }
```

or

```
set_grid_usage -type SLURM=6 -control { /nfs/slurm/bin/sbatch -p vcfM5 --
wait }
```

No control string is needed for RSH. For [LSF|SGE|RTDA], the grid submission commands/options depends on the setup of computing farms and is different from farm to farm. VC Formal displays the following:

- Current grid usage setting and allows editing
- Control strings commonly used for [LSF|SGE|RTDA]

The following are the commonly used control strings:

```
LSF: bsub -q bnormal -R arch==glinux -R rusage[mem=4000]
```

SGE: qsub -P bnormal -V -l arch=glinux,mem_free=4G

```
RTDA: nc run -e "SNAPSHOT"
```

The following are the three common use-models for running the VC Formal tool on a compute farm:

1. Main and auxiliary worker processes on the same remote machine
2. Main process on the local machine and auxiliary worker processes on remote machines

3. Main and auxiliary worker processes on different remote machines

   Note:

   Only use-model (1) and (2) supports interactive mode, and (3) always run in batch mode.

Main and Auxiliary Worker Processes on the Same Remote Machine

Submit a VC Formal job to one machine on the compute farm. By default, the main process and the four auxiliary worker processes run on the same machine. Additional workers can be launched utilizing the other CPU-cores available on the same machine. When running multiple worker processes on the same machine, make sure it has enough free memory to avoid swapping.

Example

To enable VC Formal to invoke 10 auxiliary worker processes on the machine that is running the main process, add the following line in the VC Formal Tcl script before executing any `check_fv` command.

```
set_grid_usage -type rsh=10
```

Main Process on the Local Machine and Auxiliary Worker Processes on Remote Machines

Running the main process on the local machine allows VC Formal to run in interactive mode. Use the `set_grid_usage` command to specify the compute farm type and control information such as the number of hosts.

Example

- Use the following command to configure an SGE farm for 10 Linux 64 bits machines, where each machine has at least 8G memory:

  ```
  set_grid_usage -type sge=10 -control { qsub -P bnormal -V -l
   arch=glinux,os_bit=64,mem_free=8G }
  ```

- Use the following command to configure an LSF farm for 10 Linux redhat 4 machines, where each machine has at least 8G memory:

  ```
  set_grid_usage -type lsf=10 -control { bsub -q bnormal -R arch==glinux
   -R os_version==WS4_0 -R rusage[mem=8000] }
  ```

- To set up RSH on multiple machines to enable the main process to submit auxiliary worker processes to other remote machines:

  1. Arrange a few machines on which you would like to distribute jobs (for example, consider, M1, M2, M3, M4 are the machines you arranged).
  2. Pick one machine to run FV main process (say, M_main).

3. Set up RSH permission on all the machines where auxiliary worker jobs must be distributed from the main machine, that is, from *M_main*to {*M1, M2, M3 and M4*}.
4. Set up the host-file in the following format:

```
# 1<enable>|0<disable> | host <RSH/SSH only> | number of worker(s)
 |location for tmp files | grid type | control string

The following is an example of the host-file (say, my_hostfile):

1 | M1 | 4 | /SCRATCH | RSH | rsh

1 | M2 | 2 | /SCRATCH | RSH | rsh

1 | M3 | 5 | /SCRATCH | RSH | rsh

1 | M4 | 5 | /SCRATCH | RSH | rsh
```

5. Run the VC Formal grid command to point to the host-file:

```
set_grid_usage -file my_hostfile
```

   This command invokes a total 16 workers (=4+2+5+5) across the four specified machines, while the `check_fv` command is executed.

Main and Auxiliary Worker Processes on Different Remote Machines

When running regression tests, all the processes need to be run on grid machines. The steps to do this are given below:

- Create a Tcl file with the setup and run commands, including `set_grid_usage` command in the Tcl script before the `check_fv` command specifying the grid type and number of hosts.
- Submit the Tcl script to the compute farm in batch mode using a `qsub/bsub` command.

The VC Formal main process submits auxiliary worker processes to other remote machines using the options specified in the `set_grid_usage` command.

Example

If you have 12 core machines and an SGE compute farm, the following command submits the `run_vcst.csh` Tcl script to the farm and requests a machine with 12 CPU's available:

```
qsub -V -P <job_queue> -pe mt 12 run_vcst.csh
```

Note:

The -V option makes sure SGE runs the run_vcst.csh in an environment that is similar to the current one.

```
% cat run_vcst.csh
vcf -f ipName.tcl -batch
```

6.2.2 Getting a Report of the Grid Configuration

You can use the `report_grid_usage` command to get a report of the latest grid configuration set up using the `set_grid_usage` command.

Example

```
vcf> report_grid_usage
[Info] GRID_TYPE_STRING: -type SGE=9
[Info] GRID_CONTROL_STRING: -control ' qsub -P bnormal -V -l
 arch=glinux,os_bit=64,mem_free=4G '
[Info] GRID_CONFIG_FILE_INFO: -file <not set>
```

6.2.3 Controlling Grids

The VC static shell and the GUI report the status of SolverJobs. For each SolverJobs, you can know the goal and the engine are assigned to the task. You can also control the engines that must be run or the goals that must be resolved. See section Viewing Progress of check_fv Runs for more details on the SolverJobs.

6.2.4 Prefetching Workers in a Grid

Use the `[-prefetch_pct <percent>]` option in the `set_grid_usage` command to set the percentage of requested workers to pre-fetch in the grid (Range: 0 to 100).

Use the following formal variable to specify the timeout period (in seconds(S), minutes(M), or hours(H)) for the number of workers to pre-fetch. For example, 37M, 12H

```
set_fml_var fml_grid_prefetch_timeout 30M
```

Example

Successful prefetch example

```
set_grid_usage -level 0 -prefetch_pct 80 -type SGE=10 -control { qsub -P
 bnormal -l mem_free=8G -l qsc=s -l arch=glinux }

set_grid_usage -level 1 -prefetch_pct 50 -type SGE=10 -control { qsub -P
 bnormal -l mem_free=16G -l qsc=s -l arch=glinux }

set_fml_var fml_grid_prefetch_timeout 30M
```

[Info] TW_INFO_MSG : Successfully prefetched 13 workers in 20 secs.

Unsuccessful prefetch example

Feedback

```
set_grid_usage -level 0 -prefetch_pct 100 -type SGE=100 -control { qsub
 -P bnormal -l mem_free=8G -l qsc=s -l arch=glinux }

set_grid_usage -level 1 -prefetch_pct 100 -type SGE=50 -control { qsub -P
 bnormal -l mem_free=16G -l qsc=s -l arch=glinux }

set_fml_var fml_grid_prefetch_timeout 2M
```

[Info] TW_INFO_MSG: Failed to prefetch 150 workers in 120 secs. Only got 54 workers.

6.2.5 Debugging the Grid Setup

When you run the `check_fv` command, VC Formal provides the grid job submission error report, which contains the complete details on how and why the grid submission fails.

- If a grid submission is rejected, the VC Static [shell|GUI] displays the exact message returned by the computing farm and then suggests the next step, if possible.
- If workers are queued, the VC Static [shell|GUI] does not notify you automatically. However, you can find out whether any SolverJobs is queued using the *Solver Task View*. See section The report_fml_jobs Command. This feature displays the status of solver tasks in the VC Static GUI and also allows you to query the same in the VC Static shell.

6.2.6 Viewing Workers Status Grid Report in GUI

Once the grid job spawns, use the `report_fml_hosts` command for dynamic information about the hosts on which worker processes are running.

The `report_fml_hosts` command gives following information:

- *HostId*: Unique id assigned to each worker
- *Alive*: Indicates if the worker is still available (1) or not (0)
- *PID*: Process id
- *Job id*: Solver job currently running on the worker CPU-user/CPU-sys: CPU usage on the worker
- *Mem-total/Mem-used*: Total memory and used memory on the machine
- *Host*: Hostname
- *Job-name*: The job identifier

If the workers are heavily loaded and/or memory utilization on the worker machine is very high, then this can negatively impact the performance.

The `report_fml_hosts -totaltime` option gives useful information regarding *#workers requested*, *#workers actually received*, *max total host time* (hypothetical maximum compute time if we got all requested workers immediately), *actual total host time* (sum total of compute time based on workers actually available).

If the actual total host time is significantly smaller than max total host time, then it means the engines are not getting enough time to run, and this can negatively impact performance.

The following is an example output of the `report_fml_hosts` command:

```
Host Summary (4 workers)
-------------------------------------------------------------------------------------------------
  HostId |  Alive |    PID |      JobId | CPU-user |  CPU-sys |  Mem-total |   Mem-used |          Host |
-------------------------------------------------------------------------------------------------
       1 |      1 |   1260 |          2 |        0 |        0 |          0 |          0 | odcgen-odcdev-2328-0
       2 |      1 |   1259 |          1 |        0 |        0 |          0 |          0 | odcgen-odcdev-2328-0
       3 |      1 |   1261 |          0 |        0 |        0 |          0 |          0 | odcgen-odcdev-2328-0
       4 |      1 |   1262 |          3 |        0 |        0 |          0 |          0 | odcgen-odcdev-2328-0
-------------------------------------------------------------------------------------------------
```

6.2.7 Debugging Grid Setup Using Scripts

The *grid_debug.pl*script provides debugging capabilities for the grid setup. You can use the *grid_debug* script to ensure that the grid control string is working before applying it to FV. This script works in a standalone mode, and does not need VC Formal installation and licenses. You can run this script on Pearl versions 5.8 and above.

The *grid_debug* script is present in the following location:

`$VC_STATIC_HOME/doc/vcst/examples/GRID_DEBUG`

Syntax

```
usage: [--style sge|lsf][--workers <n>][--mem <n>][--log <dir>][--cmd
 <cmd>] --params "<parameter string>"
            --? --h --help  Prints this usage information.
            --style         Grid system to use sge/lsf (default: sge)
            --workers   Number of workers/jobs to start (default: 1,
 maximum: 31)
            --mem        Memory in GB for each job (default: 1)
            --time         Additional time to be spent on grid cpu in
 secs (default: 0)
            --log            Directory for logfiles, must be present on
 grid machines (default: /remote/us01home40/stangier)
            --cmd          Overwrite grid submission command
            --params    Control string to be passed to sge/lsf
```

Examples

- Basic test for qsub, 2 jobs, 2GB each:

```
grid_debug.pl --workers 2 --mem 2  --params "-P bnormal   -V -l
 arch=glinux,mem_free=1G "
```

- Basic test for lsf, 4 jobs, 8GB each:

```
grid_debug.pl --style lsf --workers 4 --mem 8 --time 60 --param '-q
 bnormal -I -R "rusage[mem=1000]" '
```

- Test for qsub, 2 jobs, 2GB each, compute at least 1800s:

```
grid_debug.pl --workers 2 --mem 2  --time 1800 --params "-P bnormal
 -V -l arch=glinux,mem_free=1G "
```

- Basic test for lsf, 4 jobs, 8GB each, use a different log directory:

```
grid_debug.pl --style lsf --workers 4 --mem 8 --log /u/bob --param '-q
 bnormal -I -R "rusage[mem=1000]" '
```

- Test for qsub, 2 jobs, 2GB each, use a different grid command:

```
grid_debug.pl --workers 2 --mem 2  --cmd /installs/our_qsub/qsub
 --params "-P bnormal   -V -l arch=glinux,mem_free=1G "
```

6.2.8 Diagnosing the Run Directory For Grid Configuration Issues

You can diagnose the run directory using the *grid_debug* script. Diagnosing the run directory provides the following advantages:

- Helps you to debug problematic FV runs.
- Helps you to fix grid-related issues.
- Helps in speeding up the turn-around time by providing more details on the erroneous scenarios to designers.

To diagnose the run directory using the *grid_debug* script:

1. Set the following environment variables to set up the Python environment:

```
setenv PYTHONHOME $VC_STATIC_HOME/auxx/yoda/python
setenv PATH $PYTHONHOME/bin:$PATH
setenv LD_LIBRARY_PATH $PYTHONHOME/lib:$LD_LIBRARY_PATH
```

2. Run the diagnose script on the FV run directory:

```
$VC_STATIC_HOME/doc/vcst/examples/GRID_DEBUG/diagnose_fv_run.py
 --directory <the directory contains 'vcst_rtdb'>
```

3. Look for the following text in the report which provides details on the results of the diagnosis:

   < scanning sub-directories ... >

   [Final report]

   Good! worker status: all xxx/xxx worker(s)

   Good! backend task: xxx task(s) ran

   Good! verification problem: xxx DFG xxx SAIG model(s) being generated

6.3 Running VC Formal on Scale-Out Computing on AWS (SOCA)

VC Formal can leverage Amazon's Scale-Out Computing on AWS (SOCA) to run complex Formal runs by accessing cloud-based compute instances for parallel computing.

The user requests a predetermined number of cloud-based compute instances with specific memory and number of cores attributes. Once ready (this step can take several minutes), VC Formal tasks that require workers like property verification, fault analysis, and Formal Core computation, uses those instances through a scheduler such as PBS (Portable Batch System) if available on the system.

All VC Formal applications can run on SOCA directly on a compute machine or by leveraging cloud-based compute instances via a scheduler.

6.3.1 Use Model

1. Requesting cloud-based compute instances:

   VC Formal provides a add_nodes.sh script to request compute instances on SOCA in the terminal window directly.

   % add_nodes.sh r5.large 20

   (A full set of instance types are shown at https://aws.amazon.com/ec2/instance-types/r5.)

2. Use the following commands to enable and control the grid usage:

   `set_grid_usage`: Set grid configuration for VC Formal. Use the `-level` option to specify the order of the grid when using multiple grids along with `-memory`.

   Note:

   You can also use `set_backup_grid_usage` command to set backup grid configuration for VC Formal. However, using `set_grid_usage -level` is recommended for creating multiple grids.

Example

```
set_grid_usage -type PBS=10 -control { qsub -q alwayson -l
instance_type=r5.large,select=1:ncpus=1 }
```

Special considerations:

- When the requested cloud-based compute instances become available, they will timeout and go offline if idle for a given time (default: 1 hour). This behavior is to manage costs on the system.
- Once jobs have run on available compute instances, they will timeout and go offline if idle for a given time (default: 5 minutes). This behavior is to manage costs on the system.
- When compute instances are not ready or go offline, VC Formal jobs are queued until compute instances become available or until the command times out.

6.4 Synopsys Grid Reservation System

The Synopsys Grid Reservation System is a utility that can be used to create a virtual reserved grid on top of any existing Grid infrastructure such as (LSF/SGE/SLURM and so on.). This can be used in scenarios where you want to reserve certain number of workers from the primary grid, but not release these workers after each job. You can then submit further jobs to the virtual grid, without any wait/queue time. The reserved grid can only be used for VCF runs, other commands are rejected.

Note:

Reserving a grid can lead to under utilization of the grid resources.

6.4.1 The snps_grs utility

A single cmdline utility is provided to both start and manage the Synopsys Grid Reservation System:

```
$> snps_grs --help
```

Syntax

```
Usage:
  snps_grs [Options] <Command> [Command Options]

Commands:
  add           Add another grid pool to the Synopsys Grid Reservation
 System
  display       Open a dynamic Synopsys Grid Reservation System
 monitoring display
```

```
  ping          Try connecting to Synopsys Grid Reservation System
  run           Run a task to the Synopsys Grid Reservation System.
 Blocks until task is               completed
  start         Start the Synopsys Grid Reservation System
  stop          Stop the Synopsys Grid Reservation System
  submit        Submit a task to the Synopsys Grid Reservation System

Tip:
  Use `snps_grs <CMD> --help' to see command specific help

Options:
  --help               Display this help menu
  --version            Display the tool version
```

6.4.2 Starting the Grid Reservation Server

The `snps_grs` command start can be used to launch a grid reservation server. The full set of options available for the server are:

Syntax

```
Usage:
  snps_grs start [Options]


Options:
  --help                 Display this help menu
  --version              Display the tool version
  --hostconfig file      CDPL compatible host configuration file for the
 grid
  --autostop duration    Automatically shutdown the Grid after given
 interval.
                         E.g. "1d 4h 10m"
  --workers N            Number of workers to acquire from the grid
  --minworkers N         Minimum number of workers to acquire from the
 grid.
                         Can't use with --workers
  --maxworkers N         Maximum number of workers to acquire from the
 grid.
                         Can't use with --workers
  --timeout duration     Timeout interval for acquiring workers from the
 grid.
                         0 implies do not prelaunch. Defaults to infinite
 if
                         not specified.
  --refresh seconds (=5) Display refresh interval. 0 implies no status
 display
  --outdir dir (=PWD)    Output directory for the log files
  --port port            Start the server on a specific port
  --lic_wait             Wait for the licenses if unavailable
```

Example

```
snps_grs start --hostconfig sge.cfg --workers 20 --outdir
output_directory
```

Note:

When the Synopsys Grid Reservation System and VC Formal sessions are not run in the same directory, the output directory must be specified using an absolute path.

After sourcing the setup script generated with `snps_grs start ...`command, use command `snps_grs display`to show the current status.

```
% snps_grs display
```

Example

```
% snps_grs display
Synopsys Grid Reservation System
--------------------------------
Workers Total   :    12
Workers Idle    :    12
Workers Busy    :    0
Workers Pending :    0
Tasks Waiting   :    0
Tasks Submitted :    0
Tasks Failed    :    0
Tasks Completed :    0
Runtime         : 1m 35s
--------------------------------
Host Name : host01234
Workers   : 12
CPU Load  : %5
Mem Used  : %33.7296
Swap Used : %30.4092
```

Host Configuration File:

The server requires a CDPL compatible host configuration file which specifies the underlying grid infrastructure from which the workers will be created. The host configuration file looks like:

1 | | 100 | /SCRATCH | SGE | qsub -P bnormal -l mem_free=16G -l arch=glinux

Note:

The number of workers in the config file (100) represents the maximum limit for the grid. You can request fewer workers using command line options, but will not be able to exceed this maximum limit.

6.4.3 Submitting Jobs

Once the virtual grid is up and running, VC Formal can submit its jobs onto the grid.

Environment Setup

The server generates setup.sh/setup.csh files in the outdir. Source these files, before you interact with the Grid Reservation System.

Example

```
$ source output_directory/setup.csh
```

Verifying Setup

To verify and check if there is a problem in communicating with the server, use the following commands:

Example

```
$ snps_grs ping
Pinging Synopsys Grid Reservation System...
Received OK. Response time = 13.07 ms
```

VC Formal Setup

VC Formal can be instructed to submit jobs to the virtual grid using the set_grid_usage Tcl command. For example, the following command would use 12 workers from the virtual grid.

```
set_grid_usage -type snps_grs=12
```

6.4.4 Shutting Down

Use the following command to shutdown the server:

```
$ snps_grs stop
```

You can also kill the server process using the Ctrl+C on the terminal where Grid Reservation System is running.

6.5 Controlling check_fv Runs

VC Formal provides different ways to control the run. It allows stop, resume, cancel and setting time or memory limit to control the run of the `check_fv` command.

Feedback

6.5.1 Stopping/Resuming check_fv Command

The running `check_fv` command can be stopped to provide an opportunity to make changes to the setup. This is very effective when working with the tool interactively. The following command can be used to stop the running process:

```
vcf(check_fv[00:04:30])> check_fv -stop
```

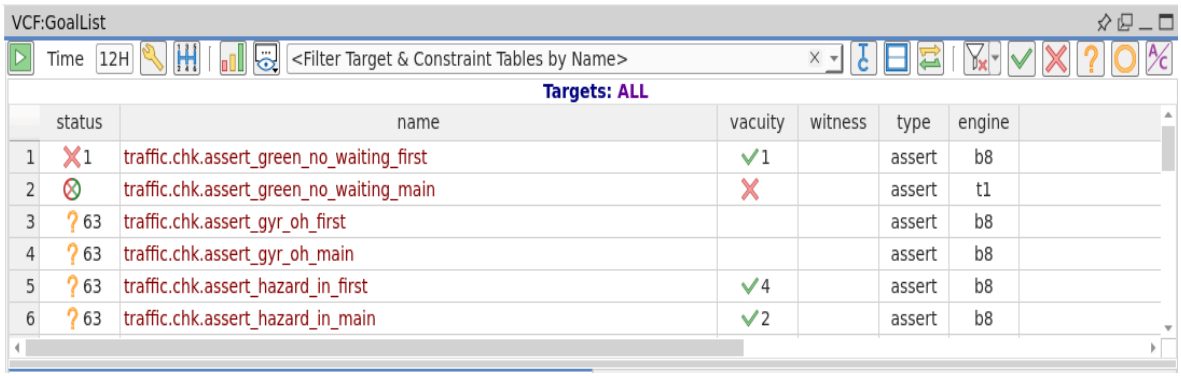
Here, (`check_fv[00:04:30]`) at the `vcf` indicates the run servers are actively working in the background and the run process has started four and a half minutes ago.

To resume from stop, simply re-enter `check_fv`:

```
vcf(check_fv[00:09:39]> check_fv
```

This can also be done in the GUI as shown in #unique_208/ unique_208_Connect_42_i1046336:

Figure 26: Run Stop/Resume From GUI



6.5.2 Setting Limits for the check_fv Run

If no limits are set, the formal engines continue to work on goals that are still unresolved. VC Formal enables you set the time limit and memory limit for the formal engines to work on the goals.

For example, to set run time to be a maximum of 10 hours wall clock time:

```
set_fml_var fml_max_time 10H
```

To set memory usage to be a maximum of 8GB per auxiliary worker process:

```
set_fml_var fml_max_mem 8GB
```

When the max memory limit is hit, the run servers will go on to the next part of the recipe to continue the proof processes. By default, the `fml_max_mem` application variable is set to 8GB.

Feedback

6.5.3 Managing Time Limits with fml_max_check_time

The `fml_max_check_time` formal variable provides a way to manage time limits in formal verification tasks. Unlike the existing `fml_max_time` variable, which only considers verification time, `fml_max_check_time` accounts for all steps in the flow, including auto-reset simulation, model build, and preverification checks.

The benefits of `fml_max_check_time` include:

- *Comprehensive Time Tracking*: Includes all steps in the flow, not just verification time.
- *Safe Execution Stops*: Stops execution at a safe location when the time limit is exceeded. For example, as model build cannot be interrupted, if the `fml_max_check_time` value is lower than model build time, the tool waits for model build to complete before safely stopping execution.
- *Enhanced Debugging*: Provides meaningful messages when the time limit is exceeded, aiding in troubleshooting.

The `fml_max_check_time` formal variable currently supports the following commands:

- `check_fv`
- `compute_reduced_constraints`
- `compute_over_constraints`

Note:

- Default Behavior: If `fml_max_check_time` is not specified, the tool defaults to using `fml_max_time`.
- When both `fml_max_time` and `fml_max_check_time` are specified, `fml_max_check_time` takes precedence, and `fml_max_time` is ignored.

Enabling and Using fml_max_check_time

- Use the `set_fml_var` command to specify the maximum check time.

  ```
  set_fml_var fml_max_check_time <time_limit>
  ```

  Example:

  ```
  set_fml_var fml_max_check_time 0.2M
  ```

- Execute formal verification commands such as `check_fv`, `compute_reduced_constraints`, or `compute_over_constraints`. The tool will monitor the total time from the point the compute command is issued.

- If the total time exceeds the specified limit, the tool will stop execution at a safe point and display an appropriate warning message.

  Example Output Message:

```
[Warning] SPECIFIED_MAX_CHECK_TIME_EXCEEDED: Max effort limit of
 '0.2M' reached. For unconverged properties set a higher effort limit
 with fml_max_check_time and rerun.
```

Limitations

Currently, the VC Formal GUI does not reflect the `fml_max_check_time` value, and only the `fml_max_time` value is shown. This will be addressed in a future releases.

6.5.4 Ensuring Executing Set of Commands Post check_fv with -run_finish Option

VC Formal DPV enables to use the `-run_finish` option to execute a specified set of commands when the `check_fv` run is finished. You can work with the shell in an interactive mode while the `check_fv` run is in progress, and also schedule the set of commands post `check_fv` when the run is completed.

In the following example, `check_fv` for P1 is executed in the non-blocking mode and the listlemmas and listassumes commands in the block are executed when `check_fv -run_finish` is completed.

```
check_fv -run_finish {
  listlemmas
  listassumes
}
gen_proof P1
check_fv P1
```

6.5.5 Specifying Maximum Progress Time Limit

You can specify the maximum progress time limit using the `fml_progress_time_limit` formal variable. The maximum progress time is the time for which the `check_fv` command does not receive any useful information from the Formal engines. The `fml_progress_time_limit` variable acts as a clock that gets reset when the `check_fv` command receives an useful information (such as proof, falsification, increase in bounded depth, information related to constraints, and so on) from the Formal engines.

```
set_fml_var fml_progress_time_limit 2H
```

If `check_fv` command does not receive any useful update in the specified time limit, the following warning message is issued, and the `check_fv` command stops:

[Warning] SPECIFIED_PRGS_TIME_EXCEEDED: Specified max progress time limit of '2H' reached. For unconverged properties set a higher effort limit and rerun.

By default, the `fml_progress_time_limit` variable is set to -1, which means that it is set to infinite time, and the unproductive timeout feature is disabled.

6.5.6 Restoring Session from Abnormal Termination of check_fv Run

You can recover a session if there is an accidental tool crash (for example, if a job gets killed on a server farm because of the IT resource utilization policy) while executing the `check_fv` command. To enable this feature, set the following application variable to true.

```
set_app_var fml_auto_save true/false
```

When the `fml_auto_save` application variable is set to true, if VC Formal crashes while executing the `check_fv` command, you can restore the run using the `-restore` option and return the prompt to run the subsequent report commands or re-initiate the `check_fv` command.

To restore the session, you must open the `vcf` shell in the restore mode in the same location and session name as in their primary run.

```
vcf -session <session_name> -restore
```

This command returns VC Formal to the last-known-good-state and gives the prompt back to the user.

VC Formal does not restore the session if it terminates before the execution of the `check_fv` command begins, or before the first check-pointing is done in the `check_fv` command. The first check-pointing is done after the formal model creation and before VC Formal starts solving any goal. The following information message is displayed when the first checkpoint is done.

INFO: Automatic checkpointing has been activated.

6.5.7 Improving Performance of Subsequent check_fv runs

The progress of the `check_fv` command can be resource controlled using the *set_fml_var fml_max_time* formal variable. When the time specified in this formal variable has reached, the `check_fv` command stops. You can also interactively stop the `check_fv` command using the `check_fv -stop` option.

You can resume the `check_fv` run from where it had stopped by setting the `fml_enable_resume` formal variable to `true`.

By default, the `fml_enable_resume` application variable is set to `false`.

When the `fml_enable_resume` application variable is set to true, the subsequent `check_fv` command resumes the formal verification strategies (engine orchestration) from where it had stopped in the last run and thus improves the overall performance. If there is any change in the problem definition or model (for example, addition/deletion of constraints, snip points, new properties, abstractions, change at, and so on), then VC Formal detects such changes and restarts from the scratch.

6.5.8 Resuming check_fv from the Last Known Maximum Bounded Depth

When you resume `check_fv` on a set of un-converged properties, their last known bounded depth is preserved in the property database. However, the engines start exploring again from depth 0 in the resumed session.

Set the `fml_enable_resume_depth` application variable to true when you want the engines to restart from the last known maximum bounded depth in the resumed session.

```
set_fml_var fml_enable_resume_depth true
```

By default, the `fml_enable_resume_depth` application variable is set to `false`.

Note:

Starting from last known bounded depth is not always useful, as 'learning' from lower depths helps the engine to reach higher depths. Hence, performance improvement is not guaranteed with incremental BMC.

6.5.9 Using Proven Assertions for Converging on Subsequent Assertions

You can enable VC Formal to use every proven assertions to help converge on the other subsequent assertions. This helps improving the performance of the `check_fv` run. To enable this functionality, use the following command:

```
vcf> set_fml_var fml_enable_assume_proof true
```

Note:

While this may help performance in some cases, it may also impact performance in few other cases.

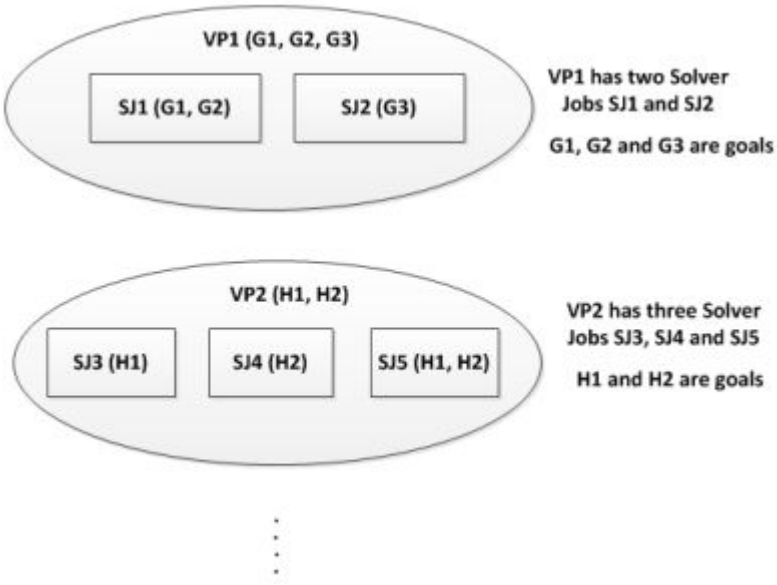
6.6 Viewing Progress of check_fv Runs

Each `check_fv` command creates a goal that is handed to the orchestration. Each goal is a proof object and has its set of constraints (assumptions, witness and vacuity) and assertions (can include different types such as SVA asserts, vacuity, SVA covers and so on). Each goal has a unique integer ID to distinguish it from other goals.

The orchestration solves a goals by creating multiple SolverJobs. Each SolverJob has following characteristics:

- It is a unit of work dispatched on local/remote machine or grid.
- It runs on a single core and has an unique integer ID.
- It has a label (not unique) and can employ one or more engines but they run sequentially.
- It has a subset of goals in VP.
- It eventually gets assigned to a worker on some machine.

Figure 27: Two Verification Problems VP1 and VP2 and the Solver Jobs Working in those Verification Problems



6.7 Viewing Details for VC Formal Verification Goals

VC Formal provides the `report_fml_engines` and `report_fml_jobs` commands to view the following information on the goals that helps you debug and converge on issues.

- The number of proven, falsified, bounded properties (along with depth).
- The SolverJobs that are working for a given goal and their status:
  - You can know which engines are unproductive or what is the best engine for a given problem. For hard goals, you can know the bounded depth reached by different engines.
- Goals that are being worked on by orchestration at any given time:
  - You can know how goals are being prioritized by orchestration. For example, you can see that safety goals are given more priority to the liveness goals. This enables you to instruct orchestration to solve goals in a different order by creating multiple user level tasks.
- Whether the SolverJobs are running out of memory/inconclusive/error:
  - You can know which engines are running out of memory or crashing.
- Number of workers available/in-use:
  - You can know how many grid resources are actually allocated and how the resources change over time. For example, if you asked for 100 slots and you only got 10, then you can take appropriate action such as ask for more resources or simplify the problem to get acceptable run times.
- The progress of the engine run on a goal.
- Viewing the Goals Depths Distribution and Viewing the Grid Workers Summary reports in the VC Formal GUI that can be analyzed easily.

6.7.1 The report_fml_engines Command

The `report_fml_engines` command reports the different SolverJobs that goal is present in and the status of that goal in each SolverJob that contains that goal, in a table format for each goal. By default, `report_fml_engines` command prints a summary for the latest `check_fv` command in the current verification task.

The summary includes total number of goals in a verification problem, number of proven goals, number of falsified goals, number of bounded goals. Summary by itself does not provide any more information over `report_fv`.

Syntax

```
report_fml_engines [-no_summary] [-list] [-verbose] [-depthCounts]
 [-depthVsTime] \
[-status <proven, falsified, bounded>] \
[-jobId <num>] [-goalId <num>] [-engine <string>] \
[-subtype <vacuity,witness,->] [-jobStatus <scheduled,running,completed>]
```

Where,

- `-no_summary:` Does not print summary information.
- `-list:`When this option is specified, one row is printed for each goal. The row contains a conclusive result proven/falsified for a property if available. If no conclusive result is available it contains the best bounded proof depth (if available), if this is not available it contains `no_status`. The entries in the row are as follows:

  `GoalId, Status, Engine name, Time, SDepth, FDepth, JobId, SubType, PropertyName`

  - The *PropertyName* column contains the name of the property.

    For a given property, there can be multiple goals generated; one corresponding to the actual property, one for witness (if enabled) and one for vacuity (if enabled). The *GoalId* column contains a unique id assigned to each goal arising from a property. The *SubType* field contains "-" for actual property, *witness* for witness version of the property, and *vacuity* for vacuity check corresponding to the property.
  - The *JobId* gives the number of the solver job that got a proven/falsified result for the goal or the best bounded result. In addition, the *JobId* field has an indicator to indicate the status of the job (s) for scheduled, (r) for running, (c) for completed.
  - The *Engine* column gives the engine name that got proven/falsified result for a goal or best bounded result.
  - Status can be one of the following

    `no_status, proven, falsified, bounded`
  - *SDepth* is the safe depth of the goal. This is the depth for which the goal has a bounded proof. *Depth* is the depth at which a goal is falsified.
  - *Time* column is the sum of time taken by all engines working on this property in seconds. For the SEQ application, sub-proofs named `sdcp_*` and `edcp_*` are created to simplify the problem. The engine column for SEQ can contain names of these sub-proofs. The runtime for a sub-proof is a sum-total of all engines that were run in that sub-proof. It is not wall-clock time.
- `-verbose:`Prints multiple lines per goal. Each line corresponds to one engine working on a goal. The format of the line is same as the `-list` option.

  In the verbose report, the *status, time, SDepth, FDepth* of that particular engine is reported.
- -`depthCounts:` Displays two tables, one for falsified goals and one for bounded goals. Each row in the table has two columns. The first column is the depth and the second column contains the number of properties that are falsified/bounded at that depth.

Feedback

- `-depthVsTime:` Reports the change in BMC depth with respect to time for a particular goal (for each engine that worked on that goal). This option works only when the application variable `fml_orc_bmc_depth_profile` is set to true and the `-goalId` option is also given.
- `-status <val>:` Only lists the goals that have a particular status *proven*, *falsified* and *bounded*.
- `-jobId <id>:` Shows only goals where solver job <id> provided a conclusive result or provided the best bounded depth.
- `-goalId <gid>:` Lists only information for a particular goal id.
- `-engine <name>:` Lists only goals where engine with given name provided conclusive result or best bounded depth.
- `-subtype <stype>:` Lists only goals with a matching subtype *-, vacuity, witness*. The meaning is as follows:
  - - refers to original assertion
  - *vacuity* refers to vacuity component of an assertion
  - *witness* refers to witness component of an assertion
- `-jobStatus <js>:` Shows only rows where the corresponding job has the specified status `<js>`, where `<js>` can be scheduled, running, completed. Note that with `-list` option is given the job that is shown is the job that has provided best result so far.

Example

An example of the output in shown in Figure 28.

Figure 28: Goal View for a Particular VP

```
Summary for verification problem vp0 (id=0) in verification task _default
#NumGoals      :    46
    #no_status     :     0
    #falsified     :    25
    #proven        :    21
    #bounded       :     0

List of goals for verification problem vp0 (id=0) in verification task _default
-------------------------------------------------------------------------------------------------------------------------------
GoalId |     Status | EngName |     Time | SDepth | FDepth |      JobId |  SubType | PropertyName
-------------------------------------------------------------------------------------------------------------------------------
     0 |  falsified |      s1 |        0 |      - |      3 |      2 (c) |  vacuity | traffic.chk.assert_green_no_waiting_first
     1 |  falsified |      b1 |        0 |      0 |      1 |      5 (c) |        - | traffic.chk.assert_green_no_waiting_first
     2 |     proven |      e2 |        0 |     10 |      - |      4 (c) |  witness | traffic.chk.assert_green_no_waiting_first
     3 |     proven |      t3 |        0 |      - |      - |      3 (c) |  vacuity | traffic.chk.assert_green_no_waiting_main
     4 |     proven |      t3 |        0 |     10 |      - |      3 (c) |        - | traffic.chk.assert_green_no_waiting_main
     5 |     proven |      t3 |        0 |     10 |      - |      3 (c) |  witness | traffic.chk.assert_green_no_waiting_main
     6 |     proven |      t3 |        0 |     10 |      - |      3 (c) |        - | traffic.chk.assert_gyr_oh_first
     7 |  falsified |      b1 |        0 |      0 |      1 |      5 (c) |  witness | traffic.chk.assert_gyr_oh_first
     8 |     proven |      t3 |        0 |     10 |      - |      3 (c) |        - | traffic.chk.assert_gyr_oh_main
     9 |  falsified |      b1 |        0 |      0 |      1 |      5 (c) |  witness | traffic.chk.assert_gyr_oh_main
    10 |  falsified |      s1 |        0 |      - |      5 |      2 (c) |  vacuity | traffic.chk.assert_hazard_in_first
    11 |     proven |      e2 |        0 |     10 |      - |      4 (c) |        - | traffic.chk.assert_hazard_in_first
```

Feedback

The table in Figure 28 hows information about goals 0 to 11. Goal 0 corresponds to vacuity component of the `traffic.chk.assert_green_no_waiting_first` property, while goal 1 corresponds to the `traffic.chk.assert_green_no_waiting_first itself` property. Goal 0 was falsified using engine *s1* in 3 clock cycles (*FDepth*) using solver job number 2. You can filter the table based on column values.

6.7.2 The report_fml_jobs Command

The `report_fml_jobs` command reports the status of each solver job. It displays the solver job status (scheduled/running/completed), machine name, time, memory, and the number of goals assigned to that solver job in a table format.

Syntax

```
Usage: report_fml_jobs    # Reports information about solver jobs running
 locally or on grid.
        [-status <status-attributes>]
                               (Running status of a solver job:
                                Values: "", completed, running,
                                scheduled)
        [-reason <result-attributes>]
                               (Reason for completion of a solver job:
                                Values: -, crash, memout, misc_error,
                                normal, orc_killed, sig_killed,
                                start_fail, timeout)
        [-list]                (Outputs detailed report in list form)
        [-totaltime]           (Reports the total jobs time)
        [-no_summary]          (Do not print summary report)
        [-engine <string>]     (list information belonging to a
 particular engine.)
        [-host <string>]       (list information belonging to a
 particular machine.)
        [-jobId <integer>]     (list information for a particular job.)
        [-profile]             (Profile the memory and time for jobs
 based on engine tag and summarize job count based on memory usage)
        [-memBucket <integer>] (Control size of memory buckets. Default
 is 8 (unit is in GB).)
   [-grid_level <integer>]     (Reports information associated with a
 particular grid level)
```

Where,

- `-status <val>`: Shows only solver job matching a particular status.
- `-reason <val>`: Shows only solver job matching a particular reason for completion.
- `-list`:Displays one line per solver job. The line contains the solver job id, status (scheduled/running/completed), reason (`normal/timeout/memout/ orc_killed,sig_killed,crash, start_fail, misc_error`) for completed solver jobs, wall-clock time in seconds, CpuUtil (ratio of CPU time over wall-clock time for a

job expressed as a percentage), memory consumption in MB, number of goals in the solver job, engine id in the solver job, engine name in the solver job, timestamp for when the solver job was started, the id for the physical machine where the solver job is being run, machine name.

In machines with heavy loads a job may not get enough CPU cycles leading to CpuUtil less than 100%. A low CPU utilization can negatively impact the performance of the tool. It is recommended to switch to dedicated machines or less-loaded grids or run check_fv for longer time in such cases.

  - *normal*: Solver job that completed normally.
  - *timeout*: Solver job that timed out. The time limit is specified by orchestration in the recipes. It is normal for some of the solver job to timeout. This is because orchestration tries to run engines with small time limits first, and if the property is hard, such solver job may run out of time before the property converges.
  - *memout*: Solver job that had memory out. The memory limit can be specified by orchestration in the recipes or provided by user using `fml_max_mem` application variable. If a solver job is killed because memout limit is reached, the result is reported as *memout*.
  - *orc_killed*: Solver job that were killed by orchestration because another solver job solved the goal(s). This is quite normal because many jobs simultaneously try to solve a goal. When goal is converged, all other jobs working on the same goal are killed by the orchestration and their status is *orc_killed*.
  - *sig_killed*: Solver job that were killed by some external signal. For example, you can specify the memory limit on a compute grid (for example, LSF). If the job exceeds that memory limit, the job is killed by the grid. In such cases, the status of the job is `sig_killed`.
  - *crash*: Solver job that had a crash. In general, orchestration is designed to be tolerant to such jobs. If many jobs have this status, reach out to Synopsys support.
  - *start_fail*: Solver job that could not be started. Only used in SEQ app.
  - *misc_error*: Solver job that had completed with unclassified error. Only used in SEQ app.

- `-no_summary:` Does not print summary.
- `-engine <string>:`Shows only solver job that are using a specified engine name.
- `-host <string>:`Shows only solver job that were run or are running on a particular host.
- `-jobId <id>:` Shows only the solver job `<id>.`

Feedback

- `-profile`: When this option is specified, the command reports the following:
  - For each engine tag, the command prints minimum memory and maximum memory, number of jobs of this engine that stopped due to memory out/crash/external signal, the total time spent for all jobs matching the given engine tag.
  - `NumSolvedGoals`: Number of goals solved by the engine.
  - `NumSuccessGoals`: Number of goals with success classification solved by engine. Meaning of success/failed is same as in `report_fml_engines`.
  - `NumFailedGoals`: Number of goals with *failed* classification solved by engine.
  - `NumBestDepthGoals`: Number of goals for which this engine got best bounded depth (meaningful for safety engines).
- Prints a breakdown of memory used for completed jobs in ranges of 8 GB.
- Prints the average and maximum memory used among all completed jobs.
- `-memBucket integer`: Controls the size of memory buckets. Default is 8 (unit is in GB).
- `-grid_level integer`: Sorts the output based on the grid levels (applicable when multiples queues are used).

  Note:

  When a job (engine) is started, the initial number of goals assigned to that job is reported in the *#Goals* column. For certain engines, it is possible that the engine can work on more goals than it was assigned to improve QoR. In such cases, the *#Goals*reported in the `report_fml_jobs -jobId <j>` command may not match with the number of rows reported in the `report_fml_engines -verbose -jobId <j>` command.

Example

Figure 29 illustrates few example outputs of `report_fml_jobs`.

Figure 29: Examples for report_fml_jobs

```
vcf>report_fml_jobs -list
Solver jobs summary for verification task FPV (91 solver jobs)
    #scheduled     :     0
    #running       :    12
    #completed     :    79
        #normal        :    26
        #timeout       :    51
        #memout        :     0
        #orc_killed    :     2
        #sig_killed    :     0
        #crash         :     0
        #start_fail    :     0
        #misc_error    :     0
Solver jobs report for verification problem vp2 (id=2) in verification task FPV
-------------------------------------------------------------------------------------------------------------------------------------------------
  JobId |      Status |       Reason |     Time | CpuUtil |   Memory |   #Goals | EngName |             Started | HostId |            Host |           FvOrc |
-------------------------------------------------------------------------------------------------------------------------------------------------
      0 |    completed |       normal |        1 |       0 |       70 |        0 |    INIT |  2023-06-07 10:30:33 |      1 | us01odcres-ayann-0110 |               - |
      1 |    completed |       normal |        0 |       - |      107 |        0 |      cc |  2023-06-07 10:30:34 |      2 | us01odcres-ayann-0110 |               - |
      2 |    completed |       normal |       10 |     100 |      488 |        0 |      t1 |  2023-06-07 10:30:35 |      3 | us01odcres-ayann-0110 |               - |
      3 |    completed |       normal |        0 |       - |       70 |        0 |      t0 |  2023-06-07 10:30:35 |      7 | us01odcres-ayann-0110 |               - |
      4 |    completed |      timeout |      300 |     100 |      499 |      134 |      b8 |  2023-06-07 10:30:36 |      8 | us01odcres-ayann-0110 |               - |
      5 |      running |            - |        0 |       - |        0 |      134 |      b1 |  2023-06-07 10:30:36 |      4 | us01odcres-ayann-0110 |               - |
```

Feedback

```
vcf>report_fml_jobs -profile
Solver jobs summary for verification task FPV (98 solver jobs)
    #scheduled     :     0
    #running       :    12
    #completed     :    86
        #normal        :    30
        #timeout       :    52
        #memout        :     0
        #orc_killed    :     4
        #sig_killed    :     0
        #crash         :     0
        #start_fail    :     0
        #misc_error    :     0
Completed jobs profile for verification problem vp2 (id=2) in verification task FPV
Memory/time profile by engine
-----------------------------------------------------------------------------------------------------------------------------------------------------
 Engine |   MinMem(MB) |   MaxMem(MB) |  NumJobs | NumStoppedJobs |  Time(secs) | NumSolvedGoals | NumSuccessGoals |  NumFailedGoals | NumBestDepthGoals |
-----------------------------------------------------------------------------------------------------------------------------------------------------
   INIT |           70 |           70 |        2 |              0 |           2 |              0 |               0 |               0 |                 0 |
     b1 |          206 |          432 |        3 |              0 |         148 |             42 |              42 |               0 |                 2 |
     b8 |          499 |          499 |        1 |              0 |         300 |             27 |              27 |               0 |                 0 |
     cc |          107 |          107 |        1 |              0 |           0 |              0 |               0 |               0 |                 0 |
     e1 |          107 |          142 |       17 |              0 |        2235 |             24 |              24 |               0 |                 0 |
    e14 |          137 |          146 |        6 |              0 |         770 |              0 |               0 |               0 |                 0 |
    e15 |          250 |          265 |        3 |              0 |         517 |              1 |               1 |               0 |                 0 |
    e16 |          151 |          163 |        6 |              0 |        1539 |              0 |               0 |               0 |                 0 |
     e2 |          186 |         1060 |        5 |              0 |         351 |             21 |              21 |               0 |                 0 |
     e3 |          420 |         1035 |        9 |              0 |         829 |              2 |               2 |               0 |                 0 |
     e4 |          195 |          270 |        2 |              0 |         310 |              0 |               0 |               0 |                 0 |
     e7 |          259 |          402 |        6 |              0 |         288 |              0 |               0 |               0 |                 0 |
     e8 |          132 |          143 |        6 |              0 |         288 |              0 |               0 |               0 |                 0 |
     f5 |           70 |          159 |        8 |              0 |           9 |              0 |               0 |               0 |                 0 |
     s1 |          163 |          179 |        6 |              0 |         364 |              1 |               1 |               0 |                 0 |
     t0 |           70 |           70 |        1 |              0 |           0 |              0 |               0 |               0 |                 0 |
     t1 |          488 |          488 |        1 |              0 |          10 |             14 |              14 |               0 |                 0 |
     t3 |          109 |          109 |        3 |              0 |           3 |              0 |               0 |               0 |                 0 |
-----------------------------------------------------------------------------------------------------------------------------------------------------
Memory distribution
-------------------------------
          Range |      NumJobs |
-------------------------------
      0GB - 8GB |           86 |
-------------------------------
Avg. Memory among completed jobs : 222 MB
Max. Memory among completed jobs : 1060 MB
1
```

6.7.3 Reporting Peak Engine Memory

The `get_resource_cost` command reports the wall time, cpu time, peak memory, and current set resident memory size used in real time.

Syntax

```
vcf> get_resource_cost -help
Usage: get_resource_cost    # Get Wall/Cpu time, peak memory, or current
 resident set memory size
        [-tcl]                  (Return data only in a tcl array/string.
 {Wall Cpu Mem Rss})
        [-cpu]                  (Return cpu time in seconds since startup)
        [-wall]                 (Return wall time in seconds since
 startup)
        [-mem]                  (Return current total memory footprint in
 MB)
        [-rss]                  (Return current memory Resident Set Size
 in MB)
        [-mallocmem]            (Return current malloced memory in MB)
        [-engine]               (Return backend engine wall time and peak
 memory)
        [<cmd-to-execute>]      (Command to execute and time.  Returned
 data is delta for this command only)
```

Examples

The following is an example output of the `get_resource_cost` command.

```
vcf> get_resource_cost
=======================================================
Total Time(S)  :2373.97
CPU Time(S)    :2.71
Peak Memory(MB):1682
Current RSS(MB):620
Current Malloc(MB):407
=======================================================
```

6.7.4 Bound Statistics for a Goal

The goal bound statistics reports how the proof depth changes for a goal over time. To enable the profiling mode, the following application variable must be set to true.

```
set_app_var fml_orc_bmc_depth_profile true
```

An example of the output is shown in Table 7

Table 7: Bound Statistics for Goals

Goal ID 0	
Time	**Depth**
10	1
20	2
30	3
40	4
50	5
60	6
70	7
80	8
90	9
100	10

Feedback

6.8 Viewing the Task Progress Report

The task progress report provides the intermediate progress of jobs that are currently being run. Using the task progress report, you can check the health and result of jobs.

Note:

Viewing the jobs that are in progress is supported only in the FPV and SEQ application modes.

6.8.1 Viewing the Job Summary Report

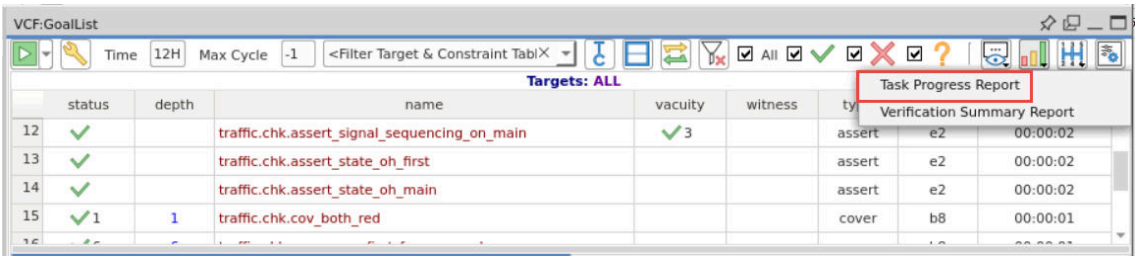
The **Job Summary** report contains the following:

- *Job Status*: Shows the Job status (scheduled/running/completed), machine name, time, memory, and the number of goals assigned to that Job in a table format.
- *Job Details Table*: Prints multiple lines per goal. Each line corresponds to one engine working on a goal. Shows the Status, Reason, Time, Memory, #Goals, EngName, and so on of that particular engine. You can also filter out unrelated data item using the filter option.

To view the **Job Summary** Report:

1. Click and select **Task Progress Report**.

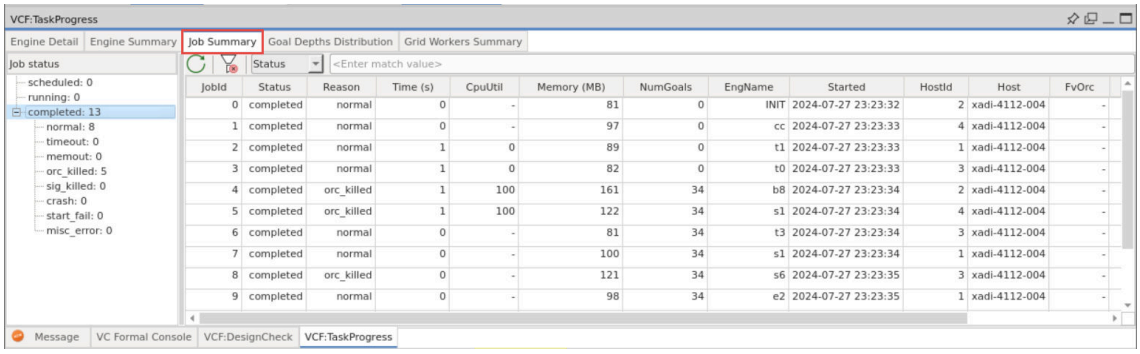
Figure 30: Task Progress Report



Feedback

2. The *VCF:TaskProgress* report is displayed. Click **Job Summary** as shown in the following figure:

Figure 31: Viewing the Job Summary Report



3. You can select the **EngName** from the list and view the job details per engine as well. Or **Enter match value** in the text box to filter data.

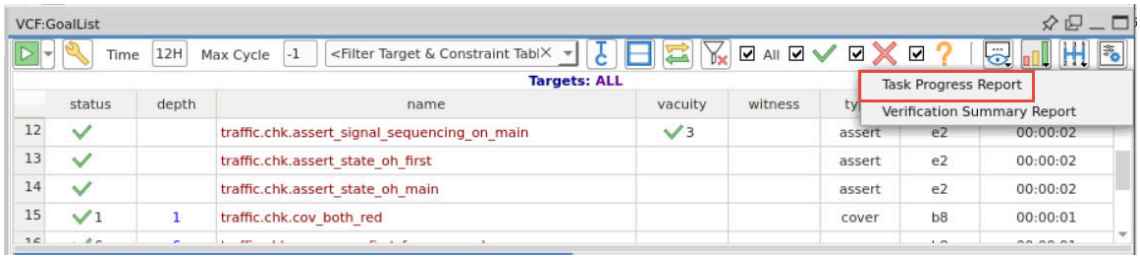
6.8.2 Viewing the Engine Details

The *Engine Detail* table indicates which engines are running and its status for each goal.

To view the engine details:

- Click in the **GoalList** tooolbar, and select **Task Progress Report**.

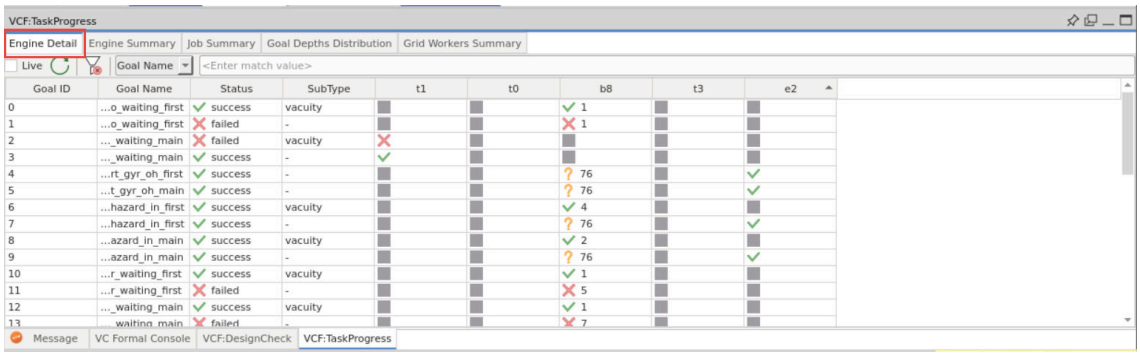
Figure 32: Navigating to Engine Details



Feedback

The *VCF:TaskProgress* report is displayed. The **Engine Detail** table appears as illustrated in the following figure. For more details on the engines, see section Orchestration in VC Formal.

Figure 33: Engine Details Report



6.8.3 Viewing the Goals Depths Distribution

You can view the engines' performance, goals achieved over depths, and the workers and grid utilization for each of the properties.

You can view the progress of a property at two levels:

1. The first level is to view the task progress for one or more properties or all the properties for a run. This can be invoked from the GoalList toolbar. For more details on how to open the task progress for all the properties, see section Using the GoalList.
2. The second level is done at the individual property level which provides the progress details for a selected property. The following sub-sections describe how you can view the task progress for individual properties. You can generate the following reports for each of the properties:
   - Goals Depths Distribution
   - Grid View
   - Engine Summary View

This graphs describes how many goals are solved at what depth. This is a quick eye chart to see how quick progress is being made by Formal. This information can also be used to optimize the time allotted for formal regressions.

The *Goal Depths Distribution* graph is reported for a full task instead of a single property.

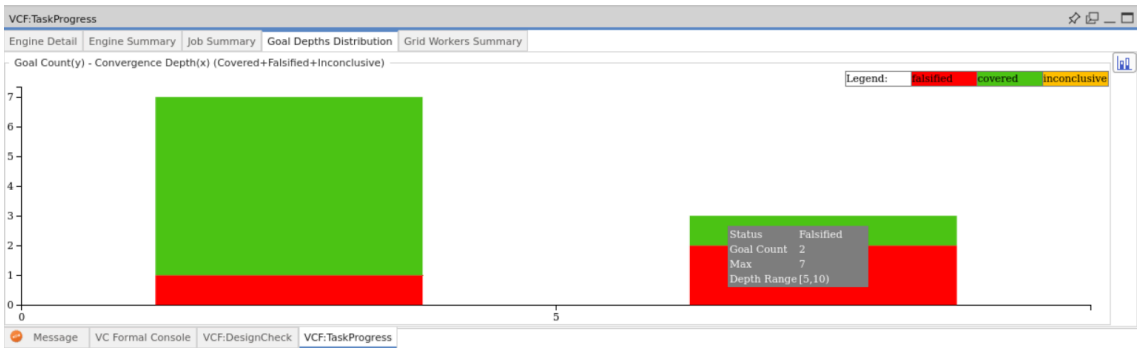
Feedback

To view the *Goals Depths Distribution*graph:

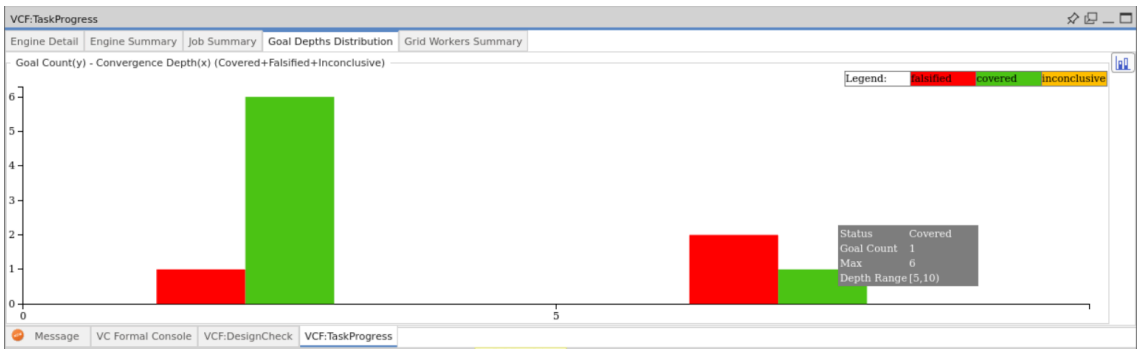
1. In the **TaskList**, select a property.
2. In the **GoalList** toolbar, click and select **Task Progress Report**.

   The **VCF:TaskProgress** report is displayed. To view the distribution of the goal depths, click **Goals Depths Distribution** as shown in the following figure:

Figure 34: Goals Depths Distribution Graph



3. Use the button to switch between the 3-bar mode view and the stacked bar mode view.



6.8.4 Viewing the Grid Workers Summary

The *Grid Workers Summary* graph provides details of the grid utilization over a period of time.

You can use the *Grid Workers Summary* graph to see the number of grid workers over a period of time, the available grid workers, and the running workers that are out-of-memory or out-of-time.

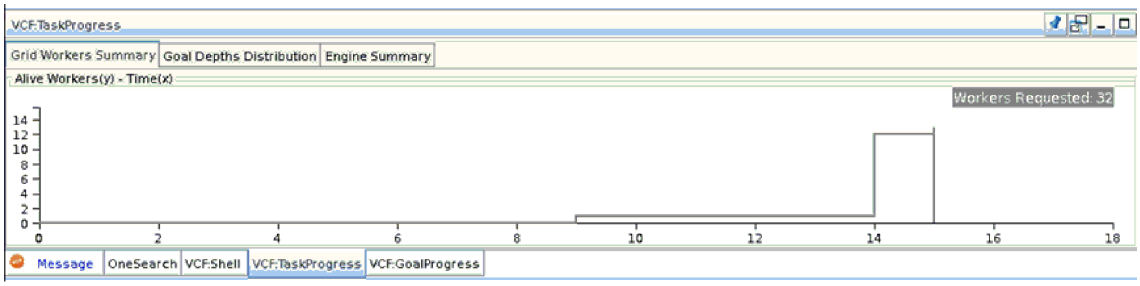
Feedback

To view Grid Workers Summary:

1. In the **TaskList**, select a property.
2. In the **GoalList** toolbar, click and select **Task Progress Report**.

   The **VCF:TaskProgress** report is displayed. To view the grid utilization, click **Grid Workers Summary** as shown in the following figure:

Figure 35: Grid Workers Summary Report



3. Click to refresh the *Grid Workers Summary*graph.

The following is an example output of the `report_fml_hosts -history` Tcl command which shows a tabular view of the graph shown in Figure 35:

```
Timestamp    WorkerCount
10                     1
100                    2
120                    3
140                    2
…
```

6.9 Orchestration in VC Formal

VC Formal uses a rich set of formal and static algorithmic engines. These engines are sequenced in different ways depending on the task at hand. This is termed as orchestration. Different orchestrations are called recipes. Different modes use different recipes. For example, FPV mode uses different recipes as compared to COV mode.

6.9.1 Configuring Orchestration in VC Formal

You can specify effort levels or the type of effort required instead of specifying the recipes.

Specifying the effort level or the type of effort required:

- Improves the performance of the `check_fv` command over the given set of properties, and helps in finding easy ways to solve properties.
- Enables you to perform a quick run on COI properties to find root causes of proof complexity.

You can specify the effort levels using the `fml_effort` application variable.

```
vcf> set_fml_var fml_effort <default high bounded bug_hunting discovery
bitlevel_hard no_decompose_expensive userorc>
```

This application variable can take the following values:

- `default`: The default value of `fml_effort` is *default*. If set to default or omitted, it activates the current default recipe for the `check_fv` command.
- `easy`:Set the effort level to easy when you have many easy properties and only few workers to work on them.
- `high`: If set to high, the `prop_check_heavy` recipe is triggered.
- `bug_hunting`: If set to `bug_hunting`, the `check_fv` command prioritizes finding falsifications and reachable coverage targets. You can also configure bug hunting for effort level. For more details, see sections Configuring Engines for VC Formal Orchestration, Generating Cover Properties for Bug Hunting, and Providing User-Defined Property Order to Improve Bug Hunting.
- `discovery`:If set to discovery, a large number of different strategies are applied for solving a single hard property. When you set this option, the recommended number of workers to use is 20 per property.
- `no_decompose_expensive`: The `no_decompose_expensive` effort level avoids the decomposition step. This effort level uses the FPV style of recipe, in some rare cases useful for finding failures.
- `userorc`: Set the effort level to user orchestration.
- `bitlevel_hard`: Use the `bitlevel_hard` effort level when you have SEQ designs with very little word-level correspondence.

The following table contains the different effort levels that can be defined for each of the VC Formal applications.

Application Name	Configurable Efforts Levels
FPV	`default high bug_hunting discovery userorc`

Feedback

Application Name	Configurable Efforts Levels
AEP	`default high bug_hunting discovery userorc`
FCA (COV)	`default high bug_hunting discovery userorc`
CC	`default high bug_hunting discovery userorc`
FRV	`default high bug_hunting discovery userorc`
FTA	`default high bug_hunting discovery userorc`
FuSa	`default easy high bounded bug_hunting no_decompose_expensive userorc`
SEQ	`default easy high bounded bug_hunting discovery bitlevel_hard no_decompose_expensive`
FXP	`default easy high bounded bug_hunting no_decompose_expensive`
FSV	`default easy high bounded bug_hunting no_decompose_expensive`

The *high* and *discovery* effort levels are targeted for future deprecation. If you set *high* or *discovery* as `fml_effort` value, the FML_EFFORT_FUTURE_DEPRECATION warning is reported during `check_fv`. It is recommended to use the *default* effort value instead.

Examples

[Warning] FML_EFFORT_FUTURE_DEPRECATION: fml_effort level high will be deprecated soon. We recommend default should be tried first.

[Warning] FML_EFFORT_FUTURE_DEPRECATION: fml_effort level discovery will be deprecated soon. We recommend default should be tried first.

6.9.2 Configuring Engines for VC Formal Orchestration

VC Formal reports the success/failure of the engine and the engines' usage statistics. You can use this information to attain more focused performance results in some cases by selectively enabling and disabling specific engines. This enables you to save session results, and then improve performance in subsequent runs of the same design. This is helpful when verification of test cases are run in regression after the initial verification has been performed.

The `set_engine` command can be used to enable/disable specific engine IDs for use in the upcoming `check_fv` command. Start the first run in the default mode. Try to reuse productive engines and disable the unproductive ones for the same design in the future runs.

Feedback

Syntax

```
vcf> set_engine -help
Usage: set_engine    # set engine on/off or score
        [-on]                   (turn on the engine)
        [-off]                  (turn off the engine)
        <id>                    (engine id)
```

VC Formal employs state-of-the-art engines to prove or falsify a property. You can enable or disable multiple engines at a time with the `set_engine` command. For example:

```
set_engine -on {e* b1 b6}
set_engine -off {e* b1 b6}
```

The engine names in VC Formal are reported in various places such as GUI and TCL commands. These engine labels are described in Table 8.

Table 8: Engine Labeling in VC Formal

Engine Labels	Description
e* (e1,…,e18)	Proof Engines
b* (b1, ..., b7)	Bounded Proof Engines
r* (r1, ..., r5)	Abstraction Engines
l* (l1,…,l12)	Liveness Engines
s* (s1,…,s12)	Simulation Engines
t* (t1,…,t9)	Transformation Engines
k* (k1, k2, k3, k4)	Livelock/Deadlock Engines
de	Deadend Analysis Engine
cc	Conflicting Constraints Checker
nc	Reduced Constraints Engine

Note:

You cannot disable the transformation engines (*t**) and the conflict checker engines (*cc*) using the `set_engine -on/off` command. The conflict checker engine is always enabled. You can control the strength of the conflict checker engine using the Formal application variables described in section Viewing Progress of check_fv Runs.

Feedback

6.10 Viewing the Solve/Engine Time for Proven/Falsified Goals

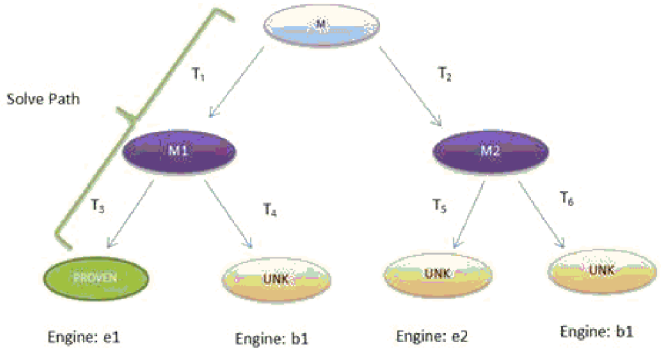
VC Formal reports the solve/engine time taken to solve individual properties. The solve/engine time is the time spent in a successful attempt while directly working on the property in the context of the current run. In few other contexts, for example, the property in isolation or with a different recipe, the solve/engine time is potentially different but is at best a rough estimate.

Example:

Suppose you want to solve a property p1. Orchestration may apply different strategies to solve the goal. As you see in Figure 36, VC Formal applies different transformations to the verification problem and running different engine (e1, e2 and b1) on the transformed model. Eventually e1 proves the property. Therefore, the solve time for p1 is as follows:

(Time taken to create transformed model M1 (T1) + Time taken by e1 in the context of M1 (T3))

Figure 36: Solve Time Calculation Example



VC Formal reports the *solve/engine* time in textual report as well as in the VC Static GUI:

1. During `report_fv/cov -verbose` Tcl commands:

```
> Assertion
    # Assertion: 1
    > ID: [570] proven
     - name          : p1
     - language      : SVA
     - type          : assert
     - usage         : assert
     - update_time   : 18:05:36 - Jan 15
     - elapsed_time  : 01:06:14
     - solve_time    : 00:00:47
     - location      : test.v:182
     - engine        : e1
```

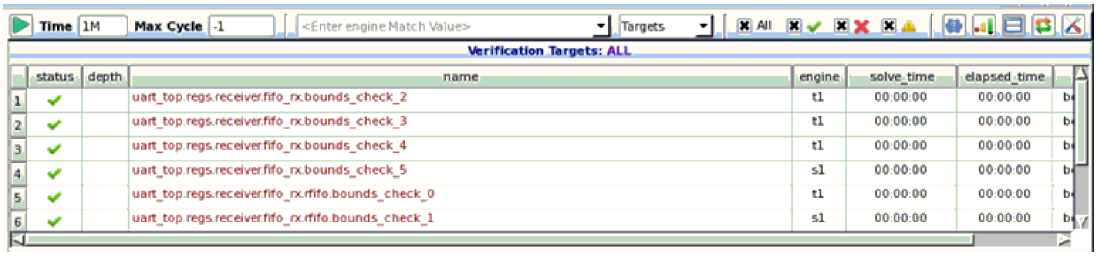
Feedback

Note:

In the textual report, the solve_time is reported as 00:00:00 for unconverged properties and this is just a place holder.

2. In VC Formal GUI,
   - In full-field GoalList view, the *solve_time* column reports the solve time.

Figure 37: Reporting Solve Time in Full-Field GoalList View



6.10.1 Limitations

- Solve time is not supported for SEQ application.
- When you disable all goals except one, that property is not guaranteed to be solved during reported solve time. The solve time can potentially change (increase or decrease) with respect to the setup including the number of the active goals, that is, it can happen that a goal is solved faster when it is being run without other goals, as it can reuse some information that it learns while solving rest of goals.

6.11 Viewing Bad Code

VC Formal helps you view the bad code in the design with a special attribute named simplication. VC Formal analysis the Line and Condition FCA goals. If FCA goals are trivially proven, then they are considered to be uncoverable. This is used as a pointer for separating the bad code in a design.

To decide on the trivially proven properties, t3 engine is used by performing logic reductions on the design. If the constant is reduced to 0 (uncoverable) through this engine, all of those goals are reported as trivially uncoverable.

Note:

The t3 engine must run in blocking mode and finish for each goal before any other engine works on that goal.

6.11.1 Use Model

You can access this feature by enabling the following fml var:

```
fml_fca_enable_bad_code_check false
```

By default, this variable set to false. If you want to activate this feature, set it to true. Set this formal variable to true before running the `check_fv` command in the COV app mode for the most effective output. If set after the `check_fv` command, there are chances that the bad code can get missed as proper orchestration of formal engines would not have happened.

The t3 engine checks each Line and Condition coverage goals to see if any of the coverage goals are trivially uncoverable. If yes, it marks the goals with the attribute `reason` as `simplication`, besides being marked as uncoverable. This attribute marking is readable if the `fml_fca_enable_bad_code_check` variable is set to true.

When the first trivially uncoverable goal is found, the following warning message is displayed:

```
[Warning] FCA_TRIVIAL_UNCOVERABLES_FOUND: Trivially uncoverable line
 and/or condition goals are found, indicating possibility of bad code.
```

The details of these goals are written to the file: `vcst_rtdb/logs/trivialUncoverables_COV.rpt.`

As mentioned in the warning message, the trivially uncoverable goals are written with their details and location till they are found.

At the end of the `check_fv` run, the following warning message is displayed:

```
[Warning] FCA_TRIVIAL_UNCOVERABLES_FOUND_WRITTEN: Trivially uncoverable
 line and/or condition goals were found, indicating possibility of bad
 code.
```

The details of these goals are written to this file: `vcst_rtdb/logs/trivialUncoverables_COV.rpt.`

From the vcf command line, you can use the engine attribute to filter and collect all these properties and view their results. Example,

```
vcf> get_props -filter "reason  == simplification"
```

If required, you can also write it in an el file as follows, which can then be loaded into Verdi GUI.

```
save_cov_exclusion -file trivials.el -targets [get_props -status
 uncoverable -filter {reason == simplification }]
```

Feedback

6.12 Viewing ProofTree (Orchestration) Details

You can view the details of a Proof tree in the Verdi GUI. The ProofTree view allows you to perform the following activities:

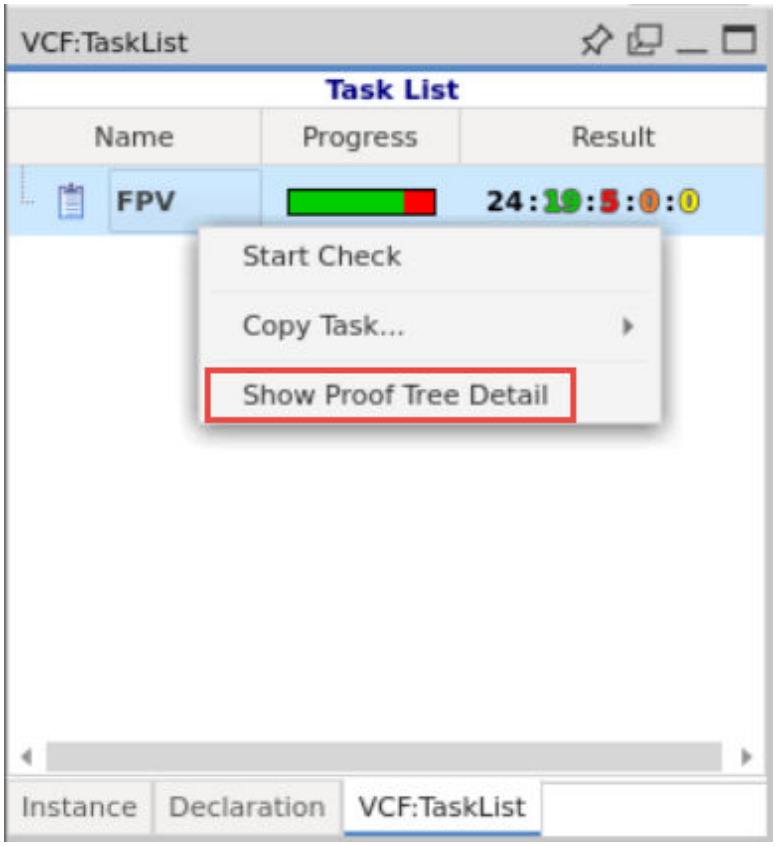
- Refresh
- Toggle
- Enable Highlight
- Clear All Filters

Note:

The ProofTree Detail view feature is available for all except FuSa, AEP, CC, and FCA.

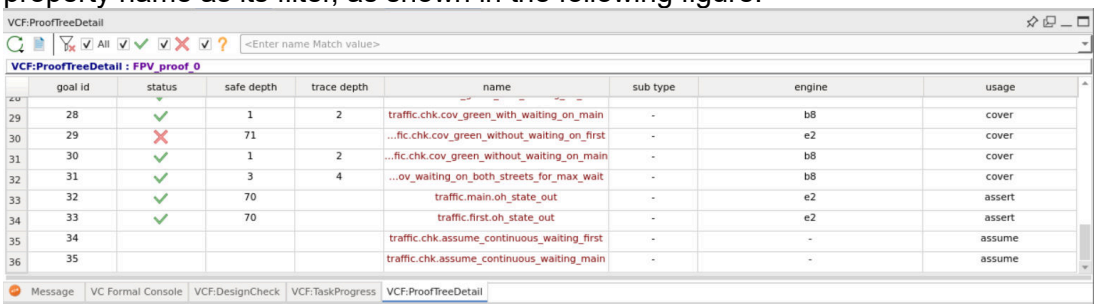
To filter the visualization of orchestration for a property:

1. Select a property in the GoalList for which you want to view the orchestration structure.
2. From the RMB menu of the Tasklist, click **Show Proof Tree Detail**.



Feedback

- The **VCF:ProofTreeDetail** dialog box appears for the selected property name as its filter, as shown in the following figure.



- You can customize the **VCF:ProofTreeDetail** and VCF:Constraints tables in the following ways:
  - Reorder and resize columns as required
  - Auto-saved-and-restored column settings

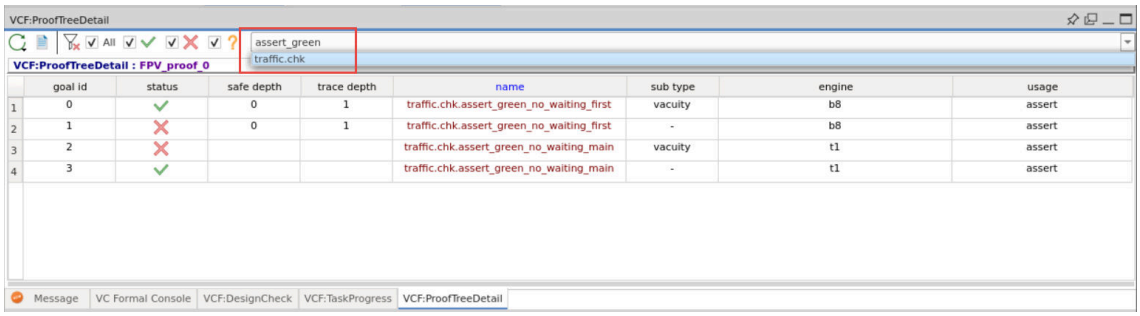
    Depth: Depth column shows the larger of `safe_depth` and `trace_depth` details. You can mouse over an entry in the depth column to see a ToolTip displaying both `safe_depth` and `trace_depth` details.

    Status: You can see status of GoalList and Constraints as follows:
  - Covered
  - Uncoverable
  - Non_vacuous
  - Vacuous
  - Proven
  - Falsified
  - Bounded

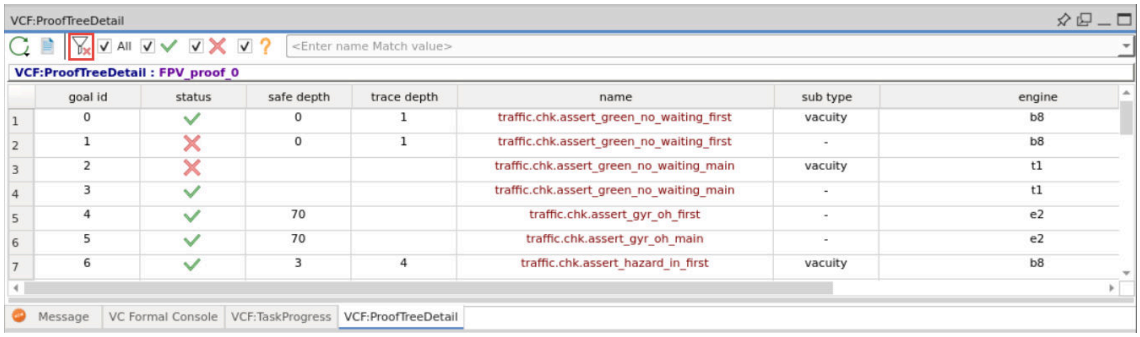
3. Click the arrow to view the filter history.

   A menu appears listing the history of filtered names.

Feedback



4. Click the (*Clear All Filters*) icon on VCF:Orchestration toolbar to clear all the current filter settings, including name filter and column filter by value. Note that this button does not remove the name filtered history. You can still view the name filter settings of the previous one in the history list.

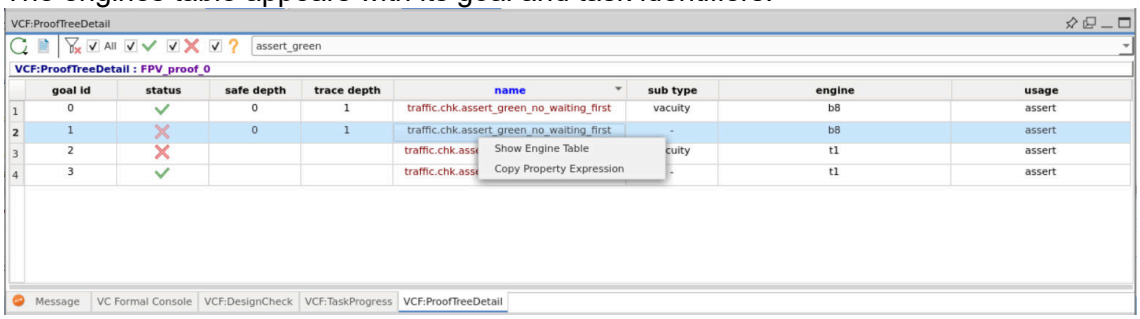


To view engine table for a property goal:

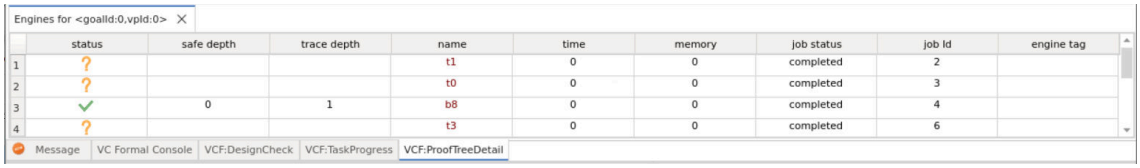
1. Select a signal for which you want to view the engine table.
2. Right-click on the goal and select **Show Engine Table**.

Feedback

The engines table appears with its goal and task identifiers.



The engine details appear as shown in the following figure:



6.13 Viewing Bit Level Model Statistics in VC Formal

Bit level model statistics gives an overall idea of the netlist size which are submitted to the backend engines. This statistics contains the number of gates, inputs, registers and constraints in the bit-level representation of the problem. The bit level model statistics are generated by default during a `check_fv` run,

For example,

[Info] BITLEVEL_MODEL_STATS: Generated model with 683 gates, 2 inputs, 87 registers, 0 initial constraints, 2 constraints.

You can disable the statistics report by setting the formal variable to false.

```
set_fml_var fml_show_orch_stat false
```

The message may appear multiple times with multiple values across messages as the orchestration generates multiple models.

The values may not be exactly same as the `report_fv`/`report_fv_complexity` output because of optimizations. In case of, the values shown in the message is much higher than the expected values, you can review the `report_fv_complexity` to identify the logic that can be abstracted.

Feedback

6.14 Viewing the Engine Summary in VC Formal GUI

VC Formal has many engines that run in parallel to solve a goal. You can use the *Engine Summary* graph to check the performance of each engine and using these details you can turn on or turn off an engine to get a better runtime performance.

To view Engine Summary:

1. In the TaskList, select a property.
2. In the GoalList toolbar, select *Engine Summary* from the drop-down list.

   The *Engine Summary* is displayed in the VC Static message view area.
3. Click to refresh the *Engine Summary* view.

Figure 38: Engine Summary View

The following is an example output of the engine reference from the `report_fml_engine` command.

```
report_fml_engines -engineStats
-------------------------------------------------------------------
      Name |     Runs | Proven | Falsified | Bounded | Solved% |
-------------------------------------------------------------------
        b1 |      992 |      0 |         0 |     382 |       0 |
        b2 |      992 |     77 |         2 |     913 |       7 |
        e1 |      992 |     22 |       283 |     686 |      30 |
-------------------------------------------------------------------
```

The bounded depths are not included in the Engine Summary view.

Feedback

6.15 Defining User Orchestration for VC Formal Runs

User-controlled orchestration enables the users to define a customized orchestration to run a set of specific engines on a set of properties with different engine options, with user-controlled resource limits, and with user defined scheduling of the jobs. We often find that the user-controlled orchestration is useful for customizing performance on specific test cases.

User controlled orchestration can be specified using the `fvorc` command.

Note:

For the `fvorc` command to take effect, the effort level must be set to user orchestration using the following formal variable: `set_fml_var fml_effort userorc`

The `fvorc` command is supported for following applications: *FPV, FRV, FTA, FUSA (only control run_mode), CC, COV, AEP.*

Syntax

```
fvorc [-property <list-of-names-or-ids-of-properties>] [-name <string>]
 [-time <string>] [-mem <string>] [-block][-engine_list] [-order <seq |
 par | par_blk_begin | par_blk_end>] [-model <def | optA>] [-engine_opts
 <string>] [-all] [-clear] [-delete] [-update] [<list-of-engine-tags>]
```

Where,

- `-property <list-of-names/ids-of-properties>`: Work on inconclusive properties in the list. If not specified, work on all inconclusive properties.
- `-name <string>`: Specify the name of the `fvorc` for future reference/debugging purpose. When this option is not specified, default name is assigned for each `fvorc` command in sequence. Example: fvorc1, fvorc2, …
- `-time <string>`: Timeout for each property and engine pair. Units in seconds (S), minutes (M) or hours (H) (e.g. 37M)
- `-mem <string>`: Specify the memory limit for each task. Units are GB or MB (for example, 12GB, 12000MB)
- `-block`: Blocks next fvorc till tasks from current fvorc is complete.
- `-engine_list`: Show the list of supported engines for fvorc.
- `-order <seq/par/par_blk_begin/par_blk_end>`: Order of execution of engines over properties. By default, it is set to par. The values “par_blk_begin/par_blk_end” are used for creating a par block where multiple fvorc commands can be specified to be run in parallel over properties.

- `-model <def | optA>`: Specify the model to run the engine. Possible values: def (default), optA (optimized). By default, it is set to *def*.
- `-engine_opts`: Specify the engine options for the engines in the list of engine tags. Engine options are described in Appendix G - Engine Controls for User Controlled Orchestration.
- `-all`: All the properties are handled in a single engine job.
- `-clear`: Clear all specified fvorcs in current user task.
- `[-delete]`: Deletes specified fvorc in current user task with '-name'
- `[-update]`: Updates specified fvorc and options in current user task with '-name'.
- `<list-of-engine-tags>`: Can be one of the following: `b1, b10, b11, b12, b13, b14, b15, b16, b17, b20, b3, b4, b5, b6, b7, b8, b9, e1, e10, e12, e13, e14, e15, e16, e17, e18, e19, e2, e23, e24, e25, e3, e4, e7, e8, e9, fc1, l1, l10, l11, l12, l13, l14, l16, l17, l18, l2, l22, l3, l4, l5, l6, l7, l8, l9, s1, s12, s7`

  For more information on the Engine controls, see section Appendix G - Engine Controls for User Controlled Orchestration.

Examples

fvorc usage:

```
> set_fml_var fml_effort userorc
> fvorc { b1 e1 } -order par
> fvorc { b4 e4 } -order par
> check_fv
```

Orchestration with round-robin loop:

The script below describes a round-robin orchestration that runs e1, b1 engines in parallel with $i*10 time limit in each iteration $i.

```
set_fml_var  fml_effort  userorc
for { set i 1} { $i < 10 } { incr i } {
            fvorc { e1 b1 } -time [expr $i*10]s -order par -block }
```

Engine b1 orchestration with different engine options:

The user orchestration below runs b1 engines with different initial depths, end depths, and time limits. This is a good technique for bug hunting especially when many workers are available.

```
set_fml_var fml_effort userorc
fvorc b1 -engine_opts "-initDepth 11 -endDepth 20 -timePerCycle 1000"
fvorc b1 -engine_opts "-initDepth 21 -endDepth 30 -timePerCycle 2000"
fvorc b1 -engine_opts "-initDepth 31 -endDepth 40 -timePerCycle 3000"
```

Feedback

Engine s7 bug-hunting orchestration:

User can specify a bug-hunting strategy which runs each worker with specific parameters and time setting.

```
set_fml_var fml_effort userorc
fvorc s7 -engine_opts "-seed 503411 -strategy 1 "
fvorc s7 -engine_opts "-seed 43453 -initDepthHeavy 5"
fvorc s7 -engine_opts "-seed 324234 -lookback 5"
fvorc s7 -engine_opts "-seed 56765 -lookahead 5"
fvorc s7 -engine_opts "-seed 5011 -helpers 1"
```

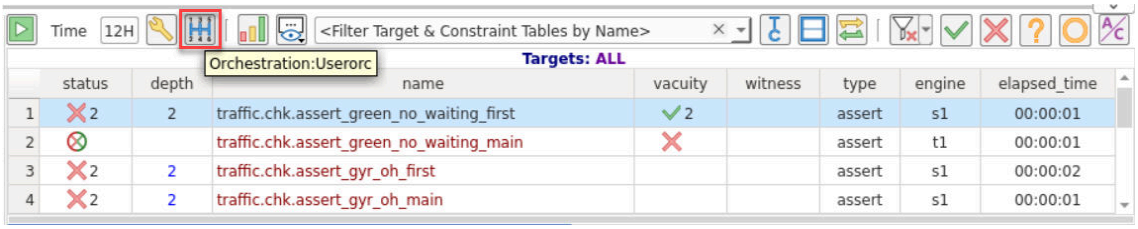
Engine b1 bug-hunting orchestration in a par block with multiple fvorc commands:

```
fvorc -order par_blk_begin
fvorc b1 -engine_opts { -initDepth 10 }
fvorc b1 -engine_opts { -initDepth 20 }
fvorc b1 -engine_opts { -initDepth 30 }
fvorc -order par_blk_end
```

6.15.1 Defining User Orchestration in GUI

To define user orchestration:

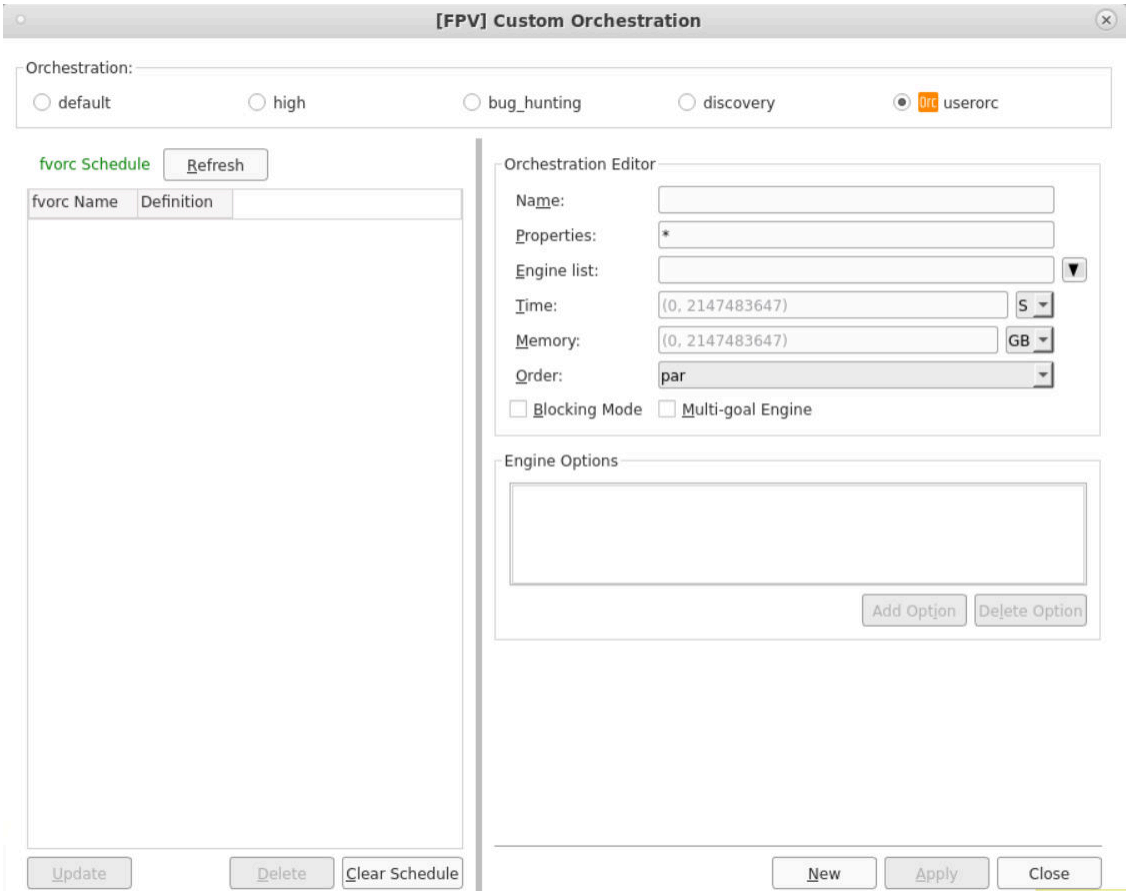
1. Select **User orc** from the **Effort Level** drop-down list.



Feedback

The **Custom Orchestration** dialog box appears.

Figure 39: Custom Orchestration Dialog Box

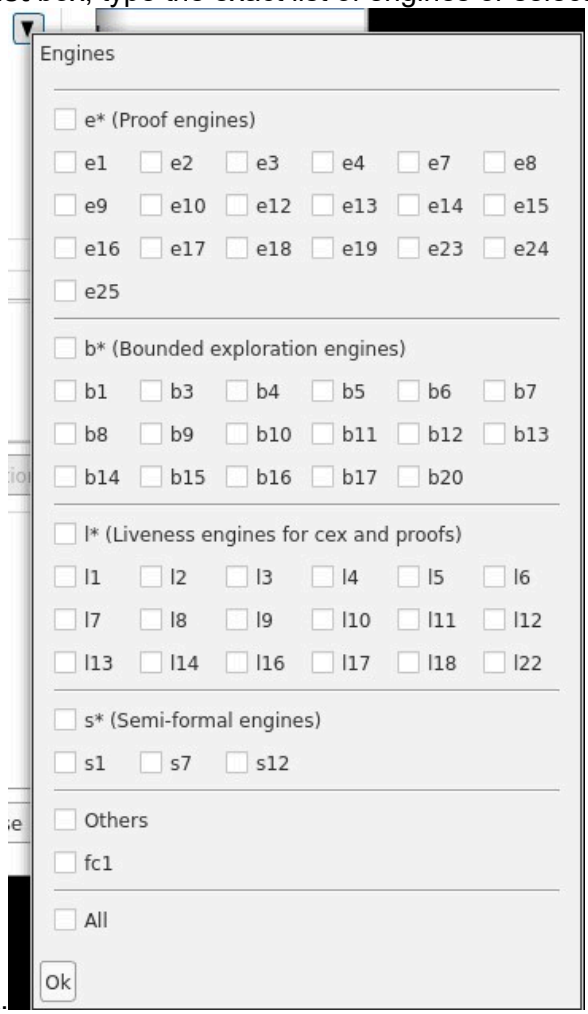


1. In the **Name** box, specify the name for the `fvorc` command.
2. In the **Properties** box, specify the properties to solve.

   Each individual property can be specified in the **Properties** box either by typing it or by copying it from the property table. The symbol asterisk (*) can be used to specify all properties or multiple properties with the same pattern.

Feedback

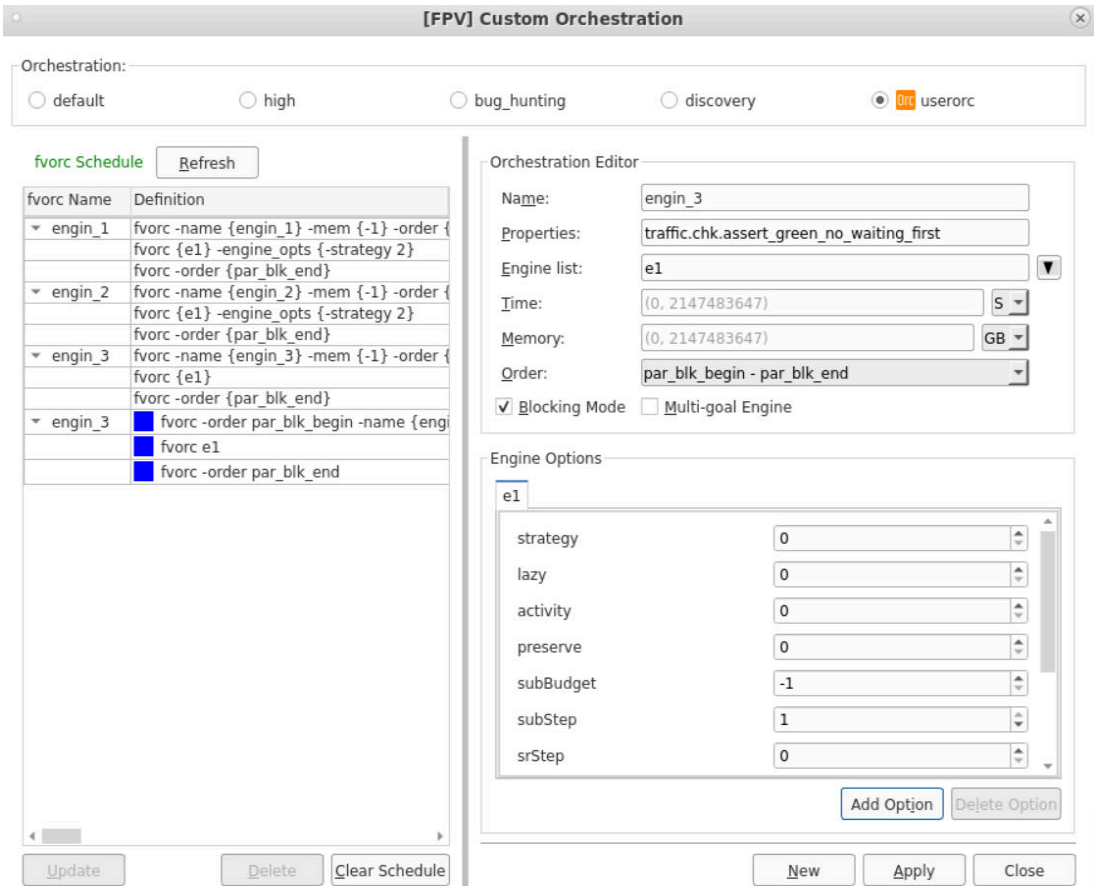
3. In the **Engine List** box, type the exact list of engines or select the engine from drop-down check box.



4. In the **Time** box, specify the runtime for each solver job.
5. In the **Memory** box, specify the memory in GB or MB.
6. From the **Order** drop-down list, specify if the engines must be executed sequentially or in parallel for the properties.
   - When set to **par or par_blk_begin - par_blk_end**, the first property p1 is run with respect to all engines available/mentioned in the command, then property p2 is run with respect to all engines available/mentioned in the command, and so on.
   - When set to **seq**, all properties p1, p2, p3 are run with respect to first engine available, and then p1 p2 p3 are run with respect to the second engine available, and so on.

Feedback

7. When the **Order** is **par_blk_begin – par_blk_end**, the **Add Option** button is enabled. This enables the user to specify engine options for the fvorc schedules inside of the par block that will be executed in parallel for each property.
   - Specify the engine options for each engine, and click **Add Engine** Option to record current engine options. You can specify multiple engine options and click **Add Option** to create multiple `par_blk_begin`/`par_blk_end` `fvorc` command. The created command shows in the schedule with a blue box to indicate that it is not real schedule command as shown in the following figure.



   - To delete engine options, select the engine and click **Delete Option**.
   - Click **Apply**. The fvorc commands with the blue box get finalized after clicking **Apply**.
8. Select the **Blocking Mode** option to stop the execution of the next schedule till all the tasks in the current `fvorc` are completed.
9. Select the **Multi-goal Engine** option to add all the properties in a single engine job.

Feedback

10. Click **Apply** to append the new `fvorc` schedule to the existing schedule. Or, click **New** to reset the Orchestration Editor for creating a new schedule.
11. Click **Update** in the left pane to update an existing schedule.

    Click **Delete** in the left pane to update an existing schedule.

    Click **Clear Schedule** in the left pane to remove all the properties queued up to be solved.
12. Click **Close** after you have set the required orchestration configurations.

6.15.2 Getting a Report of the Orchestration

Use the `report_fvorc` command to report of the fvorc commands that are defined in the current verification task.

The following is an example output of the `report_fvorc` command.

```
vcf> fvorc { b1 b6 } -engine_opts { -initDepth 10 } -time 10M -mem 16GB
vcf> fvorc { e1 e9 } -engine_opts { -strategy 1 } -model optA -time 5M -mem 8GB
vcf> report_fvorc
### Number of fvorc specified for current task: 2

FVORC # 1:
  Name           : _fvorc_1
  Engines        : b1 b6
  Property List  :
  Time           : 10M
  Mem            : 16GB
  Order          : par
  Model          : def
  Eng Options    :  -initDepth 10
  Block          : 0
  All            : 0

FVORC # 2:
  Name           : _fvorc_2
  Engines        : e1 e9
  Property List  :
  Time           : 5M
  Mem            : 8GB
  Order          : par
  Model          : optA
  Eng Options    :  -strategy 1
  Block          : 0
  All            : 0
```

6.16 Applying User-defined Orchestration on Top of Another Effort Level

You can apply user-defined orchestration on top of another effort level specified in *fml_effort* using *fvorc_config*. This helps in better convergence by combining user-defined orchestration and any of the effort level (*default* | *high* | *bug_hunting* | *discovery*).

When *fvorc_config* is enabled, the workers are assigned to the jobs for the *fvorc* and the remaining workers are assigned to the jobs for any of the specified effort level. When the jobs for the *fvorc* are finished, then the workers are assigned to the jobs for the other effort level.

Syntax

```
set_fml_var fml_effort [default | high | bug_hunting | discovery]
fvorc_config -include_effort_level [true | false]
```

By default, the *fvorc_config* is set to false.

Example

Consider you set the effort level to default and 12 workers available

```
set_fml_var fml_effort default
fvorc_config -include_effort_level true
fvorc { b1 b6 } -property { p1 p2 }
```

4 workers are reserved for `fvorc` and the remaining 8 workers are used for the default effort level.

6.17 Supporting the Bug Hunting Feature

Bug hunting enables you find deep bugs on hard inconclusive properties. When the effort level is set to bug hunting, VC Formal uses a combination of semi-formal and exhaustive engines to find bugs. You can also provide intermediate helper goals to guide the bug hunting mode in the following ways:

- Generating cover property
- Providing user-defined property order to improve bug hunting

6.17.1 The bug_hunting_config Command

Configure the `bug_hunting` effort level using the `bug_hunting_config` command.

Syntax

```
vcf> bug_hunting_config [-random <0|1>] [-saveDir dir]
```

Where,

- `[-random <0|1>]`: Specify this option to control randomness in bug hunting search engines. When you specify this option, the results of this option vary from run to run. By default, it is set to 0.
- `[-saveDir dir]`: Specify the directory location where the bug hunting configuration log files are saved. The logs are written by s7 engine only. By default, the log files are not saved.

6.17.2 The report_fml_bug_hunting command

When you use the `-saveDir` switch while configuring bug hunting, the `bug_hunting_config` logs are saved in the specified directory location. View the saved log details using the `report_fml_bug_hunting` command.

Syntax

```
vcf> report_fml_bug_hunting [-jobId <id>] [-status <0|1>] [-nlines <n>]
[-pos <head|tail|all>]
```

Where,

- `[-jobId <id>]`: Specify the job id for which you want to view the details. By default, the details of all the job ids are displayed.
- `[-status <0|1>]`: Specify this option to view the status for the specified job ids. By default, the status is set to 1.
- `[-nlines <n>]`: Specify the integer to print in the log file. By default, it is set to 0.
- `[-pos <head|tail|all>]`: Specify this option to print details in the log file. By default, it is set to tail.
  - `head`: Prints the first n lines in the log report.
  - `tail`: Prints the last n lines in the log report.
  - `all`: Prints the entire report in the log file.

6.17.3 Providing User-Defined Property Order to Improve Bug Hunting

When the effort level is set to bug hunting, VC Formal finds deep bugs in the design. In this mode, VC Formal generates, uses, and prioritizes helper goals to improve the

search. However, due to large state space, the bug hunting may not succeed within a given resource limit.

To overcome the limitation, the `fvorder` command enables you to specify partial search order of the cover and assert properties. The bug hunting engine uses the information provided in the `fvorder` command to prioritize the witness states corresponding to the specified helper properties.

The fvorder Command

Use the `fvorder` command to specify a partial order on properties or on a collection of properties. You can also use this command multiple times to provide information incrementally. The `fvorder` command allows you to specify the ordering of properties to be used in the backend search engines.

```
vcf> fvorder [-clear]
 [-print]<list-of-names-ids-or-collections-of-properties>
```

Where,

- `[-clear]`: Removes all the partial orders specified for the current task.
- `[-print]`: Prints the integer number of the property corresponding to the `fvorder` commands, including the current command.
- `<list-of-names-ids-or-collections-of-properties>`: Lists the collection of valid properties.

Examples

The following examples show the various use cases of `fvorder`.

```
vcf> fvorder {C1 C2}
vcf> fvorder {C2 C3}
vcf> fvorder {B1 B2}
vcf> fvorder {C1 C3} -print
=====Current Order=======
1: C1
1: B1
2: C2
2: C3
2: B2
```

Limitations

- The user-defined property order is supported only for bug hunting engine.
- Only cover and assert properties can be specified in the ordered list.
- Cyclic order is not permitted.
- The user-specified partial order is honored, but not always.

Feedback

6.18 Generating Cover Properties for Bug Hunting

You can provide cover properties to guide the bug hunting mode. VC Formal enables you to automatically generate effective cover properties from the following:

- Signal value range under a condition (using The gen_covers_for_range command)
- COI counter values (using The gen_covers_for_coi_counters command)
- Coverage goals from COV app (using The gen_covers_from_cov_targets command)

6.18.1 The gen_covers_for_range command

Use the `gen_covers_for_range` command to generate cover properties for a specified signal range under a given condition.

Syntax

```
vcf> gen_covers_for_range -signal <sigName> [-name <covName>] [-condition
<covName>] -hi <integer> -lo <integer> [-increment <integer>]
```

Where,

- `-signal <sigName>`: Specify the signal name for which you want to generate the cover properties for a range.
- `[-name <covName>]`: Specify the prefix to be added for the generated cover properties.
- `[-condition <covName>]`: Specify the condition to be used for the cover property.
- `-hi <integer>`: Specify the maximum value of the range for the generated cover property.
- `-lo <integer>`: Specify the minimum value of the range for the generated cover property.
- `[-increment <integer>]`: Specify the amount by which the signal value should increase in the successively generated cover properties. Default value is 1.

Examples

Run the command after loading the design:

```
  set_fml_appmode FPV
  set design traffic
  read_file -top $design -format sverilog -sva -vcs
{-f ../design/filelist ../sva/traffic.sva ../sva/bind_traffic.sva}
  ..
```

Feedback

```
 ..
 gen_covers_for_range <>
```

To generate cover for counter "traffic.first.timer_sub.timer_d[3:0]" with values incremented from 0 to 4. Properties will be named script_prop_*:

```
 gen_covers_for_range -signal "traffic.first.timer_sub.timer_d[3:0]" -hi
5 -lo 0
```

To generate cover for counter "traffic.first.timer_sub.timer_d[3:0]" on specific condition. Values are incremented by 2 from 0 and maximum is 4:

```
 gen_covers_for_range -signal "traffic.first.timer_sub.timer_d[3:0]"
-name first_timer_range -hi 5 -lo 0 -increment 2 -condition
"traffic.first.timer_sub.state_out_d == 4"
```

6.18.2 The gen_covers_for_coi_counters command

Use the `gen_covers_for_coi_counters` command to generate cover properties based on the counters in the COI for specified properties.

Syntax

```
vcf> gen_covers_for_coi_counters [-property <propList>] [-name <covName>]
[-condition <covName>] [-maxWidth <integer>] [-increment <integer>]
```

Where,

- `[-property <propList>]`: Specify a property list to generate cover properties based on the counters.
- `[-name <covName>]`: Specify the cover name for the cover properties.
- `[-condition <covName>]`: Specify the condition to use for the cover property.
- `[-maxWidth <integer>]`: Specify the maximum width of counters that should be considered for the cover property. If this option is not specified, a default width of 8 is considered.
- `[-increment <integer>]`: Specify the value by which the cycle must increase in the successive cycles while generating covers in range. Default value is 1.

Examples

Run the command after loading the design:

```
 set_fml_appmode FPV
 set design traffic
 read_file -top $design -format sverilog -sva -vcs
{-f ../design/filelist ../sva/traffic.sva ../sva/bind_traffic.sva}
 ..
```

```
  ..
  gen_covers_for_coi_counters <>
```

To generate cover for counters in the COI of traffic.chk.assert_honor_waiting_main & traffic.chk.assert_honor_waiting_first properties on specific condition and limiting generation of cover properties to counters having maximum width of 10 bits. Values of counters used for coverage will be incremented by 2 (0,2,4...). Properties will have name assert_honor_waiting_main_cntr_mul_*:

```
  gen_covers_for_coi_counters -property
{traffic.chk.assert_honor_waiting_main
traffic.chk.assert_honor_waiting_first} -name
assert_honor_waiting_main_cntr_mul -condition
"traffic.main.timer_sub.state_out_d == 4" -increment 2 -maxWidth 10
```

6.18.3 The gen_covers_from_cov_targets command

Use the `gen_covers_from_cov_targets` command to generate cover properties based on the coverage targets in COV App.

Syntax

```
vcf> gen_covers_from_cov_targets [-property <propList>] [-match
<pattern>]
```

Where,

- `[-property <propList>]`: Specify the list of properties to generate cover properties based on the coverage targets.
- `[-match <pattern>]`: Specify the pattern to be matched in the property list.

Examples

Run the command after loading the design (enable coverage instrumentation):

```
  set_fml_appmode FPV
  set design traffic
  read_file -top $design -format sverilog -sva  -cov line -vcs
{-f ../design/filelist ../sva/traffic.sva ../sva/bind_traffic.sva}
  ..
  ..
  gen_covers_from_cov_targets <>
```

To copy all coverage goals from COV app to current task:

```
  gen_covers_from_cov_targets
```

To copy all coverage goals for an assertion to current task:

```
  gen_covers_from_cov_targets -property
traffic.chk.assert_honor_waiting_main
```

Feedback

To generate coverage properties matching first or toggle in COV property name:

```
 gen_covers_from_cov_targets -property
{traffic.chk.assert_honor_waiting_main
traffic.chk.assert_honor_waiting_first} -match {*first* *toggle*}
```

To generate coverage property matching *first*toggle*:

```
 gen_covers_from_cov_targets -property
{traffic.chk.assert_honor_waiting_main
traffic.chk.assert_honor_waiting_first} -match {*first*toggle*}
```

7

Debugging Results of Formal Verification

After obtaining the results of the `check_fv` run using the `report_fv` command, you must debug the results for all the failing assertions.

This section contains the following topics:

- Property Status Description
- Viewing Results of check_fv Runs
- Generating a report_fv Output in XML Format
- Debugging VC Formal Results in VC Formal GUI
- Support for Unified Debug in Formal
- Using VC Formal Console in GUI
- Using OneSearch for Design Information
- Debugging Conflicting Constraints
- Identifying Reduced Constraints for Proofs
- Exporting FSDB for Property Traces
- Saving and Replaying Traces
- Evaluating External FSDB
- Signal Groups for Debugging

7.1 Property Status Description

The `report_fv` command and the **GoalList** table in the VC Formal GUI provide the status and results for various verification goals, including:

- SVA properties specified in RTL or specified within the tool through the run script/shell.
- SVA asserts, SVA covers, script asserts, script covers, AEP, extracted coverage for line-cond, SEQ generated miters, CC goals from csv and so on.
- Assumptions or Constraints such as SVA assume, Script assume.

Feedback

The output of the `report_fv` command contains the following sections:

- *Assert*: Properties (including CC connections) used as assertions in the current task.
- *Assume*: Properties used as constraints.
- *Cover*: Properties (including extracted ones) used as cover properties.

Each section on the `report_fv` output provide the following status fields for each the goals:

- *Status*: Primary status
- *Depth*: Progression (Bounded or Falsification) Depth
- *Vacuity*: Vacuity Sub-Goal Primary Status
- *Witness*: Witness Sub-goal Primary Status

Goals may have vacuity and witness sub-goals, so multiple run status fields may be present for each goal. These fields collectively reflect the primary status, progression, and sub-goal statuses.

Note:

- Constraints properties do not have run status fields.
- Vacuity checks are not supported for cover properties in VC Formal.

7.1.1 Property Status

Each goal has a primary status that reflects its current or known state. This field shows the initial, intermediate, and terminal run statuses. The specific status values depend on the application, goal type, and sub-goal statuses such as vacuity.

The status of a property can be influenced by the result of its witness check. A witness property (or witness check) is used to confirm that there exists at least one scenario where the main property (assertion) does not fail, that is, it is possible for the property to pass. If the witness check is uncoverable, it means there is no scenario where the property can pass—this could indicate issues such as tautology, inconsistencies, or setup problems with the property or the environment.

The primary status does not reflect the relative progression. Possible values include:

- **Initial Status**

  NOTRUN: The goal has not started running and has not been attempted by any engine. This is the starting status for all enabled goals and sub-goals.

- **Intermediate Status**

Feedback

CHECKING: The goal is currently being processed by a formal engine. This status is shown until a terminal status is reached, and progression is tracked in the depth field.

- **Terminal Status Values**

  The terminal status reflects the final status for a goal. It is dependent on the application and the type of goal.

Table 9: Terminal Status Values

Value	Explanation	Application
FALSIFIED	Counter example found for the goal	FPV, SEQ, FRV, and AEP (FSM livelock/deadlock checks)
PROVEN	Goal has been proven exhaustively	FPV, SEQ, FRV, and AEP (FSM livelock/deadlock checks)
VACUOUS	Goal has been proven vacuously. Vacuity Status for such cases would also show vacuous status.	FPV, CC, FRV, and AEP (FSM livelock/deadlock checks)
COVERED	Trace has been found for the coverage goal	FPV, COV, FRV, and AEP (FSM livelock/deadlock checks)
UNCOVERABLE	No trace exists that reaches the coverage goal	FPV, COV, FRV, and AEP (FSM livelock/deadlock checks)
CONNECTED	Connection/Path exists between design nodes	CC
UNCONNECTED	No connection/Path found between design nodes	CC
INCONCLUSIVE	Goal wasn't solved because resource limit was reached	All applications

7.1.2 Vacuity Goal Status

The vacuity field shows the main status of a vacuity goal. A vacuity goal, which checks whether the antecedent of an assertion or constraint can ever be true, is created when vacuity checks are enabled. The vacuity status is separate from the main status of the assertion, but it does affect the final result. Specifically, an assertion’s final status will be PROVEN or VACUOUS depending on the outcome of its vacuity check.

Example

Feedback

Suppose you have the following property:

```
property p;
  @(posedge clk) (a == 1) |-> (b == 1);
endproperty
```

If, due to constraints or design logic, the condition `(a == 1)` can never be true, the vacuity check will detect this. The vacuity field for this property will show VACUOUS, indicating that the property was proven only because the antecedent never occurred. As a result, the assertion's main status will also be set to VACUOUS, not PROVEN.

Enabling/Disabling Vacuous Goals

In VC Formal, vacuity goals are enabled by default . You can control the enabling or disabling of vacuity goals using the formal variable `fml_vacuity_on`. This can be set on a per-task basis using the `set_fml_var` command.

To disable vacuity goals:

```
set_fml_var fml_vacuity_on false
```

To enable vacuity goals (default):

```
set_fml_var fml_vacuity_on true
```

You can check the current value with: `get_fml_var fml_vacuity_on`

These settings apply to the current verification task unless you use the `-global` option to apply them to all tasks.

Note:

Since there is no separate Usage Field for Vacuity, the Vacuity Field can reflect both usage and run status.

Vacuity Goals Statuses

Table 10: Vacuity Fields Values

Value	Explanation
VACUOUS	Vacuity check failed. No witness possible. Goal's primary status would show terminal status as VACUOUS
NONVACUOUS	Vacuity check was successful
INCONCLUSIVE	Vacuity check could not complete as the resource limit was reached or the run was stopped by user

Feedback

7.1.3 Witness Goal Status

The witness field shows the result of the witness check for a property, if a witness check is applicable. The status of the witness depends on the outcome of the vacuity check: if the vacuity check result is VACUOUS, then a witness is not generated.

Example

Suppose you have an assertion property in your design. After running formal verification, the vacuity field for this property shows "VACUOUS," indicating that the antecedent condition can never be true. In this case, the witness field will be empty, because a witness trace (a scenario where the property holds) cannot exist if the property is vacuously true.

Enabling/Disabling Witness Goals

By default, witness trace generation is disabled. To enable witness generation and checks for all properties, set the following formal variable before running `check_fv`:

```
set_fml_var fml_witness_on true
```

This enables witness trace generation for all properties specified by `fvassert`.

Witness Goal Statuses

Table 11: Witness Goal Statuses

Value	Explanation
COVERED	Witness check successful. Trace available
UNCOVERABLE	No witness exists.
INCONCLUSIVE	Vacuity check could not complete as the resource limit was reached or the run was stopped by user

7.1.3.1 Vacuity Definition for Uncoverable Witness

VC Formal provides a more comprehensive approach to identifying vacuous properties during formal verification to quickly identify over-constrained scenarios that may otherwise be overlooked, ensuring a more robust verification process.

A property is considered vacuous if either of the following conditions is met:

- No trace satisfies the antecedent of the property.
- The property is proven, but the witness is uncoverable.

Feedback

If the vacuity or witness goals are inconclusive or in-progress, the reporting will remain unchanged. Likewise, if the property goal is falsified, inconclusive, not run, or checking, and the witness goal is uncoverable, the reporting will not change.

Enabling/Disabling Vacuity Detection for Uncoverable Witness

By default, the properties with uncoverable witnesses are marked as vacuous.

To disable this feature, use the `fv_config -enable_vacuous_witness false` variable before running formal verification checks with `check_fv`.

Note:

If `fml_witness_on` is set to `false`, the witness checks are not performed, and the witness field in the property status and `report_fv` output will remain blank. Change this variable to true and rerun the analysis to update the property status and reporting accordingly.

Example

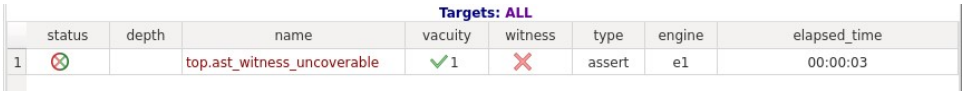
Consider the following properties:

```
assert property (@(posedge clk) a |-> ##4 b);
assume property (@(posedge clk) a ##3 1 |-> 0);
```

In this setup, the assertion requires that whenever signal `a` is true, signal `b` must be true exactly 4 cycles later. However, the assumption enforces that any trace where `a` is true will dead-end (i.e., become impossible to continue) after 3 cycles. This is because the assumption prevents the system from progressing beyond 3 cycles after `a` is true.

As a result, any counter-example (cex) or witness trace for the assertion would need to be at least 4 cycles long to demonstrate the required behavior. However, due to the assumption, it is impossible to generate such a trace—the system cannot reach the 4th cycle after `a` is true.

The witness is reported as UNCOVERABLE because it is not possible to generate a valid trace that demonstrates the property being exercised (that is, a scenario where `a` is true and `b` is checked 4 cycles later). By default, VC Formal marks the properties with uncoverable witnesses as vacuous.



The vacuous status will not be limited to properties with implication operators. For example:

```
assert property (@(posedge clk) ##4 b);
assume property (@(posedge clk) a ##3 1 |-> 0);
```

Feedback

If a property is proven but lacks a witness (uncoverable), it will be marked as vacuous, even if the antecedent is not satisfied in the traditional sense. For example:

```
assert property (@(posedge clk) a |-> ##4 b);
assume property (@(posedge clk) a ##3 1 |-> 0);
```

To disable this feature and make the properties proven for uncoverable witness, use the `fv_config -enable_vacuous_witness false` variable before running formal verification checks with `check_fv`.

Targets: ALL

	status	depth	name	vacuity	witness	type	engine
1	✓		top.ast_witness_uncoverable	✓1	✗	assert	e1

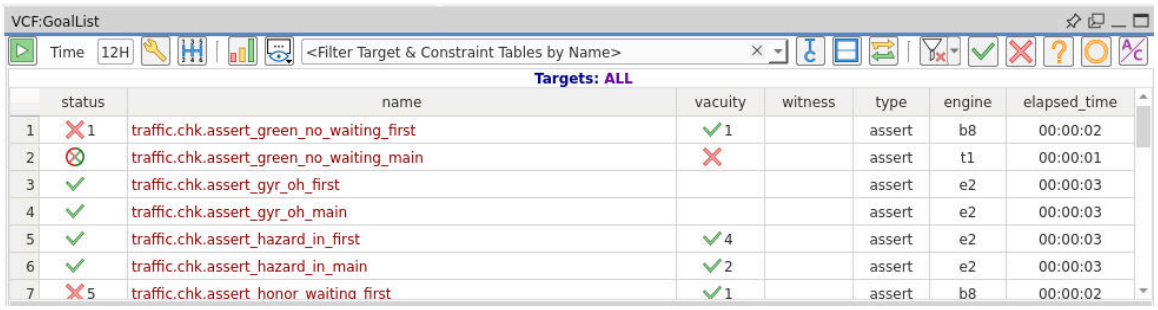
7.1.4 Conditional Proven Status

Apart from the statuses mentioned in Table 9, the *cond_proven* status, when enabled, is reported to represent following special cases.

- When the main goal is proven, but vacuity is not yet determined or disabled.
- When the main goal is proven and vacuity is not vacuous, but the witness goal is uncoverable, not yet determined, or disabled.

Conditional Proven status is shown as ✓ in Status field of *VCF:GoalList*, and it is treated under *Inconclusive* category in `report_fv` and *VCF:TaskList* task bar.

Figure 40: Conditional Proven Status



	status	name	vacuity	witness	type	engine	elapsed_time
1	✗1	traffic.chk.assert_green_no_waiting_first	✓1		assert	b8	00:00:02
2	⊗	traffic.chk.assert_green_no_waiting_main	✗		assert	t1	00:00:01
3	✓	traffic.chk.assert_gyr_oh_first			assert	e2	00:00:03
4	✓	traffic.chk.assert_gyr_oh_main			assert	e2	00:00:03
5	✓	traffic.chk.assert_hazard_in_first	✓4		assert	e2	00:00:03
6	✓	traffic.chk.assert_hazard_in_main	✓2		assert	e2	00:00:03
7	✗5	traffic.chk.assert_honor_waiting_first	✓1		assert	b8	00:00:02

Enabling/Disabling Conditional Proven Status

Set the following formal variable to enable the *cond_proven* status:

```
set_fml_var fml_cond_proven_status_on <off Vacuity Witness>
```

Feedback

Where,

- `off`: cond_proven status is turned off.
- `Vacuity`: cond_proven status is determined by the vacuity goal. Properties with a vacuity goal are not considered fully proven until the vacuity is non_vacuous.
- `Witness`: cond_proven status is determined by the witness goal. Properties with a witness goal are not considered fully proven until the witness is covered.

By default, the `fml_cond_proven_status_on` formal variable is set to `off`.

7.2 Viewing Results of check_fv Runs

To view the results of the `check_fv` run, use the `report_fv` command.

Syntax

```
vcf> report_fv -help
Usage: report_fv    # Reports formal information
        [-class <list-of-class-attributes>]
                               (property class selection:
                                Values: aep, connection, coverage, fusa,
                                no_class, register, script, security,
                                seq, source, xprop)
        [-type <list-of-type-attributes>]
                               (property type selection:
                                Values: arith_oflow, assert, assume,
                                boundconstraint, bounds_check, branch,
                                condition, conflict_driver,
                                constconstraint, control, cover,
                                covergroup, detect, envconstraint,
                                envstableconstraint, exclude,
                                fault_check, floating_bus, fsm_deadlock,
                                fsm_livelock, fsm_state, fsm_transition,
                                fsm_unionlock, full_case, injectx,
                                injectx_oba, injectx_reset,
                                injectx_undef, injectx_xassign,
                                internal, line, mark, multi_driver,
                                no_type, nox, nox_reset, observe,
                                one_hot, parallel_case, resetconstraint,
                                restrict, seq_gcc, set_reset,
                                stableconstraint, structural, toggle,
                                , x_assign)
        [-usage <list-of-usage-attributes>]
                               (property type selection:
                                Values: assert, assume, cover)
        [-status <list-of-status-attributes>]
                               (property status selection:
                                Values: "", checking, cond_proven,
                                connected, covered, detected, falsified,
```

Feedback

```
                                inconclusive, non_activated,
                                non_detected, non_formalcore,
                                non_propagated, not_run, proven,
                                unconnected, uncoverable, vacuous)
       [-vacuity <list-of-vacuity-attributes>]
                               (property vacuity selection:
                                Values: "", checking, cond_non_vacuous,
                                inconclusive, non_vacuous, not_run,
                                vacuous)
       [-filter <filterExpr>] (Filter expression to apply)
       [-regexp]              (Use regular-expression instead of glob
filtering)
       [-exact]               (No pattern matching performed)
       [-xml]                 (Outputs in a XML format)
       [-no_summary]          (Suppresses summary information)
       [-verbose]             (Outputs detailed report)
       [-list]                (Outputs one-line report per check)
       [-min_proof_depth]     (Returns the minimum proof depth)
       [-format <tcl|sva>]    ((FPV) Specialized Output format:
                               Values: sva, tcl)
       [-module_name <modulename>] (module name)
       [-no_line_intent]      ((COV) Filters out coverage properties
which are unintentionally covered or intentionally uncoverable)
       [-line_intent]         ((COV) Only reports coverage properties
which are unintentionally covered or intentionally uncoverable)
       [-formatCC <csv|tcl|path>]
                               ((CC) Output format for Connectivity
Checks:
                               Values: csv, path, tcl)
       [-report_type <setup|warning>]
                               ((CC) Report type:
                               Values: setup, warning)
       [-reached_enable]      ((CC) Shows connectivity checks whose
enable condition can be true (non vacuous check). Needs fml_vacuity_on
to be set)
       [-unreachable_enable]  ((CC) Shows connectivity checks whose
enable condition is never true (vacuous check). Needs fml_vacuity_on to
be set))
       [<list-of-names-ids-or-collections-of-properties>]
                               (List of property names, name patterns, or
property collections)
```

Example

- By default, a summary of the status of properties is printed using the `report_fv` command. The `report_fv` command supports live status update, and provides details of each goal from the Properties and each of the sub groups.

```
vcf>report_fv
Summary Results
   Property Summary: FPV
   -----------------
   > Assertion
```

Feedback

```
    - # found        : 16
    - # proven       : 12
    - # falsified    : 3
    - # vacuous      : 1
  > Vacuity
    - # found        : 10
    - # non_vacuous  : 9
    - # vacuous      : 1
  > Cover
    - # found        : 8
    - # covered      : 7
    - # uncoverable  : 1
  > Constraint
    - # found        : 3
```

- To get a more detailed report including proof engine, elapsed and convergence times, and safe depth information, use `report_fv -verbose`:

```
vcf>report_fv -verbose -no_summary  traffic.chk.assert_hazard_in_first
  Verbose Results
   Property Verbose List:
   ----------------------
   > Assertion
     # Assertion: 1
     > ID: [4] proven
      - name           : traffic.chk.assert_hazard_in_first
      - vacuity        : non_vacuous
      - witness        : covered
      - language       : SVA
      - type           : assert
      - usage          : assert
      - update_time    : 12:06:42 - May 17
      - elapsed_time   : 00:00:02
      - converge_time  : 00:00:00
      - location       : ../sva/traffic.sva:38
      - engine         : e2
      - safe_depth     : 90
      - vacuity_engine: b8
      - vacuity_safe_depth: 3
      - vacuity_depth : 4
      - witness_engine: b8
      - witness_safe_depth: 3
      - witness_depth : 4
```

- To get a one line status report of each property, use `report_fv -list -no_summary`:

```
vcf>report_fv -list -no_summary
List Results
 Property List:
 --------------
 > Assertion
   # Assertion: 16
   [   0] falsified   (depth=3) (non_vacuous)-
 tr.assert_green_no_waiting_first
```

Feedback

```
   [   1] vacuous               (vacuous)     - tr.assert_green_no_waiting_main
   [   2] proven                              - tr.assert_gyr_oh_first
   [   3] proven                              - tr.assert_gyr_oh_main
   [   4] proven                (non_vacuous)- tr.assert_hazard_in_first
   [   5] proven                (non_vacuous)- tr.assert_hazard_in_main
   [   6] falsified   (depth=5) (non_vacuous)- tr.assert_honor_waiting_first
   [   7] falsified   (depth=8) (non_vacuous)- tr.assert_honor_waiting_main
   [   8] proven                              - tr.assert_no_both_green
   [   9] proven                              - tr.assert_no_both_yellow
   [  10] proven                (non_vacuous)-
 tr.assert_signal_sequencing_on_first
   [  11] proven                (non_vacuous)-
 tr.assert_signal_sequencing_on_main
   [  12] proven                              - tr.assert_state_oh_first
   [  13] proven                              - tr.assert_state_oh_main
   [  24] proven                              - tr..oh_state_out
   [  25] proven                              - tr.t.oh_state_out
 > Cover
   # Cover: 8
   [  16] covered     (depth=1)               - tr.cov_both_red
   [  17] covered     (depth=28)              -
 tr.cov_green_first_for_one_cycle
   [  18] covered     (depth=3)               - tr.cov_green_main_for_one_cycle
   [  19] covered     (depth=5)               -
 tr.cov_green_with_waiting_on_first
   [  20] covered     (depth=2)               -
 tr.cov_green_with_waiting_on_main
   [  21] uncoverable                         -
 tr.cov_green_without_waiting_on_first
   [  22] covered     (depth=2)               -
 tr.cov_green_without_waiting_on_main
   [  23] covered     (depth=4)               -
 tr.cov_waiting_on_both_streets_for_max_wait
 > Constraint
   # Constraint: 3
   [  14] constrained           (non_vacuous)-
 tr.assume_continuous_waiting_first
   [  15] constrained           (non_vacuous)-
 tr.assume_continuous_waiting_main
   [  26] constrained                         - constant_26 - rst==0
```

- You can view the results of the properties by class, type, usage, or status of the properties. For example, to view results for assert properties, use `report_fv -usage assert`:

```
vcf>report_fv -usage assert
  Summary Results
   Property Summary: FPV
   -----------------
   > Assertion
     - # found        : 16
     - # proven       : 12
     - # falsified    : 3
     - # vacuous      : 1
   > Vacuity
     - # found        : 8
     - # non_vacuous  : 7
     - # vacuous      : 1
```

- To save results to a file, a standard file output `redirect` or `append` (unix command) can be used across the platform.:

```
vcf> redirect -file falsification_list.txt {report_fv}
vcf> report_fv -list -status proven > proven_list.txt
vcf> report_fv -list -status proven >> all_list.txt
vcf> report_fv -list -status falsified >> all_list.txt
```

7.3 Generating a report_fv Output in XML Format

VC Formal enables you to generate XML report for the `report_fv`commands. To generate XML report, you must use the new switch `-xml` with the respective commands.

In the XML output, there are four tags named <fv>, <cov>, <cc> and <seq>. All the information which are generated in the normal report are present under certain fields in each of these tags. The XML report for particular `report_fv` command is updated under its respective node only.

Example

```
vcf>report_fv -xml
```

Output

Consider that the `report_fv` command has the following output:

```
  Summary Results
   Property Summary:
   -----------------
   > Assertion
     - # found         : 6
     - # proven        : 4
     - # falsified     : 1
     - # vacuous       : 1
   > Vacuity
     - # found         : 6
     - # non_vacuous   : 5
     - # vacuous       : 1
   > Cover
     - # found         : 16
     - # covered       : 16

   > Constraint
     - # found         : 2
```

The corresponding xml report for the above results is as follows:

```
<?xml version="1.0"?>
<properties>
  <scope>counter</scope>
  <summary task_name="_FPV" task_app_name="FPV" task_count="1">
```

Feedback

```
  <fv>
    <num_active_goals>28</num_active_goals>
    <num_converged_goals>27</num_converged_goals>
    <num_scheduled_goals>1</num_scheduled_goals>
    <bounded_proof_depths min="0" max="0" change="0"/>
    <assertions>
      <found>6</found>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="falsified" count="1"/>
      <status value="proven" count="4"/>
      <status value="vacuous" count="1"/>
      <status value="inconclusive" count="0"/>
    </assertions>
    <vacuities>
      <found>6</found>
      <status value="not_run" count="0"/>
      <status value="vacuous" count="1"/>
      <status value="non_vacuous" count="5"/>
      <status value="checking" count="0"/>
      <status value="inconclusive" count="0"/>
    </vacuities>
    <witnesses>
      <found>0</found>
      <status value="" count="0"/>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="covered" count="0"/>
      <status value="uncoverable" count="0"/>
      <status value="inconclusive" count="0"/>
    </witnesses>
    <covers>
      <found>16</found>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="falsified" count="0"/>
      <status value="proven" count="0"/>
      <status value="inconclusive" count="0"/>
      <status value="uncoverable" count="0"/>
      <status value="covered" count="16"/>
    </covers>
    <constraints>
      <found>2</found>
    </constraints>

  </fv>
  <cc>
    <num_active_goals>0</num_active_goals>
    <num_converged_goals>0</num_converged_goals>
    <num_scheduled_goals>0</num_scheduled_goals>
    <connections>
      <found>0</found>
      <status value="not_run" count="0"/>
```

Feedback

```
      <status value="checking" count="0"/>
      <status value="vacuous" count="0"/>
      <status value="inconclusive" count="0"/>
      <status value="connected" count="0"/>
      <status value="unconnected" count="0"/>
    </connections>
    <enables>
      <found>0</found>
      <status value="not_run" count="0"/>
      <status value="vacuous" count="0"/>
      <status value="non_vacuous" count="0"/>
      <status value="checking" count="0"/>
      <status value="inconclusive" count="0"/>
    </enables>

  </cc>
  <cov>
    <num_active_goals>0</num_active_goals>
    <num_converged_goals>0</num_converged_goals>
    <num_scheduled_goals>0</num_scheduled_goals>
    <line>
      <found>0</found>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="falsified" count="0"/>
      <status value="proven" count="0"/>
      <status value="inconclusive" count="0"/>
      <status value="uncoverable" count="0"/>
      <status value="covered" count="0"/>
    </line>
    <condition>
      <found>0</found>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="falsified" count="0"/>
      <status value="proven" count="0"/>
      <status value="inconclusive" count="0"/>
      <status value="uncoverable" count="0"/>
      <status value="covered" count="0"/>
    </condition>
    <toggle>
      <found>0</found>
      <status value="not_run" count="0"/>
      <status value="checking" count="0"/>
      <status value="falsified" count="0"/>
      <status value="proven" count="0"/>
      <status value="inconclusive" count="0"/>
      <status value="uncoverable" count="0"/>
      <status value="covered" count="0"/>
    </toggle>

    <fsm_state>
      <found>0</found>
```

Feedback

```
        <status value="not_run" count="0"/>
        <status value="checking" count="0"/>
        <status value="falsified" count="0"/>
        <status value="proven" count="0"/>
        <status value="inconclusive" count="0"/>
        <status value="uncoverable" count="0"/>
        <status value="covered" count="0"/>
      </fsm_state>
      <fsm_transition>
        <found>0</found>
        <status value="not_run" count="0"/>
        <status value="checking" count="0"/>
        <status value="falsified" count="0"/>
        <status value="proven" count="0"/>
        <status value="inconclusive" count="0"/>
        <status value="uncoverable" count="0"/>
        <status value="covered" count="0"/>
      </fsm_transition>
      <fsm_livelock>
        <found>0</found>
        <status value="not_run" count="0"/>
        <status value="checking" count="0"/>
        <status value="falsified" count="0"/>
        <status value="proven" count="0"/>
        <status value="inconclusive" count="0"/>
        <status value="uncoverable" count="0"/>
        <status value="covered" count="0"/>
      </fsm_livelock>

      <waived>
        <found>0</found>
        <assert>0</assert>
        <assume>0</assume>
        <cover>0</cover>
      </waived>
    </cov>
    <seq>
      <num_active_goals>0</num_active_goals>
      <num_converged_goals>0</num_converged_goals>
      <num_scheduled_goals>0</num_scheduled_goals>
      <assertions>
        <found>0</found>
        <status value="not_run" count="0"/>
        <status value="checking" count="0"/>
        <status value="falsified" count="0"/>
        <status value="proven" count="0"/>
        <status value="vacuous" count="0"/>
        <status value="inconclusive" count="0"/>
      </assertions>

    </seq>
  </summary>
</properties>
```

Feedback

7.4 Debugging VC Formal Results in VC Formal GUI

The VC Formal GUI offers an integrated environment for analyzing and debugging formal properties and verification tasks. With seamless integration to Verdi, users can access advanced GUI features to efficiently debug formal properties and tasks.

The GUI layout typically includes:

- The Formal toolbar for project and setup management.
- The **GoalList** pane for property organization and filtering.
- The **Task Summary** table for an overview of verification results.
- Tooltips in the **TaskList** pane for quick status updates on each task.

Watch this video to learn about the three key components of the VC Formal GUI: the TaskList, the GoalList, and the VC Formal console.

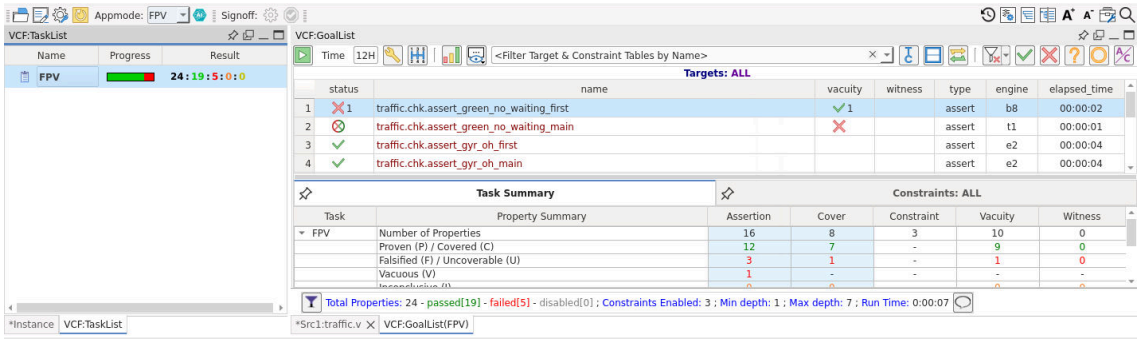
Watch this video to understand the design navigation and debug aids, and how to browse and search across design hierarchies, view signals, and find different scopes per module in VC Formal GUI.

The VC Formal GUI provides a layout as shown in Figure 41, which contains the following panes:

- VC Static and Formal toolbar (see section Using the VC Formal Toolbar for more details)
- GoalList (see section Managing and Analyzing Properties Using the GoalList for more details)

Feedback

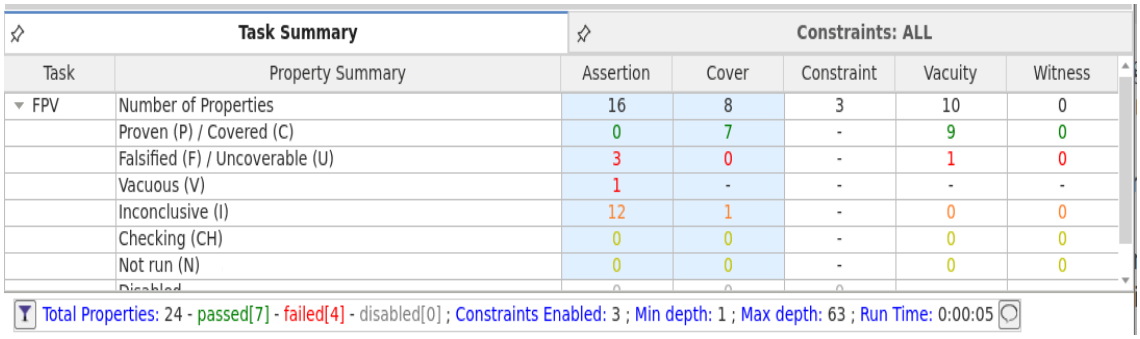
Figure 41: VC Formal GUI



- **Task Summary** table: This table as shown in Figure 42 provides the summary of the current task. You can pin the **Task Summary** table.

  The **Vacuous (V)** / **Falsified (F) / Uncoverable (U)** row is only for Assertions and Constraints. The actual cover property results are factored in the **Proven (P) / Covered (C)** & **Falsified (F) / Uncoverable (U)** rows.

Figure 42: VC Formal Task Summary



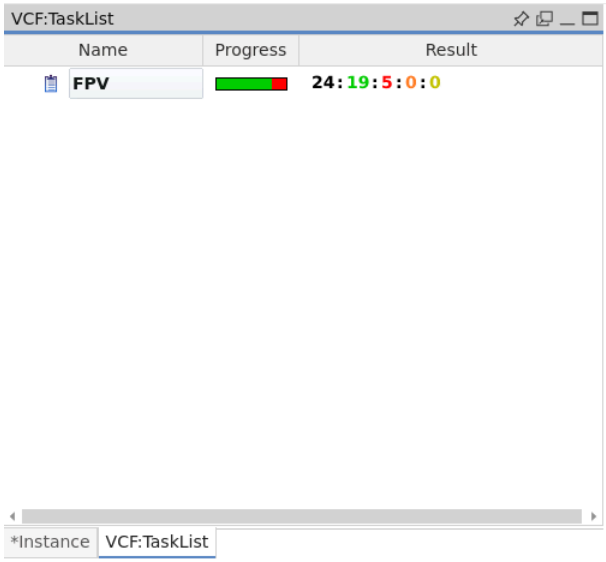
Note:

Status `cond_proven` is shown for assertions when `fml_cond_proven_status_on` set to “vacuity” or “witness” when those have

Feedback

no status for a proven property. You can hide the “Conditionally Proven” row by setting the `fml_cond_proven_status_on` to off.

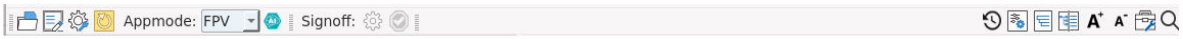
The same summary is also available as a tooltip in the **Task List** pane as shown in the following figure:



7.4.1 Using the VC Formal Toolbar

The VC Formal toolbar is as shown in Figure 43.

Figure 43: VC Formal Toolbar



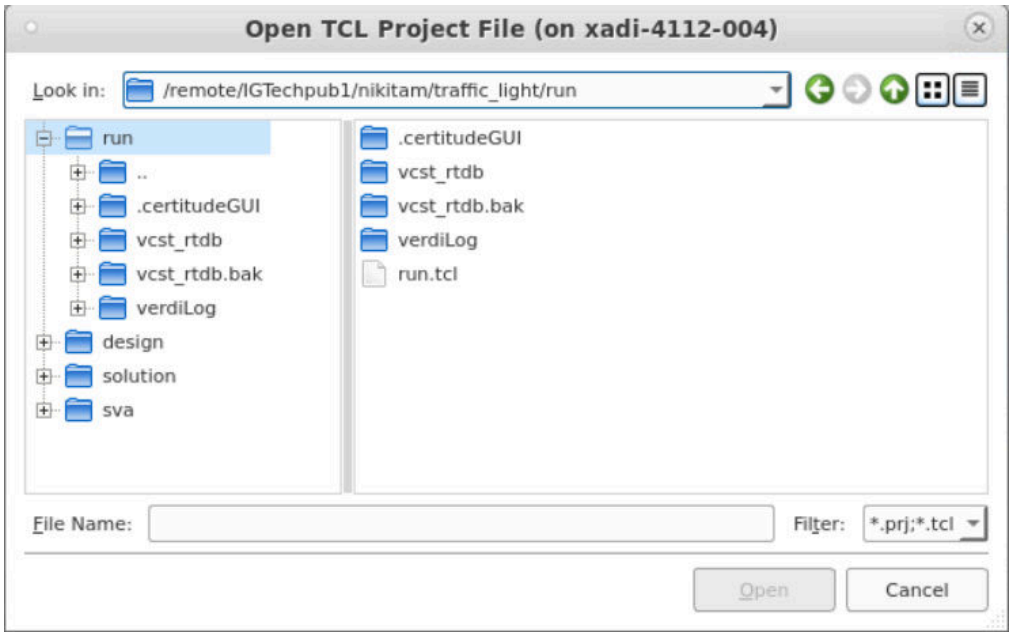
From the VC Static and Formal toolbar, you can perform the following tasks:

- Click to open a Tcl project file.

  The *Open TCL Project File* dialog box appears. You can open files having .prj/.tcl/*. extensions.

Feedback

Figure 44: Open TCL Project File Dialog Box



Select a project file from the current directory (default) or any other directory.

- Click to open the VC Formal Setup Wizard. See section VC Formal Setup Wizard for more details.
- From the drop-down list, select the required application mode.

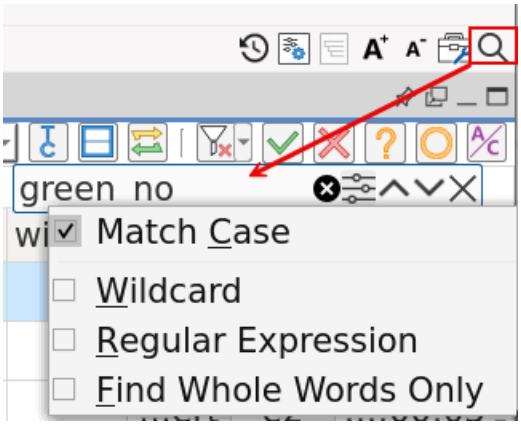
  By default, the FPV App mode is shown in the default task (FPV). If you specify the `-fmode` option or the `set_fml_appmode` command, the specified application mode is set to the current view with its own default task (for example, AEP, SEQ, CC, and COV). Only one application mode is seen at a time in the TaskList.

- Restart the VC Formal session using the icon.
- Click to open AI based GenFV application for formal verification.

  See section VC Formal GenFV® for more details on VC Formal GenFV.
- The other available features on the VC Formal toolbar for
  - : View the history of actions performed on the GUI.
  - : Open the VC Formal Settings dialog box.

- : View the signal list.
- : Increase/decrease the size of the fonts/GUI elements in the GUI.
- : Perform a global search across the VC Formal GUI.

  The search provides the following advanced search operations:



7.4.2 Managing and Analyzing Properties Using the GoalList

The GoalList in VC Formal offers a dynamic, interactive interface for monitoring, sorting, and analyzing the status of formal verification properties during a run. This feature gives you real-time visibility into property progress, supports efficient debugging, and enables flexible result management. Using the GoalList, you can streamline your verification workflow, quickly identify issues, and improve property convergence.

The VC Formal GoalList provides the following features:

- View live updates of property status, depth, engine, vacuity, and witness columns during formal runs.
- Sort and filter properties to focus on specific verification outcomes.
- Access advanced debugging tools, including trace viewing and source code navigation.
- Customize the display and organization of property data for improved clarity.
- Save session-specific settings for consistent user experience across tasks and applications.
- Viewing targets and constraints
- Viewing progress and complexity reports in the Verdi shell and message view

Feedback

Accessing the VC Formal GoalList

To access the VC Formal GoalList:

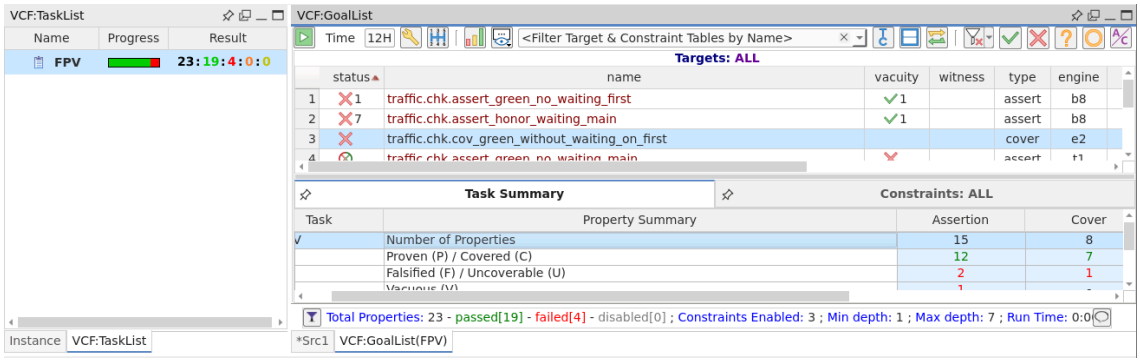
- Start VC Formal in GUI mode:

  ```
  vcf -gui
  ```

- Run a Formal verification tasks
- Execute a formal run to populate the GoalList with property data.

  ```
  check_fv
  ```

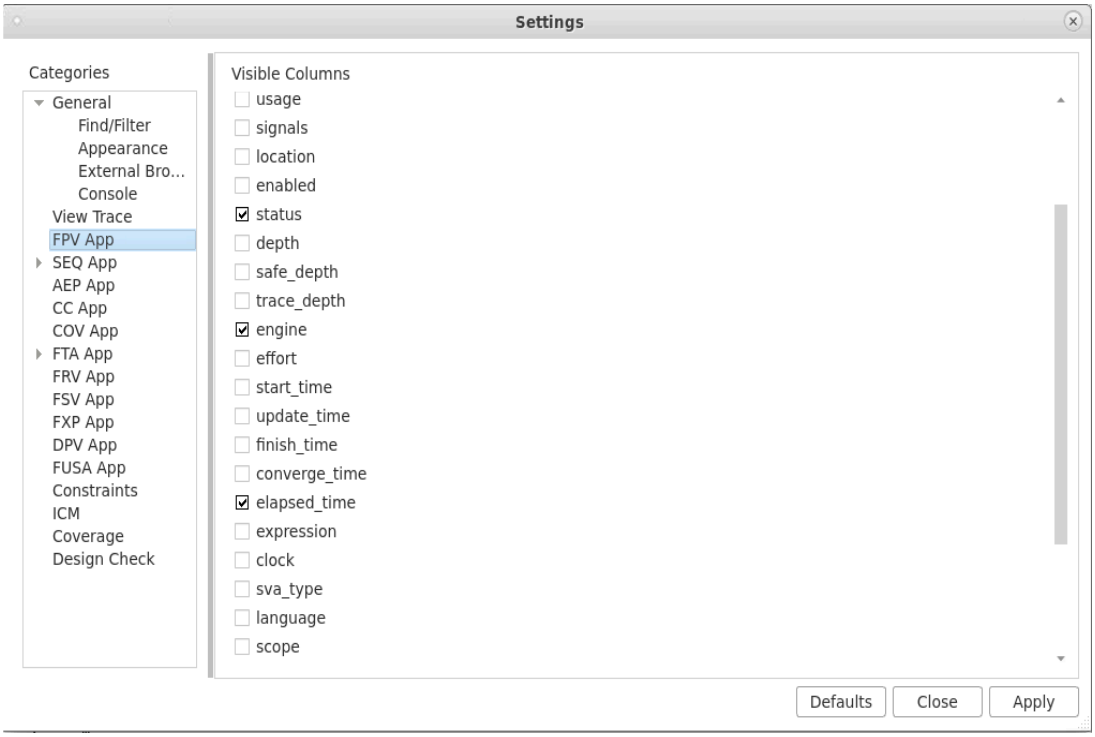
  The GoalList and TaskList update live during the run.



Configuring the GoalList Display

1. Customize the visible columns
   - Click the settings icon in the VC Formal toolbar to open the Settings dialog.
   - Select which columns to display based on your application mode.

Feedback



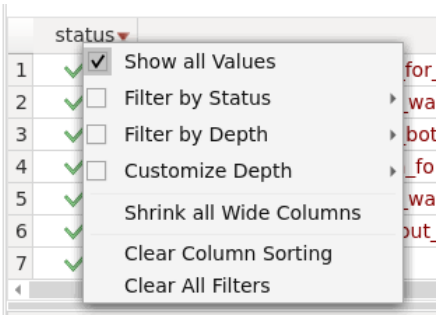
Columns automatically resize to fit the table width.

2. Hide or pin the Constraints table
   - Click the icon to toggle the **Constraints** table visibility.
   - You can also pin the **Constraints** table for persistent access.
3. Adjust Column Order
   - Drag and drop columns to rearrange their order.
   - Changes are saved in the session and applied to future tasks.

Feedback

Sorting and Filtering Properties

1. You can sort the columns using the column headers (for example, **Status, name, vacuity. witness**) to sort properties as shown in the following figure.



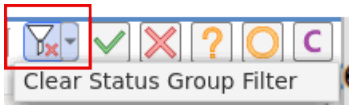
   Use the right-mouse-button menu on the column headers for the filtering with different available options.

2. To sort the columns in ascending or descending order, double-click on each column header.

3. You can also use the filter icons in the GoalList toolbar to filter properties by status (success, failure, inconclusive, not run, asserts, cover) or by custom conditions.

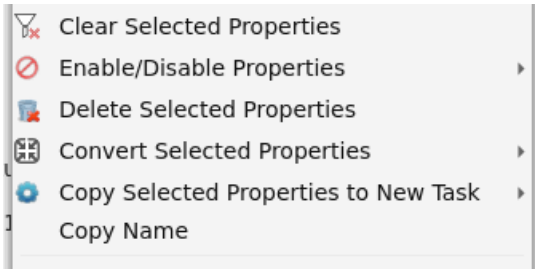
4. You can also double-click a row in the **Task Summary** tab to view the corresponding rows in **Targets** tab.

5. To clear the filtering across all columns, click the clear filter icon.



Selecting Properties from the GoalList

You can select specific properties from the GoalList and perform actions using the right mouse button (RMB) menu. The GoalList in the GUI displays all available properties, and the RMB menu provides context-sensitive options for property management.



The following RMB Menu Operations in the GoalList:

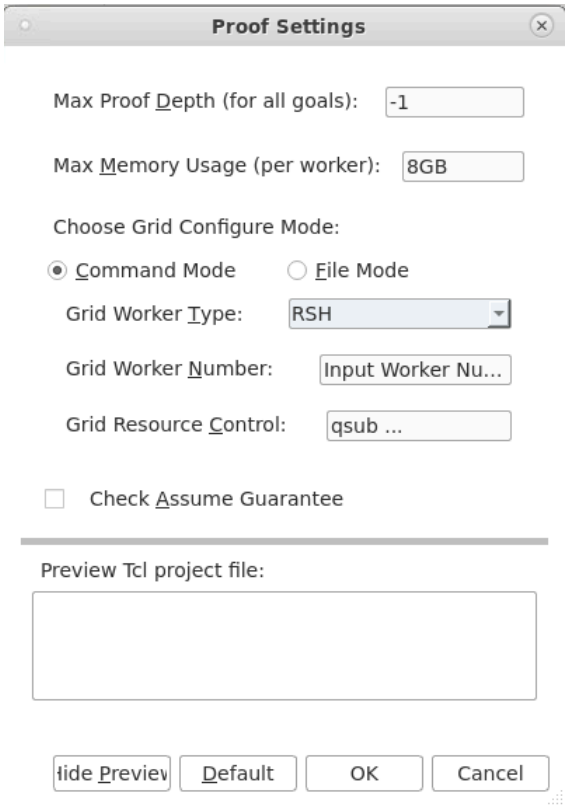
- **Enable/Disable Properties**: Use this option to enable or disable selected properties. This feature allows you to focus formal analysis on a targeted set of properties or exclude certain properties from the current run. The enable and disable actions correspond to the fvenable and fvdisable commands, which update the enabled attribute of the selected properties.
- **Delete Selected Properties**: Use this option to remove properties that are no longer needed in your analysis. This helps maintain an organized and relevant property list.
- **Convert Selected Properties**: Use this option to change the type of one or more selected properties (assert, assume, cover) in your design.
- **Copy Selected Properties to New Task**: Use this option to create a new verification task containing only the selected properties from an existing task.
- **Copy Name**: You can copy the names of selected properties to the clipboard for use in scripts, documentation, or additional command-line operations.
- **Export to CSV**: Use this option to export the GoalList contents, including selected properties, to a CSV file. This enables external analysis, sharing with team members, or archiving for future use.

Setting Up Proofs

You can specify the proof depth for all goals, maximum memory usage, and configure the grids for checks.

1. To configure several important parameters for your formal verification run, click .

   The **Proof Settings** dialog appears.



2. **Maximum Proof Depth**: Enter the maximum proof depth for all goals. This controls how deep the engines will attempt to prove properties.
3. **Maximum Memory Usage (per Worker)**: Set the maximum memory usage for each worker. The default is 8GB.
4. **Choose Grid Configure Mode**
   - To configure the grid in command mode, select Command Mode, and choose the grid type, specify the number of workers, and set control options.
   - If you already have a grid configuration file, select **File Mode** and browse to the configuration file.

Debugging Properties from the GoalList

The VC Formal GoalList provides the following options for debugging.

1. View and save traces

   For more details on viewing and saving traces, see section Saving and Replaying Traces.

- Right-click a property with a failing status and select **View Trace** to open waveforms for debugging.
- Right-click a property with a failing status and select **Save Trace** to save waveforms for debugging.
- The console displays the path to the saved trace and instructions for loading it in Verdi.

2. Open source code
   - Double-click the property name in the GoalList to view the source code.
   - Alternatively, use the **Show property source** option from the RMB menu in other columns.
3. Viewing Task Progress reports
   - Select one or multiple properties (using CTRL), and from the RMB menu, select **Property Progress Report** to view the progress report.
   - You can also use the icon to open the TaskProgress report.

     For more details on the task progress report, see section Viewing the Task Progress Report.
4. Viewing property expressions and statuses
   - Hover over a goal to display a tooltip with the property expression and detailed status.

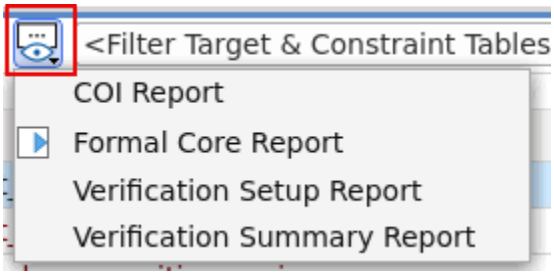
Managing GoalList Sessions

The GoalList saves column order, width, and sorting in the `novas.rc` file.

These settings persist across sessions and are specific to each VC Formal application.

Viewing Other Reports from the GoalList

To view the other verification reports, use the following options from the GoalList toolbar, or from the RMB menu of the GoalList.



1. Complexity Reports. See section Viewing Complexity Report (COI) in VC Formal GUI
2. Formal Core Report. See section Formal Core (FC) Analysis.
3. Verification Setup Report. See section Verifying VC Formal Designs
4. Verification Summary Report.

Accessing VC Formal Reports and Related Files

- **Session Log**: Results are saved in `vcf.log`, detailing the number of proven, falsified, vacuous, inconclusive, and covered properties.
- **Trace Files**: Saved traces (e.g., `.fsdb` files) for properties can be loaded in Verdi for waveform analysis.
- **Settings File:** The `novas.rc` file stores GoalList configuration and column order.

7.4.3 Customizing the View Settings in VC Formal

You can customize the view settings of widgets in VC Formal. You can select the font type and size for the VC Formal widgets as required. The customized settings is applicable for progress and goal progress reports as well.

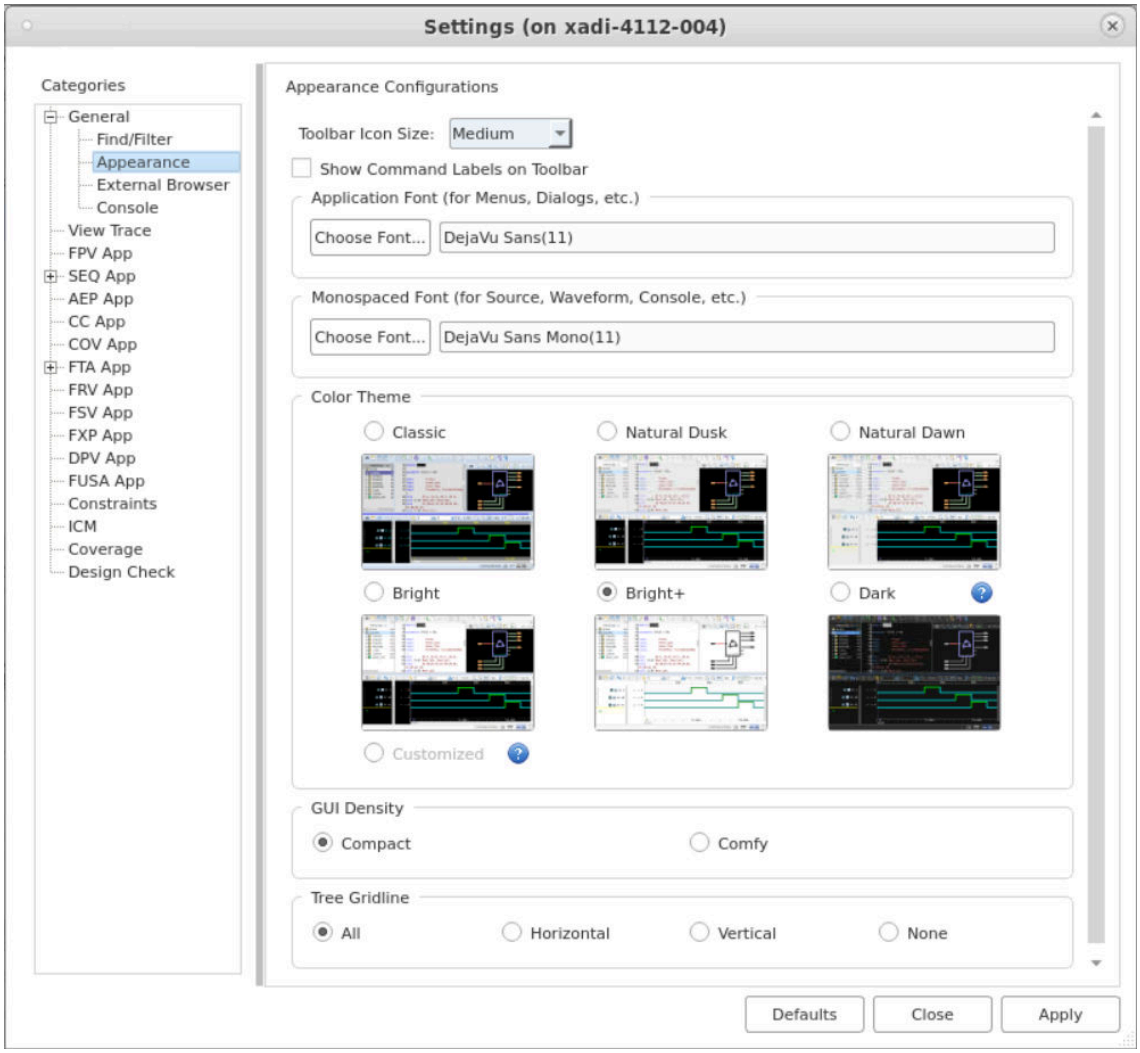
Changing Font Size and Color for VC Formal Widgets

To change font size and color for VC Formal widgets:

1. Click the (*Settings*) icon on the toolbar.

   The *Settings* dialog box appears.

Feedback

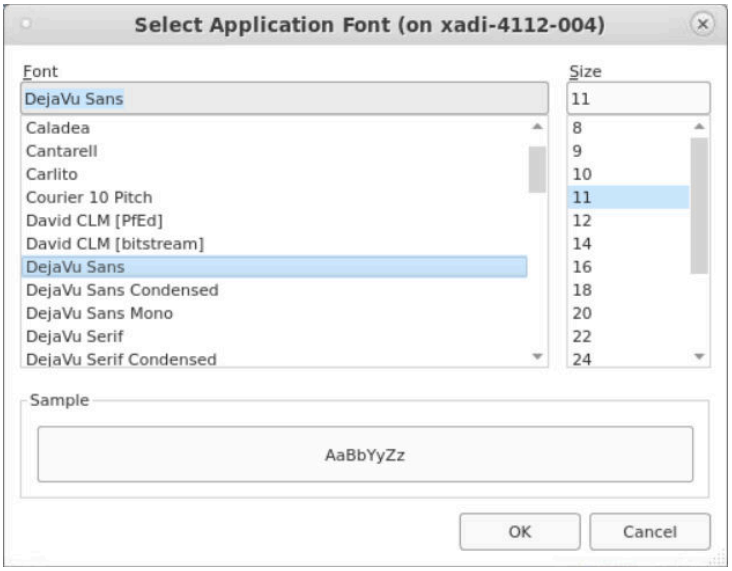


2. Select *General* under *Appearance* list to set the font type and size.
3. Click *Choose Font*.

   Note:

   This Settings is supported for all application modes in VC Formal.

   The *Choose Font* dialog box appears.



4. Select the *Font, Font style*, and *Size* as required.
5. Click *OK* and *Apply*. This font setting is applied for all the widgets in VC Formal.
6. Click *Close* to close the *Settings* dialog box.

Showing and Hiding Waived Properties in GoalList

By default, the waived properties appear in the *GoalList* for VC Formal Apps that support waivers and `fvwaive`/`fvunwaive` commands.

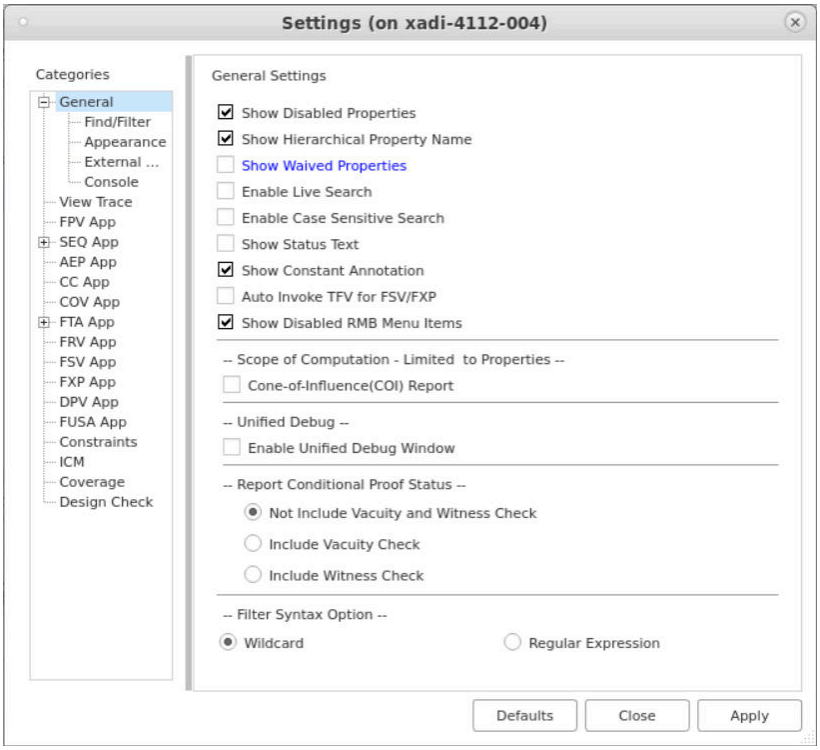
The following applications support waiving properties

- AEP (under `fml_aep_enable_waiver_flow` fml var),
- FCA (COV)
- FTA

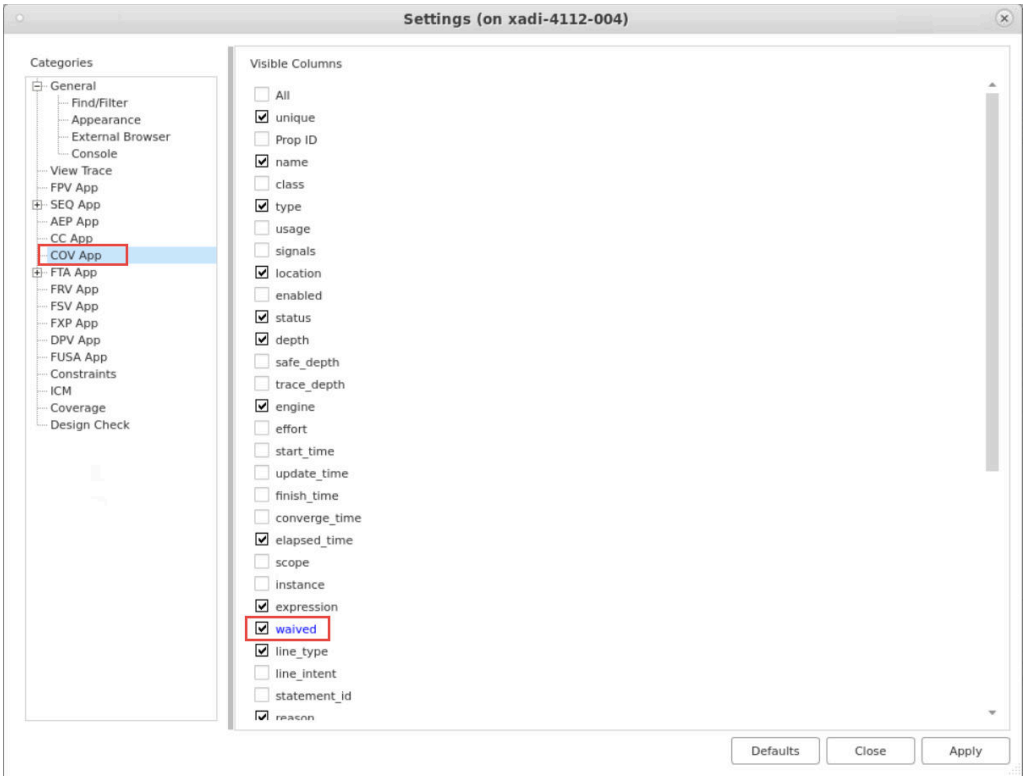
The waived properties appear with a light gray color background similar to disabled properties in the *GoalList*.

Use the *Settings window to hide waived properties by unselectingShow Waived Properties* option.

Feedback



Using the *Settings*, the *Waived* column can be added for VC Formal applications that support waivers and fvwaive/fvunwaive commands. The *Waived* column shows the waived status for each property with true indicating a waived property.

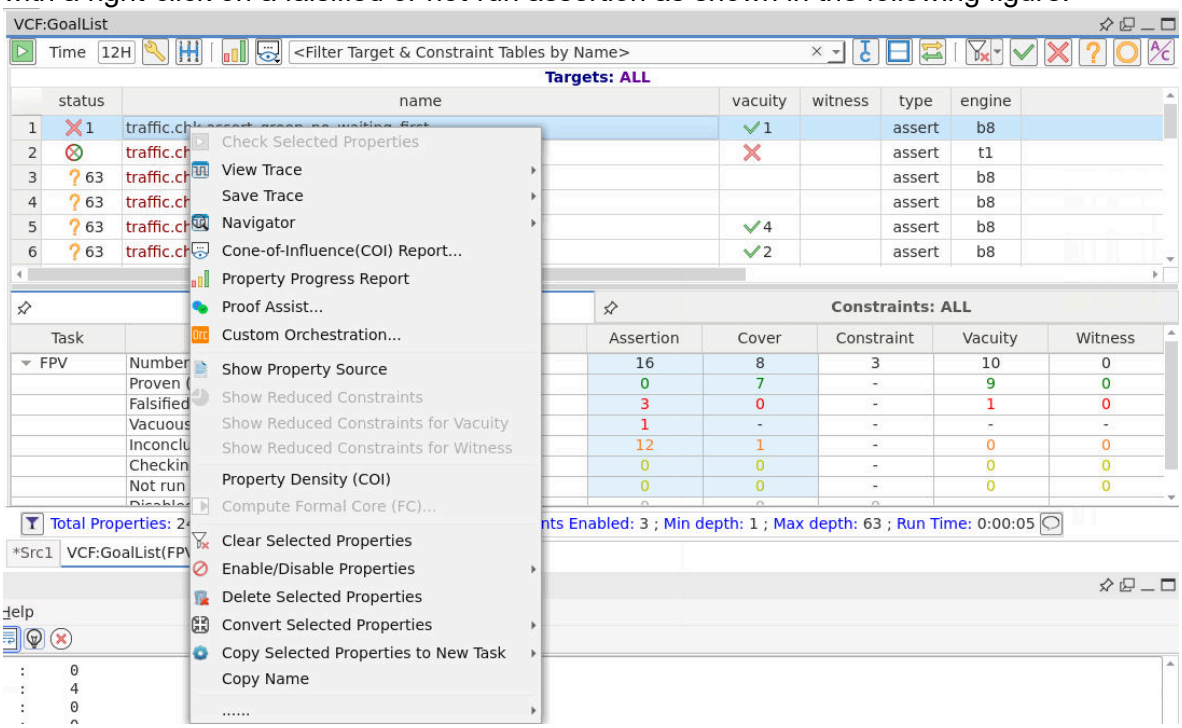


Showing and Hiding Context-Sensitive RMB Menu Options

By default, the context-sensitive RMB menu options not applicable for the current right-click target are visible but disabled. Disabled context-sensitive menu items are grayed out. For example, the *Compute Formal Core* RMB menu option is visible, but disabled

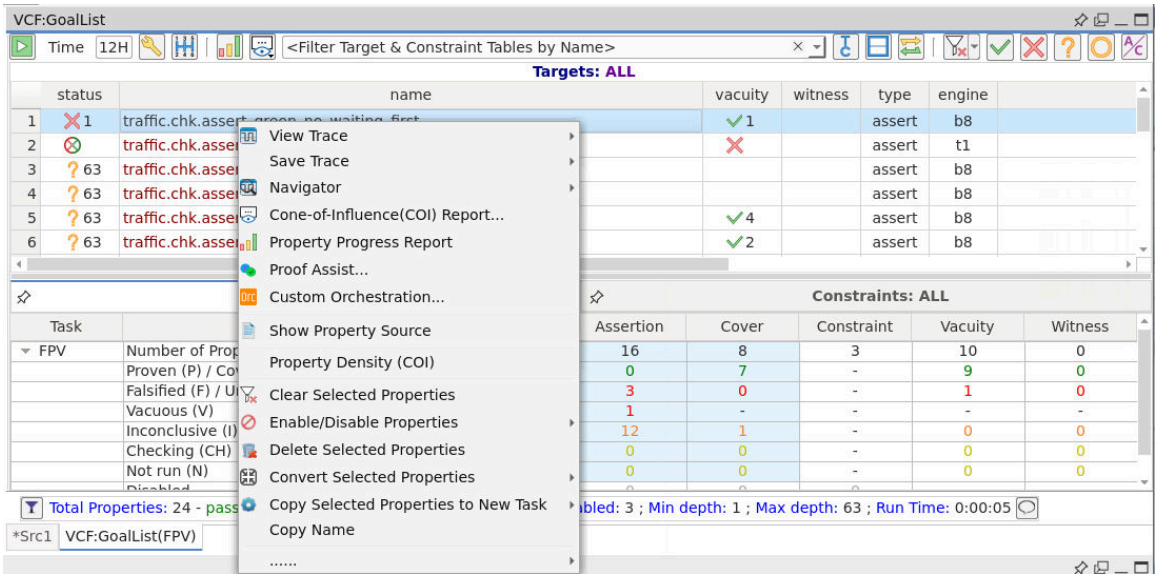
Feedback

with a right-click on a falsified or not-run assertion as shown in the following figure:



To hide disabled context-sensitive menu items instead of showing them grayed out, unselect the *General> Show Disabled RMB Menu Items* option in the *Settings* window.

When the option is unselected, the context-sensitive RMB menu options disappear as shown in the following figure:

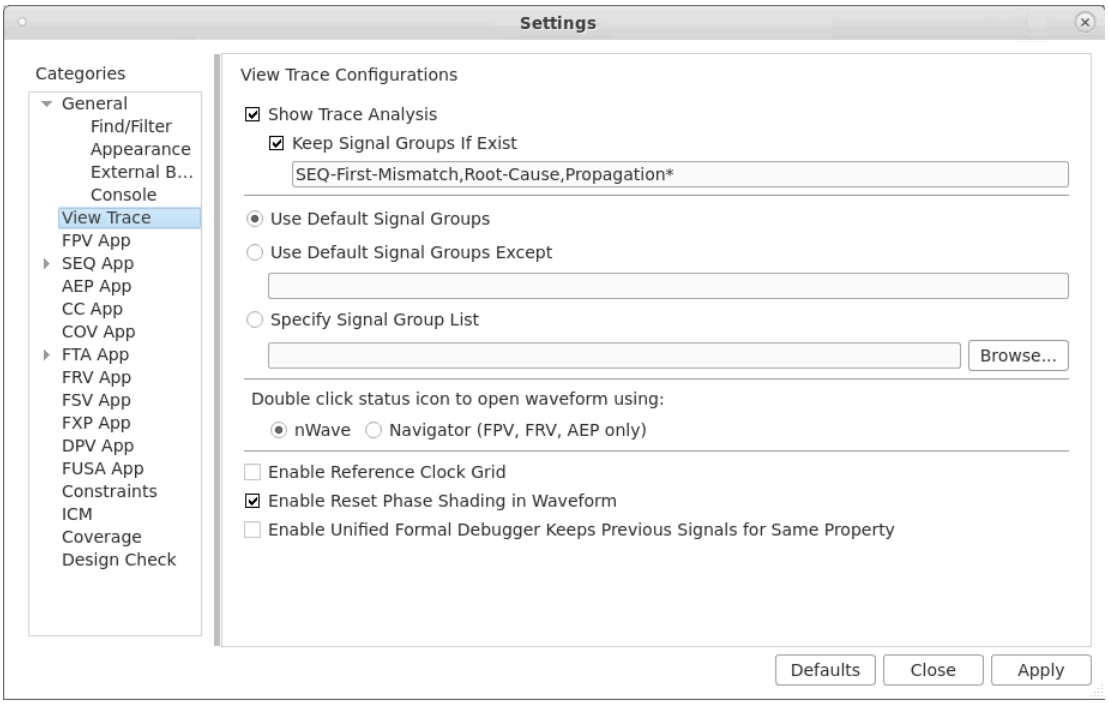


Feedback

Reusing .RC Files in Subsequent Runs

To reuse the *rc* file in subsequent runs, from the *View_Trace> Specify Signal Group List*option browse to the required rc file.

The Tcl command to perform the same is `view_trace <prop> -rcfile <rcfile>`.



Changing Color of Error/Warning/Info Messages

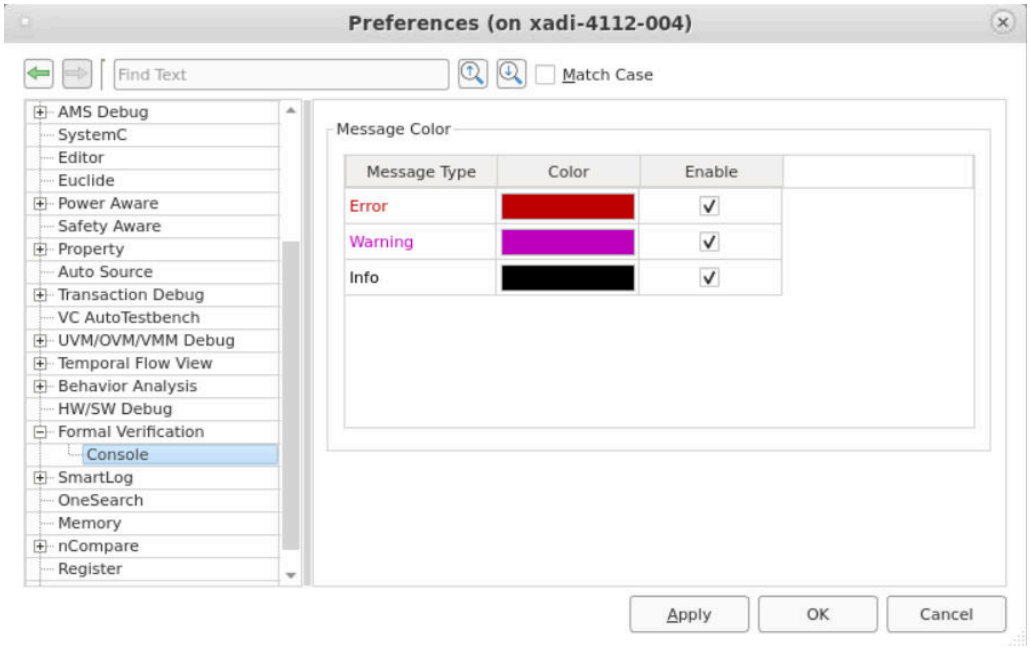
You change the color of the error, warning, and info messages displayed on the VC Formal console.

To change the color of the error, warning, and info messages:

1. Click *Tools* > *Preferences*.
2. Select *Formal Verification* > *Console* and set the colors for the errors, warnings, and info messages.

Feedback

By default, errors are in red, warnings are in magenta and info in black.



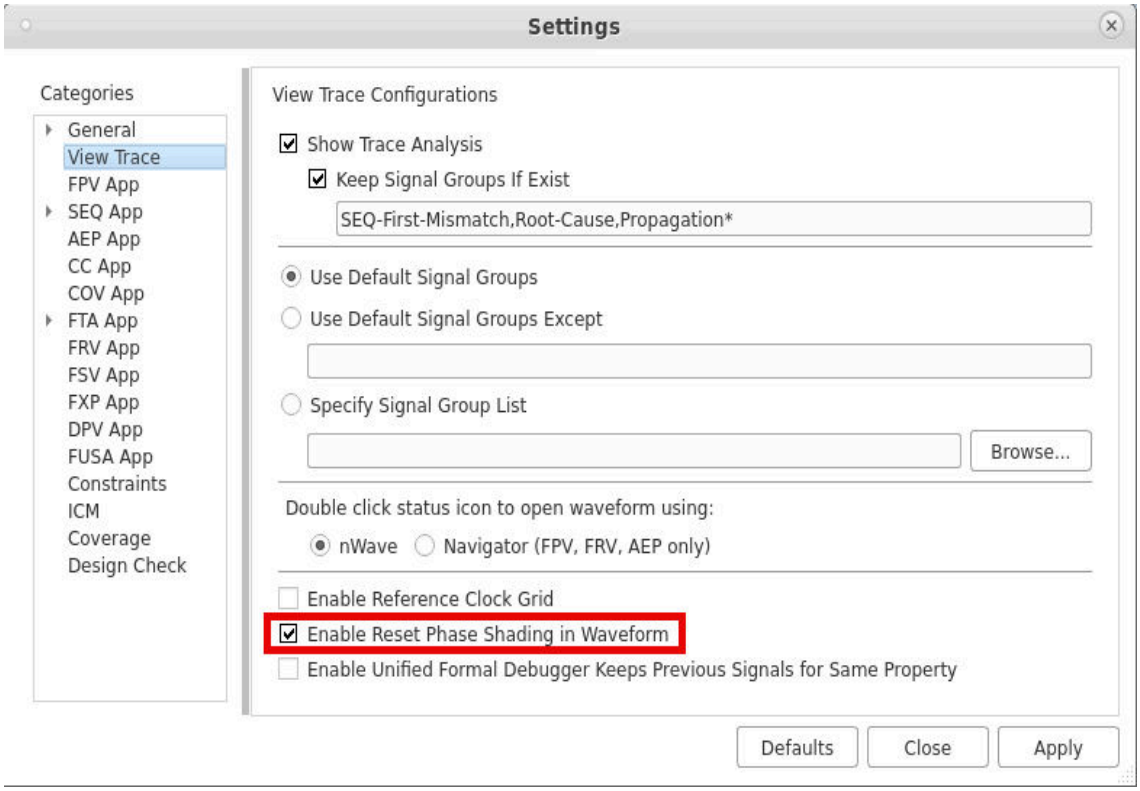
Enhancing Waveform Analysis by Shading the Reset Phase

By default, the reset phase in simulation waveforms is shaded. This shading allows you to clearly separate the reset period from other operational cycles, supporting efficient analysis of signal activity before and after reset. The feature is especially helpful for debugging or verifying initialization and reset-specific properties.

Feedback

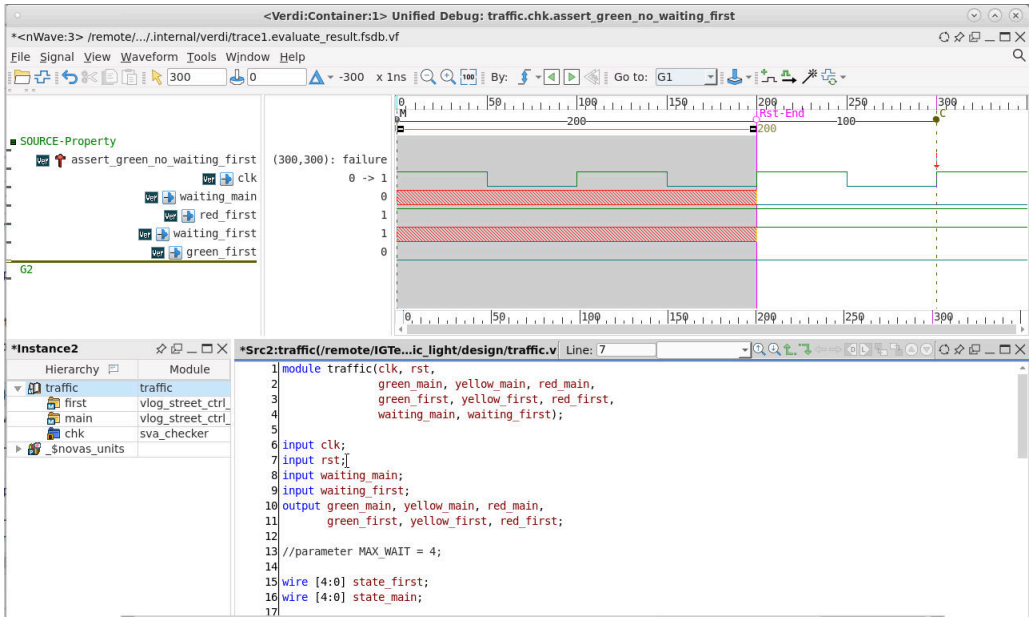
To disable the shading of the reset phase:

1. Open the Settings menu located at the top right of the property table in the Verdi GUI.



Feedback

2. In the **View Trace** category, deselect the **Enable Reset Phase Shading in Waveform** option. This option is enabled by default.



7.4.4 Viewing Witness Traces

A witness trace is a waveform showing a passing assertion. It provides a way to visualize and review whether the properties are written as intended. By default, witness trace generation is disabled. Only one witness trace for a passing property will be generated. It is not possible to view more than one witness trace at a time. The following command can be used to enable witness trace generation.

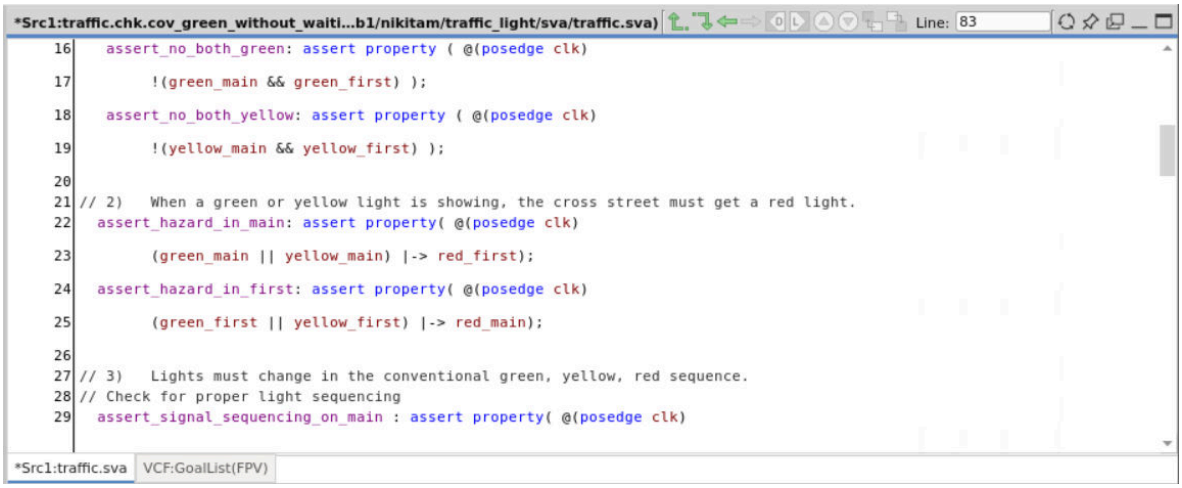
```
set_fml_var fml_witness_on true
```

This option enables witness trace generation for all properties specified by `fvassert`.

Witness trace in the VC Formal tool can be brought in the VC Formal GUI as seen in #unique_261/unique_261_Connect_42_i1031554.

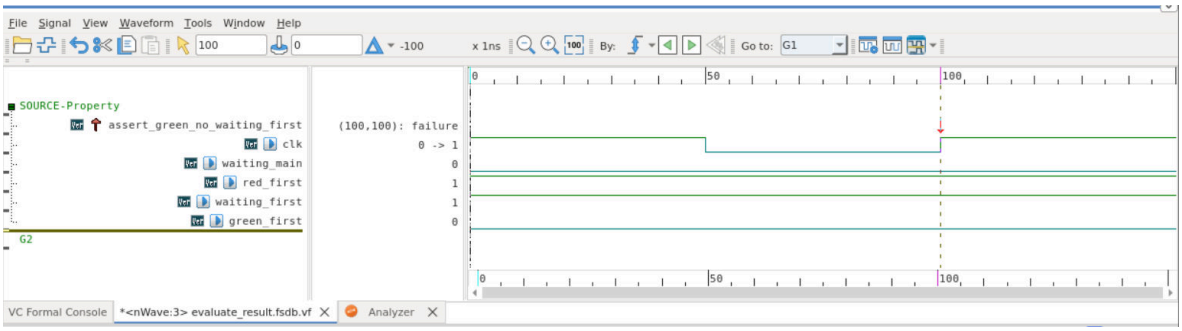
Feedback

Figure 45: Debugging Property Failure From GUI



The nWave window shows the source code, a property analyzer and the waveform as shown in Figure 46:

Figure 46: Falsification Trace in VC Formal



The assertion fails at the first rising edge of the clock. It is checking for one-hot behavior, that is exactly one bit asserted at all times. All grant signals de-asserted is a valid reset state but it violates the property which means the assertion is incorrectly coded. It should check for zero-one-hot behavior, that is, maximum of one bit asserted at all times. The design input `stall_i` shown in the waveform is shown as X because it is not a required signal for the property. Setting it to 0 or 1 has no impact on the property passing or failing.

Update the property to:

```
assert_gnt_zero_one_hot: assert property (@(posedge clk)
$onehot0({gnt3,gnt2,gnt1,gnt0}));
```

Rerun the check and this time the property passes.

```
[ 0] proven - assert_gnt_zero_one_hot
```

By default, VC Formal replays formal trace only in the counter example waveform. You can select the option to show the reset trace in the waveform. You can also set up the tool so that the counter example shows the composite reset and formal trace. By default, the composite waveform generation is set to `true`. You can see a marker on the waveform indicating the end of the reset sequence. You can also see the complete trace including the reset phase.

To disable this capability, use the following commands:

```
set_app_var fml_composite_trace false
sim_config -rst_wave OFF
```

7.4.5 Viewing Abstract Counters in VC Formal GUI

VC Formal allows for abstraction of counters by partitioning their value space by key values (using the `set_abstractions -using keyvals`command). While this causes an over-approximation of the counter's behavior, it can sometimes help with proving assertions which are otherwise extremely difficult to prove. Currently, the traces involving such abstracted counters can be difficult to interpret because the values of a counter transitions freely within each partition, like a snip. VC Formal only cares for the interval in which the counter belongs, as these intervals represent different abstract states in the design. The nWave is enhanced to display an abstract representation, depicting the value range, along with each counter signal. The abstract representation appears like a signal which shows the value interval of the corresponding counter signal.

You can see both counter and abstract-count signal in nWave. This helps in understanding the state/status at different times which enables you to identify the errors and the data sequence easily.

VC Formal tool dumps out the original counter signal, and it will also dump out the abstract counter signal in the FSDB for debug. When Verdi reads in the FSDB with abstract counter signals, you can see the new abstract counter signals in nWave.

To view the abstract signals in nWave:

1. Click *Signal* > *Get Signals*.

   The *Get Signals Form* appears.

2. Add the counter signal to the waveform.

   When counter signals are added in nWave, the abstract counter signal is also added into nWave automatically.

Alternatively, you can select the abstract signal and add it into nWave using the *Add Selected Signals to Wave* in nTrace and nSchema option from the RMB menu.

7.4.6 Debugging False Proofs

It is important to make sure the results reported by the tool are what you expected. Assertions that fail have a counter example waveform to show the violating behavior. It is clear that the assertion failed. The counter example can be debugged to determine the root cause of the failure. Assertions which are reported as proven may need further analysis since there are several possible reasons for the proven status. An assertion is reported as proven in formal analysis if it is not possible to generate a failing counter example. In effect, a proof is an absence of a failure. Different reasons for a passing status are:

- *Incorrect reset state* - Setting the correct reset state is extremely important. As incorrect reset state can often lead to false failures and on rare occasions can also lead to false proof.
- *Incorrect assertion* – there may be errors in the assertions as well as in the design. The assertion may incorrectly be coded to check a condition that cannot fail.
- *Over constraining*– there may be overly restrictive constraints that prevent valid chip behavior from occurring. Assertions that check these behaviors cannot fail. For example, a constraint to disable scan means that an assertion that is only triggered in scan mode will pass because it cannot fail.
- *Vacuity* – constraints may be incorrect such that no input sequences exist that obeys all constraints at the same time. If there are no valid input sequences to the design, no assertion can fail.

For more details on how to debug the Reset State, see section Specifying the Initial State.

7.4.7 Checking Vacuity Goals

By default, VC Formal automatically creates a vacuity goal and reports vacuous proofs for SVA asserts and assumes which use the |-> or |=> implication operators. A vacuous proof is one where the antecedent or the left side of the implication can never be true. When vacuous proofs are reported, you should review the code to determine why this occurred, and modify your assertion or design accordingly before rerunning your test.

To disable vacuity reporting, use the following command:

```
set_fml_var fml_vacuity_on false
```

Alternatively, set the `fv_create_vacuity_cover_properties` formal variable to `true` to create an explicit cover property for the vacuity term of FPV asserts and assumes. These vacuity cover properties are named *${original_property_name}_vacuity* and an integer is appended in case of conflict with an existing property. Similar to the vacuous proofs mentioned above, when uncoverable proofs are reported for these vacuity cover

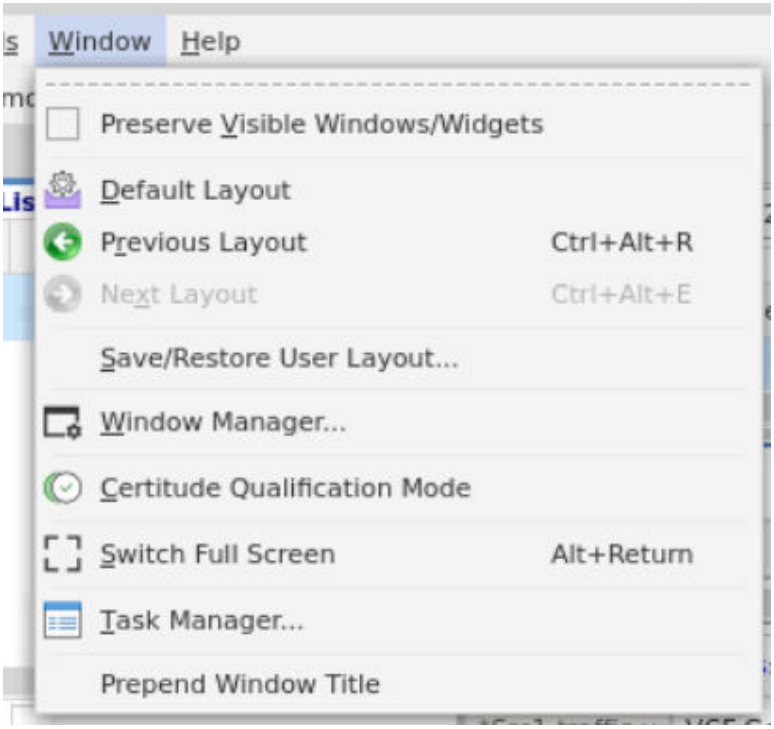
properties, you should review the code to determine why this occurred and modify your assertion or design accordingly before rerunning your test.

By default, `fv_create_vacuity_cover_properties` is set to false.

7.4.8 Preserving Multiple GUI Layouts

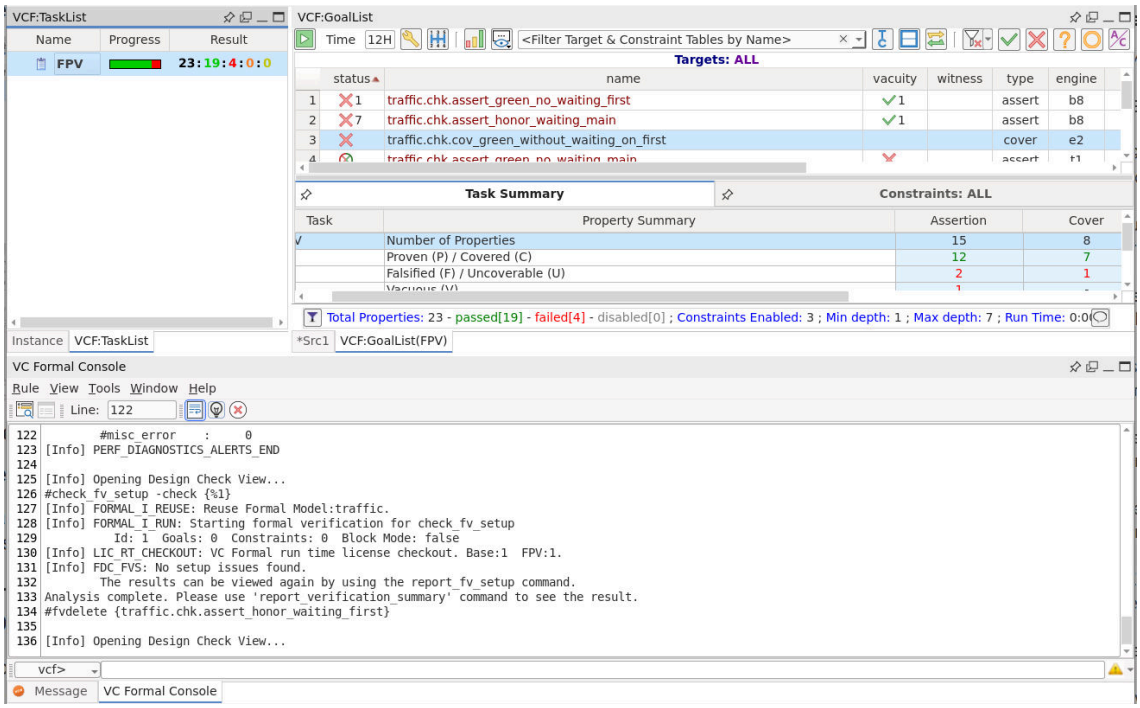
Rearranging and resizing of the GUI widgets/windows is a common user behavior while performing debug operations. VC Formal enables you to save/restore your customized GUI layouts and switch between layouts for various scenarios. You can use either of the following menu options for switching between multiple layouts:

Figure 47: Using Windows Menu



Feedback

- **Preserve Visible Windows/Gadgets**: Preserves the current visible layout in the novas.rc files and restores this layout in future sessions.
- **Default Layout**: Resets to the default layout as shown in the following figure:



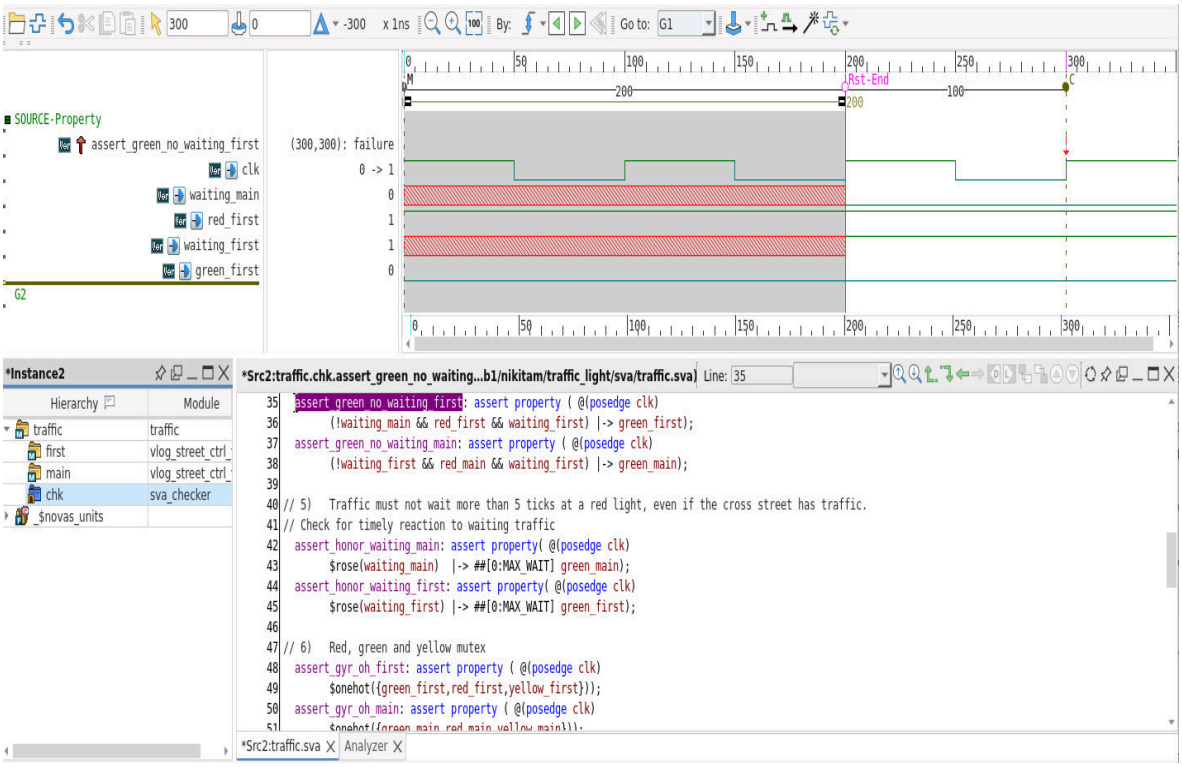
- **Previous Layout**: Restores the previously used layout.
- **Next Layout**: Acts like a Redo operation, and restores to the next layout.
- **Save/Restore User Layout**: You can save and restore customized layouts for various sessions based on your requirements.

7.5 Support for Unified Debug in Formal

VC Formal displays debug information—including the source window, waveform, and instance hierarchy—in a single, unified debug window. By default, the unified debug window is enabled across all VC Formal applications.

Feedback

The following is example of the unified debug window:



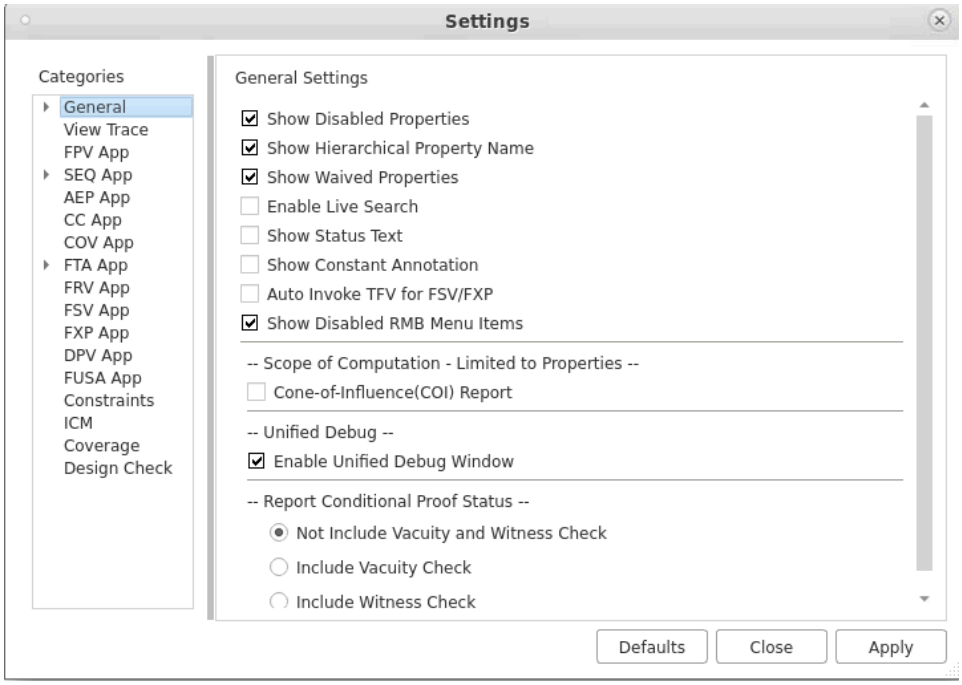
7.5.1 Opening the Unified Debug Window

You can enable/disable the unified debug window flow. You can also configure the default windows to be opened in the unified default window. For example, you can choose if the Instance Hierarchy must be opened by default in the unified debug window. The Navigator also opens in the Unified Debug mode.

Feedback

To open the unified debug window:

1. In the VC Formal Debug mode, click *Customize View Settings -> General*.

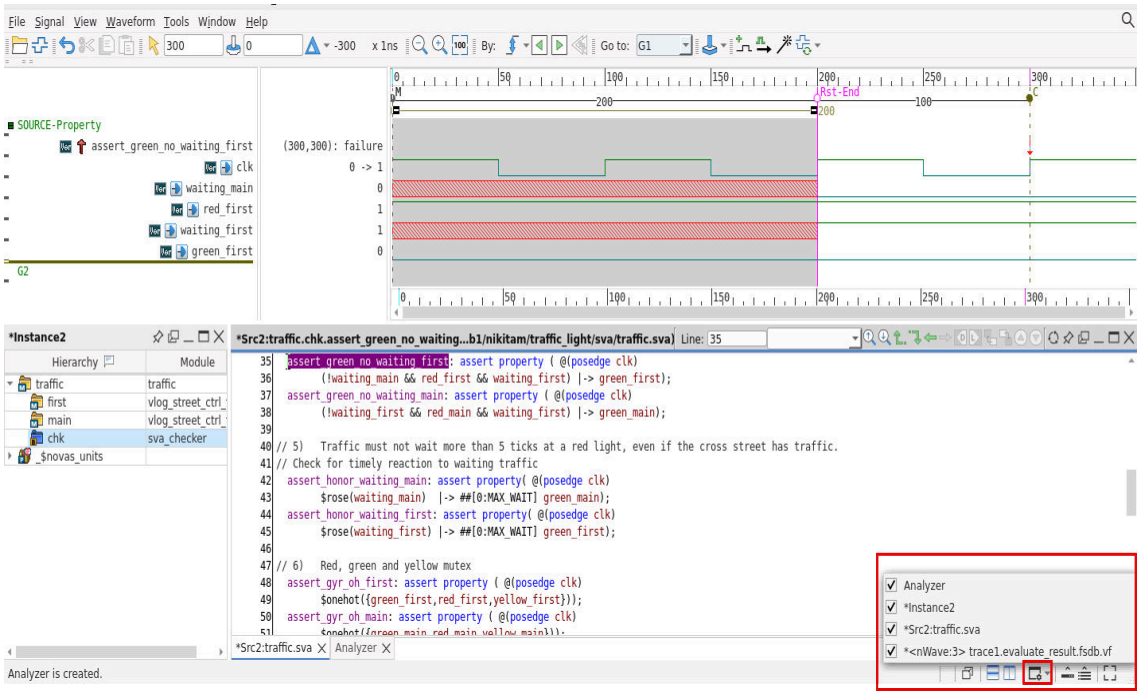


2. If you want the unified debug window to be opened by default in all sessions, select the *Enable Unified Debug Window* option.

7.5.2 Other Uses for Unified Debug Window

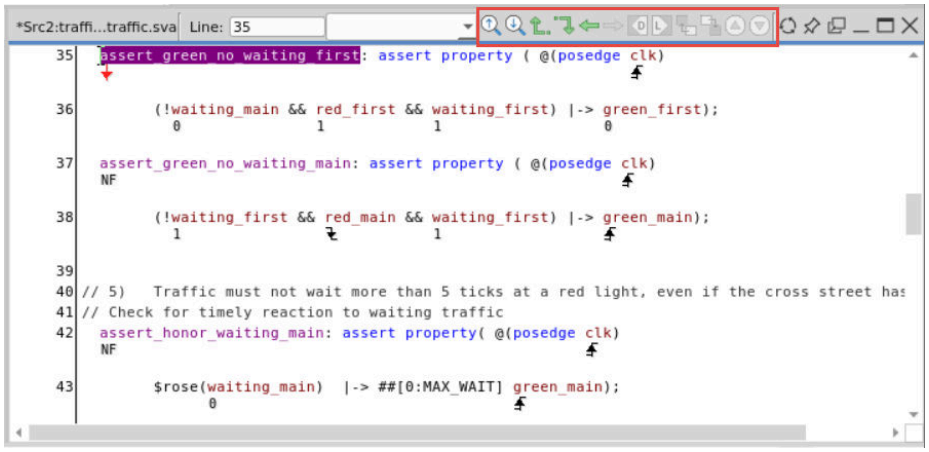
- You can also use hotkeys (for example, ctrl+w, ctrl+h, ctrl+x), PageUp/PageDown, and arrow keys in the unified debug window.
- You can show or hide window in the unified debug windows as shown in the following figure.

Feedback



- The unified debug window is restored when you restore the session. Each time unified debug window closed, the layout is saved. When the unified window is opened again, the same layout is restored.
- The source widget has the following toolbar which helps you to navigate:

  Calling, Definition, Backward History, Forward History, D, L, Show Previous, Show Next, Show Previous in Hierarchy, Show Next in Hierarchy

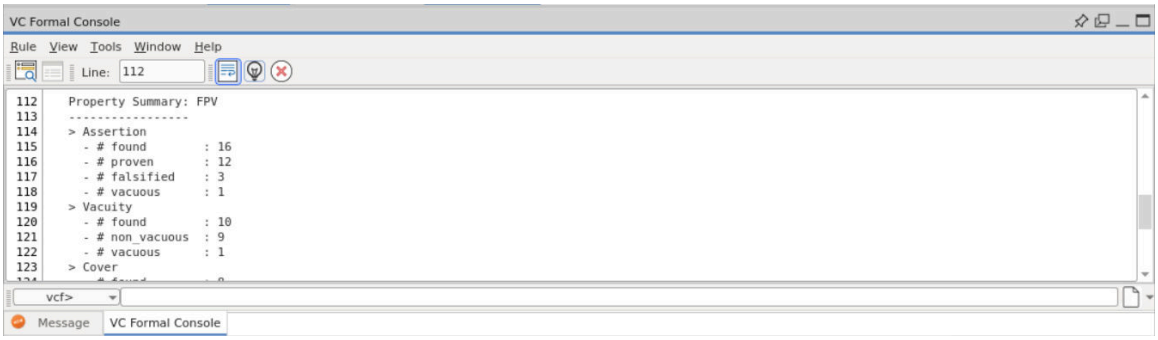


- You can drag and drop signals from source window to nWave window. You can select signals and then drag them to nWave in the same debug session.
- You can open multiple unified debug sessions. Each session will be opened with the name of the parent property.

7.6 Using VC Formal Console in GUI

VC Formal Console enables you view the VC Formal output incrementally and read and traverse log files in a readable view. VC Formal Console loads a log file with a specific format that shows a structured presentation of the log contents.

Figure 48: VC Formal Console



VC Formal Console offers the following features:

- Identify and filter out the desired log messages
- Auto-completion of commands and options
- Open the Smart Log viewer
- Automatically copies the text when user selects the text

7.6.1 Searching with Strings

You can search the log file to highlight a string and filter out the blocks that do not contain the matched string.

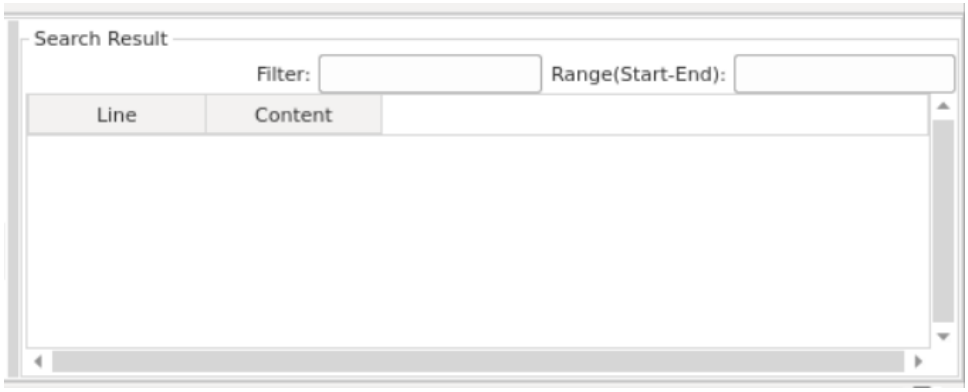
To search the log file with strings:

- Click .

  As illustrated in Figure 49, only the blocks with messages that meet the searched criteria are displayed.

Feedback

Figure 49: Search with Strings



- Type the value and the range in the Filter and Range boxes.

7.6.2 Locating a Specified Line Number

The line numbers are displayed in the VC Formal Console.

- In the *Line* box, specify a line number and press Enter.

  The cursor appears at the specified line and highlights the same.

7.6.3 Executing Commands from the VC Formal Console

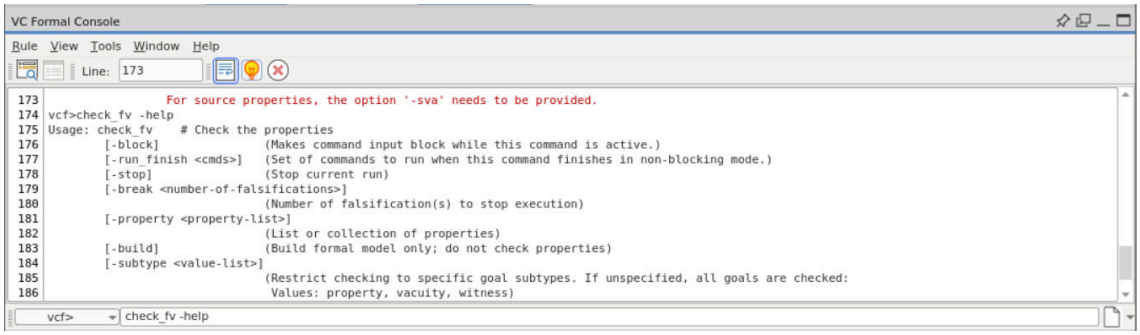
You can execute commands from the VC Formal Console. The VC Formal Console supports auto-completion of command and commands' options.

- Type the vcf commands in the *vcf*> box and press Enter, or click the *vcf*>to execute the commands.

  Use the *Tab* key to enable auto-completion of the commands and commands' option.

  The command string and returned result are displayed in the interactive console frame.

Feedback



- To get the historical typed-in commands, use the up "↑" and down "↓" keys in the command entry.

7.6.4 Opening Smart Log

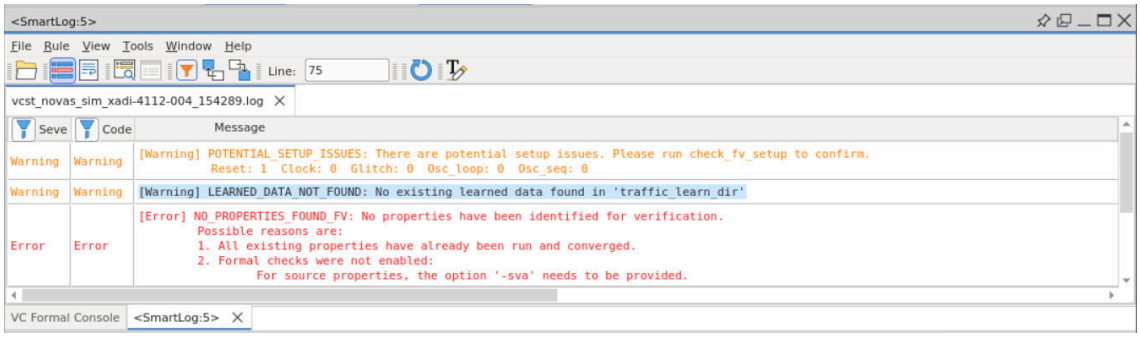
You can open the Verdi Smart Log viewer from the VC Formal Console.

- To open the Verdi Smart Log, in the VC Formal Console toolbar, click .

  The *SmartLog* window appears as shown in Figure 50.

  Whenever a new error message is reported, the icon changes to in the VC Formal console toolbar to indicate that new error messages are reported.

Figure 50: Verdi SmartLog



For more details on how to use the Verdi Smart Log, see section *Smart Log Tutorial* in the design Verdi® User Guide and Tutorial.

7.7 Using OneSearch for Design Information

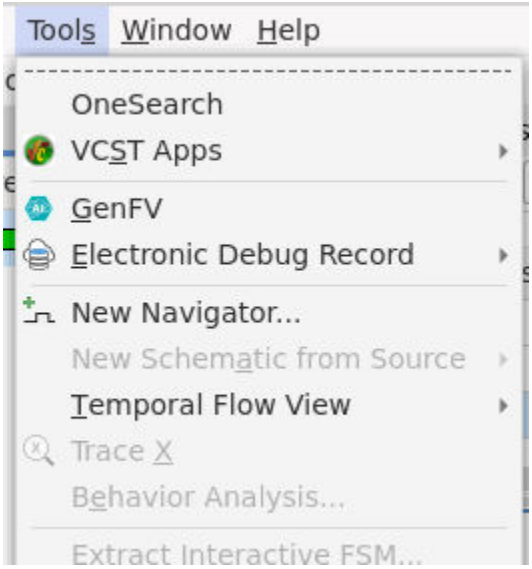
The OneSearch capability in Verdi works like a web search engine and helps you to search through various aspects or domains within your design. The OneSearch interface

Feedback

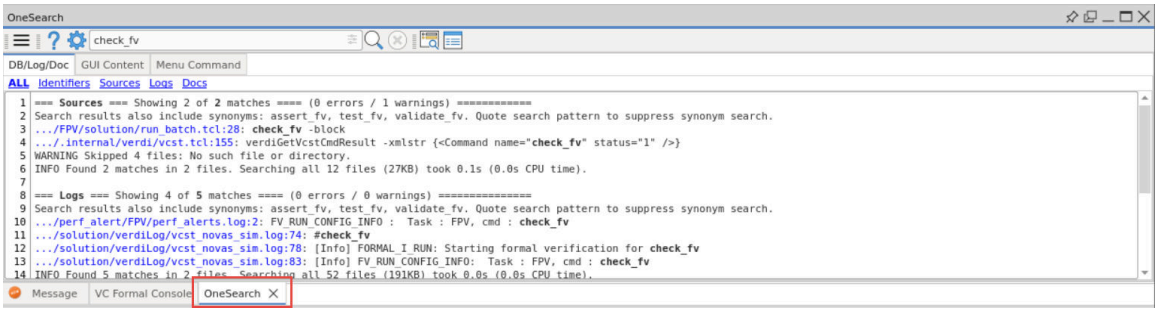
is simple, intuitive, and easy to use; wherein you can enter a search query and get ranked and sorted results, view the results, modify your query, and refine your search.

OneSearch supports search in **Sources**, **Identifiers**, and **Docs** domains with domain-specific search engines, generates ranked matches, and stores results in domain-specific results file.

To open Onesearch in VC Formal GUI, select **Tools > OneSearch** from the Verdi toolbar as shown in the following figure:



The OneSearch Window appears as illustrated in the following figure:



For more information on OneSearch, see the Verdi® User Guide and Tutorial.

7.8 Debugging Conflicting Constraints

When the `check_fv`command is run, the design and the constraints are analyzed for conflicts. If conflicts are found, the following message is displayed:

```
[Error] CONSTRAINT_CONFLICT: Found conflicting (contradictory)
 constraints.
[Info] CONFLICT_DBG:
[Info] CONFLICT_DBG: Conflict occurs at depth 0
[Info] CONFLICT_DBG: List of assumptions involved in conflict
 design.foo_assume (id=4)
```

This means that there is no legal input sequence that can satisfy the constraints. Either the constraints themselves conflict or they conflict due to the values assigned in RTL. You must look at the list of assumptions given below to debug constraints/RTL.

The constraint conflict message means that the design has conflicting constraints.

The `CONFLICT_DBG` message tells you the depth (in terms of clock-cycles) at which conflict occurs. The next `CONFLICT_DBG` message gives the list of assumptions that participate in conflict (and their property ids).

The assumptions given as SVA properties and script properties (without the `-stable`, `-env` switches) are reported. Only one set of conflicting constraints are reported. You must fix them and then run the tool again. This command does not report any constants (specified using the `set_constant` command) even if they are involved in conflict.

The conflict engine has a label *cc*. To see if the conflict engine is running, use the following command.

```
report_fml_jobs -engine cc -list
```

Note:

The report command does not work for `check_fv`command in the SEQ application mode. That is, the `report_fml_jobs` command does not report `cc` engine for SEQ application.

When conflicting constraints are detected an error message is issued.

7.8.1 About Conflicting Constraints

Conflicting constraints at a depth D mean that there is no legal input sequence that can satisfy all constraints at that depth. This indicates a serious problem with the constraints and/or RTL as the design is severely (100%) over-constrained from that depth onwards. As there is no legal input sequence, then there is nothing to verify from that depth D onwards. The engines trivially (vacuously) report proofs after depth D. In order to guard you against vacuous proofs the `check_fv`commands perform conflict checking, by default

Feedback

up to a depth of two cycles. To increase the depth for this check you can use the formal application variables mentioned in the following examples.

When conflicting constraint message is seen you must debug the constraints and/or RTL.

A small/minimal set of constraints that are responsible for conflict are reported. This helps when there are hundreds of constraints and only few of them are causing conflicting constraints. In addition, if reset state of certain registers is involved in the conflict, then those registers are reported as well. You must identify which parts of RTL are involved in the conflict. You must look at the signals in conflicting constraints and understand how are they modified by the RTL. The conflict may be solely due to constraints or it may involve both constraints and RTL.

Examples

- The following script properties are conflicting at depth 0.

```
fvassume -expr {in1 == in2 }
fvassume -env -expr {in1 == 1}
fvassume -depth 1 -expr {in2 == 0}
```

  Note the conflict does not involve RTL.

- The following script property and RTL is conflicting at depth 0.

  RTL line:

```
assign dummy_reg = 3'b000;
```

  constraint

```
fvassume -expr {dummy_reg > 3'd3}
```

  This is because the constraint is requiring `dummy_reg` to be greater than `3'd3`. However, `dummy_reg` is a constant `3'b000` in the RTL.

  There is no legal input sequence that satisfies the constraint.

- Following script property and RTL are conflicting at depth 0.

```
 constraint:
   fvassume {A == 1}
 RTL
  always @(posedge clk or negedge rst)
```

```
if (!rst)
A <= 0;
else
......
```

  The reset state of A is 0 as per RTL. However, the constraint requires it to be 1.

- Following script property and RTL are conflicting at depth 7 (clock starts in low phase).

Feedback

```
constraint: fvassume {count < 4}

RTL
 reg [3:0] count;
always @(posedge clk or negedge rst)
 if (!rst)
  count <= 0;
 else
  count <= count + 1;
```

The RTL keeps incrementing counter count from 0, 1, ..., 15.

However, the constraint requires it to be always lesser than 4. This constraint cannot be satisfied with the given RTL as the count is 4 after fourth positive clock edge.

- Following SVA property and script assumption and RTL conflict.

  Script property:

  ```
  fvassume {in == 0}
  ```

  SVA property:

  ```
  assume property @(posedge clk) disable iff (reset == 1'b0) (RA == 1)
  ```

  RTL:

  ```
  reg RA;
  always @(posedge clk or negedge reset)
  if (~reset)
  RA <= 1;
  else
  RA <= in;
  ```

  For SVA property to hold RA, it must be 1 at posedge of the clock. This is only possible if input `in` was 1 before the clock edge happens. However, the script property requires`in` to be always 0. The constraints are conflicting as there is no legal input sequence that satisfies both of them.

7.8.2 Performing Conflicting Constraints Checks Up to a Specified Depth

By default, conflicting constraint checks are performed up to a depth of two clock cycles. If there are no conflicting constraints till the depth of two, it does not mean that there are no conflicting constraints at all. It is possible that conflict happens at a later depth, say at the depth of fifty.

The conflict checks can be expensive, by default, a conflict check is done till a small depth. If you want to check for conflicts at a greater depth, you can use the following formal variables:

- `set_fml_var fml_conflict_check_depth <num>`Checks for contradictory constraints up to a given depth. The default value for `<num>` is 2.

  Conflicting constraints can occur at any depth. By default, conflicting constraints check is performed up to a depth of 2 so it is quick. No engine is started until this check has succeeded. You can increase value by`set_fml_var fml_conflict_check_depth 20`

  It is best to do conflicting constraints check for higher depth to ensure constraints are ok. Alternatively, run check_constraints command with higher value of -ccDepth.

- `set_fml_var fml_conflict_debug_minimal <true/false>`Reports small sets of conflicting constraints. The default value is true. Computing small set of constraints can be expensive. If you experience this, then you can set this variable to `false`.

- `set_fml_var fml_conflict_debug_reset <true/false>`Reports the list of register names whose reset state contribute to the conflict. The reported list is not necessarily exhaustive. The default value is false as it is more expensive to compute.

  Note:

  Conflict checking is done in a blocking manner, that is, orchestration first runs conflict check, if there are no conflicts only then other engines are started. If there are conflicts, the orchestration stops and reports conflicting constraints.

7.8.3 Viewing the Reports of Conflicting Constraints and Dead-ends

Use the `report_constraints` command to get both the results of constraints check as well as the dead-ends, if enabled. Dead-ends are explained in more detail in section Dead-ends due to Constraints.

The `report_constraints`command reports the summary of conflicting constraints check. It reports the status of conflict check which can take following values:

- `[-list]`: Outputs one-line report per deadend. By default we only report deadends with distinct set of involved constraints/registers (root cause).)
- `[-verbose]`: Outputs detailed report per deadend. By default we only report deadends with distinct set of involved constraints/registers (root cause).)
- `[-no_summary]`: Do not print summary report
- `[-allDeadends]`: Provide information about all deadends (not just the ones with distinct root cause)

Feedback

- `[-deadendId <id>]`: Filter on a particular deadend id
- `[-dict]`: Returns selected output as a Tcl dictionary.

The depth is also printed. In case, conflicting constraints are found, the set of constraints involved and registers involved are reported as controlled by formal application variables described above. For each constraint, its *property id* is also reported.

The constraints in an FV setup can cause dead-ends in the design. A dead-end state is a state

- That is reachable from a reset state.
- Has no legal next state. That is, no successor state exists where all constraints can be met.

Example

The following an example of report for `report_constraints`:

```
check_constraints -block -ccDepth 4
[Info] FORMAL_I_REUSE: Reuse Formal Model:simple1.
[Info] FORMAL_I_RUN: Starting formal verification for check_constraints
      Id: 1  Goals: 2  Constraints: 3  Block Mode: true
[Info] LIC_RT_CHECKOUT: VC Formal run time license checkout. Base:1
 FPV:1.
[Error] CONSTRAINT_CONFLICT: Found conflicting (contradictory)
 constraints at depth 4.
    There is no legal input sequence that can satisfy the constraints.
    Either the constraints themselves conflict or they conflict due to
 the
    values assigned in RTL.
    List of assumptions involved in the conflict:
      counter (id=5)
1
report_constraints -verbose
Constraints summary for verification problem vp1 (id=1) in verification
 task FPV
Conflicting constraints check summary
    Status        : conflict
    Depth         :     4
    Involved constraints
        counter  (id=5)
Deadend check summary (distinct)
    NumDeadends   :     1
    Max length    :     4
    Min length    :     4
    Avg length    :     4
    Deadend engine has completed running.
--------------------------
Deadend verbose report (distinct)  for verification problem vp1 (id=1) in
 verification task FPV
DeadendId :   143
```

Feedback

```
Length    :     4
    Involved constraints
        counter  (id=5)
1
```

The example output as a Tcl dictionary:

```
set result [report_constraints -dict]
cc_status conflict cc_depth 4 cc_constraints "counter " de_num 1
 de_len_min 4 de_len_max 4 de_len_avg 4
puts [dict get $result "cc_status"]
conflict
puts [dict get $result "cc_depth"]
4
puts [dict get $result "cc_constraints"]
counter
puts [dict get $result "de_num"]
1
puts [dict get $result "de_len_min"]
4
puts [dict get $result "de_len_max"]
4
puts [dict get $result "de_len_avg"]
4
```

7.8.4 Dead-ends due to Constraints

Dead-ends in a formal verification setup can point to badly written constraints or over-constraints. In some cases dead-ends can be intentional in nature. Depending on the methodology you employ, dead-ends may or may not be a bad thing.

For example, consider a four-stage pipelined design that passes input packets from one stage to another. Suppose you have a constraint that packet always have a specific address in pipeline stage four. Then, any input packet where the address does not match the specific address will cause a dead-end in the design. That is design can go to a state which has no legal next state, that is, a state where our constraint can be met.

The VC Formal tool provides an ability to find dead-ends in the setup.

Dead-end Engine

To enable a dead-end engine, use following command before `check_fv`command.

```
set_engine -on de
```

The dead-end engine has a label *de*. To see if the dead-end engine is running or not, use the following command:

```
report_fml_jobs -engine de -list
```

Feedback

The following application variables control the effort spent in the dead-end engine:

- `set_fml_var fml_deadend_check_maxcycles <num>`

  Controls the total number of cycles for which dead-end engine is run.

- `set_fml_var fml_deadend_check_depth <num>`

  Controls the max depth for a dead-end (that is, do not report dead-ends of longer length).

- `set_fml_var fml_deadend_debug_minimal <true/false>`

  Reports small sets of constraints causing dead-end. The default value is true. Computing small sets of constraints can be expensive. If you experience this, then you can set this variable to false.

- `set_fml_var fml_deadend_debug_reset <true/false>`

  Reports the list of register names whose reset state contribute to the dead-end. The reported list is not necessarily exhaustive. The default value is false as it is more expensive to compute.

The register list is not necessarily exhaustive. The register list is expensive to compute and is computed only when this variable is set to true. To see the default value of these formal variables use the `report_fml_var` command.

Even while the dead-end engine is running it can start producing information about dead-ends. To see if any dead-ends were found and get more information about them use following command:

```
report_constraints [-list] [-verbose] [-allDeadends]
```

This command first reports the results/summary of conflicting constraints check. Next, it reports the summary of dead-ends found. This includes number of dead-ends found, min/max/average length of dead-end. The `report_constraints` with `-list` option reports a table with dead-end ID and dead-end length. The `report _constraints` with `-verbose` option reports for each dead-end the constraints that are involved in the dead-end (not necessarily minimal) and the registers that are involved.

By default summary, `-list, -verbose` options only report information about dead-ends that have a distinct root cause, that is, only dead-ends which have a different set of constraints/registers responsible for the dead-end. In order to see all dead-ends you can pass `-allDeadends` option.

7.8.5 Debugging Dead-end Engines

The `view_trace` command can be used to generate an FSDB for a dead-end and open it in Verdi. The FSDB extends the last legal (dead-end) state with half clock-cycle more to also show one illegal state that follows.

The option `-deadendID <id>`, where `deadendId` is what is reported by `report_constraints -list`.

Note that:

- The dead-end analysis is not guaranteed to find or report all dead-ends or to report specific dead-ends that you may already know about. It may report zero dead-ends even if the design has dead-ends.
- Dead-end states are not the same as deadlock/livelock states. The dead-end states are caused by user constraints, but deadlock/livelock depends on the RTL.

Limitations

- The `check_fv` command in the SEQ mode does not support the dead-end engine.
- The `check_fv` command requires problem to have at least one goal (property) to check.

7.8.6 Command Dedicated For Checking Constraints

You can also check the conflicting constraints and dead-ends up to a desired depth using the `check_constraints` command. By default, the `check_fv` does a conflicting constraints check for a depth of two. Also, if you want to perform a deadend check with `check_fv`, you must enable deadend engine *de*, and the `check_fv` command stops if there are no goals (assertions/covers and so on.). However, the `check_constraints` command can be used even if there are no goals. The depth specified in the `check_constraints` command overrides the values provided in the `fml_deadend_check_depth` and `fml_conflict_check_depth` formal variables.

7.9 Identifying Reduced Constraints for Proofs

Avoiding over constraining in the design is important, as it may lead to missed bugs and vacuous results. An under constrained environment that does not lead to false failures is desirable as it usually has faster run-time and the amount of verification goes beyond the minimal set of legal inputs. To optimize the constraint environment, you must:

- Remove the constraints that are not needed for proving a property
- Identify the constraints that are responsible for vacuous proofs

Feedback

The constraint analysis is done *on one or more proven*properties in the following two steps.

1. A reduced constraint set *is computed for each specified property using*the `compute_reduced_constraints` command.
2. The result is reported with the `report_reduced_constraints` command.

7.9.1 The compute_reduced_constraints Command

Given a list of proven properties or a Tcl collection of proven properties <tclListOrCol>, you can compute the reduced constraints for the specified properties using the following command.

Syntax

```
compute_reduced_constraints -property <tclListOrCol> [-subtype <
 vacuity,witness>] [-block] [-stop]
```

Example

```
vcf> compute_reduced_constraints -property arb.arb_sva_inst.gnt_oh0
[Info] FORMAL_I_CREATE: Create Formal Model:arb.
[Info] FORMAL_I_RUN: Starting formal verification for
 compute_reduced_constraints
Id: 1 Goals: 1 Constraints: 4 Block Mode: false
```

This computes the set of constraints that are required for the property to be proven, that is, all the other constraints can be disabled when proving this property.

The `compute_reduced_constraints`command can also be used to identify which constraints are causing a property to be vacuously proven. For example:

```
vcf> compute_reduced_constraints -subtype vacuity -prop
 AXI_128_BIT_INCR_RD_ALIGNED_64_128BIT
```

7.9.2 The report_reduced_constraints Command

To know the result of the `compute_reduced_constraints` command, you must use the `report_reduced_constraints` command.

Syntax

```
report_reduced_constraints [-property <tclListOrCol>] [-subtype <
 vacuity,witness>]
```

An example of the output of the `report_reduced_constraints` command for a proven and a vacuous property is shown below:

The following is an example of the output of the `report_reduced_constraints`:

```
vcf> report_reduced_constraints
Property : main.bar.genblk1.dasrt
SubType : -
Needed constraints
a1
a3
vcf> report_reduced_constraints
Needed constraints
Property : AXI_128_BIT_INCR_RD_ALIGNED_64_128_BIT
SubType : vacuity
Needed constraints : 3
AXI_MST_128_BIT_INCR_RD_ALIGNED_64_128_BIT
AXI_MST_ASIZE_NOT_TO_EXCEED_RD_BUS_WIDTH
AXI_MST_AVALID_RST_LOW
```

7.9.3 The get_reduced_constraints Command

To get a TCL collection of constraints present in the reduced set of constraints for specified properties, you must use the `get_reduced_constraints`command.

Syntax

```
vcf> get_reduced_constraints [-property <tclListCol>]
```

If no option is given a union of reduced constraints for all the properties where reduced constraints is available is displayed.

Limitations

- The set of constraints computed for a property is not necessarily minimal. A constraint may be in the structural COI of a property but not needed to prove the property.
- Constraints in the form of `set_constant` is not supported.
- Environment constraints of the form `fvassume -env`are not supported.
- You cannot use this feature with the GUI, it must be run using the tcl commands.

7.10 Exporting FSDB for Property Traces

You can save the FSDB files in the `vcf` for a desired list of properties for validating and debugging the results in Verdi. These FSDB files can be imported in Verdi to view the debug waveform and validate the design.

The following are the prerequisites to save the FSDB files in the `vcf`:

1. Ensure that the VC Formal run is successful.
2. Run the following TCL commands on the `vcf` or in the TCL command file:

   ```
   set_app_var fml_mode_on true
   ```

3. `set_app_var enable_verdi_debug true`Specify the directory where the FSDB files must be dumped:

   ```
   set_app_var verdi_export_dir <dir path>
   ```

   The default value of the `verdi_export_dir`application variable is './'. All FSDB files are dumped in specified directory in the following directory structure:

   ```
   <verdi_export_dir >/traces/
   ```

4. Ensure that the Verdi compiled design is copied into `<Verdi_export_dir>/work.lib++`

7.10.1 Saving FSDB Traces in VC Formal

You can use the `fvtrace` command to save the FSDB file for the desired list of properties.

Use Model

```
vcf> fvtrace -help
Usage: fvtrace    # Create an FSDB trace of selected property.
        [-reset]                (Create an FSDB of the reset trace)
        [-property <propName>] (Create an FSDB for property <propName>.)
        [-vacuity <propName>]  (Create an FSDB for vacuity trace for
 <propName>.)
        [-witness <propName>]  (Create an FSDB for witness trace for
 <propName>.)
        [-inteq <id>]          ((SEQ) Create an FSDB for internal
 equivalence <id>)
        [-file <fileName>]     (Filename for the resulting FSDB.)
        [-composite]           (Write out the combined reset+property
 trace)
        [<propName>]           (Property name)
```

The default value for the `-file` option is *<propname>_timestamp.fsdb*. You can save the traces after restoring a previous saved session into VC Formal.

Note:

By default, the `-composite` option is not supported. To generate a composite trace, set the following environment variable: `% setenv SNPS_VCST_DISABLE_VF 1 % Launch VC Formal session % fvtrace -composite <property_name>`

7.10.2 Reading FSDB in Verdi

Use the following command to run the Verdi compiler and load the FSDB files saved by VC Formal in Verdi:

```
setenv VCS_HOME $VC_STATIC_HOME/vcs-mx
verdi -lib <export_dir>/work.lib++ -ssf
 <export_dir>/traces/<fsdb-file-name> -play
 <export_dir>/traces/p.fsdb.tcl
verdi -dbdir ./vcst_rtdb/.internal/design/simv.daidir -ssf
 <export_dir>/traces/p.fsdb -play <export_dir>/traces/p.fsdb.tcl
```

Where *simv.daidir*can be named differently based on design import commands. The exact paths depend on the design and the export directory.

Note:

Use an equivalent set of files and command line options as provided to VC Formal build. Any change in the command line can lead to a failed FSDB load.

7.10.3 Limitations

- The debug capabilities when you load the FSDB in Verdi is a basic waveform debug as available in stand-alone Verdi waveform viewer.
- No formal-aware debug features of VC Formal or Verdi are available in this waveform.
- You cannot merge FSDB files into a single file as the list of signals are different for each FSDB file for different properties.
- You cannot load a FSDB into a Verdi session that is outside VC Formal, if you have performed any snipping or abstraction in the VC Formal setup.

7.11 Saving and Replaying Traces

VC Formal enables you to save and replay its cover and falsification traces across sessions. This enables quick turnaround time when you want to reproduce the failures from a previous VCF run.

Saving Traces

Use the `save_trace` command to save the traces that can be replayed later. By default, traces for non-vacuous failures are saved. Specify the `-full` switch to save all the traces.

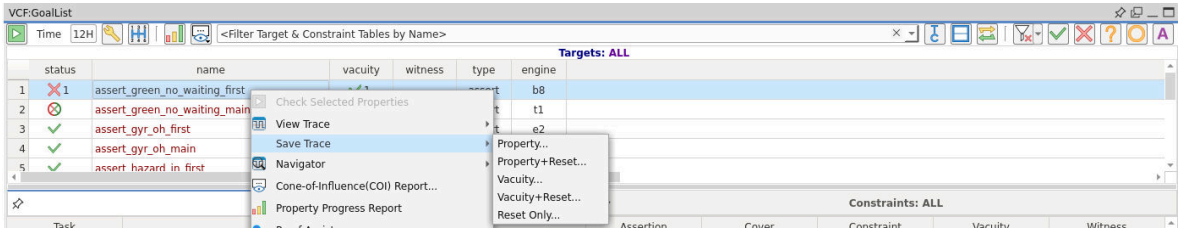
Syntax

```
vcf>save_trace -help
Usage: save_trace    # Save VC-Static Formal traces for later replay with
 replay_trace.
```

Feedback

```
        -dir <tracesDirName>    (Traces will be stored in this directory.)
        [-props <list-of-properties>]
                                (Specifies a list of properties whose
 traces are to be saved)
        [-full]                 (Include witness and vacuity. Also include
 covers (unless -props is used))
        [-filter <expr>]        (filters the collection with the value of
 the expression)
vcf>save_trace -help
Usage: save_trace    # Save VC-Static Formal traces for later replay with
 replay_trace.
        -dir <tracesDirName>    (Traces will be stored in this directory.)
        [-props <list-of-properties>]
                                (Specifies a list of properties whose
 traces are to be saved)
        [-full]                 (Include witness and vacuity. Also include
 covers (unless -props is used))
        [-filter <expr>]        (filters the collection with the value of
 the expression))
```

You can save traces directly from the GUI by right-clicking a property in the GoalList table and selecting the **Save Trace** option from the RMB menu.



Replaying Traces

Use the `replay_trace` command to replay Formal traces saved previously using the `save_trace` command. By default, this command replays all the saved traces.

Syntax

```
vcf>replay_trace -help
Usage: replay_trace    # Replay VC-Static Formal traces saved previously
 with save_trace
        [-property <property-list>] (List or collection of properties)
        -dir <trace-dir-list>  (Traces will be replayed from the
 specified directory list)
        [-cexall]              (Replay all available traces)
        [-parallel <true|false>]   (Enable parallel trace replay. Default
 value is false)
```

Without the `-cexall` option, a saved trace for which the original property has a converged status (falsified assertion or covered cover property) is skipped. The trace does not replay for other properties in that task either.

Feedback

With the `-cexall` option, the trace is replayed even if the original property has a converged status, thus allowing the trace to be replayed against other properties in that task.

Examples

- The `replay_trace` command allows you to specify multiple directories with the `-dir` switch. When multiple directories are specified, traces are replayed with each directory processed sequentially.

```
vcf>save_trace -dir trace1 -props traffic.chk.assert_green_no_waiting_first
1
vcf>save_trace -dir trace2 -props traffic.chk.assert_green_no_waiting_first
1
vcf>save_trace -dir trace2 -props traffic.chk.assert_honor_waiting_first
1
vcf>save_trace -dir trace2 -props {traffic.chk.assert_honor_waiting_main traffic.chk.cov_green_with_waiting_on_main}
1
vcf>fvclear
1
vcf>replay_trace -dir {trace1 trace2}
[Info] FML_PROP_ENABLE_COUNTS: Properties selected with the -property switch: #Assert:18 #Cover:8 #Assume:3.
       Proof constraints are unchanged; selecting assumptions targets their vacuity and witness goals.
[Info] REPLAY_NUM_CEX: Scheduling replay of 1 counterexample trace(s).
```

- To replay all available formal traces from trace_dir1 and trace_dir2

```
replay_trace -cexall -dir {trace_dir1 trace_dir2}
```

  Traces should have been saved by previous vcf run before replaying the traces using the following command.

```
save_trace -props {traffic.cov.cov_both_red
 traffic.cov.cov_green_with_waiting_on_first} -dir ./my_local_cexs
```

- To process traces in parallel, use the following command:

```
replay_trace -dir <trace-dir-list> -parallel true
```

  The default value for the `-parallel` option is `false`, which processes traces sequentially.

  The number of `rf1` jobs displayed by the `report_fml_jobs` command is determined by the smaller of two values: the number of traces to process or the number of workers specified with the `set_grid_usage` command (default: 4). If parallel processing is not enabled, the number of `rf1` jobs is set to 1.

- When using the `replay_trace -dir <trace-dir-list>` command with multiple directories, workers are allocated once and remain active until all directories are processed. Thus, reducing the inefficiencies by avoiding repeated worker allocation and deallocation, which can be time-intensive.

  Because Trace Replay has low processing requirements, it is typically sufficient to allocate workers on the local machine where the main VC Formal session is running. Use the `set_grid_usage -type RSH=<N>` command to allocate `N` workers.

Use Model for Saving and Replaying Traces

- Run VCF and save trace.

  ```
  save_trace -dir ./my_saved_traces
  ```

- Debug and fix RTL/FV testbench.
- Invoke VCF with updated RTL/FV testbench.
- As part of the new RTL VCF session, evaluate the checker (FV setup as well) by replaying the trace generated from the older RTL-VCF session.

  ```
  replay_trace -dir ./my_saved_traces
  ```

Example

In the initial VCF run, save the traces:

```
read_file …
<formal_setup>
check_fv
save_trace -prop {p1 p2} -dir "./my_saved_traces"
view_trace -prop p1
exit
```

After changing the design or setup, replay the traces saved in the previous run:

```
read_file …
<formal_setup>
replay_trace -prop {p1} -dir "./my_saved_traces"
```

Note:

When you use the following use model, `save_trace [...] -> fvclear -> replay_trace […]`the status of the properties are updated as follows: *Asserts*: Previously falsified, are marked as falsified. Previously proven, are marked inconclusive. *Covers*: Previously falsified, are marked as inconclusive. Previously proven, are marked as proven. Any inconclusive property remains inconclusive

7.12 Evaluating External FSDB

You can also load FSDB waveform file from external sources into a VC Formal setup and get a report on any falsifications. This feature is only planned for compatible top level in the FSDB. Meaning, the design top provided with read_file/ elaborate should be compatible with the top level of the design hierarchy in the FSDB.

Feedback

Use the `replay_trace_fsdb` command to replay a trace from FSDB. This command can automatically spawn a new fvtask and update results in that fvtask if the switch -newtask is passed. The name of the task is generated automatically by the tool.

Syntax

```
replay_trace_fsdb  -f  <path_to_fsdb_file> [-newtask] [-property
 collection of properties]
```

In an errorless execution, this command reads the fsdb, and update the status and internal trace of the specified properties that get falsified or covered by replaying the input sequence specified in the FSDB.

7.12.1 Use Model

- Save the trace generated from external source (any other tool).
- Invoke the VCF setup.
- Evaluate the VCF setup by loading the external trace.

  ```
  replay_trace_fsdb -f ./external_tool.fsdb
  ```

  [Info] FORMAL_I_CREATE: Create Formal Model:dutt.

  [Info] FORMAL_I_RUN: Starting formal verification for replay_trace_fsdb

  Id: 0 Goals: 216 Constraints: 0 Block Mode: true

  [Info] APP_LIC_CHKOUT: Checkout 1 app license(s).

  [Info] PROP_I_RESULT: i_mask_test_block..ast_rw_bdr_initial_check falsified 00:00:00

  [Info] PROP_I_RESULT:i_mask_test_block.ast_rw_bdr_initial_check falsified 00:00:00

7.13 Signal Groups for Debugging

The VC Formal and Verdi tools provide intelligent guidance for a replay trace opened up in the waveform viewer that lets you quickly identify the cause of the failure that the replay represents. VC Formal pre-loads signals in the replay waveform display, ordered/ranked based on the likelihood of each signal causing the property to fail

This means that you can directly load the trace and start debugging without first opening the signals, or choosing which signals to add. The pre-loaded signals provide a thorough understanding of the assertion firing and what stimulus the formal tool provided to create the firing. You only need to start adding additional signals to the waveform display when you are actively debugging the design once you understand the firing.

The following signal or signal groups are created in the Verdi's *nWave* waveform.

Note:

Some signal groups are collapsed by default to reduce clutter. These are likely to be the next signals that you might need to see to continue the debug process.

- Reset Marker

  The reset marker that marks the end of the reset. This is for combined trace.

- Support Signal Group (Expanded by default)
  - All the signals in support of the property.
  - The actual design signals in the support of the property.
- Last-Toggled Input Signal Group (Collapsed by default)

  This is a heuristic to detect input signals that have a sudden increase of toggle counts in the last W cycles to the assertion failing cycle. Top K such signals are displayed. By default, K is 3 and W is 5.

- Most-Toggled Input Signal Group (Collapsed by default)

  The top K most-toggled input signals of the trace, where the signal is in the COI and it is not part of a vector with bit-width greater than 3. By default, K is 5.

- Most-Toggled Registers Group (Collapsed by default)

  The top K most-toggled sequential signals of the trace in the COI. By default, K is 5.

- Last-Toggled Registers Group (Collapsed by default)

  This is a heuristic to detect sequential signals that have a sudden increase of toggle counts in the last W cycles to the assertion failing cycle. Top K such signals are displayed. By default, K is 3 and W is 5.

- X-Injection Signal Group (Collapsed by default)
  - Uninitialized registers.
  - Undriven signals
    - BB outputs
- X-Drivers (Fail Nodes)
  - Snip drivers
  - Combinational loops

    - Multi-drivers
    - X assignments
- Constant Signal Group (Collapsed by default)
    - Input signals in the COI that have constant values in the trace.
    - Input signals in the COI that are constrained by some invariants, for example, a==b. (Optional)

For each of the groups, the order of signals is activity based, that is, the last toggled.

8

Managing Formal Work Environment

This section describes how you can effectively use the VC Formal work Environment.

- Creating Verification Tasks
- VC Formal Regression Mode Accelerator
- Saving and Restoring Sessions
- Support for Synopsys Testcase Packager (STP)
- VC Formal Setup Wizard

8.1 Creating Verification Tasks

A verification task encapsulates the setup information for a particular invocation of the `check_fv`command. There is always one default verification task and the initial setup commands apply to the default task.

When a new task is created, some or all of the setup information is copied into the new task. The subsequent setup commands apply to the new task; the previous task(s) retain their prior setup information and are not modified unless the current task selection is reset.

Once multiple verification tasks are created, you can switch between multiple tasks to modify setups, run checks, view results, and debug counter-examples. The design data and any properties compiled as part of the design are common to all tasks. The initial state is global to all tasks.

Each Verification task has an unique set of results. When a new task is created, results from the predecessor task (property or coverage status for example) are not copied into the new task. The results are created in a task only when a `check_fv` command is run in that task. A specific design property may be proven in one task and falsified in another because of differences between the constraints enabled in each task.

This section discusses the following topics:

- Advantages of Using Verification Tasks
- Limitations of Verification Tasks
- Adding Verification Tasks

- Getting a list of Verification tasks
- Getting Report of the Verification Tasks
- Verification Task Use Model
- Using Verification Tasks in the VC Formal GUI
- Running Parallel check_fv Commands for Different Tasks
- Controlling Task Specific Variables and Attributes

8.1.1 Advantages of Using Verification Tasks

During design verification with formal tools, there are many situations where the problem setup must be partitioned or temporarily modified. Some examples include:

- Partitioning a group of properties into subsets to be solved separately.
- Assume-guarantee methodology where a property is proven in one formal run and then used as a constraint to help prove other properties.
- Temporary over constraint to improve formal engine performance or validate property correctness. In other runs, these constraints may be removed.
- Trials with various effort levels or resource limitations.

All of these processes can be performed by copying and editing VC Static project scripts, but it is much more convenient and efficient to use the Verification Task feature to encapsulate the concept of multiple derivative verification scenarios in a single script.

8.1.2 Limitations of Verification Tasks

The following are the limitations of Verification tasks.

- Verification tasks are supported only for formal properties, formal coverage, and sequential equivalence. The other static applications in VC Static do not support tasks.
- All verification tasks share the same design data. A single invocation of `vcf`can only have one `read_file`command.
- All verification tasks share the same initial state.
- The script properties created by the `fvassume -env` and `fvassume -stable`are global to all tasks.
- Script property names must be unique across all tasks.

Feedback

- Clock and reset specifications are global to all tasks.
- Only one task at a time can run `check_fv`.

8.1.3 Adding Verification Tasks

To add a verification task, use the `fvtask` command. The default initial task cannot be deleted. The default task name is dependent on the current application, if application mode is set to FPV, the default task name is FPV. Similarly, if the application mode is set to COV, the default task name is COV and so on.

You can have an alias for each application default task. It can be renamed using the following command:

```
set_app_var fml_default_task_alias <aliasname>
```

Syntax

```
vcf> fvtask -help
Usage: fvtask    # command to set/create/delete a user verification task
        [-create]              (create a new task and make the active
 task)
        [-delete]              (delete the specified task)
        [-limit_copy]          (limits the copy to
 only the -asserts/-assumes/-covers and the specified
 -constants/-snips/-changeats/-abstractions. Requires -asserts, -assumes,
 or -covers to be specified.)
        [-copy <copy_task>]    (copy objects of the specified task to the
 current/new task)
        [-appcopy <app_name>]  (copy objects from the specified app to
 the current task)
        [-attributes <name/value pairs list>]
                               (list of attribute value pairs)
        [-asserts <property-list-or-collection>]
                               (List of names or collection of assertions
 to be copied from the specified task.)
        [-assumes <property-list-or-collection>]
                               (List of names or collection of
 assumptions to be copied from the specified task.)
        [-covers <property-list-or-collection>]
                               (List of names or collection of covers to
 be copied from the specified task.)
        [-constants <constant-list-or-collection>]
                               (List of existing constant signals or
 collection of constant signals to be copied to new task)
        [-snips <snip-list-or-collection>]
                               (List of existing snip signals or
 collection of snip signals  to be copied to new task)
        [-changeats <changeat-list-or-collection>]
                               (List of existing changeat signals or
 collection of changeat signals to be copied to new task)
```

Feedback

```
        [-keep_status]          (Keep the property goal status on task
creation from the parent task)
        [-keep_task]            (Keep the original task as the
active task. If task_name is provided, the specified task will be
created/modified, but will not be set as the active task.)
        [<task_name>]           (task name (with no options sets the
current active task))
```

Example

- To create a new task:

  ```
  fvtask -create <taskName>
  ```

  The new task becomes the current task and subsequent setup commands apply to this task. It is an error to create a task with the same name as another task. By default, a new task inherits all of the configuration data from the previous task, but a variety of switches are provided to control exactly which optional data are copied.

- To limit the assertions enabled in the new task to those matching a specific name:

  ```
  fvtask -create <taskName> -asserts top_check*
  ```

- Though the default source of setup information for a new task is the previous current task, you can copy setup data from a different task with the `-copy` option:

  ```
  fvtask -create <taskName> -copy <task_name>
  ```

  Note:

  The fvorc settings is copied from the parent task to the child task by default while copying the task.

- To delete a non-default task which is no longer needed:

  ```
  fvtask -delete <taskName>
  ```

- To keep the original task as the current active task instead of switching to the new task once it is created:

  ```
  fvtask -create <taskName> ... -keep_task
  ```

Note:

Task-specific attributes are not copied to the new task unless explicitly listed with option '`-attributes`'. The following message is reported by the `fvtask` command when the `-limit_copy` or `-attributes` options are used during task creation.

[Info] FV_TASK_LIMIT_ATTR_COPY: Either option '-limit_copy' or '-attributes' is used. Task-specific attributes will not be copied to the new task unless explicitly listed with option '-attributes'.

Feedback

8.1.4 Getting a list of Verification tasks

To get the name of the verification tasks created, use the `get_fvtask` command.

Syntax

```
vcf> get_fvtask -help
Usage: get_fvtask    # Returns a collection of fvtasks
        [-filter <attribute>]  (Returns only the tasks filtered by
 <attribute>)
        [tasks]                (List of task names)
```

Example

- To retrieve the current task:

  ```
  get_fvtask
  ```

- To retrieve all the tasks:

  ```
  get_fvtask *
  ```

8.1.5 Getting Report of the Verification Tasks

To get a report of the statuses of all the verification task, use the `report_fvtask` command.

Syntax

```
vcf> report_fvtask -help
Usage: report_fvtask    # Returns a report of the specified tasks
        [-filter string]       (Filter by attribute)
        [-xml]                 (List the task report in xml format)
        [tasks]                (List of task names)
```

Example

- The following command prints a summary of the tasks matching the selection criteria:

  ```
  report_fvtask *
  Formal Verification Tasks
  Task Name          Status
  ---------------- ---------
  _default
  mycov
  myprops            (active)
  ```

8.1.6 Verification Task Use Model

The following is an example of the Verification task use model:

```
#-- global app vars
set_app_var fml_mode_on true

#-- compile and load the design
read_file -top dut -sva -format -sverilog -vcs "…"

#-- Clocking and reset
create_clock clk -period 100
create_reset resetn -sense low

#-- generate the reset state
sim_run -stable
sim_save_reset

#-- set up task to handle assertions and run
fvtask prop_task -create -copy FPV -asserts * -assumes *
   -attributes {{fml_max_time 1H} {fml_witness_on true}}
set_grid_usage -type RSH=6

check_fv -block
report_fv -list

#-- create a new tasks to look at unconverged properties
#   and add new pin constraint and then run with new resources
set unconv [get_props -status { timed_out bounded }]
fvtask prop_unconv -create -copy prop_task -assumes * \
   -asserts $unconv \
   -attributes {{fml_max_time 8H} {fml_orc_tactic prop_heavy}}
set_constant free_config -value 0
check_fv -block
report_fv -list
```

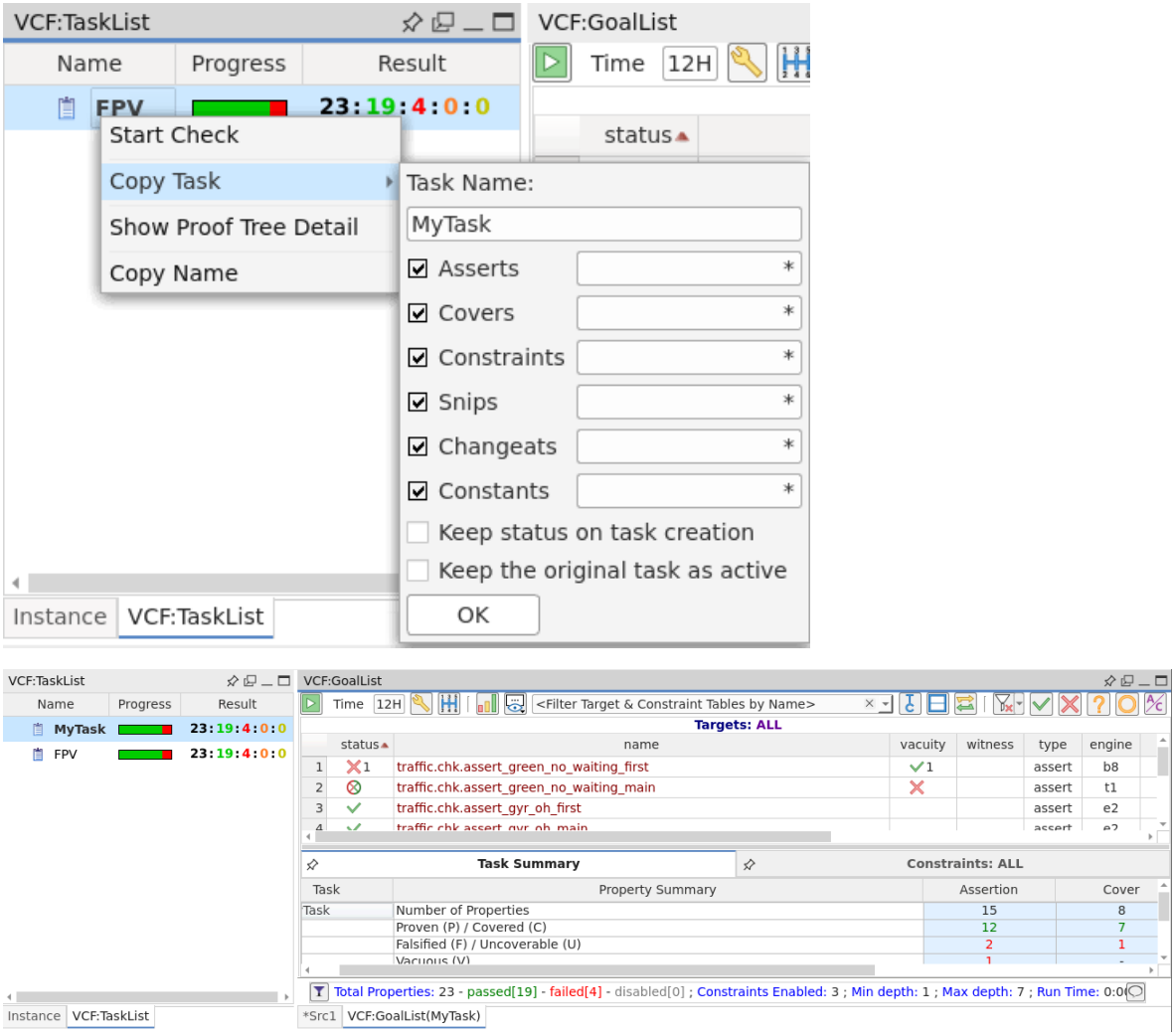
8.1.7 Using Verification Tasks in the VC Formal GUI

Verification tasks can be created, selected, and managed in the VC Static GUI as well as from the TCL script interface. As new tasks are created, they appear in the *Instance* tree on the left side of the window. The task summary appears in the *Task List* pane as shown in #unique_49/unique_49_Connect_42_i1048773.

Right-click a task, and select *Copy Task* for creating new tasks and selecting the properties and constraints that must be included in that task.

Feedback

Figure 51: GUI for Creating Verification Task



Using the TaskList

The TaskList summarizes the activity of a property for a specific application mode. The TaskList is a separate view that can be docked in the area of the Verdi instance tree as shown in #unique_49/unique_49_Connect_42_i1048785.

By default, the *FPV* application mode is shown in the default task. If you specify the `-fmode` option or `set_fml_appmode` command, the specified application mode is set to the current view as the default task (for example, AEP, SEQ, CC, and COV).

Note:

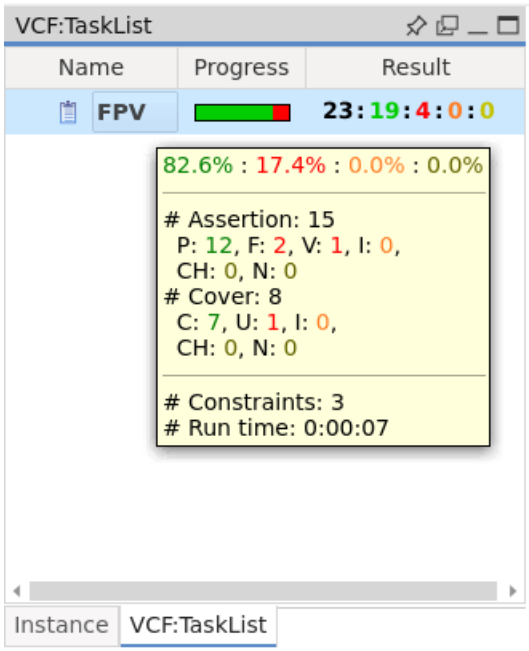
You can view only one application mode at a time in the TaskList.

Feedback

From the TaskList:

- You can view the property progress of a given task in the TaskList. The total count and the counts of all the different categories are displayed in the TaskList. Different colors are assigned to different results as:
  - Constraint count.
  - Success count (green) including Proven, Covered, and Connected.
  - Failure count (red) including Falsified, Uncoverable, Unconnected, and Vacuous.
  - Inconclusive count (Orange)
  - Checking and Not-run (yellow)
  - Disabled count.
- When you create a new task for an application mode, the new task is always set as the active task for the new application mode. The other tasks in the same application mode are listed below the current active task. To make a task as the current active task, double-click the corresponding task item.
- The default filter groups are shown, however, you can toggle them from the RMB menu. These settings are saved in a session.
- You can view details like the allocated runtime, elapsed time, and worker status using the tooltip of a task node as shown in Figure 52.

Figure 52: Tool Tip of the TaskList



- You can perform the following from the RMB menu of a task item:
  - Switch application mode, that is, you can set a specific application mode to the current view with its own default task (for example, AEP, SEQ, CC, and COV).
  - Start the check. When you start a check, the `check_fv` command runs to check the current task.
  - Copy a new task based on a selected task.
  - Import user-defined groups.

    Note:

    You cannot run multiple tasks in parallel and you cannot see the orchestration recipe in the GUI.

8.1.8 Running Parallel check_fv Commands for Different Tasks

You can run multiple `check_fv` commands in parallel for different tasks. For example, you can start a `check_fv` command on all properties and as the results of these commands are received, you can start navigator tasks on specific failing properties while the original run continues.

You can:

- Run multiple `check_fv` commands on different tasks in parallel. However, you can run only one check per task.
- Switch between tasks during a `check_fv`command, when the check is run as an asynchronous thread.
- Create new tasks while an existing task is running.
- View the summary of the updates for all running tasks.
- Control the number of parallel `check_fv` tasks that can be run.

  You can use the `async_command_max_parallel_count` application variable to control the maximum number of `check_fv` commands that can be run at the same time. By default, one `check_fv` command can be run at a time. To enable multiple parallel `check_fv` commands, set the value to a bigger number. You can set any value between 1 <= val <= 64.

  Example

  ```
  set_fml_var async_command_max_parallel_count 4
  ```

Feedback

8.1.9 Controlling Task Specific Variables and Attributes

Any formal application variables set using the `set_app_var`command are global to all tasks. To enable task-specific variable values, the `set_fml_var` command is provided. For example, to turn off vacuity checking in the current task, use the following command:

```
set_fml_var fml_vacuity_on false
```

The `get_fml_var` and `report_fml_var` commands are provided to query a specific formal variable or generate a report of all formal variables. Table 12 lists some task-specific formal variables.

Refer to Appendix F - Formal Variables for the language.complete list of documented formal variables.

Table 12: Tasks Specific Variables

Attribute Name	Value Type	Default Value	Constraints
fml_conflict_check_depth	int	2	
fml_conflict_debug_minimal	bool	true	
fml_conflict_debug_reset	bool	false	
fml_cov_gen_trace	string	default	{default on off}
fml_max_mem	string	8GB	
fml_max_time	string	-1H	
fml_min_unk	int	0	
fml_progress_time_limit	string	-1H	
fml_quiet_trace	bool	true	
fml_orc_tactic	string	prop_check	
fml_vacuity_on	bool	true	
fml_witness_on	bool	false	

It is also possible to access task variables as attributes on the task object. For example, the following command returns the value of the `fml_vacuity_on` variable:

```
get_attribute [get_fvtask] fml_vacuity_on
```

Feedback

To get a list of complete attributes for a task, use the following command:

```
list_attributes -application -class task
```

It is also possible to use the `fv_task` command to set attributes on a task. For example, the following command sets the current task to task2 and sets the maximum memory available for formal searches to 2Gb.

```
fvtask task2 -attributes {{fml_max_mem 2GB}}
```

When you set formal variables using the `set_fml_var` command, values apply to the current task only in most cases. That implies that they apply only to the current application mode as well. For example, if you set a formal variable in FPV task, the value applies to the FPV task in the FPV application mode only. If you want the formal variable to apply to all tasks and application modes, use option `-global` with the `set_fml_var` command. You can always revert the behavior of a formal variable to apply to a specific task by setting the formal variable again without the `-global` option.

Note that some formal variables are global by default and their value applies to all tasks. The `report_fml_var` command identifies such formal variables with a (global) string in their description.

Example

```
vcf> report_fml_var fml_defer_property_compile
Attribute                       Value      Type    Default
 Constraints
------------------------------- ---------- ------- ----------
 ----------------------
fml_defer_property_compile      false      bool    false      (global)
```

By default, most of the formal commands apply only to the current task. Therefore, if you want the command to apply across all application mode, you must specify the `-global` option in the commands. The following commands support the `-global` option:

- set_fml_var
- set_constant
- snip_driver
- set_changeat
- fvassert
- fvassume
- fvcover

Note:

For improved usability, the `set_fml_var`, `report_fml_var`, and `get_fml_var` commands also apply to FV (Formal Verification) application variables as listed with `report_app_var -app FV`. These variables are always applied globally by default.

8.2 Saving and Restoring Sessions

VC Formal enables you to save a compiled design and then restore it whenever you want to. This enables you to save considerable runtime when multiple sessions are used to analyze a design. For example, if you save and exit one session, you can display schematic paths after reloading the design database. Save and restore allows a session to be restarted without incurring the burden of reloading the design. This results in a significant performance boost on subsequent design reloads.

Another benefit comes from the ability to fork a session for exploring several different scenarios. In this model, you can load your design, save the session, make copies of the session, and then for each variation, restore a copy and continue with your tests. VC Formal does not automatically save the session when you quit. When you quit from the vcf, the current session results is not automatically saved. If you wish to save the session set up and run data, use `save_session` command.

```
vcf -f tcl -verdi
vcf> save_session
```

Or

```
vcf -f tcl -verdi
vcf>save_session -session S1
```

The information from the run is then stored in the *vcst_rtdb*folder in the current working directory. You see the following message on the screen.

```
[Info] SAVING_SESSION: Saving session (vcst_rtdb).
[Info] SAVE_I_FACET: 'FV' saved.
[Info] SAVE_I_FACET: 'design' saved.
```

The information from a previous run in the `vcst_rtdb` folder can be restored with a new invocation of the `vcf`.

```
vcf -restore -verdi
[Info] RESTORING_SESSION: Restoring session (vcst_rtdb)
```

Or

```
vcf -verdi
vcf>restore_session -session S1
```

Feedback

All the data from the previous run can be obtained using query time commands such as `report_fv, get_props, report_formal_source,` and so on.

There are some limitations to the save and restore feature. Not eve rything from previous session is saved. The `sim_force/sim_monitors` are not restored. Sim is in reset state and Tcl variables are also not restored.

Saving the session upon exiting can take some time. This may not be desirable in the initial setup stage when frequent changes are required. An option to the quit command tells the tool to bypass the save session step and exit quickly.

```
vcf> quit -force
```

Note:

The Tcl variable `is_in_restored_session` is set to 1 in a successful restored session. By default, it is set to 0.

8.2.1 Saving and Restoring Sessions in VC Formal GUI

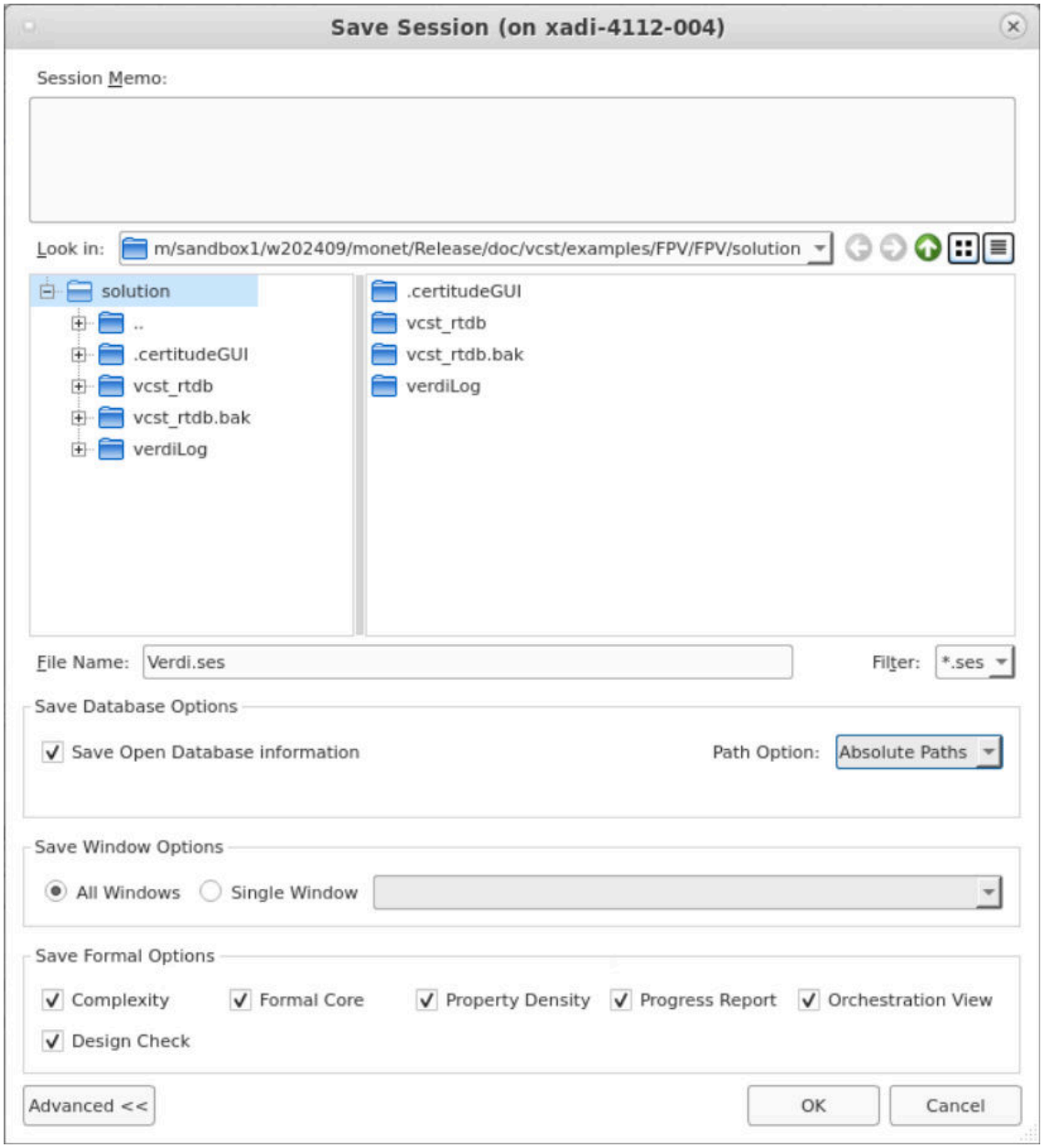
You can also save and restore sessions in the VC Formal GUI.

To save a session

1. From the Verdi Menu bar, click **File> Save** Session.

   The **Save Session** dialog box appears.

Feedback



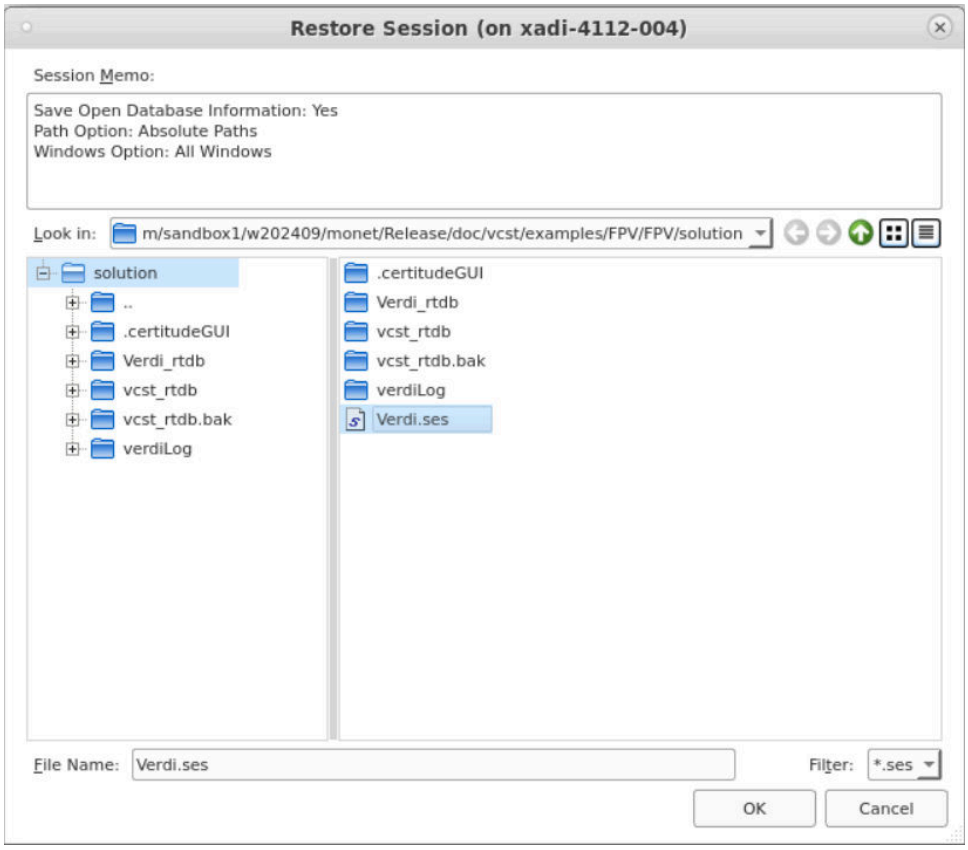
2. Browse to the path where you want to save the **Verdi.ses** file.

3. Choose from the required options to be saved:
   - **Save Database Options**: Choose from the relative path or the Absolute paths to be saved.
   - **Save Window Options**: Choose the windows you want to save.
   - **Save Formal Options**: Choose the VC Formal reports/checks that you want to save.
4. Click **OK**.

To restore a session

1. Open the vcf in the GUI mode.
2. From the Verdi menu bar, click **File > Restore** Session.

   The **Restore Session** dialog box appears.



3. Browse to the path and select the **Verdi.ses** file. Click **OK**.

Feedback

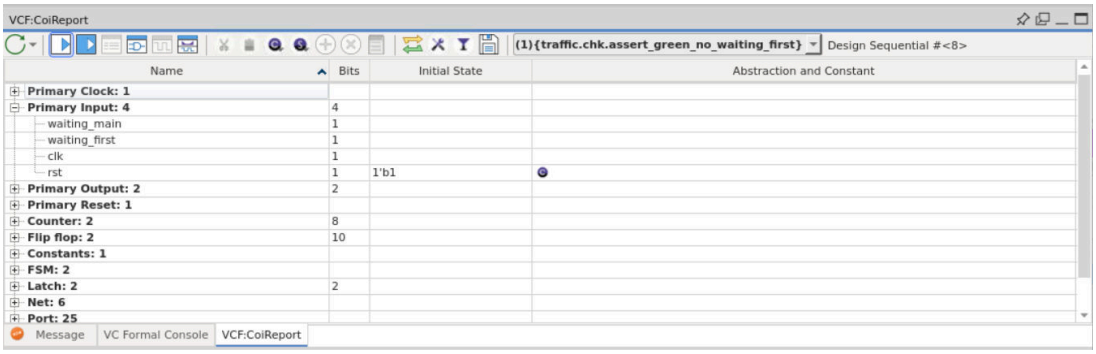
4. If you want to restore a session from a different user, then you must copy all *Verdi*.ses* (the file that was saved while saving the session) files to new directory, and then perform steps 1 to 3.

```
cp -r <>/Verdi*
```

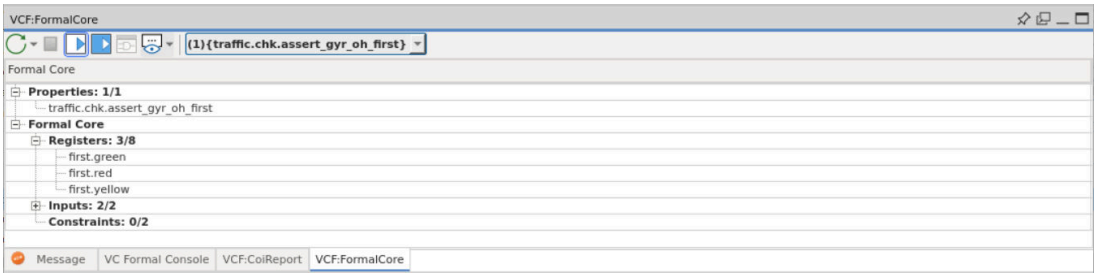
8.2.2 Reports Saved and Restored in VC Formal GUI

The following reports are saved and restored when you save and restore a session in VC Formal GUI:

- When you perform a save and restore session in the VC Formal GUI, the restore sessions will also restore the complexity report, formal core, progress report, goal progress, orchestration view, and property density views.
  - Complexity report (COI report) in the VC Formal GUI is restored based on the last design/property by calling `report_fv_complexity <propertyName>` when restoring.

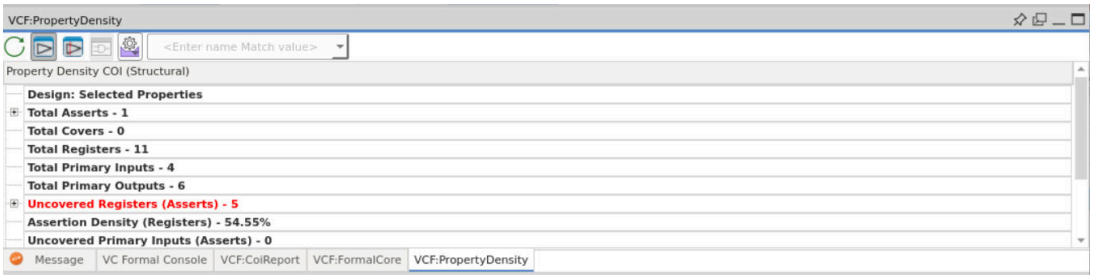


  - The Formal core in the VC Formal GUI is restored based on the last Tcl command showing formal core.

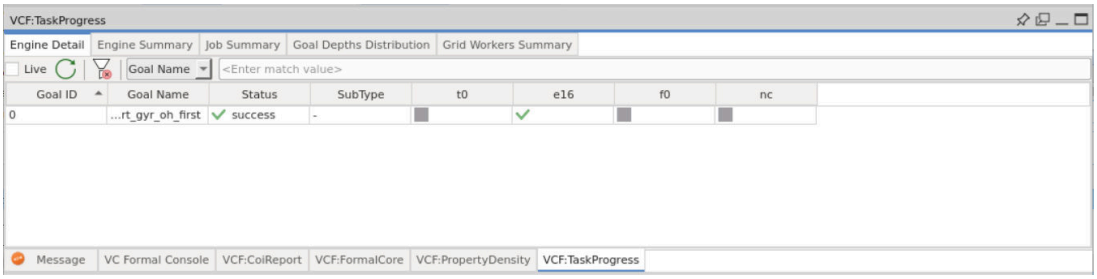


  - The property density in the VC Formal GUI will restore the property density based on the last Tcl command showing property density.

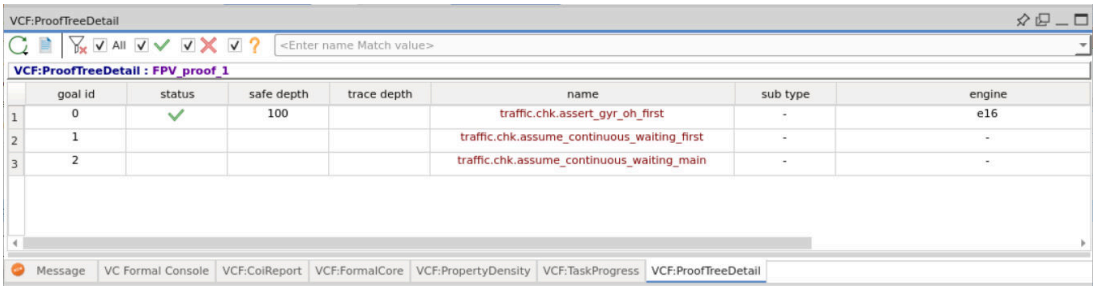
Feedback



- The progress report in the VC Formal GUI will be restored.



- The orchestration view will be restored.



8.3 Support for Synopsys Testcase Packager (STP)

The Synopsys Testcase Packager (STP) is the standard tool for extracting test cases in customer sites and bringing it to Synopsys.

To enable VC Static and Formal users to extract test cases using STP, the `stp_extract` command is introduced. When you use the command, the STP is run, and a compressed file is created.

Syntax

```
vcf> stp_extract -help
```

Usage

```
Usage: stp_extract    # extract the current testcase using STP
        [-path <directory path>]
```

Feedback

```
                                (Directory path to extract the testcase)
       [-exclude_file_path <file path>]
                                (File name which contains the absolute
paths of files to be skipped)
```

Note:

`stp_extract` does not support test case extraction and packing when the VC Formal shell is invoked with the option`-out_dir <dir_name>`. When the `-out_dir <dir_name>` option is used along with `stp_extract`, the following error message is reported:

[Error] VCST_TEST_EXTRACTION_FAILED: Automatic test extraction command failed.

8.4 VC Formal Setup Wizard

VC Formal Setup Wizard is an interactive GUI to guide VC Formal users in creating Tcl project files for their formal verification environment through step-by-step instructions. As it can be difficult for new users to find out all the commands and variables from scratch, the wizard provides an interactive capability to help create the required setup for formal verification.

Note:
The VC Formal Setup Wizard is a Limited Customer Availability (LCA) feature. Contact Synopsys support for more information.

Using VC Formal Setup Wizard, you can:

- Select the target VC Formal application between FPV, AEP, and COV (FCA)
- Select or create a Tcl project file
- Define the application variables and formal variables
- Specify top module name and compile options
- Configure AIP protocols to be analyzed for FPV App
- Configure signoff options for FPV App
- Configure coverage options for COV (FCA) App
- Configure property types for AEP App
- Configure signoff options
- Import design and SVA properties
- Set up clocks, resets, and constants

Feedback

- Configure runtime limits and grid options
- Preview and Edit Tcl commands
- Save and run Tcl commands

Note:

VC Formal Setup Wizard only supports FPV, AEP, and COV (FCA) application modes.

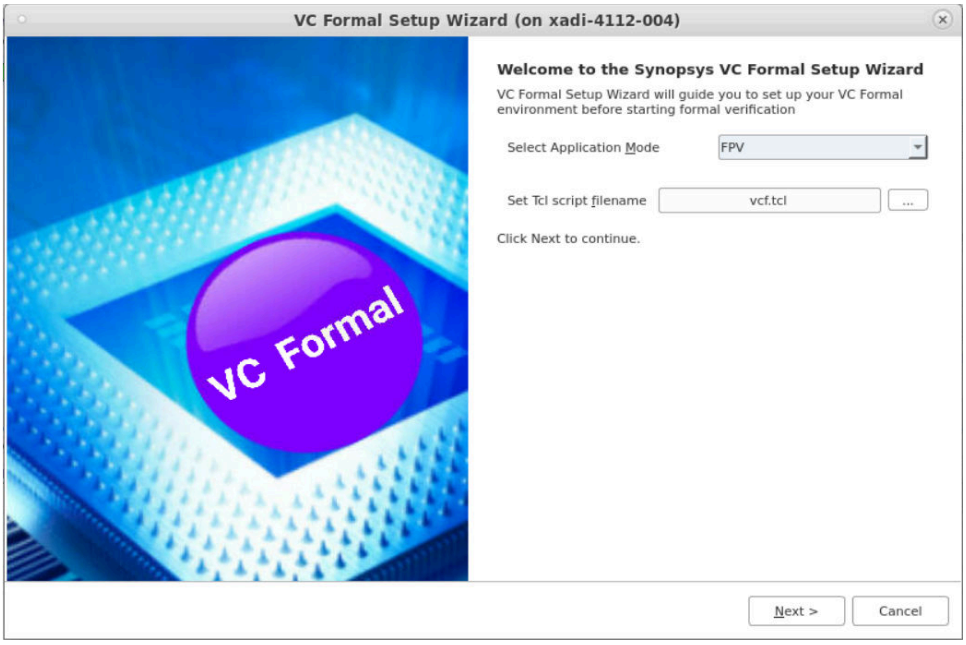
8.4.1 Opening VC Formal Setup Wizard

VC Formal Setup Wizard can be started in the following ways:

1. When VCF GUI is started without loading a Tcl project file, click in the GoalList to open Setup Wizard. Alternatively, click in the Verdi tool bar to open the Setup Wizard.

```
vcf -verdi
vcf -gui
vcf -verdi -out_dir out_dir
```

The *VC Formal Setup Wizard* welcome window appears.

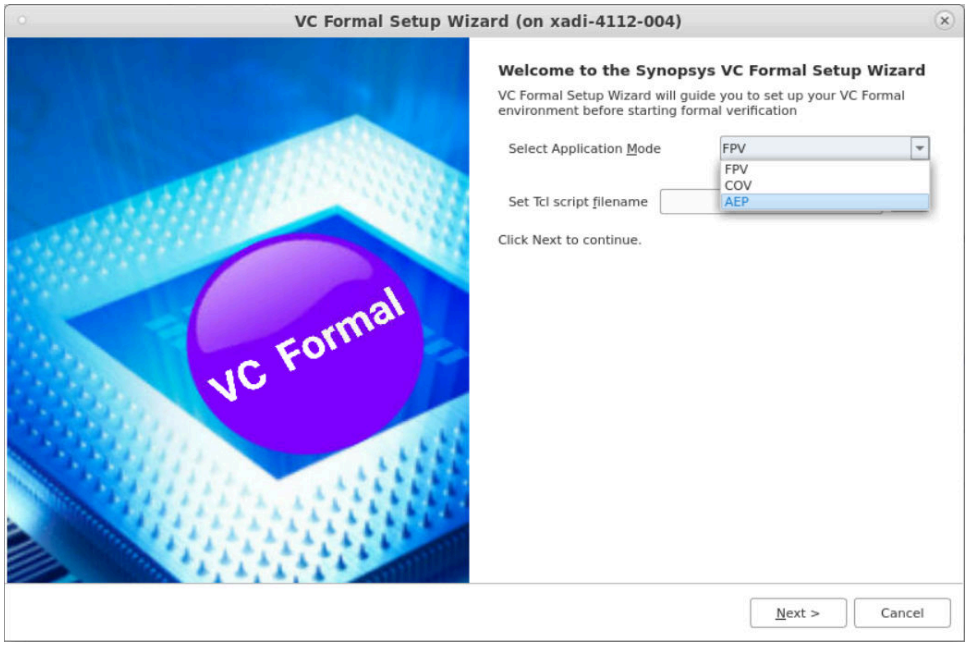


Feedback

Note:

VC Formal Setup Wizard can only show the commands and variables which are newly created. You cannot parse or modify existing Tcl project files using the Setup Wizard.

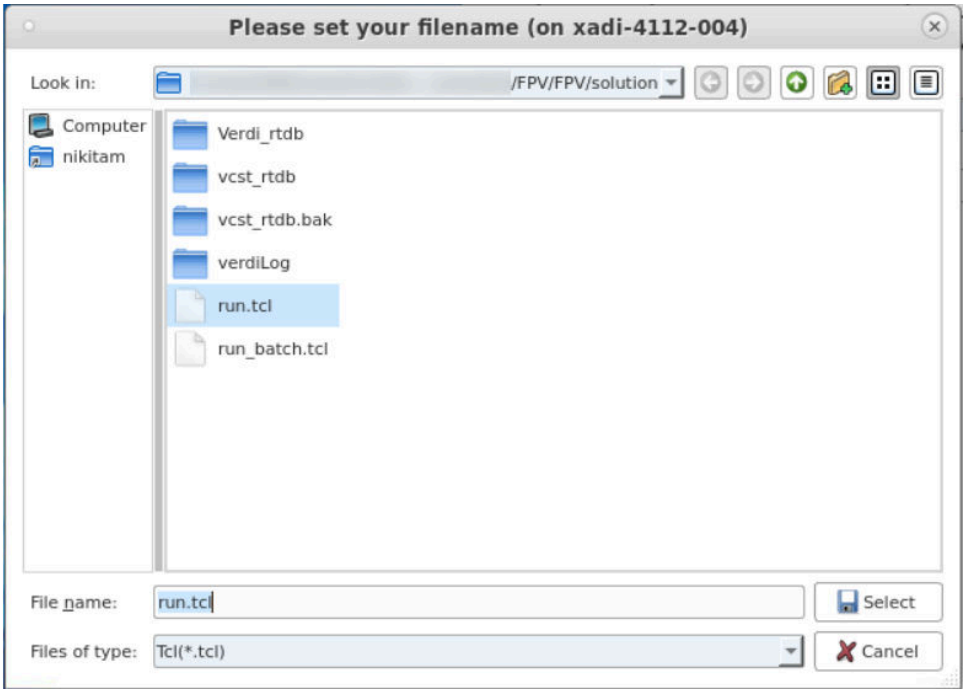
2. From the *Select Application Mode* option, select the target VC Formal application.



3. Browse and select a Tcl project file to be created.

   If the Tcl project file already exists, a dialog box appears asking for confirmation to overwrite it.

Feedback



4. Click *Next* in the Setup Wizard.

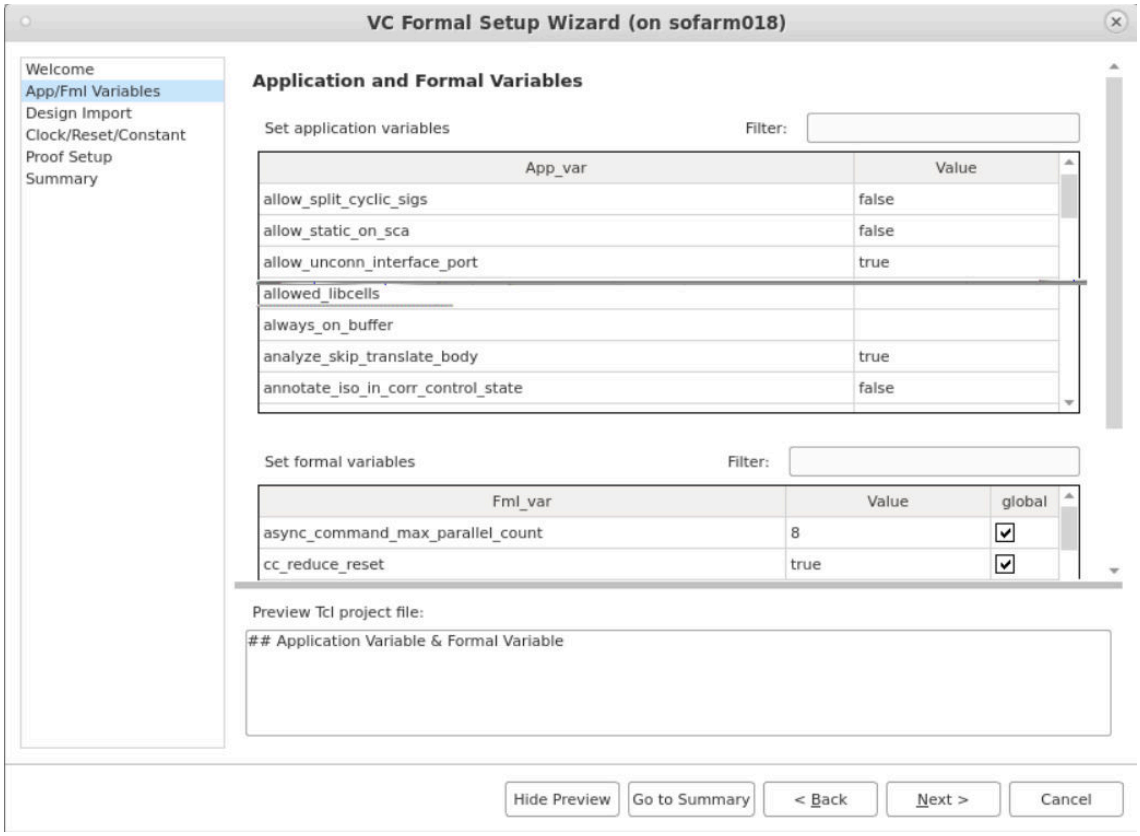
   The *Application and Formal Variables* screen appears.

8.4.2 Defining Application and Formal Variables

In the *Application and Formal Variables* screen, you can change the default value of application and formal variables in the table.

1. Set the default values for the variables required for your Tcl project file.

   These settings correspond to the `set_app_var` and `set_fml_var` commands respectively.



2. Select *global* check box if you want the values to be applied for all tasks.
3. Click *Next*.

   The *Design Import Settings* screen appears.

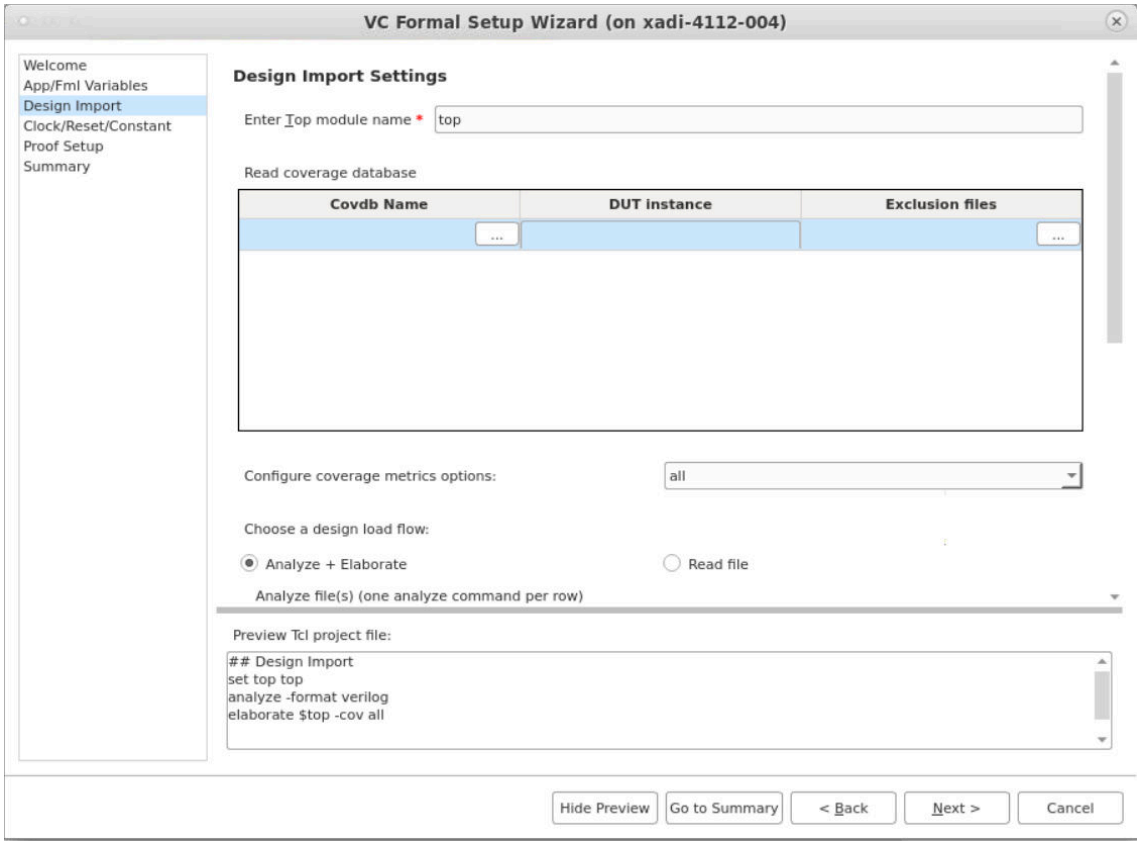
Feedback

8.4.3 Importing Design and Properties

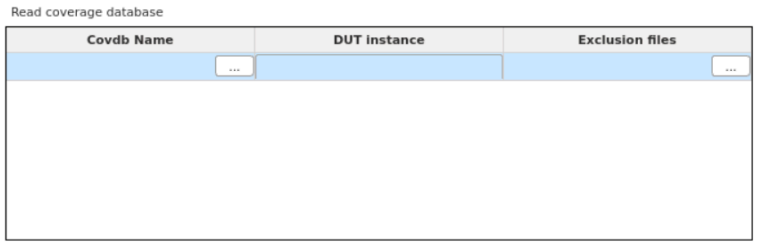
In the *Design Import Settings* screen, perform the following steps:

1. Type the name of the top module.

Figure 53: Design Import Settings



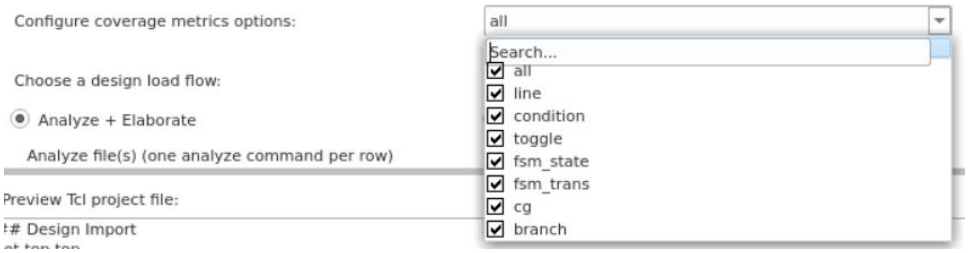
2. Import the coverage database and the exclusion files:



3. Specify the coverage metric options for analysis. You can select multiple signoff options. The `signoff_config -type < all | line | cond | toggle |`

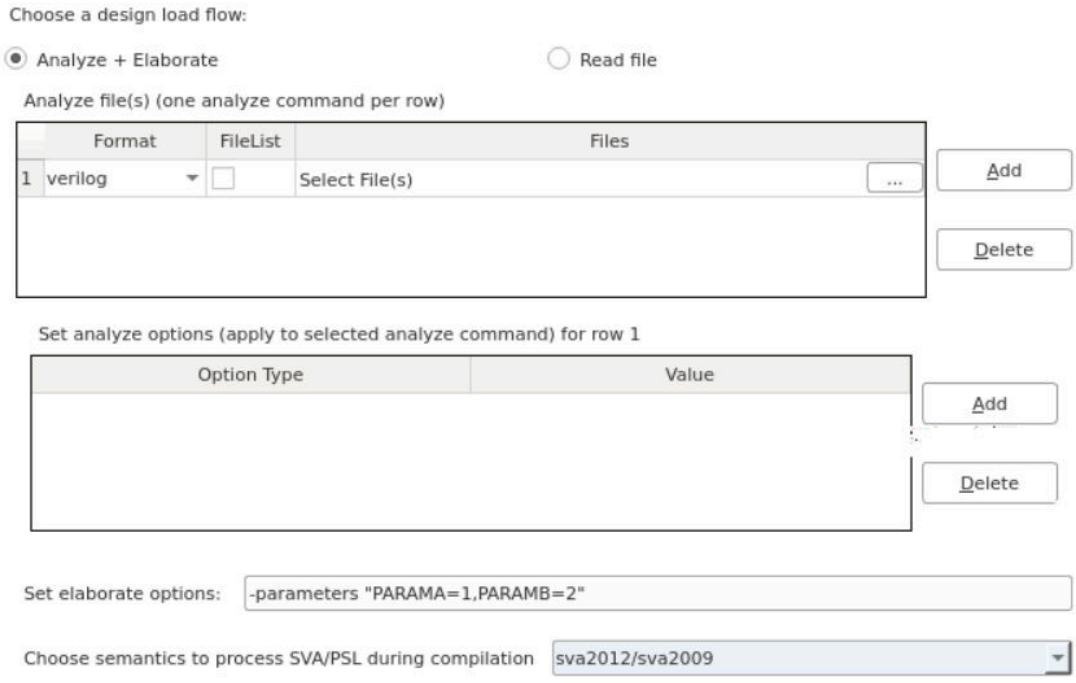
Feedback

`fsm_state | fsm_trans | branch | cg | fault_rtl | fault_conn>` command appears in the Tcl project file.



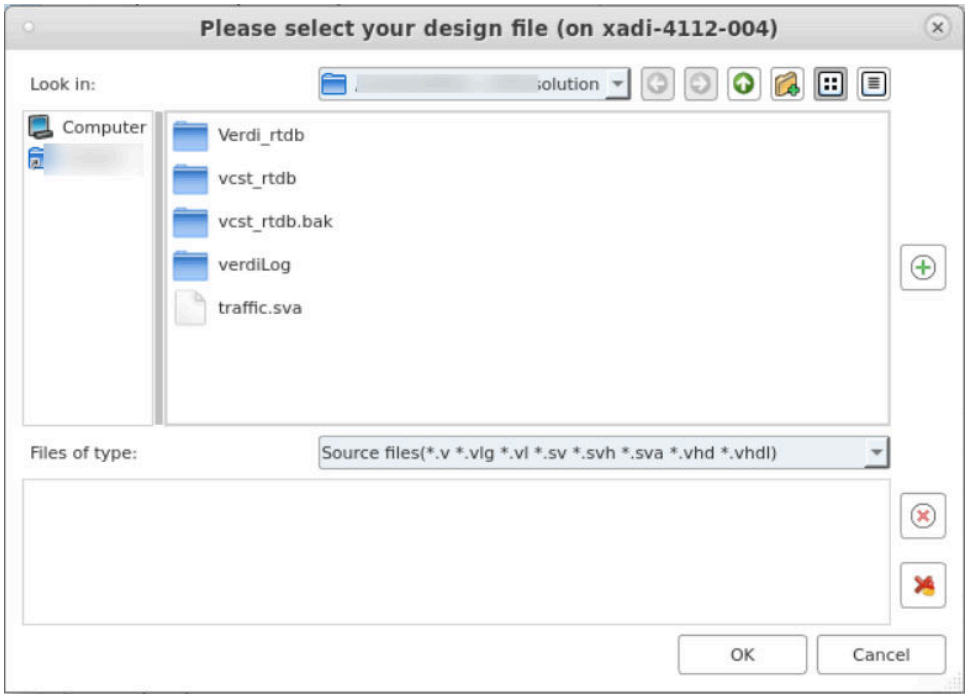
4. To import design and SVA properties, select the language in the *Format* column for each row.

   The `analyze -format <verilog | sverilog | vhdl>` command appears in the Tcl project file.



5. Browse to the design files in the *Files* column for each row.

   The *Please select your design file* window appears.

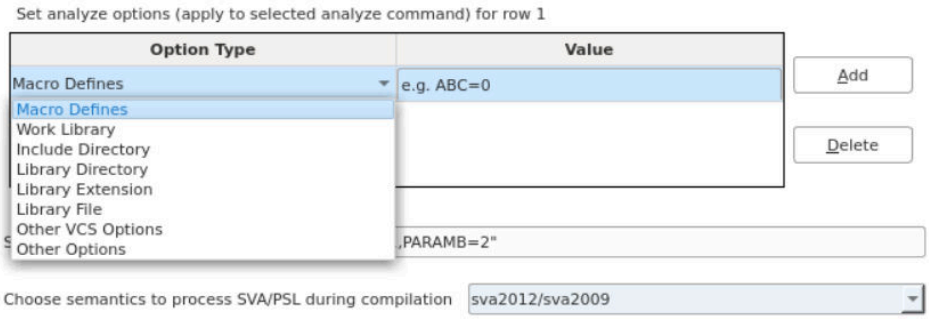


6. Browse to the location of the design files, and click to add the files.

   Click to delete the files if you do not need them.

7. Click *OK*.

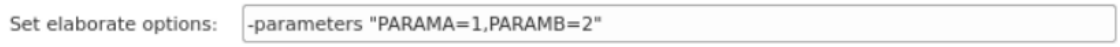
8. Set the analyze options by specifying the required *Option type*, including *Macro Defines*, *Include*, *Directory*, *Library Directory*, and so on, and specify their value.



   Use the *Add* option to add multiple rows and multiple options. Specify the *Value* for each option.

9. Specify any user-defined elaborate options in the *Set elaborate options* box.

Feedback



10. Specify semantics to process SVA/PSL during compilation.

11. To go back to the previous step, click *Back*.
12. Click *Next* to proceed.

    If the configure signoff options are selected, a dialog box appears asking *Do you want to run signoff_config command first?*.

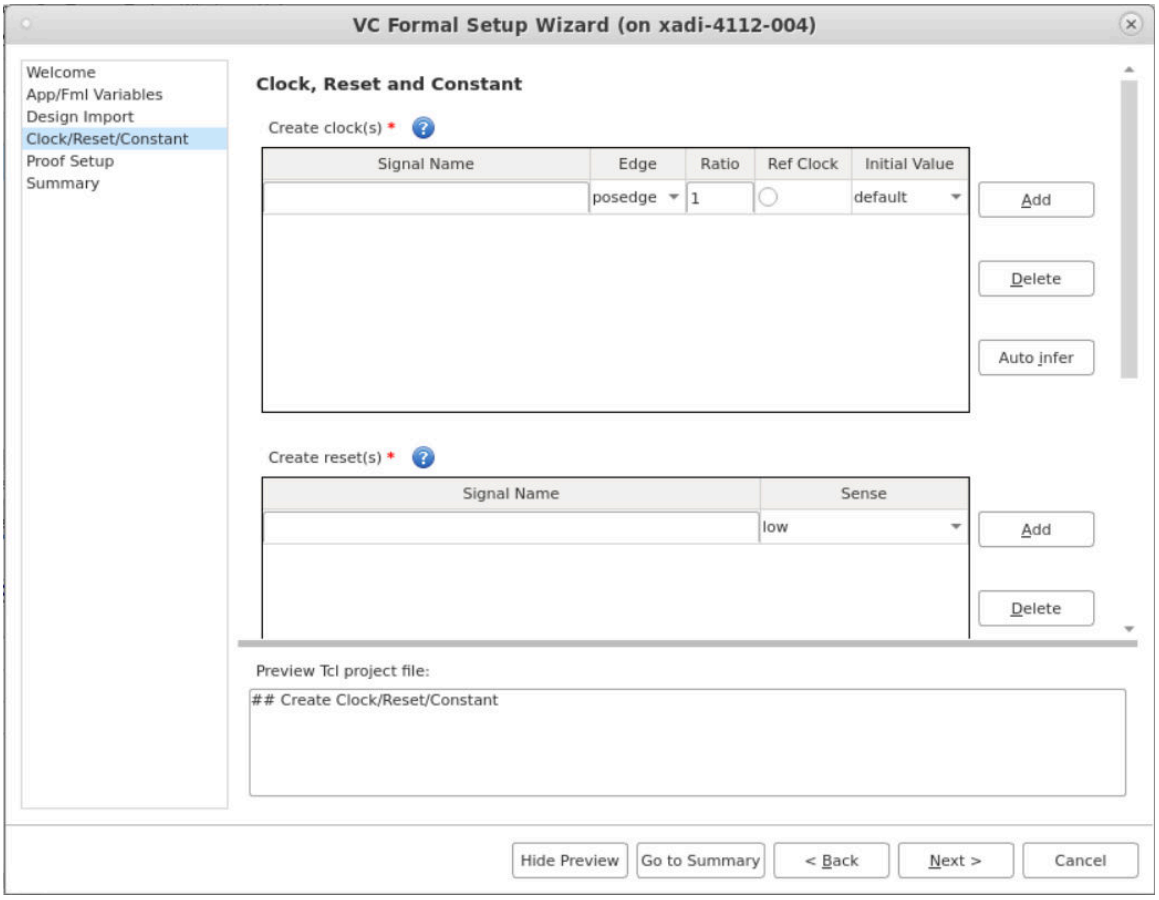
    Select *OK* to run the `signoff_config` command.

    The *Clock, Reset and Constant* screen appears.

8.4.4 Setting Up Clocks, Resets and Constants

In the *Clock, Reset and Constants* screen, you can add, delete, or edit clocks, resets, and constants in their respective table. The settings are updated in the `create_clock`, `create_reset` and `set_constant` commands respectively.

Feedback



Setting Up Clocks

1. To infer clocks first, select *Auto Infer*, and then review the results. To infer clocks automatically, you should load the design first. Click *OK* when the *Auto Infer will run after loading design. Do you want to load design first?* message appears.

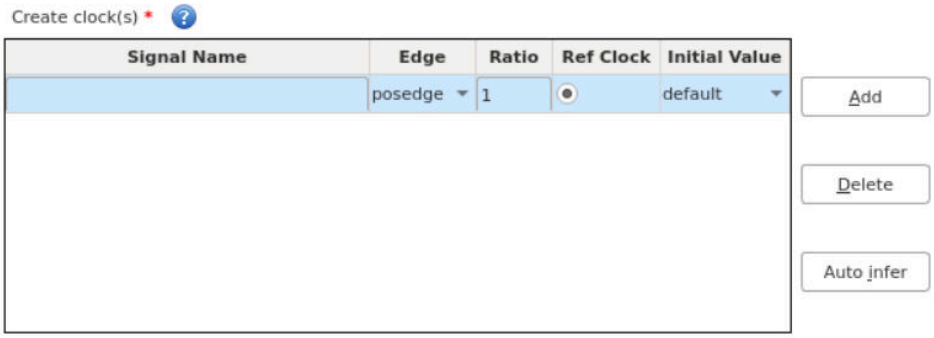
   Note:

   It is not recommended to do Auto Infer and configure signoff options at the same time. The Auto Infer results may not be correct, if fault injection is turned on.

2. To create a clock, click *Add*.
3. Specify the clock name in the *Clock Signal Name* column. You can also drag and drop design signals from nTrace if design is loaded.
4. Specify the clock edge in the *Edge* column.
5. Specify the clock ratio in the *Ratio* column. Clock ratio is set to 1 by default, which corresponds to `create_clock -period 100.`

Feedback

6. Click *Ref Clock* to add command `set_change_at -default` based on that clock.



If you specify same clock/reset more than once, the rows with these signals will be highlighted in red.

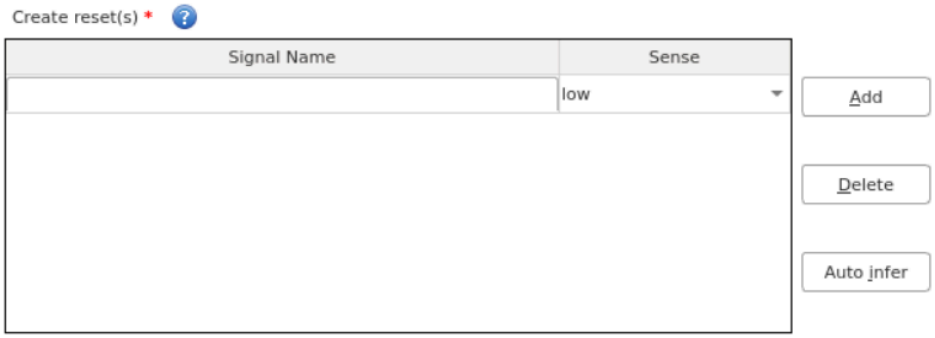
Setting Up Resets

1. To infer resets first, select *Auto Infer*, and then review the results. To infer resets automatically, you should load the design first. Click *OK* when message *Auto Infer will run after loading design, do you want to load design first?* appears.

   Note:

   It is not recommended to do Auto Infer and configure signoff options at the same time. The Auto Infer results may not be correct, if fault injection is turned on.

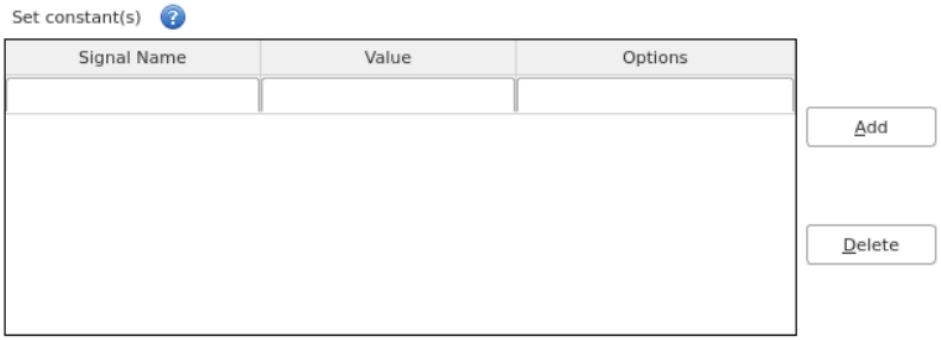
2. To create a reset, click *Add*.
3. Specify reset name in the *Reset Signal Name* column. You can also drag and drop design signals from nTrace if the design is loaded.
4. Specify the reset active level in the *Sense* column.



Feedback

Setting Constants

1. To set a constant, click *Add*.
2. Specify signal name in the *Signal Name* column. You can also drag and drop design signals from nTrace if the design is loaded.
3. Specify the `set_constant` command options in the *Options* column.



4. When clocks, resets, and constants setup is completed, click *Next*.

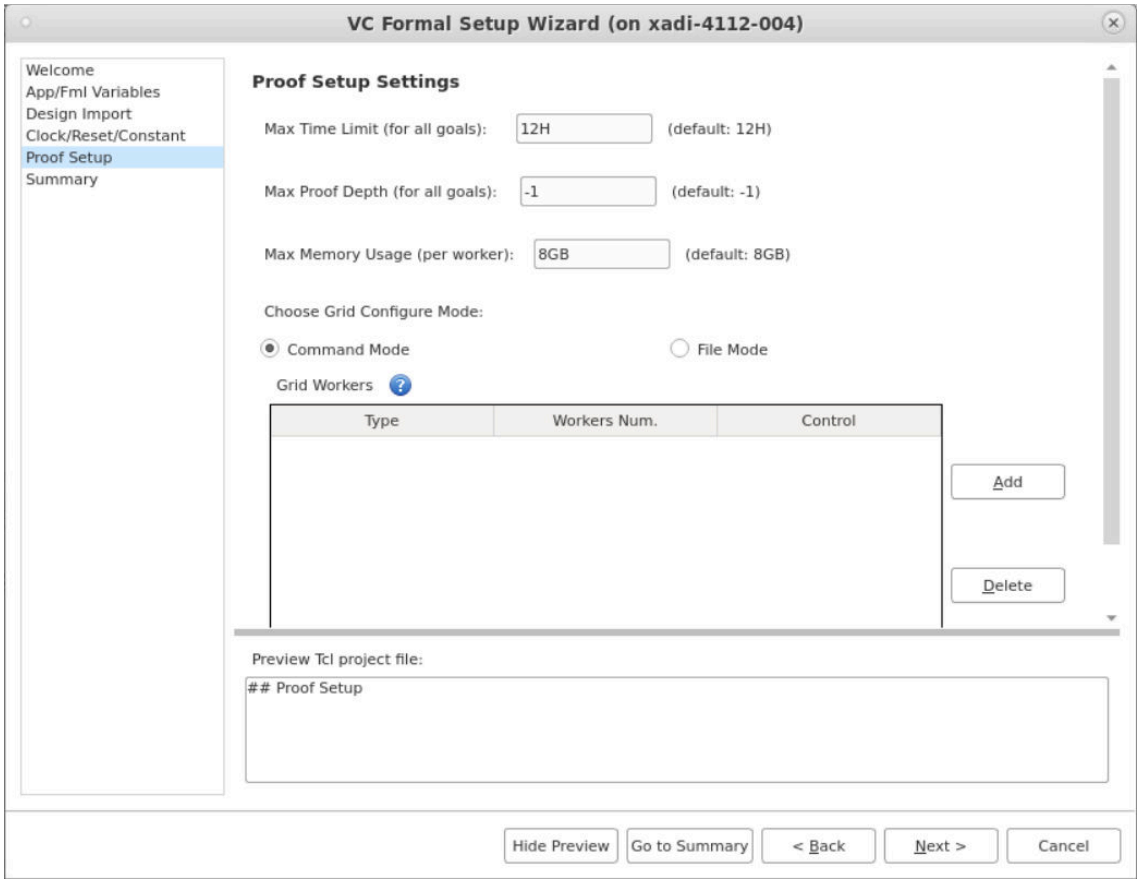
   The *Proof Setup Settings* screen appears.

8.4.5 Setting Up Proofs

In the *Proof Setup Settings page*:

1. Type the maximum time limit for the formal engines to work on the goals. By default, the maximum time limit set is 12H.

   See section for moreSpecifying Maximum Progress Time Limit.



2. Type the maximum proof depth for all goals.

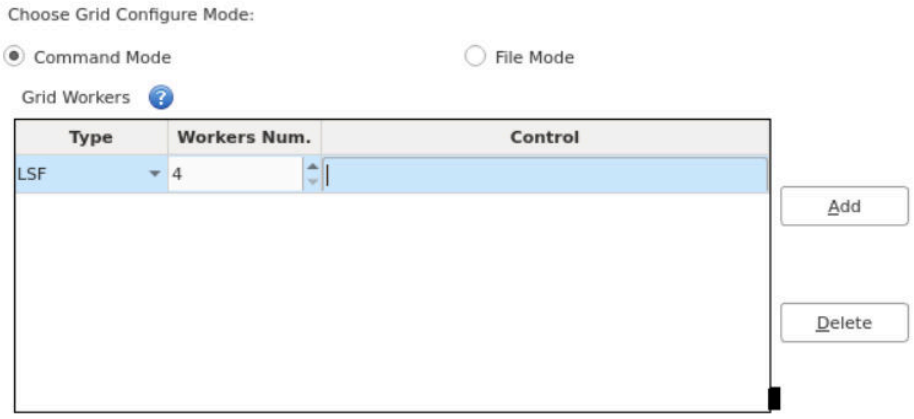
   See section Configuring Orchestration in VC Formal for more details.

3. Type the maximum memory usage per worker. By default, the maximum time limit set is 16GB.

4. To configure the grid in command mode, select *Command Mode*.

Feedback

a. Click *Add*.



b. Select the type of grid to be configured.

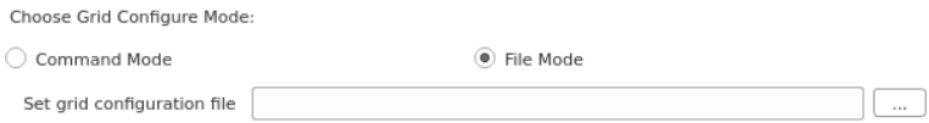
c. Specify the numbers of workers for the grid.

d. Specify the *Control* options for the grid.

Note:

See section Running on Compute Farm for more details on how to configure the grid.

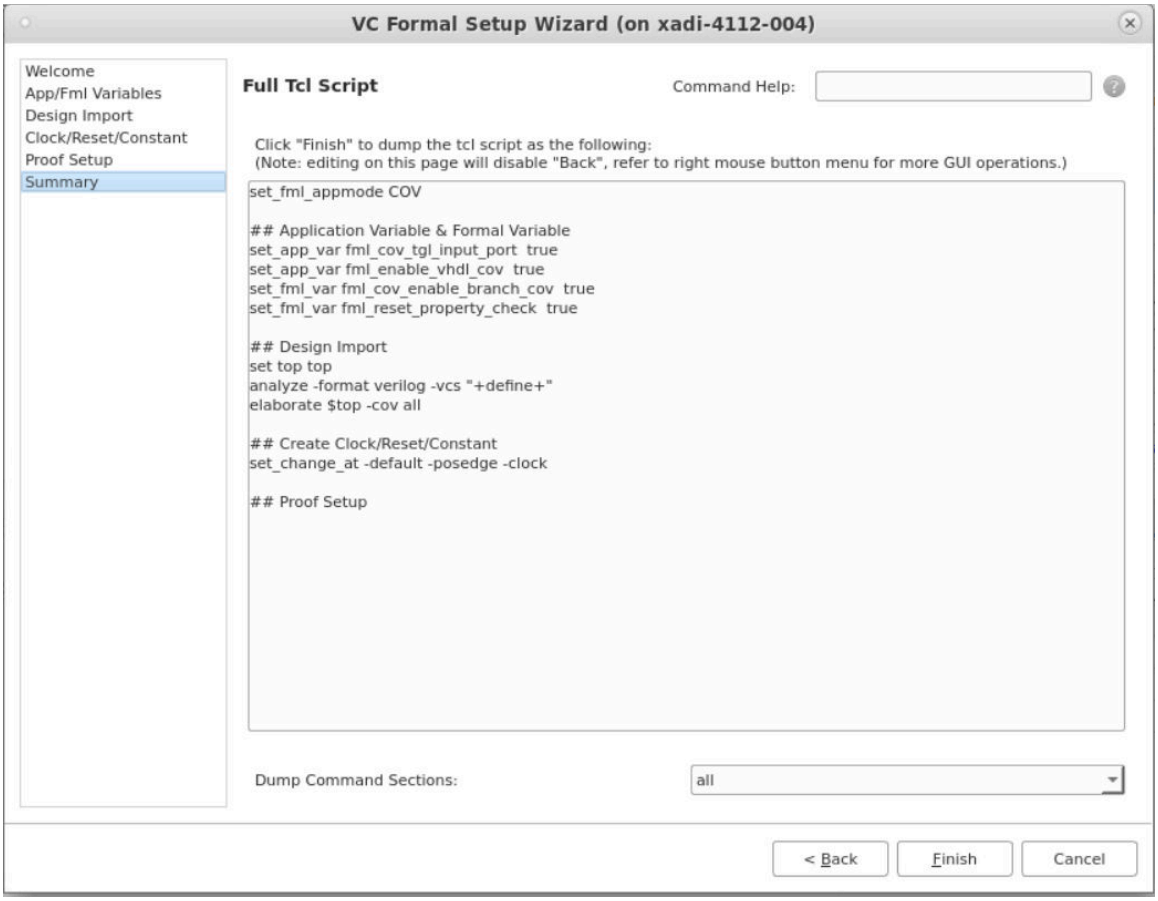
5. If you already have a grid configuration file, select *File Mode*, and browse to the grid configuration file.



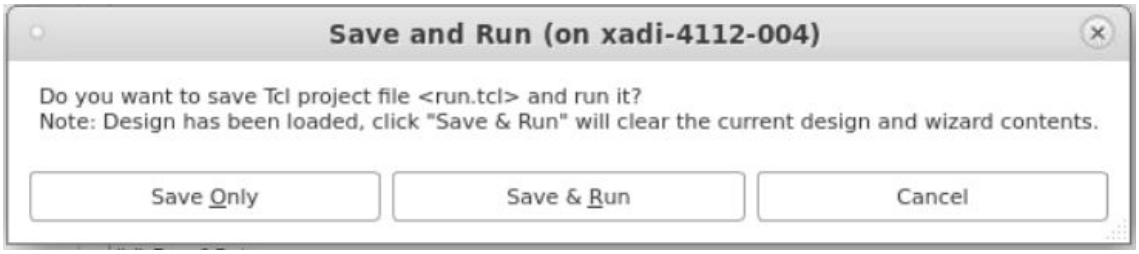
The *Summary* Screen appears.

8.4.6 Tcl Project Summary

You can make changes to the Tcl script if necessary in the *Summary* screen. If the Tcl is modified, you can click on RMB 'undo' (ctrl+z) to go back to the original settings. You can also select the required sections using the *Dump Command Sections* drop-down.



1. After you review all the steps, click *Finish.*



2. Click *Save Only* to save the Tcl project file.
3. Click *Save & Run* to save and run the Tcl project file. If the design has been loaded, *Save & Run* clears the current design and uses the Tcl project file you have created to restart.
4. Click *Cancel* to go back to Setup Wizard and make changes.

9

Exploring Design Using Navigator

The section describes how to explore designs using Navigator.

This section contains the following topics:

- About Navigator
- Opening Navigator
- Opening Multiple Navigator Sessions
- Exploring Properties/Design Using Navigator
- Saving/Exporting Navigator Properties
- Saving/Exporting Navigator Tcl
- Closing Navigator

9.1 About Navigator

Navigator is an interactive debug environment created as an extension of Verdi to aid designers and verification engineers to understand design behavior, debug assertion failures, check for potential traces and refine constraints. The explore and plot capabilities enables you to do a “What If?” analysis and observe design behavior to define and refine asserts, covers and constraints.

It is difficult to explore intended design behavior and corner case scenarios in a traditional simulation environment. Navigator eliminates the need for a simulation testbench environment to explore design behavior. It provides an interactive capability to explore a design and develop formal verification properties. It provides the flexibility to explore the design in a step by step fashion. You can modify the waveform for one scenario (or add a new condition in the form of constraints) and then incrementally explore the next set of scenarios. You can use Navigator in the following use models.

- Design Exploration: Check design behavior without testbench and test scenarios
- Property Visualization: Analyze the property behavior
- Post-silicon Debug: Reproduce post-silicon bug

Feedback

Using Navigator, you can:

- Analyze the failure, property, signal waveform
- View information of the property being debugged like Primary Property, Referenced Clock, Property list and so on.
- Create and modify assertions, covers and constraints
- Plot and run these new behaviors in relation to the Primary Property

Note:

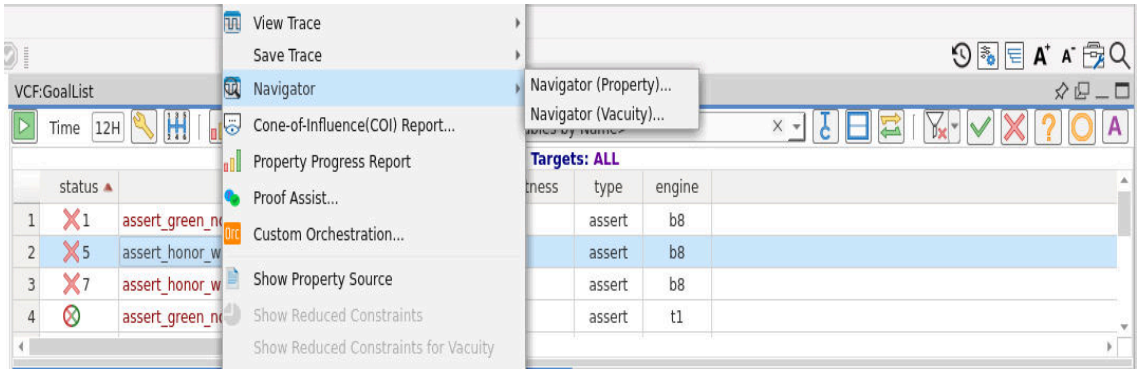
Navigator is currently supported for the FPV, FRV, and AEP applications.

This video explains how to use the Navigator feature in VC Formal to debug properties, create new properties, and analyze RTL behavior. Navigator is particularly useful in property verification and offers three primary use cases: exploring RTL behavior without a testbench, debugging failing properties, and replicating post-silicon issues for effective debugging.

9.2 Opening Navigator

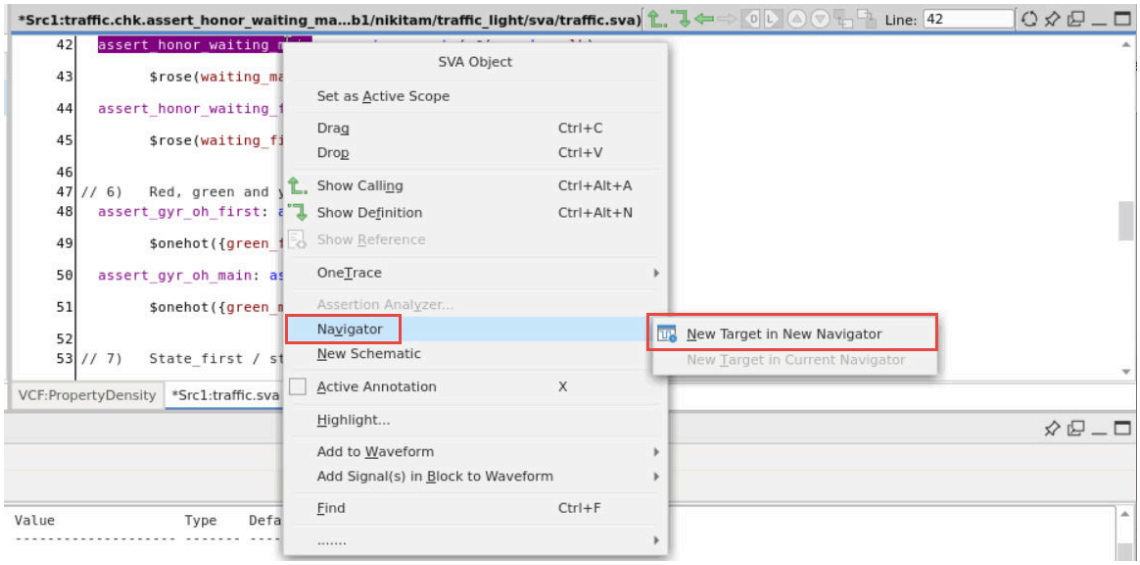
You can open Navigator in the following ways:

- To analyze properties using Navigator, from the GoalList, select a cover or assert property in the GoalList, and from the RMB menu, click *Navigator (Property)*. For vacuity and witness properties, select *Navigator (Vacuity)* and *Navigator (Witness)*.

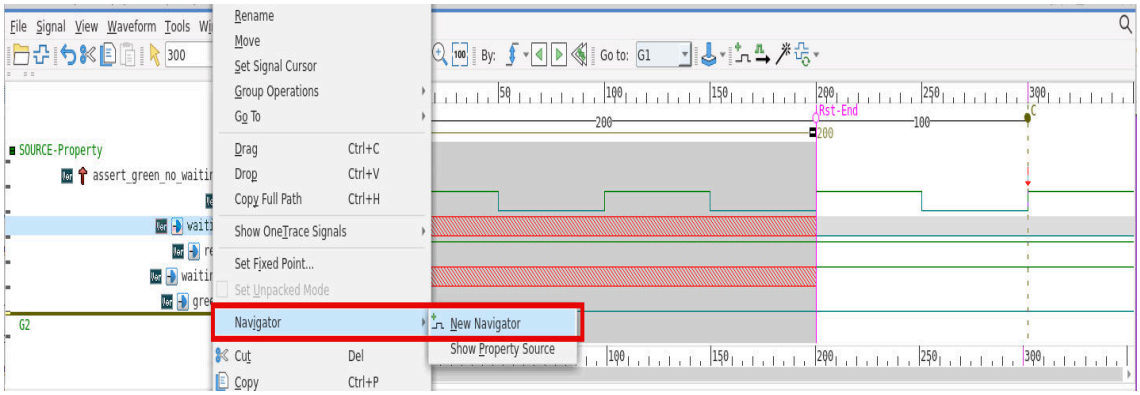


Feedback

- To explore design behavior, select a signal in the source view and from the RMB menu, click *New Target in New Navigator*.



- From the trace window, click , or from the RMB menu of the waveform signal, select *New Navigator*.

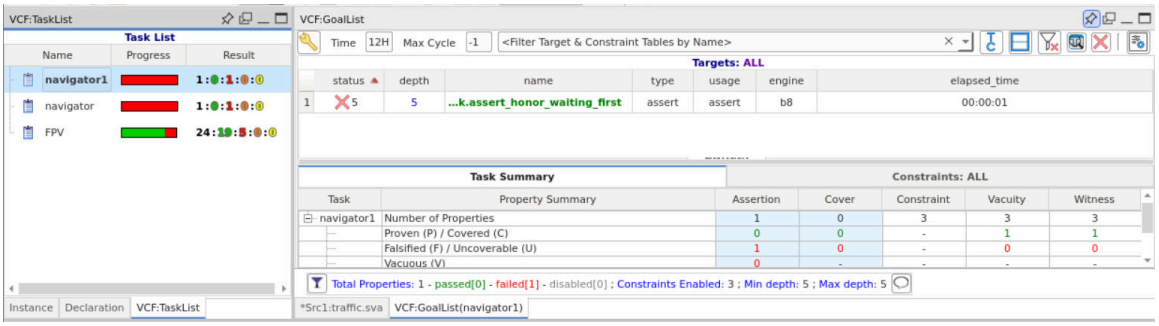


When you open a Navigator session for a property/signal, a new Navigator task and a corresponding session is created for that property/signal. The property/signal being explored is added in the target list of the Navigator task. Assumptions are inherited from the parent task.

Feedback

9.3 Opening Multiple Navigator Sessions

You can open multiple Navigator sessions from the parent task in any of the ways described in . For every Navigator session, the tool creates a separate navigator task as shown in the following figure:



9.4 Exploring Properties/Design Using Navigator

After you create a new Navigator task for a property/signal, you can perform the following:

1. Add/edit/delete properties (See section Add/Edit/Delete Properties in Navigator)
2. Edit the waveform to explore design behavior (See section Edit Waveform Mode)
3. Extend waveform cycles for further analysis (See section Extend Cycles)
4. Apply changes to waveforms and plot them again. (See section Plotting the Waveform)
5. Explore behavior based on a design snapshot (See section Snapshot of a Waveform)

The video focuses on the Navigator feature in VC Formal, which aids in debugging, property creation, and understanding RTL behavior. Navigator is particularly useful in the Formal Property Verification (FPV) application. It supports three primary use cases: analyzing RTL without properties, debugging failing properties, and replicating post-silicon issues for effective debugging.

The Navigator provides a waveform, property editor, and source window for interactive debugging. You can create multiple Navigator sessions to explore different scenarios in parallel, edit waveforms to simulate assumptions, and extend traces to analyze signal behavior. Properties can be enabled, disabled, or deleted as needed.

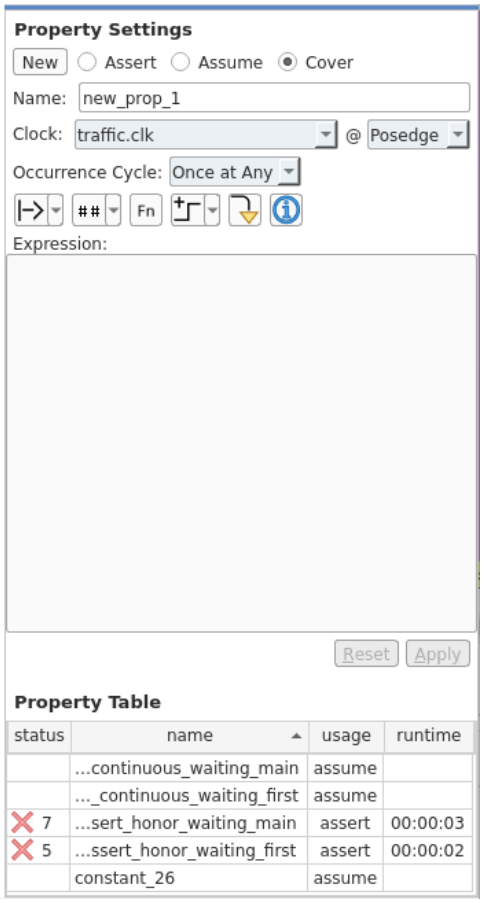
9.4.1 Add/Edit/Delete Properties in Navigator

The Navigator window consists the following

Feedback

- Navigator Panel: From the Navigator Panel, you can add or edit properties, save the properties, perform a plot, take a snapshot and so on.

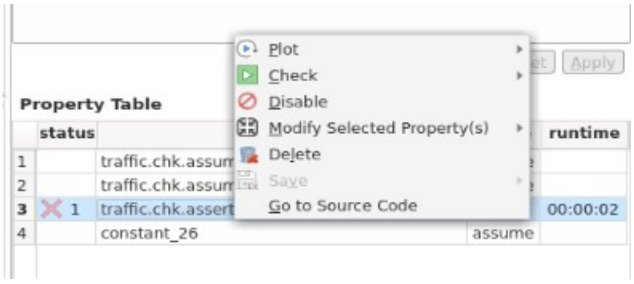
Figure 54: Navigator Panel



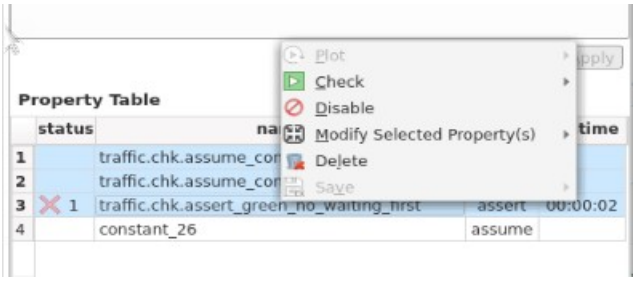
- Navigator Toolbar: This toolbar contains all the shortcut buttons to access the option on the menu bar.

- **Property Table**: This table list all the target properties created. You can also use the RMB menu from the Property Table to plot, delete, disable the properties and view the source code of the properties.

Feedback



When multiple properties are selected, you can disable, delete or save the selected properties from the RMB menu.

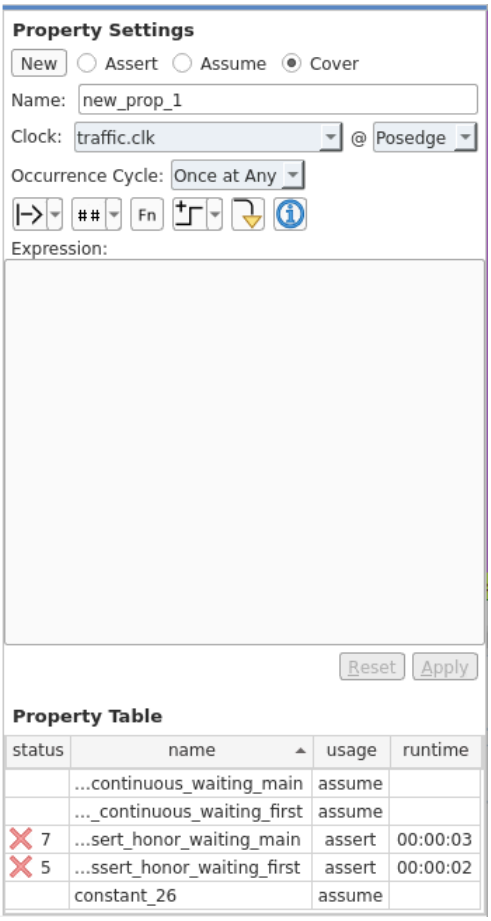


To add a new property, perform the following steps:

1. Click `New` to create a new property.
2. In the *Name:* box, type a name for the newly created property.
   - New property is created under a top scope.
   - Name can only start with [a-z], [A-Z] and “_”, followed by 0 or [a-z], [A-Z], [0-9] and “_”.
3. Select a *Type* for the newly created property. By default, *cover* is selected.
4. From the *Clock* drop-down list, select the reference clock of this new created property. In the example shown in Figure 54 above the default reference clock is traffic.clk. You can select another reference clock if needed.
5. If you select an assume or assert, you can specify a target using the following options:
   - If you want to add an assume property, select the *Assume* option. If you want to add an assert property, select the *Assert* option.
6. To specify the occurrence of target expression, select one of the following options from the *Occurrence Cycle* drop-down list.

Feedback

Figure 55: Choose Cycle for Target Expression



- To specify that the occurrence can be in any one of the cycle, select *One at Any*. By default, *Once at Any* is selected.
- To specify the minimum number of cycles, select *Minimum*.
- To provide a cycle range, select *Range*. Type the start and end cycle for the expression.
- To directly put cycles using the cursor and marker of a selected row, click .

  The left side of cursor or marker position is updated in the *Cycle* drop-down list, and the distance between cursor and marker is updated in the *Repeat* combo box. For example, if the cursor is kept on cycle 3 and marker is kept on cycle 6, the cycle option is changed accordingly, 3~3 is added in the *Cycle* drop-down list, and (5-3+1) in added in the *Repeat* combo box.

Feedback

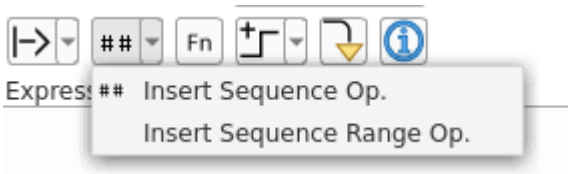
7. Provide the updated SVA expression for the newly created property. Note that a runtime signal cannot be used in the expression.

   Note:

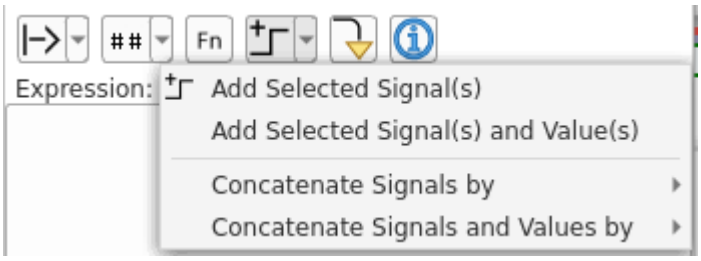
   Navigator supports all legal SVA syntax for expressions.

   a. To add implication operators, select overlapping and non-overlapping implication from the following drop-down list.

   b. Use the sequence operators drop-down to insert the sequence operator or sequence operator range from the following drop-down:

   c. Click to add selected signals in primary nWave.

      i. By default, the signal name is added and a disjunction is performed when multiple signals are selected.

      ii. Use the *Add Selected Signals and Values* option menu to add both signal name and the old value of signals at cursor time. The signal name and value are concatenated with ==.

      iii. Use the *Concatenate Signals by* and *Concatenate Signals and Values by* sub-menu to choose the concatenation for *Add Selected Signals* and *Add Selected Signals and Values* separately.

8. Click *Apply*.

Feedback

If there are no issues with the specifications, the results are reflected in the property list with the selection. If not, an error message is reported. Fix the errors and click *Apply* to view the results.

Note:
You can modify the expression and click **Plot** to view the updated waveform in nWave. It is not necessary to click **Apply** before plotting.

9.4.2 Edit Waveform Mode

After adding the properties to explore behavior, you can also modify the waveform using the *Edit Waveform* option for further analysis in the Navigator window.

To edit a waveform in nWave:

1. Select a signal in nWave, and on the RMB menu, select *Edit Waveform by Cycle*.

   The following dialog box appears.

Figure 56: Edit Waveform

   The **Start** field displays the first start cycle time at the cursor position. The **End** field shows the last cycle time of the selected range. For example, if you right-click at cursor time 270 and the clock cycle is 100, the dialog will display "200" for Start and "300" for End.

   Alternatively, you can *Shift+drag* to modify waveform by range.

2. Click *Apply*.
3. Click *OK* and the waveform draws the modified waveform range with a red rectangle and updates the value you provided as shown in Figure 57.

   Once you have done all modifications, you can plot the waveform for the updated values using the button.

4. To remove the edited value on the RMB menu in the waveform, click *Remove Edit Value*.

Feedback

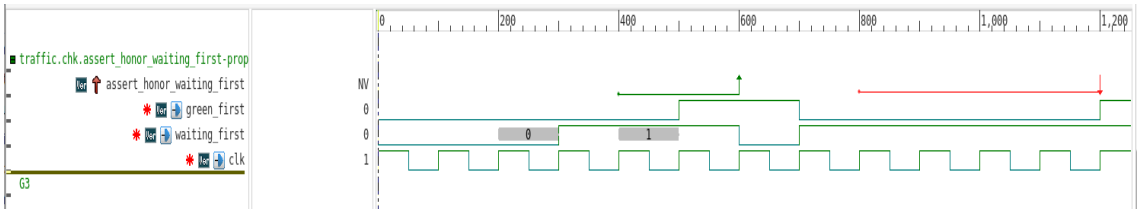
Click again to exit from the *Edit Waveform Mode*. After you exit the *Edit Waveform Mode*, and if there is at least one modification, then the following dialog box appears.

If you click *Apply*, then several assume properties are created. You can manually do a plot after the addition is done.

The newly created assume property is composed by current waveform and edited value.

After editing the waveform, you get a new waveform after a plot, nWave masks the old editing waveforms with a gray rectangle, you can easily identify them as shown Figure 57, and then modify it further as required.

Figure 57: Example for enum or localparam Modified Waveform



9.4.3 Extend Cycles

The Navigator waveform can be extended in the following ways:

- Before: Cycles are extended prior to a target being reached. This is enabled when Navigator is launched from Source View by selecting a signal.
- After: Cycles are extended after the target is reached. This option is enabled in the waveform window for the trace of a property generated by view_trace.

The Extend Cycles feature can be invoked in many ways.

1. On the nWave toolbar, click the (*Extend Cycles*) icon.

2. The **Extend Cycles** dialog box is displayed as shown in Figure 58.

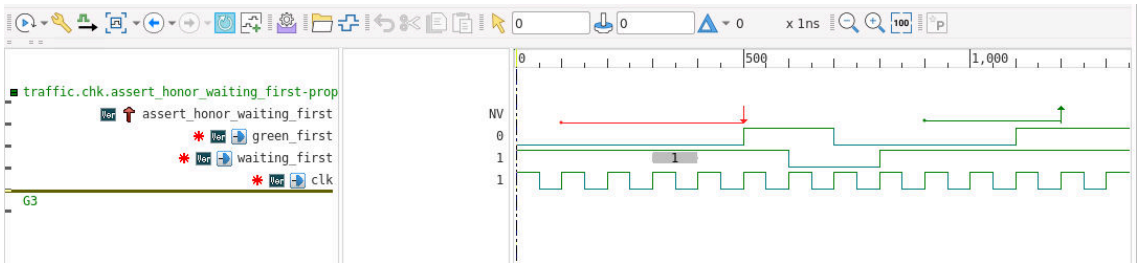
Feedback

Figure 58: Extend Cycles for a New Prop

3. Type the number of cycles you want to extend in the *Extend* box. Or click the up/down arrow to increase/decrease the number.
4. In the case of an existing waveform, you will not be able to select Before from the Cycles list. Only After is available as an option.

   Once extended the waveform is as shown in Figure 59.

Figure 59: Extended Cycles in Waveform



Note:

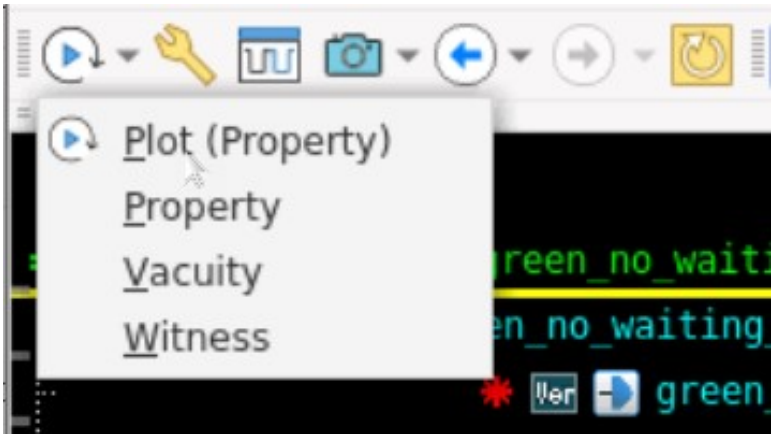
If the tool is unable to extend the trace because of conflicting or dead-end constraints, or insufficient time allocated for trace generation, an error message is reported. After acknowledging the message by clicking **OK**, check the VC Formal console for detailed error information. The console provides logs or messages to help you identify the root cause, such as conflicting constraints or a timeout, so you can address the issue and try extending the trace again.

9.4.4 Plotting the Waveform

The Plot button enables you to generate a new waveform based on the current primary property.

Feedback

Figure 60: Plotting Results

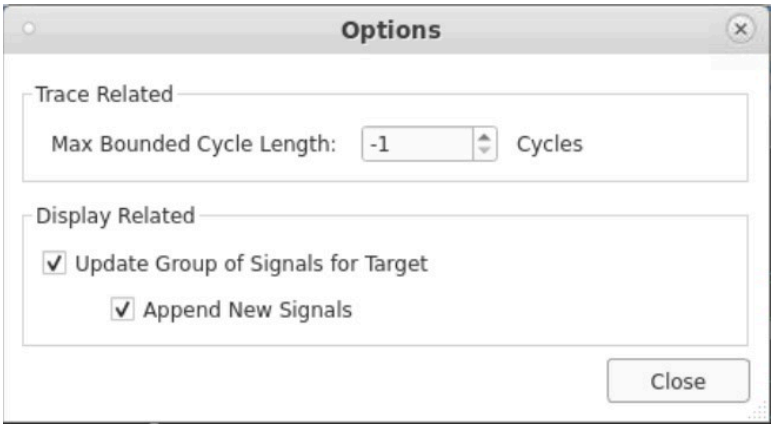


The property and signals are added into a group at the bottom with into a new group.

Updating Signals After Plot

Navigator enables you to update signals after being plot by default. Any changes to the Navigator panel are reflected in the waveform. If you do not want to update the signals, go to *Tools* > *Options* in the Navigator panel shown in Figure 60 and uncheck the option highlighted in Figure 61.

Figure 61: Plotting Results

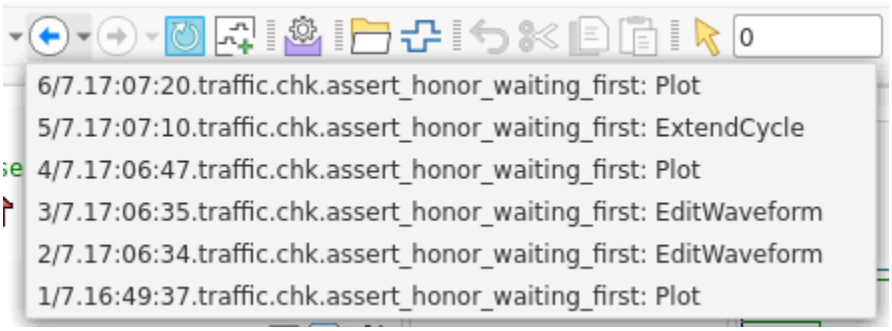


1. Deselect the *Update Group of Signals for Target* check box. The additional signals are not added to the target's name.

Undo and Redo

You can use the undo and redo functions highlighted in the following figure to go to the previous or the next step. Undo and redo support for the operations such as plotting traces, adding/editing/removing waveform values, extending cycles, and snapshot. Note that after an undo, performing any other operation disables the redo option.

Feedback



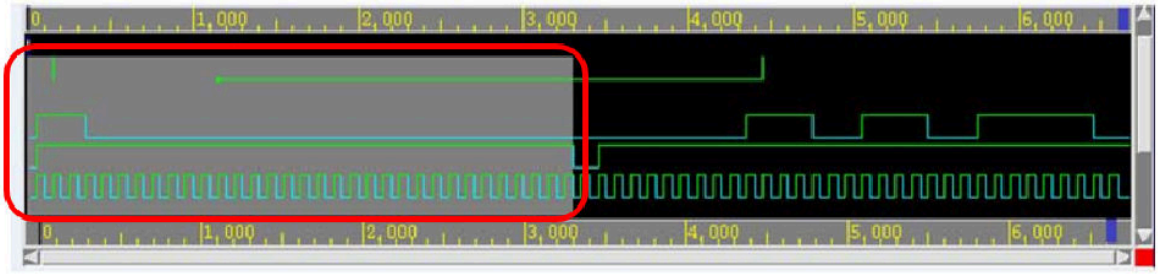
9.4.5 Snapshot of a Waveform

Snapshot captures a selected part of the current waveform. Using it as a precondition, Navigator evaluates a new target based on the captured waveform. The snapshot portion of the waveform is covered in a gray background and cannot be modified. Here are the recommended use models for using snapshots:

- After a snapshot, all the targets keep the original enable/disable status. You must either specify a new target or plot with an existing target manually. Before the next trace is generated, the following commands are disabled:
  - Extend Cycles
- All cycle-based constraints are removed after a snapshot.

The snapshot is enabled immediately after being plot. It will be disabled when you perform any trace manipulation operation.

Figure 62: Captured Waveform Colored with Dark Gray Background



9.4.6 Reloading RTL in Navigator Sessions

VC Formal Navigator enables efficient updates and debugging of RTL code within an active session. Navigator allows you to reload modified RTL code instantly, eliminating the need to restart the VC Formal session or repeat Navigator setup. The Restart button in the Navigator toolbar provides a fast way to apply RTL changes and immediately update property tables and signal lists.

Feedback

- Reduces downtime caused by session restarts after RTL updates.
- Retains the current context for property tables and signal lists.
- Improves productivity by allowing quick debug and verification iterations.
- Makes the workflow easier for users who regularly modify RTL.

Reloading Modified RTL in Navigator

To reload the modified RTL in Navigator:

- Open your VC Formal session and start the Navigator.
- Click the Restart button in the Navigator toolbar to reload the updated RTL code.

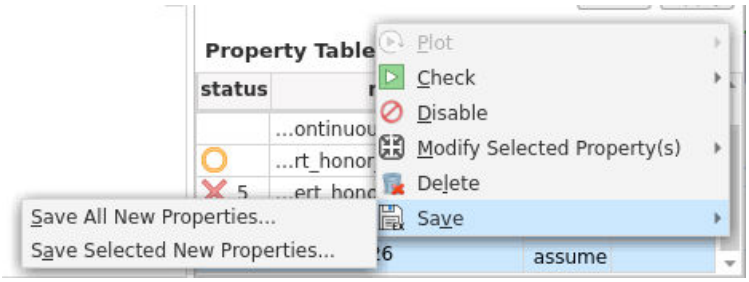
  The Navigator reloads the modified RTL automatically, refreshing the property table and signal lists to show the latest changes.

- You can continue debugging or analyzing without restarting the session.

9.5 Saving/Exporting Navigator Properties

To save your work in the Navigator window, right-click on the navigator property and export properties.

- From RMB menu, select *Save All New Properties* to export all new properties to Tcl or report format.
- From RMB menu, select *Save Selected New Properties* to export all selected new properties to Tcl or report format.



The default file format is Report Format (*.rpt). You can select Tcl format (*.tcl) or SVA Formal (*.sv) optionally as shown in the following figure:

9.6 Saving/Exporting Navigator Tcl

You can export and replay a Navigator waveform as a Tcl file using force, deposit, and run commands. This approach offers several advantages when replaying traces on a modified design:

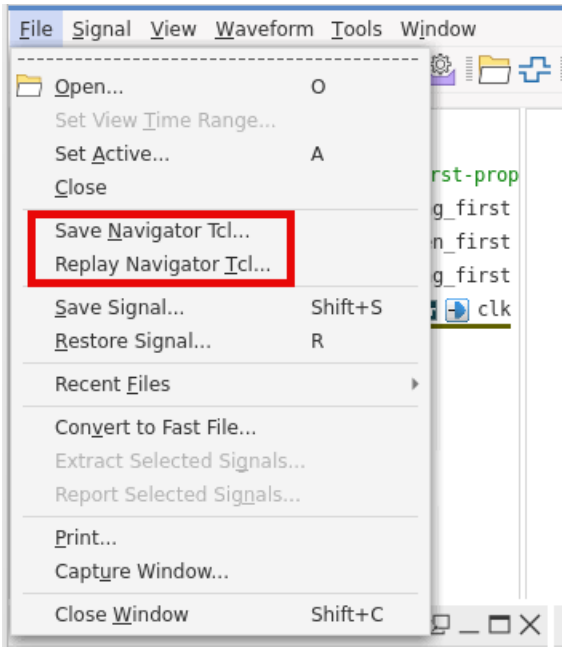
1. **Consistent Scenario Reproduction**: Exporting the waveform as a Tcl script captures the exact stimulus, including input values and signal changes, that produced a specific behavior or property failure. This enables you to replay the scenario on a modified design and verify whether the issue is resolved or if the design change impacts the behavior.
2. **Streamlined Automation**: The Tcl file can be integrated into automated regression or batch runs, allowing you to apply the same test scenario across multiple design revisions without manual effort.
3. **Efficient Debugging**: Replaying the waveform on a modified design helps determine if a property failure persists, changes, or is resolved. This supports effective debugging and validation of fixes.
4. **Testbench Independence**: The force, deposit, and run commands in the Tcl file provide a self-contained test scenario, removing the need for a simulation testbench and simplifying scenario reproduction.
5. **Easy Collaboration**: The Tcl file can be shared with team members or used in different environments, ensuring consistent scenario reproduction for collaborative debugging and review.
6. **Incremental Verification Support**: As constraints are refined or the design is updated, previously captured scenarios can be quickly replayed to check for regressions or improvements, enabling efficient incremental verification.

The save and replay feature enhances formal verification, particularly during iterative design and debug cycles.

Exporting and Replay a Navigator Waveform as a Tcl file

To export and replay a Navigator waveform in Tcl format, select the highlighted option from the **File > menu**.

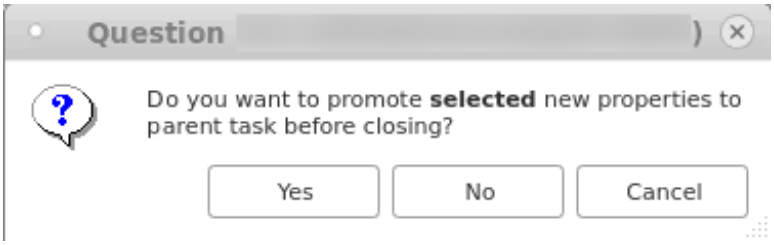
Feedback



9.7 Closing Navigator

When you close the unified navigator window, the following dialog box appears as shown in Figure 63:

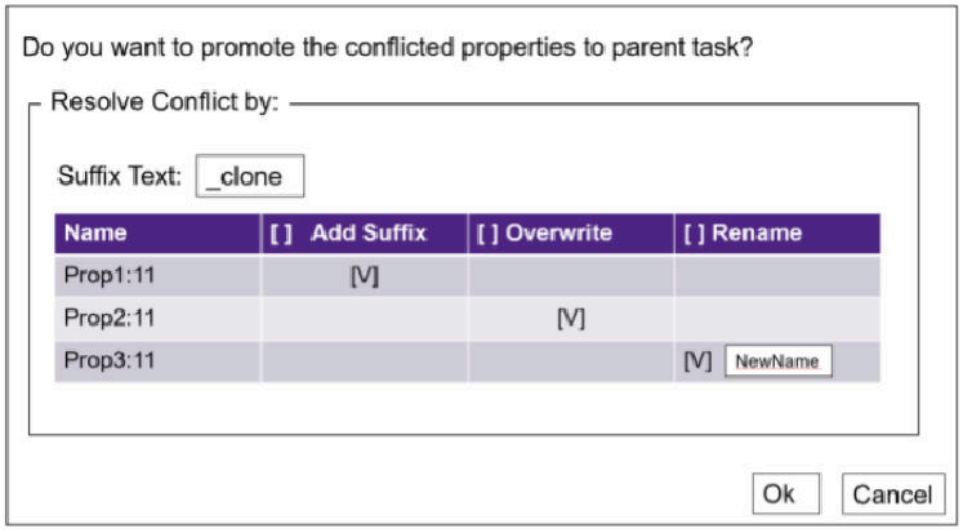
Figure 63: Quitting Navigator



1. Click *Yes* to save the newly created properties to the parent task. To quit Navigator, click *Yes*.
2. If the promoted properties have the same name as the parent properties, a dialog box to rename or overwrite the property appears. All properties with naming conflicts are shown in the dialog box.
   - The default option is <Add Suffix> with keywords *_clone*. To add the default suffix for all properties, just select the *Add Suffix* option.
   - To overwrite the properties, select the *Overwrite* option.

Feedback

- To use your own names, click the *Rename* option, and type the name.



3. Click *OK*.

10

Automatically Extracted Properties Application

The VC Formal Automatically Extracted Properties (AEP) application automatically extracts and verifies properties from a design. VC Formal generates these properties to verify the functionality independent of the conditions in RTL which can lead to incorrect behavior.

Getting Started • *What is VC Formal AEP?* • *Accessing and Working with AEP Lab Example*	**Improving Performance of AEP Analysis** • *Setting Up Runtimes* • *Setting Up Grids* • *Setting Up AEP Tasks*
Enabling AEP Checks • *Supported AEP Checks* • *Enabling AEP Checks for VHDL, VHDL Bus checks, VHDL FSM checks*	**Debugging, Reviewing and Saving Results** • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving and Restoring Sessions*
Setting Up and Running AEP • *Enabling AEP Application mode* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Verifying Design Setup* • *Running AEPs in TCL and GUI*	**Getting Help on Commands** • *AEP Tcl Commands* • *Setting AEP Specific Application Variables*

10.1 About AEP Application

The Automatically Extracted Properties (AEP) application mode can be used to automatically extract properties from the design and verify them. AEPs are properties generated by the tool after analyzing the design. VC Formal extracts AEPs from the design, and creates specially instrumented RTL signals that can be asserted or de-asserted in the formal models.

The AEP application helps you find functional design bugs easily in the early stage of design cycle. To use this application you do not require prior knowledge of formal verification or assertions.

Feedback

10.1.1 Supported AEPs and Switches to Extract

The following table lists the types of AEPs that can be extracted by VC Formal and the switches to use to extract them.

Table 13: Supported AEPs and Switches to Extract

Property Type	Switches	Description and Example

Array boundaries	`-aep bounds_check`	For arrays with indexes which cannot be computed when the module is compiled, VC Formal can test that accesses to the array use an index within the array bounds.
		Example
		`wire [1:0] sigB;`
		`reg [1:0] memA [1:0];`
		`wire [1:0] memOut;`
		`assign memOut = memA[sigB] ;`
		Falsified bounds_check: Bad data may be read.
		Failure example: If sigB's value is equal to 2 or 3, but the max index for memA is 1.
		Note:
		In Verilog designs, if `fml_aep_unique_name` is enabled, then bounds_check AEP indicates the associated dimension by showing the expression inside the array dimension for which the AEP is generated.

Arithmetic overflow	`-aep arith_oflow`	Checks the result of an arithmetic operation:
		Example
		`always @(posedge clk)`
		`if (reset)`
		`counter = 0;`
		`else`
		`counter <= counter + 1;`
		Falsified arith_oflow: Indicates the correct value of the operation would not be stored, and the result would get truncated.
		Example: If there is no condition to limit counter addition.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example

Shift overflow operations	`-aep shift_oflow`	Checks the result of the Shift overflow operations:
		Example
		`reg counter [7:0];`
		`reg [7:0] counter1;`
		`unsigned [2:0] Y;`
		`always @(posedge clk) begin`
		`counter[0] = counter[0] << Y;`
		`counter1 = counter1 << Y;`
		`end`
		`end`
		Falsified shift_oflow: indicates that all the bits of the left operand of shift operator have been shifted out.
		In the above example, the `shift_oflow` property on first shift operator is falsified because counter[0] is only 1 bit and Y can have a value greater than 1.
		The `shift_oflow` property on second shift operator is proven because counter 1 is 8 bit wide and all its bits cannot be shifted out as maximum value Y can have is 7.
		Note:
		Currently, shift_oflow checks are not supported in VHDL.
		Note:
		The shift_oflow checks is a beta feature. For more details on this support, contact Synopsys support.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example

Cast overflow checks	`-aep cast_oflow`	Cast overflow checks are created when $signed/$unsigned cast function call is the outermost operation on the right-hand side of an assignment.
		Example
		`reg [7:0] X;`
		`reg [2:0] Y;`
		`assign X = 4'b1000;`
		`always @(posedge clk) begin`
		`  Y = $signed(X);`
		`end`
		Falsified cast_oflow: indicates that leftmost set bit of the argument of $signed/$unsigned function cannot be stored in the signal on the left-hand side of the assignment.
		In the above example, the `cast_oflow` property is falsified because the left most set bit X[3] cannot be stored in Y. If the value of X is less than 4'b1000, then it will be proven
		Note:
		cast_oflow checks are not supported in VHDL as of now.
		Note:
		The cast_oflow checks is a beta feature. For more details on this support, contact Synopsys support.

Conflict driver bus checks*	`-aep conflict_driver`	Falsified conflicting-driver property indicates bus contention, x propagation, and reason for simulation/synthesis mismatch.
		Example
		`wire sigA;`
		`assign sigA = oe1? 1'b0: 1'bz;`
		`assign sigA = oe2? 1'b1: 1'bz;`
		Falsified conflict_driver: When there are conflicting values driven by multiple tristate drivers onto the same bus.
		Example: sigA is assigned twice when oe1=oe2=1, and sigA will have conflicted value 0/1 at the same time.No failure is reported if the multiple driven value has the same logical value.
		Note:
		This AEP will not be instrumented for non-tristate drivers.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example

Floating_bus*	`-aep floating_bus`	Verify that there is at least one driver active at all times for any bus in the design with multiple tri-state drivers.
		Example
		`wire sigA;`
		`assign sigA = oe1? 1'b0: 1'bz;`
		`assign sigA = oe2? 1'b1: 1'bz;`
		Falsified floating_bus: Indicates reason for possible x propagation and simulation/synthesis mismatch.
		Note:
		This AEP will not be instrumented for non-tristate drivers.

Multiple driver bus*	`-aep multi_driver`	For buses with multiple tri-state drivers, VC Formal can check that only one driver is active at a time.The property fails even if the driven value is the same.
		Example
		`wire sigA;`
		`assign sigA = oe1? 1'b1: 1'bz;`
		`assign sigA = oe2? 1'b1: 1'bz;`
		`wire sigB;`
		`assign sigB= oe1? 1'b1: 1'bz;`
		`assign sigB = oe2? 1'b0: 1'bz;`
		Falsified multi-driver: Indicates bus contention, x propagation, and may be a reason for sim/synmismatch. Multiple driver bus checks is a surperset of the conflict driver bus checks.
		Note:
		This AEP will not be instrumented for non-tristate drivers.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example

full_case	`-aep full_case`	full_case AEPs are created for case statements to verify that the case expression matches at least one case item at every cycle. Similarly, full_case AEPs are created for if statements to verify that at least one leg of the ifelse ladder always handles every possible scenario.
		There are two methods for generating full_case checks:
		1.With pragma 'synopsys full_case':
		`case (din) //synopsys full_case`
		`if (din) //synopsys full_case`
		2.priority construct in System Verilog (no need to specify any pragma because priority construct is functionally same as full_case):
		`priority case (din)`
		`priority if (din`
		Examples
		`reg [1:0] in, out;`
		`case (in) // synopsys full_case`
		`  0: out = 2;`
		`  1: out = 3;`
		`  2: out = 0;`
		`Endcase`
		or
		`reg [1:0] in, out;`
		`if (in == 0) out = 2;//synopsys full_case`
		`  else if (in == 1) out = 3;`
		`   else if (in ==2) out =0;`
		Falsified full_case: This can cause simulation behaving differently from synthesis. For example, if signal "in" is 3, the behaviour is undefined because n ==3 condition is not handled, and there is no default in case statement and no final else in the ifelse ladder.
		Note:
		Currenty, full_case checks are not supported in VHDL.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example

Parallel_case	`-aep parallel_case`	parallel_case AEPs are created for a case statement to verify that case expression matches no more than one case item at every cycle.
		Similary, parallel_case AEPs are created for an ifelse statement to verify that no two if conditions are true simultaneously.
		There are three methods for generating parallel_case checks:
		1.With pragma 'synopsys parallel_case':
		`casex (din) //synopsys parallel_case`
		`if (din) //synopsys parallel_case`
		2.unique0 construct in System Verilog (no need to specify pragma because unique0 construct is functionally same as parallel_case):
		`unique0 casex (din)`
		`unique0 if (din)`
		3.Unique construct in System Verilog (no need to specify any pragma because unique construct is functionally same as parallel_case and full_case combined):
		`unique casex (din)`
		`unique if (din)`
		Example
		`reg [3:0] current_state, next_state;`
		`parameter state1 = 2'b01,state2=2'b10;`
		`always @(din or state1 or state2)`
		`casex (din) //synopsys parallel_case`
		`2'b1x: next_state = state1;`
		`2'bx0: next_state = state2;`
		`endcase`
		`end`
		or
		`reg [3:0] current_state, next_state;`
		`parameter state1 = 2'b01,state2=2'b10;`
		`always @(din or state1 or state2)`
		`if (din == 2'b1x) next_state = state1;`
		`// synopsys parallel_case`
		`else if (din == 2'bx0) next_sate = state2;`
		`end`
		Falsified parallel_case: This can cause simulation behaving differently from synthesis. For example, if signal "din" has value "2'b10", it meets both the conditions "2'b1x" and "2'bx0".
		Note:
		Currenty, parallel_case checks are not supported in VHDL.

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example
Simultaneous set/reset	`-aep set_reset`	Check for set and reset signals never asserted at the same time for sequential elements with asynchronous set and reset inputs (for example, SR flip-flops). **Example** `always @(posedge clk or posedge set or posedge reset) begin if (set) counter = 3'b111; else if (reset) counter = 3'b000; else counter <= counter + 1;`

X_assignment reachability	`-aep x_assign`	VC Formal can verify that X assignments to signals are never activated.
		Example
		sigX = 1'bx;
		Use the `set_fml_var fml_x_in_waveform true` with x_assignnment reachability analysis to view x in waveform instead of 0 incase of property failures.
		`always @(posedge clk or negedge resetn) begin if (!resetn) state <= IDLE; else state <= next_state; end always_comb begin case(state) IDLE : if (valid_in) next_state <= RUN ; RUN : if (counter == 0) next_state <= END ; END : next_state <= IDLE; endcase end endmodule`
		See section Support for VHDL AEPs for more details.

FSM Deadlock	`-aep -fsm_deadlock`	To check for FSM infinite wait in a single state • Each FSM state has its own deadlock assertion • Property name convention: <hierarchical position>_I_Fsm_lock_AEP_prop.<fsm reg name>_<fsm state value>_deadlock_safe

Feedback

Table 13: Supported AEPs and Switches to Extract (Continued)

Property Type	Switches	Description and Example
FSM Livelock	`-aep -fsm_livelock`	To check for FSM getting stuck between multiple states • Each FSM state has its own livelock assertion • Property name convention: <hierarchical position>_I_Fsm_lock_AEP_prop.<fsm reg name>_<fsm state value>_livelock_safe
FSM Unionlock	`-aep -fsm_unionlock`	To check for any existing lock issue for a FSM, covering both deadlock and livelock • Each FSM has single unionlock assertion to check any of its states lock condition • Property name convention: <hierarchical position>_I_Fsm_lock_AEP_prop.<fsm reg name>_unionlock_safe

Note:

The `fsm_deadlock`, `fsm_livelock`, and `fsm_unionlock` checks are resource intensive, and hence these are not created with `-aep all`. See section Enabling fsm_deadlock, fsm_livelock, fsm_unionlock Type of Checks for more details on how to enable them.

10.1.2 Support for VHDL AEPs

The following AEPs checks are supported in VHDL.*

- X-Assignment
- Array bounds check
- multi_driver
- conflict_driver
- floating_bus
- Arithmetic Overflow
- Simultaneous Set/Reset Check
- FSM Deadlock
- FSM Livelock
- FSM Unionlock

Currently, parallel_case, full_case, shift_oflow, and cast_oflow AEP checks are not supported in VHDL as of now.

Note:

To enable multi_driver, conflict_driver and floating_bus AEP checks, enable the following application variables in the Tcl file:

```
set_app_var fml_enable_vhdl_aep true
set_fml_var fml_enable_vhdl_bus_checks true
```

10.2 Methodology for AEP Application

When the RTL is in development stage, AEP can be run to check for some functional issues or coding inconsistencies. Since this application helps in boosting productivity, it can be used as long as RTL is ready or has been changed. AEP is often used when fully developed formal testbench with all the constraints is not available yet. Therefore, some falsifications can occur due to lack of proper constraints for the design input space.

The AEP flow includes the following steps:

1. Set the application mode to AEP
2. Add AEP options to design read
3. Set up clock, reset and other constraints
4. Run AEP
5. Review results
6. Debug

10.2.1 Setting up and Running AEP

Setting up and Running AEP requires:

1. RTL code
2. Tcl file to set up the tool to run AEP

10.2.2 Tcl Setup for AEP

At the design read step, that's when AEP goals can be defined, and the tool will instrument those goals accordingly. You can choose to define all of the AEP type of checks using the `-aep all` option. You can also choose to define the AEP checks individually, for example, using the `-aep bounds_check+x_assign` option.

Once the design is read and AEP goals are defined, an AEP task is automatically created with all the properties extracted by the tool. Additional information such as clocks and

Feedback

resets and initial state needs to be set up as well. After the setup is done, AEP check can be started.

The following are few example AEP Tcl files:

1. For verilog/system verilog design:

```
######## Setup Phase ######
#Turn on the app AEP
set_fml_appmode AEP
#Read design with aep option
read_file -top $design -format <verilog/sverilog> -aep all \
-vcs "-f ../design/filelist"
#Clock and Reset settings
create_clock <clockName> -period <timePeriod>
create_reset <resetName> -sense high/low
#Other constraints
#Setting up initial state
sim_run -stable
sim_save_reset
######## Run Phase ######
#Formal Run
check_fv -block/-run_finish
#Reporting Command
report_fv <-list/-verbose>
```

2. For VHDL/Mixed design:

```
######## Setup Phase ######
set_app_var fml_mode_on true
set_fml_appmode AEP
#Enable VHDL AEP
set_app_var fml_enable_vhdl_aep true
set_fml_var fml_enable_vhdl_bus_checks true
analyze -format vhdl test.vhd
elaborate top -aep x_assign+bounds_check+…
#Clock and Reset settings
create_clock <clockName> -period <timePeriod>
create_reset <resetName> -sense high/low
#Other constraints
#Setting up initial state
sim_run <no_of_clock_cycles_to_run>
sim_save_reset
######## Run Phase ######
#Formal Run
check_fv -block/-run_finish
#Reporting Command
report_fv <-list/-verbose>
```

Feedback

10.2.3 Enabling AEP Properties Present in the COI of FPV Goal

Use the `enable_aep_goals_in_coi` command to enable only those AEP properties which are present in the COI (Cone of Influence) of an FPV goal or property, and disables the rest of the properties. This command works with the FPV application mode.

Syntax

```
vcf> enable_aep_goals_in_coi -help
Usage: enable_aep_goals_in_coi    # Enables the AEP goals which are in
 the COI of a FPV goal
        [-type <list-of-type-attributes>]
                                  (property type selection:
                                   Values: arith_oflow, bounds_check,
                                   conflict_driver, floating_bus,
                                   fsm_deadlock, fsm_livelock,
                                   fsm_unionlock, full_case, multi_driver,
                                   parallel_case, set_reset, x_assign)
        [-status <list-of-status-attributes>]
                                  (property status selection:
                                   Values: checking, cond_proven,
                                   connected, falsified, inconclusive,
                                   not_run, proven, unconnected, vacuous)
        [-filter <filterExpr>] (Filter expression to apply)
        [-regexp]              (Use regular-expression instead of glob
 filtering)
        [-exact]               (No pattern matching performed)
        [<list-of-names-ids-or-collections-of-properties>]
                                  (List of property names, name patterns, or
 property collections)
```

Example

If on running `fvassert -expr out>0;` command, property having name: script_prop_4 is created, and the content of the design file is as follows:

```
module test (sel, out, out2);
input  [1:0]  sel;
output [3:0]  out, out2;
reg [3:0]  out, out2;

always_comb begin
unique case (sel[1:0])
   2'b00 : out  = 4'b0001;
   2'b01 : out  = 4'b0010;
   2'b10 : out  = 4'b0100;
   2'b11 : out  = 4'b1000;
   default: out = 4'b1000;
endcase
end

wire sel00 = sel==2'b00;
```

Feedback

```
wire sel01 = sel==2'b01;
wire sel10 = sel==2'b10;
wire sel11 = sel==2'b11;

always_comb begin
unique case (1'b1)
   sel00 : out2  = 4'b0001;
   sel01 : out2  = 4'b0010;
   sel10 : out2  = 4'b0100;
   sel11 : out2  = 4'b1000;
   default: out2 = 4'b1000;
endcase
end

endmodule
```

Summary Results (before running `enable_aep_goals_in_coi script_prop_4`)

```
   AEP Summary: AEP
   ------------
   > Assertion
     - # found        : 4
     - # not_run      : 4


  List Results
   AEP List:
   ---------
   > AEP
     # AEP: 4
     [  0] not_run                                  -  test.full_case_0
 (test.sv:7)
     [  1] not_run                                  -  test.full_case_1
 (test.sv:22)
     [  2] not_run                                  -
 test.parallel_case_0 (test.sv:7)
     [  3] not_run                                  -
 test.parallel_case_1 (test.sv:22)
```

On running `enable_aep_goals_in_coi script_prop_4` command, the full case and parallel case AEP properties having underlying signal ‘out’ are enabled and those having underlying signal ‘out2’ are disabled.

Summary Results (after running `enable_aep_goals_in_coi script_prop_4`)

```
   AEP Summary: AEP
   ------------
   > Assertion
     - # found        : 2
     - # not_run      : 2

   > Disabled
     - # found        : 2
```

Feedback

```
    - # assert       : 2


 List Results
  AEP List:
  ---------
  > AEP
    # AEP: 2
    [  0] not_run                                  -  test.full_case_0
(test.sv:7)
    [  2] not_run                                  -
test.parallel_case_0 (test.sv:7)

  > Disabled
    # Disabled: 2
    [  1] assert not_run                           -  test.full_case_1
(test.sv:22)
    [  3] assert not_run                           -
test.parallel_case_1 (test.sv:22)
```

10.2.4 Enabling or Disabling AEP Checks

You can disable AEP goals by type using the `fvdisable` command when you want to run a subset of all the AEP goals that are defined by the design read. For example, if you want to disable the bounds_check from the run:

```
fvdisable -class aep -type bounds_check
```

You can also disable individual AEP properties by providing the list of property names. For example:

```
fvdisable -class aep traffic.main.arith_oflow_0
```

10.2.5 Enabling fsm_deadlock, fsm_livelock, fsm_unionlock Type of Checks

To enable fsm_deadlock, fsm_livelock, fsm_unionlock checks in AEP, specify the `-aep fsm_deadlock`, `-aep fsm_livelock`, `-aep fsm_unionlock` options in `read_file`/`elaborate` commands.

Example

```
read_file -top $design -format sverilog -aep fsm_deadlock
read_file -top $design -format sverilog -aep fsm_livelock
read_file -top $design -format sverilog -aep fsm_unionlock
read_file -top $design -format sverilog -aep fsm_deadlock+fsm_livelock
read_file -top $design -format sverilog -aep
 fsm_deadlock+fsm_livelock+fsm_unionlock
```

Feedback

or

```
set_app_var fml_enable_vhdl_cov true
analyze -format vhdl test.vhd
elaborate top -aep fsm_deadlock
```

10.2.6 Configuring AEP Instrumentation

Use the `aep_config` command to configure the instrumentation of AEPs. Use the `aep_config` command before the design is loaded, that is, before `read_file` command.

Syntax

```
vcf> aep_config -help
Usage: aep_config    # Configure AEP app settings
        [-dont_generate]   (Dont generate properties for
                            given types)
        [-type <type-list>] (List of property types for generation
                             and exclusion:   Values: bounds_check,
                             arith_oflow, conflict_driver,
                             floating_bus, multi_driver, parallel_case,
                             full_case, set_reset, x_assign,
                             fsm_deadlock, fsm_livelock, fsm_unionlock,
                             all)
        [-enable_counter_arith_oflow <true|false>](Enables extraction
                              of arith_oflow properties for counter
                              expressions. Default value is true:
                              Values: true, false)
        [-generate_only_with_fanout <true|false>]
                             (Generates only those properties for
                              which the target signal has other signals
                              in its fanout. Default value is false.
                              Values: true, false)
        [<signal-name:state>]  (Specify signal hierarchal name with an
                              optional FSM state name. Used with
                              -exclude and -type.
                               Example: {i1.reg_state:ST1})
```

Configuring Instrumentation for Fsm_* Checks

Use the `aep_config` command along with `-dont_generate` option before the design is loaded, to disable Fsm_* related instrumentation.

Examples

Consider a design with two FSM's *first.state_out*and *main.state_out*. The property types that can be generated over these FSM's are fsm_deadlock, fsm_livelock, fsm_unionlock.

1. To disable the instrumentation of specified property types for all FSMs in design (without specifying the list of FSMs and its states):

```
vcf>aep_config -dont_generate -type fsm_deadlock
```

2. To disable the instrumentation of a specific property type, for example, first.state_out (with FSM names but not stateNames)

```
vcf>aep_config -dont_generate -type fsm_deadlock {first.state_out}
```

3. To disable the instrumentation of a specific property type (first.state_out) with the RESET state (with specific FSM state names)

```
vcf>aep_config -dont_generate -type fsm_deadlock
 {first.state_out:RESET}
```

4. To disable the instrumentation of a specific property type (first.state_out) with the RESET state and a specific property:

```
vcf>aep_config -dont_generate -type fsm_deadlock
 {first.state_out:RESET main.state_out}
```

Configuring Instrumentation of arithOflow AEPs with Counter Like Pattern

Instrumentation of arithOflow AEPs with counter-like pattern (a= a + 1, a = a - 1) is enabled by default. When the instrumentation of such AEPs is not required, you can disable them using the following command:

```
aep_config -enable_counter_arith_oflow false
```

Removing arithOflow AEPs Created on Signals Not Driving Anything:

ArithOflow properties created on signals which are not driving anything do not have a significant impact. If you are not interested in viewing the results of these properties, then you can remove them using the following command:

```
aep_config -generate_only_with_fanout true
```

10.2.7 Generating FSM Fairness Constraints

Use the `aep_generate` command to generate FSM fairness constraints for the COI objects to solve any fsm_deadlock/fsm_livelock/fsm_unionlock failure induced by the FSM COI objects stuck at certain value, as fairness constraint could eventually assume them to have a change to toggle.

Before using the `aep_generate` command,

- Always ensure that fairness behavior of FSM COI objects matches design specification requirement.
- Ensure the following commands are already run: `create_clock, create_reset` and `set_constant`.

Syntax

```
vcf> aep_generate -type <list-of-type> [-input_type
 <list-of-input-types>] [-inputs <collection-or-list-of-inputs>]
 [-tcl <file_name>] [-max_width <integer>] [-bit] [-property
 <list-or-collection-of-properties]
```

- `-type <list-of-type>`: Specify the type to generate constraints. Values: `fsm_fairness, change_at`

`fsm_fairness` constraints ensures that a signal occupies all its possible values.

`change_at`constraints ensures that the value of a signal changes at a specific edge of the clock.

- `[-input_type <list-of-input-types>]`: Specify this option to generate constraints for the specified input types: Values: *in, snip, abstract, bbout*.
- `[-inputs <collection-or-list-of-inputs>]`:When specified, generate constraints for the specified inputs. By default, constraints are generated for all inputs in the COI.
- `[-tcl <file_name>]`: Specify a name for the tcl file to save the generated constraints.
- `[-max_width <integer>]`: Specify the threshold value to limit the bus bit width to generate fairness constraints.
  - When the`-inputs` option is not specified with the `-max_width` option, by default, the `max_width` is set to 2. A warning is reported if any COI inputs bit width is beyond the specified `max_width` value. An error is reported when no fairness constraint is generated.
  - When the`-inputs` option is specified with the `-max_width` option, by default, the `max_width` is set to 10. A warning is reported if any of the specified inputs' bit width is beyond the set `max_width` value. An error is reported when no fairness constraint is generated.
- `[-bit]`: Generate constraints for specified bit level input types.
- `[-property <list-or-collection-of-properties]`: Specify the list of properties to generate constraints.

Example

Feedback

- To generate fairness constraints on COI signals in the fanin of the properties P1, P2, and P3, use the following command:

```
  aep_generate -type fsm_fairness -property <P1 P2 P3>
```

- To generate `change_at` constraints on COI signals in the fanin of the properties P1, P2, and P3, use the following command:

```
  aep_generate -type change_at -property <P1 P2 P3>
```

- To generate fairness constraints for all inputs related to fsm and write them into fsm_fairness.tcl, which can be reviewed, modified and sourced again:

```
aep_generate -type fsm_fairness -tcl fsm_fairness.tcl
```

- To generate fairness constraints for the specified inputs within bit width 2 and enable them directly:

```
aep_generate -type fsm_fairness -inputs {s_axi_awvalid s_axi_arvalid
 s_axi_wvalid} -max_width 2
```

- You can now use wild card once anywhere in input signal name.

```
vcf>aep_generate -type fsm_fairness -inputs {waiting_main
waiting_first} -tcl fairness.tcl
```

  is equivalent to

```
vcf>aep_generate -type fsm_fairness -inputs {waiting*} -tcl
fairness.tcl
```

- To generate fairness constraints for the specified inputs within bit width 2 and enable them directly:

```
aep_generate -type fsm_fairness -input {s_axi_awvalid s_axi_arvalid
 s_axi_wvalid} -max_width 2
```

- To generate fairness constraints with bit option with input with range specified.

```
vcf> aep_generate -type fsm_fairness -inputs phy_rxd[3:1] -bit -tcl
 fairness.tcl
```

  Contents of fairness.tcl:

```
set_fml_var fml_defer_property_compile true
fvassume fsm_constraint_phy_rxd[1] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[1])}
fvassume fsm_constraint_phy_rxd[2] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[2])}
fvassume fsm_constraint_phy_rxd[3] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[3])}
fvcompile
```

- To generate fairness constraints with the `-bit` option with single bit input.

```
vcf>aep_generate -type fsm_fairness -inputs phy_rxd[1] -bit -tcl
 fairness.tcl
```

Contents of fairness.tcl:

```
set_fml_var fml_defer_property_compile true
fvassume fsm_constraint_phy_rxd[1] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[1])}
fvcompile
```

- To generate fairness constraints with the `-bit` option used with the complete signal, that is, without specifying any range.

```
vcf>aep_generate -type fsm_fairness -inputs phy_rxd -bit -tcl
 fairness.tcl
```

Contents of fairness.tcl:

```
set_fml_var fml_defer_property_compile true
fvassume fsm_constraint_phy_rxd[0] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[0])}
fvassume fsm_constraint_phy_rxd[1] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[1])}
fvassume fsm_constraint_phy_rxd[2] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[2])}
fvassume fsm_constraint_phy_rxd[3] -expr {@ (negedge ff_clk) 1 |=>
 s_eventually $changed(phy_rxd[3])}
fvcompile
```

phy_rxd is a 4-bit signal, which means providing a input signal without specifying its range with the `-bit`option will create property on every individual bit of that signal.

10.2.8 Recommendations for Using AEPs

If the design is big, it is suggested to limit AEP type and level of design hierarchy to improve convergence and allow more resources to focus on fewer goals. See section Enabling or Disabling AEP Checks for information on how to enable or disable AEP checks.

10.3 Standard Use Model for AEPs

AEP can be run in Tcl command interactive mode, GUI mode, batch or regression mode.

10.3.1 Running AEP in Tcl mode

After setting AEP tcl script, invoke the tool and run interactively.

Feedback

```
vcf -f <your_aep_script>.tcl
```

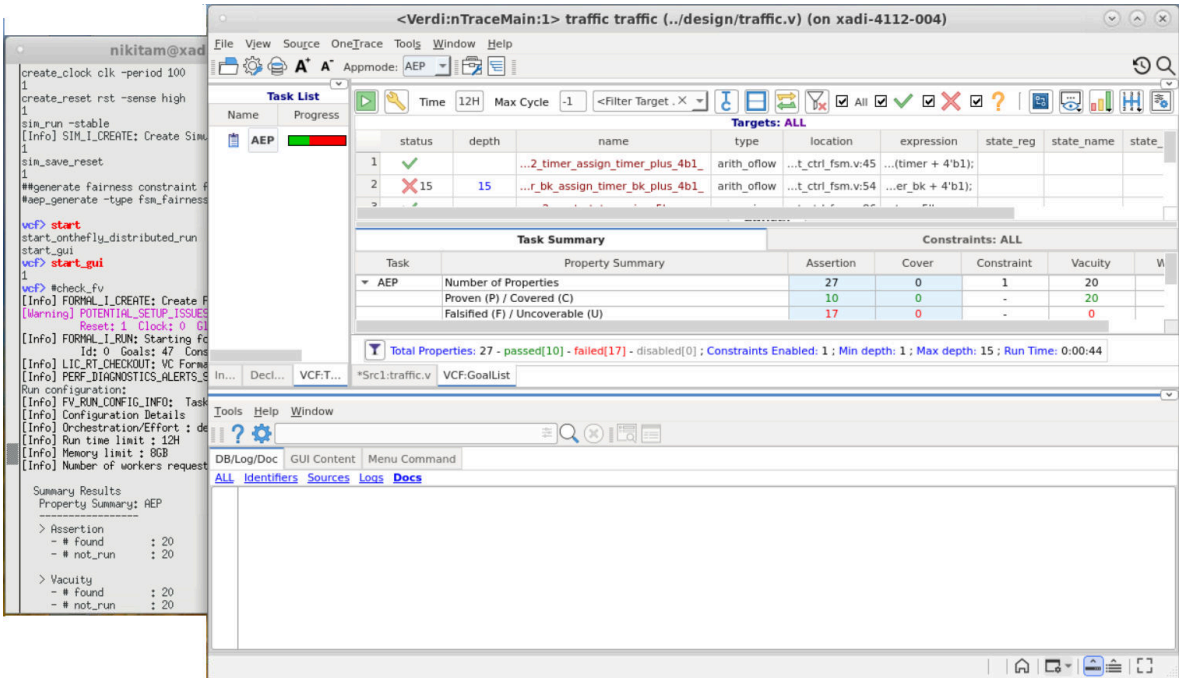
You can provide only the setup portion of the commands in the Tcl file and issue the run command check_fv interactively after invoke the tool. Or you can include the run commands in the Tcl file.

If the check_fv is not issued with -block option, you should be able to interactive the tool while it is running. If you choose to invoke the GUI during the run, you can also interactively issue the command:

```
vcf> start_gui
```

This will bring up the VC Formal GUI as shown in Figure 64 . In this mode, you will need to go back to the orginal Tcl shell window which is outside of the GUI to enter any commands.

Figure 64: AEP GUI Invoked from Tcl



10.3.2 Run AEP in GUI mode

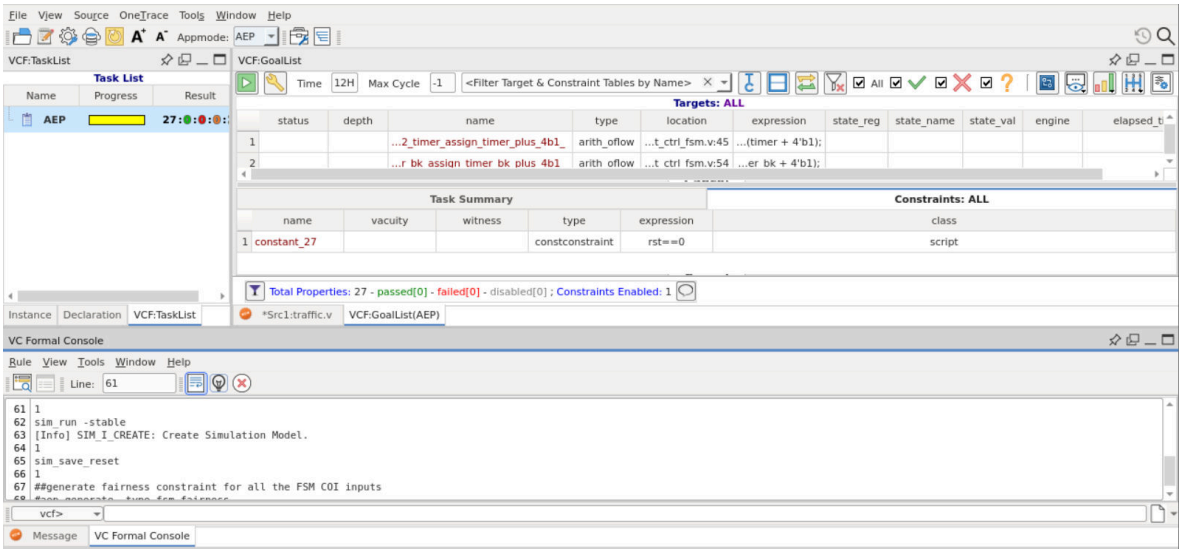
AEP can also be started in the GUI mode:

```
vcf -f <your_aep_script>.tcl -gui
```

This will bring the tool in the GUI mode, as shown in Figure 65.

Feedback

Figure 65: AEP GUI Invoked from Unix



You can see from Figure 65

- The AEP task is added in the Task List.
- The AEP properties are listed in the GoalList tab.

To run the AEP checks:

You can provide only the setup portion of the commands in the Tcl file, then hit the to start the AEP run. Or you can include the run commands in the Tcl file. The status of the properties appears in the Status column.

10.3.3 Running AEP in Regression or Batch Mode

For batch or regression mode, refer to section VC Formal Regression Mode Accelerator.

10.3.4 Running AEP on compute farm

Compute farm can provide more resources therefore better chance of improved performance in terms of run time and convergence rate. For more information on how to configure grid workers, Running on Compute Farm.

10.3.5 Setting up AEP for Multiple Tasks

Multiple tasks and parallel runs are also supported for AEP. This technique can be used to partition of the big problems to smaller ones therefore more resource can be devoted

to the partitioned problems in separate tasks. For more information on multiple tasks, see Creating Verification Tasks.

10.3.6 Saving and Restoring

AEP results can be saved the restored. For more information on saving and restoring sessions, see section Saving and Restoring Sessions.

10.3.7 Viewing Progress of Runs

The AEP run progress report is the same as the FPV run progress report. For more information, see section Viewing the Task Progress Report.

10.3.8 Reviewing Results and Report Generation

For AEP, the run results are the same as that of FPV. For more information on the run results, see section Analyzing Assertion Results.

- Proven
- Falsified
- Checking
- Inconclusive

10.3.9 Debugging AEP Failures

Debugging AEP failures is the same as for FPV. For more information on the debugging FPV failures, see section Analyzing Assertion Results.

10.3.10 Waiver Support for AEP

To enable waiver support, set the following formal variable before reading the design:

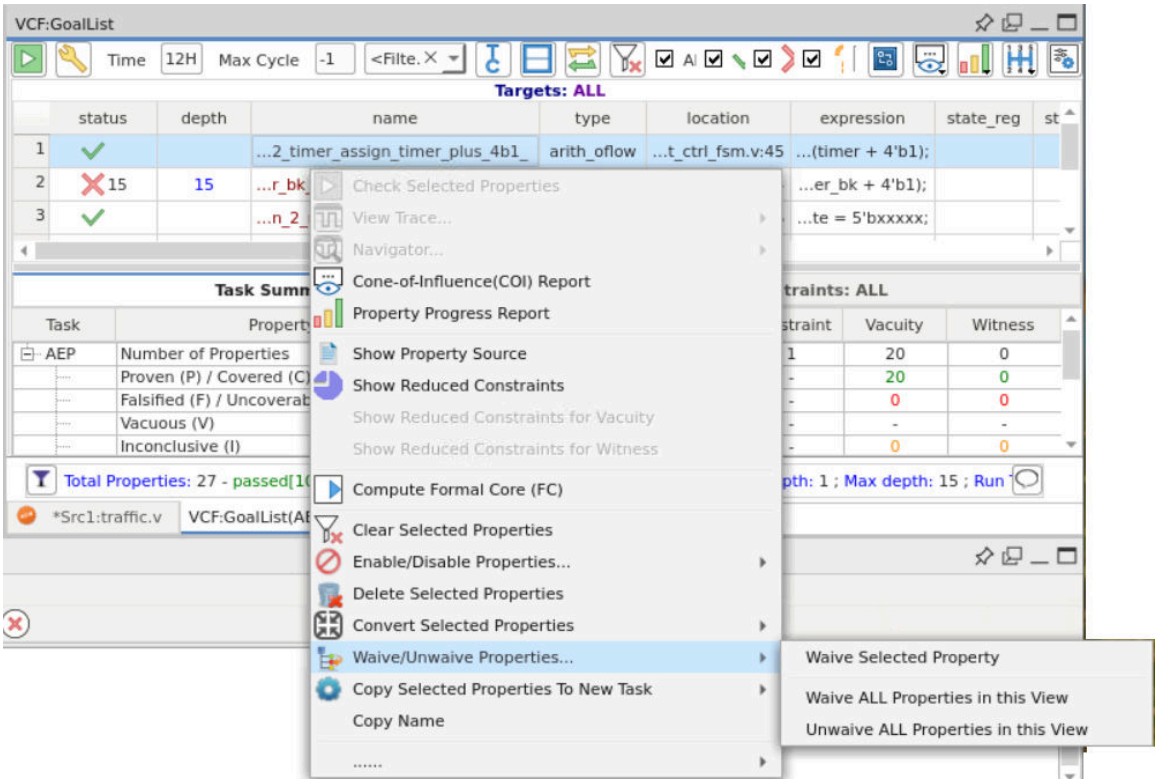
```
set_fml_var fml_aep_enable_waiver_flow true
```

After reviewing the properties, use the following commands for waive/unwaive the required properties:

```
fvwaive <property_name>
fvunwaive <property_name>
```

You can waive/unwaive properties from the GUI using the options as shown in the following figure:

Feedback



To save the waived goals in a xml file for reuse, use the following command

```
save_waiver_file -file <filename>
```

The saved waiver files can be executed before check_fv using the following command:

```
read_waiver_file -elfiles <filename>
```

Note:

The waiver mechanism is not supported for assert type AEPs.

10.4 Performance and Convergence

The performance and convergence for AEP is same as FPV. For more information, see section Formal Runtime Control.

10.5 AEP Specific Application Variables

You can use the `report_fml_var <fml_var> -verbose` command to see the default value, present setting, and the detail description.

Feedback

Table 14: AEP Specific Formal Variables

Variable Name (fml_var)	Description
fml_enable_vhdl_bus_checks	To enable the checks for vhdl like multi_driver/conflict_driver/floating_bus

You can use the `report_app_var <app_var> -verbose` to see the default value, present setting, and the detail description.

Table 15: AEP Specific Application Variables

Variable Name (app_var)	Description
fml_enable_vhdl_aep	To enables extraction of AEPs for VHDL designs.

11

Formal Coverage Analyzer Application

The VC Formal FCA application complements simulation flows and provides proof that uncovered points in coverage goals are indeed unreachable, allowing them to be removed from further analysis-saving significant manual effort.

Getting Started • *What is VC Formal FCA?* • *Accessing FCA Lab Example*	**Obtaining Signoff Coverage for FCA** • *Property Density Structural Coverage* • *Over-constraint Unreachability Coverage and Bounded proof reachability coverage* • *Formal Core Reachability Coverage* • *FTA Reachability Coverage*
Setting Up and Running FCA • *Enabling FCA Application mode* • *Supported Coverage Metrics* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Running VC Formal FCA UNR*	**Debugging, Reviewing and Saving Results** • *Generating Verification Summary* • *Intentional Uncoverable Analysis* • *Saving the Coverage Database Using TCL* • *Waiving Coverage Goals* • *Saving and Restoring Sessions*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress*	**Getting Help on Commands** • *FCA Tcl Commands* • *Setting FCA Specific Application Variables*

11.1 About Formal Coverage Analyzer Application

Coverage metrics is a well established measurement for progress and final verification signoff, that is commonly used in simulation. Coverage closure at the final signoff stage is often very difficult and time consuming.

The Formal Coverage Analyzer (FCA) application can be used to do the following:

- Assist simulation-based verification coverage signoff
  - Leverage the simulation coverage database and perform unreachability analysis on goals not covered by simulation, and provide exclusion list with coverage goals that are uncoverable for review and waive.
- Qualify formal property verification with coverage signoff
  - Over-constraint coverage unreachability analysis to identify unreachable goals because of constraints.
  - Property density coverage provides structural coverage of the design code in the COI of all the properties.
  - Bounded depth coverage reachability analysis helps to identify whether the required design code is covered within the specified proof depths.
  - Formal core coverage reachability analysis provides minimal design code that is required for the formal engines to reach the full proof or specified proof depths.
  - Fault coverage reachability analysis provides design code that is covered by the FTA application.

VC Formal FCA Demo Video

You can also view the VC FCA demo video to get hands on.

VC Formal FCA Lab Example

An example for VC Formal Coverage Analyzer is provided in the following location:

$VC_STATIC_HOME/doc/vcst/examples/FCA

11.1.1 Supported Coverage Metrics

VC Formal FCA supports the following coverage metrics. Different FCA unreachability and reachability analysis applications may support all or some of the metrics described here.

Line Coverage

Line coverage (or statement coverage) shows you which lines of code are exercised and which ones are not, by your testbench during a simulation run. VCS generates metrics for total coverage of lines, blocks, and branches that tell you how much of your code is actually executed by the tests. Line coverage is applied to signal and variable assignments in HDL code. A zero execution count, pin points a line of code that has not been exercised. This could be the source of a potential design error.

Feedback

Condition Coverage

Condition coverage monitors whether certain expressions and sub-expressions in your code evaluate to true or false.

Note:

By default, condition coverage is not supported inside functions and tasks. You must use the `-cm_cond` option in VCS to enable the same.

Toggle Coverage

Toggle coverage monitors value changes on signal bits in the design. When toggle coverage reaches 100%, it means that every bit of every monitored signal has changed its value from 0 to 1 and from 1 to 0. Missing transitions of values provide definitive conclusions about inactive elements and unexercised portions of the design. The statistics produced for each module can be examined to quickly determine areas of low coverage. This helps you write tests to address the missing activity.

Note:

Toggle coverage is not supported inside functions and tasks.

Branch Coverage

Branch coverage monitors the execution of conditional statements such as if/else statements, case statements, and the ternary operator ?: in a design. By default, branch coverage analyzes which alternative of these conditional statements is covered. It can also monitor and report the values computed by a ternary expression.

Note:

The branch coverage can be enabled either with `-cov branch`, `set_fml_var fml_cov_enable_branch_cov true`, or `-cov all`.

FSM Coverage

In hardware, a FSM is a sequential logic that outputs a current state and a combinational logic that outputs the next state. FSM coverage identifies a group of statements in the source code to be an FSM and tracks the states and transitions that occur in the FSM during simulation.

The sequential logic is driven by the next state signal, clock and reset signals. The combinational logic is driven by the current state and the inputs to the FSM.

VC Formal also supports coverage for FSMs that do not have clock for the transition coverage goals.

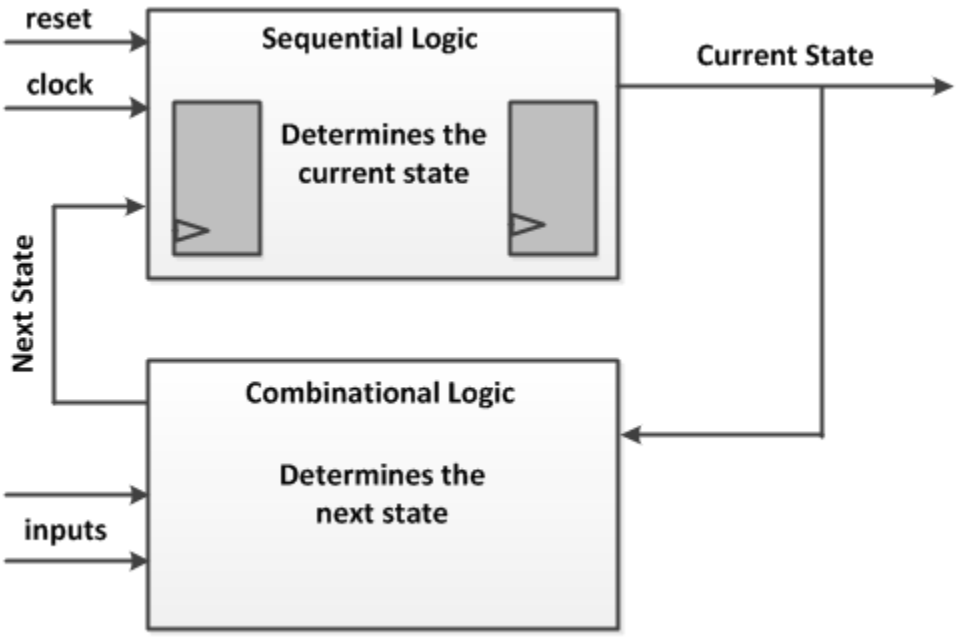
Feedback

In Verilog, a group of statements can be used to describe a higher level of abstraction of an FSM. VC Formal Coverage Analyzer treats a group of statements as an FSM if it meets the following criteria:

- The group of statements must contain a procedural assignment statement to a vector variable to assign the current state of the FSM.
- The group can also contain one of the following (though this is not required):
  - A procedural assignment to assign the next state to another variable.
  - A concurrent assignment statement to assign the next state to a net.
- In the group, a dependency exists between the current values assigned and the next value assigned. When you write such a group of statements, it is important to know if the statements made assignments to all possible states of the FSM, and all possible transitions between states.

Figure 66 shows the block diagram of FSM modeling.

Figure 66: Finite State Machine



SV Covergroups

SV covergroup coverage enables you to monitor values, transitions, and crosses for variables and signals. In the following example, SV covergroup coverage tells whether the value *0* for the coverpoint *cp* with respect to the given bin say *b1* is covered or uncoverable.

Feedback

Example

The following code defines a covergroup named cg. The covergroup cg contains a coverpoint named cp. There are two bins associated with this cover point b1 and b2 with some expected values.

```
bit cp;
covergroup cg;
coverpoint cp {
bins b1 = {0};
bins b2 = {1};
}
endgroup
```

You can instantiate a covergroup as follows:

```
cg inst = new();
```

Covergroup without coverage event is supported. For example,

```
covergroup cg;
   coverpoint add_out1
      {
              bins a = {[0:59]};
              bins a_1 = {[60:62], 63};
      }

      endgroup
cg inst = new();
always @(*) begin
    inst.sample();
end

endmodule
```

Covergroup with complex clocks is supported. For example,

```
covergroup test_cg @(count);
  option.per_instance =1;
  test_counter : coverpoint count {

}
```

Note:

Support for Covergroup with complex clocks is currently a beta feature. For more details on this feature, contact Synopsys support.

Limitations

SV covergroups are supported with the following limitations:

- Covergroups are supported only when they are defined in DUT and/or in the associated hierarchy.
- Only covergroups with simple clocking event of type @posedge OR @negedge, and complex clock @signal are supported. Multi-clocks are not supported. Similarly, the functions sample(), start(), stop(), and so on are not supported, and hence they are ignored.
- Wildcard array bins are not supported. The illegal_bins are treated as ignore_bins.
- The coverage options are not supported, except for auto_bin_max. The default values of the coverage options are supported, except the following two, where a non-default value is supported:
  - at_least = 1
  - per_instance = true
- Only the covered status of the SV covergroups is written to the VCS coverage database (covdb). The uncovered status of the SV covergroups is not written to VCS coverage database (covdb).
- Illegal_bins inside a coverpoint with guard expression is not supported.
- Clocking block is not supported.

  Example

```
clocking cb @(negedge clk iff coverage_enable);
input valid_signals;
endclocking
covergroup cg_example @(cb)
```

11.1.2 X Semantics Support

FCA supports 'X' semantics. Without this support, signals having an unknown value (X) is driven with either 0 or 1, resulting in incorrect covered goals. To overcome this issue, FCA supports X semantics, to enable correct identification of uncoverable goals.

To enable the support of X semantics, set the `fml_cov_x_support` fml variable to true, before the `read_file` command in the FCA flow.

Example

Feedback

Consider the following code snippet:

```
reg b;
always @(posedge clk)
begin
b <= en? 0 : 1;
end
always @(posedge clk)
begin
if (!b)
c <= 0;
end
```

Here, since b is initialized at reset. So, it can be 0 or 1 post reset. As a result, the highlighted line will be marked covered by FCA. With X semantic support, the same line will be uncoverable.

It is very rare case where user like to consider "x->0"/"x->1" as UNRs. VC Formal puts in extra effort to find these codes and compute for UNR. Therefore, with this feature there will be performance impact than compared to default run (X semantics check is off).

Note:

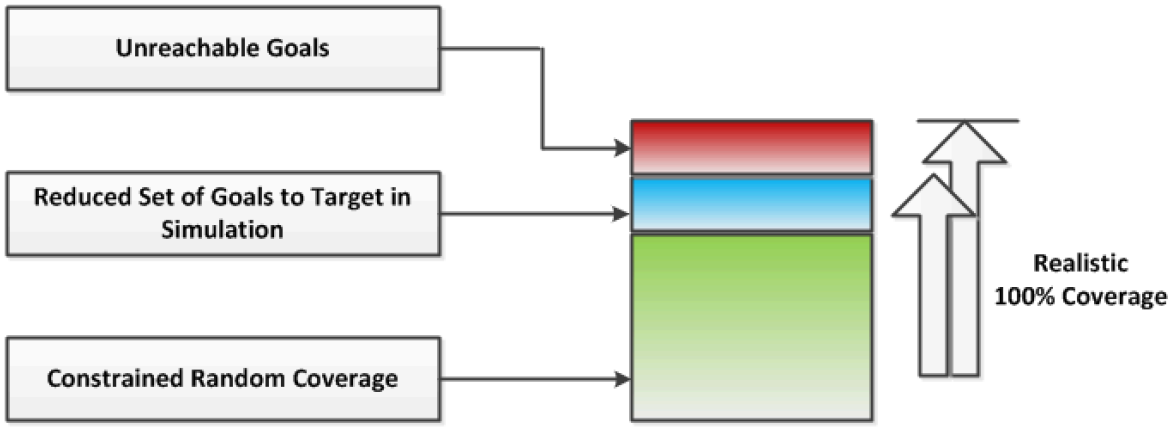
Enabling X semantics will have tremendous performance impact.

11.2 Unreachability Analysis to Accelerate Coverage Closure in Simulation (FCA UNR)

Coverage closure functionality enables you to target and achieve a realistic coverage goal. For example, consider the three colored blocks comprising of 100% of code coverage that you intend to cover, as shown in Figure 67. The green colored block shows coverage that is achievable using constrained random verification, which is about 90%. The rest of the coverage (shown in blue and red color) is difficult to achieve.

Feedback

Figure 67: Coverage Closure Block Diagram



VC Formal Coverage Analyzer exposes the unreachable goals (in red color), so that you can target only the reduced set of goals (in blue color) using directed tests in simulation. It uses the coverage database provided by you and invokes the formal engines to automatically analyze the coverage targets not covered in the simulation database. When it finds uncoverable goals, you can save them to an exclusion file. These uncoverable or dead code need to be reviewed carefully. They may be exposing hidden bugs. The simulation coverage database and the exclusion file generated by VC Formal can be loaded into coverage viewer like Verdi or URG to see the improved coverage score. FCA can save a lot of manual effort required to find the dead code and it helps to increase the overall coverage score.

VC Formal Coverage Analyzer UNR can perform coverage analysis for the following metrics:

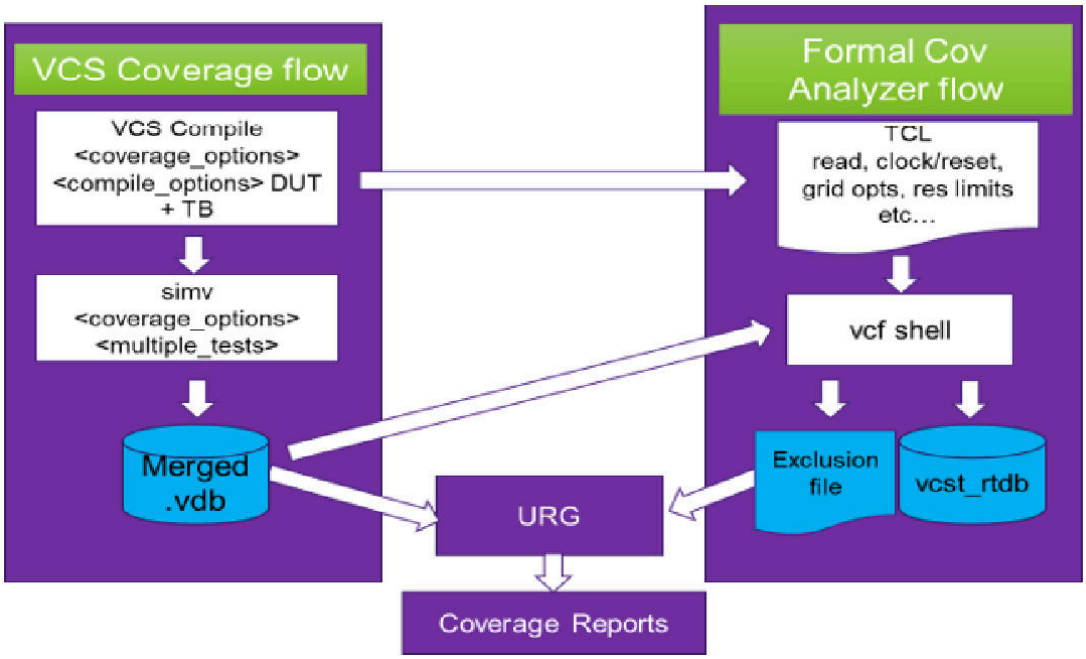
- Line
- Condition
- Toggle
- Branch
- Finite State Machine (FSM)
- SV covergroups

11.2.1 VC Formal FCA UNR Methodology

You can perform the following functions with the VC Formal Coverage Analyzer flow as shown in Figure 68. This should be done without any constraints, so that you are sure that the uncoverable logic is dead code, and can be safely excluded.

Feedback

Figure 68: Figure 10-3 FCA Unreachability Analysis



- Import a simulation database.
- Specify coverage metric for FCA UNR.
- Run VC Formal Coverage Analyzer on the rest of the coverage goals.
- Save the unreachable coverage objects (unreachables) in the form of exclusion file.
- Read the created exclusion file using Verdi or URG along with the simulation database to give a complete coverage report.
- Reuse the accepted exclusions and ignore them for coverage analysis from the next run. If the design is modified, then the same exclusions need not be revisited.
- Ensure that the unreachables have not become reachable, when the design is stable.

FCA UNR can be used in the following verification stages.

- *Early verification stage*: In the early verification stage, no coverage database is available. VC Formal Coverage Analyzer extracts all the coverage targets in the design and performs analysis for them.
- *Testbench available*: In this stage, a coverage database is available which is read into VC Formal Coverage Analyzer. VC Formal extracts the coverage holes from the

Feedback

database and performs coverage analysis for them. This can be an iterative process, where previous goals already marked uncoverable can be excluded from analysis

- *Validate exclusions*: This generally happens at a much later stage when the design is stable. VC Formal Coverage Analyzer can be used to validate any previously generated exclusions.

There are four basic use model for FCA UNR. Table 16 summarizes how each of the use model is applied during which project cycle, input required, scope and purpose. The following sections provide more detailed information to setup and run FCA UNR.

Note:

When simv.vdb is read in, then FCA generates the exclusion file with respect to the simulation hierarchy, else the exclusion file is generated with respect to the formal top from which it is elaborated.

Table 16: FCA UNR Flow Usage Models 1-4

Project Cycle	Use Model	Inputs	Scope	Purpose
Design phase – no testbench	Generate uncoverables	RTL	Block	Find bugsUnexpected dead code
Regressions Started	Generate exclusions	RTL + simv.vdb	Block/Chip	Find uncoverables.Reduce manual coverage closure effort by finding real goal.
Regressions Incremental – Closing Coverage	Exclude previous exclusions and generate new exclusions	RTL + simv.vdb + *.el	Block/Chip	
Late - Closing Coverage	Validate Exclusions	RTL + simv.vdb + *.el	Block/Chip	Validate *.el

11.2.2 VC Formal FCA UNR Setup Tcl File

Create a setup Tcl file that includes the following steps.

1. Set the application mode at the beginning of the Tcl script:

```
set_fml_appmode COV
```

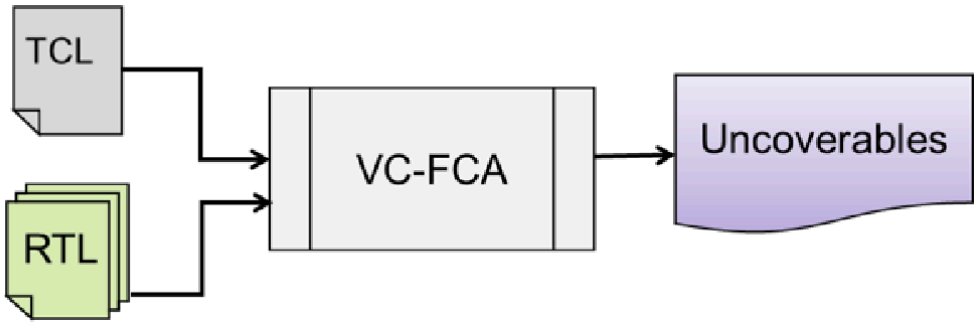
2. Import a simulation database if there is one:

```
read_covdb -cov_input <dbPath> -cov_dut
 <instancePath_to_topModule_for_FCA>
```

Feedback

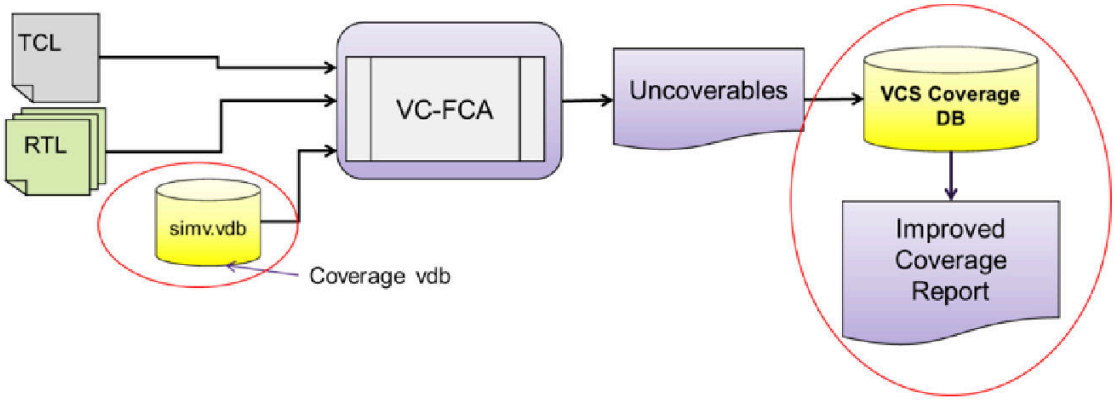
As indicated in the Table 16, in use model 1 when there is no simulation testbench with coverage database (Figure 69), you can skip this step. In this use model, it is best to run FCA UNR at the block level during the design phase.

Figure 69: FCA UNR Use Model 1: Design Phase



For use model 2 during the verification phase (Figure 70), the above command will be required to import the simulation coverage database. Use model 2 is the most common usage for FCA UNR.

Figure 70: FCA UNR Use Model 2: Verification Phase



Usage of fv_dut and cov_dut

- The `fv_dut` option of the `read_covdb` command specifies the instance path name in the formal wrapper. It tells the VC Formal the location of the synth DUT. The `cov_dut` option points to the DUT in the simulation world whereas `fv_dut` points to the DUT in the formal world.

  For example, the synth DUT is named foo.

  In Simulation: A testbench named as simTB and within simTB the hierarchy instance of foo is simTB.i1.i1.DUT.

Feedback

In Formal: the formal wrapper named fvTB and within fvTB the hierarchy instance of foo is fvTB.a1.DUT.

In this scenario, cov_dut is simTB.i1.i2.DUT, and fv_dut is fvTB.a1.DUT.

3. In general, reset is tied to its inactive state during formal. As a result, VC Formal reports code coverage goals under reset are reported as uncoverable. To enable coverage checking during reset:

```
set_fml_var fml_reset_property_check true
```

4. By default, FCA performs coverage analysis for the Verilog part of the design. To perform coverage analysis for the VHDL part, set the following application variable to true.

```
set_app_var fml_enable_vhdl_cov true
```

5. To enable coverage for ports or signals of packed struct type that have elements of enum type, set the following formal variable before `read_file/elaborate`.

```
set_fml_var fml_cov_enable_enum_in_struct true
```

6. If you choose to enable toggle coverage metric, by default, the top level input ports are not enabled, therefore they will not be analyzed. You will need set to the following application variable to true to enable toggle coverage of input ports. It may be meaningless since all the inputs are coverable in the absence of any constraints.

```
set_app_var fml_cov_tgl_input_port true
```

7. Specify coverage metrics `-cov <coverage_metric| all>` option need to be used with `read_file` or `elaborate` command.

   Single step flow:

```
read_file -cov line -top <top_module_name> -vcs "<vcs_command_line>"
```

   Two step flow:

```
analyze <design_files> <options>
elaborate <top_module_name> -cov line -vcs " <vcs_command_line> "
```

   To enable all coverage metrics:

```
read_file -cov all+branch -top <top_module_name> -vcs "
 <vcs_command_line> "
```

   The options that can be passed with the `-cov` switch are as follows:

   - `line`
   - `cond`

- `tgl`
- `fsm_state`
- `fsm_trans`
- `cg`
- `branch`

If the `all` keyword is specified with the `-cov` switch, all the coverage metrics would be enabled, except branch coverage. To enable branch coverage, `set_fml_var fml_cov_enable_branch_cov true` is needed before `read_file/elaborate` with `-cov branch` or with `-cov all`.

Note:

By default, VCS does not instrument coverage goals for library components. i.e. any files specified by -v or -y option in design file will not contain any coverage goals. To ensure coverage goals are created for all the design elements, add `-cm_libs vy` option to `-vcs` command. For example,

```
read_file -cov all -top <top_module_name> -vcs "-cm_libs vy
 <vcs_command_line> "
```

8. When the setup has non-synthesizable constructs, the Top module should be set to synthesizable DUT.

   For setup with testbench files, use the `-buildTop <module name>` in `read_file` or `elaborate` command to set the correct synthesizable DUT.

9. Set the clocks and resets

```
create_clock <clockName> -period <timePeriod>
create_reset <resetName> -sense <high/low>
sim_run <no_of_clock_cycles_to_run> or sim_run -stable
sim_save_reset
```

Supported VCS Coverage Sub Options

As mentioned above, you can specify that the VCS Coverage sub-options with the `-vcs` switch of `read_file` and `elaborate` command to read the design. Some sub options are also supported in FCA

The following are some of the VCS options that are supported in FCA.

- Line: `contassign`
- Cond: `obs, allops, allvectors, tf, std, for, scalarbitwise, ports, full, event`

Feedback

- Toggle: `portsonly, mda, fullintf`
- FSM: `reportvalues, allowtmp, report2StateFsms, reportWait, reportXassign`

Note:

- You should be cautious when using the `-cm_tgl mda` option as it is performance intensive, therefore, if needed, you can enable coverage only for ports with multi-dimensions using the `-cm_tgl mda -cm_tgl portsonly` options.
- Not all the options described in Coverage Technology User Guide are valid for FCA.
- For more details on the VCS options that are supported in FCA, see the Coverage Technology User Guide.

Automatic Extraction of Coverage Sub Options

VC Formal Coverage Analyzer, by default, automatically extracts the coverage sub options from the simulation database read during the run. These options are used to run VC Formal. This ensures that the simulation and the Formal view gets to work with the same coverage shape, and hence there are no mapping issues between these two views. You can override this behavior by setting the following application variable:

```
set_app_var fml_cov_override_db_opt true
```

When this application variable is set to true, VC Formal honors the coverage sub options sent through the `read_file/ elaborate` command.

If the user wants the tool to extract all the coverage sub options but make an exception for `-cm_hier` (if the original *hier* file isn't accessible), then they can override the *cm_hier* file by passing a new *cm_hier* file using the following command:

```
set_fml_var fml_cov_alt_hier_file <hier_file_name>
```

This setting will pick up the *cm_hier* file from the fml var `fml_cov_alt_hier_file`and allows the application of all the rest of the coverage sub options, from the simulation db.

If the user wants the tool, in certain scenarios, to have both the simulation and alt *hier* files applied, then they can use the following command:

```
set_fml_var fml_cov_hier_both true
```

The automatic extraction of coverage options is done, by default.

VC Formal Coverage Analyzer also automatically extracts hier/const file files passed through the `-cm_hier/-cm_constfile` options to simulation.

Feedback

Note:

VC Formal Coverage Analyzer does not support including/excluding interface signals using a hier file.

11.2.3 Running VC Formal FCA UNR

The FCA UNR can be run in Tcl command mode, GUI mode, batch or regression mode following the step to invoke VC Formal with the Tcl setup file created in the above section Inputs for FCA UNR.

```
vcf -f <your_fca_unr.tcl>
```

Running FCA UNR using Tcl command

Use the `check_fv` Tcl command to run FCA UNR from the Tcl shell interactively.

Running FCA UNR using GUI

The FCA UNR can also be run from the GUI. You can invoke FCA UNR run in GUI mode:

```
vcf -f <your_fca_unr.tcl> -verdi
```

The property table of the "COV" task are only visible from the "COV" Appmode.

Running FCA UNR in Batch Mode

Use the following command to run FCA in batch mode:

```
vcf -f <your_fca_unr.tcl> -batch
```

Setting Up FCA UNR to Run on Grid Compute Farm

Compute farm can provide more resources therefore better chance of improved performance in terms of run time and convergence rate. Refer to section for information on how to configure grid workers, see Running on Compute Farm.

Setting up FCA UNR for Multiple Tasks

Multiple tasks and parallel runs are also supported for FCA. This technique can be used to partition of the big problems to smaller ones, and thereby more resources can be devoted to the partitioned problems in separate tasks. For more information on verification tasks, see section Creating Verification Tasks.

Feedback

11.2.4 Saving the Exclusions Using Tcl

For FCA UNR, since there are no or minimal constraints, the covered goals are not interesting. The uncoverable goals are the important ones. VC Formal provides mechanism to save the uncoverable in the exclusion format with annotation, so that you can review, debug, and confirm the exclusion from Verdi Coverage GUI.

This can be done with Tcl commands in the following steps.

- When the generated exclusion file is too large for toggle coverage, it takes time to load in Verdi/Coverage database.

  For example, 256bit dead-code DUMMY, FCA generates 256 items of toggle exclusion:

  ```
  wire sel = 0;
  wire [255:0] DUMMY;
  assign DUMMY[255:0] = sel ? {(256){waiting_main}} : {(256){1'b1}};
  ```

  In such cases, you can compress the 256 items of toggle exclusion to 1 exclusion item, for example, *Toggle: DUMMY[255:0]*, thus reducing the exclusion file size.

  To enable the compression mode for the exclusion files for toggle coverage, use the following formal variable:

  ```
  set_fml_var fca_el_compress true
  ```

- To save the exclusion files and VDB incrementally, so that the results generated at regular intervals do not get lost, set the following formal variable:

  ```
  set_fml_var fca_save_incr_mode <el , vdb, both>
  ```

  When the fca_save_incr_mode formal variable is set, by default, the exclusion files and VDB are saved every 30 minutes (30M).

  You can control the time intervals for saving the exclusion files and VDB incrementally using the `fca_save_incr_time` formal variable:

  ```
  set_fml_var fca_save_incr_time < xS, xM, xH >
  ```

- Save the uncoverable goals to exclusion file. The `save_cov_exclusion` command is used to generate coverage exclusion file for coverage goals which have been detected as uncoverable by the VC Formal.

  When the command is applied successfully, a 1 is returned; 0 otherwise. The syntax of the command is as follows:

  ```
  save_cov_exclusion -file <filename> [-append]
   [-missing_else] [-cov <metric_type>] [-line_intent]
   [-no_line_intent] [-filter <class>] [-targets
   <list-of-coverage-goal-names-ids-or-collection-of-coverage-goals>]
  ```

Feedback

```
[-enable_top_mod_excl] [-mod_excl] [-mod_excl_scope
<list_of_modules>] [-strict] [-scope <module-instance>]
```

Where,

- `-file <{elFileName}>` : Specifies the name of the exclusion file in which the un-coverable information needs to be dumped by VC Formal. If just the file name is given then it would be saved in the current working directory.
- `[-append]`: Specifies if un-coverable information needs to be appended to an already existing exclusion file.
- `[-missing_else]`: When this option is specified, if missing else is present in the design, then that should also get covered in the exclusion file.
- `[-cov <{metric_type}>]`: Specifies the coverage metric for which the exclusion file should be written. Valid coverage property types are listed and are separated by the plus + delimiter: `line, cond, toggle, fsm_state, fsm_trans, branch, cg, assert, cover, all`. The keyword `all` is supported for dumping the exclusion file for all structural coverage property types. Since `-cov` is an optional argument, if it not specified, all coverage metrics would be dumped. This parameter is mandatory.
- `[-line_intent]`: When this option is specified, only the coverage goals which are unintentionally covered or intentionally uncoverable are saved in the exclusion file.
- `[-no_line_intent]`: When this option is specified, the coverage goals which are unintentionally covered or intentionally uncoverable are filtered in the exclusion file.
- `[-filter <class>]`: Specify the filter expression that must be used to select properties based on the various attributes and values of those properties.
- `[-targets <{list-of-coverage-goal-names-or-collection-of-coverage-goals}>]`: Specifies a list of coverage goals that need to be saved in the exclusion file.
- `[-enable_top_mod_excl]`: This option dumps all the unreachable targets for the top module and its sub-hierarchy. For the top module, it saves the exclusion with module name whereas the subhierarchy with the instance hierarchy names. Upon re-applying the module based exclusions, all unreachables can be waived off for the module as well as its self instances in one go.

  When the simulation coverage database is used, then exclusion files look like below-

  ```
  MODULE: traffic
  ANNOTATION: "VC_COV_UNR"
  Toggle 0to1 state_first [0] "net state_first[4:0]"
  CHECKSUM: "3381384760 4023159756"
  INSTANCE: TB.T1.main
  ```

Feedback

```
Fsm state_out "4023159756"
ANNOTATION: "VC_COV_UNR"
Transition RESET->SR0 "1->2"
```

When simulation coverage database is not used, exclusion file appears as following-

```
MODULE: traffic
ANNOTATION: "VC_COV_UNR"
Toggle 0to1 state_first [0] "net state_first[4:0]"
CHECKSUM: "3381384760 4023159756"
INSTANCE: traffic.main
Fsm state_out "4023159756"
ANNOTATION: "VC_COV_UNR"
Transition RESET->SR0 "1->2"
```

- `[-mod_excl]`: This option dumps the common set of unreachables for all instances of a specific module.
- `[-mod_excl_scope <list_of_modules>]`: Specifies list of modules for which common set of unreachables need to be dumped in exclusion file. To be used with -mod_excl.
- `[-strict]`: Enable strict mode of dumping exclusion file. When this option is specified, the elfile is dumped in the strict mode. Such elfiles when loaded with coverage database using URG/Verdi `-excl_strict` option, and if there is mismatch between the status of any coverage targets (for example, covered in the database and excluded in the elfile), then Verdi/URG reports a warning message.
- `[-scope <module-instance>]`: Saves Exclusion file for modules/instances.
- **Examples**
  - By default saves all UNRs

    ```
    save_cov_exclusion -file unr.el
    ```

  - saves the covered goals as el

    ```
    save_cov_exclusion -file covered.el -targets [get_props -status covered]
    ```

- While reviewing the uncoverable default case items, it can be overwhelming and uninteresting if there are too many such goals. Setting of the `set_fml_var fml_cov_ignore_case_default true` application variable helps in pruning out such default case items from coverage analysis.

  Set this application variable before the `elaborate`/`read_file` command.

Feedback

11.2.5 Saving the Coverage Database Using TCL

When a simulation database is imported into FCA, only uncovered coverage goals are run in FCA. At the end of the run, VC Formal dumps an exclusion file for the coverage goals which are found uncoverable. You can save the formal covered/uncoverable coverage targets into a new or existing coverage database using the `save_covdb` command.

```
save_covdb -status covered -name <newCovDBName >
```

It must be called before `view_coverage`. It also works with the save/restore mechanism. The detail information of the command is as follows:

Use Model

```
save_covdb
-name <covdb_name> [-update]
[-overwrite]
[-cov <metric_type>] [-line_intent]
[-no_line_intent]
[-status <list-of-status-attributes>]
```

Where,

- `-name`: Name of the coverage database.
- `-update`: Updates the new information into an existing coverage database as a new test file.
- `-overwrite`: Overwrites the existing coverage database (if exists) and creates a new one.
- `-line_intent`: Saves only coverage goals which are unintentionally covered or intentionally uncoverable.
- `-no_line_intent:` Filters out coverage goals which are unintentionally covered or intentionally uncoverable.
- `-status <list-of-status-attributes>`: Specifies the status type of coverage goals that must be saved. Values: covered, uncoverable.
- `-cov <metric type>`: Specifies the coverage metric type of excluded objects that must be saved.

  Note:

  The -cov option is an optional argument, if not specified, it would mean dumping out for all coverage metrics. However, covers and asserts will only

Feedback

be saved if we provide them explicitly in the -cov option. They are not picked up by default.

When you don't specify -status covered, the uncoverable goals will be directly removed from the coverage db. Therefore, Exclusion manager in Verdi Coverage GUI will not show the uncoverable goals excluded.

11.2.6 Review the Coverage from Verdi GUI

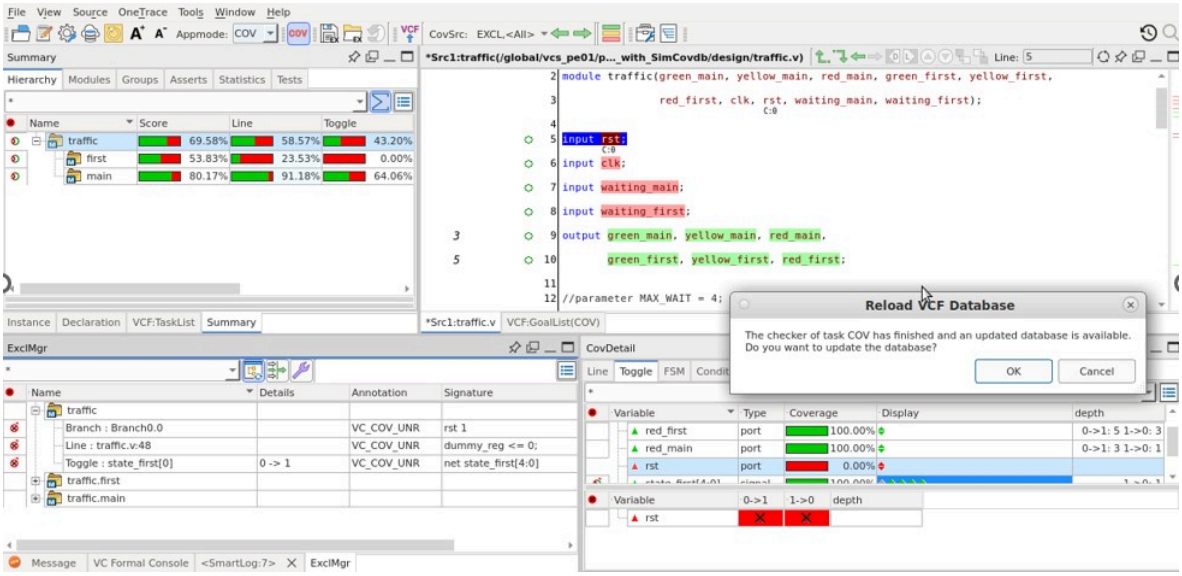
When the coverage run is ongoing, tool automatically launches the coverage view integrated in VCF GUI. Refer the “Summary” tab next to Tasklist showing the % contribution from different metrices.

Once, the run is completed, it prompts to update the existing coverage database.

For debug purpose, user can openVerdi Coverage GUI to review the coverage data with the exclusion results using the following TCL command:

```
view_coverage -cov_input <saved database> -elfile <name of exclusion
 file>
```

Figure 71: Coverage Database Update from GUI



User can review the uncoverable goals from the Exclusion Manager and decide whether they should be accepted as waiver items.

Feedback

Note:

When FCA is run from GUI, tool automatically saves the coverage database and exclusion file at following path:

```
<>/vcst_rtdb/cov_debug/COV_UA_Auto_<>.vdb
<>/vcst_rtdb/cov_debug/COV_UA_Auto_<>.el
```

11.2.7 Saving and Restoring FCA UNR Sessions

FCA UNR results can be saved and restored. For more information on saving and restoring sessions, see section Saving and Restoring Sessions.

11.2.8 Viewing Progress of FCA Run

The FCA UNR progress report is different from FPV App, where as the task progress report functionality is the same, as described in theViewing the Task Progress Report.

11.2.9 Reviewing Results of VC Formal FCA Run

For FCA UNR, the run results can have these statuses:

- covered
- uncoverable
- inconclusive

The results can be reviewed from GUI property table. User can also review the results and generate report using Tcl commands.

Use the `report_fv` Tcl command to get a report the run result:

Example of the output is as follows:

- Summary report

```
report_fv -type [all/line/condition] command by default gives a
 summary report:
  Summary Results
   Property Summary: COV
   -----------------
   > Constraint
     - # found        : 1

   Coverage Summary: COV
   -----------------
   > Line Coverage
     - # found        : 55
```

Feedback

```
    - # covered      : 46
    - # uncoverable  : 9

  > Condition Coverage
    - # found        : 18
    - # covered      : 18

  > FSM State Coverage
    - # found        : 10
    - # covered      : 10
```

- Verbose report

  Use the `report_fv -verbose -type [all/line/condition]` option to get a detailed report:

```
report_fv -verbose

  Verbose Coverage List:
  ----------------------
  > Line Coverage
    # Line Coverage: 55
    > ID: [0] covered (depth=1)
     - name          : traffic.line_0
     - type          : line
     - line_type     : ALWAYS
     - usage         : cover
     - update_time   : 00:37:50 - Oct 25
     - elapsed_time  : 00:00:01
     - converge_time : 00:00:01
     - location      : design/traffic.v:48
     - engine        : e1
     - safe_depth    : 0
     - trace_depth   : 1
```

- List report

  Use the `report_fv -list -type [all/line/condition]` command to get a list:

```
Coverage List:
   -------------
   > Line Coverage
     # Line Coverage: 55
     [    0] covered     (depth=1)                    -  traffic.line_0
 (design/traffic.v:48)
     [    1] uncoverable                              -  traffic.line_1
 (design/traffic.v:48)
     [    2] covered     (depth=1)                    -  traffic.line_2
 (design/traffic.v:49)
     [   61] covered     (depth=1)                    -
 traffic.main.line_0 (design/vlog_street_ctrl_fsm.v:30)
```

Feedback

```
    [   62] uncoverable                                -
traffic.main.line_1 (design/vlog_street_ctrl_fsm.v:31)
    [   63] covered     (depth=0)                      -
traffic.main.line_10 (design/vlog_street_ctrl_fsm.v:58)
```

11.2.10 Viewing Coverage Report Using URG

For FCA UNR, since there are no or minimal constraints, the covered goals are not interesting. The uncoverable goals are the important ones. VC Formal provides mechanismto save the uncoverable in the exclusion format with annotation, so that you can review, debug and confirm the exclusion from Verdi Coverage GUI.

1. To view html report of the formal.vdb, execute urg commands from unix terminal

   ```
   urg -dir formal.vdb
   firefox urgReport/dashboard.html
   ```

   For more information on URG, refer to the Coverage Technology Reference Manual.

2. To merge multiple vdb

   ```
   urg -dir a.vdb -dir b.vdb -dbname ab.vdb
   ```

3. To view URG report based on metric

   ```
   urg -dir formal.vdb -elfile a.el -metric line
   ```

11.2.11 Debugging VC Formal FCA Results

If you need to further analyze why a particular coverage target is uncoverable, you can run analysis on the coverage goal to compute reduced constraints that contributed to the uncoverable, or to compute formal core to get the additional logic needed for the formal engines to determine the uncoverable result.

Example

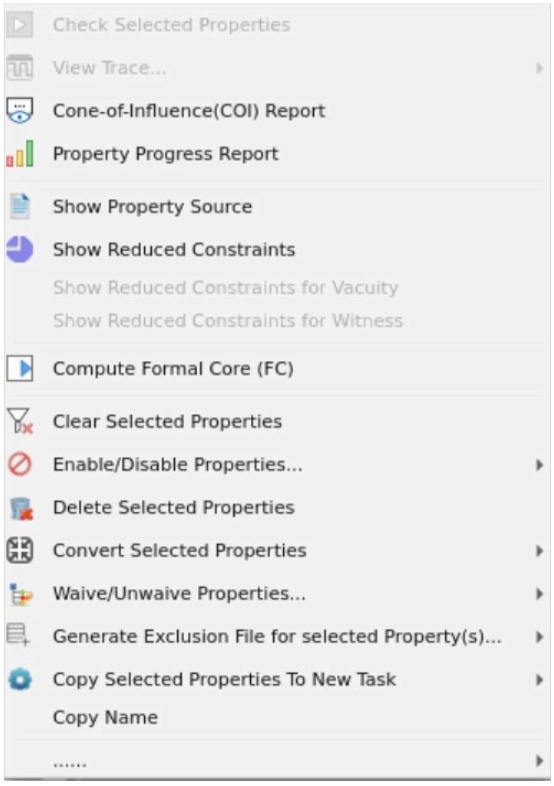
- Using Tcl command

  ```
  compute_reduced_constraint <property_name>
  ```

- Using GUI (as shown in Figure 72)

Feedback

Figure 72: Get Reduced Constraints from GUI



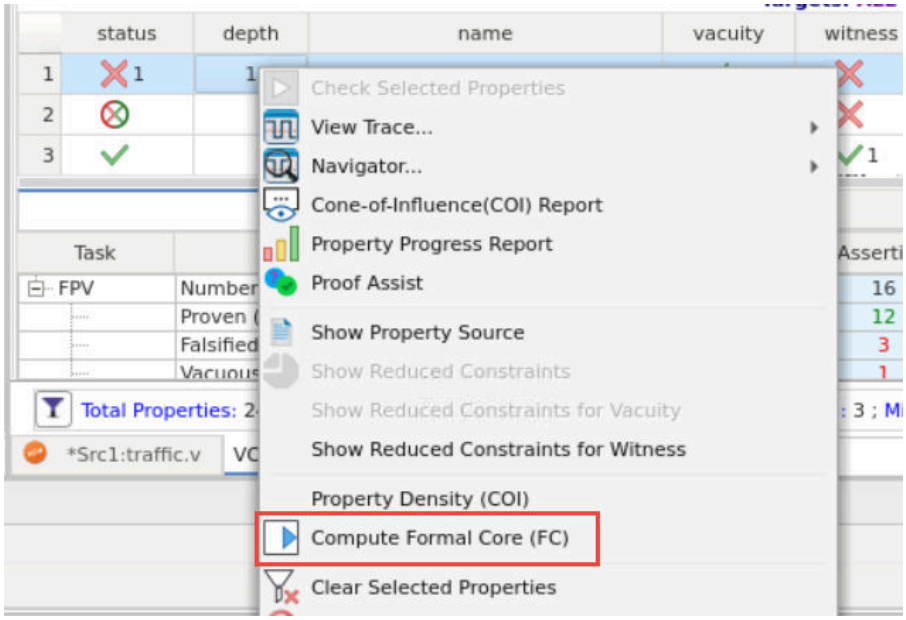
- Or, using Tcl command

```
compute_formal_core <property_name>
```

Feedback

- Using GUI (as shown in Figure 73)

Figure 73: Compute Formal Core from GUI

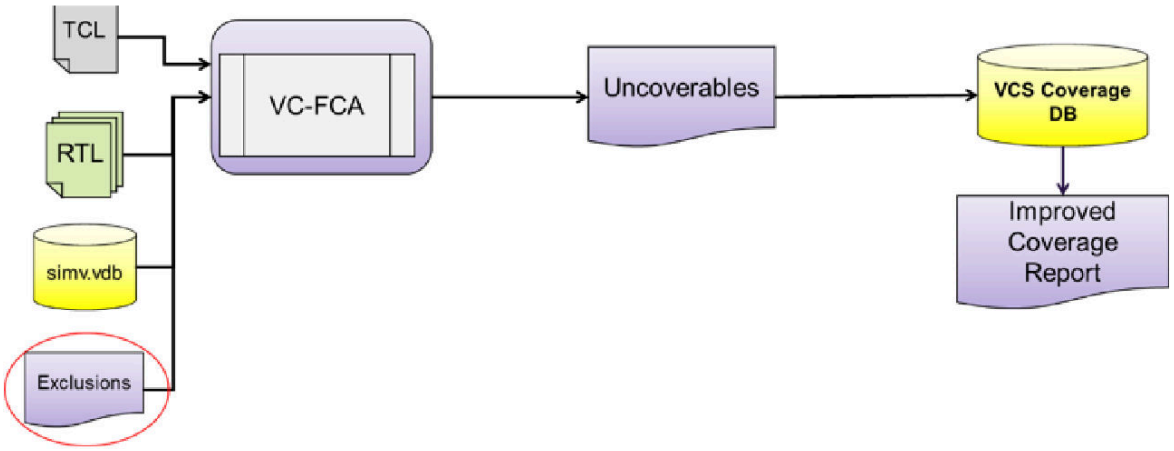


11.2.12 FCA UNR In Project Regression Phase: Ignore Accepted Exclusions

VC Formal Coverage Analyzer enables you to save the uncoverable goals in the form of an exclusion file. Initially, you may not have any exclusions, as these get generated during the initial run. After the exclusions are reviewed and accepted, VC Formal Coverage Analyzer offers the capability to read the user accepted exclusions, then ignore these coverage goals for the next run as shown in Figure 10-10. This way, you need not review the same exclusions repeatedly.

Feedback

Figure 74: FCA UNR Use Model 3: Regression Phase



You will need to modify the initial FCA Tcl file and add the -elfiles option to this command:

```
read_covdb -cov_input <dbPath> -cov_dut <instName> -elfiles
 <listOfElFiles>
```

The goals specified in the `-elfiles` will not be targeted in the current run, so more resources can be devoted to any goals that are still uncovered from the vdb database.

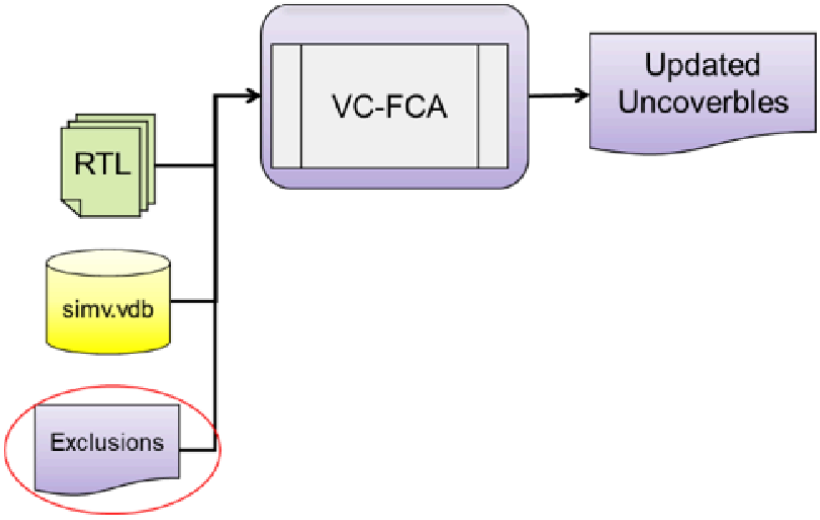
11.2.13 FCA UNR in Late Verification Phase: Validate Exclusions

As the regression tests continue to be run and FCA UNR continue to be run in regression mode, more and more exclusion items are be accumulated. If there are late stage design change, it is possible that RTL changes at the late stage after the exclusion files have been generated could have changed the status of those uncoverable goals.

In such cases, to ensure the uncoverables are still valid, VC Formal Coverage Analyzer offers the capability to read the user accepted exclusions, then check only these coverage goals as shown in Figure 10-11.

Feedback

Figure 75: FCA UNR Use Model 4: Late Phase Validating Uncoverables



You will need to add one more option to the Tcl command:

```
read_covdb -cov_input <dbPath> -cov_dut <instName> -elfiles
 <listOfElFiles> -check_el
```

You should be able to accept any uncoverable goals from this run as they have already been reviewed before.

11.3 FPV Signoff Coverage Features in FCA

In addition to UNR, FCA is also used for FPV signoff with similar setup and usage. These signoff coverage feature includes the following

- Property Density structural coverage
- Over-constraint unreachability coverage
- Bounded proof reachability coverage
- Formal Core reachability Coverage
- FTA reachability coverage

For more information on these, see section Obtaining Formal Signoff For a Design and Formal Testbench Analyzer App.

Feedback

11.4 Intentional Uncoverable Analysis

VC Formal FCA enables you perform the following types of analysis:

- Intentional Uncoverable Analysis
- Intentional Unreachability Analysis for Ternary Operators and Condition Coverage Metric
- Intention Based Analysis for Else/Default Branch Line of FSM/x-assign

11.4.1 Intentional Uncoverable Analysis

VC Formal Coverage Analyzer has the ability to determine if each line of RTL source code is covered or uncoverable, given the initial state and constraints specification. Now, VC Formal can also classify covered and uncoverable source code lines by intent, according to the customer's coding guidelines. If a subset of uncoverable source code lines are identified as intentionally uncoverable, you can optionally filter those lines out from the uncoverable line reports. This reduces the effort of engineers who need to review those reports. VC Formal also identifies covered lines, that should not be covered according to the customer's coding guidelines. This is reported as unintentional covered, for further investigation by designers.

Definition of Intentional Uncoverable: A statement can be generalized to be intentionally uncoverable when the statement is found to be uncoverable and it is:

- Non-FSM and default/branch assigning X value
  - Full condition analysis is not required
- FSM and default/branch

Definition of Unintentional Covered: A statement can be generalized to be unintentionally covered when the statement is found to be covered and it is:

- Non-FSM and default/branch assigning X value
  - Full condition analysis is not required
- FSM and default/branch

  Note:

  For this analysis, VCS rules of identifying FSM are used.

This feature is disabled by default. It can be enabled using the following command:

```
set_app_var fml_cov_enable_int_unr true
```

Feedback

Intentional Uncoverable Examples

Non-FSM

```
always @(posedge clk or negedge rst_x) begin
if (!rst_x) begin
  d2 <= 2'b00; // Intentional Covered
end else begin
  if (din==2'b00) d2 <= 2'b00; // Intentional Covered
  else if (din==2'b01) d2 <= 2'b01; // Intentional Covered
  else if (din==2'b10) d2 <= 2'b10; // Intentional Covered
  else if (din==2'b11) d2 <= 2'b11; // Intentional Covered
  else d2 <= 2'bXX; // Intentional Uncoverable
  end
end
```

FSM

```
S0 = 2'b00; parameter S1 = 2'b01; parameter S2 = 2'b10;
always @(posedge clk or negedge rst_x) begin
if (!rst_x) begin
  fsm0 <= S0; // Intentional Covered
end else begin
  case (fsm0)
  S0:fsm0 <= S1; // Intentional Covered
  S1:fsm0 <= S2; // Intentional Covered
  S2:fsm0 <= S0; // Intentional Covered
  default: fsm0 <= S0; // Intentional Uncoverable
  endcase
end
end
```

Unintentional Covered Examples

Non FSM

```
always @(posedge clk or negedge rst_x) begin
if (!rst_x) begin d4 <= 2'b00; // Intentional Covered
end else begin
case (din)
2'b00:d4 <= 2'b01; // Intentional Covered
2'b01:d4 <= 2'b10; // Intentional Covered
2'b10:d4 <= 2'b11; // Intentional Covered
default: d4 <= 2'bXX; // Unintentional Covered
endcase
end
end
```

FSM

```
parameter S0 = 2'b00; parameter S1 = 2'b01; parameter S2 = 2'b10;
 parameter SX = 2'b11; // illegal
always @(posedge clk or negedge rst_x) begin
```

Feedback

```
if (!rst_x) begin
fsm1 <= 2'b00; // Intentional Covered
end else begin
case (fsm1)
S0:     fsm1 <= S1; // Intentional Covered
S1:     fsm1 <= S2; // Intentional Covered
S2:     fsm1 <= SX; // Intentional Covered
default: fsm1 <= S0; // Unintentional Covered
endcase
end
end
```

11.4.2 Intention Based Analysis for Else/Default Branch Line of FSM/x-assign

You can set the styles for which intention based analysis must be performed using the `set_fml_var fml_cov_int_unr_style` application variable.

This application variable accepts the following values: `x-assign`, `fsm`, `all`. The default value is `all`.

- `x-assign`: If you set the application variable to x-assign, then intention analysis is performed only for the last else/default branch line of x-assign.
- `fsm`: If you set the application variable to fsm, then intentional analysis is performed only for the last else/default branch line of FSM
- `all`: If you set the application variable to all, then intentional analysis is performed for both x-assign and fsm style

Table 17: Behavior of VC Formal on Enabling/Disabling fml_cov_int_unr_style and fml_cov_enable_int_unr

fml_cov_enable_int_unr	fml_cov_int_unr_style	Output
true	not set	VC Formal performs intentional analysis for both x-assign/fsm and hence it is equivalent to setting the fml_cov_int_unr_style application variable to all.
false	set	VC Formal does not perform intentional analysis and the fml_cov_int_unr_style application variable has no effect. A warning message is issued saying that the intentional analysis style can be set only when int-unr analysis is enabled, by setting the fml_cov_enable_int_unr application variable to true.
true	set	VC Formal performs intentional analysis based on the style set using the fml_cov_int_unr_style application variable.

Feedback

11.4.3 Intentional Unreachability Analysis for Ternary Operators and Condition Coverage Metric

To perform unreachability analysis of ternary operators, you must enable condition coverage. To enable condition metric with VC Formal Coverage Analyzer, pass the `-cov cond` command to the `read_file`/`elaborate` command. This extracts condition coverage vectors which can be mapped to the control structure for the different branches of ternary operators.

This feature is disabled by default. It can be enabled using the following command

```
set_app_var fml_cov_enable_int_unr true
```

When the intentionally unreachability analysis is enabled, VC Formal performs intent based coverage analysis for condition coverage vectors.

Examples of Ternary Operators

Consider the following continuous assignment statement involving ternary operators in the RHS:

```
assign q = (a == 1'b0) ? 1'b0 : (a==1'b1) ? 1'b1: 1'bx);
```

When condition coverage is enabled, VC Formal extracts coverage goals for the two conditions (a==1'b0) ? and (a==1'b1) ? which appears in the RHS of the above assign statement. The following condition coverage vectors that are extracted for these two conditions:

- Condition coverage extracted for (a==1'b0) ?

  For the RHS of the above assignment, the condition (a==1'b0) ? can have two possible values 1 and 0. Hence, VC Formal would extract two vectors 1 and 0 for this top level condition.

- Condition coverage report for (a==1'b0) ?

  The following is the snapshot of the coverage report for the previous expression (generated by Unified Report Generator). The underline marker -----1----- refers to the actual expression for which the condition coverage is reported. The status table indicates the coverage vectors and their corresponding coverage status.

  ```
  EXPRESSION: ((a == 1'b0) ? 1'b0 : ((a == 1'b1) ? 1'b1 : 1'bx))
  -----1-----
  -1- Status
  0 Not Covered
  1 Not Covered
  ```

- Condition coverage extracted for (a==1'b1) ?

Feedback

For the RHS of the previous assignment, the condition (a==1'b1) ? can also have two possible values 1 and 0. VC Formal would therefore extract two vectors 1 and 0 for this condition.

- Condition coverage report for (a==1'b1) ?

```
SUB-EXPRESSION ((a == 1'b1) ? 1'b1 : 1'bx)
-----1-----
-1- Status
0 Not Covered
1 Not Covered
```

Mapping of Condition Coverage Goals to Branches of Ternary Operators

For the example assign q = (a == 1'b0) ? 1'b0 : (a==1'b1) ? 1'b1: 1'bx), the following top level conditions map to the two different branches of the ternary operators and out of these, (a==1'b1) ? can be a potential target for performing intent-based analysis since x-assignment happens in its false path.

```
(a == 1'b0) ? and   (a == 1'b1) ?
-----1----- ----1-----
```

Therefore, VC Formal marks the condition coverage vector 0 for the expression (a==1'b1) ? as intentionally uncoverable.

When condition coverage is enabled in VC Formal Coverage Analyzer, goals are automatically extracted for other expressions as well along with ternary operators. For example, control structure of FSM/Non-FSM performing an explicit x-assignment. They can also be a potential target for intent-based analysis.

Condition Coverage Goal Extraction

The condition coverage support set can be refined to address intent based analysis for condition coverage metric as a whole.

By default VC Formal Coverage Analyzer extracts condition coverage goals in sync with whatever is inferred by VCS when the `-cm cond` option is given to VCS.

VC Formal also supports all existing VCS sub-switches of condition coverage which can be specified by -cm_cond such as allops, obs, allvectors, scalarbitwise.

When intentional unreachability is enabled, the entire support set is refined and VC Formal Coverage Analyzer only extracts those condition coverage goals that have vectors 0 and 1; the rest is dropped.

In case a top level condition is made up of sub-conditions but does not have vectors 0 and 1, all its sub- conditions are dropped as well.

Refinement of Condition Coverage Goals Extraction for Intent Based Coverage Analysis

To perform intent based analysis for ternary operators extracting condition coverage goal only for the top level condition which has only two vectors 0 and 1 is sufficient. There is no need to get into the sub condition level.

11.4.4 Saving Intentional Uncoverable Lines in an Exclusion File

The following commands can be used to filter out the intentional uncoverable lines, and save them in an exclusion file.

- Filtering out intentional uncoverable lines while saving exclusion file:

```
save_cov_exclusion -file <elfilName> -no_line_intent
```

- Saving only intentional uncoverable lines in exclusion file:

```
save_cov_exclusion -file <elfilName> -line_intent
```

11.4.5 Reporting and Filtering

Use the following commands to filter out the intentional uncoverable/unintentional covered from the results.

- Filtering out intentional uncoverable lines:

```
report_fv -status uncoverable -no_line_intent
```

- Filtering out unintentional covered lines:

```
report_fv -status covered -no_line_intent
```

- Reporting only intentional uncoverable lines:

```
report_fv -status uncoverable -line_intent
```

- Reporting only unintentional covered lines:

```
report_fv -status covered -line_intent
```

For example, the `-no_line_intent -status covered` command, filters out the intentional uncoverable lines, and shows only the true uncoverable lines.

Similarly, the`-no_line_intent -status covered`command, filters out the unintentional covered lines, and shows only the true covered lines.

Sample Output

Feedback

- `report_fv -list -status uncoverable`

```
[ 6] Uncoverable (Line Coverage) - test (/test.sv:28)
[ 13] Uncoverable (Line Coverage) - test (/test.sv:45)
[ 24] Uncoverable (Line Coverage) - test (/test.sv:66)
[ 33] Uncoverable (Line Coverage) - test (/test.sv:85)
[ 42] Uncoverable (Line Coverage) - test (/test.sv:104)
[ 51] Uncoverable(Line Coverage) - test (/test.sv:123)
```

- `report_fv -list -status uncoverable -no_line_intent`

```
[ 6] Uncoverable (Line Coverage) - test (/test.sv:28)
[ 24] Uncoverable (Line Coverage) - test (/test.sv:66)
[ 33] Uncoverable (Line Coverage) - test (/test.sv:85)
```

The line_intent Attribute

The `line_intent` attribute is supported for both line and condition coverage. Filtering is performed based on that. Therefore, unintentionally covered and intentionally uncoverable line and condition coverage goals can be filtered using the following command:

```
report_fv -filter { ( type==line || type==condition) &&
 ((line_intent==INT_UNR_UNINT_RCH && status==covered ) ||
 (line_intent!=UNINT_UNR_INT_RCH&& status==uncoverable )) }
```

11.4.6 Limitations of Intent Based Coverage Analysis

The following condition coverage goals would be extracted by VC Formal only for intent based coverage analysis:

- Support set for condition coverage extraction:
  - Only top level conditions are extracted by the tool that has only two vectors 0 and 1
- Unsupported set for condition coverage extraction:
  - Top level conditions having vectors other than 0 and 1
- Sub conditions

Intent Based Analysis for Condition Coverage Goals

With the rules in place for condition coverage extraction, any vector whose unreachability leads to a line that is intentionally uncoverable is reported as an intentionally uncoverable condition. This is also the same for unintentionally covered condition.

Consider an example scenario of refinement of condition coverage goal extraction for intent based analysis and their corresponding marking for intentionally uncoverable/ unintentionally covered.

Top Level Condition has only Two Vectors 0 and 1

Feedback

```
if(a==1'b1)
…
else
z = 1'bx
```

In this case, by default, the tool does not extract any condition coverage target.

However, if condition coverage options such as -cm_cond allops+allvectors+... are given, the expression (a==1) would have only two vectors 0 and 1. Hence the tool extracts condition coverage goals for them and the vector 0 would be a potential target for intent based analysis since its unreachability would contribute to the line z=1'bx to be intentionally uncoverable.

If the line z=1'bx is found to be uncoverable by the tool then the vector 0 of the expression (a==1'b1) is reported as intentionally uncoverable by the report_fv -verbose command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] intentionally uncoverable
- name : top.condition_0
- type : condition
- expression : (a == 1'b1)
- condition_vector : 0
- usage : cover
- location : module.v:5
> ID: [1] Covered
- name : top.condition_1
- type : condition
- expression : (a == 1'b1)
- condition_vector : 1
- usage : cover
- location : module.v:5
```

If the line z=1'bx is found to be covered by the tool, the vector 0 of the expression (a==1'b1) is reported as unintentionally covered by the report_fv -verbose command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] unintentionally covered
- name : top.condition_0
- type : condition
- expression : (a == 1'b1)
- condition_vector : 0
- usage : cover
- location : module.v:5
> ID: [1] covered
- name : top.condition_1
- type : condition
- expression : (a == 1'b1)
- condition_vector : 1
```

```
- usage : cover
- location : module.v:5
```

Top Level Condition does not have Vector 0 and 1

```
if(a &&b)
…
else
z = 1'bx
```

In this case, the expression (a&&b) does not have the vectors 0 and 1. Instead, it has the vectors 01, 10 and 11. This does not comply with the predefined rule of extracting condition coverage goal only for top level condition having only vectors 0 and 1. The tool therefore, does not extract any condition coverage goal for (a&&b) when intent based analysis is enabled.

Top Level Condition has vector 0 and 1 Along with Sub-conditions

```
if((a &&b) ==1)
…
else
z = 1'bx
```

In this case, by default the tool does not extract any condition coverage target.

However, if condition coverage options like-cm_cond allops+allvectors+... etc are given, the expression ((a&&b) ==1) would have top level condition with vectors 0 and 1.

If the line z=1'bx is found to be uncoverable by the tool, the vector 0 of the expression ((a&&b)==1) is reported as intentionally uncoverable by the report_fv -verbose command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] intentionally uncoverable
- name : top.condition_0
- type : condition
- expression : ((a && b) == 1)
- condition_vector : 0
- usage : cover
- location : module.v:5


> ID: [1] covered
- name : top.condition_1
- type : condition
- expression : ((a && b) == 1)
- condition_vector : 1
- usage : cover
- location : module.v:5
```

Feedback

If the line z=1'bx is found to be covered by the tool then the vector 0 of the expression ((a&&b)==1) is reported as unintentionally covered by the report_fv -verbose command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] unintentionally Covered
- name : top.condition_0
- type : condition
- expression : ((a && b) == 1)

- condition_vector : 0
- usage : cover
- location : module.v:5


> ID: [1] covered
- name : top.condition_1
- type : condition
- expression : ((a && b) == 1)
- condition_vector : 1
- usage : cover
- location : module.v:5
```

Top Level Condition in Ternary Operators Having Both Vector 0 and 1 Along with Sub-conditions

assign a = (a || b) ? 1'b0 : 1'bx;

In this case, by default the tool extract condition coverage only for the top condition (a|| b) ?.

However, if condition coverage options like -cm_cond allops+allvectors+... etc are given, the expression (a||b) ? would have top level condition with vectors 0 and 1 and also a sub-condition for (a||b) with condition vectors as 0 and 1.

The tool extracts condition coverage goal only for the top level condition with vectors 0 and 1 and ignores the sub-condition and its associated condition vectors.

If the vector 0 of the expression (a||b) ? is found to be uncoverable by the tool, it is reported as intentionally uncoverable by the report_fv -verbose command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] intentionally uncoverable
- name : top.condition_0
- type : condition
- expression : (((a||b) == 1'b1) ? 1'b0 : 1'bx)
- condition : -----------
- condition_vector : 0
- usage : cover
- location : module.v:3
```

Feedback

```
> ID: [1] covered
- name : top.condition_1
- type : condition
- expression : (((a||b) == 1'b1) ? 1'b0 : 1'bx)
- condition : -----------
- condition_vector : 1
- usage : cover
- location : module.v:3
```

If the vector 0 of the expression (a||b) ? is found to be covered, it is reported as unintentionally covered by the `report_fv -verbose` command as follows:

```
> Condition Coverage
# Condition Coverage: 2
> ID: [0] unintentionally Covered
- name : top.condition_0
- type : condition
- expression : ((a == 1'b1) ? 1'b0 : 1'bx)
- condition : -----------
- condition_vector : 0
- usage : cover
- location : module.v:3

> ID: [1] covered
- name : top.condition_1
- type : condition
- expression : ((a == 1'b1) ? 1'b0 : 1'bx)
- condition : -----------
- condition_vector : 1
- usage : cover
- location : module.v:3
```

11.5 Coverage Goal Waiving Mechanisms

In general, the waive and unwaive is done through exclusion management. There are also several other different waiving mechanisms can be used to waive and unwaive coverage goals:

- On the fly waiving while session is in progress

  The advantage to waive or unwaive as the run is in progress is as follows:

  - Easy, persistent exclusion/waiver of illegal areas
  - Waive and remove from the coverage status targets that are not required, thereby saving time and effort needed to achieve coverage closure.

- Using Tcl commands

- From VC Formal GUI
- From Verdi Coverage GUI

11.5.1 Waive and Unwaive of Coverage Goals Using Tcl Commands

Reading Coverage Waiver file

You need not import a coverage database to read the waiver file. VC Formal automatically maps the code coverage targets from the waiver file to the coverage goals in VC Formal shell. The matching code coverage goals are tagged as waived, and the total goal count shown by the `report_fv` command reduces.

VC Formal also reports the applied waiver summary count, and also saves a report highlighting the details of the waived targets in an .txt file.

Use Model

```
vcf> read_waiver_file
[-elfiles <elFileName(s)>]
```

Where,

- `[-elfiles {<elFileName(s)>}]`: Specifies the path of the coverage waiver files that must be imported.

The command returns 1 if successful, else 0.

Reading Coverage Waiver file

You can use the `read_waiver_file` command to import a preexisting coverage waiver (exclusion) file without having to import a coverage database. Use this command only after the `read_file`/`elaborate` command, that is, when the goals are already present in the vcf.

Saving the Waived Code Coverage Targets

You can use the `save_waiver_file` command to save the in-memory waived code coverage targets (previously waived through the `read_waiver_file` or `fvwaive` command) in a waiver (coverage exclusion) file. The saved waiver file can be loaded in Verdi coverage GUI using the `view_coverage` command. If a waiver file from a previous version of RTL is loaded in Verdi coverage GUI, then only those exclusions would be applied for which the design matches. The remaining exclusions are dropped.

Use Model

```
save_waiver_file
-file <fileName> [-append ] [-cov <metric_type>] [-fta] [-filter <class>]
```

Feedback

```
[-targets
 <list-of-coverage-goal-names-ids-or-collection-of-coverage-goals>]
```

Where,

- `[-file]`: Specifies the file name in which to save the in-memory waived coverage targets.
- `[-append]`: Appends waived code coverage targets to an existing exclusion file.
- `[-cov <metric_type>]`: Specifies the coverage metric for which the exclusion file should be written. Valid coverage property types are listed and are separated by the plus + delimiter: (1)line (2)cond (3) toggle (4) fsm_state (5) fsm_transition. The keyword all is supported for dumping the exclusion file for all structural coverage property types. Since -cov is an optional argument, if it not specified, all coverage metrics would be dumped. This parameter is mandatory.
- `[-fta]`: Enables saving of waived faults for FTA
- `[-filter <class>]`: Specify the filter expression that must be used to select prop erties based on the various attributes and values of those properties.
- `[-targets <list-of-coverage-goal-names-ids-or-collection-of-coverage-goals>]`: Alternatively accepts a collection returned by the get_props command. The command returns 1 if successful, else 0

11.5.2 Waiving and Unwaiving of Coverage Goals Using VC Formal GUI

You can also perform an in-memory waiving and unwaiving of coverage goals from the VC Formal GUI.

- Waive/Unwaive option in Property Table RMB pop-up menu

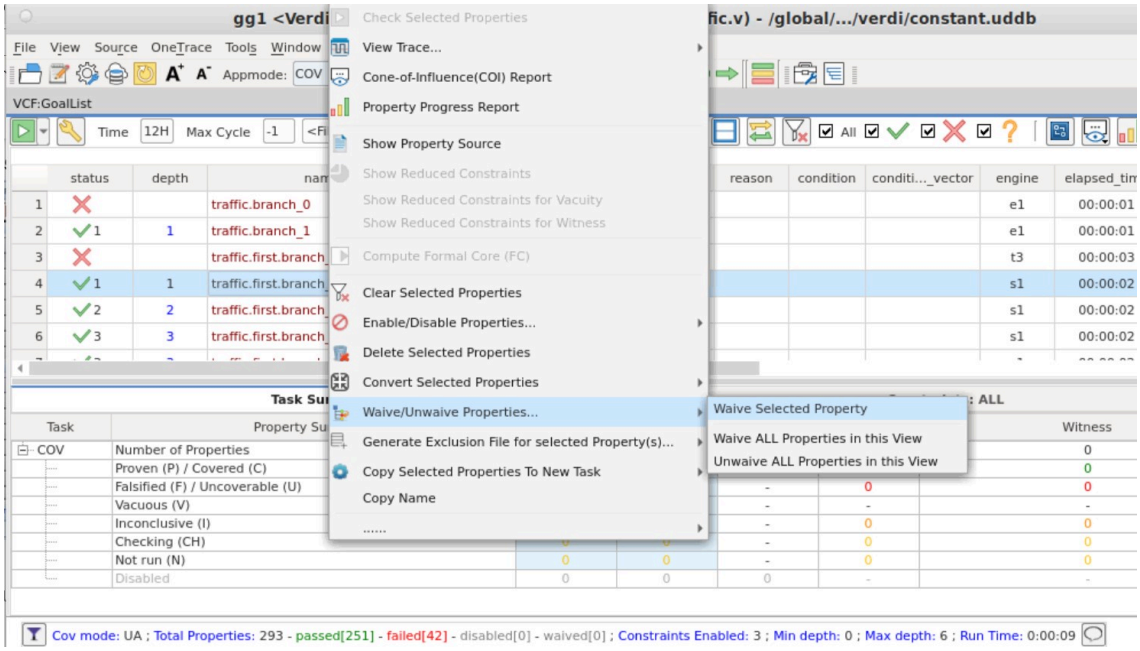
  You can use the Waive option to waive code coverage targets. These options perform an in-memory waiving of code coverage targets such that they are not targeted for further analysis. These targets are tagged as waived and the total goal count shown by the report_fv command reduces.

  You can use the Unwaive option to remove code coverage targets. This option performs an in- memory removal of code coverage targets such that they are targeted for further analysis. This option is also used to remove the waived attribute of previously waived code coverage goals, thus, the total goal count shown by the

Feedback

report_fv command increases. This command does not work for the code coverage goals that are not waived using the waive option.

Figure 76: Waive/Unwaive Option in Property Table RMB Pop-up Menu



- *Waived Column* in Property Table

  You can view the waived properties in the waived column in the property table as shown in Figure 76. By default., the waived column does not appear in the property table. Click and select the waived from the *Custom View Settings* window as shown in Figure 77.

Feedback

Figure 77: Custom View Settings Window

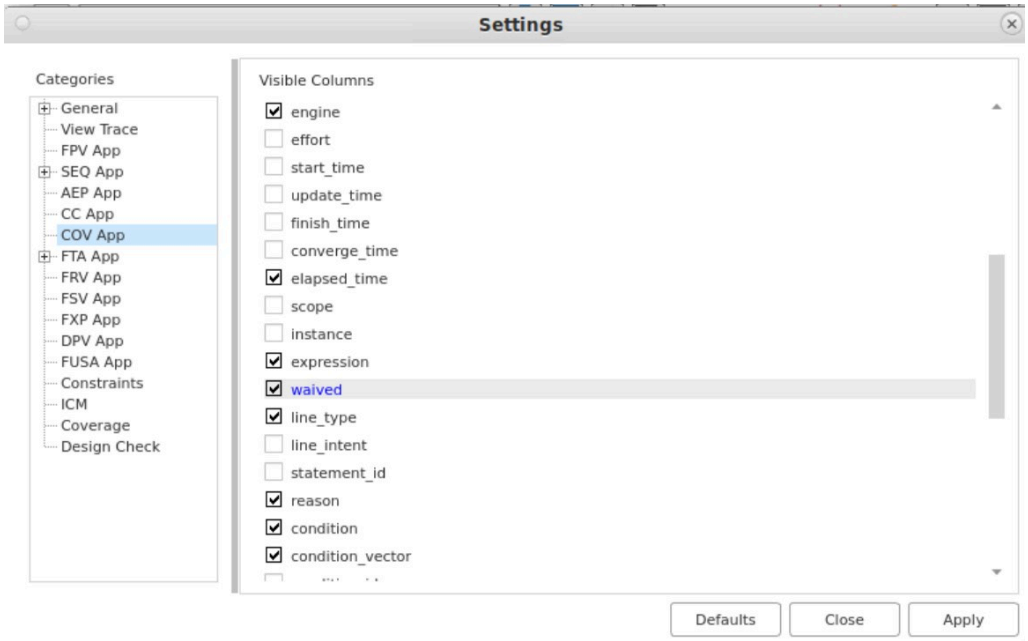
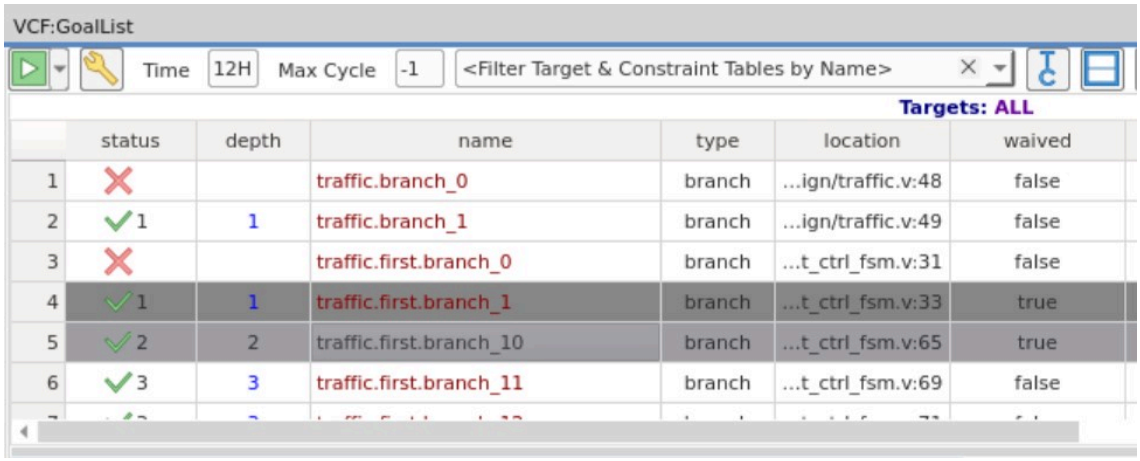
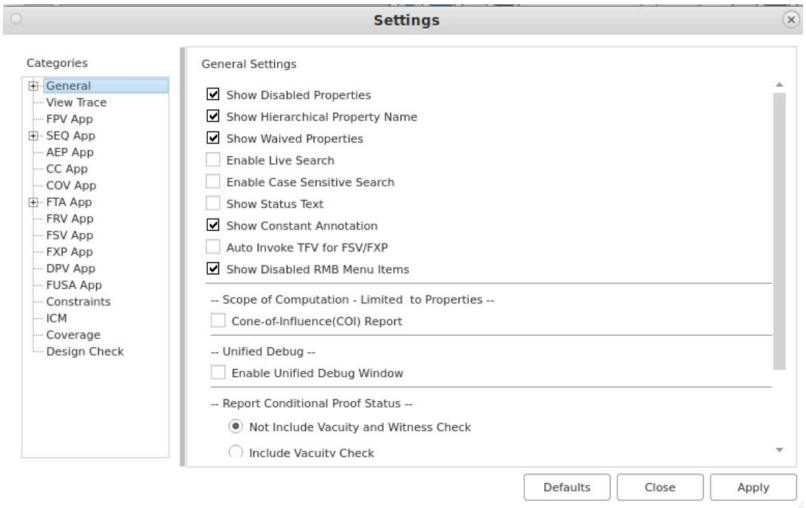


Figure 78: Waived Column in Property Table



You can view the waived properties for all applications, by default, by selecting the *Show Waived Properties* option as shown in the following figure:



11.6 Performance and Convergence

FCA reports unreachability issues using exhaustive mathematical techniques, it works best when applied at the block level. When you perform unreachability analysis on large sized blocks (or at a higher level in the design hierarchy, for example, sub-system level or SoC level), FCA may face capacity and performance challenges. This section gives recommendations to reduce the complexity and improve the runtime performance as well as the chances of getting better convergence.

- FCA -cm_hier flow
- Excluding bind hierarchies from coverage
- FCA blackbox flow
- Enable automatic abstractions
- Avoid enabling trace generation
- Running selective coverage metrices
- Using Auto Scale Flow (Very Useful – combines most of the techniques mentioned above)

11.6.1 Partition Runs by Hierarchy Trees Using -cm_hier option in VCS

Before running, check the number of coverage goals in all the hierarchies. For faster convergence create `cm_hier` files wisely so that the number of coverage goals in that is in

Feedback

the range of 50K. This is recommended way to partition the coverage goals because any constant propagation can still be carried through the design.

- Use Verdi coverage to help you identify these blocks/hierarchies
- Open the coverage database with Verdi and look into the block which you want to target
- Then in the instance hierarchy look into the sub blocks underneath and using the tool tip to find the number of coverage goals in that.
- Use this information to split up the blocks into smaller manageable chunks and then create separate `-cm_hier` files for them
- Have separate runs to target each of these `-cm_hier` files separately
- Once all the runs are completed, all the exclusion files can be fed to URG or Verdi coverage using `-elfiles` for final coverage reporting and analysis.

  The `cmhier` file can be passed in two ways during formal run:

  1. When simulation database is not read, it can be passed as an option to `-vcs "-cm_hier"` in `read_file` or `elaborate` command.
  2. When simulation database is read, and user want to override the `cm_hier` file used during the simulation then it needs to be passed using `set_fml_var fml_cov_alt_hier_file <hier_file_name>` before `read_file` or `analyze` command.

  If the user wants the tool, in certain scenarios, to have both the simulation and alt hier files applied, then they can use the following command:

  ```
  set_fml_var fml_cov_hier_both true
  ```

  Syntax for cm_hier file:

  ```
  -/+module: to exclude/include a module in the analysis
   -/+tree: to exclude/include the whole instance tree in the analysis
  ```

  Note:

  For more details on the configuration file syntax, refer to the Coverage Technology User Guide.

  The following is an example Tcl script that is used for performing hierarchical unreachability analysis.

  At sub-block level:

  ```
  set design vlog_street_ctrl_fsm
  read_covdb -cov_input simv.vdb -cov_dut TB.T1.main
  ```

Feedback

```
read_file -top $design -format verilog -cov all -vcs
 "-f ../design/filelist"
create_clock clk -period 100
create_reset rst -sense high
sim_run -stable
sim_save_reset
check_fv -block
report_fv -verbose > main.txt
save_covdb -name main.vdb -status covered
# UNRs found at sub block instance traffic.main saved in main_unr.el
save_cov_exclusion -file main_unr.el
save_cov_exclusion -file main_covered.el -targets [get_props -status
 covered]
view_coverage -cov_input main.vdb
```

At system level:

```
set_fml_appmode COV
set design traffic
set_message_severity -name COV_EXCLUSION_FILE_NOT_SAVED warning
read_covdb -cov_input simv.vdb -cov_dut TB.T1
read_file -top $design -format verilog -cov all -vcs
 "-f ../design/filelist -cm_hier hier.cfg"
create_clock clk -period 100
create_reset rst -sense high
sim_run -stable
sim_save_reset
check_fv -block
report_fv -verbose > traffic.txt
save_covdb -name traffic.vdb -status covered -overwrite
save_cov_exclusion -file traffic_unr.el
view_coverage -cov_input traffic.vdb -elfiles {traffic_unr.el
 first_unr.el main_unr.el main_covered.el first_covered.el}
urg -dir traffic.vdb -elfilelist el.f where el.f contains below info-
traffic_unr.el
first_unr.el
main_unr.el
```

//hier.cfg//

```
+module traffic
```

- When a simulation coverage database for the full block is available:

  Load the coverage database into a coverage viewer like Verdi or URG and identify the modules for which the coverage score achieved by simulation is poor. Target those modules first for unreachability analysis using the `-cm_hier` switch of VCS. You can load the module wise generated exclusion files in the coverage viewer along with the simulation coverage database to see the improved coverage score.

- When simulation coverage database is not available:

  You must avoid running FCA at a full block level and target sub-blocks of interest based on design knowledge using `-cm_hier` switch of VCS.

  Preferably in this case, the unreachability analysis must be done in early stages of design development when the simulation based verification has not been started yet.

11.6.2 Excluding Bind Hierarchies From Coverage

Formal Verification testbenches are often bound to the design using SV bind. Such testbench modules are uninteresting for coverage collection and need to be excluded. Use the `-cm_instr exclude_bind_hierarchies` option to enable the tool to automatically determine such hierarchies, and then exclude the hierarchies with the `-cm_hier` switch at compile time.

The hierarchies excluded through the `-cm_instr exclude_bind_hierarchies` switch cannot be included back through `+tree/+module` rules in the cm_hier file. The switch takes priority over the `cm_hier` rules.

The other way to ignore bind hierarchies from the coverage is to set `fca_disable_bind_cov` true. This needs to be set before the `read_file` or `analyze` command as follows:

```
set_fml_var fca_disable_bind_cov true
```

11.6.3 Blackboxing Modules Where Coverage Analysis Not Required

When there are some modules in the hierarchy for which coverage analysis is not required such as, if there are known memories, analog modules, DFT logic etc in the design blackbox them upfront. This helps in faster build time.

```
set_blackbox -designs { <list_of_memory_modules> }
```

11.6.4 Enabling Automatic Abstraction

Operators like multipliers/adders and so on can increase the complexity and make convergence difficult. In order to reduce the design complexity you can choose to abstract them selectively using `set_abstraction` command or use auto-abstraction to prune out complex operators all at once using `set_abstraction -auto on`

Using abstractions, whatever unreachable are found will still be valid. The only downside here is that not all unreachable might be reported and get converted to covered.

11.6.5 Controlling Trace Generation

The trace generation for the reached coverage targets is disabled by default in the FCA flow. It is advised to keep following fml var off as generating trace can have impact on performance.

```
vcf> set_fml_var fml_cov_gen_trace off
```

The `fml_cov_gen_trace` application variable accepts the following values:

- `default`: The initial value `default` disables trace generation for covered coverage goals for better overall performance.
- `on`: Set the value to `on` to enable the trace generation for reached coverage targets in the FCA flow.

  When set to `on` with many properties having toggle metrics, then it can lead to increased disk usage.
- `off`: Set the value to `off` to disable the trace generation for reached coverage targets in the FCA flow.

Note:
It is recommended to turn `off` this variable during the run and turn it `on` when you want to know the witness trace of a specific property. In such cases, clear the selected property and rerun the check with the `fml_cov_gen_trace` variable set to `on`.

11.6.6 Controlling Compute Resources

- Choosing max memory and time

  Unreachability analysis on big designs can be time consuming. Let it run for sufficient time (at least 24 hours) to get reasonable results. By default FCA has a default max time of 12 hrs. This can be increased as follows

  ```
  set_fml_var fml_max_time 24H
  ```

  By default, FCA uses max solver memory of 8GB. In general, memory requirement for the solver jobs should not be high and resources requested on the grid should not be in the range of 64G or more. Initially set the grid for 8/16G resources and in case of memout seen on the solver jobs, user should ask for more. This can be increased as follows:

  ```
  set_fml_var fml_max_mem 16GB
  ```

- Parallelize the runs using grid

- If grid facilities like LSF/SGE is available then use it for parallelizing by spawning off the runs on multiple cores

```
set_grid_usage  -type lsf=<number of cores> -control <resource
 control string for grid job submission, eg. { qsub -P bnormal -V
 arch=glinux }>
```

  - For auxiliary worker process,

    23 LSF slots, each having 8-32 GB memory (`set_grid_usage -type lsf=23 -control { bsub -R rusag[(mem=32]}`). 20 licenses enable 23 workers

  - For main process,

    One machine with 8-32 GB for main process. The main process is launched using VC Formal command (`bsub -I -R "rusage[mem=8000]" vcf -verdi -f < your tcl file >`)

- Make sure main and auxiliary worker processes are running in different machines
- It is recommended to have dedicated/heavy machines for auxiliary worker processes
- Run VC Formal in batch mode to avoid GUI hang due to main process machine memory limit exceeded
- To view/debug the progress in GUI from shell mode, use command:

  ```
  vcf> start_gui
  ```

- Ensure there is no mem outs using `report_fml_jobs` command. The following message should be carefully looked at in the UNR runs and improve the grid memory accordingly.

  [Warning] CANT_SOLVE_ALL_GOAL: Some of the goals couldn't be solved. Please review the status of formal jobs with 'report_fml_jobs' and try increasing the resource limits or simplifying the verification problem.

- Ensure the expected number of jobs/workers are running using `report_fml_hosts -totaltime` command. This will give the info regarding the number of workers assigned during the run

  It is advised to add `check_config -worker_stat` command before the run command so that the workers info will be logged automatically.

11.6.7 Running Selective Coverage Metrices

In most cases, the toggle and condition metrices property count are larger as compared to other coverage types. Coverage metrics like condition can make the problem harder. Complexity can increase manifold because in that case for all the expressions in the

design there would be additional logic instrumented in the design to monitor the coverage of those interesting points.

In many circumstances, for toggle coverage, simulation script has all the possible switches to exhaustively test all the design scenarios. However, you must check if the toggle coverage monitoring from Formal is required for the MDAs. Besides, since MDAs can be potentially performance intensive for toggle coverage so -cm_tgl mda should be added with caution.

You must avoid performing toggle coverage for input ports unless it is really required. By default, toggle coverage for input ports is not monitored by VCS UNR.

In above listed cases, it is advised not to run all metrics at once. It is better to run toggle and condition coverage separately.

11.7 AutoScale Flow

In large designs, handling design complexity and large number of coverage goals is the biggest challenge in the FCA/UNR setup. To address this issue, the common methodology followed is to manually partition the design and perform coverage analysis. Manually partitioning the designs is not scalable as the iterative process to define partitions for every design takes a lot of effort. Also, loss of information (for example, constants and parameters) from the top module adds to the complexity of setup.

To overcome this issue, VC Formal enables you to auto-scale to reduce the manual effort of partitioning large designs to achieve convergence. Using auto-scale, you can:

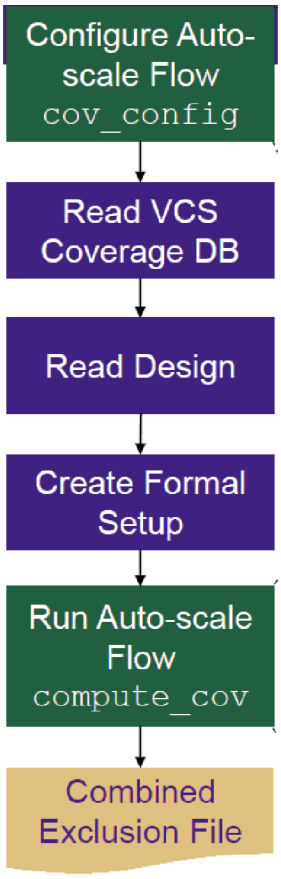
- Instrument coverage goals for simulation coverage holes only.
- Handle design complexity intelligently while maintaining consistency with UNR exclusions.
- Target coverage metrics as separate tasks and process them in parallel.

Auto-scale Methodology

It is recommended to use this flow when the number of coverage targets are more than 100K. The following figure shows the methodology to auto-scale coverage convergence with VC Formal FCA.

Feedback

Figure 79: Methodology to Auto-Scale Coverage Convergence with VC Formal FCA



11.7.1 Configuring Auto-Scale in VC Formal FCA

To configure auto-scale, use the `cov_config` command before `read_file`/`elaborate`.

```
cov_config -auto_scale
```

Syntax

```
[-auto_scale] [-line_task <line_task>] [-cond_task <cond_task>]
 [-branch_task <branch_task>] [-tgl_task <tgl_task>] [-fsm_task
 <fsm_task>] [-cg_task <cg_task>] [-limit <limit>] [-line_limit
 <line_limit>] [-branch_limit <branch_limit>] [-tgl_limit
 <tgl_limit>] [-fsm_limit <fsm_limit>] [-cond_limit <cond_limit>]
 [-module <module>] [-inst_list <inst_list>] [-start_task_cmd
 <start_task_cmd>] [-end_task_cmd <end_task_cmd>] [-start_line_task_cmd
 <start_line_task_cmd>] [-start_cond_task_cmd <start_cond_task_cmd>]
 [-start_branch_task_cmd <start_branch_task_cmd>] [-start_tgl_task_cmd
 <start_tgl_task_cmd>] [-start_fsm_task_cmd <start_fsm_task_cmd>]
```

Feedback

```
[-start_cg_task_cmd <start_cg_task_cmd>] [-exclude_assumes_covdb]
[-exclude_bb_signals]
```

Note:

You should use the following switches before the `read_file` or `elaborate` command to get the expected result. `-limit -line_limit -branch_limit -tgl_limit -fsm_limit -cond_limit -module -inst_list` Else the following warning message is reported:

[Warning] INCORRECT_CMD_USAGE: cov_config with '-limit' is used after read_file/elaborate in the auto_scale flow. To fix this, call this before reading the design file.

- `[-auto_scale]`: Specify this option to enable automatic scaling up for big designs.
- `[-line_task <line_task>]`: Specify the number of parallel tasks for running line coverage metric.
- `[-cond_task <cond_task>]`: Specify the number of parallel tasks for running conditional coverage metric.
- `[-branch_task <branch_task>]`: Specify the number of parallel tasks for running branch coverage metric.
- `[-tgl_task <tgl_task>]`: Specify the number of parallel tasks for running toggle coverage metric.
- `[-fsm_task <fsm_task>]`: Specify the number of parallel tasks for running fsm coverage metric.
- `[-cg_task <cg_task>]`: Specify the number of parallel tasks for running cover group coverage metric.
- `[-limit <limit>]`: Specify the threshold coverage percentage. By default, it is set to 70. Any module whose total coverage percentage is greater than the specified value would be ignored by FCA/UNR.
- `[-line_limit <line_limit>]`: Set the threshold value for line cov metric. Any module whose number of uncovered goals for line metric is lesser than the specified value would be ignored by FCA/UNR.
- `[-branch_limit <branch_limit>]`: Set the threshold value for branch coverage metric. Any module whose number of uncovered goals for branch metric is lesser than the specified value would be ignored by FCA/UNR.
- `[-tgl_limit <tgl_limit>]`: Set the threshold value for toggle coverage metric. Any module whose number of uncovered goals for toggle metric is lesser than the specified value is ignored by FCA/UNR.

Feedback

- `[-fsm_limit <fsm_limit>]`: Set the threshold value for fsm coverage metric. Any module whose number of uncovered goals for fsm metric is lesser than the specified value is ignored by FCA/UNR.
- `[-cond_limit <cond_limit>]`: Set the threshold value for condition coverage metric. Any module whose number of uncovered goals for condition metric is lesser than the specified value is ignored by FCA/UNR.
- `[-module <module>]`: Specify the scope from where the auto-scaling should be begin. By default, auto-scaling starts from the top module sent to VC Formal.
- `[-inst_list <inst_list>]`: Specify instance/instances in the input coverage database hierarchy from where to start doing divide and conquer. Default is to start from the top instance.
- `[-start_task_cmd <start_task_cmd>]`: Specify the command to invoke after individual task creation in the auto-scale flow and before invoking `check_fv`.
- `[-end_task_cmd <end_task_cmd>]`: Specify the command to invoke after `check_fv` is finished for each individual task in the auto-scale flow.
- `[-start_line_task_cmd <start_line_task_cmd>]`:Specify the commands to invoke after task creation of line metric in auto scale flow and before invoking `check_fv`.
- `[-start_cond_task_cmd <start_cond_task_cmd>]`:Specify commands to invoke after task creation of cond metric in auto scale flow and before invoking `check_fv`.
- `[-start_branch_task_cmd <start_branch_task_cmd>]`:Specify commands to invoke after task creation of branch in auto scale flow and before invoking `check_fv`.
- `[-start_tgl_task_cmd <start_tgl_task_cmd>]`:Specify commands to invoke after task creation of tgl in auto scale flow and before invoking `check_fv`.
- `[-start_fsm_task_cmd <start_fsm_task_cmd>]`:Specify commands to invoke after task creation of fsm in auto scale flow and before invoking `check_fv`.
- `[-start_cg_task_cmd <start_cg_task_cmd>]`: Specify commands to invoke after task creation of cg in auto scale flow and before invoking `check_fv`.
- `[-exclude_assumes_covdb]`:Saves exclusion file for assumes
- `[-exclude_bb_signals]`:Automatically exclude blackbox signals in covdb.

Example Usage

All the `cov_config` switches should be used along with the `-auto_scale` switch or use the `cov_config` switches after enabling auto-scale using the `-auto_scale` switch.

- To enable auto-scale along with the switches:

```
cov_config -auto_scale -limit 80
```

- To first enable auto-scale and then set the options:

```
cov_config -auto_scale
cov_config -limit 80
```

- If the `-auto_scale` is not enabled and you try to use the switches, you will see the following error:

```
cov_config -limit 80
```

[Error] AUTO_SCALE_NOT_ENABLED: Auto scale not enabled. '-limit' switch should be used after enabling auto scale. Auto scale can be enabled by using '-auto_scale' switch.

Auto-Scale Variable Settings

Use the following settings to auto-scale:

- To enable memory and time consumption messages

```
cov_config -debug
```

- To enable auto-scale and perform parallel processing (instead of `check_fv`)

```
compute_cov
```

- To enable abstraction of complex operators

```
set_fml_var fml_auto_scaling_abs true -global
```

- By default, model reduction is enabled. To disable model reduction, set the following variable to false

```
set_fml_var fml_auto_scaling_reduction false -global
```

11.7.2 Configuring Auto Scale in VCS UNR

Use the following settings in the VCS UNR configuration file to enable auto-scale:

- To enable auto-scale in VCS UNR

```
-autoScale
```

- To control the coverage percentage of scopes (by default, set to 70%)

```
-preUnrComp "cov_config -limit <value_like_60_70_etc>"
```

- To control the number of coverage goals of scopes

Feedback

```
-preUnrComp "cov_config -<metric_type>_limit <value>"
```

- To set the scope from where to start the auto-scaling flow. By default, auto-scale starts from the formal top:

  ```
  -preUnrComp "cov_config -module <module_name>"
  ```

- To enable memory and time consumption messages

  ```
  -preUnrComp "cov_config -debug"
  ```

- To enable abstraction of complex operators

  ```
  -preUnrComp "set_fml_var fml_auto_scaling_abs true -global"
  ```

- To enable model reduction

  ```
  -preUnrComp "set_fml_var fml_auto_scaling_reduction false -global"
  ```

Use Model: Auto-Scale Usage in VCS UNR

The following shows the settings in the configuration file that can be used to auto-scale with VCS-UNR.

```
-top traffic
-covEL exclorg.el
-covInput simv.vdb
-covDUT TB.T1
-clock clk 100
-reset rst 1
-autoScale
-preUnrComp "cov_config -limit 80"
-preUnrComp "set_fml_var fml_auto_scale_reduction false -global"
-preUnrComp "set_fml_var fml_auto_scale_abs true -global"
```

11.7.3 Example Tcl Outputs

- When the simulation vdb is loaded, the module coverage score file is created showing uncovered and covered goals, and coverage percentage for each metric type for all modules:

  ```
  Log message:
  [Info] AUTO_SCALE_INST_ALL_COV_SCORE: Instance specific coverage
   score from the simulation vdb is dumped into this corresponding file
   <path>/vcst_rtdb/logs/all_inst_score.txt .
  cat vcst_rtdb/logs/all_inst_score.txt
  Instance: TB.T1 (0)
  Instance TB.T1 average hierarchy score: 0
  No covered target exists for TB.T1
  Instance: TB.T1.main (67.11)
  Instance TB.T1.main average hierarchy score: 67.11
  ```

Feedback

```
 instance: TB.T1.main  metric: Line 28/32 (87.5)
 instance: TB.T1.main  metric: Branch 0/0 (0)
 instance: TB.T1.main  metric: Tgl 34/52 (65.38)
 instance: TB.T1.main  metric: Condition 5/9 (55.55)
 instance: TB.T1.main  metric: Fsm 9/15 (60)
Instance: TB.T1.first (66.32)
Instance TB.T1.first average hierarchy score: 66.32
 instance: TB.T1.first  metric: Line 26/32 (81.25)
 instance: TB.T1.first  metric: Branch 0/0 (0)
 instance: TB.T1.first  metric: Tgl 31/52 (59.61)
 instance: TB.T1.first  metric: Condition 7/9 (77.77)
 instance: TB.T1.first  metric: Fsm 7/15 (46.66)
```

- A `cm_hier` file is automatically generated based on the `-limit`, the`-<metric>_limit`, `-module`, and `-list_inst` options and used to configure coverage instrumentation at elaboration time:

```
Log message
[Info] AUTO_SCALE_INST_HIER_FILE: Tool has generated cm_hier file
 <path>/vcst_rtdb/logs/auto_scale_inst_hier.txt which satisfies the
 criteria passed with -limit/-<metric>_limit switch with cov_config
 command and is using this cm_hier file for coverage extraction .

cat vcst_rtdb/logs/auto_scale_inst_hier.txt
+tree traffic.main
+tree traffic.first
```

- At the completion of each task summary report is printed. The summary of all the tasks results is printed at the end.

```
Summary Results
   Property Summary: COV_COND
   ------------------
   > Constraint
     - # found        : 1

   Coverage Summary: COV_COND
   ------------------
   > Condition Coverage
     - # found        : 6
     - # covered      : 6
```

- One exclusion file is saved for each task as *<task_name>.el* and combined el file with name unreachables.el.

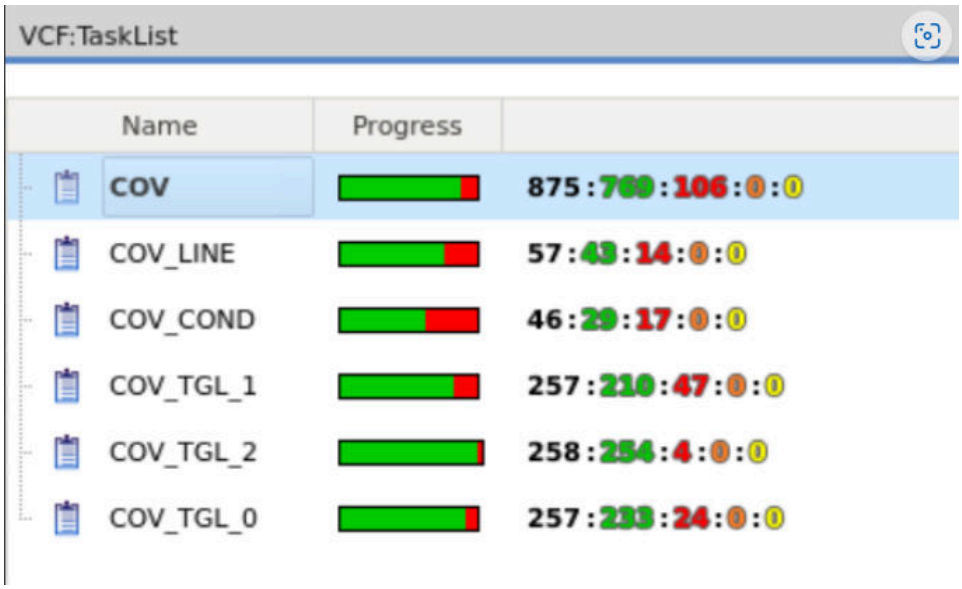
11.7.4 Viewing Autoscale Results in GUI

By default, all coverage metrics are run in separate tasks and processed in parallel. The line, condition, FSM, and branch coverage metrics get one task each named

Feedback

COV_<METRIC_TYPE>. For example, the toggle coverage metric goals are split into multiple tasks (up to 8) with name COV_TGL_<Number>

- The line, condition, FSM, and branch coverage metrics get one task each named COV_<METRIC_TYPE>. For example, the toggle coverage metric goals are split into multiple tasks (up to 8) with name COV_TGL_<Number>.



11.8 Generating VCS UNR Setup from FCA

Use the `fca_report-unr_setup` command to generate a VCS/UNR setup closely matching the current FCA session.

The `-unr_setup` option creates both the configuration files *unr_config.txt* and the C Shell script *run_unr.csh* to run VCS/UNR.

Syntax

```
vcf> fca_report -help
Usage: fca_report # Print report for FCA analysis results.
[-unr_setup] (Generate UNR setup which includes the files: unr_config.txt
 and run_unr.csh.)
```

The configuration file shows the following switches with the information matching the FCA session: `-top,-covInput, -covDUT, -fvDUT, -maxTime, -maxMemory, -clock,-reset, -constant, -covEl, -grid, -gridOpts, -gridFile, -postUnrCompsnip_driver, -fml_var, -buildTop`.

Before running VCS/UNR, edit the configuration file *unr_config.txt* and script *run_unr.csh* as needed.

Feedback

11.9 Current Limitations of FCA

VCS extracts condition coverage goals for events like "@(posedge clk or negedge rst)" or "@ (b or c)" only when the -cm_cond event option is specified. Same is the behavior for VC Formal Coverage Analyzer. However, current limitation is that VC Formal Coverage Analyzer does not extract coverage goals for edge triggered events, that is, the events of the form "@(posedge clk or negedge rst)". It extracts goals only for level triggered events.

The `cm_assert_hier` VCS sub option is not supported in FCA because it is simulation run time based.

In the default flow, to enable the FCA to map the goals in simulation coverage database (when coverage database is imported) with the ones extracted by VC Formal, you must provide consistent coverage options to both, that is, the coverage options used while generating the simulation database must match with the ones provided in VC Formal. If the flow discussed in section Automatic Extraction of Coverage Sub Options is used, the limitation is taken care of by the tool automatically.

Apart from this VC Formal Coverage Analyzer does not support the following:

- Coverage analysis for repeat and for-each loop
- Escaped name for top module
- Blackbox outputs are not currently included in coverage results independent of the value of the `fml_cov_tgl_input_port` formal variable.

11.10 Variables Related to FCA

Table 18 lists all the application variable that impact the FCA application.

Table 18: Variables Related to FCA

Attribute Name	Value Type	Suggested Value
fml_reset_property_check	bool	true: To enable the reset reachable check
fml_cov_enable_branch_cov	bool	true: To enable branch coverage
fml_track_loc_const_select	bool	true: To keep source location of constant selects, and map them appropriately
fml_track_loc_reset	bool	true: To keep source location of async reset blocks, and map reset block for formal core coverage mapping
fml_cov_tgl_input_port	bool	true: To enable toggle coverage of input ports

Feedback

Table 18: Variables Related to FCA (Continued)

Attribute Name	Value Type	Suggested Value
fml_cov_enable_int_unr	bool	see for detailed usage
fml_cov_gen_trace	string	default: To not generate waveform trace in COV mode
fml_cov_x_support	bool	true: To enable the support of X semantics

12

Formal X-Propagation Application

The VC Formal FXP application checks for unknown signal value (X) propagation through the design and allows tracing of the failed property to source of X in the Verdi schematic and waveform.

Getting Started • *What is VC Formal FXP?* • *Accessing FXP Lab Example*	**Improving Convergence of Properties** • *Adding Abstractions* • *Computing Reduced Constraints* • *Computing Formal Core for Proofs* • *Using Proof Assist*
Setting Up and Running FXP • *Enabling FXP Application mode* • *Configuring FXP Properties* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Verifying Design Setup* • *Running FXP*	**Debugging, Reviewing and Saving Results** • *Computing and Debugging FXP rootcauses* • *Viewing Traces for Debugging* • *Saving and Restoring Sessions*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	**Getting Help on Commands** • *FXP Tcl Commands*

12.1 About FXP Application

The FXP app is designed to check whether unknown signal values (X's) can propagate to dangerous points within a design. It allows the user to configure individual sources of X and the destination points to which they should not propagate. There are two points which are defined in the FXP app.

- X-injection: Point at which an unknown is introduced into a design.
- X-observation: Point at which an unknown is observed.

Feedback

The X-injection and X-observation points can be configured from a list of pre-defined types to auto-generate the FXP properties. Additionally, custom FXP injection and observation points can be defined by the user. The FXP types are discussed in detail in section fxp_generate Property Types.

You can configure the X-injection and X-observation points from a list of pre-defined types and auto generate the required properties for checking.

If a property fails, then the FXP debug environment allows you to auto-trace the failure to the source of X through both waveform and schematic views.

12.2 VC Formal FXP: Methodology

The FXP app can be run on only design files, there is no requirement to create any custom SVA or other properties. The flow to run the FXP app is as follows:

1. Set Appmode to FXP
2. Read in design files
3. Generate FXP properties
4. Run Properties
5. Review Results
6. Debug

You can switch to the FXP app using both tcl commands and interactively in the GUI.

Using Tcl

- Set the following variable to enable the FXP application mode.

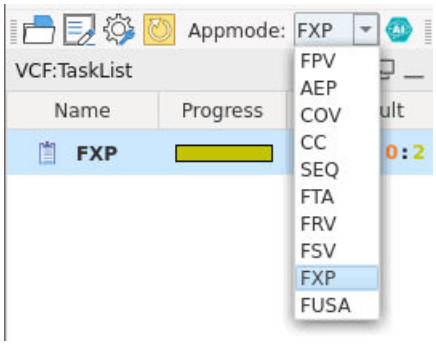
```
set_fml_appmode  FXP
```

Using GUI

Feedback

- Select the application mode as *FXP* from the *Appmode* list, as shown in the following figure.

Figure 80: Navigating to FXP Application Mode



12.2.1 Inputs

There are no pre-requisites for running FXP, the required inputs are just synthesizable design files and tcl configuration.

To read in a design for FXP use the same procedure defined in section Compiling Designs using either the read_file or analyze/elaborate commands. There are no special switches required for the FXP app.

For accurate FXP checks, the design should be placed into the reset state. This can be achieved by using the reset commands defined in section Setting Up User-defined Resets.

12.3 Standard Use - Configuring FXP

The main use case for FXP is the auto-generation of properties that will check whether unknown values can propagate to interesting areas of the design. There are three FXP specific commands which are available to generate properties.

- The `fxp_generate` Command
- The `fxp_assume` Command
- The `fxp_assert` Command

For a general, non-customized FXP run, use `fxp_generate` to create all of the properties required for FXP.

Feedback

12.3.1 The fxp_generate Command

Use the `fxp_generate` command to configure the X-injection and X-observation points at which properties are generated.

Syntax

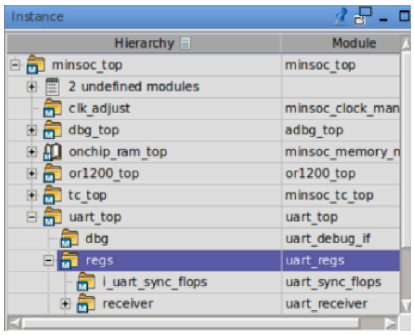
```
vcf> fxp_generate [<list-of-scopes>] [-name <my_name>][-type
 <list-of-types>]
```

Where,

- `list-of-scopes`: Specify this switch to mention the scope at which properties are generated. Specify only the instance name, not the module name. Use the hierarchical separators if the instance is not under the top module. Use {} to include more than one instance and separate them with space.

  Example

  Consider the top module is *minsoc_top* and to generate properties for instance *tc_top* and *regs*, use the following command:


  ```
  fxp_generate {tc_top uart_top.regs }
  ```

- `-name <my_name>`: This optional switch allows users to specify a common name prefix for generated properties. For example

  ```
  fxp_generate -name my_props -type {uninit}
  ```

  generates properties as follows, where M and N are integers:

  ```
  my_props_<M>_<my_signal>_<N>
  ```

- `-type <list-of-types>`: This optional switch allows the user to configure which injection and observation types to generate properties for in FXP. Possible values: `abs, absin, absout, auto, bbin, bbout, in, oba, out, snip, snipdrv, undef, undriven, uninit, unresin, unresout, xassign`

Feedback

If this switch is omitted the default generation types shown in the table below will be used.

Note:

The `-type auto` option is equivalent to `-type {abs bbin oba out snip undef undriven uninit xassign unresin absin}`. If the `-type <list-of-types>` option is not specified, by default, it is set to `-type {abs bbin oba out snip undef undriven uninit xassign absin unresin}`.

If the optional fields are omitted, then the `fxp_generate` command generates X-observation and X-injection points for default types. The default options are shown in the table below.

Table 19: Types of Properties

Name	Description	Type	Default
abs	Abstractions	Inject X Observe X	Yes
bbin	Black box inputs	Observe X	Yes
bbout	Black box outputs	Inject X	No
in	Primary inputs	Inject X	No
oba	Array bound violations	Inject X	Yes
out	Primary outputs	Observe X	Yes
snip	Snips as inputs	Inject X	Yes
snip_drv	Snips for observations	Observe X	Yes
undef	Undefined behavior (divide by 0, multi-driven)	Inject X	Yes
undriven	Undriven nets	Inject X	Yes
uninit	Uninitialized registers	Inject X	Yes
xassign	Explicit X-assignments	Inject X	Yes
unresin	Unresolved module inputs	Observe X	Yes
unresout	Unresolved module outputs	Inject X	No

fxp_generate Property Types

Abstractions (abs)

Feedback

If the abstraction property type is specified with `fxp_generate` then any abstracted construct will be considered as an X-Injection and X-Observation point.

```
set_abstraction -construct count=4
```

In the above example, an abstraction has been applied on all counters in the design with width greater than 4. This means that, for all abstracted counters, a check will be generated to see if an X can propagate to them. It will also consider these counters as a X-injection points, ie a source of an X. This property type is default in fxp_generate

Blackbox Inputs (bbin)

If the blackbox input property type is specified with fxp_generate then the inputs to any black boxed instance will become observation points to check that an X cannot propagate to these inputs.

In the above example, a blackbox has been applied to all instances of module my_design. As such all the inputs to this block are considered X-observation points and checks will be created. This property type is default in fxp_generate

Blackbox Outputs (bbout)

If the blackbox output property type is specified with fxp_generate then the outputs of any black boxed instance will become X-injection points.

```
set_blackbox -designs my_design
```

In the above example, a blackbox has been applied to all instances of module my_design. As such all the outputs of this block are considered X-injection points and constraints will be created. This property type is not default in fxp_generate.

Primary Inputs (in)

If the Primary Input property type is specified with fxp_generate then the primary inputs of the design will become X-injection points.

```
input a,
input [31:0] b,
```

All primary inputs will be considered as X-injection points. This property type is not default in fxp_generate

Array Bound Violations (oba)

If the array bound violation property type is specified with fxp_generate then any out of bounds access in the design will become an X-injection point. This property type is default in fxp_generate.

Primary Outputs (out)

Feedback

If the primary output property type is specified with fxp_generate then all primary outputs become X-observation points. This is the most commonly interesting point to ensure that X's cannot propagate to.

```
output my_op1
output [3:0] my_op2;
```

All primary outputs are considered as X-observation points. This property type is default in fxp_generate

Snips (snip)

If the snips property type is specified with fxp_generate then any snipped signals will become X-Injection points and if they can propagate to observation points will cause counterexamples.

```
snip_driver my_wire
```

This property type is default in `fxp_generate`.

Snips (snip_drv)

If the snips property type is specified with `fxp_generate` then any snipped signals' drivers becomes X observation points.

```
snip_driver my_wire
```

This property type is default in `fxp_generate`

Undefined Behavior (undef)

If the undefined behavior property type is specified with fxp_generate then any instances of undefined behavior will become X-Injection points. An example of undefined behavior would be a divide by 0. This property type is default in fxp_generate.

Undriven Nets (undriven)

If the undriven nets property type is specified with fxp_generate then any signals which are undriven will become X-Injection points and if they can propagate to observation points will cause counterexamples

```
wire my_wire
```

This property type is default in fxp_generate

Uninitialized Registers (uninit)

If the uninitialized registers property type is specified with fxp_generate then any registers that do not have an initial value in the formal analysis will be considered X-Injection points. Most often this is as registers are missing a reset condition. Uninitialized registers are a common source of X problem on the observation points.

Feedback

```
reg my_reg
always @(posedge clk)
  if (a)
    my_reg <= 1'b1;
```

In the above example, my_reg does not get a known value until a goes high, so it will be uninitialized and considered as an X-Injection point. This property type is default in fxp_generate.

Explicit X-Assignments (xassign)

If the X-Assign property type is specified with fxp_generate then any signals explicitly assigned to x will become X-Injection points and if they can propagate to observation points will cause counterexamples.

```
wire my_wire;
assign my_wire = 1'bx
```

This property type is default in fxp_generate.

Unresolved module inputs (unresin)

If the unresolved module input (inferred) property type is specified with `fxp_generate` then the inputs to unresolved instance will become observation points to check that an X cannot propagate to these inputs. This property type is default in `fxp_generate`.

Unresolved module outputs (unresout)

If the unresolved module output (inferred) property type is specified with `fxp_generate` then the outputs of any unresolved instance will become X-injection points. This property type is not default in `fxp_generate`.

12.3.2 The fxp_assume -injectx Command

Use the `fxp_assume`command to create custom control of X injection sources on specific points to inject a X from formal analysis.

Syntax

```
vcf> fxp_assume -injectx [-name <name>] [-reset] [-oba] [-xassign]
 [-undef] [-condition <expr>] [-scope <scope_name>] <signal_name>
```

Where,

- `name <name>`: Specify this switch to name the design for which you want to create custom control of X injection sources.
- `reset`: Specify this switch to initialize the X out of reset. It can only be used on sequentials.
- `oba`: Specify this switch to inject an X on out of bounds. It can only be applied to entire scope and requires –scope option.
- `xassign`: Specify this switch to inject an X on any assign X. It can only be applied to entire scope and requires –scope option.
- `undef`: Specify this switch to inject an X for undefined. It can only be applied to entire scope and requires –scope option.
- `condition <expr>`: Specify this switch to inject an X on a conditional expression.
- `scope <scope_name>`: Specify this switch to mention the scope required for oba, xassign, and undef.
- `signal_name`: Specify the signal name for which you want to create custom control of X injection sources on specific points.

Clock and reset signals defined using create_clock and create_reset commands cannot be used as X injection sources.

12.3.3 The fxp_assume -nox Command

Use the `fxp_assume` command to create custom control of X injection sources on specific points to remove an X from formal analysis.

Syntax

```
vcf> fxp_assume -nox [-name <name>] [-reset] [condition <condition>
 [<signal>]
```

Where,

- `name <name>`: Specify a name of the design for which you want to remove custom control of X injection sources.
- `reset`: Specify this switch to remove the X out of reset. It can only be used on sequentials, and the options cannot be used.
- `signal`: Specify the signal name for which you want to remove custom control of X injection sources on specific points.
- `condition <condition>`: Specify the optional combinational condition expression.

Feedback

12.3.4 The fxp_assert Command

You can also create custom observation points to check that a x cannot propagate to interesting points using the fxp_assert command.

Syntax

```
vcf> fxp_assert [-name <name>] [-condition <expr>] <signal_name>
```

Where,

- `name <name>`: Specify a name of the design to check that a x cannot propagate to interesting points.
- `condition <expr>`: Specify this switch to observe a x on a conditional expression.
- `signal_name`: Specify the signal name for which you want to observe the custom control of x injections.

  Note:

  You can also run the FXP application mode using the `check_fv` command.

  The `fvassume` command and SVA assumptions are supported in the FXP application.

12.4 Performing FXP Analysis in VC Formal GUI

VC Formal provides the capability to perform FXP checks using the VC Formal GUI.

When you run the `check_fv` command in the `vcf_shell`:

- The FXP task mode is added in the Task List.
- The FXP properties are listed in the *GoalList* tab.

To run the FXP checks:

- Click to run the properties.

  The status of the properties appears in the *Status* column.

12.4.1 Reviewing Results and Report Generation

For FXP the run results are the same as that of FPV. For more information on the run results, see section “Analyzing Assertion Results”.

Feedback

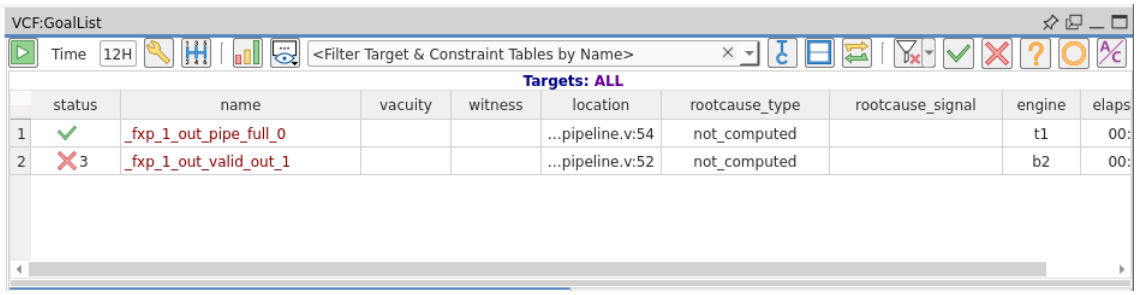
- Proven: Proven status indicates that it is not possible for any of the X-Injection sources to propagate to this observation point.
- Falsified: Falsified indicates that there is a path from an X-Injection source to an observation point.
- Checking: Checking status indicates that the properties are still being checked.
- Inconclusive: Inconclusive status indicates that in the time given, it has not been possible to fully prove or disprove a path from an X-Injection source to destination.

12.4.2 Debugging Failures

To debug failure in GUI,

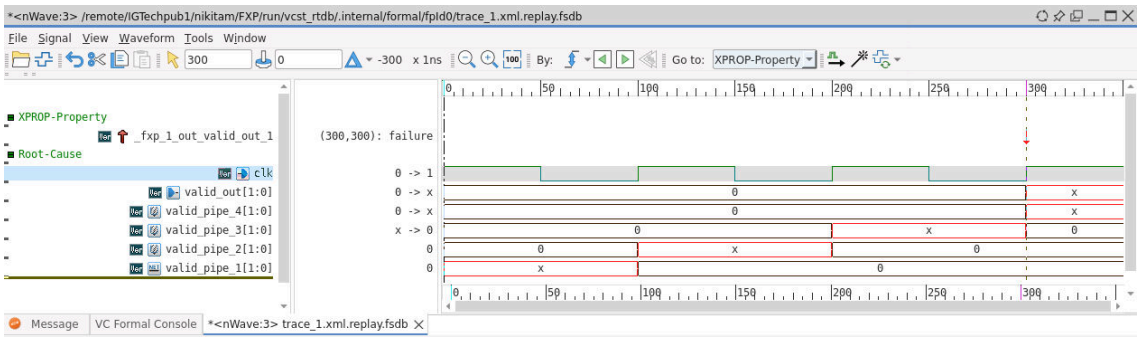
1. Click on (falsified property).

   The trace-failure with related signals in waveform appears. The point of failure is highlighted where the observability point goes to an x. The *rootcause_type* and *rootcause_signal* columns also appear at the same time as falsified waveform generation.



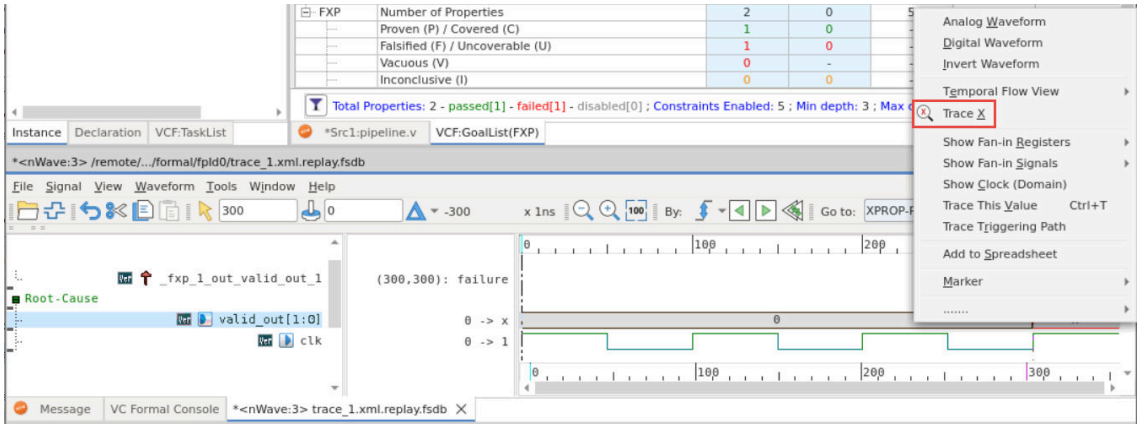
2. Right-click on the failure and select View Trace > Property.

   The nwave window appears.

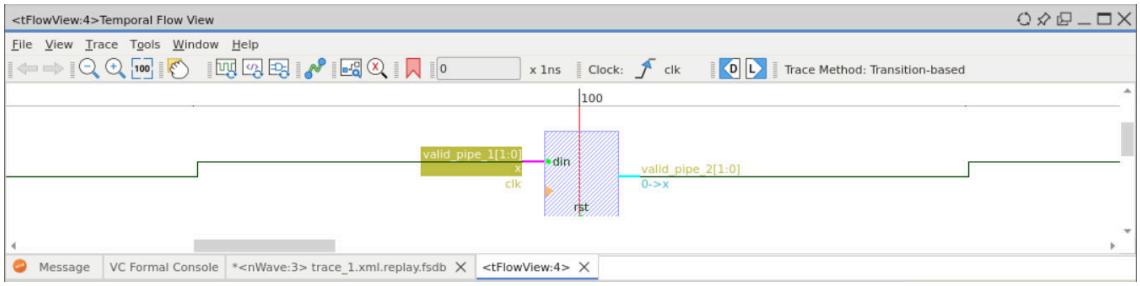


   Right-click on the failure and select *Trace X* to trace this back to the source of x.

Feedback



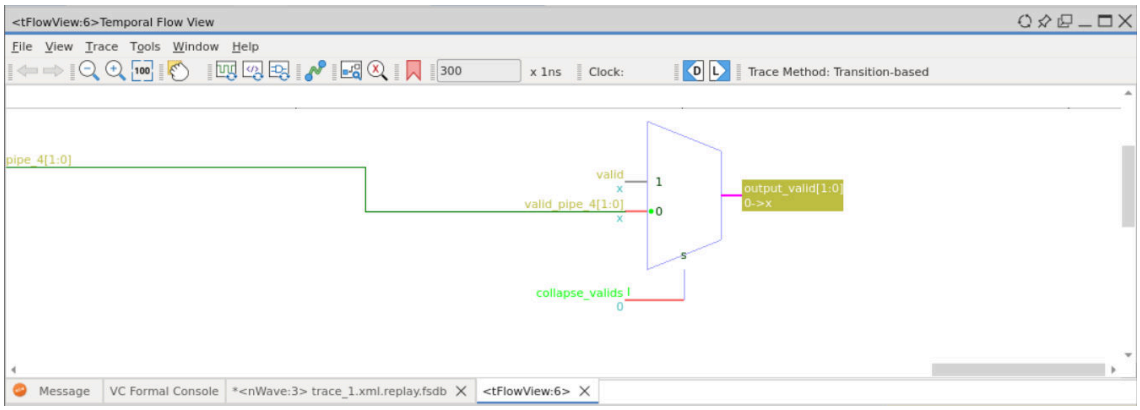
The source of X is also highlighted in schematic view as TFV.



3. Rerun the checks and ensure that all checks pass.

12.4.3 Debugging Failures with Temporal Flow View

1. To generate Temporal Flow View, RMB on the X-value signal transition > Select *Temporal Flow View* > *Auto Trace*.
2. The temporal view of X propagation is generated.



Feedback

12.4.4 Computing and Reporting Root Cause to Debug FXP Failure

FXP in VC Formal allows you to check that unknown signal values cannot propagate through a design for a falsified X-propagation property. You can get the details of the root cause signals automatically in addition to manual tracing using the following commands:

- `fxp_compute_rootcause`
- `fxp_report_rootcause`

You can compute the root cause signals for falsified FSV properties during the `check_fv` run. To enable this feature, set the following formal variable to true:

```
set_fml_var fxp_compute_rootcause_auto true
```

When the `fxp_compute_rootcause_auto` formal variable is set to true, use the `fxp_report_rootcause` command to get the report of root cause signals after the `check_fv` command.

By default, the `fxp_compute_rootcause_auto` formal variable is set to false.

Use Model

- To compute the root cause, use the following command:

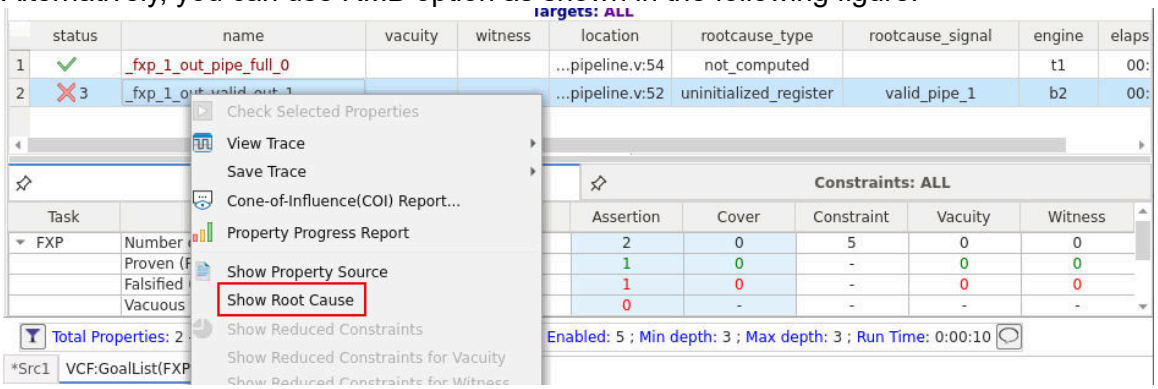
```
fxp_compute_rootcause                # Compute the root causes of the
 selected properties
[-property <list-of-property-name>] (A list of property names)
[-stop]                              (Clear the property list for
 fxp_compute_rootcause)
[-block]                              (Compute the root causes in the
 block mode)
```

- To report the root cause, use the following command:

```
fxp_report_rootcause                 # Report the root cause of the
 selected properties
[-property <list-of-property-name>] (A list of property names)
[-list]                              (Outputs one-line report per
 property)
[-no_summary]                         (Outputs without a summary of the
 root cause types)
```

Feedback

Alternatively, you can use RMB option as shown in the following figure:



Example

The `fxp_report_rootcause -property <property name> -list` followed by `fxp_compute_rootcause` gives the details about the root cause signal and root cause type.

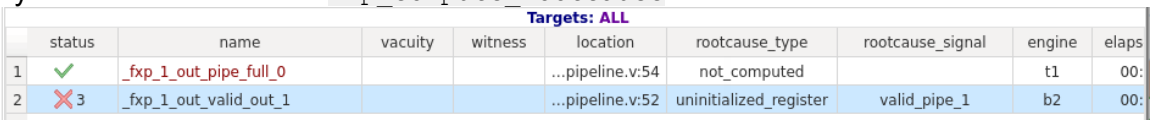
The following is an example of one such root cause information:

```
vcf>fxp_report_rootcause -property _fxp_1_out_pipe_full_0 -list
Summary of root cause information
---------------------------------
Total: 1
 uninitialized_register  :          1
List of root cause information
---------------------------------
 _fxp_1_out_pipe_full_0 |  falsified |  uninitialized_register |  pipe_counter
```

Property Name — Property Status — Rootcause Type — Rootcause Signal

This information can be used to identify and classify the issues. For example, in the above case it says, the property _fxp_1_out_pipe_full_0 is falsified due to pip_counter register being uninitialized. Now instead of debugging the entire failure trace, you can directly check if pipe_counter is non-resettable flop or its reset is gated or not declared and take corrective action.

Rootcause information is also populated in the property table by the RMB action or the `fxp_compute_rootcause` command.



13

Connectivity Checking Application

The VC Formal CC application enables verification of connectivity at the SoC level. Flexible input format ensures ease of integration. Powerful debugging, including value annotation, schematic viewing, source code browsing and analysis reporting speeds analysis. Automatic root-cause analysis of unconnected connectivity checks saves significant debug time.

Getting Started • *Understanding Connectivity Checks* • *Accessing CC Lab Example*	**Debugging, Reviewing and Saving Results** • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Analyzing Connectivity Checks Coverage* • *Saving and Restoring Sessions*
Setting Up and Running CC • *Enabling CC Application mode* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Specifying Connections* • *Initializing Design* • *Reviewing Design Complexity* • *Reviewing and Editing Properties* • *Running CC in TCL and GUI* • *Extracting Connections*	**Getting Help on Commands** • *CC Tcl Commands*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	

13.1 About Connectivity Checks

For many SoC designs, one of the common requirements is to verify SoC connectivity at the top level or connections between IP blocks. These connections may be direct wires or MUX-ed connections. Some of connections can also be I/O pad sharing or test logic. Formal Connectivity check can expedite such connectivity verification.

Feedback

Formal Connectivity Checking is about proving conditional device wiring. It checks if there is a structural connection between the source and the destination These Connectivity checks are directional in that it looks for value propagation from source to destination.

One can consider it as proving a property of the form (enable expressions) |-> (src == dest)

A given connectivity check is true if the following holds, if

- Connectivity Check Condition 1

  There is a structural path from the src to the dest. A structural path exists if:

  - There is a physical directional path from src to dest.
  - The bit width of the src and dest is equal.
  - The numbers of FFs between the src and dest is less than or equal to the path_delay.

- Connectivity Check Condition 2

  The value of the source and the destination are always the same when the enable expression, if any, is true.

  Note:

  A connectivity check which does not have a structural path is one that satisfies only Connectivity Check Condition2. This check is a functional only check. An example is when we have a = c; and b = c; A CC with a as src and b as dest will satisfy CCC2, but not CCC1.

Watch this video to explore the VC Formal Connectivity Checking (CC) application, and how to set up, execute, and debug connectivity checks.

Other Connectivity Checking Videos

Click the links to view videos on how to use the CC application:

- Connectivity Check Debug (11 mins)
- Connectivity Checking Coverage (6 mins)
- Connectivity Checking Enable Expression Coverage (5 mins)
- Connectivity Checking with Latency (9 mins)
- Connectivity Checking Extraction (8 mins)

13.2 VC Formal CC: Methodology

The CC application takes the design (RTL) and the connection specifications as inputs, and verifies if the connection specifications holds good for this design or not.

- Specifying Connectivity Properties
- Specifying CCs in CSV Format
- The load_cc CSV Fomat
- Specifying CCs in Table Format
- The report_load_cc Command
- Specifying CCs in Tcl Format
- Adding Connections With Latency
- Using Alias in the Specification

13.2.1 Specifying Connectivity Properties

A Connectivity Check (CC) contains the nine elements (name, src, dest, enable_expr, path_delay, enable_hold, enable_delay, offset_delay, clock) where:

1. A name is an alphanumeric string starting with a letter.
2. A src is a signal (net, port) (bit level, word level) of the Design Under Test (DUT) or a constant value.
3. A dest is a signal (net, port) (bit level, word level) of the DUT.
4. An enable expr is a combinational expression that enables the connection. The default enable_expr is true. (1)
5. A path_delay is a latency value or range (min:max) in the path from source to destination of the connection (default 0).
6. An enable_hold (EH) is a latency value which specifies the number of clock cycles (EH) enable must hold relative to the arrival time of source (default 0).
7. Enable_delay (ED) is the time when enable needs to be asserted relative to the source. This is negative or 0.

Feedback

8. Offset_delay (OD) is a latency value which specifies the offset from reset to start the connectivity check.
9. A clock is a signal of the DUT (typically a clock signal). The default clock is the one declared using the `create_clock` command. If a `path_delay` is present, and if a clock is not declared or included in the connectivity property, then an error is reported.

The connection and their enabling conditions are specified in a spreadsheet either in csv format or table format. This file can be loaded using the `load_cc` Tcl command. For more details, see section The load_cc CSV Fomat.

Connections can also be specified using the `add_cc` Tcl command as described in Specifying CCs in Tcl Format. Multiple load_cc commands can be used, and both `load_cc` and `add_cc` commands can be used together to specify connectivity checks.

The example design shown in Specifying CCs in CSV Format is used in many of the examples provided in the following sections.

13.2.2 Specifying CCs in CSV Format

The connectivity information is typically in the csv format. The information in the spreadsheet is saved in CSV format and read in to the tool using the `load_cc` command.

Syntax

```
vcf> load_cc [-format csv|table] [-quiet] <filename>
 [-default_param_order]
```

Where,

- `-format <format>`: The format of the file to be loaded. The two formats are csv or table. The format option is optional, and default is csv.
- `<filename>`: Path of the properly formatted csv file to read.
- `[-default_param_order]`: Loads parameters in file following the default order of fields.

  Default Order : *csv_prop_name, csv_src, csv_dest, csv_enable, csv_path_delay, csv_enable_hold, csv_enable_delay, csv_offset_delay, csv_clock, csv_num_enable_cols, csv_negedge_clock, csv_start_line, csv_end_line, csv_comment_pattern, csv_lpa_type*

  The `-default_param_order` requires the file name to be specified, specifying other fields are optional. Note that, specifying this option does NOT override the user-defined format set using `load_cc_set_param`. It enables the user to load files in default parameter ordering, even if they have a custom format defined in the script.

Feedback

- `-quiet:` Specify this option to hide the progress messages and summary results of the structural checks on the shell.

```
vcf > load_cc -quiet conn.csv
[Error] CC_UID017: Connection src__to__dest_4' is structurally
 disconnected. No path from source to target, and all signal are
 present in the instance top
        Please fix the connection
```

By default, the structural connections check progress appears and the *summary result*appears at the end of Structural Check. The maximum depth is reported for the disconnected properties.

For example,

```
vcf > load_cc conn.csv
[Info] STRUCT_CHECK_RESULT:  <PropertyName>    <Status>:Depth
 <Seconds: MilliSeconds: MicroSeconds>
-----------------------------------------------------------------
[Info] STRUCT_CHECK_RESULT: src__to__dest_0  Connected
           00:144:256
[Info] STRUCT_CHECK_RESULT: src__to__dest_1  Connected
          02:110:319
[Info] STRUCT_CHECK_RESULT: src__to__dest_2  Not applicable
 00:221:801
[Info] STRUCT_CHECK_RESULT: src__to__dest_3  Not applicable
 00:011:325
[Info] STRUCT_CHECK_RESULT: src__to__dest_4  Disconnected:8
     00:410:122
[Error] CC_UID017: Connection src__to__dest_4' is structurally
 disconnected. No path from source to target, and all signal are
 present in the instance top
        Please fix the connection
-----------------------------------------------------------------
Summary Results
---------------
 conn.csv
                         # Properties read:                         5
                         -     # Structurally Connected:           2
                         -     # Structurally Disconnected:        1
                         -     # Structural Check not applicable:  2
                         # Lines in CSV File:      6
          # Error:                  1
                         # Warning:                0
-----------------------------------------------------------------
```

In the CSV format, every line represents one CC, therefore, in general, you must specify nine columns of data and delimit them by a comma (,). However, if you do not specify the all the nine columns, default values are considered for the missing columns.

```
name,source_expr,destination_expr,enable_expr,path_delay,enable_hold,enab
le_delay,offset_delay,clock
```

Feedback

```
hier1_in1_to_hier2_in1,hier1.in2,hier1.inst.in1,
 ~hier1.sel&en&c_en,2,0,0,0,clk
hier1_in2_to_hier2_in1,hier1.in2,hier1.inst.in1,
 hier1.sel&en&c_en,2,1,0,0,clk
hier2_outv_to_hier1_outv,hier1.inst.outv,hier1.outv,hier1.en,1,1,0,0,clk
```

The mandatory columns are *src* and *dest*. If *enable_expr* is missing, VC CC assumes it is '1'.

Example

The file in CSV format can be as follows:

```
assert1, x, y, "ctl == 0", 0
assert2, x, y, 1, 2
assert3, x, y, (a == 1) && (b == 0) && (c[0] == 1), 0
```

13.2.3 The load_cc CSV Fomat

You must adhere to the following when you create a csv format for the `load_cc` command.

- Each line in the CSV file provided to the `load_cc` command is set of strings separated by commas (the delimiter). Each string is called a field. The first string is denoted by %1%, the second by %2%, and so on.
- Fields can remain empty.
- A line can contain a comment marker (at the beginning). All characters in the line after the comment marker are ignored. The comment marker is controlled by the `csv_comment_pattern` parameter. The default is "//"

  Example

  ```
  assert1, x, y, "ctl == 0", 0
  assert2, x, y, 1, 2
  // assert3, x, y, (a == 1) && (b == 0) && (c[0] == 1), 0
  ```

- When you want to load connections with duplicate names from multiple csv files using the `load_cc` command, set the following formal variable to true

  ```
  set_fml_var fml_cc_enable_csv_based_naming true
  ```

  When the `fml_cc_enable_csv_based_naming` is set to true, csv file name is appended to the property as a prefix `'<csv_file_name>__<property_name>'`

  For example,

  `file1__conn1` // load_cc file1.csv. file1.csv has property conn1

  `file2__conn2` // load_cc file2.csv. file2.csv has property conn2

  By default, the `fml_cc_enable_csv_based_naming` variable is set to false.

Feedback

By default, when there are duplicate connection names in the multiple csv files, the following error message is reported:

[Error] CMD_E_PROP_ALREADY: Property p already exists. Resolve the name collision and try again. Remove the property first if possible. Also, same connection names across csv files are supported by enabling fml_var - fml_cc_enable_csv_based_naming

Setting the Order of the Fields in the File

You can control the order of fields in the file by using the `load_cc_set_param` command.

To loads parameters in file following the default order of fields, use the `[-default_param_order]` option in the `load_cc` command.

Default Order : csv_prop_name, csv_src, csv_dest, csv_enable, csv_path_delay, csv_enable_hold, csv_enable_delay, csv_offset_delay, csv_clock, csv_num_enable_cols, csv_negedge_clock, csv_start_line, csv_end_line, csv_comment_pattern, csv_lpa_type

Use Model

```
vcf>load_cc_set_param <param-name> "%<value>%"
```

Where,

- `<param-name>`: Can be one of the following: comment, csv_prop_name, csv_src, csv_dest, comment, csv_start_line, csv_path_delay, csv_enable_hold_time.
- `<value>`: Is a positive number specifying the corresponding parameter's column location.

Table 20 provides the list of available parameters along with their default values.

Table 20: The load_cc command Parameters for CSV Fomat

Parameter name	Value type	Default location	Default Value
csv_prop_name	String	%1%	
csv_src	String	%2%	
csv_dest	String	%3%	
csv_enable	String	%4%	True
csv_path_delay	Min:max | fixed Integer		0
csv_enable_hold	Integer		0
csv_enable_delay	Integer		0

Feedback

Table 20: The load_cc command Parameters for CSV Fomat (Continued)

Parameter name	Value type	Default location	Default Value
csv_offset_delay	Integer		0
csv_num_enable_cols	Integer		1
csv_clock	String		
csv_negedge_clock	String		
csv_start_line	String		
csv_end_line	String		
csv_comment_pattern	String		//
csv_lpa_type	String		

In general, a CC requires defining at most nine parameters. The lines of the CSV file need not contain nine fields. It is recommended to define all used columns and not rely on default conditions.

- The order of the parameters can be changed by the `load_cc_set_param` command.
- In addition parameters can be defined as a concatenation of two or more fields by the `load_cc_set_param` command.

A parameter whose field is empty takes the default value. In case a parameter is constructed by two or more fields and some of these fields are empty, such parameters are skipped.

Example

- The following example sets the CSV column 3 to be the enable column, and load lines 5-10 (inclusive) from the CSV file.

```
load_cc_set_param csv_prop_name "%1%"
load_cc_set_param csv_clock "%2%"
load_cc_set_param csv_enable "%3%"
load_cc_set_param csv_enable_hold "%4%"
load_cc_set_param csv_from "%5%"
load_cc_set_param csv_to "%6%"
load_cc_set_param csv_path_delay "%7%"
load_cc_set_param csv_start_line "2"
load_cc -format csv $testDir/connections.csv
```

- Load the connectivity checks after loading the design

Feedback

```
    set_fml_appmode CC
        set design chip_top
        read_file -top $design -cov tgl -format verilog -vcs
{../design/cctest.v}
        load_cc_set_param csv_enable "%1%"
        load_cc_set_param csv_src "%2%"
        load_cc_set_param csv_dest "%3%"
        load_cc_set_param csv_prop_name "%4%"
        load_cc_set_param csv_start_line "2"
        load_cc <>
```

To load connectivity property mentioned in csv format from cctest.csv to Goallist

To report the properties after `check_fv` in CSV format

```
report_fv -formatCC report_properties.csv
```

report_properties.csv is saved in default field ordering. Default Order: csv_prop_name,csv_src,csv_dest,csv_enable,csv_path_delay,csv_enable_hold, csv_enable_delay,csv_offset_delay, csv_num_enable_cols,csv_clock, csv_negedge_clock, csv_start_line, csv_end_line, csv_comment_pattern, csv_lpa_typ

To load `report_properties.csv` after making any changes to the field values, use

```
load_cc -format csv cctest.csv
check_fv -block
report_fv -formatCC csv > report_properties.csv
fv_delete *
load_cc -default_param_order report_properties.csv
```

To load connectivity property without giving summary or structural connectivity check information about the added properties. However, errors and warnings related to this command are displayed even with -quiet option.

```
load_cc -format csv -quiet cctest.csv
```

Concatenated Parameter Fields

To specify that a parameter is composed of multiple fields, the value argument are of multiple concatenated fields. The source and destination fields may be specified as a concatenation of the values in two columns, for example the first indicating hierarchical path and the second indicating the signal name, for example,

```
load_cc_set_param csv_src "%3%.%4%"
load_cc_set_param csv_dest "%5%.%6%.%7%"
```

The concatenating character is always a hierarchy delimiter. If one of the fields is left empty on some line then the resulting field is skipped and the constructed name drops the unnecessary delimiter characters.

Feedback

For the CSV file:

```
foo,my_foo,,bar,modem,tx,en2
foo,my_bar,mid,x,router,,clk
```

And the parameter setting above, the from signal on line 1 is bar, and the to signal is modem.tx.en2. The from signal on line 2 is mid.x and the to signal is router.clk.

Comments in CSV File

Lines in the CSV file that has the character defined by `csv_comment_pattern` in the first position of the line are treated as comments, that is, they are ignored. Treating lines as comments is supported both for the CSV format and table format.

Example 1 (first field not used)

```
load_cc_set_param csv_src "%2%"
load_cc_set_param csv_dest "%3%"
load_cc_set_param csv_enable "%4%"
```

In the CSV file:

```
foo, A, B, EN1, conn34
//foo, C, D, EN2, conn35
//,E, F, EN3, conn36
```

In the above example, the first check *conn34* is included, but the second and third - *conn35* and *conn36* are skipped.

Example 2

The comment character may also be changed by setting the `csv_comment_pattern` parameter. In this example, change it to "#" instead of "//".

```
load_cc_set_param csv_comment_pattern "#"
load_cc_set_param csv_src "%1%"
load_cc_set_param csv_dest "%2%"
load_cc_set_param csv_enable "%3%"
```

In the CSV file:

```
A, B, EN1, conn72
#C, D, EN2, conn73
```

In the above example, the first connection, *conn72* is included, but the second *conn73* is skipped.

13.2.4 Specifying CCs in Table Format

The table format is an enhanced CSV Format, where you can also provide the enable conditions in several columns instead of one column in the CSV. The supported enable

Feedback

syntax is limited to a conjugation of bit – or word-level assignments, for example, *(a == 1) && (b == 0) && (c[0] == 1)*.

Every enable column in the file is placeholder for one enable signal (or signal bit). The corresponding column cell may contain a constant - which means that signal is equal a constant in the enable expressions. Otherwise, the cell must be empty - which means that the signal does not actively participate in the enable expression.

There must be two header lines in the table followed by the connectivity checks data.

- Line 1 – the enable signal names, if an enable signal is a vector of N > 1 bits, its name must appear only appear in the first column, all the following N-1 columns can be empty
- Line 2 – the bits of the enable signals

In the following example, assume ‘c’ is defined as a vector(0:2))

```
name, source, dest, ctl, a, b, c, , , source_delay
, , , , , , 0,1,2
assert1, x, y, 0, , , , , , 0
assert2, x, y, , , , , , , 2
assert3, x, y, , 1, 0, 1, , , 0
```

13.2.5 The report_load_cc Command

The `report_load_cc` command reports all status, including errors, warnings, and connected information generated from `load_cc` command. You can use the `report_load_cc` command to find issues of the `load_cc` command. The output of `load_cc` command is consolidated and presented in an organized manner using the `report_load_cc` command. You can use the following options to filter the `report_load_cc`output.

Syntax

```
vcf> report_load_cc [ -list] [-verbose] [-no_summary] [-error] [-warning]
 [-connected] [-disconnected] [-not_applicable] [filename]
```

Where,

- `[-list]`: Specify this option to report the summary results and then lists the status, line number, and connection name of each property file-wise.
- `[-verbose]`: Displays the summary of results and then provides detail information about the report such as the status, connection name, line number and message of each property file-wise.
- `[-no_summary]`: Specify this option if you do not want to see the summary information for any other switch.

Feedback

- `[-error]`: Specify this option to get a report of errors. You can view error details in file-wise. Appending the `-error` switch with `-list` and `-verbose` switches, displays all errors from the list and verbose.
- `[-warning]`: Specify this option to get a report of warnings. You can view the warnings file-wise. Appending the `-warning` switch with `-list` and `-verbose`, displays all warnings from the list and verbose.
- `[-connected]`: Specify this option to get a report of the structurally connected properties. You can view the structurally connected properties file-wise. Appending the`-connected` switch with `-list` and `-verbose`, displays all structurally connected properties from the list, and verbose results.
- `[-disconnected]`: Shows structurally disconnected properties only.
- `[-not_applicable]`: Shows properties for which structural check is not applicable (disabled/constant src).
- `[filename]`: Specify this option to view the details of the specified file.

Example

Currently, the output of this command is provided in textual format only.

```
vcf> report_load_cc
Summary Results: CSV <FileName1>
# Properties successfully read : 2
# Lines in CSV file : 6



# Error : 3
# Warning : 2
CSV <FileName2>
# Properties successfully read :
# Lines in CSV file :
# Error :
# Warning :
.....................................................
..................................................... CSV <FileNameN>
# Properties successfully read :
# Lines in CSV file :
# Error :
# Warning :
```

13.2.6 Specifying CCs in Tcl Format

In addition to loading a large number of connections from a file, you can use the `add_cc` command to specify additional connections that must be checked. The `add_cc` commands may also be loaded from a file using the Tcl source command.

Feedback

Syntax

```
vcf> add_cc
-src <source-signals>
-dest <destination-signals>
[-enable <enable-expr>]
[-path_delay <path_delay]>]
[-enable_hold <enable_hold_time>]
[-enable_delay <enable_delay>]
[-offset_delay <offset_delay>]
[-clock <clock-expr>]
[-name <name>]
```

Where,

- `src <source-signals>`: Provide an ordered list of signals to use as source of a connection to check. Wild card characters are supported.
- `dest <destination-signals>`: Provide an ordered list of signals to use as destination of a connection to check. Wild card characters are supported.
- `enable <enable-expr>`: Provide a combinational expression that enables the connection (the default value is true).
- `path_delay <min:max| fixed latency>`: The CC delay between src and dest (default 0).
- `enable_hold <enable_hold_time>`: The hold time of the enable expression (default 0).
- `enable_delay <enable_delay>`: Enable time relative to source, that is, how long before src does enable need to be asserted (default 0).
- `offset_delay <offset_delay>`: Number of cycles after reset to start connectivity check.
- `clock <clock-expr>`: Optional clocking expression for the CC. The default is to check on the posedge of the clock declared with `create_clock`.
- `name <string>`: User defined name for the check.

Note:

Any net/pin/port/operator/instance which is not encrypted is a valid input to `add_cc` and `load_cc` commands. This holds true for source, destination, enable, clock signals, ports on the interface of encrypted modules. You can specify the complete name of a net inside the encrypted module, if you want to consider the encrypted module as a valid source/destination.

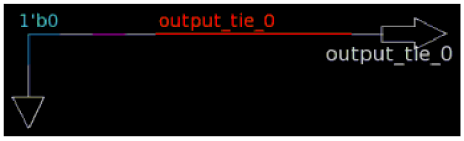
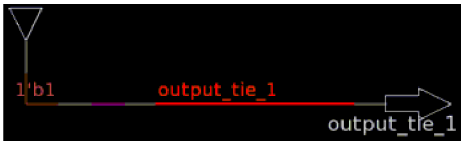
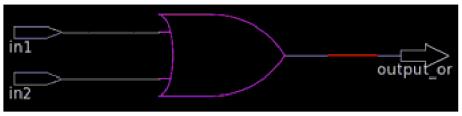
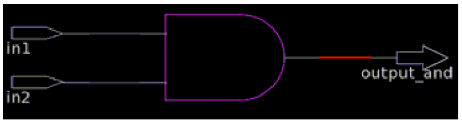
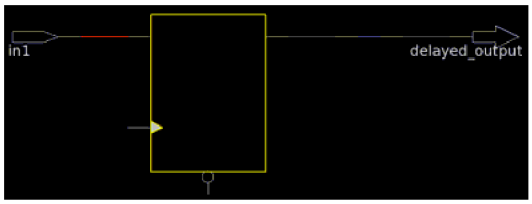
Examples

Feedback

Table 21 provides examples for adding following types of connections:

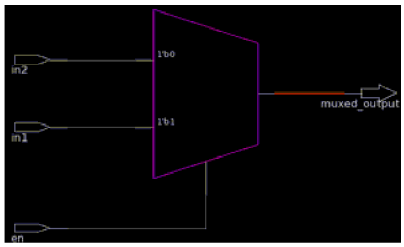
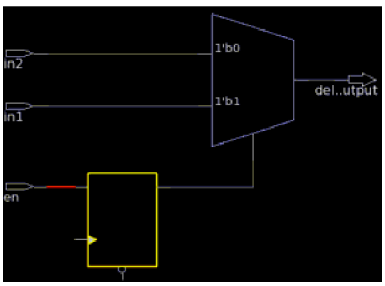
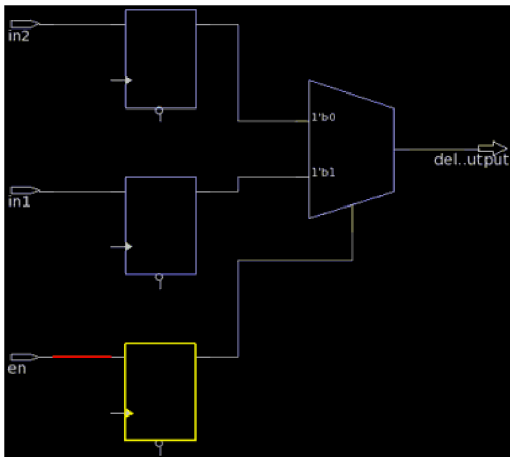
- Simple
- Gated
- Delayed
- Muxed - with different flop position on input, output, enable

Table 21: Specifying Connections Using add_cc Command

Schematic	Command
 Simple Connection	`add_cc -src 1'b0 -dest output_tie_0`
 Simple Connection	`add_cc -src 1'b1 -dest output_tie_1`
 Gated Connection	`add_cc -src "in1|in2" -dest output_or`
 Gated Connection	`add_cc -src "in1&in2" -dest output_and`
Gated Connection	`add_cc -src in1 -dest direct_output`
 Delayed Connection	`add_cc -src in1 -dest delayed_output -path_delay 1 -> delayed_output == $past(in1)`

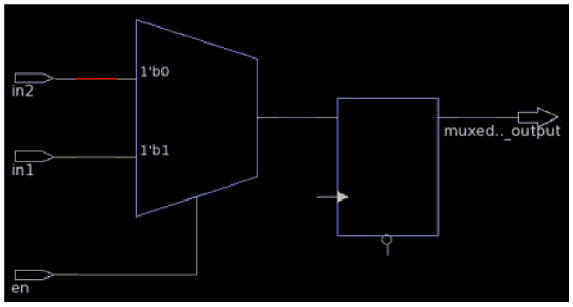
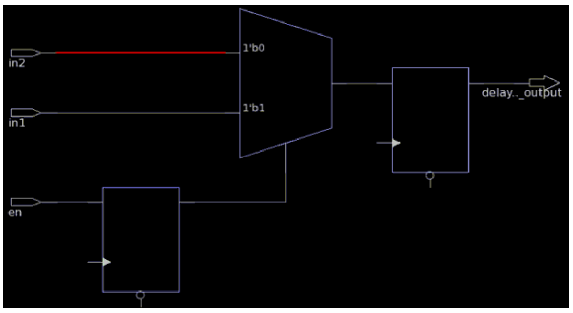
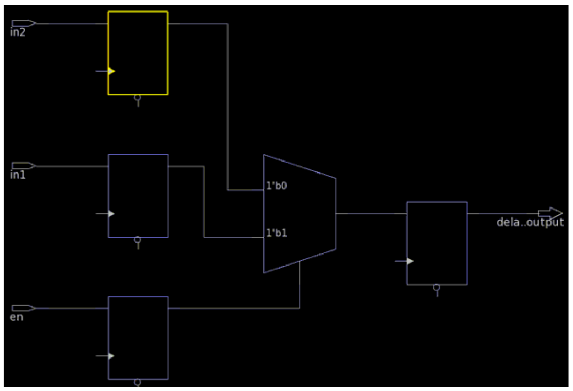
Feedback

Table 21: Specifying Connections Using add_cc Command (Continued)

Schematic	Command
 Muxed Connection	`add_cc -src in1 -dest muxed_output -enable en`
 Muxed Connection	`add_cc -src in1 -dest delayed_en__muxed_output -enable en -enable_delay -1-> ##-1 en |-> ##1 delayed_en__muxed_output == $past(in1)`
 Muxed Connection	`add_cc -src in1 -dest delayed_inputs__delayed_en__muxed_output -enable en -path_delay 1-> en |-> delayed_en__muxed_output == $past(in1)`

Feedback

Table 21: Specifying Connections Using add_cc Command (Continued)

Schematic	Command
 Muxed Connection	`add_cc -src in1 -dest muxed_delayed_output -enable en -path_delay 1-> en |-> delayed_en__muxed_output == $past(in1)`
 Muxed Connection	`add_cc -src in1 -dest delayed_en__muxed_delayed_output -enable en -path_delay 1 -enable_delay -1 ->##-1 en |-> ##2 delayed_en__muxed_output == $past(in1)`
 Muxed Connection	`add_cc -src in1 -dest delayed_inputs__delayed_en__muxed_delayed_output -enable en -path_delay 2 -> en |-> ##2 delayed_en__muxed_output == $past(in1,2)`

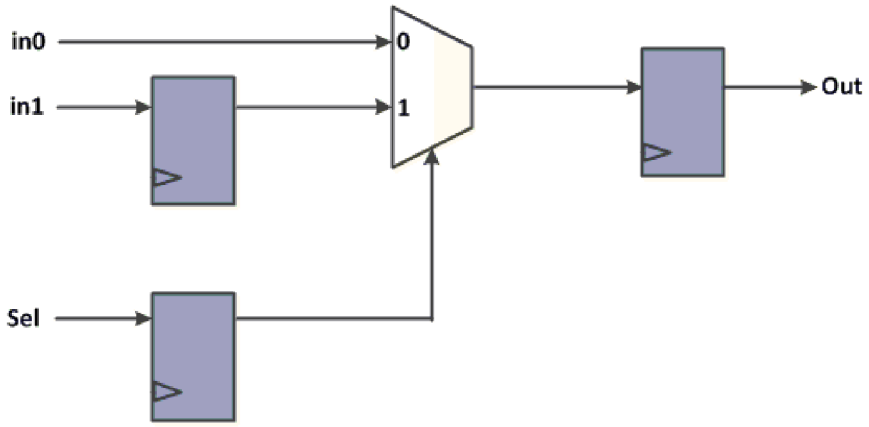
13.2.7 Adding Connections With Latency

You can specify latency in the connection between two points both in the Tcl format and the CSV format. The latency can be a fixed (or variable) number or number of cycles.

Feedback

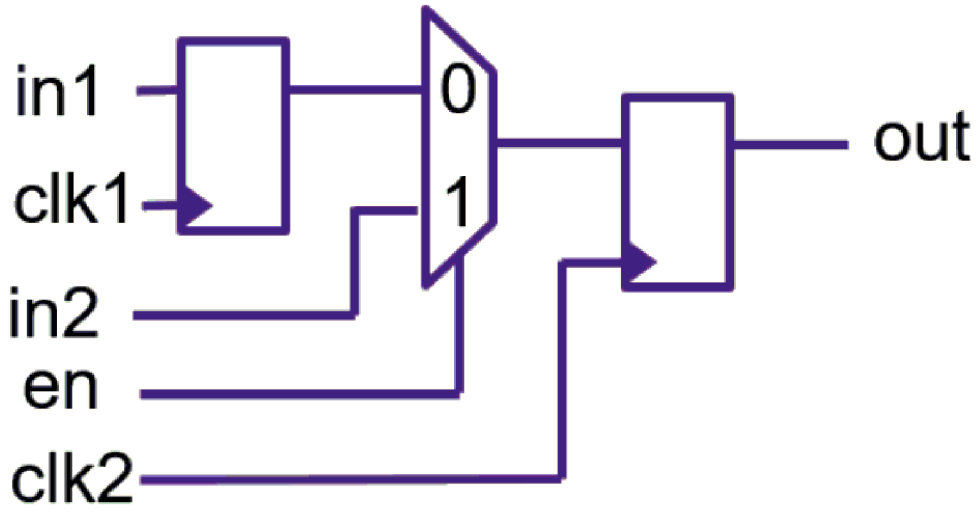
Latency can occur on the control path, that is, the enable condition, the input side or the output side of the connection as shown in Figure 81:

Figure 81: Connections With Latency



Only one clock is supported for connectivity checks with latency. A connection from in1 to out as shown in Figure 82 is illegal because there are two clocks in the path. To verify the connectivity between in1 and out, declare clk1 and clk2 the same, and then use clk1 in the property.

Figure 82: Connections With Latency Example



Watch this video to understand how to use VC Formal's Connectivity Checking application to verify connections with latency or delay.

Feedback

The `add_cc` command has additional options used to specify the latency of a connection as shown in the following code snippet. The source signal is always asserted at time 0 and this is the reference for the other delay parameters. For the *in0* path, the enable signal needs to arrive at the mux at the same time as the source so it needs to be asserted 1 cycle before in0, that is, enable_delay is -1, since the delay on the enable path is greater than the delay on the in0 path to the mux. For the in1 path, enable is asserted at the same time as the source so both enable_delay and enable_hold are 0.

```
add_cc -src in0 -dest Out -enable ~Sel -path_delay 1 -enable_hold 0
-enable_delay -1 -clock clk
add_cc -src in1 -dest Out -enable Sel -path_delay 2 -clock clk
```

Each connectivity property with latency can have an associated clock which is specified with the `-clock` option to the `add_cc` command or the `csv_clock` parameter for a csv file. If no clock is specified, the reference clock specified with the `create_clock` command is used. All the delays are specified relative to the clock of the property.

The `create_clock` command needs to be specified before `add_cc` or `load_cc` commands. It is recommended to always add the `-clock` option to `add_cc` and to specify the clock column in csv files.

Similarly, five fields are added to the CSV file format that are specified by the `load_cc_set_param` command:

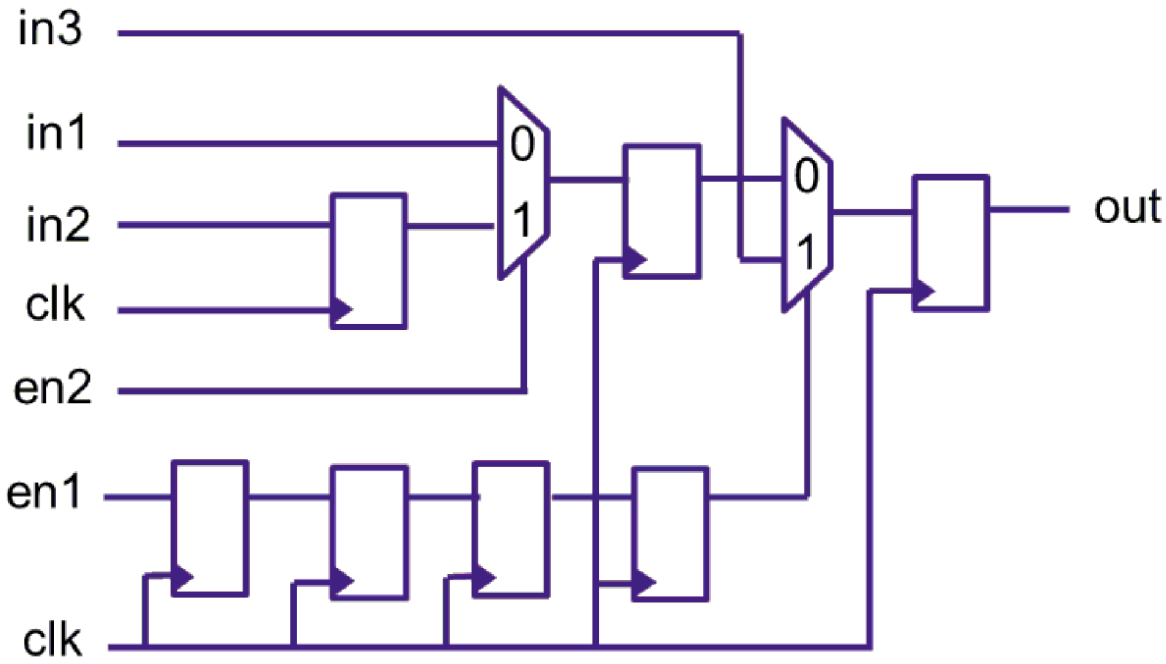
```
load_cc_set_param csv_path_delay "%5%"
load_cc_set_param csv_enable_hold "%6%"
load_cc_set_param csv_clock "%7%"
load_cc_set_param csv_enable_delay "%8%"
load_cc_set_param csv_offset_delay "%9%"
```

If the connectivity check has multiple enable signals with different delay one need to consider enable delay and enable hold for each enable and then specify the longest enable hold and earliest enable delay for the connectivity property.

In the example shown in Figure 83, for the in1 to out connection there are two enable signals, en1 and en2, with different delay.

Feedback

Figure 83: Connections With Latency Example



- *in1* and *en2* arrive at the mux at the same time so enable delay for en2 is 0.
- Enable hold for en2 is 0 because it arrives at the same time as in1 at the mux, and does not affect the connection after that time.
- en1 has delay of 4 cycles and in1 has delay of 1 cycle to reach the mux so en1 has to be asserted 3 cycles before in1.
- en1 is asserted 3 cycles before in1 and must be held for 3 cycles until in1 arrives so enable hold is 3 cycles.
- The enable delay for the connection is the earliest of the enable delays for each enable, here -3.
- Enable hold for the connection is the longest of the enable holds for each enable, here 3.

The complete connectivity check for in1 to out is:

```
add_cc -src in1 -dest out -enable {~en1 && ~en2} -path_delay 2
 -enable_delay -3 -enable_hold 3 -clock clk
```

Feedback

Specifying Undriven net(z) in Source, Destination and Enable Condition in Connectivity Check

VC Formal CC supports undriven net in source, destination, or in enable condition. This enables you to verify if a net is an undriven net or not.

For example, consider the following RTL code:

```
wire A;
assign A = 1'bz;
```

The following CC checks are proven:

```
add_cc -src 1'bz -dest A
```

Specifying a Range in the path_delay Parameter

Sometimes, with a change in design, the delay between source to destination path might change. In some cases, the actual number of cycles of this delay is not important, as long as it is within a specified range.

You can specify a range in the path delay parameter in both the CSV format, and in the `-path_delay` option of the `add_cc` command.

The range is specified as `-path_delay {min_latency : max_latency}` in the `add_cc` command.

```
add_cc -src in1 -dest out -enable {~en} -path_delay {1:3} .
```

- In the `add_cc -path_delay` command, the field is delimited by " " or {}. For example, `-path_delay {3:5}` or `-path_delay "3:5"`.
- In the `path_delay` column of the CSV file, only the format`min_delay:max_delay` is supported. For example, 3:5.
- Latency values must be positive or zero.
- The value of `min_latency` must be less than or equal to the value for `max_latency`, that is, {3:5} is legal, but {5:3} is not.
- The difference between min and max latency must not be more than 5. If the difference is more than 5, an error is reported. However, you can downgrade this error to a warning using the `set_message_severity` application variable. When this application variable is set, VC Formal reports a warning message that this is not recommended, but continues to converge the property on the range provided.
- If `min_latency` is the same as `max_latency`, this is the same as specifying a `path_delay` with no range. For example, `-path_delay {3:3}`is the same as`-path_delay 3`. Both `min_latency` and `max_latency` values must be provided. The following cases are illegal: `-path_delay { :3}` or -`path_delay {3: }`. When such

Feedback

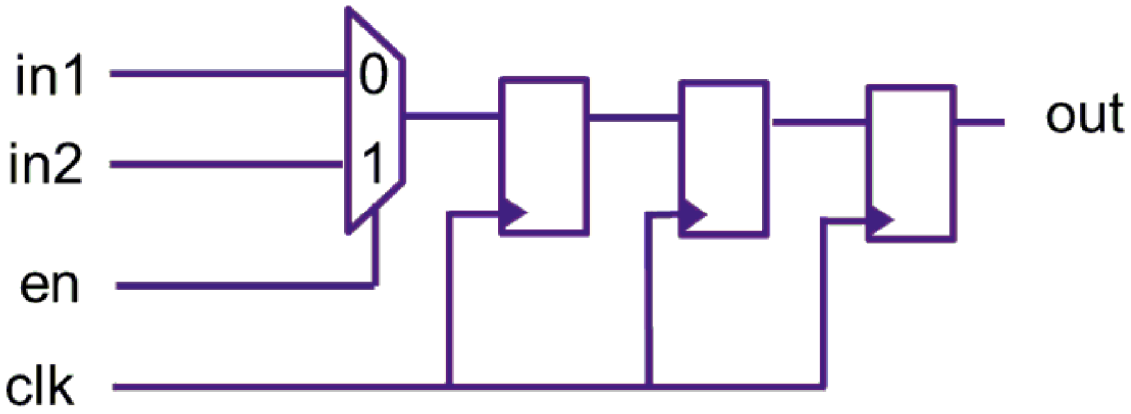
values or formats are specified for `path_delay`, an error message is reported, and the checks are not performed.

- A range is not supported for `enable_delay` and `enable_hold` parameters. These values may need to change if a connectivity property is changed from fixed latency to a range of values.

Example

In the example in Figure 84, the delay specified for `path_delay` is a fixed value. If a connection has the same latency as the number of cycles specified in the check, the check holds good.

Figure 84: Range in Path Delay Example




```
add_cc -src in1 -dest out -enable {~en} -path_delay 1       Falsified
add_cc -src in1 -dest out -enable {~en} -path_delay 2       falsified
add_cc -src in1 -dest out -enable {~en} -path_delay 3       proven
add_cc -src in1 -dest out -enable {~en} -path_delay 4       falsified
add_cc -src in1 -dest out -enable {~en} -path_delay 5       falsified
```

VC Formal supports a range of path delays. Instead of a fixed number, you can specify range, for example, {1:3}. This means that the following check will be proven:

```
add_cc -src in1 -dest out -enable {~en} -path_delay {1:3}
```

This check will also be proven because the actual delay of 3 is inside the range:

```
add_cc -src in1 -dest out -enable {~en} -path_delay {0:5}
```

The check would be proven if at least one delay value is within the range, provided by the user.

Feedback

If the actual delay (for example, 3) is outside the range specified, the check would fail. For example, the following check would fail:

```
add_cc -src in1 -dest out -enable {~en} -path_delay {4:9}
```

13.2.8 Using Alias in the Specification

Sometimes the hierarchical path to the source or destination can be quite long, and it might be difficult to create and review the specification. To make it easier, you can set an alias for the common portion of the path used in many checks. The alias can then be used to specify source, destination, enable expression and clock definition in the files given to the `load_cc` and `add_cc` commands.

You can set, report, or delete an alias using the following commands:

- `set_cc_alias [-nest]`

  Use this command to define a new alias. Example: `set_cc_alias foo top.core.bar`

  Only characters A-Z, a-z, 0-9 and _ can be used to define an alias. Special character "." is reserved for path separator, therefore do not use the "." symbol in the alias. However, multiple aliases in the same signal name is allowed. For example:

  topA.coreB.sigC is a destination signal in the RTL.

  ```
  set_cc_alias A topA
  set_cc_alias B coreB
  add_cc -src in1 -dest A.B.sigC
  ```

  Specify the`-nest` switch to enable the recursive name resolution for connectivity alias. For example,

  ```
  set_cc_alias portc port_ctl[5]
  set_cc_alias testc test_ctl
  set_cc_alias chipt1 chip_top
  set_cc_alias -nest chipt  chipt1
  set_cc_alias -nest chipt2 chipt1.port_ctl
  set_cc_alias -nest test {chipt.testc==8'h20 && chipt.portc==1'b1 &&
   chipt2==8'h0 }
  add_cc -name test123 -src chip_top.modb_in[3] -dest chip_top.ibe
   -enable test
  ```

- `report_cc_alias`

  Use this command to get a list of defined aliases. This command can be used to query individual or a group of aliases using wildcards.

- `delete_cc_alias <alias_name>`

Feedback

Use this command to delete the previously defined alias.

Note:

You can define the same alias twice. The latter alias defined will override the previous one.

13.3 Specifying Initial State

13.3.1 Specifying Clocks and Resets

Setup of clocks and resets for initialization of the design may be necessary for connectivity verification. For example, if mux control signals depends on register bits, the design must start in a legal state. If they only depend on primary inputs, initialization of the design is not needed. For example, a connection only depends on one or more primary inputs such as test mode.

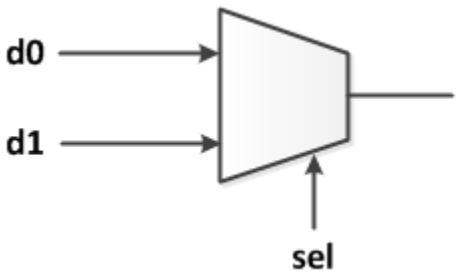
For a sequential connection, it is mandatory to have the clock setup. You can use `create_clock` command (as defined in section Setting Up User-Defined Clocks).

13.4 Specifying Constraints

If there are muxes or gates in the path being checked, the connections are only valid under certain conditions. The constraints specify these conditions.

For example, consider the connections through the mux shown in Figure 85:

Figure 85: Connectivity Through MUX Example



There are two paths from inputs to output but they are not always connected. They depend on the mux select input (sel).

- d0 to out is only connected when sel == 0
- d1 to out is only connected when sel == 1

Feedback

There are two methods for specifying constraints for connectivity checks in VC:

- Property enable expression
- The set_constant commands

The primary method to specify a constraint for a connectivity property is by using an enable expression as part of the property. For the example above, the two connectivity properties with enable expressions would be:

```
add_cc -src d0 -dest out -enable ~sel
add_cc -src d1 -dest out -enable sel
```

When the enable expression is the constraint it only applies to one connection and both connections can be proven together. Using a property enable expression is the recommended method to constraining connectivity properties.

The `set_constant` command forces a constant value on to a net or a pin that applies to all connectivity checks and assertion checks. There is no need to cut the net being forced. The `set_constant` command overrides any internal driver on the net. If the command `set_constant sel -value 1'b0` is used in the mux example given above, the d0 to out connection passes but the d1 to out connection fails since d0 is permanently selected. In order to get the d1 to out connection to pass, the command needs to be changed to `set_constant sel -value 1'b1`. Since the `set_constant` value applies to all checks, you can not get both mux checks to pass at the same time. The `set_constant` command is therefore used for constraints applicable to the entire verification run, for example, to disable the test or scan mode.

If the SVA constraints are read in, they will be applied to all connectivity check properties just like the `set_constant` constraints. SVA constraints and `fvassume` constraints are not supported when automatic blackboxing is enabled and are thus not recommended.

13.5 Structural Checks

Structural checks is by default enabled as part of the connectivity check. The structural checks can be controlled by the following formal variable:

`fml_cc_structural_check <none|path|full>`

When the `fml_cc_structural_check`application variable is set to:

- `none`: The structural checks are not performed.
- `path`:Enables checks of the existence of a structural path from source to destination.
- `full`:An additional check of the existence of a structural path from enable to destination is enabled, along with the structural path from source to destination.

Feedback

By default, the `fml_cc_structural_check` application variable is set to `path`. When the connectivity specification is read in, if there is no structural connection between and source and destination, an error message is displayed.

Example

```
add_cc -src Out2_CoreA -dest Po3 -enable ~Out2_Ctrl
```

[Error] CC_UID017: Connection 'connection_7' is structurally disconnected. No path from source to target

Please fix the connection

{"connection_7"}

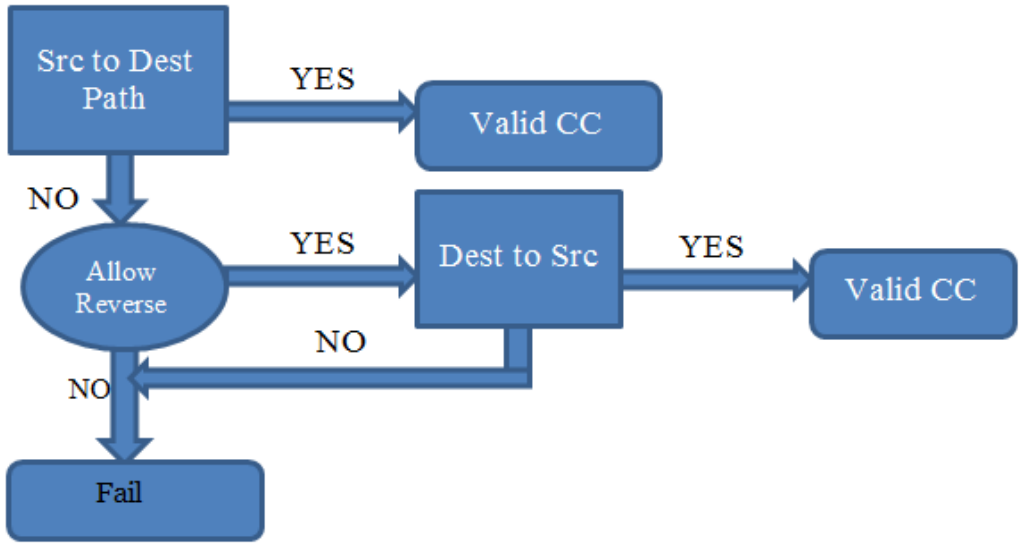
13.5.1 The fml_allow_reverse_cc_path Formal Variable

The `fml_allow_reverse_cc_path` application variable allows you to enable/disable reverse path connectivity checks. In VC Formal CC, direction is important. By default, the structural check is done in one direction, that is, from source to destination.

This is useful for legacy designs where direction was not considered in the specification but is not recommended for new designs. The connectivity property will be proven if there is a structural path from destination to source and they are always equivalent when the enable expression is true.

Figure 86 shows the flow of a structural check.

Figure 86: Structural Check Flow



To allow reverse structural check, you have to set the following application variable to true. By default, this variable is set to false.

```
vcf > set_fml_var fml_allow_reverse_cc_path true
```

Having enabled the reverse path application variable, if the default structural check fails, then the structural check is performed in the reverse direction. If the reverse structural check fails, then the connection is set to be structurally disconnected.

If the reverse structural check passes, the source and destination signals are swapped in all the reports of the application.

13.6 Automatic Black-boxing

Connectivity checks often verify top-level connectivity so that the low-level functions of the chip are not needed and must be black-boxed. This is essential in improving the design read process since SoC level elaboration can often take a very long time. The more the black-box modules and instances present before the design read commands, the better the performance. However, inappropriately black-boxing too many modules may lead to false failures as the necessary logic may be removed.

Black-boxing modules that are not needed will decrease the amount of memory used and the overall runtime. Manually determining which modules to black-box is not always easy. If you black-box too few modules, you do not get the memory and runtime improvements. If you black-box too many modules, you may get false failures because the path is broken or logic is missing in the control of the path.

VC Formal can automatically determine which modules and instances to black box at run-time. This means the user does not have to identify which modules to black box, and does not risk false failures due to incorrect black boxing. Note, automatic black boxing is done after the design is read, so it does not improve compile time or memory requirements.

13.6.1 Aggressive Automatic Black-boxing

The VC Formal CC application deploys an aggressive automatic black-boxing mechanism to optimize the connectivity verification problem, and enable efficient checking of connections on SoC level designs.

This feature is aligned with the verification requirement to verify the path exactly from source signal to destination signal. The cone of influence of the source signal has no impact on the connection status.

Feedback

13.6.2 The fml_cc_autobbox Application Variable

The CC application of VC Formal is mostly used at the SOC level. Formal verification is effort intensive at the SOC level. In the CC application mode, several optimization techniques are introduced under the application variable `fml_cc_autobbox` to handle SOC level designs. This helps in quick convergence of connectivity checks. By default, this application variable is set to true.

You can disable this application variable by setting it false.

```
vcf > set_fml_var fml_cc_autobbox false
```

SVA assume and `fvassume` constraints are not supported in the automatic black boxing flow. They will be ignored.

Also, this flow does not allow vacuity check and switching off the structural check. If vacuity check is enabled or structural check is disabled, VC Formal automatically disables the `fml_cc_autobbox` application available with a warning message, and proceeds with the run.

Vacuity Checks

By default, vacuity checking is done on abstracted model. The vacuity goals that are reported as vacuous, are genuinely *vacuous*. However, the ones reported as *inconclusive* most likely due to the nature of the model. It may or may not be non-vacuous when full model is considered, hence the *inconclusive* status. To check reachability of such an enable expression, use cover properties in the FPV application mode, or set `fml_vacuity_on` formal variable to true.

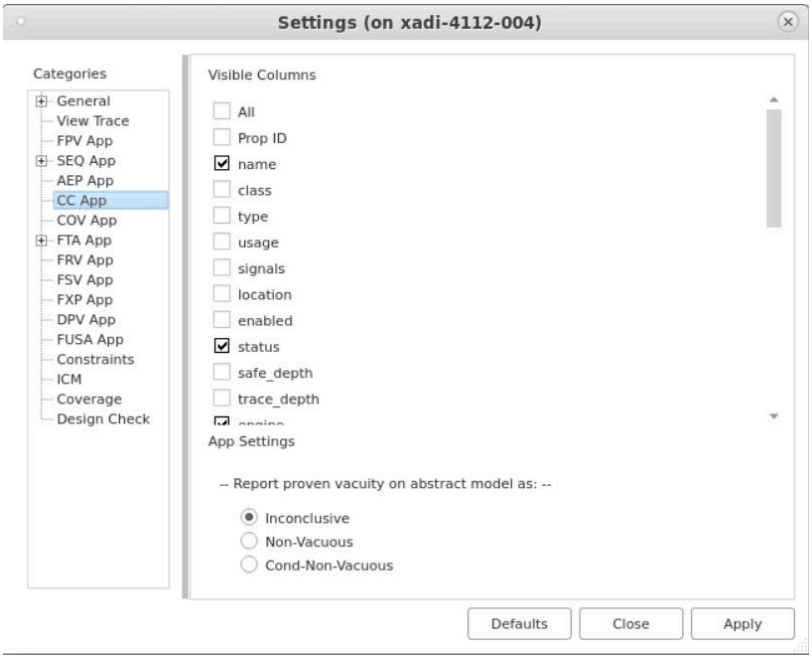
To control how vacuity results are reported on an abstracted design, use the `fml_cc_proven_vacuity_status` formal variable. The `fml_cc_proven_vacuity_status` application variable can take the following values:

```
vcf> set_fml_var fml_cc_proven_vacuity_status <Inconclusive, Non-vacuous,
Cond-Non-Vacuous>
```

- *Inconclusive*: Reports *Inconclusive* status for vacuity properties which are proven under the abstracted design. By default, the `fml_cc_proven_vacuity_status` variable is set to *Inconclusive*.
- *Non-vacuous*: Reports non-vacuous status for vacuity properties which are proven under the abstracted design.
- *Cond-Non-Vacuous*: Reports Cond-Non-vacuous status for vacuity properties which are proven under the abstracted design.

The `fml_cc_proven_vacuity_status`formal variable can also be controlled in the *Customized View Settings* window in GUI under the connectivity checking application.

Feedback



13.6.3 Snip Modeling in Connectivity Checks Application

The VC Formal CC application provides snip modeling to optimize the connectivity verification, and enable efficient checking of connections on SoC level designs.

The snip modeling support assumes that the verification requirement is to verify the path from source to destination, and any logic present in the fan-in of the source should not influence the connection status.

The snip based modeling for connectivity checks can be controlled with the `fml_snip_source_and_enable` formal variable.

The `fml_cc_snip_source_and_enable` formal variable can take the following values. By default, it is set to 1.

```
vcf > set_fml_var fml_cc_snip_source_and_enable <0, 1, 2>
```

- *0*: Old flow with *no* snipping
- 1: Snip source for each property (default value)
- *2*: Snip source and enable for each property

When set to 1, during each property verification, connectivity checks are performed after snipping the source of that particular property. This snipping is more like masking, which is only applicable at the property verification of that particular property, so it should not have

Feedback

any effect on the flow later. Note that the snip based modeling feature can affect both auto blackbox ON and OFF modes.

Note:

Use `set_fml_var fml_cc_snip_source_and_enable 0` to use the old flow with no snipping (backward compatibility switch).

13.6.4 Enable Dumping of Abstracted Model After autobbox

To enable saving of the abstracted model after black boxing, use the `fml_cc_autobbox_log [0-2]` formal variable before the `check_fv` command.

Example

```
set_fml_var fml_cc_autobbox_log 1
```

Where,

- 0: is the default value and no logs are saved.
- 1: Saves the cc_modules.txt and cc_bbox_modules.txt
- 2: Saves the cc_instances.txt, cc_bbox_instances.txt, cc_modules.txt and cc_bbox_modules.txt

The files are saved in the *vcst_rtdb/.internal/debug* location.

The following are example outputs of the logs saved:

cc_modules.txt

```
// Module Names
# modules: <count>
----------------------------
mod_1
mod_2
mod_3
mod_4
mod_5
mod_6
mod_7
```

cc_bbox_modules.txt

```
// Blackboxed module names
# Blackboxed modules: <count>
----------------------------
set_blackbox -designs { modBB_1 modBB_2
modBB_3 modBB_4 modBB_5 modBB_6 }

USER-DEFINED BBOX MODULES
<tab>modBB_2
<tab>modBB_4

UNRESOLVED MODULES
<tab>modBB_6

AUTOBBOX MODULES
<tab>modBB_1
<tab>modBB_3
<tab>modBB_5
```

Feedback

cc_instances.txt

```
// Instance Names
# Instances: <count>
----------------------------
mod_1
<Tab>inst_1
<Tab>inst_2
mod_2
<Tab>inst_3
mod_3
<Tab>inst_4
```

cc_bbox_instances.txt

```
// Blackboxed Instance Names
# Blackboxed Instances: <count>
----------------------------
set_blackbox -cells {inst_5 inst_6
 inst_7
inst_8 inst_9}

USER-DEFINED BBOX INSTANCES
<tab>modBB_2 <tab> inst_5
inst_6
<tab>modBB_4 <tab> inst_7

UNRESOLVED INSTANCES
<tab>modBB_6 <tab> inst_8

AUTOBBOX INSTANCES
<tab>modBB_6 <tab> inst_9
```

13.7 Enabling/Disabling Reset Verification on Sequential Elements

The `fml_cc_verify_rst_path` application variable allows you to enable/disable reset verification on sequential elements in connectivity checks.

By default, VC Formal does not perform reset verification on the path of sequential elements while performing connectivity checks for combinational connections in the `fml_cc_autobbox` flow. If you want to enable reset verification on the path of sequential elements, set the `fml_cc_verify_rst_path` application variable to true:

Syntax

```
vcf> set_fml_var fml_cc_verify_rst_path true
```

13.8 Standard Use Model

```
vcf -fmode CC/ vcf -f <tcl file>
set_fml_appmode CC //If you did not invoke VC formal in the CC app mode

read_file  / elaborate top //RTL read and elaboration
create_clock clock -period 100
create_reset arst -sense high
add_cc/ load_cc
sim_save_reset
check_fv -block
report_fv // To view the Connection result
```

Feedback

13.9 Proving Connectivity Properties

Verifying the connections specified in the CSV file or with the `add_cc`command is done using the `check_fv` command.

The `check_fv` command performs the following functions:

- Proves all connection properties that has their usage attribute set to assert.
- Assumes that the enable condition for each check is true.
- Proves connectivity checks.

If there are no connectivity checks and only other properties, you will only see the constraint properties.

13.9.1 Viewing Connectivity Checking Results

After `check_fv` is completed, the result is shown by the `report_fv` command.

A connectivity check passes and report is connected if the two nets always have the same value when the enable condition is true. It will be reported as follows:

```
[ 1] proven - in1B
```

The enable condition of the connectivity check is true and the result is valid.

A connectivity check passes and report is connected if the two nets always have the same value when the enable condition is true. It will be reported as follows:

```
[ 1] proven - in1B
```

A connectivity check fails if the two nets do not have the same value when the enable connection is true. It will be reported as follows:

```
[ 1] falsified - in1B
```

Another outcome is an undetermined result. The tool is not able to prove or falsify a connection property before reaching the time limit set by`app_var fml_max_time`. It is reported as follows:

```
[ 1] inclusive - in1B
```

It also issues the following warning:

```
[Warning] SPECIFIED_MAXTIME_EXCEEDED: Max effort limit of '3M' reached.
For unconverged properties set a higher effort limit and rerun.
```

Feedback

The following is an example output of the `report_fv` command:

```
vcf> report_fv -verbose

Summary Results
Connectivity Summary:
   ---------------------
   > Connection
     - # found        : 8
     - # proven       : 5
     - # falsified    : 3

Verbose Results
Verbose Connectivity List:
   -------------------------
   > Connection
     # Connection: 8
     > ID: [1] falsified
      - name              : C1
      - source_expr       : ARR_STD_LOGIC1
      - destination_expr  : ARR_STD_LOGIC
      - structural        : NO
      - enable_expr       : 1
      - location          : connectivity_check.csv:2

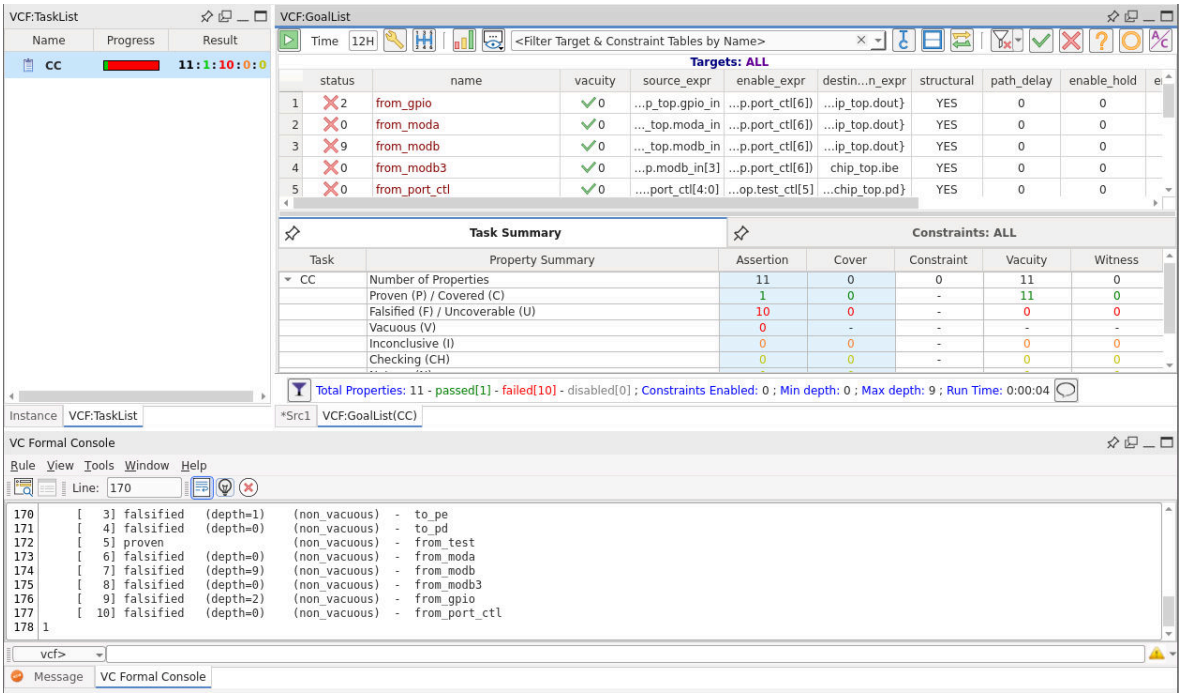
     > ID: [2] proven
      - name              : Primary_ip_instance_op
      - source_expr       : int12
      - destination_expr  : i_add_sint.add_out
      - structural        : YES
      - enable_expr       : (int12==32 && int22==0)
      - location          : connectivity_check.csv:4

     > ID: [3] proven
      - name              : Primary_ip_instance_ip
      - source_expr       : int1
      - destination_expr  : i_add_int.ip_1
      - structural        : YES
      - enable_expr       : (int12==32 && int22==0)
      - location          : connectivity_check.csv:5
```

The progress of the checks can also be viewed in the GUI as shown in #unique_439/unique_439_Connect_42_i1088946.

Feedback

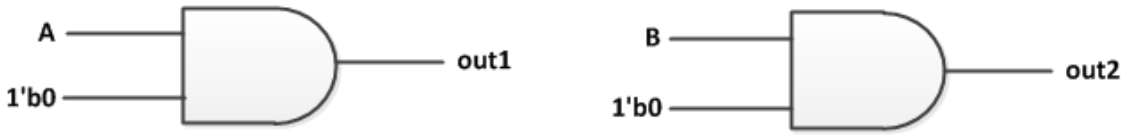
Figure 87: Viewing Connectivity Checks In GUI



13.10 Debugging Connectivity Checks

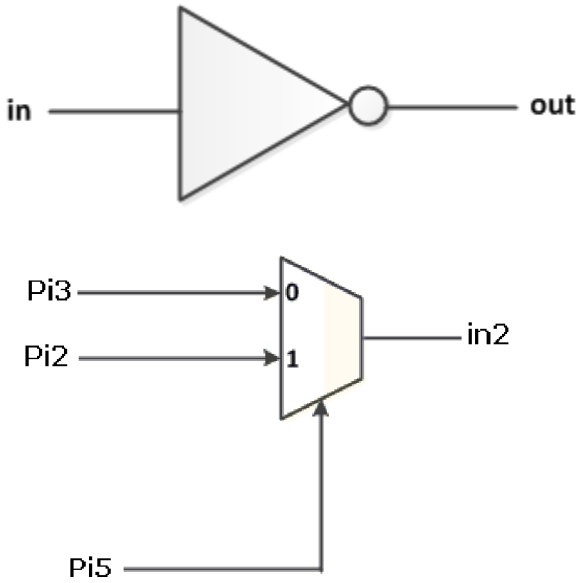
A connectivity check is proven when there is a structural directional connection between the source net and the destination net and the value of the source net and the destination net are always the same when the enable expression, if any, is true. This means that a connectivity check failure can be caused by either a missing physical connection or different values on the source and destination nets. For example, the nets out1 and out2 below are not connected even though they always have the same value (0) but there is no physical path between them. So, the following check fails:

```
add_cc -src out1 -dest out2 -enable 1
```



Similarly, a structural connection does not mean that two nets are connected in the sense that they always have the same value. For example, the nets in and out in the following diagram are structurally connected through the inverter. However, the check `add_cc -src in -dest out -enable 1` fails because they never have the same value.

Feedback



If the `report_fv` lists any of the connections as unconnected, the next step is to debug the cause. If you change the enable condition of the in2Abug check so that Pi2 can not be selected, the check fails since Pi5 always selects Pi3 as the output from the mux.

```
add_cc -src Pi2 -dest top.myCoreA.In2 -enable ~Pi5 -name in2Abug
```

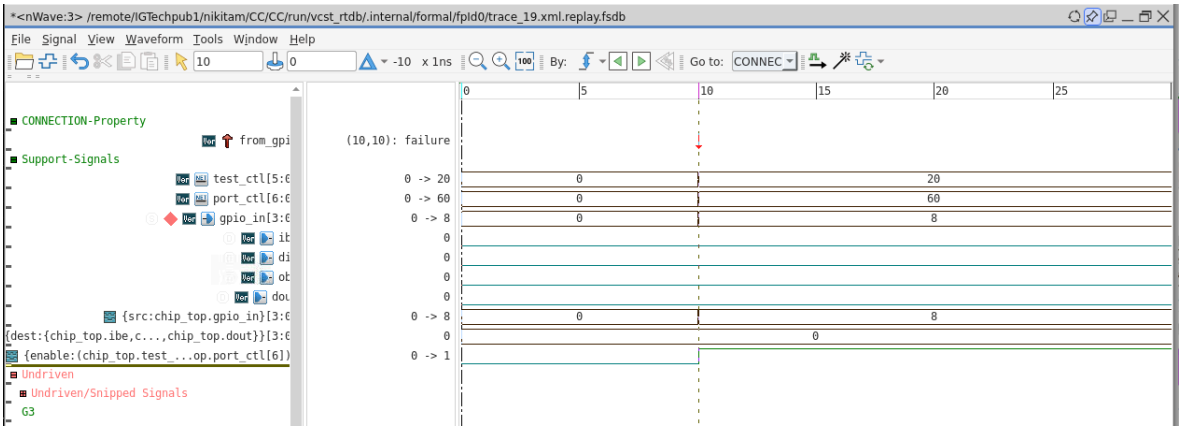
The `report_fv -list` command shows which check failed.

```
Summary Results
   Connectivity Summary:
   ---------------------
   > Connection
     - # proven: 1
     - # falsified : 1
List Results
   Connectivity List:
   ------------------
   > Connection
     # Connection: 1
     [  0] falsified (enabled)      -  in2Abug
```

The most common method to debug a failing connectivity property is to analyze a counter example waveform. This may be obtained for a failing connectivity check by starting the GUI and selecting *Debug in Verdi*. #unique_440/unique_440_Connect_42_i1053746 shows the waveform for the failing connectivity check.

Feedback

Figure 88: Waveform for a Failing Connectivity Check



However, since connectivity checks are usually not temporal properties, waveforms are not the most optimal way to view connectivity information.

There are several reasons why a connectivity check fails. The two major ones are:

- No connectivity exists between the two points because:
  - The specification may be incorrect.
  - The RTL implementation may be broken.
- There is a path, but it is broken.
  - Incorrect mux select or AND gate input tied low.
  - Incorrect constraints.

By default, VC CC replays only the CC trace in the counter example waveform. You can select the option to show the reset trace in the waveform. You can also set up the tool so that the counter example shows the composite reset and formal trace. By default, the composite waveform generation is set to false. You can see a marker on the waveform indicating the end of the reset sequence, and the complete trace including the reset phase.

To enable this capability, use the following commands:

```
set_fml_var fml_cc_composite_trace true
sim_config -rst_wave OFF
```

VC Formal provides several commands to assist and automate the debugging of connectivity check failures. The available commands are

- `analyze_cc_rootcause`
- `report_cc_rootcause`

Feedback

- `list_path`
- `view_schematic`

The first step in debugging a failing connectivity check is to run the automatic root cause analysis on one or more failing checks. If this does not identify the cause of the failure or if no structural connection exists, proceed with debugging using the `list_path` and `view_schematic` commands.

Watch this video to understand how to debug failing connectivity properties using the VC Formal CC application.

13.10.1 Determining the Root-cause for Failing Connectivity Checks

For some failing cases, the tool can determine the reason for a failing connectivity check thus making debugging much easier. Since this analysis is done automatically, it should be tried first before trying to find a path using the `list_path` command. It employs different algorithms, called tactics, to identify possible reasons for the connection check failure. The currently available tactics are:

- Structural Tactic
- Blackbox Tactic
- Blackbox Backward Tactic
- Blackbox Forward Tactic
- Invert Tactic
- Multi-driven Tactic

Structural Tactic

The structural tactic checks if the path is structurally connected between the source and destination. This tactics works when the structural checks are on, and when the connectivity is structurally disconnected, then the command reports the root cause. The reason can be one of the following:

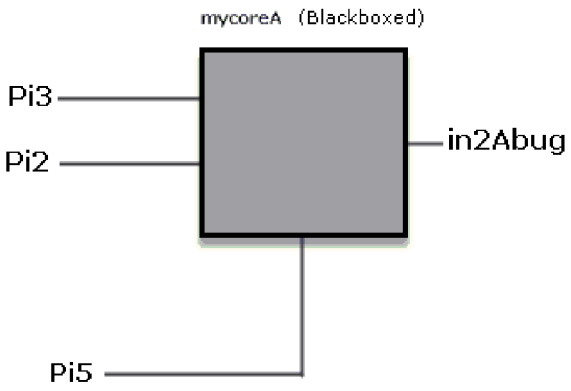
- There is no structural path
- The bit-width is different
- The delay on the path is more than the `path_delay`.

Feedback

By default, the structural tactic is used.

Black Box Tactic

If the path being checked is not connected because there is a black-boxed module in the path, this is reported by the `analyze_cc_rootcause` command. In this case, the black-boxed module is instantiated at the top level but if it is instantiated several levels of hierarchy below, it is not so easy to find.



In the example, the module myCoreA is black-boxed and connectivity is checked between Pi3 and in2Abug:

```
add_cc -src Pi3 -dest in2Abug -enable 1'b1 -name in2Abug
```

This check fails:

```
   Connectivity List:
   ------------------
   > Connection
     # Connection: 1
     [  0] falsified (enabled)      - in2Abug
```

Run the `analyze_cc_rootcause` command to debug the failure:

```
vcf> analyze_cc_rootcause
Doing tactic BlackBoxCause
Found a blackbox:mycoreA  between source:Pi3 and dest:in2Abug
   source reaches:mycoreA.Pi3
   destination reaches:mycoreA.in2Abug
Doing tactic DisconnectCause
CONNECTIVITY CHECKS ROOT CAUSE SUMMARY:
ROOTCAUSE_BLACKBOX 1 times.
```

Since the only connection between Pi1 and Po1 goes through the black-boxed module, this is the cause of the connectivity check failure.

Feedback

Blackbox Backward

If the path being checked is not connected because there is a black-boxed module in the path, this is reported by the `analyze_cc_rootcause` command. In this case, the black-boxed module is traced from the destination and traverse backwards to find any black-box before the source.

Blackbox Forward

If the path being checked is not connected because there is a black-boxed module in the path, this is reported by the `analyze_cc_rootcause` command. In this case, the black-boxed module is traced from the source and traverse forward to find any black-box before the destination.

Invert Tactic

The inverter tactic checks if there is an inverter on the path between source and destination. If an inverter is found then the output is reported as A to !B.

Broken Path Tactic

If there is a structural connection between the source and destination but the connectivity check fails, the `analyze_cc_rootcause` command determines what condition or conditions are causing the disconnect. An example of reasons is mux select having the wrong value or an AND gate having a constant 0 input.

```
=== Starting root cause analysis for in2Abug ===
Source(s) : Pi2
Destination(s) : myCoreA.In2
===============
top.v:11  at time 50 : Mux I_MUX_in_net is breaking the path because
 selector value is 1
```

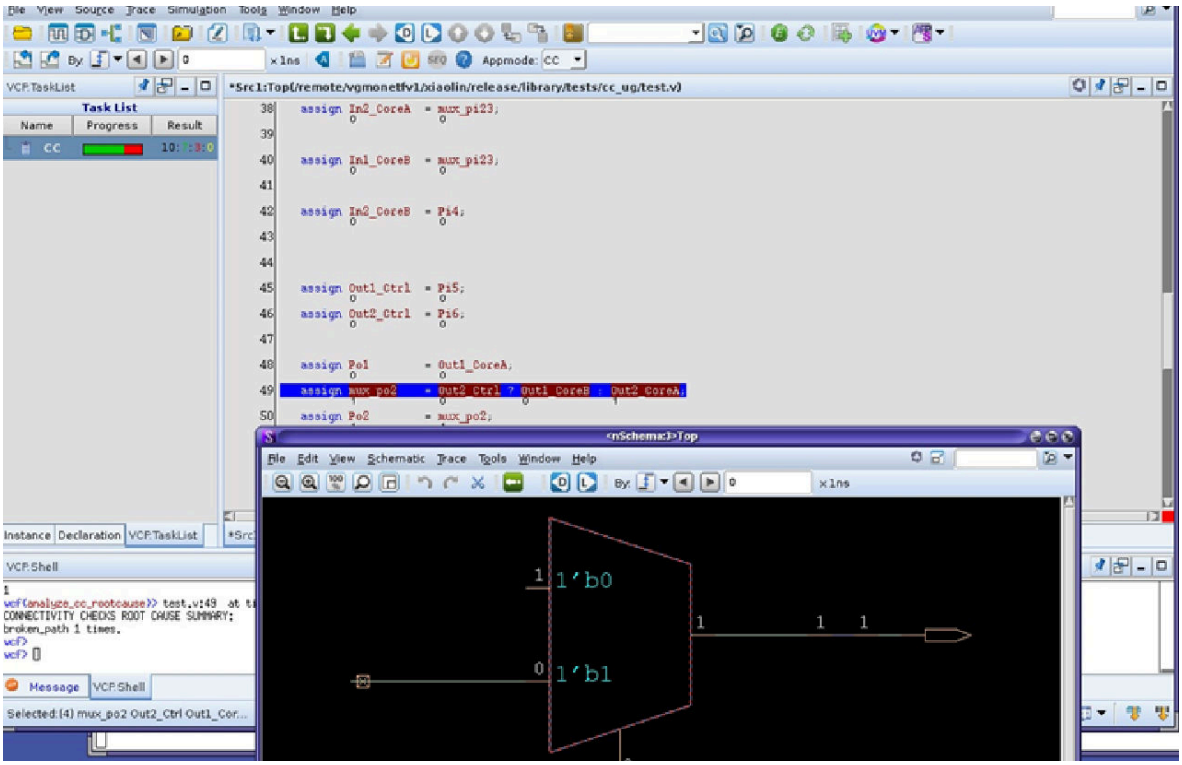
To find the reason for why the mux select has an incorrect value, start the GUI using the `start_gui` command. Select the failing property check and bring up the schematic by clicking on New Schematic Path.

Select the path to the net or cell which is breaking the path displayed by the `analyze_cc_rootcause` command.

Feedback

In the schematic view, use the find field as shown in Figure 89. Paste the cell name in the find field and press return:

Figure 89: Debug from Schematic and Source Code with Values Annotated in Both



Multiple Driver Tactic

The multiple driver tactic checks if there are multiple drivers on a net in the path between source and destination. If they drive different values, the result is unknown, therefore, the value on the source net can not propagate to the destination.

Example

```
=== Starting root cause analysis for pad3__to__out1 ===
Source(s) : pad3
Destination(s) : out1
===============
Multi driven net out1 has conlicting drivers: At time 50 pad1 drives 0
 and dummy_pad1 drives 1
```

The multiple driver tactic is not run by default. To run it, use the -tactic option:

```
analyze_cc_rootcause -tactic multi_driven
```

Feedback

13.10.2 Interpreting list_path Results

The list_path command checks for a path between the source and the destination and prints it out on the screen, if it exists. Note that the list_path command and connectivity checks are directional. Each line has three or four fields.

- The first field identifies the location in the connection. The possible values are:
  - OPERATOR
  - OPERATOR_BIT
  - PORT-IN
  - PORT_BIT-IN
  - PORT-OUT
  - PORT_BIT-OUT
  - PORT-BIDIR
  - PORT_BIT-BIDIR
  - PIN-IN
  - PIN_BIT-IN
  - PIN-OUT
  - PIN_BIT-OUT
  - PIN-BIDIR
  - PIN_BIT-BIDIR
- The second field is the full hierarchical name of the node.
- If the node is an OPERATOR, then the third field will be its type. Ports and pins do not have this field.
- The fourth field is the instance name or Hierarchical Scope (HS). If the node is a pin and it is of the top level then it prints (Root Instance).

13.10.3 Determining the Root-cause for a Subset of Checks

There may be many failing connectivity checks and running `analyze_cc_rootcause` on all of them returns too much information. To run `analyze_cc_rootcause` on a subset of connectivity check failures, specify a list of properties as an argument to the `analyze_cc_rootcause` command.

Note:

- The `analyze_cc_rootcause` does not work on a check which has encrypted instances in the path reports a error message.
- In the schematics, the encrypted instances on the path is grayed out and you cannot view the details of the instances.
- The waveform viewer does not show any encrypted signal. An exception to this is the source and destination inside an encrypted module. These values are visible to the user.

13.10.4 Getting a Reports of the Root-cause for Failing Connectivity Checks

The `report_cc_rootcause` command prints a report of results from a previously run `analyze_cc_rootcause` command. If the checks, constraints and design have not changed and the `analyze_cc_rootcause` command is run again, it will only print a summary of the result. To get the full report, use the `report_cc_rootcause` command. You can still get a summary of all the root cause analysis that has been run even if the last one ran a sub-set of the failing connections, using the `report_cc_rootcause` command.

Example

```
vcf> report_cc_rootcause
CONNECTIVITY CHECKS ROOT CAUSE:
Found RC for CC pi2poA:
BlackBox: src Pi1 dest Po1
box name: myCoreA source reached box at: myCoreA.In1 destination reached
 box at: myCoreA.Out1
Found RC for CC pi2poB:
Structural disconnect: src Pi4 dest Po3
CONNECTIVITY CHECKS ROOT CAUSE SUMMARY:
ROOTCAUSE_DISCONNECT 1 times.
ROOTCAUSE_BLACKBOX 1 times.
```

13.10.5 Debugging Connectivity Checks in GUI

If `analyze_cc_rootcause` does not find the reason for the connection being not connected, to find out if there is a structural connection between the start and end point, use the `list_path` command. It does not use constraints so it lists the path even if the connectivity check failed because of an incorrect enable signal or a set_constant on an input.

Syntax

Feedback

```
vcf> list_path -help
Usage: list_path    # Print a path between source to destination if it
 exists
        [-src string]           (the path source)
        [-dest string]          (the path destination)
        [-cc_prop string]       (the connectivity check property)
        [-cc_ignore_structural]
                                (Ignore the structural check done for CC
 and do an independent traversal)
        [-col]                  (return a collection of objects)
        [-schematic]            (show the path in schematic)
        [-word]                 (use word level traversal)
        [-no_bbox_stop]         (check for destination beyond blackbox)
```

Example

- For the failing connection check already discussed above, `list_path` returns the following path:

```
vcf> list_path -src Pi2 -dest top.myCoreA.In2
list_path: path from source Pi2 to destination top.myCoreA.In2 of
 length 4:
PORT-IN Pi2 (HS = top)
OPERATOR I_MUX_in_net MUX (HS = top)
PIN-IN myCoreA.In2 (HS = top)
PORT-IN myCoreA.In2 (HS = CoreA)
```

- This shows that there is a path between Pi2 and In2 of CoreA and it goes through a mux. In some cases a path does not exist, for example, between Pi2 and Po1:

```
vcf> list_path -src Pi2 -dest Po1
list_path: Path not found
```

- If there is no path, the command checks the reverse path to see if there is a path, since the check may have been specified backwards.

```
vcf> list_path -src myCoreA.In2 -dest Pi2
list_path: Path not found
 BUT A REVERSE PATH EXISTS!
```

- You can view the schematic of the path using the `-schematic` option in the `list_path` command.

```
vcf> list_path -src myCoreA.In2 -dest Pi2 -schematic
```

- If the connection path has black box modules or cells in the middle, the `list_path` command reports the *path not found* message.

```
vcf>list_path -src test -dest dout
list_path: Path not found
Found black boxes during search, you may want to add '-no_bbox_stop' switch to find plausible path between source and destination
0
```

- Specify the `-no_bbox_stop` option in the `list_path` command, else the paths are listed for the black box modules or cells.

Feedback

```
vcf>list_path -src test -dest dout -no_bbox_stop
list_path: path from source test to destination dout of length 3:
PIN_BIT-IN test_mux.mux_in0[0] (Blackbox)
PIN_BIT-OUT test_mux.mux_out[3:0] (Blackbox)
OPERATOR_BIT dout INDEX
0 internal objects
3
```

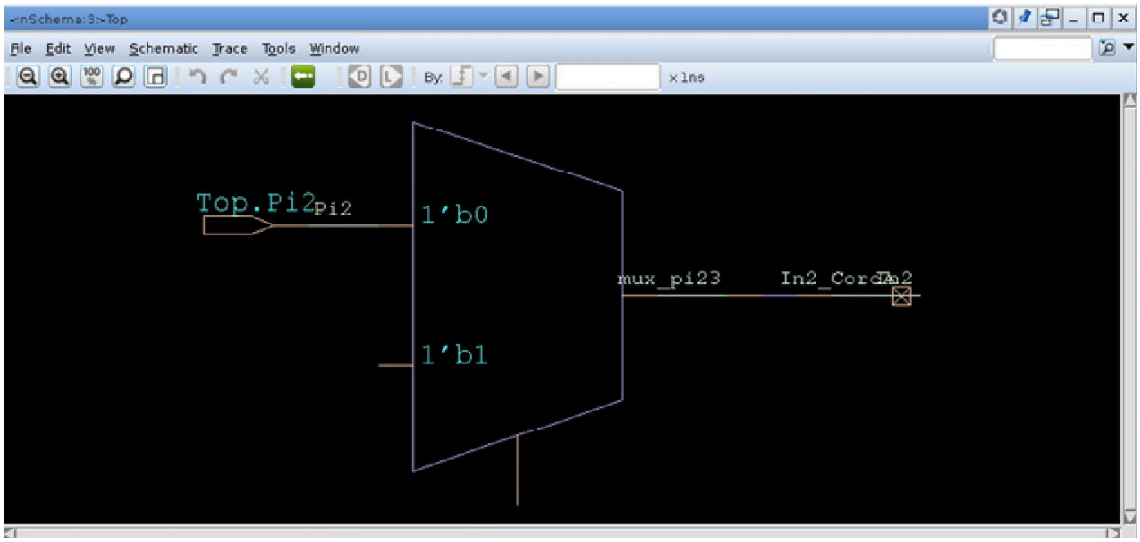
Note:

The `list_path` command might report a path, even when `add_cc` has reported that the same path is structurally unconnected due to latency. The `list_path` command does not consider latency.

- Another way of viewing the path is to generate a schematic using the `view_schematic` command as shown in Figure 90:

```
vcf> view_schematic -src Pi2 -dest top.myCoreA.In2
```

Figure 90: Schematic View



The block diagram of the example design shows that the mux select should be 0 to select the Pi2 input, hence ~Pi5 is used to enable the connectivity check.

vcf> `add_cc -src Pi2 -dest top.myCoreA.In2 -enable ~Pi5 -name in2Abug`

The check needs to be changed as follows due to the inverter in the path:

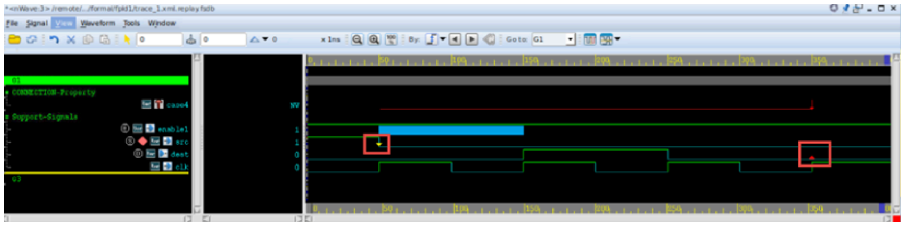
vcf> `add_cc -src Pi2 -dest top.myCoreA.In2 -enable Pi5 -name in2AnoBug`

The hardest case to debug is if there is no path. The schematic only shows a symbol for the starting point, end point and enable expression since it cannot show something that does not exist. First, verify that the connectivity check is correct against the specification. If it is, use the schematic view to try to determine where the connection was broken. Double click on the end point symbol to show what it is connected to. Continue exploring what is

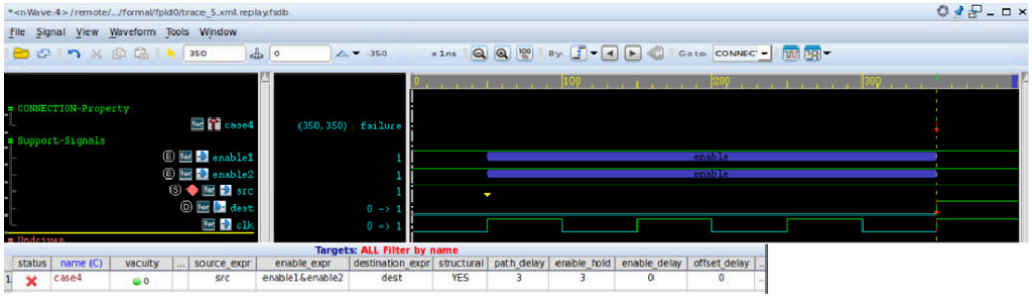
Feedback

driving it as well as exploring the fanout of the starting point. Check if they go to the same module and investigate if there should be a connection in this module.

- The nWave window displays the source sampling point and destination sampling point using the ▼ and the ▲ icons respectively as shown in the following figure:



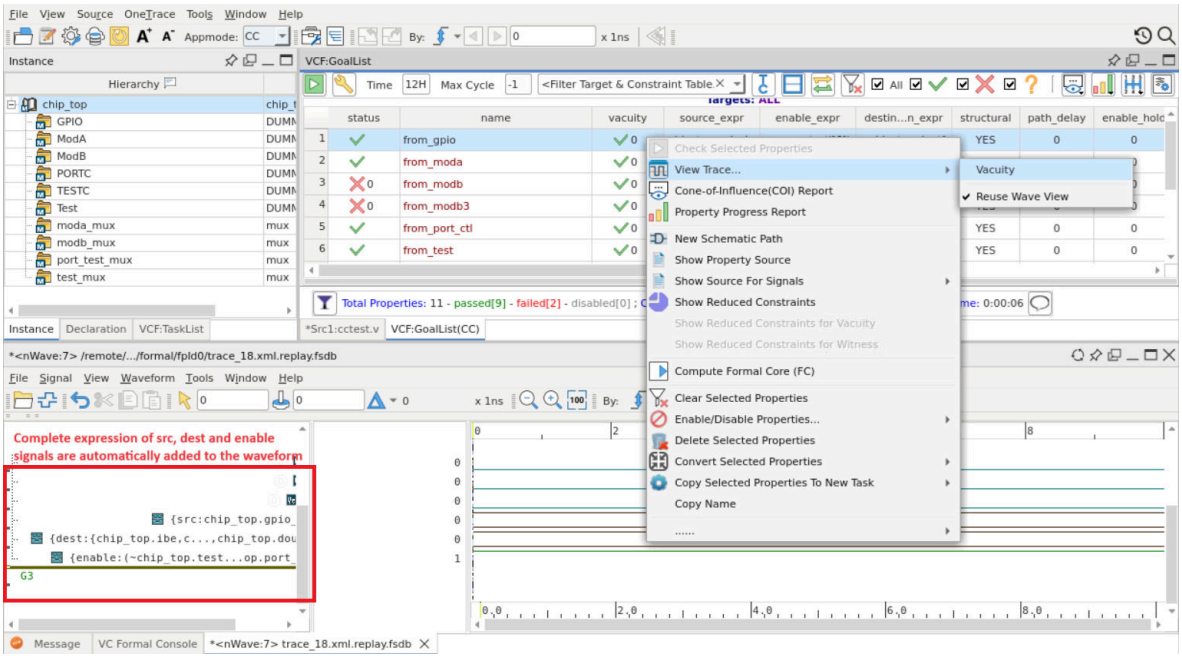
- The enable hold, enable delay and path delay timestamps are highlighted in the waveform similar to the navigator as shown in the following figure



Feedback

:

- In Connectivity Checking app mode, when src or dest or both signals are concatenated expressions, then the complete expression will be automatically shown in the waveform, as shown in the following figure:



13.11 Extracting Connections

Connectivity extraction enables you to generate connectivity properties for connections between source and destination pins, ports, nets or instances in a design. The source or destination can be a single signal, multiple signals, or a collection of signals. The extracted connectivity properties represent the actual connections present in the RTL. They must be reviewed to ensure they are consistent with the specification.

Watch this video to understand how to extract connectivity properties from a design using VC Formal CC. The video highlights the importance of verifying the correctness of the extracted connections with respect to the specification.

13.11.1 The generate_cc Command

The `generate_cc` command enables you to extract connections between a source and destination ports, pins, net or instance. The `generate_cc` command checks if

Feedback

a connection exists between a source and destination pair, and finds (extract) the corresponding enable expression. The enable expression consists of primary inputs and/or the first register in the expression, and optionally signals specified in the `-control` option of the `generate_cc` command.

Syntax

```
vcf> generate_cc
 [-src <collection-or-signal-name-or-inst-name>]
 [-dest <collection-or-signal-name-or-inst-name>]
 [-control <collection-or-signal-name-or-inst-name>]
 [-run]
 [-preserve]
 [-clearall]
 [-file <path>]
 [-limit <int>]
 [-verbose]
```

where,

- `-src`: Use this option to specify the source nets, ports or instance of a design from where the connectivity starts.
- `-dest`: Use this option to specify the destination nets, ports or instance of a design to where the connectivity ends.
- `-run`: Use this option to generate a set of connectivity properties for the specified source and destination ports.
- `-file`: By default, it saves the property in the property database. If the `-file <file name>` switch is provided, then it writes the connections in the *.tcl* file format. Or you can also save the properties in *.csv* format by using the `report_fv -formatCC csv` command.
- `-control`: Use this option to specify a signal or instance boundary for an enable expression.
- `-preserve`: Use this option to reuse a set of source and destinations for the next `generate_cc -run`.
- `-clearall`: Use this option to clear the list of control signals specified by the `-control`option along with source and destination port details.
- `-limit`: Use this option to limit the number of connections extracted to N if multiple paths exist between source and destination (default 1).

Feedback

- `-verbose`: Use this option to get a verbose report.

  Note:

  Set the following formal variable to true to extract connections only between out/inout ports of source instance to in/inout ports of destination instance:
  `set_fml_var fml_cc_instance_src_out_to_dest_in true`

13.11.2 Examples

- To extract connections for a module in a design.

```
generate_cc -src Pi1 -dest Po1
generate_cc -src Pi2 -dest myCoreA.In2
generate_cc -run
```

  Add all source and destination pairs for which you want to extract connections. No connectivity extraction is run until the `-run` switch is used.

- The set of commands:

```
generate_cc -src Pi1 -dest Po1
generate_cc -src Pi2 -dest myCoreA.In2
generate_cc -run"
```

  and

```
generate_cc -src Pi1 -dest Po1 -run
generate_cc -src Pi2 -dest myCoreA.In -run
```

  are functionally equivalent, but the first group of commands is executed faster because the extraction is run only once.

- Connectivity properties can be extracted even if the source and destination ports have different widths.

  The following example shows how to extract connection from 4 bit input *Pi7* to one bit port *In3*.

```
generate_cc -src Pi7 -dest myCoreA.In3 -run
[Info] GENERATE_CC_COMMAND: For command : generate_cc -src {Pi7} -dest
 {myCoreA.In3}
[Warning] GENERATE_CC_WIDTH_MISMATCH: The source (Pi7: width 4) and
 distination (myCoreA.In3: width 1) will be ignored
 Proceeding to bit level extraction
. . . .
4
```

  The `generate_cc` command returns the number of connections extracted as 4.

```
add_cc -src Pi7[0] -dest myCoreA.In3 -enable "{(top.Pi8 == 2'b00)}"
add_cc -src Pi7[1] -dest myCoreA.In3 -enable "{(top.Pi8 == 2'b01)}"
```

Feedback

```
add_cc -src Pi7[2] -dest myCoreA.In3 -enable "{(top.Pi8 == 2'b10)}"
add_cc -src Pi7[3] -dest myCoreA.In3 -enable "{(top.Pi8 == 2'b11)}"
```

- To extract multiple paths for each connection by using the `-limit` switch. Multiple paths are connections between the same source destination pair with different enable expressions.

  For example, in this circuit, 3 paths exist from the net padN to the output padS.

  ```
  generate_cc -src padN -dest padS
  generate_cc -run -limit 5
  ```

  The following list of connectivity properties appears.

  ```
  add_cc -src padN -dest padS -enable  {(selS==2'b01) && oeS && !oeN}
  add_cc -src padN -dest padS -enable  {(selS==2'b10) && oeS && !oeN &&
   oeW && (selW==2'b01)}
  add_cc -src padN -dest padS -enable  {(selS==2'b11) && oeS && !oeN &&
   oeE && (selE==2'b01)}
  ```

13.11.3 Connectivity Extraction and Constraints

In general, constraints are not honored when running connectivity extraction. The SVA assume, `fvassume` and `set_constant` Tcl constraints are ignored when extracting connections. However, environment constraints (`fvassume -env`) are honored.

Example

For example, two connections through a 2-1 mux controlled by the select signal SEL:

```
*Connection from Pi2 -> In2 when SEL == 0
*Connection from Pi3 -> In2 when SEL == 1
```

- If one of the following constraints is used:

  ```
  set_constant -value 1 SEL
  fvassume -expr {SEL == 1}
  ```

  And the following `generate_cc` commands are run:

  ```
  generate_cc -src Pi2 -dest myCoreA.In2 -run
  generate_cc -src Pi3 -dest myCoreA.In2 -run
  ```

  Both the connections through the mux are extracted:

  ```
  add_cc -src Pi2 -dest myCoreA.In2 -enable "{(SEL == 0)}
  add_cc -src Pi3 -dest myCoreA.In2 -enable "{(SEL == 1)}
  ```

Feedback

- If an environment constraint is used instead, for example,

```
fvassume -env -expr {SEL == 1}
```

  Only the second connection is extracted:

```
add_cc -src Pi3 -dest myCoreA.In2 -enable "{(SEL == 1)}
```

13.11.4 Controlling Enable Expression Signals

By default, the enable condition for an extracted connection is expressed in terms of:

- Primary inputs
- The first register encountered in the COI of the enable signal

This is the most common case because enable signals must be controllable for an extracted connection to be valid. It is possible to add other signals to be used in enable expression. The enable expression consists of specified signals, primary inputs, and registers as required. For example,

- Outputs of a module can be used instead of internal registers using the `-control <instance_name>` option in the `generate_cc` command.
- Mux control signals can also be used as enable expression.
- Only word level signals may be used, no bit-selects.
- One or many `generate_cc -control <sig1>` commands may be run. They are applied to all subsequent `generate_cc -run` commands, until a `generate_cc -clearall`command is issued. The multiple nets or signals can be specified, but only the related ones are used.

  Note:

  Specifying internal nets or instances as control expression for extracting connections may lead to incorrect behavior as it may not be possible set the internal nets or instances to the required value for the connection.

Watch this video to understand how to run and analyze enable expression coverage for connectivity properties using VC Formal. The demo highlights how to improve coverage by including enable expression toggle coverage for signals such as instance ports, flip-flops, and declared nets in the enable path. Proven properties are essential for contributing to coverage.

13.11.5 Limitations

- Connections with flip-flops in the path are not extracted.

Feedback

13.12 Performing Coverage Analysis for Connectivity Checks

VC Formal enables you to find the part of the design that is covered by the formal connectivity checks, identify the paths where connectivity checks are missing, and check if the verification is complete.

For all proven connectivity properties, VC Formal saves the toggle coverage results of the source, destination, module ports, sequential elements (includes latches), and nets in the structural paths. You can view the overall coverage score in Verdi Coverage GUI and identify points in the design, which are not covered by connectivity coverage. You can review and waive off some of the coverage targets (if those are not interesting for Connectivity), or add additional connectivity checks to cover them.

You can merge the connectivity coverage results with the coverage DB from the simulation connectivity verification. This helps in improving the overall coverage score, which involves measuring toggle coverage of ports, nets, and sequential elements.

You can perform toggle connectivity coverage:

- Only for the source and destination in the path
- For all the nets in the path of a proven connectivity property

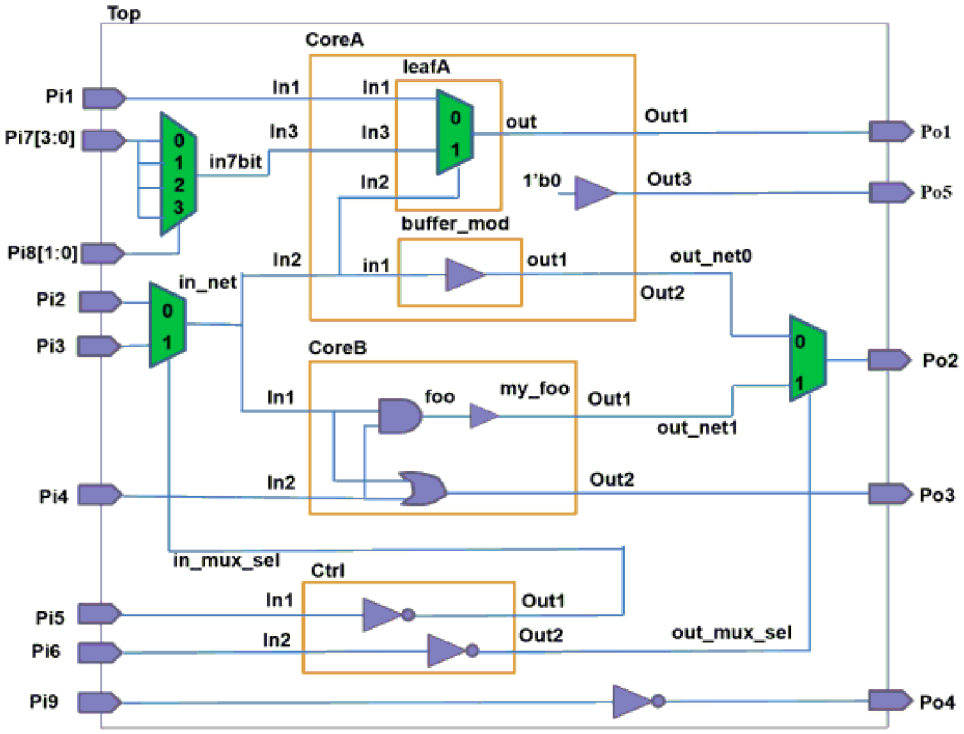
Watch this video to understand how to set up, run, and analyze toggle coverage for connectivity properties using VC Formal.

13.12.1 Example

Consider the design shown in Figure 91.

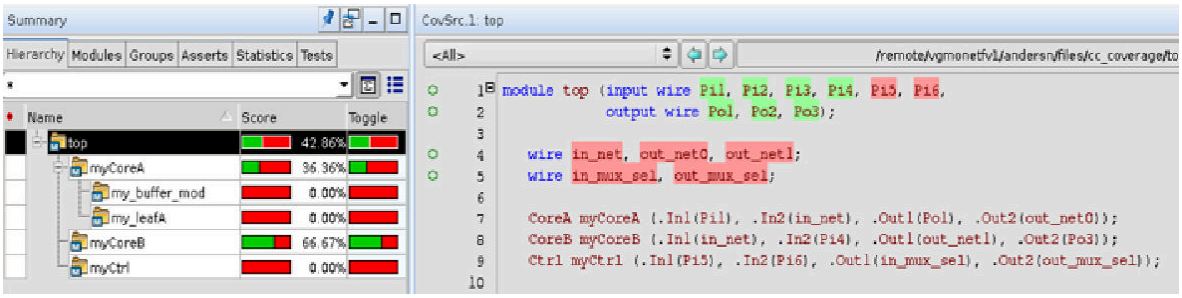
Feedback

Figure 91: Example Design




```
The following properties are defined between the primary inputs/outputs
 and the ports on the two top level modules - CoreA and CoreB:
add_cc -src Pi1 -dest myCoreA.In1 -enable 1 -name Pi1_2_CoreAIn1
add_cc -src Pi2 -dest myCoreA.In2 -enable Pi5 -name Pi2_2_CoreAIn2
add_cc -src Pi2 -dest myCoreB.In1 -enable Pi5 -name Pi2_2_CoreBIn1
add_cc -src Pi3 -dest myCoreA.In2 -enable ~Pi5 -name Pi3_2_CoreAIn2
add_cc -src Pi3 -dest myCoreB.In1 -enable ~Pi5 -name Pi3_2_CoreBIn1
add_cc -src Pi4 -dest myCoreB.In2 -enable 1 -name Pi4_2_CoreBIn2
add_cc -src myCoreA.Out1 -dest Po1 -enable 1 -name CoreAOut1_2_Po1
add_cc -src myCoreA.Out2 -dest Po2 -enable Pi6 -name CoreAOut2_2_Po2
add_cc -src myCoreB.Out1 -dest Po2 -enable ~Pi6 -name CoreBOut1_2_Po2
add_cc -src myCoreB.Out2 -dest Po3 -enable 1 -name CoreBOut2_2_Po3
```

When you perform connectivity checks for these properties, all the properties are proven. You can perform toggle coverage analysis for the source and destination points, and save the results in the coverage database, which can be viewed in Verdi coverage GUI as follows:

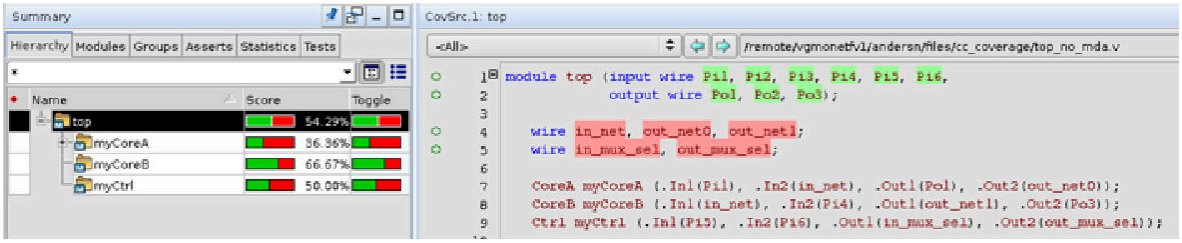


The toggle coverage score for the module top is 77.78.

To complete the coverage, you must add the following properties for Pi5 and Pi6.

```
add_cc -src Pi5 -dest myCtrl.In1 -enable 1 -name Pi5ctrl
add_cc -src Pi6 -dest myCtrl.In2 -enable 1 -name Pi6ctrl
```

After you add these properties, if you perform toggle coverage analysis, the Verdi Coverage GUI reports 100% coverage for the modules coreA and coreB.



13.12.2 Supported Connectivity Coverage

VC Formal supports toggle coverage for the following types of connections.

Inverted Connections

VC CC can generate toggle coverage for inverted connections.

Example

```
add_cc -src Pi9 -dest ~Po4 -enable 1 -name Pi9_~Po4
[  128] proven -  Pi9_~Po4
```

The ~ is not part of the destination port, it indicates that the destination is always the inverted value of the destination. There is a path from source to destination and the value is supposed to be inverted as specified by the connectivity property, therefore, it is included in the coverage analysis.

The following toggle coverage goals are covered:

```
[95] covered      (depth=0)          -  top.toggle_Pi9_24 [Pi9 0->1]
[96] covered      (depth=0)          -  top.toggle_Pi9_25 [Pi9 1->0]
```

Feedback

```
[103] covered    (depth=0)          -  top.toggle_Po4_32 [Po4 0->1]
[104] covered    (depth=0)          -  top.toggle_Po4_33 [Po4 1->0]
```

Connections from a Constant

The toggle coverage is generated on the destination port for a check from a constant, but the toggle coverage goals fails because of the constant.

```
add_cc -src 1'b0 -dest Po5 -enable 1 -name 0_2_Po5

[  131] proven                            -  0_2_Po5
```

These coverage goals on the destination fails because it is a constant:

```
[103] uncoverable                    -  top.toggle_Po5_34 [Po5 0->1]
[104] uncoverable                    -  top.toggle_Po5_35 [Po5 1->0]
```

Bit Selects of Ports

Toggle coverage on ports is generated at the bit level. If only one bit of the port has a connectivity check, toggle coverage is generated on the port that is not covered.

Sequential Elements

Toggle coverage is generated on the sequential elements (including latches) in the path of proven connectivity properties. Toggle coverage is also generated for properties with latency.

13.12.3 Commands for Computing Connectivity Coverage

To enable the formal connectivity coverage analysis, set the following application variable to true.

```
set_app_var fml_cc_coverage_analyzer true
```

You can use the following commands to compute connectivity coverage of proven connections in the Tcl flow.

- `compute_cc_cov`
- `save_cc_cov_results`

The compute_cc_cov Command

To generate the input, output, and flip-flop toggle coverage for proven connectivity properties, use the `compute_cc_cov` command. Using this command, you can generate toggle coverage only for source and destination of a property, or for all the nets in the path of the proven property. You can also directly mark the proven properties as covered in the coverage database.

Feedback

```
vcf> compute_cc_cov [-par_task parTaskName] [-endpoint] [-net]
 [-structural] [-property properties] [-block] [-run_finish finishCmds]
 [-enable]
```

Where,

- `[-par_task parTaskName]`: Specify the name of the parent CC task. If not specified, `compute_cc_cov` generates toggle coverage for the current task.
- `[-endpoint]`: Specify this option to generate toggle coverage for source and destination of the CC property.
- `[-net]`: Specify this option to generate toggle coverage for nets in the path of proven connectivity checks.
- `[-property properties]:` Specify the list of enabled proven CC properties you want to generate toggle coverage for.
- `[-structural]`: Specify this option to map the proven results from CC checks and mark the corresponding toggle coverage goals directly as covered in coverage database without performing coverage analysis.
- `[-block]`: Specify this option to ensure that the command has completed execution before accepting the next Tcl command at the shell prompt. Default is to accept the next Tcl command as soon as the command is started. This means that each command will return to the shell prompt soon after the analysis session is initiated. This allows for results analysis and debug to proceed while the check is running. In certain cases (script scenarios for example), blocking behavior is desirable; for these scenarios, use the -block switch.
- `[-run_finish finishCmds]`: Optional argument to provide a set of Tcl commands to execute when the command finishes. This switch provides a set of Tcl commands to run when the command finishes. If a number of commands and arguments are needed, then semi-colon delimiter must be used between the Tcl commands, and the command sequence must be enclosed in braces. Valid in blocking mode only.
- `[-enable]`: Specify this option to compute the toggle coverage of enable expression, and path between enable expression to the muxing point of a connectivity checking property.

The save_cc_cov_results Command

To save the results of the `compute_cc_cov` command into the coverage database, use the `save_cc_cov_results` commands.

```
vcf> save_cc_cov_results [-covdb_name ] [-exclude_disabled] [-elfile]
```

Where,

- `[-covdb_name]`: Use this option to specify the name of the coverage database to be saved. If you do not specify this option, by default, the results are saved to *cc_tgl_cov.vdb*.
- `[-exclude_disabled]`: Specify this option when you want to save the disabled/unrelated goals in an exclusion file. You can also save the exclusion file in the strict mode. This means that once the goals are considered covered, they are marked covered irrespective of the coverage status in the subsequent runs. To save the exclusion file in the strict mode, you must set the `fml_cov_enable_strict_elfile_mode` formal variable to true. By default, this variable is set to false.
- `[-elfile]`: Specify this option to save the unreachable toggle goals in exclusion file. If you have specified the option to save the elfile, the elfile is saved with the same name as covdb.

13.12.4 Computing Connectivity Coverage in Tcl Flow

The following is an example Tcl script to compute the connectivity coverage:

Note:

You must specify the `-cov tgl` option in the `read_file` or `elaborate` commands.

Note:

By default, VCS does not instrument coverage goals for library components, that is, any files specified by -v or -y option in design file lie will not contain any coverage goals. To ensure coverage goals are created for all the design elements, add `-cm_libs vy` option to `-vcs` command. For example,

```
read_file -cov all -top <top_module_name> -vcs "-cm_libs vy
 <vcs_command_line> "
```

`vcf -fmode CC/ vcf -f <tcl file>`

`set_fml_appmode CC`//If you did not invoke VC formal in the CC app mode

`set_app_var fml_cc_coverage_analyzer true //`To enable formal connectivity coverage flow

```
read_file -cov tgl / elaborate -cov tgl
add_cc/ load_cc
sim_save_reset
check_fv -block
```

Feedback

`compute_cc_cov -block <necessary options>`// To map proven connectivity checks to coverage and perform coverage analysis)

Note:

In the Connectivity Checks coverage flow, the property creation is delayed till `compute_cc_cov` is called as big SOC designs can have million of toggle coverage goals, and only a subset of them which are in the path of the proven CC properties are needed.

`save_cc_cov_results`//To save the results of formal connectivity coverage in database

`view_coverage`// To visualize the connectivity coverage results in Verdi coverage GUI

`report_fv`// To view the toggle coverage result

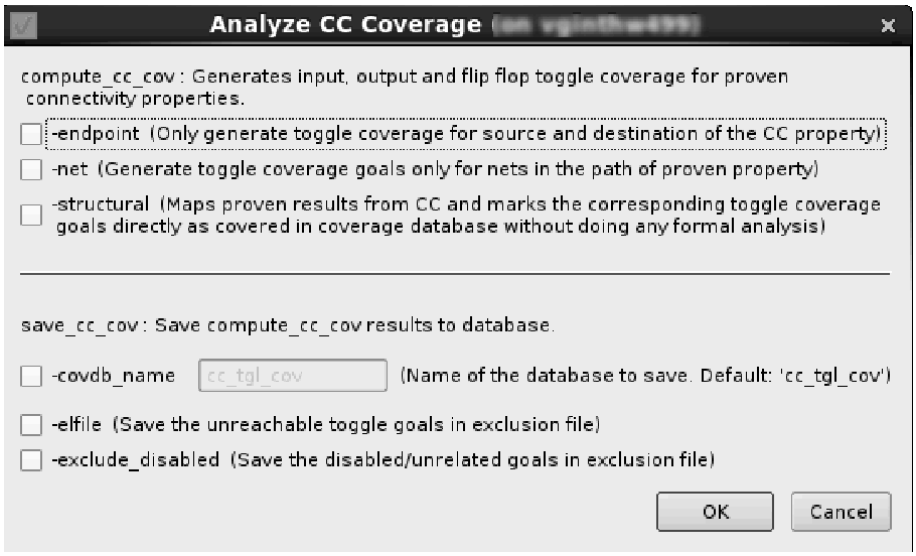
13.12.5 Computing Connectivity Coverage in VC Formal GUI

You can perform connectivity coverage analysis in the VC Formal GUI, and view the coverage score in the Verdi Coverage GUI.

To perform coverage analysis in Formal GUI:

1. Switch to the CC application mode from the *Appmode* drop-down list, or use the `set_fml_appmode CC` command.
2. Click to run the CC task to obtain the proven properties.
3. Click to compute the CC coverage analysis.

   The *Analyze CC Coverage* dialog box appears.

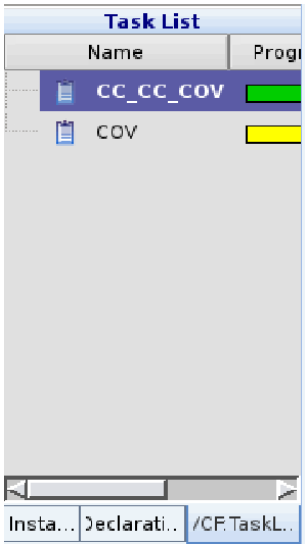


Feedback

4. Select the options for the `compute_cc_cov` and the `save_cc_cov` commands. Click *OK*.

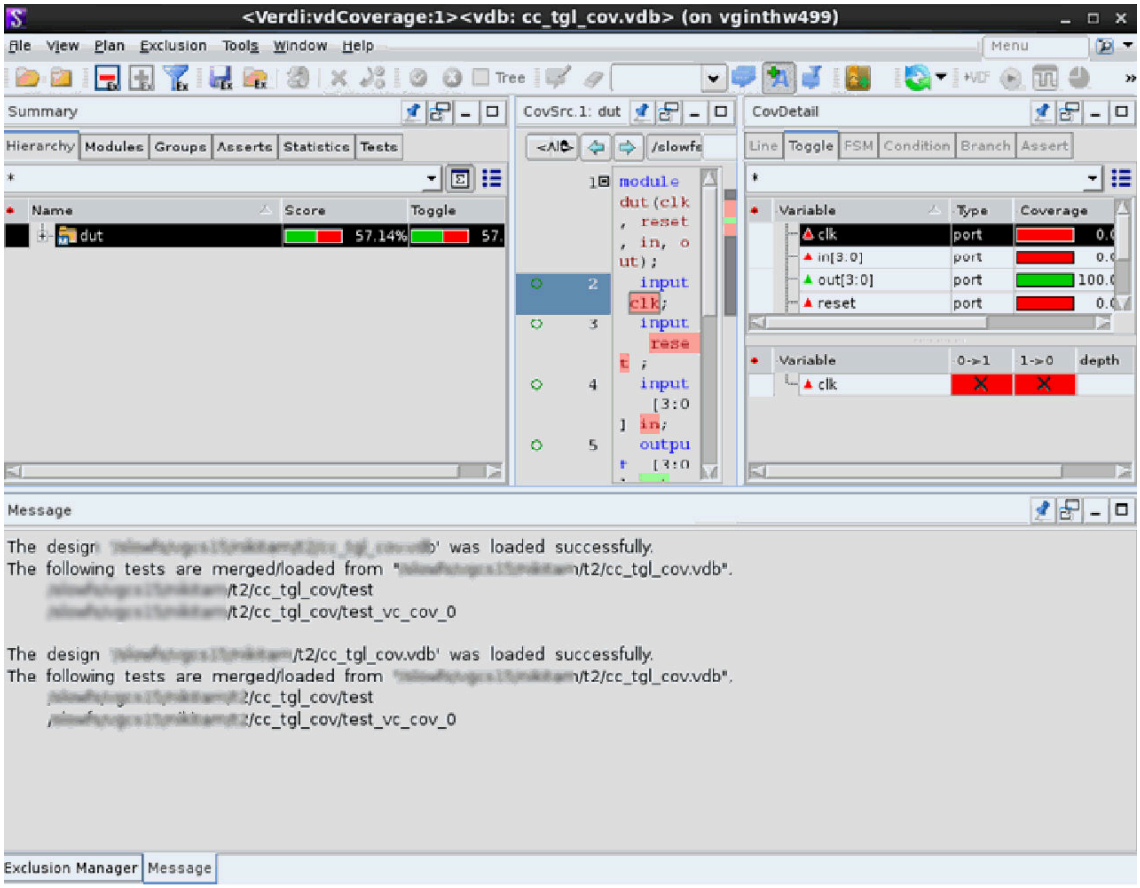
   For more details on these commands, see section Commands for Computing Connectivity Coverage.

   A new task `<task_name>_CC_COV` is created under COV appmode, and VC Formal starts to compute the toggle coverage for the proven CC properties.



   When the first covered property is computed, VC Formal automatically saves the results in the database, and opens the Verdi Coverage GUI to display the saved data.

Feedback

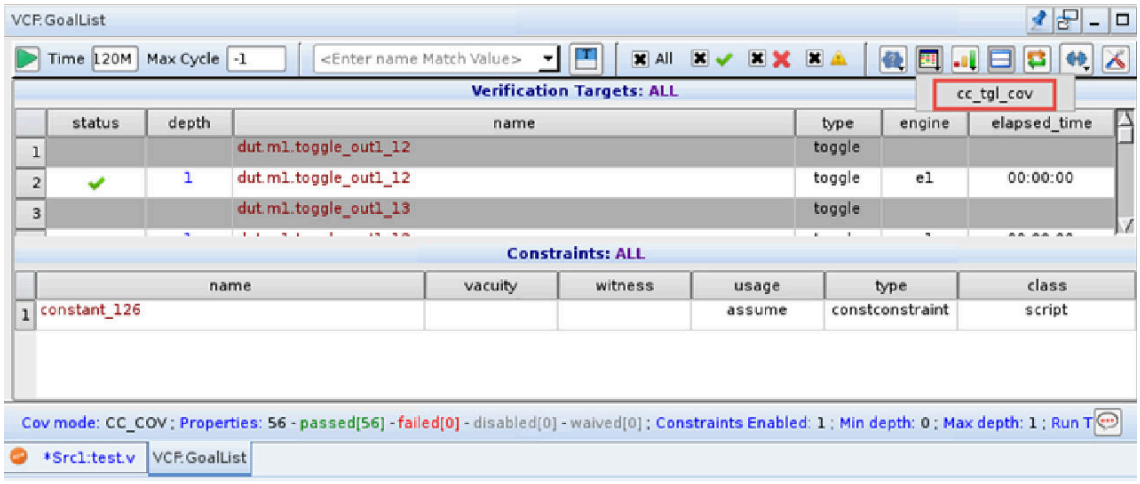


The Verdi Coverage GUI icon in the property table, pull-down menu that contains the latest saved coverage database. If you accidentally close the Verdi Coverage GUI, you click on the coverage database name to view the Verdi Coverage GUI.

If you have made any changes in the Connectivity properties, and if you perform the connectivity coverage analysis for the updated properties, the coverage database is

Feedback

updated. In such cases, the Verdi Coverage GUI pops up a dialog box asking you if you want to load an updated database.



14

Sequential Equivalence Checking Application

The VC Formal SEQ application allows comparison of designs, after register retiming, insertion of clock gating for power optimization or microarchitecture changes.

Getting Started • *What is VC Formal SEQ?* • *Accessing SEQ Lab Example*	**Improving Convergence of Properties** • *Adding Abstractions* • *Computing Reduced Constraints* • *Computing Formal Core for Proofs* • *Using Proof Assist*
Setting Up and Running SEQ • *Enabling SEQ Application mode* • *Compiling Design* • *Mapping Designs* • *Getting a Report of Mapping* • *Setting up Clocks and Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Verifying Design Setup* • *Running SEQ in TCL and GUI*	**Debugging, Reviewing and Saving Results** • *Viewing Verification Results* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving and Restoring Sessions*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	**Getting Help on Commands** • *SEQ Tcl Commands*

14.1 About VC SEQ

Formal methods are used by virtually all design teams to compare RTL designs with their synthesized gate-level versions. This approach largely eliminates the need for lengthy and time-consuming simulations of the gate-level design. It is also complete, which means that, all possible inconsistencies between the two versions of the design can be found.

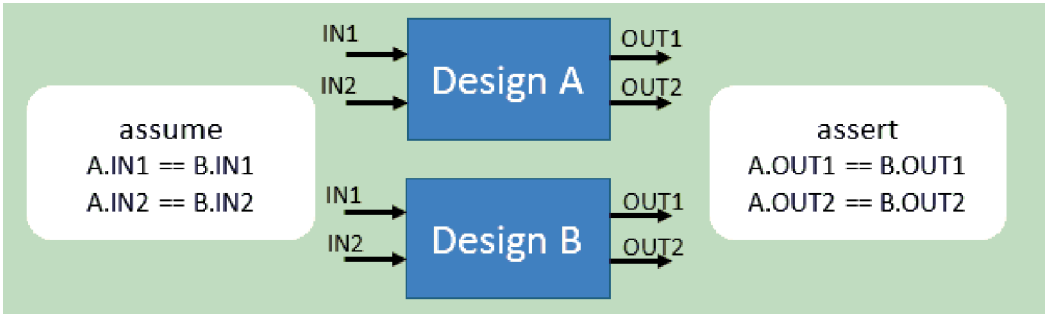
In order to address the ever-increasing consumer demand for more power efficient devices, more transformations such as clock gating are being applied directly in the RTL. When optimizations are performed at the RTL level, it becomes necessary to verify that the two versions of the RTL exhibit identical behavior. This is a far more formidable task

than verifying the equivalence of an RTL design to its gate-level implementation, but the same value proposition holds. VC SEQ enables you to compare two RTL designs and verify that they are equivalent on a cycle-by-cycle basis.

14.1.1 Equivalence Checking

VC Formal SEQ is used to compare two RTL designs written in synthesizable SystemVerilog or VHDL. It checks if two RTL designs produce identical outputs given identical inputs on a cycle-by-cycle basis. One of the designs is designated as the specification or *spec*, and the other as the implementation *impl*. Typically, the *spec* design is known to be correct and VC Formal SEQ is used to verify that the post-modification, unverified design, *impl* matches the behavior of the *spec* design. A typical use of VC Formal SEQ is when you modify a known-good design to add some transformation such as clock-gating or re-timing. You now want to verify that the RTL that results from the transformation still exhibits the same correct behavior as the RTL before the transformation.

Figure 92: Sequential Equivalence Checking



This video focuses on the Sequential Equivalence Checking (SEQ) application in VC Formal. It covers setting up the Tcl script, running SEQ properties, and debugging SEQ failures.

14.1.2 Video Links

Click the links to view videos on SEQ:

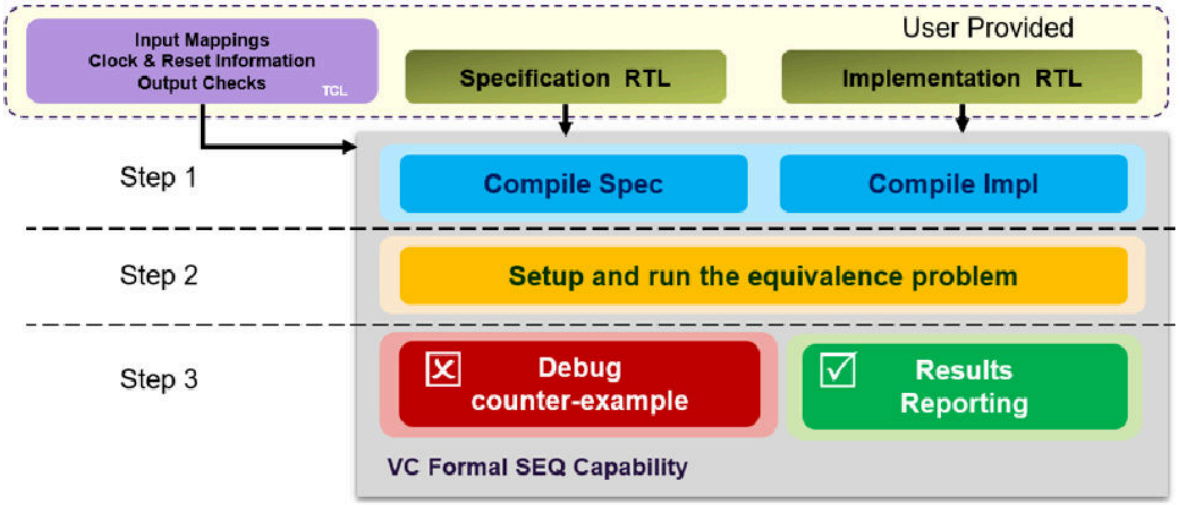
- Introduction to Sequential Equivalence Checking (4 mins)

Feedback

14.2 Methodology

VC Formal SEQ flow include the following steps:

- Set Appmode to SEQ
- Compile designs
- Map inputs and outputs
- Run SEQ
- Review results
- Debug
- Convergence
- Signoff



14.2.1 VC Formal SEQ Flow

To check for sequential equivalence in two design flow:

1. Read both the designs into VC Formal SEQ and then connect the corresponding inputs.
2. Since not all of the flip-flops in the two designs are (typically) initialized by the reset signal(s), user must initialize the reset. User can choose to set uninitialized flip-flops to the same or different starting states. Uninitialized flops can be present due to non-resettable flops or missing reset definition in the SEQ setup.

3. Instruct VC Formal SEQ on how to reset the designs. VC Formal SEQ also offers several ways to help do this efficiently.
4. It must be told to VC Formal SEQ what equivalent means, by specifying a set of assertions that it needs to prove (or disprove). If VC Formal SEQ proves all assertions, it means that the designs that are being compared are equivalent based on the definition of equivalence being provided. If it disproves one or more of the specified assertions, it generates a counter-example to expose the problem. Then debug the falsification to determine if the problem is in RTL or in the setup.
5. VC Formal SEQ may yield an inconclusive result. Generally, this happens either because of a deficiency or error in the setup, or because the problem is too large or complex for VC Formal SEQ. If the setup is not an issue, then use advanced proof techniques (discussed in later chapters) to simplify the problem and enable VC Formal SEQ to converge on a solution.

In order to get an accurate and meaningful proof of equivalence, the assertions, that is, the description of equivalence must be both correct and complete. Otherwise, the proof or counter-example generated by VC Formal SEQ will be meaningless.

An example that contains two single RTL designs can be found at

$VC_STATIC_HOME/doc/vcst/examples/SEQ

The use model for comparing the two RTL designs can be found in the

$VC_STATIC_HOME/doc/vcst/examples/SEQ/README_SEQ_setup.pdf

VC Formal SEQ Quick cheat sheet can be found under

$VC_STATIC_HOME/doc/vcst/Quick_Reference/VC_Formal_SEQ_App_Quick_Guide.pdf

14.2.2 Designs

VC Formal SEQ flow can be enabled by setting the formal mode to SEQ by using the `-fmode SEQ` option to vcf:

```
vcf -fmode SEQ
```

An alternative option is to set the appmode to SEQ in setup tcl script:

```
set_fml_appmode SEQ
```

VC Formal SEQ ideally deals with 2-designs. Each of these designs needs to be analyzed separately in different libraries. The work library name can be specified by passing `-library <library_name>` to `analyze`. The default library name for implementation and specification designs are *impl* and *spec* respectively.

```
analyze -format verilog -library spec <spec_design_file_list>
analyze -format verilog -library impl <impl_design_file_list>
```

Both the design needs to be elaborated by calling `elaborate_seq`. Implementation and specification design top needs to be passed to `elaborate_seq` with `-impltop` and `-spectop` respectively.

```
elaborate_seq -spectop <spec_design_top> -impltop <impl_design_top>
```

If a different library name is used other than *spec* or *impl* with `analyze`, `-impllib` and `-speclib` has to be specified with `elaborate_seq` to identify the library names for implementation and specification designs respectively.

```
analyze -format verilog -library mylib1 <spec_design_file_list>
analyze -format verilog -library mylib2 <impl_design_file_list>
elaborate_seq -speclib mylib1 -impllib mylib2 -spectop <spec_design_top>
 -impltop <impl_design_top>
```

If a checker file needs to bind to both *spec* and *impl* designs, and both the designs have same top level name, set the following application variable to true:

```
set_app_var enable_same_library_sv_bind true
```

Note:

Apart from `elaborate_seq`, the `elaborate` command can also handle the SEQ options, that is, you can choose to use `elaborate` instead of `elaborate_seq`. The following is the use model:

```
elaborate mul -impltop mul
```

You can observe the use of the default `design_name` option for `elaborate` instead of the `-spectop` option for `elaborate_seq`.

14.2.3 Same Design Compile

It is often seen that the design is compared to itself because of various reasons. For example, you can use one design with clock gating disabled and the other one with clock gating enabled. Instead of analyzing the same design twice, `-same_design`can be passed to `elaborate_seq`. Also, the `-library` option with `analyze` and `-impltop`option with `elaborate_seq` is no longer required:

```
analyze -format verilog <spec_design_file_list>
elaborate_seq -spectop <spec_design_top> -same_design
elaborate mul -same_design
```

Once the design is read, the rest of the flow remains the same.

14.3 Setting up the Equivalence Problem

After you have read the designs, perform the following steps to complete the task of setting up the equivalence checking problem.

- Mapping the Designs
- Setting Up Clocks and Resets
- Setting Up Initial State
- Getting a Report of Mappings

14.3.1 Mapping the Designs

To compare the two designs, VC SEQ needs to know how inputs to the two designs relate to each other. It also needs to know what equivalence means, that is, how the outputs of the two designs are to be compared to decide that they are the same in each clock cycle. Moreover, SEQ also needs to map the black boxes, abstractions, and the undriven signals of the two designs.

The simplest solution is when both the inputs and the outputs of the two designs have the same names. In this case, you must map the corresponding inputs, and compare the corresponding outputs. You can do this using the `map_by_name` command.

Syntax

```
vcf> map_by_name -help
Usage: map_by_name # Map signals by name in the two designs
        [-input]   (Automatically map inputs in the two designs)
        [-output]  (Automatically map outputs in the two designs)
        [-undriven](Automatically map undrivens in the two designs)
        [-blackbox] (Automatically create assumptions and
                     assertions for blackboxed modules)
        [-abstraction](Automatically create assumptions and assertions
                       for abstractions)
        [-unloaded] (Automatically map unloaded nets in the two designs)
        [-noenv]    (Do not create -env assumptions)
        [-verbose]  (Report all Info messages)
        [-infer_unresolved_ports] (Apply heuristics (based on fanin's)
                                   to infer ports of unresolved modules)
        [-no_undriven_assert] (Removes assertions that are undriven
                               in one design)
        [-use_case_equality] (Use case equality (===) instead of
                              equality (==) for outputs)
        [-cg_aware_clock <cgAwareClockName>] (Clock signal to create
                                              conditional mapping for
                                              compile memory
                                              abstraction clocks
                                              to take care of
```

Feedback

```
                                          clock gating)
      [-cg_type <cgCondPropType>] (Type (default: clock_only) of
                                  conditional property on
                                  the ports of compile
                                  abstraction memory:
                                   Values: clock_only, all_syn, none)
      [-exclude <list-of-signals>] (List of signals or hierarchical
                                    instances to exclude from
                                    mapping (without prefix))
      [-instances <list-of-instances>] (List of instances for mapping
                                       (without prefix))
      [-clock <clockName>]    (The clock to create the
                              concurrent assertions)
      [-edge <edgeType>]      (Type of clock edge (default: posedge):
                               Values: posedge, negedge, edge)
      [-align_mismatch_bitwidth <alignType>] (To align bits if
                                             bitwidth mismatches:
                                            Values: lsb, msb, none)
      [-condition <condition>] (Condition expression for the equality)
      [-spec_latency <list-of-latencies>] (List of latencies for the
                                          spec signal)
      [-impl_latency <list-of-latencies>] (List of latencies for the
                                          impl signal)
      [-specprefix <prefixName>] (Prefix for the specification
                                 design (default: spec))
      [-implprefix <prefixName>] (Prefix for the implementation
                                  design (default: impl))
      [-enable_exact_prefix <true|false>] (Enable matching of exact
                                          instance names given with
                                          -specprefix and -implprefix:
                                          Values: true, false)
      [-map_type <type-list>] (List of object types to be mapped:
                              Values: input, output, undriven, bb,
                              abs, unloaded, auto, all)
      [-skip_map_type <type-list>] (List of object types to be skipped
                                    from mapping: Values: input,
                                    output, undriven, bb,
                                    abs, unloaded)
```

The semantics of the `map_by_name` command is as follows:

- If no options are provided, then it enables all the mappings (input, output, black box, CMA, abstraction, and undriven).
- If an option is provided, then only the mapping for that option is done. For example, the `map_by_name -input` command adds only the primary input mappings.
- The `map_by_name` command must be placed before the `sim_run` command to ensure proper instrumentation of environment constraints for undrivens, blackbox outputs, CMA outputs, and primary inputs.

    - If you run `map_by_name` after `sim_run`, the *MAP_BY_NAME_AFTER_SIM_RUN* error is reported, and the Tcl run is terminated.
    - To downgrade the severity of *MAP_BY_NAME_AFTER_SIM_RUN* message to Warning, so that the Tcl run continues, use the following command:

      ```
      set_message_severity -names MAP_BY_NAME_AFTER_SIM_RUN warning
      ```

      However, in this case, the assumes are created for undrivens, blackbox outputs, CMA outputs, and primary inputs, instead of the environment constraints.

- The `map_by_name` command has two switches that are frequently used:
    - `-exclude <list>`: This switch excludes the signals specified in the list from the automatic mapping, so, the command `map_by_name -exclude in1` maps all signals with the same names except `in1`.
    - `-blackbox`: This switch maps all the black box instances and CMA (Compile Memory Abstractions) with the same name in the two designs. You can selectively exclude any black box or compile memory using the `-exclude` option.
        - By default, only those black box instances and CMA are mapped whose underlying modules also have the same name.

          To disable this default behavior and enable mapping between black box instances irrespective of their underlying modules, use `seq_config -map_diff_bb_designs ON` before `map_by_name`.
        - By default, `map_by_name` infers the directions for ports of unresolved modules. To disable this inference, use `seq_config -infer_unresolved_ports OFF` before `map_by_name`.
        - To enable application of COI heuristics to infer direction of ports for all blackbox modules, use `seq_config -enable_infer_blackbox_port_direction true`.

          This option helps with inferring the direction of ports for unresolved modules even when the user has explicitly specified `set_blackbox` over them.
        - The `map_by_name` command skips the mapping of the blackbox where all the output ports are unloaded. A warning message is reported, when the `-verbose` option is enabled along with the summary message for the number of such blackbox output pairs.
    - `-align_mismatch_bitwidth <lsb> <msb>`: This switch maps ports and signals that have the same name but different bit width. It aligns the mismatched bit width either by LSB or by MSB.
- Following are few other options supported by `map_by_name` which can be used based on customer requirements

Feedback

- `-undriven`: Undriven signals can lead to false failures. Use this option with `map_by_name` to automatically map undrivens in the two designs.

  Note:
  The undriven signals with snipped peers are also mapped.

- `-abstraction`: Map abstractions.
- `-infer_unresolved_ports`: Apply heuristics (based on fanin's) to infer ports of unresolved modules
- `-clock`: The clock to create the concurrent assertions. This allows the evaluation of assertions only at the occurrence of a clock tick.

  Use the setting `seq_config -enable_clock_coi_analysis true` to create an immediate assertion rather than a concurrent one for those signals whose combinational fanin contains the clock signal provided with the `-clock` option (the clock signal must be present at the sequential depth '0' of the signal fanin).

  Note:
  This is currently a Beta feature. Contact Synopsys Support for more information.

- `-edge`: Type of clock edge (default: posedge): Values: posedge, negedge, edge
- Use the `-spec_latency` and `-impl_latency` options to create the concurrent assertions with time delay (## operator).

  The following example shows how to create concurrent assertions with same latency:

```
vcf> map_by_name -map_type {output} -spec_latency 3 -impl_latency 3
                     -clock spec.clk
[Info] MAPPING_ENABLED: output.
[Info] MAPPING_DISABLED: input, blackbox, CMA, undriven,
 abstraction.
[Info] MBN_SUMMARY: '2' primary outputs are mapped as asserts.
1
vcf> report_fv -list -no_summary
  List Results
   Property List:
   --------------
   > Assertion
     # Assertion: 2
     [  0] not_run  -   _map_output_out1 - @(posedge spec.clk) (##3
                                          (spec.out1 == impl.out1))
     [  1] not_run  -  _map_output_out2 - @(posedge spec.clk) (##3
                                           (spec.out2 == impl.out2))
```

- Use the `-condition` option to specify condition expression for the equality. This allows evaluation of the assertions only when the condition expression is satisfied.

Feedback

The following example shows creating assertions based on a condition and with different latencies:

```
vcf> map_by_name -map_type {output} -spec_latency {4 5 10}
                    -impl_latency {2 4 10} -condition spec.en
                    -edge edge -clock spec.clk
[Info] MAPPING_ENABLED: output.
[Info] MAPPING_DISABLED: input, blackbox, CMA, undriven,
 abstraction.
[Info] MBN_SUMMARY: '2' primary outputs are mapped as asserts.

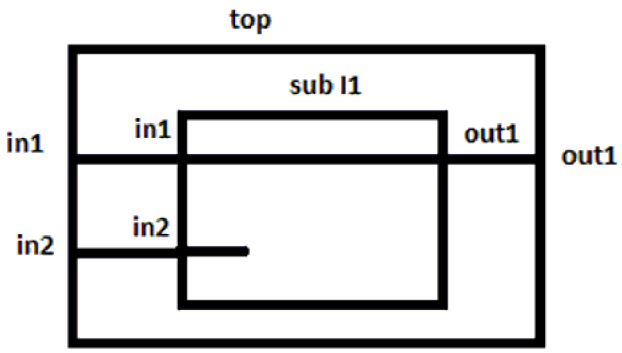
vcf> report_fv -list -no_summary

  List Results
   Property List:
   --------------
   > Assertion
     # Assertion: 2
     [  0] not_run  (not_run) -  _map_output_out1 - @(edge spec.clk)
                                 spec.en |-> ((##4 (spec.out1 ==
                                 $past(impl.out1, 2)))) or
                                 ((##5 (spec.out1 ==
                                 $past(impl.out1,
                                 1)))) or (##10 (spec.out1 ==
                                 impl.out1))
     [  1] not_run  (not_run) -  _map_output_out2 - @(edge spec.clk)
                                 spec.en |-> ((##4 (spec.out2 ==
                                 $past
                                 (impl.out2, 2)))) or
                                  ((##5 (spec.out2 ==
                                 $past(impl.out2, 1)))) or
                                 (##10 (spec.out2 == impl.out2))
```

- `-no_undriven_assert`: Removes assertions that are undriven in one design.
- `-use_case_equality`: Use case equality (===) instead of equality (==) for outputs.
- `-unloaded`: Automatically maps the unloaded nets in the two designs.

  Consider the following RTL schematic for both *spec*and *impl*. When the `-unloaded` option is specified, VC Formal creates an assertion on the ‘unloaded’ net ‘*spec.I1.in2*’.

Feedback



The assert properties are created with prefix '_map_unloaded_'. For example, `_map_unloaded_I1_in2 spec.I1.in2 == impl.I1.in2`

Alternatively, you can also set the `seq_add_unloaded_assert` formal variable to true, to enable creating assertions for unloaded nets by default.

By default, the `seq_add_unloaded_assert` formal variable is set to false.

- `-map_type <obj-types-to-be-mapped>`: This option takes a list of object types to be mapped. Allowed values are `input, output, undriven, bb, abs, unloaded, auto` and `all`. The `auto` value enables mapping of `input, output, blackbox/CMA, abstraction` and `undriven`. The `all` value enables mapping of the following map type objects: `input, output, blackbox/CMA, abstraction, undriven` and `unloaded`. The default value of this option is `auto`.
- `-skip_map_type <obj-types-to-be-skipped>`: This option takes a list of object types to be skipped from mapping. Allowed values are `input, output, undriven, bb, abs` and `unloaded`. The `-skip_map_type` option can be used to skip the mapping of any signal type. The example shows how user may skip, say output mapping.

```
vcf>map_by_name -map_type {auto} -skip_map_type {output}
[Info] MAPPING_ENABLED: input, blackbox, CMA, undriven, abstraction.
[Info] MAPPING_DISABLED: output.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'in5' is not mapped by name,
 because it is not found in the 'spec' instance.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'bbout1' is not mapped by
 name, because it is not found in the 'impl' instance.
[Info] MBN_SUMMARY: '2' primary inputs are mapped as assumes.
[Info] MBN_SUMMARY: '1' blackbox inputs are mapped as asserts.
[Info] MBN_SUMMARY: '1' blackbox outputs are mapped as assumes.
[Info] MBN_SUMMARY: '1' CMA inputs are mapped as asserts.
[Info] MBN_SUMMARY: '1' CMA outputs are mapped as assumes.
[Info] MBN_SUMMARY: '0' abstraction inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' abstraction outputs are mapped as assumes.
[Info] MBN_SUMMARY: '1' undriven nets are mapped as assumes.
```

Feedback

The `-input, -output, -blackbox, -abstraction, -undriven, -unloaded` options are legacy options which have been deprecated and using them would lead to the following warning:

```
vcf> map_by_name -output
[Warning] SEQ_OPTION_DEPRECATED: The option(s) '-output' have been
 deprecated and will be removed from future releases.
Please use the '-map_type' option to provide this information.
[Info] MAPPING_ENABLED: output.
[Info] MAPPING_DISABLED: input, blackbox, CMA, undriven,
 abstraction.
```

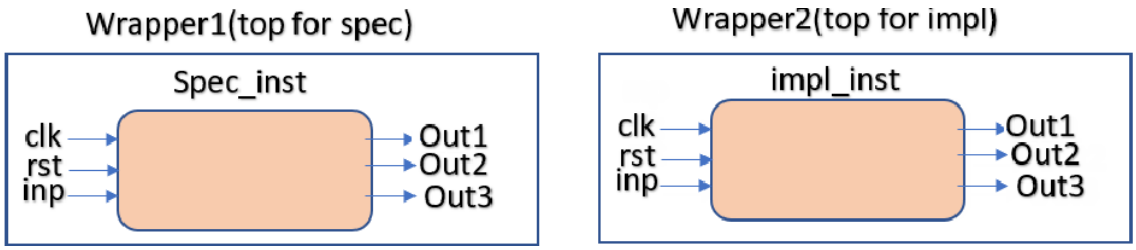
The `-map_type` option is not compatible with these legacy options, that is, `-input, -output, -blackbox, -abstraction, -undriven` and `-unloaded`. When you use these options together, the following error is reported:

```
vcf>map_by_name -input -map_type {bb}
[Error] SEQ_INCOMPATIBLE_OPTIONS: Using legacy (soon to be
 deprecated) option(s) '-input' with '-map_type' is not supported.
Use '-map_type' option to provide this information.
Error: 0
Use error_info for more info. (CMD-013)
```

- When using the `map_by_name` command, by default, the SEQ App expects and generates automatic properties for the top-level module of the specification and the implementation design.

  However, if the user wants to map an instance that is not the top-level module but one or more levels down in the hierarchy then the properties are not generated. To enable this the user needs to first configure the SEQ app. This configuration is done by using below command after elaboration stage:

  ```
  seq_config -specprefix spec.spec_inst -implprefix impl.impl_inst
  -allow_different_prefix
  ```

  When you use `map_by_name` on instances which are not top level. It expects at least one of them to be a top level, otherwise it produces the following warnings and expects the user to map the clocks and resets for both *spec* and *impl* design. In this case, you need to create clock and reset for both *spec* and *impl* design using `create_clock` and `create_reset`.

Feedback

[Warning] ENVCNSTR_W_INVALIDSIG: Invalid environment constraint signal. 'spec.spec_inst.clk'. Signals must be primary inputs, blackbox outputs, undriven or snipped environment signals (use snip_driver -env <signal>).

[Warning] ENVCNSTR_W_INVALIDSIG: Invalid environment constraint signal. 'impl.impl_inst.clk'. Signals must be primary inputs, blackbox outputs, undriven or snipped environment signals (use snip_driver -env <signal>).

[Warning] ENVCNSTR_W_INVALIDNODES: Environment constraint contains invalid signals: spec.spec_inst.clk == impl.impl_inst.clk Constraint applied on the formal model only.

If you see the above messages, you must declare the clocks for both the designs.

```
create_clock spec.spec_inst.clk -period 100
create_clock impl.spec_inst.clk -period 100
```

Similarly, if you see messages for reset you must declare the resets for both the designs.

- The compile memory abstraction identifies a memory and blackboxes it by creating a predefined box around it. SEQ mappings creates regular input/output mappings for these blackboxes. Properties on ports of the abstracted memory that are driven by same constants will be skipped from creation. This saves runtime from being spent in solving such trivially proven properties. The clock input mapping trivially fails in presence of clock gating, as the *spec* is not gated and *impl* is gated. To address this, you can create conditional mappings to check the clock signals only when enable signal to these memories is available.
  - `-cg_aware_clock`: The clock used to create conditional mapping for *clock*, *data* and *addr* ports of the synchronous read/write ports of compile abstracted memory (blackbox) to take care of clock gating.
  - `-cg_type`: Can take one of the following values:
    - `all_syn`: Conditional property creation on *clock*, *data* and *addr* ports of the synchronous read/write ports of the compile abstracted memory.
    - `clock_only`: Conditional property creation with a different flavor, only on the *clock* port of the synchronous read/write ports of compile abstracted memory.
    - `none`: No conditional mapping for any port of compile abstracted memory.

  Examples

  - The following example shows using a `cg_type` as `all_syn` with valid `cg_aware_clock`. Only relevant snippet from '`report_fv`' command is shown.

    ```
    vcf>map_by_name -cg_aware_clock spec.clk -cg_type all_syn
    [Info] MAPPING_ENABLED: input, output, blackbox, CMA, undriven,
    ```

Feedback

```
abstraction.
[Info] MBN_SUMMARY: '5' primary inputs are mapped as
assumes.
[Info] MBN_SUMMARY: '4' primary outputs are mapped as
asserts.
[Info] MBN_SUMMARY: '0' blackbox inputs are mapped
as asserts.
[Info] MBN_SUMMARY: '0' blackbox outputs are mapped
as assumes.
[Info] MBN_SUMMARY: '12' CMA inputs are mapped as asserts.
[Info] MBN_CG_AWARE_CLOCK_SUMMARY: '3' synchronous ports of
 abstracted memories are changed due to option cg_aware_clock.
[Info] MBN_SUMMARY: '1' CMA outputs are mapped as assumes.
[Info] MBN_SUMMARY: '0' abstraction inputs are mapped
as asserts.
[Info] MBN_SUMMARY: '0' abstraction outputs are mapped
as assumes.
[Info] MBN_SUMMARY: '1' undriven nets are mapped as
assumes.
vcf>  report_fv -list -regr -usage assert -no_summary

List Results
Property List:
--------------
> Assertion
# Assertion: 3
[  18] not_run   (not_run) -  _map_cma_cg_wr_f1_r2_mem_addr1 -
@(posedge spec.clk) (##1($changed(spec_f1_r2_mem_clock1_tgl) &&
 $past(spec.f1.r2.mem.enable1))
|-> ($past(spec.f1.r2.mem.addr1) == $past(impl.f1.r2.mem.addr1)))
[  19] not_run   (not_run)  -  _map_cma_cg_wr_f1_r2_mem_data1 -
@(posedge spec.clk) (##1($changed(spec_f1_r2_mem_clock1_tgl) &&
 $past(spec.f1.r2.mem.enable1))
|-> ($past(spec.f1.r2.mem.data1) == $past(impl.f1.r2.mem.data1)))
[  20] not_run             -  _map_cma_cg_wr_f1_r2_mem_clock1 -
@(posedge spec.clk) (##1(($changed(spec_f1_r2_mem_clock1_tgl) &&
 $past(spec.f1.r2.mem.enable1))
== ($changed(impl_f1_r2_mem_clock1_tgl) &&
 $past(impl.f1.r2.mem.enable1))))
```

- The following example shows using `cg_aware_clock` with default `map_by_name`. This creates conditional property only on the clock port of the synchronous read/write ports of compile abstracted memory as `cg_type` has default value of `clock_only`.

```
vcf>map_by_name -cg_aware_clock spec.clk
[Info] MAPPING_ENABLED: input, output, blackbox, CMA, undriven,
 abstraction.
[Info] MBN_SUMMARY: '5' primary inputs are mapped as assumes.
[Info] MBN_SUMMARY: '4' primary outputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' blackbox inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' blackbox outputs are mapped as assumes.
```

Feedback

```
[Info] MBN_SUMMARY: '12' CMA inputs are mapped as asserts.
[Info] MBN_CG_AWARE_CLOCK_SUMMARY: '1' synchronous ports of
 abstracted memories are changed due to option cg_aware_clock.
[Info] MBN_SUMMARY: '1' CMA outputs are mapped as assumes.
[Info] MBN_SUMMARY: '0' abstraction inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' abstraction outputs are mapped as
 assumes.
[Info] MBN_SUMMARY: '1' undriven nets are mapped as assumes.
vcf>  report_fv -list _map_cma_f1_r2_mem_clock1 -no_summary

List Results
Sequential Equivalence List:
----------------------------
> Assertion
# Assertion: 1
[  20] not_run - _map_cma_cg_wr_f1_r2_mem_clock1 -
 (spec_f1_r2_mem_enable1_reg || impl_f1_r2_mem_enable1_reg) ->
 spec.f1.r2.mem.clock1 == impl.f1.r2.mem.clock1
```

- The following example shows using `cg_aware_clock` without enabling black box/ CMA mapping, say with only `-output` switch to `map_by_name`.

```
vcf> map_by_name -output -cg_aware_clock spec.clk
[Info] MAPPING_ENABLED: output.
[Info] MAPPING_DISABLED: input, blackbox, CMA, undriven,
 abstraction.
[Warning] MAP_BY_NAME_DEPENDENT_OPTION: Option '-cg_aware_clock'
 used without enabling the '-blackbox' flow.
The option will have no effect.
[Info] MBN_SUMMARY: '4' primary outputs are mapped as asserts.
```

- The following example shows using `cg_aware_clock` with specifying `cg_type` as `none`. Warning 'SEQ_COMP_MEM_COND_SKIPPED' gets flagged to indicate that the conditional mapping has been skipped for synchronous ports of compile memory as 'cg_type' has been used with the value 'none'.

```
vcf> map_by_name -cg_aware_clock spec.clk -cg_type none
[Info] MAPPING_ENABLED: input, output, blackbox, CMA, undriven,
 abstraction.
[Warning] SEQ_COMP_MEM_COND_SKIPPED: Conditional mapping will be
 skipped for synchronous ports of compile memory as 'cg_type' has
 been used with value 'none'.
Any value specified with 'cg_aware_clock' switch will be skipped.
 Immediate assertions will be created.
[Info] MBN_SUMMARY: '5' primary inputs are mapped as assumes.
[Info] MBN_SUMMARY: '4' primary outputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' blackbox inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' blackbox outputs are mapped as assumes.
[Info] MBN_SUMMARY: '12' CMA inputs are mapped as asserts.
[Info] MBN_CG_AWARE_CLOCK_SUMMARY: '0' synchronous ports of
 abstracted memories are changed due to option cg_aware_clock.
[Info] MBN_SUMMARY: '1' CMA outputs are mapped as assumes.
```

Feedback

```
[Info] MBN_SUMMARY: '0' abstraction inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' abstraction outputs are mapped as
 assumes.
[Info] MBN_SUMMARY: '1' undriven nets are mapped as assumes.
```

- The following example shows using a `cg_type` as `all_syn` but not specifying any `cg_aware_clock`. This is an error and the flow stops.

```
vcf>map_by_name -cg_type all_syn
[Info] MAPPING_ENABLED: input, output, blackbox,
CMA, undriven, abstraction.
[Error] SEQ_OPTION_REQUIRED: Option '-cg_aware_clock'
is required with option '-cg_type'.
```

- The following example shows using a `cg_type` with `spec_latency`. That is presently not supported and gives an error and the flow stops. Similar behavior occurs with `impl_latency`.

```
vcf>map_by_name -cg_type all_syn -spec_latency {1}
Error: Cannot specify '-cg_type' with '-spec_latency'. (CMD-001)
```

- In case of a blackbox with all unloaded output ports, there will be a summary warning message for the number of pairs of such blackbox output ports skipped. If `-verbose` is specified, an additional warning message is reported describing the signals that have been skipped for this reason.

```
vcf> map_by_name -map_type {bb} -verbose

[Info] MAPPING_ENABLED: blackbox, CMA.
[Info] MAPPING_DISABLED: input, output, undriven, abstraction.
[Info] SEQ_MAPPING_START: Mapping for blackbox inputs and
 blackbox outputs has started.
[Warning] SEQ_BBOX_UNLOADED_SKIPPED: Mapping of blackbox ports
 'spec.ecodonor0.out' and 'impl.ecodonor0.out' is skipped as
 either one or both the ports are unloaded.
.
.
.
[Warning] MAP_BBOX_OUTPUT_UNLOADED_SUMMARY: '1' pairs of blackbox
 output ports are not mapped because either one or both the ports
 are unloaded.
Please use 'report_seq_mappings' for more details.
```

- If you prefer to specify output comparisons yourself and map only the inputs, use the following commands:

```
map_by_name -input
fvassert -expr {spec.sig1 == impl.sig1}
fvassert -expr {spec.sig2 == impl.sig2}
```

Feedback

```
..
.
```

In this scenario, the `map_by_name` command maps corresponding inputs and the `fvassert` commands define the outputs that you want to compare.

VC Formal has various commands to support mapping using scripts. Few examples and use cases are given below.

- Following simple script can be useful to map all output ports and create assert properties for a specific instance manually based on the requirements.

```
foreach_in_collection r [get_ports spec.* -filter direction==out
 -word] {
    set i [get_attribute $r full_name];
    if {[string range $i 0 4] == "spec."} {
            set i [string replace $i 0 4 ""];
            fvassert -expr "impl.$i == spec.$i";
        }
    }
```

- Following simple script can be useful to map all input ports and create assume properties for a specific instance manually based on the requirements.

```
foreach_in_collection r [get_ports spec.* -filter direction==in -word]
 {
    set i [get_attribute $r full_name];
    if {[string range $i 0 4] == "spec."} {
            set i [string replace $i 0 4 ""];
            fvassume -env -expr "impl.$i == spec.$i";
        }
    }
```

- If on the other hand, you want to automatically compare outputs with the same names, but manually map inputs, use the following commands:

```
map_by_name -output
fvassume -env -expr {spec.some_sig_1 == impl.other_sig_9}
fvassume -env -expr {spec.some_sig_2 == impl.other_sig_15}
..
```

In this scenario, the map_by_name command sets up equivalence properties to be checked for outputs with the corresponding names, and `fvassume` commands specifies the input mappings.

- For example, if the two designs have some input and output names in common, that is, both designs have input signals `in1` and `in2`, and output signals `out1` and `out2`, but there are additional signals with different names, use the following commands:

```
map_by_name
fvassume -env -expr {spec.some_input_1 == impl.some_other_input_5}
fvassume -env -expr {spec.some_input_2 == impl.some_other_input_43}
```

Feedback

```
...
..
fvassert -expr {spec.some_output_1 == impl.some_other_output_22}
..
```

In this scenario, the combination of the `map_by_name` command and the `fvassume/fvassert` commands set up the following input mappings:

```
spec.in1 == impl.in1
spec.in2 == impl.in2
spec.some_input_1 == impl.some_other_input_5
spec.some_input_2 == impl.some_other_input_43
```

The following output comparisons are defined:

```
spec.out1 == impl.out1
spec.out2 == impl.out2
spec.some_output_1 == impl.some_other_output_22
```

- To selectively map corresponding black-boxed modules between the two designs, use the `-instances` option along with `-blackbox` option. For example, if there is an instance M1 of module M in each design, and the modules take inputs A and B to produce an output Y, then the `-blackbox -instances M1` command causes the following mappings:

```
fvassert -expr {spec.M1.A == impl.M1.A}
fvassert -expr {spec.M1.B == impl.M1.B}
fvassume -env -expr {spec.M1.Y == impl.M1.Y}
```

A mapping summary is displayed by VC Formal SEQ once `map_by_name` command gets executed. The mapping summary shows the number of input, output, blackbox, CMA, undriven and abstraction points being mapped.

```
[Info] MAPPING_ENABLED: input, output, blackbox, CMA, undriven,
 abstraction.
[Info] MBN_SUMMARY: '5' primary inputs are mapped as assumes.
[Info] MBN_SUMMARY: '2' primary outputs are mapped as asserts.

[Info] MBN_SUMMARY: '0' blackbox inputs are mapped as asserts.

[Info] MBN_SUMMARY: '0' blackbox outputs are mapped as assumes.

[Info] MBN_SUMMARY: '0' CMA inputs are mapped as asserts.

[Info] MBN_SUMMARY: '0' CMA outputs are mapped as assumes.

[Info] MBN_SUMMARY: '0' abstraction inputs are mapped as asserts.

[Info] MBN_SUMMARY: '0' abstraction outputs are mapped as assumes.

[Info] MBN_SUMMARY: '2' undriven nets are mapped as assumes.
```

The `report_fv` command shows the properties created by `map_by_name`.

- Properties for inputs and outputs will have a prefix `_map_input_` and `_map_output_` respectively.
- Properties for blackbox inputs and outputs will have a prefix `_map_bb_`.
- Properties for CMA inputs and outputs will have a prefix `_map_cma_`.
- Properties for abstraction points will have a prefix `_map_abs_`.
- Properties for undriven signals will have a prefix `_map_undriven_`.

For more information on how to manage the properties, see section Managing Constraints and Properties for Formal Analysis.

Setting Formal Variables to Configure Splitting for Elements

Use the following formal variables to configure the splitting for elements:

- `seq_split_multi_dim_array_port`: Splits higher dimensions of a multidimensional primary output or blackbox input port into individual elements. By default, this formal variable is set to false.
- `seq_split_struct_array_port`: Splits a struct or array of struct port into individual elements. By default, this formal variable is set to true, and works only for primary output or blackbox input ports.

Example

Consider the following nested 'struct' snippet:

```
============================
typedef struct {
    logic[2:0]     mySeq;
    logic[2:0]     mySeq1;
} myNestStruct;

typedef struct {
    logic[2:0]     mySeq1;
    myNestStruct   nest ;
} myStruct;
============================
```

Consider the following output port declarations:

```
-----------------------------------------------------------
  output myStruct out1,
  output logic [7:0] out2 [7:0],
  output myStruct out3 [1:0][1:0]
-----------------------------------------------------------
```

For simplicity, the example of *Primary Outputs* is being used. The identical behavior is observed on any *Blackbox Inputs* also.

Feedback

Default flow with seq_split_struct_array_port = true and seq_split_multi_dim_array_port = false

The struct port 'out1', and array of struct port 'out3' gets split. The non-struct array port 'out2' does not get split.

```
[  6] proven          - _map_output_out1_mySeq1 - spec.out1.mySeq1 == impl.out1.mySeq1
[  7] proven          - _map_output_out1_nest_mySeq - spec.out1.nest.mySeq == impl.out1.nest.mySeq
[  8] proven          - _map_output_out1_nest_mySeq1 - spec.out1.nest.mySeq1 == impl.out1.nest.mySeq1
[  9] proven          - _map_output_out2 - spec.out2 == impl.out2
[ 10] proven          - _map_output_out3[0][0]_mySeq1 - spec.out3[0][0].mySeq1 == impl.out3[0][0].mySeq1
[ 11] proven          - _map_output_out3[0][1]_mySeq1 - spec.out3[0][1].mySeq1 == impl.out3[0][1].mySeq1
[ 12] proven          - _map_output_out3[1][0]_mySeq1 - spec.out3[1][0].mySeq1 == impl.out3[1][0].mySeq1
[ 13] proven          - _map_output_out3[1][1]_mySeq1 - spec.out3[1][1].mySeq1 == impl.out3[1][1].mySeq1
[ 14] proven          - _map_output_out3[0][0]_nest_mySeq - spec.out3[0][0].nest.mySeq == impl.out3[0][0].nest.mySeq
[ 15] proven          - _map_output_out3[0][1]_nest_mySeq - spec.out3[0][1].nest.mySeq == impl.out3[0][1].nest.mySeq
[ 16] proven          - _map_output_out3[1][0]_nest_mySeq - spec.out3[1][0].nest.mySeq == impl.out3[1][0].nest.mySeq
[ 17] proven          - _map_output_out3[1][1]_nest_mySeq - spec.out3[1][1].nest.mySeq == impl.out3[1][1].nest.mySeq
[ 18] proven          - _map_output_out3[0][0]_nest_mySeq1 - spec.out3[0][0].nest.mySeq1 == impl.out3[0][0].nest.mySeq1
[ 19] proven          - _map_output_out3[0][1]_nest_mySeq1 - spec.out3[0][1].nest.mySeq1 == impl.out3[0][1].nest.mySeq1
[ 20] proven          - _map_output_out3[1][0]_nest_mySeq1 - spec.out3[1][0].nest.mySeq1 == impl.out3[1][0].nest.mySeq1
[ 21] proven          - _map_output_out3[1][1]_nest_mySeq1 - spec.out3[1][1].nest.mySeq1 == impl.out3[1][1].nest.mySeq1
```

Flow with seq_split_struct_array_port = false and seq_split_multi_dim_array_port = false

Neither struct port 'out1' nor non-struct array port 'out2' nor array of struct port 'out3' gets split.

```
[  6] proven          - _map_output_out1 - spec.out1 == impl.out1
[  7] proven          - _map_output_out2 - spec.out2 == impl.out2
[  8] proven          - _map_output_out3 - spec.out3 == impl.out3
```

Flow with seq_split_struct_array_port = true and seq_split_multi_dim_array_port = true

The struct port 'out1', non-struct array port 'out2' and array of struct port 'out3' get split:

Feedback

```
[  6] proven      - _map_output_out1_mySeq1 - spec.out1.mySeq1 == impl.out1.mySeq1
[  7] proven      - _map_output_out1_nest_mySeq - spec.out1.nest.mySeq == impl.out1.nest.mySeq
[  8] proven      - _map_output_out1_nest_mySeq1 - spec.out1.nest.mySeq1 == impl.out1.nest.mySeq1
[  9] proven      - _map_output_out2[0] - spec.out2[0] == impl.out2[0]
[ 10] proven      - _map_output_out2[1] - spec.out2[1] == impl.out2[1]
[ 11] proven      - _map_output_out2[2] - spec.out2[2] == impl.out2[2]
[ 12] proven      - _map_output_out2[3] - spec.out2[3] == impl.out2[3]
[ 13] proven      - _map_output_out2[4] - spec.out2[4] == impl.out2[4]
[ 14] proven      - _map_output_out2[5] - spec.out2[5] == impl.out2[5]
[ 15] proven      - _map_output_out2[6] - spec.out2[6] == impl.out2[6]
[ 16] proven      - _map_output_out2[7] - spec.out2[7] == impl.out2[7]
[ 17] proven      - _map_output_out3[0][0]_mySeq1 - spec.out3[0][0].mySeq1 == impl.out3[0][0].mySeq1
[ 18] proven      - _map_output_out3[0][1]_mySeq1 - spec.out3[0][1].mySeq1 == impl.out3[0][1].mySeq1
[ 19] proven      - _map_output_out3[1][0]_mySeq1 - spec.out3[1][0].mySeq1 == impl.out3[1][0].mySeq1
[ 20] proven      - _map_output_out3[1][1]_mySeq1 - spec.out3[1][1].mySeq1 == impl.out3[1][1].mySeq1
[ 21] proven      - _map_output_out3[0][0]_nest_mySeq - spec.out3[0][0].nest.mySeq == impl.out3[0][0].nest.mySeq
[ 22] proven      - _map_output_out3[0][1]_nest_mySeq - spec.out3[0][1].nest.mySeq == impl.out3[0][1].nest.mySeq
[ 23] proven      - _map_output_out3[1][0]_nest_mySeq - spec.out3[1][0].nest.mySeq == impl.out3[1][0].nest.mySeq
[ 24] proven      - _map_output_out3[1][1]_nest_mySeq - spec.out3[1][1].nest.mySeq == impl.out3[1][1].nest.mySeq
[ 25] proven      - _map_output_out3[0][0]_nest_mySeq1 - spec.out3[0][0].nest.mySeq1 == impl.out3[0][0].nest.mySeq1
[ 26] proven      - _map_output_out3[0][1]_nest_mySeq1 - spec.out3[0][1].nest.mySeq1 == impl.out3[0][1].nest.mySeq1
[ 27] proven      - _map_output_out3[1][0]_nest_mySeq1 - spec.out3[1][0].nest.mySeq1 == impl.out3[1][0].nest.mySeq1
[ 28] proven      - _map_output_out3[1][1]_nest_mySeq1 - spec.out3[1][1].nest.mySeq1 == impl.out3[1][1].nest.mySeq1
```

Flow with seq_split_struct_array_port = false and seq_split_multi_dim_array_port = true

The struct port 'out1' and array of struct port 'out3' do not get split. However, the non-struct array port 'out2' does get split.

```
[  6] proven      - _map_output_out1 - spec.out1 == impl.out1
[  7] proven      - _map_output_out2[0] - spec.out2[0] == impl.out2[0]
[  8] proven      - _map_output_out2[1] - spec.out2[1] == impl.out2[1]
[  9] proven      - _map_output_out2[2] - spec.out2[2] == impl.out2[2]
[ 10] proven      - _map_output_out2[3] - spec.out2[3] == impl.out2[3]
[ 11] proven      - _map_output_out2[4] - spec.out2[4] == impl.out2[4]
[ 12] proven      - _map_output_out2[5] - spec.out2[5] == impl.out2[5]
[ 13] proven      - _map_output_out2[6] - spec.out2[6] == impl.out2[6]
[ 14] proven      - _map_output_out2[7] - spec.out2[7] == impl.out2[7]
[ 15] proven      - _map_output_out3 - spec.out3 == impl.out3
```

Creating SEQ Properties on the Fly

Use the `seq_assert` and `seq_assume` commands to create properties on the fly between the specified signal pair. These commands are unlike the `seqmap` command which only creates an internal mapping, and the property gets created from the following `map_by_name`.

The `seq_assert` and `seq_assume` commands take a signal pair as an argument, unlike the `seqmap` command which takes a prefix pair as its argument.

When `seq_assert`/`seq_assume` is given after `map_by_name`, ensure that both the signals are unmapped. If the signals are already mapped, the `seq_assert`/`seq_assume` commands will not work.

Feedback

14.3.2 Creating SEQ Properties on the Fly

Creating Assume Mappings

Syntax

Use the `seq_assume` command to create assume mapping between two signals. By default, the command creates environment constraints.

```
vcf> seq_assume -help
Usage: seq_assume    # Create assume mapping between two signals for SEQ.
        [-align_mismatch_bitwidth <alignType>]
                                (To align bits if bitwidth mismatches:
                                 Values: lsb, msb, none)
        [-noenv]                (Do not create -env assumptions.)
        <Signal1>               (First signal)
        <Signal2>               (Second signal)
```

Examples

- ```
  vcf> seq_assume spec.in1 impl.in1 -noenv
  [Info] SEQ_PROPERTY_CREATED: Assume property created between spec.in1
   and impl.in1
  1
  ```

- ```
  vcf> seq_assume spec.in2 impl.in2
  [Info] SEQ_PROPERTY_CREATED: Assume property created between spec.in2
   and impl.in2
  1
  ```

- ```
  vcf> report_seq_mappings -map_type input -type mapped -verbose
  ----------------------------------------------------------------
  SEQ Mappings report
  ----------------------------------------------------------------
  Task Name : SEQ
  ----------------------------------------------------------------
                       |  MAPPED        |  NOT MAPPED
  ----------------------------------------------------------------
    PRIMARY INPUT      |  2             |  0
  ----------------------------------------------------------------
  ----------------------------------------------------------------
  Primary Input Mappings report ( Number of Mappings = 2 )
  ----------------------------------------------------------------
       > ID: [0]
        - Property Name  : _map_input_seq_assume_in1
        - Type           : Primary Input
        - Signal1        : spec.in1
        - Signal2        : impl.in1
        - BitWidth       : 8
        - Usage          : assume
        - Expression     : spec.in1 == impl.in1
  ```
  ```

Feedback

```
    > ID: [1]
     - Property Name  : _map_input_seq_assume_in2
     - Type           : Primary Input
     - Signal1        : spec.in2
     - Signal2        : impl.in2
     - BitWidth       : 8
     - Usage          : envconstraint
     - Expression     : spec.in2 == impl.in2

-----------------------------------------------------------------
```

_map_input_seq_assume_in1 is an assume while *_map_input_seq_assume_in2*is an environment constraint.

- To align signals when they have different bit widths, use the following commands. For example, say, *spec.in1* and *spec.in2* are 6-bit signals (say 5:0), while impl.in1 and impl.in2 are 11-bit signals (say 10:0).

  ```
  vcf> seq_assume spec.in1 impl.in2 -align_mismatch_bitwidth lsb
  [Info] SEQ_PROPERTY_CREATED: Assume property created between
   spec.in1 and impl.in2
  1
  ```

  ```
  vcf> seq_assume spec.in2 impl.in1 -align_mismatch_bitwidth msb
  [Info] SEQ_PROPERTY_CREATED: Assume property created between
   spec.in2 and impl.in1
  1
  ```

  ```
  vcf> report_seq_mappings -map_type input -type mapped -list
  ------------------------------------------------------------------------
  SEQ Mappings report
  ------------------------------------------------------------------------
  Task Name : SEQ
  ------------------------------------------------------------------------
                       |  MAPPED        |  NOT MAPPED
  ------------------------------------------------------------------------
    PRIMARY INPUT      |  2             |  0
  ------------------------------------------------------------------------
  ------------------------------------------------------------------------
  Primary Input Mappings report ( Number of Mappings = 2 )
  ------------------------------------------------------------------------
          Id | BitWidth |  Mappings
  ------------------------------------------------------------------------
           0 |        6 |  spec.in1 == impl.in2[5:0]
           1 |        6 |  spec.in2 == impl.in1[10:5]
  ------------------------------------------------------------------------
  ```

Creating Assert Mappings

Use the `seq_assert` command to create assert mapping between two signals.

Syntax

Feedback

```
vcf> seq_assert -help
Usage: seq_assert    # Create assert mapping between two signals for SEQ.
        [-clock <clockName>]    (The clock to create the concurrent
                                 assertions)
        [-edge <edgeType>]      (Type of clock edge (default: posedge):
                                 Values: posedge, negedge, bothedge)
        [-align_mismatch_bitwidth <alignType>]
                                (To align bits if bitwidth mismatches:
                                 Values: lsb, msb, none)
        [-use_case_equality]    (Use case equality (===) instead
                                 of equality (==) for outputs)
        [-condition <Condition>]
                                (Condition expression for the equality)
        [-latency1 <Latency>]   (List of latency for the first signal)
        [-latency2 <Latency>]   (List of latency for the second signal)
        <Signal1>               (First signal)
        <Signal2>               (Second signal)
```

Examples

- To create an immediate assertion between a signal pair, use the `seq_assert` as follows:

```
vcf> seq_assert spec.out3 impl.out7
[Info] SEQ_PROPERTY_CREATED: Assert property created between spec.out3
 and impl.out7
1
vcf> seq_assert spec.out1 impl.out1
[Info] SEQ_PROPERTY_CREATED: Assert property created between spec.out1
 and impl.out1
1
vcf> report_seq_mappings -map_type output -type mapped -verbose
-------------------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------------------
  PRIMARY OUTPUT     |  2             |  0
-------------------------------------------------------------------
-------------------------------------------------------------------
Primary Output Mappings report ( Number of Mappings = 2 )
-------------------------------------------------------------------
     > ID: [0]
      - Property Name  : _map_output_seq_assert_0
      - Type           : Primary Output
      - Signal1        : spec.out3
      - Signal2        : impl.out7
      - BitWidth       : 2
      - Usage          : assert
      - Expression     : spec.out3 == impl.out7
      - Case Equality  : false
```

Feedback

```
     - Status         : not_run

    > ID: [1]
     - Property Name  : _map_output_seq_assert_out1
     - Type           : Primary Output
     - Signal1        : spec.out1
     - Signal2        : impl.out1
     - BitWidth       : 1
     - Usage          : assert
     - Expression     : spec.out1 == impl.out1
     - Case Equality  : false
     - Status         : not_run

-------------------------------------------------------------------------
```

- To create concurrent assertions, use the `seq_assert` as follows. Use `-latency1` and `-latency2` options to specify *spec* and *impl* latencies.
  - To specify same latency

```
vcf> seq_assert spec.out1 impl.out1 -clock spec.clk -latency1 2
 -latency2 2
[Info] SEQ_PROPERTY_CREATED: Assert property created between
 spec.out1 and impl.out1
1

vcf> report_seq_mappings -map_type output -type mapped -list
-------------------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------------------
  PRIMARY OUTPUT     |  1             |  0
-------------------------------------------------------------------
-------------------------------------------------------------------
Primary Output Mappings report ( Number of Mappings = 1 )
-------------------------------------------------------------------
  Id | BitWidth |  Mappings
-------------------------------------------------------------------
   0 |        1 | @(posedge spec.clk) (##2
                   (spec.out1 == impl.out1))
-------------------------------------------------------------------
```

  - To specify different latencies

```
vcf> seq_assert spec.bout2_spec[2] impl.bout2_impl[2] -clock
 impl.clk -latency1 {10 0 9} -latency2 {4 5 9} -use_case_equality
 -edge negedge -condition spec.in2

[Info] SEQ_PROPERTY_CREATED: Assert property created between
 spec.bout2_spec[2] and impl.bout2_impl[2]
```

Feedback

```
1

vcf> report_seq_mappings -map_type output -type mapped -list
------------------------------------------------------------
SEQ Mappings report
------------------------------------------------------------
Task Name : SEQ
------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
------------------------------------------------------------
  PRIMARY OUTPUT     |  1             |  0

------------------------------------------------------------
------------------------------------------------------------
Primary Output Mappings report ( Number of Mappings = 1 )
----------------------------------------------------------
 Id | BitWidth |  Mappings
----------------------------------------------------------
  0 |        1 | @(negedge impl.clk)
                              spec.in2 |-> ((##10
                              (spec.bout2_spec[2] ===
              $past(impl.bout2_impl[2], 6))))
              or ((##5 (impl.bout2_impl[2] ===
              $past(spec.bout2_spec[2], 5))))
              or (##9 (spec.bout2_spec[2] ===
              impl.bout2_impl[2]))
----------------------------------------------------------
```

- To align signals when they have different bit widths, use the following command. For example, consider spec.out1_diff is 4-bit (say 3:0) and impl.out1_diff is a scalar port.

```
vcf> seq_assert spec.out1_diff impl.out1_diff -align_mismatch_bitwidth
 lsb

[Info] SEQ_PROPERTY_CREATED: Assert property created between
 spec.out1_diff and impl.out1_diff
1

vcf> report_seq_mappings -map_type output -type mapped -list

SEQ Mappings report
------------------------------------------------------------
Task Name : SEQ
------------------------------------------------------------
------------------------------------------------------------

                     |  MAPPED        |  NOT MAPPED
------------------------------------------------------------
  PRIMARY OUTPUT     |  1             |  0
------------------------------------------------------------
------------------------------------------------------------
Primary Output Mappings report ( Number of Mappings = 1 )
```

Feedback

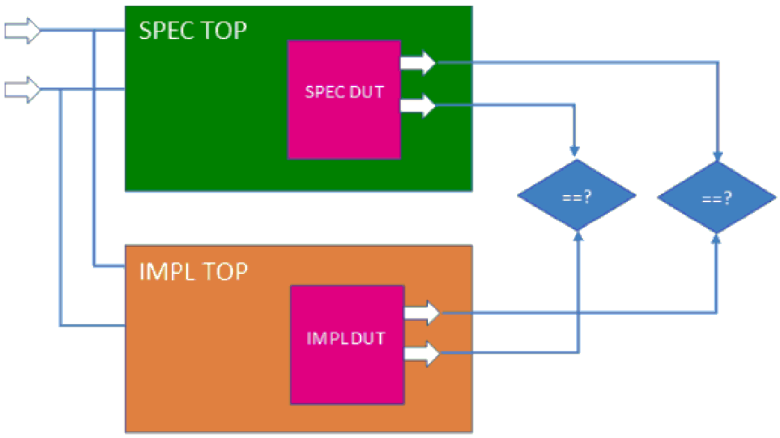
```
-------------------------------------------------------------
        Id | BitWidth |  Mappings
-------------------------------------------------------------
         0 |        1 | spec.out1_diff[0] ==
                       impl.out1_diff
-------------------------------------------------------------
```

14.3.3 SEQ Hierarchical Verification Flow

You can verify submodule of the elaborated top design against the implementation of that submodule.

Configuration 1

In this configuration, the SEQ loads two top designs (Spec. and Impl.) and compare two submodules. The inputs of top modules are mapped (assumptions) and the output of submodules are verified (assertions).




```
map_by_name -input -abstraction -blackbox -undriven
```

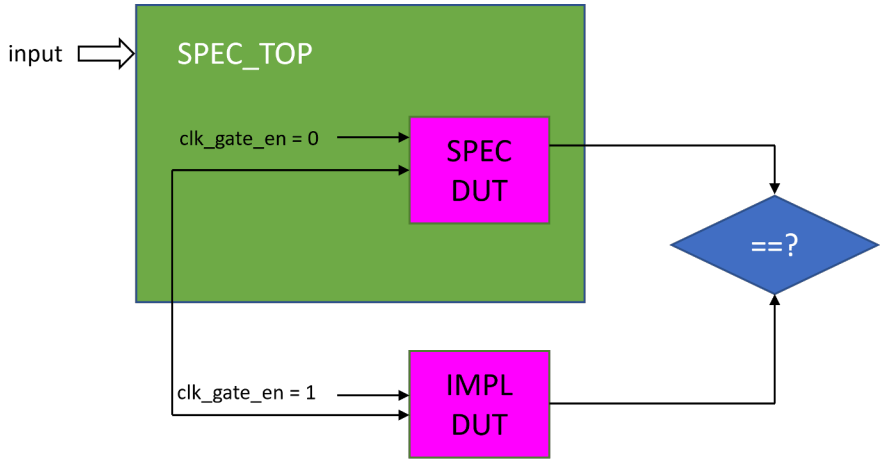
You can implement the following:

```
map_by_name -output -specprefix spec.dut -implprefix impl.dut
```

Configuration 2

In this configuration, the SEQ application loads only one design, and then binds the implementation to the design. The inputs of the DUTs are driven by the same nets so that the constraints to the DUTs are the same. The outputs of the DUTs are verified.

Feedback




```
analyze -format sverilog -vcs " test.v ";
elaborate top
seq_bind <instance_path_name> -instantiation <name>
map_by_name -input -abstraction -blackbox -undriven
map_by_name -output -specprefix top.<instance_path_name> -implprefix
 <name>
```

14.3.4 Setting Up Clocks and Resets

VC Formal SEQ requires a single primary clock, from which other clocks may be derived synchronously. In the mapping step described in section Mapping the Designs, the clock inputs to the two designs have already been mapped, but these clocks must be identified to VC SEQ using a `create_clock` command, for each clock.

Example

- If you have a single clock named *sysclk*, you can define it with the following command:

  ```
  create_clock -period 100 spec.sysclk
  ```

- The actual value of the period is only relevant if you have more than one clock. In that case, use the period to indirectly identify the relative frequencies of the two clocks. For example, that commands,

  ```
  create_clock -period 100 spec.sysclk
  create_clock -period 200 spec.auxclk
  . . .
  ```

  Identifies two clocks *sysclk* and *auxclk* with the former having twice the frequency of the latter.

  When you have more than one clock, the only constraint is that the periods of all the clocks must have a least common multiple.

Note:

In these examples, spec.sysclk and spec.auxclk has been used. Instead, impl.sysclk and impl.auxclk can be used, provided that those signal pairs have been mapped using the procedures described in section Mapping the Designs.

- Additionally, specify the reset with the following command, assuming that the reset signal in the *spec* design is named *rst*:

```
create_reset spec.rst -sense [low|high]
```

Once again, ensure that you have mapped `spec.rst` to the corresponding reset signal in your implementation design.

14.3.5 Setting Up Initial State

Once the signals are mapped, and the resets and clocks have been defined, it is necessary to run VC Formal SEQ for some number of clock cycles (from reset), until it reaches a stable state. You can then record that stable state as the initial state from which subsequent cycle-by-cycle comparisons can be done. This is done by using the following commands:

```
sim_run [-stable|N]
sim_set_state...
sim_save_reset
```

Note:
For more details, see the Specifying the Initial State section.

14.3.6 Getting a Report of Mappings

SEQ automatically makes initial hint register mappings based on name matching between the designs. These mappings are used by the solvers to guide convergence. They are not necessary for correctness, however, they have a significant impact on the convergence for hard designs. Providing accurate mapping information will guide the solvers in the right direction.

While you are setting up the design, you can view the mapping using the `report_seq_mappings/get_seq_mappings`command, if required, you can refine the mappings using the `seqmap`/`sequnmap` commands.

The get_seq_mappings command

The `get_seq_mappings` command returns a collection of the mapping information. You can apply various filters to get the appropriate information using the options in the command. You can also create various scripts based on the mapping information.

Feedback

Syntax

```
vcf> get_seq_mappings -help
Usage: get_seq_mappings    # Gets information about various SEQ mappings
                             as a tcl collection
        [-type <type-attributes>](Mapping type selection:
                                  Values: mapped, not_mapped)
        [-map_type <type-list>] (Mapped object type selection:
                                  Values: input, output,
                                  bbin, bbout, absin, absout,
                                  sequential, undriven, unloaded,
                                  explicit_x, undefined_x, net,
                                  auto, all)
        [-skip_map_type <type-list>] (List of object types
                                  to be skipped from mapping:
                                  Values: input, output, bbin,
                                  bbout, absin, absout, sequential,
                                  undriven, unloaded, explicit_x,
                                  undefined_x, net)
        [-clear_mem]             (Clear all previous collections generated
                                  through preceding 'get_seq_mappings'
                                  commands, if any)
```

Supported Attributes

The following are the attributes supported on the objects returned in collection of '`get_seq_mappings`'. The following is an example snippet of the output.

```
vcf> list_attributes -application -class mapping_object
Properties:
    A - Application-defined
    U - User-defined
    I - Importable from design/library (for user-defined)
    S - Settable
    B - Subscripted

Attribute Name            Object     Type       Properties  Constraints
-------------------------------------------------------------------------
-------
absin                     mapping_object
                                     boolean    A
absout                    mapping_object
                                     boolean    A
bbin                      mapping_object
                                     boolean    A
bbout                     mapping_object
                                     boolean    A
case_equality             mapping_object
                                     boolean    A
clock                     mapping_object
                                     string     A
condition                 mapping_object
```

Feedback

```
                                    string     A
edge                        mapping_object
                                    string     A
explicit_x                  mapping_object
                                    boolean    A
full_name                   mapping_object
                                    string     A
impl_latency                mapping_object
                                    string     A
input                       mapping_object
                                    boolean    A
mapped                      mapping_object
                                    boolean    A
mapped_signal               mapping_object
                                    string     A
name                        mapping_object
                                    string     A
net                         mapping_object
                                    boolean    A
not_mapped                  mapping_object
                                    boolean    A
not_mapped_reason           mapping_object
                                    string     A
output                      mapping_object
                                    boolean    A
prop_expr                   mapping_object
                                    string     A
prop_name                   mapping_object
                                    string     A
seq_assert                  mapping_object
                                    boolean    A
seq_assume                  mapping_object
                                    boolean    A
sequential                  mapping_object
                                    boolean    A
spec_latency                mapping_object
                                    string     A
status                      mapping_object
                                    string     A
taskid                      mapping_object
                                    int        A
undefined_x                 mapping_object
                                    boolean    A
undriven                    mapping_object
                                    boolean    A
unloaded                    mapping_object
                                    boolean    A
usage                       mapping_object
                                    string     A
width                       mapping_object
                                    int        A
```

Feedback

The following attributes are currently supported only on properties created through `seq_assert` command.

clock, edge, condition, spec_latency, impl_latency and case_equality

Querying these for signals from properties created using `map_by_name` and `seq_assume` and so on returns an empty value.

Examples

- When the `get_seq_mappings` command is run before `map_by_name` command, a warning message is reported:

```
vcf> get_seq_mappings
[Warning] SEQ_RMAP_NO_MAP_BY_NAME_DONE: Command 'map_by_name' is not
 run yet.
Please run 'get_seq_mappings' after 'map_by_name'.
```

- The following example shows default output of the `get_seq_mappings` command.

```
vcf> get_seq_mappings
{"spec.in2", "impl.in2", "spec.in3", "impl.in3", "spec.in7_ex",
 "impl.in7_ex", "spec.clk", "impl.clk", "spec.rst", "impl.rst",
 "spec.in1", "spec.in6", "impl.in6", "spec.temp", "spec.out2",
 "impl.out2", "spec.out5_und", "impl.out5_und", "spec.out7_ex",
 "impl.out7_ex", "impl.temp", "spec.out1", "spec.out3_s", "spec.out6",
 "impl.out6", "impl.out3", "spec.out5_und", "impl.out5_und",
 "impl.temp"}
```

- The following example shows the output with the `-type` and `-map_type` options:

```
vcf> get_seq_mappings -type mapped
{"spec.in2", "impl.in2", "spec.in3", "impl.in3", "spec.in7_ex",
 "impl.in7_ex", "spec.clk", "impl.clk", "spec.rst", "impl.rst",
 "spec.out2", "impl.out2", "spec.out5_und", "impl.out5_und",
 "spec.out7_ex", "impl.out7_ex", "spec.out5_und", "impl.out5_und"}

vcf> get_seq_mappings -type mapped -map_type sequential
{"spec.out7_ex", "impl.out7_ex", "spec.out2", "impl.out2"}
```

- When the `-clear_mem` option is used, the previous collection(s) are deleted, and the new collection is created according to the specified options.

```
vcf> get_seq_mappings -clear_mem
{"spec.in2", "impl.in2", "spec.in3", "impl.in3", "spec.in7_ex",
 "impl.in7_ex", "spec.clk", "impl.clk", "spec.rst", "impl.rst",
 "spec.in1", "spec.in6", "impl.in6", "spec.temp", "spec.out2",
 "impl.out2", "spec.out5_und", "impl.out5_und", "spec.out7_ex",
 "impl.out7_ex", "impl.temp", "spec.out1", "spec.out3_s", "spec.out6",
 "impl.out6", "impl.out3", "spec.out5_und", "impl.out5_und",
 "impl.temp"}
```

Feedback

- The following example shows various type of filtering, using `filter_collection` and `foreach_in_collection`.

```
vcf> set col [get_seq_mappings]
{"spec.in2", "impl.in2", "spec.in3", "impl.in3", "spec.in7_ex",
 "impl.in7_ex", "spec.clk", "impl.clk", "spec.rst", "impl.rst",
 "spec.in1", "spec.in6", "impl.in6", "spec.temp", "spec.out2",
 "impl.out2", "spec.out5_und", "impl.out5_und", "spec.out7_ex",
 "impl.out7_ex", "impl.temp", "spec.out1", "spec.out3_s", "spec.out6",
 "impl.out6", "impl.out3", "spec.out5_und", "impl.out5_und",
 "impl.temp"}

vcf> filter_collection $col {status==proven}
{"spec.out5_und", "impl.out5_und"}

vcf> filter_collection $col {status==falsified}
{"spec.out2", "impl.out2", "spec.out7_ex", "impl.out7_ex"}

vcf> filter_collection $col {status==falsified && full_name=~spec.*}
{"spec.out2", "spec.out7_ex"}

vcf> filter_collection $col {mapped}
{"spec.in2", "impl.in2", "spec.in3", "impl.in3", "spec.in7_ex",
 "impl.in7_ex", "spec.clk", "impl.clk", "spec.rst", "impl.rst",
 "spec.out2", "impl.out2", "spec.out5_und", "impl.out5_und",
 "spec.out7_ex", "impl.out7_ex", "spec.out5_und", "impl.out5_und"}

vcf> filter_collection $col {output && status==proven}
{"spec.out5_und", "impl.out5_und"}

vcf> set col1 [filter_collection $col {not_mapped}]
{"spec.in1", "spec.in6", "impl.in6", "spec.temp", "impl.temp",
 "spec.out1", "spec.out3_s", "spec.out6", "impl.out6", "impl.out3",
 "impl.temp"}

vcf> foreach_in_collection obj $col1 { puts "not mapped reason
 for [get_attribute $obj full_name] = [get_attribute $obj
 not_mapped_reason]" }
not mapped reason for 'spec.in1' is 'Not found'
not mapped reason for 'spec.in6' is 'Different bitwidth'
not mapped reason for 'impl.in6' is 'Different bitwidth'
not mapped reason for 'spec.temp' is 'Different direction'
not mapped reason for 'impl.temp' is 'Different direction'
not mapped reason for 'spec.out1' is 'Not found'
not mapped reason for 'spec.out3_s' is 'Not found'
not mapped reason for 'spec.out6' is 'Different bitwidth'
not mapped reason for 'impl.out6' is 'Different bitwidth'
not mapped reason for 'impl.out3' is 'Not found'
not mapped reason for 'impl.temp' is 'Undriven not exist or driven'
```

Feedback

The report_seq_mappings Command

Use the `report_seq_mappings` command to get information about various SEQ mappings. The `report_seq_mappings` command provides the mappings without any status.

When the `-backend` switch is specified in the `report_seq_mappings` command, the command lists the mapped pairs and the unmapped sequentials.

In the legacy flow, when the `-backend` switch is not specified, the `report_seq_mappings` lists the mapping accepted by the `seqmap` and `sequnmap` commands. In the default flow, only the summary is dumped for the various map types.

Note that SEQ uses these mappings as hints, as such they do not impact correctness, however, wrong mappings or less mappings can significantly degrade performance.

The `-map_type all` option reports the *mapped*/*not_mapped* information for all the objects `-input, output, bbin, bbout, absin, absout, sequential, undriven, unloaded, explicit_x, undefined_x, net`. The `-map_type auto` option reports the *mapped*/*not_mapped* information for the objects that is, `input, output, bbin, bbout, absin, absout` and `undriven`.

When the `seq_add_unloaded_assert` formal variable is set to `true`, *mapped*/*not_mapped* information for `unloaded` is also reported.

Syntax

```
vcf> report_seq_mappings -help
Usage: report_seq_mappings    # Reports information about various
                                SEQ mappings
        [-type <type-attributes>](Mapping type selection:
                                Values: mapped, not_mapped)
        [-map_type <signal-type>](Mapped object type selection:
                                Values: input, output, bbin, bbout,
                                absin, absout, sequential, undriven,
                                unloaded, explicit_x, undefined_x, net,
                                auto, all)
        [-skip_map_type <type-list>] (List of object types to be skipped
                                from mapping: Values: input, output,
                                bbin, bbout, absin, absout,
                                sequential, undriven, unloaded,
                                explicit_x, undefined_x, net)
        [-backend]              (Mappings effective in the backend)
        [-list]                 (Outputs one-line report per mapping)
        [-verbose]              (Outputs detailed report of each mapping)
        [-no_summary]           (Skip summary report of various
                               map_by_name commands)
        [-sort <sortType>]      (Sort output on basis of bitwidth as per
                               given selection: Values: ascending,
                               descending)
```

Feedback

Example

An example output as shown as Figure 93:

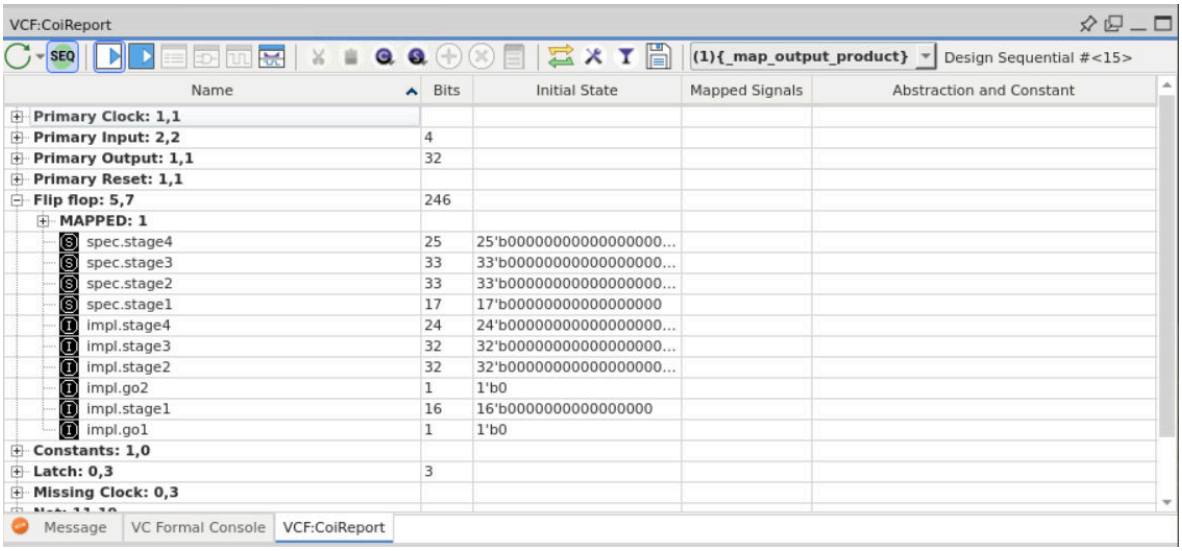
Figure 93: Output for report_seq_mappings -backend

```
vcf> report_seq_mappings -backend
-----------------------------------------------------
Intial register mappings found based on name matching.
 - Note these are hints and not assumed to hold.
-----------------------------------------------------
[Mapped]                  - impl.go1(1) == spec.go1(1)
[Mapped]                  - impl.go2(1) == spec.go2(1)
[Mapped]                  - impl.go3(1) == spec.go3(1)
[Mapped]                  - impl.go4(1) == spec.go4(1)
[Mapped]                  - impl.product(16) == spec.product(16)
[Mapped]                  - impl.stage1(16) == spec.stage1(16)
[Mapped]                  - impl.stage2(32) == spec.stage2(32)
[Mapped]                  - impl.stage3(32) == spec.stage3(32)
[Mapped]                  - impl.stage4(24) == spec.stage4(24)
[Mapped]                  - impl.valid(1) == spec.valid(1)
[Not Mapped] - Not Found - impl.cg2.en_out(1)
[Not Mapped] - Not Found - impl.cg3.en_out(1)
[Not Mapped] - Not Found - impl.cg4.en_out(1)
-----------------------------------------------------
Total (Mapped 10) (Not mapped 3)
1
```

Note that the number of internal equivalences is the same as the mapped registers in the output of the `report_seq_mappings -backend` command as shown in Figure 93.

In VC Formal SEQ, complexity (COI) report can be seen for the designs or for the properties as well. Unlike most of the VC Formal apps, since SEQ has 2 designs, the complexity report shows elements from both the designs. If the same element exists in both the designs, they appear under MAPPED section otherwise outside the MAPPED.

Figure 94: SEQ Complexity (COI) Report



Feedback

- If `report_seq_mappings` is run without executing `map_by_name`/`seq_assert`/`seq_assume` commands, then the following warning message is reported:

```
vcf> report_seq_mappings
```

[Warning] SEQ_RMAP_NO_MAP_BY_NAME_DONE: Command 'map_by_name' is not run yet. Please run 'report_seq_mappings' after 'map_by_name'. 0

- If there are valid mappings, then `report_seq_mappings` gives only the summary of the mappings when given without any arguments:-

```
vcf> report_seq_mappings
-------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------
  PRIMARY INPUT      |  4             |  4
  PRIMARY OUTPUT     |  3             |  5
  BLACKBOX INPUT     |  0             |  0
  BLACKBOX OUTPUT    |  0             |  0
  CMA INPUT          |  0             |  0
  CMA OUTPUT         |  0             |  0
  ABSTRACTION INPUT  |  0             |  0
  ABSTRACTION OUTPUT |  0             |  0
  UNDRIVEN           |  1             |  1
-------------------------------------------------------
1
```

- The `-map_type` option can be used to get the mapped/not_mapped info of the specific signal type. The following is an example for obtaining *Primary Input* mappings in the list format:

```
vcf> report_seq_mappings -map_type input -list
-------------------------------------------------------------
SEQ Mappings report
--Task Name : SEQ
-------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------------
  PRIMARY INPUT      |  4             |  4
-------------------------------------------------------------
-------------------------------------------------------------
Primary Input Mappings report ( Number of Mappings = 4 )
-------------------------------------------------------------
        Id | BitWidth |  Mappings
-------------------------------------------------------------
         0 |        1 | spec.in1 == impl.in3
         1 |        1 | spec.in2 == impl.in2
```

Feedback

```
         2 |         1 | spec.clk == impl.clk
         3 |         1 | spec.rst == impl.rst
-------------------------------------------------------------
-------------------------------------------------------------
Primary Input Not Mapped report ( Number of Not mapped Signals = 4 )
-------------------------------------------------------------
  Id | Reason              | BitWidth | Signal
-------------------------------------------------------------------------
   0 | Different bitwidth  |  1       | spec.in6
   1 | Different bitwidth  |  2       | impl.in6
   2 | Excluded            |  1       | spec.in7_ex
   3 | Different direction |  1       | spec.temp
-------------------------------------------------------------------------
1
```

- The `-skip_map_type` option can be used to skip the reporting for a particular map object type.

  The following is an example to skip the reporting for abstraction inputs/outputs:

```
vcf> report_seq_mappings -skip_map_type {absin absout}
-------------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------------
  PRIMARY INPUT      |  6             |  2
  PRIMARY OUTPUT     |  2             |  1
  BLACKBOX INPUT     |  2             |  1
  BLACKBOX OUTPUT    |  3             |  0
  UNDRIVEN           |  0             |  1
-------------------------------------------------------------
```

- It is mandatory to provide a format specifier, use either `-list/-verbose` to enable dumping of the report. If no format specifier is provided, only summary will be dumped.

```
vcf> report_seq_mappings -map_type {input output}
-----------------------------------------------------------
SEQ Mappings report
-----------------------------------------------------------
Task Name : SEQ
-----------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-----------------------------------------------------------
  PRIMARY INPUT      |  2             |  1
  PRIMARY OUTPUT     |  2             |  2
-----------------------------------------------------------
```

- Use the `-type` option to filter the summary and the report to get only the `mapped/not_mapped` information.

Feedback

```
vcf> report_seq_mappings -map_type {bbin bbout}
                             -type not_mapped -list
-------------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------
                     |  NOT MAPPED
-------------------------------------------------------------
BLACKBOX INPUT       |  1
BLACKBOX OUTPUT      |  1
CMA INPUT            |  0
CMA OUTPUT           |  0

-------------------------------------------------------------
-------------------------------------------------------------
BlackBox Input Not Mapped report ( Number of Not mapped Signals = 1 )
-------------------------------------------------------------------
        Id |     Reason | BitWidth |  Signal
-------------------------------------------------------------------
         0 |  Not found |     1    | spec.count_mod.bbin1
-------------------------------------------------------------------
-------------------------------------------------------------------
BlackBox Output Not Mapped report ( Number of Not mapped Signals = 1 )
-------------------------------------------------------------------
        Id |   Reason   | BitWidth |  Signal
-------------------------------------------------------------------
         0 | Not found |        2 | spec.count_mod.bbout1
-------------------------------------------------------------------------
-------------------------------------------------------------------
CMA Input Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
CMA Output Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
```

- The `-no_summary` option along with `-list/-verbose` can be used to skip the summary and get only the report dumped.

```
vcf> report_seq_mappings -map_type {undriven} -list -no_summary
SEQ Mappings report
-------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------
-------------------------------------------------------------
Undriven Mappings report ( Number of Mappings = 1 )
-------------------------------------------------------------
        Id | BitWidth |  Mappings
-------------------------------------------------------------
         0 |        1 | spec.out1 == impl.out1
-------------------------------------------------------------
-------------------------------------------------------------
```

Feedback

```
Undriven Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------
-------------------------------------------------------------
```

- If neither `-list` nor `-verbose` is specified, and`-no_summary` is given, then there will be no output. At least one of `-list` or -verbose is mandatory to get the report.

```
vcf> report_seq_mappings -map_type {input output} -no_summary
1
```

- The `-verbose` option can be used to get more detailed output. The following is an example of `-verbose` usage for obtaining *Undriven* mappings.

```
vcf> report_seq_mappings -map_type undriven -verbose
-------------------------------------------------------------
SEQ Mappings report
-------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-------------------------------------------------------------
  UNDRIVEN           |  1             |  1
-------------------------------------------------------------
-------------------------------------------------------------
Undriven Mappings report ( Number of Mappings = 1 )
-------------------------------------------------------------
     > ID: [0]
      - Property Name  : _map_undriven_out5_und
      - Type           : Undriven
      - Signal1        : spec.out5_und
      - Signal2        : impl.out5_und
      - BitWidth       : 1
      - Usage          : envconstraint
      - Expression     : spec.out5_und == impl.out5_und

-------------------------------------------------------------
-------------------------------------------------------------
Undriven Not Mapped report  Number of Not mapped Signals = 1 )
-------------------------------------------------------------
     > ID: [0]
      - Type           : Undriven
      - Signal         : impl.temp
      - BitWidth       : 1
      - Reason         : Undriven not exist or driven
-------------------------------------------------------------
```

- In case of blackbox with unloaded output ports, the not mapped reason *Unloaded* is reported in the *BlackBox Output Not Mapped* report.

```
vcf> report_seq_mappings -map_type bbout -type not_mapped -list
                                                    -no_summary
-------------------------------------------------------------------------
SEQ Mappings report
```

Feedback

```
-------------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------------
BlackBox Output Not Mapped report ( Number of Not mapped Signals = 4 )
-------------------------------------------------------------------
Id |          Reason       |BitWidth |  Signal
-------------------------------------------------------------------
 0 | Unloaded              |   1     | spec.ecodonor0.out
 1 | Unloaded              |   1     | impl.ecodonor0.out
 2 | Sequnmap              |   1     | spec.ecodonor4.out
 3 | Sequnmap              |   1     | impl.ecodonor4.out
-------------------------------------------------------------------
-------------------------------------------------------------------
```

- The `map_type input output bbin bbout absin absout undriven` is enabled by default. When you use just `report_seq_mappings -list`, information of all the `map_type` is obtained:

```
vcf> report_seq_mappings -list
-----------------------------------------------------------
SEQ Mappings report
-----------------------------------------------------------
-----------------------------------------------------------
                     |  MAPPED        |  NOT MAPPED
-----------------------------------------------------------
  PRIMARY INPUT      |  4             |  4
  PRIMARY OUTPUT     |  3             |  5
  BLACKBOX INPUT     |  0             |  0
  BLACKBOX OUTPUT    |  0             |  0
  CMA INPUT          |  0             |  0
  CMA OUTPUT         |  0             |  0
  ABSTRACTION INPUT  |  0             |  0
  ABSTRACTION OUTPUT |  0             |  0
  UNDRIVEN           |  1             |  1
-----------------------------------------------------------
-----------------------------------------------------------
Primary Input Mappings report ( Number of Mappings = 4 )
-----------------------------------------------------------
        Id | BitWidth |  Mappings
-----------------------------------------------------------
         0 |        1 |  spec.in1 == impl.in3
         1 |        1 |  spec.in2 == impl.in2
         2 |        1 |  spec.clk == impl.clk
         3 |        1 |  spec.rst == impl.rst
-----------------------------------------------------------
-----------------------------------------------------------
Primary Input Not Mapped report ( Number of Not mapped Signals = 4 )
-------------------------------------------------------------------------
        Id |                Reason | BitWidth |  Signal
-------------------------------------------------------------------------
         0 |    Different bitwidth |        1 | spec.in6
         1 |    Different bitwidth |        2 | impl.in6
         2 |              Excluded |        1 | spec.in7_ex
```

Feedback

```
         3 |   Different direction |        1 | spec.temp
-------------------------------------------------------------------
-------------------------------------------------------------------
Primary Output Mappings report ( Number of Mappings = 3 )
-------------------------------------------------------------------
        Id | BitWidth |  Mappings
-------------------------------------------------------------------
         0 |        1 | spec.out1 == impl.out3
         1 |        1 | spec.out2 == impl.out2
         2 |        1 | spec.out5_und == impl.out5_und
-------------------------------------------------------------------
Primary Output Not Mapped report ( Number of Not mapped Signals = 5 )
-------------------------------------------------------
  Id |                Reason | BitWidth |  Signal
-------------------------------------------------------------
   0 |   Different direction |        1 | impl.temp
   1 |             Not found |        1 | spec.out3_s
   2 |    Different bitwidth |        1 | spec.out6
   3 |    Different bitwidth |        2 | impl.out6
   4 |              Excluded |        1 | spec.out7_ex
-------------------------------------------------------------
-------------------------------------------------------------
BlackBox Input Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
BlackBox Input Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
BlackBox Output Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
BlackBox Output Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
CMA Input Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
CMA Input Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
CMA Output Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
CMA Output Not Mapped report ( Number of Not mapped Signals = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
Abstraction Input Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
-------------------------------------------------------------------
Abstraction Input Not Mapped report(Number of Not mapped Signals = 0)
-------------------------------------------------------------------
-------------------------------------------------------------------
Abstraction Output Mappings report ( Number of Mappings = 0 )
-------------------------------------------------------------------
```

Feedback

```
-------------------------------------------------------------------
Abstraction Output Not Mapped report(Number of Not mapped Signals = 0)
-------------------------------------------------------------------
-------------------------------------------------------------------
Undriven Mappings report ( Number of Mappings = 1 )
-------------------------------------------------------------
  Id | BitWidth |  Mappings
-------------------------------------------------------------
   0 |        1 | spec.out5_und == impl.out5_und
-------------------------------------------------------------
-------------------------------------------------------------
Undriven Not Mapped report ( Number of Not mapped Signals = 1 )
-------------------------------------------------------------------
 Id | Reason                        | BitWidth |  Signal
-------------------------------------------------------------------
  0 | Undriven not exist or driven |       1 | impl.temp
-------------------------------------------------------------------
1
```

Similarly, using the `report_seq_mappings -verbose` gives detailed verbose output for the default map_types.

- Using `-backend` with any of the options like`-map_type`, `-list`,`-verbose`, or `-no_summary` is an error as follows:

```
vcf> report_seq_mappings -backend -map_type output
Error: Cannot specify '-backend' with '-map_type'. (CMD-001)

vcf> report_seq_mappings -backend -list
Error: Cannot specify '-backend' with '-list'. (CMD-001)

vcf> report_seq_mappings -backend -verbose
Error: Cannot specify '-backend' with '-verbose'. (CMD-001)

vcf> report_seq_mappings -backend -no_summary
Error: Cannot specify '-backend' with '-no_summary'. (CMD-001)
```

Examples for Mapping and Unmapping Registers

- To provide signal register mappings that are not matched by name, use the following command:

```
seqmap first-signal-name second-signal-name
```

- To map all registers under a pair of instances that are not matched by name, use the following command:

```
seqmap first-instance-name second-instance-name
```

- In the case of instance mapping, all registers under the instances that match by name will be mapped. To exclude a signal register mapping, use the following command:

  ```
  sequnmap first-signal-name second-signal-name
  ```

- To exclude all register mappings under a pair of mapped instances, use the following command:

  ```
  sequnmap first-instance-name second-instance-name
  ```

- To perform an automatic map-by-name support for expressions bringing a list of signals, use the following command:

  ```
  seqmap spec.da* impl.da*
  ```

  Where, *spec.da** represents *{spec.da_a, spec.da_b, spec.da_c}*, and *impl.da** represents {*impl.da_a, impl.da_c, impl.da_d}*. Then the mappings <*spec.da_a, impl.da_a*> and <*spec.da_c, impl.da_c*> are created automatically provided their bit-width matches.

  For signals matching an expression, only the un-drivens and the registers are considered. In addition, un-driven signals map to un-driven ones. Registers map to registers. Mapping across un-driven and registers is not supported.

- To exclude all register mappings for expressions bringing a list of signals, use the following command:

  ```
  sequnmap spec.a impl.a
  ```

  If the expression brings partial signal, then an error message is reported.

  ```
  sequnmap spec.a[7:1] impl.a[6:0]
  ```

  Note:

  The usage of `sequnmap` is either by name or by expression. The following is not a valid combination, for example, `sequnmap -name ABC spec.a impl.b`

- To give a name to the mapping clause, use the following command:

  ```
  seqmap -name map1 spec.a_(\\w+) impl.b_\\1
  ```

  If you do not provide a name, then SEQ names the clauses automatically. If a clause brings more than one pairs, a subscription is appended to the names. For instance, suppose *spec.a** returns a list of *{spec.a_d1, spec.a_d2,…}* and impl.b* returns {impl.b_d1,*impl.b** returns *{impl.b_d1, impl.b_d2, …}*, then there will be mapping pairs <*spec.a_d1, impl.b_d1*> with a name *map1_1*, and <*spec.a_d2, impl.b_d2*> with a name*map1_2*. And map1 turns to be the automatically-generated name.

Feedback

An error is displayed if a name is reused. Suppose *map1_1* is assigned to a mapping clause before, then you cannot use map1_1 as a user name, like

```
seqmap -name map1_1 spec.ax impl.by
```

Examples for Mapping and Unmapping Ports

- To map the port objects, use the following command:

  ```
  seqmap -port spec.in1 impl.in2
  ```

  When you issue the next `map_by_name` command, it maps the port objects. Even if there is no call to '`map_by_name`', the '`seqmap`' commands will be considered while running '`check_fv`'.

- To specify exact pair of signals instead of prefix pair to be mapped/unmapped, use the `-exact` option in the `seqmap` and `sequnmap` commands respectively.

- Use `sequnmap` command to remove the mapping hints created by `seqmap` command as shown in the below example:

  ```
  vcf> seqmap -port spec.count_inst.bbout1 impl.count_inst.bbout2 -name
   map_b1_b2
  [Info] MAP_FOUND: 1 map(s) is(are) found between
   spec.count_inst.bbout1 and impl.count_inst.bbout2
  1
  vcf> seqmap -port spec.counter_greybox_inst.bbout3
   impl.counter_greybox_inst.bbout4
  [Info] MAP_FOUND: 1 map(s) is(are) found between
   spec.counter_greybox_inst.bbout3 and impl.counter_greybox_inst.bbout4
  1
  vcf> sequnmap -name map_b1_b2
  [Info] UNMAP_NAME_REMOVED: 1 map(s) of name map_b1_b2 is(are) marked
   to be removed in the next map_by_name or check_fv command
  1
  vcf> sequnmap -port spec.counter_greybox_inst.bbout3
   impl.counter_greybox_inst.bbout4
  [Info] UNMAP_REMOVED: 1 map(s) is(are) removed between
   spec.counter_greybox_inst.bbout3 and impl.counter_greybox_inst.bbout4
  1
  ```

- Use the `sequnmap` command to delete the created properties, as shown in the below example:

  ```
  vcf> sequnmap -port spec.counter_greybox_inst.count
   impl.counter_greybox_inst.count
  [Info] SEQ_DELETE_PROPERTY: The property
   _map_bb_counter_greybox_inst_count will be deleted
  [Info] CONSTRAINT_REMOVAL: Removal of constraint
   '_map_bb_counter_greybox_inst_count' will invalidate all proven,
   vacuous and uncoverable results.
  ```

Feedback

```
Falsified, non-vacuous and covered results will remain untouched.
        Subsequent constraint removals will not be reported.
```

- Use the `sequnmap` command to skip a mapping in the subsequent `map_by_name` command.

  For example, say, you want to skip the mapping between the primary input ports *spec.in1* and *impl.in1* in the `map_by_name` command:

```
vcf> sequnmap -port spec.in1 impl.in1
[Warning] SEQ_MAPPED_PAIR_NOT_FOUND: No mapping is found between
 'spec.in1' and 'impl.in1'.
        The mapping will be skipped in subsequent map_by_name or
 check_fv commands.
0

vcf> map_by_name
[Info] MAPPING_ENABLED: input, output, blackbox, undriven,
 abstraction.
[Warning] SEQ_SIGNAL_UNMAP: Primary input ports 'spec.in1' and
 'impl.in1' are not mapped, because they are specified in sequnmap
 command.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'out3' is not mapped by name,
 because it is not found in the 'impl' instance.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'in5' is not mapped by name,
 because it is not found in the 'spec' instance.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'out7' is not mapped by name,
 because it is not found in the 'spec' instance.
[Warning] MAP_BY_NAME_NOT_MAPPED: Port 'bbout1' is not mapped by name,
 because it is not found in the 'impl' instance.
[Info] MBN_SUMMARY: '1' primary inputs are mapped as assumes.
[Info] MBN_SUMMARY: '2' primary outputs are mapped as asserts.
[Info] MBN_SUMMARY: '1' blackbox inputs are mapped as asserts.
[Info] MBN_SUMMARY: '1' blackbox outputs are mapped as assumes.
[Info] MBN_SUMMARY: '0' CMA inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' CMA outputs are mapped as assumes.
[Info] MBN_SUMMARY: '0' abstraction inputs are mapped as asserts.
[Info] MBN_SUMMARY: '0' abstraction outputs are mapped as assumes.
[Info] MBN_SUMMARY: '1' undriven nets are mapped as assumes.
1

vcf> report_seq_mappings -list -map_type {input} -type not_mapped
SEQ Mappings report
-------------------------------------------------------------------------
-------------------------------------------------------------------------
Task Name : SEQ
-------------------------------------------------------------------------
                     |  NOT MAPPED
-------------------------------------------------------------------------
  PRIMARY INPUT      |  3
-------------------------------------------------------------------------
-------------------------------------------------------------------------
Primary Input Not Mapped report ( Number of Not mapped Signals = 3 )
```

Feedback

```
-------------------------------------------------------------------
        Id |          Reason | BitWidth |  Signal
-------------------------------------------------------------------
         0 |        Sequnmap |        1 | spec.in1
         1 |        Sequnmap |        1 | impl.in1
         2 |       Not found |        1 | impl.in5
-------------------------------------------------------------------
```

14.4 Running SEQ

In the SEQ application mode, the `check_fv` command is used to check the sequential equivalence assertions that you have specified to compare the two designs. If you have enabled model checking assertions in the setup, then the `check_fv` command also tries to check them. However, it is recommended that you use the FPV application mode for proving model checking assertions as it activates algorithms optimized for proving SystemVerilog assertions.

```
check_fv [-block] [-run_finish <cmds>] [-stop] [-property <list-of-collection-of-properties>]
```

Table 22: Options of check_fv Command

`-block`	Blocks command input while `check_fv` is active.
`-run_finish <cmds>`	Set of commands to run when `check_fv` finishes (in non-blocking mode).
`-stop`	Stops the execution of currently running `check_fv` command.
`-property <list-of-collection-of-properties>`	List or collection of properties

To add SEQ specific options, use the following command:

```
seq_config [-specprefix <prefix>] [-implprefix <prefix>] [-recipe <recipeName>] [-map_uninit] [-map_x <zero|one|safe|bestmatch|none>]
```

Table 23: Options of seq_config Command

`-specprefix <prefix>`	Prefix for registers in specification design. (Default: spec)
`-implprefix <prefix>`	Prefix for registers in implementation design. (Default: impl)
`-recipe <recipeName>`	Name of the orchestration recipe to execute. Values: orch_generic_easy, orch_generic_low, orch_generic_medium, orch_generic_hard. (Default: orch_generic_medium)

Feedback

Table 23: Options of seq_config Command (Continued)

`-map_uninit`	Automatically maps all uninitialized corresponding registers in the two designs at reset (using name matching).

`-map_x <zero|one|safe|bestmatch|none>`	Enables you to assign zero or one or map all tool generated X values. This includes undefined behavior because of array bound violations, explicit X assignments, missing case statements.
	This option accepts `[zero|one|safe|bestmatch|none]` as its values.
	By default, it is set to `none`.
	• `zero`: Assigns zero "0" to all the undefined Xs.
	• `one`: Assigns one "1" to all the undefined Xs
	• `safe`: Maps the different bit width signals with the same name.
	• `bestmatch`: Tries to match the Xs by name between the two designs, else assigns them to zero.
	• `none`: Treats them as Xs.
	Note:
	Snips are not addressed as part of this option. All snips should be manually mapped. For example,
	`snip_driver -env impl.M1.a`
	`snip_driver -env spec.M1.a`
	`fvassume -env -expr`
	`{ impl.M1.a == spec.M1.a }`

`-map_counters`	Maps abstracted counters in the two designs.
`-extended_mapping`	Allow seqmap to search peers beyond sequentials and undrivens.
`-latch_scope <design>`	The design for latch instrumentation: Values: spec, impl, none
`-auto_failure_debug <ON|OFF>`	Compute the debug information of falsified properties automatically. Values: OFF, off, ON, on.
`-enable_rca_path <ON|OFF>`	Compute the rootcause path of falsified properties: Values: OFF, off, ON, on By default, this option is set to ON.
`-map_diff_bb_designs <ON|OFF>`	Allow mapping between black box instances of same name even if their underlying modules are different (Default is 'OFF'). Values: OFF, off, ON, on
`-enable_clock_coi_analysis <true|false>`	Enable COI analysis for assertions in case of Primary Output/Blackbox Input/Abstraction Input to detect clock in their combinational fanin. Default value is false. Values: true, false.
`-infer_unresolved_ports <ON|OFF>`	Apply heuristics (based on fanin's) to infer direction of ports of unresolved modules (Default is 'ON'). Values: OFF, off, ON, on

Feedback

Table 23: Options of seq_config Command (Continued)

`-enable_infer_blackbox_port_direction`	Enable application of COI heuristics to infer direction of ports for all blackbox modules. Default value is false. Values: true, false

14.4.1 Viewing the Progress of Mappings

You can also view the progress of the mapping while the `check_fv` command is running using the `report_seq_equivalences` command. This command reports all the potential internal equivalences that `check_fv` command is working on and report their statuses. Since, it only reports the conclusive equivalences and/or real failures, you may not see similar progress as in the orchestration view. Note that SEQ does not use all the mappings in its analysis, therefore there may be some internal equivalences whose status is 'no_status'. The output of the `report_seq_equivalences` command is for each run of the `check_fv` command.

Figure 95: Output of report_seq_equivalences

```
vcf> report_seq_equivalences
Summary for verification problem (id=0) in verification task SEQ
-----------------------------------------------------------------
#InternalEquivalences :    10
#Proven               :     7
#Falsified            :     0

1
```

Figure 96: Output of report_seq_equivalences

```
vcf> report_seq_equivalences -list
Summary for verification problem (id=0) in verification task SEQ
-----------------------------------------------------------------------------------------
#InternalEquivalences :    10
#Proven               :     7
#Falsified            :     0

List of internal equivalences for verification problem (id=0) in verification task SEQ
-----------------------------------------------------------------------------------------
    Id |     Status | FDepth | Equivalences
-----------------------------------------------------------------------------------------
     0 |     proven |      - | impl.go2 == spec.go2
     1 |     proven |      - | impl.go1 == spec.go1
     2 |  no_status |      - | impl.stage3 == spec.stage3
     3 |     proven |      - | impl.go3 == spec.go3
     4 |  no_status |      - | impl.stage2 == spec.stage2
     5 |     proven |      - | impl.valid == spec.valid
     6 |     proven |      - | impl.stage1 == spec.stage1
     7 |     proven |      - | impl.go4 == spec.go4
     8 |  no_status |      - | impl.stage4 == spec.stage4
     9 |     proven |      - | impl.product == spec.product
-----------------------------------------------------------------------------------------
1
```

Feedback

You can see CEX for both the output failures and internal equivalence failures, while the tool is running in the `check_fv` command. If any of the internal equivalence points failure is unexpected, then the you can debug them using the following command:

```
view_trace -inteq <Id> # internal equivalence Id from the report.
```

The failed internal equivalence point `<Id>` needs to be passed to`-inteq`. The same information can be collected from Designer View. See section Viewing the Designer View.

These CEXs enable you to check if the changes that you have made are considered by the tool and check if the setup is not over-constrained. For example, if you make changes in the implementation design such that two registers should not be equivalent, but expect the outputs to be sequentially equivalent. Then, using this report you can see if the tool thinks the same way either by finding a CEX or not using the mapping.

The get_seq_equivalences Command

The `get_seq_equivalences` command returns a collection of the internal equivalences found in a SEQ setup. You can apply various filters to get the appropriate information using the options in the command. You can also create various scripts based on the equivalences information.

```
vcf> get_seq_equivalences -help
Usage: get_seq_equivalences    # Gets information about
                                 internal equivalences found
                                 in a SEQ setup as a Tcl collection
        [-status <status-attributes>] (Property status selection:
                                Values: success, failed, bounded,
                                 no_status)
        [-id <integer>]        (List information for a particular
                                 internal equivalence id)
```

Supported Attributes

The following are the attributes supported on the objects returned in collection of `get_seq_equivalences`. The following is an example snippet of the output.

```
vcf> list_attributes -application -class seq_equivalence_object
Properties:
    A - Application-defined
    U - User-defined
    I - Importable from design/library (for user-defined)
    S - Settable
    B - Subscripted

Attribute Name            Object     Type       Properties  Constraints
-------------------------------------------------------------------------
fail_depth                seq_equivalence_object
                                     int        A
full_name                 seq_equivalence_object
                                     string     A
```

Feedback

```
id                          seq_equivalence_object
                                      int        A
phase                       seq_equivalence_object
                                      int        A
vpid                        seq_equivalence_object
                                      int        A
```

Examples

- The following example shows the default output of the `get_seq_equivalences` command:

```
vcf> get_seq_equivalences
{"spec.stage2", "impl.stage2", "spec.stage1", "impl.stage1",
 "spec.stage3", "impl.stage3", "spec.valid", "impl.valid", "spec.go4",
 "impl.go4", "spec.product", "impl.product", "spec.stage4",
 "impl.stage4", "spec.go3", "impl.go3", "spec.go2", "impl.go2",
 "spec.go1", "impl.go1"}
```

- The following example shows the output with the `-status` and `-id` options:

```
vcf> get_seq_equivalences -status failed
{"spec.stage2", "impl.stage2", "spec.stage3", "impl.stage3",
 "spec.product", "impl.product", "spec.stage4", "impl.stage4"}

vcf> get_seq_equivalences -id 3
{"spec.valid", "impl.valid"}
```

- The following example shows various types of filtering, using `filter_collection`:

```
vcf> set col [get_seq_equivalences]
{"spec.stage2", "impl.stage2", "spec.stage1", "impl.stage1",
 "spec.stage3", "impl.stage3", "spec.valid", "impl.valid", "spec.go4",
 "impl.go4", "spec.product", "impl.product", "spec.stage4",
 "impl.stage4", "spec.go3", "impl.go3", "spec.go2", "impl.go2",
 "spec.go1", "impl.go1"}

vcf> filter_collection $col {id==2}
{"spec.stage3", "impl.stage3"}

vcf> filter_collection $col {fail_depth==4}
{"spec.product", "impl.product"}

vcf> filter_collection $col {full_name=~spec.go*}
{"spec.go4", "spec.go3", "spec.go2", "spec.go1"}
```

14.5 Viewing SEQ Results in Tcl

All assertions during a `check_fv` run, start in the checking state.

- Once an engine finishes, it updates the result with proven, falsified, or bounded.
- If no conclusive result was found for a particular assertion and the `check_fv` command finishes then the result becomes inconclusive.

SEQ orchestration under the hood automatically creates simpler proofs and/or helper assertions to check. Intuitively, we define a proof in SEQ as a formal model, a set of assertions, and a set of assumptions. When we solve a proof, the statuses of the assertions are with respect to the formal model and assumption in that proof.

For a SEQ setup, when the `check_fv` command is executed, a top-level proof (named seqdef) of the user provided assumptions and assertions is created. The SEQ orchestration then internally creates various sub-proofs. These proofs are related to each other in the form a proof tree. This is visualized in details using the orchestration view in the GUI (or using the `report_proofs` TCL command).

One main task of the SEQ orchestration is to automatically identify internal equivalence points. We call this step of finding the internal equivalence points as decomposition. The result of decomposition is a new proof (named with prefix idcp) that includes all the internal equivalence points as assertions. Another important task as the orchestration progresses is to automatically refine the decomposed proof if there are any falsification in the internal equivalences. This step creates a new refined decomposed proof (named with prefix idcp). Because this step can be expensive the SEQ orchestration produces info messages as shown below when it is performing the refine step.

```
[Info] TW_INFO_MSG: Info in TW: 'Starting refine command'.
[Info] TW_INFO_MSG: Info in TW: 'Finished refine command in 1 seconds'.
```

As the orchestration progresses, it produces more such info messages to make the user aware of it's stages and progress. The SEQ orchestration is also quite dynamic. It enables/disables various engines based on the number of properties and number of available workers. This means that for the same effort level depending on the number of properties and/or number of workers you might see different engine ordering.

This can also be visualized in Verdi. See section *Viewing the Proof Tree in Verdi*.

You can use the `report_fv` command to view the current state of the proof while VC SEQ is running and after it has completed. To get a detailed description of the numerous options for this command, you can give the following command at the `vcf` prompt:

```
report_fv -help
```

The most common options is to use the`-verbose` switch to get detailed information about the status of each check, and `-list`to get a one-line summary for each check.

14.6 Running SEQ in a Grid Environment

Just like any other VC Formal app, grid environment can be enabled in VC Formal SEQ. Refer to section for information on how to configure grid workers, See Running on Compute Farm. Most jobs generated by SEQ run very quickly and are well suited to be executed on the default queue. However, SEQ generates a small number of jobs that exceed the resource limits of the default queue and need the resource limits of the heavy queue to complete.

If you specify one host file, and if the default queue is used, the long running jobs are not completed. If the heavy queue is used, long waits for machines to be allocated often arise. To resolve this issue, you can specify two host files (as described in section Specify two host files), using different submit commands which, in turn, can specify different queues. If you specify the second host file (using the `set_backup_grid -level 1`command), then the following behavior is activated.

- All jobs are submitted to the default queue.
- Any job that is killed by the grid for exceeding resource limitations is re-scheduled using the submit command specified in the second host file.

If the host_file is not specified, VC SEQ runs the proofs on the local machine sequentially. The syntax of the host file is described in the next section.

14.6.1 Specify two host files

Set the following VC SEQ variable to specify the name of the host file:

```
set_grid_usage -file <host.default> -level 0
set_grid_usage -file <host.large> -level 1
```

All jobs are initially submitted using the command in host.default. If a job is killed by the grid for exceeding resource limitations, it is resubmitted using the command in host.large.

14.7 Resuming Previous Run

VC Formal SEQ automatically saves the back-end state at certain checkpoints to the disk. This includes the assumptions/assertions in each proof, and the status of the assertions (proven, falsified, inconclusive). It also saves the bounded proof depths or failure depths for each assertion. In other words, it saves all the current statuses of the internal equivalence points. If the execution of the `check_fv` command exits due to resource limits or other issues and there are still inconclusive goals, then you can start a re-run of the `check_fv` command from the last checked point state using the following command:

```
resume_seq
```

This restores the SEQ to the last known state, and continues running the provided script from there.

14.8 Advanced Convergence Techniques

Consider a scenario where you set up and ran the proof and VC SEQ returned an inconclusive result. This usually happens because the equivalence problem is too large or too complex. In such cases, you can use snip points and black boxes to reduce the complexity.

14.8.1 Snip Points

Snip points are used to simplify a difficult equivalence proof which helps in convergence of a hard proof. Typically, you can use snip points as follows:

1. Create assertions that characterize the behavior of some node in the design.
2. Prove these assertions.
3. Enable a snip point at that node, assume the assertions you just proved, and under those assumptions prove the remainder of the design.

This process is called assume-guarantee reasoning, and you can apply it in as many steps as necessary.

Note:

This process can be applied to either or both designs as appropriate - the optimal approach is something that needs to be arrived at through experimentation. If you can prove two designs to be equivalent using snip points, you are guaranteed that the designs are also equivalent without snip points, provided that the relationships assumed on the snip points are also proved. This is known as a safe over-approximation.

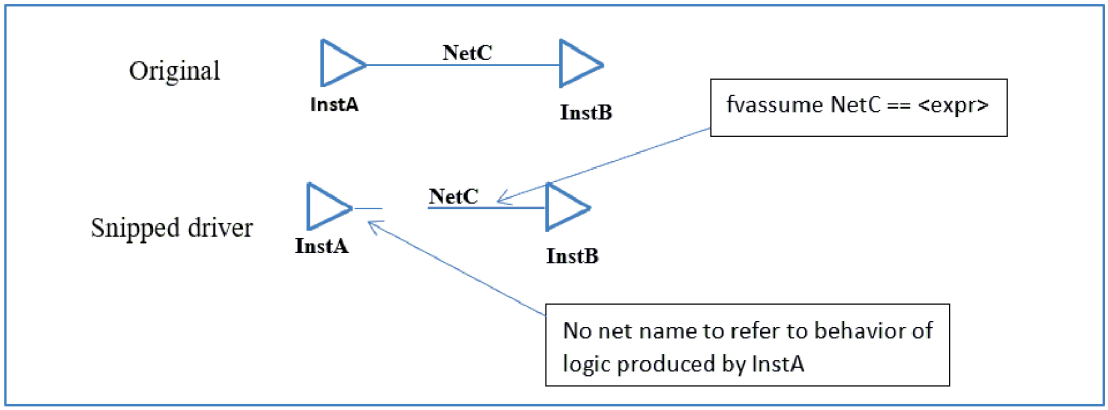
Use $driver to Access Snipped Net Driver

When a snip point is created in the model, it disconnects the driving logic, leaving the net as a free input for formal analysis.

There may be cases where the user needs to access the original net's driver, for example,

- The user snips a net, then writes a constraint for the new pseudo-input to the formal problem.
- Later, the user then wishes to verify that the driving logic also behaves in a way which is consistent with the assume.

Feedback



The $driver function takes one argument which is expected to be a snipped net. The behavior of the $driver function is that of the driver of the snipped net. As shown in the above figure, the $driver(NetC) would refer to the behavior provided by InstA.

An example usage in a script assert could be:

```
fvassert -expr {$driver(NetC) == 8'hf |=> NetD == NetF}
```

Additionally:

- It is possible and allowed for $driver to reference a net that is not snipped. In these cases, the behavior will be the same as if the net name itself were used (without $driver) in the property. So, if NetC is not snipped, these two assertions will behave exactly the same:

  ```
  fvassert -expr {$driver(NetC) == 8'hf |=> NetD == NetF}
  fvassert -expr {NetC == 8'hf |=> NetD == NetF}
  ```

- If the net is a multi-driven net, the behavior of $driver will be that of all the resolved drivers.

Leverage $next to Constrain Snip Points

One abstraction technique is to snip FF that is clock gated and map them (both in spec and impl). This technique often allows to get full convergence, however, for full correctness we need to check if the next states of the registers are always equivalent.

Consider the specification,

```
always @(posedge clk  or negedge rst) begin
   if(!rst)
         reg <= 'h0
   else
     if (ren)
          reg <= din;
end
```

Feedback

and the implementation,

```
cg cg1(clk, ren, gclk);
always @(posedge gclk or negedge rst) begin
if(!rst)
reg <= 'h0;
else begin
reg <= din;
end
```

Say, gclk is the gated clock in impl. We can snip and map using the following command:

```
snip_driver spec.reg
snip_driver impl.reg
fvassume -expr { spec.reg == impl.reg }
```

And if we add the following assertions:

```
fvassert -expr { $driver(spec.reg) == $driver(impl.reg) }
```

This assertion fails trivially because when the clock is gated in impl we hold the old value of the register in impl, however we get the new value of the snipped input in the spec. Currently, the confusion arises because in case of registers we are not snipping the register instead the output net of the register. To address this confusion, we introduce the $next function.

We can check if the next state of the registers are always same using the following assertion:

```
fvassume -expr { spec.reg == impl.reg }
     fvassert -expr { $next(spec.reg) == $next(impl.reg) }
```

Note that the use of $next, implicitly snips the register and hence we do not need an explicit snip_driver on the register.

The above assertion will be proven.

Use $next() as shown below:

$next(register_signal)

register_signal: a full word net driven directly by an inferred register in the design

Limitations:

- Multi-driven nets not supported
- Net must be driven directly by a register. If the net is word-split only the bits that are driven by registers are allowed.
- $next will not be applied if register_signal is used in an environment constraint
- Snipped signals (`snip_driver` or `set_abstractions` snips)

- If the register_signal is snipped, the snip will not be applied to the formal model if the signal is used in $next
- Constant signals (`set_constant`)
- If the register_signal is set to a constant, the constant will not be applied to the formal model if the signal is used in $next

An example of word-split is:

```
reg [7:0] reg1;
always @(posedge clk or negedge rst) begin
if(!rst)
reg1[2:0] <= 'h0;
else begin
reg1[2:0] <= din1;
end
always @(posedge clk or negedge rst) begin
if(!rst)
reg1[7:5] <= 'h0;
else begin
reg1[7:5] <= din2;
end
assign reg1[4:3] = din3;
```

In this case the two word-splits are reg1[2:0] and reg1[7:5] and you can use $next(reg1[2:0]) and $next(reg1[7:5]).

14.8.2 Black-boxing

Like snip points, black-boxing can also be used to simplify a proof by abstracting away implementation details that make the proof difficult, or are not required for the proof. You can black-box selected or all instances of a module. When a module instance is black-boxed the outputs of that instance become free inputs to the downstream logic. When a module is black-boxed, the outputs of all instances of that module become free inputs. You can access the inputs and outputs of the black-boxed module by using their hierarchical names. You can also define assumptions and assertions respectively on the inputs and outputs ports of a black-boxed module. Essentially, black-boxing is a module-level shorthand for inserting snip points. If a module has three outputs, you can achieve the same effect as black-boxing the module by placing snip points on all three outputs. For more information on black-boxing, see section Black Boxing Modules in a Design

Matching Black-boxes

If there is a 1:1 correspondence between black-boxed modules in the two designs, VC Formal SEQ will map the inputs and outputs of black-boxed modules/instances. If you want to add only a subset of those black-boxes use the following command:

```
map_by_name -blackbox -instances <list-of-blackboxes-without-prefix>
```

This command adds assertions to check that the corresponding inputs of the black-boxed modules instances are equal. It also adds assumptions that the outputs of the corresponding outputs of instances are equal. You can use the `get_blackbox` command to get a list of all black boxes.

If there are no assumptions created for a blackbox, the tool does not create assertions for its inputs.

Note:
Assumptions for unloaded output ports of a blackbox are not created.

14.8.3 SEQ Abstractions

VC Formal SEQ have support for various abstraction using `set_abstraction` command. In SEQ App, additional assumptions and assertions are added to map the abstractions using the `map_by_name` and check if the abstractions are fine. The main problem of this flow is that the user should manually refine some or all abstractions in case the abstraction checks fail.

VC Formal SEQ support automatic memory abstraction in the orchestration. If any of the abstraction failed then it is being automatically refined, thereby avoiding the manual refine steps.

The `set_seq_abstractions` command guides the orchestration to automatically do both memory and operator abstraction and refinement in the SEQ orchestration.

```
set_seq_abstractions # Sets the various abstraction settings for SEQ
 orchestration
-construct <list-of-construct-criteria> (list of design constructs
 and size thresholds to find and abstract. Each criterion is a design
 construct type and size threshold separated with = character. In each
 case, the size threshold causes constructs of that size or greater to
 be automatically abstracted from the formal model. Constructs are mem,
 mult, add, sub, div, mod, counter. Example: -construct {mult=4 mem=512})
```

Based on the selection of the options in this command some of the abstractions will be switched on/off.

- By default whenever this command is called memory abstraction is enabled. The `-construct` option takes a list of `<type>=<width>` pairs. For example,

  `set_seq_abstractions -construct {mult=4 div=8}`

  [Info] SEQ_ABSTRACT_SUMMARY: mem: <true, 512> mult: <true, 4> add: <false, 8> sub: <false, 8> div: <true, 8> mod: <false, 8>.

  Type can be one of {mem, mult, add, sub, div, mod}.

Width is an integer which means abstract all operators equal and above the specified bit width.

- The command does not overwrite settings of the types that are not mentioned in the list.

  ```
  set_seq_abstractions -construct {mult=16 add=8}
  ```

  [Info] SEQ_ABSTRACT_SUMMARY: mem: <true, 512> mult: <true, 16> add: <true, 8> sub: <false, 8> div: <true, 8> mod: <false, 8>.

- To disable a type of abstraction, use -1 for the bit width.

  ```
  set_seq_abstractions -construct {mem=-1}
  ```

  [Info] SEQ_ABSTRACT_SUMMARY: mem: <false, 512> mult: <true, 16> add: <true, 8> sub: <false, 8> div: <true, 8> mod: <false, 8>.

If any of the above abstraction is enabled, the proof tree in the orchestration view (or `report_proofs` TCL command) will have an extra entry (abs_1). The additional properties in that proof corresponds to the abstractions. If any of those properties fail, then the tool will automatically refine them.

The main reason to have a separate command is to make sure users understand the difference between the two commands. One is going to do automatic abstraction refinement in the background, it helps user to avoid manual work. However, for the same reason results might be coming little later, however, any result it provides will not require any manual work. There is no corresponding report command for the `set_seq_abstractions` command, only way to find what got abstracted is to check the orchestration view/report_proofs during/after runtime.

14.8.4 Support for Creating Properties Around Abstracted Memory

VC Formal provides the ability during compilation to have synthesis use memory inferencing to identify memories and abstract (replace) them with instrumented black box modules. The black box mapping flow can then be leveraged to automatically create the required properties around the memory black box ports.

Note:

- Currently, when using this approach, there is no way to undo the black boxing of the inferred memories other than disabling abstraction inferencing during compilation and redoing the compilation.
- 1-dimensional arrays are not identified as memories as they may cause false failures.

The set_compile_abstractions Command

Use the `set_compile_abstractions` command to enable abstraction inferencing during compilation. This command must be set before `read_file` or `elaborate` commands, else the *COMPILE_ABS_CMD_POST_ELAB* error message is reported to indicate the usage of `set_compile_abstractions` post elaborate.

Syntax

```
set_compile_abstractions [-construct <list-of-construct-criteria>]
 [-skip_mem <list-ofskip-mem>] [-skip_module <list-of-skip-modules>]
 [-force_mem <list-of-force-mem>] [-disable]
```

Where,

- `[-construct <list-of-construct-criteria>]`: List of design constructs and size thresholds to find and abstract. Each criteria is a design construct type and size threshold separated with =character. In each case, the size threshold causes constructs of that size or greater to be automatically abstracted from the formal model. Only construct 'mem' is supported for now. Example: `construct {mem=512}`
- `[-skip_mem <list-of-skip-mem>]`: List of 'module.sigName' signals to be skipped from being inferred as memory.
- `[-skip_module <list-of-skip-modules>]`: List of 'module' names within which memory inferring has to be skipped. The `-skip_module` accepts wildcard characters. For example, `-skip_module *` skips all the modules. `-skip_module {*Dummy}` skips only the modules whose names contain `"Dummy"`.
- `[-force_mem <list-of-force-mem>]`: List of 'module.sigName' signals to be forced to be inferred as memory.
- `[-disable]`: Disable memory abstraction.

The report_compile_abstractions Command

Use the `report_compile_abstractions` command to report the various compile abstractions created by the `set_compile_abstractions` command.

Syntax

```
report_compile_abstractions [-list] [-verbose] [-no_summary]
```

Where,

- `[-list]`: Outputs one-line report per instance.
- `[-verbose]`: Outputs detailed report of each abstracted instance.
- `[-no_summary]`: Skip summary.

Example

Feedback

The following is example output of the abstractions created:

```
Compile Abstraction Report
Construct | Count
-------------------------------------------------
Memory    | 2
-------------------------------------------------

> ID: [0]
- Instance   : spec.f1.r2.mem
- Construct  : Memory
- Bounds     : 15:0|7:0
- Total Bits : 128
- Threshold  : 100
- Module     : ram
- Memory     : mem
- Library    : SPEC
> ID: [1]
- Instance   : impl.f1.r2.mem
- Construct  : Memory
- Bounds     : 15:0|7:0
- Total Bits : 128
- Threshold  : 100
- Module     : ram
- Memory     : mem
- Library    : IMP
```

The get_compile_abstractions Command

Use the `get_compile_abstractions` command to report the various compile abstractions created by `set_compile_abstractions` command as Tcl collection.

Syntax

```
get_compile_abstractions
```

14.8.5 Increasing Bounded Depth in SEQ

`set_fml_var seq_bmc_depth <num> (Default = 100)`

The SEQ orchestration uses this number as a hint on how to balance the orchestration early on. This value is the number of phases the orchestration tries to bounded proof the properties.

For most designs, a cycle consists of two phases. Note that as the orchestration progresses it will try to proof higher bounded depths. The problem of setting this number too big is that it reduces the time spent by engines that tries to get a full proof for the properties. On the other hand, if you are trying to find deep falsification or increase the bounded depth of inconclusive properties then this number should be increased.

For example, if you want to find if there are any falsification within 300 cycles, then you will set this variable to 600.

14.8.6 Decomposition Using Partial Word Equivalence

Using `seqmap`, user can specify equivalence between bit-ranges of two registers. In the backend, these partial words are natively represented while solving. Saves solver time spent in "refining" the decomposed model constructed assuming entire word when only bit-ranges are expected to be equivalent. To use it you have to set the following two fml var

```
set_fml_var seq_use_decompose_partial_words true (default is false)
set_fml_var seq_use_regclasses true
```

This can be useful if the implementation has registers that are only partially equivalent to the specification.

For example, say there is a control register c which is 8 bit in the spec. In the implementation say the designer added some new features that are controlled by c. Now, c in the impl is 10 bits. The actual mapping is spec.c[7:4] == impl.c[9:6] and spec.c[3:0] == impl.c[3:0]. The tool automatically tries to find such mappings if it is set to true. Can have negative impact in scenarios where such mappings are not found.

- Use Case 1: GPU's Datapath

  GPU's often perform operations in parallel. For example, a register of 256-bits is updated using 4 parallel datapaths each of 64-bit. Current SEQ flow maps the entire 256 bit, therefore the equivalence problems is hard. Support for specifying partial words equivalences enables the user to decompose the problem into 4 smaller equivalence problem of 64-bits each, thereby making the verification problem more tractable.

  e.g., let r be a 256-bit register in specification and the implementation

  ```
  seqmap spec.r[63:0]         impl.r[63:0]
  seqmap spec.r[127:64]     impl.r[127:64]
  seqmap spec.r[191:128]     impl.r[191:128]
  seqmap spec.r[255:192]     impl.r[255:192]
  ```

- Use Case 2 : Bitwidth Optimization

  Synthesis/Designers often optimizes the bit widths of register. E.g., spec may define operations on a 32-bit registers, optimization analysis infers that only a 24-bit register is sufficient. Support for partial word equivalence enables the user to specify mapping as follows:

  ```
  seqmap spec.a[23:0]  impl.a[23:0]
  ```

- Use Case 3: Memory Splitting

- Large memories in a Spec are often partitioned into several banks of smaller memories. For example, a 4KB memory is implemented using 4 banks each of 1 KB. The address bus then can be mapped as follows:

```
seqmap spec.addr[11:10]  impl.bank_select[1:0]
seqmap spec.addr[9:0] impl.bank0[9:0]
seqmap spec.addr[9:0] impl.bank1[9:0]
seqmap spec.addr[9:0] impl.bank2[9:0]
seqmap spec.addr[9:0] impl.bank3[9:0]
```

- A 32KB memory with 256-bit word size is implemented as 4 smaller memories each of 64-bits word size

```
seqmap spec.mem[9:0][63:0]  impl.mem0[9:0][63:0]
seqmap spec.mem[9:0][127:64]  impl.mem1[9:0][63:0]
seqmap spec.mem[9:0][191:128]  impl.mem2[9:0][63:0]
seqmap spec.mem[9:0][256:192]  impl.mem3[9:0][63:0]
```

- Similarly, partial word mapping is useful in scenarios where the memories are joined.

14.8.7 Clock-Gating Verification Convergence

To achieve convergence for clock-gating verification, you can use the `seq_guidance` command.

Syntax

```
vcf> seq_guidance -help
Usage: seq_guidance    # Command to guide users to achieve convergence
 for clock-gating verification
       [-mode <value>]        (Processing mode for clock gating
 verification. Default value is identify_complex_register:
                               Values: identify_complex_register,
                               override_complex_register,
                               identify_cg_condition)
       [-report]              (List the registers identified by the
 guidance)
       [-file <filename>]     (Save the list of registers identified by
 the guidance to an XML file)
       [-enable_override_top_level_failures <true|false>]
                              (Enable computation of registers even in
 case of top-level failures. Default value is false:
                               Values: true, false)
       [-col]                 (Return list of object as a collection,
 works only with mode 'identify_cg_condition')
       [-list]                (Outputs detailed report in list form)
       [-scope <list-of-scopes>]
                              (Identify registers inside given scopes,
 works only with mode 'identify_cg_condition')
```

Feedback

```
        [<list-of-reg-patterns>]
                                (Specifies a list of name patterns
to match registers in the design (default), works only with mode
'identify_cg_condition')
```

Examples

- The following example identifies and saves all gated flops affecting the convergence to the 'flops.xml' XML file:

```
vcf> seq_guidance -file flops.xml
```

- The following example overrides the clock pin of the implementation flop with the clock pin of the specification flop for all the flops listed in the 'flops.xml' XML file:

```
vcf> seq_guidance -file flops.xml -mode override_complex_register
```

- The following example identifies registers having clock gating condition from entire design:

```
vcf> seq_guidance -mode identify_cg_condition
```

- The following example identifies registers having clock gating condition from scope ‘spec.h1’ and ‘impl.icg1’ only:

```
vcf> seq_guidance -mode identify_cg_condition -scope {spec.h1
 impl.icg1}
```

- The following example identifies registers having clock gating condition where register name should match pattern *enable or *addOut:

```
vcf> seq_guidance -mode identify_cg_condition {*enable *addOut}
```

- The following example reports registers having clock gating condition in list form:

```
vcf> seq_guidance -report -mode identify_cg_condition -detail list
------------------------------------------------------------------------
--------------------
Scopes : spec, impl
------------------------------------------------------------------------
--------------------
Number of registers identified with enable condition : 4
------------------------------------------------------------------------
--------------------
List of registers with enable condition
------------------------------------------------------------------------
--------------------
        Id |                          Register    |    Enable Condition
------------------------------------------------------------------------
--------------------
         0 |                      spec.h1.out     |    Not identified
 (enable pin unconnected)
         1 |                 spec.add1.addOut     |    spec.add1.cond
```

Feedback

```
         2 |                          spec.s_d    |    spec.inp1_d[1]
         3 |                       spec.p1.out    |    Not identified
 (enable pin unconnected)
-------------------------------------------------------------------------
--------------------
```

- The following example reports collection of registers having clock gating condition:

```
vcf> seq_guidance -report -mode identify_cg_condition -col
{"spec.h1.out", "spec.add1.addOut", "spec.s_d", "spec.p1.out"}
```

- The following example reports registers having clock gating condition in list form by matching the register pattern *.out :

```
vcf> seq_guidance -report -mode identify_cg_condition *.out -list
-------------------------------------------------------------------------
--------------------
Scopes : spec, impl
-------------------------------------------------------------------------
--------------------
Number of registers identified with enable condition : 2
-------------------------------------------------------------------------
--------------------
List of registers with enable condition
-------------------------------------------------------------------------
--------------------
        Id |                          Register    |    Enable Condition
-------------------------------------------------------------------------
--------------------
         0 |                       spec.h1.out    |    Not identified
 (enable pin unconnected)
         1 |                       spec.p1.out    |    Not identified
 (enable pin unconnected)
-------------------------------------------------------------------------
--------------------
```

14.9 Debugging SEQ Failures

When VC SEQ falsifies an assertion, a counter-example is generated to illustrate one possible condition that precipitates the failure. A counter-example consists of assignments to input and state variables. Execution of the spec and impl designs for the length of the counter-example reveals the assertion failure.

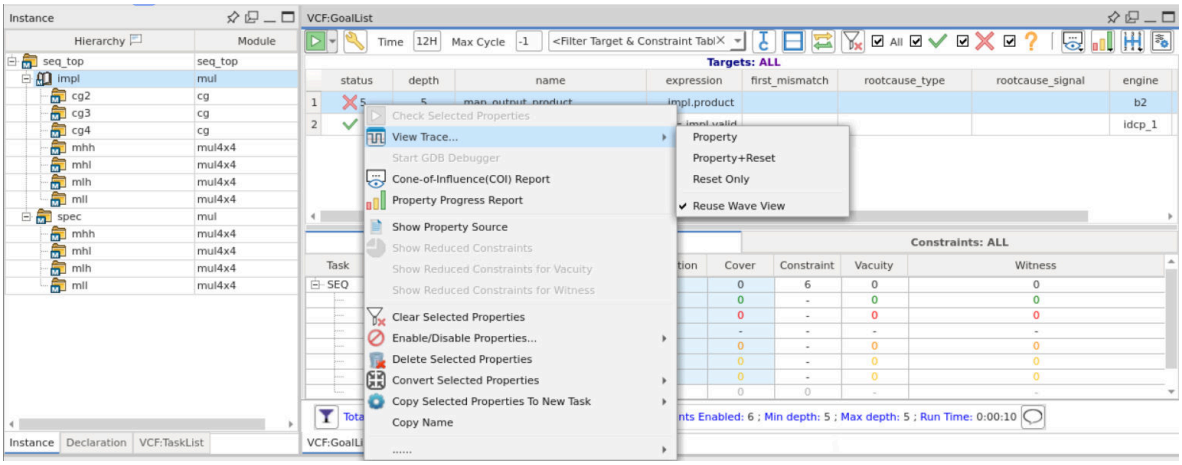
Each counter-example applies to a single assertion failure. If VC Formal SEQ finds many assertion failures, it generates an equivalent number of counter-examples. There is always a single counter-example for each failed assertion.

Any inputs or state variables that do not affect the success or failure of the target assertion are left in don't care states. When debugging in an RTL environment, don't care variables appear as X values.

Feedback

The counter-example can be replayed by double clicking on the falsification status from the property table or right clicking on the property `View Trace -> Property+Reset`

Figure 97: Replaying a Failure



14.9.1 Running SEQ in VC Formal GUI

VC Formal provides the capability to perform SEQ using the VC Formal GUI.

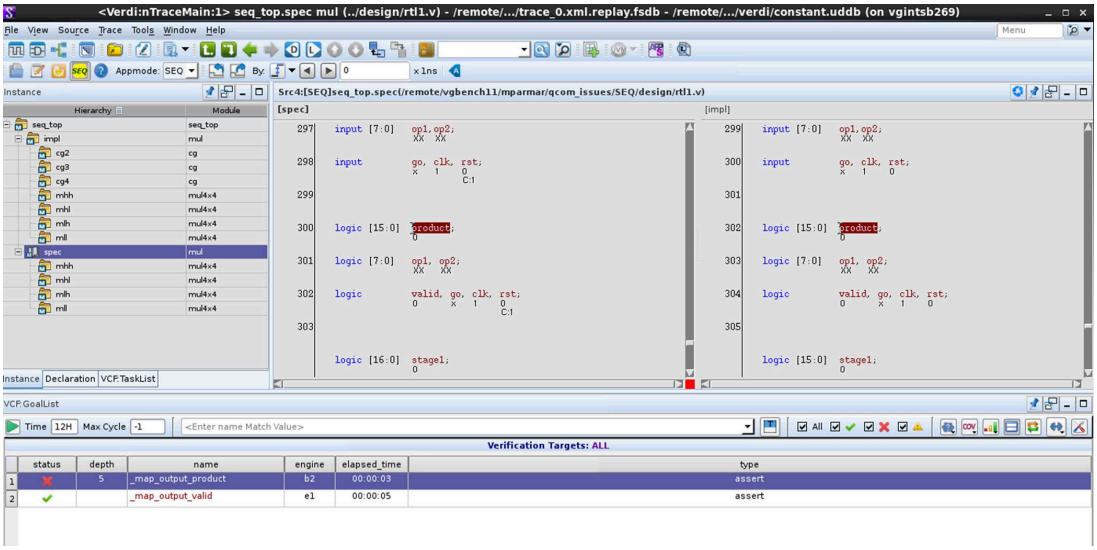
To open the SEQ Appmode in the Verdi GUI:

1. Click to run the SEQ task to get proven properties or bounded proofs.
   - The SEQ task mode is added in the Task List.
   - The properties are listed in the *GoalList* tab.

Feedback

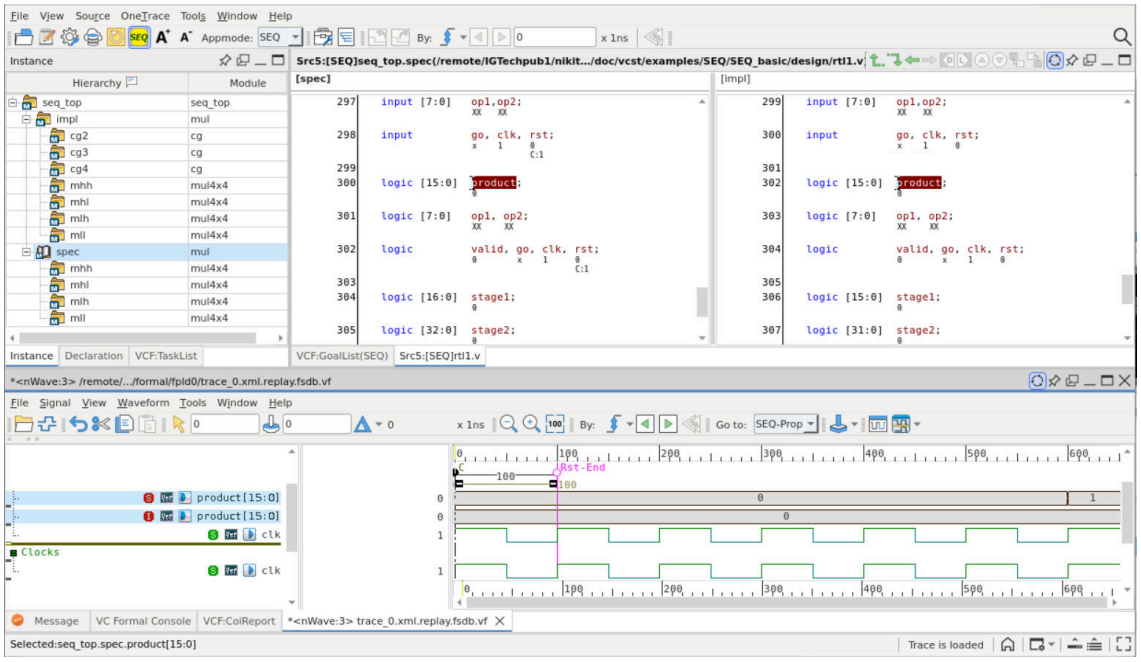
- The spec and impl source windows appear.

Figure 98: SEQ Debug Mode



1. Click the icon in the *Status* column to view the details of the property.
2. Click on the *Property* name in the property table to debug in the Verdi. This action invokes Verdi and the screen shown in Figure 99 is displayed:

Figure 99: Verdi Debug Screen



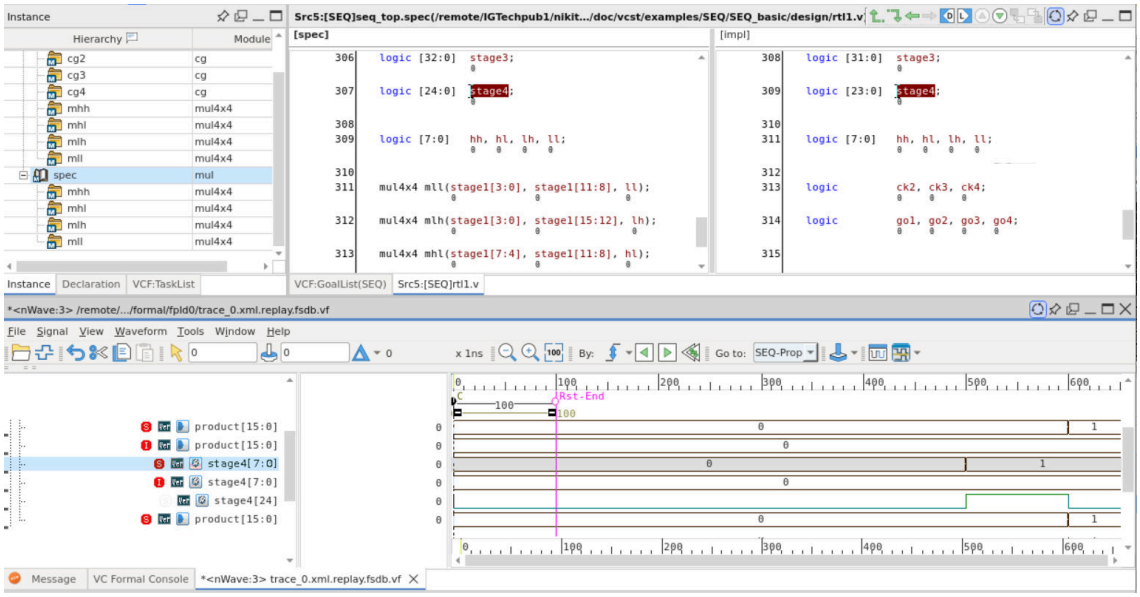
Feedback

Note the following:

- The RTL hierarchy is shown in the *Instance* pane in the top left. Here, the `seq_top` module contains an instance of the *specification* design (`spec`), and an instance of the *implementation* design (`impl`). The `impl` version has `cg2, cg3` and `cg4` modules – these are the clock gaters.
- The other two panes at the top show the source code for the two designs. The signal that failed in the output comparison is highlighted.
- The pane at the bottom shows the waveforms. The script property `_map_out_product` failed and the two signals it compares are shown under *Support-Signals*. The origins of these signals are marked with *S* (for specification) and *I* (for implementation).

1. You can right-click on the failing signal and select *Show Driver/Load Signals -> Driver Signals* to pull the signals driving the failed signals into the waveform display. This action brings in the driver signals from both the `spec` and the `impl` designs as shown in Figure 100.

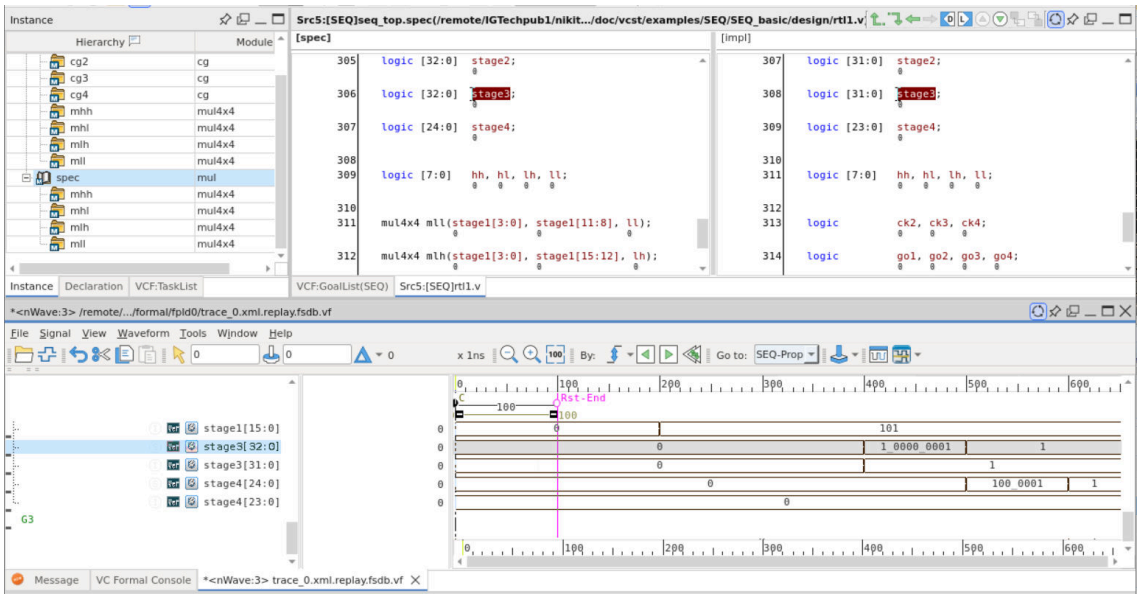
Figure 100: Driver Signals in Different Stages



Feedback

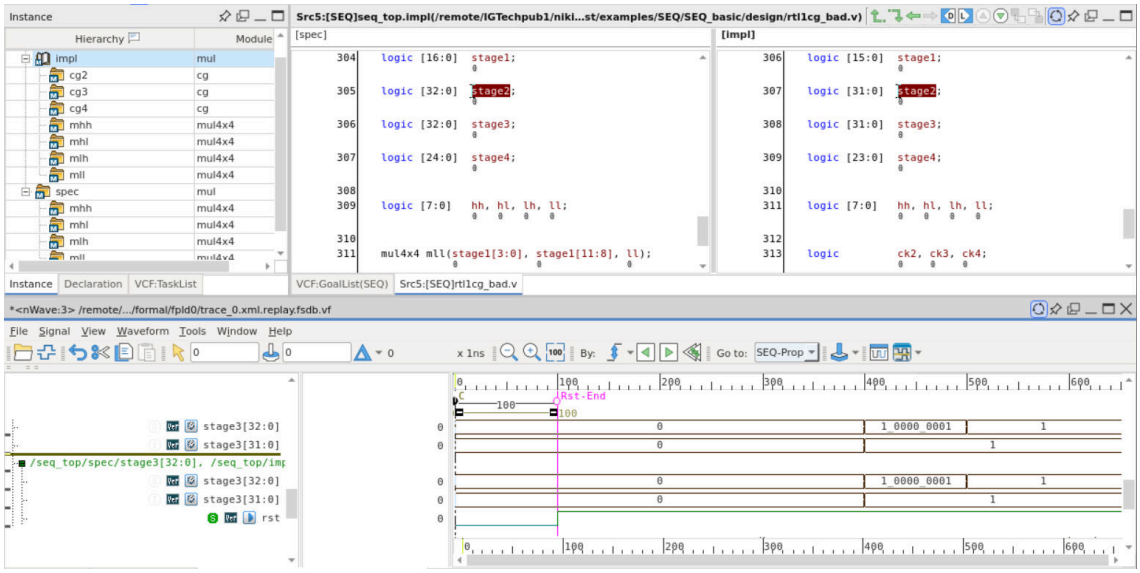
2. As you can see, the signal *stage4* is different between the `spec` and the `impl` designs. You can right-click on *stage4* and trace back further using the same method of displaying driver signals described earlier, resulting in the screen shown in Figure 101:

Figure 101: Tracing Back Signal - Stage3



3. Tracing back, you can see that *stage3* in the `impl` is different as shown in Figure 102. Tracing back further, you can see that, *stage2* is the same for both designs. Thus, it is determined that the discrepancy begins at *stage3*. Now you can easily determine the root cause of the discrepancy.

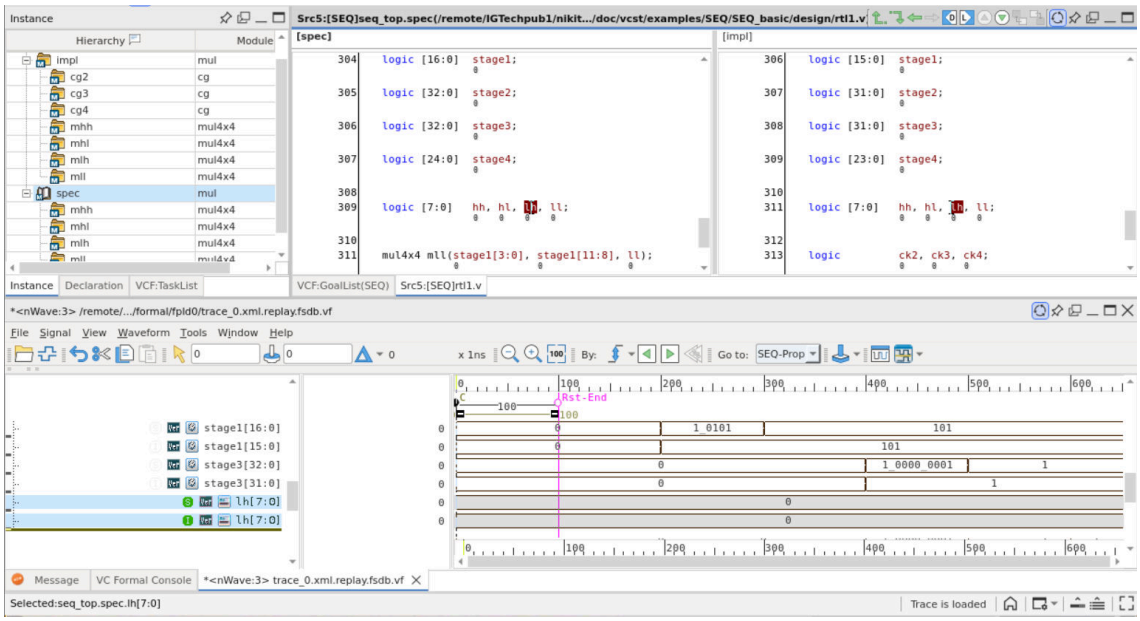
Figure 102: Tracing Back Signal - Stage2



Feedback

4. The code and waveform windows are linked, so selecting a signal in a code pane and pressing CTRL-W will add the `spec` and `impl` versions of that signal to the display. Figure 103 shows this for the 8 bit signal *ll*:

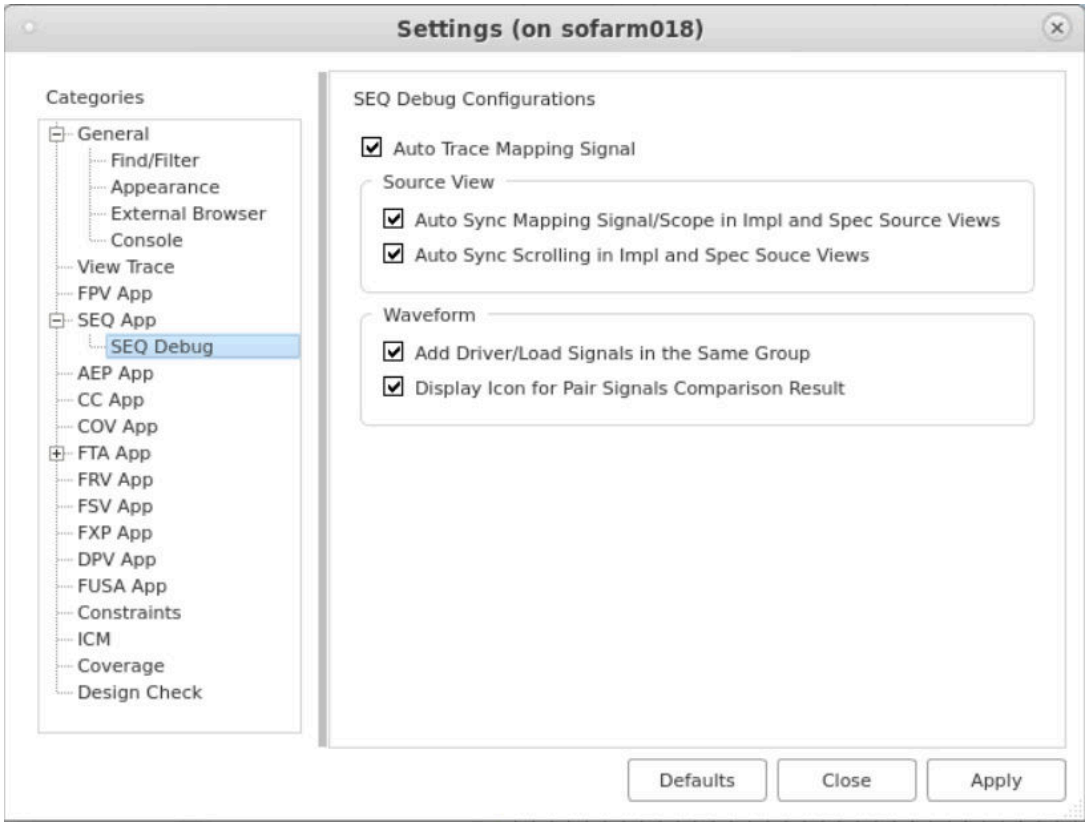
Figure 103: 8-bit Signal - ll



Feedback

5. There are several configuration options available for SEQ debugging in Verdi as shown in Figure 104:

Figure 104: Configuration Options Available for SEQ Debugging in Verdi



14.9.2 Debugging And Reporting SEQ Failures

This section provides details on how to debug and report SEQ failures using the `seq_debug` and `seq_debug_report` commands.

Debugging SEQ Failures

Use the `seq_debug` command to detect the propagation from the 1st mismatch sequential to the observation point in the waveform.

The `seq_debug` command performs root cause analysis by default, unless the `-enable_rca_path` option is set to `OFF/off` in the `seq_config` command . The root cause signals are the ones leading to the value discrepancy of the first mismatch sequentials. Each root cause signal is labeled with a root cause type, such as *driver_mismatch, unmapped, undriven, snip, abs, blackbox, input, uninit, explicitx, and driver_logic*.

Table 24: Root Cause Signal for Sequential Mismatch

Root cause type	Description
driver_mismatch	The peer of root cause signal cannot be found from the drivers of the first mismatch signal of the peer side.
unmapped	The root cause signal is unmapped.
undriven	The root cause signal is undriven.
snip	The root cause signal's driver is snipped.
abs	The root cause signal is an output of an abstraction.
blackbox	The root cause signal is an output of a blackbox instance.
input	The root cause signal is a Primary Input.
uninit	The root cause signal is an uninitialized sequential.
explicitx	The root cause signal is explicitly assigned with X.
driver_logic	seq_debug cannot find drivers of different values of the first mismatch signals. More likely, the logics driving the first mismatch signals are different.

Syntax

```
vcf> seq_debug -help
Usage: seq_debug    # Compute debug information for the specified
 properties
        [-stop]                (Stop current run)
        [-block]               (Perform action in blocking mode)
        [<property-list>]      (List of properties)
```

Reporting SEQ Failures

Use the `seq_debug_report` command to the get the debug information for the specified properties.

Syntax

```
vcf> seq_debug_report -help
Usage: seq_debug_report    # Report debug information for the specified
 properties
        [-detail <value-list>] (Specify report style and detail required.
 Default value is summary:
                                Values: list, summary, verbose)
        [-format <value>]      (Report results in specified format.
 Default value is text:
```

Feedback

```
                                Values: text, xml)
      [-group_failure <value-list>]
                               (Group failures by specified criteria:
                                Values: mismatch, depth, rootcause_type,
                                rootcause_signal)
      [<property-list>]        (List of properties)
```

Examples

- Run the `seq_debug` command after `check_fv`.

```
 set_fml_appmode SEQ

           # Compile the two designs
           analyze -format sverilog -library spec ../design/rtl1.v
           analyze -format sverilog -library
impl ../design/rtl1cg_bad.v
           elaborate_seq -spectop mul -impltop mul

           seqmap ...

           # Map primary inputs/outputs, blackbox inputs/outputs,
abstraction inputs/outputs and undrivens
           map_by_name

           ## Create clock and reset signals
           create_clock -period 100 spec.clk
           create_reset spec.rst -sense low

           ## Run reset simulation
           sim_run -stable
           sim_save_reset

           #use SEQ config to map uninitialized registers
           seq_config -map_uninit -map_x bestmatch
           check_fv -block
           seq_debug
           seq_debug_report
```

- To run `seq_debug` in the block mode and then report the debug information

```
seq_debug -block
seq_debug_report
```

- To compute and report debug report on the specified properties:

```
seq_debug _map_output_out1 -block
seq_debug_report _map_output_out1
```

- To get the debug a group of properties and get a report:

```
seq_debug -group "depth mismatch"
seq_debug_report -group_failure
```

Feedback

- The option `-group_failure` in `seq_debug_report` needs either `-detail {list}` or `-detail {verbose}` option to be enabled. If neither of these are enabled, a warning message, *'FV_OPTION_SPECIFIC_VALUE_REQUIRED'* is flagged and `-detail {list}` is automatically enabled. The following example shows the use of `-group_failure` to report the properties with respect to variant features:

  ```
  seq_debug_report -group_failure "depth mismatch"
  ```

- A SEQ CEX trace may have multiple pairs of first mismatch sequentials and their root cause signals. However, there can be only one row for a property with the command `seq_debug_report -detail list`.

  "…" is used to indicate that there are items that are not listed:

  ```
  List of debug information
  ----------------------------------
  Id property_name   depth first_mismatch  rootcause_type
   rootcause_signal
   _map_output_o       1   spec.mem[0:1]   uninit ...
   impl.mem[0:1] ...
  ```

  With `seq_debug_report -detail verbose`, all the root cause signals are reported as follows:

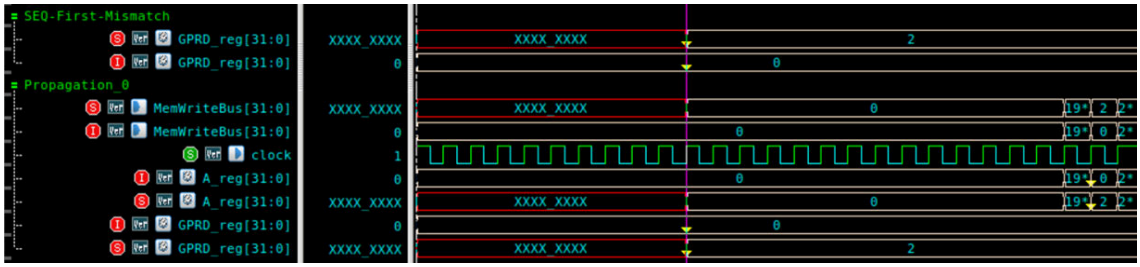
  ```
  property_name: _map_output_out28
                Id: 27
        Depth: 20
  first_mismatch: spec.inner_mod_inst.OUT7
             rootcause:
               abs                  : impl.inner_mod_inst._ASN_32 (out7)
               abs                  : spec.inner_mod_inst._ASN_63 (OUT7)
  ```

  Where, the internal signal *impl.inner_mod_inst._ASN_32* is the output of an abstraction. The load is put in signal *out7* in the parentheses.

- To calculate the debug information automatically on each falsification, use the following option:

  ```
  seq_config -auto_failure_debug ON
  ```

  By default, the `seq_config -enable_rca_path` is set to ON and you can get the signal group, listing the propagation path in a SEQ trace. For example, "Propagation_0" as shown in the following figure.

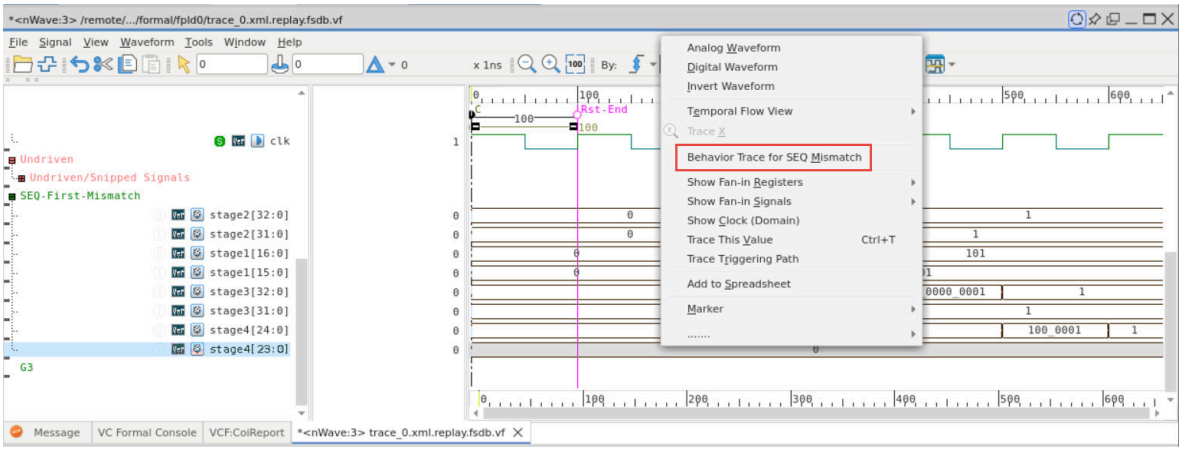


The yellow triangles mark the time instances on which the sequentials from the spec and the impl parts have different values. The signal group “SEQ-FIRST-MATCH” lists the first signal pair in the propagation path to debug the first falsified SEQ property.

14.9.3 Debugging SEQ Failures using Temporal Flow View

Failures in VC Formal SEQ can also be debugged and rootcaused using temporal flow view(TFV). After the counter-example is being replayed and the waveform appears in Verdi, right click on the failure output and select *Behavior Trace for SEQ Mismatch*.

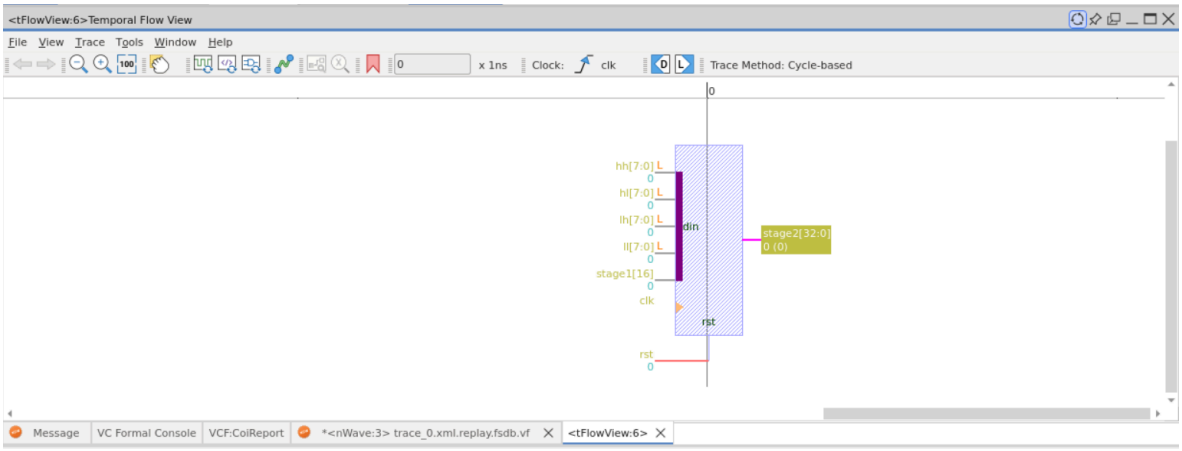
Figure 105: Enabling Temporal Flow View Debug in SEQ



This will open the temporal flow view which automatically unrolls the time from reset till the failure and traces back to the earlier mismatch. Each mismatching FF or signal will be highlighted in RED with both spec and impl values: spec_value(impl_value)

Feedback

Figure 106: Temporal Flow View in SEQ debug



14.10 Viewing Advanced SEQ Reports

After a SEQ problem is set up, the first step is to run VC SEQ's automatic orchestration (`check_fv`) with different parameters. VC SEQ internally creates simpler proofs and/or assertions to check. It is sometimes useful to view these proofs and/or assertions. Moreover, for more complex problems, there may be a few inconclusive properties. It is more beneficial to focus on one of the complex problems rather than the original assertions. These internal assertions often give clues to the root cause of the proof difficulty.

A proof in VC SEQ consists of a set of assertions and a set of assumptions and its associated results and configurations. For a VC SEQ setup, when the `check_fv` command is executed, a top-level proof of your assumptions and assertions is created.

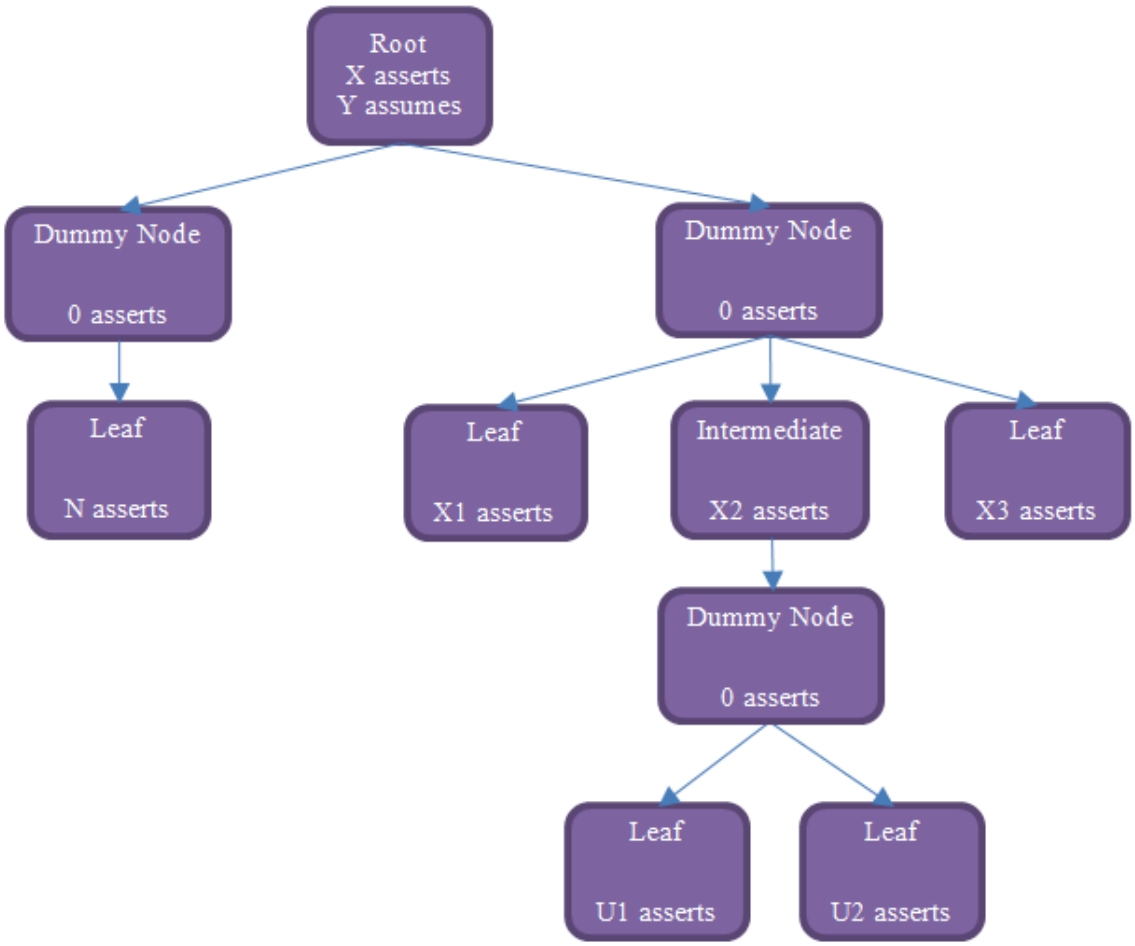
14.10.1 Proof Tree

For complex problems, VC SEQ has the option to internally create different types of sub-proofs. These sub-proofs are derived from the root proof created using the assumptions and assertions provided by you. The relation between the different proofs can be represented using a tree.

The general structure of the tree can be represented as illustrated in Figure 107.

Feedback

Figure 107: Proof Tree Template



- *Root Node*: The root of the tree is created using the assumptions and assertions provided by you during setup. Consider that this proof contains X assertions and Y assumptions as provided in the setup. This is the proof that is to be solved using VC SEQ.
- *Dummy Node*: This node does not contain any assertions or assumptions. It acts as a collector node that summarizes the results of its immediate children. There are two types of dummy nodes:
  - *Decompose*: The relation between the top assertion and the decomposed proof is complex and is handled automatically by the tool. Intuitively, it is a sound abstraction of the original proof. The key characteristics of this node are as follows:

Failures in the child proofs do not automatically mean failure in any of its parent-proof's assertions. If there is a real failure in the original root proof, it is automatically updated with the falsified status.

If all the goals in any of its immediate child-proof are proven, all the goals in its immediate parent-proof are proven too.

- *Assume-Guarantee*: The immediate child-proof contains more assumptions and less assertion then the immediate parent-proof. The key characteristics of this node are as follows:

There is a one-to-one correspondence between the goals in the immediate child-proof and the immediate parent-proof. The child-proofs preserve the original goal Ids.

A success in any of the goals in the immediate child-proofs implies a success in the same goal in the immediate parent-proof.

A failure in any of the goals in the immediate child-proofs implies a failure in the same goal in the immediate parent-proof.

- *Leaf Node*: A leaf node is created as a result of abstractions and/or assume-guarantee. This node contains non-zero assumptions and/or assertions. The parent of this type of node is always a dummy node. The number of assumption and assertions in this node may be different from the number of assumptions and assertions in the immediate parent of its dummy parent node.
- *Intermediate Node*: A leaf node is transformed into an intermediate node, when you run commands to create further child proofs from one of the leaf nodes.

14.10.2 Detailed Results for Each Proof

To print the details of all proofs in the current VC SEQ run, use the `report_proofs` command.

The `report_proofs` command can further provide details for each proof. You can use the `-list`or the `-verbose` options to report all the properties (including internal ones). Furthermore, it also reports all the engines run on a particular property and the results from each engine.

```
report_proofs # Reports information about existing proofs
[-no_summary] (Suppresses summary information)
[-verbose] (Outputs detailed report)
[-list] (Outputs one-line report per check)
[-type type-attributes] (Property type selection:
Values: assume, internal, reset,
env, stable, assert)
[-usage usage-attributes] (Property usage selection:
Values: assume, assert)
[-status status-attributes](Property status selection:
Values: proven, falsified, bounded)
[-proof proofname] (Select properties for a proof)
```

Example

Feedback

The output from the `report_proofs -verbose -usage assert -no_summary` command is as follows:

```
Verbose Proofs:
   Proof >
            VpId: 0
            Name: seqdef
            Type: root
          Parent: nil
     #Assertions: 2
    #Assumptions: 0
          Status: failed
       Converged: 2
    Assertions >
           Usage: assert
            Type: assert
          Handle: _BMS.ccId5_exp
            Name: _map_output_product
      Expression: spec.product == impl.product
          Status: failed
          SDepth: 4
          FDepth: 5
            Time: 3
          Memory: 145
        Engines >
                  Name: t1
               Options: -resource=3600-conflict=150000
                Status: bounded
                  Time: 0
                Memory: 110
                  Name: b2
                Status: bounded
                SDepth: 4
                  Time: 1
                Memory: 145
                  Name: b1
                Status: failed
                SDepth: 4
                FDepth: 5
                  Time: 2
                Memory: 145
             Usage: assert
              Type: assert
            Handle: _BMS0.ccId6_exp
              Name: _map_output_valid
      Expression: spec.valid == impl.valid
          Status: success
          SDepth: 49
            Time: 6
          Memory: 145
        Engines >
                  Name: t1
               Options: -resource=3600-conflict=150000
```

Feedback

```
              Status: bounded
                Time: 0
              Memory: 110
                Name: b2
              Status: bounded
              SDepth: 49
                Time: 2
              Memory: 145
                Name: b1
              Status: bounded
              SDepth: 41
                Time: 2
              Memory: 145
                Name: e1
              Status: success
                Time: 2
              Memory: 145
  Proof >
           VpId: 0
           Name: rw1_1
           Type: int
         Parent: seqdef-rw
   #Assertions: 2
  #Assumptions: 0
         Status: failed
     Converged: 2
   Assertions >
          Usage: assert
           Type: assert
         Handle: _BMS.ccId5_exp
           Name: _map_output_product
     Expression: spec.product == impl.product
         Status: failed
         SDepth: 4
         FDepth: 5
           Time: 1
         Memory: 121
```

The `report_proofs` command is illustrated in Figure 108:

Figure 108: The report_proofs Command Result

```
vcf>report_proofs
  Summary Proofs:
  ------------------------------------------------------------------------------------------------------------------------
   VpId |          Name |      Type |        Parent |    #A |    #C |    #S |    #F |    #I |    Status |     %
  ------------------------------------------------------------------------------------------------------------------------
      0 |        seqdef |      root |           nil |     2 |     0 |     1 |     1 |     0 |    failed |   100
```

VC Formal SEQ orchestration can internally create various sub-proofs. These proofs are related to each other in the form a proof tree. Figure 109 shows the listing from the

Feedback

`report_proofs` command when the following `check_fv` command (default recipe) is executed:

```
seq_config -map_uninit
check_fv -block
```

Figure 109: Sub-proofs Using the report_proofs Command

```
vcf>report_proofs
  Summary Proofs:
  ------------------------------------------------------------------------------------------------------------------
   VpId |          Name |       Type |          Parent |  #A |  #C |  #S |  #F |  #I |   Status |    %
  ------------------------------------------------------------------------------------------------------------------
      0 |        seqdef |       root |             nil |    2 |    0 |    1 |    1 |    0 |   failed |  100
      0 |     seqdef-rw |         or |          seqdef |    - |    - |    - |    - |    - |        - |    -
      0 |         rw1_1 |        int |     seqdef-rw |    2 |    0 |    1 |    1 |    0 |   failed |  100
      0 |      rw1_1-ur |         or |         rw1_1 |    - |    - |    - |    - |    - |        - |    -
      0 |         ur_1 |       leaf |      rw1_1-ur |    4 |    0 |    0 |    0 |    4 |  bounded |    0
      0 |      rw1_1-dcp | decompose |         rw1_1 |    - |    - |    - |    - |    - |        - |    -
      0 |        idcp_1 |       leaf |      rw1_1-dcp |    2 |    0 |    1 |    0 |    1 |  bounded |   50
  ------------------------------------------------------------------------------------------------------------------
```

Figure 109 shows three proofs. Each entry describes the following:

- The name of the proof
- The type of proof in the context of the proof tree (see section Proof Tree for details). The different possible values are:
  - `root`: This type indicates a proof that does not have a parent. This type of proof consists of the original user provided assumptions and assertions.
  - `leaf`: This type indicates a proof that does not have any sub-proofs.
  - `decompose`: This type indicates that this is a dummy proof to combine the results of the sub-proofs. The main characteristic of this node is that failures in the sub-proofs do not automatically mean failure of the top-level proof goals.
  - `or`: This type indicates that this is a dummy proof to combine the results of the sub-proofs as follows:

If a goal is proven or failed in a sub-proof, then the goal in the parent proof will be proven or failed as well.

- `intermediate`: This type indicates a proof that is not a dummy proof and has sub-proofs.
- The parent column is used to indicate if a proof is a sub-proof of some other proof. In this example `seqdef` is the root proof, so, its parent is *nil*.
- The number of assertions in the proof.
- The number of assumptions in the proof.
- Proven goals.
- Falsified goals.

- Inconclusive goals.
- The status column shows proven if all goals in the proof pass, falsified if any goal in the proof failed, bounded if there is at least one inconclusive goal and no failed goal

  Note:

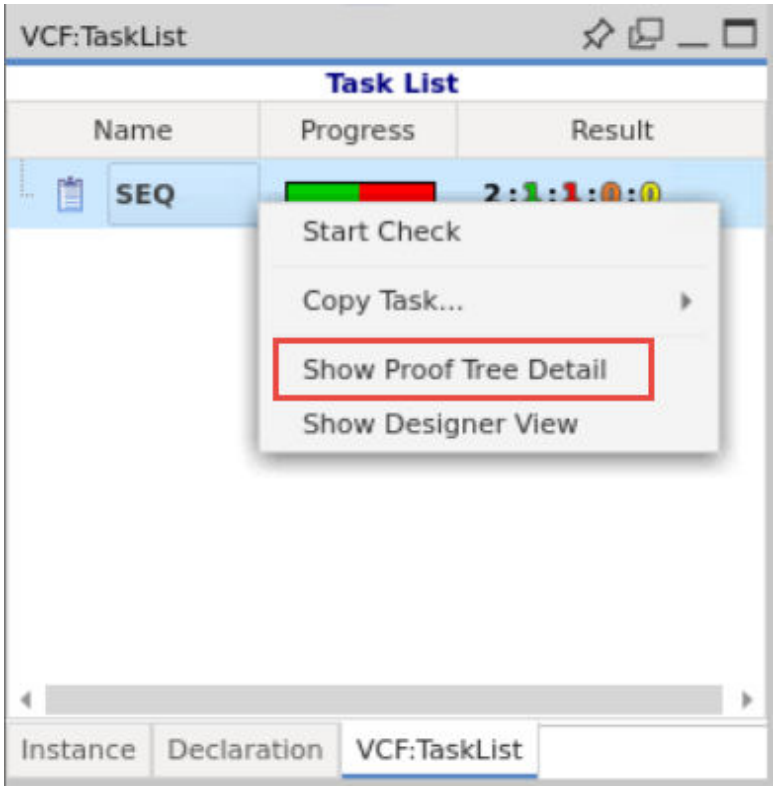
  Updating of the results is done automatically. At any point the results reported on the root proof are the actual results. In other words, if an assertion in the root proof is reported to be proven, it is actually proven given the assumptions. If it is reported to be falsified, then there exists a counter-example for that assertion.

14.10.3 Viewing the Proof Tree in Verdi

You can also view the proof tree in the VC Formal GUI.The orchestration view (or proof tree) in the Verdi GUI enables you to view the progress made by SEQ for a proof and to get information about proofs. Since the sub-proofs can be an abstraction of the original proof, the statuses of the properties can be conditional on the statuses of the other properties in the same proof. For example, it can happen that one of the output mappings in the decomposed proof is proven; however, it is conditional on all the other internal assertions being proven. In other words, if one of the internal assertions is falsified then the status of the output mapping is not valid anymore and the orchestration moves forward with the refinement step. The orchestration view contains information for each proof.

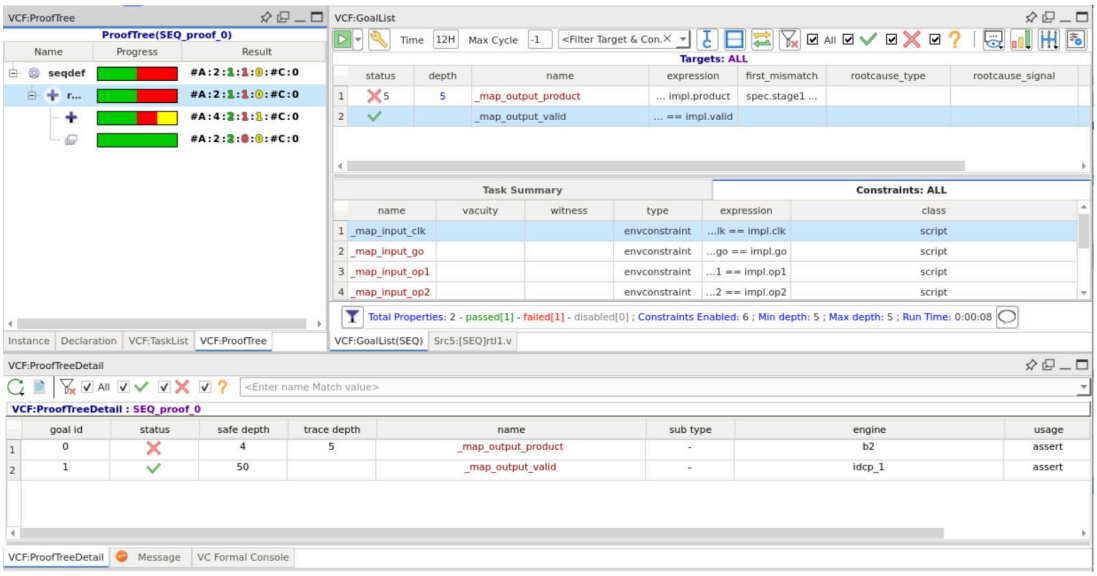
Feedback

1. Right-click a task in the task tree, and click *Show Proof Tree Detail*.



   - The Proof Tree detail opens a new view like the Instance tree as shown in Figure 110.

Figure 110: Orchestration View



- The Proof Tree view shows the proof tree based on the output of the `report_proofs -xml`command hierarchically.
- The icons of the nodes indicate the node type:

  Root:

  Decompose

  Or

The Proof Tree view indicates which proof node is active and which active proof node has ongoing progress. The orchestration view is updated automatically while checker is running.

- The progress bar in the proof tree shows percentage completed.
- The result column shows the number of assert properties (using color codes) and number of constraints.

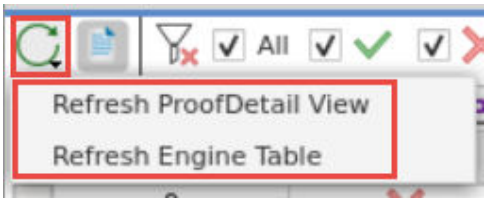
  Success count (green) including Proven, Covered, and Connected.

  Failure count (red) including Falsified, Uncoverable, Unconnected, and Vacuous.

  Inconclusive count (yellow) including Inconclusive, Checking, and Not-run.

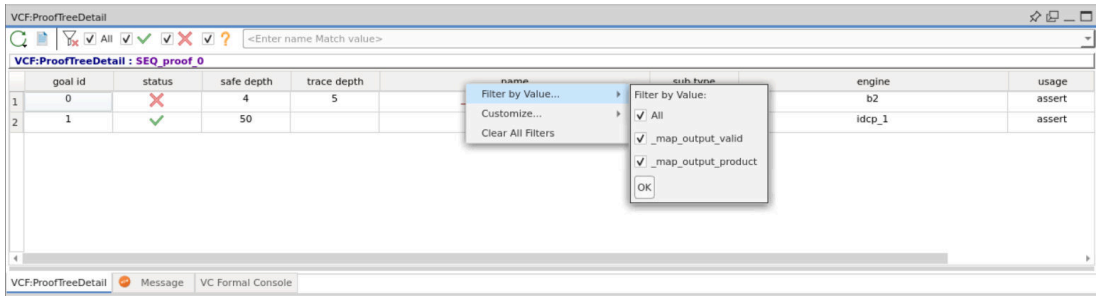
1. You can expand or collapse the nodes of the proof tree.

   The dummy nodes in the proof with no properties (from TCL command output) are merged into child nodes.
2. Click on a node of the *proof tree* item to view the property details of the node in the property table and the constraint table for that proof.
3. From the toolbar of the Proof Tree property table:
   a. Click either of the following to view the latest status of the nodes.



Feedback

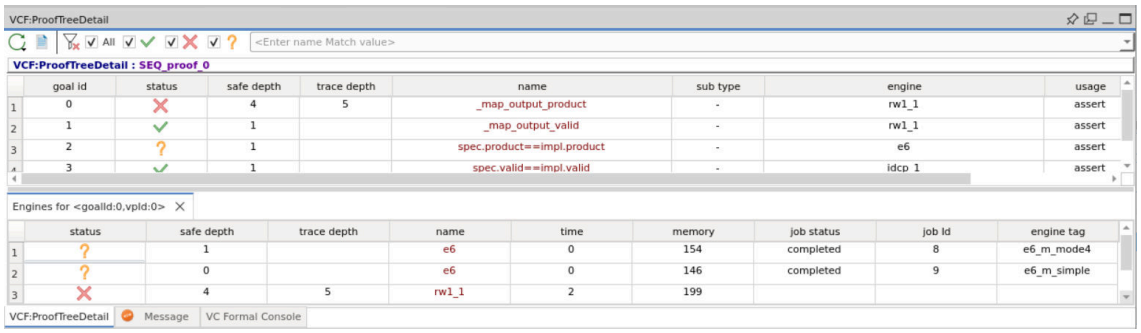
b. To filter the columns based on the values, right-click on the column header, and from the *Filter by value* box, select the values for the filter.



4. To view the engine details, right-click the engine name, and select *Show Engine Table*.

   The engines table appears as shown in Figure 111.

Figure 111: Engines Table



14.10.4 Viewing the status of the Internal Equivalence Points in the Verdi Source Browser

You can view the status of the internal equivalence points about a signal in context of a proof during a SEQ run in Verdi Source browser.

- To view information about a signal in the Verdi source browser, double-click the signal name in the orchestration view table.

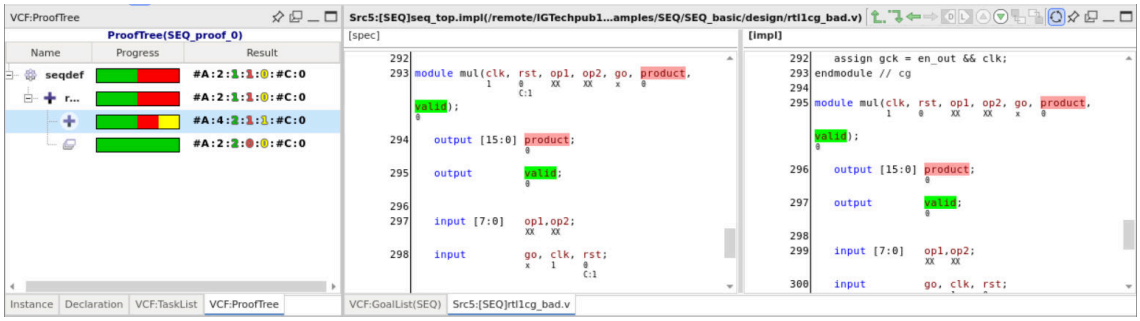
  A signal in a SEQ proof is either mapped or not mapped. For mapped signal, it can be

  - Equivalent or proven (marked in green)
  - Not Equivalent/Failed (marked in red)
  - Not Known/Inconclusive (marked in yellow)

Feedback

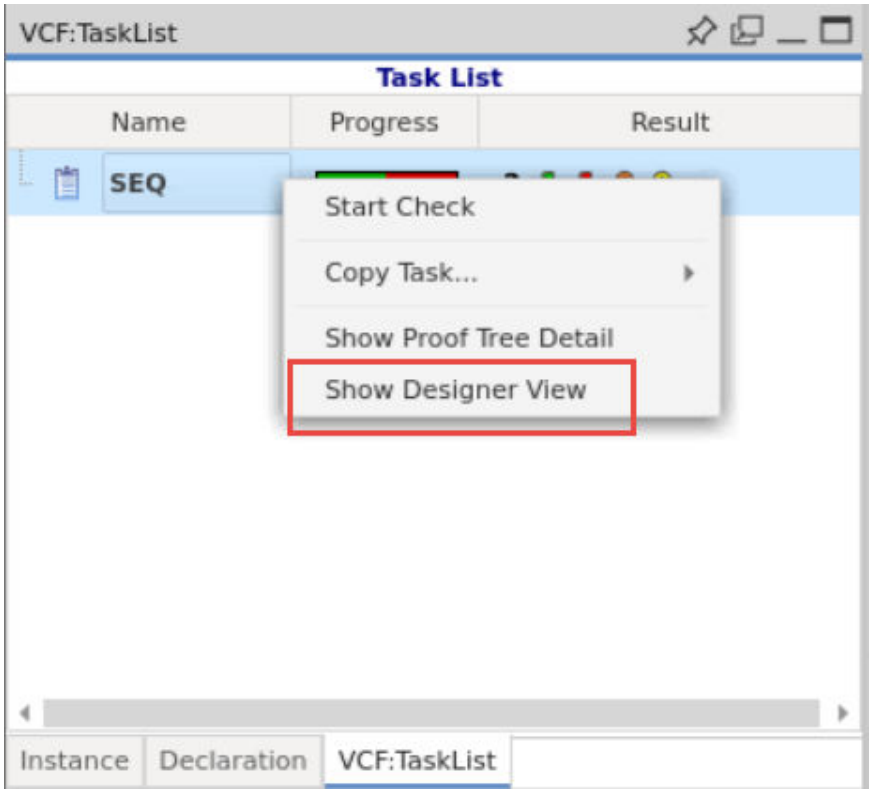
- To view the status of the internal equivalence points highlighted with the defined colors in the Verdi source browser, click in the *orchestration view* property table.

Figure 112: Status of the Internal Equivalence in Verdi Source Browser



14.10.5 Viewing the Designer View

1. In the RMB menu of task list, click *Show Designer View*.

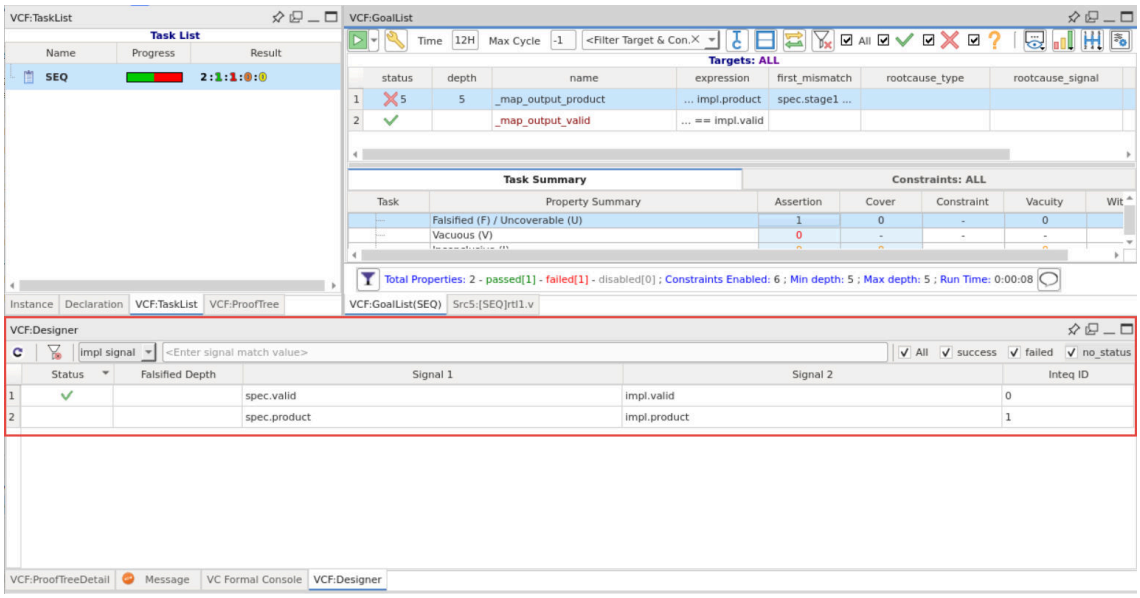


An example of the designer view is as shown in Figure 113.

Feedback

Besides the orchestration view as described in the previous section, designer view provides insight to the internal equivalence points between both the designs and the status of those while the properties on the outputs are being checked. The failures seen in designer view are true failures and can be debugged in Verdi waveform.

Figure 113: Designer View



The table shows the matched signals and the unmatched spec/impl signals, and signals outside of spec/impl.

1. To view the source (if available) of the signal name, double-click the signal name.
2. To view the trace for counter example (Engine dependency) of the falsified properties, double-click the falsified status.

14.11 Finding Root Cause of a Failure in SEQ

In SEQ, you can specify mappings on signals from the specification design to the implementation design. To debug a failure in SEQ, you can find the root cause of a failure rather than the falsified (mismatched) outputs miter.

The root cause of a failure in SEQ for a given trace is defined as:

- The first mismatched user-mapped equivalences split by the trace in the fanin cone of the failing output miter at a certain time cycle T.
- No other user-defined miters should fail at cycles less than T.

Feedback

The root cause analysis lists the registers that have different values compared to its mapped counterpart, and all the fan-in registers that have the same value compared to its mapped counterpart. The root cause can contain multiple pairs of registers. You can identify these registers in the trace waveform.

By default, the root cause analysis is enabled. You can disable this feature using the following application variable:

```
set_app_var fml_dfgsim_SEQ_root_cause_debug false
```

The root cause analysis is automatically performed when you use the `view_trace` command on a SEQ trace. The signal groups in the Verdi waveform display the root cause pairs as shown in Figure 114.

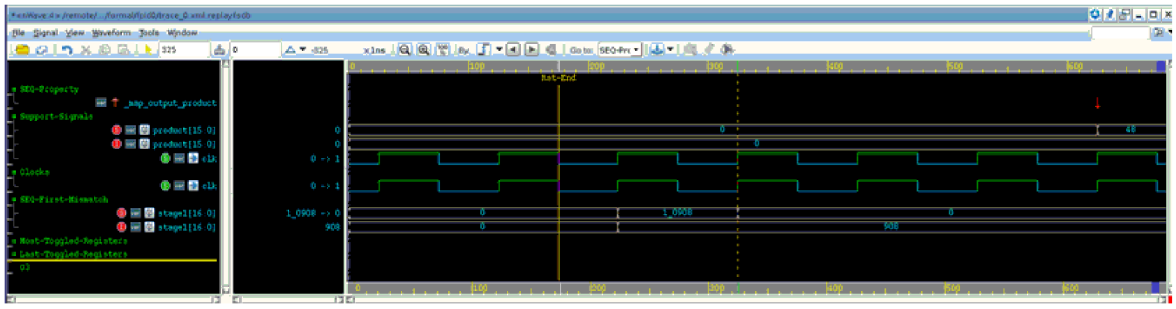
You can also specify the number of registers that must be displayed in the waveform for the root cause using the following formal variable. The default value of the `seq_root_cause_size` formal variable is 6. The `seq_root_cause_size`formal variable can take values from 0 to 100.

```
set_fml_var seq_root_cause_size < 0-100 >
```

Note:

Only registers are considered for root cause analysis.

Figure 114: Root Cause Pairs in the SEQ Trace



14.12 Performing Coverage Analysis for SEQ Mode

Coverage is the primary platform used in simulation to determine if the verification is complete. You can use the Verdi GUI to get the coverage metrics from simulation, that can be applied to SEQ verification to quantify the SEQ verification and to assist in coverage closure in conjunction with simulation based verification.

- To quantify SEQ verification, Verdi coverage GUI provides the following metrics:
  - Over-constraint Analysis: Analyze the coverage report to identify unreachable goals due to constraints.
  - Bounded Depth Coverage: Methodology to know if the required design behavior is covered within the stipulated proof depths.
- To assist simulation based verification, Verdi coverage GUI provides the unreachability analysis metrics. The unreachability analysis is a methodology to leverage the simulation coverage database and to formally generate waivers for code coverage metrics to ensure coverage closure.

You can perform coverage analysis for the SEQ application in the Verdi GUI using the VC Formal GUI. For more details on the VC Formal GUI, see section Debugging VC Formal Results in VC Formal GUI.

14.12.1 Opening Verdi Coverage GUI

To open the Verdi coverage GUI in the SEQ mode, click in the property table toolbar.

Figure 115: Property Table Toolbar

14.12.2 Types of Coverage Metrics in the SEQ Mode

The following types of analysis are available in the SEQ mode:

- Over Constraint + Bounded Unreachability Analysis (OA + BA)
- Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)

Over Constraint + Bounded Unreachability Analysis (OA + BA)

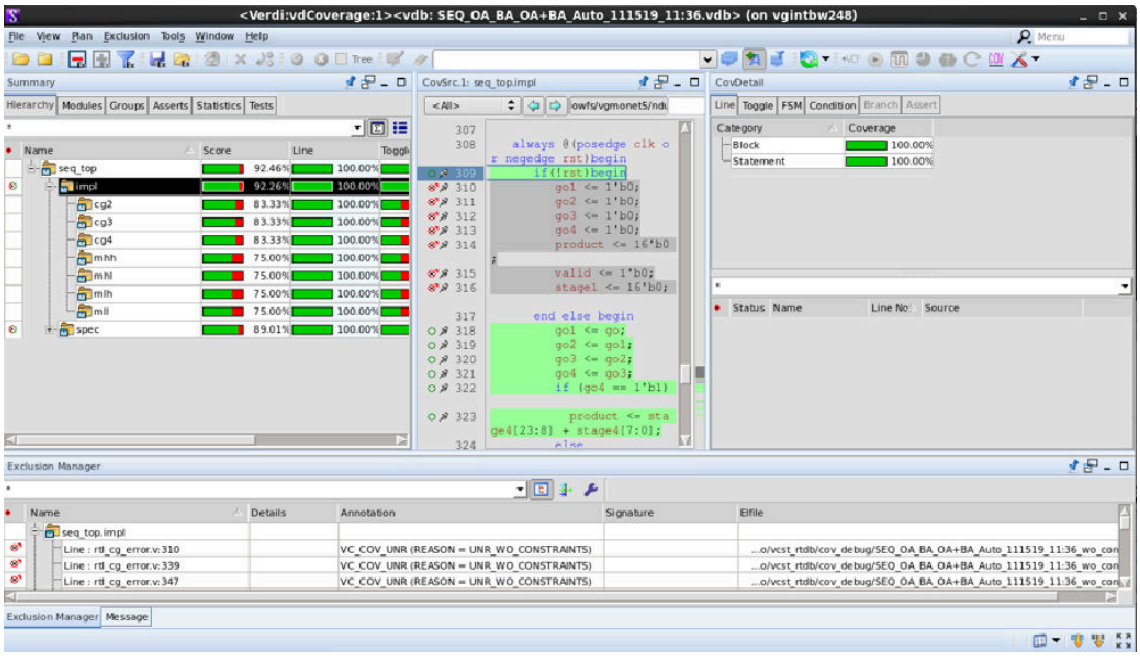
The *Unreachability + Over Constraint Analysis* (OA) enables you to identify unreachable code due to the user-defined constraints. This leads to analysis of unreachable code, which can be waived or can lead to constraint management.

The over-constraint analysis classifies the unreachable code into two categories.

- Unreachable: Signifies true unreachable (that is, dead code).
- Unreachable -over-constraints: Unreachable as an artifact of the constraint.

Feedback

Figure 116: Unreachability + Over Constraint Analysis (OA)



VC SEQ automatically saves a coverage database for the covered goals and exclusion files along with the reason attribute indicating whether the uncoverable is a due a constraint or a dead code.

Review the related constraints to determine if it is over-constrained. If it is an over-constraint, revise the constraint and reanalyze the constraint. If it is a dead code, exclude it in the Verdi Coverage GUI.

When the verification is complete, the property table is auto-refreshed. You can verify if the required properties are covered after revising the constraints. In addition, you can identify the impact of revising the constraints.

Note:

You can perform the Over Constraint analysis, by setting the depth -1.

The *Over Constraint + Bounded Unreachability Analysis (OA + BA)*enables you to identify unreachable code for a minimum bounded depth (K with respect to the reference clock) across the assertions, which have results as a bounded proof. In this mode, you can specify the depth to which the bounded coverage analysis must be performed.

Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)

The *Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)*enables you to identify unreachable code for one property for a minimum bounded depth (K with respect to the reference clock) across the assertions, which have results as a bounded proof. In this mode, you can specify the depth to which the bounded coverage analysis must be performed for one property.

14.12.3 Viewing Coverage Metrics in the SEQ Mode

When you invoke any type of analysis, VC SEQ automatically switches to COV mode and creates a new task. The OA+BA and OA+PBA analysis initialized from a SEQ task (named SEQ) is named as SEQ_OA_BA, and SEQ_OA_PBA_N (where N is an index for that increments from 0) respectively. When you perform OA+BA analysis, only one task from the same SEQ mode is created in the COV mode. Whenever you switch to a certain COV mode if the specific COV mode task is already present, then the existing one is deleted and a new task is created.

In the *Max cycle*box in the property table toolbar, configure the minimum depth for OA+BA and OA+PBA. The default value in the OA+BA mode is the minimum depth in the task, and in the OA+PBA mode, it is among the selected properties.

Note:

Automatic instrumentation is not supported. You must specify the metric using the -cov switch in the `read_file` command.

Verdi coverage is automatically invoked after the first covered property is identified. The icon is enabled in the property table toolbar. You can use this button in case you close the Verdi coverage accidentally. The Covdb and elfile are automatically saved for the current mode and is loaded into the Verdi Coverage.

Verdi coverage has a button for manually updating data from the VC Formal SEQ. As the checker progresses, you can click the button to load a newer version of covdb (elfile). After the checker is done, a dialog box appears to check if the final version of the data can be loaded.

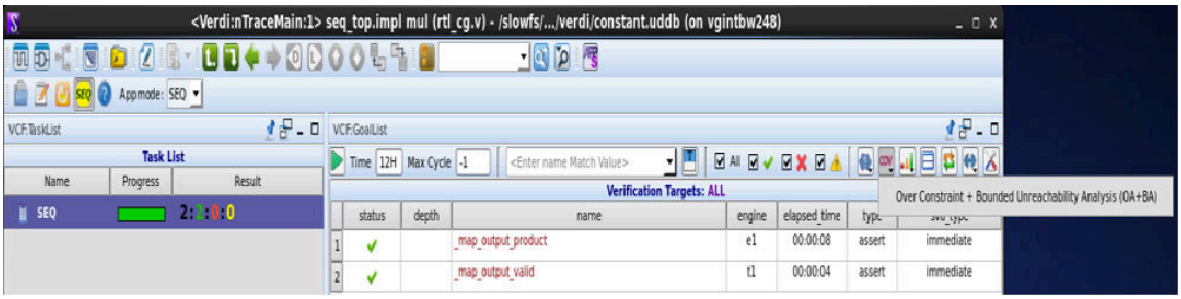
Viewing the (OA + BA) and (OA + PBA) Metrics

By default, you can view the *Over Constraint + Bounded Unreachability Analysis (OA + BA)*metrics in the SEQ mode.

From the property table toolbar, select *Over Constraint + Bounded Unreachability Analysis (OA + BA)*from the drop-down list as shown in Figure 117.

Feedback

Figure 117: Over Constraint Analysis + Bounded Unreachability Analysis (OA + BA)



When you perform *OA + BA*, a dialog box appears to query for the depth to which the SEQ run must be performed. A default value is inferred from the SEQ result.

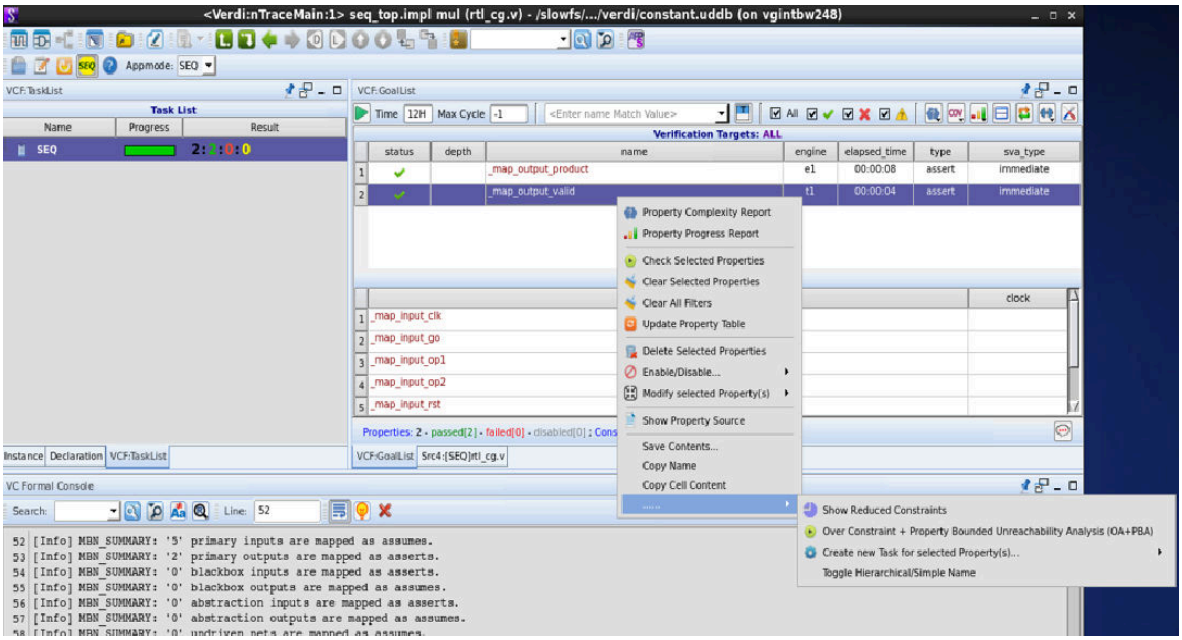
Note:

If you want to perform only OA analysis, specify the depth as -1.

To view the *Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)*for one property, from the RMB menu in the property table, select *Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)*as shown in Figure 118.

Feedback

Figure 118: Over Constraint Analysis + Per-property Bounded Unreachability Analysis (OA + PBA)



14.12.4 Viewing the Source for COV Properties

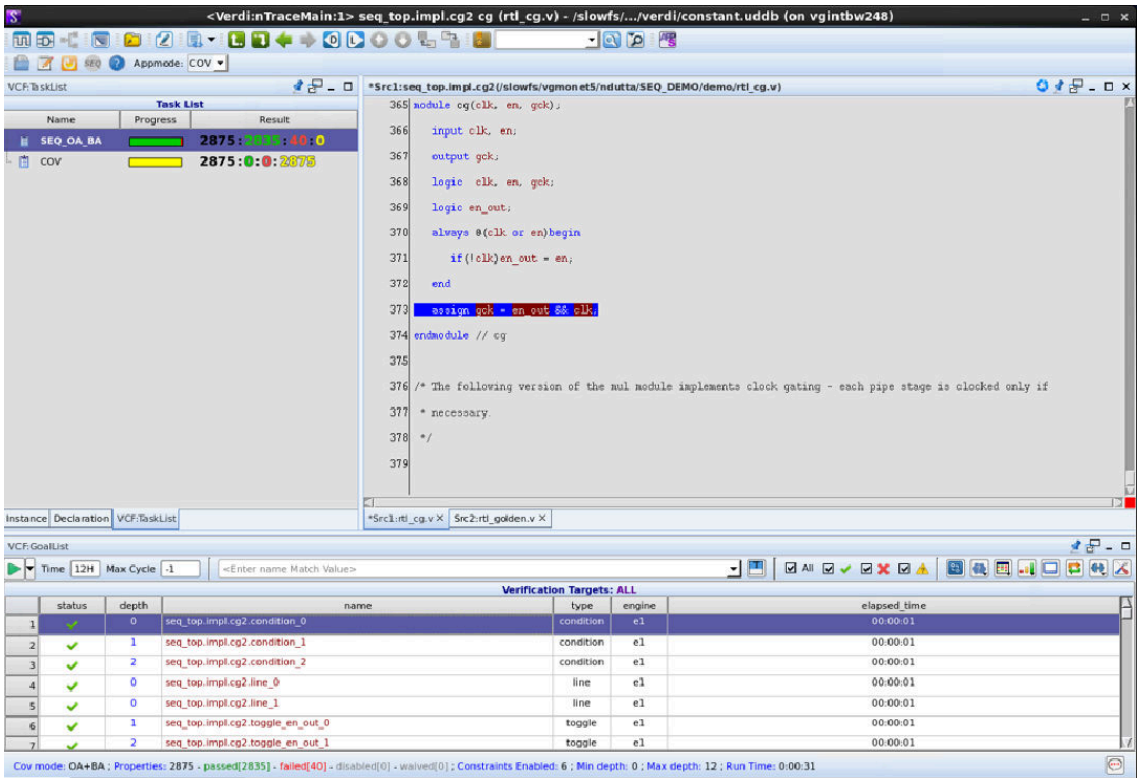
When you perform coverage analysis ((OA + BA) and (OA + PBA)) in the SEQ mode, the source code is displayed in 1-source-window mode.

By default, this mode is displayed irrespective of whether the spec and implementation design are different or not. The source of only one design, that is either the implementation or the spec design is displayed.

Feedback

- If you click on a property, the source location of that property is highlighted in the corresponding source window, that is, if you click an implementation design property, the source of the property in the implementation design source window is highlighted.

Figure 119: 1-source-window Mode



14.13 Performing Formal Core in SEQ Mode

You can review the Formal core of a property in SEQ mode.

For more information on what is formal core of a property, see section Formal Core (FC) Analysis.

To compute formal core of a property in Tcl flow, see section Use Model for Computing Formal Core. Formal core can only be run on internal equivalence points from orchestration view and not on the output property.

Feedback

14.14 VC SEQ Known Limitations

VC SEQ capabilities are available with the following limitations:

- Limited support for signal with escaped names

  Certain designs with special escaped names such as *\a&&b* are known to have problems during mapping using the `-map_uninit` option of the `seq_config` command.

14.15 SEQ Clock Gating Covers

The following two commands are introduced to add cover properties for non-clock signals present in the COI.

- `add_cg_covers`: This command adds toggle cover properties for non-clock signals present in the COI of clock nets. It will find all flops in the given scope and for each flop it will trace the driver of the clock pin. It will collect all nets in the COI of the clock pin. It will stop if it reaches an input or a sequential after number of levels are exhausted.

  For the collected nets, it will add toggle cover property.

```
vcf> add_cg_covers -help
Usage: add_cg_covers    # Adds the toggle cover properties for
 non-clock signals present in the COI of clock nets
        [-new_task <newTaskName>]
                                (Name of the new task. If not
 specified, add_cg_covers would be invoked on the current task)
        [-bit]                  (Do bit traversal instead of word for
 COI of clock nets of sequentials)
        [-scope <list-of-scopes>]
                                (Add properties for all sequentials
 inside this scope)
        [-sequentials <list-of-sequentials>]
                                (List of sequentials)
        [-nets <list-of-nets>] (List of nets)
        [-seqDepth <seqDepth-to-trace>]
                                (The number of levels of sequentials to
 trace)
        [-clean]                (Clean old cg cover data)
        [-verbose]              (Outputs verbose information about
 CG cover properties                                  which gets
 generated)
        [-prefix <prefixName>] (String to be prefixed to the name of
 the CG cover                                  properties which will
 get generated)
```

- `report_cg_covers`: This command will report the flops (as discovered by `add_cg_covers`), signals found in COI of the clock pins of these flops and toggle cover properties (as created by `add_cg_covers`), their attributes and status of properties.

Feedback

```
vcf> report_cg_covers -help
Usage: report_cg_covers    # Reports the toggle cover properties
 created for non-clock signals present in the COI of clock nets
        [-scope <list-of-scopes>]
                                (Report properties for all sequentials
 inside this scope)
        [-status <status-value>]
                                (Report properties having the given
 status:
                                 Values: covered, uncoverable,
 checking,
                                 not_run, inconclusive)
        [-sequentials <list-of-sequentials>]
                                (Report properties for the given list
 of sequentials)
        [-nets <list-of-nets>] (List of nets)
        [-seqDepth <seqDepth-to-trace>]
                                (Report properties created for the nets
 which are present                                in the COI upto
 given seqDepth)
        [-verbose]              (Outputs detailed report)
        [-show_origin]          (Outputs origin sequential/net)
```

15

Supported AIPs

This section describes AMBA Bus protocol checkers also known as Assertion IP (AIP) and its supported protocols available in compliance with Formal verification environment.

This section includes the following topics:

- About AIPs
- Methodology
- Supported Protocols and Verification Features
- Performance and Convergence
- Formal Variables Relevant for AIP

15.1 About AIPs

Assertion IPs have pre-built assertions that help to verify and signoff the IPs in System on Chip (SoC) designs. They can significantly reduce verification effort and improve the design quality by monitoring a target design without modifying the corresponding RTL, and they can be applied in formal verification without simulation test-benches. In addition, AIPs help to setup Formal environment quickly, because of the benefit of plug-in usage and sanity test on various configurations. AIP is a perfect solution to a multi-manager to multi-subordinate bridge design or scenario. Using the supported AIPs not only reduce the effort of designing the interface protocol checkers but also minimize the concerns of AIPs' quality.

All the VC Formal applications require specific license keys. For more information on the license keys required for each of the VC Formal applications, see the *VC Static Platform Installation Notes*.

Note:

The AIP packages is available in the VC Static installation directory at the following location: *$VC_STATIC_HOME/packages/aip/*

15.2 Methodology

15.2.1 Inputs to this AIP

Before using this AIP, a clearly defined specification for design is required. The specification must contain the following components:

- Protocol of the Interface
- Compliance version of the Interface
- What is not used in the protocol
- What is beyond the standard protocol

In the list above, firstly, the protocol of the interface states which AIP you need to use. It is the basic knowledge of the design interface. And if there are multiple interfaces exist, each of them should be defined. A table to collect all the protocols are suggested.

Secondly, it must also be clearly specified which protocol version is implemented on this design. E.g., there is no wid in AXI4; however, AXI3 has it. The unambiguity must be as documented clearly to avoid any unexpected extra efforts between designers and verification engineers.

Thirdly, although AMBA protocol provides a strong prototype for the IP development, sometimes they become overdesign to meet your design. For example, if the lock function defined in AXI protocol is not required in your design, it should be put in the specification. This rule should be strictly followed such that any function configured to be disable in the verification environment can map to a custom function explicitly. It can reduce the risk of missing verification due to lacking clear documentation.

Lastly, anything goes beyond the standard protocol is another topic to be written down in the document. It is because the standard AIP provided to you may not cover the function required in your design. This part will make a clear statement that should be taken care of by verification engineers in addition to applying the AIP on the verification testbenches. For any complicated modification or extension on the AMBA standard protocol, VC Formal AE team also provides service on consulting. AEs provide solutions to your question dedicatedly. If you have such requirement, contact your AE for more information.

After you got a full specification with the items listed in above, see section Supported Protocols and Verification Features for next step. There are usage documents for every protocol, in which the signal names and configurations are provided in detail. Set those configurations to meet your specification requirement.

Feedback

15.2.2 What is look for and what to avoid

In the verification testbench with AIP, the early run is very likely to report several of failures due to design's low maturity or low testbench quality. The first step to a successfully AIP-applying story in your testbench is to start from the vacuity checks. The failure of vacuity checks implies unreachable scenarios in your testbench. They can be misused of the assumptions in the testbench or due to design implementation. It is a chance to conduct a review of the verification plan with specification architects, RTL designers, and verification engineers.

For example, Manager DUT with AXI interface can always accept BVALID; therefore, BREADY is tied to 1. In this case, ast_snps_axi_bresp_stable will be vacuous because every B-channel handshaking is finished in one cycle. So, it is impossible to reach the condition to check the stability of BRESP. User needs ignore this according to their design specification. In summary, user is suggested to debug or ignore vacuous depending on case.

And it is not necessary to look at AIP internal signals during debugging. The most important thing is to determine what the protocol violation is. In another words, which signal's behavior of the interface causes protocol violation. User must focus on bus interface signals.

15.2.3 The aip_load Command

Use the `aip_load` command to preload AIPs for analyze before elaboration. With this command, you need not specify AIP files manually in the `analyze` command.

Syntax

```
vcf> aip_load -help
Usage: aip_load    # To pre-load aip files.
        [-protocol <List of protocol(s)>] (Specifies the aip protocol:
                                Values: APB, AHB, AXI3, AXI4, AXI5,
                                AXI_STREAM, ACE, ACE5, CHI, LPI, GIC,
                                RISCV, I2C, SPI, SPMI, FSCB, all)
        [-aip_src_path <aip_src_path>] (Specifies the source directory
 for AIP)
```

Feedback

Where,

- `protocol <Bus Protocol>`: Use this option to pass the bus protocol(s) expected to pre-load. If no protocol is given then all protocols would be picked up for pre-loading and the following message is reported:

  [Info] AIP_LOAD_ALL_PROTOCOLS_TOBE_PICKED: The command 'aip_load' has been called without specifying a protocol('-protocol' option not used), hence all supported protocols (APB, AHB, AXI3, AXI4, AXI5, AXI_STREAM, ACE, ACE5, CHI, LPI, GIC, RISCV, I2C, SPI, SPMI, FSCB) will be picked up for pre-loading.

- `aip_src_path <Path to load AIPs>`: Use this option to specify the path (directory) from where the user wants to preload the AIPs.

Example 1

The following commands

```
analyze -format sverilog -vcs " -f aip.f "
aip.f
incdir${AIP_PATH}/APB_AIP/src
${AIP_PATH}/APB_AIP/src/snps_apb_aip_pkg.sv
${AIP_PATH}/APB_AIP/src/snps_apb_aip.sv
```

can be replaced by

```
aip_load -protocol APB
```

or

```
aip_load
…
elaborate -sva top
```

Example 2

- To pre-load all Synopsys AIPs under *$VC_STATIC_HOME/packages/aip* and analyze user's bind files:

  ```
  aip_load
  analyze ...
  elaborate ...
  ```

- To specify a user-defined AIP directory, use the following:

  ```
  aip_load -aip_src_path ../AIP_patch/
  analyze ...
  elaborate ...
  ```

Feedback

15.3 Supported Protocols and Verification Features

Table 25 provides the list of supported AIPs and the supported protocols that are available in compliance with Formal verification. All packages of AMBA AIPs has its' corresponding usage document. It is available under the path: $VC_STATIC_HOME/packages/aip/XXX_AIP/doc/XXX_AIP.pdf. Every usage document includes the following sections.

- Introduction
  1. Compliance Version of AMBA Protocol
  2. Available Features
- Installation and Setup
  1. Workstation Requirements
  2. VC Formal Version Requirements
  3. OS Requirements
  4. License Policy
- The AIP in the Formal Verification Environment
  1. How It Looks Like to Use This AIP in VC Formal
- The AIP Configuration
  1. All Available Configurations
  2. Configuration Variables Explanation
- The AIP use case
  1. An example verification Test-bench with Referenced RTL and This AIP

Table 25: Supported AIP packages

Assertion IP Protocol	Specification Compliance	Usage Documents ($VC_STATIC_HOME/packages/aip/)
APB	AMBA specification: ARM IHI 0011A APB2 ARM IHI 0024B APB3 ARM IHI 0024C APB4	APB_AIP/doc/APB_AIP.pdf
AHB	AMBA specification: ARM IHI 0011A AHB ARM IHI 0033A AHB-Lite ARM IHI 0033B.b AHB5	AHB_AIP/doc/AHB_AIP.pdf
AXI3	AMBA specification: ARM IHI 0022E AXI3	AXI3_AIP/doc/AXI3_AIP.pdf

Feedback

Table 25: Supported AIP packages (Continued)

Assertion IP Protocol	Specification Compliance	Usage Documents ($VC_STATIC_HOME/packages/aip/)
AXI4	AMBA specification: ARM IHI 0022E AXI4 ARM IHI 0022E AXI4-Lite	AXI4_AIP/doc/AXI4_AIP.pdf
AXI5	AMBA specification: ARM IHI 0022H.c AXI5 ARM IHI 0022H.c AXI5-Lite	AXI5_AIP/doc/AXI5_AIP.pdf
AXI_STREAM	AMBA specification: ARM IHI 0051B	AXI_STREAM_AIP/doc/AXI_STREAM_AIP.pdf
ACE	AMBA specification: ARM IHI 0022E ACE-Lite ARM IHI 0022E ACE	ACE_AIP/doc/ACE_AIP.pdf
ACE5	AMBA specification: ARM IHI 0022H.c	ACE5_AIP/doc/ACE5_AIP.pdf
CHI	AMBA specification: ARM IHI 0050A CHI ARM IHI 0050B CHI ARM IHI 0050C CHI ARM IHI 0050D CHI ARM IHI 0050E.b CHI	CHI_AIP/doc/CHI_AIP.pdf
LPI	AMBA specification: ARM IHI 0068D	LPI_AIP/doc/LPI_AIP.pdf
GIC	GICv3, GICv4 ARM IHI 0069H	GIC_AIP/doc/GIC_AIP.pdf
RISC-V	The RISC-V Instruction Set Manual Volume I: Unprivileged ISA Document Version 20191213.	RISCV_AIP/doc/RISC-V_AIP.pdf
I2C	UM10204 I2C-bus specification and user manual (Rev. 7.0 – 1 October 2021) Synopsys DesignWare DW_apb_i2c Databook (Version 2.04a - April 2023)	I2C_AIP/doc/I2C_AIP.pdf
SPI	Microchip Advance Information Section 23. Serial Peripheral Interface (SPI) Texas Instruments SPRUHM8I Chapter 18. Serial Peripheral Interface (SPI) (Revised September 2019)	SPI_AIP/doc/SPI_AIP.pdf
SPMI	MIPI® Alliance Specification for System Power Management Interface (SPMI) Version 2.0 – 14 March 2012	SPMI_AIP/doc/SPMI_AIP.pdf

15.4 Performance and Convergence

Formal verification exhaustively verified all the possible state-space. It can be easily imagined the complexity of formal verification grows exponentially. At the background of formal engine, it is a NP-complete problem. So, it is natural when the complexity is high (e.g., FIFO depth is high or outstanding number is big) the properties in the AIP may bound to certain depth rather than reaching the proven status. See section Considerations for Convergence Improvement to leverage several skills for improving convergence.

In this chapter, solving convergence problems might not be the only way to complete your verification. For example, considering a design that is a 8-AXI-managers to 1-AXI-subordinate arbiter. And let's assume the protocol at the AXI subordinate side cannot be fully proven. How can we proceed our formal verification on this module?

Although the property is not fully converged at AXI subordinate interface. Instead of pushing the limit on those inconclusive properties, alternatively, you can use formal scoreboard to prove that any transaction issued from these 8 AXI manager interfaces can be transferred to the subordinate side without being manipulated, discarded, wrongly ordered. Then it also implies the AXI protocol is proven because these 8 AXI managers are assumed to follow the AXI protocol.

This is a high-level illustration to reach a high quality of verification without solving the original problems of convergence. By carefully analyzing the verification goals, a verification problem can be addressed from different aspects. Since it is an advanced usage with AIP, if you need the methodology beyond the Considerations for Convergence Improvement for convergence issue, contact your AE for comprehensive solutions.

15.5 Formal Variables Relevant for AIP

To enable the checkers of no X propagation on the interface with AIPs, set this variable: `fml_x_detect_mode` to 2, or use the VC Formal FXP application.

```
set_fml_var fml_x_detect_mode 2
```

For more details on VC Formal FXP, see section About FXP Application.

For other formal variables, see the corresponding chapter of each VC Formal application. For example, if you are using FTA, see the Application Variables and Formal Variables for FTA section for the applicable variables in FTA.

16

Formal Register Verification Application

The VC Formal FRV application helps to formally verify behavior of configuration registers for respective attributes like “read only”, “read/write” or “reset value” eliminating the need for directed simulation tests.

Getting Started • *What is VC Formal FRV?* • *Vendor Extensions Supported in the IP-XACT File* • *Vendor Extensions Supported in the RALF File* • *Accessing FRV Lab Example*	**Obtaining Signoff Coverage for FRV** • *Generating Verification Summary* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving and Restoring Sessions*
Setting Up and Running FRV • *Enabling FRV Application mode* • *Loading IPXACT Files* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Verifying Design Setup* • *Reviewing and Editing Properties* • *Running FRV in TCL*	**Getting Help on Commands** • *FRV Tcl Commands*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Enabling Regression Mode Accelerator* • *Using Proof Assist* • *Using Proof Architect*	

16.1 About Formal Register Verification

VC Formal support Formal Register Verification (FRV) application mode. The FRV application mode enables you to automatically create various formal verification checks for registers in the design. You can provide the register verification information in the IP-XACT format where all the registers and/or reg-fields are defined. VC Formal performs various checks on the registers based on the descriptions provided and reports the results of the

Feedback

checks. VC Formal also enables you to get a report of the register and/or reg-fields in the IP-XACT file.

Note:

Currently, FRV supports IP-XACT versions IEEE 1685-2014 and IEEE 1685-2009.

This video focuses on the Formal Register Verification (FRV) app in VC Formal, designed to ensure register models align with their specifications, for example, IP-XACT, CSV, RALF. Traditional simulation-based verification is manual, time-consuming, and non-exhaustive. In contrast, FRV leverages formal methods for faster, exhaustive verification, identifying corner-case bugs efficiently.

16.1.1 Video Links

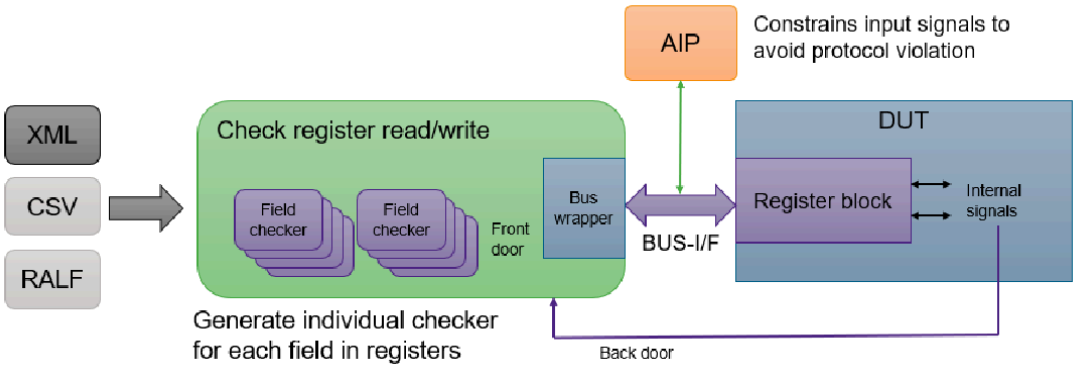
Click the links to view videos on FRV:

- Introduction to FRV (5 mins)

16.2 Methodology

The FRV application takes in RTL along with a register specification as input. You can specify register definitions in the form of an XML, CSV or RALF format and pass it to the tool as shown in Figure 120. The tool then extracts assertions and covers based on the RTL and the register specification and verifies if register intent for example RO/RW etc... is being honored. In most cases you will also need a Formal Verification environment, or an AIP as shown in Figure 120 to constrain the input signals to avoid false failures from protocol violations. VC Formal FRV is best applied at the block level. You can also apply it at the SoC level, but complexity issues might prevent convergence at this level.

Figure 120: VC Formal FRV Methodology



Feedback

16.3 Performing FRV on your Design

To use VC Formal FRV application:

- Set the application mode to FRV using the `set_fml_appmode FRV` command.
- Use the `frv_load` command to load the register specification file into VC Formal, use `-auto_load`to automatically load the checker file and all relevant FRV checker library files.

```
frv_load -ipxact/-ral/-csv <IP-XACT/RALF file/csv file> -auto_load
```

- If there are any modules/instances to be black boxed, specify them before the `read_file/analyze` command.

```
  set_blackbox -design <module_names>
      set_blackbox -cells <hierarchical instance_names>
      report_blackbox
```

- Create a bind file to bind FRV bus wrapper with the design. You can include it using the following command or you can also include it in the next step.

```
frv_load -ipxact/-ral/-csv <IP-XACT/RALF file/csv file> -auto_load
 -bind <bind_file>
```

- Include the following files for compilation with `read_file/analyze`:
  - Design files
  - SVA checker files
  - Bind file, if not provided in `frv_load`.
- Specify clocks, resets and all other initial states. For more information, see sections
  - Setting Up User-Defined Clocks
  - Setting Up User-defined Resets
  - Specifying the Initial State
- The operations of Proof and Debug are also like FPV. See sections Controlling check_fv Runs and Debugging VC Formal Results in VC Formal GUI respectively for more information.
- Use the `report_fv` command to get the results. Use the `frv_report` command to get register data.

Feedback

16.3.1 FRV Example

To get hands on with the FRV application, use the FRV example in the VC Formal installation directory.

```
<VC_STATIC_INSTALL>/doc/vcst/examples/FRV
```

16.4 Commands for Performing Register Checks

VC Formal provides the following commands for performing checks on the registers.

- `frv_load`
- `frv_report`
- `frv_config`

16.4.1 The frv_load Command

Use the `frv_load` command to provide the register specification to VC Formal for verifying the registers. This command creates the required checkers for verifying the registers. This command returns 1 if successful, else returns 0.

The `frv_load` command reports the following message, if successful:

[Info] SVA_NEW_CHECKER_FILE_SAVED: SVA Checker file ‘<work_dir>/frv.sv' generated successfully

The assertions generated by the `frv_load` command are shown in Table 26.

Table 26: VC Formal FRV Assertions

Assertion Name	Description	Check Path	Conditions to Check
`ast_<attr>_read_data_check`	Read field should match with:reset value for the first read before writethe latest written field after write	Bus	When initial value specified and COMPOSITE is set to 1, read-only, read-write
`ast_<attr>_initial_read_check`	Read field should match with reset value for the first read before write	Bus	When initial value specified and COMPOSITE is set to 0, read-only, read-write

Feedback

Table 26: VC Formal FRV Assertions (Continued)

Assertion Name	Description	Check Path	Conditions to Check
`ast_<attr>_write_read_check`	Read field should match with the latest written field	Bus	When no initial value specified or COMPOSITE is set to 0, read-only, read-write
`ast_<attr>_bdr_initial_check`	Internal signal should match with the initial value at the first cycle after reset	Backdoor	When initial value specified, Any access type
`ast_<attr>_bdr_read_check`	Read field should match with internal signal before specified cycle	Bus, Backdoor	When backdoor signal specified, read-only, read-write
`ast_<attr>_bdr_write_check`	Write field should be reflected to internal signal after specified cycle	Bus, Backdoor	When backdoor signal specified, write-only, read-write
`ast_out_adblk_read_check`	Verify that the read data is equal to specified value for out of addressBlock range. **Note:** If design returns an ERROR response for out of addressBlock access, the assertion becomes vacuous. In such cases, set OUTABLK_MD to 2 to disable the assertion.	Bus	Read occurs for out of addressBlock range

The covers generated by the frv_load command are shown in Table 27.

Table 27: VC Formal FRV Covers

	Property Name	Description
1	`cov_first_read_before_write`	the first target register read before the target register write
2	`cov_first_read_after_write`	the first target register read when the target register write already occurred
3	`cov_read_after_read_before_write`	the target register read when the target register read already occurred, but the target register write hasn't occurred yet

Feedback

Table 27: VC Formal FRV Covers (Continued)

	Property Name	Description
4	`cov_read_after_read_and_write`	the target register read when both the target register write and read already occurred
5	`cov_diff_addr_write_between_write_read`	the sequence of the target register write -> different address register write -> the target register read
6	`cov_diff_addr_read_between_write_read`	the sequence of the target register write -> different address register read -> the first target register read
7	`cov_out_addr_write_between_write_read`	the sequence of the target register write -> write for out of addressBlock range address -> the target register read
8	`cov_out_addr_read_between_write_read`	the sequence of the target register write -> read for out of addressBlock range address -> the target register read
9	`cov_diff_addr_read_before_first_write`	the sequence of different address register read -> the first target register write
10	`cov_out_addr_read_before_first_write`	the sequence of read for out of addressBlock range address -> the first target register write
11	`cov_diff_addr_write_before_first_read`	the sequence of different address register write -> the first target register read
12	`cov_out_addr_write_before_first_read`	the sequence of write for out of addressBlock range address -> the first target register read
13	`cov_read_only_write`	the target register write to make sure write occurred for read-only register
14	`cov_write_only_read`	the target register read to make sure read occurred for write-only register
Not Generated As default		
15	`cov_user_addr_write_between_write_read`	the sequence of the target register write -> write for user specified address -> the target register read
16	`cov_user_addr_read_between_write_read`	the sequence of the target register write -> read for user specified address -> the target register read
17	`cov_user_addr_read_before_first_write`	the sequence of read for user specified address -> the first target register write

Feedback

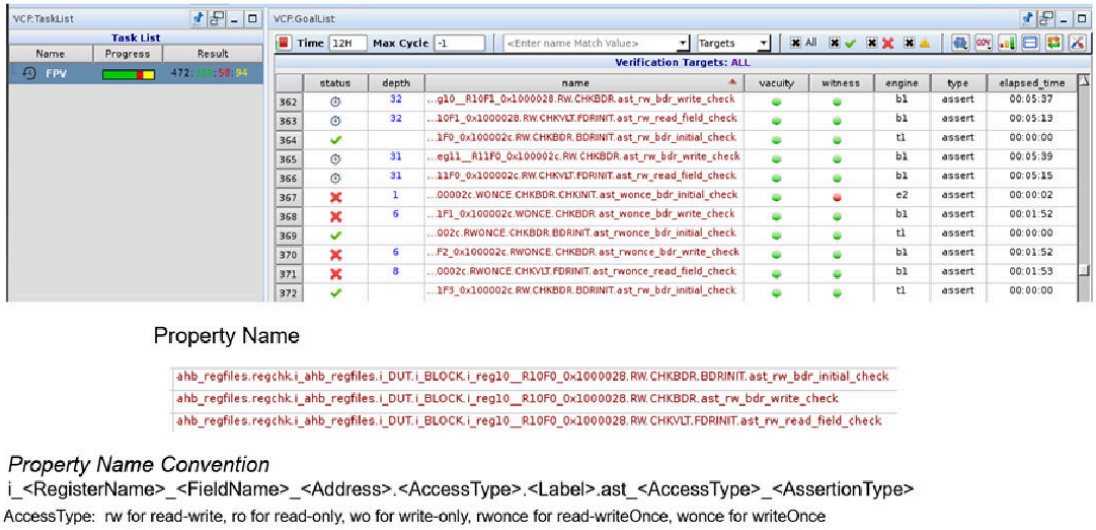
Table 27: VC Formal FRV Covers (Continued)

	Property Name	Description
18	`cov_user_addr_write_before_first_read`	the sequence of write for user specified address -> the first target register read
New Properties from R-2020.12-SP2		
19	`cov_diff_addr_write_before_first_write`	the sequence of different address register write -> the first target register write
20	`cov_out_addr_write_before_first_write`	the sequence of write for out of addressBlock range address -> the first target register write
21	`cov_user_addr_write_before_first_write`	the sequence of write for user specified address -> the first target register write
22	`cov_diff_addr_read_before_first_read`	the sequence of different address register read -> the first target register read
23	`cov_out_addr_read_before_first_read`	the sequence of read for out of addressBlock range address -> the first target register read
24	`cov_user_addr_read_before_first_read`	the sequence of read for user specified address -> the first target register read
New Properties from U-2023.03-SP2		
25	cov_valid_byte_lane_updated	indicates whether valid byte lanes are updated for given register field

The assertion notation generated by the tool is shown in Figure 121.

Feedback

Figure 121: FRV Assertion Notation



Syntax

```
frv_load  [-ipxact <IP-XACT File> [-ral <RALF File>] [-csv <file-name>]
 [-bind <Bind File>] [-top [comp|mmap|block]:<Component_or_mmap_or_block
 Name>] [-checker_file <Checker File>] [-include <Include type list>]
 [-auto_load] [-parameters <List of parameter(s)>] [-verbose]
```

Where,

- `[-ipxact <IP-XACT File>]`: Use this option to provide the IP-XACT file that specifies the register descriptions.
- `csv <file name>`: Provide the input CSV file containing register description. For more details on the format of the CSV file, see section CSV Format for Register Specification.
- `[-ral <RALF File>]`: Use this option to specify the input RALF file containing register description.
- `[-bind <Bind File>]`: Use this option to provide the bind file that binds the register verification checkers to the design for the specified protocol.
- `[-top [comp|mmap|block]:<Component_or_mmap_or_block Name>]`: Use this option to limit the register checks to a specific component, memory map or address block name in the input file. If the subtype `(`that is,`comp|mmap|block)` is not provided in the `-top` option, then the lookup order is:*comp, mmap, block*.

  Examples

  ```
  frv_load -top comp:<component_name>
  frv_load -top mmap:<memory_map_name>
  frv_load -top block:<block_name>
  ```

Feedback

If there exists multiple component/memoryMap/addressBlock with the same names, you can specify hierarchical name with the `-top` option.

Example

For multiple addressBlocks with the same name, the hierarchical top name can be specified as follows:

```
frv_load -top block:compName.mmapName.blockName
```

- `[-checker_file <Checker File>]`: Use this option to provide specific file names for the checkers created. The checker files are used by VC Formal to perform checks on the registers. By default, the file name for the checker created is *frv.sv*.

  If you add the frv.sv in the filelist, and provide a register specification with IP-XACT format, the only required option of the frv_load command is the -ipxact option. If you add the frv.sv in the filelist, and provide register specification with RALF format, the only required option of the frv_load command is -ral.

  The FRV application can generate the SVA checker file with the same interface regardless of the DUT bus interface. In case, the bus interface is AMBA such as apb, ahb or axi, the bus wrappers are available in *$VC_STATIC_HOME/packages/Frv* directory, and you do not have to create a wrapper.

  $VC_STATIC_HOME/packages/Frv/snps_frv_apb_wrapper.sv for AMBA APB bus.

  $VC_STATIC_HOME/packages/Frv/snps_frv_ahb_wrapper.sv for AMBA AHB bus.

  $VC_STATIC_HOME/packages/Frv/snps_frv_axi_wrapper.sv for AMBA AXI bus.

  When the bus interface is not AMBA and bus interface is simple, you can use generic bus wrapper *$VC_STATIC_HOME/packages/Frv/snps_frv_gen_wrapper.sv*.

  When the bus interface is not AMBA and bus interface is complex, you can create own bus wrapper. The FRV application provides an example for bus wrapper *$VC_STATIC_HOME/packages/Frv/snps_frv_gen_wrapper.sv*. You can declare ports to match with design, and write glue logic to connect to the FRV module.

  You should replace snps_*_rd_wr.sv file with above or user created file in the compilation filelist.

- `[-include <Include type list>]`: Specify the list of additional types to be included for generation of register properties. Values: `unmapped, unmapped_reg, unmapped_field, reserved, reserved_reg, reserved_field`:
  - `unmapped`: Checks are enabled on all the unmapped registers and fields.
  - `unmapped_reg`: Checks are enabled on all the unmapped registers.
  - `unmapped_field`: Checks are enabled on all unmapped fields.

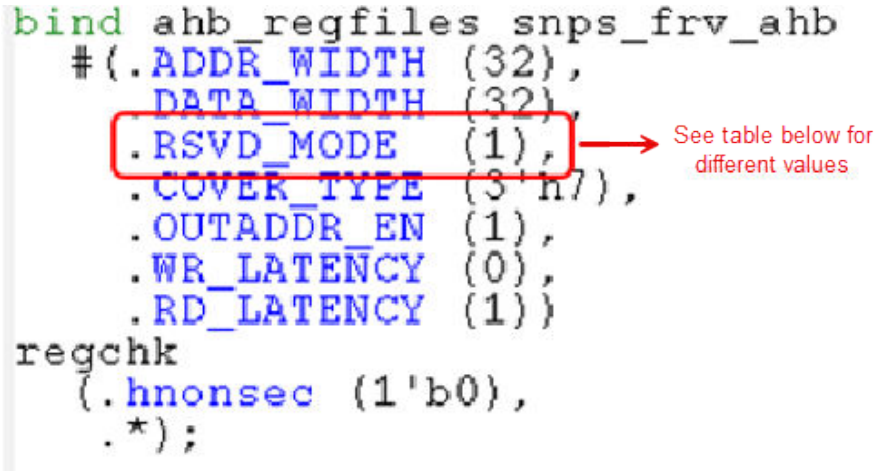
Feedback

- `reserved`: Checks are enabled on all reserved registers and fields.
- `reserved_reg`: Checks are enabled on all the reserved registers.
- `reserved_field`: Checks are enabled on all reserved fields.

Example

```
frv_load -ipxact …. -include unmapped
frv_load -ipxact …. -include {unmapped_reg reserved_field}
```

The generated checkers by `frv_load` command has parameter `RSVD_MODE` which specifies what value is expected as the read data. If the `-include`option is specified, the `RSVD_MODE` parameter is required to be set to enable unmapped or reserved checks.



RSVD_MODE	Behavior
0	No Check
1	Expect all 0 for read data
2	Expect all 1 for read data
3	Expect custom value for all read data

- If `RSVD_MODE` is 0, assertion is not generated.
- In case *all 0* is expected as read data for unmapped or reserved register/field, set `RSVD_MODE` with 1.

Feedback

    - In case *all 1* is expected as read data for unmapped or reserved register/field, set `RSVD_MODE` with 2.
    - In case custom value is expected as read data for unmapped or reserved register/field, set `RSVD_MODE` with 3. The custom value can be provided through vendorExtension *snps:outAddrValue*.

- `[-auto_load]`: Enables automatic loading of the generated checker file, bind file (if specified) and other required FRV checker library files.
- `[-parameters <List of parameter(s)>]`: Specifies the bind parameters to be passed to the checker file. The following are the parameters that can be used in the checker file. See section Checkers parameters for detailed description of these parameters.
    - ADDR_WIDTH
    - DATA_WIDTH
    - RSVD_MODE
    - COMPOSITE
    - BUS_LATENCY
    - WR_LATENCY
    - RD_LATENCY
    - COVER_TYPE
    - COVER_WRITE
    - COVER_READ
    - COVER_BYTE_UPDATE
    - OUTADDR_EN
    - LATENCY_MD
    - OUTABLK_MD
    - SNPS_AD_ABS
    - COVER_ALLOW_ERROR

  Examples

Feedback

Use the following commands to bind specific parameters to the checker files:

- To mark a field as volatile:

  ```
  frv_load -parameter {VOLATILE=1}
  ```

- To disable all write access-related cover properties:

  ```
  frv_load -parameter {COVER_WRITE=0}
  ```

- To enable all read access-related cover properties:

  ```
  frv_load -parameter {COVER_READ=1}
  ```

- To disable out-of-address cover properties:

  ```
  frv_load -parameter {OUTADDR_EN=0}
  ```

- To enable the read data check only when the reset value is specified:

  ```
  frv_load -parameter {RO_RD_CHK=0}
  ```

- To set the read data to 0 for out-of-address block range:

  ```
  frv_load -parameter {OUTABLK_MD=1}
  ```

- To enable cover regardless of response status:

  ```
  frv_load -parameter {COVER_ALLOW_ERROR=1}
  ```

- `[-verbose]`: When this option is specified, information messages for unsupported constructs are displayed, instead of silently ignoring them.

  The recommended usage of the `frv_load` is as follows:

  ```
  vcf> frv_load -ipxact <IP-XACT File> -auto_load
  ```

  or

  ```
  vcf> frv_load -ral <RALF File> -top <Component_or_mmap_or_block Name>
  -auto_load
  ```

  And specify `-include`, `-parameters` or `-verbose` if necessary.

16.4.2 The frv_config Command

The `frv_config` command is used to configure the various options of the FRV application. The `frv_config` command should be used before the `frv_load` command.

Syntax

Feedback

```
frv_config [-generate_backdoor_checks <value>] [-generate_covers <value>]
[-generate_out_of_block_range_checks [true|false]:<ExpectedValue>]
 [-ignore <Ignore type list>] [-disable_backdoor_checks <value>]
 [-generate_wo_read_check <value>] [-generate_wo_read_check_value
 <value>] [-generate_bdrsig_from_name] [-enable_fiv <value>]
 [-enable_volatile_check <value>]
```

Where,

- `[-generate_backdoor_checks <value>]`: Use this option to control the generation of checks for backdoor access: Values: true, false, 1, 0.
- `[-generate_covers <value>]`: Use this option to control the generation of all cover properties: Values: true, false, 1, 0
- `[-generate_out_of_block_range_checks <[true|false]:<ExpectedValue>>]`: Specify this option to control the generation of checks for 'out of AddressBlock range'.

  Can be specified in the form `<flag>:<expectedValue>`.

  Flag (can be true or false only) specifies whether to generate the checks or not. *Expected value* (can be 0 or 1 only, default is 0) specifies the expected value to be used in the check.
- `[-ignore <Ignore type list>]`: Specify the list of types that should be ignored by the frv_load command. Values: unknown_access
- `[-disable_backdoor_checks <value>]`:Used to control the generation of checks for backdoor access for specified type of construct: Values: reg_array, all)
- `[-generate_wo_read_check <value>]`: Used to control the generation of read data check for write-only register fields: Values: true, false, 1, 0. The default value is 0.
- `[-generate_wo_read_check_value <value>]`:Specifies the read value for write-only register fields to be used in the read data checks. It is mandatory to use `-generate_wo_read_check` option before the `-generate_wo_read_check_value`option.
- `[-generate_bdrsig_from_name]`: Specify this option to generate the backdoor signal path from the name attribute of *addressBlock*, *register* and *field*. This option can be used only when the name attribute represents actual RTL signal.
- `[-enable_fiv <value>]`: Specify this option to enable checks for formal interrupt verification. Values: `true`, `false`.

  For more details on interrupt verification, see section Formal Interrupt Verification.
- `[-enable_volatile_check <value>]`: Specify this option to generate read data checks for volatile fields. Values: `none, with_hw_update, all`. The default value is: `none`

- To enable generation of checks for volatile fields with hwUpdate, use the following command:

  ```
  frv_config -enable_volatile_check with_hw_update
  ```

- To enable generation of checks for all volatile fields, use the following command:

  ```
  frv_config -enable_volatile_check all
  ```

- To disable generation of checks for all volatile fields, use the following command:

  ```
  frv_config -enable_volatile_check none
  ```

Example

- When you do not specify any options, the command shows the default values.

```
vcf> frv_config
Backdoor Access Checks: TRUE
Cover Property Checks: TRUE
Out of AddressBlock Range Checks: FALSE
Ignore Unknown Access Checks: FALSE
Disable Backdoor Checks for: NONE
Read Data Checks for Write-Only Register Fields: FALSE
Value for Read Data Checks for Write-Only Register Fields: NONE
Generate backdoor signal from the name attribute: FALSE
Enable checks for formal interrupt verification: FALSE
Enable Volatile Checks: NONE
```

- `vcf> frv_config -generate_covers 0`
- `vcf> frv_load -ipxact …`

16.4.3 The frv_report Command

Use the `frv_report` command to get a report of the register data.

Syntax

```
vcf> frv_report [-hide_fields] [-no_summary] [-top
 [comp|mmap|block]:<Component_or_mmap_or_block Name>] [-vendor_extensions
 <[comp|mmap|block]:ComponentName.MemorymapName.BlockName.RegisterName.Fi
eldName>] [-filter <specify the filter option>]
```

Where,

- `hide_fields`: Specify this option to hide the individual field information in the generated report.
- `no_summary`: Specify this option to generate the report without the summary section.

Feedback

- `top [comp|mmap|block]:<Component_or_mmap_or_block Name>`: Use this option to get a report of the registers in a specific component, memory map or address block name in the input file. If the subtype `(`that is,`comp|mmap|block)` is not provided in the `-top` option, then the lookup order is:*comp, mmap, block*.
- `[-vendor_extensions <[comp|mmap|block]:ComponentName.MemorymapName.BlockName.RegisterName.FieldName>]`: Specify the name of a register or a field from the top to get the vendor extensions for the register or a field.
- `filter <specify the filter option>`:Specify the filter to sort the data according to register or field access type. The supporter parameter for filtering the data are regAccess or fieldAccess. The supported operation for filtering the data are == and !=.

The following is an example output of the `frv_report` command:

```
vcf> frv_report
Register Summary:
  Component       : regs
  MemoryMap/System: regs_map
  Address Block   : registers
  Base Address    : 0x0
  Register Count  : 3
  Assertions      : 48/48/0/0
  Covers          : 49/49/0/0
```

The following is an example output of `frv_report -filter { regAccess != rsvd }`:

```
vcf> frv_report -filter { regAccess != rsvd }
Register Summary:
  Component       : regs
  MemoryMap/System: regs_map
  Address Block   : registers
  Base Address    : 0x0
  Register Count  : 3
  Assertions      : 48/48/0/0
  Covers          : 49/49/0/0
```

The following is an example output of `frv_report -filter { regAccess == rw }`:

```
vcf> frv_report -filter { regAccess == rw }
Register Summary:
  Component       : regs
  MemoryMap/System: regs_map
  Address Block   : registers
  Base Address    : 0x0
  Register Count  : 3
  Assertions      : 48/48/0/0
  Covers          : 49/49/0/0
```

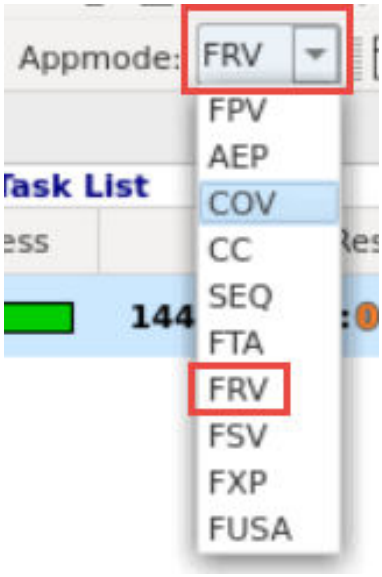
Feedback

16.5 VC Formal FRV GUI and Debug

VC Formal FRV has a unique GUI that helps in running register checks and debugging failures using Verdi. The FRV GUI will be invoked when,

- VC Formal GUI can be opened with `vcf -f … -fmode FRV -gui`
- Tcl file has `set_fml_appmode FRV`
- Change the appmode to FRV in the GUI as shown in Figure 122.

Figure 122: VC Formal FRV GUI Selection



16.5.1 Running Register Checks in the Formal GUI

When you run the `frv_load` command in the `vcf_shell`:

- The FRV task is added in the *Task List.*
- The properties created for the registers are listed in the *Verification Targets* tab.
- The registers are listed in the *Registers* tab.
- The constraints are listed in the *Constraints* tab.

Figure 123 shows the layout of FRV GUI.

Feedback

To perform the register checks in the FRV mode:

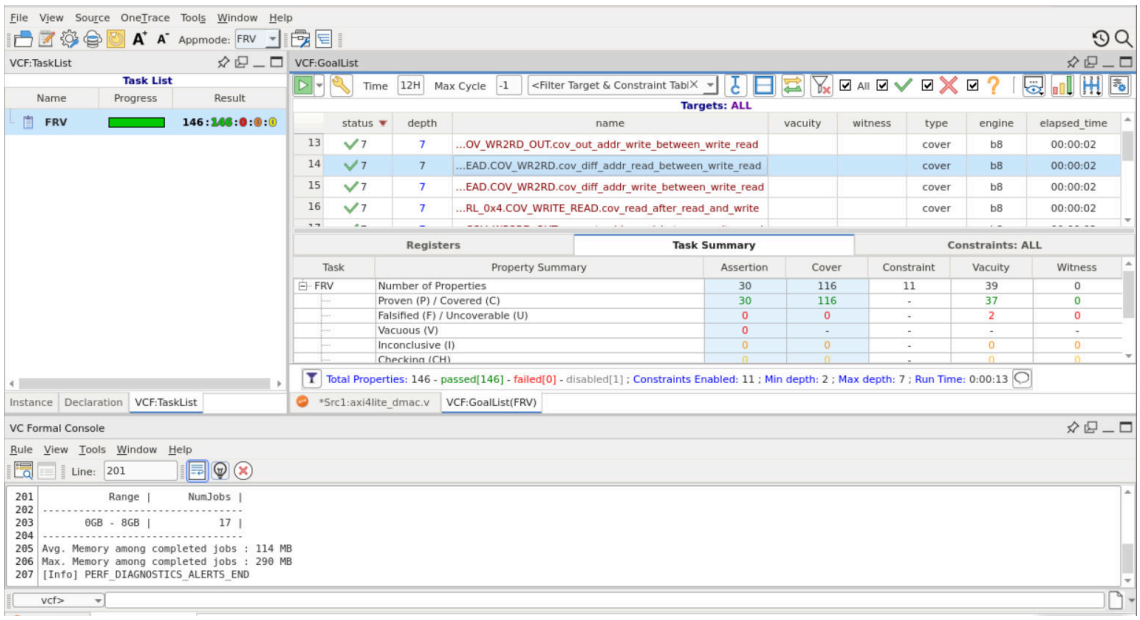
- Click in the GoalList toolbar.

  Th results of the properties and registers are updated in the *GoalList* table.

  Note:

  The results of the registers and properties are updated in the GUI as when the checks are performed.

Figure 123: FRV Mode in VC Formal GUI



16.5.2 Review Results and Report Generation

For FRV, the run results are the same as that of FPV. For more information on the run results, see section Analyzing Assertion Results.

- proven
- falsified
- checking
- inconclusive

Note:

You can also follow the tooltip of a falsified assert or an uncoverable cover for the next steps during debug.

Feedback

Falsified for Backdoor Series Assertions

When the backdoor signals has a typo, the backdoor assertion is falsified, and the following Lint message is reported after compiling the design:

Lint-[XMRE-L] Cross-module reference resolution error

If the backdoor assertion is falsified, once of possible reasons could be that there is typo in the backdoor signal. You can check if the above message is present in the log file, and fix the typo in the backdoor signals in RALF/IP-XACT file.

16.5.3 Configuring Registers Tab

The *Registers*tab lists the registers with the corresponding registers attributes provided in the IP-XACT file. By default, the following register attributes are displayed:

- Status
- Address Block
- Register Name
- Field Name
- Address
- Bit Range
- Type

The following optional columns are displayed only when they have content in the IP-XACT file provided:

- Reset
- Reset Mask
- Read Mask
- Write Mask
- Backdoor

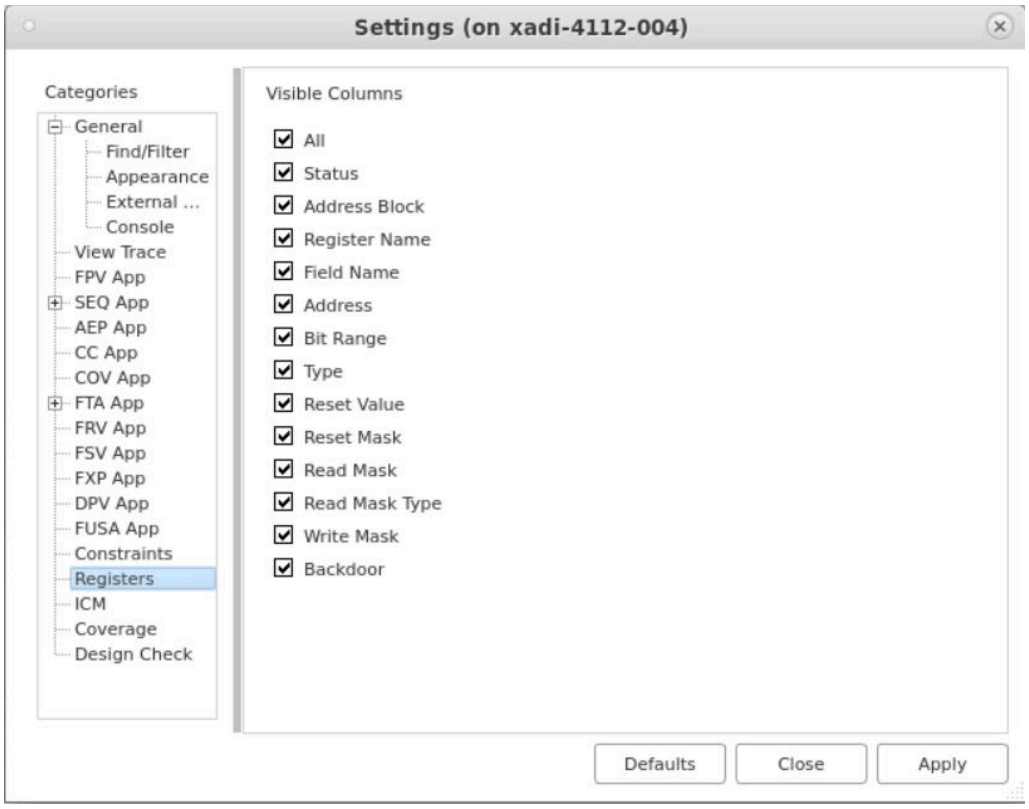
To configure the columns to be displayed in the *Registers* tab:

- Click in the GoalList toolbar.

  The *Customize View Settings* dialog box appears.

Feedback

Figure 124: Customize View Settings



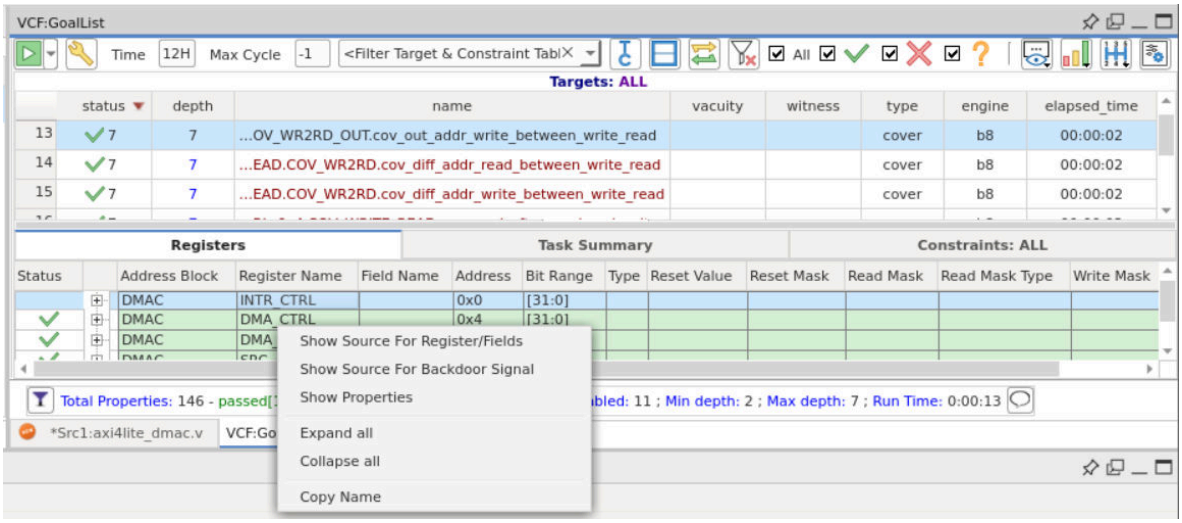
- Select the columns that must appear in the *Registers* tab and click *Apply*.

16.5.4 Using RMB Menu Options in the Registers Tab

You can perform the following RMB menu operations on the registers in the *Registers* tab as shown in Figure 125.

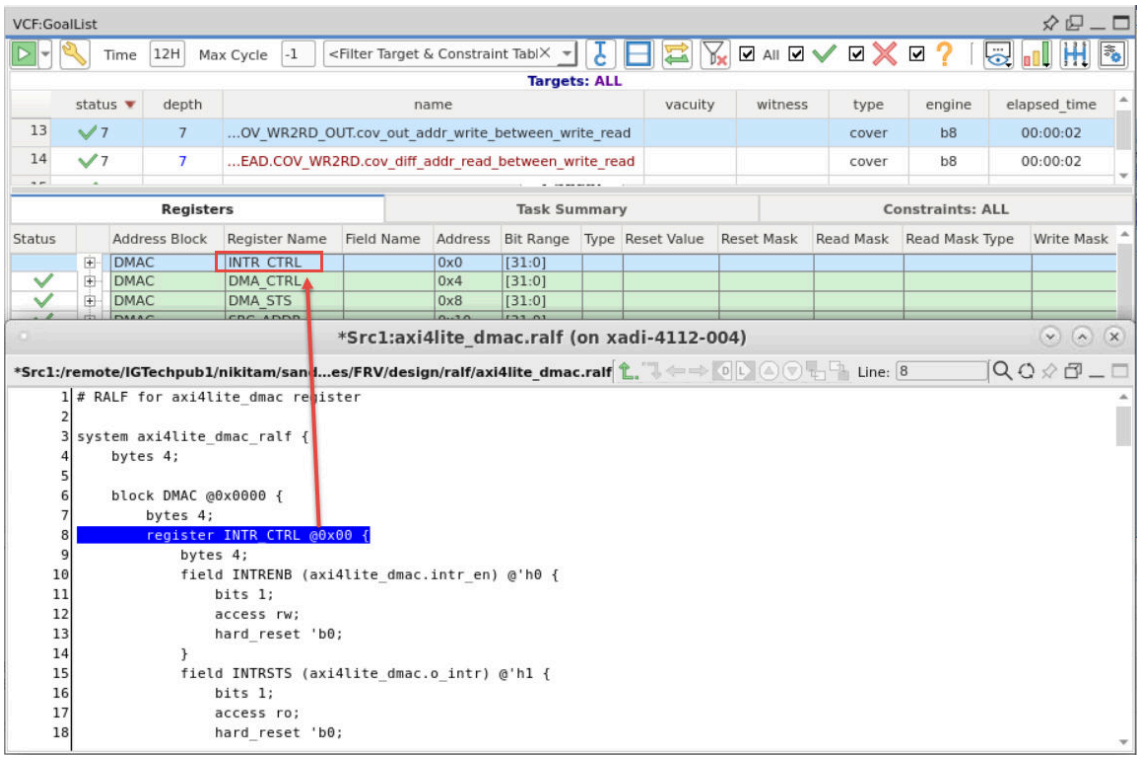
Feedback

Figure 125: RMB Menu Options in Registers Tab



- To view the source of the registers and fields, right-click the register/field and select Show Source For Register/Fields.

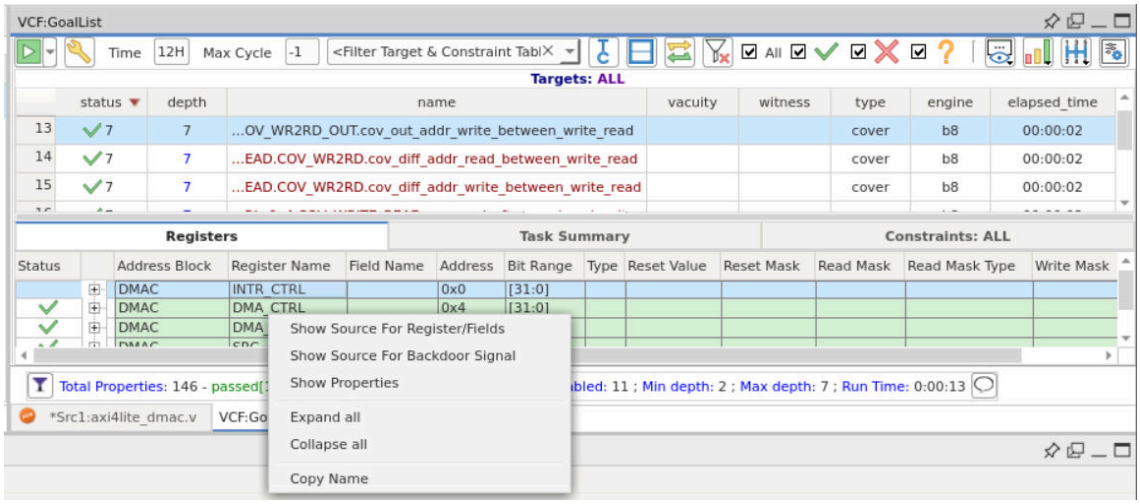
  Alternatively, you can double-click the register name in the *Register Name* column. The source pane appears with the register highlighted.



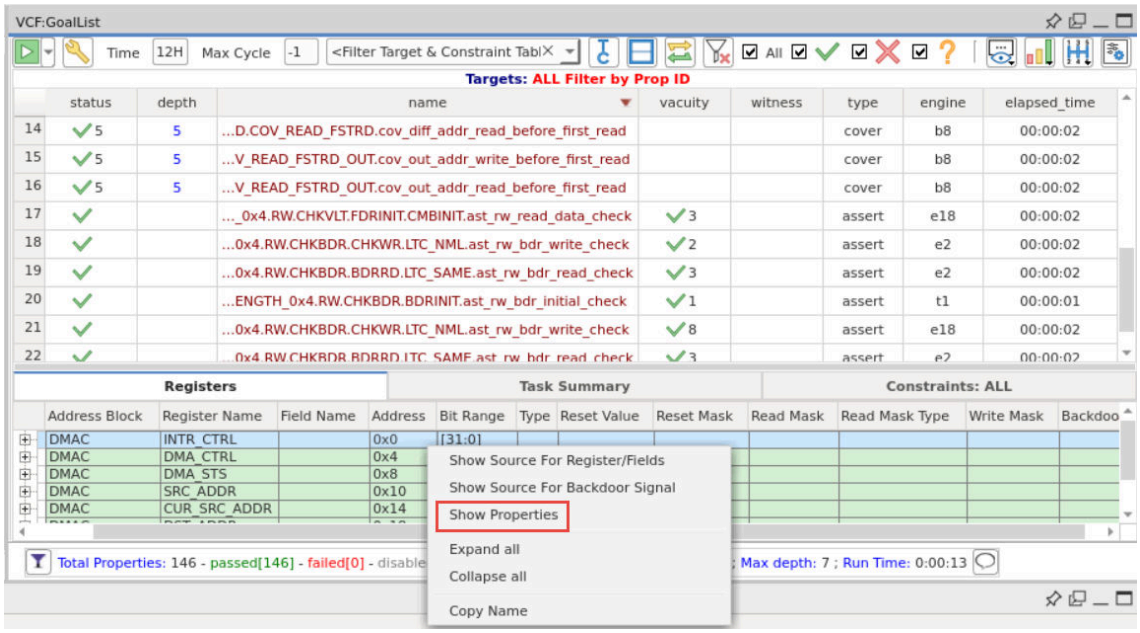
Feedback

- To view the RTL signal of the register, click **Show Source For Backdoor Signal.**

Figure 126: RMB Menu Options in Registers Tab



- To view the properties created for a selected register in the **Verification Targets** tab, right-click the corresponding register, and **Show Properties**.



- To view all the fields for all the registers, right-click any row, and select **Expand all**.
- To hide all the fields of the registers, right-click any row, and select **Collapse all**.
- To copy the name of the register, right-click a register and select **Copy Name**.

Feedback

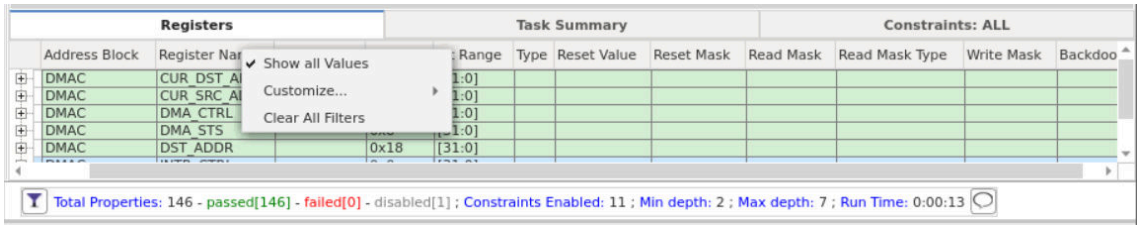
16.5.5 Filtering the Registers Tab Based on the Register Attributes

You can filter the register attributes using the column headers in the *Registers* tab.

To filter the *Registers* tab based on the register attributes:

- Right-click the header of the column you want to filter.

  The options to filter the register attributes appear.



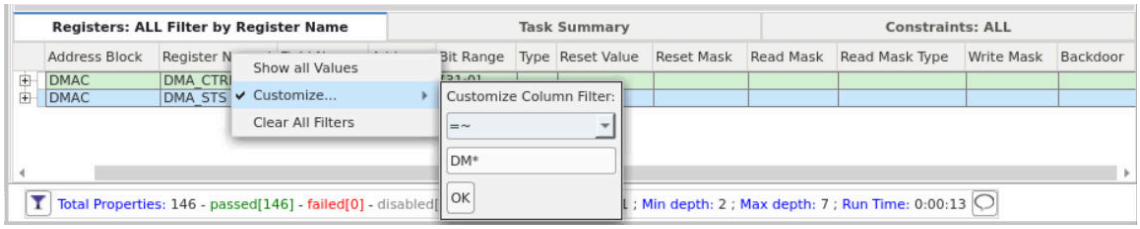
- To apply a filter, click **Customize**.

  The **Customize Column Filter** dialog box appears.

- Select the filter operator from the drop-down list, and type the desired value in the text box.

- Click **OK**.

  The column is filtered and the column header appears in blue.



- To clear all the filters, click **Clear All Filters**.

- To view all the values, click **Show all Values**.

  Note:

  When the table is filtered by *Address Block* or *Register Name*, the matched *Register* names along with the field names are displayed. This is not true for other columns.

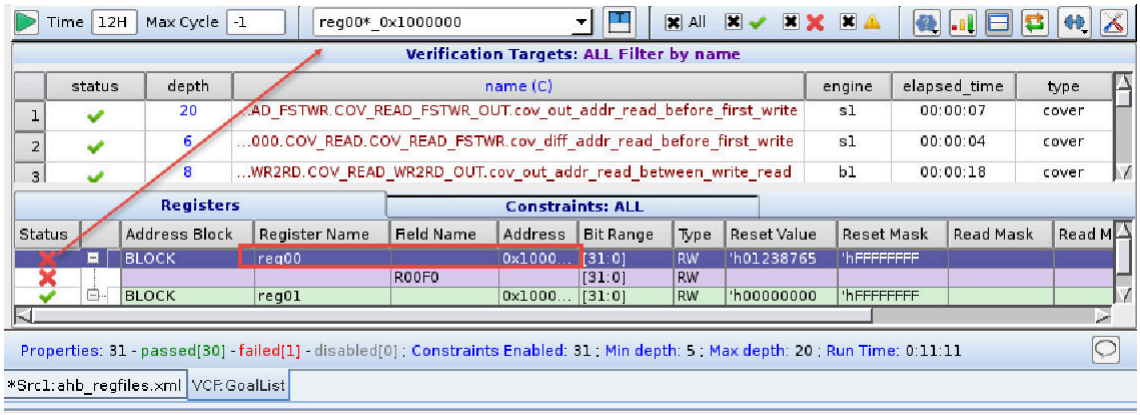
16.5.6 Viewing the Properties of Register

You can filter the properties in the *Verification Targets* tab for a selected register.

Feedback

- To view the properties of a particular register, double-click the **status** in the **Status** column for the corresponding register.

  The properties for the corresponding register is displayed. The register name along with its address is displayed in the search field.



16.5.7 Debug a Falsified Property of a Register Check

Opening the trace

You can view the trace for the falsified properties in nWave.

For more information on debugging falsified properties, see section Analyzing Assertion Results.

To view the proof trace for a falsified property:

- Double-click the status in the **status** column for the corresponding falsified property in the **Verification Targets** tab. The nWave window appears with the waveform.

FRV Violation Trace Debug

Most register blocks have a standard protocol interface like APB, AHB or AXI4-Lite or a simple proprietary handshake interface. To make the FRV debug more productive, perform the following:

1. Ensure you have an AIP or a set of user-defined assumes or constraints at the interface constraining the environment correctly.
2. Check the values of the parameters in bind statements.
3. Check clock and reset declarations and any potential violations using the `check_fv_setup` and `report_fv_setup` commands.
4. Add the protocol signals on the interface of the block to every CEX you debug.

Feedback

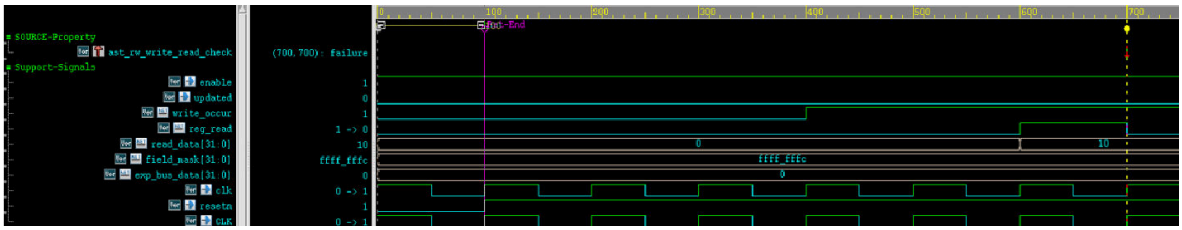
Hints for Debugging

In nWave window, the trace for failed assertion is shown with some useful signals for debugging. The cycle in which *reg_read* is High indicates register read completes.

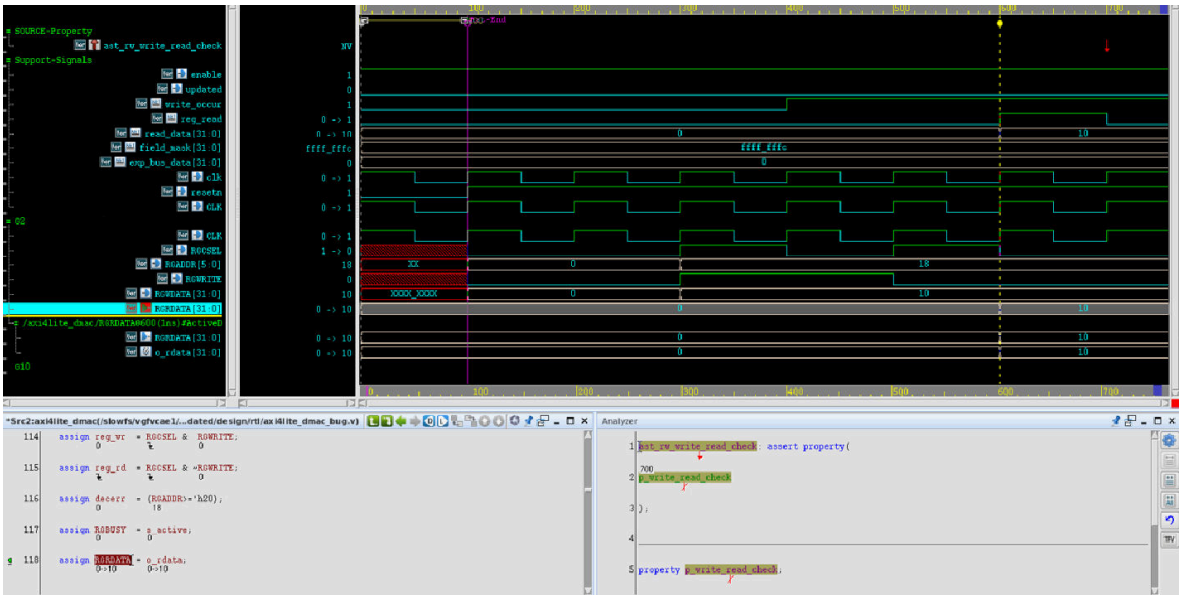
The *read_data* indicates read data driven by the design.

The *field_mask* indicates which bits within data correspond to target field. If *field_mask* is 32'hffff_fffc as shown below, bit slice [31:2] within bus data is this field, and assertion checks these bits.

The *exp_bus_data* indicates expected data. The assertion fails when *reg_read* and *exp_bus_data* are different.



To debug efficiently, adding bus interface signals in the trace is recommended. Next step is to back trace read data driven by design, 'REGRDATA' as shown in the following figure.



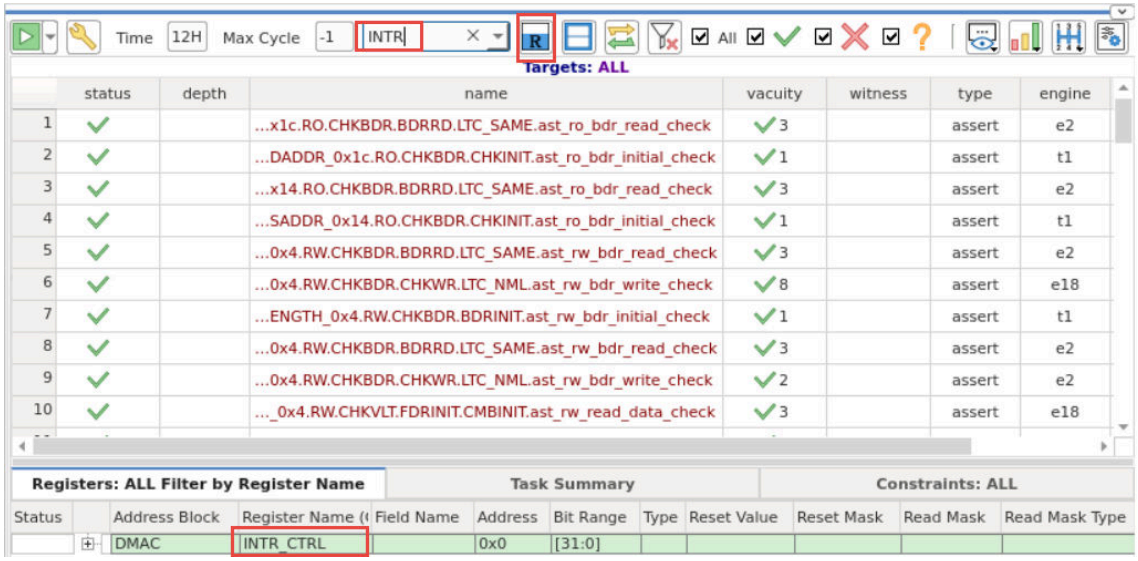
Feedback

16.5.8 Searching for Registers, Assertions, Constraints in the GoalList Table

You can search for properties, registers, and constraints using the search button in the **GoalList** toolbar. You can also sort the columns using the column headers in the respective tabs.

- To toggle between the tables to be searched, use the button. By default, the icon appears and you can search for the properties in the both the **Targets** and **Registers** table.
- To search for a register in the *Registers* tab, click .

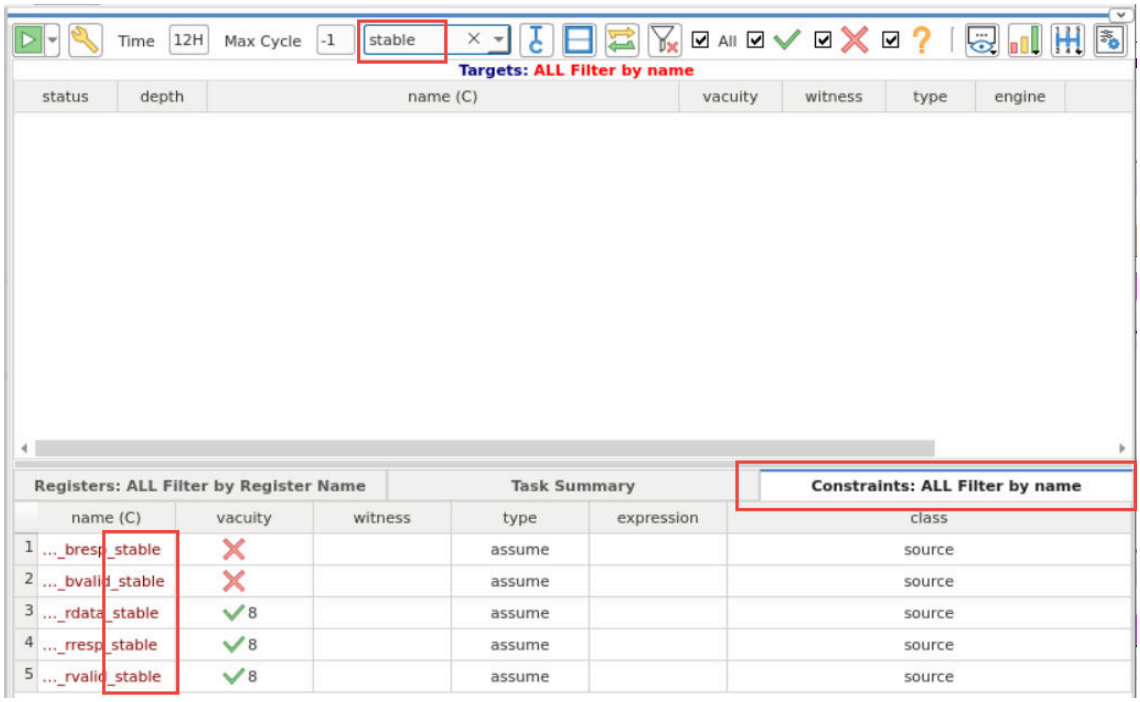
  The icon changes to . You can search for the registers in the *Registers* tab.



- To search for constraints, open the **Constraints** tab.

  The button appears. You can search for the constraints in the *Constraints* tab.

Feedback



Note:

Each of the tabs in the GoalList tab (property, register, or constraint) keeps its own filtering status.

16.6 CSV Format for Register Specification

A CSV file is a delimited text file that uses a comma to separate values. Each line of the file is a data record and each record consists of one or more fields separated by commas.

Each data record in the CSV file supports following fields:

Row identifier, Name/Keyword,baseAddress/addressOffset/bitOffset/write_mask/id/priority/ wrLatency, width/size/bitWidth/read_mask/trigger/rdLatency, access/read_mask_type/ value, accessHandle/mask, range/reset-value/latency, reset-mask/func, volatile, readAction, modifiedWriteValue

The first row in the CSV file denotes the columns, that is, what data is provided in each row and to which category it would belong.

Feedback

In the following example, the *REGISTER* is a row identifier:

REGISTER,REG0,0x000,32,,,,,,

The data for the register is:

```
Name : REG0
addressOffset : 0x00
size : 32
```

Example

The row should be in a sequence in which data needs to be interpreted.

```
REGISTER, REG0,,,,,,
FIELD, REG0_F0,,,,,,,
FIELD, REG0_F1,,,,,
REGISTER, REG1,,,,,
FIELD, REG1_F0,,,,,,
FIELD, REG1_F1,,,,,,
VENDOREXT VE_REG1_F1,,,,,
```

The equivalent IPXACT for above CSV would be:

```
<ipxact:register>
    <ipxact:name>REG0</ipxact:name>
    <ipxact:field>
         <ipxact:name>REG0_F0</ipxact:name>
     </ipxact:field>
     <ipxact:field>
           <ipxact:name>REG0_F1</ipxact:name>
      </ipxact:field>
</ipxact:register>
<ipxact:register>
    <ipxact:name>REG1</ipxact:name>
    <ipxact:field>
         <ipxact:name>REG1_F0</ipxact:name>
     </ipxact:field>
     <ipxact:field>
           <ipxact:name>REG0_F1</ipxact:name>
      </ipxact:field>
      <vendorExtension>VE</vendorExtension>
</ipxact:register
```

Row Identifiers

You can specify the following row identifiers:

- *COMPONENT*: represents component
- *MMAP*: represents memoryMap
- *BLOCK*: represents addressBlock

Feedback

- *REGISTER*: represents register
- *FIELD*: represents vendorExtension
- *VENDOREXT*: represents vendorExtension

Example

```
REGISTER,REG0,0x000,32,,,,,,,
```

Here, Register is the row identifier and further data identifies that register, that is, the name, addressOffset, and size.

The following can be the columns specifications:

- **Field Related Columns**
  - Name/keyword
  - bitOffset
  - bitwidth
  - access
  - accessHandle
  - reset
  - resets
  - volatile
  - readAction
  - modifiedWriteValue
  - testable
  - reserved
  - writeValConstraints
  - VendorExtensions
- **Register Related Columns**
  - Name
  - addressoffset
  - Size
  - access

- accesSHandles
- Volatile
- reset
- resets
- vendorextensions
- dim
- Alternate registers

- **AddressBlock related columns**
  - Name
  - baseaddress
  - range
  - width
  - access
  - accesshandles
  - register
  - Registerfile
- **VendorExtensions related columns**
  - read_mask
  - write_mask
  - read_mask_type
  - read_mask_rdvalue
  - fieldHWupdates
    - fieldHWupdate
    - id
    - trigger
    - value
    - latency

Feedback

        - function
        - priority
    - fieldLatencies
        - rd_latency
        - wr_latency
    - disableChecks
        - inhibitVal
        - allowVal

The following figure illustrates an example of the CSV file created for a register specification:

Figure 127: CSV file for Register Specification

Row Identifier	Name/Keyword												
COMPONENT	name												
BLOCK	name	baseAddress	width	access	accessHandle	range							
REGISTER	name	addressOffset	size	access	accessHandle	reset-value	reset-mask						
FIELD	name	bitOffset	bitWidth	access	accessHandle	reset-value	reset-mask	volatile	readAction	modifiedWriteValue	testable	writeValueConstraint minimum	writeValueConstraint maximum
VENDOREXT	Mask	write_mask	read_mask	read_mask_type									
VENDOREXT	csrSetting	type (*1)											
VENDOREXT	Alias	field-path											
VENDOREXT	HWupdate	id	trigger	value	mask	latency	func						
VENDOREXT	HWupdates	priority											
VENDOREXT	Latency	wrLatency	rdLatency										
VENDOREXT	DisableChecks	allowValue	inhibitValue										
*1) either one of NO_CSR_TEST, NO_CSR_W_TEST or NO_CSR_R_TEST													

16.7 Extensions Supported in the IP-XACT File

The VC Formal FRV vendor extension format supported was different from standard IP-XACT convention. Starting with this U-2023.03-SP2-6 and V-2023.12-SP2, release, VC Formal FRV supports the standard vendor extension format.

The original (previous) format is still supported for backward compatibility, however using standard format is strongly recommended. This section provides the details of the standard vendor extension format.

16.7.1 Write and Read Mask Capability

The write and read mask capability that is not directly supported by IP-XACT, is supported through vendor extension in VC Formal. Some registers may be restricted to write or read an entire word. The value or signal for write/read mask can be specified and the value to be read for the masked bit can be specified (either 0, 1 or don’t care).

Feedback

XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
  <snps:writeMask>
    <mask>(writeMask)</mask>
  </snps:writeMask>
  <snps:readMask>
    <mask>(readMask)</mask>
    <maskType>(readMaskType)</maskType>
    <maskRdValue>(readMaskRdValue)</maskRdValue>
  </snps:readMask>
</ipxact:vendorExtensions>
```

- `writeMask`: bitwise mask for write access, 1 implies enable write, 0 implies mask write. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field. To specify this, you should use a prefix `REGFIELD::,` for example, to use value of field 'F1' of register 'R1' as the `writeMask`, specify as follows:

    ```
    <ipxact:vendorExtensions>
      <snps:writeMask>
        <mask>REGFIELD::R1.F1</mask>
      </snps:writeMask>
    </ipxact:vendorExtensions>
    ```

- `readMask`: bitwise mask for read access, 1 implies enable read, 0 implies mask read. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field. To specify this, you should use a prefix `REGFIELD::`, for example, to use value of field ‘F1’ of register ‘R1’ as the `readMask`, specify it as follows:

    ```
    <ipxact:vendorExtensions>
      <snps:readMask>
        <mask>REGFIELD::R1.F1</mask>
        <maskType>0</maskType>
      </snps:readMask>
    </ipxact:vendorExtensions>
    ```

- `readMaskType`: indicate what data must be read when it is masked
  - 0: 0 should be returned as masked data
  - 1: 1 should be returned as masked data

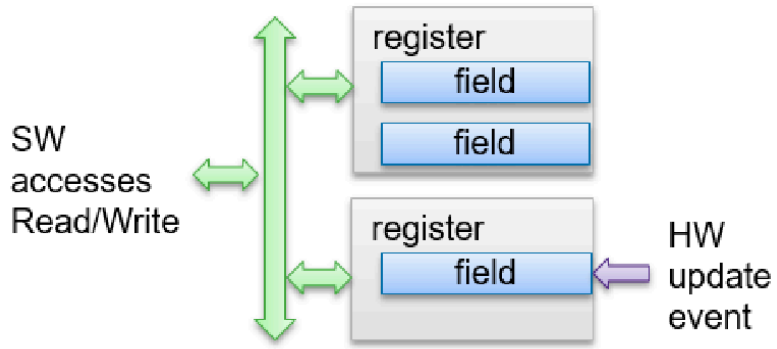
- 2: don’t care and no check (default)
- 3: when value other than 0 or 1 is to be returned as masked data, `readMaskRdValue` should be specify the masked data.

- `readMaskRdValue`: indicates what data should be read when it is masked. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field, using a prefix `REGFIELD::` (as mentioned above)

16.7.2 HW Update Capability

The HW update capability which is not directly supported by IP-XACT, is supported through vendor extension. Usually register/field values are updated by a software write and by design internal logic. The latest written data by software may not be read due to a hardware update. You must ensure that the value for these types of register/field is updated with the expected value when a hardware update event occurs.

Figure 128: Example for HW Update



XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
  <snps:hwUpdates>
    <snps:hwUpdate> <!-- Specify 1 HW update functionality--->
      <id>….</id>
      <trigger> …. </trigger>
      <value> …. </value>
      <mask> …. </mask>
      <latency> …. </latency>
      <function> …. </function>
    </snps:hwUpdate>
    <snps:hwUpdate> <!-- Specify 1 HW update functionality--->
```

Feedback

```
      <id> …. </id>
      <trigger> …. </trigger>
     <value> …. </value>
      <mask> …. </mask>
      <latency> …. </latency>
      <function> …. </function>
    </snps:hwUpdate>
<!-- Repeat until all HW update functionalities are specified--->
…
    <priority> …. </priority>
  </snps:hwUpdates>
</ipxact:vendorExtensions>
```

Where,

- `id`: ID (need to be unique within the same field) (string). If the ID is not specified, then unique IDs are automatically assigned. For example, hw0, hw1, and so on.
- `trigger`: Trigger signal for HW update (string). It can also be a value of any other field. To specify this, you should use a prefix `REGFIELD::`, for example, to use value of field 'F1' of register 'R1' as the `trigger`, specify as follows:

  ```
  <trigger>REGFIELD::R1.F1</trigger>
  ```

- `value`: Value or signal to be updated (string)
- `mask`: Bit of 1 will be updated (string)
- `latency`: Specify the number of cycles required to be updated (integer > 0)
- `function`: Part of module name to represent functionality (string)

Function	Library module name
load	snps_regchk_load
pulse	snps_regchk_pulse
inc	snps_regchk_inc
dec	snps_regchk_dec

- `priority`: Specify priorities between SW-Write, SW-Read, and HW triggers. The predefined ID for SW-Write is “sww”. The predefined ID for SW-Read is “swr”. For example, “sww, hw0, swr, hw2, hw1” (string).

Feedback

If sww and/or swr is not specified explicitly in the `priority`, then the highest priority is implicity considered as sww, swr, ..., ..., and so on.

For example, if the priority is specified as hw0, hw2, hw1, then the priority interpreted is sww, swr, hw0, hw2, hw1.

Example

In case FLD_R1_0 field is updated by gpio_wren with value gpio_data.

```
<ipxact:field>
  <ipxact:name>FLD_R1_0</ipxact:name>
  <ipxact:bitOffset>0</ipxact:bitOffset>
  <ipxact:bitWidth>16</ipxact:bitWidth>
  <ipxact:access>read-write</ipxact:access>
  <ipxact:resets>
    <ipxact:reset>
      <ipxact:value>0x9876</ipxact:value>
      <ipxact:mask>0xFFFF</ipxact:mask>
    </ipxact:reset>
  </ipxact:resets>
  <ipxact:vendorExtensions>
    <snps:hwUpdates>
      <snps:hwUpdate>
        <id>hw_r1_0</id>
        <trigger>hwupdate_aliased.gpio_wren</trigger>
          <!-- This field is updated when gpio_wren==1 -->
        <value>hwupdate_aliased.gpio_data</value>
          <!-- This field value will be updated to gpio_data -->
        <latency>1</latency>
          <!-- latency=1 : update occurs at the next cycle of trigger -->
        <function>load</function>
          <!-- Use snps_regchk_load module -->
      </snps:hwUpdate>
      <priority>hw_r1_0, sww, swr </priority>
        <!-- HW update has the highest priority than software accesses
 -->
    </snps:hwUpdates>
  </ipxact:vendorExtensions>
</ipxact:field>
```

16.7.3 Aliased Registers Capability

Alias registers update capability which is not directly supported by IP-XACT, is supported through vendor extension. The same register field may be accessed using multiple addresses. One register field can be shared by multiple register fields, and access type may be different in each register field.

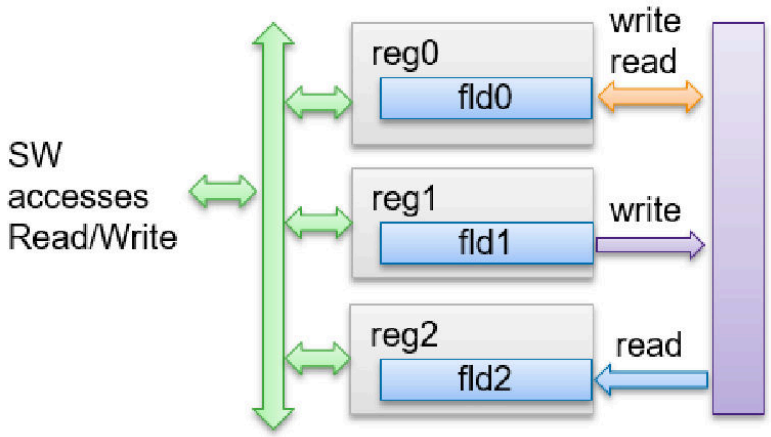
Example

Feedback

reg0.fld0 can be write-read, reg1.fld1 can be write-only, reg2.fld2 can be read-only

- If write reg1.fld1, written value can be read using reg0.fld0 and reg2.fld2
- If write reg0.fld1, written value can be read using reg0.fld0 and reg2.fld2

Figure 129: Example of Alias Register



XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
<snps:alias>
   <field> hierarchicalPath.fieldNameRef </field>
</snps:alias>
</ipxact:vendorExtensions>
```

If the referred field is	The hierarchical path
Within the same register	is not required
Within the same addressBlock but different register	should be <register name>

Example 1

Consider the following scenario:

reg0.fld0 is aliased by reg1.fld1 and reg2.fld2, reg0.fld0 : write-read, reg1.fld1 : write-only, reg2.fld2 : read-only

```
<ipxact:register>
   <ipxact:name>reg0</ipxact:name>
   <ipxact:addressOffset>0x00</ipxact:addressOffset>
   <ipxact:field>
      <ipxact:name>fld0</ipxact:name>
```

Feedback

```
      <ipxact:bitOffset>0</ipxact:bitOffset>
      <ipxact:bitWidth>4</ipxact:bitWidth>
      <ipxact:access>read-write</ipxact:access>
   </ipxact:field>
</ipxact:register>
<ipxact:register>
   <ipxact:name>reg1</ipxact:name>
   <ipxact:addressOffset>0x08</ipxact:addressOffset>
   <ipxact:field>
      <ipxact:name>fld1</ipxact:name>
      <ipxact:bitOffset>10</ipxact:bitOffset>
      <ipxact:bitWidth>4</ipxact:bitWidth>
      <ipxact:access>write-only</ipxact:access>
      <ipxact:vendorExtensions>
              <snps:alias>
                 <field>reg0.fld0</field>
              </snps:alias>
      </ipxact:vendorExtensions>
   </ipxact:field>
</ipxact:register>
<ipxact:register>
   <ipxact:name>reg2</ipxact:name>
   <ipxact:addressOffset>0x14</ipxact:addressOffset>
   <ipxact:field>
      <ipxact:name>fld2</ipxact:name>
      <ipxact:bitOffset>16</ipxact:bitOffset>
      <ipxact:bitWidth>4</ipxact:bitWidth>
      <ipxact:access>read-only</ipxact:access>
      <ipxact:vendorExtensions>
              <snps:alias>
                 <field>reg0.fld0</field>
              </snps:alias>
      </ipxact:vendorExtensions>
   </ipxact:field>
</ipxact:register>
```

Example 2

Consider the following scenario:

REGA is aliased by REGB

```
<ipxact:register>
<ipxact:name>REGA</ipxact:name>
<ipxact:addressOffset>0x00</ipxact:addressOffset>
<ipxact:access>read-write</ipxact:access>
<ipxact:field>
       …
</ipxact:field>
</ipxact:register>
<ipxact:register>
<ipxact:name>REGB</ipxact:name>
<ipxact:addressOffset>0x08</ipxact:addressOffset>
```

Feedback

```
<ipxact:access>read-only</ipxact:access>
<ipxact:vendorExtensions>
        <snps:alias>
            <register> REGA </register>
        </snps:alias>
</ipxact:vendorExtensions>
<ipxact:field>
        …
</ipxact:field>
</ipxact:register>
```

Note:

In case, the field that you want to specify as the aliased field of some other field, belongs to a register having 'dim' index as N, specify the field alias as follows:

```
<snps:alias>
    <field><regName>_<dim(N-1)>.<fieldName> </field>
</snps:alias>
```

Example

To specify field named ‘FLD1’ as the aliased field of some another field, which belongs to a register named ‘REG_RW1’, having dim (array) index as 5, use the following:

```
<snps:alias>
    <field>REG_RW1_dim4.FLD1<fieldName> </field>
</snps:alias>
```

Examples

16.7.4 Latency Update Capability

The latency update capability which is not directly supported by IP-XACT, is supported through vendor extension. You can specify latency parameters using the bind statement. However, in this method, one value is used for all fields. In actual design, latencies can be different in each field. This enhancement enables you to specify latency cycles per field, and also specify a range for some of the fields that might not have fixed cycle latency.

For example, if you specify write latency with 3:5, then it is okay if expected value is seen at least once within 3 to 5 cycles.

XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
  <snps:latencies>
    <wrLatency>2</wrLatency>
    <rdLatency>1:5</rdLatency>
  </snps:latencies>
</ipxact:vendorExtensions>
```

Feedback

Note:

- Latency is supported at both field and register level.
- Field-level latency takes priority over register-level latency when both are specified.
- You can either specify only *wrLatency*, *rdLatency* or *rdActLatency* or *all*.

Where,

- *wrLatency*: This indicates how many cycles to take from bus write to actual field value update. Specify an integer. If you want to specify a fixed latency specify a single value. In case there is range in latency, specify a range of values using ":". For example, 2:5
- *rdLatency*: This indicates how many cycles to take from actual field value to bus read. Specify an integer. If you want to specify a fixed latency specify a single value. In case there is range in latency, specify a range of values using ":". For example, 2:5
- *rdActLatency*: This is applicable only for field with readAction. If readAction is specified for field, bus read changes field value. This indicates how many cycles to take from bus read to actual field value update. Specify an integer. Only a single value is supported.

16.7.5 Disable Checks Capability

Disable checks capability which is not directly supported by IP-XACT, is supported through this vendor extension. The disableChecks vendor extension can be one of two types, `allowValue` or `inhibitValue`. Only one of these can be specified at any point of time.

- `inhibitValue`: Disable all assertions when a specific value is written to a specific register field
- `allowValue`: Disable all assertions when a value other than the specified value is written to a specific register field

XML Format to be specified in the IPXACT file.

- `disableChecks: inhibitValue`

```
<ipxact:vendorExtensions>
  <snps:disableChecks>
    <inhibitValue>(specify inhibit value)</inhibitValue>
  </snps:disableChecks>
</ipxact:vendorExtensions>
```

- `disableChecks: allowValue`

```
<ipxact:vendorExtensions>
  <snps:disableChecks>
    <allowValue>(specify allow value)</allowValue>
```

Feedback

```
  </snps:disableChecks>
</ipxact:vendorExtensions>
```

Examples

- disableChecks: inhibitValue

  Consider the case of register FLD_21_WO, where all the assertions need to be disabled, if 1'b1 is written to it.

```
<ipxact:field>
  <ipxact:name>FLD_21_WO</ipxact:name>
  <ipxact:bitOffset>19</ipxact:bitOffset>
  <ipxact:bitWidth>1</ipxact:bitWidth>
  <ipxact:access>write-only</ipxact:access>
  <ipxact:vendorExtensions>
    <snps:disableChecks>
      <inhibitValue>1'b1</inhibitValue>
    </snps:disableChecks>
</ipxact:vendorExtensions>
</ipxact:field>
```

- disableChecks: allowValue

  Consider the case of register FLD_32_WO, where all the assertions need to be disabled for any value other than 2'b0.

```
<ipxact:field>
```

```
<ipxact:field>
  <ipxact:name>FLD_32_WO</ipxact:name>
  <ipxact:bitOffset>14</ipxact:bitOffset>
  <ipxact:bitWidth>2</ipxact:bitWidth>
  <ipxact:access>write-only</ipxact:access>
  <ipxact:vendorExtensions>
    <snps:disableChecks>
      <allowValue>2'b0</allowValue>
    </snps:disableChecks>
  </ipxact:vendorExtensions>
</ipxact:field>
```

16.7.6 alternateRegisters Capability

The FRV application supports the alternateRegisters capability. An alternateRegister element contains an alternate definition for the containing register. Each alternateRegister element contained within the same alternateRegisters element provides an alternate description for the containing register element.

Example

```
<register>
<alternateRegisters>
```

Feedback

```
    <alternateRegister>
     ....
     ...
    </alternateRegister>
</alternateRegisters>
</register>
```

FRV enables you to specify the condition to enable alternateRegisters through vendor extension. To enable an alternate register, specify the condition with *signal* and *enVal* attributes. You can specify a signal as either a physical signal name, or a register field name by adding *REGFIELD::* keyword.

When the *signal* matches with *enVal*, the alternate register is enabled. If the `vendorExtension` is missing, the default register is enabled by default.

Let us assume alternateRegisters *REG_RW1_ALT* is enabled, when field *FLD_RW_01* in register *REG_RW0* is *1'b1*.

The example vendor extension in this case is as follows.

```
<ipxact:register>
  <ipxact:alternateRegisters>
    <ipxact:alternateRegister>
      <ipxact:name>REG_RW1_ALT</ipxact:name>
      <ipxact:alternateRegisterDefinitionGroup>
        …
      </ipxact:alternateRegisterDefinitionGroup>
      <ipxact:vendorExtensions>
        <snps:enableAlternateRegisters>
          <signal>REGFIELD::REG_RW0.FLD_RW_01</signal>
          <enVal>1'b1</enVal>
        </snps:enableAlternateRegisters>
      </ipxact:vendorExtensions>
    </ipxact:alternateRegister>
  </ipxact:alternateRegisters>
</ipxact:register>
```

Consider alternateRegisters REG_RW2_ALT is enabled, when signal dut.ctrl_reg is 2'b01.

```
<ipxact:alternateRegisters>
    <ipxact:alternateRegister>
      <ipxact:name>REG_RW2_ALT</ipxact:name>
      <ipxact:alternateRegisterDefinitionGroup>
      …
      </ipxact:alternateRegisterDefinitionGroup>
      <ipxact:vendorExtensions>
        <snps:enableAlternateRegisters>
          <signal>dut.ctrl_reg</signal>
          <enVal>2'b10</enVal>
        </snps:enableAlternateRegisters>
      </ipxact:vendorExtensions>
```

Feedback

```
    </ipxact:alternateRegister>
  </ipxact:alternateRegisters>
```

16.7.7 Variable Reset Capability

Variable reset capability which is not directly supported by IP-XACT, is supported through this vendor extension. With this vendor extension, you can set a reset value for a register, whose reset value changes depending on the input value.

Example

Consider the scenario, where *simple_test.sel* can set the reset value of the field.

- If *simple_test_sel == 1*, reset value for the field is 32'hdeadbeaf
- Else, the reset value is 32'h0

  The example vendor extension in this case is as follows:

```
<ipxact:vendorExtensions>
  <snps:variableReset>
    <signal>simple_test.sel</signal>
    <mask>001</mask>
    <value>1:32'hdeadbeaf;32'h0</value>
    <mode>1</mode>
  </snps:variableReset>
</ipxact:vendorExtensions>
```

Where,

- `signal`: Specify the select signal which updates reset value.
- `value`: Specify the reset value.
- `mask`: Specify the mask. Default is all 1's for the field, if it is not specified.
- `mode`: This indicates when expected reset value is determined. Values: 0 or 1.
  - *0*: Directly refer signal specified by `signal` (default).
  - *1*: Use signal value specified by `signal` during reset.

Notes

- If both standard reset and `variableReset` are specified, then the standard reset is overridden, and an informational message is reported.
- It is also mandatory to mention both `signal` and `value`, if only one of them is mentioned, an error message is reported.
- The `mode` and `mask` fields are optional.

Feedback

16.7.8 Array Support Capability

FRV supports unrolling of an array using array vendorExtension. It supports both 2D as well as 3D arrays.

XML format to be specified in the IP-XACT file for 2D array:

2D array: `reg [0:31] fld_rw1 [0:1] [0:3];`

```
<ipxact:vendorExtensions>
  <snps:array>
    <xFrom>0</xFrom>
    <xTo>3</xTo>
    <yFrom>0</yFrom>
    <yTo>1</yTo>
    <offsetCalc>'h50+'b100*x+'b400*y</offsetCalc>
    <hdlPattern>(ABC_%x_%y,x,y)</hdlPattern>
  </snps:array>
</ipxact:vendorExtensions>
```

XML format to be specified in the IP-XACT file for 3D array:

3D array: `reg [0:31] fld_rw0 [0:1][0:1][0:3];`

```
<ipxact:vendorExtensions>
  <snps:array>
    <xFrom>0</xFrom>
    <xTo>3</xTo>
    <yFrom>0</yFrom>
    <yTo>1</yTo>
    <zFrom>0</zFrom>
    <zTo>1</zTo>
    <offsetCalc>'h10+'b100*x+'b400*y+'b800*z</offsetCalc>
    <hdlPattern>(ABC_%x_%y_%z,x,y,z)</hdlPattern>
  </snps:array>
</ipxact:vendorExtensions>
```

16.7.9 Individual Reset for Field Capability

By default, FRV expects all the register fields to reset with the bus reset. However, some fields may reset with a different reset source depending on the design. To support such designs, FRV supports individual reset capability for register fields through vendor extension in IP-XACT. Through this vendor extension, you can specify individual reset signal for each register field.

By default, bus reset is treated as the primary reset. If the reset source vendor extension is specified for a field, the specified reset source is used to reset the field.

Feedback

XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
  <snps:resetnSource>
    <signal>hierarchicalPath.resetSignal</signal>
  </snps:resetnSource>
</ipxact:vendorExtensions>
```

Note:

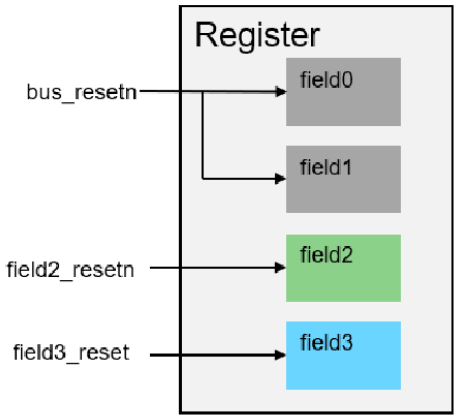
Specify the `resetSignal` along with the path. If the `resetSignal` is active high reset, add ~/! along with the path.

Example

Consider the case as shown in the Figure 130, where

- *field0* and *field1* get initialized with bus reset
- *field2* gets initialized with own reset source *field2_resetn* (Active Low)
- *field3* gets initialized with own reset source *field3_reset* (Active High)

Figure 130: Example of Individual reset for field



The `resetnSource` vendor extension can be specified as follows.

- For *field2*:

```
<ipxact:vendorExtensions>
  <snps:resetnSource>
    <signal>top.rstblk.field2_resetn</signal>
  </snps:resetnSource>
</ipxact:vendorExtensions>
```

Feedback

- For *field3*:

```
<ipxact:vendorExtensions>
  <snps:resetnSource>
    <signal>top.rstblk.field3_resetn</signal>
  </snps:resetnSource>
</ipxact:vendorExtensions>
```

16.7.10 Out of Address range Capability

The out of address range capability enables you to specify an exact value to be returned when read access occurs outside the defined address range. This vendor extension works in conjunction with RSVD_MODE = 3 to provide precise control over out-of-address range access.

XML format to be specified in the IP-XACT file:

```
<ipxact:vendorExtensions>
    <snps:outAddrValue>
        <value>1'b0</value>
    </snps:outAddrValue>
</ipxact:vendorExtensions>
```

16.7.11 Support for registerFile in IP-XACT

VC Formal FRV supports the *registerFile* defined in IEEE standard.

Example

The following example shows the case that *CTRL_STATUS* registerFile has 4 dimensions and each contains 2 registers *CTRL* and *STATUS*.

```
<ipxact:addressBlock>
  <ipxact:name>adblockname</ipxact:name>
  <ipxact:baseAddress>0x10000000</ipxact:baseAddress>
  <ipxact:range>0x100</ipxact:range>
  <ipxact:width>32</ipxact:width>
  <ipxact:registerFile>
    <ipxact:name>CTRL_STATUS</ipxact:name>
    <ipxact:dim>4</ipxact:dim>
    <ipxact:addressOffset>0x0</ipxact:addressOffset>
    <ipxact:range>8</ipxact:range>
    <ipxact:register>
      <ipxact:name>CTRL</ipxact:name>
      <ipxact:addressOffset>0x0</ipxact:addressOffset>
      <ipxact:size>32</ipxact:size>
      <ipxact:access>read-write</ipxact:access>
      <ipxact:field>
        <!-- ... -->
      </ipxact:field>
```

Feedback

```
    </ipxact:register>
    <ipxact:register>
      <ipxact:name>STATUS</ipxact:name>
      <ipxact:addressOffset>0x4</ipxact:addressOffset>
      <ipxact:size>32</ipxact:size>
      <ipxact:access>read-only</ipxact:access>
      <ipxact:field>
        <!-- ... -->
      </ipxact:field>
    </ipxact:register>
  </ipxact:registerFile>
```

The following example shows the case that *CHAN*registerFile has 2 dimensions and each contains register file *ARRAY*. *ARRAY* has 32 dimensions and each contains register *REG*.

```
<ipxact:addressBlock>
  <ipxact:name>adblockname</ipxact:name>
  <ipxact:baseAddress>0x10000000</ipxact:baseAddress>
  <ipxact:range>0x100</ipxact:range>
  <ipxact:width>32</ipxact:width>
  <ipxact:registerFile>
    <ipxact:name>CHAN</ipxact:name>
    <ipxact:dim>2</ipxact:dim>
    <ipxact:addressOffset>0x0000</ipxact:addressOffset>
    <ipxact:range>0x80</ipxact:range>
    <ipxact:registerFile>
      <ipxact:name>ARRAY</ipxact:name>
      <ipxact:dim>32</ipxact:dim>
      <ipxact:addressOffset>0x0</ipxact:addressOffset>
      <ipxact:range>0x4</ipxact:range>
      <ipxact:register>
        <ipxact:name>REG</ipxact:name>
        <ipxact:addressOffset>0x0</ipxact:addressOffset>
        <ipxact:size>32</ipxact:size>
        <ipxact:access>read-write</ipxact:access>
        <ipxact:field>
          <!-- ... -->
        </ipxact:field>
      </ipxact:register>
    </ipxact:registerFile>
  </ipxact:registerFile>
```

16.8 Extensions Supported in the RALF File

16.8.1 Write and Read Mask Capability

The write and read mask capability that is not directly supported by RALF, is supported through vendor extension in VC Formal. Some registers may be restricted to write or read

Feedback

an entire word. The value or signal for write/read mask can be specified and the value to be read for the masked bit can be specified (either 0, 1 or don't care).

```
register REG_RW1_0 @0x010 {
            bytes 4;
            field FLD_RW_0 @'h0 {
                bits 32;
                access rw;
                hard_reset 'h05bc0000;
                attributes {
                  write_mask "{32{test.fld_rw_0[0]}}",
                  read_mask "{32{test.fld_rw_0[0]}}",
                  read_mask_type "1"
                }
```

- `write_mask`: bitwise mask for write access, 1 implies enable write, 0 implies mask write. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field. To specify this, you should use a prefix `REGFIELD::,` for example, to use value of field 'F1' of register 'R1' as the `read_mask`, specify as follows:

    ```
    write_mask "REGFIELD::R1.F1"
    ```
- `read_mask`: bitwise mask for read access, 1 implies enable read, 0 implies mask read. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field. To specify this, you should use a prefix `REGFIELD::`, for example, to use value of field 'F1' of register 'R1' as the `read_mask`, specify it as follows:

    ```
    read_mask "REGFIELD::R1.F1"
    ```
- `read_mask_type`: indicate what data must be read when it is masked
  - 0: 0 should be returned as masked data
  - 1: 1 should be returned as masked data
  - 2: don't care and no check (default)
  - 3: when value other than 0 or 1 is to be returned as masked data, `read_mask_rdvalue` should be specify the masked data.

- `read_mask_rdvalue`: indicates what data should be read when it is masked. It can be:
  - signal name with path or
  - value in Verilog format or
  - value of any other field, using a prefix `REGFIELD::` (as mentioned above)

16.8.2 Hardware Update Capability

The hardware update capabilities that is not directly supported by RALF file, is supported through vendor extension in VC Formal. You can specify hardware update information using a comma separated list of key-value pairs for a field in the `attributes` construct.

The following is the use model to specify a hardware update vendor extensions.

- Specify `start`and `end`in the construct to provide the boundaries.
- Specify a key or a value within quotes if it contains a space, “.”, “:”, and so on.

```
field IP @'h0 {
bits 48;
reset 'h0;
access rw;
attributes {
  fieldHWupdates start,
     fieldHWupdate start,
        id hw_r1_0,
        trigger "hwupdate_aliased.gpio_wren",
        value "hwupdate_aliased.gpio_data",
        latency 1,
        function load,
     fieldHWupdate end,
     fieldHWupdate start,
        id hw_r1_1,
        trigger "hwupdate_aliased.gpio_wren",
        value "hwupdate_aliased.gpio_data",
        latency 1,
        function load,
     fieldHWupdate end,
     priority "hw_r1_1, hw_r1_0, sww, swr",
  fieldHWupdates end
   };
};
```

16.8.3 Latency Update Capability

The field latency update capabilities that is not directly supported by RALF file, is supported through vendor extension in VC Formal. You can specify the field latency

Feedback

information using a comma separated list of key-value pairs for a field in the `attributes` construct.

The following is the use model to specify the field latency vendor extensions.

- Specify `start`and `end`in the construct to provide the boundaries.
- Specify a key or a value within quotes if it contains a space, ".", ":", and so on.

```
field IP @'h0 {
bits 48;
reset 'h0;
access rw;
attributes {
    fieldLatencies start,
       rdLatency "0:4",
       wrLatency "0:5",
    fieldLatencies end,
  }
}
```

16.8.4 Disable Checks Capability

Disable checks capability which is not directly supported by RALF, is supported through this vendor extension. The disableChecks vendor extension can be one of two types, `allowValue` or `inhibitValue`. Only one of these can be specified at any point of time.

The following is the use model to specify disableChecks vendor extension.

```
disableChecks start,
allowValue " ",
inhibitValue " ",
disableChecks end,
```

Examples

- disableChecks: inhibitValue

  Consider the case of register FLD_21_WO, where all the assertions need to be disabled, if 1'b1 is written to it.

```
field FLD_21_WO @'h00000001 {
             attributes {
                           disableChecks start,
                           inhibitValue "1'b1",
                           disableChecks end
                        };
              bits 1; access w1c; reset 'h001;
};
```

- disableChecks: allowValue

Feedback

Consider the case of register FLD_32_WO, where all the assertions need to be disabled for any value other than 2'b0.

```
field FLD_32_WO @'h00000001 {
              attributes {
                              disableChecks start,
                              allowValue "2'b0",
                              disableChecks end
                          };
              bits 1; access w1c; reset 'h001;
};
```

16.8.5 Variable Reset Capability

Variable reset capability which is not directly supported by RALF, is supported through this vendor extension. Use the `variableReset` attribute to set a reset value for a register, whose reset value changes depending on the input value.

Example

Consider the scenario, where *simple_test.sel* can set the reset value of the field.

- If *simple_test_sel == 1*, reset value for the field is 32'hdeadbeaf
- Else, the reset value is 32'h0

```
block simple_test {
  bytes 4;

  register REG_RW0 @'h40C {
    bytes 4 ;
    field FLD_RW_0    (simple_test.fld_rw_0) @'h00000000 {
                              attributes {
                                              variableReset start,
                                              signal "simple_test.sel",
                                              value "0:32'h00000000;1:32'hdeadbeaf;2",
                                              mask "32'hffffffff",
                                              mode "1",
                                              variableReset end };

                              bits 32 ; access rw  ;reset 'hdeadbeaf ;


    }
  }
}
```

16.8.6 Individual Reset for Field Capability

By default, FRV expects all the register fields to reset with the bus reset. However, some fields may reset with a different reset source depending on the design. To support

Feedback

such designs, FRV supports individual reset capability for register fields through vendor extension in RALF. Through this vendor extension, you can specify individual reset signal for each register field.

By default, bus reset is treated as the primary reset. If the reset source vendor extension is specified for a field, the specified reset source is used to reset the field.

Example

```
block simple_test {
  bytes 4;

  register REG_RW0 @'h40C {
    bytes 4 ;
    field FLD_RW_0    (simple_test.fld_rw_0) @'h00000000 {
                                attributes {
                                       resetnSource start,
                                       signal "simple_test.sel",
                                       resetnsource end
                                     };


   }
  }
}'
```

16.8.7 Support for registerFile in IP-XACT

VC Formal FRV supports the *registerFile* defined in IEEE standard.

Example

The following example shows the case that *CTRL_STATUS* registerFile has 4 dimensions and each contains 2 registers *CTRL* and *STATUS*.

```
<ipxact:addressBlock>
  <ipxact:name>adblockname</ipxact:name>
  <ipxact:baseAddress>0x10000000</ipxact:baseAddress>
  <ipxact:range>0x100</ipxact:range>
  <ipxact:width>32</ipxact:width>
  <ipxact:registerFile>
    <ipxact:name>CTRL_STATUS</ipxact:name>
    <ipxact:dim>4</ipxact:dim>
    <ipxact:addressOffset>0x0</ipxact:addressOffset>
    <ipxact:range>8</ipxact:range>
    <ipxact:register>
      <ipxact:name>CTRL</ipxact:name>
      <ipxact:addressOffset>0x0</ipxact:addressOffset>
      <ipxact:size>32</ipxact:size>
      <ipxact:access>read-write</ipxact:access>
      <ipxact:field>
```

Feedback

```
        <!-- ... -->
      </ipxact:field>
    </ipxact:register>
    <ipxact:register>
      <ipxact:name>STATUS</ipxact:name>
      <ipxact:addressOffset>0x4</ipxact:addressOffset>
      <ipxact:size>32</ipxact:size>
      <ipxact:access>read-only</ipxact:access>
      <ipxact:field>
        <!-- ... -->
      </ipxact:field>
    </ipxact:register>
  </ipxact:registerFile>
```

The following example shows the case that *CHAN* registerFile has 2 dimensions and each contains register file *ARRAY*. *ARRAY* has 32 dimensions and each contains register *REG*.

```
<ipxact:addressBlock>
  <ipxact:name>adblockname</ipxact:name>
  <ipxact:baseAddress>0x10000000</ipxact:baseAddress>
  <ipxact:range>0x100</ipxact:range>
  <ipxact:width>32</ipxact:width>
  <ipxact:registerFile>
    <ipxact:name>CHAN</ipxact:name>
    <ipxact:dim>2</ipxact:dim>
    <ipxact:addressOffset>0x0000</ipxact:addressOffset>
    <ipxact:range>0x80</ipxact:range>
    <ipxact:registerFile>
      <ipxact:name>ARRAY</ipxact:name>
      <ipxact:dim>32</ipxact:dim>
      <ipxact:addressOffset>0x0</ipxact:addressOffset>
      <ipxact:range>0x4</ipxact:range>
      <ipxact:register>
        <ipxact:name>REG</ipxact:name>
        <ipxact:addressOffset>0x0</ipxact:addressOffset>
        <ipxact:size>32</ipxact:size>
        <ipxact:access>read-write</ipxact:access>
        <ipxact:field>
          <!-- ... -->
        </ipxact:field>
      </ipxact:register>
    </ipxact:registerFile>
  </ipxact:registerFile>
```

16.8.8 Latency Setting for Read Action

VC Formal FRV supports latency for Read Action. You can specify after how many cycles the tool should reflect the read action on the field after the read on the bus. By default, the Read Action latency is set to 1.

Example

Feedback

The following example shows the case in which the Read Action latency for field `rega_fldb` is 2.

```
field rega_fldb @'d20 {
      bits 4;
      access rw;
      reset 'h0;
      attributes {
        fieldLatencies start,
          rdActLatency 2,
        fieldLatencies end
      }
    }
```

16.8.9 Support for regfile in RALF

VC Formal FRV supports the *regfile* defined in RALF.

Example

The following example shows the case that *CTRL_STATUS* registerFile has 4 dimensions and each contains 2 registers *CTRL* and *STATUS*.

```
block ADBLK @0x10000000 {
    bytes 4;
    regfile CTRL_STATUS[4] @0x00 {
        register CTRL @'h0 {
            bytes 4;
            field FLD @'h0 {
                bits 32;
                access rw;
                hard_reset 'b0;
            }
        }
        register STATUS @'h4 {
            bytes 4;
            field FLD @'h0 {
                bits 32;
                access ro;
                hard_reset 'b0;
            }
        }
    }
```

Note:

Currently, FRV does not support nested regfile, that is, regfile under regfile.

Feedback

16.9 Register/Field Access Types

Each register/field has the required access type along with optional read action, and write modified values.

Note:

You must check the script ware for correct integer mappings.

Access Attribute:

```
access := { U, RO, WO, RW, RW1, WO1, RSVD}
```

- U – undefined
- RO – read-only
- WO – write-only
- RW – read-write
- RW1 – read-writeOnce
- WO1 – writeOnce
- RSVD – reserved

Optional Read Action Attribute:

```
read_action := { U, C, S, M, 0, 1 }
```

- U – undefined
- C – clear
- S – set
- M – modify
- 0 – zero
- 1 – one

Optional Modified Write Value Attribute:

```
mod_write :={ U, 1C, 1S, 1T, 0C, 0S, 0T, C, S, M }
```

- U – undefined
- 1C – oneToClear
- 1S – oneToSet
- 1T – oneToToggle

Feedback

- 0C – zeroToClear
- 0S – zeroToSet
- 0T – zeroToToggle
- C – clear
- S – set
- M - modify

For display purposes an access type attribute will be defined that combines access, read action and modified write values:

Access Type Attribute:

```
access_type := access[,[read_action][,mod_write]]
```

The following is a mapping from the IP-XACT data and RALF access.

Table 28: Mapping from the IP-XACT data and RALF access

Access Type	**IP_XACT**			**RALF**
	access	**readAction**	**modifiedWriteValue**	
RO	read-only			ro
WO	write-only			wo
RW	read-write			rw
RW1	read-writeOnce			w1
WO1	writeOnce			wo1
RW,,1C	read-write		oneToClear	w1c
RW,,1S	read-write		oneToSet	w1s
RW,,1T	read-write		oneToToggle	w1t
RW,,0C	read-write		zeroToClear	w0c
RW,,0S	read-write		zeroToSet	w0s
RW,,0T	read-write		zeroToToggle	w0t
RW,,C	read-write		clear	wc
RW,,S	read-write		set	ws

Feedback

Table 28: Mapping from the IP-XACT data and RALF access (Continued)

Access Type	IP_XACT			RALF
	access	readAction	modifiedWriteValue	
RW,C	read-write	clear		wrc
RW,S	read-write	set		wrs
RW,C,S	read-write	clear	set	wsrc
RW,S,C	read-write	set	clear	wcrs
RW,C,1S	read-write	clear	oneToSet	w1src
RW,S,1C	read-write	set	oneToClear	w1crs
RW,C,0S	read-write	clear	zeroToSet	w0src
RW,S,0C	read-write	set	zeroToClear	w0crs
WO,,C	write-only		clear	woc
WO,,S	write-only		set	wos
RO,C	read-only	clear		rc
RO,S	read-only	set		rs

16.9.1 RISC-V CSR Specific Access Types

FRV supports the following RISC-V CSR specific access types:

- Reserved Writes Preserve Values, Reads Ignore Values (WPRI)
- Write / Read Only Legal Values (WLRL)
- Write Any Values, Reads Legal Values (WARL)

To specify these access types in IP-XACT, use the `specialAccess` vendor extension.

Example

To set access type WARL when legal value is 2’b11, 2’b01, 2’b00, use the following:

```
<ipxact:vendorExtensions>
       <snps:specialAccess>
          <access>WARL</access>
          <legalValues>2'b11, 2'b01, 2'b00</legalValues>
       </snps:specialAccess>
</ipxact:vendorExtensions>
```

Feedback

16.10 IPXACT Attributes Support

The following figure provides the list of IPXACT attributes supported by VC Formal.

M: Mandatory, O: Optional.

addressBlock		register		field		Comments
name		name		name		
baseAddress	M	addressOffset	M	bitOffset	M	
range	M	accessHandles	O	accessHandles	O	
width	M	size	M	bitWidth	M	
		volatile	O	volatile	O	"true" indicates there is no guarantee read is consistent with previous write or read. Default: false
		access	O	access	O	read-only, write-only, read-write, writeOnce, read-writeOnce
				modifiedWriteValue	O	clear, set, oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle
				readAction	O	clear, set
				testable	O	"true" indicates the field is testable by a simple automated register test. Default: true
				writeValueConstraint	O	Specify a set of values that are acceptable to be written to this field. Takes a minimum and maximum value range. Enums are not supported.
		reset	O	reset	O	
		value	M	value	M	
		mask	O	mask	O	
		vendorExtensions	O	vendorExtension	O	

16.10.1 DTD Support for IPXACT Attributes

The IPXACT supports DTD entity in xml for the *accessHandles* attribute. The *accesshandle* can use aliasName, which would parsed to the actual name of the signal.

Note:

The DTD entity support is available for the accessHandles attribute only.

Feedback

The xml format for the DTD entity is as follows:

Syntax

```
<? xml version = '1.0' encoding ='UTF-8'?>
<! DOCTYPE ipxact: component [
<! ENTITY  aliasName  "actualName">
]>
```

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE ipxact:component[
<!ENTITY sig_sample test>
]>
<ipxact:register>
            <ipxact:name>REG_WONLY0</ipxact:name>
            <ipxact:addressOffset>0x004</ipxact:addressOffset>
            <ipxact:size>32</ipxact:size>
            <ipxact:access>write-only</ipxact:access>
            <ipxact:field>
              <ipxact:name>FLD_WO_0</ipxact:name>
              <ipxact:bitOffset>0</ipxact:bitOffset>
              <ipxact:bitWidth>32</ipxact:bitWidth>
              <ipxact:access>write-only</ipxact:access>

 <ipxact:accessHandles>@sig_sample;.fld_wo_0</ipxact:accessHandles>
            </ipxact:field>
          </ipxact:register>
```

16.11 Checkers parameters

The checkers generated by `frv_load` command has the parameters described in Table 29. Set these parameters based on design specification and verification needs.

Table 29: Checker Parameters

Parameter name	Description	Default Value
ADDR_WIDTH	Address bit width	32
DATA_WIDTH	Data bit width	32

Feedback

Table 29: Checker Parameters (Continued)

Parameter name	Description	Default Value
RSVD_MODE	Specify how to check reserved and/or unmapped registers and/or fields: 0: no check when ACCESS_TYPE==ACS_RSVD 1: 0 should be read when ACCESS_TYPE==ACS_RSVD and CHECK_INIT==0 2: 1 should be read when ACCESS_TYPE==ACS_RSVD and CHECK_INIT==0 3: Custom value should be read when ACCESS_TYPE==ACS_RO and CHECK_INIT==1	1
COMPOSITE	Specify type of assertions per field for initial value and after write: 1: Generate one common assertion (better convergence) 0: Generate two individual assertions	1
BUS_LATENCY	Effective only in generic wrapper: Latency from read enable signal to read data valid on bus	1
WR_LATENCY	Write Latency from bus write data propagates to internal field	1
RD_LATENCY	Read Latency from internal field to bus read data	1
COVER_TYPE	When COVER_WRITE==13'bxx1: cover write between write to read window3'bx1x: cover write before first write3'b1xx: cover write before first readWhen COVER_READ==13'bxx1: cover read between write to read window3'bx1x: cover read before first write3'b1xx: cover read before first read	3'h7
COVER_WRITE	Specify enable or disable write access related cover properties 0: Disable all write access related cover properties 1: Enable all write access related cover properties	1
COVER_READ	Specify enable or disable read access related cover properties 0: Disable all read access related cover properties 1: Enable all read access related cover properties	1
COVER_BYTE_UPDATE	Generate cover properties to see valid byte lane update for a field 0: Disable covers for a field 1: Enable covers for a field	0

Feedback

Table 29: Checker Parameters (Continued)

Parameter name	Description	Default Value
OUTADDR_EN	Specify enable or disable out of address range access related cover properties 0: Disable out of address range access related cover properties 1: Enable out of address range access related cover properties	1
LATENCY_MD	Specify Latency handling 0: RD_LATENCY parameters are used only in backdoor checks 1: RD_LATENCY parameters are used in all checks	0
OUTABLK_MD	Specify how to check out of address range accesses 0: check read data should be 0 for out of addressBlock range 1: check read data should be 1 for out of addressBlock rangeother: no check for out of addressBlock range	2
SNPS_AD_ABS	Synopsys Non-Deterministic Abstractions 0: check all possible scenarios 1: check only specific scenario in each step	0
COVER_ALLOW_ERROR	Controls if cover is included/excluded during errors: 0: Exclude cover when errors occur 1: Include cover when errors occur	0

16.11.1 Support for AIP FRV

Only some of the AIPs support the FRV interface and work only in new flow. In the old flow, even the AMBA bus interface, you can prepare quite similar, but slightly different two bind statements, one is for AIP and the other is FRV checker. In the new flow, when you use AIP with the FRV interface, you do not have to prepare bind statement for FRV checker file. You can prepare only a bind statement for AIP, and set the `FRV` parameter with 1. AIP instantiates the FRV checker module when `FRV` is set with 1. The bind example is shown below.

```
bind dut_top snps_apb_aip
  #(.APB_VERSION     (snps_apb_aip_pkg::APB3),
    .AGENT_TYPE      (snps_apb_aip_pkg::MASTER),
    .ENABLE_ASSERT   (0),
    .ENABLE_ASSUME   (1),
    .ENABLE_COVER    (0),
    .PSEL_WIDTH      (1),
    .ADDR_WIDTH      (16),
```

Feedback

```
    .WDATA_WIDTH      (32),
    .RDATA_WIDTH      (32),
    .CONFIG_LOWPOWER (0),
    .CONFIG_X_CHECK   (0),
    .FRV              (1))
APB_SLAVE
  (.pclk    (CLK),
   .presetn (RESETN),
```

This makes it easier to apply to FRV.

Table 30: AIP Wrapper and Bind

Bus protocol	Use AIP	Bus wrapper	bind	Add in analyze/read_file
APB	Yes	No need	Bind AIP with setting FRV=1	Only bind file *1
	No	snps_frv_apb_wrapper.sv	Bind bus wrapper	Only bind file *2
AHB	Yes	No need	Bind AIP with setting FRV=1	Only bind file *1
	No	snps_frv_ahb_wrapper.sv	Bind bus wrapper	Only bind file *2
AXI	Yes/No	snps_frv_axi_wrapper.sv	Bind bus wrapper	Only bind file *2
Simple bus	N/A	snps_frv_gen_wrapper.sv	Bind bus wrapper	Only bind file *2
Complex bus	N/A	Need to create	Bind bus wrapper	Bus wrapper and bind files

Note:

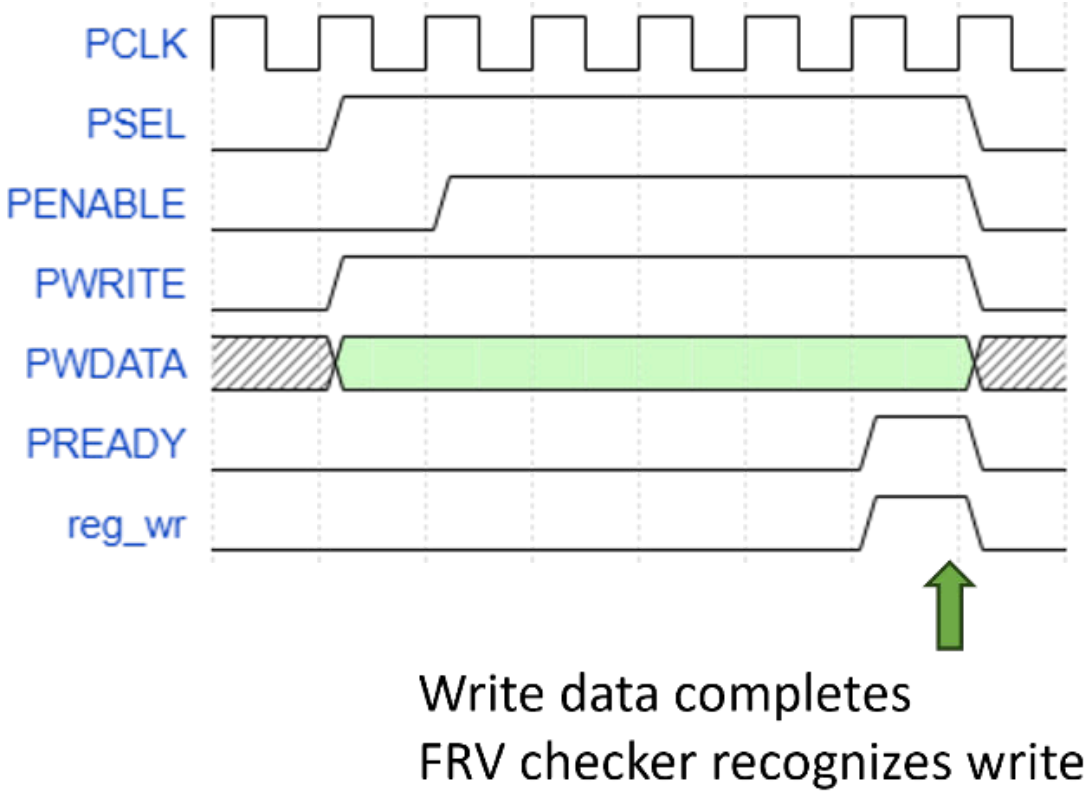
snps_frv_(apb/ahb/axi/gen)_wrapper.sv’ files are in $VC_STATIC_HOME/packages/Frv directory *1) use ‘aip_load ’ and 'frv_load ... -auto_load' *2) use ‘frv_load ... -auto_load'

Feedback

16.12 Considerations While Specifying Write Data Transaction for FRV APB Bus Wrapper

Write data transaction completes when PSEL & PWRITE & PENABLE & PREADY on APB bus as shown in the following waveform. The FRV APB bus wrapper considers write occurs at this cycle.

Figure 131:



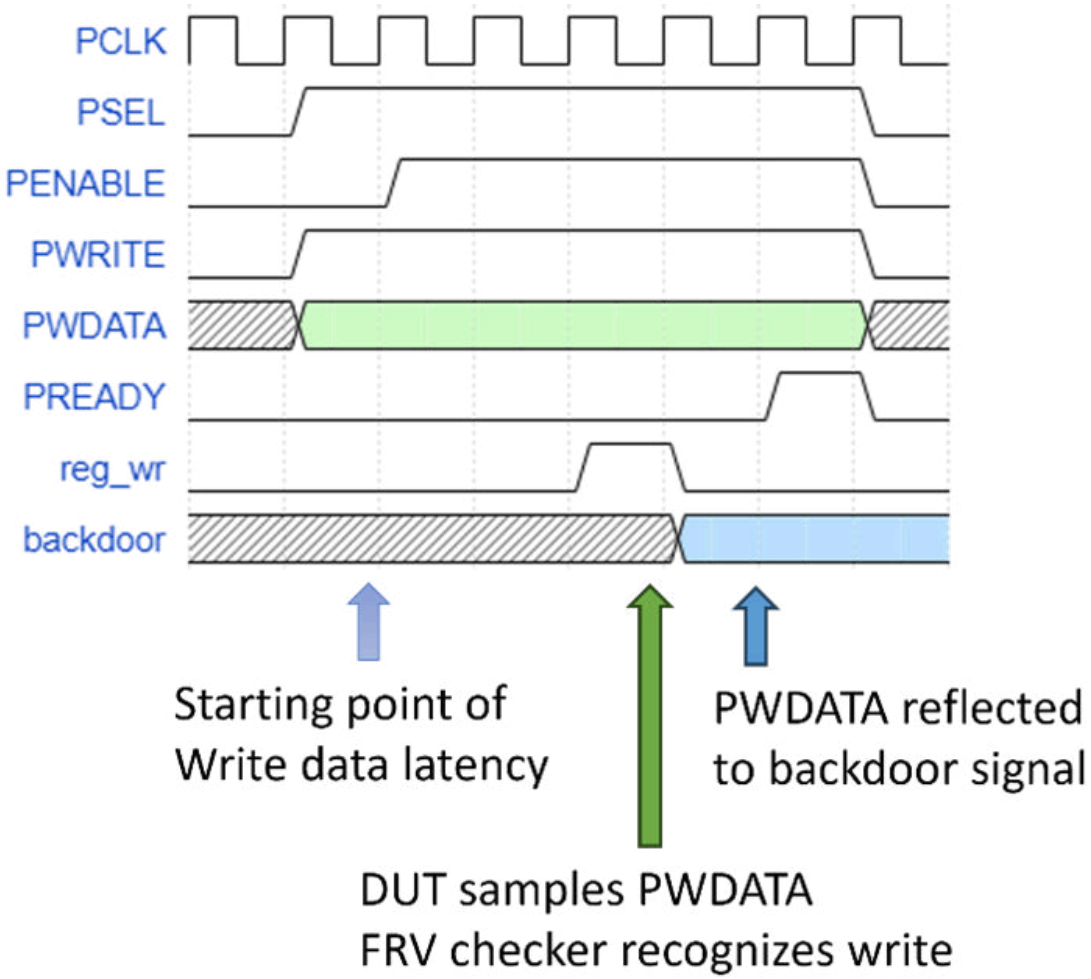
This works in normal case, however, this scheme does not work in cases when write latency affects the check, such as backdoor write check, and HW update event is specified.

In APB protocol, PWDATA is valid when PSEL & PWRITE until PREADY goes High, and DUT can sample PWDATA at any cycle. And backdoor signal is updated based on the cycle DUT samples PWDATA. The backdoor signal may be updated earlier than write data completion, that is, PSEL & PWRITE & PENABLE & PREADY depending on DUT. The backdoor check assertion gets false failure in such case. To avoid this scenario, FRV APB bus wrapper supports the capability user can specify when DUT samples PWDATA on

Feedback

APB. The starting point is the first cycle of write transaction, that is, PSEL & PWRITE & ~PENABLE.

Figure 132:



For example, if DUT samples PWDATA after 3 cycles from PSEL & PWRITE & ~PENABLE and the sampled write data is reflected at the next cycle to backdoor signal, set WDATA_DELAY with 3 and WR_LATENCY with 1 in bind for `snps_frv_apb_wrapper.sv`.

Also, this write latency can be set per address range. You can specify address range using the following parameters.

- `NUM_ADDR_RANGE`: The number of address ranges
- `START_ADDR`: Start address of each address range

- `LAST_ADDR`: Last address of each address range
- `WDATA_DELAY`: Write data latency in each address range
- `MAX_WDATA_DELAY`: The maximum value of write data latency

For example, let assume there are 3 address ranges, and each write latency is as described in the following table.

Start Address	Last Address	Write data latency
16'h0000	16'h01ff	3
16'h0200	16'h03ff	1
16'h0400	16'h1fff	5

The expected parameter setting is as follows.

NUM_ADDR_RANGE	3
START_ADDR	{16'h0000, 16'h0200, 16'h0400}
LAST_ADDR	{16'h01ff, 16'h03ff, 16'h1fff}
WDATA_LATENCY	{3, 1, 5}
MAX_WDATA_LATENCY	5

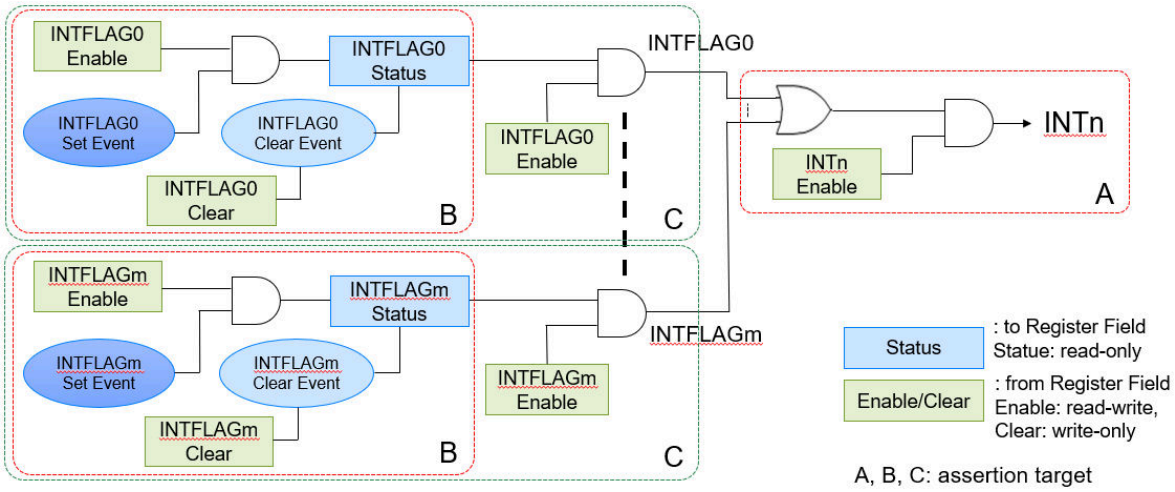
16.13 Formal Interrupt Verification

VC Formal FRV is enhanced to verify interrupts. Interrupt signals are generated by various events, and enabling or clearing interrupt is controlled through register access. And also, software checks interrupt status through status register read. This implies that, interrupt control is tightly connected with register accesses. Therefore, you can use FRV to verify interrupt control.

Figure 133 illustrates a typical interrupt generation process. Interrupt flag is enabled or disabled per event. Interrupt flag can be cleared directly by interrupt clear register or can be combined with design internal event. Interrupt flag for each event can be referred through interrupt status register.

Feedback

Figure 133: Generating Interrupt Flow



Rectangles with light blue or light green indicate registers related to interrupt control. These registers are specified by IP-XACT, RALF or CSV when applying FRV. You can specify which register is corresponding to which interrupt or interrupt flag. And you should specify the interrupt set and clear event for each interrupt flag. FRV automatically generates assertions to verify interrupt control.

To enable interrupt verification in FRV, use the `frv_config [-enable_fiv <value>]` command as described in The frv_config Command.

FRV generates assertions for 3 groups: A, B and C as illustrated in Figure 133.

- A is for main interrupt signal.
- B is for interrupt flags contributed to main interrupt signal.
- C is optional when you want verify each interrupt flag signal explicitly.

FRV generates the following assertions for each main interrupt signal.

Table 31: Interrupt Signal Assertions

Assertion Name	Description	Target
ast_interrupt_assert_expected	If interrupt is enabled and at least 1 interrupt flag is asserted (interrupt flag is enabled and set event occurred), interrupt signal should be asserted.	A
ast_interrupt_assert_allowed	If interrupt signal is asserted, it should be enabled and at least 1 interrupt flag should be asserted (interrupt flag is enabled and set event occurred).	A

Feedback

Table 31: Interrupt Signal Assertions (Continued)

Assertion Name	Description	Target
ast_interrupt_deassert_expected	If interrupt is not enabled or none of interrupt flag is asserted, interrupt should be de-asserted.	A
ast_interrupt_deassert_allowed	If interrupt signal is de-asserted, interrupt should not be enabled or all interrupt flag should be de-asserted.	A
ast_flag_status_assert_expected	If interrupt flag is enabled, interrupt flag set event occurred and it's not clearing, interrupt flag status should be asserted at the next cycle.	B
ast_flag_status_assert_allowed	If interrupt flag status is asserted, it should be enabled, interrupt flag set event occurred and it's not clearning at the previous cycle.	B
ast_flag_status_deassert_expected	If interrupt flag clear event occurred, interrupt flag status should be de-asserted at the next cycle.	B
ast_flag_status_deassert_allowed	If interrupt flag status is de-asserted, interrupt flag clear event should have occurred at the previous cycle.	B
ast_int_flag_assert_expected	If interrupt flag is enabled and interrupt flag status is asserted, interrupt flag signal should be asserted.	C
ast_int_flag_assert_allowed	If interrupt flag signal is asserted, it should be enabled and interrupt flag status should be asserted.	C
ast_int_flag_deassert_expected	If interrupt flag is not enabled or interrupt flag status is de-asserted, interrupt flag signal should be de-asserted.	C
ast_int_flag_deassert_allowed	If interrupt flag signal is de-asserted, interrupt flag should not be enabled or interrupt flag status should be de-asserted.	C

FRV generates the following cover properties for each main interrupt signal.

Feedback

Table 32: Interrupt Signal Covers

Cover Property Name	Description
cov_interrupt_enable_assert	Interrupt is enabled and interrupt is asserted.
cov_interrupt_enable_deassert	Interrupt is enabled and interrupt is de-asserted.
cov_interrupt_disable_deassert	Interrupt is disabled and interrupt is de-asserted.
cov_flag_status_value	The combination of {status, enable, set, clear} for interrupt flag.
cov_int_flag_enable_assert	Interrupt flag is enabled and interrupt flag is asserted.
cov_int_flag_enable_deassert	Interrupt flag is enabled and interrupt flag is de-asserted.
cov_int_flag_disable_deassert	Interrupt flag is disabled and interrupt flag is de-asserted.

Specifying Interrupt Signals in IP-XACT

Use the following IP-XACT format to describe interrupt information.

```
      (Interrupt ID)
          (Interrupt Signal)
          (Interrupt Enable)
              (Interrupt Flag ID)
              (Interrupt Flag signal)
              (Interrupt Flag Enable)
              (Flag Enable)
              (Flag Status)
              (Flag Set Condition)
              (Flag Clear Condition)
              (multiple intFlag)
          (multiple interrupt)
```

Example

```
<ipxact:addressBlock>
 <ipxact:name>FIV_EXAMPLE</ipxact:name>
 <ipxact:baseAddress>0x1000</ipxact:baseAddress>
 <ipxact:range>0x100</ipxact:range>
 <ipxact:width>32</ipxact:width>
 <ipxact:vendorExtensions>
     <snps:interrupts>
       <snps:interrupt>
         <intId>INT0</intId>
         <intSignal>fiv_dut.INT0</intSignal>
         <intEnable>REGFIELD::INT_CTRL.IT0EN</intEnable>
         <snps:intFlags>
```

Feedback

```
          <snps:intFlag>
            <flagId>IT0IC0</flagId>
            <flagSignal>fiv_dut.int_flag[0]</flagSignal>
            <intFlagEnable>REGFIELD::INT_FLAG_CTRL.IF0EN</intFlagEnable>
            <flagEnable>REGFIELD::FLAG_ENABLE.FE0</flagEnable>
            <flagStatus>REGFIELD::INT_FLAG_STATUS.IFS0</flagStatus>
            <flagSet>fiv_dut.flag_set[0]</flagSet>
            <flagClear>REGFIELD::INT_FLAG_CLEAR.IF0CLR</flagClear>
          </snps:intFlag>
        </snps:intFlags>
      </snps:interrupt>
   <snps:interrupts>
<ipxact:vendorExtensions>
</ipxact:addressBlock>
```

Where the mandatory fields are:

- `<snps:interrupt>` is mandatory inside `<snps:interrupts>`.
- `<snps:intFlags>` is mandatory inside `<snps:interrupt>`.
- `<intId>` and `<intSignal>` in `<snps:interrupt>` are mandatory.
- `<snps:intFlag>` in `<snps:intFlags>` is mandatory.
- `<flagId>` and `<flagStatus>` in `<snps:intFlag>` are mandatory.

Default Values:

- If `<intEnable>` is not specified, it can be assumed as "1".
- If `<intFlagEnable>` or `<flagEnable>` is not specified, it can be assumed as “1”.
- If `<flagSet>` or `<flagClear>` is not specified, it can be assumed as “0”.

17

Formal Property Verification Application

The VC Formal FPV application uses formal proof-based techniques to verify SystemVerilog Assertion (SVA) properties to ensure correct operation across all possible design activity even before the simulation environment is available. Advanced assertion visualization, property browsing, grouping and filtering allow simple concise access to results. Formal navigator enables debug and "what-if-analysis" with only the RTL design in the Unified Verdi GUI with/without assertions or Testbench.

Getting Started • *What is VC Formal FPV?* • *Coding Properties in SVA for VC Formal FPV* • *Accessing FPV Lab Example*	**Improving Convergence of Properties** • *Adding Abstractions* • *Leveraging Formal Scoreboard and FIFO/Memory Abstraction Models* • *Computing Reduced Constraints* • *Computing Formal Core for Proofs* • *Using Proof Assist* • *Using Proof Architect*
Setting Up and Running FPV • *Building Your Own Script with Setup Wizard* • *Enabling FPV Application mode* • *Loading Assertion IPs (AIPs)* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Verifying Design Setup* • *Reviewing and Editing Properties* • *Running FPV in TCL and GUI*	**Debugging, Reviewing and Saving Results** • *Generating Verification Summary* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving and Restoring Sessions*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	**Getting Help on Commands** • *FPV Tcl Commands*

17.1 About FPV Application

The Formal Property Verification (FPV) application mode can be used to verify the following types of properties in the design.

- Assertions to check design behavior
- Assumptions to constrain the formal verification environment
- Coverage property to monitor interesting/expected events

For more details on the supported properties, see section About Properties.

17.1.1 Formal Property Verification

The FPV app is an application used to prove properties which verify a DUT. These properties are to be prepared by users, and/or those defined in the commercial AIPs for interface protocol or function blocks.

The FPV app uses various powerful VC Formal engines to work towards exhaustively proving or disproving a property. If an assertion fails, the FPV app provides a counter-example to show a violating trace. In some cases, it may not be possible to definitively prove or disprove a property. In such cases, a bounded proof result will be given showing that no falsification can be found up to a specific depth from the initial state.

For a set of assertions, proofs mean that under the constraints given, there is no way to falsify the properties. In the FPV application, signing off verification means checking that enough assertions have been written and that there are no over constraints in the design. For information on this process see section Obtaining Formal Signoff For a Design for more information.

A challenge which may be encountered when using the FPV app is that of property convergence. In the event that the tool cannot reach a conclusive result for a property, advanced convergence techniques can be used to try and reach full convergence. There are several verification approaches to tackle this issue such as nondeterminism and abstraction which are discussed in section Considerations for Convergence Improvement. Choosing the right design for formal is also important.

17.1.2 Video Links

Click the links to view videos on FPV:

- FPV Setup and Falsification Debug (4 mins)
- Formal Run Progress Monitor (6 mins)
- FPV Coverage Analysis (10 mins)

17.2 Methodology

This section describes how to bring up and execute an FPV testbench, and analyze the results.

- Inputs for FPV
- Creating FPV Testbench
- Executing FPV Testbench
- Analyzing Results

17.2.1 Inputs for FPV

Required inputs to FPV app are the following:

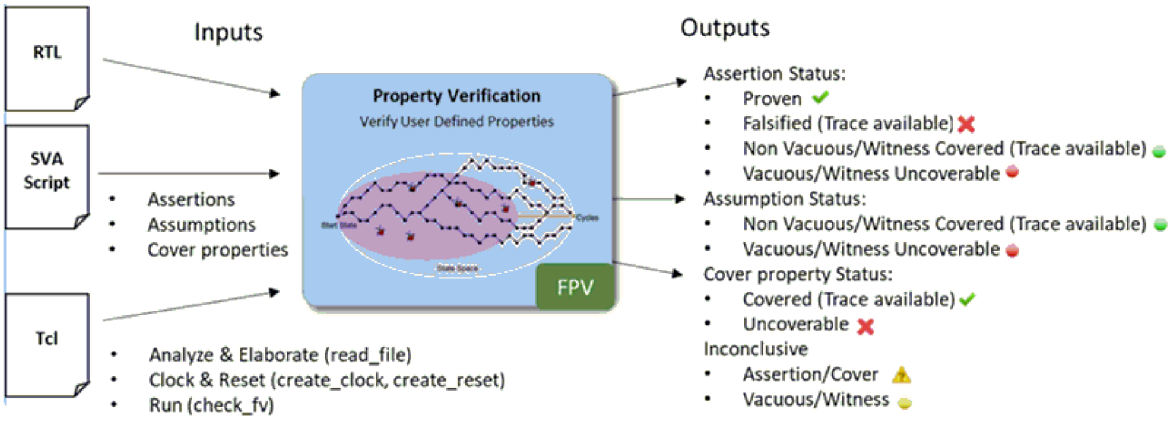
- RTL design files (Verilog/SystemVerilog/VHDL)
- Properties
  - A set of assertions (SVA and script property: `fvassert`)
  - A set of assumptions (SVA and script property: `fvassume`)
  - A set of cover properties (SVA and script property: `fvcover`)
- VC Formal compile and run commands
  - Analyze and elaborate commands (example, `read_file`)
  - Clock and reset commands (example, `create_clock`, `create_reset`)
  - Execute commands (example, `check_fv`)
  - Report commands (example, `report_fv`)

Outputs from FPV app are the following:

1. Property Status
   - Assertion status: Proven, Falsified, Vacuous, Witness-Coverable, Uncoverable and Inconclusive
   - Assumption status: Non-Vacuous, Vacuous, Witness-Coverable, Uncoverable and Inconclusive
   - Cover status: Coverable and Uncoverable

Feedback

- Available Traces
  - Assertion: Falsified, Witness and Vacuity traces
  - Assumption: Witness and Vacuity traces
  - Cover property: Covered trace



17.2.2 Creating FPV Testbench

The below shows an example of a simple traffic light controller design to illustrate how to create a setup for FPV. To load the design in FPV there needs to be a tcl file with the relevant options. For this example, we will use a file, run.tcl

1. Set the application mode to FPV

   ```
   set_fml_appmode FPV
   ```

   See section Setting the Application modes for alternative way to specify the application mode.

2. To use the read_file command to specify design and property file paths. The following command shows how to compile design whose top module name is *traffic*.

   ```
   read_file -top traffic -format verilog -sva -vcs
    "-f ../design/filelist"

   See section Compiling Designs for more information.
   ```

3. Use the `create_clock` command to specify a clock as show below. *clk* is a clock input to the top module traffic, and the following is an example of how to set it up:

   ```
   create_clock clk -period 100

   See section Specifying User-defined Clocks for more information.
   ```

4. Use the `create_reset` command to create reset. See section Setting Up User-defined Resets for details.

   The reset signal is a reset input to the top module traffic and the design is to be reset when it is high. The following example shows how to create a reset:

```
create_reset rst -sense high
```

   The overall tcl file should look like below

```
set_fml_appmode FPV
read_file -top traffic -format verilog -sva -vcs
 "-f ../design/filelist"
create_clock clk -period 100
create_reset rst -sense high
```

17.2.3 Executing FPV Testbench

The following steps show how to execute run.tcl in the interactive mode. See section Setting The VC_STATIC_HOME Environment Variable for more information about how to start VC Formal.

1. Start VC Formal in interactive mode using run.tcl as the script

```
% vcf -f run.tcl
```

   Another common use model is to start the tool in the Verdi GUI. This can be done by adding the -verdi switch to the vcf command:

```
% vcf -f run.tcl -verdi
```

2. It is recommended to check your setup before running a proof. Use the `check_fv_setup` and the `report_fv_setup` commands:

```
vcf> check_fv_setup
```

3. You can run this in blocking or non-blocking (default) mode.

```
vcf>report_fv_setup
  Design/Setup analysis results
  -----------------------------
  Reset violations                 :    0     running 1(1)
  Clock violations                 :    0     done
  Glitch violations                :    0     done
  Oscillating pure comb. loops     :    0     done
  Oscillating latch/ff comb. loops :    0     done
  Combinational loops              :    0
  Multidriven nets                 :    0
  Multidriven output modports      :    0
  > Checking in progress ...
0
```

Feedback

```
[Info] BITLEVEL_MODEL_STATS: Generated model with 0 gates, 0 inputs, 0
 registers, 0 initial constraints, 0 constraints.
[Info] BITLEVEL_MODEL_STATS: Generated model with 0 gates, 0 inputs, 0
 registers, 0 initial constraints, 0 constraints.
[Info] FDC_FVS: No setup issues found.
 The results can be viewed again by using the report_fv_setup command.
```

Any clock or reset violations would be shown in this report, as would other setup issues. In the report above, there are no identified issues. See section Verifying VC Formal Designs for more information.

4. You can use the `check_fv` command to prove the properties. In non-Verdi interactive mode, the property results will be displayed on the command line as they are solved. In the Verdi GUI, the results are shown using icons as explained in section Analyzing Results.

   The default output of the property results is as follows:

```
check_fv -block
[Info] FORMAL_I_CREATE: Create Formal Model:traffic.
[Warning] POTENTIAL_SETUP_ISSUES: There are potential setup issues.
 Please run check_fv_setup to confirm.
          Reset: 1  Clock: 0  Glitch: 0  Osc_loop: 0  Osc_seq: 0
[Info] FORMAL_I_RUN: Starting formal verification for check_fv
          Id: 0  Goals: 34  Constraints: 2  Block Mode: true
[Info] LIC_RT_CHECKOUT: VC Formal run time license checkout. Base:1
 FPV:1.
[Info] PERF_DIAGNOSTICS_ALERTS_START
Run configuration:
[Info] FV_RUN_CONFIG_INFO:  Task : FPV, cmd : check_fv
[Info] Configuration Details
[Info] Orchestration/Effort : default
[Info] Run time limit : 12H
[Info] Memory limit : 8GB
[Info] Number of workers requested : 4
[Info] FPV flow ver : v7
[Info] PERF_DIAGNOSTICS_ALERTS_END
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_green_no_waiting_main
 t1  vacuous  vacuity  00:00:00
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_green_no_waiting_main
 t1  vacuous  property  00:00:00
[Info] BITLEVEL_MODEL_STATS: Generated model with 633 gates, 12
 inputs, 93 registers, 0 initial constraints, 2 constraints.
[Info] BITLEVEL_MODEL_STATS: Generated model with 392 gates, 12
 inputs, 86 registers, 0 initial constraints, 2 constraints.
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_green_no_waiting_first
 checking  vacuity  00:00:01
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_green_no_waiting_first
 checking  property  00:00:01
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_gyr_oh_first  checking
 property  00:00:01
```

Feedback

```
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_gyr_oh_main  checking
 property  00:00:01
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_hazard_in_first
 checking  vacuity  00:00:01
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_hazard_in_first
 checking  property  00:00:01
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_hazard_in_main  checking
  vacuity  00:00:01

…
…
…
[Info] PROP_I_RESULT: FPV
 traffic.chk.assert_signal_sequencing_on_main  e2  proven  property
 00:00:02
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_hazard_in_first  e2
 proven  property  00:00:02
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_hazard_in_main  e2
 proven  property  00:00:02
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_no_both_green  e2
 proven  property  00:00:02
[Info] PROP_I_RESULT: FPV  traffic.chk.assert_no_both_yellow  e2
 proven  property  00:00:02
[Info] PROP_I_RESULT: FPV
 traffic.chk.cov_green_without_waiting_on_first  e2  uncoverable
 property  00:00:02
```

- When the `fml_enable_verbose_depth` formal variable is set to true in the Tcl file, the timestamp is reported in the PROP_I_RESULT messages from `check_fv`.

  ```
  set_fml_var fml_prop_result_timestamp true
  ```

  The following is an example output of the PROP_I_RESULT message when `fml_prop_result_timestamp` is set to true:

  ```
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.assert_hazard_in_first  e2  proven  property  00:00:02
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.cov_green_without_waiting_on_first  b8  depth=71
   property  00:00:02
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.assert_hazard_in_main  e2  proven  property  00:00:02
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.assert_no_both_green  e2  proven  property  00:00:02
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.assert_no_both_yellow  e2  proven  property  00:00:02
  [Info] PROP_I_RESULT: [2024-11-13 14:31:41]  FPV
   traffic.chk.cov_green_without_waiting_on_first  e2  uncoverable
   property  00:00:02
  ```

- When the `fml_enable_verbose_depth` formal variable is set to true, VC Formal displays extra run-time messages related to the bounded proof depths. Each time

Feedback

a greater depth is reached for a given property, a message is printed in the shell transcript, and logged in the message database. The property name, its new depth, and the elapsed wallclock time are displayed in the message.

```
set_fml_var fml_enable_verbose_depth true
```

The following is an example output of the PROP_I_RESULT message when `fml_enable_verbose_depth` is set to true:

```
[Info] PROP_I_RESULT: FPV traffic.main.oh_state_out b8 depth=63
 property 00:00:02
```

Note that the "traceon" and "traceoff" continues to work in cases where you want to pause the verbose screen reports during a run.

See section Controlling check_fv Runs for how to control a formal run.

17.2.4 Analyzing Results

This section covers the below topics:

- Analyzing Assertion Results
- Analyzing Cover Property Results
- Analyzing Assumption Results

After you have verified the properties, use the `report_fv` command to review the run status.

```
vcf> report_fv
Summary Results
   Property Summary: FPV
   -----------------
   > Assertion
     - # found         : 14
     - # proven        : 10
     - # falsified     : 3
     - # vacuous       : 1
> Vacuity
     - # found         : 10
     - # non_vacuous   : 9
     - # vacuous       : 1
> Cover
     - # found         : 6
     - # covered       : 5
     - # uncoverable   : 1
> Constraint
     - # found         : 3
```

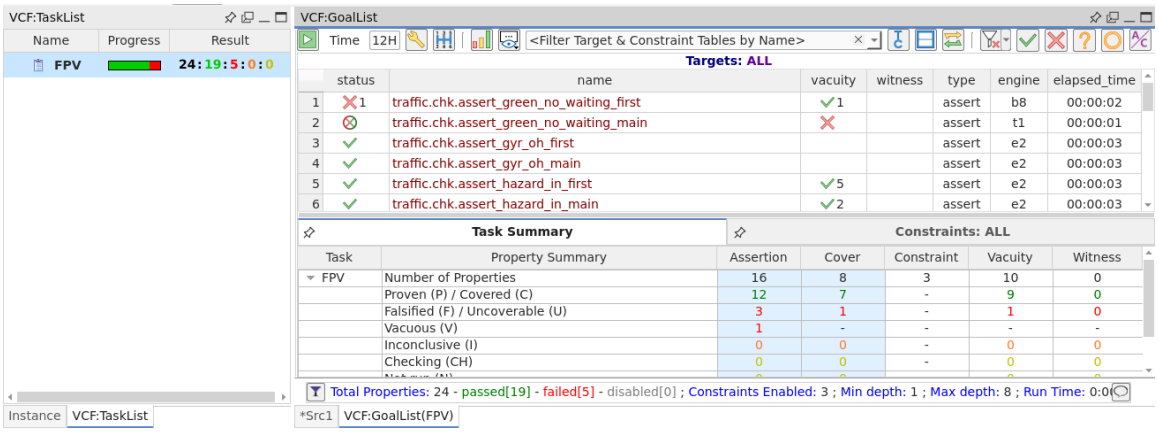
See section .

Feedback

If you have not launched the tool in GUI mode then to analyze a trace you first need to execute the `start_gui` command to invoke the Verdi GUI.

```
vcf> start_gui
```

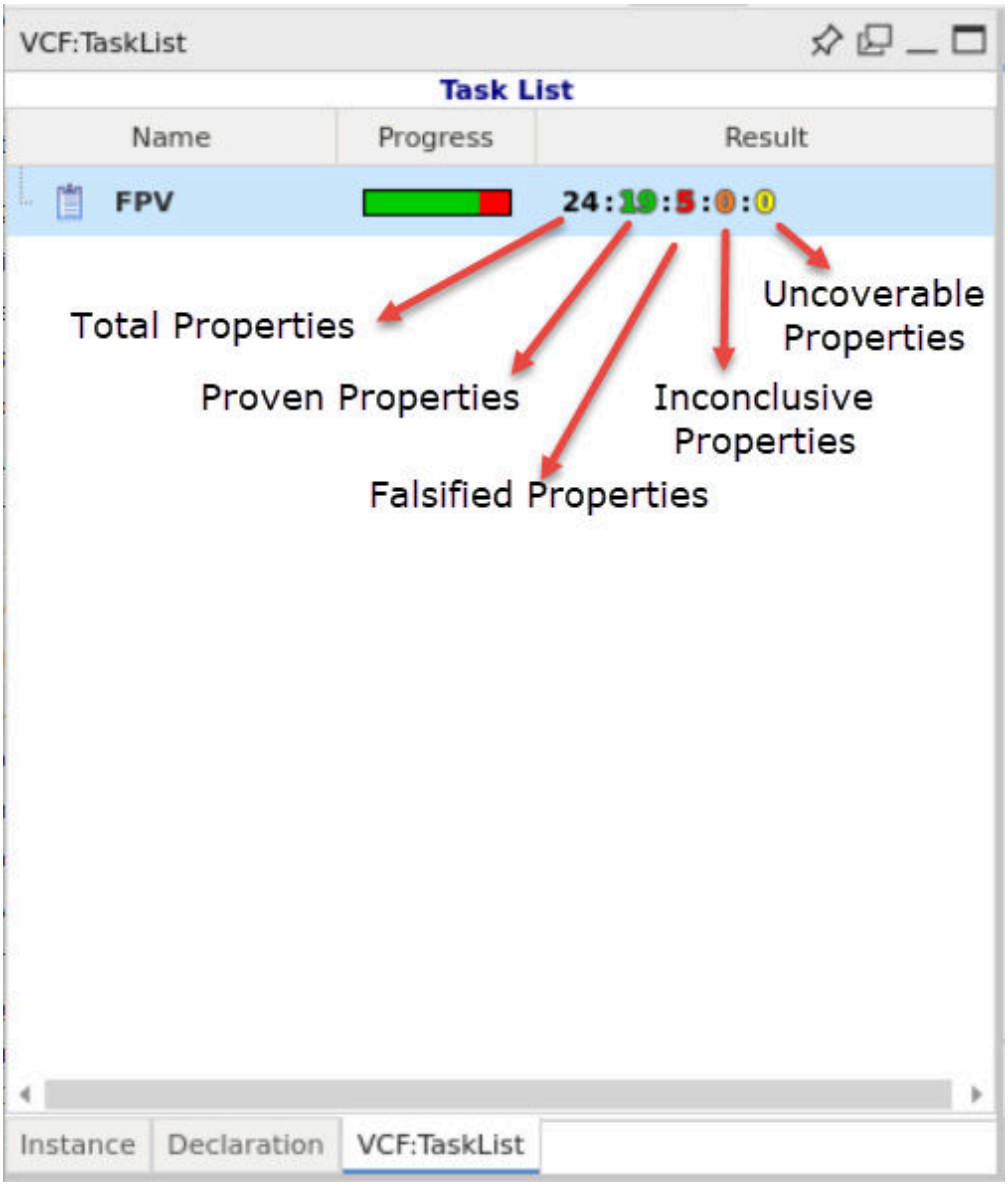
VC Formal Verdi GUI view will be launched:



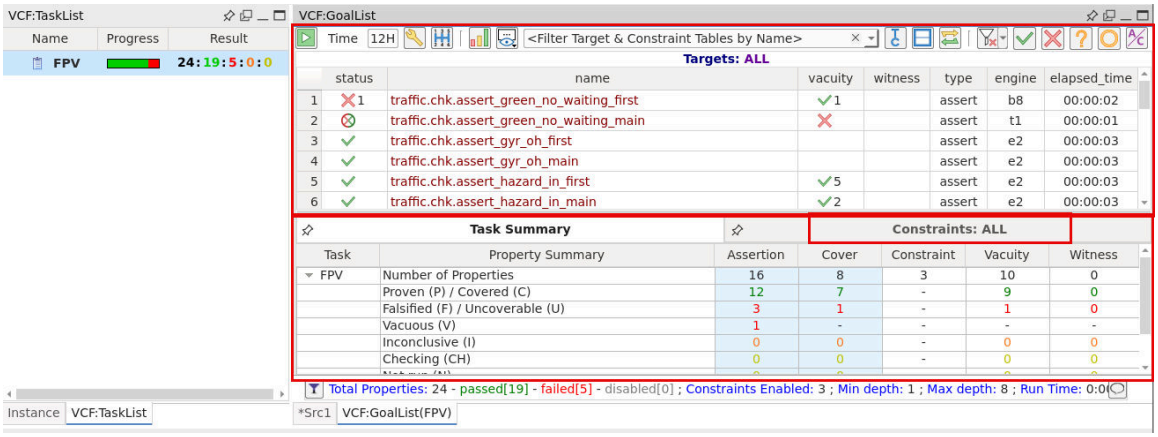
If the view is different from the above example, make sure the "VCF:TaskList" and "VCF:GoalList" tabs are selected.

The TaskList shows the proof status summary:

Feedback

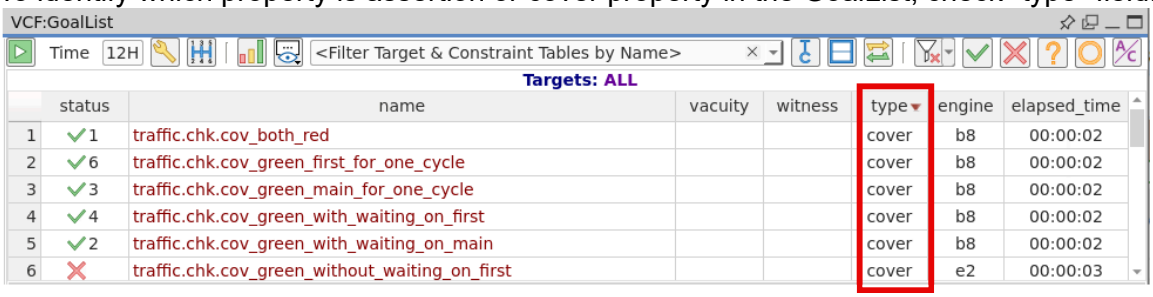


The GoalList consists of Verification Target pane which shows list of assertions and cover properties, and Constraints pane which shows assumptions in your testbench.



See section Using the GoalList for more details.

To identify which property is assertion or cover property in the GoalList, check "type" field.

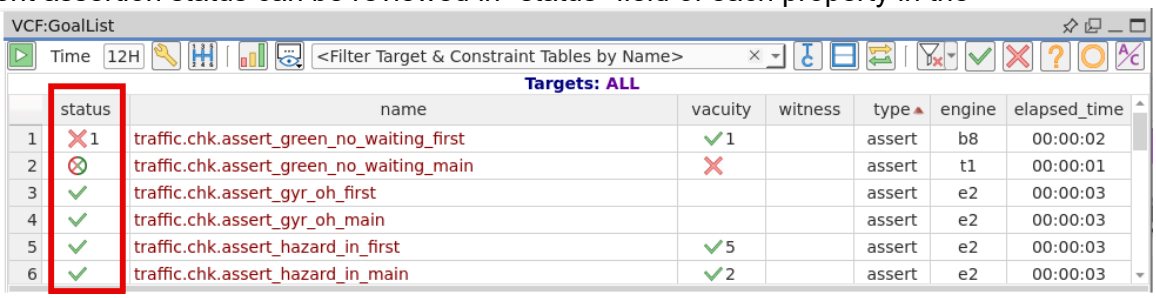


17.2.5 Analyzing Assertion Results

After running check_fv, assertion status can be one of PROVEN, VACUOUS, FALSIFIED and INCONCLUSIVE. See also for more information.

Feedback

The current assertion status can be reviewed in "status" field of each property in the GoalList:

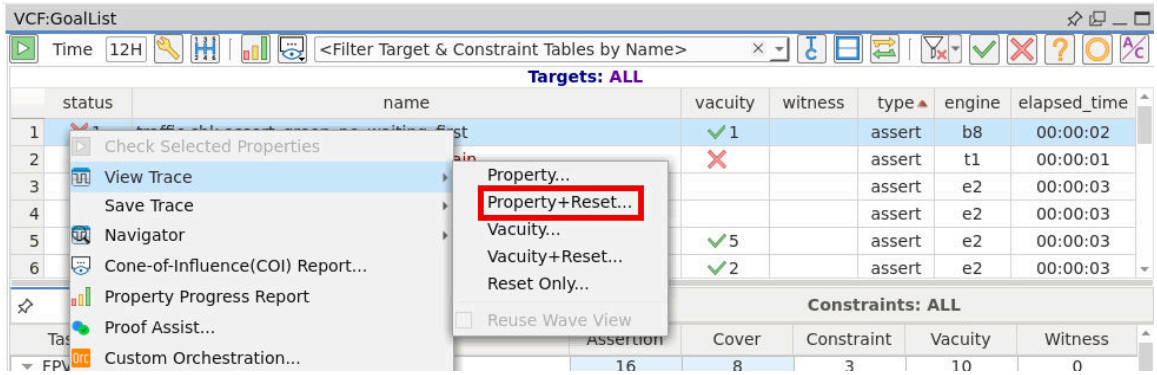


The meaning of each status icon is the following:

- FALSIFIED: Counter example found for the goal.
- VACUOUS: Goal has been proven vacuously. Vacuity Status for such cases would also appear red.
- PROVEN: Goal has been proven exhaustively.
- INCONCLUSIVE: Goal was not solved because resource limit was reached.

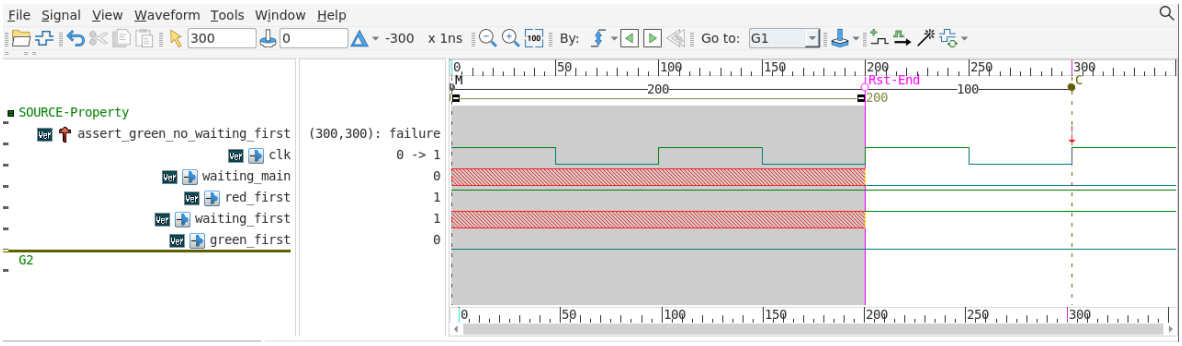
How to Analyze and Debug Falsified Assertion

A falsified waveform or counter example can be reviewed by double-clicking on or right-click on the assertion and choose *View Trace* > *Property+Reset*in the RMB menu as shown in the following figure:



Falsified trace will be displayed in the nWave as in below:

See section Debugging Results of Formal Verification for more details on how to use VC Formal Debug feature.



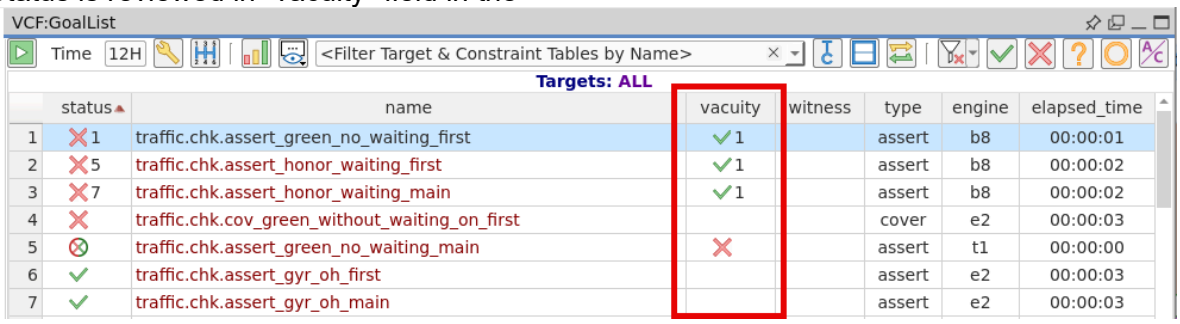
Note:

If you encounter an issue that signals change at wrong clock edge making it difficult to analyze, consider using the set_change_at command. See section Stabilizing Constraints in Multi-Clock Designs for details. This is useful even if your design is single-clock design.

You can use the fvtrace command to save the FSDB file. See section Exporting FSDB for Property Traces.

How to Analyze and Debug a Vacuous Assertion

Vacuity Status is reviewed in "vacuity" field in the GoalList:



The meaning of each vacuity status icon is the following:

- NONVACUOUS: Vacuity check was successful. Trace is available and it's 3-cycle length.
- VACUOUS: Vacuity check failed. No witness possible. Goal's primary status would show terminal status as VACUOUS.
- INCONCLUSIVE/NOTRUN: Vacuity check could not complete as the source limit was reached or the run was stopped by user. Or vacuity checking has not been tried.

See section for more information.

Feedback

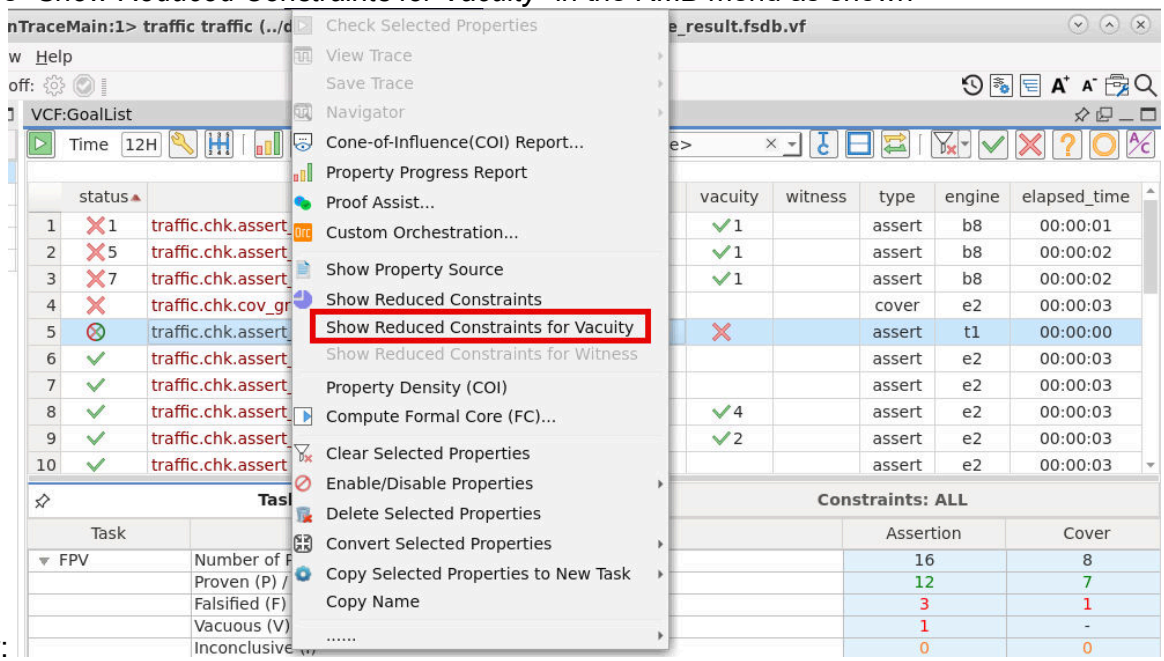
If the vacuous status of assertion is one of the three above, the assertion must consist of implication and have an antecedent. Otherwise, if the "vacuity" field is blank, there is no antecedent in the assertion and vacuity check is not applicable to the assertion.

The antecedent in SVA assertion definition can be reviewed in the source view. Double-clicking on the name of an assertion of interest in the GoalList will display its property definition in the same pane as the GoalList is located:

Select “VCF:GoalList” tab, if you wish to go back to the GoalList pane again.

The above example shows the antecedent of the assertion assert_green_no_waiting_main is uncoverable. It means no trace is available for this assertion.

To debug this vacuity check failure, use the `compute_reduced_constraints` command to see if any constraints might be over-constraining the antecedent. To execute this command, right-click on the name of the assertion then choose *“Show Reduced Constraints for Vacuity”* in the RMB menu as shown below:



The computation result will be shown in the terminal window in which you executed the vcf command for launching VC Formal:

vcf(compute_reduced_constraints[0][00:00:00])>

[Info] No constraints were used for traffic.chk.assert_green_no_waiting_main

Feedback

Otherwise if you invoked VC Formal GUI with `-gui` or `-verdi` switch in the `vcf` command, the message will be displayed in VCF Console pane as shown below:

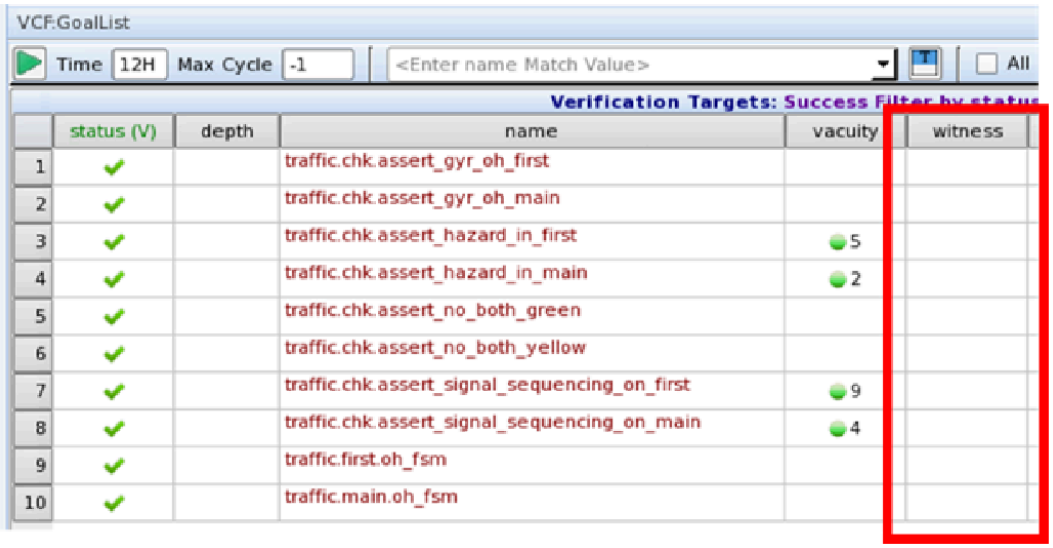
The above result indicates there are no assumptions in this testbench causing this vacuous status. Therefore, the design itself or the definition of the assertion must cause this issue. To look in more detail at the design constructs involved in this, the `compute_formal_core` command can be used. See section Formal Core (FC) Analysis.

For more information about the `compute_reduced_constraints` command, see section Identifying Reduced Constraints for Proofs.

How to Analyze a Proven Assertion

If an assertion is proven, you can still review a witness trace in nWave which shows one possible covered scenario examined of the assertion. If the assertion is defined by "A |=> B", then the witness trace shows a scenario at least once covering the sequence of "A ##1 B".

To check witness trace availability of each assertion, you can check "witness" field in the GoalList. If the `fml_witness_on` parameter is not set to true before the `check_fv` command run, the "witness" field is blank as in the following example:

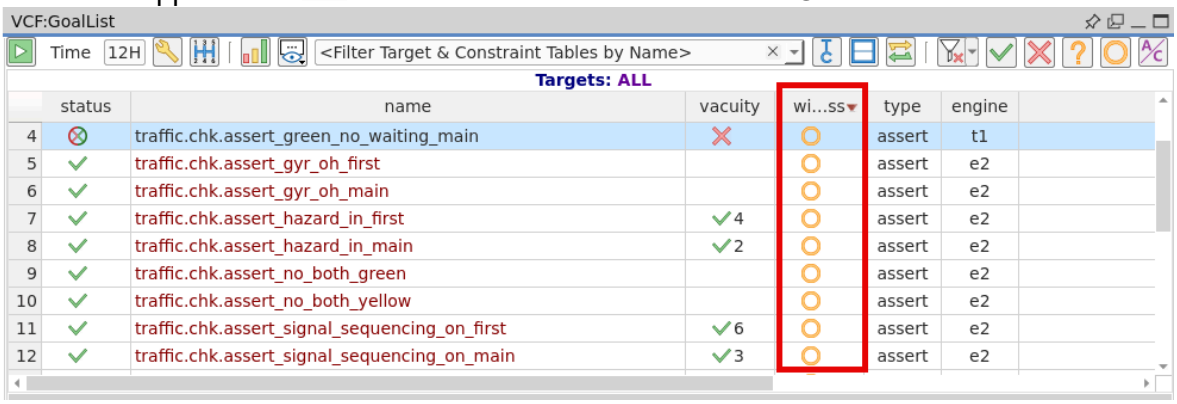


The `fml_witness_on` parameter is set to false by default, therefore you need to explicitly set it to true in your Tcl script, or execute the command in the terminal of VC Formal Console beforehand:

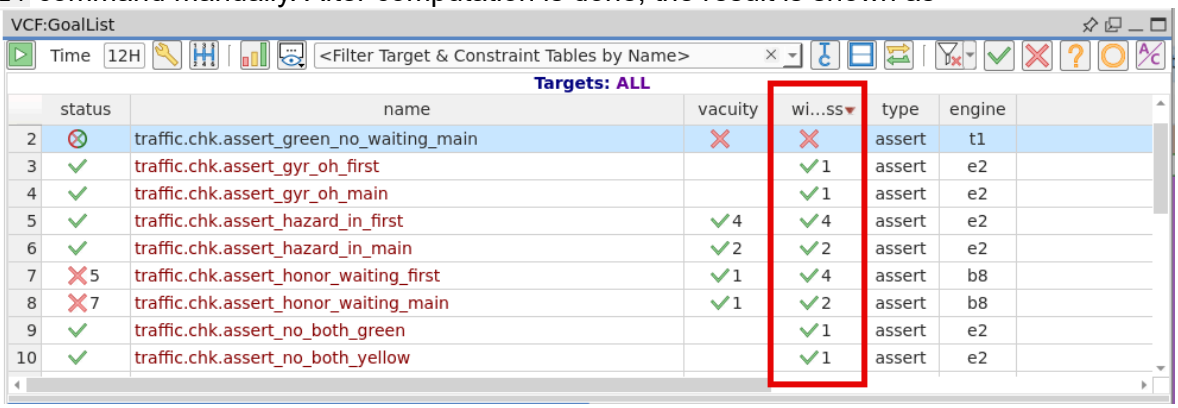
```
vcf> set_fml_var fml_witness_on true
```

Feedback

Now there appears the status icon in "witness" field in the GoalList:



You can execute the `check_fv` command to compute the witness status of the assertions. Click on at the left top in the GoalList, or you can also execute the `check_fv` command manually. After computation is done, the result is shown as follows:



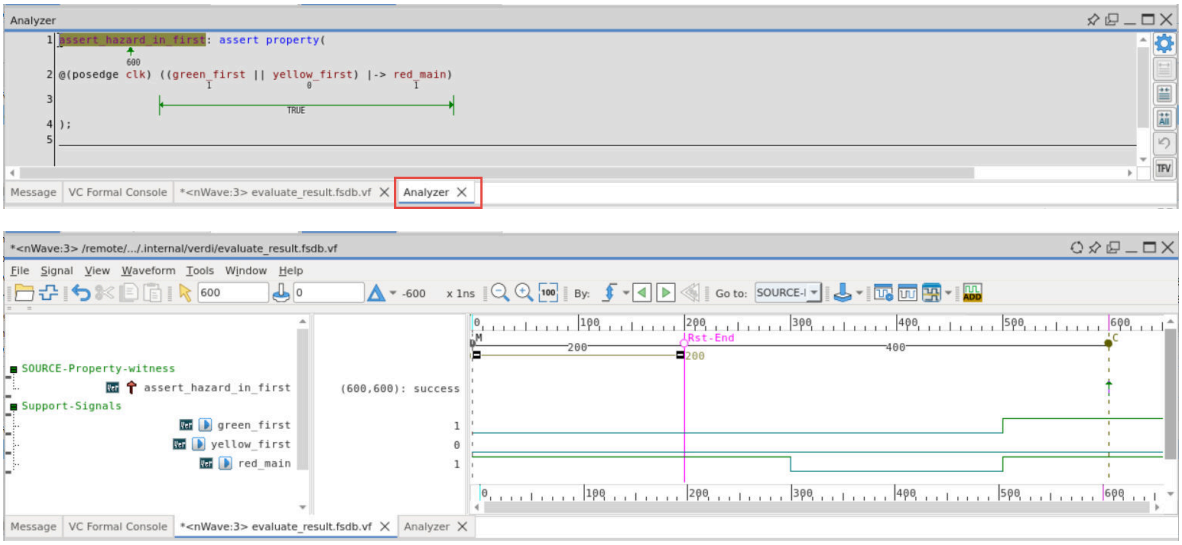
The meaning of each witness status icon is the following:

- 12 COVERED: Witness check successful. Trace available. Trace is available and it's 12-cycle length.
- UNCOVERABLE: No witness exists.
- 21 INCONCLUSIVE/NOTRUN: Witness check could not complete as the source limit was reached or the run was stopped by user. Or witness checking has not been tried.

See section for more information.

For example, you can double-click on the COVERED witness status icon of the assertion traffic.chk.assert_signal_sequencing_on_first to display its trace in nWave:

Feedback

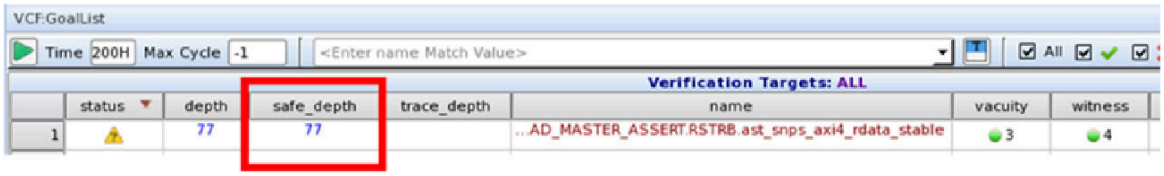


The analyzer pane shows the assertion definition and other useful information for analysis:

See section Using the GoalList for more information.

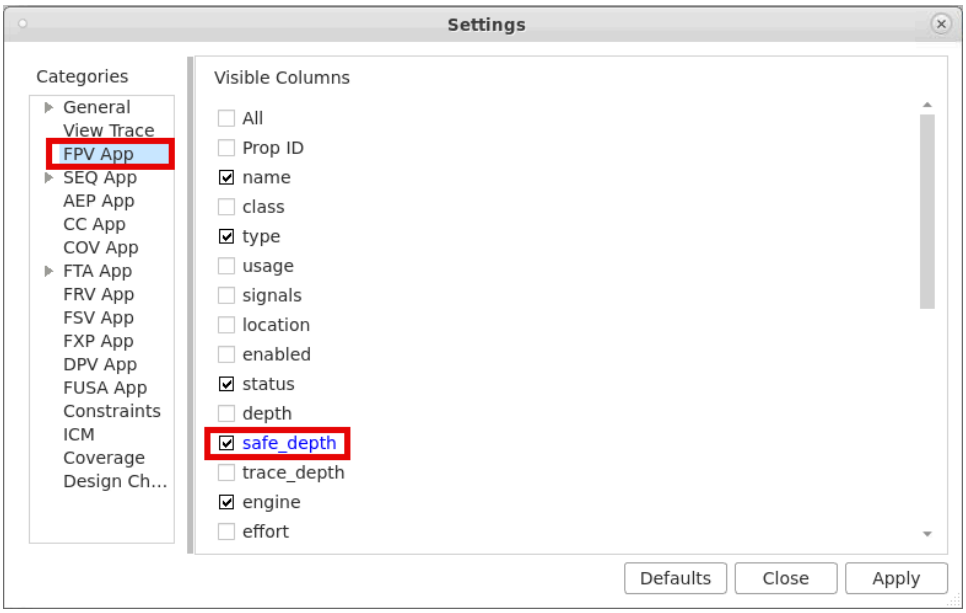
How to Analyze Inconclusive Assertion

If assertion is inconclusive after a long run, you still can check the value of the assertion's *safe_depth* to confirm that the assertion is proven for all possible scenarios up to the *safe_depth* cycles after reset. It's not a full proof, but it still gives you some level of confidence in what has been explored. The *safe_depth* can be reviewed in the *safe_depth* field in the GoalList:



Feedback

The safe_depth field is not shown by default. Therefore, you need to configure the view, by clicking on , then selecting *FPV app -> safe_depth -> Apply* in the RMB menu:



Another suggestion for dealing with an inconclusive assertion is to compute formal core of the assertion and analyze the result. It reports which part of the design and which assumptions are involved in the proof computation and gives you the insight of the reason of the complexity. With this information, you may be able to come up with another verification plan for a better chance of convergence. See section Formal Core (FC) Analysis for the details.

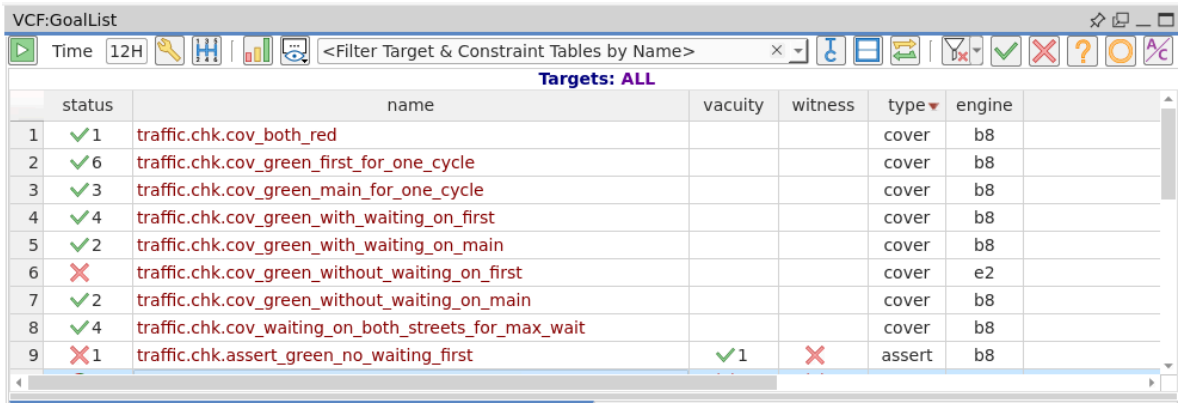
17.2.6 Analyzing Cover Property Results

This section covers the below topics:

After running `check_fv`, cover property status can be one of COVERED, UNCOVERABLE and INCONCLUSIVE. See section for more information.

Feedback

The current cover property status can be reviewed in "status" field of each property in GoalList:



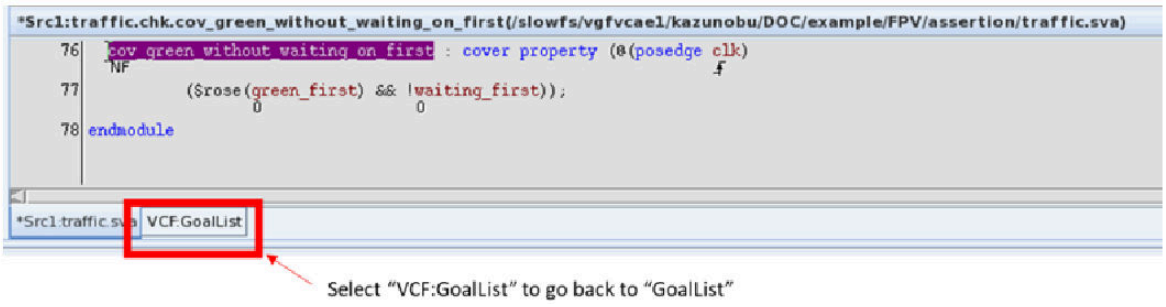
	status	name	vacuity	witness	type	engine
1	✓1	traffic.chk.cov_both_red			cover	b8
2	✓6	traffic.chk.cov_green_first_for_one_cycle			cover	b8
3	✓3	traffic.chk.cov_green_main_for_one_cycle			cover	b8
4	✓4	traffic.chk.cov_green_with_waiting_on_first			cover	b8
5	✓2	traffic.chk.cov_green_with_waiting_on_main			cover	b8
6	✗	traffic.chk.cov_green_without_waiting_on_first			cover	e2
7	✓2	traffic.chk.cov_green_without_waiting_on_main			cover	b8
8	✓4	traffic.chk.cov_waiting_on_both_streets_for_max_wait			cover	b8
9	✗1	traffic.chk.assert_green_no_waiting_first	✓1	✗	assert	b8

The meaning of each status icon is the following:

- ✗ UNCOVERABLE: No trace exists that reaches the coverage goal.
- ✓ COVERED: Trace has been found for the coverage goal.
- ? INCONCLUSIVE: Goal was not solved because resource limit was reached.

How to Analyze and Debug Uncoverable Cover Properties

The SVA cover property definition can be reviewed in the source view. Double-click on the name of cover property with uncoverable state in the GoalList, then its property definition is shown:

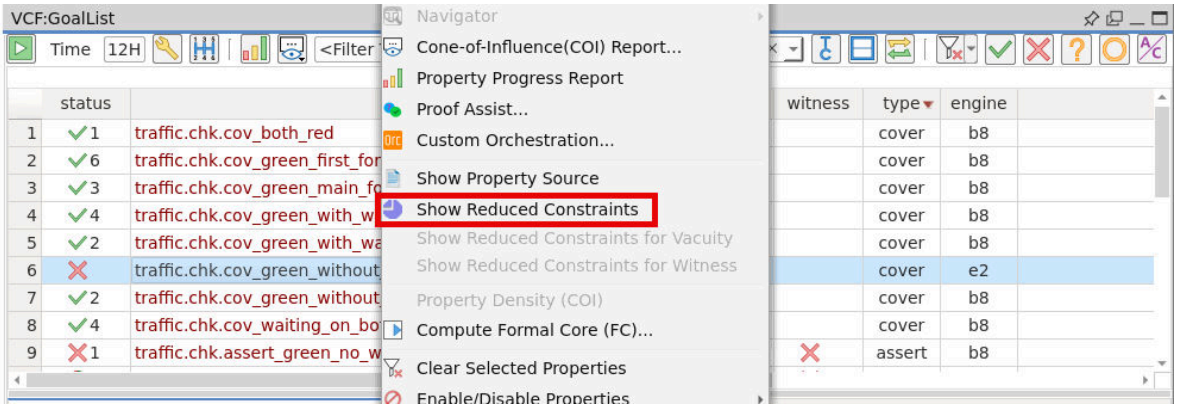


Because the cover property cov_green_without_waiting_on_first is marked as uncoverable, the sequence of "$rose(green_first)&&!waiting_first" is now confirmed to be uncoverable.

The reason for the uncoverable result could be that the testbench is over-constrained, the cover property is wrongly defined, the design has a bug which is causing an unreachable status, or some of or all these combined.

Feedback

Now that you can use the `compute_reduced_constraints` command to find the root cause of this uncoverable status. The command can be executed in the Verdi GUI. You can right-click on the name of the cover property then choose *"..." -> "Show Reduced Constraints"* in the RMB menu:



The computation result will be shown in the terminal window in which you executed the vcf command:

```
vcf(compute_reduced_constraints[0][00:00:00])>
```

[Info] Reduced constraints for traffic.chk.cov_green_without_waiting_on_first:
traffic.chk.assume_continuous_waiting_main traffic.chk.assume_continuous_waiting_first

Otherwise if you invoked VC Formal GUI with `-gui` or `-verdi` switch in the `vcf` command, the message will be displayed in VCF Console pane as shown in below:

The name of the assumptions in blue reported in the VC Formal Console is also a link to its corresponding property definition in the source view. By clicking on the name, the cover definition can be easily reviewed:

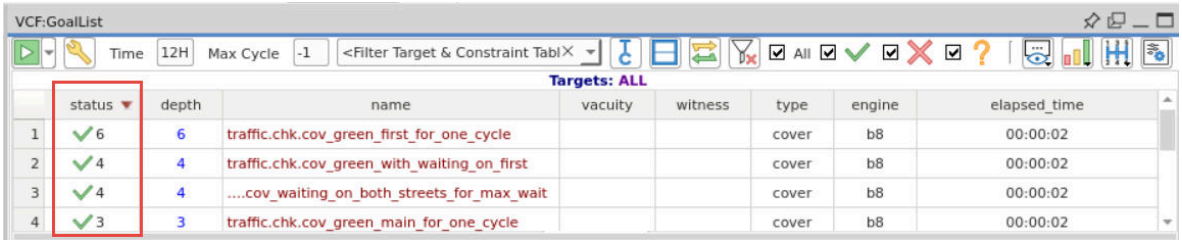
It helps understanding the assumptions involved in the uncoverable cover property.

Feedback

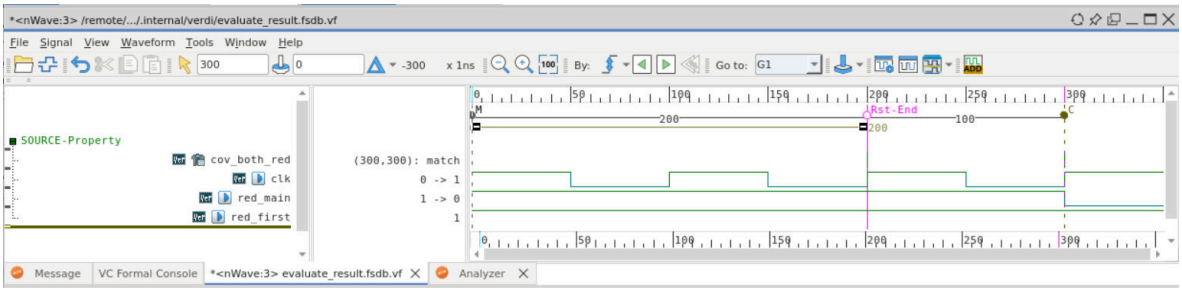
Also, for further investigation, you can use the `fvdisable` command on assumption(s) and execute the `check_fv` command to see how the status will change without them. See section Changing Property Usage for more information about the `fvdisable` and `fvenable` commands.

How to Analyze Covered Cover Properties

Similar to analyzing witness trace for proven assertion, it's important to review a covered trace. A cover trace can be reviewed only when the status of the cover property is COVERED.



Double-clicking on the COVERED status icon shows a covered trace in nWave:



Double-clicking on the name of cover property shows its property definition in the source view for review:

17.2.7 Analyzing Assumption Results

How to Analyze and Debug Vacuous and Uncoverable Witness of Assumptions

Similar to analyzing assertions, checking vacuity and witness statuses of assumptions is important to understand the verification environment and result. If any of them are unreachable, it's an indication that the testbench may be over-constrained, the assumption

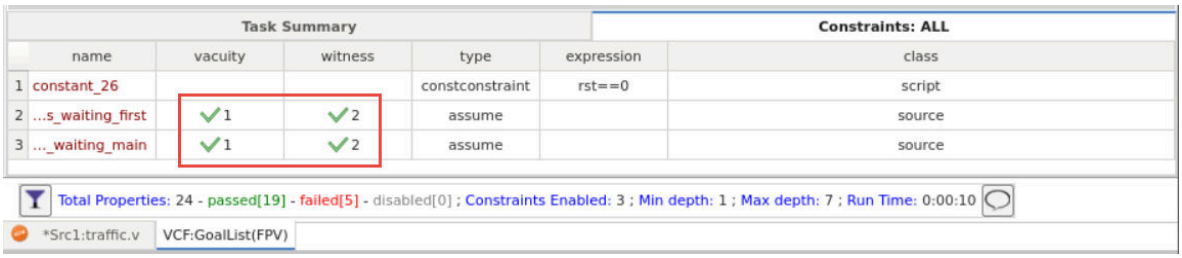
Feedback

may be wrongly defined, the design has a bug which is causing an unreachable status, or some of or all these combined.

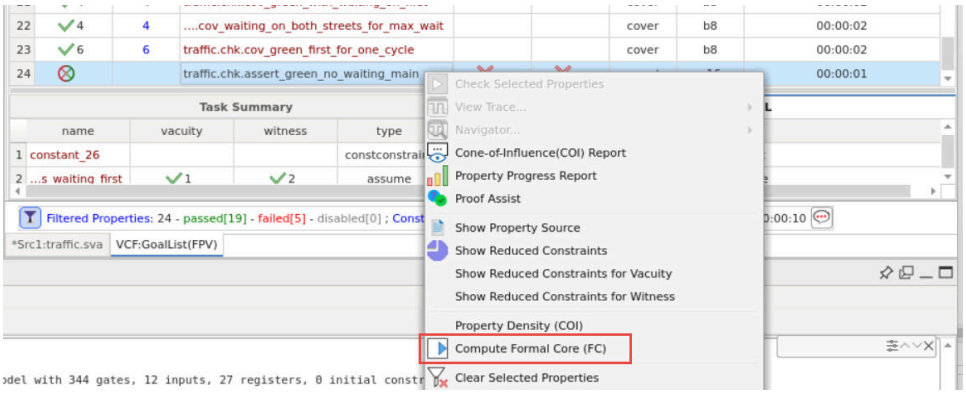
After running the check_fv command, the status of vacuity and witness can be one of NONVACUOUS/COVERED, VACUOUS/UNCOVERABLE and INCONCLUSIVE. See also for more information.

To analyze the witness and vacuity statuses of assumptions, the parameters fml_witness_on and fml_vacuity_on must be both set to true. See Viewing Witness Traces and Checking Vacuity Goals for more details.

Vacuity and witness statuses of assumptions can be reviewed in the Constraints table in the GoalList:

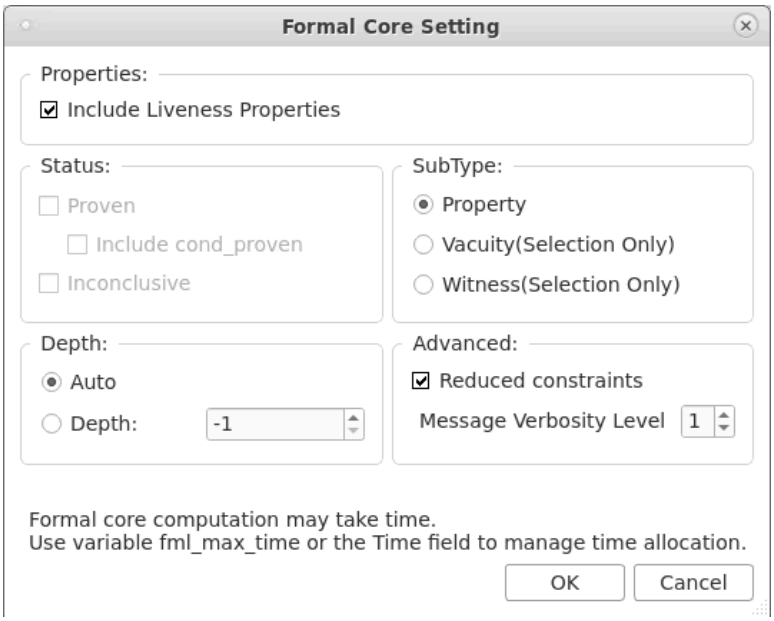


If vacuity status is vacuous or witness is unreachable, use compute_formal_core to analyze which assumptions may be causing the issue. To do that, right-click on the property with vacuous or uncovered witness status, then choose "Compute Formal Core" in the RMB menu:

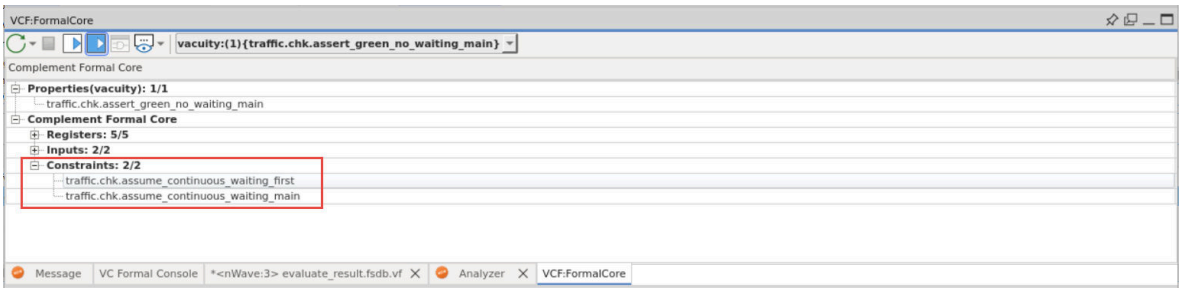


Feedback

Choose "Vacuity" or "Witness" and click on "OK" in the popup menu as in below:



Now the tool computes Formal Core which also lists up the constraints involved in the vacuity/uncovered witness status under Constraints:



Assumption(s) listed under Constraints could be a root cause of vacuity or uncoverable witness status. See section Formal Core (FC) Analysis for more information about Formal Core.

How to Analyze and Debug Conflicting Constraints

See section Debugging Conflicting Constraints for dead-end debugging

How to Reduce Constraints for Optimizing Proofs

See section Identifying Reduced Constraints for Proofs for proof optimization.

17.3 Property Control

This section covers the below topics:

- Adding Script Property
- Disabling and Enabling Property
- Changing Property Attributes

17.3.1 Adding Script Property

There are commands available to add script properties. Use the fvassert, fvassume, fvcover commands to add script assertion, assumption and cover property, respectively. See section Script Properties and Constraint Examples for more information.

17.3.2 Disabling and Enabling Property

To disable or enable a property in the GoalList, use the fvdisable command or the fvenable command, respectively. See Source Properties for more information

17.3.3 Changing Property Attributes

There are also commands available to change the attribute of a property among assumption, assertion and cover. See Source Properties for more information.

17.4 Formal Runtime Control

17.4.1 Controlling Formal Run

The `check_fv` command accepts various options for controlling a run. See section Performing and Configuring VC Formal Checks for details. Some useful options are:

```
check_fv -block
check_fv -property  <list-or-collection-of-properties>
check_fv -run_finish <cmds, ex. Report_fv>
```

17.4.2 Controlling Stop

You can specify when to stop proof run based on the time limit specified, the number of falsifications, and/or the status of vacuity and witness of given properties.

Feedback

Stop on Time Limit

You can control when to stop a formal run by specifying maximum run time. The command and parameters used for this control are:

```
set_fml_var fml_max_time <time, ex. 24H>
set_fml_var fml_progress_time_limit <time, ex. 100M>
```

See section Setting Limits for the check_fv Run and Specifying Maximum Progress Time Limit for more information.

Stop on Number of Falsifications

You can also configure to stop a current run when the number of falsifications reaches the maximum limit specified. The commands and option used for this control are:

```
check_fv -break <# of falsification(s)>
check_fv_setup -break <# of falsification(s)>
```

See section Performing VC Formal Checks and Running Formal Design Checks for more information.

Stop on Vacuous or Uncoverable Witness

You can also configure to stop a current run when a vacuous or uncoverable witness is found. The command and option used for this control are:

```
check_confing -stop_on <stop_condition>
```

See section Stopping the Run When Constraints are Bad for more information.

17.4.3 Controlling Resume

Resuming runs from where it previously stopped is a useful option and VC Formal supports this feature. Command and parameters needed for this control are:

```
set_fml_var fml_enable_resume true
set_fml_var fml_enable_resume_depth true
```

See section Improving Performance of Subsequent check_fv runs and Resuming check_fv from the Last Known Maximum Bounded Depth for more information.

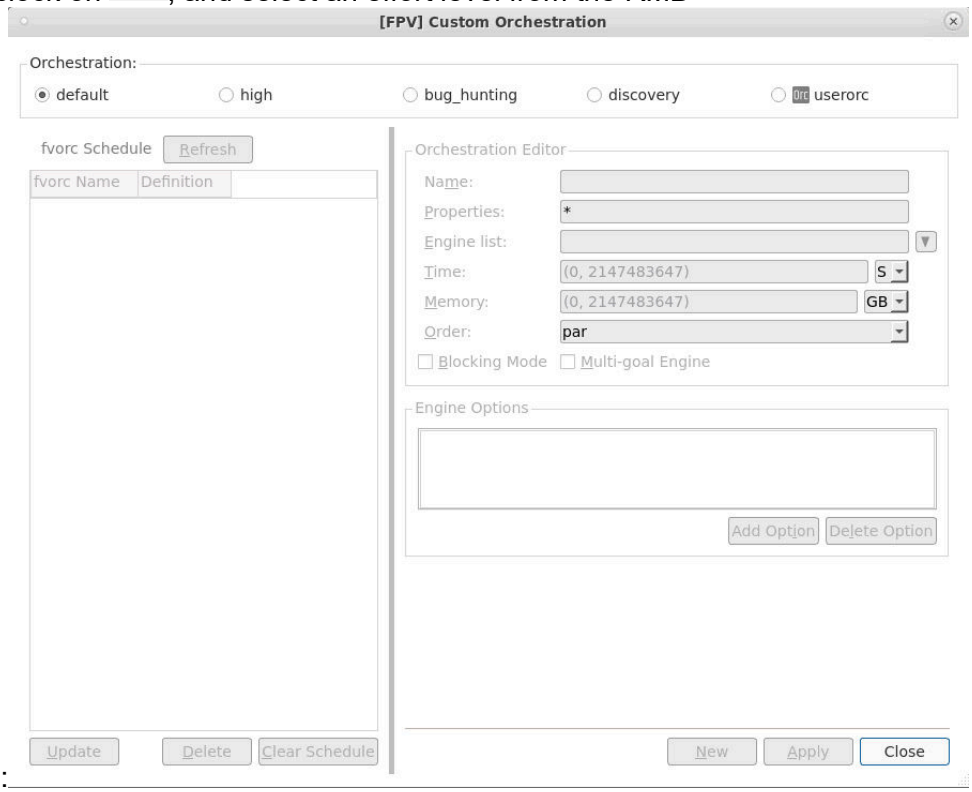
17.4.4 Controlling Engine Effort

You can control engine recipes and effort. The command and parameter used for this control are:

```
set_engine [-on|-off] <engine id>
set_fml_var fml_effort <effort level, ex. high>
```

Feedback

Engine effort can be also selected in the Verdi GUI. You can right-clock on , and select an effort level from the RMB menu:



See Orchestration in VC Formal for specifying engine effort levels or the type of effort required instead of specifying the recipes for more information.

17.5 Proof Acceleration

This section covers the below topics:

- Controlling Resources for a Run
- Using Machine Learning
- Using Proven Assertions as Assumptions
- Turning off Trace Generation

17.5.1 Controlling Resources for a Run

You can also specify memory and grid usages for the check_fv command.

Controlling Memory Usage

To set a maximum memory limit, see section Setting Limits for the check_fv Run for how to control max memory usage per auxiliary worker process. The command and parameter used for this control are:

```
set_fml_var fml_max_mem <Maximum memory size>
```

Controlling Grid Usage

To set a grid resource usage, see section Running on Compute Farm for how to launch multiple workers. The commands and parameters used for this control are:

```
set_grid_usage -type [LSF|SGE|RTDA]=<#_of_workers> …
report_grid_usage
```

17.5.2 Using Machine Learning

You can enable VC Formal to learn information about falsifications and proofs and leverage the learnings for the subsequent check_fv runs. The command and parameter used for this control are:

fvlearn_config -local_dir <dir_path> …

See section VC Formal Regression Mode Accelerator for the details.

17.5.3 Using Proven Assertion as Assumption

When using proven assertions as assumption, it could enhance performance depending on the current environment. The commands and parameters used for this control are:

```
check_fv -assume
set_fml_var fml_enable_assume_proof true
```

See Performing VC Formal Checks and Using Proven Assertions for Converging on Subsequent Assertions for more information.

Note:

It is not always guaranteed to improve the performance by using proven assertions as assumption. In fact, there is also a chance to degrade the computation performance. It's recommended to run a proof without using this option at first then try using it when further improvement is not expected.

Feedback

17.5.4 Turning off Trace Generation

When executing the `check_fv` command without saving trace information, the performance can be enhanced. The command and parameter used for this control are:

```
set_fml_var fml_no_trace true
```

Note:

The disadvantage of using this option is that creating a trace is not available after executing the check_fv command. Therefore, you should rerun the check_fv on properties of interest after the fml_no_trace parameter being reset to false in order to create a trace of the properties.

17.6 Convergence Improvement

One of the biggest challenges when applying Formal Verification is convergence issues. The effective solution depends on design and properties, and there's no standard solution that works for all cases.

For more information on convergence improvements, see section Considerations for Convergence Improvement for the details.

Determining the type of method to be applied can be done through deep understanding of the RTL, including diagnosing:

- The overall complexities of the design or the COI of the property using Complexity Report (see section Performing Complexity Analysis in Tcl).
- Engine performance using Proof Assist (see section Support for Proof Assist).
- Induction convergence though ICM (see section Iterative Convergence Methodology), or other means.
- Applying compositional verification using Proof Architect (see section Compositional Verification Using Proof Architect) for more details.

The commands used for abstractions are:

- `snip_driver`
- `set_blackbox`
- `set_abstractions`
- `get_abstractions`
- `report_abstraction`

18

Considerations for Convergence Improvement

This section contains the following topics:

- Convergence Challenges When Applying Formal Verification
- Performing Complexity Analysis in Tcl
- Viewing Complexity Report (COI) in VC Formal GUI
- Using Automatic Abstraction To Reduce Complexity
- Reviewing the Formal Core of Property
- Support for Proof Assist
- Iterative Convergence Methodology
- Using Proof Architect for Faster Convergence

18.1 Convergence Challenges When Applying Formal Verification

One of the biggest challenges when applying formal verification are convergence issues. The effective solution depends on design and properties, and there is no standard solution that works for all cases. This chapter describes some guidelines to improve convergence and VC Formal features to address convergence improvement.

In general, the solutions for convergence improvement can be categorized as follows.

- Re-write assertions.
- Tune Formal engines: Change Effort Level.
- Abstraction: Reduce design complexity.
- Utilize invariants.

18.1.1 Re-write Assertions

The following are some well know techniques for re-writing assertions:

- Case Splitting
- Property Decomposition
- Symbolic Variables

These are independent from VC Formal and out of scope of this document. You can contact the Synopsys support team for the training and queries on these techniques.

18.1.2 Tune Formal Engines

One solution to improve convergence is to optimize Formal engine selection. However, it is not easy to select the most appropriate engines based on design information alone, since there are many factors to consider. For example, which engine or combination of engines to run and for how long, when to switch between engines, what engine options to apply, and so on.

VC Formal runs powerful engine orchestrations packaged as effort levels, and provides good out-of-the-box performance. The `default` effort level is being continuously improved, and provides good results in many cases, therefore, it is recommended that you start with the `default` effort level.

In case, the `default` effort level cannot solve all properties, select other effort levels based on the intent. For example, use the `bug_hunting` effort level to focus on finding additional counter examples.

The following are the recommended effort levels for FPV:

- Use `default` effort level, by default, as it provides good out-of-the-box performance.
- Use `bug_hunting` effort level, if you are looking for deep sequential bugs.
- Use `Discovery` effort level, if only a few properties are inconclusive and many workers are available. (Not commonly used and is scheduled for deprecation.)
- Use `High` effort level when there are inconclusive properties with default effort level. (Not commonly used and is scheduled for deprecation.)

Specifying Effort levels (for FPV App):

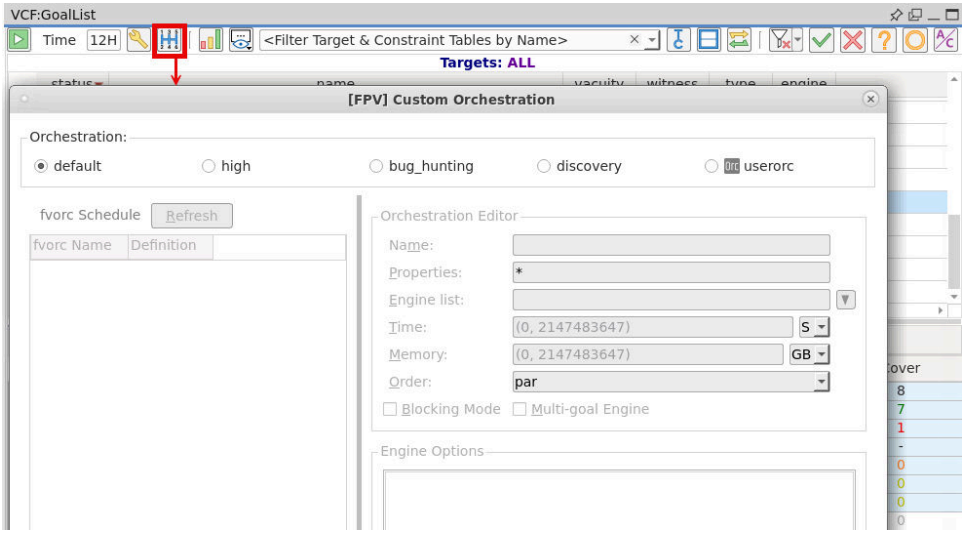
To specify the effort level in the command line or in Tcl, use formal variable `fml_effort`:

```
set_fml_var fml_effort [default|high|bug_hunting|discovery]
```

You can also specify the effort level in the GUI directly as shown in Figure 134.

Feedback

Figure 134: Changing Effort Level



Refer to section Configuring Orchestration in VC Formal for more information on effort levels, and available options for each VC Formal App.

For custom engines, user-defined orchestrations (or recipes) can be created with engines that worked well for already converged properties or for specialized engines using effort level `userorc` and the `fvorc` command. Refer to sections Configuring Orchestration in VC Formal and Defining User Orchestration for VC Formal Runs for more information.

Prepackaged engine orchestrations are continuously improved with a special focus on `default` and `bug_hunting` effort levels. If changing the effort level or using custom user-defined orchestrations did not help with convergence, you can try the version of the effort orchestration currently under development to see if it helps getting better results.

- To enable the version of the `default` orchestration under development for FPV, use:

```
set_fml_var fml_effort default
set_fml_var fpv_flow_version v8 (default: v7)
```

- To enable the version of the `bug_hunting` orchestration under development for FPV, use:

```
set_fml_var fml_effort bug_hunting
set_fml_var fpv_bh_flow_version v4 (default: v3)
```

- To enable the version of the `default` orchestration under development for SEQ, use:

```
set_fml_var fml_effort default
set_fml_var seq_flow_version v5 (default: v4)
```

Note:

Further changes can be made to those versions under development before they are made default, and results can change across VC Formal versions. The default version can also be the newest version available until a new version under development gets added.

18.1.3 Abstraction

The convergence issues are mostly caused due to design and Formal testbench complexity. To reduce Formal testbench complexity, it requires to re-write properties and optimize modeling logic in Formal testbench. How to reduce Formal testbench complexity is out of scope of this document, but we provide the training and consulting. This section describes abstractions to reduce design complexity.

Abstraction doesn't modify design at all, it replaces part of design with simpler model in Formal model used under the hood of tool. There are two important factors in abstraction. The first factor is to identify where complexity exists, in other words, which part of design to be abstracted. The second factor is how to abstract the identified part, in another words, what kind of abstraction to be applied.

The abstraction techniques discussed below will include:

- Automatic abstraction
- Using VC Formal abstraction models
- Abstract initial state
- Abstract logic elements

Identify which part of design to be abstracted.

There are several methods to identify complexed logic as follows.

- Formal Complexity Report
- Formal Core
- Bounded coverage analysis

The first step is to use Formal Complexity Report. This provides not only overall complexity of design but also complexity for COI of target property. See section Viewing Complexity Report (COI) in VC Formal GUI.

Formal Core provides one of metric used in Formal Coverage, but it also provides lots of information related to complexity.

For example, let us assume the target property is inconclusive at depth 24. If user applies Formal Core for the target property specifying depth=24, it provides the information

which registers affect to target property and which register don't. It is likely, though not guaranteed, that there are a number of registers within the COI of the target property but outside the formal core. These may contribute to complexity. Of course, many registers within COI but out of Formal Core may be irrelevant to target property. But it would be worth to investigate registers within COI but out of Formal Core as abstraction candidates. One useful trial is to snip all registers within COI but out of Formal Core, then run proof. If it provides proven, it's done! If it provides falsified, user would have idea which snipped register causes false failure. It also provides hints what kind of abstraction to be applied. If it still doesn't converge, the complexity may exist inside of Formal Core. As mentioned above, applying Formal Core provides lots of hints for abstraction. See section Formal Core (FC) Analysis.

Bounded coverage analysis also provides valuable information for convergence. For example, let assume the target property is inconclusive at depth 24. The result of bounded coverage analysis provides the information which lines of code are not reachable with depth=24. It doesn't guarantee but RTL logic with reachable depth more than inconclusive property depth could be candidates for abstraction.

Abstraction Techniques

There are many abstraction techniques and those are categorized such as manual abstraction, automatic abstraction and abstraction models.

The simplest abstraction is to set module or instance as blackbox and snip signals. These reduce design complexity and contribute in convergence but it may cause false failure results. Output signals of blacboxed instances and modules are treated in the same way as primary inputs and can take any value. Snipped signal value can be any value. User may want to specify simple constraints for output signals of blackboxed module or instance and snipped signals. In this case, it is important to ensure that added constraints are valid by proving those constraints as assertion.

Manual abstraction is out of scope of this document, but automatic abstraction is useful method and relatively easy to apply. See section Using Automatic Abstraction To Reduce Complexity for automatic abstraction available in VC Formal.

If design contains FIFO or memory, it most likely increases complexity. VC Formal provides abstraction models for simple memories and FIFOs. However, usage of abstraction models is case by case and it's tightly related to the assertions in most cases. The abstraction models reduce complexity by storing only limited information necessary for assertion.

For example, consider the case that formal optimized assertion checks only an arbitrarily selected packet at a time. In such case, if assertion informs which packet is selected as target against abstraction model, then the abstraction model can store incoming packet only when the selected packet is seen in input port. And abstraction model can drive stored value only at the timing the selected packet should be seen in output port. In

another words, there should be communication between the assertion and abstraction model.

See *"$VC_STATIC_HOME/packages/formal_sva_lib/fml_abstracted_models/ram_and_fifo/doc/Formal_RAM_FIFO_Model.pdf*for more detail of the abstraction models available in VC Formal.

The abstraction models locate under the directory "$VC_STATIC_HOME/packages/formal_sva_lib/fml_abstracted_models/ram_and_fifo/sva".

Another type of abstraction is called initial state abstraction or reset value abstraction where a register value after reset simulation is set to X (uninitialized state) or to a different value. This technique is particularly useful for counters and FSMs where setting their initial state to X is a way to reach deeper states faster. See sections Specifying the Initial State and Viewing Complexity Report (COI) in VC Formal GUI for methods to apply initial state abstraction with the `sim_set_state` command or directly through the GUI.

18.1.4 Utilize Invariants

Invariants are a powerful technique to improve convergence. However, it's not easy to find out appropriate invariants and write effective helper assertions since it depends on design and properties. Utilizing Invariants is ultimate and final solution to get convergence, but even advanced users spend lots of time to find invariants and write helper assertions. There was strong demand to provide functionality to help this process. VC Formal provides powerful functionality called Iterative Convergence Method (ICM) to find invariants and help writing helper assertions. See section Iterative Convergence Methodology for details of ICM.

18.2 Performing Complexity Analysis in Tcl

When an assertion does not converge, that is, it is neither proven nor falsified within a set time, the assertion complexity might be the cause for long run times and/or very high memory utilization. To fix assertions that do not converge, you must first determine the Cone of Influence (COI) of the property and its complexity.

Use the following two commands to analyze the Cone of Influence (COI) of a property:

- `report_fv_complexity`
- `compare_coi`

Feedback

18.2.1 The report_fv_complexity Command

The `report_fv_complexity` command generates information about the COI for a single or a set of nets or properties. For a set of properties or nets, the union of COI objects for each net or property is reported.

Syntax

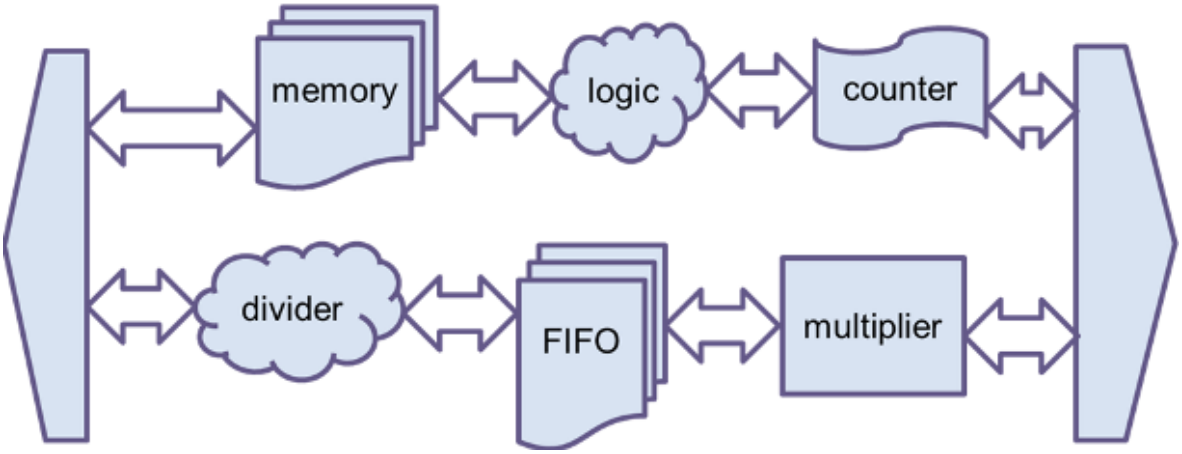
```
vcf> report_fv_complexity -help
Usage: report_fv_complexity    # Reports complexity of the COI of a list
 of nets or properties, or complexity of a design instance
        [-stopif <expression>] (Stop if object matches property
                                expression)
        [-stops <collection-or-list-of-nets-or-ports-or-instances>]
                               (Specify the collection of
                                nets/ports/instances on which to stop
                                COI exploration (region definition))
        [-list]                (Report objects in a list format)
        [-verbose]             (Report objects in a list format including
                                additional information like width and
                                source location)
        [-col]                 (Return list of object as a collection)
        [-limit <integer>]     (Limit the number of items to be listed,
                                works only with -list, -verbose or -col
                                option. By default, tool reports 100
                                items, use '-limit -1' to report all
                                items)
        [-level <integer>]     (Generate Complexity report till the
                                given level in the hierarchy. This
                                switch is valid only for the iteration
                                case and fails if start point(s)
                                are given)
        [-xml <file_name>]     (Generates Complexity report in XML format
                                in the given file)
        [-filter <expression>] (Filter objects matching expression)
        [-regexp]              (Use regular expressions for pattern
                                matching operators =~ and !~)
        [-nocase]              (Case insensitive pattern matching for =~
                                and !~ operators)
        [-bit]                 (Report bit level COI (more precise
                                but slower) instead of word level COI
                               (less precise but faster). By default,
                                tool reports word level COI)
        [-disable_abstraction] (Disable abstraction of complex operators
                               (FIFO, MEM, COUNTER...))
        [-design]              (Report design objects with -list
                                or -verbose (on by default))
        [-instrumented]        (Report tool instrumented objects)
        [-gui]                 (Show report in Verdi complexity report
                                view)
        [-include <include-type-list>](for including objects of given
                               type in the report: Values: leaf_module,
```

Feedback

```
                              constraint, latch_loop, combo_loop)
        [-complement]          (Report design objects which are
                               in the complement of the COI)
        [-property <list-or-collection-of-properties>] (List or
                               collection of properties)
        [-output_mode <mapped | unmapped>](This is to set the output
                              mode, i.e. mapped/unmapped. This
                              switch is valid only for SEQ app mode
                              (default is mapped):
                               Values: mapped, unmapped)
        [-subtype <value-list>](Property subtype (default is property):
                               Values: vacuity, property, witness, all)
        [-usage <value-list>]  (Property usage (default is all):
                               Values: assert, assume, cover, all)
        [<list-of-names-ids-or-collections>](List of names or collection
                               of properties, nets or instances)
```

If the COI of the property involves complex operators, the chance of the property proof converging could diminish. Complexity analysis can be performed on the whole design or a subpart of the design after it has been read in. Analysis can be done for a net, a property or an instance. In addition to reporting all the objects found in the formal data model, the analysis can also recognize complex operators such as memories, counters, FIFOs, and asynchronous or latched feedback loops. Based on the COI analysis, you can decide on the next steps to be taken, such as to simplify the property, abstract the complex operators, and so on.

Figure 135: Complex Operators In Design



Before starting formal analysis, knowing the complexity of the design can help estimate the degree of difficulty of verifying the block using formal technology.

Examples

- The following is an example of a complexity report of a design where many counters were identified:

```
vcf> report_fv_complexity
Design Complexity Report:
       Design Unit/Module                    :  4
       Instance (instance)                   :  3
       Primary Input (in)(words/bits)        :  5/5
       Primary Output(out)(words/bits)       :  6/6
       Register      (reg)(words/bits)       :  11/25
            Latch    (latch)(words/bits)     :  6/6
            Flipflop (ff)   (words/bits)     :  3/11
            Counter  (counter)(words/bits)   :  2/8
       StateMc       (fsm)   (words/bits)    :  2/10
       Operator      (operator)              :  70
            Arithmetic                       :  2
            Other                            :  68
       Net            (net)                 :  7
       Port           (port)                :  42
       Constant       (constant)            :  1
       Primary Clock  (clk)                 :  1
       Primary Reset  (rst)                 :  1
```

- The `report_fv_complexity` command displays the inputs and outputs for instances beneath the top module in the design hierarchy. This approach improves insight into submodule complexity and connectivity, enabling more effective design analysis and optimization.

```
vcf> report_fv_complexity main
Design Complexity Report:
 Design Unit/Module             : 1
 Instance          (instance)   : 1
 Instance Input    (instin) (words/bits): 5/9
 Instance Output   (instout)(words/bits): 4/8
 Register          (reg)    (words/bits): 5/12
      Latch        (latch)  (words/bits): 3/3
      Flipflop     (ff)     (words/bits): 1/5
      Counter      (counter)(words/bits): 1/4
 StateMc           (fsm)    (words/bits): 1/5
 Operator          (operator)   : 34
      Arithmetic                : 1
      Equality                  : 15
      Mux                       : 18
 Net               (net)        : 2
 Port              (port)       : 9
```

- By default, a summary of the properties is reported by the `report_fv_complexity` command.

```
vcf> report_fv_complexity traffic.chk.assert_green_no_waiting_first
Design Complexity Report:
       Design Unit/Module                    :  4
```

Feedback

```
      Instance          (instance)              :  3
      Primary Input     (in) (words/bits)       :  4/4
      Primary Output    (out)(words/bits)       :  2/2
      Register          (reg)(words/bits)       :  6/20
           Latch        (latch)(words/bits)     :  2/2
           Flipflop     (ff)  (words/bits)      :  2/10
           Counter      (counter)(words/bits)   :  2/8
      StateMc           (fsm) (words/bits)      :  2/10
      Operator          (operator)              :  51
           Arithmetic                           :  2
           Other                                :  49
      Net               (net)                   :  6
      Port              (port)                  :  25
      Constant          (constant)              :  1
      Primary Clock     (clk)                   :  1
      Primary Reset     (rst)                   :  1
```

- The `-list` option reports all objects that are declared in the RTL that are contained in the COI. It reports the hierarchical name and attribute (operator, net, port & instance) for each object. If the `-verbose` option is added, along with the hierarchical name and attribute (operator, net, port & instance) for each object, the width, the seqdepth, the source code location, and so on is also included.

```
vcf> report_fv_complexity -property conn_5 -verbose -filter {port ||
 net || operator || instance} -limit 25
#1: out2 { port out seqdepth=0 srcinfo="./top.v:1"}
#2: out1 { port out seqdepth=0 srcinfo="./top.v:1"}
#3: d1.out { port seqdepth=0 srcinfo="./dest_ip.v:1"}
#4: d1.out { operator optype=MUX seqdepth=0 srcinfo="./dest_ip.v:4"}
#5: d1.sel { port seqdepth=0 srcinfo="./dest_ip.v:1"}
#6: sel { net seqdepth=0 srcinfo="./top.v:5"}
#7: sel { operator optype=MUX seqdepth=0 srcinfo="./top.v:28"}
#8: dly { port in constant seqdepth=0 srcinfo="./top.v:1"}
#9: d1.in1 { port seqdepth=0 srcinfo="./dest_ip.v:1"}
#10: temp_in1 { net seqdepth=0 srcinfo="./top.v:4"}
#11: d1.in2 { port seqdepth=0 srcinfo="./dest_ip.v:1"}
#12: temp_in2 { net seqdepth=0 srcinfo="./top.v:4"}
#13: temp1 { net constant undriven seqdepth=0 srcinfo="./top.v:3"}
#14: temp { net seqdepth=1 srcinfo="./top.v:4"}
#15: clk { port in clk seqdepth=1 srcinfo="./top.v:1"}
#16: rst { port in constant rst seqdepth=1 srcinfo="./top.v:1"}
#17: in { port in constant seqdepth=2 srcinfo="./top.v:1"}
#18: mem_xor { net seqdepth=2 srcinfo="./top.v:3"}
#19: s1.mem_xor { port seqdepth=2 srcinfo="./src_ip.v:1"}
#20: s1.mem { net width=64 seqdepth=2 srcinfo="./src_ip.v:3"}
#21: s1.N_1 { operator optype=MUX width=64 seqdepth=3
 srcinfo="./src_ip.v:7"}
#22: s1.rst { port seqdepth=3 srcinfo="./src_ip.v:1"}
#23: s1.temp_mem { net width=64 seqdepth=3 srcinfo="./src_ip.v:4"}
#24: s1.clk { port seqdepth=3 srcinfo="./src_ip.v:1"}
#25: d1 { instance module=dest_ip srcinfo="./top.v:8"}
```

Feedback

```
...
1
```

- You can use the `-usage` switch with the `report_fv_complexity/get_fv_complexity` commands, to report property COI information by usage (assert, assume, and cover).

  The `-usage` switch accepts the following values: `assert`, `assume`, `cover`, and `all`.

  - To displays the COI for all assert properties in the design.

    ```
    report_fv_complexity -property * -usage {assert}
    ```

  - To displays the COI for all assert properties in the design.

    ```
    report_fv_complexity -property * -usage {assert}
    ```

  - To displays the COI for assert properties among p1, p2, and p3.

    ```
    report_fv_complexity -property {p1 p2 p3} -usage {assert}
    ```

  - To display the COI for assert properties from the provided list.

    ```
    report_fv_complexity {p1 p2 p3} -usage {assert}
    ```

  - To display the COI for assert and cover properties from the provided list.

    ```
    report_fv_complexity {p1 p2 p3} -usage {assert cover}
    ```

  Note:

  - The `-usage` switch is valid only when used with properties.

    If you use the `-usage` switch with objects other than properties, the tool displays an error:

    ```
    vcf> report_fv_complexity first.next_state -usage {assert}
    [Error] RFC_INVALID_USE_OF_USAGE: The switch -usage is only
     valid with a property.
    ```

  - You may use the `-usage` switch with or without the `-property` switch. When used with `-property`, the filter applies to the specified properties. When used without, it applies to all properties in the design.

- The `report_fv_complexity -complement` command reports the design objects which are in the complement of the COI.

```
vcf> report_fv_complexity traffic.chk.assert_green_no_waiting_first
 -complement
Design Complexity Report:
       Primary Input  (in)   (words/bits) :  1/1
       Primary Output (out)  (words/bits) :  4/4
       Register       (reg)  (words/bits) :  5/5
            Latch     (latch)(words/bits):  4/4
            Flipflop  (ff)   (words/bits) :  1/1
```

Feedback

```
       Operator       (operator)          :  19
            Other                         :  19
       Net            (net)               :  1
       Port           (port)              :  17
```

- The `-limit` option reports objects only when `-list`, `-verbose` or `-col` option is given with `report_fv_complexity`.

```
vcf> report_fv_complexity traffic.chk.assert_green_no_waiting_first
 -list -limit
-1 -complement
#1: main.red { port}
#2: main.yellow { port}
#3: main.green { port}
#4: main.red { optype=SEQ_LAT}
#5: main.N_35 { operator optype=MUX}
#6: main.N_6 { operator optype=EQUAL}
#7: main.N_7 { operator optype=EQUAL}
#8: main.N_8 { operator optype=EQUAL}
#9: main.N_24 { operator optype=MUX}
#10: main.N_10 { operator optype=EQUAL}
#11: main.N_9 { operator optype=EQUAL}
#12: main.yellow { optype=SEQ_LAT}
#13: main.N_47 { operator optype=MUX}
#14: main.N_27 { operator optype=MUX}
#15: main.green { optype=SEQ_LAT}
#16: main.N_48 { operator optype=MUX}
#17: main.N_12 { operator optype=MUX}
#18: main._ASN_105 { operator optype=ONE_HOT}
#19: first.yellow { port}
#20: first.yellow { optype=SEQ_LAT}
#21: first.N_47 { operator optype=MUX}
#22: first.N_27 { operator optype=MUX}
#23: first._ASN_105 { operator optype=ONE_HOT}
#24: chk.rst { port}
#25: chk.green_main { port}
#26: chk.yellow_main { port}
#27: chk.yellow_first { port}
#28: chk.red_main { port}
#29: chk.state_main { port}
#30: chk.state_first { port}
#31: chk._ASN_49 { operator optype=ONE_HOT}
#32: chk._ASN_51 { operator optype=ONE_HOT}
#33: chk._ASN_55 { operator optype=ONE_HOT}
#34: chk._ASN_57 { operator optype=ONE_HOT}
#35: green_main { port}
#36: yellow_main { port}
#37: red_main { port}
#38: yellow_first { port}
#39: s_reg { net}
#40: s_reg { optype=SEQ_FF}
1
```

Note:

Abstraction of complex operators such as FIFOs and counters can increase the overall time taken for generating the complexity report. You can disable the abstraction mechanism using the `-disable_abstract` option in scenarios where detection of these operators are not needed.

- The following example illustrates the usage of the `-level` switch in the `report_fv_complexity` command. The `-level` option generates the complexity report until the specified level in the hierarchy.

  Consider the following example module hierarchy:

```
module top (input a, input b, output c);
submodule in0 (a, b, c);
endmodule

module submodule (input a, input b, output c);
andlib in0 (a, b, w1);
andlib in0(w1,w2,w3);
orlib in1 (a, b, w2);
assign c = w3 ^ w2;
endmodule

module andlib (input a, input b, output c);
assign c = a & b;
endmodule

module orlib (input a, input b, output c);
assign c = a | b;
endmodule
```

  - To get the complexity of the module top, use `report_fv_complexity -level 0`.
  - To get the complexity report of the modules that are instantiated in the top module's complexity, use `report_fv_complexity -level 1`.

    Specifying `-level 1` in this example reports the complexity from module top till module submodule, and does not report complexity for modules andlib and orlib which are instantiated in submodule.
  - To get the complexity report till the leaf module, use `report_fv_complexity -level 2`.

    This reports the complexity until the leaf module, that is, the complexity from module `top` till module `andlib` and `orlib`.

Feedback

18.2.2 The get_fv_complexity Command

Use the `get_fv_complexity` command to get a collection of objects for the complexity of the COI of a set of nets or properties, or complexity of a design instance.

Syntax

```
vcf>get_fv_complexity -help
Usage: get_fv_complexity    # Return collection of object for complexity
 of the COI  of a set of nets or properties, or complexity of a design
 instance
        [-stopif <expression>] (Stop if object matches property
                                expression)
        [-stops <collection-or-list-of-nets>](Collection of nets
                                to stop on)
        [-list]                (List nets and sequential objects
                                in the COI)
        [-verbose]             (Verbose output generates more
                                information like source position)
        [-limit <integer>]     (Limit the number of items to be
                                listed. By default, tool reports
                                all items, use '-limit N' to
                                report N number of items)
        [-level <integer>]     (Generate Complexity report till
                                the given level in the hierarchy.
                                This switch is valid only for
                                the iteration case and fails
                                if start point(s) are given)
        [-filter <expression>] (Report only object which properties
                                match expression)
        [-regexp]              (Use regular expressions for pattern
                                matching operators =~ and !~)
        [-nocase]              (Case insensitive pattern matching
                                for =~ and !~ operators)
        [-bit]                 (Report bit level COI
                               (more precise but slower) instead
                                of word level COI (less precise
                                but faster). By default,
                                tool reports word level COI)
        [-disable_abstraction] (Disable abstraction of complex
                                operators (FIFO, MEM, COUNTER...))
        [-design]              (Report design objects with -list
                                or -verbose (on by default))
        [-instrumented]        (Report tool instrumented objects)
        [-include <include-type-list>] (for including objects of
                                given type in the report:
                                Values: leaf_module, constraint,
                                latch_loop, combo_loop)
        [-complement]          (Report design objects which are
                                in the complement of the COI)
        [-output_mode <mapped | unmapped>](This is to set the
                                output mode, i.e. mapped/unmapped.
```

Feedback

```
                                This switch is valid only for
                                SEQ app mode(default is mapped)
                                Values: mapped, unmapped)
      [-property <list-or-collection-of-properties>](List or
                                collection of properties)
      [-subtype <value-list>] (Property subtype (default is property):
                                Values: vacuity, property, witness, all)
  [-usage <value-list>]  (Property usage (default is all):
                                Values: assert, assume, cover, all)
      [<list-of-names-ids-or-collections>] (List of names or collection
                                 of properties, nets or instances)
```

Examples

- To generate complexity report for module named *main*:

```
get_fv_complexity -design main

{"main.clk", "main.rst", "main.waiting", "main.waiting_cross",
 "main.state_cross", "main.red", "main.yellow", "main.green",
 "main.state_out", "main.next_state", "main.timer", "main.red",
 "main.N_35", "main.N_6", "main.state_out", "main.next_state",
 "main.N_13", "main.N_23", "main.N_14", "main.N_22", "main.N_15",
 "main.N_16", "main._IF_58", "main.N_21", "main.N_17", "main.N_18",
 "main.N_29", "main.timer", "main.N_42", "main.N_25", "main.N_31",
 "main.N_32", "main.N_33", "main.N_26", "main._ASN_43", "main.N_20",
 "main.N_19", "main.N_7", "main.N_8", "main.N_24", "main.N_10",
 "main.N_9", "main.yellow", "main.N_47", "main.N_27", "main.green",
 "main.N_48", "main.N_12", "main._ASN_105"}
```

- To generate a complexity report for a net named *timer*:

```
get_fv_complexity main.timer

{"main.timer", "main.timer", "main.N_42", "main.N_25", "main.N_31",
 "main.state_out", "main.state_out",
 "main.N_32", "main.next_state", "main.next_state", "main.N_13",
 "main.N_23", "main.N_14", "main.N_22",
"main.N_15", "main.N_16", "main._IF_58", "main.state_cross",
 "state_first", "first.state_out", "first.state_out",
 "main.N_21", "main.N_17", "main.N_18", "main.waiting",
 "waiting_main", "main.N_29", "main.waiting_cross",
"waiting_first", "main.N_20", "main.N_19", "main.N_33", "main.N_26",
 "main._ASN_43", "main.clk", "clk",
"main.rst", "rst", "first.next_state", "first.next_state",
 "first.N_13", "first.N_23", "first.N_14",
"first.N_22", "first.N_15", "first.N_16", "first._IF_58",
 "first.state_cross", "state_main", "main.state_out",
"first.N_21", "first.N_17", "first.N_18", "first.waiting",
 "first.N_29", "first.timer", "first.timer",
"first.waiting_cross", "first.N_20", "first.N_19", "first.clk",
 "first.rst", "first.N_42", "first.N_25",
```

Feedback

```
"first.N_31", "first.N_32", "first.N_33", "first.N_26",
 "first._ASN_43"}
```

- To generate complexity report for specific properties:

```
get_fv_complexity {traffic.chk.assert_honor_waiting_main
 traffic.chk.assert_honor_waiting_first}

{"chk.waiting_main", "waiting_main", "chk.green_main",
 "green_main", "main.green", "main.green", "main.N_48", "main.N_6",
 "main.state_out", "main.state_out", "main.N_7", "main.N_8",
 "main.N_12", "main.N_10", "main.N_9", "main.next_state",
 "main.next_state", "main.N_13", "main.N_23", "main.N_14",
 "main.N_22", "main.N_15", "main.N_16", "main._IF_58",
 "main.state_cross", "state_first", "first.state_out",
 "first.state_out", "main.N_21", "main.N_17", "main.N_18",
 "main.waiting", "main.N_29", "main.timer", "main.timer",
 "main.waiting_cross", "waiting_first", "main.N_20", "main.N_19",
 "main.clk", "clk", "main.rst", "rst", "first.next_state",
 "first.next_state", "first.N_13", "first.N_23", "first.N_14",
 "first.N_22", "first.N_15", "first.N_16", "first._IF_58",
 "first.state_cross", "state_main", "main.state_out", "first.N_21",
 "first.N_17", "first.N_18", "first.waiting", "first.N_29",
 "first.timer", "first.timer", "first.waiting_cross", "first.N_20",
 "first.N_19", "first.clk", "first.rst", "main.N_42", "main.N_25",
 "main.N_31", "main.N_32", "main.N_33", "main.N_26", "main._ASN_43",
 "first.N_42", "first.N_25", "first.N_31", "first.N_32", "first.N_33",
 "first.N_26", "first._ASN_43", "chk.clk", "chk.waiting_first",
 "chk.green_first", "green_first", "first.green", "first.green",
 "first.N_48", "first.N_6", "first.N_7", "first.N_8", "first.N_12",
 "first.N_10", "first.N_9"}
```

- To generate a complexity report at bit level including instrumented objects for a specified property:

```
get_fv_complexity -property traffic.chk.assert_honor_waiting_main
-instrumented -bit
```

- To generate the witness properties of a specific property:

```
get_fv_complexity -subtype witness -property
traffic.chk.assert_green_no_waiting_first
```

- To list the design objects which are in the complement of the COI:

```
get_fv_complexity -complement {traffic.chk.assert_honor_waiting_main}

{"main.red", "main.yellow", "main.red", "main.N_35", "main.N_24",
 "main.yellow", "main.N_47", "main.N_27", "main._ASN_105",
 "first.red", "first.yellow", "first.red", "first.N_35", "first.N_24",
 "first.yellow", "first.N_47", "first.N_27", "first._ASN_105",
 "chk.rst", "chk.yellow_main", "chk.yellow_first", "chk.red_main",
 "chk.red_first", "chk.state_main", "chk.state_first", "chk._ASN_49",
```

Feedback

```
 "chk._ASN_51", "chk._ASN_55", "chk._ASN_57", "yellow_main",
 "red_main", "yellow_first", "red_first", "s_reg", "s_reg"}
```

- To list the counters:

```
get_fv_complexity -filter {optype==counter}
```

- To list the counters with 5 bits or more:

```
get_fv_complexity -filter {optype==counter && width>=5}
```

- To list the memories with size of 1024 or more:

```
get_fv_complexity -filter {optype==memory && size>=1024}
```

- To do case-sensitive filtering of all signals in complexity report with string *main.N_(_<Numbers>)*:

```
get_fv_complexity -regexp -filter {name =~ main.N_([10-20])*}
{"main.N_22", "main.N_21", "main.N_20", "main.N_10", "main.N_12"}
```

- To get the number of sequential (fifo, memory, counter) (not number of bits):

```
sizeof_collection [get_fv_complexity -filter {seq}]
2
```

- To get the complexity report until the specified level in the hierarchy, use the `-level` switch. The following example illustrates the usage of the `-level` switch in the `get_fv_complexity` command.

  Consider the following example module hierarchy:

```
module top (input a, input b, output c);
submodule in0 (a, b, c);
endmodule

module submodule (input a, input b, output c);
andlib in0 (a, b, w1);
andlib in0(w1,w2,w3);
orlib in1 (a, b, w2);
assign c = w3 ^ w2;
endmodule

module andlib (input a, input b, output c);
assign c = a & b;
endmodule

module orlib (input a, input b, output c);
assign c = a | b;
endmodule
```

- To get the complexity of the module top, use `get_fv_complexity -level 0`.
- To get the complexity report of the modules that are instantiated in the `top` module's complexity, use `get_fv_complexity -level 1`.

  Specifying `-level 1` in this example reports the complexity from module `top` till module `submodule`, and does not report complexity for modules `andlib` and `orlib` which are instantiated in `submodule`.
- To get the complexity report till the leaf module, use `get_fv_complexity -level 2`.

  This reports the complexity until the leaf module, that is, the complexity from module `top` till module `andlib` and `orlib`.

18.2.3 The compare_coi Command

The `compare_coi` command compares the COI of two properties or sets of properties. There are two main use-models for this command.

Syntax

```
vcf> compare_coi -help
Usage: compare_coi    # Compare one or more coi to a given coi and report
 those which are included, intersect or disjoint from the main coi
        [-prop1 <property name or id>](Name or Id of the main property
                                       to be compared with other(s))
        [-set1 <COIObject collection>] (COI object set (created by
                                        report_coi)that we want to
                                        compare with other(s))
        [-props <Property collection>] (Collection of properties for
                                        which we want to know if their
                                        COI intersects with prop1 or
                                        set1)
        [-set2 <COIObject collection>] (COI object set (created by
                                        report_coi) that we want to
                                        compare with prop1 or set1)
        [-prop2 <property name>] (Name of the property to be compared
                                  with prop1 or set1)
        [-list]                 (Reports design objects in the COI)
        [-verbose]              (Reports all objects in the COI,
                                 including internals with additional
                                 information like source position)
        [-col]                  (Returns list of objects as a collection)
        [-bit]                  (Use bit level COI (more precise but
                                slower) instead of word level COI
                                (less precise but faster).
                                By default, tool uses word level COI)
        [-limit <integer>]     (Limit the number of items to be listed.
                                By default, tool reports 1000 items,
```

Feedback

```
                                use '-limit -1' to report all items)
       [-filter <expression>] (Report only object which properties match
                               expression)
```

- The first use model is to find properties that are intersecting a given property. For example, for the property above, which constraints are in its cone of influence?

  The design has four assume properties:

  ```
  vcf> get_props -type assume
  {"assume_req0_until_gnt", "assume_req1_until_gnt",
   "assume_req2_until_gnt", "assume_req3_until_gnt"}
  ```

  Using the `compare_coi` command you see that all four constraints are contained in the COI for the assertion.

  ```
  vcf> compare_coi -prop1 assert_no_gnt0_if_no_req
  -props [get_props -type assume]
  [INFO] Computing COI of assert_no_gnt0_if_no_req
  [INFO] Computing COI of assume_req0_until_gnt
  [INFO] Computing COI of assume_req1_until_gnt
  [INFO] Computing COI of assume_req2_until_gnt
  [INFO] Computing COI of assume_req3_until_gnt
  ---------------------------------------------------
  COI Comparison report for : assert_no_gnt0_if_no_req
  compared with 5 properties
  ---------------------------------------------------
  Properties for which the COI is contained in current property COI (0):
  ---------------------------------------------------
  Properties for which the COI directly intersects the current property
   COI(4):
  ---------------------------------------------------
  assume_req0_until_gnt
  assume_req1_until_gnt
  assume_req2_until_gnt
  assume_req3_until_gnt
  Properties for which the COI indirectly intersects the current
   property COI(0):
  ```

- The second use-model is comparing two COI sets. The command returns the number of intersecting items, the number of items only in the first set and the number of items only in the second set. The COI sets can be either obtained with the `report_fv_complexity` command (with –col option) or you can use `-prop1` and `-prop2` to specify a property name. In such case the two COIs will be computed with default options. For example, using two properties from the previous example:

  Example 1

  ```
  vcf> compare_coi -set1 [report_fv_complexity -property
   assert_no_gnt0_if_no_req -col] -set2 [report_fv_complexity -property
   assume_req0_until_gnt -col]
  ```

Feedback

```
-----------------------------------------
Compare COI report for two COI object sets:
-----------------------------------------
Number of object which are only in set1: 0
Number of object in the intersection  : 404
Number of object which are only in set2: 0
```

Example 2

```
vcf> compare_coi -prop1 assert_no_gnt0_if_no_req -prop2
 assume_req0_until_gnt
[INFO] Computing COI of assert_no_gnt0_if_no_req
[INFO] Computing COI of assume_req0_until_gnt
-----------------------------------------
Compare COI report for two COI object sets:
-----------------------------------------
Number of object which are only in set1: 0
Number of object in the intersection : 404
Number of object which are only in set2: 0
```

18.2.4 The is_in_fanin Command

Use the `is_in_fanin` command to get a report of whether a given lookup signal is found in the fanin of given signal.

Syntax

```
vcf> is_in_fanin -help
Usage: is_in_fanin    # Reports whether a given lookup signal is found
                        in the fanin of given signal
        [-signals <list-of-signals>] (List of starting signals)
        -lookUpSignals <list-of-lookUpSignals> (List of lookup signals)
        [-property <list-of-properties>](List of properties)
        [-seqDepth <seqDepth>] (Desired seqdepth)
        [-col] (Returns a list of lookup signals (found in the fanin)
                as a collection)
        [-bit] (Reports lookup signals found in bit level coi of
                given signals)
        [-coi_match <value>] (Mode to match lookup signals in the COI
                  of specified signals. Value 'any' means a lookup
                  signal will be marked found if present in the
                  COI of any signal, Value 'all' means a lookup signal
                  will be marked found only if present in the COI of
                  all the specified signals (default value is any):
                  Values: any, all)
        [-file <file-name>] (Save report in specified file)
```

Examples

```
is_in_fanin -signals m0.b0.in1 -lookUpSignals clk
is_in_fanin -signals m0.b0.in1 -lookUpSignals clk -seqDepth 2
```

```
is_in_fanin -property _map_output_product -lookUpSignals {spec.mhl.b
 impl.mhl.b impl.go}
is_in_fanin -lookUpSignals first.timer -signals {green_first clk}
 -coi_match all
```

18.2.5 Limitations

All the recognized design constructs depend on RTL coding style and the synthesis process. After `read_file`/`elaborate`, synthesized operators can be understood by looking at the `ref_name` attribute for cell objects using the `get_attribute` command as follows:

```
vcf> get_attribute [get_cell foo.I_ADD_N_14] ref_name
ADD
```

Given this understanding, the following limitations are observed:

- Gate-level inferencing of math operators is not supported. If, for example, the RTL uses the following add operator,

  ```
  a <= b+c;
  ```

  An adder is recognized. If the adder is specified in the design as lower-level gates, then VC Formal does not recognize the adder.

Counters and FIFOs depend on a heuristic implemented by the `report_fv_complexity` command in order to be recognized.

18.3 Viewing Complexity Report (COI) in VC Formal GUI

The Formal complexity report (also known as the Cone-of-Influence (COI) report) provides the design statistics and design structures that are responsible for the formal complexity. Design complexity can cause state space explosion impacting formal engine performance as well as a convergent outcome for the properties. Using this report, you can make decisions regarding formal verification planning and reducing COI complexity of one or a group of properties to get better performance.

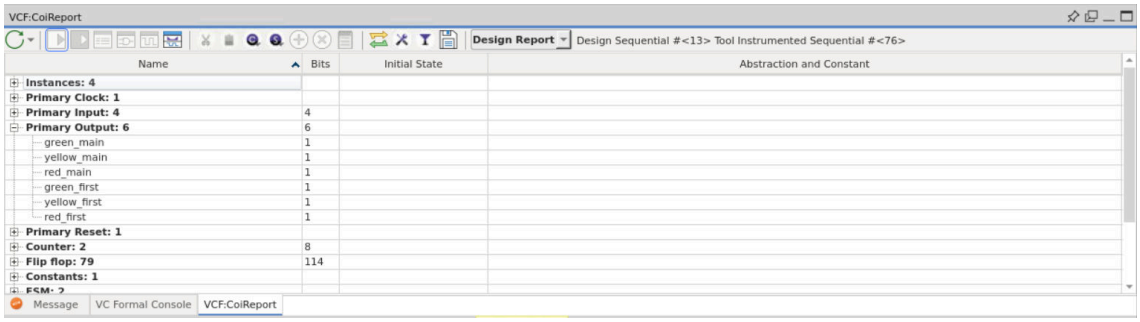
The COI report enables you:

- Locate sets of objects within your design in a quick and intuitive manner
- Find the COI included from a particular object faster

Feedback

- Cross-link from properties to objects

Figure 136: Formal COI Report

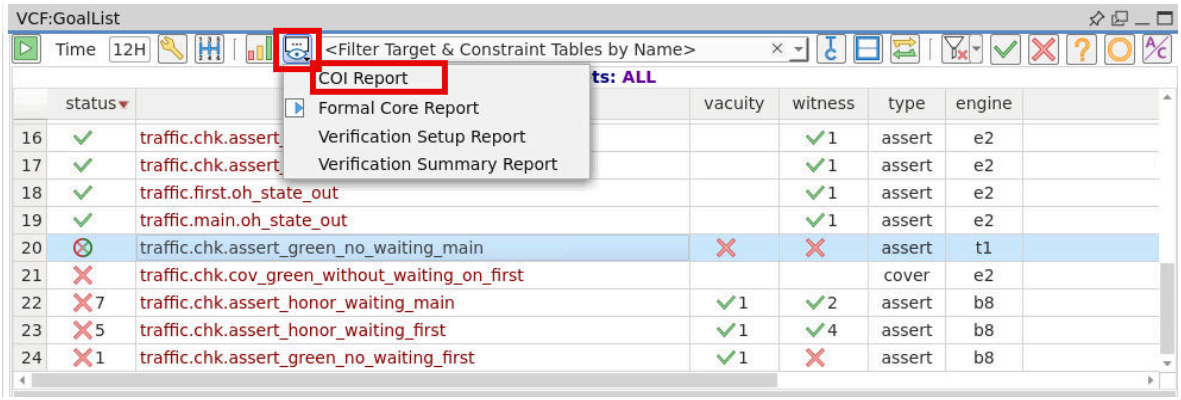


This video focuses on generating and analyzing complexity reports in VC Formal to understand and optimize design components. The complexity report provides detailed insights into the design, including clocks, inputs, outputs, resets, counters, flip-flops, memory, and mathematical constructs. It highlights issues such as missing prompts or resets, aiding in convergence improvement. You can abstract or snap signals using the complexity report.

18.3.1 Generating COI Report

To generate a COI report for the design, select *COI Report* from the drop-down list in the GoalList toolbar as shown in Figure 137.

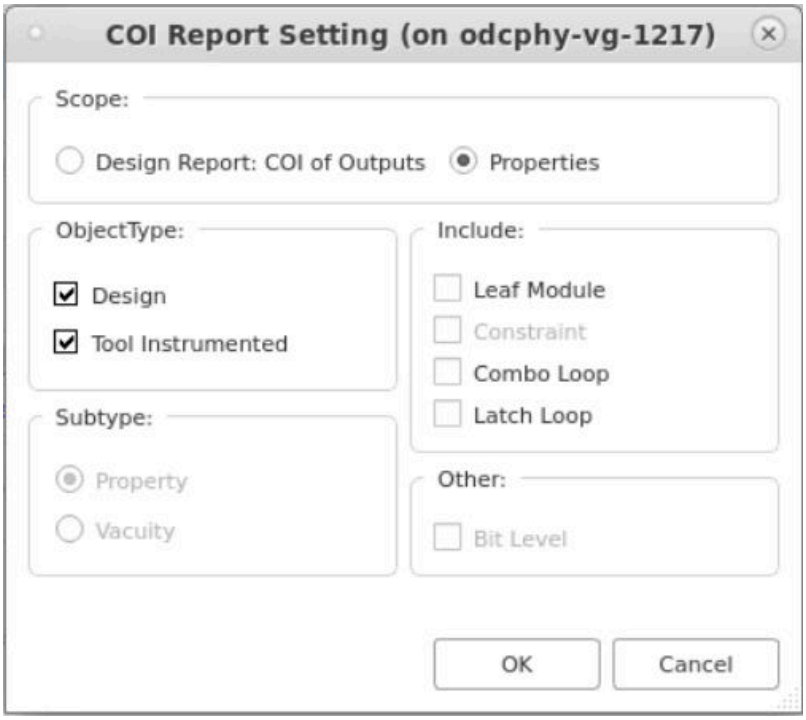
Figure 137: Generate Formal Coi Report for a whole design



The **COI Report Setting** dialog box appears as shown in Figure 138.

Feedback

Figure 138: COI Report Setting

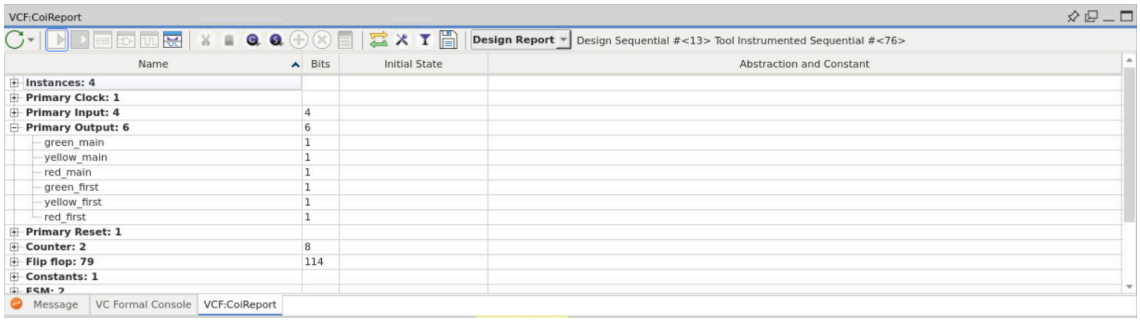


1. Select the scope - *Design Report: COI of Outputs* or *Properties*.
2. To get the COI report of the design objects, select *Design*.
3. To get the COI report of the tool instrumented objects, select *Tool Instrumented*.
4. For including the objects of given type in the report, select *Leaf Module*, *Constraints*, *Combo Loop*, *Latch Loop* as required.
5. To filter the COI report based on the Goals, select *Property* or *Vacuity* as required.
6. To view the COI report at the bit level COI, select *Bit Level.*
7. Click *OK*.

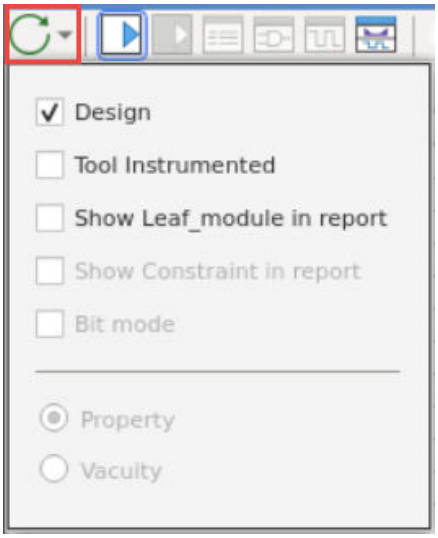
Feedback

Based on the options selected, the COI report is displayed as shown in Figure 139.

Figure 139: Design Report: COI of Outputs



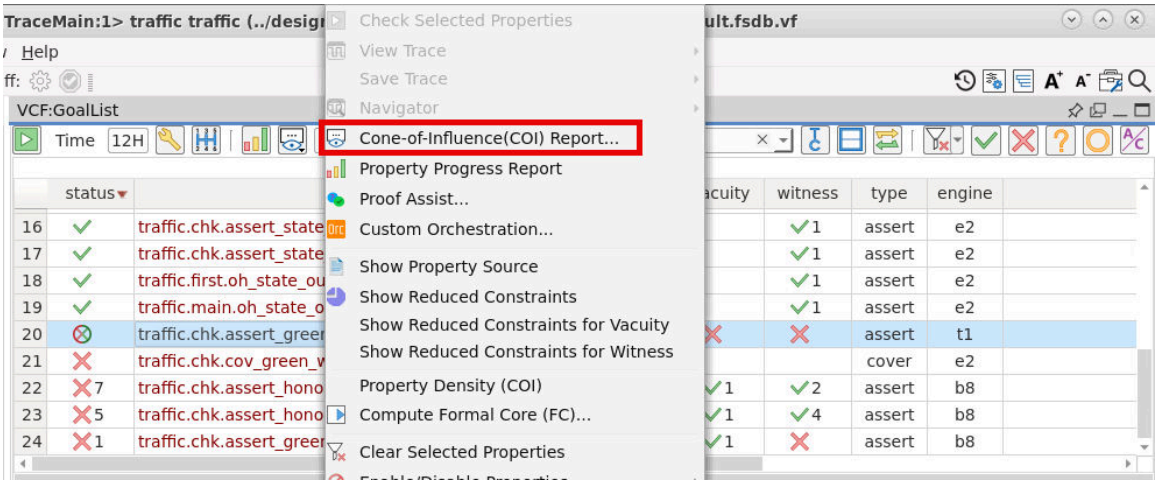
You can also filter the COI report using the drop-down list.



You can generate reports for one property as shown in Figure 140.

Feedback

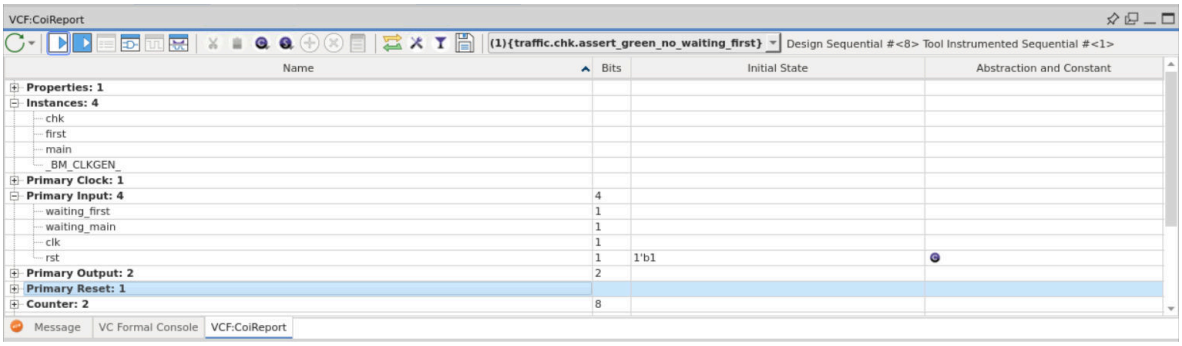
Figure 140: Generate COI Report for a Property



The **COI Report Setting** dialog box appears as shown in Figure 138.

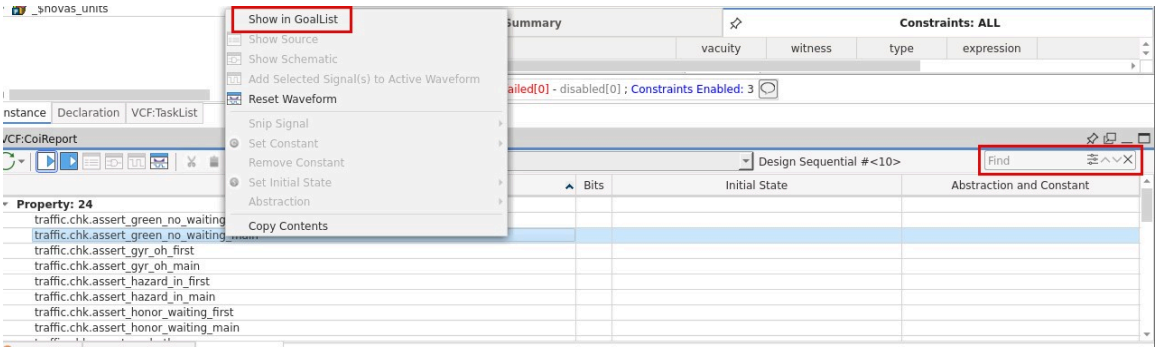
The COI of the property appears as shown in the following figure:

Figure 141: COI Report for a Property

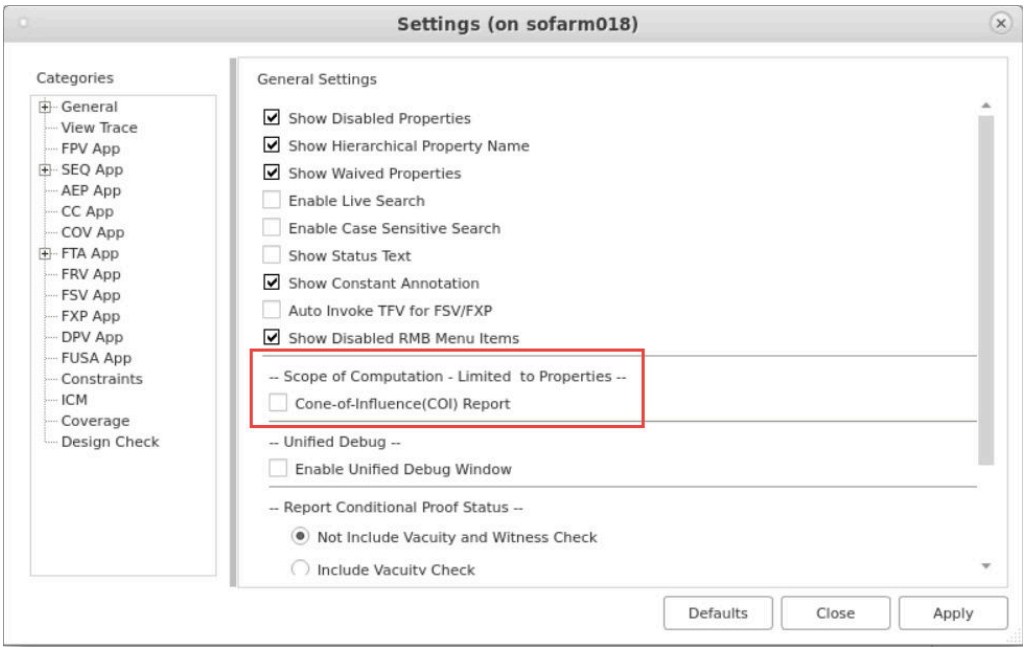


The COI report table has a link in the GUI showing where the elements are in the structure.

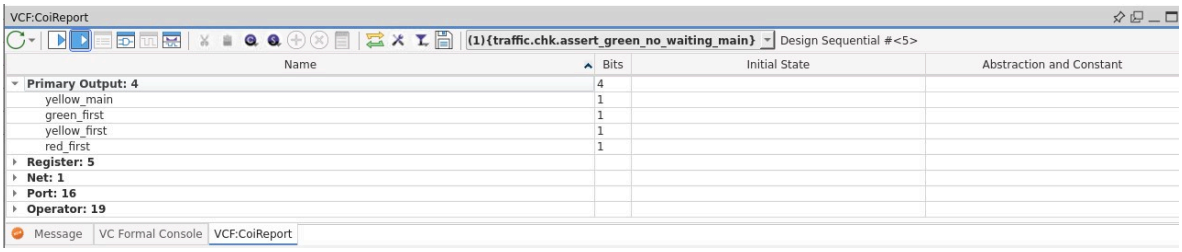
The CoiReport includes a right-mouse button (RMB) menu item, **Show in GoalList**, enabling you to display selected properties in the GoalList. Alternatively, to find a property in the CoiReport from the GoalList, use Ctrl+F to access the search bar and locate the required property.



You can limit the scope of computation of COI for properties using the following setting in the *Customize View Settings* dialog box as shown in the following figure:



To generate a COI report for the complement of the COI for a property, click *Show Complement COI Report* from the *CoiReport* toolbar as shown in the following figure:



Feedback

18.3.2 Viewing Source, Waveform, and Reset waveform of an Element

After you generate a COI report for a property, you can view the source, waveform, and reset waveform for an element.

- To view the source view, select an element and click .

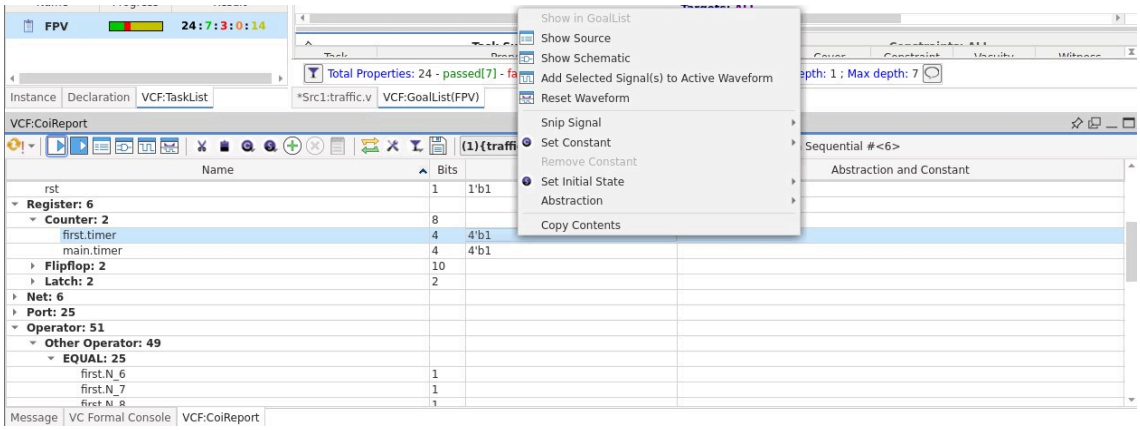
  Alternatively, you can double-click the signal name in the COI report to view the source.

- To open schematic, select a signal and click .

- To open the waveform, select an element and click .

- To open the reset waveform, select an element and click .

Figure 142: RMB Menu of CoiReport



Alternatively, from the RMB menu of an element, select *Show Source*, *Show Schematic*, *Add to Wave, and Reset Waveform* to view the source of the element, to view the schematic, to open the waveform, and reset the waveform respectively.

18.3.3 Adding Snips, Setting Constants, Setting State Of an Element, and Setting Abstractions

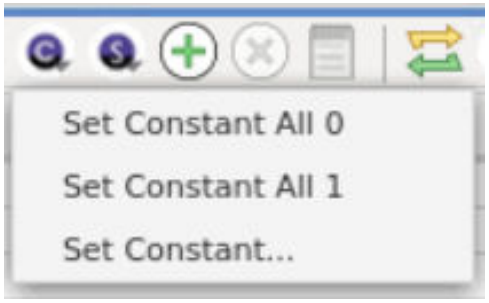
From the **CoiReport** toolbar, you can perform the following functions on each of the elements:

- Snip the elements
- Set constant

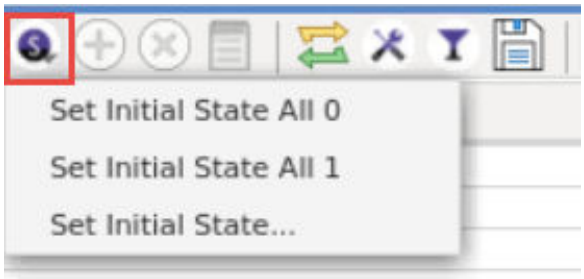
- Set state
- Set Abstractions

Perform the following to snip the elements, set a constant for an element, set state for an element, and abstract an element:

- To snip an element, select an element and click .
- To unsnip an element, select an element and click .
- To set constant for the element, select the element and select from the following drop-down:



- To set state for the element, select an element and select the state from the following drop-down:



- To set abstraction for the element, select an element and click .
- To delete abstraction for the element, select an element and click .

  Note:

  Snipping of single bit or part of select signals from the COI view is not supported as it impacts the performance of updating the COI view.

Alternatively, from the RMB menu of an element, select *Snip Signal*, *Set Constant*, *Set State and Set Abstractions* to snip the element, set constant0/1 on an element, set a state for the element, and set abstractions respectively.

The *Abstraction* column in the **CoiReport** indicates if an element is snipped, abstracted, or set to a constant.

After you modify any of the values above, click to save the changes, else the value is not updated.

18.3.4 Filtering the COI Report

You can filter the COI report for different design constructs.

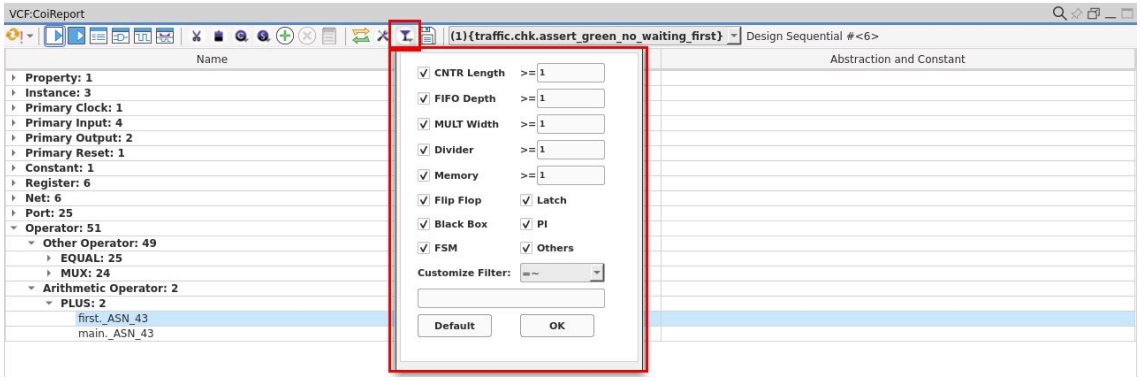
1. To filter the COI report, click in the **CoiReport** toolbar.

   The filter appears as shown in Figure 143.

2. In the box that appears in the right-hand side, select the design constructs and specify the range for each of the elements.

   The COI report is updated based on the filter.

Figure 143: Filtering CoiReport

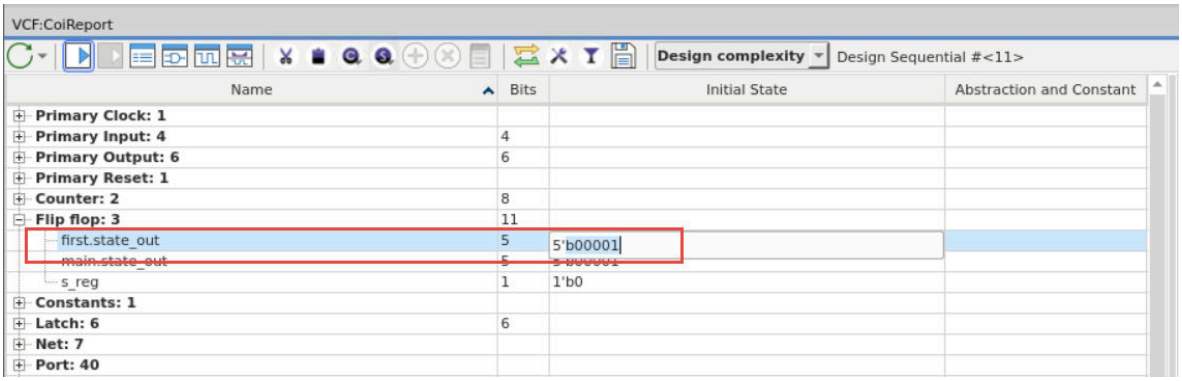


18.3.5 Setting State By Inline Editing

You can set state of an element by editing the state in the *Initial State* column as shown in Figure 144.

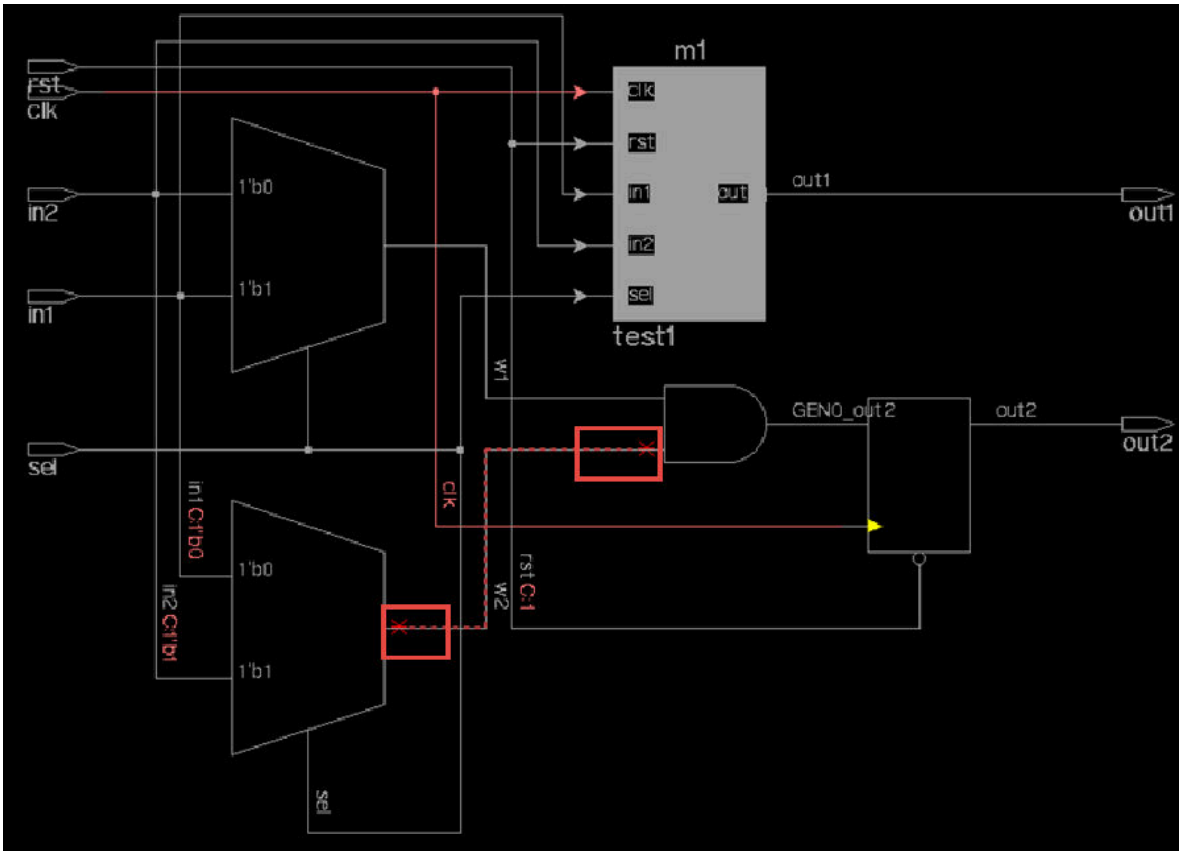
Feedback

Figure 144: Setting State By Inline Editing



18.3.6 Viewing Snip Icon in nSchema

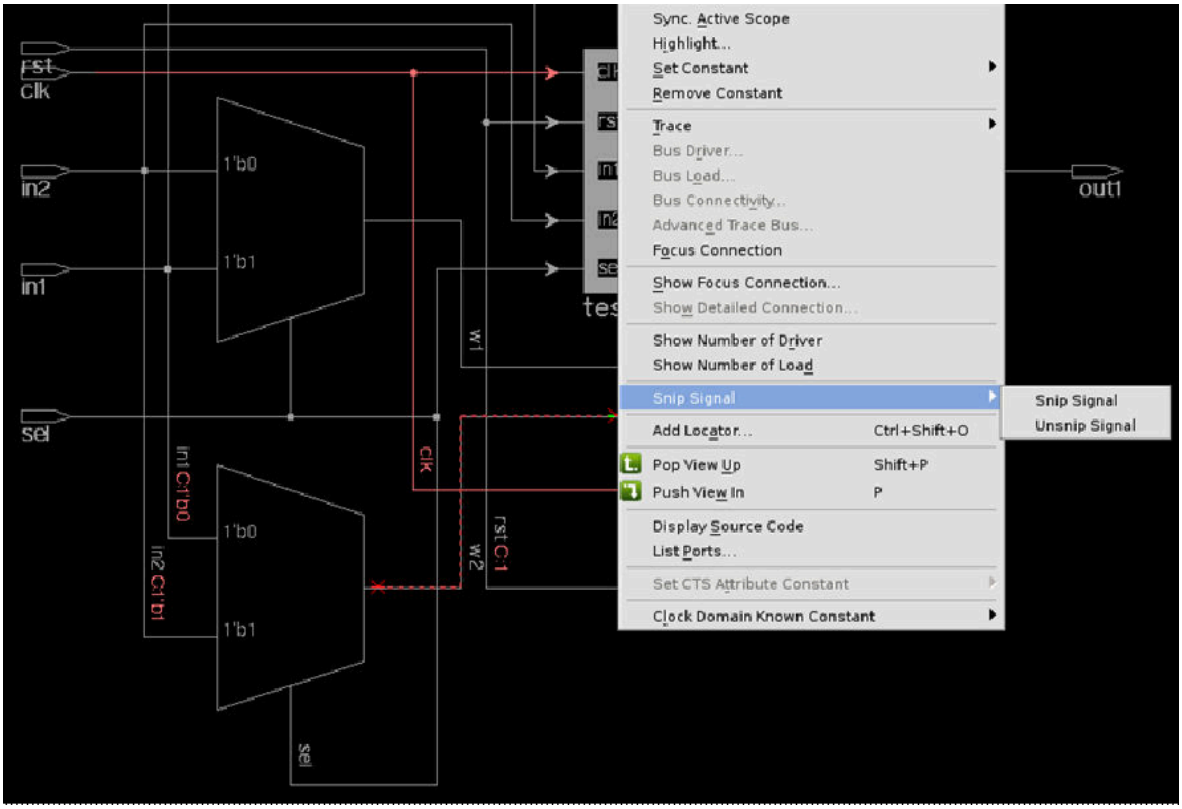
The Verdi schematic displays the snip icon near the driver pin on the signals which are snipped in schematic. This is supported for both module schematic and path schematic.



Feedback

Snipping and Unsnipping Signals from nSchema

You can snip/unsnip the selected signal from the nSchema view as supported in the Verdi Source view and SignalList view. You must right-click on the selected signal, and select Signal > Snip Signal or Unsip signal.



18.4 Using Automatic Abstraction To Reduce Complexity

Certain design constructs are known to cause computational difficulties for formal algorithms under certain scenarios. Sometimes these constructs can be selectively removed from the design, replacing them with an approximation. In many cases, run-time is improved with an acceptable loss in accuracy.

VC Formal provides automation to abstract such elements from the design in a semi-automated way using the following commands:

- `set_abstractions`
- `report_abstractions`
- `get_abstractions`

Ensure that you use these commands after design load (`read_file` or `elaborate`). They do not have an impact on design load time. Formal model build time and size generally improves if the `set_abstractions` command is used.

The `set_abstractions` command provides the ability to replace certain key complex design topologies with simpler *abstractions* for use by the formal engines. This command can be run many times in any given formal session.

The initial level of abstraction provided is to remove the elements from the formal model by *snipping* them from the design. This mechanism leverages the existing `snip_driver` mechanism internally for removing the snips. This form of abstraction is designed and intended to work with the snip-point mechanisms in VC Formal, but are presented as abstractions.

18.4.1 The set_abstractions Command

The `set_abstractions` command searches for user-requested design constructs, and replaces them with the desired abstraction. The default abstraction model provided is to remove the element from the formal model (also known as snip) in such a way that the complex behavior is replaced by a free variable. You can then provide constraints in the cases where a free variable is too under-constrained. This is very similar to providing constraints when the `snip_driver`command is used.

The `set_abstractions` command returns a collection of nets which are driven directly by the abstracted region. For snip type abstractions, these nets may be used in commands in a manner exactly as other `snip_driver` nets might be (`fvassume/fvassert, $driver`).

The `set_abstractions` command specifies the following:

- Which design constructs and their size thresholds should be replaced in the searched regions (the `-construct` switch and `-remove` switches).
- Which abstraction model should be used to replace the identified design constructs (the `[-using <snip, keyvals, keyvals+linearize, keyvals+user, keyvals +range, keyvals+linearize+range, keyvals+user+range>]` switch.
- Where in the design to search for constructs and counters to simplify for the formal engines.
- Where in the design to exclude specified element from abstraction (the `-remove` switch).
- Which task(s) the abstractions should be applied to. By default, the `set_abstractions` command only creates the abstraction in the current task. Use the `-global` switch to create the abstraction in all tasks (including other application modes).

When the `-using keyvals` switch is used to abstract counters, they are transformed into simple non-deterministic automata. To construct these automata, some key values are

Feedback

used to divide the value space of the counter into non-overlapping regions. The transitions of the original counter are then re-defined as non-deterministic transitions between these regions. This leads to an over-approximation of the behavior of the original counter.

The keyvals based counter abstraction applies only to counters with a width > 3. Counters with a width <= 3 are not considered significant for keyvals based counter abstraction. When you use the `report_abstractions` command to print key values for these counters, "NA" is displayed for those with a width <= 3.

The default key values that can be used in the `-using keyvals` option are as follows:

- Value of any constant value that the counter is compared with (using relation operators <, <=, = , >, >=) is a key value.
- The value zero is a key value.
- If counter has N bits, then its max value 2^N - 1 is a key value.
- The reset value for the counter is a key value.
- Any constant value that the counter is assigned to through a procedural assign is a key value.

You can also provide additional key values as discussed in the examples in the following section.

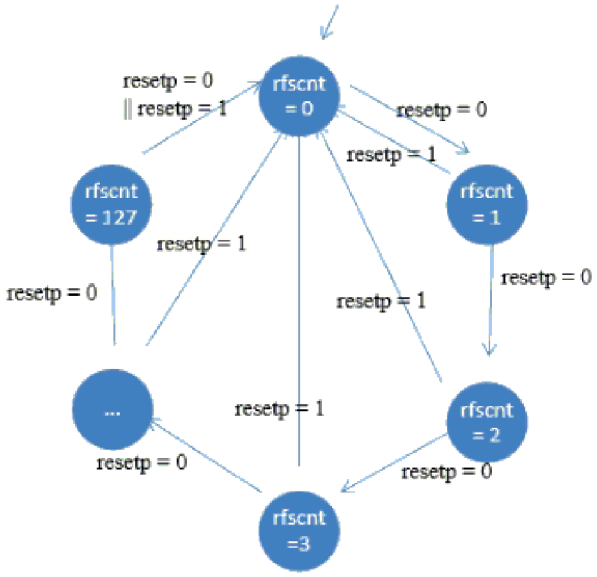
Linearized Mode for Keyvals Based Abstraction

VC Formal supports a *linearized* mode for keyvals based abstraction. In this mode, restrictions are added on value transitions allowed for counters when they belong to ranges like [2-127]. In the the linearized mode, VC Formal does not allow snip like behavior, and this helps reduce the number of false negatives (incorrect falsification of valid assertions).

In the linearized mode, the value progression of counter is tied to be directionally consistent with the increment/decrement operation. As long as the condition for increment is enabled, the value of the counter is strictly increased (and vice versa for decrement). That is, the value of rfscnt in the example shown in Figure 145 can transition in a strictly increasing sequence like 2->3->10->127 as long as the condition for increment is active.

Feedback

Figure 145: State machine for Refresh Counter



Note that the value increment/decrement can be arbitrarily large as long as the next value does not skip over the next logical range.

- To enable this linearize mode, the `+linearize` option must be added to the `set_abstractions` command:

```
set_abstractions -construct count=3 -using keyvals+linearize
set_abstractions -construct count=3 -using
 keyvals+{1,9}\\{5,6}+linearize
```

Examples

- To abstract one construct, say a multiplier of width 8, use the following:

```
vcf> set_abstractions -construct {mult=8}
```

- To abstract more than one construct, use the following:

```
vcf> set_abstractions -construct {mult=8 div=8 add=8 sub=8 mod=8
 count=8 array=8 vector=8 array_comb=32 vector_comb=32}
```

- To remove an abstraction, use the `-remove` option:

```
vcf> set_abstractions -construct {mult=8} -remove
```

- To enable VC Formal to abstract the counters in the design, use the following command:

`set_abstractions -construct count=3 -using keyvals` //Abstracts all counters of size 3 or greater than 3 having default key values.

The option `-construct count=N`must always be used in the `set_abstractions`command to enable counter abstraction, where ideally N >= 3.

- To enable VC Formal to abstract particular counters, use the following command:

  `set_abstractions -construct count=3 -using keyvals -filter { name== top.COUNT1.counter1 }`// Abstracts only the counter whose name matches with having default key values

- To add extra keyvals manually for counters, use the following command:

  `set_abstractions -construct count=3 -using keyvals+{1,3,9,12-14}` // Introduces extra key values in addition to default key values.

- To exclude keyvals manually, use the following command:

  `set_abstractions -construct count=1 -using keyvals+{2-11}\\{5-9,13} -filter { name == top.COUNT1.counter1 }`// This command adds key values 2,3,4,10,11 and it deletes key value 13, if 13 is the default key value for the counter.

- To abstract the counter with the key values specified by the user, use the following command:

  ```
  set_abstractions -construct count=4 -using keyvals+{3,5,11}+user
  ```

  This command adds key values 0, 3, 5, 11, 15 for the counter. Here, 0 and 15 are the default key values for a 4-bit counter.

- To enable the linearize mode, the `+linearize` option must be added to the `set_abstractions` command:

  ```
  set_abstractions -construct count=3 -using keyvals+linearize
  set_abstractions -construct count=3 -using
   keyvals+{1,9}\\{5,6}+linearize
  ```

- To apply the abstractions to all the tasks, use the `-global` option:

  `set_abstractions -construct {mem=1 array=1 vector=1 array_comb=1 vector_comb=1} -global`

18.4.2 The report_abstractions Command

The `report_abstractions` command reports all currently active abstractions in the design. By default, the command reports a count of the requested constructs which are abstracted. Value returned: 1 if successful, 0 otherwise.

Examples

Feedback

- The following is an example of the `report_abstractions` command:

```
vcf> report_abstractions
Abstraction Report
Construct                Count
-------------------------------------------------------------------------
---------------
Multiplier                3
Counter                   2
1
```

- The following is an example of the output when the `-list` switch is used:

```
vcf> report_abstractions -list
Abstraction Report
Construct                Size              Model
 Design Net
-------------------------------------------------------------------------
---------------
Multiplier                64                  snip
   a.b.mult_out
Counter                   16                  snip
    p.q.op_cnt
1
```

- In the above `-list` report, the construct type, actual sizes, abstraction model being used, and the driven net are all reported. For the snip type abstraction model, the net can be further constrained and accessed as if the `snip_driver`command were used.
- In the verbose scenario, the input nets for the abstracted design construct are also reported. The following is an example of the scenario:

```
vcf> report_abstractions -verbose -construct mult=16
Abstraction Report
Construct                Size              Model
 Design Net
-------------------------------------------------------------------------
------------
Multiplier                64                  snip
   a.b.mult_out
Input: a.b.net_in1
Input: a.b.net_in2
1
```

- For keyvals based counter abstraction, the key values are printed for counters in `-verbose` mode. The counters with a width <= 3 are not considered significant for keyvals based counter abstraction, therefore, `NA` is displayed for such counters in the abstraction report.

```
vcf> report_abstractions -verbose
Abstraction Report
Construct         Count
```

Feedback

```
----------------  ------
Counter                4

Construct             Size      Model     Design Net
--------------------  ------    --------  ------------
Counter                  4      keys+lin_usr  cnt1.cnt4
  Key Values: 0, 15, 25, 34
Counter                  3      keys+lin_usr  cnt2.cnt3
  Key Values: NA
Counter                  6      keys+lin_usr  cnt7.cout
  Key Values: 0, 25, 34, 63
Counter                  3      keys+lin_usr  cnt8.cout1
  Key Values: NA
1
```

- The `report_snips` command does not report nets which are affected by the abstraction system.

18.4.3 The get_abstractions Command

The `get_abstractions` command returns a collection of nets driven by the various abstractions which are remodeled. The command is similar to `report_abstractions`, except that a collection is returned.

18.5 Reviewing the Formal Core of Property

The formal core of any proven property consists of the following three components:

- A set of registers in the design that are used by the formal engines to prove the property.
- A set of constraints that are used to prove the property.
- A set of inputs that are involved in proving the property.

The above definition of formal core also applies to the properties with bounded proofs. If a property has a bounded proof up to a depth k (no counter examples up to depth k), then you can get the formal core for the property for a depth of up to k. In such cases, the formal core contains the registers, constraints, inputs used by the formal engines for the bounded proof up to the specified depth k.

Formal core can be used for several applications and is most commonly used as part of the formal signoff flow described in section Formal Core (FC) Analysis, however, it can also be used to assist with convergence as described below.

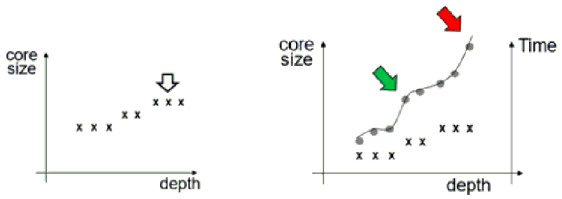
18.5.1 Formal Core for Convergence

The formal core for bounded proofs can help assist in achieving convergence by narrowing down the portion of designs and constraints that are making proof difficult. On a bounded result, the formal core gives the list of all registers, inputs and constraints required to reach depth k. This is useful as it identifies any potentially problematic constructs that could be making convergence difficult - for example large counters, memories, multipliers and so on. In addition, the formal core shows which constructs are not required for the proof. If such constructs are inside the COI but outside of the formal core, they are potential candidates for abstraction.

18.6 Support for Proof Assist

Proof Assist provides significant visibility into the formal runs to assist in convergence of safety properties. Proof Assist uses the formal core to assess the relation between the complexity of various design components and their influence on solving the properties.

The formal core reports the design components that are needed for proving the property for the given depth (in some cases, minimally but not always). The formal core focuses on a small subset of design elements when trying to improve the run time of a given property, making it advance to a higher depth, or helps in proving it for any depth. As the property is shown to hold at a higher depth, the formal core builds up, and its size is a step function, and it is always expanding (never shrinks).



Furthermore, the up-to-date core represents the new set of elements required to prove or falsify the property at the specified depth. Proof Assist shows the sequence of cores in full, using which you can focus exclusively on the last few cores presented for the depth of interest.

Proof Assist gives further insight on the difficulty with advancing the proof deeper. Using this, you can determine whether specific design elements (newly introduced to the core) are changing the run-time drastically, or alternatively the slow-down is due to an exponential state space explosion on the current core elements, and act accordingly.

Proof Assist labels each component (within a core or not), provides run-time information, the depth/cycle information (relevant for the change in the core), and engine-related effort information. This enables you to characterize the hard properties, determine whether the speed can be improved through remodeling or abstraction, and determine how much

resources (including the compute time manually set by the user) are needed to improve the run time, if possible.

With Proof Assist, you can zoom in on the possible root causes of the performance issues, through what-if analysis on the core elements, or automatically through characterizing combinational elements contribution to the solving effort of multiple registers.

Proof Assist uses the `compute_proof_assist` and `report_proof_assist` commands for assisting in the convergence of safety properties. Using these commands, you can depict the core, depth, time, and engine information to prioritize the elements within the core, based on their influence on the run time at (or close to) the depth at which the property starts hanging, and leverage the design knowledge to remodel. To re-prioritize the elements, the `report_proof_assist` command provides additional hints on the effort one or more engines spent on each design element within the formal core or its fan-in cone.

Proof Assist provides concrete hints to improve the convergence of the given properties. These hints can empower both advanced Formal users as well as novice users.

Note:
Proof Assist is currently supported only for the VC Formal FPV and VC Formal FRV applications.

18.6.1 Use Model for Computing Proof Assist

The following is an example use model to run Proof Assist.

```
set_fml_appmode FPV

read_file -sva -format sverilog -top tb_alu -vcs
 {-sverilog ../rtl/alu.sv ../work/tb_alu.sv}

create_clock clk -period 100
set_change_at random_start -clock clk
set_change_at random_addr -clock clk
set_fml_var fml_witness_on true

create_reset rst_n -low
sim_run 1
sim_save_reset

set_fml_var fml_max_mem 64GB
set_fml_var fml_formal_core_reduced_constraints false
set_engine_options fc1 -engOpts "-strategy 2"
compute_proof_assist -prop tb_alu.ast_add_check -ignore_status -depth 5
 -block

report_proof_assist
report_proof_assist -comb -constraints
report_proof_assist -comb -constraints -verbose -advisor
report_proof_assist -depth 5
```

Feedback

```
report_proof_assist -verbose -comb -last_depth
report_proof_assist -target tb_alu.alu_state -verbose -comb -last_depth
report_proof_assist -verbose -comb -depth 5 -coi
```

18.6.2 Running Proof Assist for the First Time

For an initial run on a given new design, it is recommended to use the following settings:

```
set_fml_var fml_formal_core_reduced_constraints false
set_engine_options fc1 -engOpts "-strategy 2"
compute_proof_assist -property <property> -profiler p1 -depth 20
report_proof_assist -last_depth -verbose
```

It is also possible to run engine strategies 0 and 1 in parallel, and examine the best result (largest core/depth).

18.6.3 Computing Proof Assist

To run Proof Assist, use the *compute_proof_assist* command.

Syntax

```
compute_proof_assist [-block] [-stop] [property
 <list-or-collection-of-properties>][-depth <int>] [-minimize
 <minimization-attribute>] [-profiler <string>] [-ignore_status]
 [-live_update <core|depth|all>] [-config <config-options>]
```

Where,

- `[-block]`: Specify this option to run `compute_proof_assist` command in block mode. When you specify this option you cannot interact with the `vcf` shell when the `compute_proof_assist` is in progress.
- `[-stop]`: Specify this option to stop the execution of currently running *compute_proof_assist* command.
- `[-property <list-or-collection-of-properties>]`: Specify the property or list of properties to work on.
- `[-depth <int>]`: Specify the depth consistent with the previous run. The property should have been run with `check_fv` for at least a depth of 1, else the command will not run on the property, unless `-ignore_status` is specified. The given depth has to be less or equal to the last proven depth if the property is inconclusive or falsified. If no depth is provided, a default depth of 1,000 is considered.
- `[-minimize <minimization-attribute>]`: Attempt to minimize formal core. Values: regs

- `[-profiler <string>]`: Specify the option depending on what information to gather (including how to re-run). Values: p0 and p1. By default, the profiler is set to p1.
  - p0: Computes the core, depth and time for the properties
  - p1: Computes the core, depth, time, and effort for the properties
- `[-ignore_status]`: Specify this option to ignore the status of property, that is, allow command to proceed even if the property is not in inconclusive state.
- `[-live_update <core|depth|all>]`: Specify this option to get the live update of the notifications when a new core or depth or both is explored.
- `[-config <config-options>]`: Specifies the configuration for subsequent Proof Assist runs in the current session, in the format `{param=value,...,param=value}`, with possible options: `profiler=p0|p1[default], depth=(int) [default 1000]`.

  If a parameter is repeated, only the last value provided is considered. When `-config` is used, this command returns the up-to-date configuration after applying the change. Use an empty list {} to return the current configuration without making any change.

18.6.4 Viewing Proof Assist Report

Use the `report_proof_assist` command to get a report of the `compute_proof_assist` command.

Syntax

```
report_proof_assist [-xml][-verbose][-depth
 <int>][-constraints][-comb][-bit][-target <register-name>]
 [-last_depth][-top_signals <int>][-last_cores
 <int>][-explicit_cycles][-property <property-name>] [-coi]
 [-note_truncation <option>] [-advisor] [-advisor_level <int>]
 [-report_cutoff <int>] [-engine <engine-option>] [-model <model-option>]
 [-intent <intent-option>][-mode <mode>][-config <config-options>]
```

Where,

- `[-xml]`: Specify this option to get the performance analysis report in XML format.
- `[-verbose]`: Specify this option to get a verbose performance analysis report.
- `[-depth <int>]`: Specify a value to limit the reporting depth to the given value.
- `[-constraints]`: Specify this option to include the constraints (marked as [C]) in the report.
- `[-comb]`: Specify this option to report the combinational logic in the fan-in of the core's registers (marked as [R]), including gates [G] and inputs [I].
- `[-bit]`: Specify this option to get a report at the bit-level.

- `[-target <register-name>]`: Specify the list or collection of registers to be the target of focused performance analysis.
- `[-last_depth]`: Specify the reporting depth to the last depth with available data.
- `[-top_signals <int>]`: Specify the option to report the highest N signals in terms of effort.
- `[-last_cores <int>]`: Specify the option to report the last N cores available (latest in terms of cycles).
- `[-explicit_cycles]`: Show all cycles without merging. When not specified, cycles with similar cores (no new signals) are merged.
- `[-property <property-name>]`: Report cores for the specified property.
- `[-coi]`: Reports information on the elements in the bounded Cone-of-Influence (COI) but not in the core at a given depth.
- `[-note_truncation <on/off>]`: Specify this option to control (on or off) the truncation for notes in the report. Default is on, to truncate at 80 characters.
- `[-advisor]`: Reports advisor notes along with the recommended action items you can take to fix various scenarios of performance bottle necks based on the Proof Assist analysis.

  ;;[<ID>] <*Text*>. Where, *ID* of the notes start with 0 and the *text* contains useful information about the characteristics of the property, depth, design element, and so on. The notes are truncated by default.
- `[-advisor_level <int>]`: Enable/disable advisor notes based on a 0/1 bit-mask, from lsb to msb: Complexity notes, Tcl next-step notes. -advisor_level provides further control over the notes that should be included in the report. Possible values: 0,1,2,3, representing a 2-bit binary bitmask of two options:
  - bit0 controls the filter of complexity-related notes, such as counters.
  - bit1 controls “TCL code” within next-step steps of the advisor.

  By default, the `-advisor_level` is set to 2, also known as mask 10, where the complexity-related notes are disabled, while next-step TCL code notes are enabled.
- `[-report_cutoff <int>]`: Defines the threshold of the report length, measured in terms of the number of signals displayed.
- `[-engine <engine-option>]`: Specify the engine to produce the report. Possible values: `all`, `deepest (default)`, or `<fc1, fc2, fc3, fc4, fc5>`.

  Safety properties use engines fc1, fc3, and fc4, while liveness properties use engines fc2 and fc5. Use `-engine deepest` to focus on the reports with the deepest core.

- `[-model <model-option>]`: Choose the model result to report. Possible values: `all`, `def`, `deepest (default)`, or `<model name>`.

  The default model (called `def`) applies the formal core engines on the original design, while `optB` applies the engines on speculatively abstracted version of the design based on snipping high-effort nodes. If the analysis determines that the latter model is useful, corresponding reports will include *Note 1060* (see section Proof Assist Notes for more details).

- `[-intent <intent-option>]`: Use the `-intent` option to further control the notes being displayed. Use it to describe concrete intent of your verification (proof, coverage, or bug-hunting), or leave as 'all' (default) when unknown. Possible values: `bug-hunting`, `coverage`, `proof`, or `all [default]`.

  Note that both `advisor_level` and `intent` can be independently configured using `-config` option as described below.

- `[-mode <mode-option>]`: Use this option to view the report with different formats. Possible values: `core-buildup[default], comparative`.

  The default mode is set to `-mode core-buildup` that reports the core buildup over cycles as described above. For selected engines, the -mode comparative, with a given `-depth`, allows showing all relevant signals and their corresponding effort for all engines and models.

- `[-config <config-options>]`: Configures the default arguments for this command. `-config` allows configuring:

  - The `depth` and `profiler` for `compute_proof_assist`
  - The `advisor_level` and intent for `report_proof_assist`
  - and printing the up-to-date configuration.

  Usage: `-config {<arg>=val,...,<arg>=<val>}`. Possible values: `intent=bug-hunting|proof|coverage|all[default]`.

  When an empty list is used (for example, `report_proof_assist -config {}`), it prints the up-to-date configuration, without altering it. You can change a single flag or more (comma delimited), for example:

```
report_proof_assist -config {intent=bug-hunting}
compute_proof_assist -config {depth=100}
```

By default, the `report_proof_assist` command reports the core flops (registers) with the list of elements added each time the core expanded with time at each depth. To get a report of the different element types (constraints, inputs, and gates), use the required options.

Feedback

The report starts with the high-effort signals across all hierarchies. The top signals section is reported if the p1 profiler option is specified in the `compute_proof_assist` command, without specifying a depth or target. The number of signals is the lowest of {the argument specified by `-top_signals`}, and {the number of elements with non-zero effort}.

Examples

- To view the *Summary View* report showing the depths per cycles in which new flops were added to the core, use the following commands. The report shows the core build-up, as well as the top effort signals in the last core, alongside with their effort percentages.

```
compute_proof_assist
report_proof_assist
Proof Assist Summary View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
Top Effort Signals in Last Core (Depth=66)
-------------------------------------------
(0.31%) [1]    testbench.my_data_is_written
(0.15%) [1]    testbench.DUT.initiator
(0.1%)  [9]    testbench.DUT.count
Depth                          | Performance Complexity Buildup
-------------------------------|-----------------------------------
-----------------
D=3..66 (9s):                  |
                               |     [9]    testbench.DUT.count
D=2 (0s):                      |
                               |     [1]
 testbench.my_data_is_written
D=1 (0s):                      |
                               |     [1]    testbench.DUT.initiator
D=0 (6s):                      |
=========================================
1
```

- To view the other element types such as constraints, inputs, and gates, specify the `-comb` and `-constraints` options.

```
report_proof_assist -comb -constraints
```

Each element is marked as [R], [I], [G], or [C], representing registers (or flops), inputs, gates, or constraints, respectively in the report. The top-effort signals are always Registers.

```
Proof Assist Summary View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
```

Feedback

```
Model:    def
Engine:   fc1
Top Effort Signals in Last Core (Depth=66)
-------------------------------------------
(0.31%) [1]    testbench.my_data_is_written
(0.15%) [1]    testbench.DUT.initiator
(0.1%)  [9]    testbench.DUT.count
Depth                            | Performance Complexity Buildup
---------------------------------|-------------------------------------
-----------------
D=3..66 (9s):                    |
                                 |    [R]   [9]    testbench.DUT.count
D=2 (0s):                        |
                                 |    [R]   [1]
 testbench.my_data_is_written
D=1 (0s):                        |
                                 |    [R]   [1]
 testbench.DUT.initiator
                                 |    [G]   [1]    testbench.DUT.clk
                                 |    [G]   [1]    testbench.DUT.reset
D=0 (6s):                        |
                                 |    [I]   [9]    testbench.addr
                                 |    [I]   [1]    testbench.wen
                                 |    [G]   [1]    testbench.clk
                                 |    [G]   [32]
 testbench.expected_data
                                 |    [G]   [32]   testbench.rdata
                                 |    [G]   [1]    testbench.reset
                                 |    [G]   [9]    testbench.symb_addr
                                 |    [G]   [9]    testbench.DUT.addr
                                 |    [G]   [9]
 testbench.DUT.init_addr
                                 |    [G]   [9]
 testbench.DUT.mem_addr
                                 |    [G]   [32]   testbench.DUT.rdata
                                 |    [G]   [7]
 testbench.DUT.my_mem.addr
                                 |    [G]   [4096]
 testbench.DUT.my_mem.mem
                                 |    [G]   [32]
 testbench.DUT.my_mem.rdata
===========================================
1
```

- To get a detailed cycle-by-cycle information, including the effort information, the overall list of elements relevant for performance analysis, and all the elements in the final core, use the `-verbose` option.

  The core is explicitly reported for each depth, sorted in descending order of the depth. At each depth, elements newly introduced in the core (with respect to, the previous depth) are marked with ** to the left of the element. The elements are sorted according

Feedback

to effort, the modules with higher effort appear first, and within each module and type, signals with higher effort appear first.

The report may also include warnings that are worth considering when determining performance bottlenecks for each reported property. For example, the following report warns about a spike in the effort during the core build-up, specifically in depth 5. Other warnings may be: expensively articulated property SVA (effort greater than 5% of the total effort), a falsified or proven property during the course of applying Proof Assist's analysis, or others.

```
Proof Assist Verbose View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
;; [M-0010] Property effort is very high. Consider rewriting.
;; [M-0020] Effort spiked throughout the computation. See specific
 notes per depth(s) below.
Effort within Property SVA:
---------------------------
(99.4%)
Top Effort Signals in Last Core (Depth=66)
------------------------------------------
(0.31%) [1]    testbench.my_data_is_written
(0.15%) [1]    testbench.DUT.initiator
(0.1%)  [9]    testbench.DUT.count
Core Build-Up
-------------------------------------------
Depth 3..66 (9s):
;; [M-0021] Effort spiked at this depth. Examine the core here more
 closely.
                      (0.31%) testbench
                                          (0.31%) [R]    [1]
 testbench.my_data_is_written
                                                  [G]    [1]
 testbench.clk
                                                  [G]    [32]
 testbench.expected_data
                                                  [G]    [32]
 testbench.rdata
                                                  [G]    [1]
 testbench.reset
                                                  [G]    [9]
 testbench.symb_addr
                                                  [I]    [9]
 testbench.addr
                                                  [I]    [1]
 testbench.wen
                      (0.25%) testbench.DUT
                                          (0.15%) [R]    [1]
 testbench.DUT.initiator
```

Feedback

```
                                        **      (0.1%)  [R]    [9]
 testbench.DUT.count
                                                        [G]    [1]
 testbench.DUT.clk
                                                        [G]    [1]
 testbench.DUT.reset
                                                        [G]    [9]
 testbench.DUT.addr
                                                        [G]    [9]
 testbench.DUT.init_addr
                                                        [G]    [9]
 testbench.DUT.mem_addr
                                                        [G]    [32]
 testbench.DUT.rdata
                       (0.0%)  testbench.DUT.my_mem
                                                        [G]    [7]
 testbench.DUT.my_mem.addr
                                                        [G]    [4096]
 testbench.DUT.my_mem.mem
                                                        [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 2 (0s):
                       (67.2%) testbench
                                        **      (67.2%) [R]    [1]
 testbench.my_data_is_written
                                                        [G]    [1]
 testbench.clk
                                                        [G]    [32]
 testbench.expected_data
                                                        [G]    [32]
 testbench.rdata
                                                        [G]    [1]
 testbench.reset
                                                        [G]    [9]
 testbench.symb_addr
                                                        [I]    [9]
 testbench.addr
                                                        [I]    [1]
 testbench.wen
                       (32.8%) testbench.DUT
                                                (32.8%) [R]    [1]
 testbench.DUT.initiator
                                                        [G]    [1]
 testbench.DUT.clk
                                                        [G]    [1]
 testbench.DUT.reset
                                                        [G]    [9]
 testbench.DUT.addr
                                                        [G]    [9]
 testbench.DUT.init_addr
                                                        [G]    [9]
 testbench.DUT.mem_addr
```

Feedback

```
                                                    [G]    [32]
 testbench.DUT.rdata
                        (0.0%)  testbench.DUT.my_mem
                                                    [G]    [7]
 testbench.DUT.my_mem.addr
                                                    [G]    [4096]
 testbench.DUT.my_mem.mem
                                                    [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 1 (0s):
                        (100%)  testbench.DUT
                                    **      (100%)  [R]    [1]
 testbench.DUT.initiator
                                    **              [G]    [1]
 testbench.DUT.clk
                                    **              [G]    [1]
 testbench.DUT.reset
                                                    [G]    [9]
 testbench.DUT.addr
                                                    [G]    [9]
 testbench.DUT.init_addr
                                                    [G]    [9]
 testbench.DUT.mem_addr
                                                    [G]    [32]
 testbench.DUT.rdata
                        (0.0%)  testbench
                                                    [G]    [1]
 testbench.clk
                                                    [G]    [32]
 testbench.expected_data
                                                    [G]    [32]
 testbench.rdata
                                                    [G]    [1]
 testbench.reset
                                                    [G]    [9]
 testbench.symb_addr
                                                    [I]    [9]
 testbench.addr
                                                    [I]    [1]
 testbench.wen
                        (0.0%)  testbench.DUT.my_mem
                                                    [G]    [7]
 testbench.DUT.my_mem.addr
                                                    [G]    [4096]
 testbench.DUT.my_mem.mem
                                                    [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 0 (6s):
                        (0.0%)  testbench
                                    **              [G]    [1]
 testbench.clk
                                    **              [G]    [32]
 testbench.expected_data
```

Feedback

```
                                        **              [G]    [32]
 testbench.rdata
                                        **              [G]    [1]
 testbench.reset
                                        **              [G]    [9]
 testbench.symb_addr
                                        **              [I]    [9]
 testbench.addr
                                        **              [I]    [1]
 testbench.wen
                        (0.0%)  testbench.DUT
                                        **              [G]    [9]
 testbench.DUT.addr
                                        **              [G]    [9]
 testbench.DUT.init_addr
                                        **              [G]    [9]
 testbench.DUT.mem_addr
                                        **              [G]    [32]
 testbench.DUT.rdata
                        (0.0%)  testbench.DUT.my_mem
                                        **              [G]    [7]
 testbench.DUT.my_mem.addr
                                        **              [G]    [4096]
 testbench.DUT.my_mem.mem
                                        **              [G]    [32]
 testbench.DUT.my_mem.rdata
================================================
1
```

- To get recommendations for next steps, use the `-advisor` option. The notes are displayed for every warning in the report.

```
report_proof_assist -comb -constraints -verbose -advisor
```

```
Proof Assist Verbose View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
;; [M-0010] Property effort is very high. Consider rewriting.
;; [M-0020] Effort spiked throughout the computation. See specific
 notes per depth(s) below.
;; [L-1001] Confirm core includes only what is necessary to prove or
 falsify the property
;;          for each depth. Otherwise, this may point to design
 behavior not consistent
;;          with the verification intent. Use cut points on expensive
 variables and apply
;;          assumptions to remain within sound behavior.
;; [M-1010] Reduce the property's effort by simplifying it,
 decomposing it into smaller
;;          parts, or using native functions within the SVA.
```

Feedback

```
Effort within Property SVA:
---------------------------
(99.4%)
Top Effort Signals in Last Core (Depth=66)
------------------------------------------
(0.31%) [1]    testbench.my_data_is_written
(0.15%) [1]    testbench.DUT.initiator
(0.1%)  [9]    testbench.DUT.count
Core Build-Up
-------------------------------------------
Depth 3..66 (9s):
;; [M-0021] Effort spiked at this depth. Examine the core here more
 closely.
;; [M-1020] Focus on this core more than others, as the spike in the
 effort indicates
;;          significant complexity was introduced. See [1000] for more
 recommendations.
;;          Additionally, consider case-splitting for newly introduced
 variables in this
;;          core, and conditions leading to them. If you are focused
 on bug-hunting,
;;          consider reducing the behavioral space to deepen the reach
 of Formal.
                      (0.31%) testbench
                                          (0.31%) [R]   [1]
 testbench.my_data_is_written
                                                  [G]   [1]
 testbench.clk
                                                  [G]   [32]
 testbench.expected_data
                                                  [G]   [32]
 testbench.rdata
                                                  [G]   [1]
 testbench.reset
                                                  [G]   [9]
 testbench.symb_addr
                                                  [I]   [9]
 testbench.addr
                                                  [I]   [1]
 testbench.wen
                      (0.25%) testbench.DUT
                                          (0.15%) [R]   [1]
 testbench.DUT.initiator
                                    **    (0.1%)  [R]   [9]
 testbench.DUT.count
                                                  [G]   [1]
 testbench.DUT.clk
                                                  [G]   [1]
 testbench.DUT.reset
                                                  [G]   [9]
 testbench.DUT.addr
                                                  [G]   [9]
 testbench.DUT.init_addr
```

Feedback

```
                                                    [G]    [9]
 testbench.DUT.mem_addr
                                                    [G]    [32]
 testbench.DUT.rdata
                       (0.0%)  testbench.DUT.my_mem
                                                    [G]    [7]
 testbench.DUT.my_mem.addr
                                                    [G]    [4096]
 testbench.DUT.my_mem.mem
                                                    [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 2 (0s):
                       (67.2%) testbench
                                   **      (67.2%) [R]    [1]
 testbench.my_data_is_written
                                                    [G]    [1]
 testbench.clk
                                                    [G]    [32]
 testbench.expected_data
                                                    [G]    [32]
 testbench.rdata
                                                    [G]    [1]
 testbench.reset
                                                    [G]    [9]
 testbench.symb_addr
                                                    [I]    [9]
 testbench.addr
                                                    [I]    [1]
 testbench.wen
                       (32.8%) testbench.DUT
                                              (32.8%) [R]    [1]
 testbench.DUT.initiator
                                                    [G]    [1]
 testbench.DUT.clk
                                                    [G]    [1]
 testbench.DUT.reset
                                                    [G]    [9]
 testbench.DUT.addr
                                                    [G]    [9]
 testbench.DUT.init_addr
                                                    [G]    [9]
 testbench.DUT.mem_addr
                                                    [G]    [32]
 testbench.DUT.rdata
                       (0.0%)  testbench.DUT.my_mem
                                                    [G]    [7]
 testbench.DUT.my_mem.addr
                                                    [G]    [4096]
 testbench.DUT.my_mem.mem
                                                    [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 1 (0s):
                       (100%)  testbench.DUT
```

Feedback

```
                                    **      (100%)  [R]    [1]
 testbench.DUT.initiator
                                    **              [G]    [1]
 testbench.DUT.clk
                                    **              [G]    [1]
 testbench.DUT.reset
                                                    [G]    [9]
 testbench.DUT.addr
                                                    [G]    [9]
 testbench.DUT.init_addr
                                                    [G]    [9]
 testbench.DUT.mem_addr
                                                    [G]    [32]
 testbench.DUT.rdata
                        (0.0%)  testbench
                                                    [G]    [1]
 testbench.clk
                                                    [G]    [32]
 testbench.expected_data
                                                    [G]    [32]
 testbench.rdata
                                                    [G]    [1]
 testbench.reset
                                                    [G]    [9]
 testbench.symb_addr
                                                    [I]    [9]
 testbench.addr
                                                    [I]    [1]
 testbench.wen
                        (0.0%)  testbench.DUT.my_mem
                                                    [G]    [7]
 testbench.DUT.my_mem.addr
                                                    [G]    [4096]
 testbench.DUT.my_mem.mem
                                                    [G]    [32]
 testbench.DUT.my_mem.rdata
Depth 0 (6s):
                        (0.0%)  testbench
                                    **              [G]    [1]
 testbench.clk
                                    **              [G]    [32]
 testbench.expected_data
                                    **              [G]    [32]
 testbench.rdata
                                    **              [G]    [1]
 testbench.reset
                                    **              [G]    [9]
 testbench.symb_addr
                                    **              [I]    [9]
 testbench.addr
                                    **              [I]    [1]
 testbench.wen
                        (0.0%)  testbench.DUT
```

Feedback

```
                                        **              [G]    [9]
 testbench.DUT.addr
                                        **              [G]    [9]
 testbench.DUT.init_addr
                                        **              [G]    [9]
 testbench.DUT.mem_addr
                                        **              [G]    [32]
 testbench.DUT.rdata
                        (0.0%)  testbench.DUT.my_mem
                                        **              [G]    [7]
 testbench.DUT.my_mem.addr
                                        **              [G]    [4096]
 testbench.DUT.my_mem.mem
                                        **              [G]    [32]
 testbench.DUT.my_mem.rdata
=============================================
;; [W-0002] Advisor notes are just recommendations, and do not replace
 design knowledge nor in-depth coverage analysis.
1
```

- To get a report for specific depth(s), use the `-depth` option.

```
report_proof_assist -depth 5 -verbose
```

Alternatively, you can specify the `-last_depth` to see the information for the last depth for each data is available.

```
vcf>report_proof_Assist -depth 5 -verbose
Proof Assist Verbose View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
;; [M-0010] Property effort is very high. Consider rewriting.
Effort within Property SVA:
--------------------------
(99.4%)
Core Build-Up
-----------------------------------------
Depth 5 (0s):
                        (0.31%) testbench
                                                (0.31%)  [1]
 testbench.my_data_is_written
                        (0.25%) testbench.DUT
                                                (0.15%)  [1]
 testbench.DUT.initiator
                                                (0.1%)   [9]
 testbench.DUT.count
=============================================
1
```

Feedback

- You can zoom in on the root-cause of what is thought to be the reason for the slow-down using the `-depth <D> -comb` option, and specify a set of target registers using the `-target` option. The report then shows the relevant parts of the core for the said subset.

```
report_proof_assist -target tb_alu.alu_state -verbose -comb
 -last_depth

Proof Assist Target View
****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
;; [M-0010] Property effort is very high. Consider rewriting.
Effort within Property SVA:
---------------------------
(99.4%)
Core Build-Up
----------------------------------------------
Depth 66 (4s):
                        (0.31%) testbench
                                                (0.31%) [R]    [1]
 testbench.my_data_is_written
                                                        [G]    [1]
 testbench.clk
                                                        [G]    [1]
 testbench.reset
                                                        [G]    [9]
 testbench.symb_addr
                                                        [I]    [9]
 testbench.addr
                                                        [I]    [1]
 testbench.wen
                        (0.25%) testbench.DUT
                                                (0.15%) [R]    [1]
 testbench.DUT.initiator
                                                (0.1%)  [R]    [9]
 testbench.DUT.count
==============================================
1
```

- The results show that tb_alu.start is within the “performance complexity cone” of the said registers in this run is at depth 5, is therefore a candidate for remodeling or abstraction, if possible.

  Given the report ends in a certain less-than-desired target cycle, to help shed more light on the next cycle, use the `-coi` option. Design elements that are in the cone-of-influence of the property, but not yet in the core in said depth, will be listed as [COI].

```
report_proof_assist -verbose -comb -depth 5 -coi
```

Feedback

```
Proof Assist Verbose View
*****************************************
* Inspect other engine/model reports for a similar depth.
Property: testbench.ast_write_read_check
Model:    def
Engine:   fc1
;; [M-0010] Property effort is very high. Consider rewriting.
Effort within Property SVA:
---------------------------
(99.4%)
Core Build-Up
---------------------------------------------
Depth 5 (0s):
                       (0.31%) testbench
                                              (0.31%) [R]    [1]
 testbench.my_data_is_written
                                                      [G]    [1]
 testbench.clk
                                                      [G]    [32]
 testbench.expected_data
                                                      [G]    [32]
 testbench.rdata
                                                      [G]    [1]
 testbench.reset
                                                      [G]    [9]
 testbench.symb_addr
                                                      [I]    [9]
 testbench.addr
                                                      [I]    [1]
 testbench.wen
                                                      [Coi]
 testbench.wdata$U
                                                      [Coi]
 testbench.addr$U
                                                      [Coi]
 testbench.wdata
                                                      [Coi]
 testbench.wen$U
                       (0.25%) testbench.DUT
                                              (0.15%) [R]    [1]
 testbench.DUT.initiator
                                              (0.1%)  [R]    [9]
 testbench.DUT.count
                                                      [G]    [1]
 testbench.DUT.clk
                                                      [G]    [1]
 testbench.DUT.reset
                                                      [G]    [9]
 testbench.DUT.addr
                                                      [G]    [9]
 testbench.DUT.init_addr
                                                      [G]    [9]
 testbench.DUT.mem_addr
```

Feedback

```
                                                          [G]    [32]
 testbench.DUT.rdata
                          (0.0%)  testbench.DUT.my_mem
                                                          [G]    [7]
 testbench.DUT.my_mem.addr
                                                          [G]    [4096]
 testbench.DUT.my_mem.mem
                                                          [G]    [32]
 testbench.DUT.my_mem.rdata
=============================================
1
```

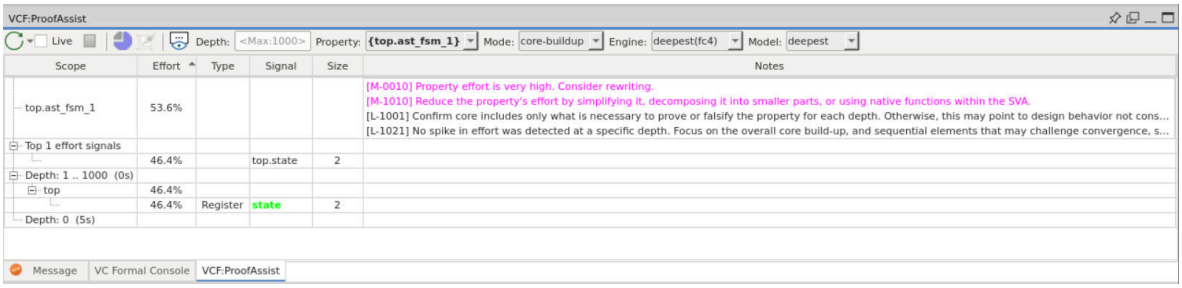
- Another example of zooming-in on specific cores is by using `-last_cores`. For example, given that there are cores available in cycles 0, 1, 3, 5, and 7, use `-last_cores 3` to get information on the last three distinct cores, that is, cores in cycles 3, 5, and 7, as follows:

```
report_proof_assist -verbose -depth 7 -last_cores 3
```

18.6.5 Proof Assist Notes

The `report_proof_assist` command generates the Proof Assist report which includes “Notes” useful to debug the performance of the given property.

The following is an example of Proof Assist report.



Notes start with an ID of the format `[T-Nnnn]`, where:

- `T` is a type, which can be a priority (H/M/L for High, Medium, or Low) or W or informative report warnings (empty report, and so on.)
- `N` is 0 or 1 for *default* notes versus *advisor* notes respectively. Advisor notes are triggered using `-advisor`, and include “next-step” actions, and some advisor notes include actual TCL code for said steps.
- `nnn` is a numeric index.

Table 33: Advisor Notes

Code	Notes	Flag Pre-requisities
0	This property has been proven during Proof Assist analysis.	default
1	This property has been falsified during Proof Assist analysis. Re-run with check_fv to get a trace.	default
10	Property effort is very high. Consider rewriting.	default
20	Effort spiked throughout the computation. See specific notes per depth(s) below.	default
21	Effort spiked at this depth. Examine the core here more closely.	default
40	Engines reached memory threshold. This can lead to incomplete or meaningless results.	default
1000	Confirm core includes only what is necessary to prove or falsify the property for each depth. Use cut points on expensive variables and apply assumptions to remain within sound behavior. Alternatively, use -minimize within compute_proof_assist to compute the minimal core (requires an expensive computation). If the latter does not remove a certain signal as expected, this may point to design behavior not consistent with the verificaiton intent.	-advisor
1001	Confirm core includes only what is necessary to prove or falsify the property for each depth. Otherwise, this may point to design behavior not consistent with the verification intent. Use cut points on expensive variables and apply assumptions to remain within sound behavior.	-advisor
1010	Reduce the property's effort by simplifying it, decomposing it into smaller parts, or using native functions within the SVA.	-advisor
1020	Focus on this core more than others, as the spike in the effort indicates significant complexity was introduced. See [1000] for more recommendations. Additionally, consider case-splitting for newly introduced variables in this core, and conditions leading to them. If you are focused on bug-hunting, consider reducing the behavioral space to deepen the reach of Formal.	-advisor

Table 33: Advisor Notes (Continued)

Code	Notes	Flag Pre-requisities
1021	No spike in effort was detected at a specific depth. Focus on the overall core build-up, and sequential elements that may challenge convergence, such as counters.	-advisor
1040	Increase engine memory allocation using fml_max_mem.	-advisor
1050	Use bug-hunting recipe:	-advisor -intent bug-hunting
1060	Snip variables:	-advisor
1070	Randomize large inputs:	-advisor -intent bug-hunting
1080	Abstract large counters:	-advisor
1090	Generate covers for counters:	-advisor -intent bug-hunting or all

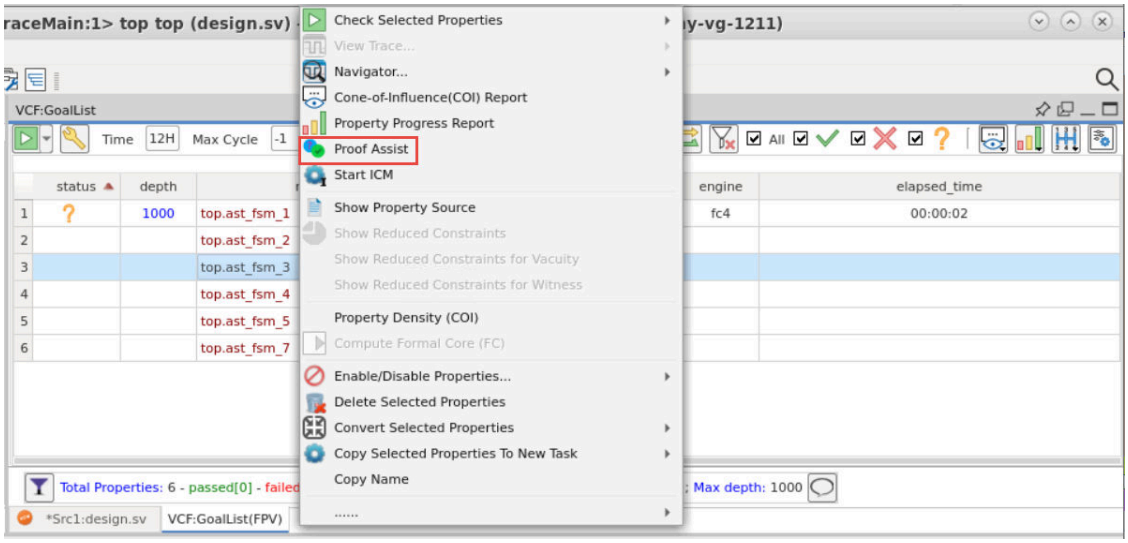
18.6.6 Computing and Viewing Proof Assist Reports in VC Formal GUI

You can compute Proof Assist from the VC Formal GUI, and view the Proof Assist reports in *ProofAssist* window.

Feedback

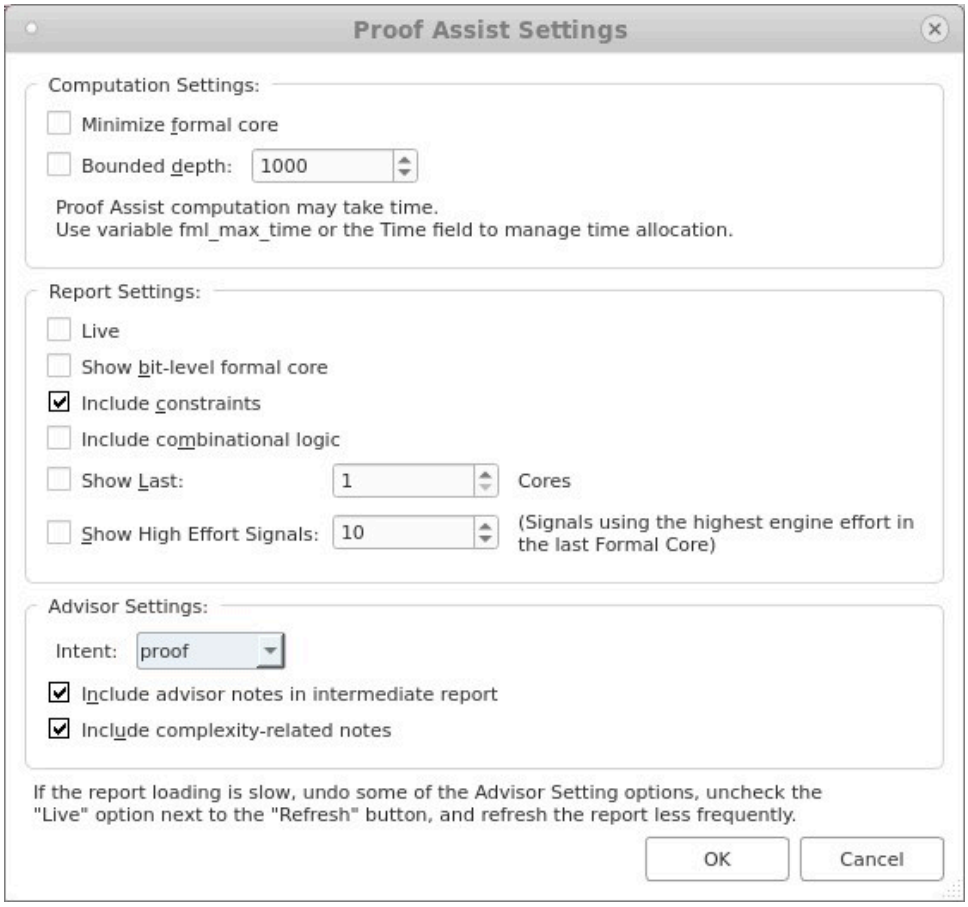
To compute Proof Assist data from the GUI and view its results:

1. Right-click on a property and select *Proof Assist*.



The *Proof Assist Settings* dialog box appears.

Feedback



Note:

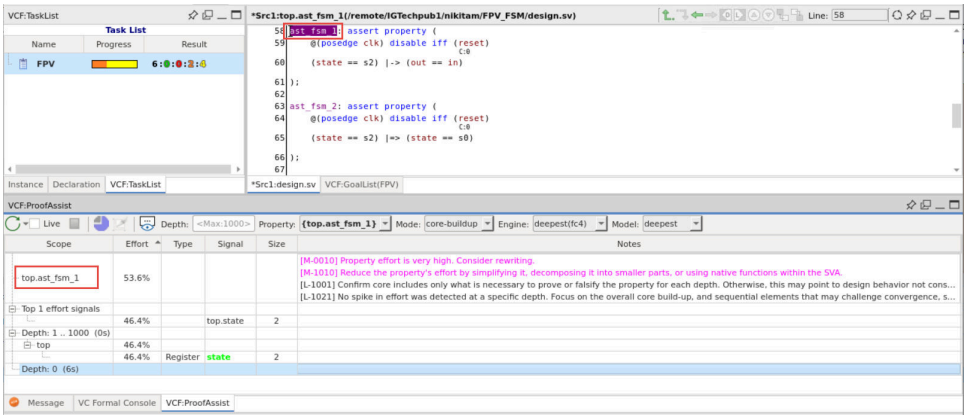
Alternatively, you can drag and drop a property from the *Source Code* view or the *GoalList* table to the Proof Assist window to automatically trigger Proof Assist computation for that property with the current settings. Dragging and dropping a constraint to the Proof Assist window has no effect.

Note:

You can drag and drop a signal from the **Source Code** view to an opened **Proof Assist** to automatically add the signal in the search bar and highlight the first matched result. You can use the search navigation arrows to continue finding further matches in the target window. Alternatively, you can

Feedback

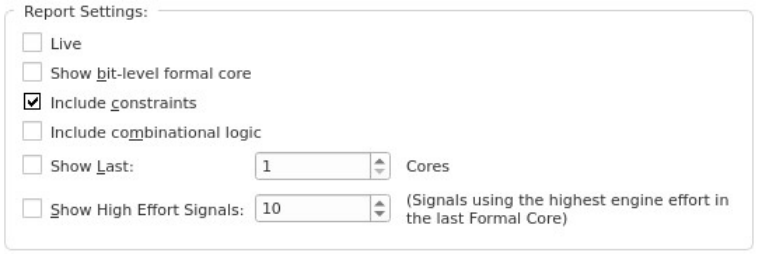
type the string in the search bar to enable search in Proof Assist windows. An example is shown in the following figure:



2. Select the *Computational Settings* option depending on what information to gather (including how to re-run).

   - To minimize formal core, select the *Minimize formal core* option.
   - Select the value to limit the reporting depth to the given value in the *Bounded depth* box.

3. Select the *Report Settings* option depending on what information to gather.



   a. To enable live update of the Proof Assist report, select the *Live* option.

   b. To view the bit-level report, select the *Show bit-level formal core* option.

   c. To include combinational logic in the report, select the *Include combinational logic* option.

Feedback

Note:

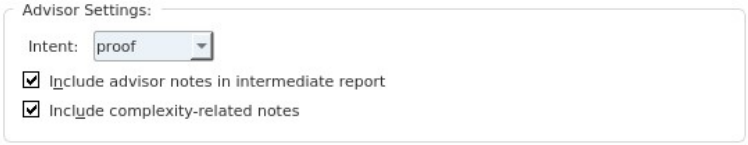
The *Show bit-level* option and *Include combinational logic* option are mutually exclusive. You can select only one of them.

d. To include constraints, select *Include Constraints*.

e. Specify the value to report the last N cores available in the *Show Last* box.

f. Specify the value to report the highest N signals in terms of effort in the *Show High Effort Signals* box.

4. Select the *Advisor Settings* option depending on what information to gather.



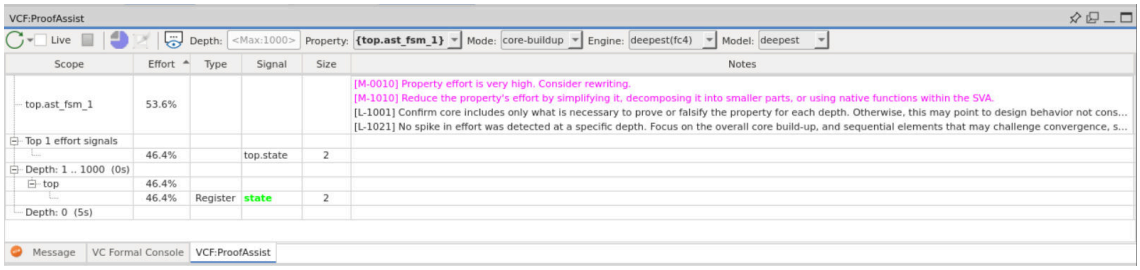
a. Select the Intent from one the following **proof, bug hunting, coverage**.

See section [-intent <intent-option>] for more details.

b. To include advisor notes in intermediate report, select **Include advisor notes in intermediate report**.

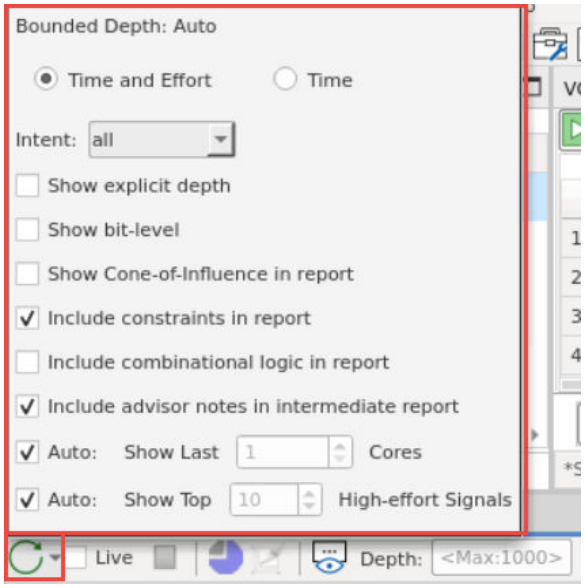
c. To include complexity-related notes, select **Include complexity-related notes**.

The Proof Assist report appears in the *ProofAssist* window as follows:



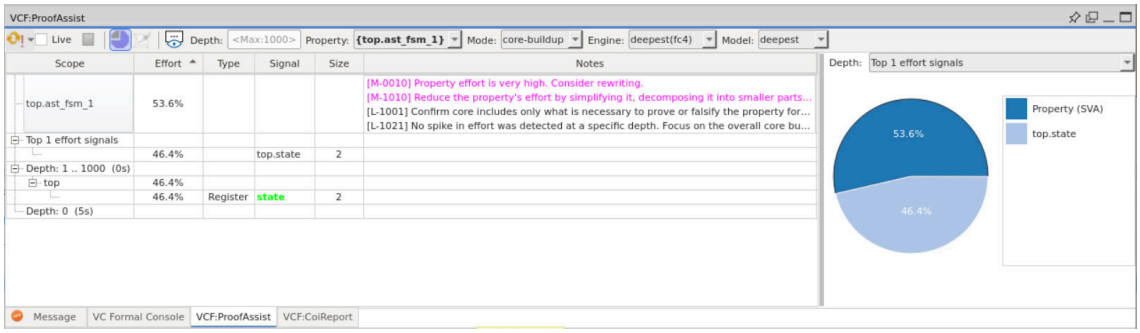
5. To review and reselect the *ProofAssist* report options, click the drop-down list.

Feedback



Select the required options from the drop-down list. The drop-down appears orange if you make changes to the options.

6. Click to view the updated report.
7. Click to view the complexity report.
8. Click to view the Proof Assist report in a pie-chart.

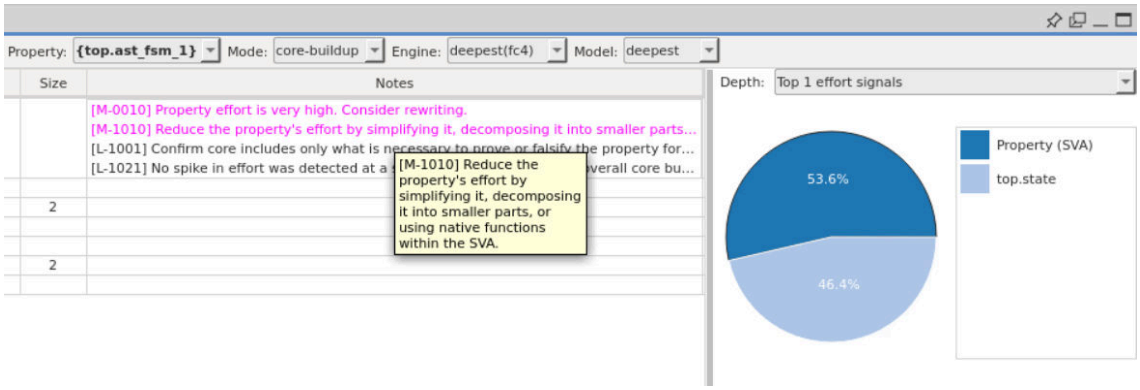


9. To get recommendations for next steps, see the *Notes* column.

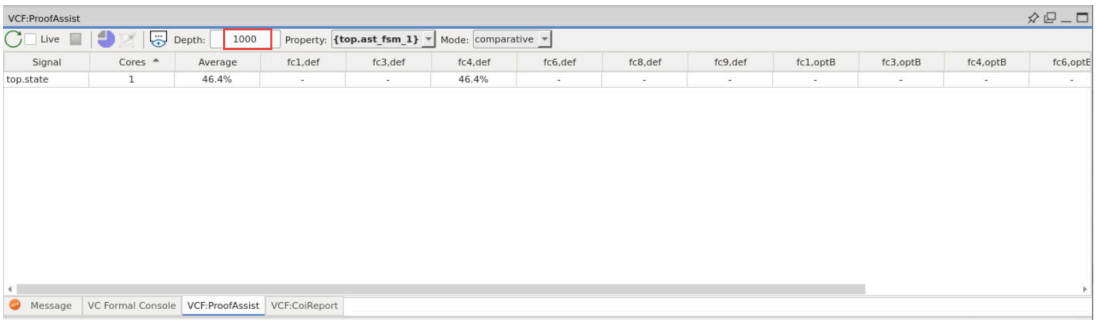
   See section Proof Assist Notes for more details on the advisor notes.

   Hovering over the *Notes* column provides recommendation for every warning in the report.

Feedback



10. From the *Mode* drop-down, select the required format of the report.
    - Select *core-buildup* to report the core buildup over cycles.
    - For selected engines, select comparative, with a given `-depth`, allows showing all relevant signals and their corresponding effort for all engines and models.



11. To filter the report based on the engines used for computation: *all*, *deepest (default)*, or *<engine name>*, select the required option from *Engine*.

    Safety properties use engines fc1, fc3, and fc4, while liveness properties use engines fc2 and fc5. Use *deepest* to focus on the reports with the deepest core.

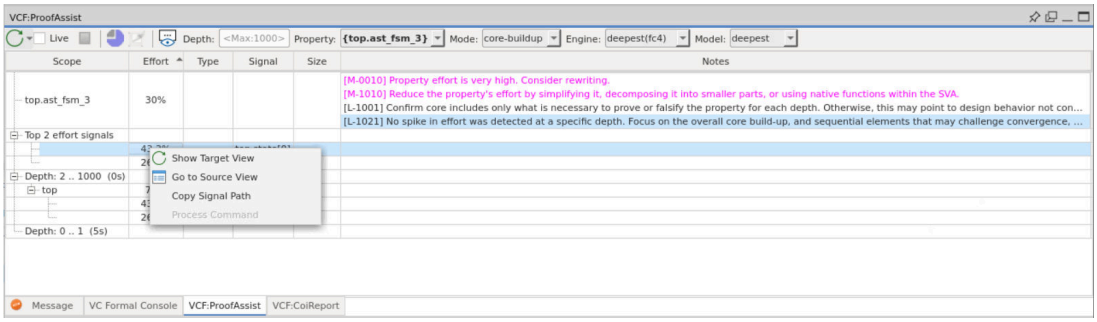
12. To filter the report based on the model applied, select the required option from *Model* list.

    Possible values: *all*, *def*, *deepest*, or *optB*.

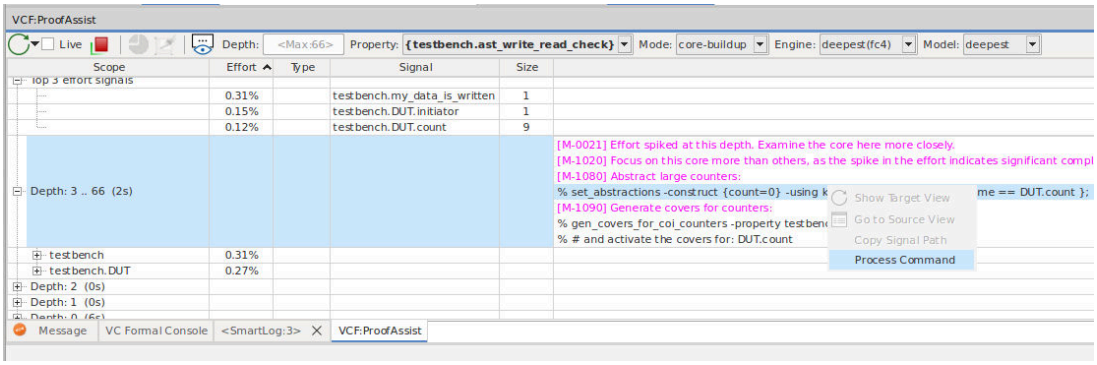
    By default, the `def` model is used which applies the formal core engines on the original design, while `optB` applies the engines on speculatively abstracted version of the design based on snipping high-effort nodes. If the analysis determines that the latter model is useful, corresponding reports will include *Note 1060* (see section Proof Assist Notes for more details).

Feedback

13. Right-click on a signal to perform any of the following:
    - *Show Target View*
    - *Go to the source view*
    - *Copy Signal Path*

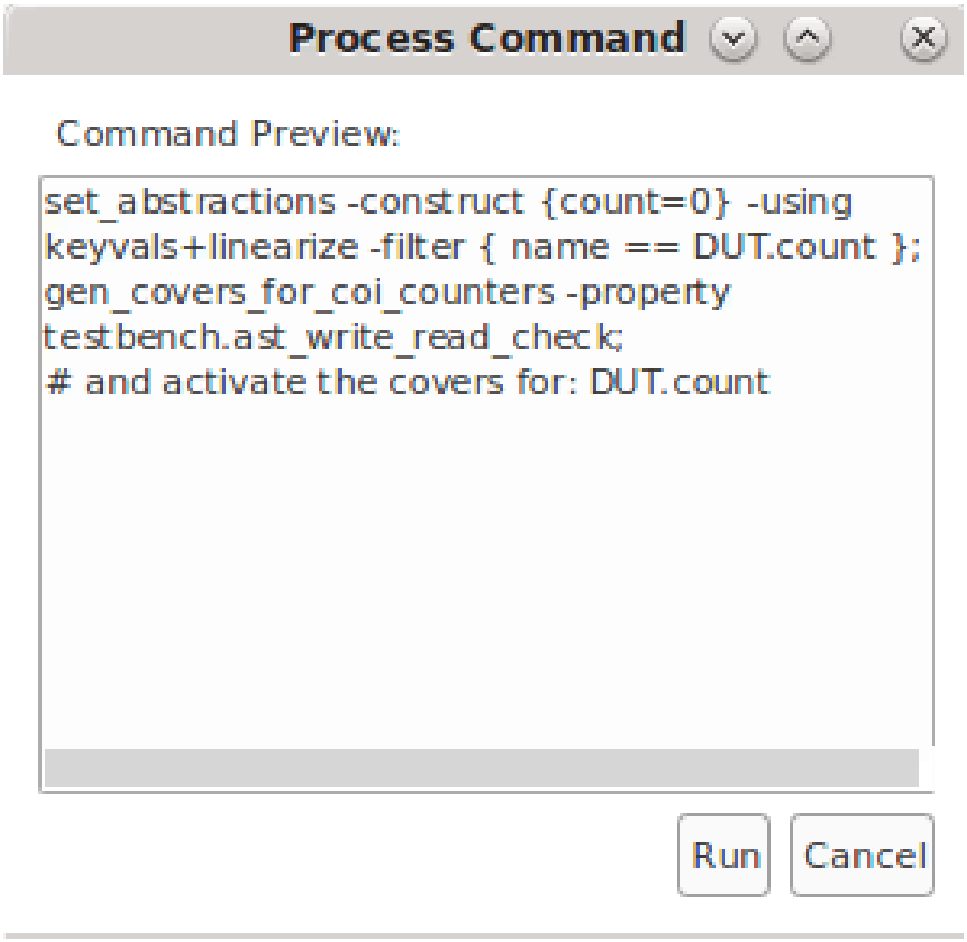


- Select *Process Command* option from the RMB menu to directly run recommended commands.



Feedback

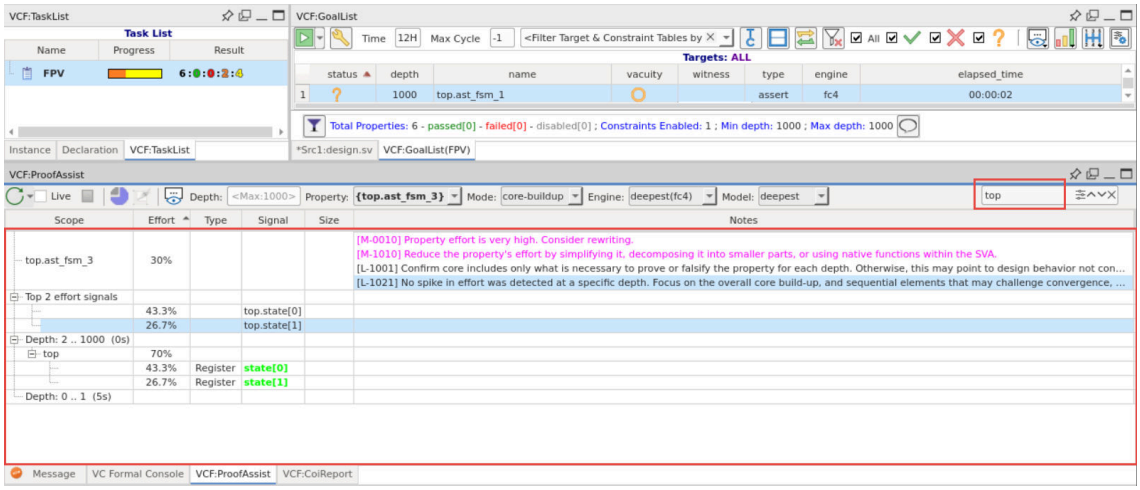
The *Process Command* dialog provides editable command preview to confirm and run the recommended commands directly from the *Proof Assist* report.



14. You can view size of the signals to know its complexity in the *Size* column.

Feedback

15. In the Proof Assist GUI, you can search for signals in the search bar and use the button to view the next or previous result as shown in the following figure:



18.7 Iterative Convergence Methodology

Iterative Convergence Methodology (ICM) is a tool which assists in the creation of helper properties (invariants) to help converge on hard properties. An invariant is a property which, once proven, can be safely used as a constraint to reduce the state space the tool needs to explore. Invariants can be very powerful but thinking of the right invariants is often a difficult task. ICM provides hints to the user about which areas of the design would be helpful targets for invariants.

In ICM, VC Formal displays counterexamples for hard properties that do not start from the reset state. These are called step counterexamples. The step counterexamples show scenarios that may be illegal, which cannot occur for the given setup. You must debug these step counterexamples and come up with helper assertions that exclude such scenarios. You can also control the length of the step counterexamples. The intent here is to find assertions which, if they were constraints would stop the behavior you see in the step counterexample from occurring. Really this is just a way to build relationships between signals at different stages of the proof depth.

Once you add a sufficient set of helper assertions, the property is likely to become easy for solvers to prove. You can also decide to stop the ICM task at any point, import all of the helper assertions to the main FPV task, and run various orchestration effort levels.

Feedback

Note:

Currently, the ICM feature is only supported in VC Formal FPV for safety properties,

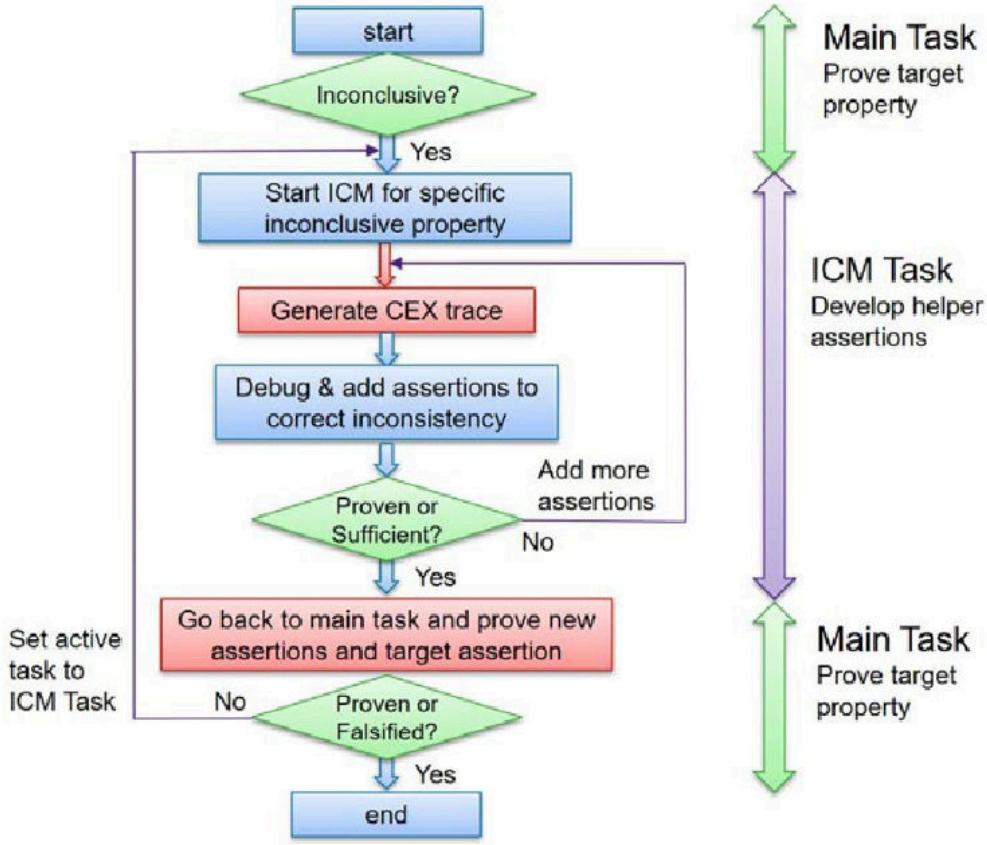
Note:

The ICM feature is only supported in VC Formal FPV and VC Formal FRV.

18.7.1 ICM Debug Flow

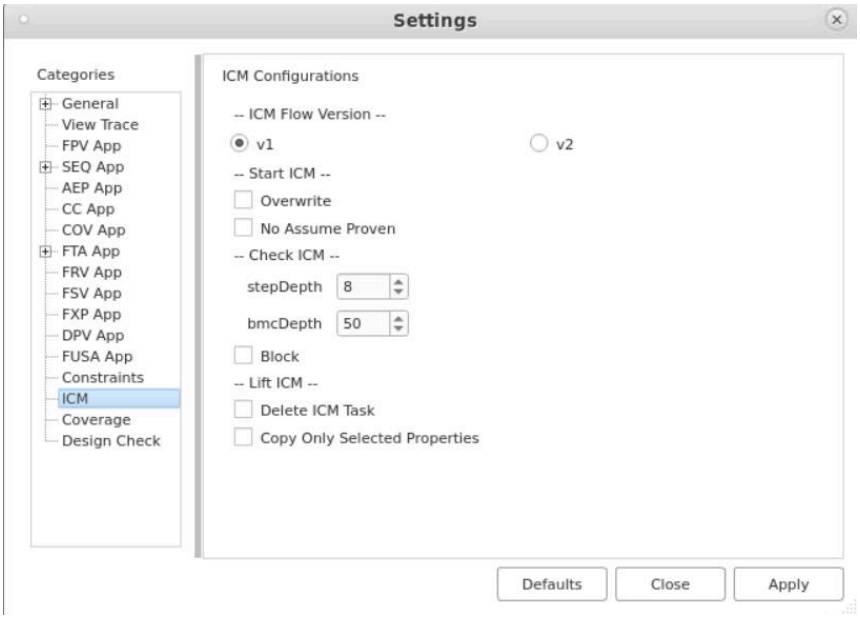
Figure 146 shows the ICM debug flow in VC Formal.

Figure 146: ICM Debug Flow



18.7.2 Configuring ICM View Settings

Configure the following settings before you perform the ICM check.



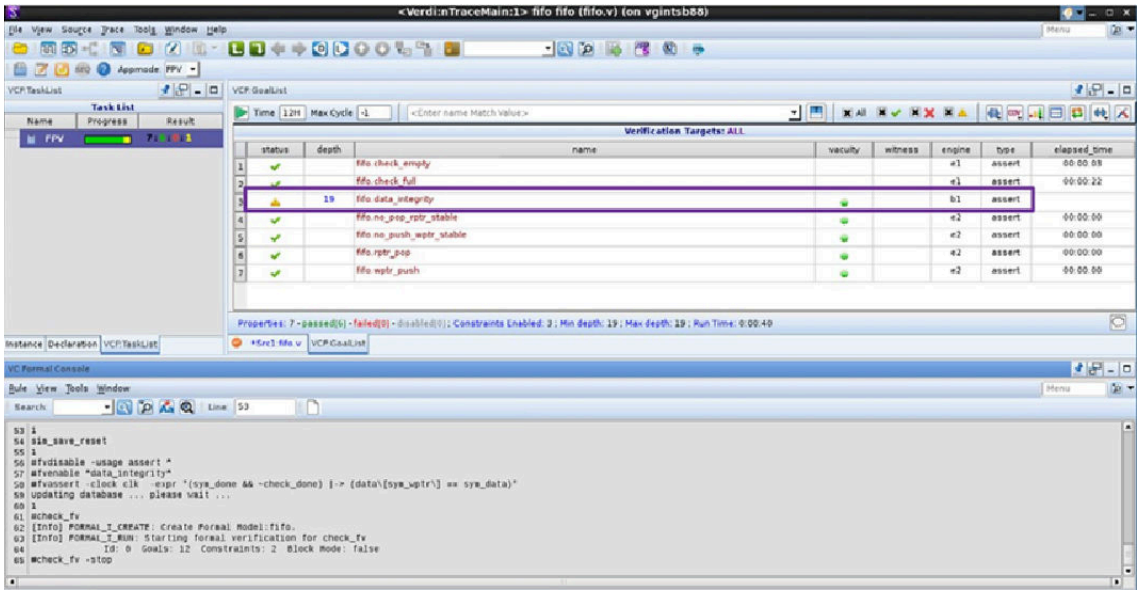
- Select the required **ICM Flow Version**:
  - Flow version **v1** (default) runs both the step engine up to **stepDepth** and the BMC engine up to **bmcDepth**.
  - Flow version **v2** runs both the step engine up to **stepDepth** and the default effort level orchestration. In flow version **v2**, the BMC engine used with ICM does not run, and the **bmcDepth** is ignored.
- **Overwrite**: Delete the existing ICM task with the same name. By default, they are not deleted.
- **No Assume Proven**: Do not make the proven properties as assumptions from the FPV task in the ICM task. By default, the proven properties of FPV task are made as assumptions in the ICM task.
- **stepDepth**: Specify the depth of step counterexamples. By default, the **stepDepth** is set to 8.
- **bmcDepth**: Specify the depth for BMC engine to find the real failures in helper assertions, when running the ICM check. By default, the **bmcDepth** is set to 50.
- **Block**: Select this option to run ICM in blocking mode.
- **Delete ICM Task**: Delete the ICM task after moving the helper assertions to the FPV task.
- **Copy Only Selected Properties**: Select this option to copy only the selected properties to the FPV task. By default, this option is disabled, and all not failing helper assertions are copied to the FPV task.

Feedback

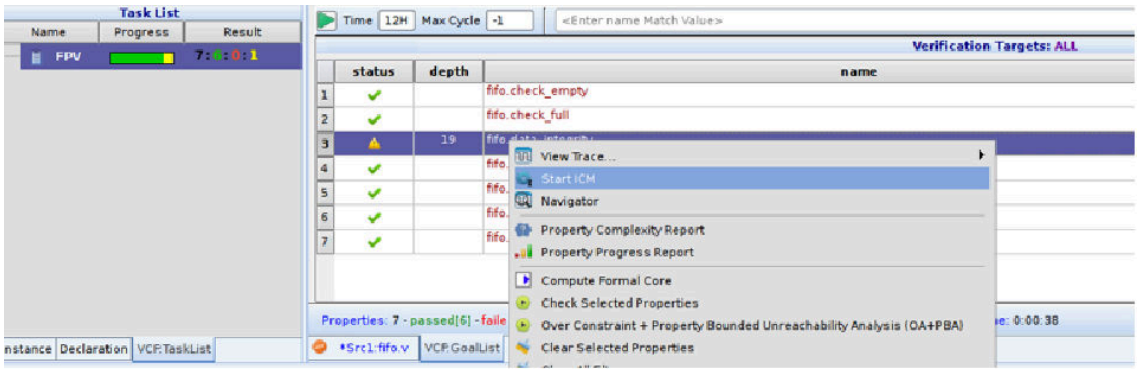
To perform ICM in the VC Formal GUI:

1. Click to run the FPV task to get proven properties, bounded proofs, and inconclusive properties.

   The status of the properties appears in the *Status* column as shown in the following image.



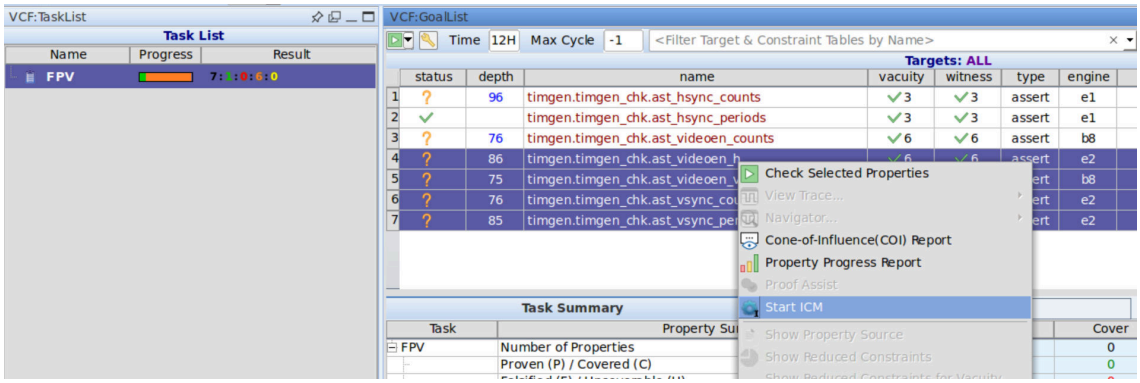
2. Right-click on the inconclusive property, and from the RMB menu, select *Start ICM*.



   Or, if multiple assertions are inconclusive, and you would like to apply ICM on multiple assertions, select multiple assertions, and from the RMB menu, select *Start ICM*.

   Note:

   Liveness properties are not supported with ICM.

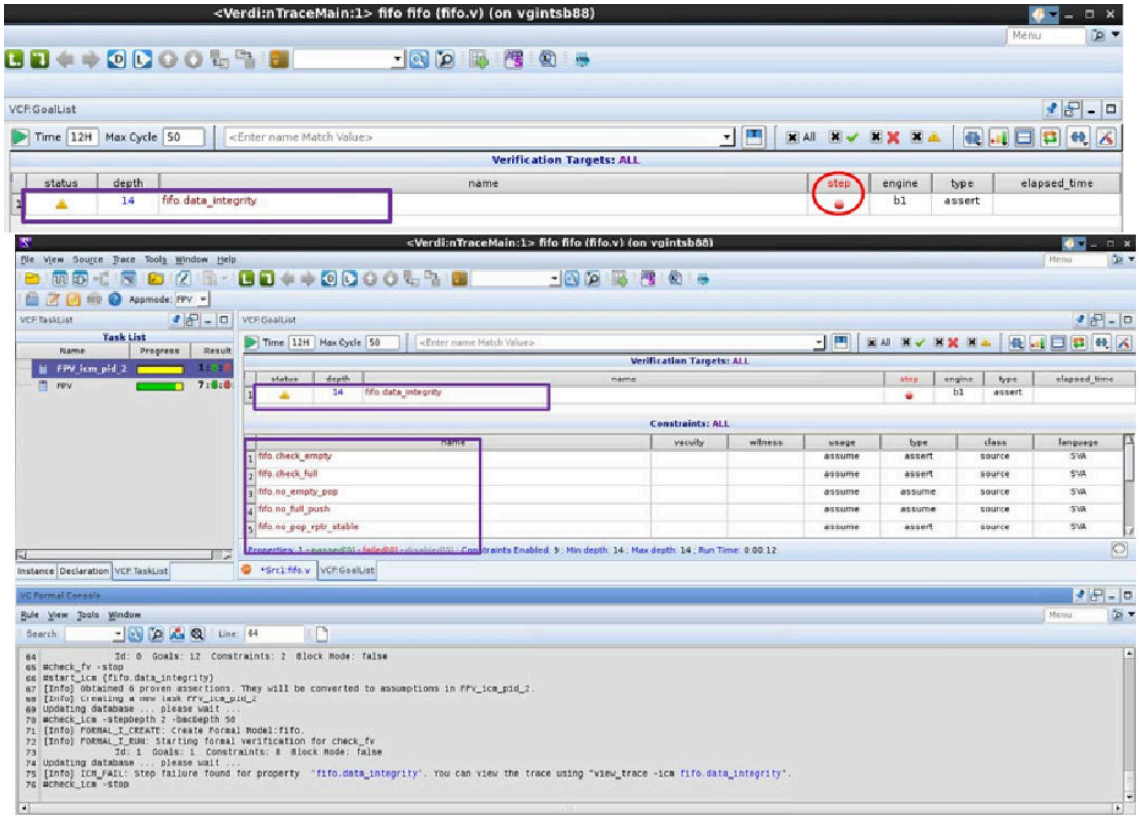


A new verification task is created with the selected inconclusive property. For example, the FPV_icm_pid_2 task is created where the `fifo.data_integrity` is listed as an inconclusive property, and the proven properties of the FPV task have become assumptions in the FPV_icm_pid_2 task by default.

Note:

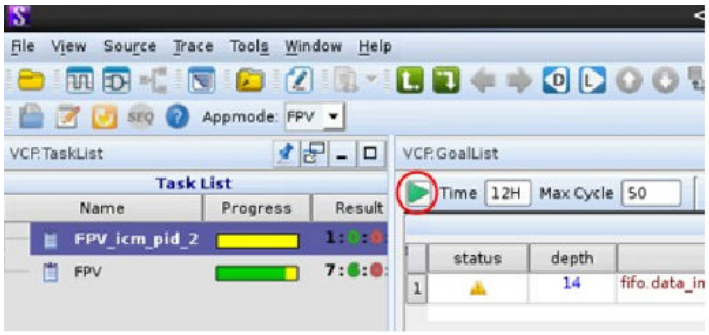
When Proof Architect is enabled, the ICM task is created under a **Copy** Proof Strategy, to allow seamless propagation to the parent task. For more information on Copy Proof Strategy, see section Copy.

Feedback



In the ICM task you see that we have both a step column and a status column. The step column indicates if we have reached a step failure that we want to debug to help guide with invariants. If however you see a failures in the status column, this indicates that the property has failed from the standard initial state. ICM always also runs a BMC engine from reset, in addition to the step CEX to see whether properties can have real failures. This is useful on the helper properties, as a failing helper property cannot be used.

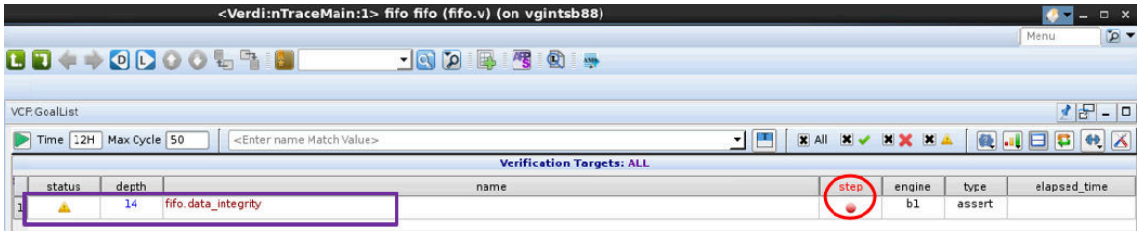
3. Click to run the ICM task.



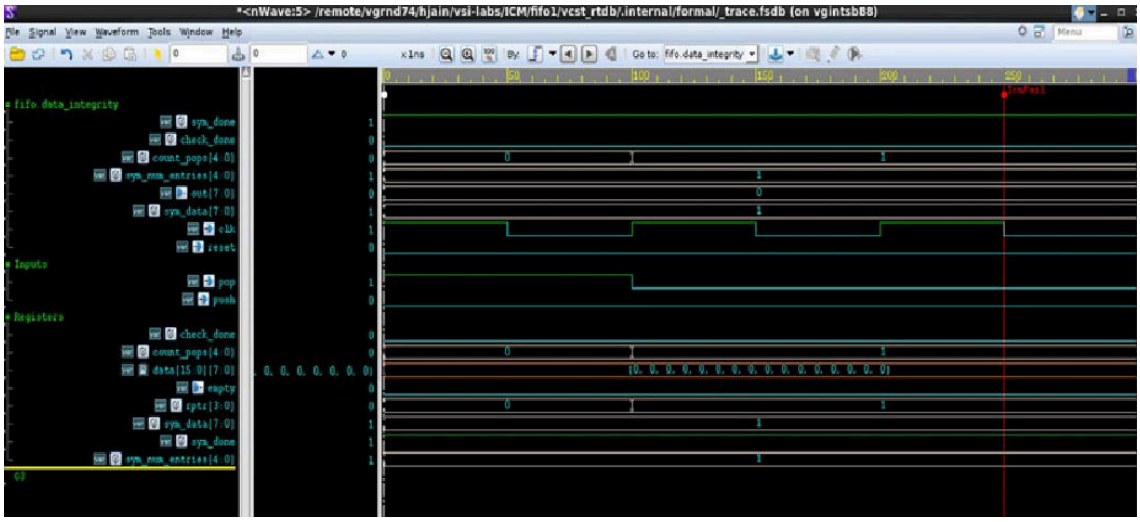
Feedback

4. Double-click the icon to trace the step failure for the ICM Task.

   It is possible that one of the helper assertions may fail through the BMC check. These real failures show up in *status* column. When the real failures show up, they need to be debugged first. The outcome of this is to remove the helper assertion or rewrite it.



   The trace for the step failure appears as shown in the following image. The `FIFO data_integrity` does not contain `sym_data`.



5. Write helper assertions to express why this scenario in step trace is not possible.

   In the FIFO example, the step failure occurs because the `sym_data` is written to FIFO. However, the `sym_data` is not present in the FIFO data array. This is an illegal scenario that must not happen. Add the following helper assertion to note that `sym_data` exists in FIFO data array at index `sym_wptr`.

```
fvassert -expr " ... -> data[sym_wptr] == sym_data"
```

6. Repeat steps 2 to 5 until the property is proven.
7. Copy all the helper assertions to the parent FPV task.
8. Run the FPV task for the selected property to see if it is proven with the helper assertions.

18.8 Using Proof Architect for Faster Convergence

You can leverage Proof Architect to converge on complex verification problems that benefit from abstraction, constraint manipulation, divide-n-conquer, and other advanced modeling and engine tuning approaches. You can create different Proof Strategies through Proof Architect to define the logical relation between tasks. Proof Architect uses these logically connected tasks to perform modeling changes to speed up performance, without losing or mixing up the already confirmed results, and without the need of manual book-keeping across tasks.

For more details on the benefits of Proof Architect and how to apply different Proof Strategies, see section Compositional Verification Using Proof Architect.

Note:
Proof Architect is currently supported only in the VC Formal FPV application.

19

Compositional Verification Using Proof Architect

Compositional verification can go a long way towards formally verifying hard goals. Combined with scalable management of thousands to tens of thousands of workers through the cloud or other scalable infrastructures, compositional verification and parallelism is the new Moore's Law in the face of the exponential increase of complexity of blocks and formally verifying them. Compositional verification allows scaling to much larger designs that would otherwise fail when verified directly without such methodology.

Proof Architect, a new feature in VC Formal, enables compositional verification by replacing the brute-force formal checking of hard goals with verifying abstracted, partitioned, or decomposed goals, along with enabling automatic reasoning of the original goals. Proof Architect uses logically connected tasks to perform modeling changes to speed up performance, without losing or mixing up the already confirmed results, and without the need of manual book-keeping across tasks.

Note:

Proof Architect is currently supported only for the VC Formal FPV and VC Formal FRV applications. Proof Architect is a Limited Customer Availability (LCA) feature. Contact Synopsys support for more information.

To converge on hard properties, model changes such as abstraction (through snipping), addition or removal of constraints, or addition of helper properties, are often required. Determining the type of modeling change required can be done through deep understanding of the RTL, including diagnosing:

- The overall complexities of the design or the COI of the property using Complexity Report (see section Performing Complexity Analysis in Tcl).
- Engine performance using Proof Assist (see section Support for Proof Assist).
- Induction convergence though ICM (see section Iterative Convergence Methodology), or other means.

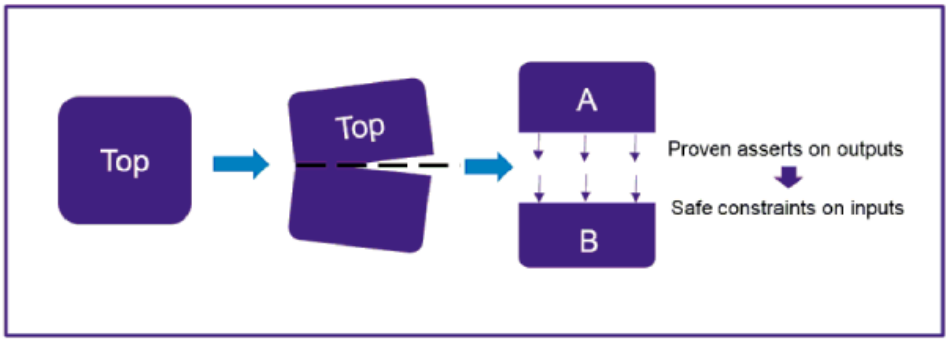
Regardless of the approach used, when you apply these model changes on the design, the status of the already converged properties are no longer valid. In the case of snipping this happens because abstraction relaxes the model and adds more behavior; therefore, a proof that used to hold true before, does not hold true after abstraction, and the proven results are removed and recalculated based on the updated model. Dually, when you add a constraint, the trace results or the falsifications are cleared and recalculated. Many times

Feedback

both modeling operations have to be applied, in which case *all* results are reset. This can lead to major loss of productivity for any realistic design, and is highly error-prone.

To overcome iterative aggregation of results and various sessions to achieve complex performance improvements, VC Formal allows the use of Tasks (see section Creating Verification Tasks), which provide a safe “context” for modeling operations. Using derived tasks, you can maintain multiple models in parallel, without needing to erase past results within the main task. However, coordination of results for related properties across tasks is still needed, along with safe-guarding against certain actions that can affect soundness, and automatically deploying sophisticated divide-n-conquer strategies.

Proof Architect provides an automated way of creating separate tasks, synchronizing, and propagating the model changes hierarchically across tasks, such that only sound results are propagated to the main goal. Proof Architect enables you to try different modeling heuristics while still not worrying about the bookkeeping across the different tasks. Note that, through Proof Architect, the tool ensures that the irrelevant or unsound results are not propagated, and enforces consistent modeling and prevents human induced errors.



Additionally, Proof Architect facilitates advanced proof techniques such as Assume Guarantee and Case-Splitting. Assume Guarantee enables you to define a chain of dependencies between sets of properties. This is useful when you partition the properties within the same module, and define the order in which they should be solved to speed up the total run-time. It also as been used for hierarchical verification across modules, through an assume/assert relation on the interface.

Proof Architect Approach

Proof Architect defines the logical relation between tasks through Proof Strategies. A Proof Strategy within a given task defines how child tasks are related to a parent task enabling the tool to co-ordinate the statuses across tasks.

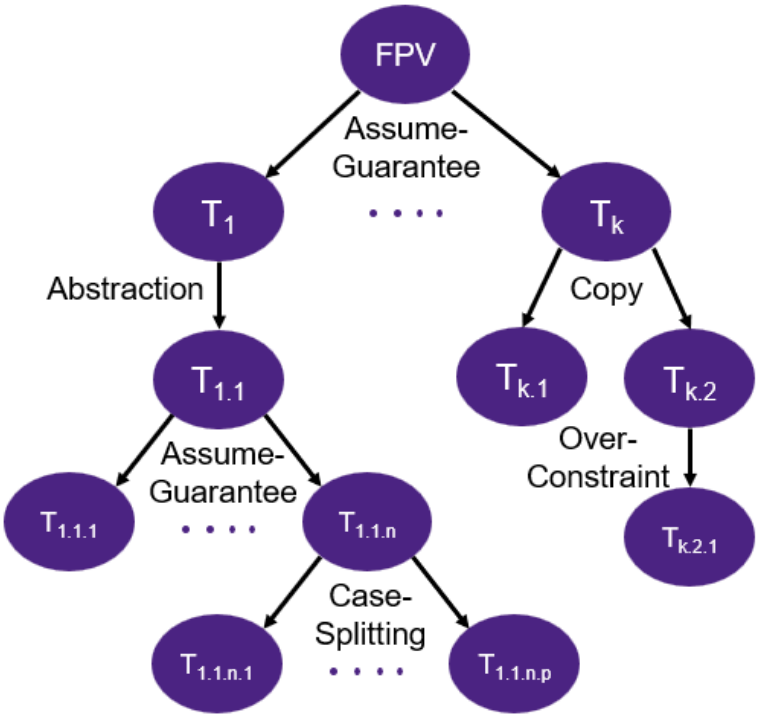
Proof Architect provides the following Proof Strategies to apply modeling changes:

- Copy
- Abstraction

Feedback

- Over-Constraint
- Assume-Guarantee
- Case-Split

Figure 147: Proof Strategies Types and Tasks



Demo: Benefits of Applying Proof Architect on a DMAC Design

Watch this video to explore how Proof Architect accelerates performance and verification of a DMAC IP design with over 1000 properties to converge.

19.1 Advantages of Proof Architect

Well-behaved Decompositions With Model Reference

Proof Architect ensures that the modeling changes across Proof Strategies when the main task is decomposed into smaller tasks are propagated accurately. These well-behaved decompositions ensure that parent-child task relationships are honored without compromising flexibility. You can define different Proof Strategies and rely on the tool

to enforce soundness, preventing conflicts caused by commands that alter the model inappropriately.

In summary, through well behaved decompositions, you can

- Enforce task relationships automatically
- Maintain flexibility in using commands within child tasks.
- Prevent conflicts and ensure soundness.

Multi-layered Hierarchical Propagation and Verification

Multi-layered hierarchical propagation allows recursive propagation and verification across multiple layers of child tasks. Proof Architect enables a seamless verification experience across the multiple layers of child tasks through well behaved decompositions.

Compliance with General Verification Tasks

Enabling Proof Architect does not impact the usage of the other general verification tasks that are not logically connected. You can continue using the other general verification tasks within VC Formal even when Proof Architect is enabled using the `fv_proof_architect` formal variable. See section Creating Verification Tasks for more details.

Auditing Proof Strategies

Though modeling commands can be used freely across Proof Strategies, the modeling delta (that is, the modeling change across time) is examined when Proof Architect verification is launched through `check_fv`. An inconsistent modeling across tasks blocks Proof Architect verification. For example, adding constraints within a model that was intended for abstraction (snipping) is not allowed by default. Nonetheless, Proof Architect does provide the flexibility to bypass the modeling safe-guards by marking the Proof Strategy as unsafe and proceeding with the verification, with or without status propagation based on your judgment.

See section Speculative Tasks Marking for more details on how to mark tasks unsafe and enable/disable propagation.

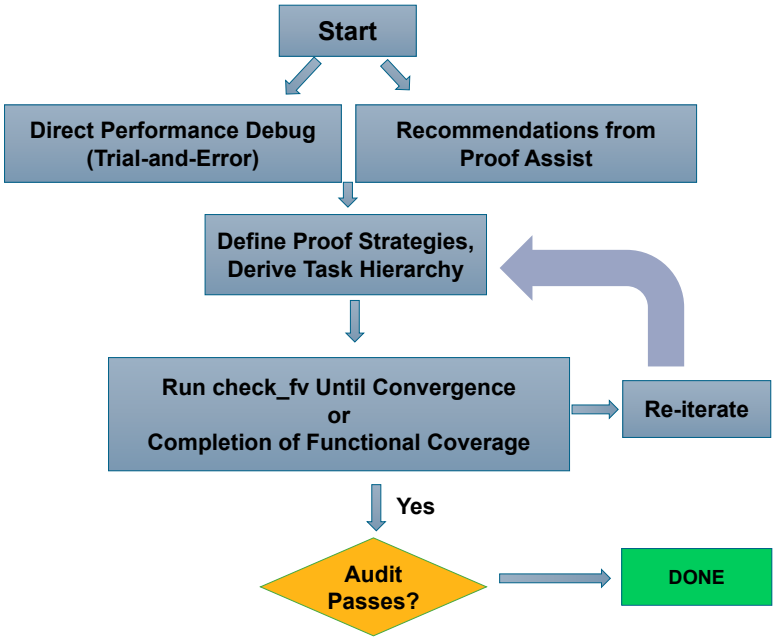
You can also view and identify the modeling delta across the hierarchy for your reference. See section Verifying Modeling Consistency and Soundness for more details.

With multiple layers of child tasks, Proof Architect enables you to audit the hierarchy for model consistency to ensure that during final verification all the unsafe markings or tasks that needed attention can be easily identified and corrected.

Feedback

During verification using Proof Architect, you can use either of the following flows:

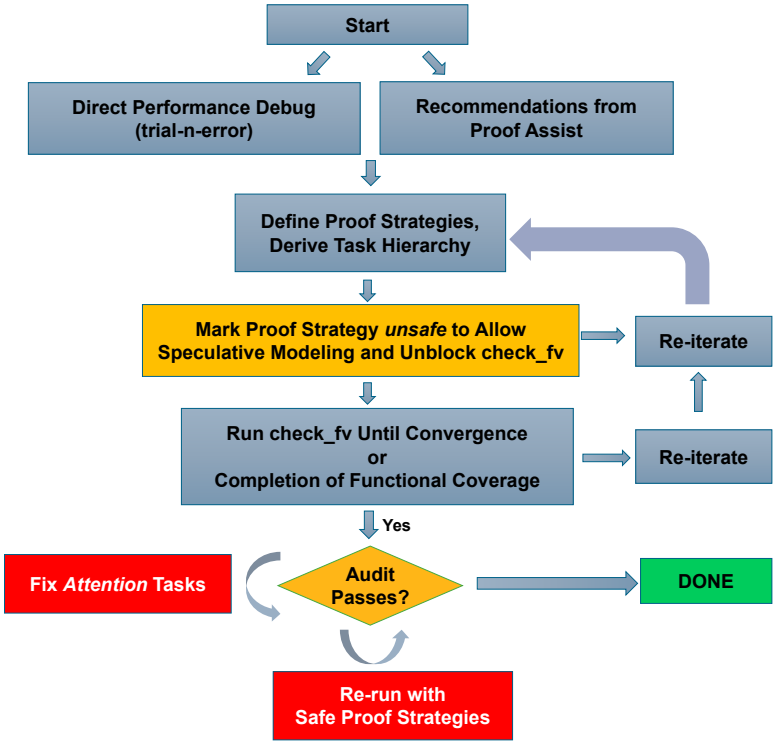
- In a typical flow with no exceptions or errors from `check_fv`, there is no need to mark a Proof Strategy as unsafe. You can continue verifying using Proof Strategies until convergence or functional coverage, and then perform audit using `fv_proof_arch -audit`.



- In advanced usage, a need may arise for deviating from the standard semantics of Proof Strategies. For example, if you are interested to speculatively perform an abstraction operation within a Copy Proof Strategy. In this case, you can mark the Proof Strategy as unsafe (Speculative Tasks Marking), to indicate a conscious decision to make an exception. This will unblock `check_fv`, but you should eventually

Feedback

reexamine all the relevant tasks and Proof Strategies to pass the audit, and be assured of a fully sound result.



19.2 Proof Architect Usage Scenarios

Proof Architect can be used in a number of scenarios, including:

- *Decomposition of goals*: Useful when there is a known ordering in the complexity of properties. In this case, it is possible to assume certain goals while verifying others, as long as the overall composition in the original set guarantees accuracy. This can be done using an Assume-Guarantee Proof Strategy. It is also possible to categorize the properties and prove them in a certain order, using a Copy Proof Strategy, or to split individual goals to sub-goals using Case-Splitting.
- *Hierarchical verification using assume-guarantee:* Useful when a decomposition across the hierarchy requires a test plan to be created for integration, in which certain attributes can be assumed on inputs while verifying properties, and then guarantee the attributes in a larger context. For example, interface protocol SVA properties can be *guaranteed* through assertions in one task, and assumed in another task while verifying the functionality of the DUT.

- *What-if analysis without changing the model*: Useful when different solving strategies are to be tried, on the whole set (or subset) of the properties, without changing the model, such as trying different orchestrations or engines.
- *Remodeling*: Useful when abstraction or over-constraining can help converge the property. This usually arises during deep performance analysis for hard properties, after diagnosis using Complexity Report, Proof Assist, or ICM.

19.3 Types of Proof Strategies

The Proof Architect enables you create the following types of Proof Strategies for faster verification.

- Copy
- Abstraction
- Over-Constraint
- Assume-Guarantee
- Case-Split

19.3.1 Copy

A Proof Strategy of type `copy` allows passing on all (inconclusive) properties, or a subset of them, to a child task, without any modeling changes in the environment (such as constraints, abstraction, snips, constants, change at), to explore different solving heuristics.

In the copy method, the child tasks promote (or propagate) all status changes from the child tasks to parent tasks automatically and without any change. For example, (bounded) proofs in the child propagates to (bounded) proofs in the parent, and falsifications, alongside their traces, also propagates without a change to the parent.

Table 34: Copy Proof Strategy Propagation Rules

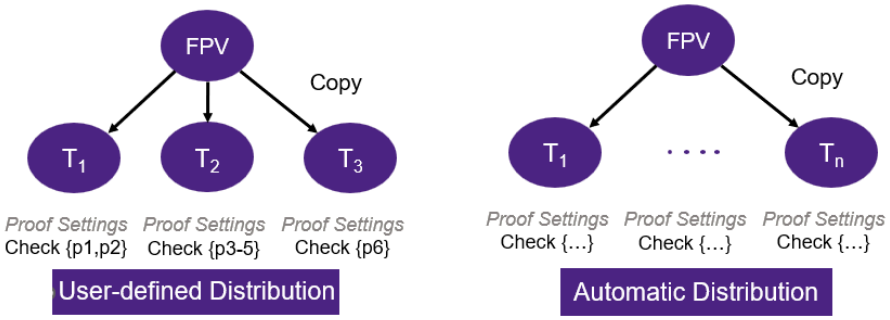
Status	Propagated	Condition(s)
Proven Unreachable	Yes	n/a
Falsified Covered	Yes	n/a
Inconclusive	Yes	n/a

Feedback

Note that due to its simplistic model, Copy Proof Strategy is used as a basis of a number of other functionalities within VC Formal, such as, ICM. Particularly, when Proof Architect is enabled and an ICM task is being generated for one or more properties, a Copy Proof Strategy is created automatically, so that the results can propagate to the parent seamlessly. Note also that within the ICM task itself, no Proof Strategy is allowed to be created. For more information on ICM, see section Iterative Convergence Methodology.

With the copy Proof Strategy, you can either distribute the tasks automatically or define custom distribution across tasks.

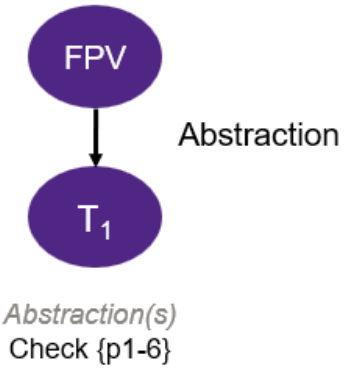
For example, you copy properties p1-p6 from original task, and create one or multiple new tasks using Copy Proof Strategy with user-defined distribution (T1, T2, T3) or automatic distribution (T1, ..., TN).



19.3.2 Abstraction

Use the Abstraction Proof Strategy when you want to snip, black-box, or abstract elements in the COI of target properties to limit state-space and reduce complexity of verification problem, thus increasing probability to converge.

For example, you can add abstractions, and check properties p1-p6 only from original task in a new task using Abstraction (T1) Proof Strategy.



Feedback

In the new task T1, you can snip variables, remove constraints, and so on. Note that proofs and safe depth information will propagate from T1 to FPV, while traces and falsified status will not.

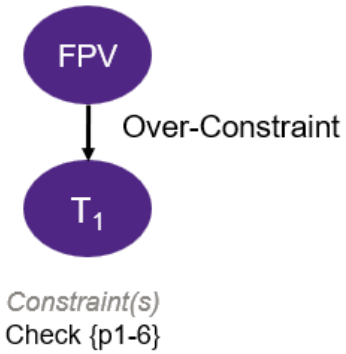
Table 35: Abstraction Proof Strategy Propagation Rules

Status	Propagated	Condition(s)
Proven Unreachable	Yes	n/a
Falsified Covered	No	n/a
Inconclusive	Yes	n/a

19.3.3 Over-Constraint

Use the Over-Constraint Proof Strategy when you want apply *strong* constraints to limit state-space and reduce complexity of Verification Problem, thus increasing probability to find deep traces.

For example, you can apply constraints, and check properties p1-p6 in a new task using Over-constraint (T1) Proof Strategy.



In the new task T2, you can add constraints, add constraints, remove snips, and so on. Note that traces and falsified status will propagate from T1 to FPV, while proofs and safe depth information will not.

Table 36: Over-Constraint Proof Strategy Propagation Rules

Status	Propagated	Condition(s)
Proven Unreachable	No	n/a
Falsified Covered	Yes	n/a
Inconclusive	No	n/a

19.3.4 Assume-Guarantee

A Proof strategy of type `assume_guarantee` enables you to define a chain of dependencies between sets of properties. This is useful when you partition the properties within the same module, and define the order in which they should be solved to speed up the total run-time.

When tackling complex properties in formal verification, a two-part proof strategy can be highly effective. This Proof Strategy leverages the concepts of "Assume" and "Guarantee" to manage complexity and ensure the soundness of proof results:

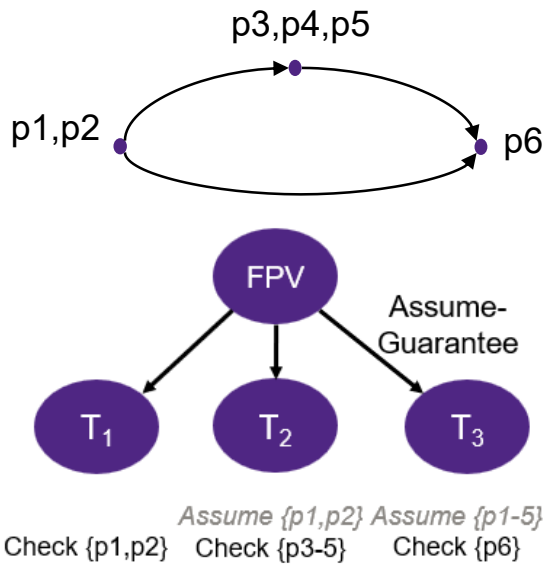
1. Assume: By applying constraints to the Cone of Influence (COI) of the properties, the state space is restricted, enabling faster proofs. These constraints act as assumptions that simplify the verification process by focusing only on the relevant portions of the design.
2. Guarantee: To ensure the validity and reliability of the proof results, the constraints used in the "Assume" phase are subsequently proven to be valid. This step guarantees that the assumptions do not introduce inaccuracies or false proofs into the verification process.

There are three types of Assume-Gaurantee Proof Strategies- Chain, Acyclic, and Grouped Induction

- **Chain**

  In this approach, you can create a dependency graph to infer relationships between properties without explicitly listing assumptions. Tasks are generated based on the graph's structure, making it ideal for straightforward dependencies.

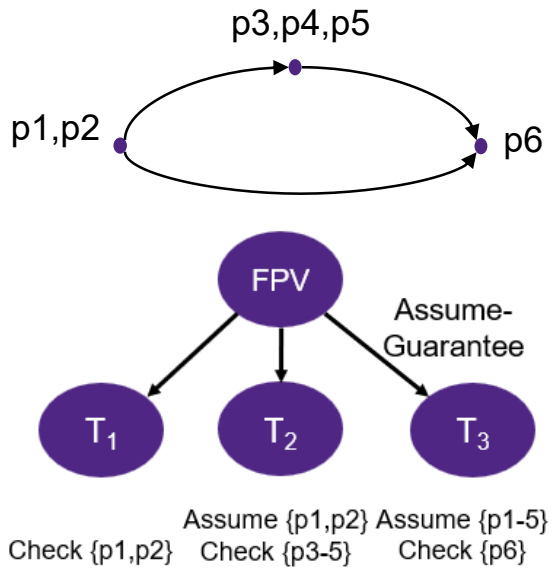
  For example, you can divide the original collection of inconclusive properties into three sets - T1, T2, and T3, and solve them in order, rather than solving them together; that is, solve {p1,p2}, solve {p3-p5} while assuming {p1,p2}, and solve {p6} while assuming {p1-5}.



Note that Assume-Guarantee acts logically like an over-constraint. Therefore, traces and falsifications propagate to the parent. However, proofs and safe depth information propagates only if all assumed asserts in the task are fully proven.

- **Acyclic**

  In this type, you can explicitly define assumptions for each property while creating the dependency graph. This approach offers precise control over property relationships.

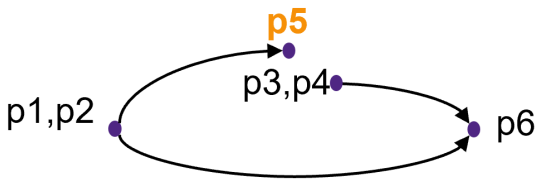


Feedback

The checked properties are not required to serve as assumptions for other properties when the following validation rules are met:

- There should be no cycles.
- Each property must have a designated "Source."
- A property must not be assigned to more than one task.

For example, Property p5 is checked using {p1,p2) as assumes BUT not used as assume to check p6



Acyclic Assume-Guarantee behaves like Chain with respect to propagations; that is, traces and falsifications propagate to the parent, while proofs and safe depth information propagates only if all assumed asserts in the task are fully proven.

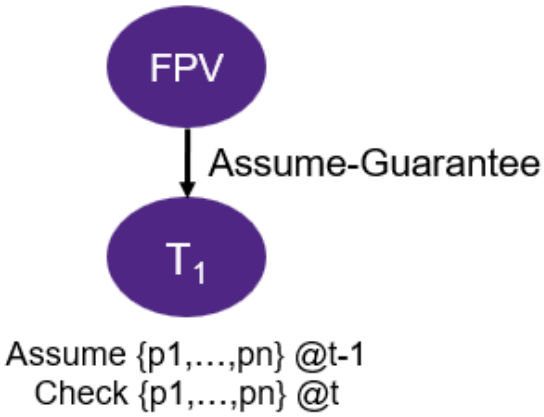
- **Grouped Induction**

  Grouped induction is a specialized form of assume-guarantee reasoning designed for verifying sets of interdependent properties. In this strategy, you identify a group of related properties, denoted as `{p1, …, pn}`, that are co-dependent and must be verified together.

  The grouped induction approach can also be a special case of Circular Assume-Guarantee reasoning, where the verification process involves assuming the group of properties `{p1, …, pn}` at a previous time step (`t-1`) and then checking the same group of properties at the current time step (`t`). Grouped Induction verifies co-dependent properties by assuming their delayed versions hold true. Grouped Induction approach ensures efficient status propagation and stops verification upon property violation.

  Grouped induction is particularly effective in situations where the properties being verified are tightly coupled and cannot be independently proven. By treating the group as a single unit, this strategy ensures that the inter-dependencies are preserved and verified collectively.

Feedback



Since the formulation within Grouped Induction is dependent on the entire group of properties involved, a falsification or trace will propagate to the parent, and that will lead to the solving to stop. Dually, full proofs will not propagate unless all properties are proven, and changes to the safe depth will lead to propagating the minimal safe depth among all properties.

For the same reason, creating child tasks of a Grouped Induction task is not recommended in general, and creating an ICM within said task is not allowed.

Table 37: Assume-Guarantee Proof Strategy Propagation Rules

Status	Propagated	Condition(s)
Proven	Yes	*Chain/Acyclic*: All constraints proven *Grouped Induction*: All properties proven
Falsified	Yes	*Grouped Induction*: Stop proof at 1st falsified/covered property
Inconclusive	Yes*	*Chain/Acyclic*: All constraints proven **Grouped Induction*: Not propagated

19.3.5 Case-Split

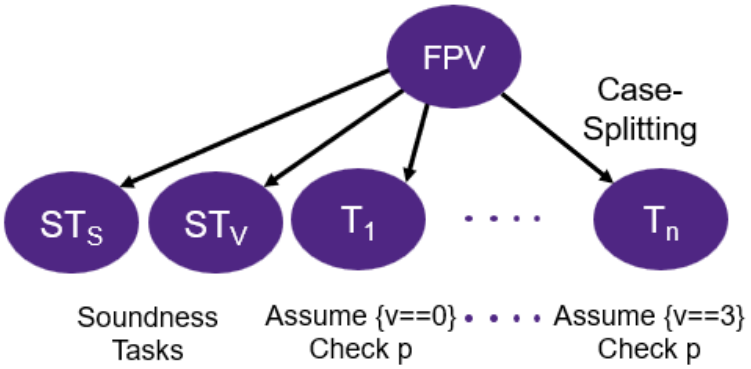
With Case-Splitting Proof Strategy, a complex proof is decomposed into a set of proofs based enumeration on design elements. Each branch acts as an "over constraint" with respect to the original property verification, that is, falsifications and traces propagate

Feedback

to the parent task, while proofs will propagate only if all branches are proven. To guard against missing branches and unsound propagation in those cases, soundness tasks are created to insure that it's safe to propagate even if all branches are proven.

Branching using the default enumeration method allows specifying a signal to branch on (using `-signal`), and a set of values (using `-values`) given in binary, decimal, or hexadecimal formats as specified by SystemVerilog standards. Other enumeration methods are possible, such as enumerating on SystemVerilog expressions, or enumerating on signals using automatically generated patterns. For more information on the enumeration methods, see the *VC Formal Beta Features Guide*.

The default semantics of branching is that of assumptions, and is suitable generally for modes of operation specified by stable values on inputs to the DUT. Systematically enumerating stable signal values can lead to performance boosts for proving or bug-hunting, since the state space in each *subspace* within the target task is exponentially smaller. Other semantics of branching is possible, such as those defined by implication in the property. For more information, see the *VC Formal Beta Features Guide*.



As described in the above figure, in addition to the branching tasks, 1-2 soundness tasks are added. For the default case-splitting using custom enumeration (values provided by user) and assume-based branching (subspace), two soundness tasks are added:

- A *Value Exhaustiveness* task verifies a new property that conjoins the given values, and proves that in each cycle, at least one of the values hold true. This allows quickly catching missing values that are allowed by the environment constraints.
- A *Space Exhaustiveness* task verifies a criterion that guarantees the original property cannot be violated *outside* the conjunction of all subspaces given in the branches. This, for example, catches the cases where the signals are not stable, despite their values being fully covered in each cycle, and in turn additional input patterns have to be checked which otherwise would lead to unsound propagation.

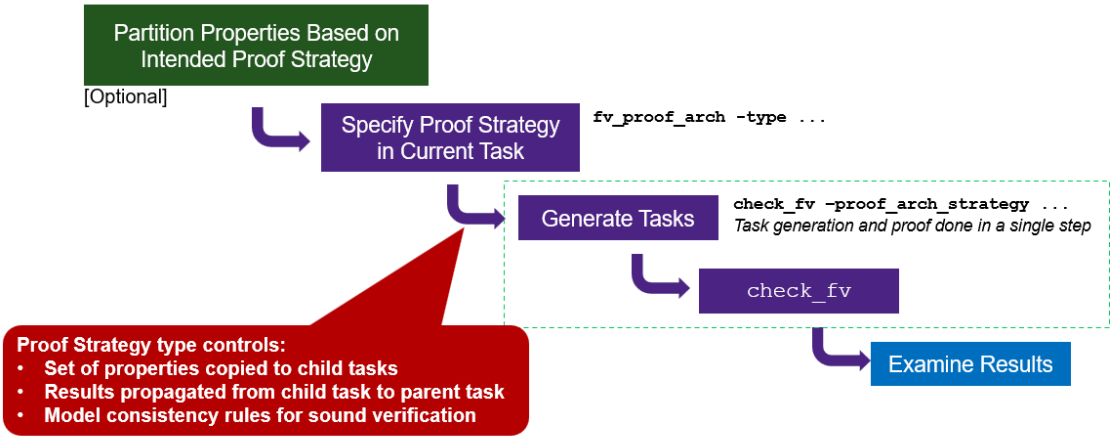
19.4 Proof Architect: Use Model

Proof Architect performs compositional verification using Proof strategies and subsequent set of a tasks and manages goal status propagation across tasks. Proof Strategies are created and managed through the `fv_proof_arch` command.

Note:
Proof Architect is currently supported only in the VC Formal FPV application.

The following figure summarizes the Proof Architect use model:

Figure 148: Proof Architect Use Model



To perform verification using Proof Architect:

1. Enable Proof Architect after reading in the design and setting up clocks and resets.

   See section Enabling Proof Architect Verification on how to enable the Proof Architect verification.

2. Create any of the following Proof Strategies types and generate tasks using the `fv_proof_arch` command or GUI.
   - Copy
   - Abstraction
   - Over-Constraint
   - Assume-Guarantee
   - Case-Split

See sections Tcl: Creating and Reporting Proof Strategies and Corresponding Tasks and GUI: Creating Proof Strategies and Corresponding Tasks for more details.

3. Generate model summary to verify the soundness of the parent and child tasks relationships and status propagation. See section Verifying Modeling Consistency and Soundness.
4. If required, mark tasks as unsafe and exclude them from verification. See section Speculative Tasks Marking for more details.
5. Audit all the Proof Strategies until you clean up all the unsafe tasks. See section Auditing Proof Strategies and Tasks for more details.
6. Recursively perform formal verification on the different tasks for different proof strategies. See section Performing Proof Architect Verification.

   See section Auditing Proof Strategies and Tasks for the two use models of recursively completing the audit along with verification.

19.4.1 Example Use Model for Compositional Verification Using Proof Architect

The following is an example use model for applying Proof Architect on a FIFO design implemented as a cascade of two FIFOs. Using Proof Architect, you can verify the global properties of the cascaded FIFO by assuming (and guaranteeing) the local properties of the smaller FIFOs.

```
set_fml_var fv_proof_architect true

set_fml_var async_command_max_parallel_count 10

# Cascaded FIFO setup
source -echo fifo_solution.tcl

# Create partitions
set vcf_partition_fifo_1 [get_props -usage assert
 {fifo_cascaded.fifo_1.i_fifo_bind.*}]

set vcf_partition_fifo_2 [get_props -usage assert
 {fifo_cascaded.fifo_2.i_fifo_bind.*}]

set vcf_partition_fifo_cascaded [get_props -usage assert
 {fifo_cascaded.i_fifo_cascaded_bind.*}]

# A - Create Proof Strategy - Acyclic Assume-Guarantee using partitions
fv_proof_arch -name AG -type assume_guarantee -subtype acyclic
fv_proof_arch -name AG -append -property "$vcf_partition_fifo_1
 $vcf_partition_fifo_2"
```

Feedback

```
fv_proof_arch -name AG -append -property $vcf_partition_fifo_cascaded
 -assume "$vcf_partition_fifo_1 $vcf_partition_fifo_2"

fv_proof_arch -name AG -generate
fv_proof_arch_report -name AG

# B - Create Proof Strategy - Custom Copy using partitions
fvtask AG_0
fv_proof_arch -name AG_0_CP -type copy -distribute_custom
fv_proof_arch -name AG_0_CP -append -property "$vcf_partition_fifo_1"
fv_proof_arch -name AG_0_CP -append -property "$vcf_partition_fifo_2"
fv_proof_arch -name AG_0_CP -generate

# C - Create Proof Strategy - Grouped Induction Assume-Guarantee on all
 properties in the task
fvtask AG_0_CP_0
fv_proof_arch -name AG_0_CP_0_GI -type assume_guarantee -subtype
 grouped_induction

fv_proof_arch -name AG_0_CP_0_GI -generate

# D - Create Proof Strategy - Grouped Induction Assume-Guarantee on all
 properties in the task
fvtask AG_0_CP_1
fv_proof_arch -name AG_0_CP_1_GI -type assume_guarantee -subtype
 grouped_induction
fv_proof_arch -name AG_0_CP_1_GI -generate
# AUDIT FLOW: Snip not allowed in AG Proof Strategy
fvtask AG_0_CP_1_GI_0
snip_driver fifo_2.pop_req

# E - Create Proof Strategy - Over-constraint
fvtask AG_1
fv_proof_arch -name AG_1_OC -type over_constraint
 -property {fifo_cascaded.i_fifo_cascaded_bind.prop_proven
 fifo_cascaded.i_fifo_cascaded_bind.prop_fault}

fv_proof_arch -name AG_1_OC -generate
fvtask AG_1_OC_0
#Constraint on the design
fvassume -expr {dummy_var == 1}
fvassume -expr {check_output == 1}

# F - Create Proof Strategy - Abstraction
fvtask AG_1
fv_proof_arch -name AG_1_AB -type abstraction -property
 {fifo_cascaded.i_fifo_cascaded_bind.prop_trial
 fifo_cascaded.i_fifo_cascaded_bind.as_no_push_ack_on_full}

fv_proof_arch -name AG_1_AB -generate
# Abstracting using snip_driver on signal check_output which contributes
 to the verification of the property
fvtask AG_1_AB_0
```

Feedback

```
snip_driver check_output

# G - Create Proof Strategy - Custom Copy
fvtask AG_1
fv_proof_arch -name AG_1_CP -type copy -distribute_custom
fv_proof_arch -name AG_1_CP -append -property
 {fifo_cascaded.i_fifo_cascaded_bind.as_empty_flag_correct
 fifo_cascaded.i_fifo_cascaded_bind.as_data_integrity}

fv_proof_arch -name AG_1_CP -append -property
 {fifo_cascaded.i_fifo_cascaded_bind.as_no_pop_ack_on_empty
 fifo_cascaded.i_fifo_cascaded_bind.as_full_flag_correct}

fv_proof_arch -name AG_1_CP -generate

# Set FPV as current task
fvtask FPV
```

19.5 Enabling Proof Architect Verification

To enable the proof architect verification, set the following formal variable to true because creating Proof Strategies

```
set_fml_var fv_proof_architect true
```

19.6 Tcl: Creating and Reporting Proof Strategies and Corresponding Tasks

Use the `fv_proof_arch` command to define, create, and configure Proof Strategies. See section Types of Proof Strategies for more details on the different Proof Strategies available.

Creating Proof Strategies

Use the `fv_proof_arch` command to define Proof Strategies, generate tasks for each Proof Strategy, and define the relation between the parent task and child tasks for each Proof Strategies.

Syntax

```
fv_proof_arch [-name <string>] [-type <value>] [-subtype <value>]
 [-append] [-generate] [-property <property-list>] [-assumes
 <list-of-assumes>] [-ignore_status] [-unsafe] [-propagate]
 [-distribute_max_tasks <int>] [-distribute_max_props <int>]
 [-distribute_custom] [-model_summary] [-model_ref <string>] [-audit]
 [-signal <string>] [-values <string>] [-enumerate <value>]
```

Arguments

- `[-name <string>]`: Specify a name for the Proof Strategy to be created, modified, or reported. This is a mandatory argument for all operations. Proof Strategy names are global and unique across tasks.
- `[-type <value>]`: Specify the type of the Proof Strategy to be created. Possible values are: `copy`, `assume_guarantee`, `abstraction`, `over_constraint`, `case_split`.
- `[-subtype <value>]`: The subtype of the Proof Strategy to be created. Relevant for assume_guarantee or case_split Proof Strategies only.
  - Possible values for `assume_guarantee` are: `acyclic (default), chain, grouped_induction`.
  - Possible values for `case_split` are: `subspace (default)`, `event`. For more information on subtype event, which uses implication precondition for branching, see the *VC Formal Beta Features Guide*.
- `[-append]`: Allows appending fields to an existing Proof Strategy with the given name.
- `[-generate]`: Generate tasks for the given Proof Strategy, and lock it to prevent future changes. The generation is dependent on the type of the Proof Strategy and its other fields specified up to that point.
  - Copy Proof Strategies create a duplicate of the parent task, including all enabled assumes, as well as the specified properties using`-properties` (or all enabled properties that are inconclusive if not specified).
  - Assume Guarantee Proof Strategy creates a task for each assumes/asserts pairs of sets specified using the `-assumes/-property` flags. Note that `check_fv -proof_arch` also generates children tasks if not already generated.
- `[-property <property-list>]`: The name of the asserts to be appended to the Proof Strategy. <property-list> can be a list of property/cover names, or patterns or a collection of properties/covers.
- `[-assumes <list-of-assumes>]`: The name of the assumes (assumptions) to be appended to the Proof Strategy. <name> can be a list of property names, or patterns or a collection of properties.
- `[-ignore_status]`: Allows including properties despite being in an inconclusive state.
- `[-unsafe]`: Mark this Proof Strategy as unsafe. This will affect consistency checking and the audit flow.
- `[-propagate]`: Determine whether to propagate through this Proof Strategy. Possible values: true, false.
- `[-distribute_max_tasks <int>]`: Distribute the goals within the given number of tasks.

- `[-distribute_max_props <int>]`: Distribute the goals such that each task will have the given number of goals, at most.
- `[-distribute_custom]`: Allow specifying a custom distribution.
- `[-model_summary]`: Calculates and prints the model summary in the current task, which by default represents the changes in the model compared to the parent task. It then determines if the model is consistent with the Proof Strategy for this task. If a reference Proof Strategy is provided using `-model_ref`, the comparison and consistency check is made with respect to the model for said Proof Strategy.
- `[-model_ref <string>]`: Defines the Proof Strategy owned by the current task, which will be used to compare models for consistency checking.
- `[-audit]`: Performs an audit check on the tasks and Proof Strategies.
- `[-signal <string>]`: Defines a signal for a `case_split` Proof Strategy.
- `[-values <string>]`: Defines a space-delimited values of a given variable for a case_split Proof Strategy.
- `[-enumerate <value>]`: The type of signal enumeration for a `case_split` Proof Strategy. Default enumeration is custom, allowing listing values to be assigned when branching on the signal. For more details on the other available enumeration types, see the *VC Formal Beta Features Guide*.

19.6.1 Tcl Examples - Creating Copy Proof Strategy

- To automatically generate tasks for a `copy` Proof Strategy (with all assertions in the design):

```
fv_proof_arch -name pa1 -type copy -generate
```

- To automatically generate tasks for a `copy` Proof Strategy for only one property:

```
fv_proof_arch -name pa2 -type copy -property {mydesign.prop} -generate
```

- To automatically generate tasks for a `copy` Proof Strategy for only one property and one cover:

```
fv_proof_arch -name pa2 -type copy -property {mydesign.prop
 mydesign.cov} -generate
```

- To automatically create a `copy` Proof Strategy, and then generate child tasks while appending properties:

```
fv_proof_arch -name pa2 -type copy
fv_proof_arch -name pa2 -append -property mydesign.basic -generate
```

- To specifying a custom distribution of tasks to `copy` Proof Strategy, and generate tasks:

```
fvtask FPV_PS_8
fv_proof_arch -name FPV_PS_8 -type copy -distribute_custom
fv_proof_arch -name FPV_PS_8 -append -property "testbench.fifo_f_e1
 testbench.fifo_f_e testbench.fifo_temp testbench.fifo_full_asrtd
 testbench.fifo_rst"
fv_proof_arch -name FPV_PS_8 -generate
```

19.6.2 Tcl Examples - Creating assume_guarantee Proof Strategy

- To automatically generate tasks for `assume_guarantee` of subtype `chain` Proof Strategy (with all assertions in the design):

```
fv_proof_arch -name FPV_PS_8 -type assume_guarantee -subtype chain
 -property "$vcf_partition_property_selection"
fv_proof_arch -name FPV_PS_8 -generate
```

- To automatically generate tasks for `assume_guarantee` Proof Strategy of subtype `acyclic`:

```
fv_proof_arch -name FPV_PS_5 -type assume_guarantee -subtype acyclic
fv_proof_arch -name FPV_PS_5 -append -property
 "testbench.fifo_data_integrity" -assume "testbench.fifo_empty_asrtd"
fv_proof_arch -name FPV_PS_5 -generate
```

- To automatically generate tasks for `assume_guarantee` Proof Strategy of subtype `grouped_induction`:

```
fv_proof_arch -name FPV_PS_6 -type assume_guarantee -subtype
 grouped_induction -property "testbench.fifo_data_integrity"
fv_proof_arch -name FPV_PS_6 -generate
```

- To create `assume_guarantee` Proof Strategy of `-subtype chain` and then generate tasks while appending properties and assumes:

```
fv_proof_arch -name FPV_PS_8 -type assume_guarantee -subtype chain
 -property "$vcf_partition_property_selection testbench.fifo_f_e
 testbench.fifo_data_integrity_fpc testbench.fifo_empty_asrtd
 testbench.fifo_rst"
fv_proof_arch -name FPV_PS_8 -append -property "testbench.fifo_temp
 testbench.fifo_full_asrtd testbench.fifo_empty_asrtd
 testbench.fifo_rst"
fv_proof_arch -name FPV_PS_8 -generate
```

- To create a `assume_guarantee` Proof Strategy of `-subtype acyclic`, and then generate tasks while appending properties and assumes:

```
v_proof_arch -name FPV_PS_8 -type assume_guarantee -subtype acyclic
fv_proof_arch -name FPV_PS_8 -append -property
 "$vcf_partition_property_selection testbench.fifo_f_e
 testbench.fifo_data_integrity_fpc testbench.fifo_empty_asrtd
```

Feedback

```
 testbench.fifo_rst" -assume "testbench.fifo_f_e
 testbench.fifo_data_integrity_fpc testbench.fifo_empty_asrtd"
fv_proof_arch -name FPV_PS_8 -append -property "testbench.fifo_temp
 testbench.fifo_full_asrtd testbench.fifo_empty_asrtd
 testbench.fifo_rst" -assume "testbench.fifo_temp
 testbench.fifo_data_integrity testbench.fifo_full_asrtd"
fv_proof_arch -name FPV_PS_8 -generate
fv_proof_arch_report
check_fv -proof_arch pa3 -block
```

- To create a *assume_guarantee* Proof Strategy of `-subtype grouped_induction` for a partition and selected properties, and generate tasks :

```
fv_proof_arch -name FPV_PS_8 -type assume_guarantee -subtype
 grouped_induction -property "$vcf_partition_property_selection
 testbench.fifo_f_e1 testbench.fifo_f_e testbench.fifo_temp
 testbench.fifo_full_asrtd testbench.fifo_data_integrity_fpc
 testbench.fifo_empty_asrtd"
fv_proof_arch -name FPV_PS_8 -generate
```

- Creating cycles in *assume_guarantee* Proof Strategy of subtype `chain` is prohibited. For example, if you create a *assume_guarantee* Proof Strategy with assume *mydesign.assert_a0_pass* and assert *mydesign.assert_a1_failc15*, and assume *mydesign.assert_a1_failc15* and assert mydesign.assert_a0_pass:

```
fv_proof_arch -name pa0 -type assume_guarantee -subtype chain
fv_proof_arch -name pa0 -append -assumes mydesign.assert_a0_pass
 -property mydesign.assert_a1_failc15
fv_proof_arch -name pa0 -append -property mydesign.assert_a0_pass
 -assumes mydesign.assert_a1_failc15
fv_proof_arch -name pa0 -generate
```

  Then the following error message is reported:

  CODE_ID:PROOF_ARCHITECT_AG_CYCLE, Severity: Error, Primary: "A cycle exists in this Proof Strategy", Secondary: "Assume/Guarantee Proof Strategies have to be cycle-free. Check the dependencies between the assumes and asserts."

19.6.3 Tcl Examples - Creating Abstraction Proof Strategy

1. To automatically create a `abstraction` Proof Strategy and generate tasks (with all assertions in the design):

```
fv_proof_arch -name pa1 -type abstraction -generate
```

2. To automatically create a `abstraction` Proof Strategy, and generate tasks ignoring the status of the property:

```
fv_proof_arch -name FPV_PS_8 -type abstraction -property
 "testbench.fifo_f_e" -ignore_status
fv_proof_arch -name FPV_PS_8 -generate
```

Feedback

19.6.4 Tcl Examples - Creating Over-Constraint Proof Strategy

1. To automatically create a `over_constraint` Proof Strategy for the specified properties, and generate tasks:

```
fv_proof_arch -name FPV_PS_10 -type over_constraint -property
 "$vcf_partition_property_selection testbench.fifo_f_e1
 testbench.fifo_empty_asrtd"
fv_proof_arch -name FPV_PS_10 -generate
```

19.6.5 Tcl Examples - Creating Case-Split Proof Strategy

1. To automatically generate tasks for `case_split` Proof Strategy of `subtype subspace` and perform checks:

```
fv_proof_arch -name cs -type case_split -subtype subspace -property p
 -signal v -values {0 1 2 3}
fv_proof_arch -name cs -generate
check_fv -proof_arch_strategy cs
```

19.6.6 Reporting Proof Strategies

Use the `fv_proof_arch_report` command for reporting the Proof strategies created using the `fv_proof_arch` command.

Syntax

```
fv_proof_arch_report [-name <string>] [-format <value>]
```

Arguments

- `[-name <string>]`: The name of the Proof Strategy to be reported. If not specified, all Proof Strategies will be reported.
- `[-format <value>]`: The format of the report: text (default), xml, or collection.

The report from the `fv_proof_arch_report` command contains the following:

- The name given to the Proof Strategy: `Name: <FPV_PS_3>`
- The name of the owner task: `Task: <FPV>`
- The type of Proof Strategy: `Type: <Over-Constraint>`
- The status of Proof Strategy - whether it is marked safe/unsafe for Propogation: `Safe: <yes>`
- Whether the results of the Proof Strategy has propagated to the parent task: `Propagate: <yes>`

Feedback

- When the tasks have already been generated for the Proof Strategy: `[Locked]`
- The list of properties appended to the generated tasks in a Proof Strategy:

```
Assert                              Assume                                Task
=============================================================================
===
testbench.fifo_data_integrity
testbench.fifo_data_integrity_fpc
-----

 FPV_PS_3_0
```

Examples

- The following is an example report of the Over-constraint Proof Strategy:

```
vcf>fv_proof_arch_report -name FPV_PS_3
*******************************************************
Proof Architect Report
*******************************************************
Name:       FPV_PS_3
Owner Task: FPV
Type:       Over-Constraint
Safe:       yes
Propagate:  yes
[Locked]
Assert                              Assume                                Task
=============================================================================
===
testbench.fifo_data_integrity
testbench.fifo_data_integrity_fpc
testbench.fifo_empty_asrtd
testbench.fifo_f_e
testbench.fifo_f_e1
testbench.fifo_full_asrtd
testbench.fifo_rst
testbench.fifo_temp
-----

 FPV_PS_3_0
-----------------------------------------------------------------------------
---
1
```

- The following is an example report of the Assume Guarantee Proof Strategy of subtype `grouped_induction`.

```
fv_proof_arch_report -name FPV_PS_8

*******************************************************
Proof Architect Report
*******************************************************
```

Feedback

```
Name:        FPV_PS_8
Owner Task:  FPV
Type:        Assume-Guarantee (Grouped Induction)
Safe:        yes
Propagate:   yes
[Locked]
Assert                              Assume                                Task
=============================================================================
===
testbench.fifo_data_integrity
testbench.fifo_data_integrity_fpc
testbench.fifo_empty_asrtd
testbench.fifo_f_e
testbench.fifo_f_e1
testbench.fifo_full_asrtd
testbench.fifo_temp
-----

 FPV_PS_8_0
-----------------------------------------------------------------------------
---
1
```

19.7 GUI: Creating Proof Strategies and Corresponding Tasks

You can also create, customize, and modify Proof Strategies in the GUI.

19.7.1 Creating Partitions for Properties

You can create property partitions for Proof Architect to group a set a properties. Properties can be partitioned on-the-fly. These created partitions are available for selection in the GUI while creating Proof Strategies.

To partition the properties:

1. Select a group of properties, and from the RMB menu, select **Create Partition**.

   Provide a name for the partition in the dialog box. The partition is created with a prefix **vcf_partition_<name>** for the selected properties.

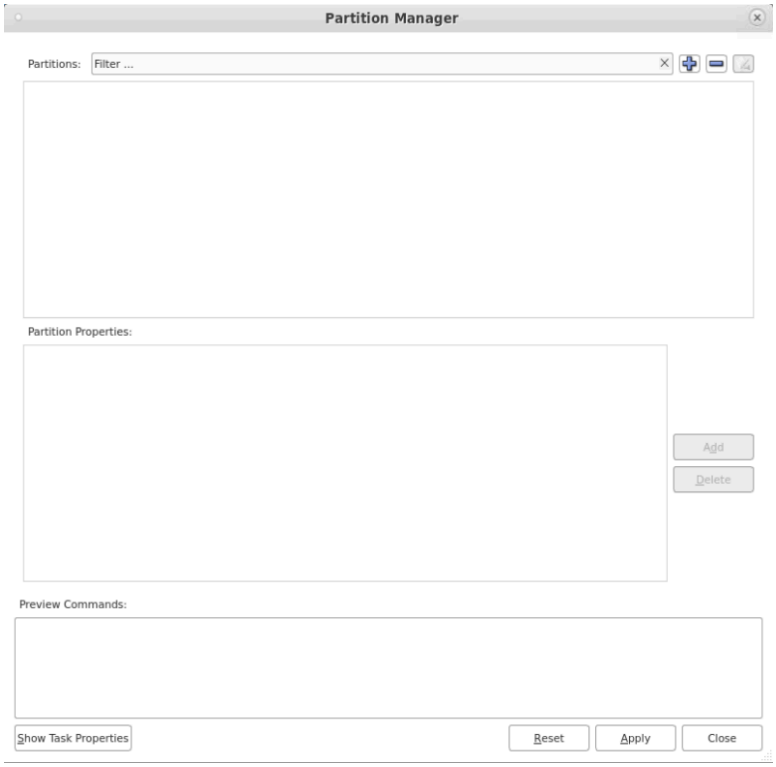
   ```
   set vcf_partition_<name> {property1 property2 …}
   ```

2. To customize the already created partitions, from the RMB menu, select **Partition Manager**.

   The **Partition Manager** dialog appears.

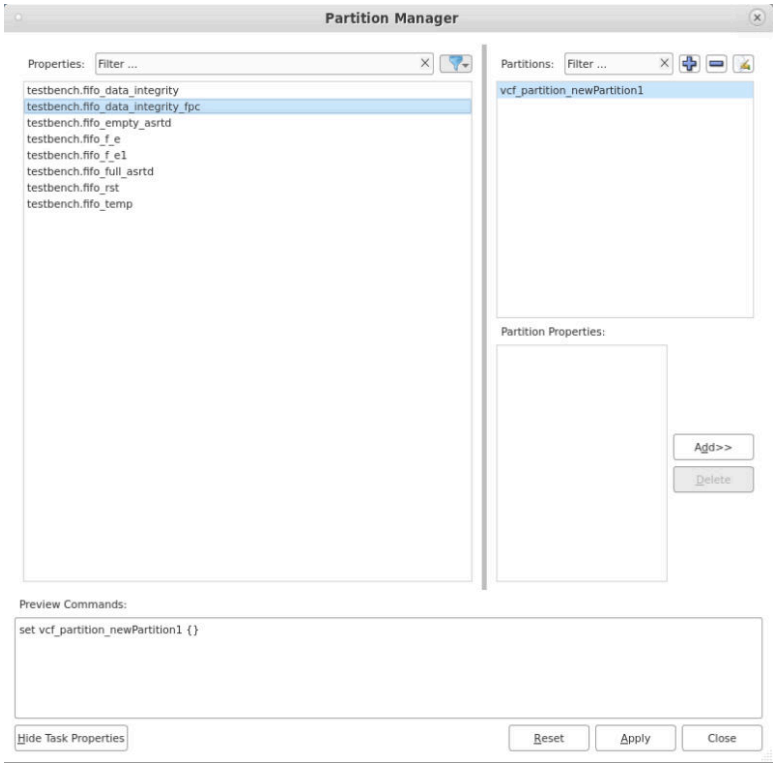
Feedback

Figure 149: Partition Manager



3. Click **Show Task Properties** to show/hide the properties for selection and the already existing partitions.

Feedback



4. To add new partitions, click , and add the required properties from the **Properties** pane, or select the properties from the **GoalList**.

   Or, select the required properties in the **Properties** pane, and click **Add/Delete** to add/delete properties to the existing partitions.

   You can add multiple partitions from the **Partition Manager** window.

5. To delete partitions, select a partition and click .

6. To rename the partitions, select a partition and click .

7. Review the commands in the **Preview Commands** pane, and click **Apply**.

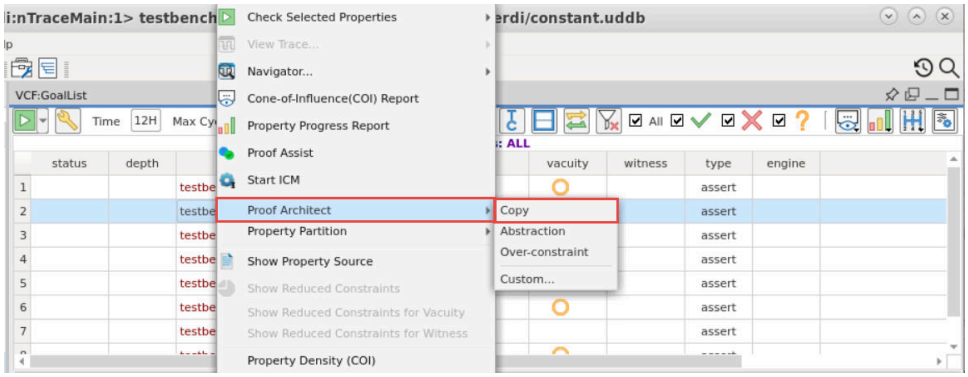
Feedback

19.7.2 Creating Copy Proof Strategies

To create Copy Proof Strategy from GUI:

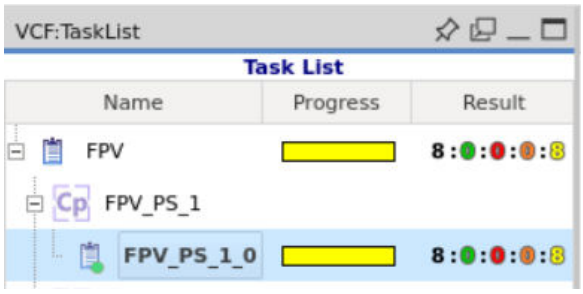
1. To create Copy Proof Strategy for a selected property, select the property in the **GoalList**, and from the RMB menu, select **Proof Architect > Copy**:

Figure 150: RMB Menu Proof Architect



A corresponding strategy for the selected property is created with the default values. The strategy appears in the Task List as FPV_PS_XX.

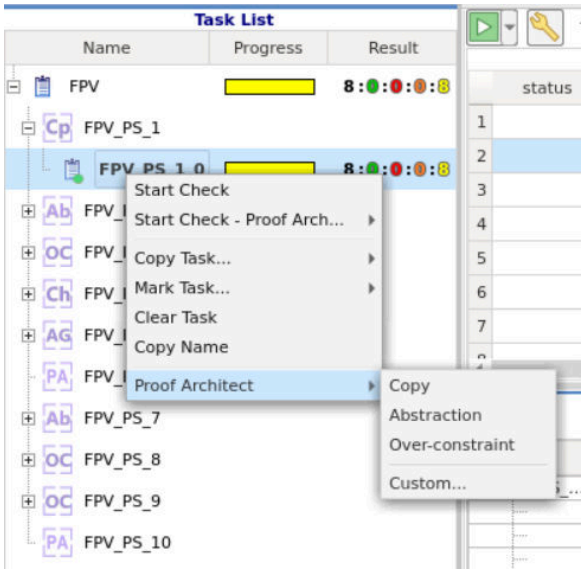
Figure 151: RMB Menu Proof Architect



Feedback

2. You can also create a copy Proof Strategy for a task from the **Task List**, select **Proof Architect > Copy**:

Figure 152: RMB Menu Proof Architect



A corresponding strategy for all the properties is created and appears in the Task List as **FPV_PS_2**.

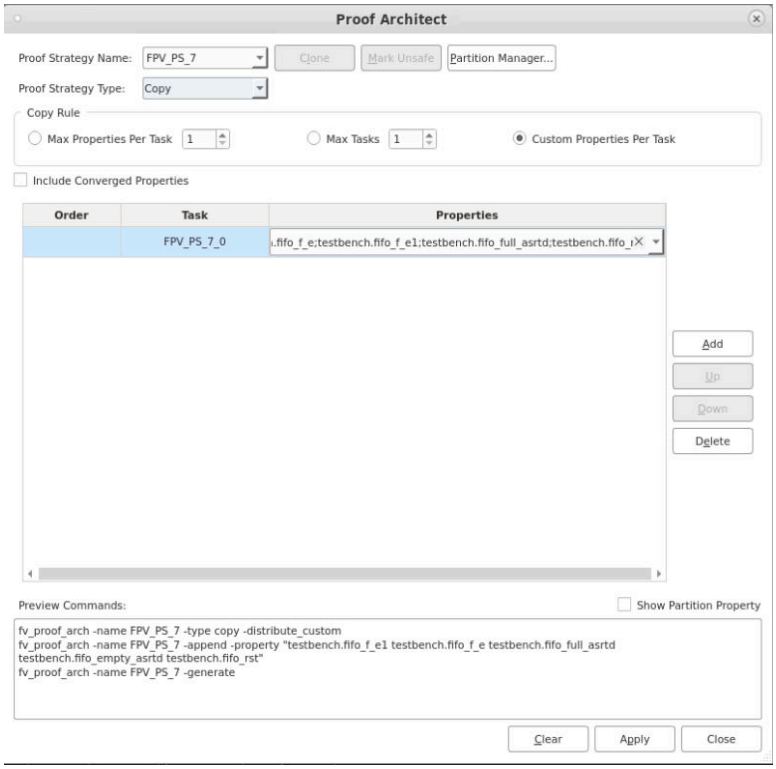
3. To create custom Copy Proof Strategy, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**.

   The **Proof Architect** Window appears.

Feedback

4. To define custom distribution of the tasks, provide the following details:

Figure 153: Proof Architect Copy



a. Select **Custom Properties per Task**.

b. **Max Properties Per Task**: Create multiple sub-tasks with the specified number of properties

c. **Max Tasks**: Creates up to the specified number of tasks and distribute the properties equally.

d. Select **Include Converged Properties** option to include the converged properties. By default, the converged properties are excluded.

e. Click **Add** to specify the custom properties.

f. Review the commands in the **Preview Commands** pane, and click **Apply**.

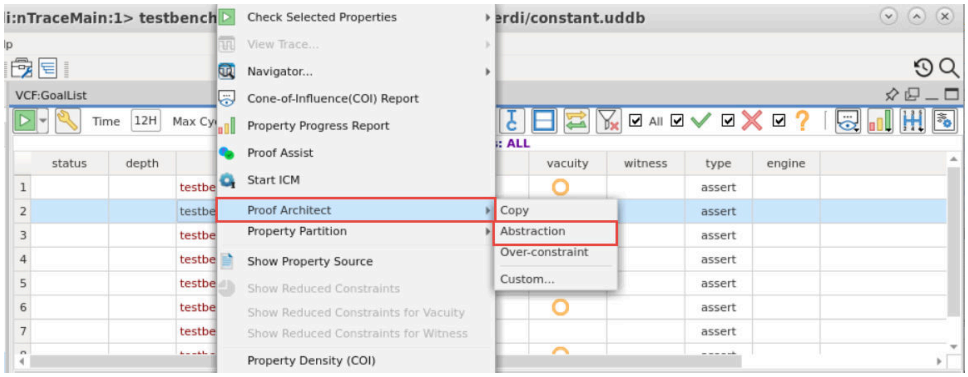
Feedback

19.7.3 Creating Abstraction Proof Strategies

To create Proof Strategies from GUI:

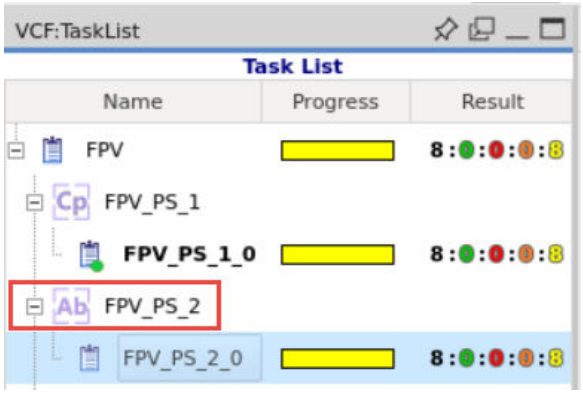
1. To create Proof Strategies for a selected property, select the property in the **GoalList**, and from the RMB menu, select **Proof Architect > Abstraction**:

Figure 154: RMB Menu Proof Architect



A corresponding strategy for the selected property is created with the default values. The strategy appears in the Task List as FPV_PS_XX.

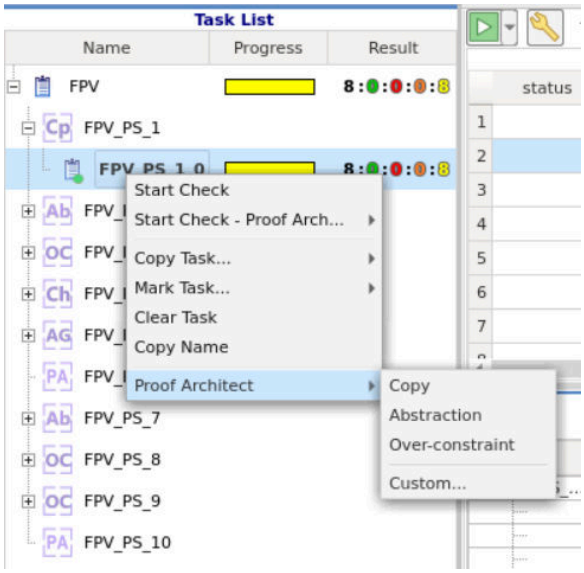
Figure 155: RMB Menu Proof Architect



Feedback

2. To create Proof Strategies for all properties, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Abstraction**:

Figure 156: RMB Menu Proof Architect



A corresponding strategy for all the properties is created and appears in the Task List as **FPV_PS_2**.

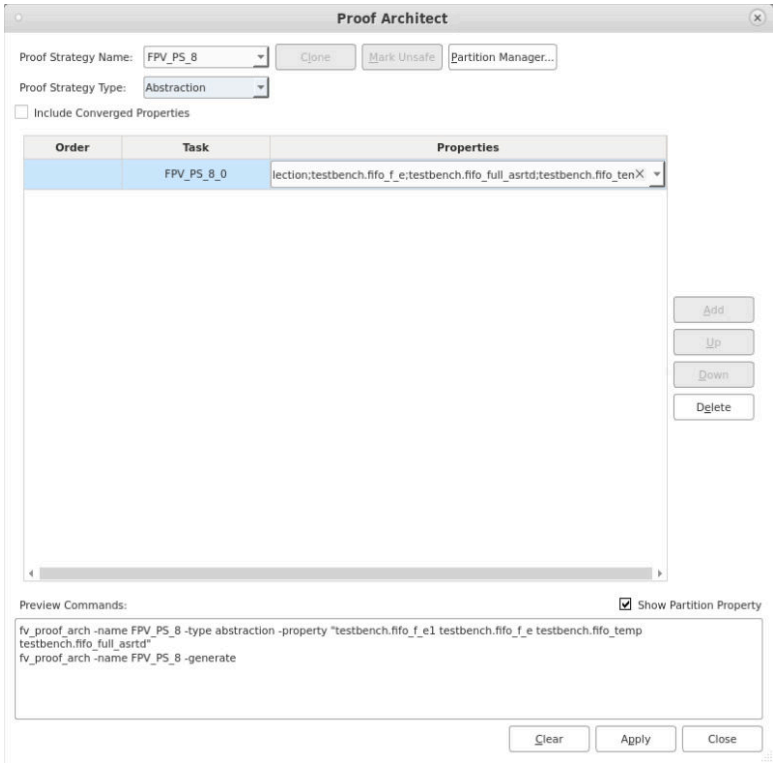
3. To further customize the strategies, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**.

   The **Proof Architect** Window appears.

Feedback

4. Customize the **Abstraction** strategy using the following options:

Figure 157: Proof Architect Abstraction



a. Click **Add** to append multiple properties.

b. Select the required properties from the drop-down list.

c. Select **Include Converged Properties** option to include the converged properties. By default, the converged properties are excluded.

d. Review the commands in the **Preview Commands** pane, and click **Apply**.

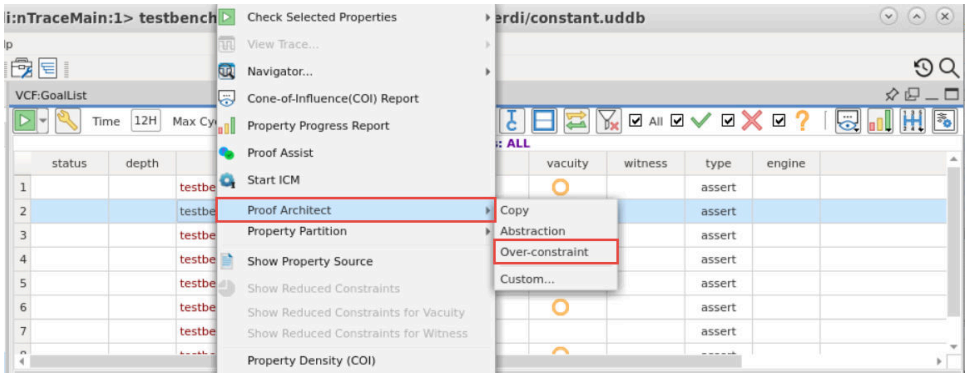
Feedback

19.7.4 Creating Over-Constraint Proof Strategies

To create Proof Strategies from GUI:

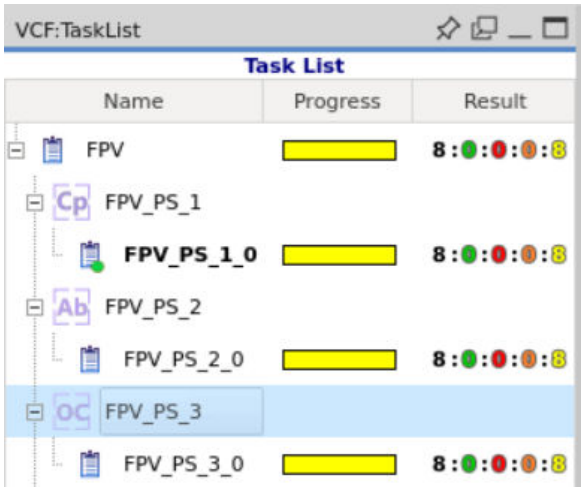
1. To create Proof Strategies for a selected property, select the property in the **GoalList**, and from the RMB menu, select **Proof Architect > Over-constraint**:

Figure 158: RMB Menu Proof Architect



A corresponding strategy for the selected property is created with the default values. The strategy appears in the Task List as FPV_PS_XX.

Figure 159: RMB Menu Proof Architect



2. To create Proof Strategies for all properties, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Over-constraint**:

   A corresponding strategy for all the properties is created and appears in the Task List as **FPV_PS_2**.

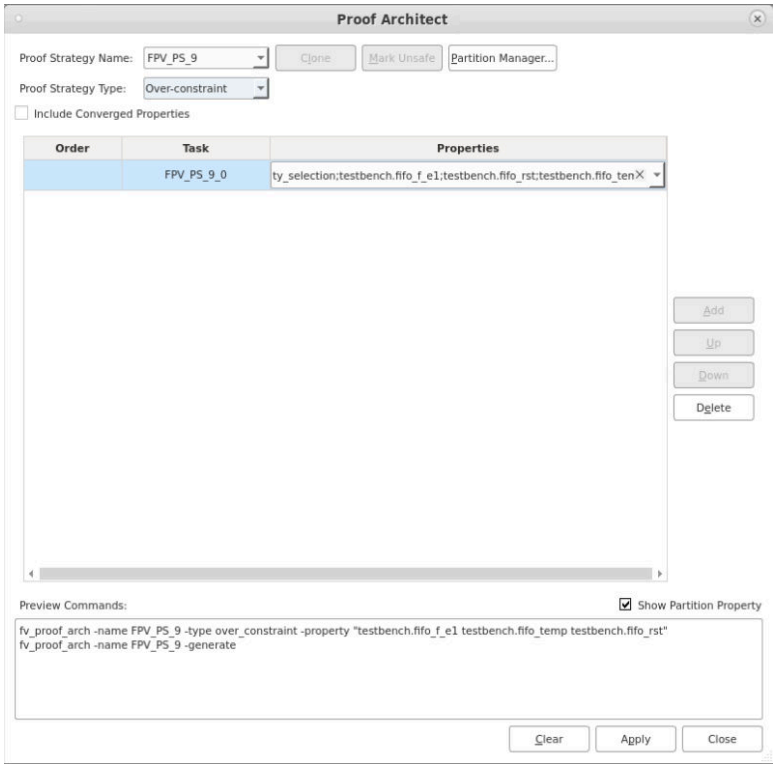
Feedback

3. To further customize the strategies, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**.

   The **Proof Architect** Window appears.

4. Customize the **Over-constraint** strategy using the following options:

Figure 160: Proof Architect Over-constraint



   a. Click **Add** to append multiple properties.

      If no properties are added, then the Abstraction strategy is applied to all properties if Proof Architect GUI is invoked from the **Task List**.

   b. Select **Include Converged Properties** option to include the converged properties. By default, the converged properties are excluded.

   c. Select the required properties from the drop-down list.

   d. Review the commands in the **Preview Commands** pane, and click **Apply**.

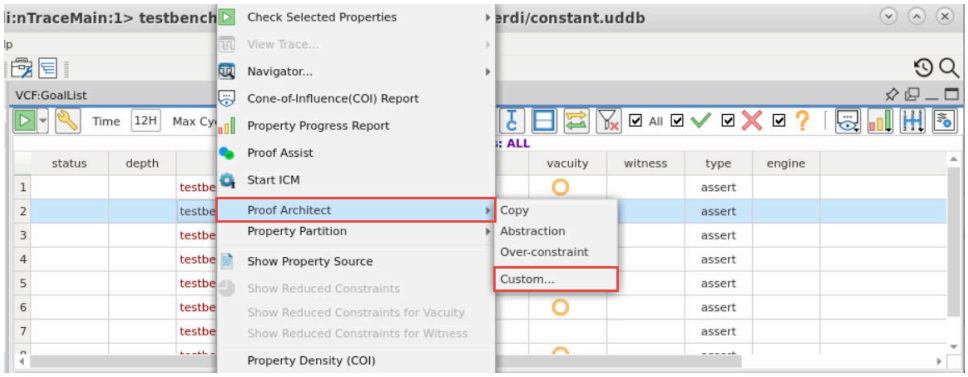
Feedback

19.7.5 Creating Assume Guarantee Proof Strategies

To create Proof Strategies from GUI:

1. To create Proof Strategies for a selected property, select the property in the **GoalList**, and from the RMB menu, select **Proof Architect > Custom**:

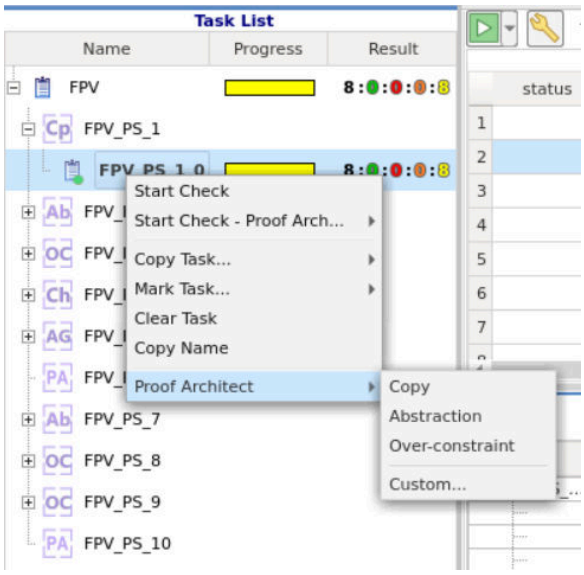
Figure 161: RMB Menu Proof Architect



A corresponding strategy for the selected property is created with the default values. The strategy appears in the Task List as FPV_PS_XX.

2. To create Proof Strategies for all properties, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**:

Figure 162: RMB Menu Proof Architect



A corresponding strategy for all the properties is created and appears in the Task List as **FPV_PS_2**.

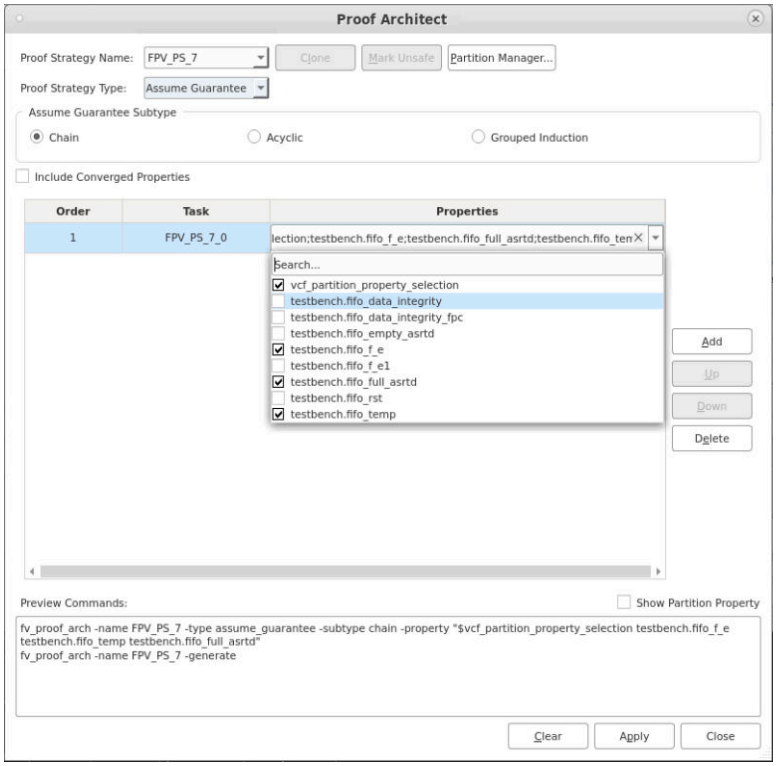
Feedback

3. To further customize the strategies, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**.

   The **Proof Architect** Window appears.

4. Customize the **Assume-Gaurantee** strategy using the following options:

Figure 163: Proof Architect Assume Gauranteee



   a. Select the Assume-Guarantee subtype - **Chain**, **Acyclic**, or **Grouped Induction**.

   b. Select **Include Converged Properties** option to include the converged properties. By default, the converged properties are excluded.

   c. Click **Add** to append multiple properties.

   d. Select the required properties and the assume from the drop-down list.

   e. Click **Up** and **Down** to change the order of the subtasks

   f. Review the commands in the **Preview Commands** pane, and click **Apply**.

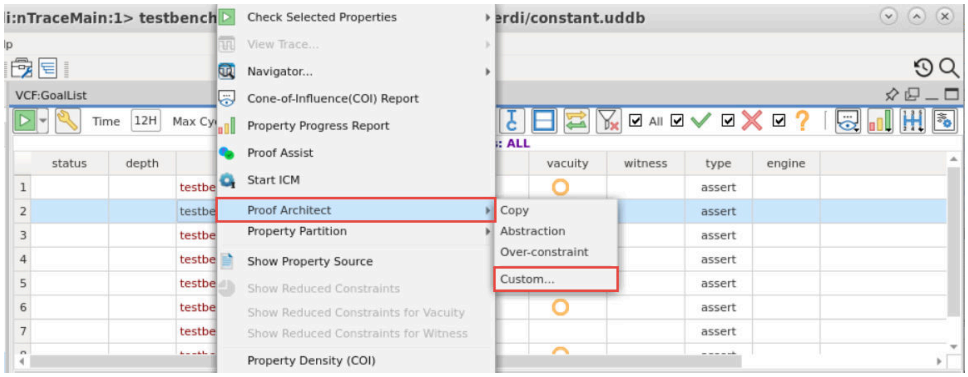
Feedback

19.7.6 Creating Case-Split Proof Strategies

To create Proof Strategies from GUI:

1. To create Proof Strategies for a selected property, select the property in the **GoalList**, and from the RMB menu, select **Proof Architect > Custom**:

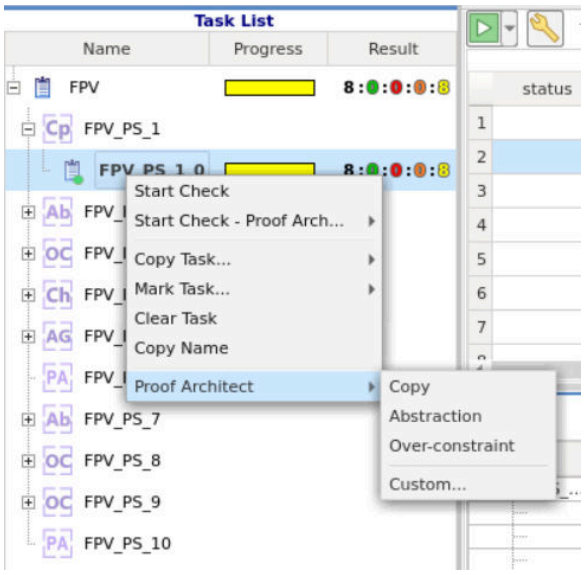
Figure 164: RMB Menu Proof Architect



A corresponding strategy for the selected property is created with the default values. The strategy appears in the Task List as FPV_PS_XX.

2. To create Proof Strategies for all properties, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**:

Figure 165: RMB Menu Proof Architect



A corresponding strategy for all the properties is created and appears in the Task List as **FPV_PS_2**.

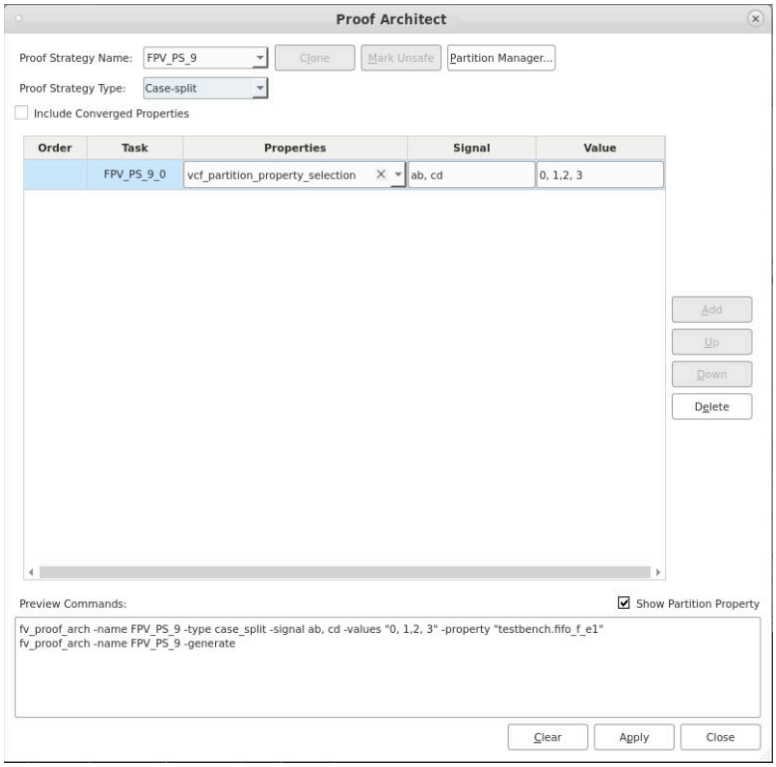
Feedback

3. To further customize the strategies, right-click FPV in the **Task List**, and from the RMB menu, select **Proof Architect > Custom**.

   The **Proof Architect** Window appears.

4. Customize the **Case-Split** strategy using the following options:

Figure 166: Proof Architect Case-Splitting



   a. Click **Add** to append multiple properties.

   b. Select **Include Converged Properties** option to include the converged properties. By default, the converged properties are excluded.

   c. Select the required properties from the drop-down list.

   d. Review the commands in the **Preview Commands** pane, and click **Apply**.

19.8 Verifying Modeling Consistency and Soundness

By default, `fv_proof_arch` tracks the detailed changes in the model with respect to the parent task at the time of child task generation, and performs the consistency/safety checks. Proof Architect checks for the following model changes:

- Snips
- Constraints
- Constants
- ChangeAts

The model consistency checks automatically take place prior to `check_fv`, and prevent it from proceeding if failing. This will be applied to the current task as well as all parent tasks, since the statuses propagate along that path. In a given task of a Proof Strategy, running `check_fv` ensures that:

- `fv_proof_arch -model_summary` passes within the task, and all parent tasks up to (but excluding) FPV. Passing means that the model within the task is consistent with its base model upon task creation.
- `fv_proof_arch -model_summary -model_ref <Proof Strategy>` passes within all parent tasks up to, and including, FPV. Passing means that the model within the task is equal to the base model of the Proof Strategy, for all given Proof Strategies that are owned by each task.

You can also use the following command to explicitly determine the modeling soundness and propagation across the hierarchy.

```
fv_proof_arch -model_summary [-model_ref <Proof Strategy name>]
```

Bypassing Soundness Enforcement

You can bypass parent/child consistency checks by marking the relevant Proof Strategy as unsafe using Speculative Tasks Marking.

19.9 Speculative Tasks Marking

Speculative task marking allows users to temporarily model and experiment with tasks created through Proof Architect. This feature enables users to mark tasks with caution levels, facilitating later auditing and confirmation. It is particularly useful for managing complex verification flows where temporary assumptions or speculative modeling are required.

The following are the benefits of speculative task marking:

- **Persistent Marking**: You can assign caution levels {none low medium high} to tasks for future reference.
- **Task Filtering**: You can get a ;ist all tasks with {low medium high} caution levels for easy identification.
- **Tcl and GUI Support**: The feature is accessible through both Tcl commands and the graphical user interface.

Speculative task marking can lead one of the following three things:

- If the consistency checks done by `-model_summary` fails, verification will not be blocked, if the corresponding Proof Strategy is marked as unsafe.
- Status propagation through Proof Strategy does not take place if it is marked unsafe with `-propagate false` before starting verification. You can override that with `fv_proof_arch -unsafe -propagate true` to request propagation.
- Unsafe marking for a Proof Strategy will lead to marking the task “task_attention = high”.

19.9.1 Tcl: Enabling Speculative Task Marking

To enable speculative task marking in Tcl, you can use the following methods:

1. Mark the Proof Strategy as unsafe using the `fv_proof_arch` commands:

   ```
   fv_proof_arch -name FPV_PS_1 -unsafe -propagate false
   fv_proof_arch -name FPV_PS_1 -unsafe -propagate true
   ```

   Note:

   Unsafe marking for a Proof Strategy will lead to marking the task `fv_task_attention high`.

2. **Set the Caution Level for a Task**: Use the `report_fml_var fv_task_attention {none low medium high}` command to assign a caution level to a task.

   For example, to set the caution level of the task to `high`.

   ```
   report_fml_var fv_task_attention high
   ```

3. **Add Task Comments**: Use the `report_fml_var fv_task_comment` command to view comments associated with tasks.

   ```
   report_fml_var fv_task_comment <comments>
   ```

4. **Filter Tasks by Attributes**: Use the `get_fvtask` command with filters to list tasks based on specific attributes.

Feedback

```
vcf>get_fvtask -help
Usage: get_fvtask # Returns the current FV task, or a collection of
 tasks meeting specified criteria
[-filter ] (Filter expression based on task attributes)
[] (List of task names or patterns)
```

This lists all tasks where the application is FPV.

5. **Retrieve Task Attributes**: Use the `get_attribute` command to fetch specific attributes of tasks.

   ```
   get_attribute [report_fml_var] fv_task_attention
   ```

   This retrieves the caution levels of all tasks.

- Windows

  Instructions for Windows

- Linux

  Instructions for Linux

- Mac

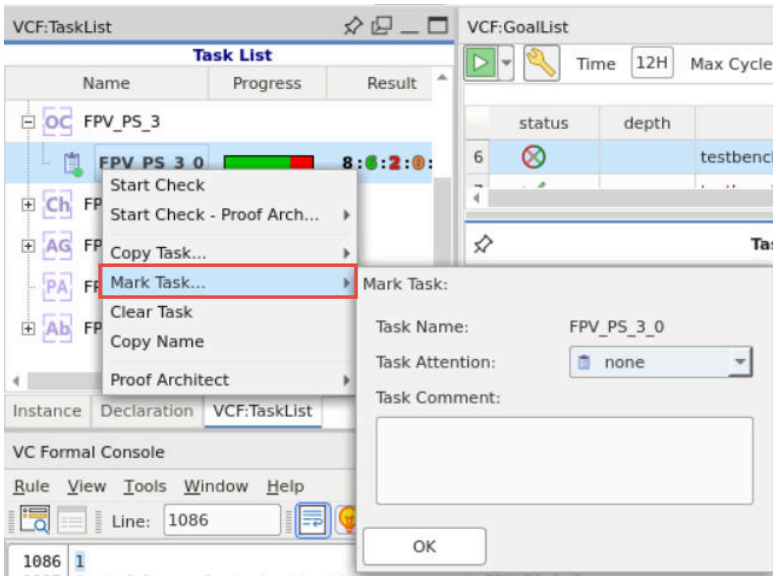
  Instructions for Mac

Feedback

19.9.2 GUI: Enabling Speculative Task Marking

To mark tasks in GUI:

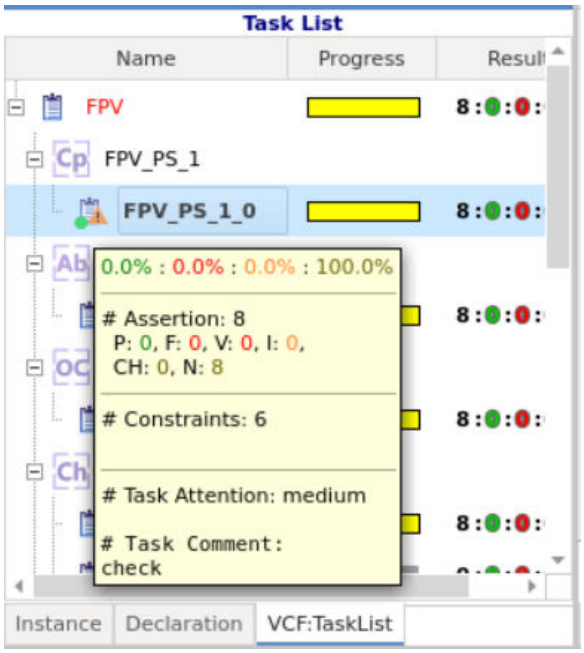
1. Select the task that requires attention, and from the RMB menu, select **Mark Task**.



2. In the **Mark Task** dialog, select the caution level from the **Task Attention** drop-down, and leave a comment in the **Task Comment** box.

Feedback

The tool tip of the Task shows the attention level and comments included for the tasks.



19.10 Auditing Proof Strategies and Tasks

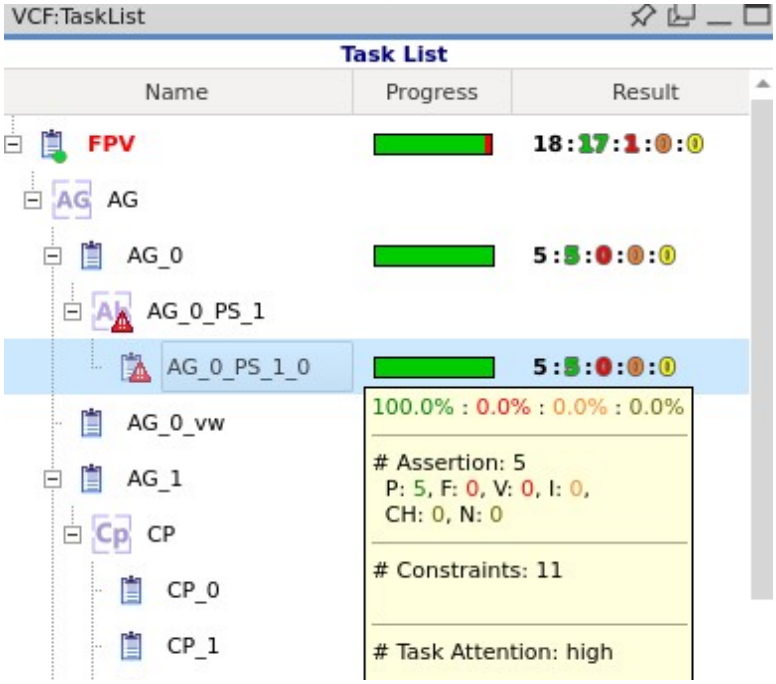
Use the `fv_proof_arch -audit` option to check whether there are any tasks that are unsafe or need attention. This command reports all the unsafe tasks and Proof Strategies.

```
vcf>fv_proof_arch -audit
2/7 unsafe Proof Strategies
FPV_PS_1
FPV_PS_2
1/9 tasks pending attention
FPV_PS_1_0
Audit: FAIL
1
```

Feedback

A failing audit shows:

- Live in the task panel (top task is in red color) as shown in the following figure:



- Within `report_fv` as shown in the following figure:

```
vcf>report_fv
   **Proof Architect audit failed. For more details: fv_proof_arch -audit.
  Summary Results
   Property Summary: FPV
   -----------------
   > Assertion
     - # found        : 18
```

By default, `check_fv` performs audit checks before verification. You can cancel the auditing checks in `check_fv` by setting hidden variable `fv_proof_arch_audit_flow` to false. When the `fv_proof_arch_audit_flow` is set to true, the unsafe marking and `-audit` functionality will remain unchanged, but `check_fv` does not consider model consistency/safety when running the checks.

19.11 Performing Proof Architect Verification

After you have created the Proof Strategies, you can verify the Proof strategies/tasks individually or recursively, from the GUI and Tcl.

Feedback

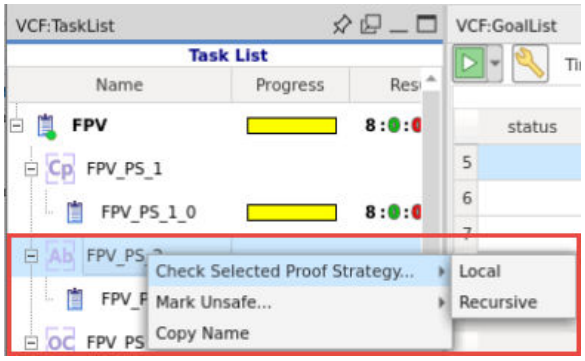
Firstly, you can run `check_fv` normally in any task, including a parent task or a child task that was created through a Proof Strategy.

Secondly, from Tcl, use the following options in the `check_fv` command to run groups of solves associated with a single Proof Strategy:

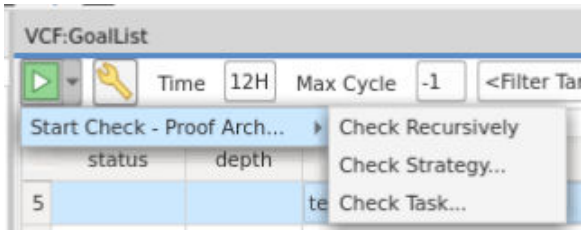
- `[-proof_arch_strategy <proof-strategy-names>]` : Launch Proof Architect verification through the given Proof Strategy(s)).
- `[-proof_arch_task <proof-task-names>]`: Launch Proof Architect verification through the given Task(s)).
- `[-proof_arch_recursion]`: Launch Proof Architect verification recursively, through the given Proof Strategy(s) or Task(s).

From the GUI:

- To check a specific Proof strategy, from the RMB menu of a selected Proof Strategy, select either **Check Selected Proof Strategy > Local** or **Check Selected Proof Strategy > Recursive** to run the selected Proof Strategy or the entire hierarchy under the specific node.

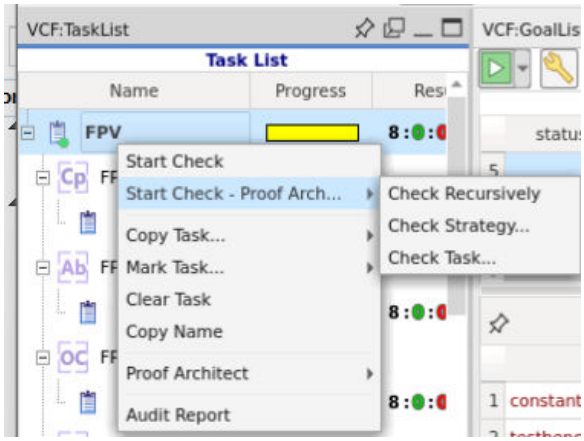


- To check a selected property from the GoalList, right-click on the selected property, and select either of the following:
  - **Start Check - Proof Arch > Check Recursively**.
  - **Start Check - Proof Arch > Check Strategy**.
  - **Start Check - Proof Arch > Check Task**.



Feedback

- To check all the properties, from the **TaskList**, right-click on the selected Proof Strategy in the **Task List**, and select either of the following.
  - **Start Check - Proof Arch > Check Recursively**.
  - **Start Check - Proof Arch > Check Strategy**.
  - **Start Check - Proof Arch > Check Task**.



19.12 Enabling RMA With Proof Architect

Proof Architect supports Regression Mode Acceleration (RMA). RMA enables VC Formal to learn information about falsifications and proofs, and leverage the learnings for the subsequent `check_fv` runs.

When Proof Architect is enabled, the learning is *local*, and each subtask has its own RMA database, separate from the parent and each other. The learnings from each sub task is applied in the subsequent `check_fv` run.

RMA also supports merging of results across tasks that enables you to fully utilize the learnings from properties in different tasks.

For more details on RMA support in Proof Architect, see the *VC Formal Beta Features Guide*.

19.13 Proof Architect: Known Limitations

The following are the known limitations of the Proof Architect feature:

- Session save/restore is not supported.
- Deletion of Proof Strategies or tasks created through Proof Strategies is not supported.
- RMA sharing across task is not supported. For more information on the Proof Architect RMA support, see the *VC Formal Beta Features Guide*.

20

Obtaining Formal Signoff For a Design

This section describes the VC Formal Signoff process and contains the following sections: the

- Introduction to Signoff
- Methodology
- Standard Use
- COI (Property Density) Analysis
- Over-constraint (OC) Analysis
- Formal Core (FC) Analysis
- Formal Testbench Analyzer (FTA)
- Handling Waivers in Coverage Flow
- Using the VC Formal Signoff Dashboard

20.1 Introduction to Signoff

The VC Formal signoff flow has been developed to allow for measurement of quantifiable metrics on the quality of an FPV environment. The signoff flow is a set of steps which provide different pieces of information and levels of confidence in the quality of a formal environment. Each of the steps will be discussed in this chapter.

20.2 Methodology

There are five different aspects to the signoff flow:

- Property Density
- Over-constraint Analysis
- Bounded Analysis

Feedback

- Formal Core
- FTA

Each of these steps can be run independently, there are no dependencies on one another. However, there is a recommended use flow discussed in section Standard Use.

Note:

You should enable the witness as the first step to signoff if you have done so during the FPV testbench development phase. If not, there may be hidden issues with the constraints and the environment that can impact the results. You should also debug any witness trace that is not coverable either in the checker or constraint.

20.2.1 Inputs

The signoff flow is designed to work on an FPV environment. The inputs for each stage of the signoff flow differ slightly and the requirements for each are detailed in the Table 38.

Table 38: Inputs for Signoff Flow

Step	Min Required	Optional	Result
Property Density	Set of assertions	set_fml_var fml_enable_prop_density_cov_map_true-cov [opts] switch with read_file/elaborateIncluding the above options enables coverage mapping from Property Density	COI coverage db (with optional command)COI text report
Over-constraint Analysis	Set of constraints-cov [opts] switch with read_file/elaborate		Over-constraint coverage db
Bounded Analysis	-cov [opts] switch with read_file/elaborate Set of assertions		Bounded reachability coverage db

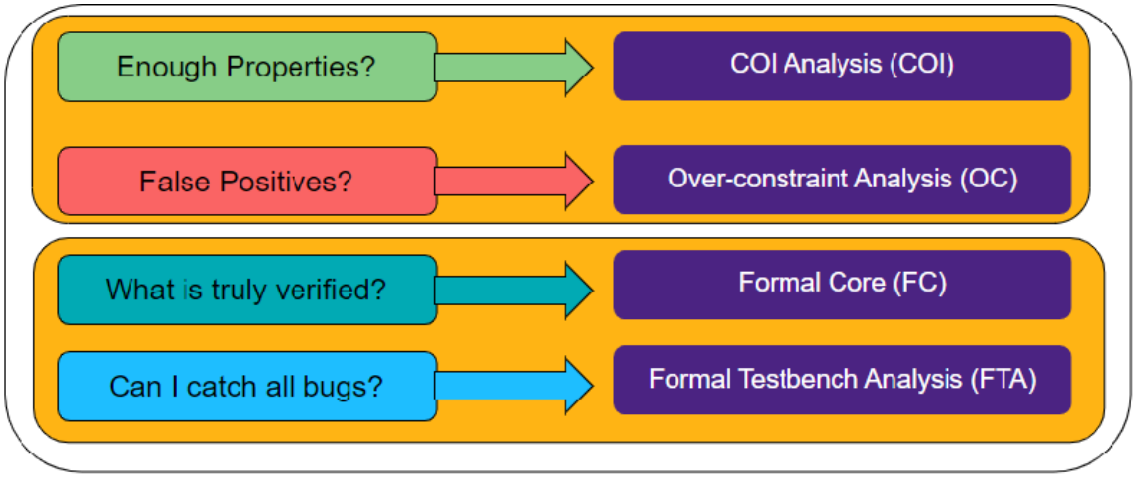
Feedback

Table 38: Inputs for Signoff Flow (Continued)

Step	Min Required	Optional	Result
Formal Core	Set of proven or bounded assertions	-cov [opts] switch with read_file/elaborateI ncluding the above option enables coverage mapping from Formal Core	Formal core coverage db (with optional command)Formal core text reportCompliment formal core text report
FTA	Set of proven or bounded assertions-inject_fa ult [opts] switch with read_file/elaborate	See FTA chapter for more details	See FTA chapter for more details

20.3 Standard Use

The following figure provides the recommended use model for the signoff flow.



For more details on the complete VC Formal signoff flow, see the *VC Formal Signoff Methodology Guide*.

20.3.1 Signoff Setup

Use the `signoff_config` command to specify the coverage metrics and fault models you would like to run. The advantage of using the `signoff_config` command is that it sets the following commonly used variables in the formal signoff flow automatically.

```
set_fml_var fml_cov_enable_branch_cov true (if -type branch is used)
set_app_var fml_cov_tgl_input_port true
set_fml_var fml_enable_prop_density_cov_map true
set_fml_var fml_reset_property_check true
set_fml_var fml_formal_core_on_the_fly true
set_fml_var fml_formal_core_reduced_constraints false
set_fml_var fml_track_loc_reset true
set_fml_var fml_track_loc_const_select true
set_app_var keep_source_loc_expr true
set_fml_var fml_cov_par_block_analysis true
set_fml_var  fml_qual_fault_in_fcore true
set_fml_var fml_fta_fast_activate true
set_fml_var fml_multi_step_fta_flow true
fta_init -fast_sanity
```

Syntax

```
vcf>signoff_config -help
Usage: signoff_config    # Configures the options for signoff
 computation.
        -type <line | cond | toggle | fsm_state | fsm_trans | branch | cg
 | fault_rtl | fault_conn | cov_all | fault_all | all>
                                (The metrics you want to monitor for
 signoff.)
        [-hierarchy <hierarchy>] (Flexibility to enable/disable coverage
 on specific modules/instances in signoff flow. The file specified has
 the same syntax as hier file passed using -cm_hier switch. Please refer
 to the coverage user guide for more information.)
        [-enable_coft_flow <true|false>](Enable new Signoff flow.
 Default: false.)
```

Examples

- To run COI/OC:

  ```
  signoff_config -type "line cond toggle branch"
  ```

- To run FTA:

  ```
  signoff_config -type "line branch fault_rtl fault_conn"
  ```

  Warning: The FTA signoff metrics are not supported in the SEQ flow. Therefore, the `-type all` option for the `signoff_config` command is ignored and the `-type cov_all` option is used. The message tag applicable is `SEQ_SIGNOFF_TYPE_ALL`.

20.4 COI (Property Density) Analysis

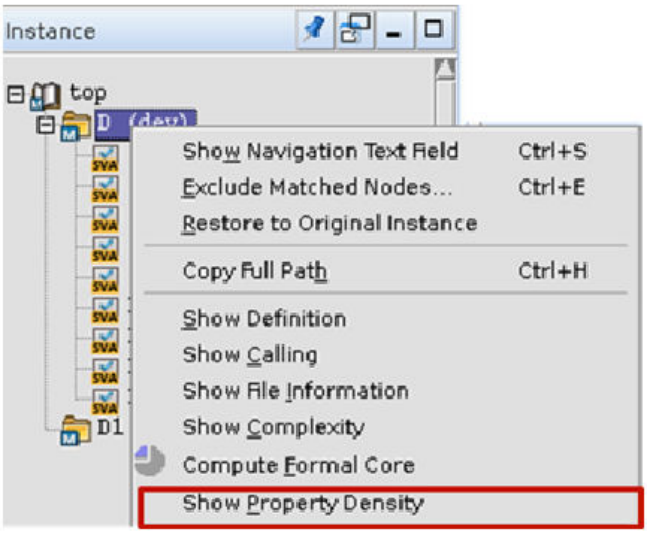
Property density performs a structural analysis of the design to determine the Cone of Influence (COI) of the assertions. Property density is intended to find the easiest to find holes in the verification environment. If something is within the property density report, it does not mean that it is fully verified, however, something which is outside of the report

Feedback

has not been tested by any assertion. Property density is structural and has a quick runtime, so it is always recommended to run the step first. To run property density in the VC Formal GUI, select the following:

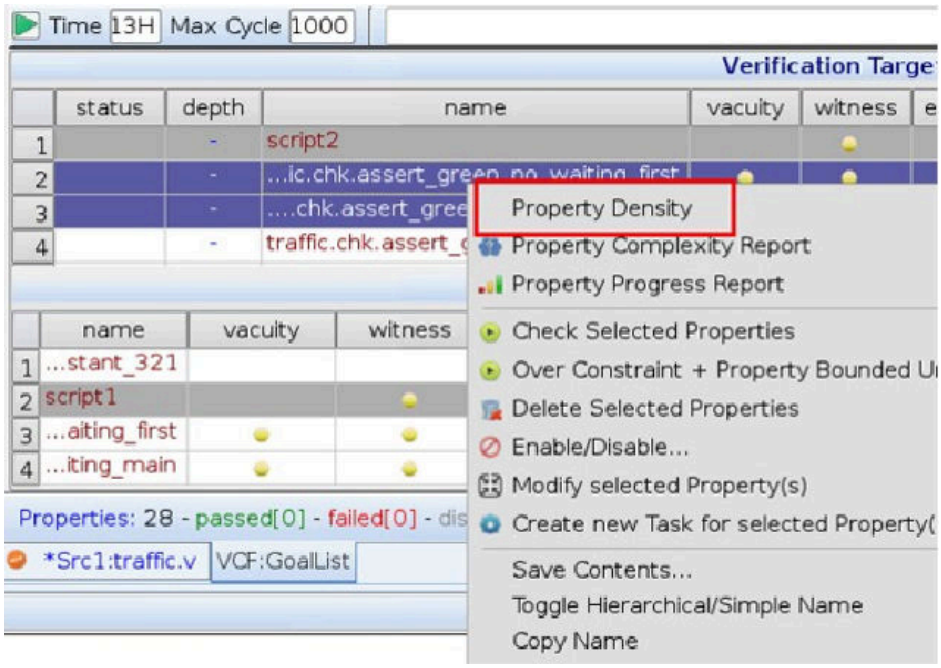
20.4.1 Viewing the Property Density Report

- You can also view the property density for a specific instance by selecting property density from the instance tree hierarchy as shown in the figure below.



- Run property density for selected properties from the RMB menu.

Feedback



- Generate the property density report in text format using the following command:

```
report_assertion_density
```

Syntax

```
vcf> report_assertion_density -help
Usage: report_assertion_density    # Report line count and number of
 assertions/covers in given scope
        [-scope <scope>]        (Starting scope)
        [-top <top>]            (Design to be considered as top)
        [-abstracted]           (Report abstracted COI which
                                 considers the snips applied on the
                                 design. By default, tool reports
                                 structural COI.)
        [-xml <file_name>]      (Generates the report in XML format in
                                 the given file)
        [-file <file_name>]     (Dump the report in the given file)
        [-gui]                  (Show report in Verdi property
                                 density view)
        [-property <list-or-collection-of-properties>]
                                (List or collection of properties)
        [-skip <skip-type-list>]
                                (List of object types to be skipped
                                 from report: Values: noLoadReg)
        [-exclude_instances <list-of-instances>]
                                (List of instances to be excluded
                                 from the report)
        [-include <include-type-list>]
```

Feedback

```
                                (List of objects to be included:
                                 Values: scriptProps)
        [-list]                 (Reports the output in a list format)
        [-type <list-of-signal-types>]
                                (List of signal types to be included:
                                 Values: all, REG, PI, PO, reg, pi, po)
        [-status <list-of-status-types>]
                                (List of status types to be included:
                                 Values: all, covered, uncovered,
total)
```

- When the `-scope` option is specified, only the internals as well as properties which lie inside the scope are considered when calculating property density. Nothing outside is considered.

  When the `-top` option is specified, the internals of the module/instance given as the argument, and properties of the whole design are picked when calculating the property density. In the `-top` option, mention the module name as an argument if only single instance of that module is present in design. If multiple instances are there, then specify the module and the instance in following format: `-top module_name:instance_name.`

- The `report_assertion_density` command reports the COI in structural mode by default, and in the abstracted mode when the *-abstracted* option is specified explicitly. If you want to switch back to structural mode while in abstracted mode, run the `report_assertion_density` command again.

- To get the total registers, primary inputs, and primary outputs, use the `-status total` option:

  Examples

  - `report_assertion_density -status total`

    ```
    vcf> report_assertion_density -status total
    ****************************************
    Design:                                 traffic
    Total Asserts:                          18
    Total Covers:                           10
    Total Registers:                        13
    Total Primary Inputs:                   4
    Total Primary Outputs:                  6
    ****************************************
    1
    ```

  - `report_assertion_density -status total -type all`

  - `report_assertion_density -status total -type reg`

Feedback

```
vcf> report_assertion_density -status total -type reg
****************************************
Design:                                  traffic
Total Asserts:                           18
Total Covers:                            10
Total Registers:                         13
****************************************
```

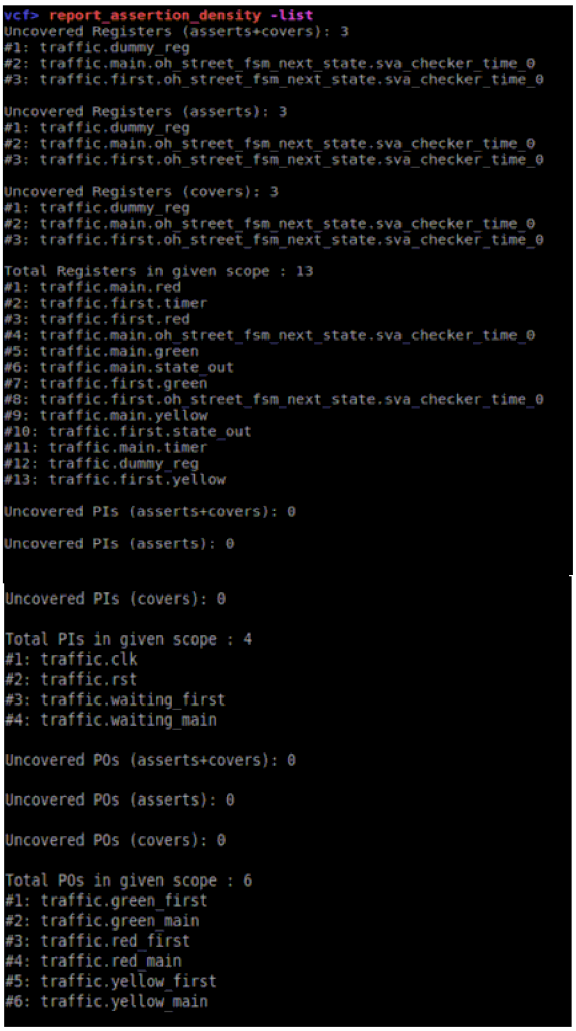
    - `report_assertion_density -status total -type pi`
    - `report_assertion_density -status total -type po`

- To get a report of the total registers, primary inputs, primary outputs in default mode along with uncovered registers, primary inputs, primary outputs, use the following commands:

  Examples

Feedback

- `report_assertion_density -list`



Feedback

- `report_assertion_density`

```
vcf> report_assertion_density
****************************************
Design:                                 traffic
Total Asserts:                          18
Total Covers:                           10
Total Registers:                        13
Total Primary Inputs:                   4
Total Primary Outputs:                  6
Uncovered Registers(Asserts+Covers)     3
Total Register Density(Asserts+Covers) 76.92%
Uncovered PI(Asserts+Covers)            0
Total PI Density(Asserts+Covers)        100.00%
Uncovered PO(Asserts+Covers)            0
Total PO Density(Asserts+Covers)        100.00%
Uncovered Registers(Asserts):           3
Assertion Density(Registers):           76.92%
Uncovered PI(Asserts):                  0
Assertion Density(PI):                  100.00%
Uncovered PO(Asserts):                  0
Assertion Density(PO):                  100.00%
Uncovered Registers(Covers):            3
Cover Density(Registers):               76.92%
Uncovered PI(Covers):                   0
Cover Density(PI):                      100.00%
Uncovered PO(Covers):                   0
Cover Density(PO):                      100.00%
****************************************
```

- The `-status` and `-type` options are independent of the -list option, that is, when the `-status` and`-type`option is specified, and the `-list`option is not specified, the output is reported in summary format. When the `-list` option is used along with these switches, then output is reported in the list format.

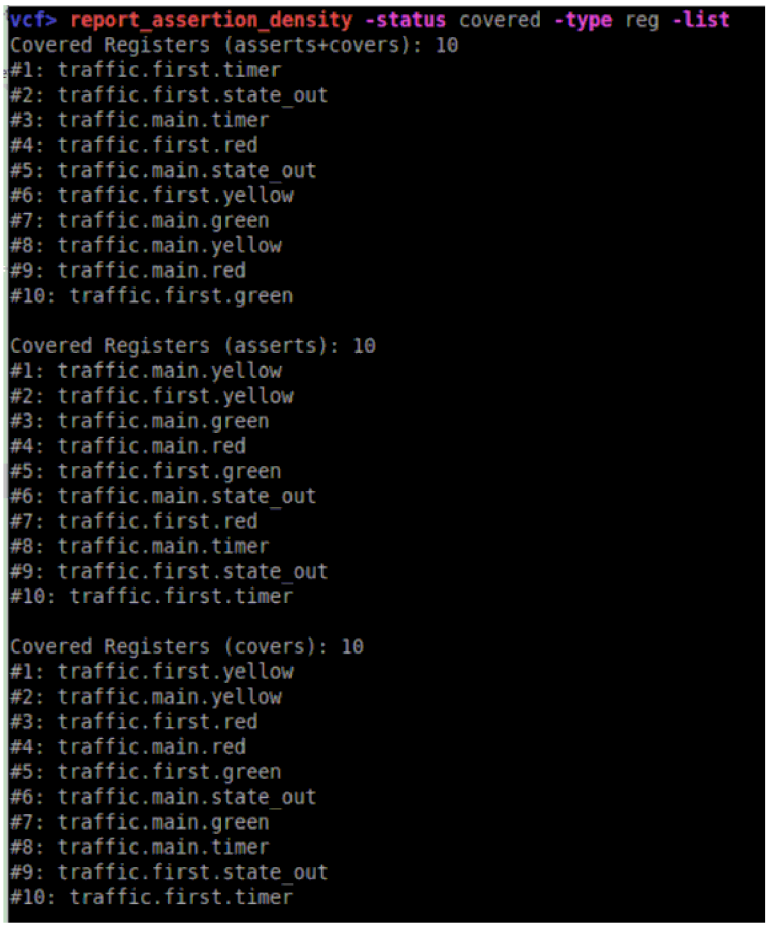
  `Examples`

  - `report_assertion_density -status covered -type reg`

```
vcf> report_assertion_density -status covered -type reg
****************************************
Design:                                 traffic
Total Asserts:                          18
Total Covers:                           10
Covered Registers(Asserts+Covers)       10
Covered Registers(Asserts)              10
Covered Registers(Covers):              10
****************************************
```

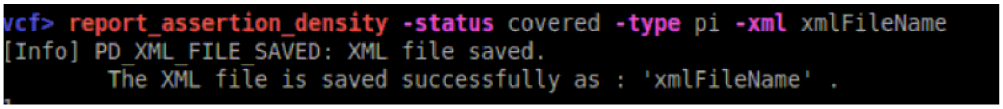
Feedback

  - `report_assertion_density -status covered -type reg -list`



- To get the xml report according to values given by the `-status` and `-type` option:

```
report_assertion_density -status covered -type pi -xml xmlFileName
```



- The following is an example output in Tcl format where the covered and uncovered information is grouped separately for the asserts and covers:

```
vcf>report_assertion_density -list -status all -type reg
Uncovered Registers (asserts+covers): 0
Uncovered Registers (asserts): 0
Uncovered Registers (covers): 3
#1: arbiter.snps_arb_i.prev_req
#2: arbiter.snps_arb_i.gnt_p1_last_seen
```

Feedback

```
#3: arbiter.snps_arb_i.last_gnt
Covered Registers (asserts+covers): 5
#1: arbiter.rotate_ptr[1:0]
#2: arbiter.grant
#3: arbiter.snps_arb_i.prev_req
#4: arbiter.snps_arb_i.last_gnt
#5: arbiter.snps_arb_i.gnt_p1_last_seen
Covered Registers (asserts): 5
#1: arbiter.snps_arb_i.last_gnt
#2: arbiter.snps_arb_i.gnt_p1_last_seen
#3: arbiter.snps_arb_i.prev_req
#4: arbiter.grant
#5: arbiter.rotate_ptr[1:0]
Covered Registers (covers): 2
#1: arbiter.rotate_ptr[1:0]
#2: arbiter.grant
Total Registers in given scope : 5
#1: arbiter.snps_arb_i.gnt_p1_last_seen
#2: arbiter.snps_arb_i.last_gnt
#3: arbiter.snps_arb_i.prev_req
#4: arbiter.grant
#5: arbiter.rotate_ptr[1:0]
```

Feedback

The property density report is displayed as follows:

VCF:PropertyDensity

<Filter Uncovered Registers, PIs and POs>

Property Density COI (Structural)	Rate
Design: traffic	
Total Asserts - 16	
Total Covers - 8	
Total Registers - 11	
Total Coverable Lines - 67	
Total Primary Inputs - 4	
Total Primary Outputs - 6	
Asserts+Covers	Covered Rate (%)
Uncovered Registers - 1	90.91%
traffic.s_reg	
Uncovered Lines - 6	91.04%
Uncovered Primary Inputs - 0	100.00%
Uncovered Primary Outputs - 0	100.00%
Asserts	Covered Rate (%)
Uncovered Registers - 1	90.91%
traffic.s_reg	
Uncovered Lines - 6	91.04%
Uncovered Primary Inputs - 0	100.00%
Uncovered Primary Outputs - 0	100.00%
Covers	Covered Rate (%)
Uncovered Registers - 3	72.73%
traffic.s_reg	
traffic.first.yellow	
traffic.main.yellow	
Uncovered Lines - 12	82.09%
Uncovered Primary Inputs - 0	100.00%
Uncovered Primary Outputs - 2	66.67%
traffic.yellow_first	
traffic.yellow_main	

This report gives several pieces of information. For formal signoff, the most interesting section is the *Uncovered Registers (asserts)*section. This gives details of all registers in the design space that have not been tested by any assertion. In the report above, only 45.11% is shown as being within the property density report, so more assertions are needed.

20.4.2 Property Density Toolbar

The property density toolbar is as illustrated Figure 167.

Figure 167: Property Density Toolbar

Using the toolbar (left to right), you can:

- Refresh the property density report
- Switch the report between abstracted COI and structural COI (default)
- View the schematic

Feedback

- Map the property density results to coverage
- Filter properties or registers

Refreshing the Property Density Report

When you change the asserts or covers, the property density report must be updated to reflect this change.

The refresh button turns orange color, , whenever there is an update to the property density report, indicating that the property density report must be refreshed to get the updated values.

When you snip the registers, the abstracted COI can change, and the refresh button turns to orange color indicating that the property density report must be refreshed to obtain the updated values.

Switching between Abstracted COI and Structural COI

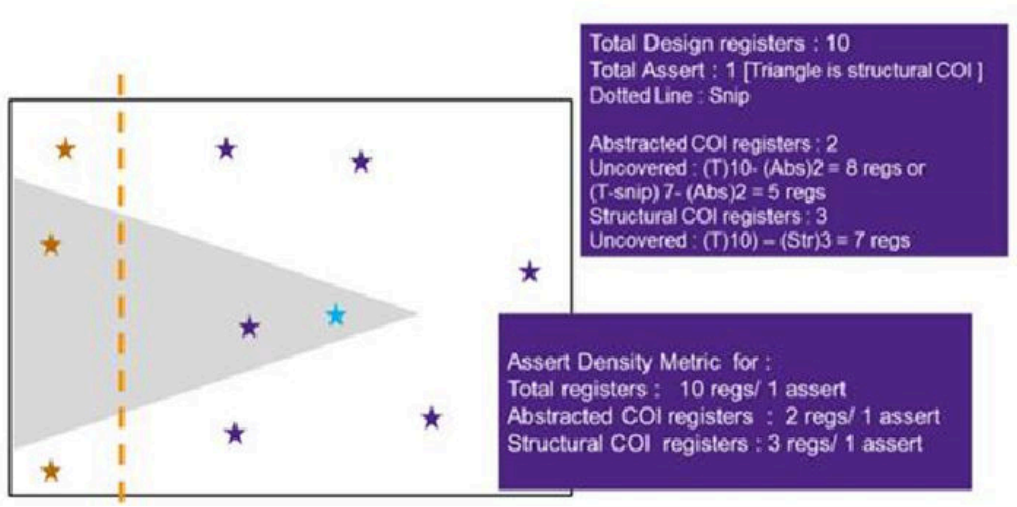
- To view the property density for abstracted COI, click .
- To view the property density for structural COI, click .

You can view the property density of the abstracted COI and structural COI separately. By default, the property density for structural COI is displayed, since structural COI analyzes the whole design. The abstracted COI considers the snips in the design, whereas the structural COI ignores them. For basic usage, structural COI is easy to use and relatively straightforward to understand. But if some parts of design are not going to be considered in the verification scope. Snip points can be widely applied to reduce the design. And abstracted COI can automatically ignore those parts for review efficiency. Figure 168 describes the difference between abstracted COI and structural COI in a graphical illustration.

The design has one assert property, and it covers three registers. If you use snip_driver to simplify the property checker, less registers are tested by the property.

Feedback

Figure 168: Difference Between Abstracted COI and Structural COI



Viewing Source and Schematic

You can view the source and schematic for registers from the property density report.

Note:

You can open the schematics of registers only. Hence, the schematic option is available only for registers.

To open the source of asserts, covers, registers:

- Select an assert, cover, register in the property density report, and from the RMB menu, select *Open Source*.

  Alternatively, you can double-click the property or register.

To open the schematic of a register:

- Select a register in the property density report, and from the RMB menu, select *Open Schematic*.

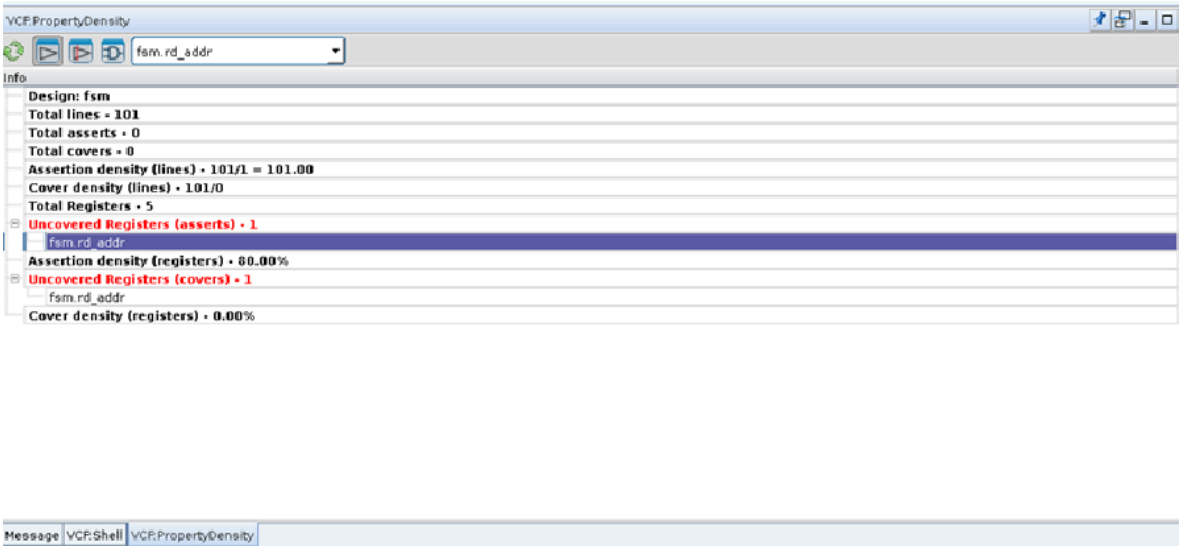
  Alternatively, you can click in the property density toolbar.

Filtering Properties and Registers

You can use the text box in the property density toolbar to filter properties and registers based on the names.

Feedback

Figure 169: Filtering Asserts and Registers



Mapping Property Density to Coverage

It is possible to map property density with the coverage database and review the regions of the RTL that are not covered in the Verdi Coverage GUI.

Use Model

- To enable mapping property density results to coverage, set the following formal variable to true:

  ```
  set_fml_var fml_enable_prop_density_cov_map true
  ```

  Note:

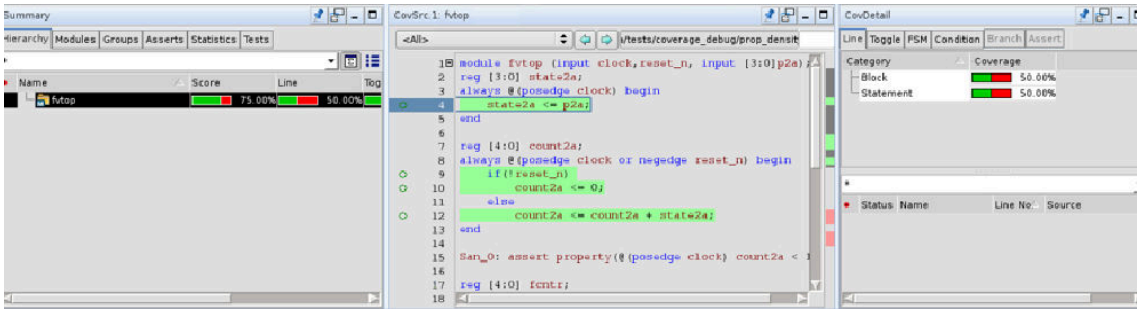
  By default, only assert properties are mapped to coverage. To map the cover properties, set the following formal variable to true: `set_fml_var fml_enable_covers_in_prop_density_cov true`

- Click to map the property density to coverage, and load it into the Verdi Coverage GUI.

  The property density (fan-in cone of user properties) is mapped to coverage metrics such as line, tgl, cond. The results are saved in the coverage database. You can view

Feedback

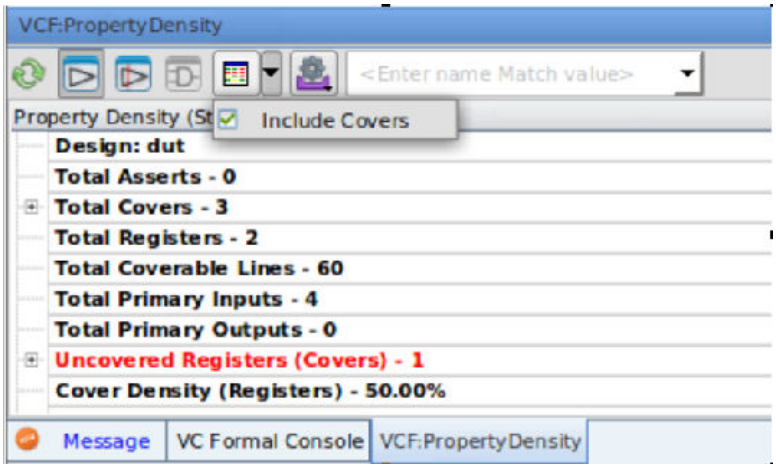
the results in Verdi Coverage GUI, and find holes in the RTL, which are not covered by user properties.



Viewing Covers in the Property Density Coverage Report

To add cover properties in addition to the assert properties in the coverage report, select *Include Covers*.

Figure 170: Adding Covers in the Coverage Report



20.4.3 Running Property Density (COI) in Command Mode

Use the following commands to perform the COI analysis:

```
save_property_density_results -db_name COI
view_coverage -cov_input COI -mode PD -reload
```

Use the `-include_covers` option in the `save_property_density_results` command to include cover properties for mapping to coverage. By default, only assert properties are mapped to coverage database.

The COI analysis results are saved in the Verdi coverage database that can be opened and viewed later.

Feedback

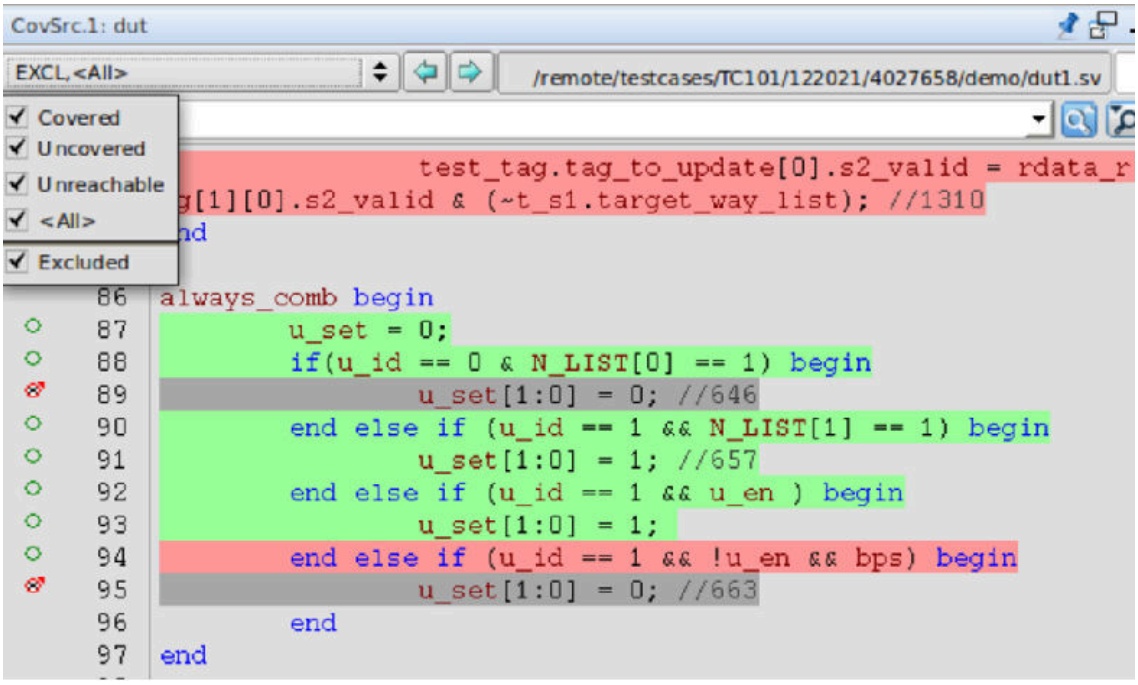
This video covers how to analyze property density and the Cone of Influence (COI) using VC Formal. The video demonstrates how to assess the testbench status and identify verification gaps. This ensures comprehensive coverage and helps refine your testbench for robust verification.

20.4.4 Running Deadcode Checks in COI Analysis

To perform a light weight deadcode checks in COI, and generate dead code information (FPV_PD_deadcode.el), set the following variable before the `save_property_density_results` command.

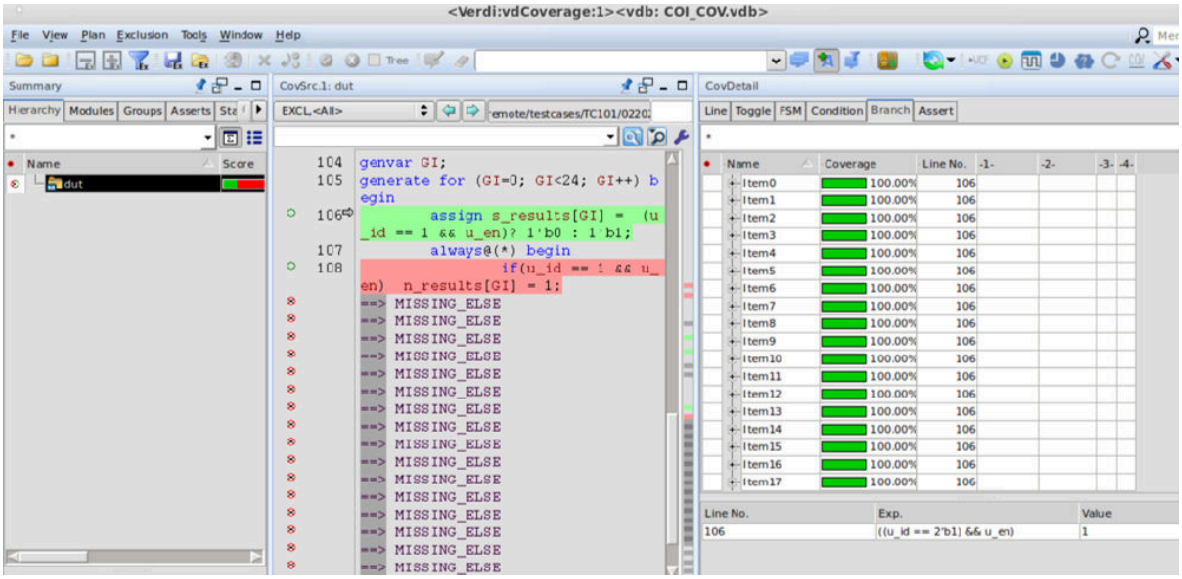
The `save_property_density_results` command creates and reads in the waiver file *FPV_PD_deadode.el* automatically. You need not specify the `-elfiles` switch in the `view_coverage` command.

```
set_fml_var fml_enable_coi_deadcode_check true -global
save_property_density_results -par_task FPV -db_name COI
view_coverage -cov_input COI -mode PD
```



With deadcode analysis, the MISSING_ELSE are excluded by FPV_PD_deadcode.el too.

Feedback



20.4.5 Running Bit-Level Checks in COI Analysis

By default, COI analysis is word-level. To enable bit-level checks in COI, set this variable before `report_assertion_density`.

```
set_fml_var fml_enable_prop_density_cov_bittravs true -global
```

Note:

This is a costly process and may take more time to run COI analysis and the mapping.

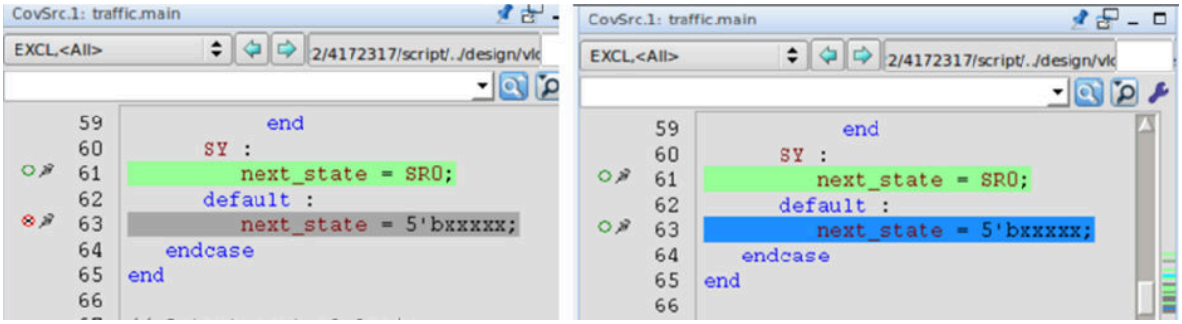
20.4.6 Reviewing COI Results in Verdi Coverage

You can find the following statuses after COI results are mapped to Verdi Coverage. The following figure shows the color coding applied for each status.

	Meaning	Color coding
COI	Inside COI	Covered
	Outside COI	Uncovered
	UNR WO Constraints (With FPV_PD_Deadcode.el)	Unreachable
	UNR WO Constraints (Unexclude)	Unreachable

Feedback

The following is an example snippet of the COI results with color coding.

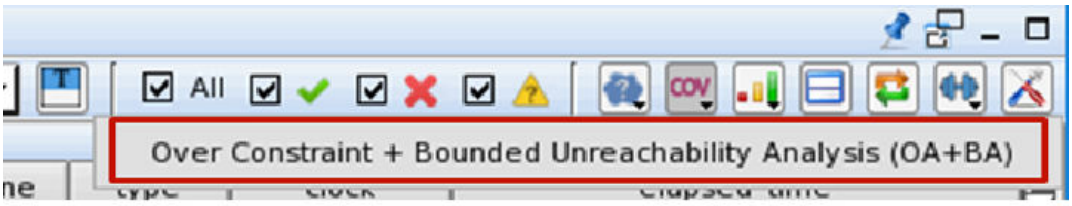


This is a video tutorial on how to enable the property density coverage option to view the coverage metrics in the Verdi GUI to show covered, uncovered, and excluded metrics.

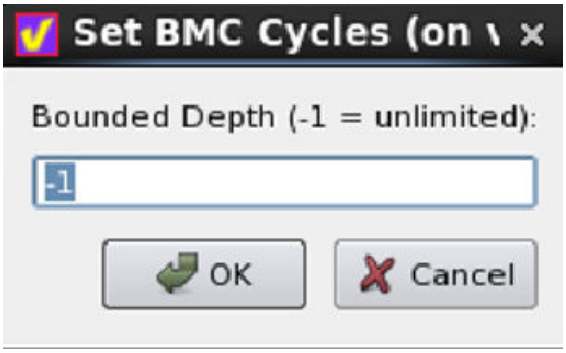
20.5 Over-constraint (OC) Analysis

Over-constraint (OC) analysis is designed to ensure that there are no constraints in the design preventing legal areas of code from being exercised. The analysis works much in the same way as an FCA reachability analysis, however for over-constraint analysis, VC Formal differentiates between areas of code that are unreachable with constraints and unreachable without constraints. Over-constraint analysis is an important step as while under-constraints may cause false property failures, over-constraints can cause false proofs.

To run over-constraint analysis in the VC Formal GUI, ensure that the correct coverage metrics are specified in the `signoff_config` command. VC Formal performs analysis on the defined metrics. To start the analysis, select the option below:

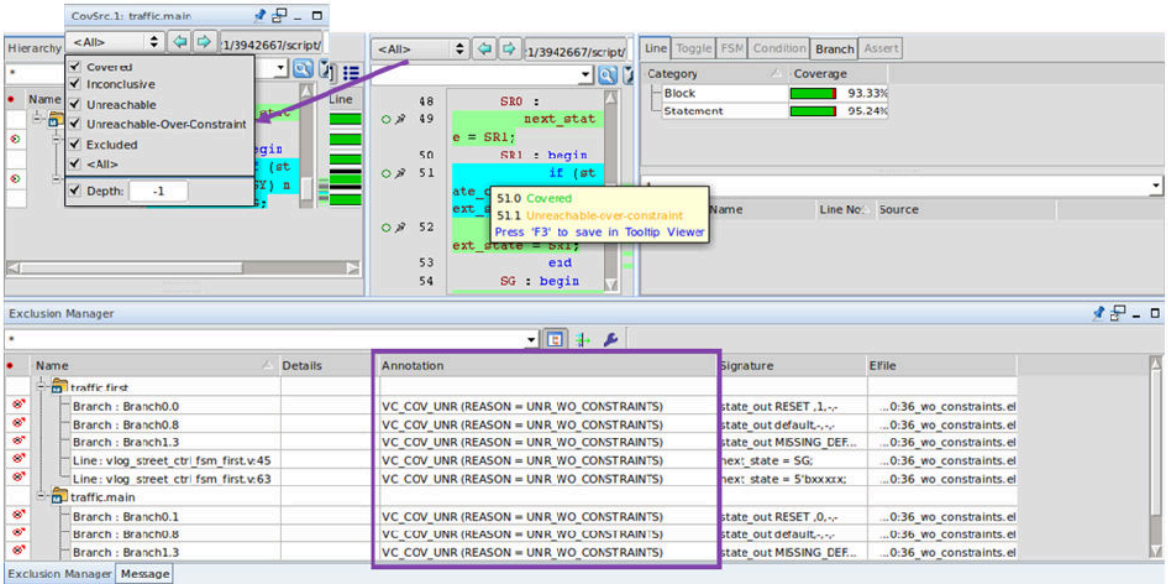


This will bring up a prompt asking the number of cycles for evaluation.



For unlimited over-constraint analysis, specify -1 in the *Bounded Depth* box. This represents a completely unbounded analysis of the state space. If you want to explore up to a specific depth then enter the depth you would like to explore up to here.

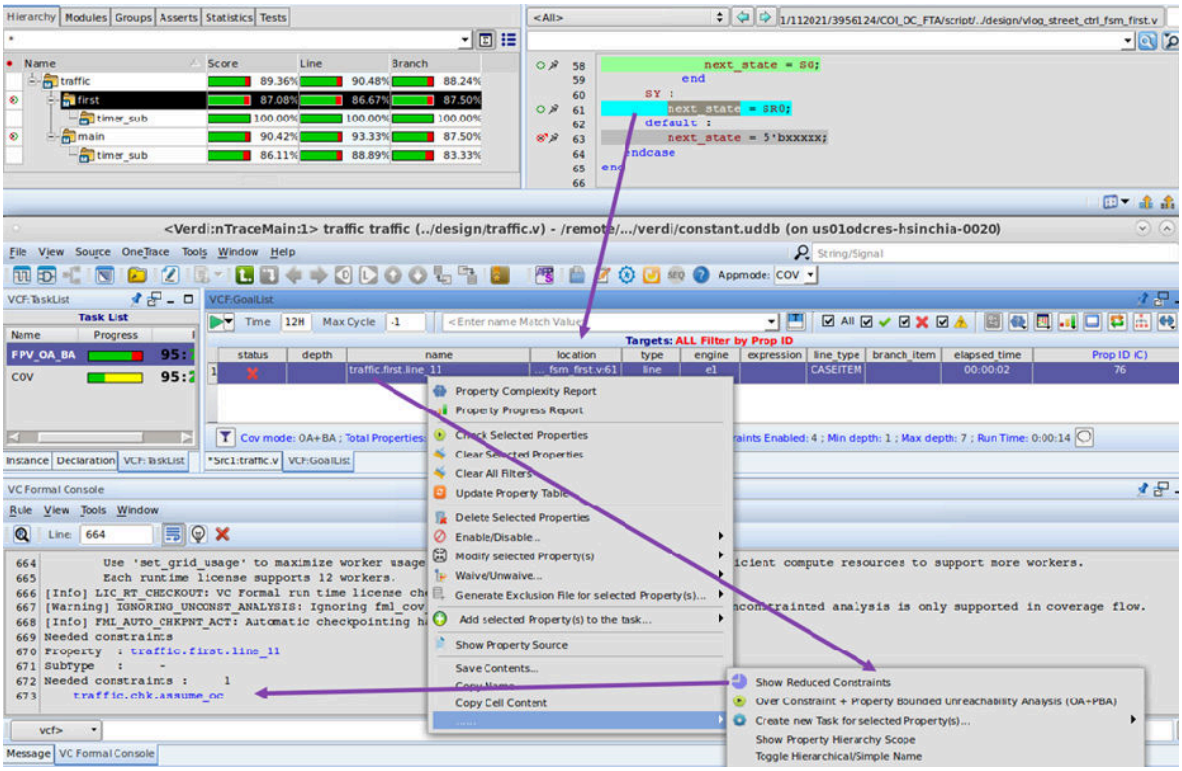
After selecting this a new task will open in the COV app and the analysis will start. While the analysis is running, the Verdi Coverage GUI will also open. Initially this will not be populated with much coverage data as the task has not completed. Once the analysis completes a prompt will appear asking if you would like the database to be updated.



In the Verdi GUI it is possible to review the results and visualize which areas of the design are unreachable. Unreachable and unreachable due to over-constraint are highlighted in different colors, and hovering over the code will pop up a message as to what the result is. To map an unreachable target back to its corresponding cover property, select the target and click *Show Reduced Constraints*.

This will show a single property back in the VC Formal GUI target pane. To understand which constraints caused this property to be unreachable right click on the property and select *Show Reduced Constraints*.

Feedback



The tool will then compute a reduced set of constraints that have influenced this property, and they will be the constraints remaining in the constraint pane. This helps to understand what may be over-constaining the design space.

20.5.1 Running Over-constraint Analysis in Command Mode

Over-constraint analysis can also be performed using the following Tcl commands:

1. Run and save reset information

```
sim_run -stable
sim_save_reset
```

2. Running over-constraint analysis

```
compute_bounded_cov -par_task FPV -max_proof_depth -1 -block
```

3. Save coverage database (OC.vdb) and exclusion files

```
save_bounded_cov_results -db_name OC
```

4. View coverage results

```
view_coverage -cov_input OC -mode OA+BA -elfiles "
 FPV_OA_BA_unr.el_wo_constraints.el "
```

Feedback

5. Running over-constraint for assertion COI only

```
compute_per_prop_bounded_cov -prop [get_props -usage assert] -par_task
 FPV -max_proof_depth -1 -block
```

20.5.2 Reviewing OC Results in Verdi Coverage

You can find the following statuses after OC results are mapped to Verdi Coverage:

Figure 171: Covered Status

	Meaning	Color coding
OC	Reachable	Covered

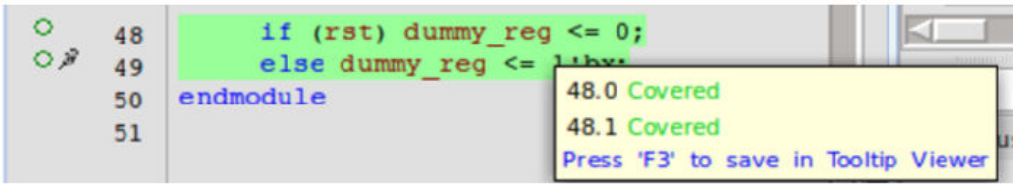
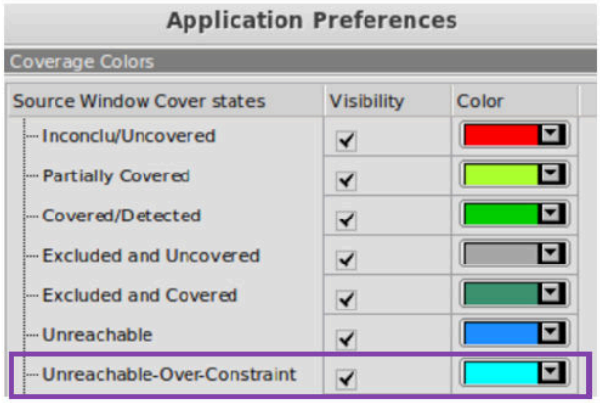
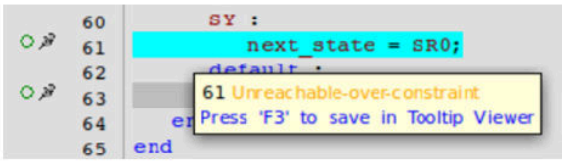


Figure 172: Unreachable-Over-Constraints

	Meaning	Color coding
OC	Unreachable W/ Constraints	Unreachable-Over-Constraints





Feedback

Figure 173: Unreachable/Excluded and Uncovered

	Meaning	Color coding
OC	Unreachable WO Constraints, excluded by FPV_OA_BA_unr.el_wo_constraints.el	Unreachable
	Unreachable WO Constraints, unexclude/disable auto loading FPV_OA_BA_unr.el_wo_constraints.el	Unreachable

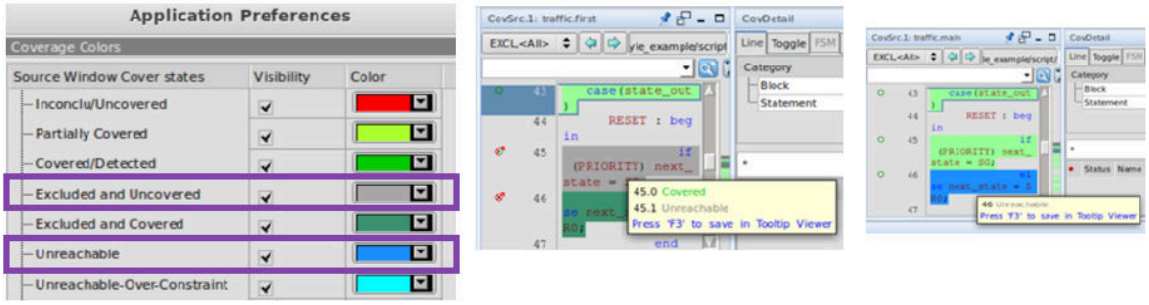
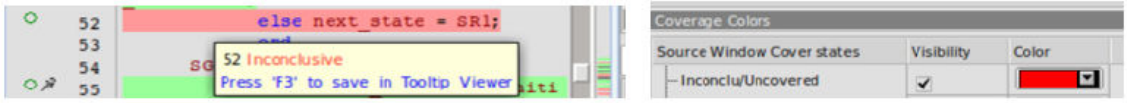


Figure 174: Inconclusive

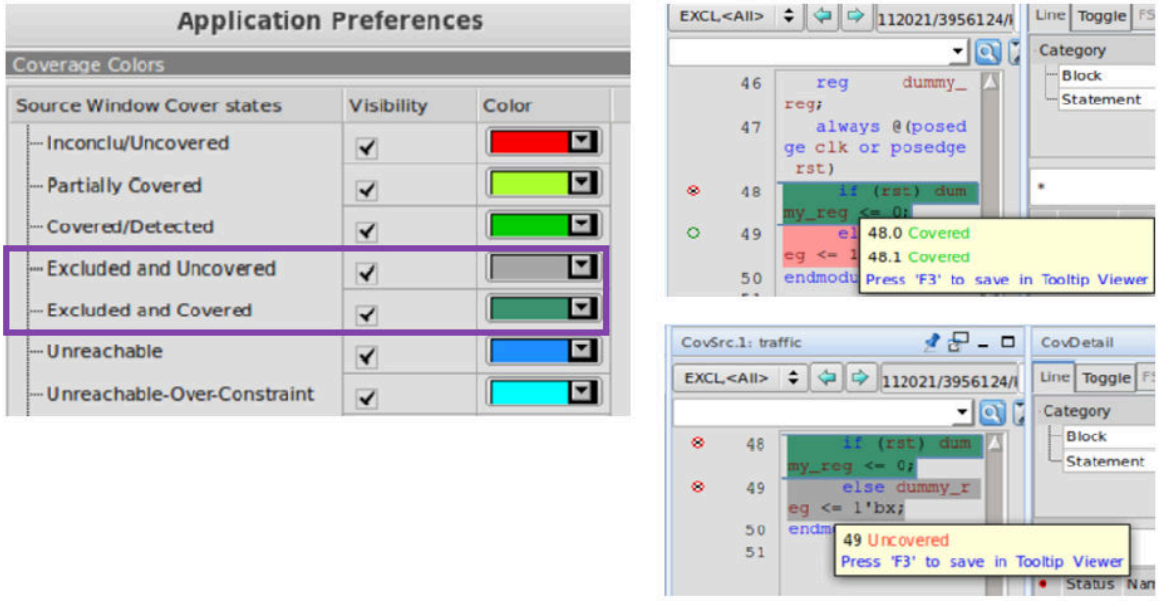
	Meaning	Color coding
OC	Inconclusive (Including unreachable with unknown reason)	Inconclusive



Feedback

Figure 175: Excluded Uncovered and Excluded and Covered

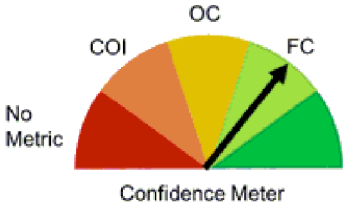
	Meaning	Color coding
OC	User Exclude and originally Covered (after recalculate)	Covered
	User Excluded and originally Unreachable-OC/Uncovered/Inconclusive	Unreachable-OC/ Uncovered/Inconclusive



20.6 Formal Core (FC) Analysis

Formal Core (FC) is defined as a sub-set of COI that is used by a formal engine to verify a given property. It is always less than or equal to the COI of the property. If there are multiple properties being verified, the formal core is the union for all the properties. In formal core analysis, the engine traverses through the hierarchy and extracts registers, inputs, and constraints responsible for the proof of an assertion.

Feedback



The above definition of formal core also applies to the properties with *bounded* proofs. If a property has a bounded proof up to a depth k (no c/ounter examples up to depth k), then you can get the formal core for the property for a depth less than equal to k. In such cases, the formal core contains the registers, constraints, inputs used by the formal engines for the bounded proof up to the specified depth k.

Formal core is a more accurate measure of which areas of the design have been involved in the proof (or bounded proof) of a property. The formal core will be a subset of the COI and is more accurate than property density, however it performs a formal analysis so the recommendation is to flush out early verification holes using property density first.

Note:

The formal core can over-approximate the set of registers, constraints, and inputs that are needed to prove a property. There is no guarantee that VC Formal will report the minimal set of registers, constraints, and inputs. Different formal engines can report a different formal core for the same property. There is no guarantee that the smallest or the most accurate formal core is always reported.

20.6.1 Use Model for Computing Formal Core

Perform the following steps to compute formal core:

1. Configure signoff setup (see section Signoff Setup)
2. Read design and properties (see section Compiling Designs)
3. Set the clock and resets (see section Setting Up User-Defined Clocks, Setting Up User-defined Resets)
4. Perform formal checks (see section Formal Runtime Control)
5. Compute formal core (see section Computing Formal Core in Tcl)
6. Get a report of the formal core (see section Reporting Formal Core in Tcl)

Feedback

20.6.2 Computing Formal Core in Tcl

Use the `compute_formal_core` command to compute the formal core for a proven property, a bounded proven property, and vacuity components in constraints.

By default, VC Formal computes formal core for both proven and inconclusive properties together and provides intermediate bounded formal cores during the computation.

You can also use `-minimize regs` to further reduce the number of registers used in the formal core. Note that this option may take more time to compute formal core, and the result is still minimal but not necessarily minimum.

To compute the formal core in SEQ application mode, you can use `-proof <name>` and `-goalIds <list-of-goal-ids>` options.

To see progress messages when computing formal core either as part of `check_fv` (on-the-fly enabled) or `compute_formal_core`. Set the following formal variable:

```
set_fml_var fml_formal_core_verbose_level <0|1|2>
```

Where,

0: Do not print any messages.

1: Print messages when formal core is derived from an engine proof. Default value.

2: In addition to level 1 messages include bounded formal core update messages.

By default, VC Formal generates formal core for liveness properties. To disable generation of formal core for liveness properties, set the following formal variable to false:

```
set_fml_var fml_formal_core_liveness false
```

Use Model

```
vcf> compute_formal_core -help
Usage: compute_formal_core    # Compute the formal core used in proving a
 property
        [-block]               (Makes command input block while this
 command is active.)
        [-stop]                (Stops execution)
        [-property <list-or-collection-of-properties>]
                               (List or collection of properties)
        [-subtype <subtype-attribute>]
                               (Specify sub type of the property:
                                Values: vacuity, witness)
        [-depth int]           (Bounded depth to reach to report bounded
 results.)
 [-goalIds <list-of-goal-ids>]
                          (List of goal ids (for SEQ app only.))
   [-proof string]        (Proof name (for SEQ app only.))
   [-minimize <minimization-attribute>]
```

Feedback

```
                              (Attempt to minimize formal core.:
                               Values: regs)
```

Note:

When -depth option is not specified, only the proven properties in the specified set of properties are considered. Error messages are reported for the properties that are not in proven state. When -depth <k> option is specified, in addition to proven properties, formal core is computed even for inconclusive properties which have bounded proof of depth >= k from the specified set of properties.

Examples

- To compute formal core on all proven assertions:

```
compute_formal_core
```

- To compute formal core for vacuity properties in constraints:

```
compute_formal_core -property <constraint_name> -subtype vacuity
 -block
report_formal_core -subtype vacuity
```

Note:

Formal core must be computed before you can execute the report command.

- By default, VC Formal computes the formal cores for both proven and inconclusive properties. To compute formal core either of the proven or inconclusive properties, use the following commands.

```
set_fml_var fml_formal_core_incremental false
compute_formal_core -property [get_props -usage assert -status
 {inconclusive}]
```

- To compute formal core up to a depth 5 for inconclusive properties:

```
compute_formal_core -property [get_props -status inconcl] -depth 5
```

Note:

The inconclusive properties that have depth > 5 are ignored.

- To compute formal core for the property names matching a pattern:

```
compute_formal_core -property *abc*
```

- By default, VC Formal generates formal core for liveness properties. To disable generation of formal core for liveness properties, set the following formal variable to false:

  ```
  set_fml_var fml_formal_core_liveness false
  ```

  Note:

  `The fml_formal_core_reduced_constraints` application variable is ignored for liveness properties.

20.6.3 Reporting Formal Core in Tcl

Use the `report_formal_core` command to get a list of registers, constraints, inputs, and vacuous components present in the formal core of one or more specified properties. By default, the `report_formal_core`commandreports a summary of the following:

- The number of properties the `compute_formal_core` command is working/worked on.
- The number of properties for which the formal core was generated from a proof.
- The number of properties for which formal core was generated from a bounded proof.
- The number of properties for which the formal core is being computed (or could not be computed due to resource limits being reached).

You can use the `-verbose` option to get a detailed report of the formal core.

In addition, if the formal core is from proven (or vacuous) properties, then the status is shown as success and the depth field is null. If the formal core is from uncoverable, falsified properties, the status is shown as failed. If the formal core is from a bounded proof, then the status is shown as bounded and the depth field specifies the depth.

Note that constraints such as environment (`-env`) constraints or `set_constant` constraints are implicitly part of formal core of every property and are not reported explicitly by the `report_formal_core` command. Any registers in formal core that are present in an encrypted region of the design are not reported.

To obtain registers, inputs and constraints, which are not in the formal core of any property, use the `-complement` switch with the `report_formal_core` command.

- `report_formal_core -complement`

To obtain the formal core for a subset of properties, you can use the `-property` switch followed by a list of properties.

- `report_formal_core -property <list-of-properties>`

Feedback

You can also use this command with the `-complement` switch to report all the registers, inputs, and constraints outside of the formal core for a subset of this properties. The `-union` switch reports all the registers, inputs, and constraints that are in the formal core.

You can use the `-bit` option to get the report in bit-level. When using the `-bit` option, the following reporting options are affected.

- In the summary view, the total number of registers reported is the summation of each bit in each register that is in the formal core. Similarly, the total number of inputs reported is the summation of each bit in each input that is in the formal core. Therefore, the number of registers and inputs in bit level are greater than or equal to that of in word level.
- When the `-verbose` option is used, each bit in registers or inputs are reported in one line. Therefore, if a register has 1000 bits and if all of them are in formal core, then 1000 lines are reported for this register.
- When `-complement` option is used, the total number of registers is the summation of each bit in each register that is not in the formal core with respect to the given property specification. Similarly, the total number of inputs is the summation of each bit in each input that is not in the formal core with respect to the given property specification. Note, since a register or input may have some bits in the formal core and some bits are not, thus, the register or input may be shown up in both regular and complement form.
- When `-union` option is used, then the union of the given property specification is reported in bit level.

In addition to registers, inputs, and constraints, you can also use `-comb` option to report combinational logic used in the formal core. By default, this option is set to off. For performance reasons, the `-bit` switch must not be used with the `-comb` switch.

To report formal core of sub-proofs in SEQ application mode, you can use `-proof <name>` and `-goalIds <list-of-goal-ids>` options.

To include the reporting of combinational logic in formal core, use the `-comb`optionin conjunction with the `-union` or`-complement` option in the `report_formal_core` command.

```
report_formal_core [-comb] [-union] [-complement]
```

Use Model

```
vcf> report_formal_core -help
Usage: report_formal_core    # Report formal core for the selected
 properties.
        [-subtype <subtype-attributes>]
                                (List information for goals with a
 particular subtype:
                                 Values: -, vacuity, witness)
        [-property <property-name>]
```

Feedback

```
                                (List or collection of properties for
which formal core should be reported)
       [-list]                  (Outputs one-line report per check)
       [-no_summary]            (Do not print summary)
       [-verbose]               (Print verbose report)
       [-complement]            (Report parts not covered in formal core)
       [-union]                 (Report union of formal core of individual
properties)
       [-runStatus run-status-attributes]
                                (Filter on run status.:
                                 Values: "", completed, scheduled)
       [-status status-attributes]
                                (Filter on status.:
                                 Values: proven, bounded, no_status)
  [-exclude_constraints] (Exclude constraint only contribution from
registers/inputs)
     [-goalIds <list-of-goal-ids>]
                              (List of goal ids (for SEQ app only.))
     [-proof string]          (Proof name (for SEQ app only.))
     [-comb]                  (Report all signal names in formal core
including combinational logics.)
     [-bit]                   (Report formal core in bit level.)
```

Example

- To get a summary of the formal core:

```
report_formal_core
```

```
vcf(compute_formal_core[00:00:44])> report_formal_core
Formal core summary view
#Properties   :    10
    #scheduled    :     6
    #completed    :     4
        #proven       :     4
        #bounded      :     0
        #no_status    :     0

    #Registers (union)        :     8
    #Inputs (union)           :     2
    #Constraints (union)      :     0

1
```

- To get a verbose report of the union of the formal core for all properties (without summary):

```
report_formal -union -verbose -no_summary
```

Feedback

```
vcf> report_formal_core -union  -verbose  -no_summary
Formal core union
#Registers (union) :     8
    first.green
    first.red
    first.state_out
    first.yellow
    main.green
    main.red
    main.state_out
    main.yellow
#Inputs (union) :     2
    waiting_first
    waiting_main
#Constraints (union) :     0

1
```

- To get a verbose report of the formal core for a specified property (without summary):

```
report_formal_core -property myproperty -verbose
```

```
vcf(compute_formal_core[00:01:30])> report_formal_core -property traffic.chk.assert_hazard_in_main -verbose  -no_summary
Formal core verbose view
Property      : traffic.chk.assert_hazard_in_main
SubType       :     -
Status        : proven
Depth         :     -
#Registers    :     5
    first.red
    first.state_out
    main.green
    main.state_out
    main.yellow
#Inputs       :     2
    waiting_first
    waiting_main
#Constraints  :     0
```

-no_summary

- To get the verbose report of the complement of the formal core:

```
report_formal_core -complement -verbose
```

Feedback

```
vcf> report_formal_core -complement  -verbose
Formal core summary view
#Properties   :    10
    #scheduled    :     0
    #completed    :    10
        #proven       :    10
        #bounded      :     0
        #no_status    :     0

    #Registers (complement)    :     3
    #Inputs (complement)       :     1
    #Constraints (complement)  :     2

Formal core complement
#Registers (complement) :     3
    dummy_reg
    first.timer
    main.timer
#Inputs (complement) :     1
    dumm
#Constraints (complement) :     2
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main

1
```

- To include combinational logic signals in the formal core report:

```
vcf> report_formal_core -comb -union
```

```
Formal core summary view
#Properties   :     3
    #scheduled    :     0
    #completed    :     3
        #proven       :     3
        #bounded      :     0
        #no_status    :     0

    #Registers (union)         :     1
    #Inputs (union)            :     2
    #Constraints (union)       :     0
    #Comb. Logics (union)      :     2
```

Using the snip_formal_core_complement Command

The `snip_formal_core_complement` command is used as a convergence debug aid. It is particularly useful when you want to create a new verification task where all registers outside the formal core of a specific property are snipped. This can help in determining if a property can be proven or if the bound for the property can be improved as a result of snipping. If a failure occurs after using this command, it may or may not be spurious due to the snipping of signals. The user can inspect the waveform to make this determination.

Arguments

`-property` `<propName> [required]`

Name of the property

`-par_task <parTaskName> [optional]`

Name of the parent task to use

`-new_task <newTaskName> [optional]`

Name of new task to be created

Examples

Can be executed after check_fv command.

```
  set_fml_appmode FPV
  set design traffic
  set_app_var fml_enable_idx true
  read_file -top $design -format sverilog -sva -vcs
 {-f ../design/filelist ../sva/traffic.sva ../sva/bind_traffic.sva}
  check_fv
  compute_formal_core -property {assert_green_no_waiting_first}
```

To take FPV as a parent task and create its own new task.

```
  compute_formal_core -property {assert_green_no_waiting_first}
  snip_formal_core_complement -property {assert_green_no_waiting_first}
```

To create the new task with desired name.

```
  check_fv
  compute_formal_core -property {assert_green_no_waiting_first}
  snip_formal_core_complement -property {assert_green_no_waiting_first}
 -par_task FPV -new_task snipped_fc_complement
```

20.6.4 Getting Formal Core in Tcl

Use the `get_formal_core` command to get the collection of registers, constraints and inputs that is present in the formal core.

Use Model

```
vcf> get_formal_core -help
Usage: get_formal_core    # Returns a collection of nets in the
 formal-core.
        [-property propName]    (Name of the property)
        [-subtype subtypeName] (Subtype for the property (vacuity,
 witness, -))
        [-complement]           (Return complement collection.)
        [-registers]            (Return registers in formal core.)
        [-inputs]               (Return inputs in formal core.)
        [-constraints]          (Return constraints in formal core.)
```

Note:

One of the option -registers, -inputs, -constraints is mandatory.

If `-complement` option is provided, a collection of registers, inputs, constraints that are not present in the formal core is reported.

Examples

- To get the collection of registers in the formal core:

```
get_formal_core -registers
```

- To get a collection of inputs in the formal core:

```
get_formal_core -inputs
```

- To get a property collection of constraints:

```
get_formal_core -constraints
```

- To get a collection of inputs in complement:

```
get_formal_core -complement -inputs
```

- To get a collection of registers in complement:

```
get_formal_core -complement -registers
```

- To get a collection of constraints in complement:

```
get_formal_core -complement -constraints
```

20.6.5 Generating Formal Core On-the-fly

To improve the performance of formal core generation, in terms of run time and the number of formal cores is generated, you can turn on on-the-fly formal core generation when executing the `check_fv` command.

Syntax

```
vcf> check_config [-formal_core <status>]
```

Where,

- `[-formal_core <status>]`: Values can be *off, proven, inconclusive,* or *both*. If it is set to *proven* (*inconclusive*) before executing the `check_fv` command, then orchestration attempts to compute formal core for a property that has *proven* (*bounded*) status. If it is set to *both*, then formal core will be computed for both the statuses. If it is set to *off*, then no formal core is generated during the `check_fv` command.

Note that,

- There is some overhead in the usual orchestration due to this option. To manage overhead, the formal core computation is stopped for a property if it does not finish within the reasonable threshold.
- Once the `check_fv` command finishes, it is possible that only for a subset of properties, the formal core is computed.You must call the `compute_formal_core` command later to compute formal core for the remaining properties.
- Depending on which engine solved the property first, the formal core results may differ.
- In the `check_fv` command, some rewrites/optimizations are used to prove properties. Some registers may get merged or some registers may disappear in the process of proving a property after constant propagation.

  Because of rewrites/optimizations, the disappeared registers do not get listed in the formal core report. If you can snip out any of the register that is not listed in the formal core, which is generated from `compute_formal_core` command, and the properties can still be proven. If you snip out any of the register that is not in the formal core generated using the `check_fv` command, then the properties may fail to be proven.

  Note:

  This option only works in FPV application mode.

20.6.6 Formal Core Known Issues and Limitations

- Registers that appear in `set_constant` or environment assumptions (`fvassume -env`) may not be reported in the formal core even if they are required. There is a performance and debug ability trade-off done here. The `set_constant` and environment assumes can sometimes offer performance benefit at the cost of making debug difficult. If you want to debug, then you should remove the `-env` switch from `fvassume`. Similarly, replace `set_contant` with the `fvassume` (without `-env`).
- The results from report commands can over-approximate the set of registers, constraints, inputs that are needed to prove a property. There is no guarantee to report minimal set of registers. VC Formal runs multiple engines (and engine options) in parallel when computing formal core. The formal core is reported for the engine that finishes first. The same formal core might not be reported every time. This is important while performing regressions, and you must only check those registers, constraints that are required for (bounded) proof are part of formal core.
- Formal core computation is a compute-intensive step and can potentially be more complex than the actual proof thus taking longer. One factor affecting formal core

Feedback

computation time is the engines involved in the analysis. It is recommended to set a max time limit with `set_fml_var fml_max_time <time>` for the formal core computation to avoid unexpected run times. Formal core computation results are not guaranteed to be minimal. There are several factors that affect the results such as engines used for proof, performance trade off, and so on.

- By default, certain VC Formal core engines identify a reduced set of constraints for bounded proof up to some depth, whose computation can be expensive. To disable reduced constraints computation during formal core generation, set the following formal variable to false:

  ```
  set_fml_var fml_formal_core_reduced_constraints false
  ```

- When reporting the complement, VC Formal does not report the signals in the constraints that are implicitly part of formal core of each property such as -env constraints or set_constants. Similarly, clock, reset signals are not reported in the complement as these are always implicitly considered to be part of the formal core. Similarly, VC Formal does not report when reporting complement of constraints, the -env constraints as they are implicitly part of the formal core. Note that -env switch can be applied to any constraint, however, VC Formal may not be able to apply the -env constraint at the compile time. In such cases a warning message is displayed and the constraint is passed to the backend (solvers). Such constraints are treated as if no -env switch is given to them when it comes to formal core reporting.
- Latches are not reported as part of the complement of formal core. This is because even if the latch is not in the `report_formal_core` output it may still be used in the proof. This is related to how latches get modeled.
- The formal core is reported at the granularity of registers. If the property is purely combinational, then formal core may be empty.
- On-the-fly formal core computation during the analysis step is stopped when the analysis completes, thus the results might be incomplete; use command `compute_formal_core` to resume computation and get complete formal core results.
- On-the-fly formal core computation is limited to the set of engines that provide suitable information.

20.6.7 Computing and Reporting Formal Core in GUI

You can compute the formal core from the GoalList for:

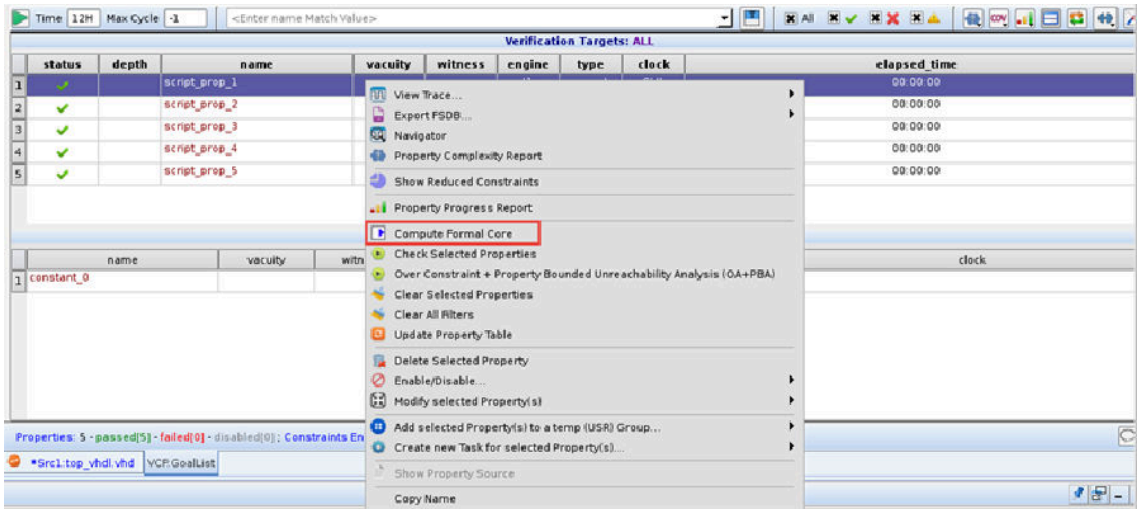
- One or multiple properties
- All the properties
- All the properties in an instance

Feedback

By default when we run formal core on the entire task by default we only pick proven assertions (inconclusive asserts will also be selected based on -depth option).

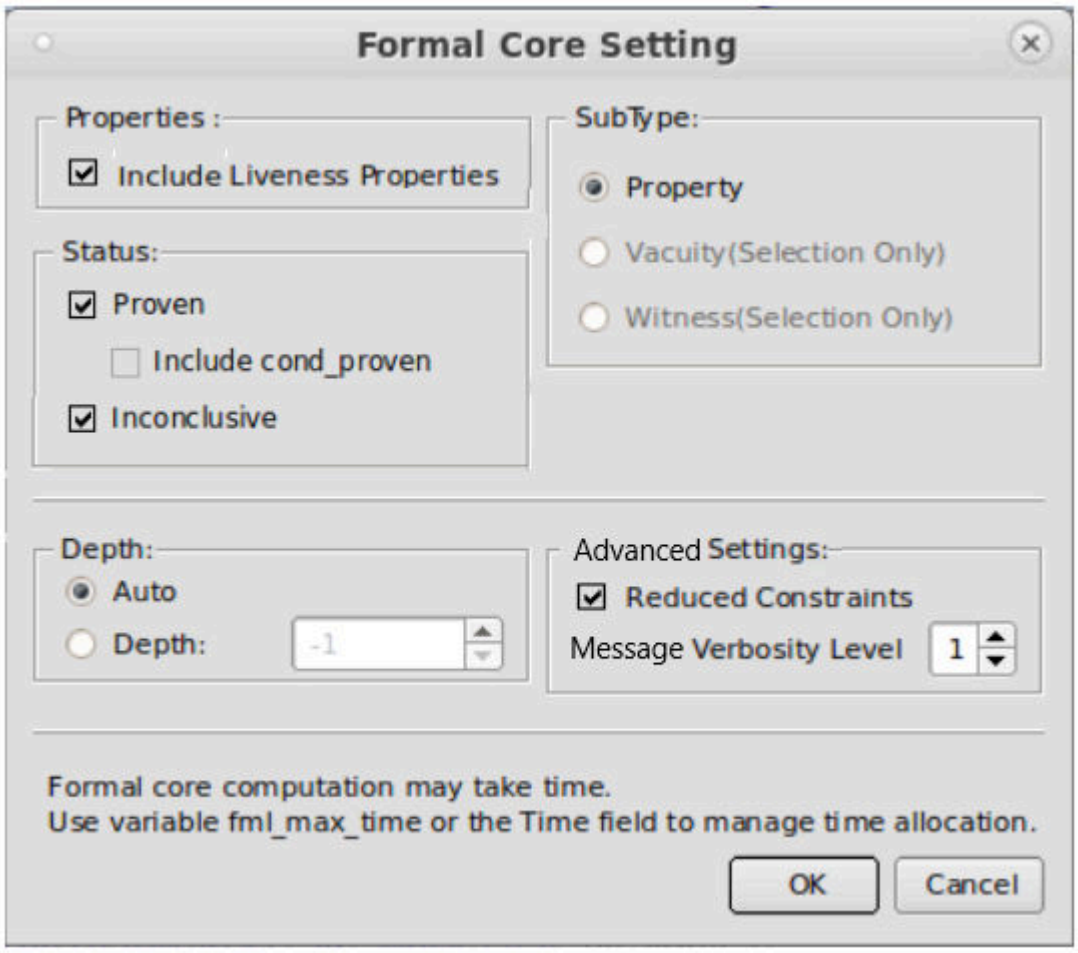
Computing Formal Core for One or More Properties

- Select a property, multiple properties, or all the properties from the GoalList, and from the RMB menu, select *Compute Formal Core*.



the *Formal Core Setting* dialog box appears.

Feedback



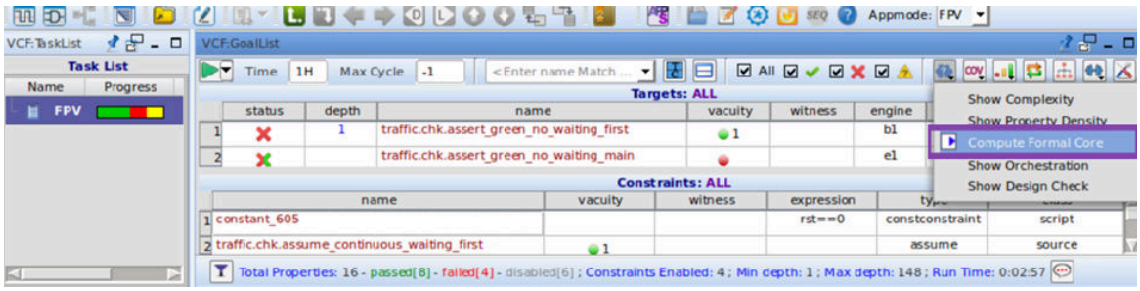
- If the property is proven, uncoverable, or vacuous, the *Auto* option is selected by default. You can specify the depth up to which the formal core must be computed by typing a value in the *Depth* box.
- If the property is inconclusive, then the *Auto* option is grayed out and you must specify the depth in the *Depth* box with the required depth. The maximum value of the depth is limited to depth of that property.
- By default, *Auto* is selected in the *Depth box.*You can select *Proven* and *Include cond_proven* or the *Inconclusive* check box in the *Status*box.
- If the vacuity/witness of the property is vacuous/uncoverable, then the *Vacuity* (Selection Only)/*Witness* (Selection Only) can be selected to compute the formal core for the vacuity/witness.

  After setting the depth, the formal core report appears as shown in Figure 177.

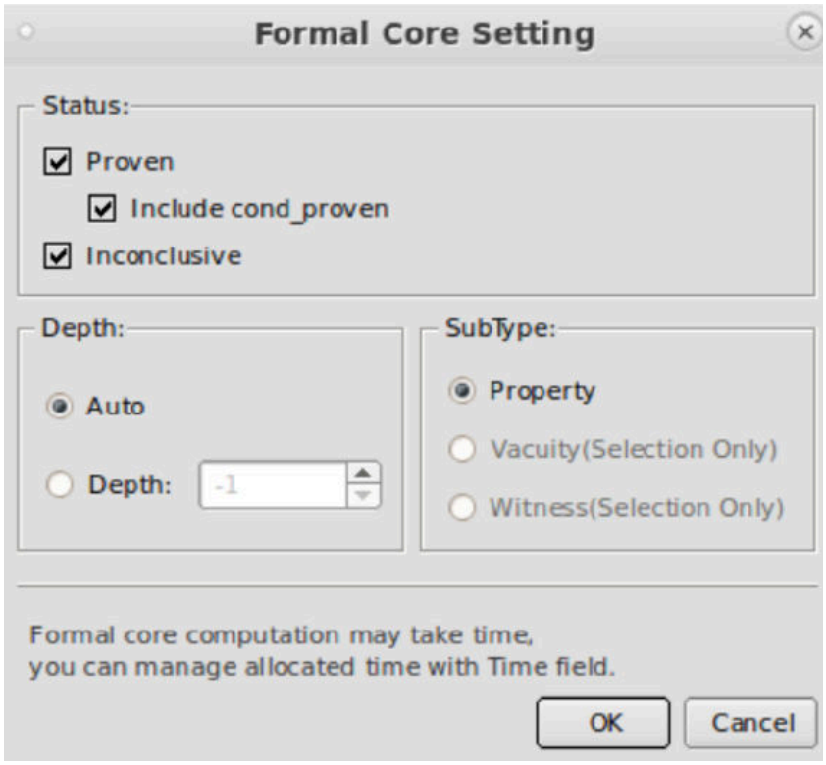
Feedback

Computing Formal Core for all Properties

- From the GoalList toolbar, click and select *Compute Formal Core*.



the *Formal Core Setting* dialog box appears.



  - If all the properties are either proven, uncoverable, or vacuous, then the *Auto* option and the *Proven* status can be selected. By default, the *Auto* option and the *Proven* status are selected, and the *Inconclusive* status is grayed out.
  - You can also specify the depth up to which the formal core must be computed by typing a value in the *Depth* box.

Feedback

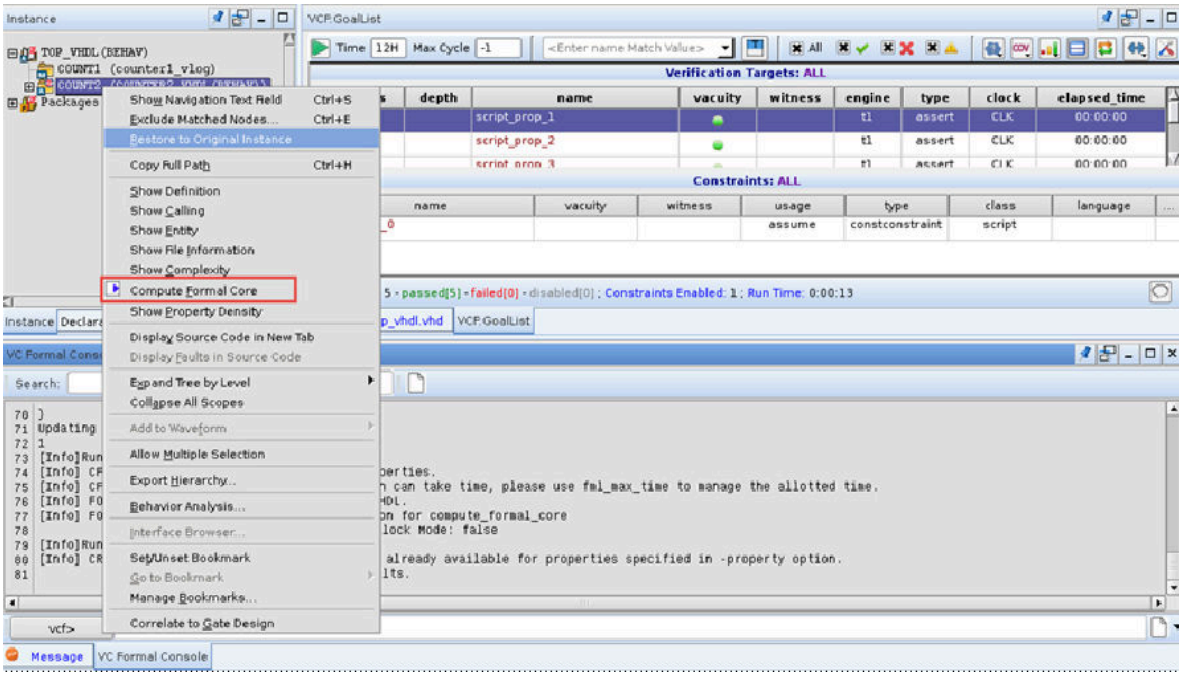
- The *Inconclusive* status can be selected when the depth is specified. The maximum value of the depth is limited to the minimal depth of the inconclusive properties.
- By default, *Auto* is selected in the *Depth* box. You can select *Proven* and *Include cond_proven* or the *Inconclusive* check box in the *Status* box.

Computing Formal Core For all Properties of an Instance

From the instance tree, select an instance and from the RMB menu, click *Compute Formal Core*.

Alternatively, you can click to sync the properties of an instance, then select *Instance* from instance pane. Click *Compute Formal Core* from the drop-down list.

Figure 176: Computing Formal Core for Instances



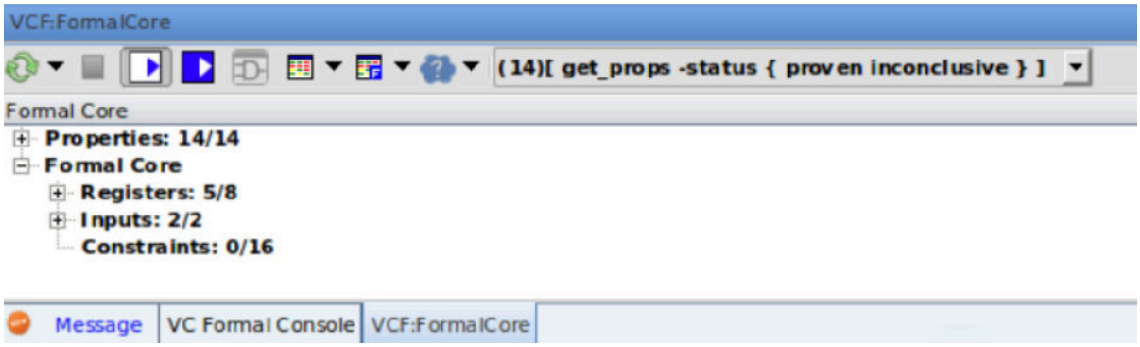
20.6.8 Formal Core Report

The Formal Core report reports a summary of the following as shown in Figure 177:

- The number of properties the `compute_formal_core` command is working/worked on.
- A set of registers in the design used by the formal engines to prove the property.
- A set of constraints used to prove the property.

- A set of inputs involved to prove the property.

Figure 177: Formal Core Report



You can also switch between formal core report of a particular property and all properties using the

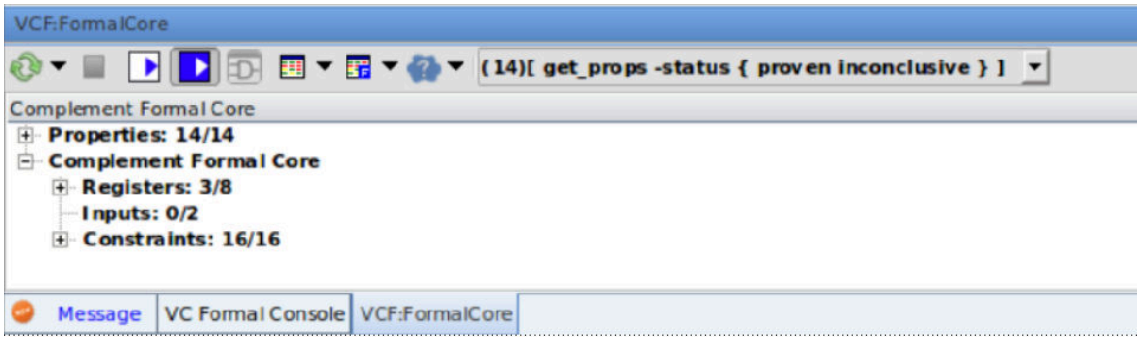
drop-down list.

- Select * from the drop-down list to view the formal core of all properties.
- Select the property name from the drop-down list to view the formal core of a particular property.

Viewing Formal Core and Complement of the Formal Core

- To view the complement of the formal core, click .

Figure 178: Complement of the Formal Core



- To view the formal core, click .

Refreshing the Formal Core Report

The formal core report is updated after computing the formal core of the first property. The formal core report is not updated automatically if there are changes while computing the formal core for the subsequent properties. The refresh button changes to orange

Feedback

whenever there is a change in the formal core computed for the subsequent properties indicating that the formal core report is not the latest.

- To view the updated report of the formal core, click .

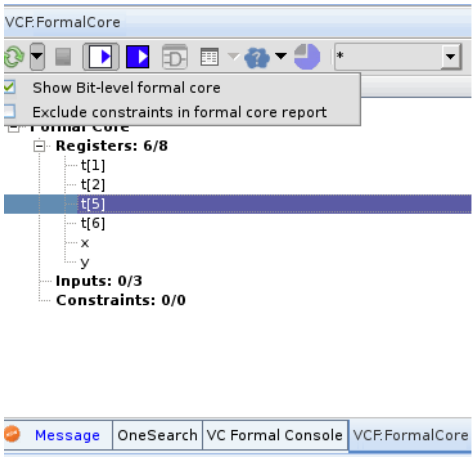
Viewing Bit-level/Exclude Constraints of Formal Core Reports

You can view formal core reports for the followings:

- *Bit-level*: Shows the formal core report in bit-level.
- *Exclude Constraints*: Shows the formal core report of exclude constraints.
- *Both*: Shows the formal core report of bit-level and exclude constraints.

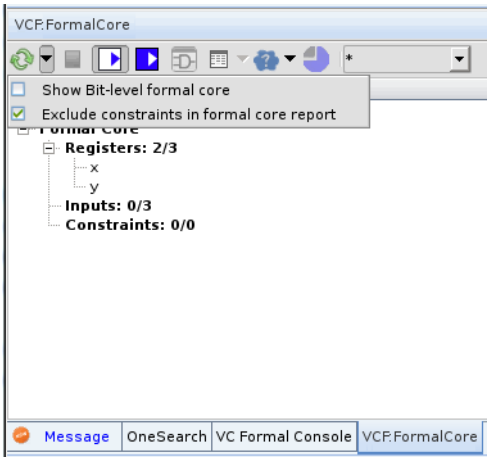
To generate bit-level/exclude constraints of formal core report:

1. Compute the formal core for a property, an instance, or all properties for which you want to generate bit-level formal core report.
2. Click the arrow and select the *Show Bit-level formal core* from the list. The bit-level of formal core report appears.

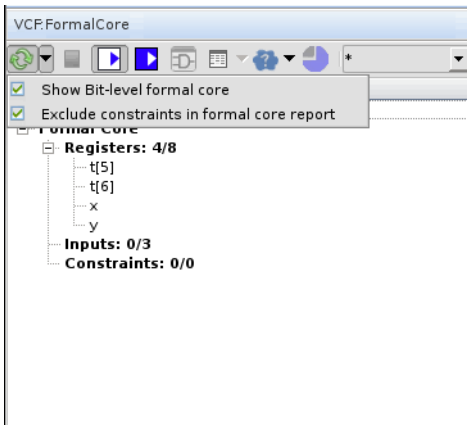


3. Click the arrow and select the *Exclude constraints in formal core report* from the list. The formal core report appears excluding the constraints.

Feedback



4. Click the arrow and select the *Show Bit-level formal core* and *Exclude constraints in formal core report* from the list. The formal core report appears including both bit-level and exclude constraints.

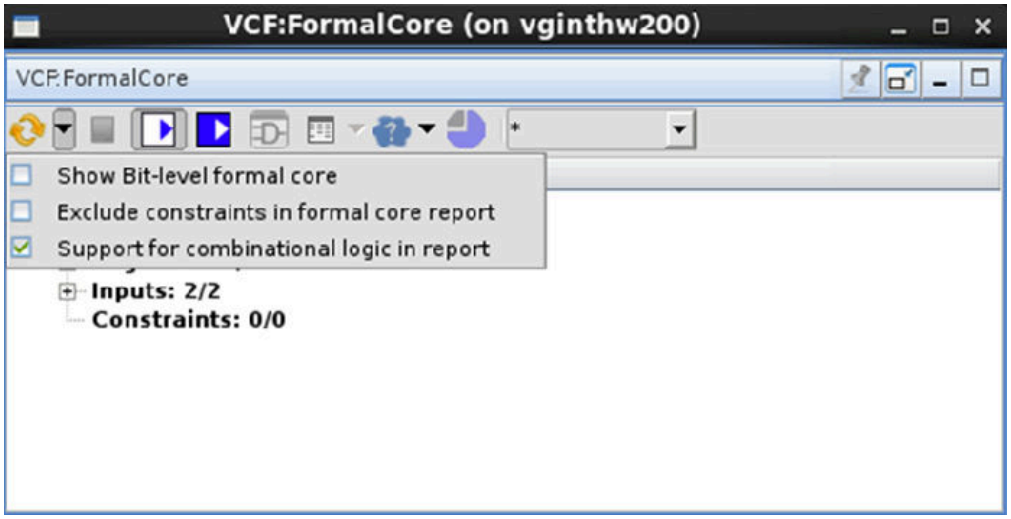


Feedback

Viewing Formal Core Report

To view the formal core report with combinational signals:

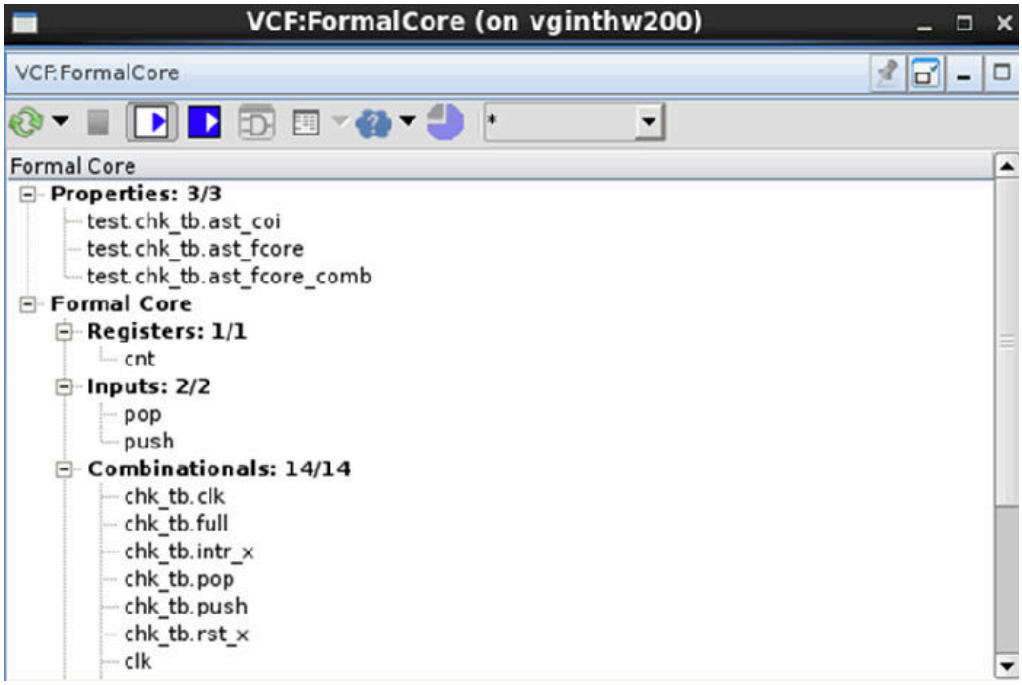
1. Select *Support for combinational logic in report*.



2. Click the button.

   The formal core report displays the combinational logics as shown in Figure 179.

Figure 179: Example for Generating Combinational Logic Report Using GUI



Feedback

Stopping the computation of Formal Core

The formal core report is updated after the computation of the formal core of the first property, and there after the formal core results are updated incrementally. You can stop the formal core computation of the subsequent properties, and view the current results.

- To stop the computation of the formal core of the subsequent properties and to view the current

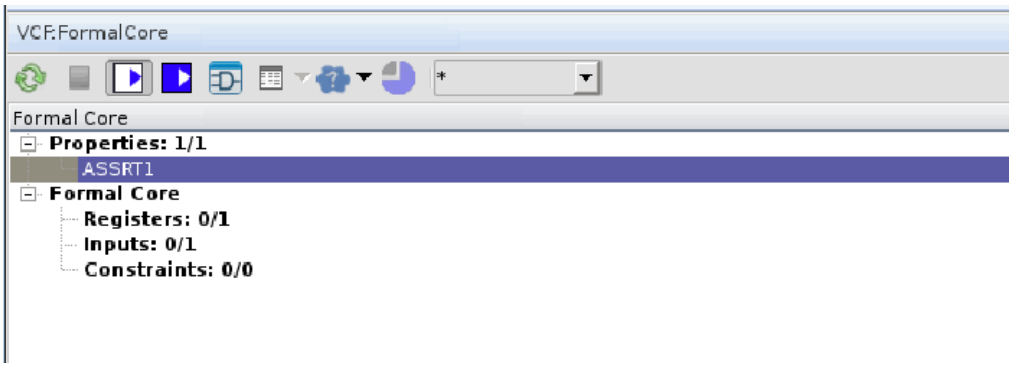
  results, click .

Viewing Formal Core Schematic

You can view the formal core schematic in the formal core report. The Formal core schematic generates accurate schematic for the property based on the computation. You can use it to debug inconclusive properties.

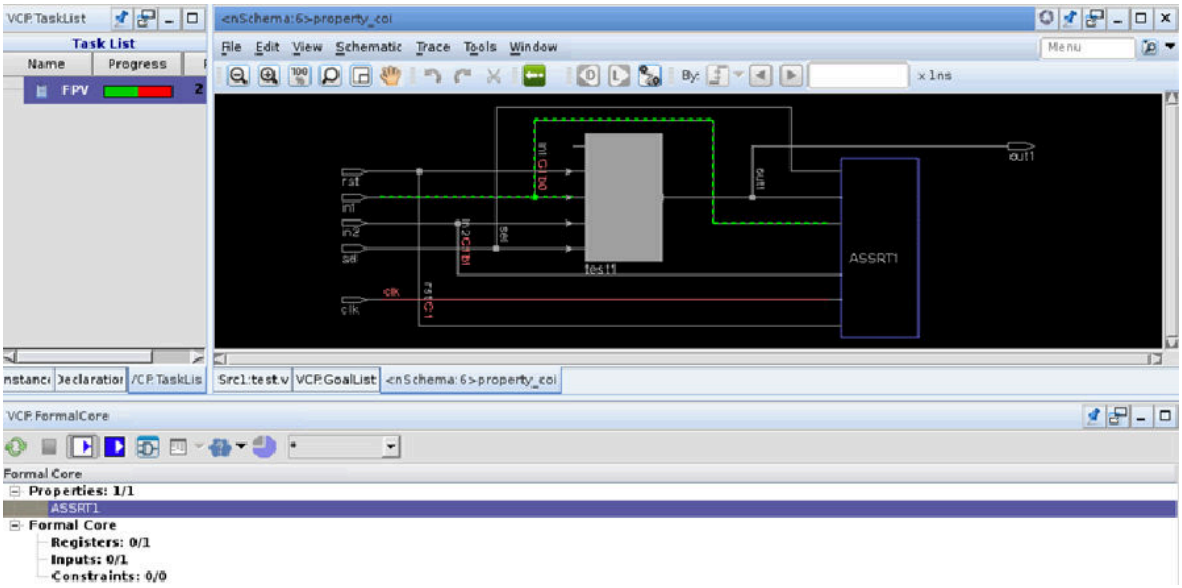
To view the formal core schematic:

- Select one proven property in the *Properties* list and click .



The formal core schematic view appears.

Feedback



Note:

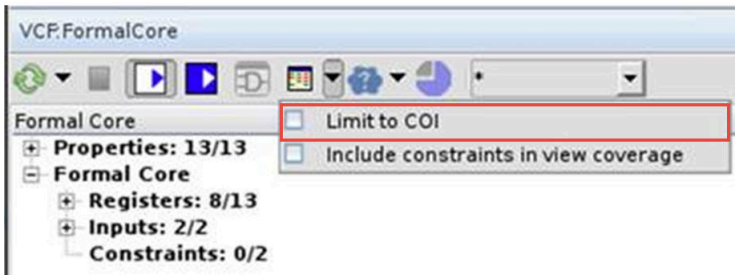
This feature applies to FPV properties only in the current phase.

Viewing Formal Core Coverage Reports

Results from formal core can be mapped onto coverage metrics that are viewable in the Verdi Coverage GUI. In order to use the coverage mapping tool it is a requirement that -cov with the relevant metrics is supplied with the read_file/elaborate command. The coverage results generated can be saved in a database which may then be used as any other coverage database. It is important to understand the difference between coverage metrics generated through simulation and those through formal, however formal core can be represented in this way.

To generate formal core coverage report,

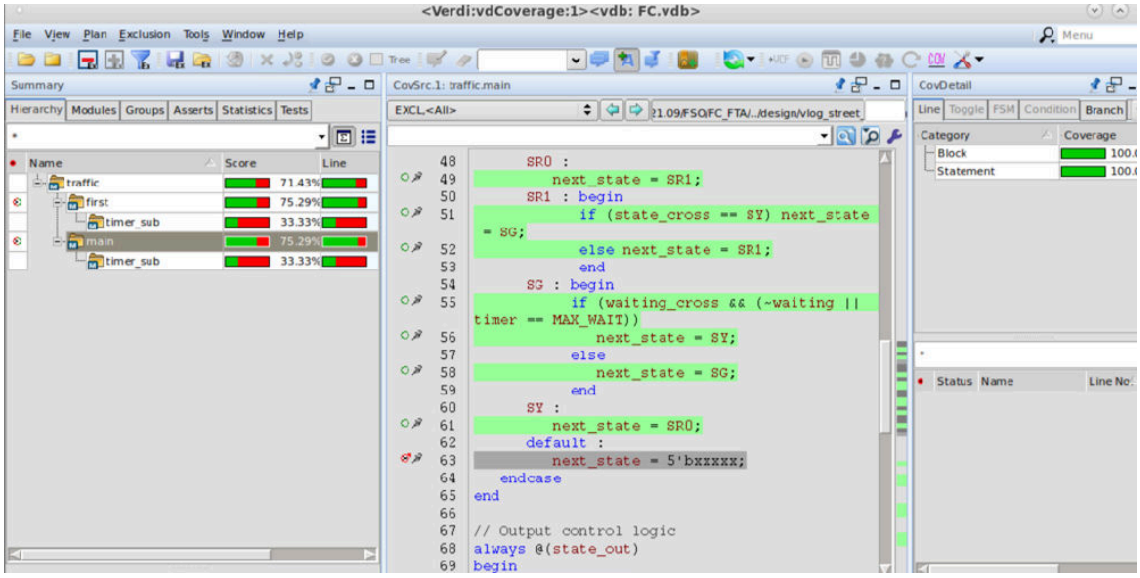
1. Click down arrow on the formal core toolbar and select the *Limit to COI*and *Include constraints in view coverage*check boxes.



2. Click the (*Coverage*) icon.

Feedback

The formal core coverage report is displayed.



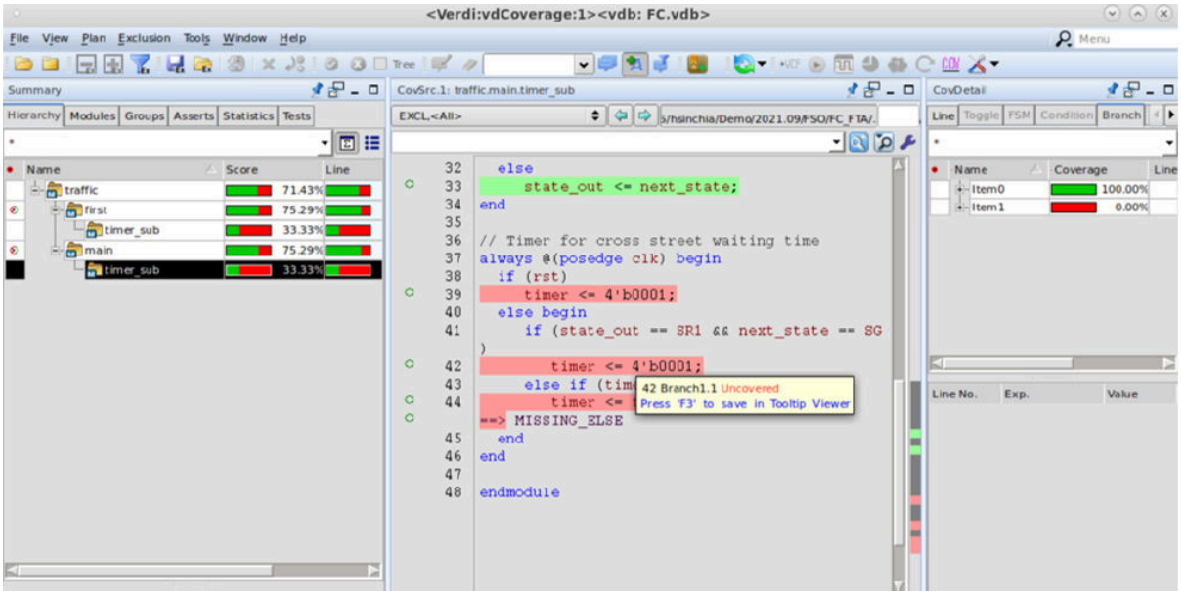
The line, conditional, toggle, cover, cover groups, and branch coverage are supported in formal core coverage. The extracted line, conditional, toggle, cover, cover groups, and branch coverage can be either directly marked as covered in the coverage database. Or you can run formal analysis and then save the results in the coverage database.

Note:

The cover groups coverage information is under the *Groups* tab of *Summary* section.

Feedback

You can mark specific statement in the branch, which is in the formal core as covered in the coverage database, as shown in the following figure:



20.6.9 Viewing Formal Core Coverage Report in Signoff Mode

The formal core coverage in the *Signoff Mode* shows the coverage report for the entire design. The coverage report shows the source code in different colors as described:

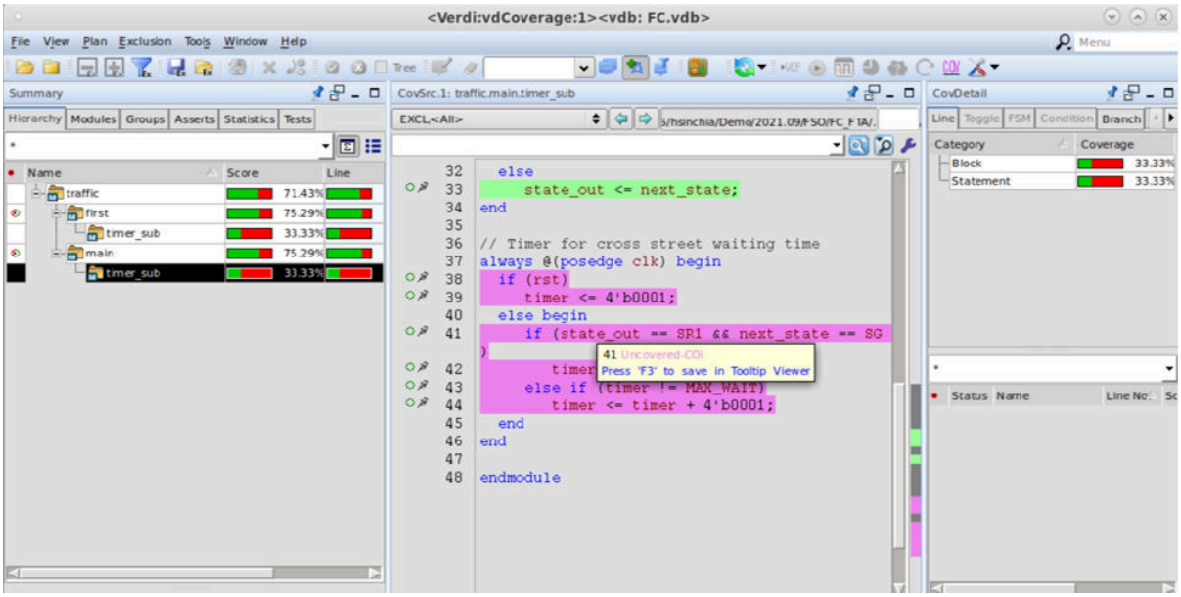
- *Inside formal core*: Marked as covered in green
- *Outside formal core, but part of COI*: Marked as Uncovered-COI in red
- *Outside COI*: Marked as Uncovered in red

You can find the following status after formal core results are mapped to Verdi Coverage.

Meaning	Color coding	Legend			
Inside FC	Covered	O	C	F	T
Outside COI	Uncovered	O	C	F	T
Outside FC inside COI	Uncovered-COI	O	C	F	T
Inconclusive	Inconclusive	O	C	F	T
Excluded by FPV_COV_FCORE_w_fcore.el	Unreachable	O	C	F	T

Feedback

Figure 180: Viewing Formal Core Coverage Report in Signoff Mode



20.6.10 Supported Coverage Metrics and Formal Core Coverage

VC Formal supports line, conditional, toggle, cover, cover groups, and branch coverage in formal core coverage. The extracted line, conditional, toggle, cover, cover groups, and branch coverage can be either directly marked as covered in the coverage database. Or you can run formal analysis and then save the results in the coverage database.

VC Formal maps formal core to line and branch coverage goals inside functions, if the corresponding function calls is in the driving logic of formal core signals. To enable this feature, set the following formal variable to true:

```
set_fml_var fml_enable_fcore_cov_function true
```

Use Model

There are optional variables which can be set to configure the coverage mapping (1-3 are set in signoff_config):

1. Set the following fml var to keep source location of constant selects, and map them appropriately:

   ```
   set_fml_var fml_track_loc_const_select true
   ```

2. Set the following fml var to keep source location of async reset blocks, and map reset block for formal core coverage mapping:

   ```
   set_fml_var fml_track_loc_reset true
   ```

Feedback

3. Set the following application variable to true to enable toggle coverage of input ports so that it can be mapped for formal core coverage:

```
set_app_var fml_cov_tgl_input_port true
```

4. To map the formal core results to coverage using tcl commands, add the following commands after computing the formal core

   a. Compute formal core coverage results

```
compute_formal_core_coverage -par_task <parent task name>
```

Syntax

```
vcf> compute_formal_core_coverage  -help
Usage: compute_formal_core_coverage    # Extract line, toggle,
 condition, SV-covergroup coverage for the formal core of the
 property.
       -par_task parTaskName  (Name of the parent task on which to
 run formal core coverage analysis)
       [-property property]   (Property for which you'd like to
 compute_formal_core_coverage for.)
       [-block]               (Makes command input block while this
 command is active)
       [-run_finish finishCmds] (Set of commands to run when this
 command finishes)
       [-structural]          (Marks the
 toggle/line/condition/SV-covergroup coverage goals in the formal
 core directly as covered in coverage database without doing any
 formal analysis.)
       [-add_constraints]     (Include constraints involved in
 formal core)
```

   b. Save formal core results in coverage database

```
save_formal_core_results
```

Syntax

```
vcf> save_formal_core_results -help
Usage: save_formal_core_results    # Saves database for formal core
 results.
       [-db_name dbName]      (Name of the coverage database)
       [-elfile_name elFileName]
                              (Name of the exclusion file)
       [-show]                (Open Verdi coverage on the generated
 database)
       [-mark val]            (Mark formal core or mark formal core
 plus coi (default fcore_only): Values: fcore_only, fcore_plus_coi)
       [-add_constraints]     (Include constraints involved in
 formal core)
```

Feedback

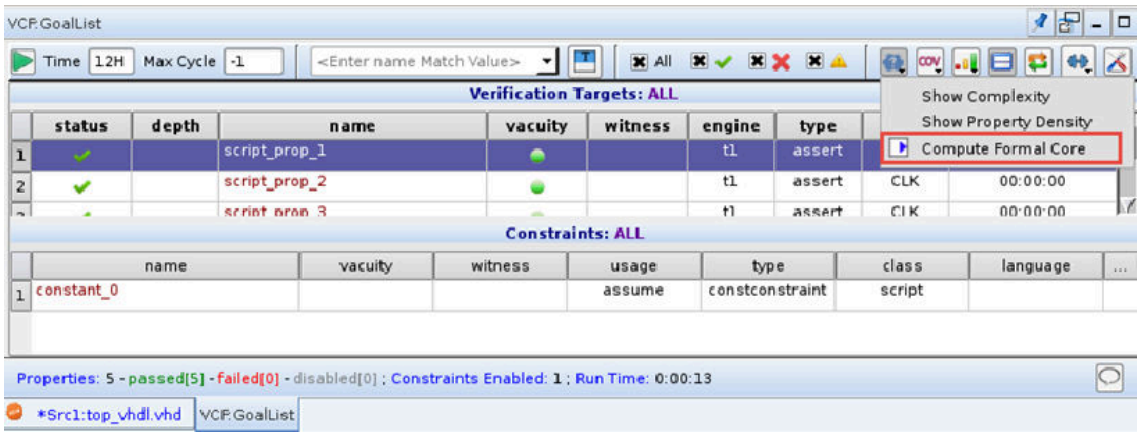
```
        [-test test]            (If this option is specified, then
testdata will be created with specified test name in atthe coverage
database.)
```

If the `save_formal_core_results` command is called directly without calling `compute_formal_core_coverage`, it generates the line coverage results and saves the result as covered directly. To generate the toggle and condition coverage goals, the `compute_formal_core_coverage -structural` command is used to save any condition/toggle goals if any, and update the line coverage database with any of the new metrics.

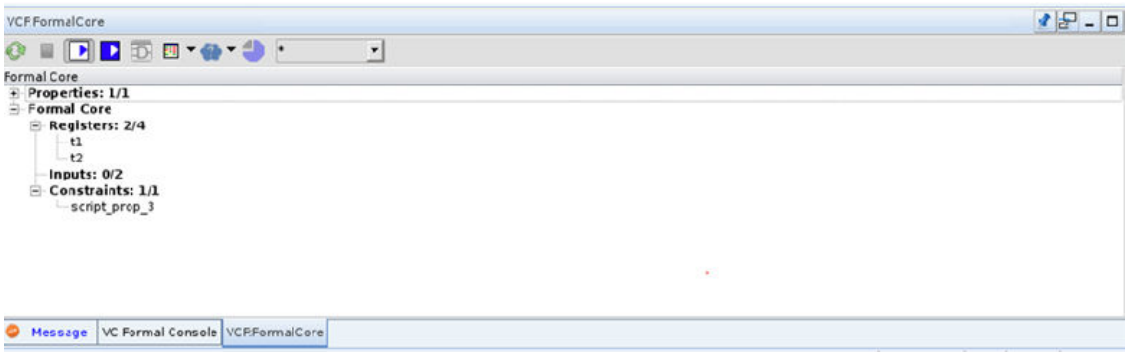
Generating Formal Core Coverage in VC Formal GUI

To generate formal core on the FPV toolbar in the GUI.

1. On the *GoalList* toolbar, click and select *Compute Formal Core*.



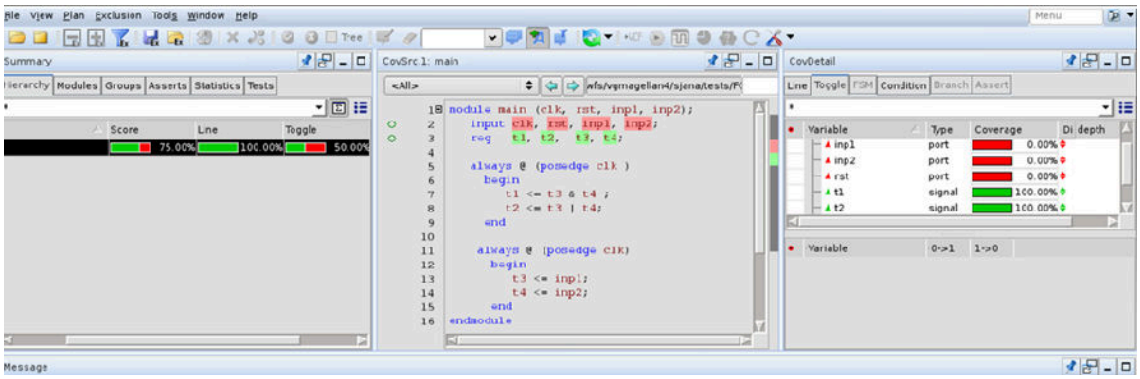
The *VCF:FormalCore* is displayed, as shown in the following figure.



2. Click the arrow on the Formal Core toolbar, and select the *Formal Analysis on the Coverage Goals in Formal Core* check box.

Feedback

The formal analysis is done on the coverage goals in the formal core and then result is saved in the coverage database. Or the coverage targets in formal core is directly saved in the coverage database.



20.7 Formal Testbench Analyzer (FTA)

The final step in the signoff flow is to perform a detailed analysis of the quality of the assertions in the environment. The FTA app has been developed to perform fault injection analysis on a design to check whether or not the formal environment can catch injected issues. If a property fails in the presence of an artificial fault then this is a good sign of testbench quality. If all assertions prove in the presence of a fault, then this is a sign of a potential verification hole.

The FTA App can be used after you think that the formal testbench has a comprehensive set of properties that are proven or inconclusive on the original RTL in the FPV mode. You can use the same constraints and target properties in the FTA App on a fault injected RTL model to qualify whether an injected fault can be detected.

When the FTA App is invoked, first, the FTA determines whether a fault belongs to the cone of influence logic (COI) of any property to be considered for FTA analysis. When a fault is in the COI of one of the properties specified for FTA analysis, it is qualified as activated. When a fault is not in the COI of any of the properties specified for FTA, it is qualified as non-activated. When a fault is non-activated, it means that the list of proven (and inconclusive, if specified) assertions in the testbench does not cover the part of the logic in the design where the non-activated faults are located. These non-activated faults should be reviewed to determine whether more assertions should be written to verify this part of the design, or they can be waived because they are covered in the other steps in the verification plan.

The FTA analysis then determines whether an activated fault can be caught by any of the target properties changing the status from either proven or inconclusive to falsified. If any property is falsified with this fault, the fault is qualified as detected. If none of the activated properties is falsified, this property becomes non-detected. A fault can also

be inconclusive. The focus should be on the non-detected faults found in the formal testbench, since it implies weakness in the verification environment. The causes can be due to:

- Missing or incorrect property
- Over-constraint environment

For more details on how to use the FTA Application mode, see section Formal Testbench Analyzer App.

20.8 Handling Waivers in Coverage Flow

All verification tasks go through the development, validation process, and eventually reach sign off stage. The progress measurement and signing off process is often measured by the coverage metrics. In addition to marking what has been done and covered in the verification scope, waiving mechanism is frequently required during the review and signoff process. So that at each stage, a waiver can be created, and applied before the next step.

There are different stages in the signoff review process, when you use coverage to measure the process leading to signoff, waivers can be incrementally added. The creating of the waivers sometimes can be automatic, sometimes can be a manual process. To make it more seamless and easier to use, VC Formal has defined a unique naming conventions for waivers generated at different stages so that they can be applied in the reviewing process or removed from the next run.

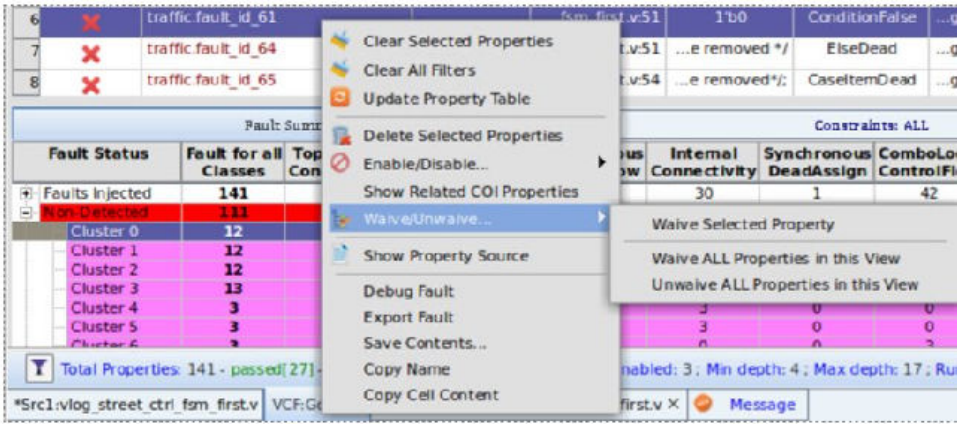
20.8.1 Waiving Versus Excluding

You can waive or exclude the objects to be removed, that are not of interest from coverage. Waiving is an action applied in VC Formal to disable the properties from computing FSO flow. You can

- Waive fault properties in FTA using `fvwaive`/`fvunwaive` commands
- Waive cover properties in OC using `fvwaive`/`fvunwaive` commands

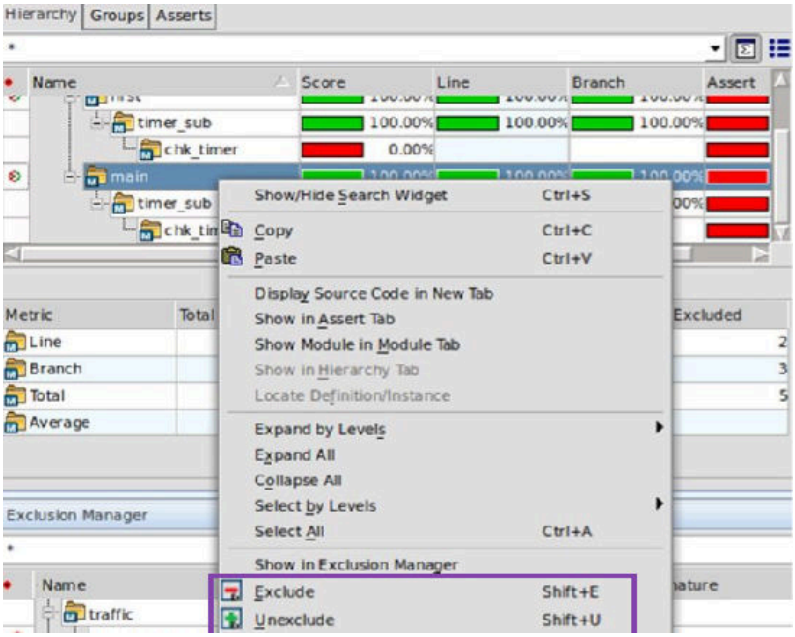
Waived properties can be saved to the *elfile* file using the `save_waiver_file` command. You can also waive properties from the GUI as shown in the following figure:

Feedback



Excluding is an action applied in Verdi Coverage to remove the coverage items from calculating coverage scores. Excluded coverage items can be saved to exclusion files (elfiles). For FSO flow, *elfiles* can be applied to:

- COI, OC flow using `read_waiver_file -elfiles`
- FTA flow using `read_fta_waiver -elfiles`
- Verdi coverage from VCF using `view_coverage -elfiles`



Feedback

You can also exclude properties from the GUI as shown in the following figure:

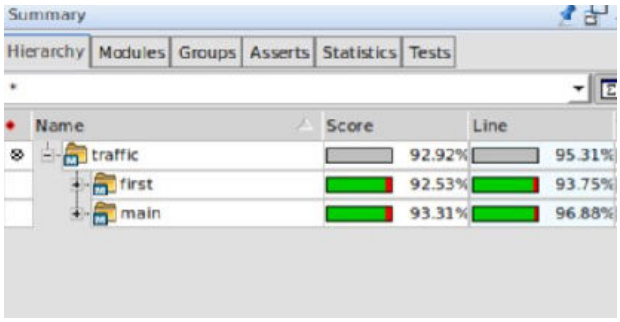
20.8.2 Waiving Out-of-Scope Instances

- To waive out-of-scope instances, declare the target instances in compile stage for coverage. Use `-cm_hier` in VCS option during `elaborate` or `read_file`

```
elaborate $top -sva -vcs " $parameters -cm_hier hier.cfg …"
```

The *hier.cfg* file

```
-tree traffic
+tree traffic.first
+tree traffic.main
```



Note:

`+tree instance_name [level_number]` VCS compile is only for the specified instance and the instances under it for coverage.

`-tree instance_name [level_number]` excludes this instance from coverage and other instances under it.

- To exclude specific scopes to an elfile, use the following command

```
save_cov_exclusion -scope traffic.first -file first.el
```

20.8.3 Applying the Elfiles in Signoff Coverage

- Loading Existing Exclusion File

  To apply elfiles to the subsequent FSO flows and Verdi Coverage, use the following command:

```
read_waiver_file -elfiles { first.el … }
```

When you load an existing *elfile* with `read_waiver_file`, it can waive the cover properties related to the excluded items and improve performance as well.

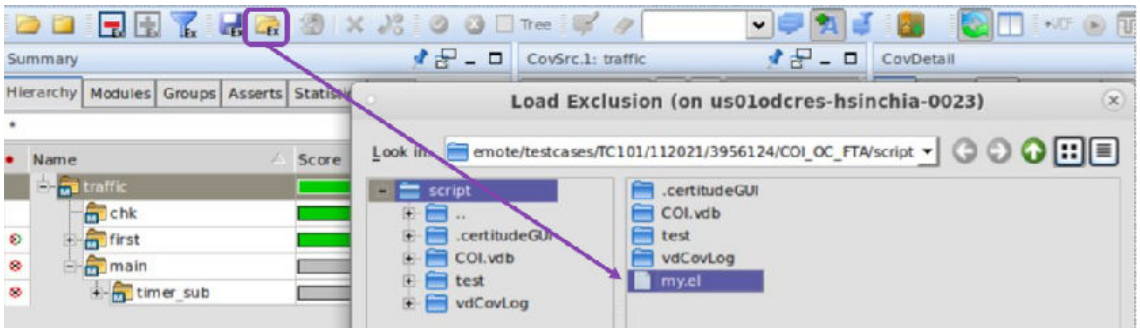
Feedback

To remove elfiles to the subsequent FSO flows and Verdi Coverage, use the following command:

```
remove_waiver_file -elfiles { first.el … }
```

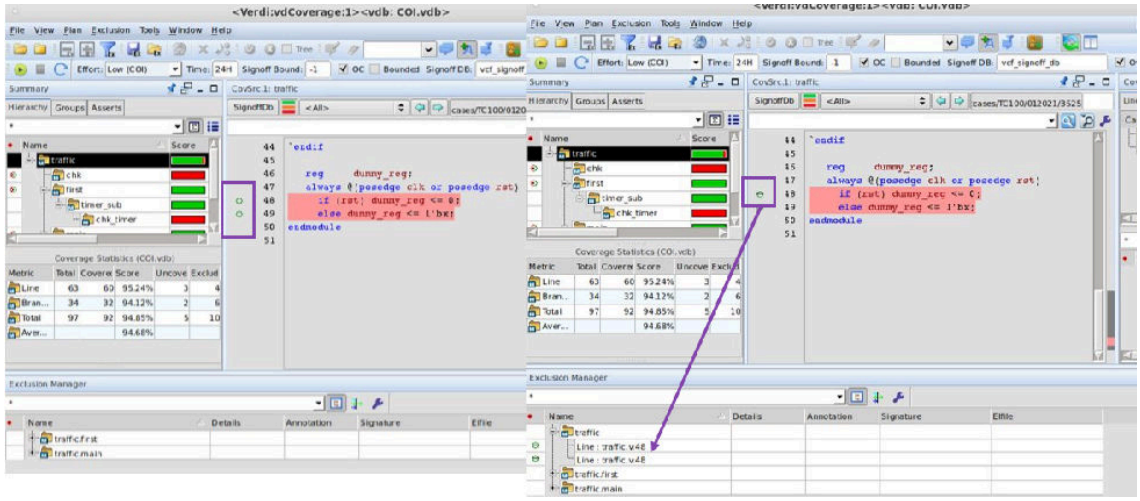
When you remove an existing elfile with remove_waiver_file, it will unwaive the cover properties related to the excluded items.

The user-defined elfile can be loaded by Verdi Coverage as shown in the following figure:



- Excluding a Line

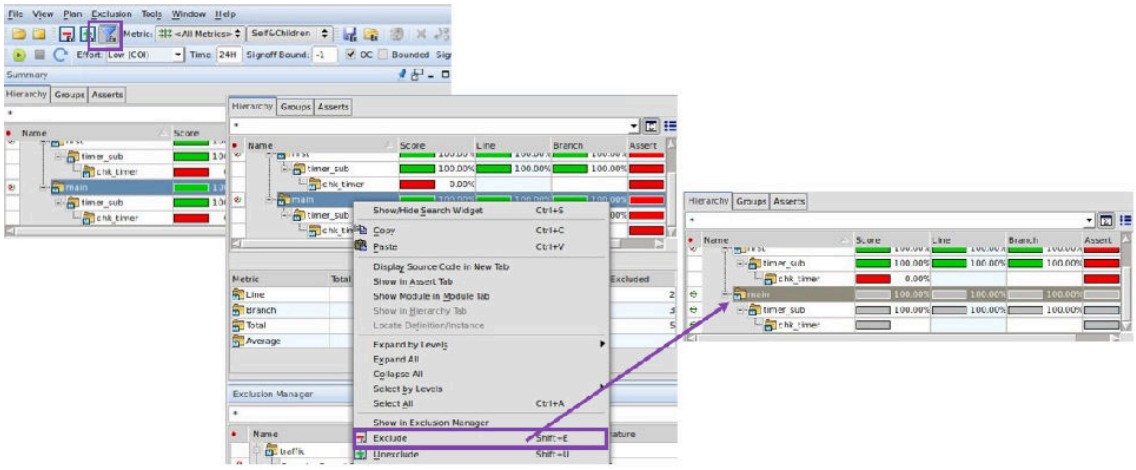
  You can exclude uncovered (red) items after review. The excluded lines appear in *Exclusion Manager* as shown in the following figure:



- Excluding an instance

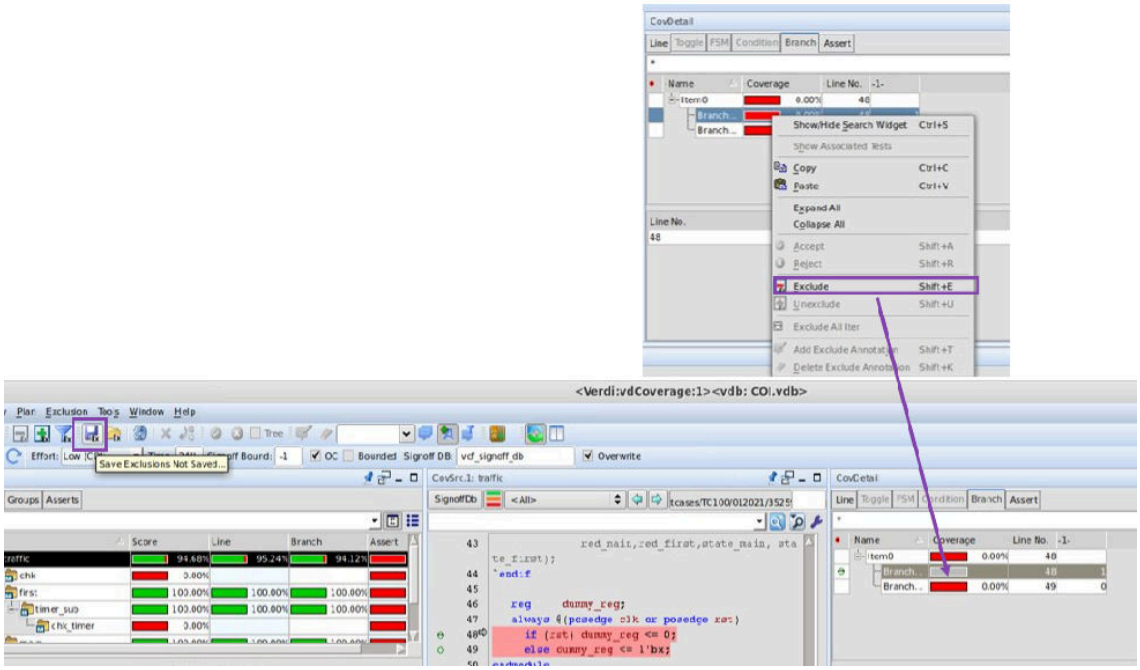
  To exclude all hierarchies under certain instance, select *Self&Children* under *Metric* before selecting *Exclude*.

Feedback



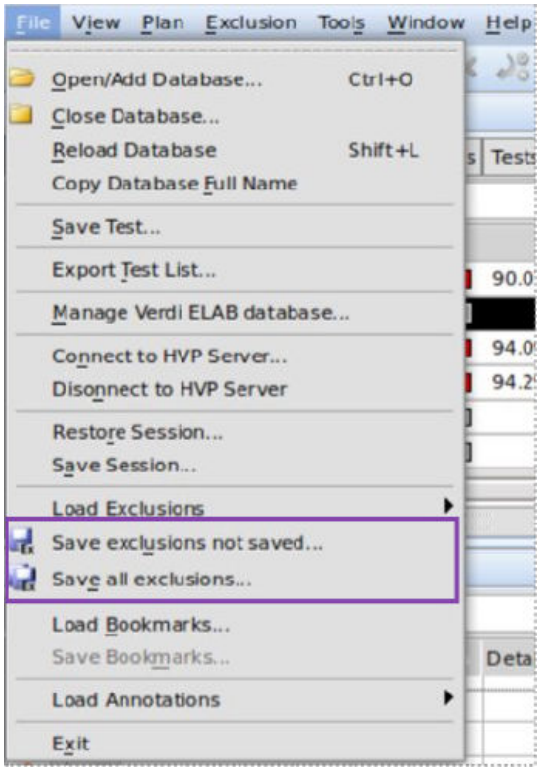
- Excluding an Item and Save Exclusion Files

  To exclude an item, select the item and select *Exclude.*The excluded items appear in Gray.



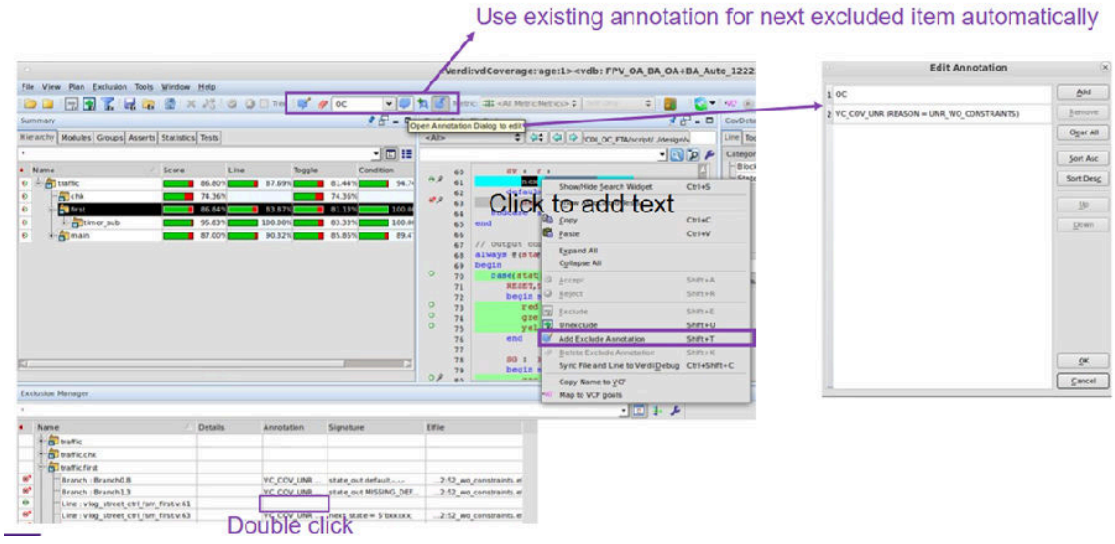
  By default, the exclusions items, which are not saved in existing elfiles, are saved. You can merge all exclusion items to another elfile using *Save all exclusions*option.

Feedback



- Add Annotation to Elfiles

  You can add annotation for the saved Elfiles as shown in the following figure:



- Automatic Loading Exclusion Files

Feedback

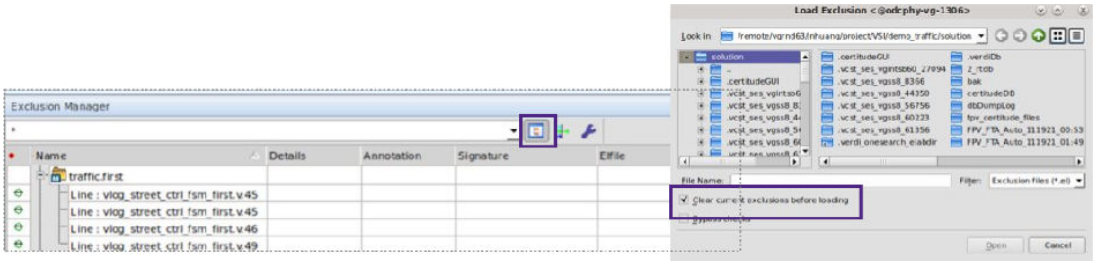
The predetermined elfile (COI.el, PD.el, OC.el, OA.el, FC.el) are read automatically during `compute_fta`, `compute_bounded_cov` and `save_property_density_results`.

To disable automatically loading predetermined elfile, set the following formal variable before running the commands

```
set_fml_var fml_cov_overwrite_waiver true -global
```

To removing elfiles from Verdi Coverage:

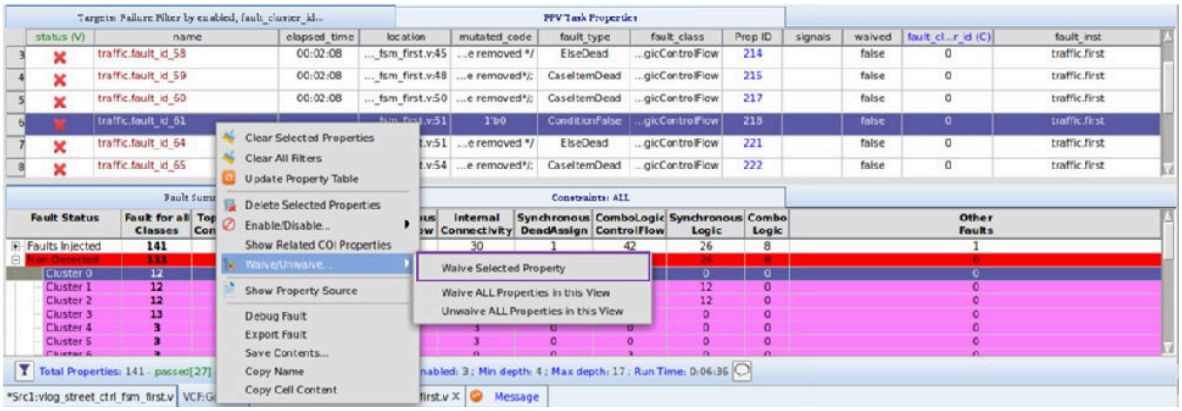
- Check *Clear current exclusions before loading* while importing the exclusion to clear current exclusion file under exclusion manager
- Unexclude exclusion items under exclusion manager flatten mode



20.8.4 Apply Waivers in FTA

To waive fault properties in FTA, waive or unwaive selected faults using the following commands:

```
fvwaive traffic.fault_id_61
fvunwaive traffic fault_id_61
```



Waiving Faults Commands

Feedback

- To save waived faults to waiver file:

```
save_waiver_file -fta -file fta.el
```

- To waive corresponding fault properties from the elfiles with previous coverage database (will be effective until compute_fta)

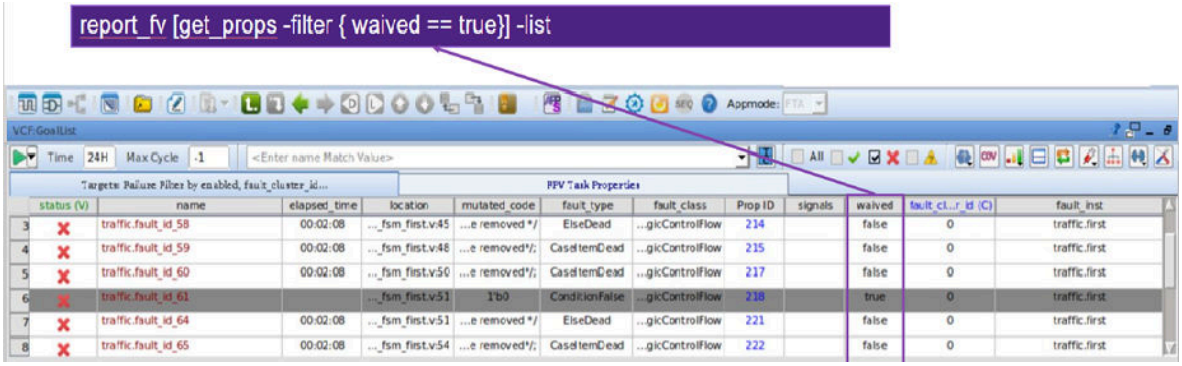
```
read_fta_waiver -elfiles fta.el -cov_input COI.vdb
```

- To unwaive corresponding fault properties from the elfiles (will be effective until compute_fta)

```
remove_fta_waiver -elfiles fta.el
```

- Report waived properties

```
report_fv [get_props -filter { waived == true}] -list
```



20.9 Using the VC Formal Signoff Dashboard

Use the VC Formal Signoff Dashboard to facilitate easy formal verification signoff on your designs. The Signoff Dashboard provides a single interface where you can:

- Revisit your FPV setup and instrument coverage and faults for analysis
- Rerun FPV and compute formal core to assess code coverage of proven assertions
- Find the root cause formal core holes and add assertions if needed
- Invoke and run FTA
- Root cause non-detected faults and add assertions if needed.
- Analyze coverage with integrated vdCov view in VC Formal GUI

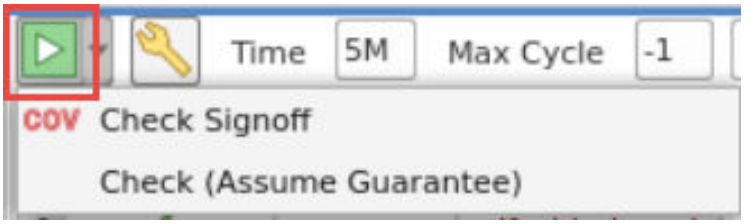
Feedback

The Signoff Dashboard provides the following effort levels:

1. *Low Effort*: Performs structural analysis, runs fast, and finds easy holes in the COI, including over-constraint analysis.
2. *Medium Effort*: Perform formal core coverage analysis, including over-constraint analysis. Performs a stronger analysis than the low effort analysis.
3. *High Effort*: Perform FTA and formal core coverage analysis, including over-constraint analysis. Performs a stronger analysis than the medium effort analysis.

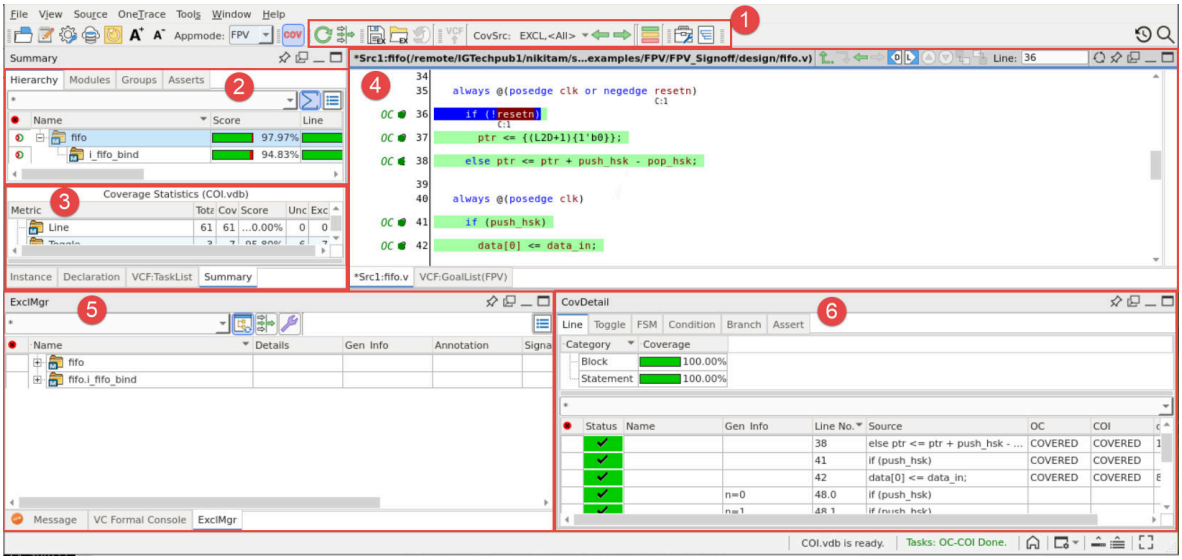
20.9.1 Opening Signoff Dashboard

After you run FPV analysis, select *Signoff Dashboard* from the drop-down list.



The *Signoff Dashboard* appears as illustrated in Figure 181.

Figure 181: Formal Signoff Dashboard



- **1**: SignOff Taskbar
- **2**: Hierarchy View of Coverage figures
- **3**: Coverage Statistics Summary for Signoff

Feedback

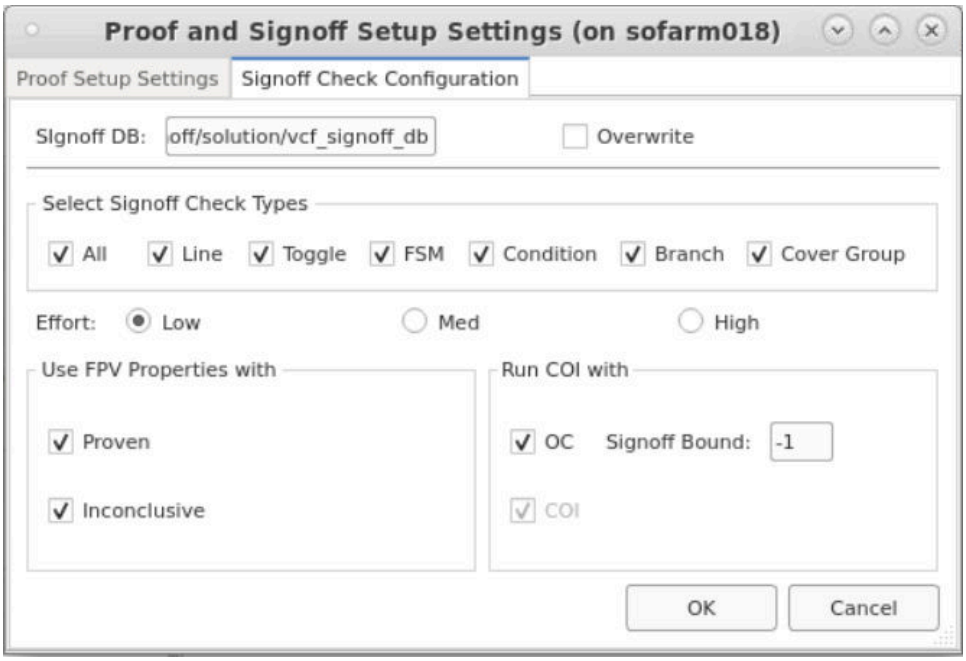
- **4**: Source Browser for Signoff Analysis
- **5**: Exclusion Manager for Signoff
- **6**: Coverage Detail for Selected Hierarchy

20.9.2 Configuring Signoff Dashboard

Configuring Signoff Dashboard in GUI

Click in *VCF:GoalList* to open the *Signoff Dashboard* configuration dialog box.

Figure 182: Signoff Dashboard Configuration Options



Where,

- **Signoff DB**: Specify the directory to store signoff database. For example, if OC + COI is already run, you will be storing the results and reusing them for subsequent stages like FC & FTA. Default is *vcf_signoff_db*.
- **Overwrite**: Forces fresh analysis at each stage.
- **Select Coverage Types**: Specify the metrics for signoff.
- **Effort**: Specify the effort for signoff.

Feedback

    - **Low**: Over-constraint Analysis+ COI (default)
    - **Med**: Over-constraint Analysis + Formal Core
    - **High**: Over-constraint Analysis + Formal Core + FTA
- **Run COI with**
    - **OC**: Enable and disable OC analysis. If it is disabled, signoff_bound will be disabled too.
    - **Signoff_bound**: Bound to compute signoff metrics. Default is “-1”, that is, “no bound limit”.
    - FTA marked as non_detected up to bound N.
    - **OC** marked as Uncovered if not covered up to a depth of N
- **Use FPV Properties with**
    - **Inconclusive**: Bounded results for FC and FTA analysis are included by default. Use this option to exclude them.
    - **Proven**: Proven properties for FC and FTA analysis are included by default. Use this option to exclude them.

Specifying Metrics for Signoff

Use the `signoff_config -type < >` command to specify the coverage metrics. Specify this command before the `read_file` command.

Syntax

```
signoff_config -type <line | cond | toggle | fsm_state | fsm_trans |
 branch | cg | fault_rtl | fault_conn | cov_all | fault_all | all>
```

Configuring Signoff Dashboard in Tcl

You can also use the `signoff_check` command in the Tcl script to specify these options.

```
vcf>signoff_check -help
Usage: signoff_check    # Executes the signoff flow in various efforts.
        [-effort <value-list>] (Signoff effort for analysis. Default is
 low. Values: low, med, high, coi, oc, fc, fta)
        [-time <Units in seconds(S), minutes(M) or hours(H)>]
                               (Timeout for command (default: 24H).)
        [-signoff_db <directory_name_or_path>]
                               (Directory to save the signoff flow
 analysis database in. Default is <vcf_signoff_db>.)
        [-no_overwrite]        (Disables fresh analysis at each stage.
 Default is to overwrite any existing results/database of a previous run
 with fresh run.)
```

Feedback

```
        [-signoff_bound <int>] (Bound to compute signoff metrics. Default
is -1 i.e. no bound limit)
        [-property <list-or-collection-of-properties>]
                               (List or collection of properties.)
        [-no_oc]               (Disable OC analysis.)
        [-no_fc]               (Disable FC analysis.)
        [-run_finish finishCmds]
                               (Set of commands to run when the signoff
effort concludes)
        [-type <line | toggle | fsm_state | fsm_trans | branch | cg |
all>]
                               (Specify the coverage metrics to be
monitored)
        [-signoff_dashboard]   (Load the coverage database in the GUI)
        [-add_constraints]     (Include constraints involved in formal
core)
```

20.9.3 Reporting Signoff Results using Tcl Command

The results generated during each effort level can be dumped using `signoff_report` command.

Syntax

```
vcf> signoff_report -help
Usage: signoff_report    # Reports signoff information
        [-all_instances]         (Output coverage % summary of all
instance)
        [-all_modules]           (Output coverage % summary of all
module)
        [-list]                  (Output one-line report of coverage
items)
        [-verbose]               (Output detail report of coverage items)
        [-no_summary]            (Output report without summary details
of coverage items)
        [-collection]            (Output report of collection of coverage
items)
        [-excluded]              (Output one line report of all excluded
coverage items)
        [-instance <instanceName>] (Output coverage % summary of a
specific instance)
        [-module <moduleName>]   (Output coverage % summary of a specific
module)
        [-effort <effortLevel>]  (Output coverage % summary of specified
effort level.:
                                   Values: low, med, high)
        [-report_type <reportType>](Output coverage % summary or
properties based on
     provided option. Values: percentages, properties)
        [-type <typeValue>]      (Output coverage summary based on
provided type:
                                   Values: line, cond, toggle, fsm_state,
```

Feedback

```
                                    fsm_trans, branch, cg, fault_rtl,
                                    fault_conn, cov_all, fault_all, all)
       [-property <propertyList>](Output coverage summary based on
provided properties)
```

20.9.4 Performing Low Effort Signoff

You can perform low effort COI analysis and over-constraint analysis.

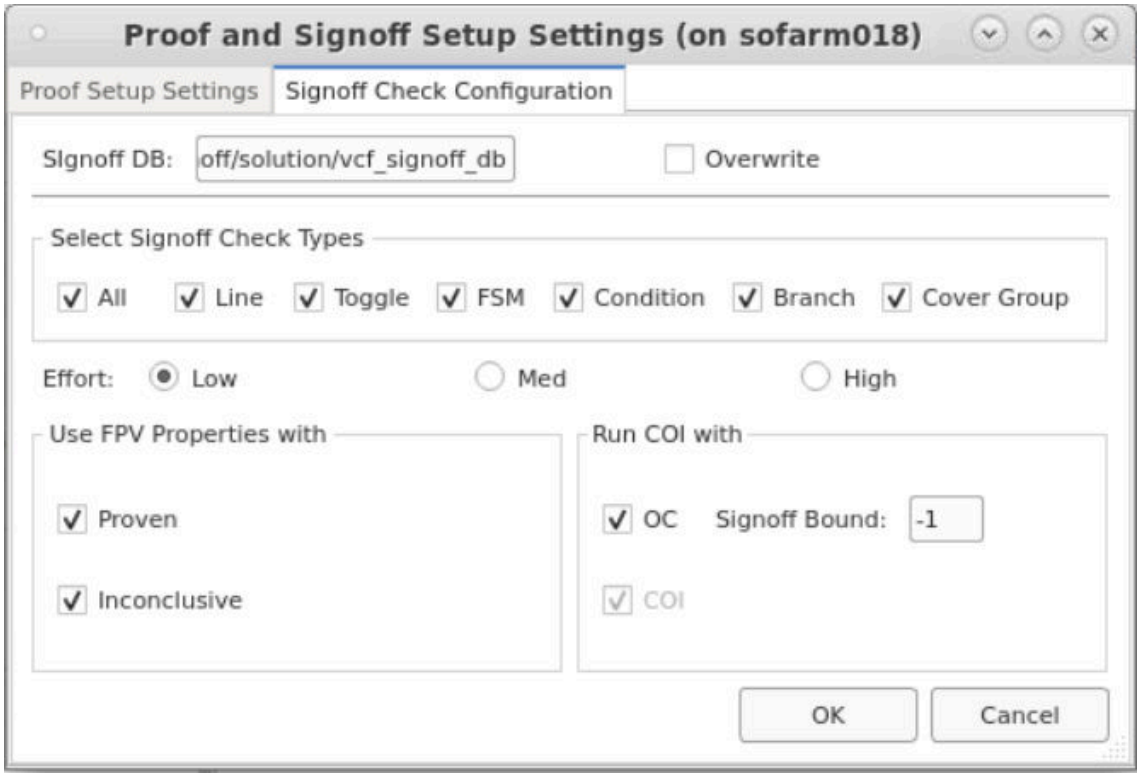
- *COI analysis*: Formal assertions are not activated by driving stimulus. The tool instead analyzes logic which feeds into an assertion and ensures that it cannot violate the property. The logic structurally connected to a property is known as the Cone of Influence (COI).
- *Over-Constraint Analysis*: Over-constraint analysis is designed to ensure that there are no constraints in the design preventing legal areas of code from being exercised. The analysis works much in the same way as an FCA reachability analysis, however for over-constraint analysis, VC Formal differentiates between areas of code that are unreachable with constraints and unreachable without constraints. Over-constraint analysis is an important step as while under-constraints may cause false property failures, over-constraints can cause false proofs.

To run over-constraint analysis in the Signoff Dashboard ensure that the correct coverage metrics have been provided with the `signoff_config` command before the `read_file`/`elaborate`.

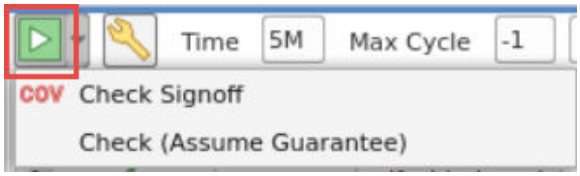
Feedback

To perform low effort signoff:

1. Select **Low** from the **Effort** list.

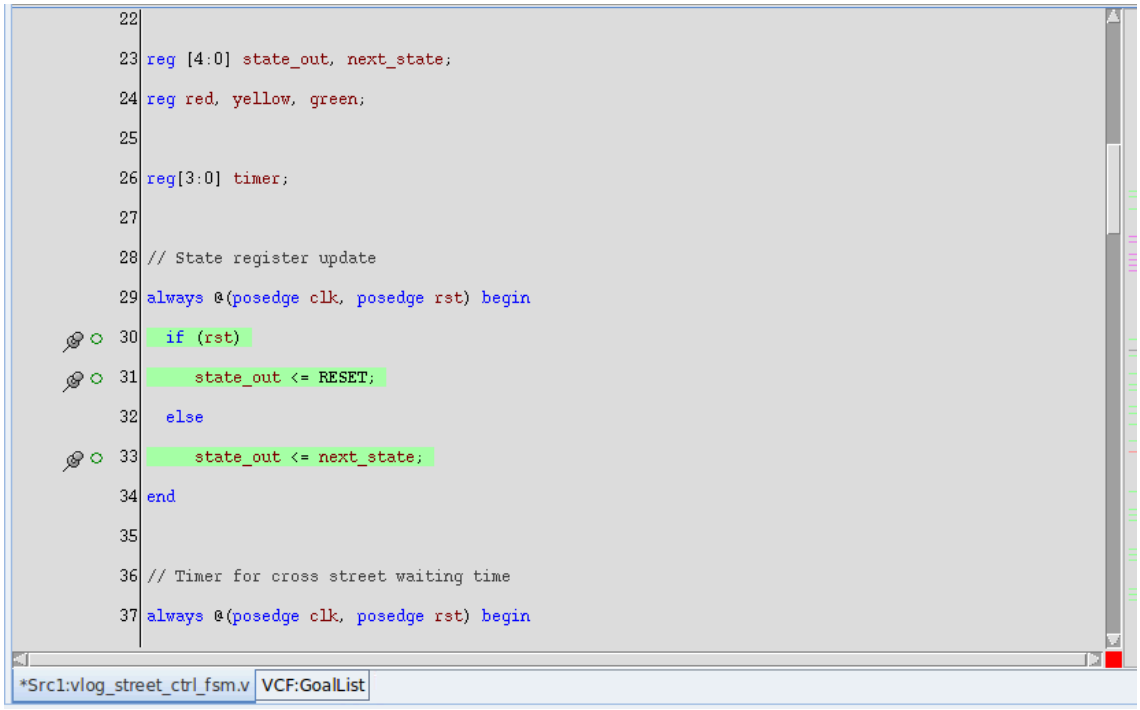


2. Configure the Signoff Dashboard as described in Configuring Signoff Dashboard.
3. Click *OK*.
4. Click *Check Signoff* to start computing signoff effort.



The coverage view of the OC analysis is displayed. Analyze the source code for the over-constraints and fix your assumes.

Feedback



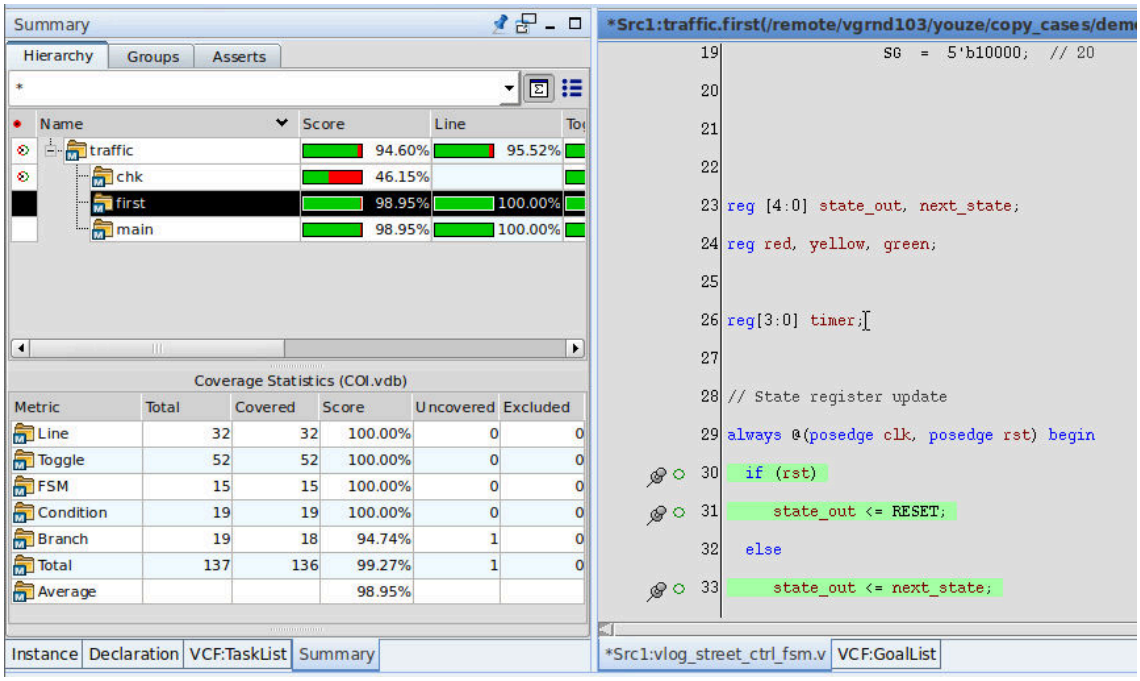
5. You can stop computing signoff effort by clicking the icon.

   The *Load Signoff Database* dialog box appears to enable you to load the new DB. Click *OK* to load the database. Click *Cancel* to continue.

6. Click check signoff to start computing signoff effort. *Check Signoff* to start computing signoff effort.

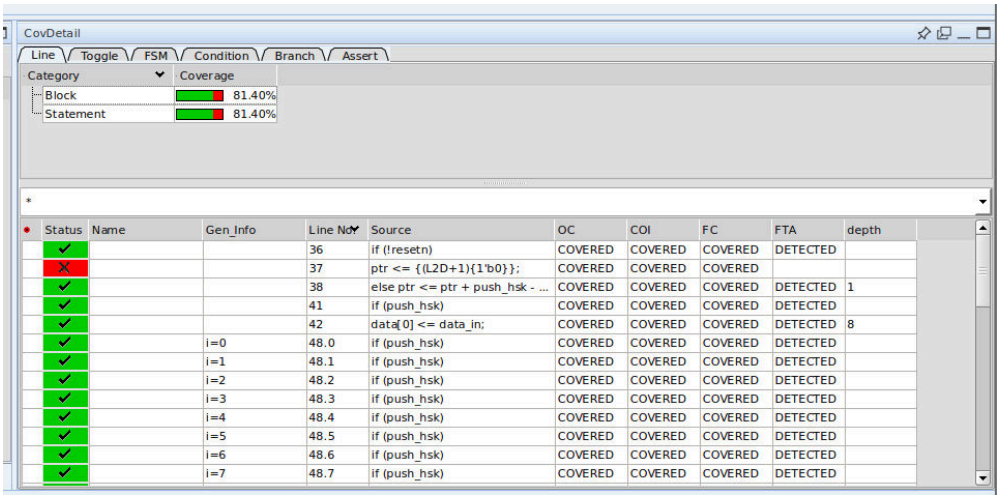
   The COI Analysis is applied on over constraint analysis database to generate COI Coverage. Use the Signoff Dashboard Taskbar to load the generated vdb to debug.

Feedback



Note:

The CovDetail view includes the following columns: OC, COI, FC, FTA (shows corresponding states) and depth (shows maximum depth number of the cover items for any stages) as shown in the following figure:



Similarly, following batch mode command can be used

```
signoff_check  -effort low -time 5M -signoff_bound -1
 -signoff_dashboard
```

20.9.5 Performing Medium Effort Signoff

You can perform medium effort signoff with the Formal Core Analysis.

The formal core of any proven property consists of the following three components:

- A set of registers in the design that are used by the formal engines to prove the property.
- A set of constraints that are used to prove the property.
- A set of inputs that are involved in proving the property.

The above definition of formal core also applies to the properties with bounded proofs. If a property has a bounded proof up to a depth k (no counter examples up to depth k), then you can get the formal core for the property for a depth less than equal to k. In such cases, the formal core contains the registers, constraints, inputs used by the formal engines for the bounded proof up to the specified depth k.

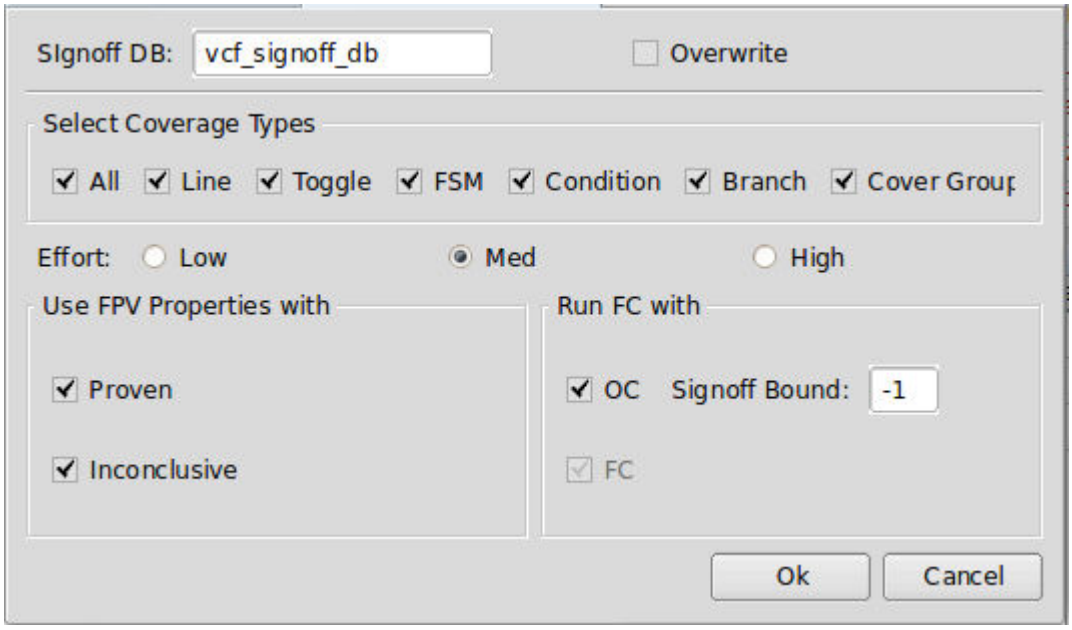
Formal core is a more accurate measure of which areas of the design have been involved in the proof (or bounded proof) of a property. The formal core will be a subset of the COI and is more accurate than property density, however it performs a formal analysis so the recommendation is to flush out early verification holes using property density first.

To run formal core analysis in the Signoff Dashboard firstly ensure that the correct coverage metrics have been supplied with the `signoff_config` command before the `read_file`/`elaborate`.

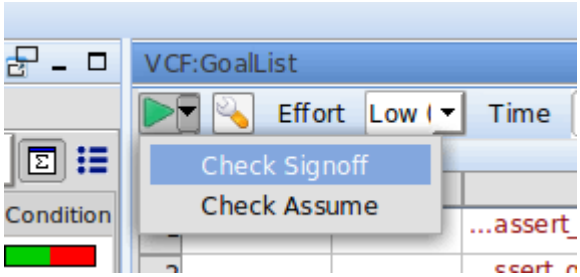
Feedback

To perform medium effort signoff with formal core analysis:

1. Select *Med* from the *Effort* list.

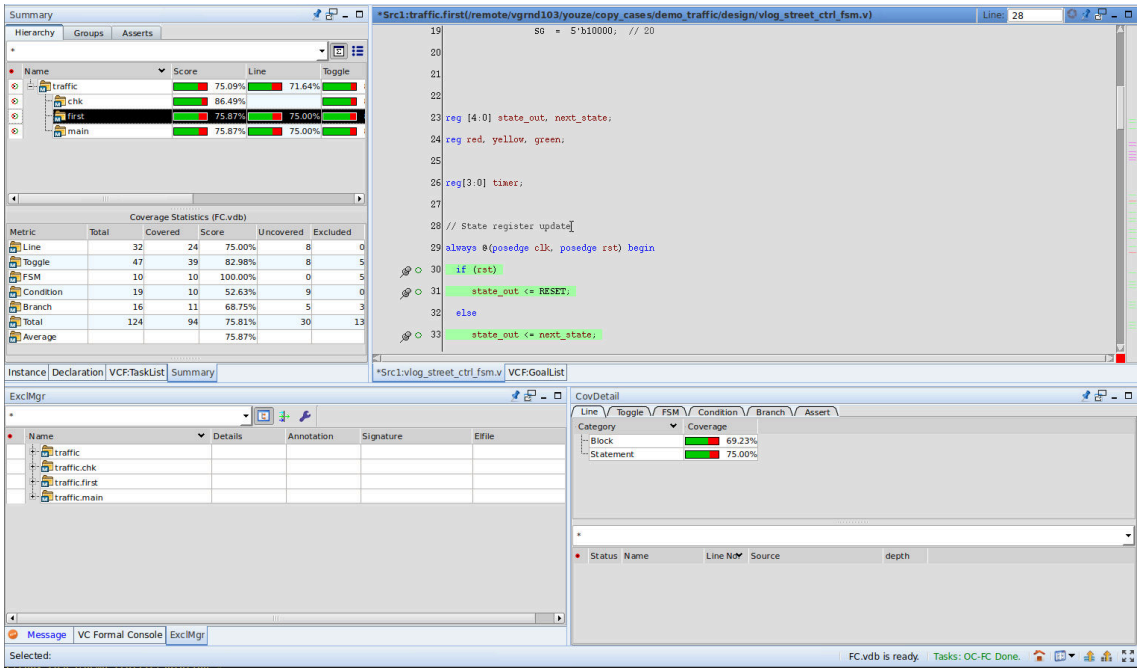


2. Configure the Signoff Dashboard as described in Configuring Signoff Dashboard.
3. Click *OK*.
4. Click *Check Signoff* to start computing signoff effort.



The formal core refines the coverage metrics from the COI Analysis. Use the Signoff Dashboard Taskbar to load the generated vdb to debug.

Feedback



5. You can stop computing signoff effort by clicking the icon.

   The *Load Signoff Database* dialog box appears to enable you to load the new database.

6. Click *OK* to load the new database for analysis. Click *Cancel* to continue.

7. Similarly, following batch mode command can be used:

```
signoff_check  -effort med -time 5M -signoff_bound -1
 -signoff_dashboard
```

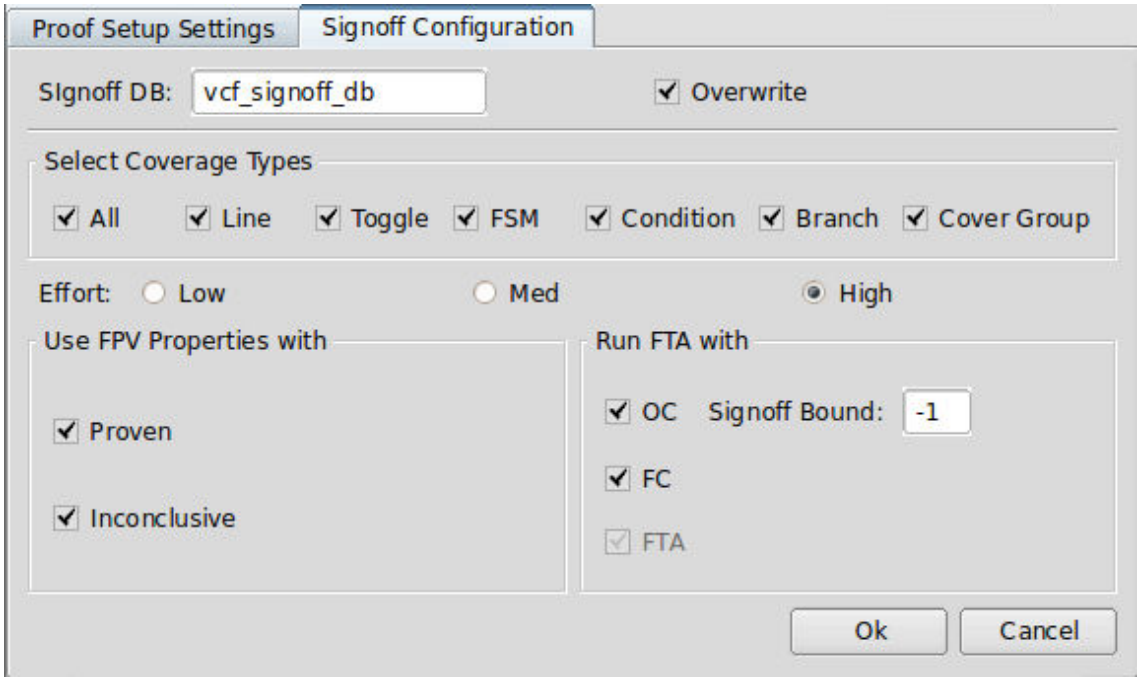
20.9.6 Performing High Effort Signoff

The high effort signoff uses FTA. To run FTA analysis in the Signoff Dashboard, firstly ensure that the correct coverage metrics have been supplied with the `signoff_config` command before the `read_file`/`elaborate`.

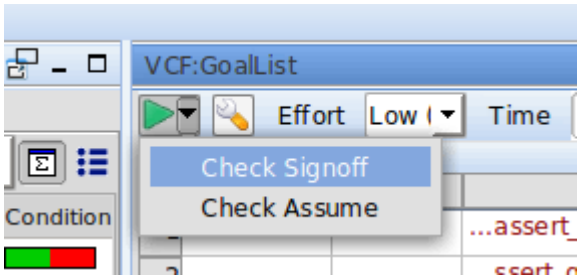
Feedback

To perform high effort signoff with FTA:

1. Select *High* from the *Effort* list.

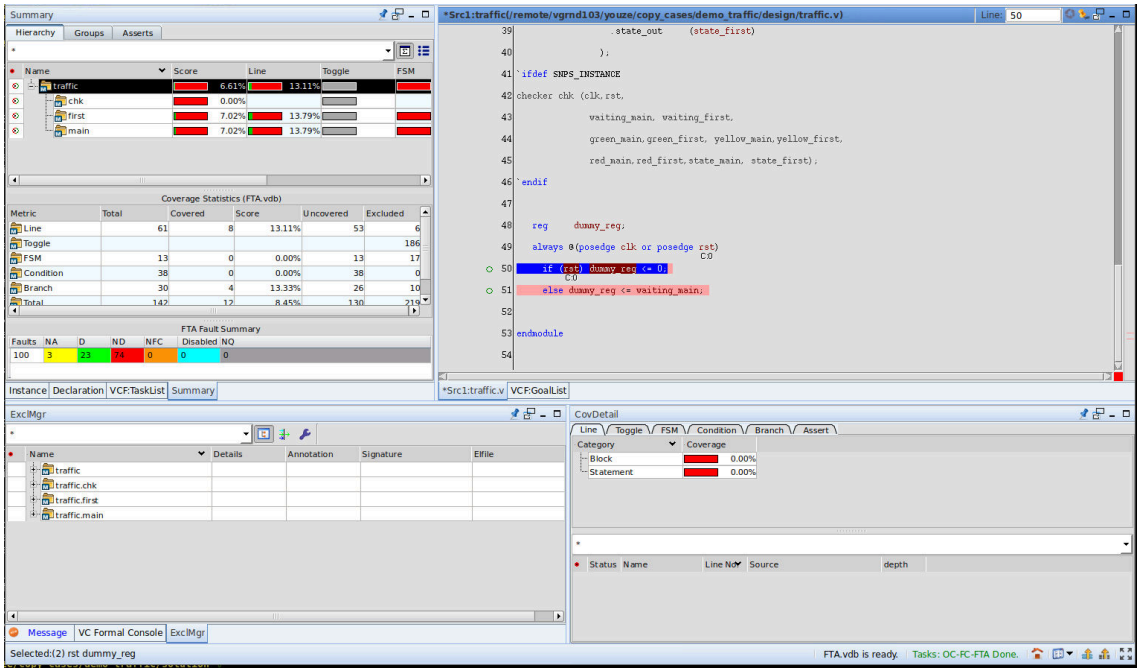


2. Configure the Signoff Dashboard as described in Configuring Signoff Dashboard.
3. Click *OK*.
4. Click *Check Signoff* to start computing signoff effort.



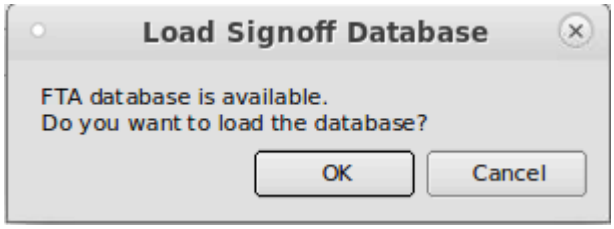
The FC+FTA refines the coverage metric further from formal core and FTA analysis. Use the Signoff Dashboard Taskbar to load the generated vdb to debug.

Feedback



5. You can stop computing signoff effort by clicking the icon.

   The *Load Signoff Database* dialog box appears to enable you to load the new DB. Click *OK* to load the database. Click *Cancel* to continue.



6. Similarly, following batch mode command can be used

```
signoff_check  -effort high -time 5M -signoff_bound -1
 -signoff_dashboard
```

20.9.7 Signoff Dashboard Taskbar

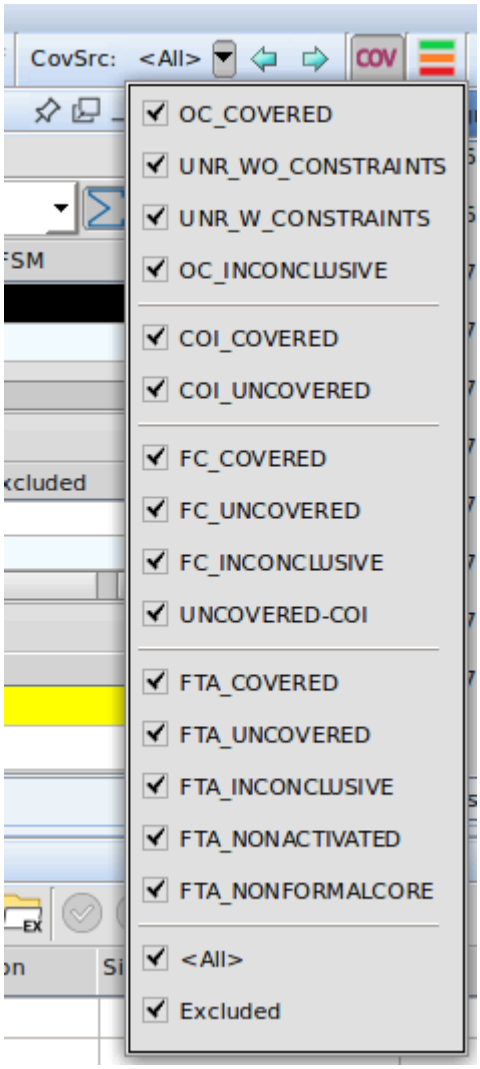
After performing the coverage analysis, use the Signoff Dashboard taskbar to debug the updated vdb..

Figure 183: Signoff Taskbar

Feedback

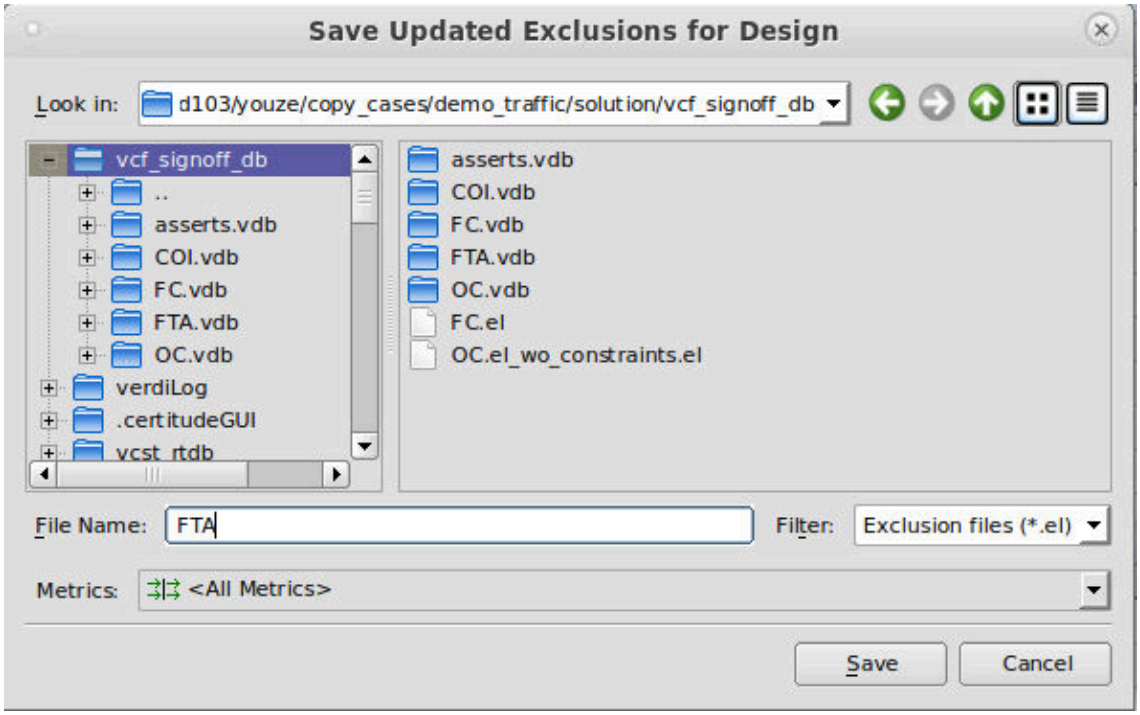
- *Reload Coverage Database*: Reload the generated signoff database.
- *Signoff Database Filter*: Open OC, COI, FC and FTA databases and choose types.

  Note:
  Hover over each goal status to view its description in a tooltip.



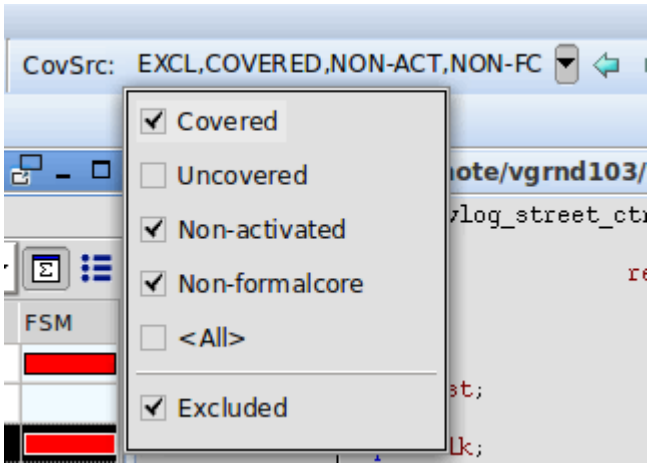
- *Save Exclusion*: Save exclusion file.

Feedback

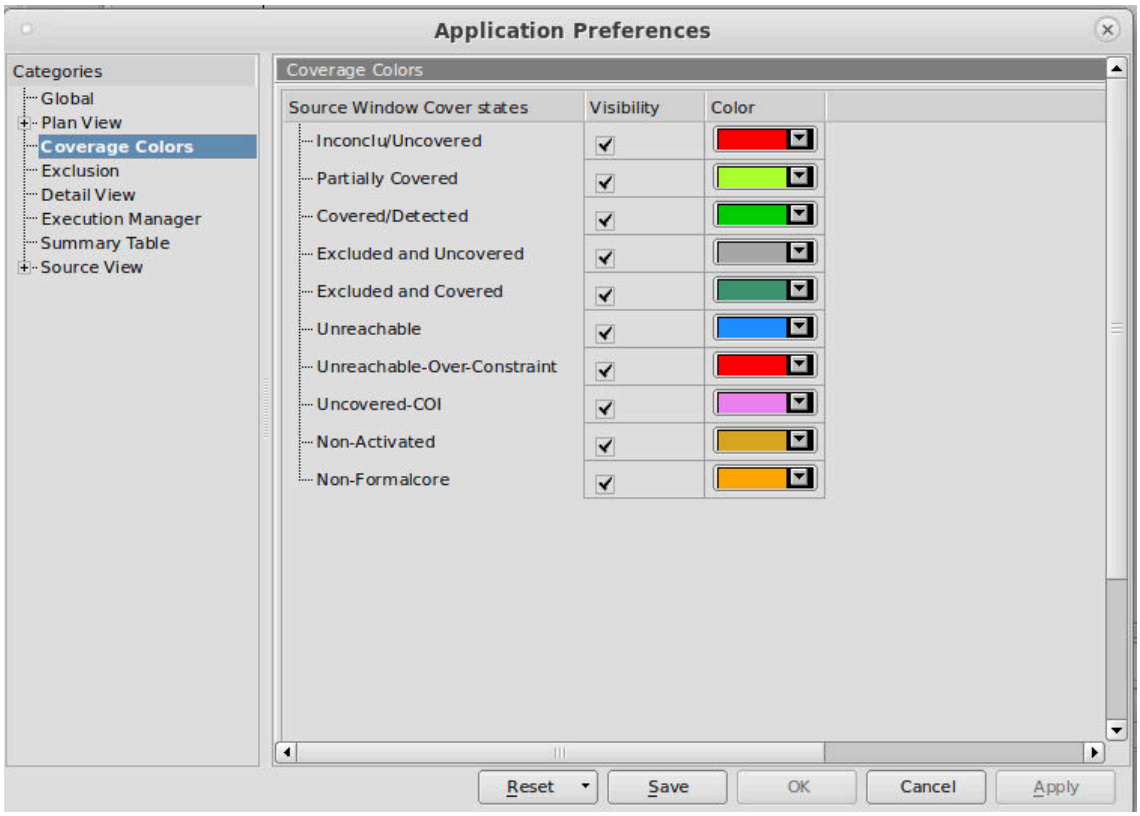


- Load Exclusion: Opens the load exclusion file dialog.
- Recalculate: Recalculate the scores after exclusions.
- Map to VCF goals: Map the items to VCF goallist.
- CovSrc: EXCL,<All> Source Browser: Choose status which want to check then click arrow to navigate forward and backward.

Feedback



- COV button: Raise or hide VC Formal views.
- Coverage color preferences: Choose custom colors for different coverage statuses.



Feedback

20.9.8 Understanding OCFT Annotation

The OCFT annotation is used to showcase the merged result in the VDB from all the signoff stages. This helps to visualize the results cumulatively without much effort. It takes the intersection of all VDBs generated during the run.

For each analysis being run, it maps to OCFT notation corresponding to each line target.

Figure 184: OCFT Annotation in GUI

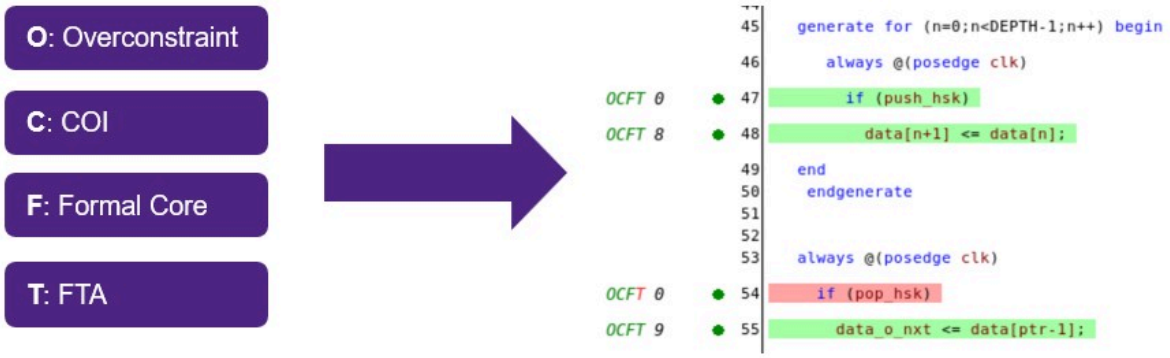
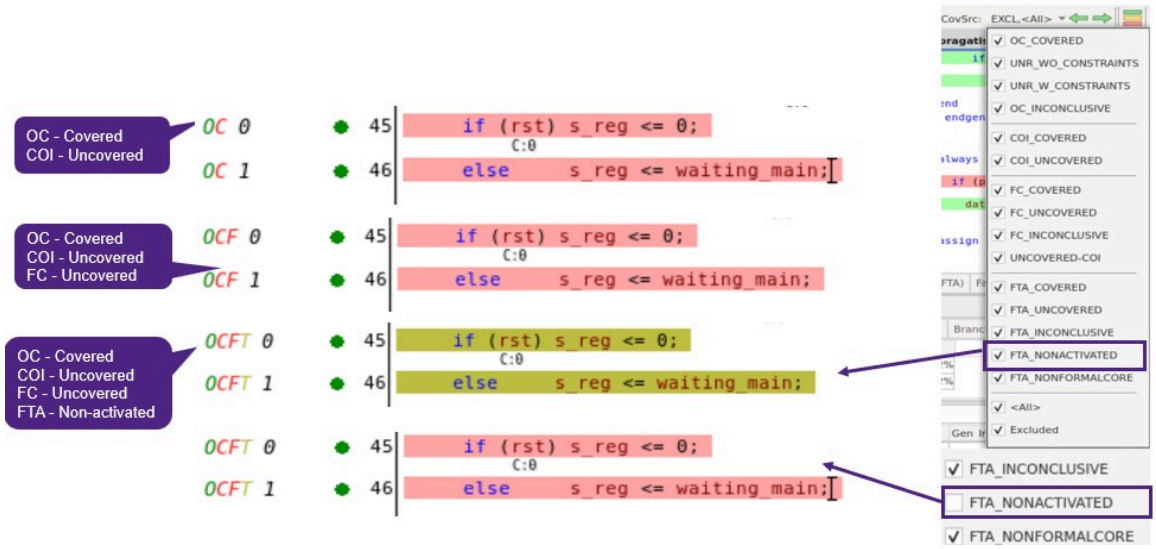


Figure 185: Meaning of the Annotation for Each Scenario



21

Formal Testbench Analyzer App

VC Formal FTA application provides unique capability to measure the quality of the formal verification environment. Combining VC Formal with Testbench Quality Assurance Fault Injection through RTL mutation, FTA can be used to check if the injected faults can be detected by the formal testbench, thus identify serious weakness in the formal verification environment.

Getting Started • *What is the VC Formal FTA App?* • *Formal Signoff with VC Formal FTA* • *Accessing FTA Lab Example*	**Debugging, Reviewing and Saving Results** • *Generating Verification Summary* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Viewing Witness Traces* • *Clustering Faults for Analysis* • *Debugging Non-Detected Faults with SEQ App* • *Exporting Faults for Analysis* • *Mapping FTA Results to Coverage Database* • *Saving and Restoring Sessions*
Setting Up and Running FTA • *Enabling FTA Application mode* • *Configuring Fault Instrumentation* • *Running Fault Analysis in TCL* • *Running Fault Analysis in GUI*	**Getting Help on Commands** • *FTA Tcl Commands*
Accelerating Proofs • *Enabling Fast Fault Activation Analysis*	

21.1 About Formal Testbench Analyzer

The key for obtaining a formal signoff of a design block is to have a formal testbench environment that ensures:

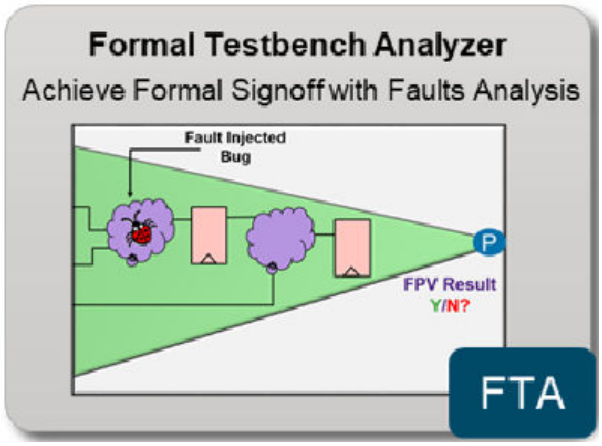
- Enough assertions are written to cover the complete design functionality.
- The set of properties are correct and complete.

There are code coverage metrics such as the property density, over-constraint and formal core (Obtaining Formal Signoff For a Design) coverage analysis that can be used in the

Feedback

signoff flow. However, although code coverage does provide a quantitative measurement, it does not provide qualitative measurement of the verification environment. Even if 100% formal core coverage is achieved, there could still be design bugs that can escape the formal testbench environment and make it all the way to silicon. The cost of these bugs can be very high.

Figure 186: FTA App



VC Formal FTA application provides an orthogonal metric to measure the quality of the formal verification environment. Combining VC Formal with Testbench Quality Assurance Fault Injection through RTL mutation, FTA can be used to check if the injected faults can be detected by the formal testbench, thus identify serious weakness in the formal verification environment. This is an essential step in achieving signoff to ensure the formal verification environment is of the possible highest quality.

Note:

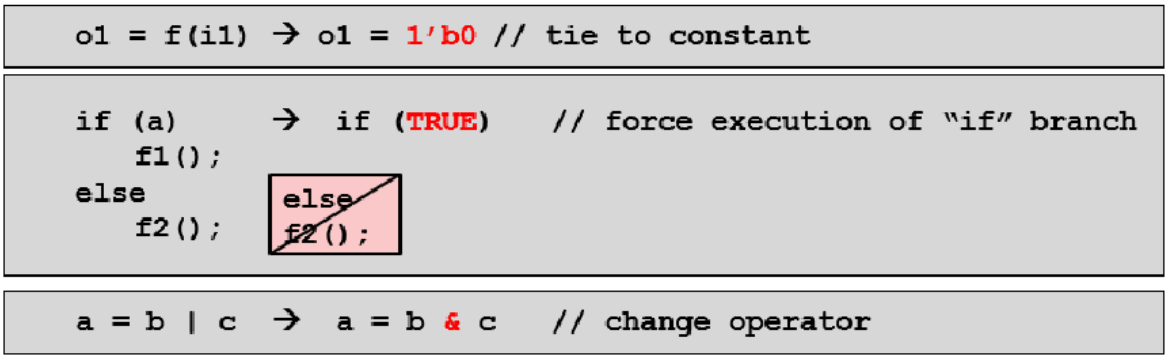
All the VC Formal applications require specific license keys. For more information on the license keys required for each of the VC Formal applications, see the VC Static Platform Installation Notes. In addition, a Testbench Quality Assurance license key (certitude-base-unlimited-sim) is required to use the FTA App. The vcf shell checks out a single certitude-base-unlimited-sim license on need basis and once done it is checked back in.

21.1.1 Fault Injection

Testbench Quality Assurance Fault Injection works by injecting behavioral faults into the design code. It does this by making systematic mutation to the design. For example, in the following RTL code, there are several faults injected to mutate the logic and functionality of the code.

Feedback

Figure 187: Example Fault Injection Using RTL Mutation



One of the faults is to mutate the code and make o1 tie to a constant; another fault is to mutate the logic flow by forcing the “if” branch to be always true; the last one is to mutate the operator from an “|” to an “&” operation.

21.1.2 Fault Classes

The faults that Testbench Quality Assurance injects in the design are categorized and listed by the fault type and their location in the functional structure of the design. These two characteristics make up the 'class' of the fault. The fault class helps to know whether the fault should be analyzed earlier or later when it turns out to be Non-Detected. This methodology presents the basic characteristics of the nine fault classes and provides interpretation keys for each one.

The nine fault classes with a brief description are listed in the table below, where the priority ranking is given to the most important fault class to start debugging the failures first.

Figure 188: Fault Classes

Fault Set	Faults type	Description	Priority
Connectivity	TopOutputsConnectivity	Output stuckAt 0, 1 or negative	1
	InternalConnectivity	Internal connectivity stuckAt 0, 1 or negative	3
RTL	ResetConditionTrue	Make part of design is always stay in reset condition	2
	SynchronousControlFlow	Chang control condition to 0, 1 or negative	4
	SynchronousDeadAssign	Remove assignment to synchronous logics	5
	ComboLogicControlFlow	Change condition to dead or negative	6
	SynchronousLogic	Convert the expression in synchronous logic	7
	ComboLogic	Convert the expression in combo logic	8
	OtherFaults	The faults not belong to the type above	9

Feedback

Detailed explanation of each false classes are given below, in the order of their importance.

- TopOutputsConnectivity

  This class contains output connectivity faults from the TopName configuration option.

  When connectivity faults fail to be detected, they have a strong impact on the qualification performance. This is another reason why all the faults of this class must be resolved (either made detected or disabled) before proceeding to the rest of the analysis.

- ResetConditionTrue

  This class contains all faults of the type ConditionTrue located in a reset condition. This class of faults does not make sense in formal setup.

- InternalConnectivity

  This class contains all other connectivity faults.

  As with the other connectivity faults, the connectivity input faults are amongst the easiest to analyze.

  When the verification is observing internal parts of the design, the corresponding connectivity input faults must be analyzed in priority, and must be resolved before the analysis can proceed further.

- SynchronousControlFlow

  This class contains the faults called control flow. This class contains all faults of the following types: ConditionTrue, ConditionFalse, NegatedCondition, CaseItemDead, ForDead, WhileDead, and ElseDead, that are located in clocked process (VHDL) or always (Verilog) structures.

  Faults located in synchronous structures generally reveal weaknesses that are more significant from a verification point of view, and are also easier to analyze. In this set of faults, the control flow faults have a more general and significant impact on the functional behavior of the design. Therefore, faults that are both of control flow type and located in clocked processes are the first ones to be analyzed after the ones in the first two sets.

- SynchronousDeadAssign

  This class contains all the faults of the DeadAssign type located in clocked process (VHDL) or always (Verilog) structures.

  This class of faults typically points to signals that are sometimes or always cut off from the rest. The analysis should be directed towards a resolution of Non-Detected faults of this class when it shows that the signal is completely cut off.

- ComboLogicControlFlow

  This class contains the faults called control flow. This class contains all faults of the following types: ConditionTrue, ConditionFalse, NegatedCondition, CaseItemDead, ForDead, WhileDead, and ElseDead. The classes above are located in combinatorial process (VHDL) or always (Verilog) structures.

  These faults should be analyzed when the faults of the previous classes have been resolved.

- SynchronousLogic

  This class contains the faults of the following type: DeadOperator, DeadStatement, Operator, SwapOperand, FlipFirst, FlipLast, BitFlip, ForSkipFirst, and ForSkipLast. The classes above are located in combinatorial process (VHDL) or always (Verilog) structures.

  These are typically the last faults to be analyzed. They are the most difficult to interpret and typically address corner cases.

- ComboLogic

  This class contains the faults of the following type: DeadOperator, DeadStatement, Operator, SwapOperand, FlipFirst,FlipLast, BitFlip, ForSkipFirst, and ForSkipLast. The classes above are located in combinatorial code. As these faults are not of control flow type or in synchronous processes, they should be analyzed last.

- OtherFaults

  This class contains all faults that are not included in the previous eight classes. By default, this class should essentially contain faults that are disabled by Testbench Quality Assurance .

21.1.3 Fault Qualification Analysis Process

If the goal of functional verification is defined as, "finding functional faults in designs", then the following sequence must occur to find a fault:

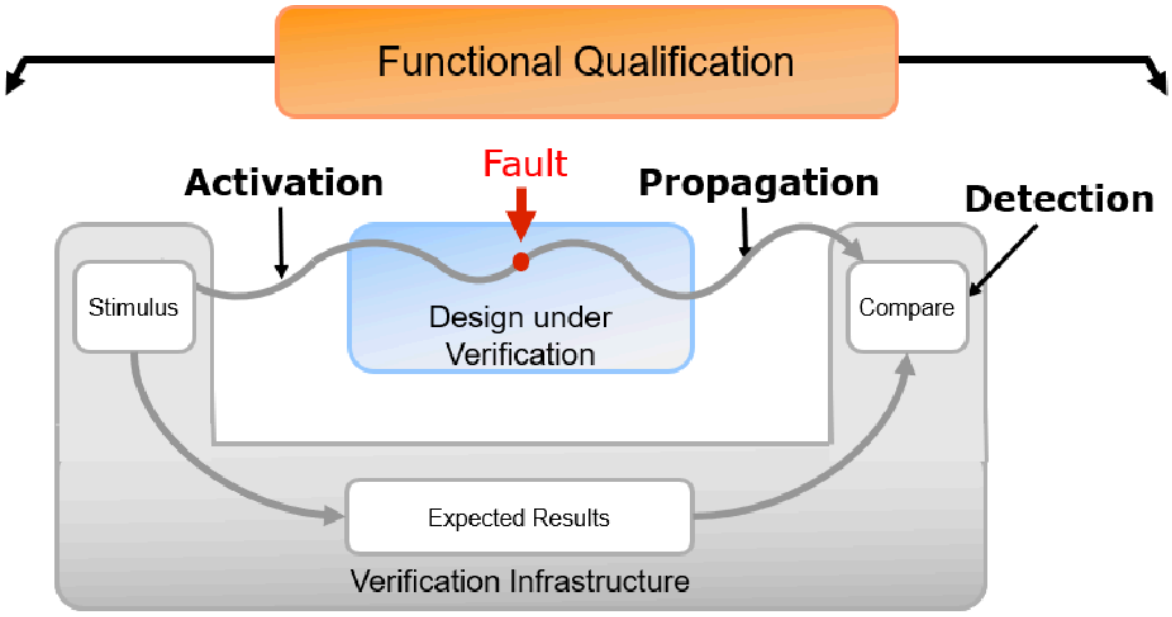
ACTIVATE fault -> DETECT fault

- 'Activating' a fault means exercising the part of the design that contains the fault.
- 'Detecting' a fault means making a comparison between the design and a reference behavior, and then reporting that the fault has been found.

The figure below describes the general functional qualification through fault analysis.

Feedback

Figure 189: Functional Qualification Through Fault Analysis



21.1.4 Fault Qualification Analysis with FTA

In VC Formal, the general concept is the same. However, because formal property verification is assertion based, the same sequence for detecting a fault is defined as follows:

- *'Activating' a fault:*When the FTA App is invoked, first, the FTA analysis determines whether a fault belongs in the cone of influence logic (COI) of any properties. When a fault is in the COI of any property, it is qualified as activated.

  When a fault is not in the COI of any of the properties, it is qualified as non-activated. When a fault is non-activated, it means that the list of proven and inconclusive assertions in the testbench do not involve the part of the logic in the design where the non-activated faults are located. These non-activated faults should be reviewed to determine whether more assertions should be written to verify this part of the design or they should be waived because they are covered in the other steps in the verification plan.

- *'Detecting' a fault:*Run all proven (sometimes bounded proven as well) assertion properties on the original design with each of the faults activated to see if this fault can be detected by one of the properties getting falsified. If not, this fault is non-detected.

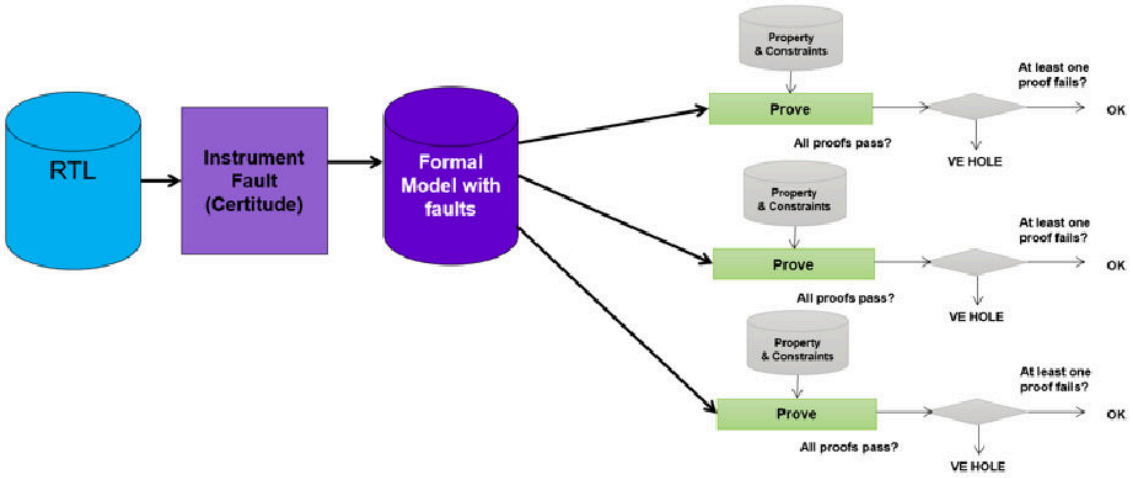
  The non-detected faults can help identify design logic where faults are not caught by any of the existing checkers therefore quantify the quality of the formal property verification environment. If all the checkers pass in the presence of a fault, it indicates

Feedback

weakness in the formal verification environment, either missing checker, incomplete specification or over-constraining. It provides guidance for further examination of the environment, for example, to add missing checker to catch the bug injected.

The figure below describes how FTA helps identify weakness in formal verification environment.

Figure 190: Fault Qualification in Formal Verification



This video focuses on The Formal Testbench Analyzer (FTA) application in VC Formal that is designed to evaluate the quality of your formal testbench by identifying verification holes. It achieves this by injecting artificial faults into the design using Synopsys Certitude technology and analyzing whether existing assertions can detect these faults. The FTA flow involves two steps: setting up the fault injection environment and running FTA.

21.1.5 Video Links

A demo video on how to use the FTA application is available on SolvNetPlus.

21.2 Methodology

The key for obtaining a formal signoff of a design block is to have a formal testbench environment that ensures the quality by catching as many bugs as possible.

The FTA App can be used after you think that the formal testbench has a comprehensive set of properties that are proven or inconclusive on the original RTL in the FPV mode. You can use the same constraints and target properties in the FTA App on a fault injected RTL model to qualify whether an injected fault can be detected.

Feedback

Ideally, you should already have achieved 100% property density coverage, 100% over-constraint coverage, and 100% formal core coverage before you start FTA analysis. Otherwise, you will need to spend time to debug the non-activated faults which should be already identified by property density or formal core. Also, if the RTL code is unreachable code due to over-constraint, it will be easier to identify the root cause in an environment without fault injection. When you are satisfied with the formal core coverage, then it is time to setup for fault analysis.

FTA flow includes the following steps:

- Setup fault injection environment
- Setup formal environment with fault detection
  - Set Appmode to FPV
  - Add fault injection setup to design read
  - Enable FPV properties for fault detection qualification, disable others
  - Sanity run of chosen FPV properties
  - Change Appmode to FTA
  - Run FTA
  - Review results
  - Debug
  - Improve FTA results by adding missing checkers in FPV
- Coverage
  - Fault coverage with Testbench Quality Assurance database
  - Code coverage from FTA

The results of the FTA can have 3 major outcomes:

- The fault is detected by at least one of the proven or inconclusive FPV properties getting falsified. The FTA analysis for this fault will stop at the first falsification, then mark the fault detected.
- None of the FPV properties is falsified in the presence of an injected fault, this fault becomes non- detected.
- A fault can also be inconclusive.

Feedback

The focus should be on the non-detected faults found in the formal testbench, since it implies weakness in the verification environment. The causes can be due to:

- Missing or incorrect property.

  Note:

  The effort level `bounded` and `bug hunting` are undefined for the FTA Application mode. The supported effort level for the FTA application is `default`. Set the effort level to default as follows for the FTA application mode: `set_fml_var fml_effort default`

  In some cases, you can choose to bypass sanity check. See section Bypass Sanity Check.

21.2.1 Inputs to Setup and Run FTA

Prerequisites for using FTA include having a comprehensive setup of FPV properties; configure fault injection; and configuration for FTA.

1. FTA App requires FPV setup with comprehensive set of assertions properties and run results and you are at the final stage of signoff. FTA App setup can be added to the existing FPV environment, so the specified properties from FPV task can be run with fault injected model in FTA.
2. Fault injection configuration set up files.

   FTA supports for native injection of faults for Verilog, System Verilog, and VHDL. There are 2 files to set up and control the fault injection.

   a. certitude_config.cer

   When the certitude_config.cer file is not present, VC Formal automatically creates this file when you use the `read_file` command. This file specifies the top-level model and the FTA mode.

   For Verilog or SystemVerilog designs, it must contain the following commands.

```
setconfig -TopName=work.<TopModulefromFPVTask>
setconfig -Simulator=native
```

   For VHDL designs, it must contain the following commands. For example:

```
setconfig -TopName=work.<TopModulefromFPVTask>
setconfig -Simulator=vcstatic setconfig -FaultSet=[list RTL ]
 setconfig -OptimizerAnalysis=true
setconfig -FormalFlowInstrumentType=ExternalPort
```

   b. certitude_hdl_files.cer

Feedback

This file specifies the part of RTL to qualify for fault injection. For example, to specify a by file name:

```
addverilog -qualify -library="work"
 -file="../vlog_street_ctrl_fsm.v"
addsystemverilog -qualify -library="work" -file="../rtl/bridge.sv"
```

or

```
addvhdl -qualify -library="c7amba_dacif" -file="rtl/dacif_pkg.vhd"
```

To specify by a file list:

```
analyzeverilog -qualify -library="work" -f="<filelist_filename>"
```

To ignore for fault injection for specific RTL code, –ignore option must be used with file name. A list of line numbers or a range of lines can be specified after the tool auto generated command. e.g.,

qualify -verilog -unitlist=[list *]

```
addverilog -qualify -library="work" -file="../rtl/bridge.sv"
 -ignore=(1,3)
addvhdl -qualify -library="work" -file="../rtl/bridge.sv"
 -ignore=(1,3,7..10)
```

When the certitude_hdl_files.cer file is not present, VC Formal will automatically create this file and injects faults for all the modules in the design.

Alternatively, you can also specify the module(s) to inject faults by using this FTA command in Tcl:

```
vcf> fta_init -scope <list-of-modules>]
```

Note:

Use the `fml_fta_alt_config_dir` formal variable before `read_file`/`analyze`/`elaborate`, to specify the location from where VC Formal should use the certitude_config.cer/certitude_hdl_files.cer files.

3. Tcl setup for FTA

   Because FTA is for FPV signoff, therefore, you should start with the current FPV Tcl file and modify to enable FTA App. The steps are:

   a. Setup to enable fault injection

      Existing FPV `read_file`/`elaborate` command needs to be modified by added the option `-inject_fault.`

Feedback

Note:

When you use `analyze`/`elaborate` command in the FTA flow, VC Formal reports the following error if you do not set the `fta_init -top $top` *CER-Info : Reading config file: '/global/apps/vcstatic_2020.12-1/certitude/tclExtension/certitude_config.cer'*

CER-Warning : [0022] No configuration file could be found in the current directory. Error: 0 Use error_info for more info. (CMD-013) Information: script 'run.tcl' stopped at line 10 due to error. (CMD-081) Extended error info:0 while executing "analyze -format sverilog -vcs " -f ../design/filelist " " (file "run.tcl" line 10)

- For Verilog/Sverilog designs:

```
read_file -inject_fault all|RTL|Connectivity|none …
```

Or

```
set_app_var fml_multi_step_fta_flow true
analyze …
elaborate -inject_fault all|RTL|Connectivity|none …
```

Note:

When you use the `-inject_fault` in `read_file/elaborate`, VC Formal reports the following warning. *Warning-[SM_MLAC] Modeling latched logic for always_comb top.sv, 53 Modeling latched logic for always_comb for signal 'check[2]'.*The SM_MLAC messages are coming because of dead assign type of faults as for such faults synthesis creates latch behavior. However these messages have no negative impact on the correctness of the results. If there is no SM_MLAC warning in the previous FPV run, you can add suppress_message SM_MLAC before `read_file`/`analyze` to suppress these messages.

Note:

When you use your own certitude_hdl_files.cer file, and when both `-inject_fault all|RTL|Connectivity`, except for `-inject_fault none`, and `setconfig -FaultSet` is specified, the `-FaultSet` setting takes precedence over the `-inject_fault` setting. For example, when both `-inject_fault all` and `setconfig -FaultSet=[ list RTL]` are used together, only RTL faults are injected. The `-inject_fault none` has more precedence over the `-FaultSet` setting.

- For VHDL designs:

```
set_fml_var fml_enable_vhdl_fta true
set_app_var fml_multi_step_fta_flow true
```

Note:

`set_app_var fml_enable_vhdl_fta true` must be set before `set_fml_var fml_multi_step_fta_flow true`, if VHDL is used.

```
analyze …
elaborate -inject_fault all|RTL|Connectivity|none …
```

b. Run FPV sanity check

After adding `-inject_fault`, the next step is to do the sanity check. A sanity check is to do `check_fv` and run the same properties in the FPV setup with the faults injected model, only the faults are all disabled internally by the tool. In this case, the FPV results should be the same as before adding the `-inject_fault` to the `read_file`/`elaborate` command to re-establish the same baseline.

See next section item b to see how to do this in GUI mode.

c. Run Formal Core computation and setup to limit FTA to Formal Core Logic only

By default, all activated faults are considered in FTA analysis. The number of faults can be further reduced by limiting to Formal Core Logic only. First formal core will need to be computed in FPV task before they can be used to qualify faults. This can be done in blocking or non-blocking mode.

```
compute_formal_core -block
```

See next section item c to see how to do compute formal core in GUI mode.

After formal core is available, a formal variable needs to be set to limit the FTA to run in logic in formal core only. This is done in Tcl only.

```
set_fml_var fml_qual_fault_in_fcore true
```

This formal variable can be set any where in the tcl before starting run in FTA application mode. Default is false.

d. Setup to configure FTA

By default, only Proven properties are included in the FTA analysis. However, fault classes and selection of properties can be configured using the `configure_fta_props` command. We will describe the options that are most commonly used for this command to configure what fault classes to be used for FTA qualification and also to specify property selection to be considered for FTA analysis.

Feedback

The option to configure fault classes is `-fault_class`. This is to give finer control to the fault classes already specified with `-inject_fault`. For example,

```
configure_fta_props -fault_class {SynchronousControlFlow
ComboLogicControlFlow} -par_task FPV
```

The option to configure list of properties to be used by FTA can be -type, -usage, -status. For example, to also include the inconclusive properties,

```
configure_fta_props -status {proven inconclusive} -par_tack FPV
```

There is also `-min_proof_depth` option you can use to specify the minimum proof depth for the inconclusive properties. When inconclusive properties are being configured to use for FTA analysis, by default, the tool will automatically select the minimum proof depth in all of the inconclusive properties specified for the FTA run.

Both configurations can be done in GUI as well. See next section item c.

e. To waive Faults in FTA

- Use the `-cov` line option to use VCS exclusion for saving FTA waiver file:

  ```
  read_file -inject_fault all -cov line -top top -vcs "..."
  elaborate top -inject_fault all -cov line -vcs "..."
  ```

- Waive specific faults

  ```
  fvwaive <fault_prop_name>
  fvwaive [get_fta_faults traffic.first]
  ```

- Save waived faults to an exclusion file.

  ```
  save_waiver_file -fta -file fta.el
  ```

  Waive the faults in the exclusion file for compute_fta or compute_signoff with high effort in Signoff Dashboard.

  ```
  read_fta_waiver -elfiles fta.el
  ```

  Syntax

  ```
  read_fta_waiver # Reading FTA waiver file
  [-elfiles exclusion_file_list] (fta waiver file to be read)
  ```

  Faults waived with `save_waiver_file -fta` associates each fault with a unique key - based on its class, the original RTL construct on which the fault was injected, and other unique attributes. If the design has changed when reading the waiver file with command `read_fta_waiver`, there are two scenarios

Feedback

  - The key for a fault remains unchanged and the waived status is re-applied.
  - The key changes and the waived status is not re-applied.

    For example, the RTL construct changes when operator “==” is replaced with “!=” on a line, results in the key for the faults on that line to change and the waived status not being re-applied.

- **Note:**
  If you have saved COI.el/FC.el under the current working directory, then the faults in COI.el/FC.el are automatically waived by compute_fta or compute_signoff with high effort in Signoff Dashboard.
- To view the waived faults:

  ```
  report_fv -filter {waived == true} -list
  ```

f. To get collection of FTA properties inside the instance list, use the following command:

   ```
   get_fta_faults <list-of-instances>
   ```

g. Setup to enable vacuity checks in FTA

   FTA is a powerful tool to boost testbench quality and to find undetectable bugs lurking inside a design. The proven and bounded properties in FPV are used in FTA. If a fault can make any of them fail, the fault can be captured by the assertions in this formal testbench. To maximize the efficacy of FTA, VC Formal supports a smarter way to extract properties from FPV. Not only the assertions coded for FPV are included in FTA, but the vacuous checks generated by VC Formal are now part of the detecting function. If a proven property in FPV becomes vacuous in a presence of a fault in FTA, then this fault is deemed detected.

   The following formal variable needs to be set to enable this feature.

   ```
   set_fml_var fml_fta_enable_vacuity_check true
   ```

   By default, the `fml_fta_enable_vacuity_check` formal variable is set to false.

h. Setup to enable cover properties in FTA

   You can include cover properties in FTA analysis and find out if the injected faults can be detected. Set the following variable to true to enable this feature.

   ```
   set_fml_var fml_fta_enable_cover true
   ```

   If the status for cover properties becomes uncoverable, the fault is detected for a particular block or instance. If the status becomes coverable, the fault is non-detected for a particular block or instance.

Feedback

i. Setup to disable properties not required for FTA analysis

If there are properties in the FPV environment that you don't want to use for the FTA run, you can simply disable them.

```
fvdisable <properties not required for FTA>
```

To exclude faults from specific instances, use the following commands:

```
fvdisable [ get_fta_faults <list-of-instances>]
```

To exclude all faults outside specific instances, use the following commands:

```
fvdisable -type fault_check
fvenable [ get_fta_faults <list-of-instances>]
```

j. Get a report of the instance based faults

```
fta_report <instName> -list/-verbose
```

Example

```
fta_report -instance {traffic.main traffic.first}
```

k. You can distinguish faults in different instances but the same module. To distinguish faults in different instances, set the following formal variable to true before the read_file/elaborate command:

```
set_fml_var fml_fta_inst_qual true
```

Example

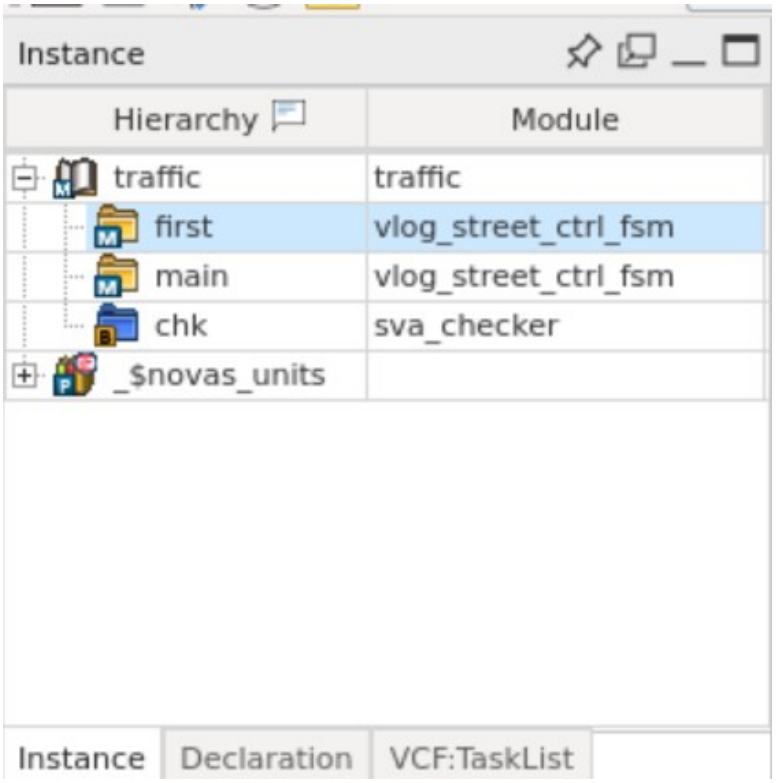
To view same faults from `get_fta_faultstraffic.first` and `get_fta_faults traffic.main`

```
set_fml_var fml_fta_inst_qual false
```

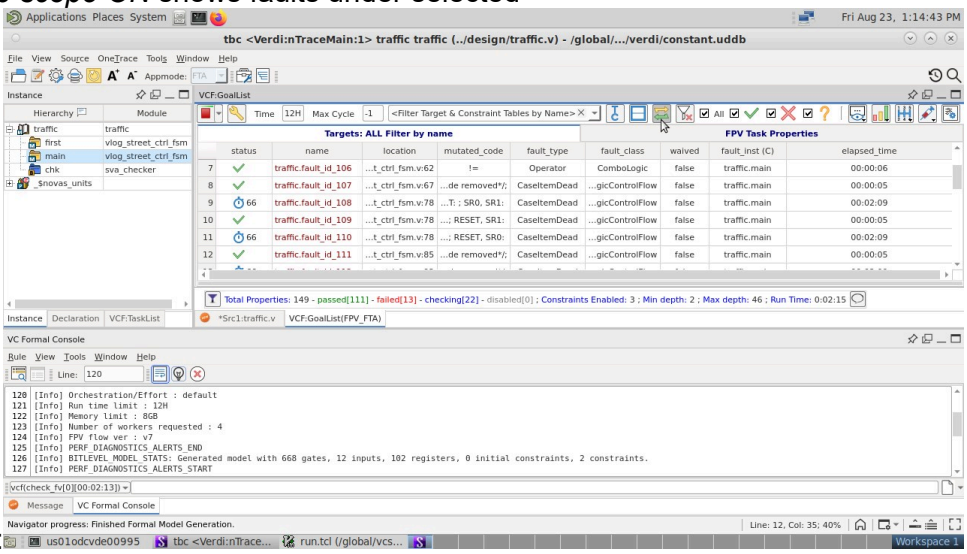
To view different faults from `get_fta_faults traffic.first` and `get_fta_faults traffic.main`:

```
set_fml_var fml_fta_inst_qual true
```

Feedback



The *Sync-scope ON* shows faults under selected instance:



l. To get a report non_detected faults under different instances, use the `get_fta_faults -status` option and `report_fv`.

Feedback

```
vcf>get_fta_faults traffic.first -status non_detected
vcf>report_fv [get_fta_faults traffic.first -status non_detected]
vcf>report_fv [get_fta_faults traffic.first -status non_detected]
 -list
vcf>report_fv [get_fta_faults traffic.first -status non_detected]
 -verbose
```

m. To enable faults only where line coverage properties are enabled using the `-cov line/all`, use `set_fml_var fml_fta_limit_to_cov true -global` before `compute_fta`.

n. To disable fault properties which are injected in clock trees during `compute_fta` (specified before `compute_fta`)

```
set_fml_var fta_disable_clk_fault true -global
```

4. GUI setup and run FTA

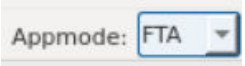
The setup labeling here will correspond with the same in the Tcl setup for FTA section above

a. Invoke fault injection is the same as in Tcl step above.

b. Sanity Check: After enabling fault injection, click in the FPV task to run sanity check.

c. Enter Tcl command to set fml var. See item c from previous section.

d. Switch to the FTA Appmode either from the drop-down list in the Verdi toolbar or from the RMB menu of TaskList.



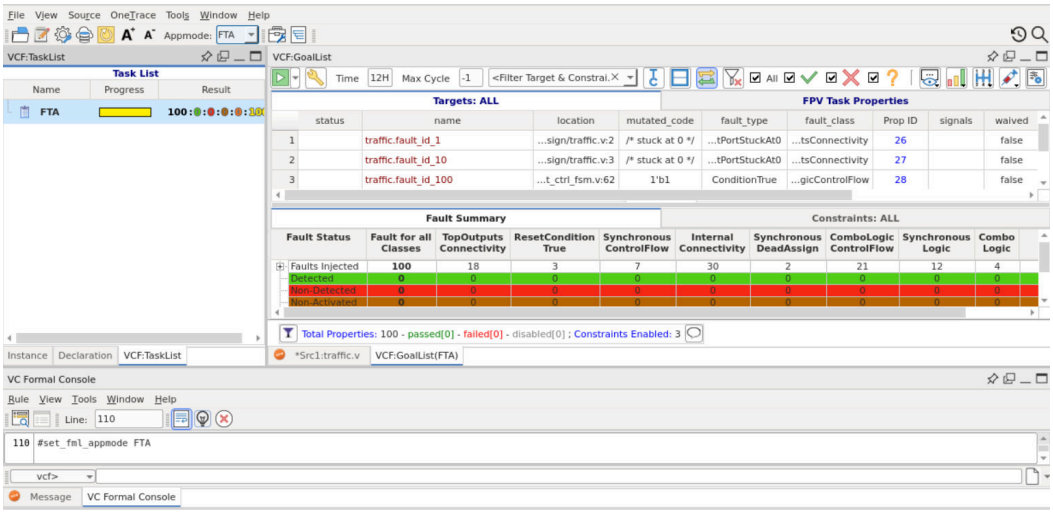
When you are in FTA Appmode the GUI layout view looks like this:

- The FTA default task is shown in the *Task List*.
- The fault properties are listed in the *Verification Targets* tab.
- The FPV properties used to qualify the faults are listed in the *FPV Task Properties* tab. It contains the properties from the current FPV parent task. It is read-only. You cannot modify the properties from this table.
- The Constraints in FPV are listed in the *Constraints* tab. It contains the constraints from the current FPV parent task. It is also read-only.

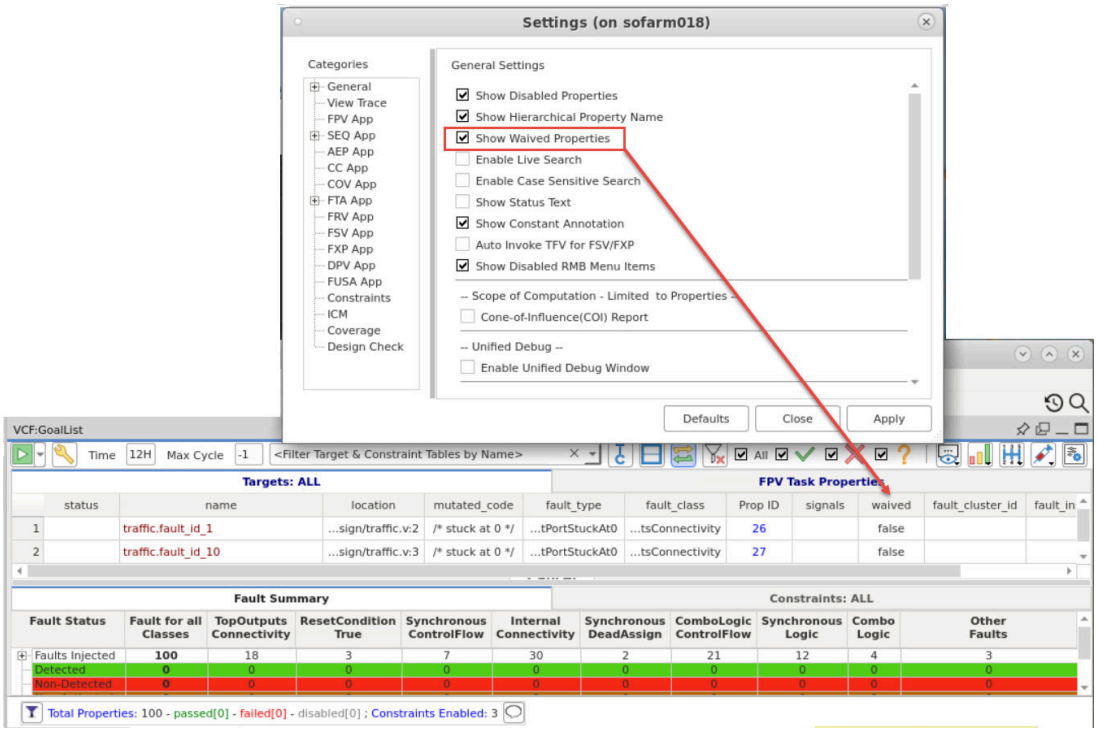
Feedback

- The fault classes and run status of the faults is listed in the *Fault Summary* tab.

Figure 191: FTA Appmode View



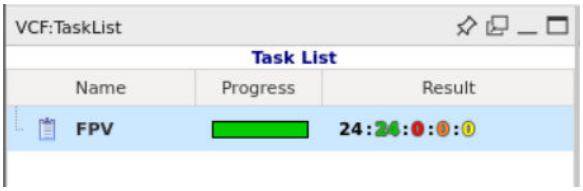
e. View the waived faults in GUI:



f. Running multiple FTA tasks using same FPV parent task.

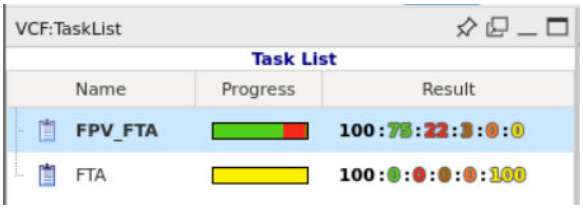
Feedback

- Complete the FPV run.



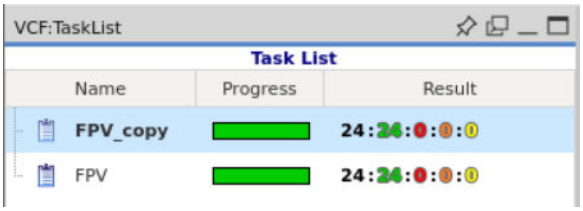
- Switch to the FTA appmode, execute the first FTA run:

```
configure_fta_props -status {proven inconclusive} -type assert
-min_proof_depth <number> compute_fta -par_task FPV
```

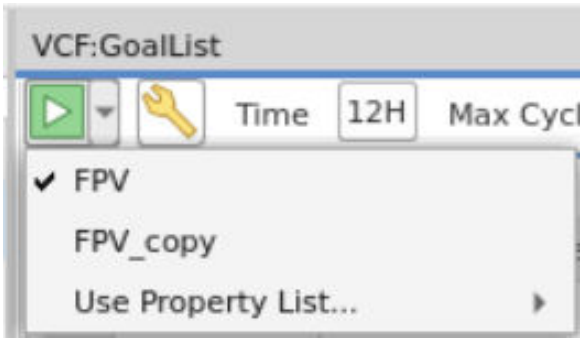


- To run a new FTA run for different faults or at different inconclusive depths, switch back to FPV to create a copy of FPV par task:

```
fvtask FPV_copy -create -copy FPV -keep_status
```

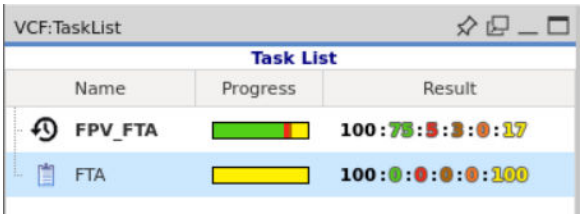


- Switch back to FTA to run FTA on FPV_copy (Run 2). The drop-down menu under run button shows the copied FPV task



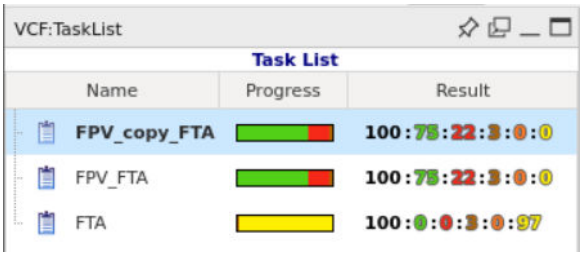
- Set FTA as the active task, and select a subset of faults or delete the old FPV_FTA run.

Feedback



- VC Formal runs the new FTA on the FPV copy:

```
configure_fta_props -status {inconclusive} -type assert
-min_proof_depth <number>
```



21.3 Using GUI to Configure Properties and Fault Classes

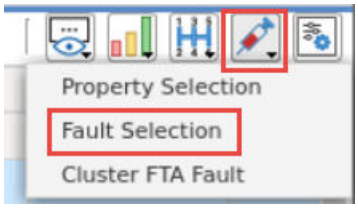
You can configure FPV properties (in the FPV Properties tab) and faults (in the Verification Tasks tab):

- Based on the proven or inconclusive status
- Based on the fault types

21.3.1 To perform FTA based on the statuses (proven or inconclusive):

- Click in the toolbar, and select *Property Selection*.

Figure 192: Configure FPV Properties in FTA Mode



The *Property Selection* dialog box appears.

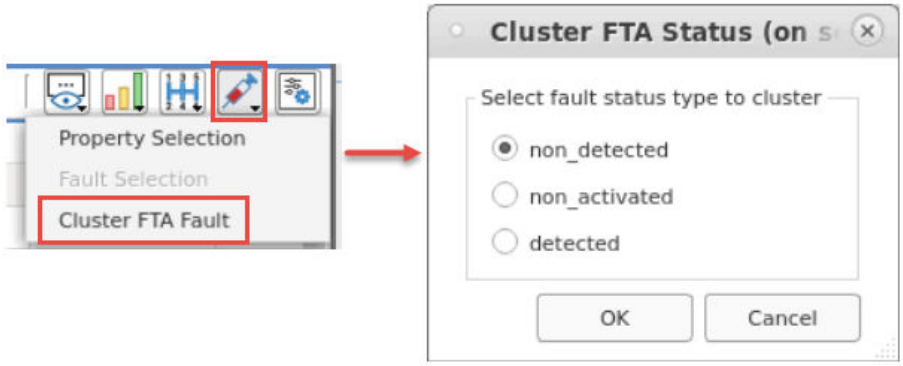
Feedback

1. Select *Proven* to use the proven properties for the FTA run.
2. If inconclusive properties are to be used as well, select Inconclusive with given depth.

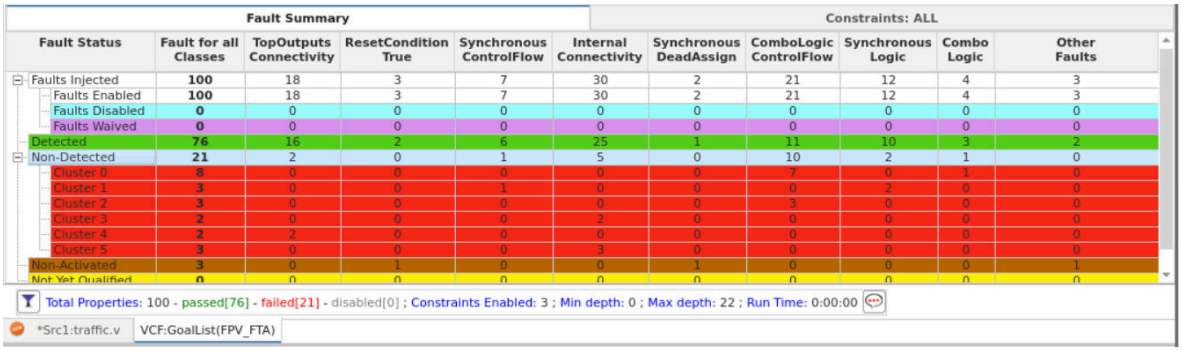
   In the *Depth* box, type the depth up to which you want to use the properties. The depth provided must not be greater than the minimum depth of the inconclusive properties in the parent FPV task.
3. Click *OK*.

21.3.2 To perform clustering of FTA Faults:

You can cluster FTA faults using the following option in the FTA application mode:



Select the fault type to cluster to view the changes in the GUI:



21.3.3 To perform FTA based on the fault types

- Click in the toolbar, and select *Fault Selection*.

  The *Fault Selection* dialog box appears.

Feedback

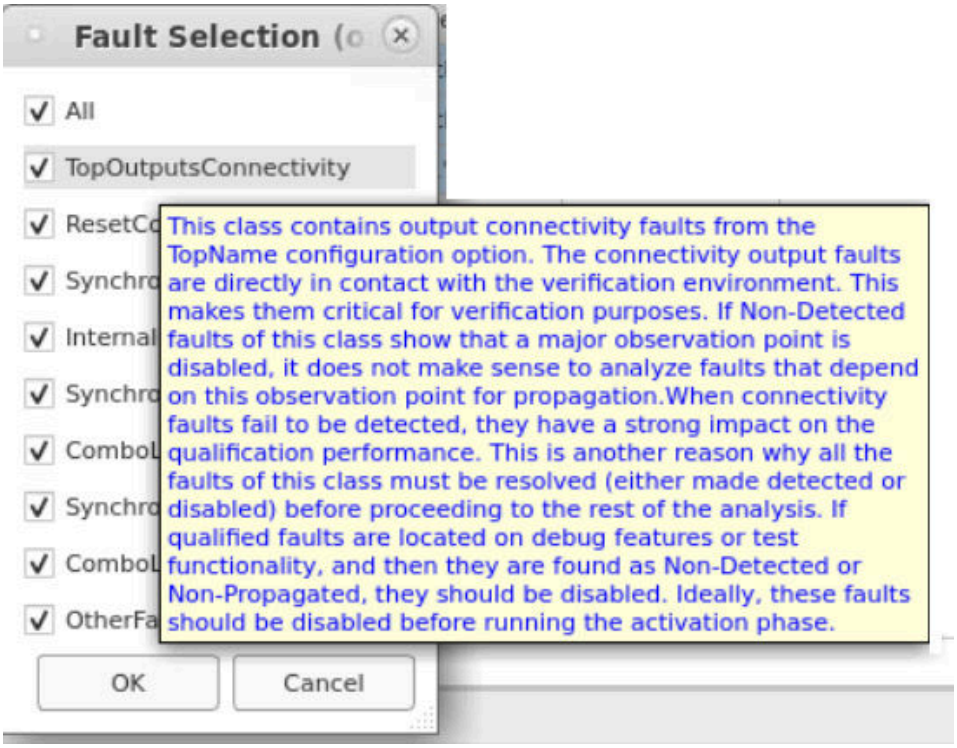
Figure 193: Fault Selection

- Select the types of the faults you want to see in the table and click *OK*.
- Note that by selecting a class from Fault Selection here, previously disabled fault properties (for example, from fvdisable command) will be re-enabled.

  Note:

  You can hover over each fault type to view the definition of the fault type.

Feedback



e. Enable cover property is the same as in Tcl command flow in above section item e.

21.4 What to Look for and What to Avoid

You can control to avoid running the sanity check every time after you performed sanity check once. You will need to save the set of the properties in FPV Appmode you used for sanity check and use it to configure the subsequent FTA runs.

Note:

Sanity check can only be done once without changes to the FPV environment and fault injection. Use save_session/restore_session in subsequent runs. Sanity check step is crucial because it provides the necessary learning that will tremendously improve FTA performance.

To save the properties in FPV Appmode, you will first need to save a list of property names. This can be done using a Tcl procedure. For example, the following proc will take the FTA proven assertions and save to a file “props_for_fta.txt”.

```
proc report_props {{status proven} {type assert} } {
foreach_in_collection my_prop [get_props -status {proven}] {
set my_prop_name [get_attribute $my_prop name]
echo "$my_prop_name"
```

Feedback

```
}
}
report_props > props_for_fta.txt
```

After this list has been collected, you should comment out the both of the command for FPV run “check_fv” and “report_props > props_for_fta.txt“. Then you need to configure subsequent FTA runs to use the list of properties you just saved.

```
configure_fta_props [exec cat "props_for_fta.txt"] -par_task FPV
```

This will bypass sanity check for all subsequent FTA runs. You can save time when there are no changes in the RTL and properties.

There are more techniques and recommendations on what to do and what to avoid are covered under following sections.

Bypass Sanity Check

You can control to avoid running the sanity check every time after you performed sanity check once. You will need to save the set of the properties in FPV Appmode you used for sanity check and use it to configure the subsequent FTA runs.

To save the properties in FPV Appmode, you will first need to save a list of property names. This can be done using a Tcl procedure. For example, the following proc will take the FTA proven assertions and save to a file “props_for_fta.txt”.

```
proc report_props {{status proven} {type assert} } {
    foreach_in_collection my_prop [get_props -status {proven}] {
       set my_prop_name [get_attribute $my_prop name]
       echo "$my_prop_name"
    }
}
report_props > props_for_fta.txt
```

After this list has been collected, you should comment out the both of the command for FPV run “check_fv” and “report_props > props_for_fta.txt“. Then you need to configure subsequent FTA runs to use the list of properties you just saved.

```
configure_fta_props [exec cat "props_for_fta.txt"] -par_task FPV
```

This will bypass sanity check for all subsequent FTA runs. You can save time when there are no changes in the RTL and properties.

Recommendations

There are more techniques and recommendations on what to do and what to avoid are covered under following sections.

Feedback

21.5 Testbench Quality Assurance Fault Coverage Database

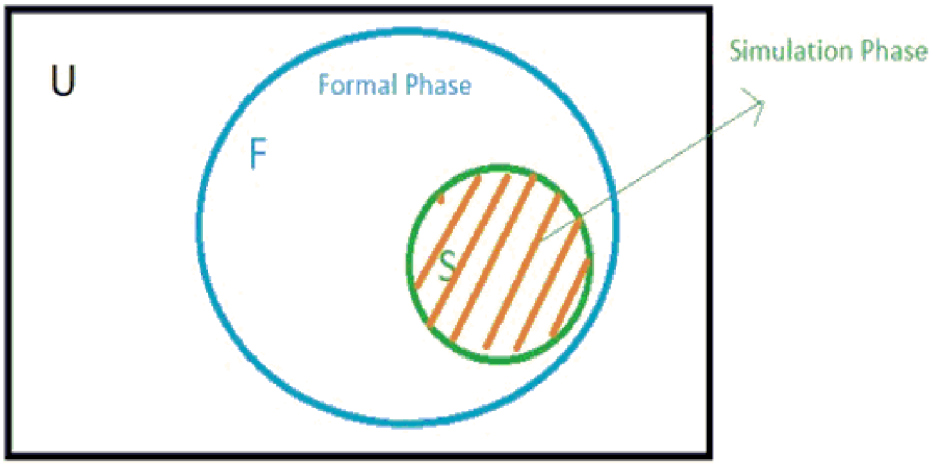
If a Testbench Quality Assurance fault coverage database from simulation is available, it can be read into FTA setup. This way, FTA will not consider any target marked “Detected” in the database, focus only on the not yet qualified faults.

Similarly, FTA results can also be saved into Testbench Quality Assurance fault coverage database.

21.5.1 Leverage Simulation Testbench Quality Assurance Fault Database

Consider the following diagram. Here, U is the set of all faults required to be converged. Let F be the set of faults targeted to be converged by Formal Methods. Out of the set F, let S be the set of faults already converged using simulation methods.

Figure 194: Leverage Simulation Fault Database



Previously, the set S which were already converged through simulation were again converged by formal methods. Starting with this release, you can exclude the set S already converged in simulation for FTA analysis in VC Formal.

In other words, simulation data after using Testbench Quality Assurance for fault analysis for a design, is already available in Testbench Quality Assurance DB. Consider that you already have converged on faults during simulation using Testbench Quality Assurance. You can leverage this data and get faster convergence in VC Formal by not performing FTA on the already converged faults.

The command to import a Testbench Quality Assurance database is:

```
read_faultdb -name <path_to_certitude_db>
```

Feedback

This command must be applied before `read_file`/`elaborate` command. that is, before step 3 item b. above. For example:

```
set_fml_appmode FPV
read_faultdb -name user.my_certitude_db read_file -inject_fault RTL -sva
 …
……

…

compute_fta -par_task <FPV_task>
```

21.5.2 Saving FTA Results Into Testbench Quality Assurance Database

FTA results can be saved into Testbench Quality Assurance DB. You can merge the saved Testbench Quality Assurance database to the simulation Testbench Quality Assurance DB to increase coverage. Use the `save_faultdb` command to save the FTA results in a Testbench Quality Assurance database.

Syntax

```
save_faultdb [-name <path_to_save_certitude_db>] .
```

21.5.3 View FTA Results in Testbench Quality Assurance Database

After saving the FTA results into Testbench Quality Assurance database, Testbench Quality Assurance can be invoked separately to see the results in Testbench Quality Assurance. The steps are:

1. Invoke Testbench Quality Assurance.

   ```
   $VC_STATIC_HOME/certitude/bin/certitude
   ```

2. Import FTA results using Testbench Quality Assurance commands at Testbench Quality Assurance prompt:

   ```
   cer> package require CerFTAUtility
   cer> ::CerFTAUtility::importFTAResults
   ```

3. View FTA results

   ```
   cer> viewreport
   ```

Feedback

21.5.4 Merge FTA Database with Testbench Quality Assurance Database

FTA database can also be merged with Testbench Quality Assurance database from other FTA sessions or from simulation. This can be done in Testbench Quality Assurance.

```
cer> resultsmerge -dblist=[list <fullpath_certDB1> <fullpath_certDB2>]
-mergetodb=mergedDB
```

21.6 Mapping FTA Results to Coverage Database

FTA is very powerful at identifying verification holes, however for signoff, a consolidated coverage view is often required. This provides a holistic quantitative measurement in terms of coverage metric that contains coverage data from all verification tools, for example, simulation, formal or fault analysis. This section will describes how to map FTA analysis results to the same coverage metric used for simulation, property density, and formal core.

The steps to map FTA results to coverage metrics are:

1. Specify coverage options in Tcl file.

   If not already defined, modify the existing tcl file to add -cov <cov_metrics> option to the read_file/elaborate command, in addition to the fault types already there. For example,

   ```
   read_file -cov line+branch+tgl+fsm_trans+fsm_state -inject_fault all|
   RTL|Connectivity|none …
   ```

   Or

   ```
   elaborate -cov line+branch+tgl+fsm_trans+fsm_state -inject_fault all|
   RTL|Connectivity|none …
   ```

   If `-cov <cov_metrics>` is already defined previously, skip to step Issue the command to map FTA results to coverage

2. Repeat the run steps described in section Tcl setup for FTA for FPV sanity check and formal core results

3. Repeat run step to run FTA as described in Standard Use.

4. Save the FTA results to coverage database

   ```
   save_fta_cov_results [ -db_name <vdb_name> ]
   ```

   If no vdb_name is provided, the coverage database will be saved under fpv_fta.vdb

5. View results in Verdi COV

```
view_coverage
```

Note:

Current supported coverage metric option is line, branch, tgl, fsm_trans, and fsm_state.

21.7 Standard Use

FTA can be run in Tcl command mode, GUI mode, batch or regression mode following the step to invoke VC Formal with the Tcl setup file created in the above section Tcl setup for FTA.

```
vcf -f <your_fta_setup.tcl>
```

21.7.1 Running FTA using Tcl command

The Tcl command to run FTA from the Tcl shell interactively is

```
compute_fta -par_task <FPV_task>
```

By default, only the proven properties are used for the FTA analysis. In Configure FTA section above, we already described the selection of properties and classes of faults can be configured. There is another run command option to configure the section of properties by using -property option to change the default.

For example,

```
compute_fta -par_task FPV -property [get_props -status {proven
inconclusive}]
```

21.7.2 Running FTA using GUI

FTA can also be run from the GUI.

```
vcf -f <your_fta_setup.tcl> -verdi
```

Click to run FTA.

- The FPV_FTA task is added in the Task List.
- The results of the faults are updated live in the *Fault Summary* tab and the *Verification Targets* tab.

Feedback

21.7.3 Running FTA in regression or batch mode

```
vcf -f <your_fta_setup.tcl> -batch
```

For more information on batch or regression mode, refer to section VC Formal Regression Mode Accelerator.

21.7.4 Running FTA on compute farm

Compute farm can provide more resources therefore better chance of improved performance in terms of run time and convergence rate. Refer to section for information on how to configure grid workers, See Running on Compute Farm.

21.7.5 Setting up FTA for Multiple Tasks

Multiple tasks and parallel runs are also supported for FTA. This technique can be used to partition of the big problems to smaller ones therefore more resource can be devoted to the partitioned problems in separate tasks. See topics on Creating Verification Tasks

21.7.6 Save and Restore

FTA results can be saved the restored. See section Saving and Restoring Sessions.

21.7.7 Progress of run

FTA progress report is different from FPV App, where the task progress report functionality are the same, as described in theGetting Report of the Verification Tasks, there is no progress report of any individual fault property, as user will not be able to do the same as in FPV. For example, techniques often used in FPV such as analyze the complexity of the property to identify potential cause for performance bottleneck in order to apply advanced convergence techniques such as abstraction, using symbolic variable, ICM Iterative Convergence Methodology do not apply here. This is because FTA is a totally different situation, where all the properties are being evaluated against all the faults at the same time, therefore the progress report on an individual fault property will not make sense here.

21.7.8 Review Results and Report Generation

For FTA, the run results can have these statuses:

- non-activated
- detected

Feedback

- non-detected
- inconclusive

If FTA is performed on formal core logic only, one more status is added:

- non_formalcore (when formal core FTA is enabled)

Review Results and Report Generation Using Tcl Commands

Similar to all VC Formal report generation, FTA also uses `report_fv` command for reporting results. For example,

The `report_fv` command can be used to get a summary of the results:

```
vcf> report_fv
```

Summary Results

```
Property Summary: FPV_FTA
-----------------
> Constraint

- # found : 19

Fta Summary: FPV_FTA

------------
> Fault
- # found : 412
- # inconclusive : 89
- # non_activated: 33
- # detected : 289
- # non_detected : 1
```

To get detailed of the fault, the location of the RTL, mutated code:

```
vcf> report_fv -verbose <prop_id>
```

For example,

```
> Fta
# Fta: 1
> ID: [129] non_detected
- name : bridge.fault_id_1
- type : fault_check
- fault_type : InputPortStuckAt0
- fault_location: (50,18)(50,20)
- mutated_code : /* stuck at 0 */
- original_rtl : ---
- changed_rtl : ---
- fault_class : TopOutputsConnectivity
- usage : assert
- update_time : 23:16:47 - Jan 22
```

Feedback

```
- elapsed_time : 00:00:02
- converge_time : 00:00:00
- location : ../rtl/bridge.sv:50
- engine: b1
```

Review Results and Report Generation in GUI

FTA results are much more comprehensive in the GUI. The status is updated in the Fault Summary tab and the Verification Target tab.

- Fault Summary tab

  The Fault Summary tab reports the summary of the results of the nine fault classes.

  Figure 195 illustrates an example of the fault summary in the *Fault Summary* tab.

  The FTA run status of each fault class is displayed in the following columns:

  - Faults in Design
  - Faults in Report
  - Non-Activated
  - Detected
  - Non-Detected
  - Disabled by User
  - Not Yet Qualified

Figure 195: Fault Summary Tab

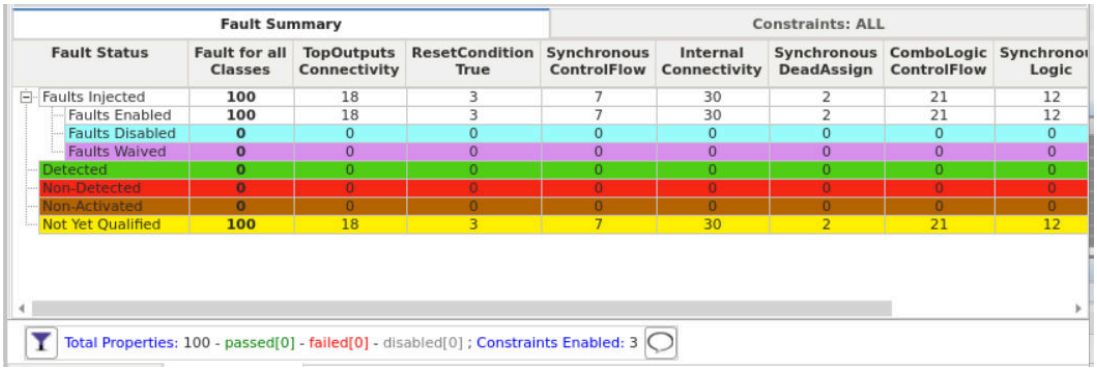


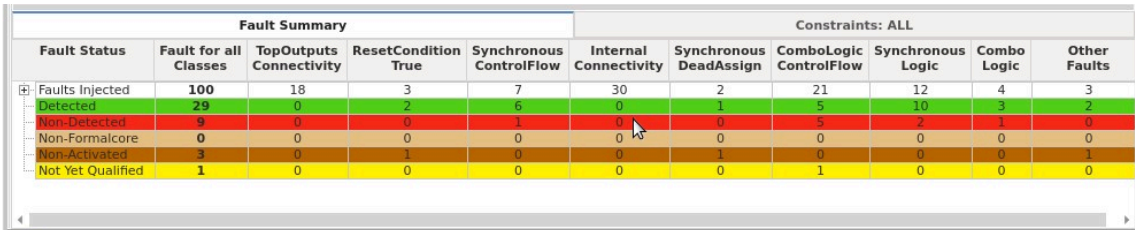
Figure 196 illustrates an example of the fault summary with formal core FTA enabled.

Feedback

The FTA run status of each fault class with formal core enabled is displayed in the following columns:

- Faults in Design
- Faults in Report
- Non-Activated
- Detected
- Non-Detected
- Non-FormalCore
- Disabled by User
- Not Yet Qualified

Figure 196: Fault Summary with Formal Core FTA

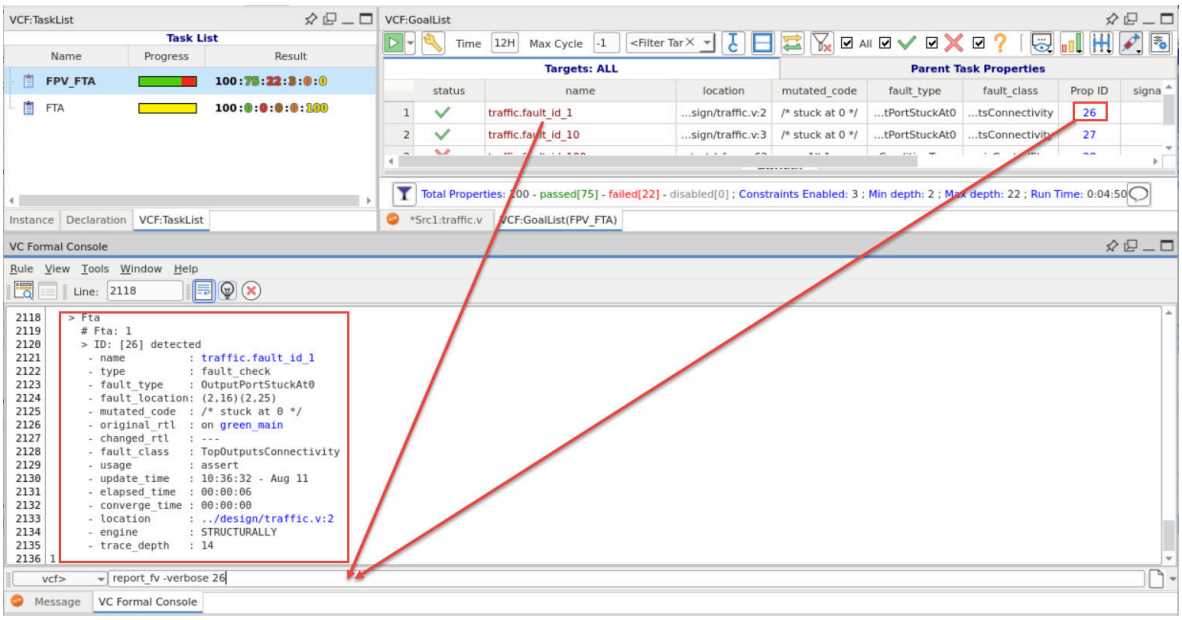


Fault Summary — Constraints: ALL

Fault Status	Fault for all Classes	TopOutputs Connectivity	ResetCondition True	Synchronous ControlFlow	Internal Connectivity	Synchronous DeadAssign	ComboLogic ControlFlow	Synchronous Logic	Combo Logic	Other Faults
Faults Injected	100	18	3	7	30	2	21	12	4	3
Detected	29	0	2	6	0	1	5	10	3	2
Non-Detected	9	0	0	1	0	0	5	2	1	0
Non-Formalcore	0	0	0	0	0	0	0	0	0	0
Non-Activated	3	0	1	0	0	1	0	0	0	1
Not Yet Qualified	1	0	0	0	0	0	1	0	0	0

Figure 197 shows a relationship of the FPV properties and FTA fault status.

Figure 197: FPV Property and FTA Fault Status Relationship

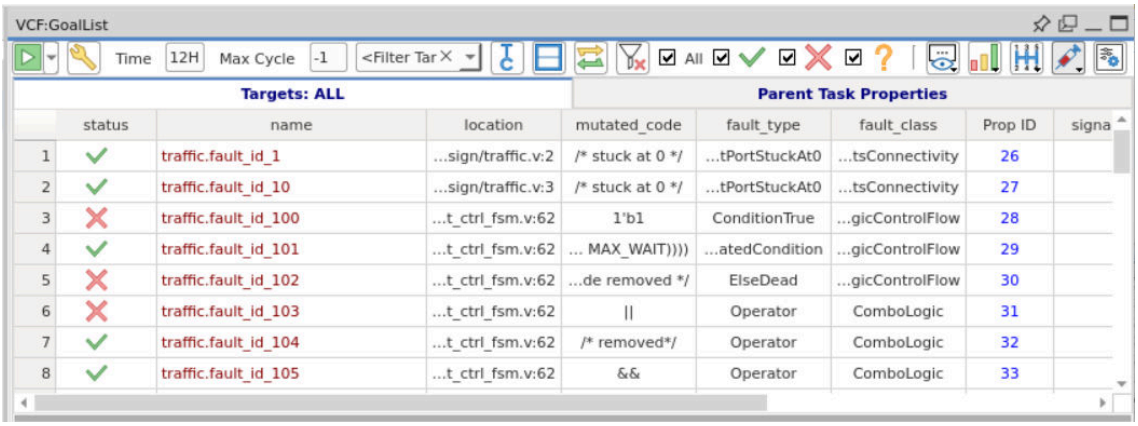


Feedback

- Verification Targets Tab

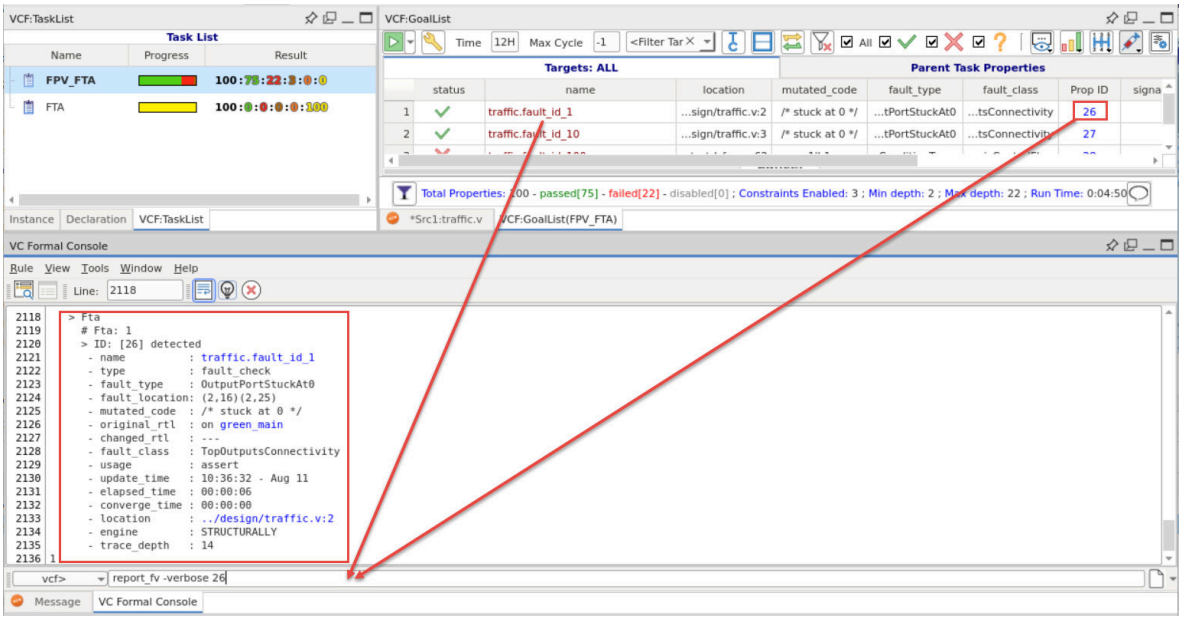
  The Verification Targets tab displays all the faults injected and their details such as the status, fault_class, fault_type, and so on.

Figure 198: Verification Target Tab



As the run progresses, the fault status field in both the *Verification Targets* tab and *Fault Summary* tab are updated.

Figure 199: Fault Detail Report using Tcl Command



- You can perform the following on the FPV properties table.
  - Click on the column header to filter and sort the FPV properties in the table.
  - Double-click the activated fault number to open the fault properties that are activated by this FPV property in the Verification Targets tab.
  - Double-click the name column to open the source for the FPV property.

21.7.9 Debug in FTA GUI

Debugging is made easier with the GUI through links between the faults in the Fault Summary tab, Verification Target tab, RTL code with mutation annotation in Testbench Quality Assurance Source Code View, and the FPV Properties tab.

Basic Debug Steps

The main debug focus should be on reviewing the details of the non-detected faults. There are links in the GUI that allows you to start from the Fault Summary, follow into the Fault Targets, then to the Testbench Quality Assurance Source Code View.

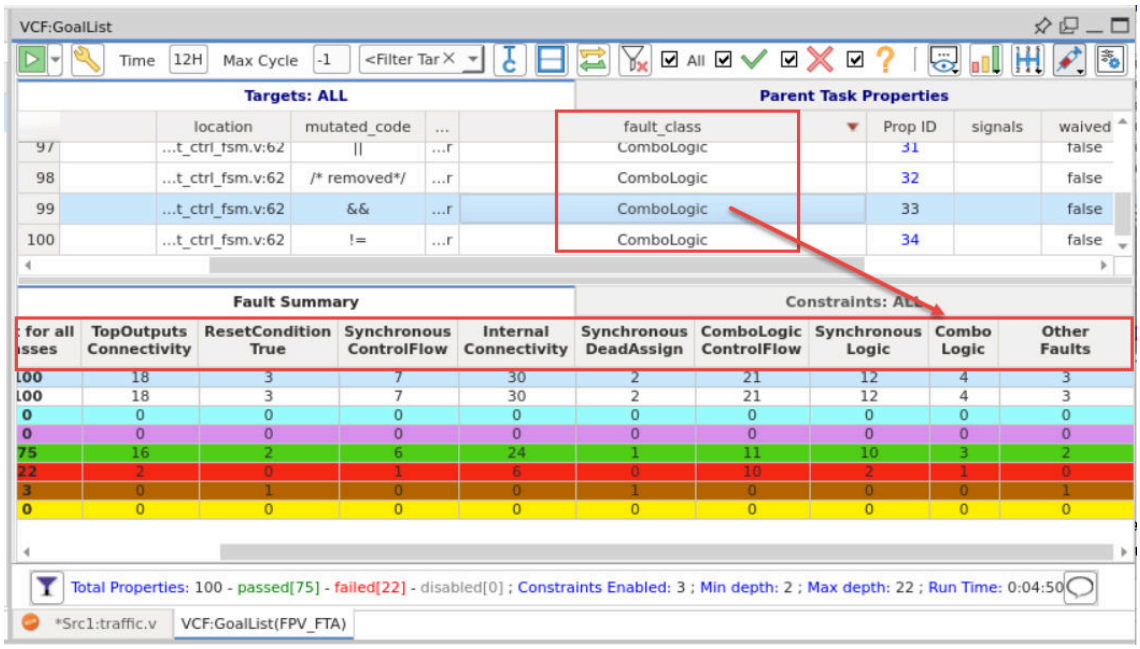
1. Link to filter by fault class

   Double-click the Class Name in the Fault Summary tab to show the faults in the Verification Targets tab based on the fault class name.

Feedback

For example, as shown in the following figure, when you double-click ComboLogic in the Class Name column, only the details of the ComboLogic fault class appear in the Verification Targets tab.

Figure 200: Filter by Fault Class



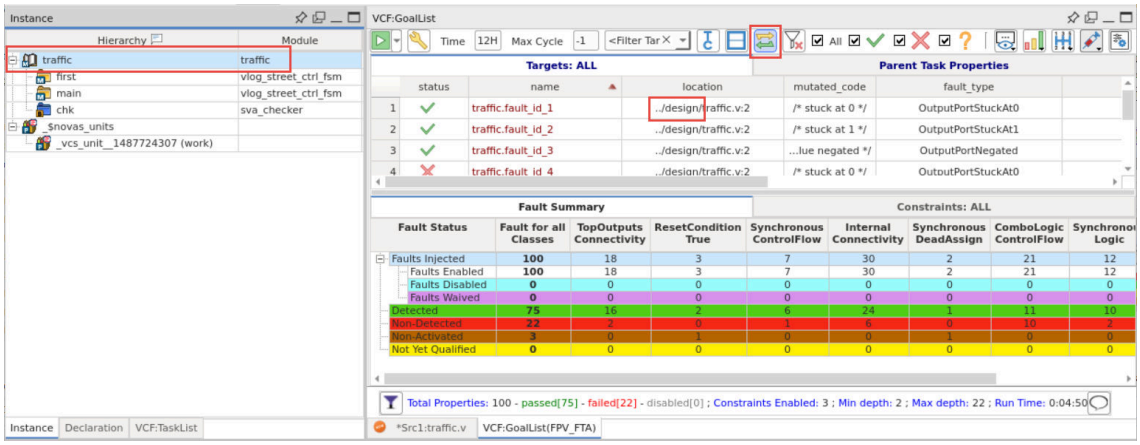
1. View the *Fault Summary* based on the instance tree.

   You can analyzes the results of *Fault/Non-Activated/Detected/Non-Detected/Not Yet Qualifed* status based on instance hierarchy at a glance. To enable this feature,

   1) Set the `set_fml_var fml_fta_inst_qual true` application variable before `read_file`/`elaborate`.

   2) Click *Sync Scope* icon the top right of the *GoalList* window.

Feedback

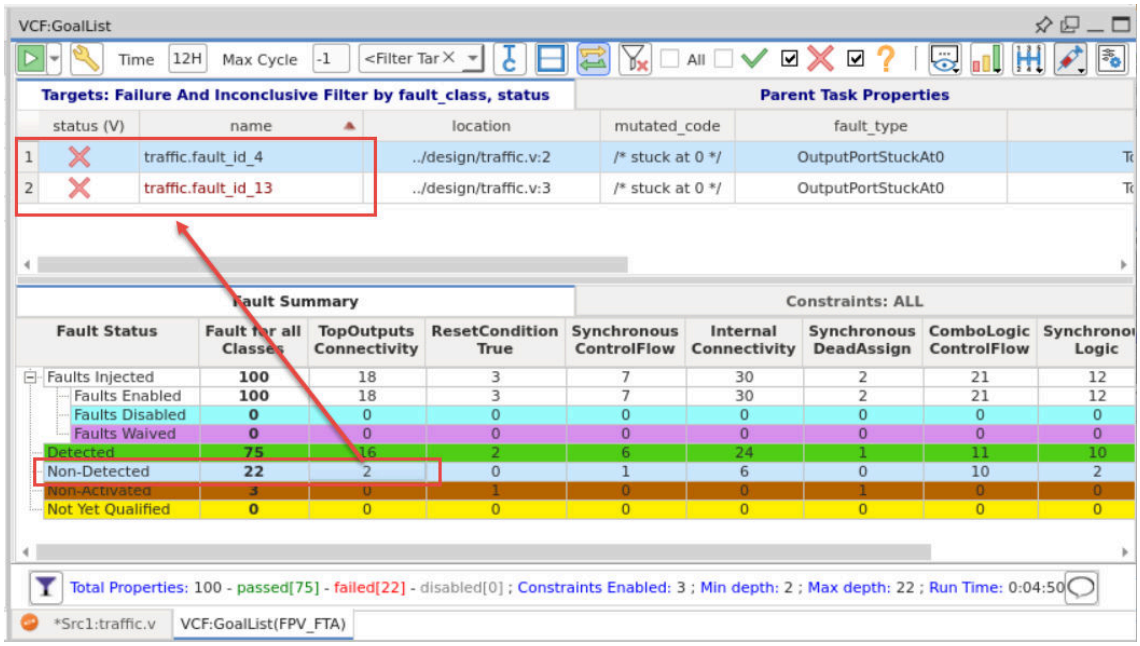


2. Link to filter by fault class and status

   Double-click a number in the *Fault Summary* tab to show the faults in the *Verification Targets* tab based on the fault class and status.

   For example, as shown in the following figure, when you double-click 2 in the TopOutputsConnectivity column in the Non-Detected row, only the details of the 2 selected faults show up in the Verification Targets tab.

Figure 201: Filter by Fault Class And Status



Feedback

1. Link to Testbench Quality Assurance fault source

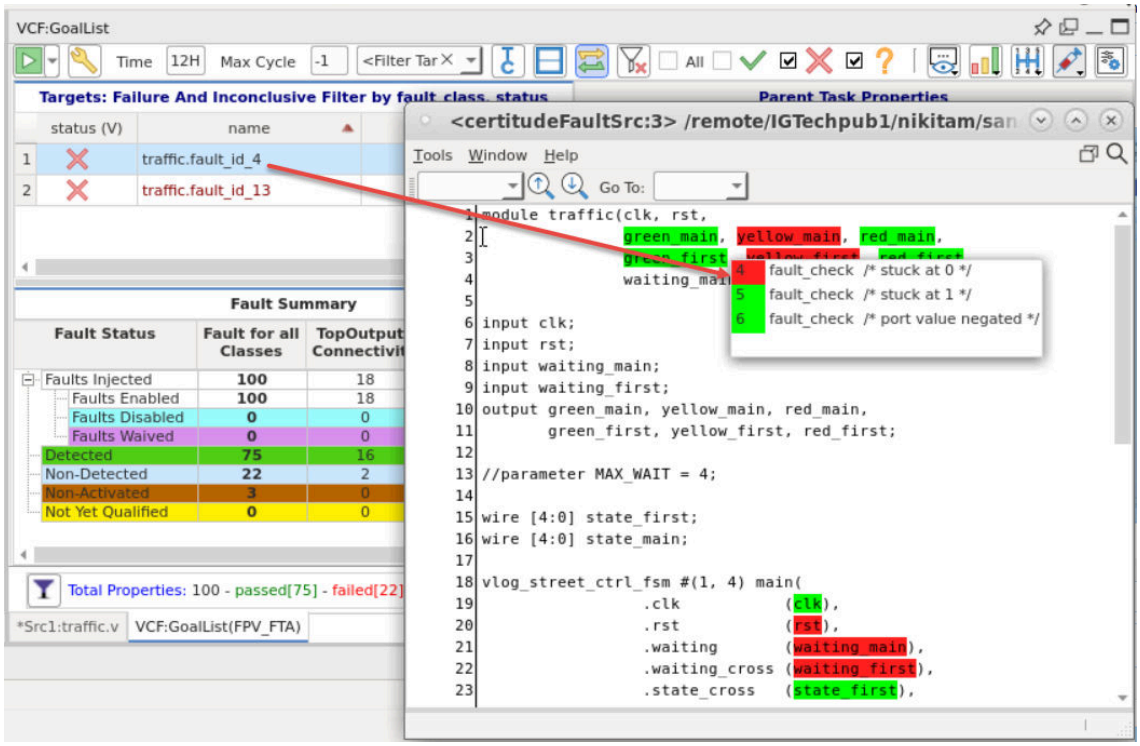
   Double-click the Name in the Verification Targets tab to show the selected fault effect on the RTL code in the FaultInSrc.

   For example, as shown in the following figure, when you double-click on the Name in the Verification Targets tab, the FaultInSrc tab will open. The fault signals are highlighted with a different background color. The background color is based on the FTA run status of the faults and is the same as in the Fault Summary view.

   If there are multiple fault properties linked to the same signal, the status' color of the highest severity is used. For example, if there are two faults, one non-detected and the other detected, the background color of the non-detected (that is, red) severity is used.

   Hover the mouse over the fault signal to view the details of the fault injected on this signal. The tool tip as shown in Figure 202 displays the FTA fault id (shown in the name column in the Verification Targets tab) with the background color indicating its status, the fault type string, and the fault status.

Figure 202: Fault in Source View



By reviewing the fault, you should start thinking why none of the properties in the verification environment can fail in order to catch this fault.

Feedback

1. Show property in relation to faults

   To see what properties are related to a fault, a Tcl command can be used to do the quarry.

   ```
   get_activated_props <fault_id>
   ```

   This will output a collection of properties whose COI included this fault. This will be useful to debug the non-detected faults. It is possible the next step is to either modify the list of properties related to this fault or to create new properties.

Figure 203: FPV Property to FTA Fault Relationship

1. Link to the list of faults in relation to certain constraints

   When non-detected faults are found, you can determine whether those faults are because of certain constraints. To enable this support, set the following formal variable to true

   ```
   set_fml_var fml_fta_constraint_check false
   ```

   Use the `report_fta_constraints` command to view the constraints with respect to fault property, present in the COI.

   Example

   ```
   vcf> report_fta_constraints
   Fault Property: traffic.fault_id_1, Constraints:
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_10, Constraints:
    traffic.chk.assume_continuous_waiting_first
   Fault Property: traffic.fault_id_11, Constraints:
    traffic.chk.assume_continuous_waiting_first
   Fault Property: traffic.fault_id_12, Constraints:
    traffic.chk.assume_continuous_waiting_first
   Fault Property: traffic.fault_id_19, Constraints:
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_2, Constraints:
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_20, Constraints:
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_21, Constraints:
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_22, Constraints:
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main
   Fault Property: traffic.fault_id_23, Constraints:
    traffic.chk.assume_continuous_waiting_first
    traffic.chk.assume_continuous_waiting_main
   ```

Feedback

```
Fault Property: traffic.fault_id_24, Constraints:
 traffic.chk.assume_continuous_waiting_first
 traffic.chk.assume_continuous_waiting_main
Fault Property: traffic.fault_id_25, Constraints:
 traffic.chk.assume_continuous_waiting_first
 traffic.chk.assume_continuous_waiting_main
```

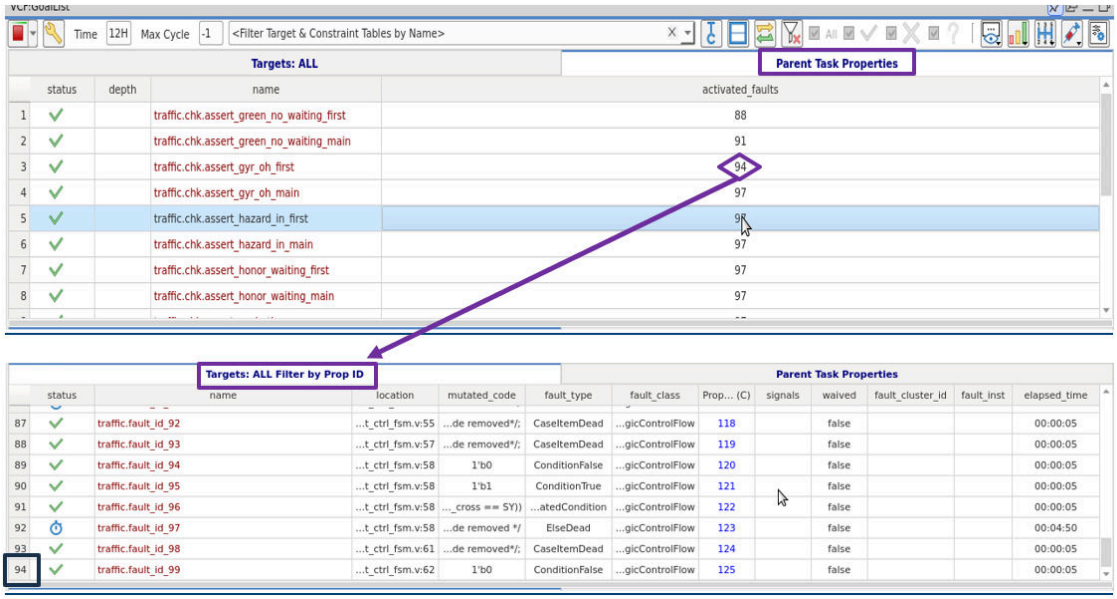
Other Features in GUI for Debug

There are other features in the GUI that can provide useful debug information.

1. Link to faults in relation to the COI of a property

   Given any FPV property, it is useful to find out how it is related to the faults in the FTA. From the **FPV Task Properties** tab, the number of activated faults is listed by every property. Double-click on the number and it will show the list of faults which are in the COI of the selected property in the **Targets** tab.

Figure 204: FPV Property to FTA Fault Relationship



1. Using RMB on the fault targets

   Functions such as enable, disable, clear status, etc. are available by using the RMB as shown below.

Figure 205: RMB Menu For Fault Properties

21.8 Performance and Convergence

When there are non-converging fault properties in your run, there are techniques you can try to improve the performance.

21.8.1 Fast Activation Analysis

To enable fast fault activation analysis (the step to find out non_activated faults), the following formal variable can be set:

```
set_fml_var fml_fta_fast_activate true -global
```

There are techniques mentioned in other sections that apply to improving performance and convergence. For more information, see section Performing and Configuring VC Formal Checks.

21.8.2 Partition the Runs

In section Creating Verification Tasks, there some techniques specific to the FTA you can apply. When there are large number of faults, you can partition the runs with separate tasks. Put focus on the more important faults first.

21.8.3 Improve Fault Check Performance

To improve FTA fault detection checking performance, the following formal variable can be set after the first run of FTA to improve performance for the future runs:

```
set_fml_var fml_fta_high_effort true
```

Default value is set to false.

21.8.4 Viewing Convergence Report

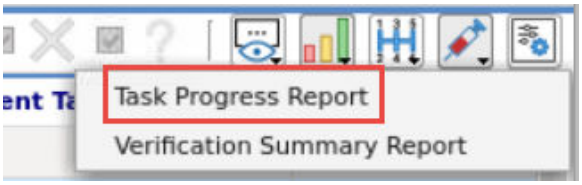
The Formal Testbench Analyzer (FTA) application is used during signoff of Formal verification to find out if enough assertions have been written covering complete design functionality. The tool injects artificial bugs called faults in the design and uses formal verification to find out if the formal testbench (assertions) can catch those bugs. Faults

Feedback

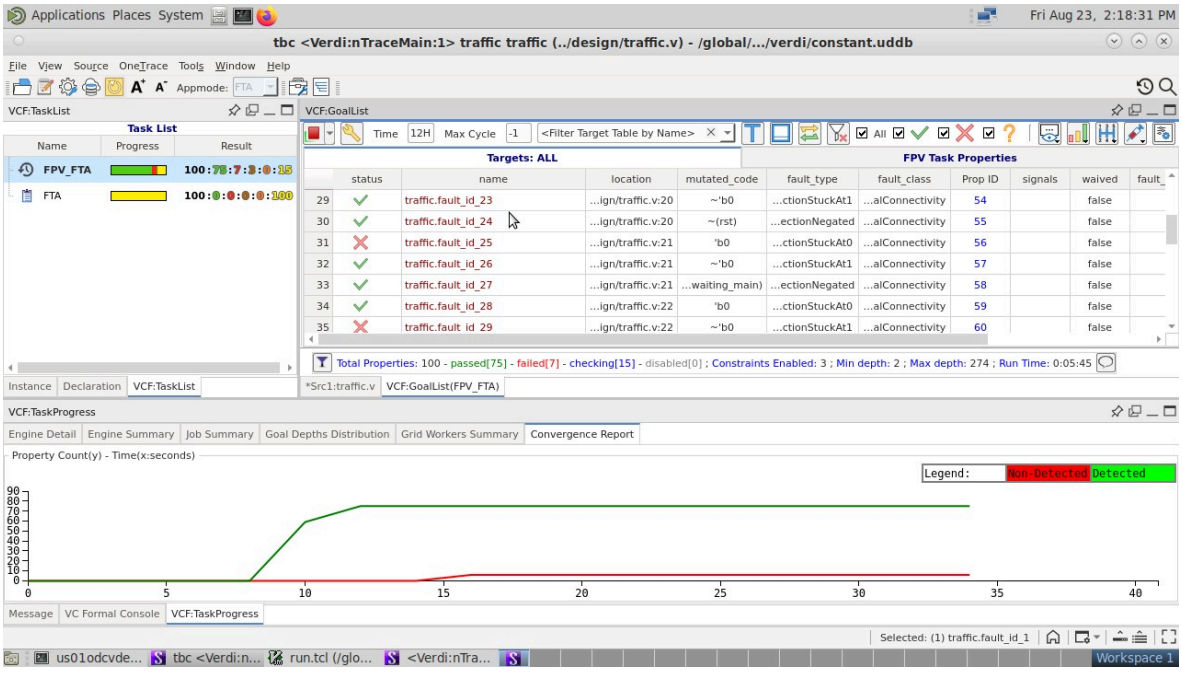
which are found to be outside the fan in of user assertions are reported as Non-Activated, those which are within the fan in and which can falsify the user assertions are reported as Detected and the remaining ones which can't falsify the user assertions are reported as Non-Detected.

From the verification Engineer, point of view, they need to closely see the non-detected faults and take appropriate action to plug in the verification holes. But sometimes the number of non-detected faults may be very high. And sometimes it can take significant time for the run to converge. However generally ~98% of the non-detected faults are reported during the first 1-2 hrs. You can start reviewing the non-detected faults as soon as they start com ing rather than waiting for the entire run to converge. Starting with this release, the convergence report features is introduced. This report shows a convergence graph for the non-detected faults, so that you can easily understand how the non-detected faults are reported from the tool side.

You can view the convergence graph from the GoalList tool bar as shown in the following figure:



The convergence graph appears as illustrated in the following figure which shows the total converged faults with time.



Feedback

21.9 Advanced Techniques

This section describes the advanced techniques to debug non-detected faults, how to cluster faults, including how to check for whether non-detected faults can propagate to any outputs, and debug the non-detected faults.

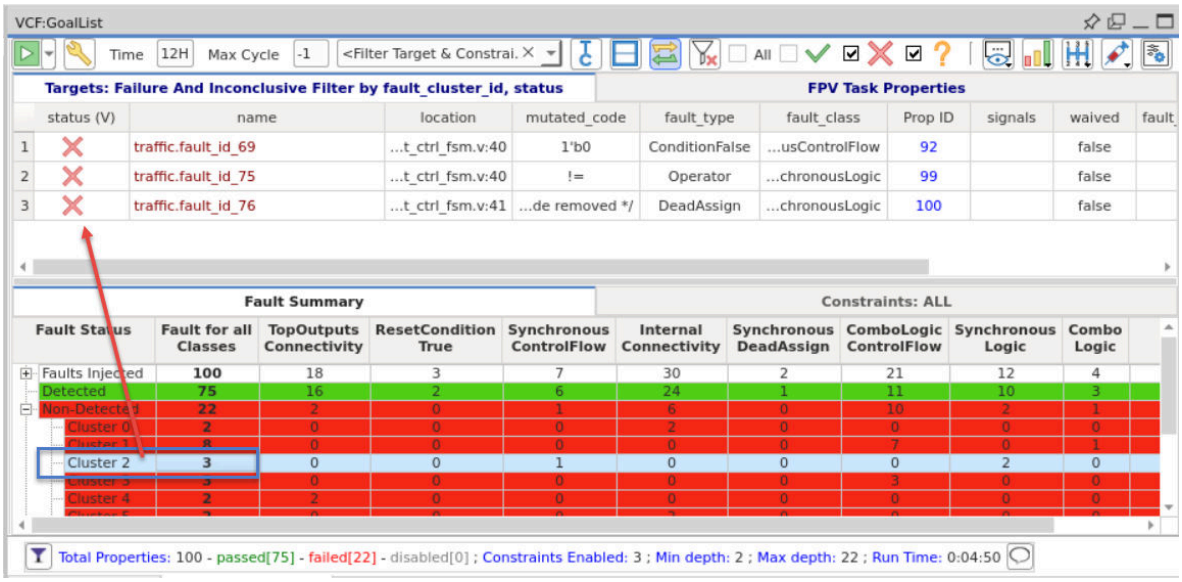
21.9.1 Cluster Non-detected Faults to Reduce Debug Time

When the number of non-detected faults is very high, you can group similar non-detected faults into clusters. As the faults in a cluster are similar, addressing any one of the faults in a cluster may have a high probability of addressing all other faults in that cluster. Instead of looking through a large number of non-detected faults, you can concentrate on each cluster whose number is less than the initial number of faults.

To create clusters of non-detected faults, use the `cluster_fta_faults` command.

To view the clusters in the report, use the `report_fta_fault_clusters` command.

You can view the clustering results from the *Fault Summary* tab in the property table.



21.9.2 Automated Debugging of Non-detected Faults

The main advantage of FTA is finding faults that are not detected by the existing set of user assertions. These non-detected faults point to a weakness in the Formal testbench, that is, to the missing properties.

The VC Formal FTA application provides `debug_fta` command to debug non-detected faults by analyzing if those faults can propagate to any of the primary outputs. The

Feedback

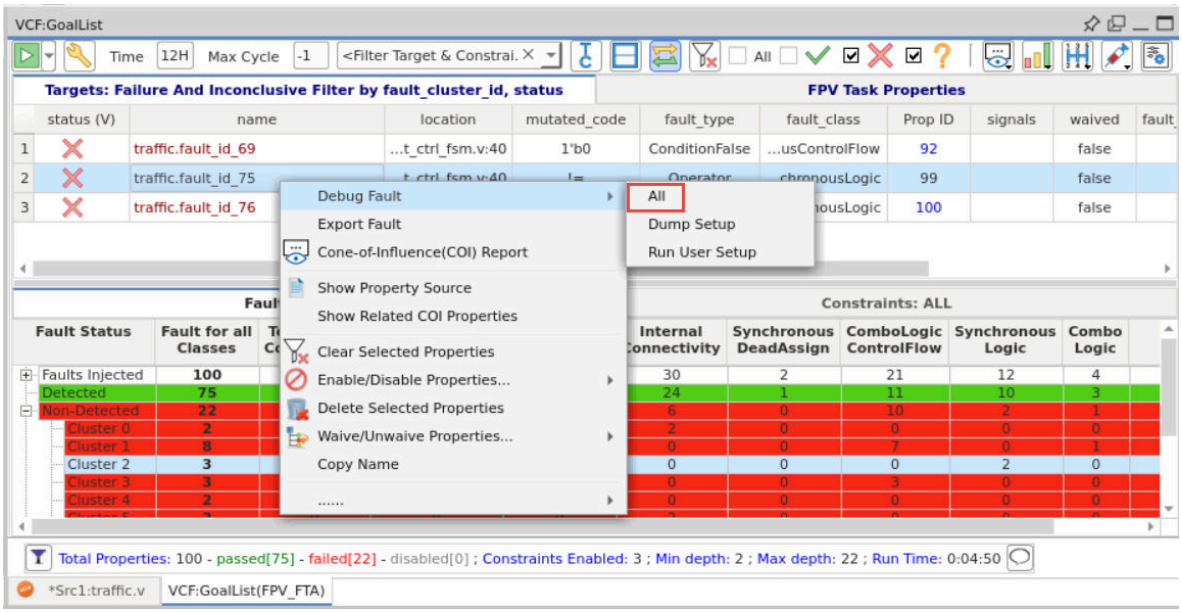
`debug_fta` command leverages the SEQ application to compare the actual design against a version of the design instrumented with the targeted faults. The user should prioritize the faults that can propagate to design outputs, and add adequate properties to detect them.

Use Model

By default, VC Formal automatically extracts several settings from FPV_FTA environment and creates a template tcl file (*fta_ndf.tcl*) and runs it when the `debug_fta` command executed.

```
debug_fta [-property propName]
```

From the GUI, once the FTA fault analysis completes, right-click on the target non-detected fault(s), and select *Debug Fault > All.*



Editing the Settings in the fta_ndf.tcl Template

If you want to modify any settings in the *fta_ndf.tcl* template:

1. Run the `debug_fta -dump_setup` command which saves the *fta_ndf.tcl*template.

   Or

   Right-click on the target non-detected fault(s), and select *Debug Fault > Dump Setup* to save the template from the GUI.

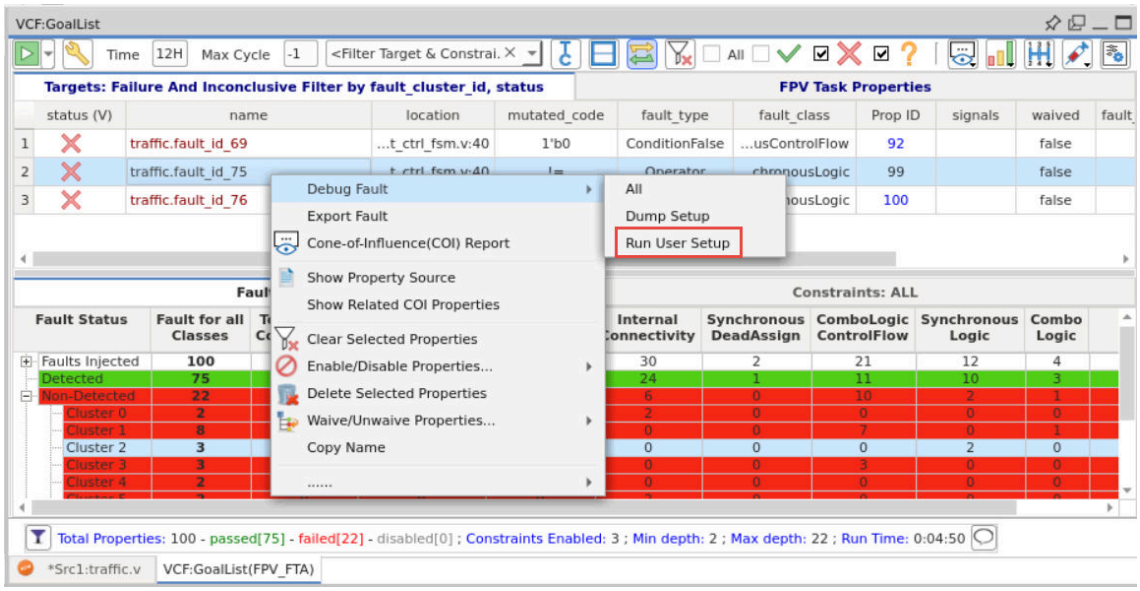
2. Modify the generated *fta_ndf.tcl* file following the instructions described in the file template.

Feedback

3. Run the `debug_fta -run_user_setup` command. This invokes the `debug_fta` flow with SEQ using the modified *fta_ndf.tcl*and *./run_ndf_propagation* present in the working directory.

   Or

   Right-click on the target non-detected fault(s), and select *Debug Fault* > *Run user Setup* to trigger the `debug_fta` command from the GUI.



Notes:

- The `debug_fta` command automatically opens a VC Formal SEQ application session in Verdi mode for the analysis of non-detected faults, thus requiring the necessary tool licenses and display settings. This also applies to the case where the FTA application was opened in batch or non-Verdi modes.
- When FTA-SEQ is run, VC Formal creates a separate *certitude_config.cer* file in the current working directory, and the *certitude_config.cer* file from FPV-FTA run is moved to the *fpv_certitude_files* directory. So, if you want to go back to FPV environment, and want to rerun FPV-FTA again, then the *certitude_config.cer* from *fpv_certitude_files* directory should be copied to the current working directory.
- When using the `debug_fta` command, the reset initialization of the design in the VC Formal SEQ application is done without enabling the targeted faults. This behavior allows for multiple faults to be debugged concurrently, while the reset state might not

Feedback

reflect the impact of those faults. This can also lead to faults remaining not detected as they only effect the reset phase of the design.

- During the analysis phase, the reset conditions declared with the `create_reset` command are disabled in a way consistent with the VC Formal FTA flow. This behavior can lead to some faults not being detected during the analysis phase.

21.9.3 Checking for Non-detected Fault Propagation

1. Exporting non-detected fault properties to parent FPV task

   You can export the non-detected fault properties to the parent FPV task. Exporting non-detected fault properties can be helpful for debugging using the witness trace in the FPV application mode. You can use the navigator feature in the FPV application mode to see what needs to be done to make the fault detected.

   The `export_fault` command enables you to add specific faults and user properties to a parent FPV task.

Syntax

```
vcf> export_fault -par_task <FPV> -property <fault_property_name>
```

1. Where,

   - `-par_task parTaskName`: Specify the name of the parent task. If it is not specified, the fault is enabled in the parent task on which FTA analysis was performed.
   - `-property propName`: Specify the name of the fault property that must be exported.

2. Check FTA Non-Detect Faults for Propagation

   When non-detected faults are found, we need a way to identify whether these faults are important. There is an advanced technique that can help identify whether a non-detected fault is observable at the RTL output, and affect the RTL behavior in any way.

   If it cannot propagate to the output, then this fault is not important, and can be ignored or waived. You can shift your focus to other non-detected faults.

   If it can propagate to the output, you will get a waveform to see how this fault can affect the output behavior compared to the original RTL code without the fault injected.

   Use model:

   a. Generate the waiver file for faults not observable at the outputs by using the `export_non_obs_fault` command. The `export_non_obs_fault`command exports the non-observable faults for incorporating in parent run. This command generates

Feedback

a Tcl file *ndf_no_disable.tcl* which can be imported in FPV FTA session to disable the non observable faults.

b. For the faults that can propagate to the outputs, you must export the fault and check for propagation using the following command:

```
export_fault -par_task <SEQ> -property <fault_prop_name>
```

You can export any type of fault (detected or non detected). For detected, it can be used to find which parent property gets falsified. For non-detected fault, it can be used to debug witness trace.

Note:

You can export faults for the FPV and SEQ modes. The use model for both the FPV and SEQ modes are the same.

c. Run the propagation of the fault check: check_fv

d. Use the waveform to see different behavior of the same output on regular RTL, and one of the exported faults. This may provide information to see what assertions must be added to catch this fault.

21.10 Use Cases

There is more information about the FTA and formal verification in this section Signoff Obtaining Formal Signoff For a Design.

For more information on how to use the App, you can copy the example testcase in the tool release area $VC_STATIC_HOME/doc/vcst/examples/FTA. Follow the instruction there to get familiar with the setup and the flow.

21.11 Application Variables and Formal Variables for FTA

The table below shows the list of application variables and formal variables for FTA and their default value.

Table 39: Application Variables and Formal Variables for FTA

Type	Name	Default
app_var	fml_multi_step_fta_flow	false
fml_var	fml_fta_fast_activate	false
fml_var	fml_qual_fault_in_fcore	false
fml_var	fml_fta_enable_cover	false

Feedback

Table 39: Application Variables and Formal Variables for FTA (Continued)

Type	Name	Default
fml_var	fml_fta_par_app_high_effort	false
fml_var	fml_fta_enable_vacuity_check	false
fml_var	fml_fta_seq_debug	false

For more detailed descriptions see Appendix Appendix E - Application Variables and Appendix F - Formal Variables

21.12 Backward Compatibility

The current mode of integration of Testbench Quality Assurance with VC Formal or any other formal tool continues to exist when you invoke Formal from the Testbench Quality Assurance prompt.

22

VC Formal Security Verification Application

The VC Formal FSV application helps formally verify that secure data should not reach non-secure destinations and ensures data integrity, where non-secure data should not over-write (or reach) secure destinations.

Getting Started • *What is VC Formal FSV?* • *Accessing FSV Lab Example*	**Debugging, Reviewing and Saving Results** • *Generating Verification Summary* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Finding Root Cause Signal for Failing Assertions* • *Saving and Restoring Sessions*
Setting Up and Running FSV • *Enabling FSV Application mode* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Verifying Design Setup* • *Creating Security Properties* • *Reviewing and Editing Properties* • *Running FSV in TCL and GUI*	**Getting Help on Commands** • *Setting FSV Specific Application Variables* • *FSV Tcl Commands*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	

22.1 Introduction FSV Application

Hardware security requires that some data is protected and cannot be read or modified unless some condition is true. This is facilitated by SoC's having secure / trusted modules and operating modes. Access control for secure data range from no access outside the module to read and write access when the requesting module is operating in secure mode to no access control. Reading or writing secure areas on chip obeying security protocols

Feedback

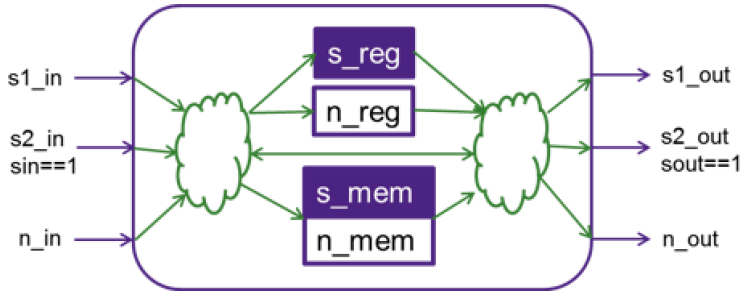
can be verified using simulation or Formal Property Verification but how do you verify that there is no access in all other cases? These unexpected paths to accessing secure data may be caused by hardware bugs or architectural oversights. If we have secure and non-secure areas in an SoC then FSV ensures that unexpected data propagation does not happen from secure to non-secure areas and vice versa. These two scenarios are referred to as data leakage and data integrity.

- Data Leakage: secure data must not reach or influence non-secure areas of the design
- Data Integrity: non-secure data must not reach or influence secure areas of the design

FSV performs the data propagation analysis between a source and a destination. The data propagation path may be direct or have delay. The source and destinations can be inputs, outputs, nets, registers, memories or instances.

In the following example, all ports, registers, and memories whose name starts with s_ refer to secure areas and those starting with n_ refers to non-secure areas of the design.

Figure 206: Example of Secure and Non Secure Areas of the Design



For example, the data propagation analysis in FSV can verify that the non-secure input n_in cannot affect the secure register s_reg. And, that data from the secure address range of memory s_mem cannot affect the non-secure output n_out.

The user specifies data flow that are not allowed using a special property format. These security properties automatically generates constraints and assertions which are subsequently proven in FSV.

You enable the FSV application using the following application variable in Tcl:

```
set_fml_appmode FSV
```

You can also select the application mode FSV from the Appmode list in the GUI, as shown in the following figure:

Figure 207: Changing the Appmode to FSV

Note:

All the VC Formal applications require specific license keys. For more information on the license keys required for each of the VC Formal applications, see the *VC Static Platform Installation Notes*.

This video introduces the Formal Security Verification (FSV) application in VC Formal, focusing on verifying secure data integrity and preventing illegal data propagation. FSV ensures that confidential data does not reach non-secure destinations or get overwritten. It achieves this by defining sources (secure or non-secure data) and destinations (secure or non-secure locations) as registers, memory, nets, or ports.

22.1.1 Video Links

Click the links to view videos on FSV:

- FSV Introduction (4 mins)
- Blackboxing in FSV (8 mins)
- Property Progress Analysis in FSV (11 mins)

22.2 FSV Security Verification Methodology

The required inputs to running formal security verification are:

- RTL Design
- Security Properties
- Clock and Reset Specification
- Constraints

The following diagram shows the required steps to verify security properties in FSV.

Figure 208: FSV Flow

22.2.1 Reading Design

The RTL design is compiled using the `read_file` or `analyze` and `elaborate` commands. For more details on how to compile designs, see section Compiling Designs.

Security properties in FSV are not specified in SVA, therefore, the "-sva" option of read_file, and elaborate is normally not used. If modules or instances are to be structurally blackboxed using the `set_blackbox` command, they need to be specified before compiling the design. Use the `report_link` command after reading the design to ensure no modules are missing.

22.2.2 Setting up the Verification Environment

To ensure correct verification results, the verification environment in which the design under test operates must be specified. The setup is the same as for other VC Formal Apps and includes:

- Set up user defined clocks using the create_clock command. See section Setting Up User-Defined Clocks.
- Set up user defined resets using the create_reset command. See section Setting Up User-defined Resets.
- Create and save reset state.
- Specify constraints.

  Constraints may be needed to avoid false data propagation paths, to enable or disable clock gating or verifying the design in test or functional mode. Normally, there are fewer constraints compared to Formal Property Verification (FPV) since data propagation analysis should take into account illegal or unexpected propagation or control paths.

22.2.3 Validating Setup

Correct clock and reset signal definitions are required to build a correct formal model for verification. Missing clock and reset signals may lead to incorrect property proofs or failures. Use the check_fv_setup and report_fv_setup commands to validate the clock and reset signals in the design and to find other design issues such as oscillating combinatorial loops. See sectionVerifying VC Formal Designs.

22.2.4 Create Security Properties

Security properties that specify data flows that are not allowed are defined using the `fsv_generate` command. See section Specifying Security Properties.

22.2.5 Proof Setup

Before proving properties, additional setup may be required. This includes setting up grid, setting proof effort and specifying secure blackboxes. See section Black Boxing and Security Verification.

22.2.6 Proving Properties

After assertions are created using the `fsv_generate` command, they are proven by running the `check_fv` command. This command is common to all VC Formal applications, and runs all enabled assertions in the active application.

22.2.7 Performance Considerations

If assertions are not proven or falsified after running for a long time, it is necessary to review progress of assertions and optimize the setup for performance. See section Advanced Techniques.

22.2.8 Debugging

The last step in the FSV security verification flow is debugging falsified properties. See section Debugging the Failing Security Properties.

22.2.9 Signoff Review

When all properties are proven the setup and commands needs to be reviewed to ensure the proofs are valid. See section Signoff Review.

22.3 Specifying Security Properties

You use the `fsv_generate` command to define the properties that you want to verify in FSV. These properties verify that data cannot propagate from a source to a destination. Specifying the source and destination of secure properties in the `fsv_generate` command is mandatory, all of the other options are optional.

Each `fsv_generate` command creates a group of properties that are independent from properties generated from other `fsv_generate` commands, unless the same zone name is used. They are verified together with the same constraints. You can specify many sources and destinations in the `fsv_generate` command, and one property for each destination is generated.

Syntax

```
vcf> fsv_generate -src <signal-list> [-src_condition <expression>] -dest
 <signal-list> [-dest_condition <expression>] [-exclude <signal-list>]
 [-name <name>] [-zone <string>][-condition_cover]
```

Feedback

Where,

- `src <signal-list>`: Specify one or more sources for no data propagation check. Source can be input, instance output, instance, register or net.
- `src_condition <expression>`: The source propagation starts once the src_condition expression is true.
- `dest <signal-list>`: Specify one or more destinations for no data propagation check. Destination can be output, instance input, instance, register or net.
- `dest_condition <expression>`: Checks if source can propagate to destination when the dest_condition expression is true.
- `exclude <signal-list>`: Do not consider propagation through any of the specified inputs, nets, registers or instances.
- `name <name>`: Specify the name for the `fsv_generate` property. <name> mentioned will be the prefix to the name of the properties and constraints.
- `zone <string>`: Specify a name to group the generated properties and constraints under a specific category. Default zone name is the same as the name specified with the -name option. All constraints with the same zone will be considered as sources for all assertions in the zone.

When specifying multiple instance, port, net or register names to `fsv_generate` options, enclose list in { } and separate items with whitespace. For example

```
-exclude { top.A top.B core.C }
```

Clock and reset signals are required for correct formal analysis and they cannot be used as sources in `fsv_generate` properties. Signals declared with `create_clock` or `create_reset` commands are automatically excluded from generated assertions if they are included in the signal list for source and destination in the `fsv_generate` command, instead a warning is printed:

[Warning] MARK_IGNORE_SIGNAL: The mark over clk will be ignored. It is not allowed to mark a clock signal

[Warning] MARK_IGNORE_SIGNAL: The mark over rst_n will be ignored. It is not allowed to mark a reset signal

22.3.1 Excluding legal paths from consideration in Security Properties

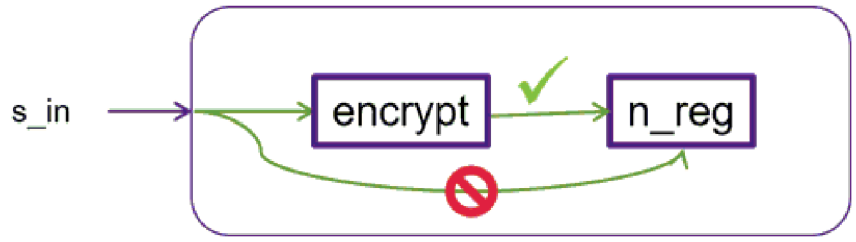
Legal propagation paths may exist in the design under test which cause security properties to fail. For example, the data on s_in may overwrite the value in n_reg but only

Feedback

if it goes through the encryption block in the picture below. This means that the following security property will fail:

```
fsv_generate -src s_in -dest n_reg
```

However this is the intended behavior of the design so the property failure is not valid. To ensure the data on s_in cannot bypass the encryption block and still overwrite the n_reg register we need to exclude the legal path using the -exclude option to fsv_generate.



We can exclude propagation through an input, net, register or instance from consideration. The following property will fail if a path exist from s_in to n_reg that doesn't go through the encrypt block.

```
fsv_generate -src s_in -dest n_reg -exclude {encrypt}
```

The property will only be proven if no path exist between s_in and n_reg or if the only path goes through the encrypt block.

Using the `-exclude` option will have the same effect on the property as black boxing the encrypt instance or cutting the input net to the encrypt instance. The difference is that -exclude only affects the assertions created by the `fsv_generate` command where as `blackboxing` and cutting the net affects all properties.

22.3.2 Specifying Source and Destination Conditions

The source and destination conditions (`-src_condition <expression>` and `-dest_condition <expression>`) in the `fsv_generate` command perform as constraints for the properties generated by the `fsv_generate` command, but they do not behave like other constraints.

- *Source Condition*: The source signal specified in the `-src <signal-list>` switch starts its propagation when the source condition expression is set to `true`. The source condition expression has to be true for the cycle when the propagation starts. It does not have to be true in any other cycle. The source condition is used when the source signal is only secure or non-secure under some conditions.

Feedback

Note:

The same signal cannot be used both as a source and a source condition in the same or in different fsv_generate properties.

The source and destination conditions only apply to the `fsv_generate` command where they are specified, not to others with the same zone name.

- *Destination Condition*: The destination signal checks for the propagation of the source when the condition expression specified with the `-dest_condition <expression>` switch is `true`. The condition must be true for the cycle when the check is performed. It does not have to be true in any other cycle. The destination condition is used when the destination is valid under some conditions.

  The source and destination conditions affect the assert and mark properties generated by the `fsv_generate` command.

- *Constraints versus Source and Destination Conditions*: Constraints (`fvassume` and `set_constant`) are not equivalent to `fsv_generate` source and destination conditions. Source and destination condition expressions do not have to hold always or simultaneously. `fvassume` and `set_constant` constraints hold all the time and they apply to all properties equally. Source and destination conditions only apply to the properties generated by one `fsv_generate` command.

In some cases, it might be valid to change a constraint to a source condition or a source condition to a constraint, but it might lead to incorrect results. If a constraint is changed to a source condition, it might be an under-constrain design. Therefore, the source condition has to hold for one cycle, and it might lead to a false failure. If a source condition is changed to a constraint, it might be an over-constrain design. Therefore, the constraint has to hold in every cycle, and it might lead to a false proof.

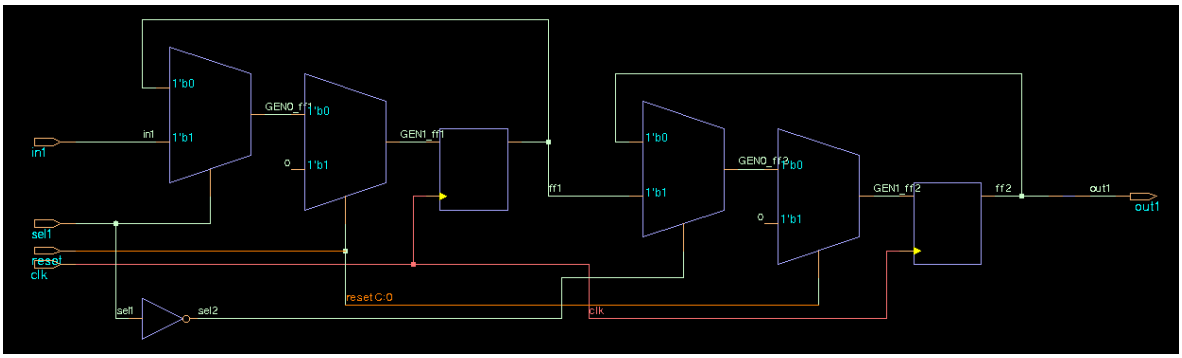
It is recommended to start verification without source and destination constraints and only add them to address false failures.

Example for Condition Design

In the following design, can data propagate from the input in1 to the output out1? It depends on the signals *sel1* and *sel2* where *sel2* is the inversion of *sel1*. *Sel1* must be asserted for data to reach the first flip-flop, and then *sel2* must be asserted for data to reach the second flip-flop.

Feedback

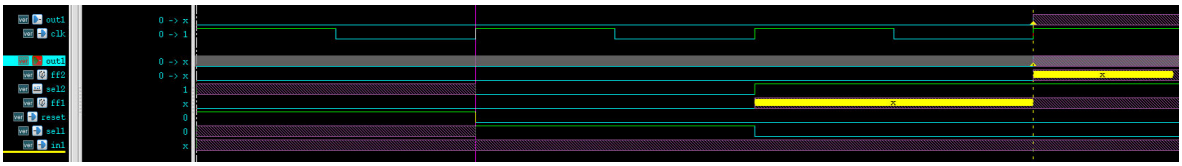
Figure 209: Design Example



The following property fails, that is, data can propagate:.

```
fsv_generate -src in1 -src_condition sel1 -dest out1 -dest_condition sel2
```

Figure 210: Data Propagation Examples



Source and destination conditions are not generally interchangeable. If you swap the source and destination expressions, then the property becomes proven where the data cannot propagate.

```
fsv_generate -src in1 -src_condition sel2 -dest out1 -dest_condition sel1
```

Now the property does not fail because when sel2 is asserted sel1 is 0 and hence data cannot propagate.

Example for Over-constraint

Now the property does not fail because sel2 can never be asserted because of the inversion.

```
fsv_generate -src in1 -src_condition sel1 -dest out1 -dest_condition sel2
```

To: `fvassume -expr {sel1 == 1'b1}`

The *sel1*property does not fail so *sel2* can never be asserted because of the inversion. The data never reach the second flip-flop.

Feedback

22.3.3 Source and Destination Condition Vacuity Checks

To detect an over-constrained environment or invalid source and destination conditions, vacuity checks on the expressions are automatically added. The vacuity check fails if the expressions are not coverable. For example,

```
fsv_generate -src in1 -src_condition sel1 -dest out1 -dest_condition sel2
```

In this example, two vacuity checks are added: one to cover {sel1 == 1} and one to cover {sel2 == 1}. The result is shown with the `report_fv -list` command.

```
[  2] constrained              (non_vacuous)  -  fail_src_in1 - (sel1) ->
 MARK(in1 )
[  1] falsified   (depth=13)   (non_vacuous)  -  fail_dest_out1 - (sel2)
 -> ASSERT(out1 )
```

If the vacuity check fails, the status of the property appears, as shown in the following figure.

Figure 211: Example for Vacuity Check

	status	name	depth	zones	engine	elapsed_time
1	✘	fail_dest_out1		fail	e2	00:00:04

The vacuity check for the source condition only checks the constraint property and the vauity for the destination condition only checks the assertion property. Both needs to be non_vacuous for the result to be valid. For example, the following `fsv_generate` property:

```
fsv_generate -src in1 -src_condition {sel1 && sel2} -dest out1
 -dest_condition sel2 -name bad_cond
```

The security property and the assetion are both proven and it is non_vacuous, indicated by the green check mark in the status column. However it is necessary to also check the constraints. The constraint is vacuous because sel1 and sel2 cannot both be 1 in the design.

22.4 Property Specification Use Case Examples

22.4.1 Specifying Multiple Sources and Destinations

The source and destination signal-list in the `fsv_generate` command can be a simple list of ports (For example, `in1`, `in2`, and `in3`). You can also create the simple list of ports using Tcl.

Examples

- No data propagation from any primary input to any primary output of the design.

```
fsv_generate -src [get_attribute [get_ports -filter {direction==in}]
 full_name]
-dest [get_attribute [get_ports -filter {direction==out}] full_name]
```

- No data propagation from the instance `cipher_load` to the instance `cipher_core`. This is equivalent to specifying all `cipher_load` outputs as source and all `cipher_core` inputs as destinations.

```
fsv_generate -src cipher_load -dest cipher_core
```

- No data propagation from any register to the `rd_data` output.

```
fsv_generate -src [get_attribute [get_sequentials -hierarchical -word]
 full_name]
-dest rd_data
```

- No data propagation from the `wr_data_secure` input to any net in the `cipher_core` instance.

```
fsv_generate -src wr_data_secure -dest [get_attribute [get_nets
 cipher_core.* - word] full_name]
```

22.4.2 Creating One Assertion for Each Source-Destination Pair

FSV creates one assertion per destination specified in `fsv_generate` command so that all propagation paths can be identified. If a `fsv_generate` command has multiple sources the assertion fails if any of the sources can propagate to the destination. This is normally a good trade-off between debug granularity and number of assertions and performance.

One can easily generate one assertion per source - destination pair using the`foreach_in_collection` tcl command.

Example: The `cipher_load` instance has 4 inputs and more than 10 outputs, but we are only concerned about two of the outputs here.

The following code will generate one assertion from each of the 4 inputs to 2 destinations.

Feedback

```
set allIn [get_ports cipher_load.* -word -filter {direction==in}]
    foreach_in_collection myPort $allIn {
 fsv_generate -name [get_attribute $myPort name] \
              -src [get_attribute $myPort full_name] -dest
 {dbg_interrupt rd_data}
    }
```

This will create 8 assertions:

```
[  1] not_run       -  cipher_key_valid_dest_dbg_interrupt -
 ASSERT( dbg_interrupt )
[  2] not_run       -  cipher_key_valid_dest_rd_data - ASSERT( rd_data )
[  4] not_run       -  cipher_read_en_dest_dbg_interrupt -
 ASSERT( dbg_interrupt )
[  5] not_run       -  cipher_read_en_dest_rd_data - ASSERT( rd_data )
[  7] not_run       -  cipher_read_addr_dest_dbg_interrupt -
 ASSERT( dbg_interrupt )
[  8] not_run       -  cipher_read_addr_dest_rd_data - ASSERT( rd_data )
[ 10] not_run       -  cipher_key_dest_dbg_interrupt -
 ASSERT( dbg_interrupt )
[ 11] not_run       -  cipher_key_dest_rd_data - ASSERT( rd_data )
```

Note:

This may create a very large number of assertions so run times may be long.

22.4.3 Viewing and Reporting Security Properties

Security properties are specified using the `fsv_generate` command, and the properties and the generated assertions and constraints are shown in the security table, property table and constraints table. Information about the security properties can also be reported with the `fsv_report` command.

Syntax

```
vcf> fsv_report [-list] [-verbose] [-no_summary] [-blackbox]
 [path_config] [path] [col] [allow_empty_assert][-gen_id <list-of-ids>]
 [-property <list-of-property-name>]
```

Where,

- `list`:Specify this option to generate one-line report for each `fsv_generate` command.
- `verbose`: Specify this option to generate a detailed report for each `fsv_generate` command.
- `no_summary`: Specify this option if you do not want to print the summary report.
- `blackbox`: Specify this option to generate a report of all the `fsv_blockbox` command.
- `path_config`: Specify this option to get a report of the user configuartion of blackbox paths.

Feedback

Report configuration and all paths inthe cipher_load instance

```
fsv_report -blackbox -path_config -instance cipher_load
```

- `path`: Specify this option to get a report of the blackbox paths from the inputs to the outputs
- `col`: Specify this option to get a return of the collection of instances specified with fsv_blackbox.
- `allow_empty_assert`: Specify this option to include the fsv_generate without active assertions.
- `gen_id <list-of-ids>`: Specify this option to provide a list of Ids of `fsv_generate` command.
- `property <list-of-property-name>`: Specify a list of property names for which you want to view the details. Note, currently, specifying a collection of properties is not supported. You can use * in the property names, namely`-property {sbox2sub_inst*_sub1_* sbox2mix*}`
- `[-instance <list-of-instance-names>]`: Specify the list of instance names.

  Reports the list of cipher_ctrl instance if it is fsv_blackboxed.

  ```
  fsv_report -blackbox -instance cipher_ctrl
  ```

The following report displays an example for `-list` option of the `fsv_generate` command where it lists all information about a secure property such as TaskId, GenId, Name, Zone and so on.

```
-------------------------------------------------------------------------
--------------
TaskId |  GenId |                  Name |                  Zone |  #Dest |
  #Src |  #Excl |
-------------------------------------------------------------------------
--------------
     7 |      2 |                  aes1 |                  aes1 |      1 |
     1 |      0 |
     7 |      3 |               aes_key |               aes_key |      1 |
     2 |      0 |
     7 |      0 |                in2out |                in2out |      1 |
     1 |      0 |
     7 |      4 |       key_cipher_ctrl |       key_cipher_ctrl |      1 |
     1 |      1 |
     7 |      1 |                  prev |                  prev |      1 |
     1 |      0 |
-------------------------------------------------------------------------
--------------
```

Feedback

You can also use the `report_fv -list`command to report all the generated assertions and constraints. The following is a list of all the constraints and assertions that are currently active in FSV. This report includes the reset constraints and other user-defined constraints as well. However, not all constraints are used for all assertions like Formal Property Verification (FPV) does. Only script constraints and source constraints from the same `fsv_generate` command apply to each assertion generated by the `fsv_generate` command.

```
Property List:
--------------
> Constraint
  # Constraint: 8
  [   2] constrained                   -  in2out_src_wr_data_secure -
 MARK( wr_data_secure )
  [   4] constrained                   -  prev_src_cipher_load_key -
 MARK( cipher_load.key )
  [   6] constrained                   -  aes1_src_aes_cipher -
 MARK( aes_cipher )
  [   8] constrained                   -  aes_key_src_cipher_load_key -
 MARK( cipher_load.key )
  [   9] constrained                   -  aes_key_src_aes_cipher -
 MARK( aes_cipher )
  [  11] constrained                  -
 key_cipher_ctrl_src_cipher_load_key - MARK( (secure_mode) ->
 cipher_load.key )
  [  12] constrained                  -
 key_cipher_ctrl_exclude_cipher_ctrl_cipher_key -
 EXCLUDE( cipher_ctrl.cipher_key )
  [   0] constrained                   -  constant_0 - rst_n==1

Security Property List:
-----------------------
> Assertion
  # Assertion: 5
  [   1] falsified   (depth=1)           -  in2out_dest_rd_data -
 ASSERT( rd_data )
  [   3] checking    (depth=8)        -
 prev_dest_cipher_ctrl_rd_data_secure_prev -
 ASSERT( cipher_ctrl.rd_data_secure_prev )
  [   5] proven                                  -
 aes1_dest_cipher_load_key - ASSERT( cipher_load.key )
  [   7] falsified   (depth=5)            -  aes_key_dest_dbg_interrupt -
 ASSERT( dbg_interrupt )
  [  10] checking    (depth=7)        -  key_cipher_ctrl_dest_rd_data -
 ASSERT( rd_data )
```

Each `fsv_generate` property has a unique GenId which identifies it in reports and in the property table.

Feedback

You can use the `fsv_report -verbose` option to know the correlation between the `fsv_generate` command and its corresponding constraints and assertions, since they all have the same GenId.

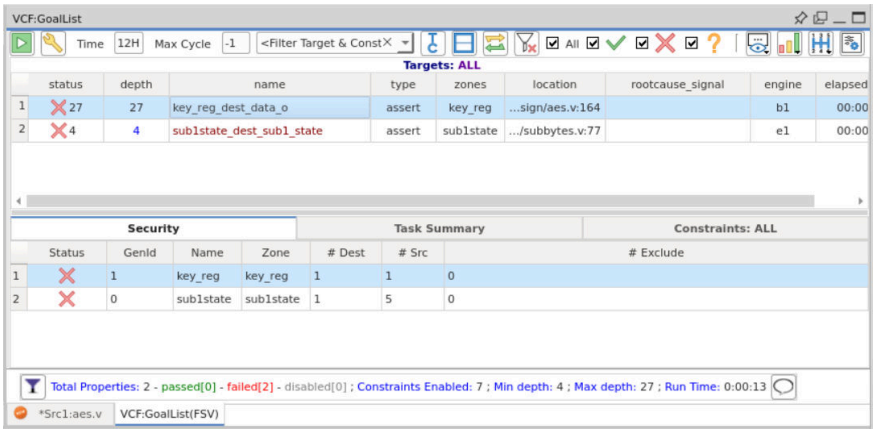
```
Generate >
     Name: key_cipher_ctrl
     TaskId: 7
     GenId: 4
     Zone: key_cipher_ctrl
 Destination >
     key_cipher_ctrl_dest_rd_data - checking - ASSERT( rd_data )
 Source >
     key_cipher_ctrl_src_cipher_load_key - MARK( (secure_mode) ->
cipher_load.key )
 Exclude >
     key_cipher_ctrl_exclude_cipher_ctrl_cipher_key -
EXCLUDE( cipher_ctrl.cipher_key )
```

The "Name" is the name specified in the `fsv_generate` command. The Source, Destination and Exclusion fields are identified by the name of the generated assertions and constraints and the signals specified in the `fsv_generate` command. Since security property may have many source and many destination signals, there will be one assertion or constraint reported per signal.

This `fsv_generate` command (GenId == 4) creates three properties that the `report_fv -list` command generates. The property name is the first part listed under Destination, Source, and Exclude. The signal following ASSERT, MARK, and EXCLUDE are the source, destination, and excluded signals.

The following figure shows the security properties, assertions, and constraints and their correlation.

Figure 212: Correlation of the Properties, Assertions, and Constraints



If you select a property in the assertion table, the corresponding security property in the security tab and constraint in the Constraints tab are highlighted too. Likewise, if you

Feedback

select a security property in the Security table, all assertions that are generated and the corresponding constraints are highlighted for easy understanding.

22.5 Black Boxing and Security Verification

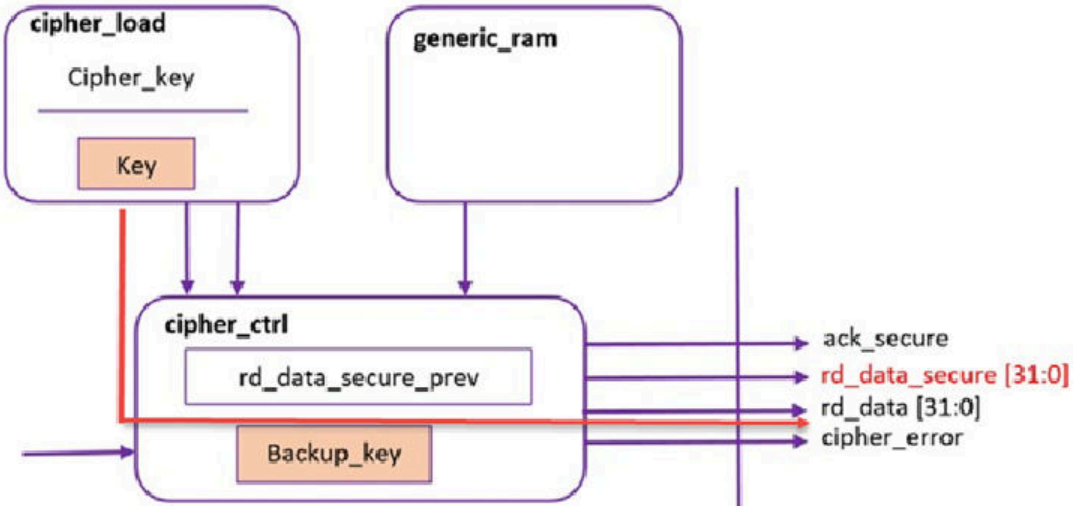
Blackboxing modules and instances that are not required for proving the security properties is an important technique to reduce complexity and enable SoC level verification. There are two types of black boxes in FSV:

- Structural black boxes. Exclude all logic in module or instance. Specify which module or instance to blackbox using `set_blackbox` command before `read_file` / `analyze` commands
- Secure black boxes. Exclude all logic in instances but allow data from sources specified in `fsv_generate` commands to pass through. Specify after reset state is defined (post `sim_save_reset`). Specify instance to be secure blackboxed using `fsv_blackbox` command.

Using structural blackboxes will decrease compile time and memory requirements. If the black boxed module or instance is not in the path between the source and destination, the property result is the same, as if the instance is not black boxed. If the black boxed module or instance exists in the path from source to destination, the property might be proven incorrectly. By black boxing one or more instances, it can remove paths that exist in the design, so the proven result is invalid.

For example, the following figure shows that the key register propagates to the output `rd_data` through the `cipher_ctrl` block, and the security property fails.

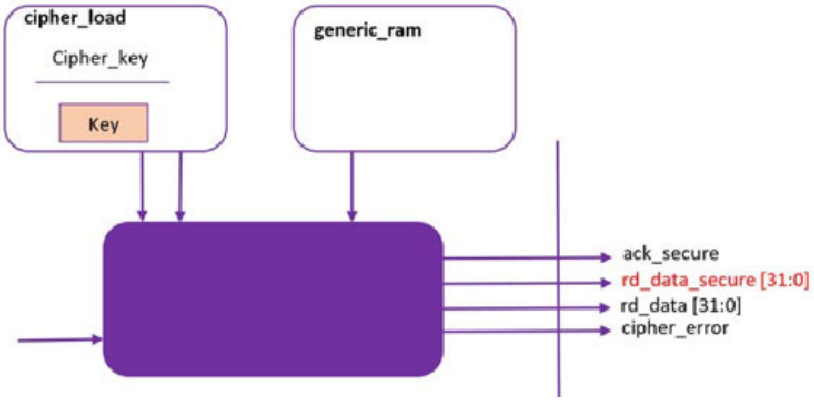
Figure 213: Example of Security Property Fail



Feedback

The following figure shows that if the cipher_crtl block is structurally black boxed, the path is broken and the security property is incorrectly proven.

Figure 214: Example of an Incorrectly Proven Security Property



If only the generic_ram module is black boxed, it would not have affected the result of the security property since it is not in the propagation path.

To verify that a property proof is valid in the presence of structural_blackboxes, create assertions on all inputs of structural blackboxes to verify that the source of the security property cannot reach the blackbox. Create the assertions using the `fsv_blackbox` command:

```
fsv_blackbox -add_input_assertion generic_ram
```

If all generated assertions are proven, then the structural blackbox does not cause false proofs of the property.

Note:

No input assertions will be generated for unconnected inputs of fsv_blackboxed instances since there cannot be any propagation through unconnected inputs.

This video focuses on using black boxing to achieve property convergence in VC Formal's Formal Security Verification (FSV) application. Black boxing simplifies complex RTL designs by replacing modules with abstract models, improving runtime and convergence. The following are the two types of black boxes discussed:

- Structural black boxes replace modules with empty models, offering faster runtime but risking false proofs.
- Secure black boxes allow data flow through models, ensuring valid proofs but risking false failures due to approximations.

Feedback

22.5.1 The fsv_blackbox Command

To avoid invalid proofs when instances are structurally black boxed, use the `fsv_blackbox` command to create a secure blackbox abstract model instead, that allows data to propagate through the blackbox.

Syntax

```
Usage: fsv_blackbox    # Create a secure blackbox for FSV App.
        [-all]                  (Create fsv_blackbox for all blackboxes)
        [-all_auto_path]        (Automatically generate paths from the
 inputs to the                                  outputs)
        [-no_auto_path]         (Do not automatically generate the path
 from the inputs                                  to the outputs)
        [-add_path]             (Add paths from -src to -dest)
        [-remove_path]          (remove paths from -src to -dest)
        [-src <list-of-input-ports>]  (The blackbox input ports
 considered as the                                  sources of the paths)
        [-dest <list-of-output-ports>]
                                (The blackbox output ports considered as
 the destinations                                  of the paths)
        [-add_input_assertion] (Add input assertions for the blackbox
 inputs)
        [-seq_path]             (Enforce sequential propagation paths from
 the inputs to                                 the outputs)
        [-comb_path]            (Enforce combinational propagation paths
 from the inputs                                 to the outputs)
        [-ignore_unresolved]    (Ignore unresolved instances)
        [-data_path_only]       (Block the propagation through the mux
 select signals, but                                allow the propagation
 from the true and false branches of a                                mux)
        [-remove]               (remove the instance from Security
 blackboxing)
        [<instanceName>]        (Blackbox instance name.)
```

- `[-all]`: Create secure blackbox for all structural blackboxes)
- `[-all_auto_path]`: Generate paths from the inputs to the outputs) - default
- `[-no_auto_path]`: Do not automatically generate the path from the inputs to the outputs
- `[-add_path]`: Manually add paths from -src to -dest in secure blackbox
- `[-remove_path]`: Manually remove paths from -src to -dest in secure blackbox
- `[-src <list-of-input-ports>]`: The blackbox input port considered as the source of the path to add or remove
- `[-dest <list-of-output-ports>]`: The blackbox output port considered as the destination of the path to add or remove.

Feedback

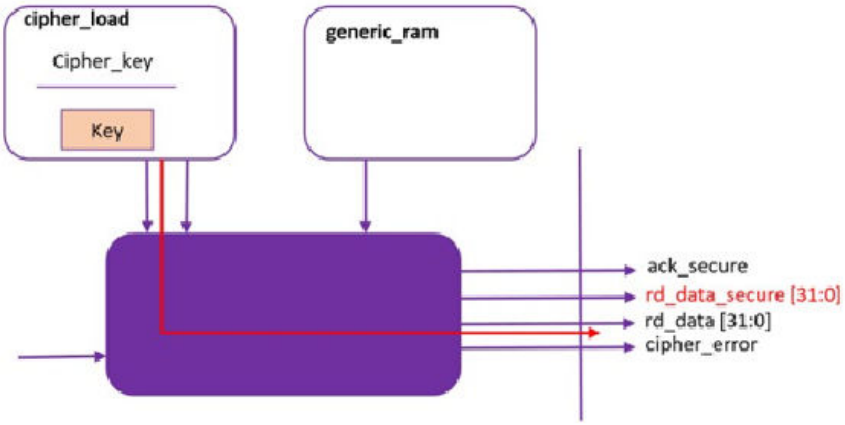
- `[-add_input_assertion]`: Add input assertions for the blackbox inputs.
- `[-seq_path]`: Enforce sequential propagation paths from the inputs to the outputs
- `[-comb_path]`: Enforce combinational propagation paths from the inputs to the outputs
- `[-ignore_unresolved]`: Do not create an `fsv_blackbox` for unresolved instances.
- `[-remove]`: Specify this option to remove the instance from security blackboxing.
- `[<instanceName>]`: Specify the instance name for which you want to create an abstract model.

In the previous example, if you run the command: `fsv_blackbox -seq_path -all_auto_path cipher_ctrl`.

This will create a secure blackbox of the instance. The security property checking for data propagation from key to rd_data correctly fails because the path now exists through the module.

Using the `-seq_path` to create a one cycle delay in the blackbox model is recommended to avoid combinatorial loops.

Figure 215: Example of Creating an Abstract Model



Note:

Note that the `fsv_blackbox` command might create paths that do not exist in the original design, and false failures might occur.

Structural and secure blackbox commands may be used together but the behavior of the default `-all_auto_path` option to the `fsv_blackbox` command is different. Normally a secure blackbox abstraction model is created without using the structural blackbox command `set_blackbox` first.

- For a structural blackbox, the `-all_auto_path` option create paths from all inputs to the all outputs
- For a security blackbox, the `-all_auto_path` option creates an abstraction model that contains all structural paths from inputs to outputs that are present in the RTL

If an instance is structurally blackboxed using `set_blackbox`, and then a secure blackbox is created of the same instance, it will have paths from all inputs to all outputs. This will likely lead to many false data propagation failures since too many paths are being added.

This may occur if there are undefined modules that are thus automatically structurally blackboxed and subsequently secure blackboxes are created using the -all option. This will create secure blackboxes with paths from all inputs to all outputs for all undefined modules. Use the `report_link` command following design read to review any undefined modules.

This is avoided by using the`-ignore_unresolved` option. In general, all modules should be defined even if they are blackboxed so that the inputs and outputs are defined.

An empty blackbox abstraction model can be created using the `fsv_blackbox -no_auto_path` command. This will behave exactly like a structural blackbox, that is, no data propagation is possible through the blackbox. One advantage of using an empty secure blackbox instead of a structural blackbox is that the structural paths from the RTL can be added to the secure blackbox using the `fsv_blackbox -all_auto_paths` command at run time.

22.5.2 Customizing Secure Blackboxes

A structural path in the RTL may not be a functional path from a data propagation point of view. These additional paths in the blackbox model may cause false data propagation failures. If one or many paths in the model are deemed incorrect, they can be removed so further false failures are avoided.

For example, an instance (my_inst) has a structural path from input ack_i to output req_o that is not a valid path. Since it is structural it is included when the secure blackbox model is created:

```
fsv_blackbox -all_auto_path my_inst
```

The invalid path can be removed:

```
fsv_blackbox -remove_path -src ack_i -dest req_o my_inst
```

22.5.3 Reporting Secure Blackboxes

To report attributes about secure blackboxes, use the `fsv_report -blackbox` command.

Feedback

Syntax

```
vcf> fsv_report -blackbox
     [-list]                prints a list of all fsv_blackboxes
     [-path_config]         (Outputs the user configuration of blackbox
 paths)
     [-path]                (Outputs the blackbox paths from the inputs
 to the outputs)
     [-col]                 (return the collection of instances specified
 with                              fsv_blackbox)
```

For example, to report the configuration of a secure blackbox created by the command:

```
fsv_blackbox -all_auto_path -remove -src fll1_ack -dest fll1_req
```

Will return:

```
vcf>fsv_report -blackbox -path_config
FSV Blackbox Report >
----------------------------------------
apb_fll_if_i
  ** Path Configuration:
      * AutoPath: all
      * PropagationType: CombPath
      * AddPath:
      * RemovePath:
          dest: apb_fll_if_i.fll1_req
            src: apb_fll_if_i.fll1_ack
1
```

To report all the paths present in a secure blackbox model use the command:

```
fsv_report -blackbox -path
```

For example

Feedback

```
fsv_report -blackbox -path
FSV Blackbox Report >
-----------------------------------------------
apb_fll_if_i
  ** IO connectivity:
    apb_fll_if_i.PRDATA is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll1_lock
      apb_fll_if_i.fll1_r_data
      apb_fll_if_i.fll2_lock
      apb_fll_if_i.fll2_r_data
    apb_fll_if_i.PREADY is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll2_ack
    apb_fll_if_i.fll1_add is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll2_ack
    apb_fll_if_i.fll1_data is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWDATA
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll2_ack
    apb_fll_if_i.fll1_req is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll2_ack
    apb_fll_if_i.fll1_wrn is connected with
      apb_fll_if_i.HRESETn
      apb_fll_if_i.PADDR
      apb_fll_if_i.PENABLE
      apb_fll_if_i.PSEL
      apb_fll_if_i.PWRITE
      apb_fll_if_i.fll2_ack
```

22.6 Abstractions

The FSV application does not support automatic abstractions. Using the `set_abstraction` command will create an incorrect formal model and will result in falsely proven or falsified properties. Therefore, this command must not be used in the FSV application mode.

Another type of abstraction is manually cutting nets using the snip_driver command. Since snipping a net is removing logic in the design it may lead to false positives. Any properties that are proven when snip_driver is used may be falsely proven. Any falsified properties when snip_driver is used are OK since a propagation path exists despite the cut. Due to the risk of false proofs it is recommended to avoid using snip_driver in FSV.

Feedback

22.7 Proving Security Properties

The FSV application proves the data propagation properties that the `fsv_generate` command specifies. If you want to prove SystemVerilog assertions and `fvassert` script properties, use the FPV mode to prove.

For example, a design has 1 SVA and 8 security properties. The following figures show the assertions and security properties that are applicable to the selected applications. In this example, the FPV application mode lists only the assertions, and the FSV application mode lists only the security properties.

Figure 216: Listing the Assertions in FPV Mode

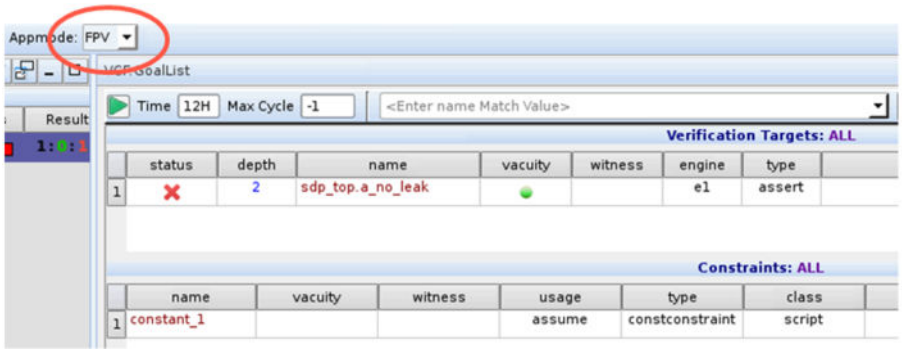
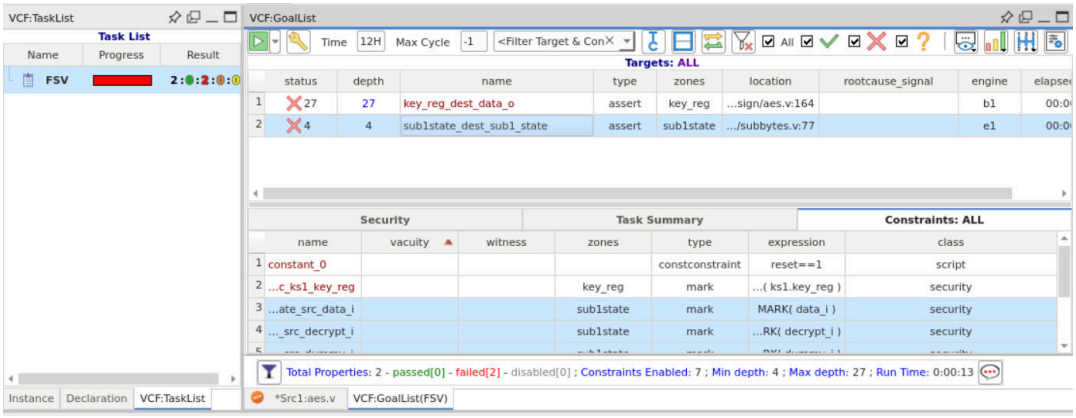


Figure 217: Listing the Security Properties in FSV Mode

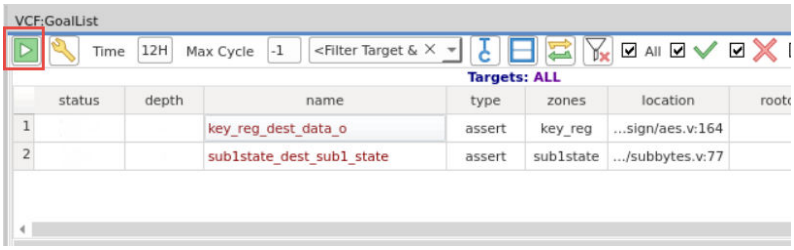


SystemVerilog constraints and fvassume script constraints do apply in all application modes because they describe the verification environment. For example, constraints in protocol assertion IP might be used for security verification. Note that any SVA or fvassume constraints might over-constrain the security properties and hide failures.

Feedback

To prove the security properties, use the `check_fv` command in the VC Formal Console or click (Run) button as shown in the following figure:

Figure 218: Proving the Security Properties



To run a subset of assertions, select the properties in the GoalList, in the RMB menu, select *Check Selected Properties*or, use the tcl command:

```
check_fv -property "property_name or ID"
```

VC Formal provides different ways to control the run. See section “Controlling check_fv Runs” for more details on how to stop, cancel, resume and setting time and memory limits for the proof run.

You can compute the root cause signals for falsified FSV properties during the check_fv run. To enable this feature, set the following formal variable to true:

```
set_fml_var fsv_compute_rootcause_auto true
```

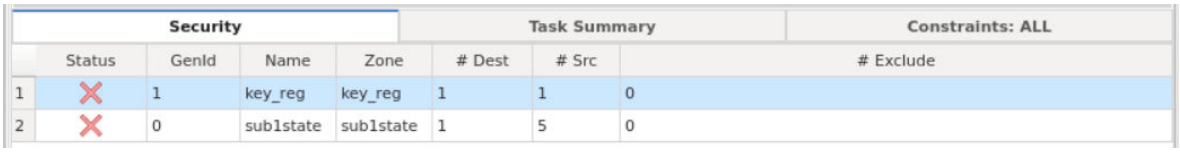
When the `fsv_compute_rootcause_auto` formal variable is set to true, use the `fsv_report_rootcause` command to get the report of root cause signals after the `check_fv` command.

By default, the `fsv_compute_rootcause_auto` formal variable is set to false.

22.7.1 Configuring the Customized View Settings for FSV

You can select the table colums that you prefer to view in the security table. By default, the security table lists the Status, GenId, Name, Zone, #src, #dest and #exclude columns.

You can select the columns to show by clicking the *Customized View Settings* button on the toolbar.

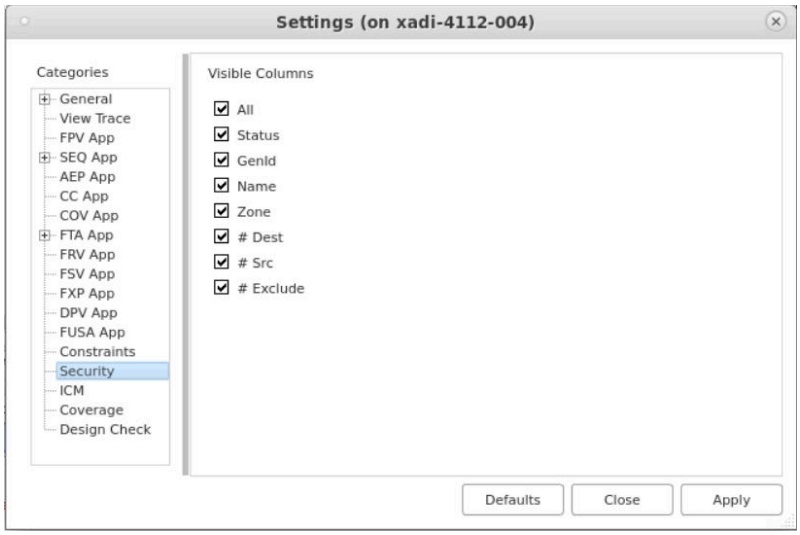


Feedback

The following ones listed under Categories applies to the FSV App:

- General
- FSV App
- Security
- Constraints

Figure 219: Configuring the Customized View Settings



22.8 Grid Usage

Running FSV on a compute grid (e.g. LSF) is essential to reach convergence of security propagation properties. Multiple engines are run per property and orchestration may divide proofs into sub-proofs. This results in many jobs that needs to be run in parallel for efficient verification.

FSV supports the same `set_grid_usage` command as other VC Formal applications.

Since the memory requirement for each engine job is not known, users often request grid machines with more memory than necessary, just in case. However, grid machines with lots of memory are often scarce and lead to long wait time for resource or fewer resources than requested.

A more efficient strategy is to request more machines with less memory and a few machines with lots of memory in case some engine jobs needs them. Use the `set_grid_usage -level <>` option to create a main and backup grid request.

For example:

Use 50 8GB memory grid machines on main grid and 5 32GB memory grid machines as backup.

```
set_grid_usage -level 0 -type LSF=50 -control {bsub -q normal -R
 'select[type==LINUX64
                                      && mem>=8000] rusage[mem=8000]'}
set_grid_usage -level 1 -type LSF=5 -control {bsub -q priority -R 'select
                                    [type==LINUX64 && mem>=32000]
 rusage[mem=32000]'}
```

Set the max amount of memory per job to the memory available on the backup grid machines:

```
set_fml_var fml_max_mem 32GB
```

This means that all engine jobs will be run on 8GB memory grid machines on the main grid. If they run out of memory, the jobs will be re-run on the 32GB memory backup grid machines to ensure the proof will make progress.

To minimize the number of engine jobs that run out of memory on the lower memory machines and thus needs to be re-started, set the following variable to always run certain critical memory intensive jobs on the backup grid:

```
set_fml_var seq_use_bg_for_critical_jobs true
```

Keep the variable `seq_use_bg_for_critical_jobs` as false if the wait time to get the larger memory machines is large. Critical jobs will wait for the backup grid machine if it is set to true and this will stall progress if no machines are available.

Only request as many grid machines as you know you have access to. Beware of limits of number of machines per user ID. The requested number of machines will affect engine orchestration and progress will be slower if much fewer machines are available than are requested.

Recommendation: Do not request more than 2X the number of grid machines you estimate you can get access to

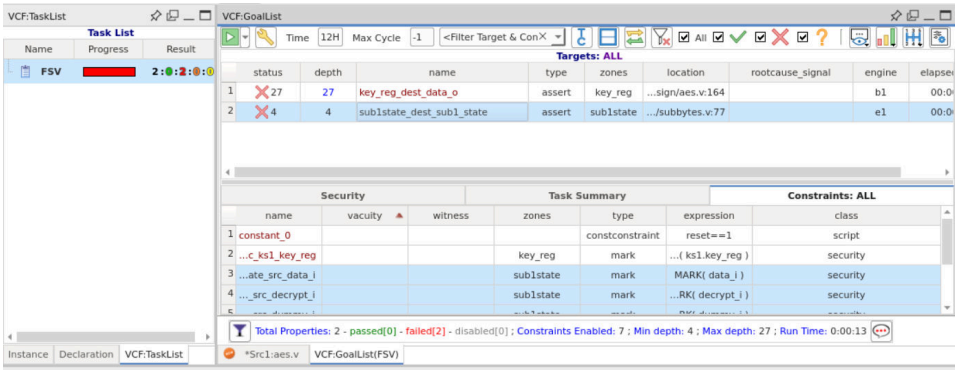
For more information, see section Running on Compute Farm.

22.9 Viewing the Status of Security Properties

You can view the status of the generated assertions under the GoalList in the GUI or use the report_fv - list command. The status of the assertions can be proven, falsified, inconclusive or not-run.

Feedback

Figure 220: Reporting and Viewing the Status of Security Properties



The *Security* table lists the status of each `fsv_generate` command. The status depends on the status of the generated assertions. Depending on the number of assertions generated, there are two cases:

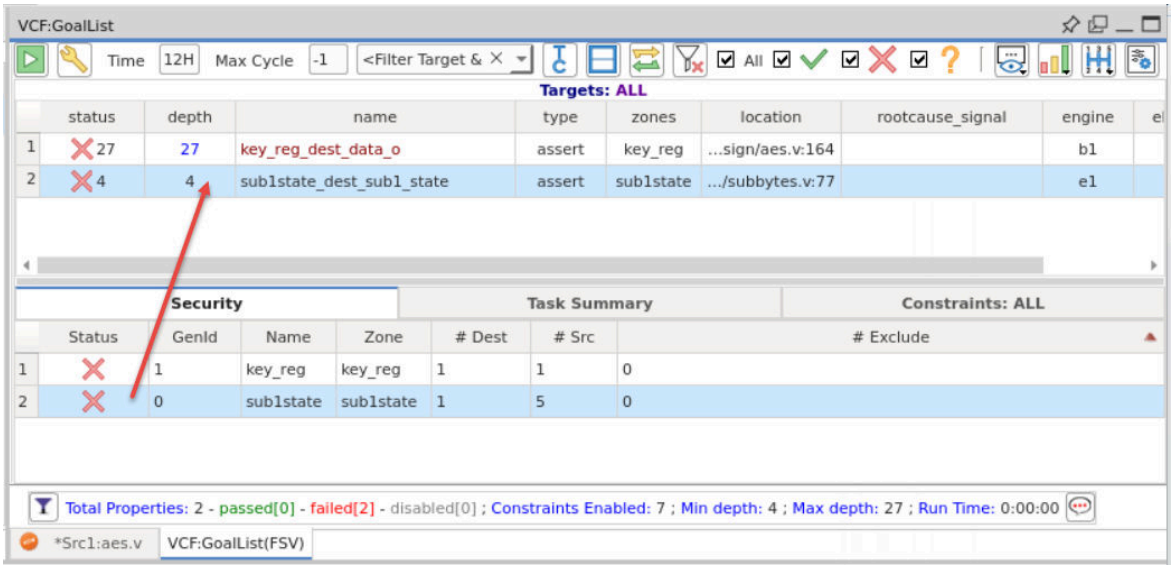
- *One Assertion*: One assertion is generated for each `fsv_generate` command. The status for both is always the same.
- *Multiple Assertions*: If the `fsv_generate` command creates multiple assertions, the status of `fsv_generate` is,
  - Proven if all generated assertions are proven
  - Inconclusive if all generated assertions are inconclusive or if some are inconclusive and some proven
  - Falsified if at least one generated assertion is falsified

22.9.1 Relation Between Security and Property Tables

You can find which assertions in the property table are generated by a specific `fsv_generate` command by double-clicking at the status column of the property in the security table. There will be one assertion in the property table selected for each destination in the corresponding `fsv_generate` command. There is also one constraint generated per source. The constraint is shown in the constraint table and the constraints corresponding to the selected `fsv_generate` command is highlighted.

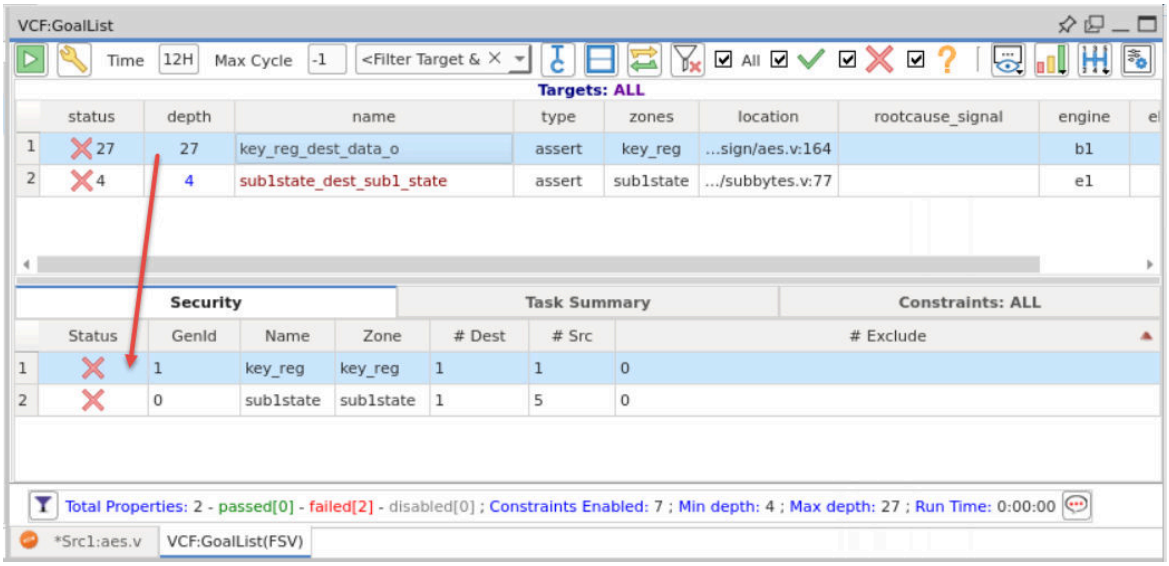
Feedback

Figure 221: Viewing the Interconnection from Security Table to Property Table



You can also find the related item in Security table when selecting assertions in property table. When you single click on a property in the property table, the corresponding property is highlighted in the security table and the constraints table.

Figure 222: Viewing the Interconnection from Property Table to Security Table



22.9.2 Status of Security Properties

The following table lists the status of each property indicated by different icons.

Feedback

Table 40: Status of Security Properties

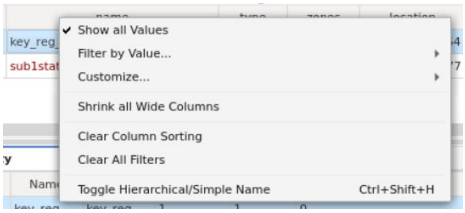
Icons	Description
	Indicates that all assertions are proven.
	Indicates that at least 1 assertion is falsified.
	Indicates that no assertion is falsified, but at least 1 assertion is in checking or inconclusive state.

In security table, the Status for properties is decided by the status of related assertions in the property table (verification targets). Live update for both security table and property table are supported. When the `check_fv` command is run, the status column of these two tables are updated after every two seconds.

22.9.3 Filtering and Viewing the Columns in Security Table

You can also filter the security columns and view the filtering from the RMB menu at column headers.

Figure 223: Filtering and Viewing the Security Table



Each table in VCF:GoalList (property, security or constraint) keeps its own filtering status. The filtering criteria is shown in the table header for the property table, or the tab header for security/constraint table. Different colorings in column headers also hint that some columns are used as filtering criteria.

Figure 224: Viewing Different Color Codes in the Table Header

The following table lists the icons that indicate the filtering criteria of tables.

Feedback

Table 41: Viewing the Filtering Icons

Filtering Icons	Description
	Indicates that the default Property table filter
	Indicates that the default Security table filter
	Indicates that the default Constraint table filter

You can click on the button to switch between the upper and lower tables. When the status of a lower table is shown, the button and filter criteria are consistent with the visible table. If you switch the tab between constraint and security table, the toolbar changes in accordance with the visible table.

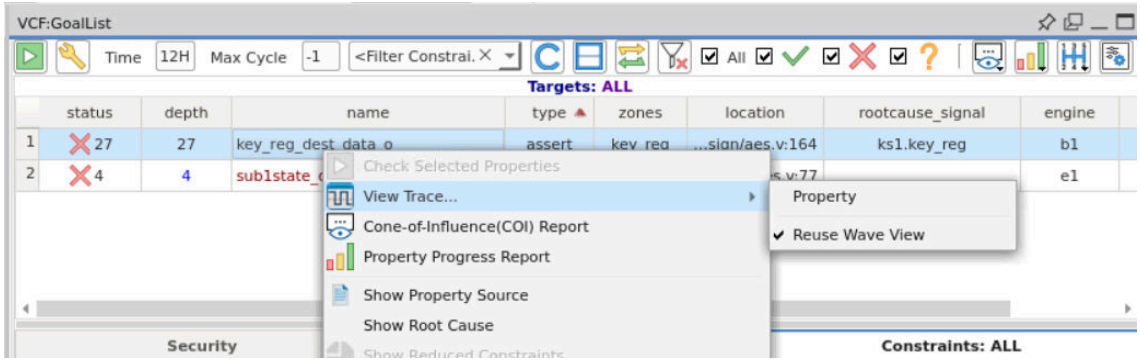
22.10 Debugging the Failing Security Properties

You can view the status of all security properties under *Verification Targets* in the VCF GoalList. The status is shown as proven, falsified, inconclusive or no status if *check_fv* is not run.

To debug a failing security property,

1. Double-click the red *X* mark in the status column. Or right-click on the failing property and select *View Trace* > *Property*.

   The waveform appears for the selected property and shows one example of how data propagates from source to destination. There may be many possible propagation paths and the first one found is shown. The trace shown is not guaranteed to be the shortest one.



   The following waveform shows how a value on source has propagated to the destination signal. The failed propagation is marked by a red arrow and the value at the

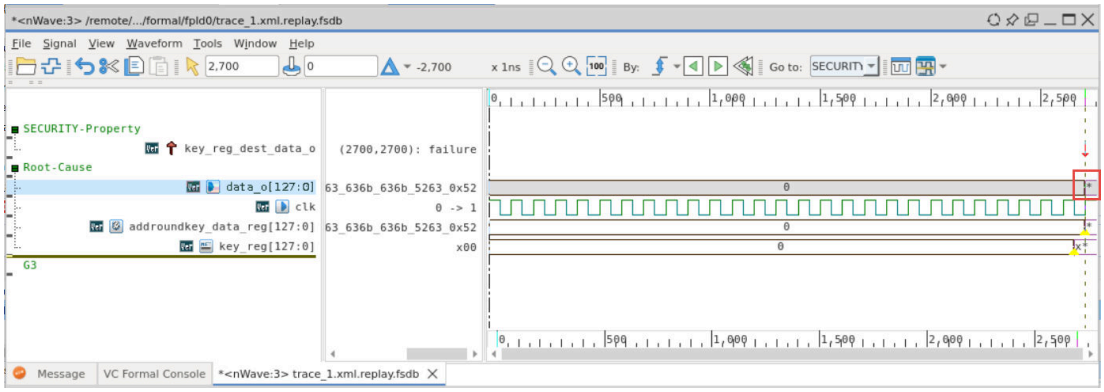
Feedback

time it propagates to the destination is shown in shaded purple. The shaded purple or the purple X for multi bit signals can be viewed as a symbolic value since FSV verifies than no value can propagate, not just specific values.

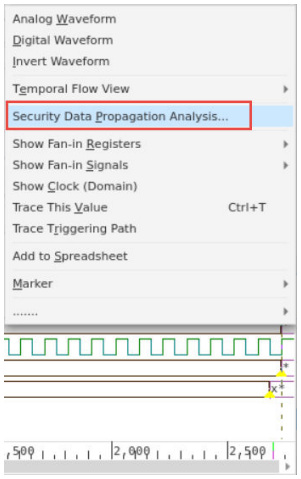
The reset part of the trace is normally not required for debug and it is not shown in the waveform. To view both the reset and formal failure trace together, set the following variable:

```
set_fml_var fsv_composite_trace true
```

The FSV-Root-Cause waveform group shows the destination signal first, then all the registers in the propagation path followed by the source signal.



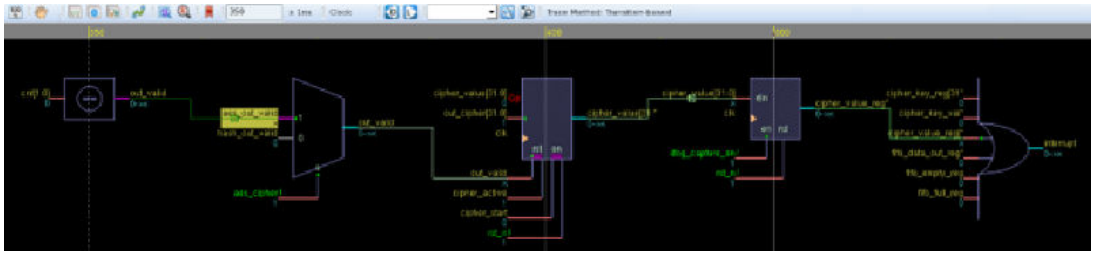
2. To view the intermediate signals between flip flops involved in the data propagation, select the destination signal and right-click the transition at the red arrow to view the menu.



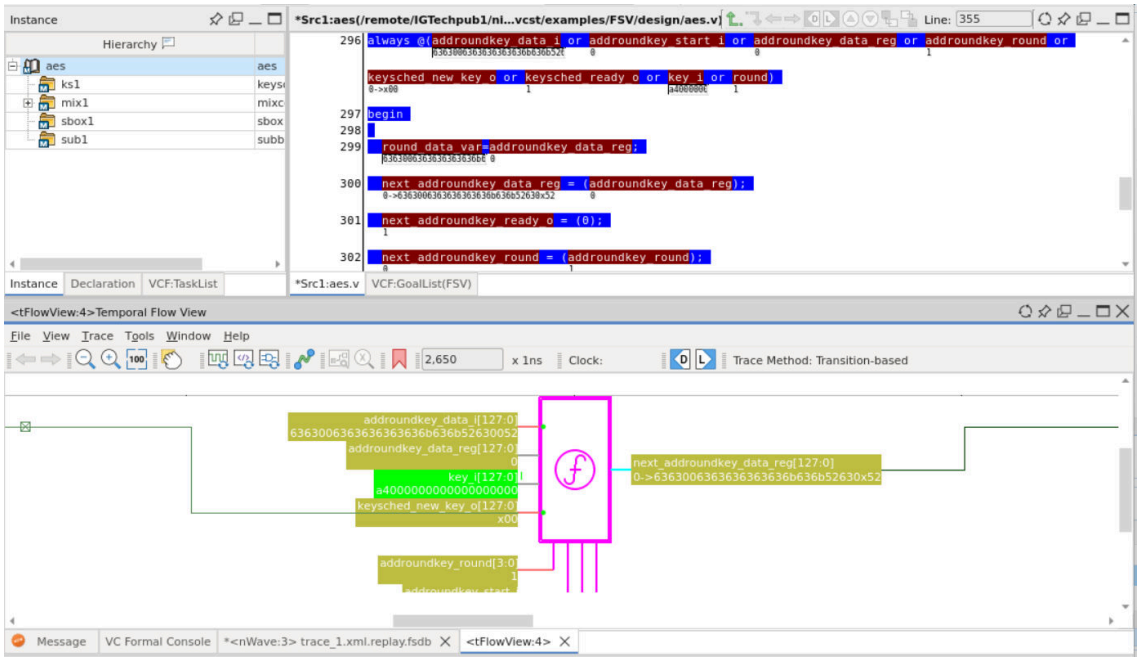
3. Select *Security Data Propagation Analysis*. It will show the data propagation in the Temporal Flow View under the FlowView tab. It shows how data propagate structurally and over time. The registers shown in Temporal Flow view will be the same ones as in

Feedback

the FSV-Root-Cause waveform group and the time when data propagates will also be the same..



4. In Verdi, the three views: Waveform, source view and temporal flow view are synchronized in time. The annotated values in the source view and temporal flow view refer to the time of the marker in the waveform view. The temporal flow view gives a good overview of how data propagates from source to destination but in order to debug why the propagation happened it is often easier to continue debugging in the source code view. After finding an interesting register or signal in the temporal flow view, drag and drop signals and functions from the temporal flow view to the source view to see the corresponding RTL code.



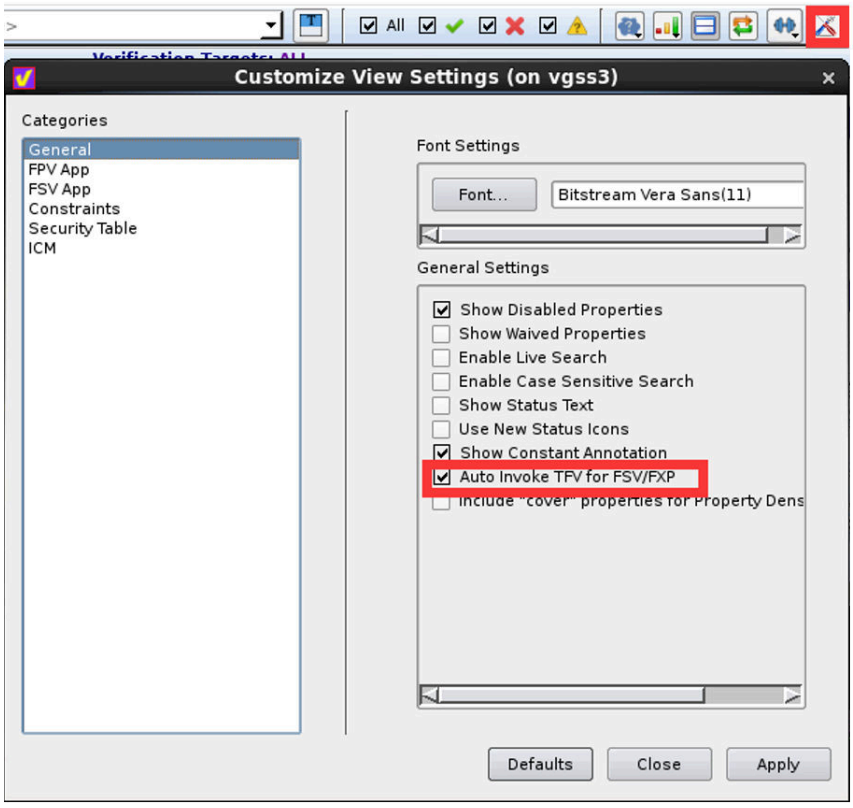
22.10.1 Enabling the Auto Invoke TFV for FSV

To enable the auto invoke TFV for FSV option,

1. Click the *Customize View Settings* icon on the toolbar.

   The *Customize View Settings* dialog box appears, as shown in the following figure.

Feedback



2. Select the *Auto Invoke TFV for FSV/FXP* check box.

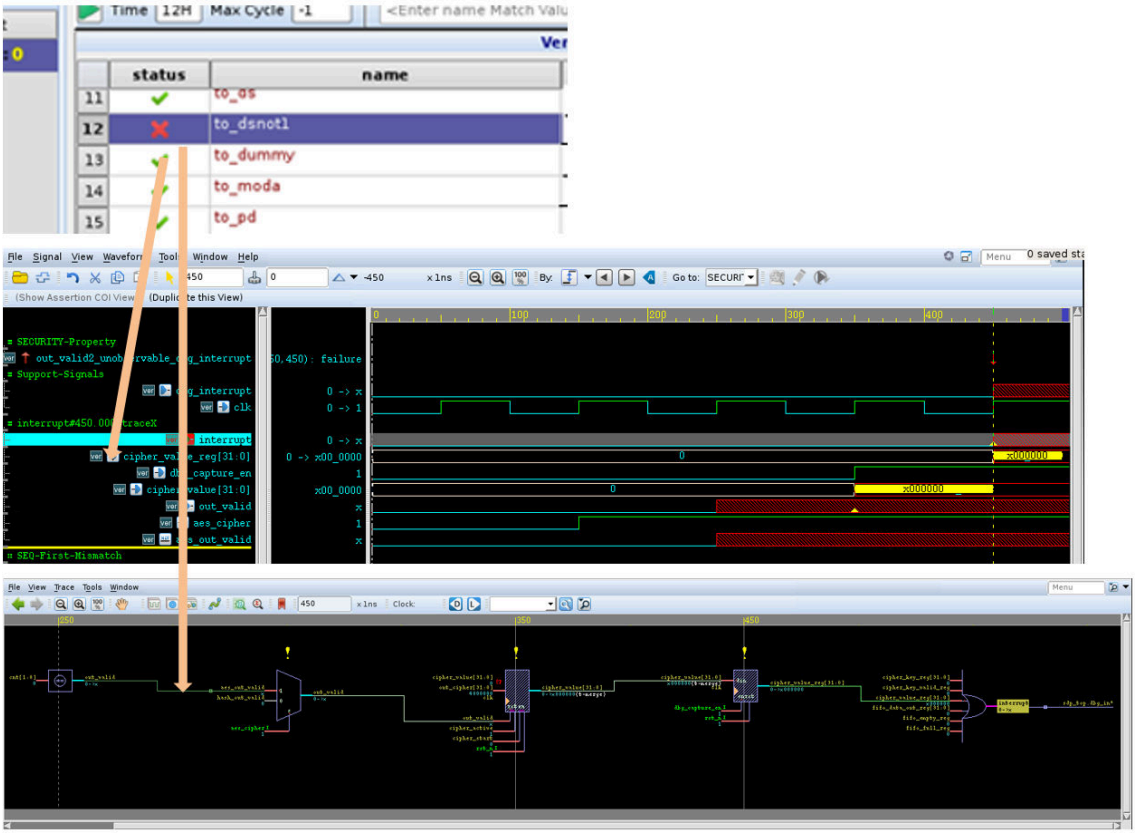
   Note:

   If you select the *Auto Invoke TFV for FSV/FXP* check box in the *Customize View Settings* dialog box, and then double-clicking the status column displays the *Temple Flow* view and the *nWave* view automatically.

3. Click *Apply* and *Close*.

Feedback

4. Double-click on the failing property in property table and see both the waveform and temporal flow view window by default, as shown in the following figure.



For more information about debugging in Verdi, see the *Verdi User Guide and Tutorial*.

22.10.2 Finding Root Cause Signal for Failing Assertions

FSV ensures that the unexpected data propagation does not happen from secure to non secure areas and vice versa in a design. For a falsified FSV property, you can get the details of the root cause signals automatically in addition to manual tracing by using the following commands.

- `fsv_compute_rootcause`
- `fsv_report_rootcause`

Use Model

Feedback

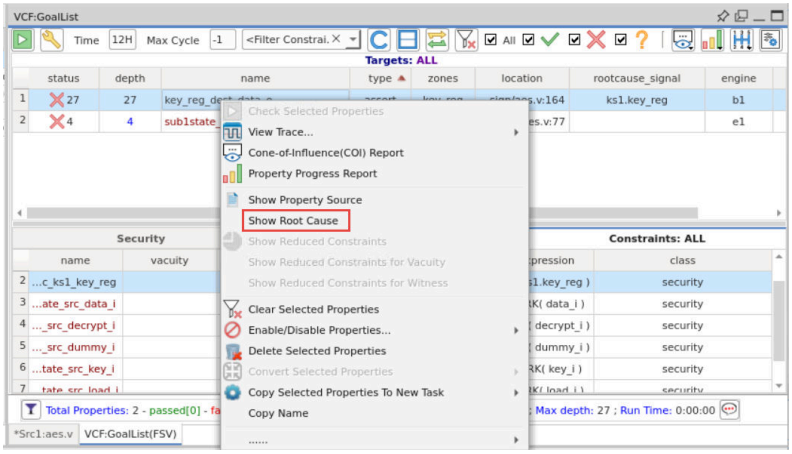
- To compute the root cause, use the following command:

```
fsv_compute_rootcause     # Compute the root causes of the selected
 properties
[-stop]                   (Clear the property list for
 fsv_compute_rootcause)
[-block]                  (Compute the root causes in the block mode)
[<list-of-property-name>]
```

- To report the root cause, use the following command:

```
fsv_report_rootcause     # Report the root cause of the selected
 properties
[-list]                   (Outputs one-line report per property)
[-no_summary]             (Outputs without a summary of the root cause
 types)
[<list-of-property-name>] (A list of property names)
```

Alternately, you can use RMB option as shown in the following figure:



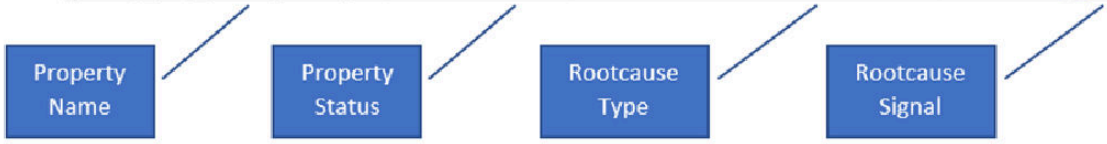
Example

The `fsv_report_rootcause -property <property name> -list`followed by `fsv_compute_rootcause` gives the details about the root cause signal and root cause type.

Feedback

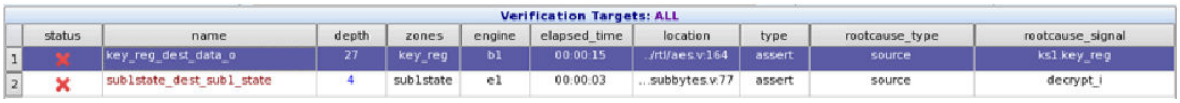
The following is an example of one such root cause:

```
vcf>fsv_report_rootcause -property key_reg_dest_data_o -list
Summary of root cause information
-----------------------------------
Total: 1
 source                    :          1
List of root cause information
-----------------------------------
 key_reg_dest_data_o |  falsified |                  source |  ks1.key_reg
```



This information can be used to identify and classify the issues. For example, in the above case it says, the property key_reg_dest_data_o is falsified due to there is a leakage from ks1.key_reg which is the defined source to the destination.

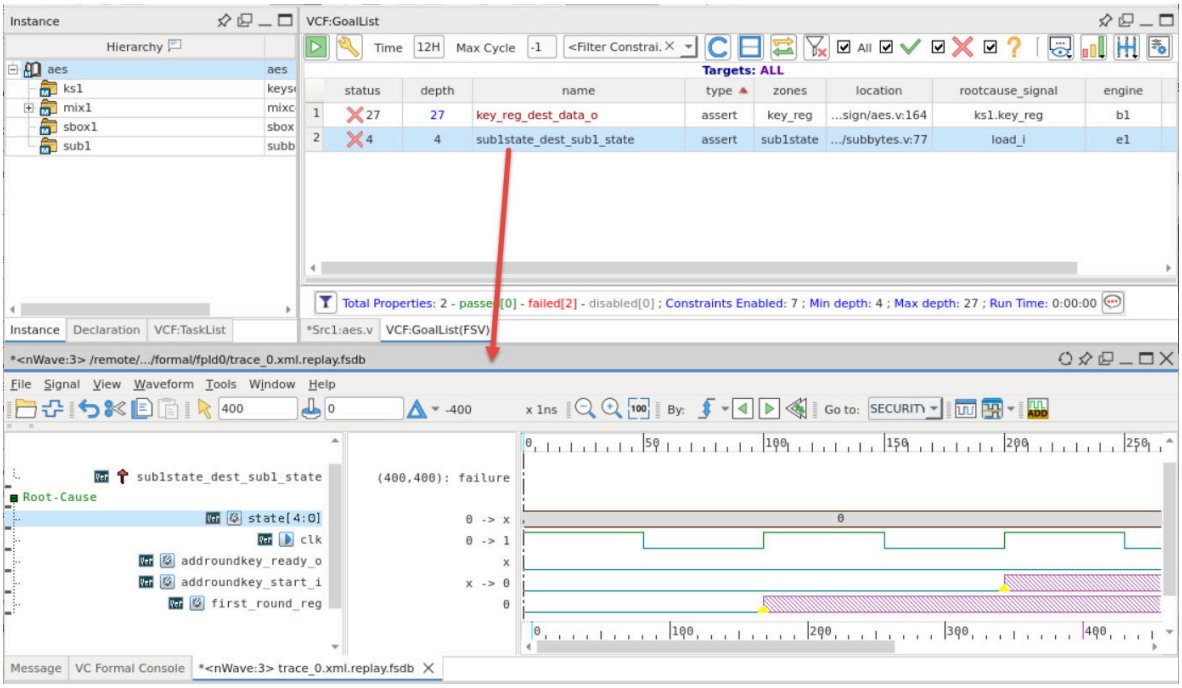
Rootcause information is also populated in the property table by the RMB action or `fsv_compute_rootcause` command. Rootcause information is populated in the property table as well.



Verification Targets: ALL

	status	name	depth	zones	engine	elapsed_time	location	type	rootcause_type	rootcause_signal
1	✘	key_reg_dest_data_o	27	key_reg	b1	00:00:15	../rtl/aes.v:164	assert	source	ks1.key_reg
2	✘	sub1state_dest_sub1_state	4	sub1state	e1	00:00:03	...subbytes.v:77	assert	source	decrypt_i

For the falsified properties, double-click ✘ to generate CEX. The new group of signals “FSV-Root-Cause” automatically added to the waveform.



22.11 Property Convergence

If property proofs are not converging after a long time, what can I do to optimize performance?

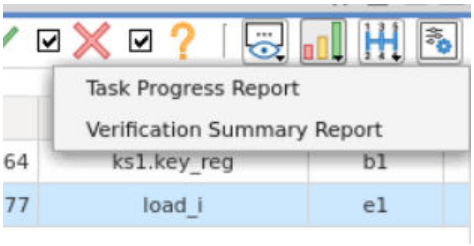
The first step is to ensure that the formal engines are running as expected. This includes checking that the requested grid resources are available and that engine jobs are not terminated by errors. Secondly, review security properties to identify if they can be written more efficiently. If many properties are not converging, run them one by one so that they are not competing for compute resources. Thirdly, if resources are available and proofs are running, determine if they are making sufficient progress or if it is better to terminate the run and try another approach. Lastly, it may be necessary to minimize the complexity of the verification task through structural or secure blackboxing of modules or applying manual path partitioning.

22.11.1 Are Formal Jobs Running as Expected?

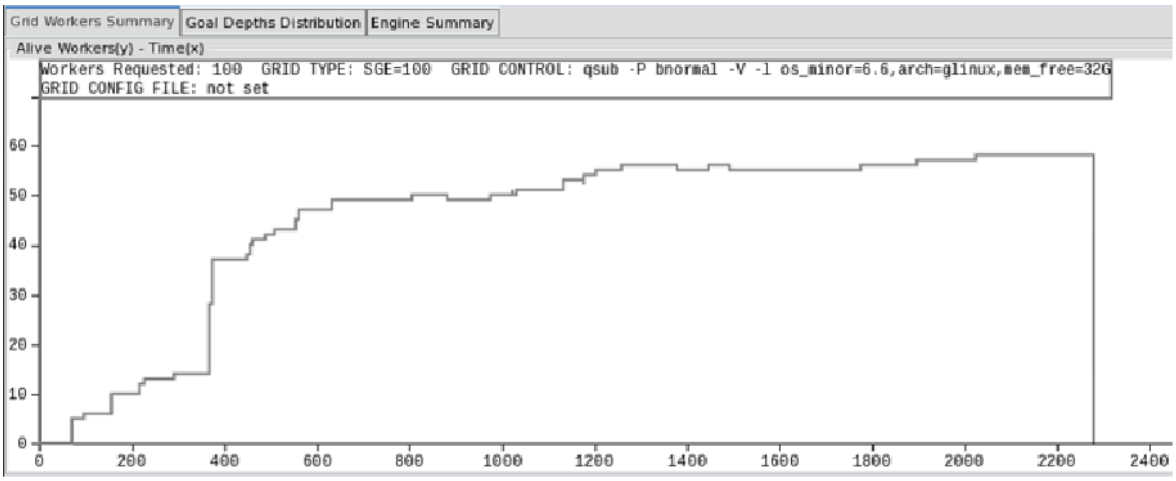
Multiple formal engine jobs may be run in parallel on a compute farm. The number of grid machines to use is requested with the `set_grid_usage` command. However, the requested number of grid machines may not be available so much fewer jobs are being executed than expected, causing longer run times.

Feedback

To view how many grid machines are being used, select *Grid Workers Summary* in the GUI:



This will show how many machines are being used over time:



In this case, 100 grid machines were requested but it took 10 minutes to get 50 machines, and no more than 55 machines are ever available.

If grid resources are available, ensure engine jobs are completed successfully. Run the `report_fml_jobs` command to see the status of all engine jobs:

```
vcf>report_fml_jobs
Solver jobs summary for
verification task FSV
    #scheduled     :     0
    #running       :     0
    #completed     : 10161
        #normal        :  5729
        #timeout       :    24
        #memout        :     3
        #orc_killed    :   329
        #sig_killed    :  3971
        #crash         :     0
        #start_fail    :     0
        #misc_error    :     0
```

Feedback

First look at the sig_killed category. This means that the grid system or operating system killed the job. This may be due to access permissions, incorrect grid control parameters or exceeding memory or run time limits. A few sig_killed jobs is no problem, these jobs will be re-started and run on other machines but if a large number of jobs are killed, the proof may not make progress. The memout category means that some jobs needed more memory than specified with `fml_max_mem` and `set_grid_usage` command. Use `report_fml_jobs -list -reason memout` to see how much memory was used and increase the amount of memory requested.

22.11.2 Optimizing Security Properties for Performance

Use word level source and destination signals in the `fsv_generate` command. Data propagation verification from every bit of the source to every bit of the destination will be performed even when word level signals are used. For example:

```
reg [7:0] key_data;
output [31:0] hrdata;
```

The following property will check if any bit of key_data can propagate to any bit of hrdata:

```
fsv_generate -name word -src key_data -dest hrdata
```

Specifying for example, each bit in the source separately will generate 8 properties instead of one so run time and convergence may be worse.

```
fsv_generate -name bit0 -src key_data[0] -dest hrdata
fsv_generate -name bit1 -src key_data[1] -dest hrdata
fsv_generate -name bit2 -src key_data[2] -dest hrdata
fsv_generate -name bit3 -src key_data[3] -dest hrdata
fsv_generate -name bit4 -src key_data[4] -dest hrdata
fsv_generate -name bit5 -src key_data[5] -dest hrdata
fsv_generate -name bit6 -src key_data[6] -dest hrdata
fsv_generate -name bit7 -src key_data[7] -dest hrdata
```

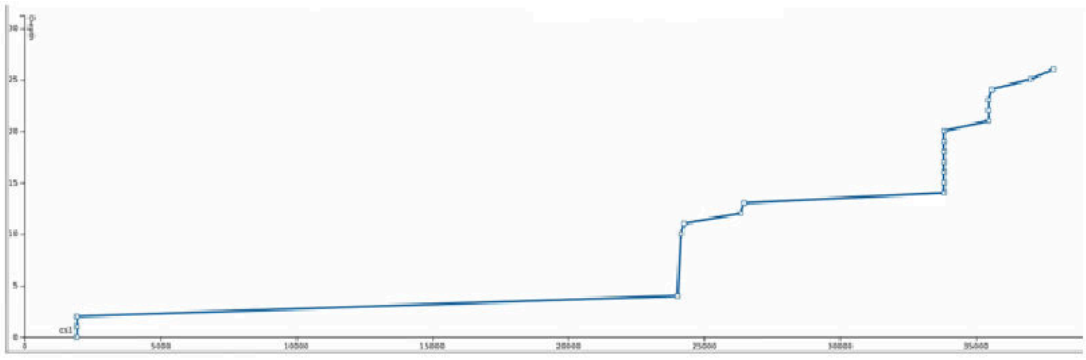
Debug granularity will be finer i.e. if many bits can propagate, there will be more counter examples shown, but at the expense of performance.

22.11.3 Are Proofs Making Progress

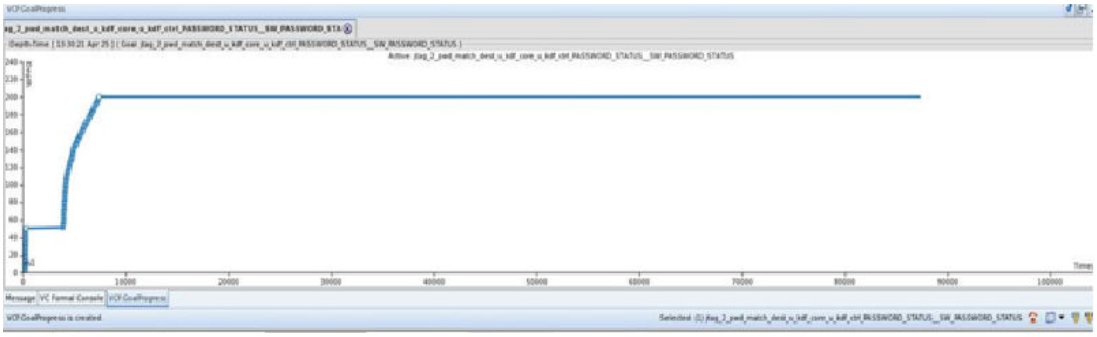
If a property proof has been running for multiple hours, how do you decide between letting it run further or, cancel it and restart with some setup change?

Feedback

Use the "Property Progress Report" . Select a property in the property table, right mouse click menu and select "Property Progress Report. The graph will show the bounded depth achieved so far for the selected property. For example:



In the above case, the formal engines are still making progress and the time to reach one more cycle deeper is not increasing for every step so this proof is likely to converge.



In this example, the engines reach depth 200 pretty quickly but then there is no further progress so property is unlikely to converge in the current setup. One indication is that the time to reach one more cycle is increasing for every cycle.

Engine orchestration is balancing between running formal engines that prove and running engines that can falsify properties. If the bounded depth is increasing to a point and then makes no more progress. One option is to increase the focus on bug hunting by setting effort to bug_hunting

```
set_fml_var fml_effort bug_hunting
```

Another option is to increase the bug hunting depth i.e. continue running engines that can find failures to deeper depth. Use `set_fml_var seq_bmc_depth 400` to increase the depth to 400 for example.

Feedback

This video focuses on property progress analysis in the Formal Security Verification (FSV) application of VC Formal. It explains how to monitor and troubleshoot the progress of formal verification tasks, particularly in grid environments. The video emphasizes the importance of proper grid configuration and resource allocation to ensure efficient formal verification progress. It illustrates defining the property progress analysis and introducing key Tcl commands such as `report_fv`, `report_fml_jobs`, and `report_fml_hosts`. These commands help analyze property statuses, engine job progress, and grid resource utilization.

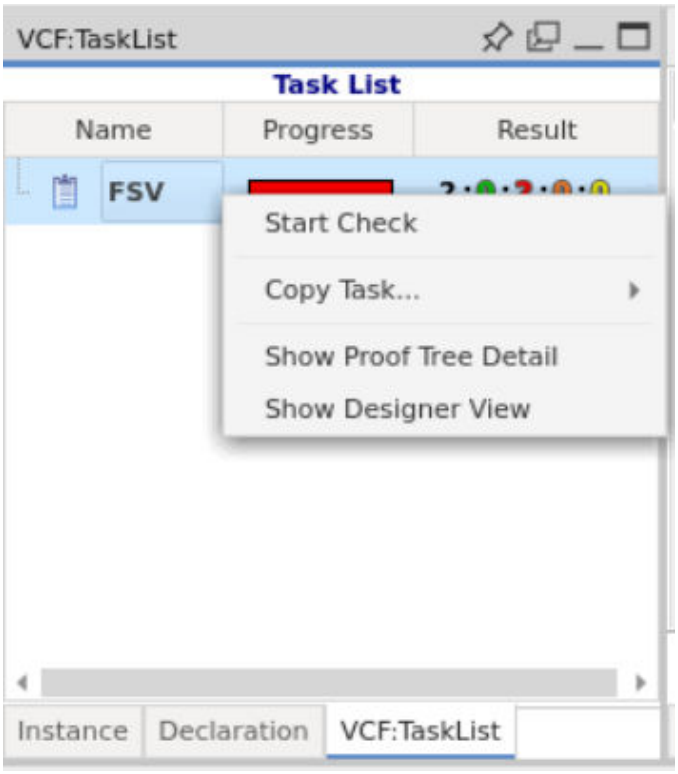
22.11.4 Reducing Complexity

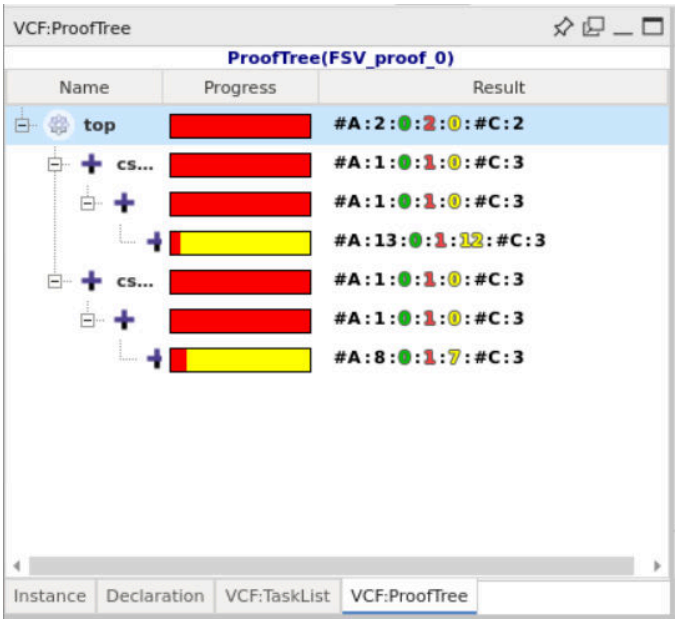
Using structural blackboxing, see section Property Specification Use Case Examples is an effective way to reduce the complexity of a formal problem to achieve convergence. If properties are not converging and are not making progress, structural blackbox (`set_blackbox` command) several modules that appear to be not related to the property to verify. If any property is falsified, this is a valid failure that needs to be debugged. If any property is proven with several modules structurally blackboxed, the proof is probably not valid and caused by the blackboxed module.

To proceed when no more failing assertions are found, use secure blackboxes instead of structural blackboxes which will allo data to pass through. This means that any proven property is valid.

In order to prove that a source signal cannot propagate to a destination, FSV needs to verify that no propagation path exists. To minimize complexity it may divide the proof into sub-proofs to find intermediate points the source can and cannot propagate to. The progress of the sub-proofs can be viewed in orchestration view. In the Task List, select the current task and from the right mouse button menu select, *Show Orchestration View*.

Feedback





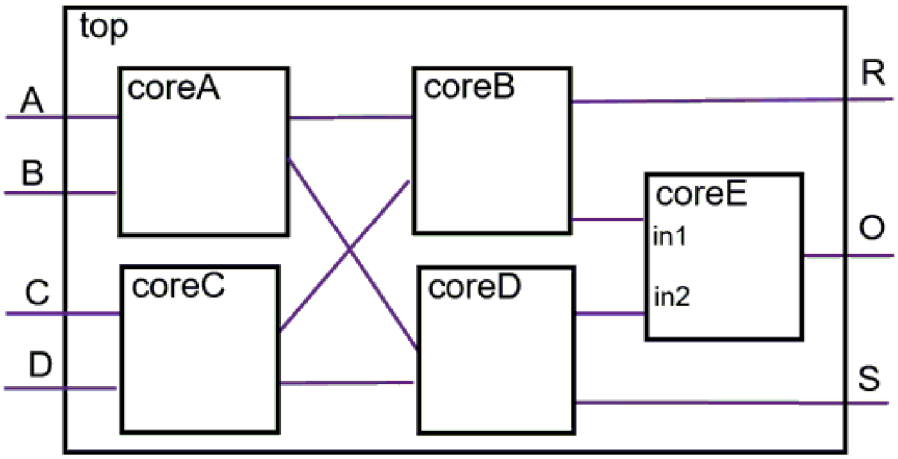
If there are many failures, there are many intermediate points the source can propagate to and more propagation paths needs to be analyzed. The automatic partitioning may be disabled by setting `set_fml_var fsv_use_casesplit false`.

Feedback

In addition to the tool supported automatic partitioning, the user can manually partition the path and reach faster convergence by applying design insights. For example, in the design below the property:

```
fsv_generate -src [get_attribute [get_ports -filter {direction==in}
-word] full_name] -dest O
```

is not converging. Instead of starting at each source by analyzing the design it is apparent that if there is no propagation path through the module *coreE*, all properties must be proven.



Start for example with analyzing all propagation paths through the instance *coreE*. If all properties are proven, there are no propagation paths through *coreE* and the original property is proven since no input can propagate past *coreE*. If some of the properties through *coreE* are falsified, check propagation from all inputs as in the original property but to the inputs of *coreE* that can propagate to the final destination O. This eliminates the complexity of *coreE* and may enable the new properties to converge. If the original property only has one source, it may be better to start the analysis there.

22.12 Signoff Review

When all security properties are proven, it is important to review the setup and steps taken to achieve the proofs in order to ensure the proofs are valid. Compared to FPV and CC, it is easier to get a false proof since removing logic through blackboxing or abstractions in FPV and CC leads to falsified properties but in FSV removed logic will prevent data propagation leading to a proof instead. Proofs may be invalid due to setup or user commands during the verification process.

Feedback

Before considering security verification complete, review the following items and if necessary, modify the setup and re-run:

1. Are any modules structurally blackboxed, that is, `set_blackbox` command was used before `read_file` / `analyze` command? Either create a secure blackbox for each module or use the`-add_input_assertion` option to `fsv_blackbox` to ensure no sources can propagate to structural blackboxes.
2. Are there any undefined modules? They are automatically blackboxed and can prevent data propagation that may exist in the complete design. Run `report_link` after reading the design to identify any undefined modules. Include the missing modules, or if not available, create a stub model of just the input and output ports and then follow the steps listed for structural blackboxes.
3. Are there any cut nets, that is, `snip_driver` command used? Any cut net may break a propagation path that exist in the design. It is recommended to re-verify the proven properties without using `snip_driver` command
4. Are clocks and reset signals correctly specified. Missing or incorrect clock and reset setup can lead to both false failures and false proofs. Run `check_fv_setup` to ensure clocks and resets are defined.
5. Is the design over-constrained through `set_constant` or `fvassume` constraints? Constraints will limit the legal state space and may prevent signal propagations that would otherwise happen in the design. It is recommended to minimize the use of constraints and to re-run proven properties without constraints. If they fail and the reason for failure is not a valid path in the design and it is caused by the missing constraint, then the original proof is OK.
6. Are abstractions used? The `set_abstraction` command will replace part of the design with an abstracted model. Propagation paths that exist in the design may not exist in the abstraction model and will thus lead to a false proof. It is recommended to not use the `set_abstraction` command.

22.13 Advanced Techniques

22.13.1 Security Assert and Assume Properties

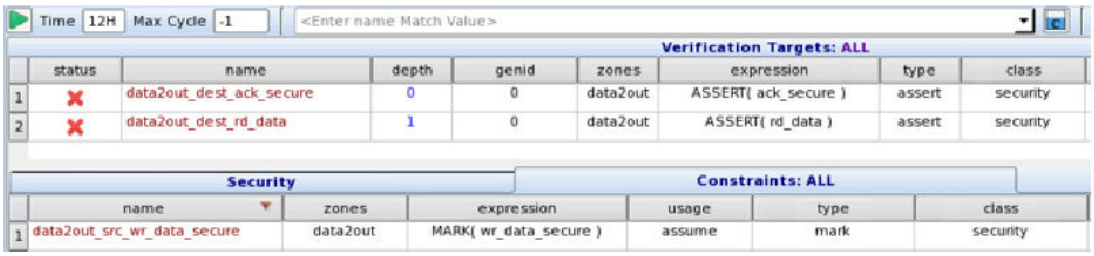
Security properties are specified using the `fsv_generate` command. The `fsv_generate` command creates assume properties for all the source signals and assert properties for destination signals. The generated properties are identified by the class attribute set to "security" and the type attribute set to "assert" or "mark".

Feedback

The following `fsv_generate` statement:

```
fsv_generate -name data2out -src wr_data_secure -dest {rd_data
 ack_secure}
```

Will generate the properties shown in the following figure. One constraint is generated since there is one source and two assertions are generated since there are two destinations specified in the `fsv_generate` statement. The zone attribute is the same for all three properties which means they will be verified together.



The fields shown for each assertion above is also available using tcl commands.

```
vcf>get_attribute [get_props data2out_src_wr_data_secure] class
security
vcf>get_attribute [get_props data2out_src_wr_data_secure] type
mark
vcf>get_attribute [get_props data2out_src_wr_data_secure] zone
data2out
vcf>get_attribute [get_props data2out_src_wr_data_secure] gen_id
0
```

These properties can also be created directly using the `fsv_assume`and `fsv_assert` commands instead of using `fsv_generate`. The properties generated by the `fsv_generate` command above can also be generated by:

```
fsv_assume -zone data2out -name src -mark wr_data_secure
fsv_assert -zone data2out -name dest1 rd_data
fsv_assert -zone data2out -name dest2 ack_secure
```

Note: The zone name is used to associate one or more sources through the `fsv_assume` property with one or more destinations specified with the `fsv_assert` command.

```
Usage: fsv_assume    # Create a constraint for the specified signal.
        [-name <name>]          (Name of the constraint.)
        [-zone <zone>]          (Zone that the constraint belong to.)
        [-condition <condition>]      (Optional combinational condition
 expression.)
        [-mark]                 (This is a mark type assumption)
        [-exclude]              (This is a exclude type assumption)
        <signal>                (Signal name.)
Usage: fsv_assert    # Create a security assert property for the signal.
        [-name <name>]          (Name of the property.)
```

Feedback

```
        [-zones <zones>]        (Zone(s) that the assert signal belong
to.)
        [-condition <condition>]        (Optional combinational condition
expression.)
        <signal>                (Signal name.)
```

It is recommended to use the `fsv_generate` command to specify security properties instead of specifying them separately using multiple `fsv_assert` and `fsv_assume` commands.

One use case for using separate commands is when checking propagation to all inputs of blackboxed instances. These assertions are automatically generated when the -add_input_assertion option is used with fsv_blackbox (see section Property Specification Use Case Examples). These assertions all have the zone attribute set to "blank". This match all possible zones i.e. each blackbox input assertion checks for propagation from all sources, regardless of zone. Instead of using `fsv_generate` properties just to define sources when checking for propagation to all blackbox inputs, specify sources using the `fsv_assume` command:

```
fsv_assume -mark signalname
```

The source and destination condition options to the `fsv_generate` command creates conditional constraints and assertions. For example, the following `fsv_generate` command creates the conditional MARK constraint seen in the following figure.

```
fsv_generate -name key_cipher_ctrl -src_condition {secure_mode==1} -src
cipher_load.key -dest rd_data
```

22.13.2 Grouping Properties Using Zone Name

The `-zone` option in the `fsv_generate` command can be used to group properties generated by multiple `fsv_generate` commands, so they are verified together. By default, the zone name is the same as the default name, or the name specified using the`-name`option.

If all zone names are different, this is equivalent to not using the `-zone` option.

Example

If you are using the same zone name for multiple `fsv_generate` commands:

```
fsv_generate -name A1 -zone z1 -src src1 -dest dest1 -exclude {foo}
fsv_generate -name A2 -zone z1 -src src2 -dest dest2
```

Feedback

This is equivalent to:

```
fsv_generate -name A12 -zone z1 -src {src1 src2} -dest {dest1 dest2}
-exclude {foo}
```

This feature allows you to write many smaller lines instead of one big line. All source, destination and exclude from all `fsv_generate` commands with the same zone are combined. The `-src_condition` and `-dest_condition` are only applied to the `fsv_generate` command they are included with, not others with the same zone.

This may cause additional assertions to fail since additional source are considered. For example, if assertion A1 above is proven and A2 is falsified when they have different zone names, they will both be falsified when they have the same zone name

22.14 Application Variables Specific to FSV

The following table lists the application variables that affect the behavior of FSV. They are set with the `set_fml_var` command.

Table 42: Application Variables For FSV

Variable	Default	Behavior
fsv_use_structural	true	Check if structural path exists before running formal engines
fsv_use_casesplit	true	Divide proof by finding intermediate propagtation points
fml_coi_reduction	true	Automatic model reduction
fsv_max_num_casesplit	10	Maximum number of parallel case splits
fml_auto_save	true	Save database on every property state change
seq_use_bg_for_critical_jobs	true	Use backup grid for large jobs that must complete in order to make progress
fml_enable_resume	false	Continue check_fv of bounded property at reached depth instead of starting at depth 0
seq_bmc_depth	100	Upper bound for BMC engine
fsv_composite_trace	False	Show reset trace for failing property

23

Functional Safety Application

Functional safety verification is an essential requirement for automotive SoC and IP designs. This App formally identifies and classifies faults based on observability or detectability criteria. Complementing fault simulation with Z01X™, the combined solution reduces the effort and time required to achieve the test coverage closure and meet functional safety objectives.

Getting Started • About VC Formal FuSA • Accessing FuSA Lab Example	**Debugging, Reviewing and Saving Results** • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving FuSA Faults* • *Saving and Restoring Sessions*
Setting Up and Running FuSA • *Enabling FuSA Application mode* • *Configuring and Loading FuSA Faults* • *Compiling Design* • *Setting up Clocks* • *Setting up Resets* • *Initializing Design* • *Reviewing Design Complexity* • *Verifying Design Setup* • *Generating FuSA Checks* • • *Running FuSa Controllability Analysis* • *Running FuSa Observability/Detectability Analysis* • *Running FuSa in TCL Mode*	**Getting Help on Commands** • *Setting FuSA Specific Application Variables* • *FuSA Tcl Commands*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress*	

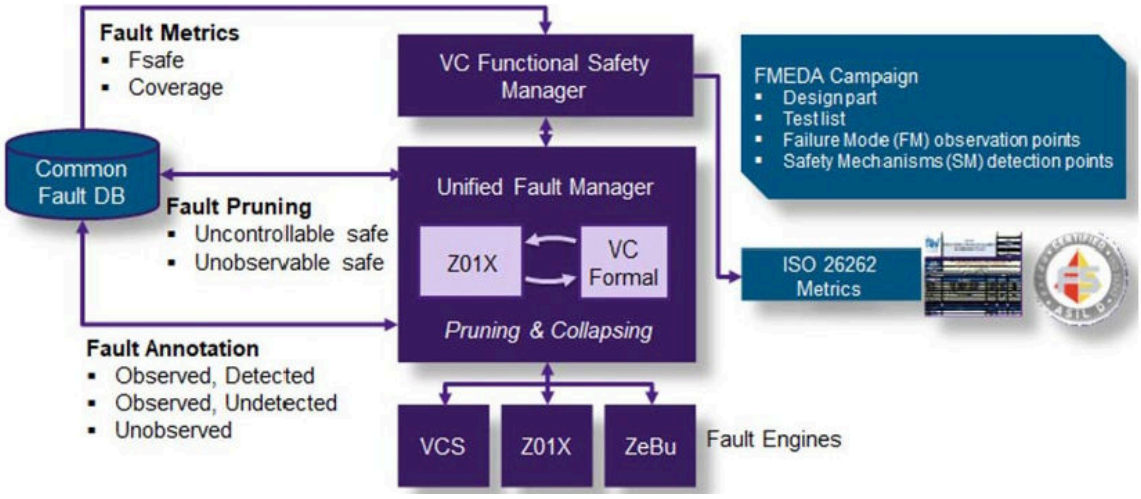
Feedback

23.1 Introduction

Functional Safety is defined as the absence of unreasonable risk due to hazards caused by malfunctioning of electrical/electronics systems. In other words, functional safety is about ensuring the safe operation of systems even when they go wrong.

The following diagram of Synopsys Functional safety verification flow.

Figure 225: Functional Safety Verification Flow



As part of the FMEDA (Failure Mode Effects and Diagnostic Analysis) campaign, Synopsys uses it's in-house tool Z01X, which is a model based Verilog software fault injection simulator for evaluating the effectiveness of safety mechanisms in automotive diagnostic tests, software test libraries, and applications. The simulation engine generates coverage results from the fault injection that can be used in calculating FMEDA metrics.

Z01X fault campaign however leaves the user with lots of NT NC NI NA ND NS NO type of faults which need manual reviews to rule out if these faults are actually safe or dangerous. Performing the safety analysis of the final 5% such faults may take 80% of the effort. Manually classifying remaining faults is very time consuming, error prone, and effort-intensive process.

To solve this issue, VC Formal has introduced the FuSa (Functional Safety) application which provides a solution for performing the controllability and observability analysis of the faults. This helps in fault reduction by detecting which faults are not controllable and not observable, and such faults can be marked as safe.

Note:

All VC Formal applications require specific license keys. For more information on license keys required for each of the VC Formal applications, see the VC Static Platform Installation Notes.

Feedback

23.1.1 Functional Safety Analysis

VC Formal provides the following types of analysis for the FuSa application.

- Structural Analysis
- Controllability Analysis
- Observability Analysis
- Detection Analysis

Structural Analysis (COI-based Fault Pruning)

In the structural analysis mode, VC Formal FuSa determines which faults are not in the COI of observation/detection points and marks them as untestable. Faults which are found to be in the COI of observation/detection points are passed through for further analysis.

Any changes to observation and detection points post invocation of structural analysis should be followed by rerunning structural analysis.

Note:

This step being purely structural scales up well from IP to SoC level.

The VC Formal FuSa structural check is done in two phases. The first phase is for observation points and the second phase is for detection points. The tool marks a fault 'UU' in the following scenarios:

- When both the observation and detection points are present, and the fault is outside the COI of both observation and detection points.
- When both observation and detection points are present, and the fault is inside the COI of both observation and detection points, but there is no structural path for propagation due to user specified constants.
- When no observation points are present and fault is outside COI of detection points.
- When no observation points are present, and the fault is inside COI of detection points, but there is no structural path for propagation due to user specified constants.
- When no detection points are present, and the fault is outside COI of observation points.
- When no detection points are present, and the fault is inside COI of observation points, but there is no structural path for propagation due to user specified constants.

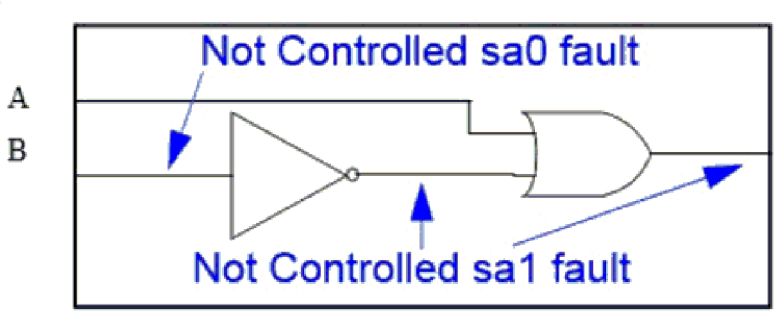
Controllability Analysis

Location of a fault is said to be non-controllable if the signal on the location of the fault cannot transition from one known state to another known state.

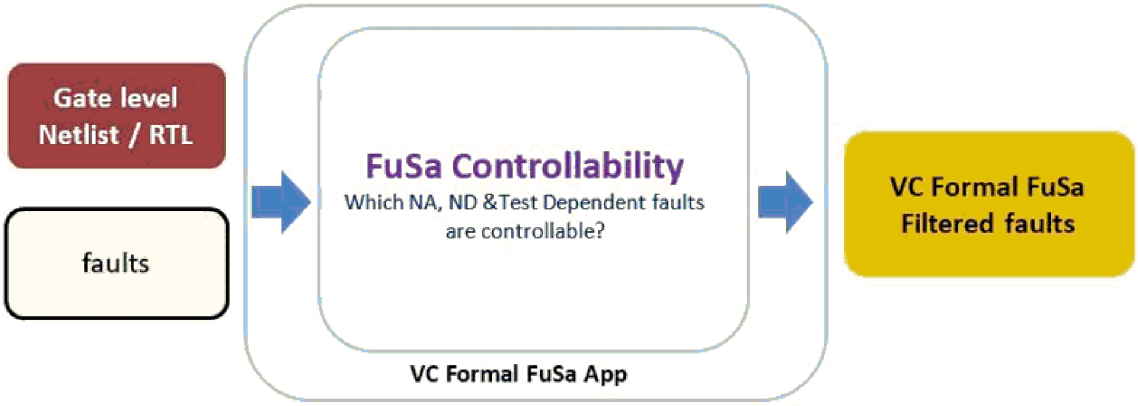
Feedback

Consider the example in Figure 226. If input B stays at 0 throughout, then the faults sa0 and sa1 as shown in Figure 226 are non-controllable.

Figure 226: Controllability Analysis Example



For performing controllability analysis, VC Formal FuSa creates cover properties for each fault location. Cover properties which are found to be unreachable, marks the corresponding fault as non-controllable. Cover properties which are covered or inconclusive (because of resource limits) are fed through for observability analysis.



Faults from Z01X

CELL	NO	0	test.risc1.pgmctr.m5.in1
CELL	NC	0	test.risc1.pgmctr.m5.in0
CELL	NC	0	test.risc1.pgmctr.d5.d
CELL	NO	0	test.risc1.pgmctr.m4.in1
CELL	--	0	test.risc1.pgmctr.m1.in1
CELL	--	0	test.risc1.pgmctr.m5.sel
CELL	NC	0	test.risc1.mux2x5.b[4]
CELL	ND	0	test.risc1.pgmctr.cnt[5]
CELL	NC	0	test.risc1.pgmctr.m3.in1
CELL	--	0	test.risc1.pgmctr.m2.in1

VC Formal Filtered

CELL	NO	0	test.risc1.pgmctr.m5.in1
CELL	UT	0	test.risc1.pgmctr.m5.in0
CELL	UU	0	test.risc1.pgmctr.d5.d
CELL	NO	0	test.risc1.pgmctr.m4.in1
CELL	--	0	test.risc1.pgmctr.m1.in1
CELL	--	0	test.risc1.pgmctr.m5.sel
CELL	NC	0	test.risc1.mux2x5.b[4]
CELL	ND	0	test.risc1.pgmctr.cnt[5]
CELL	UT	0	test.risc1.pgmctr.m3.in1
CELL	--	0	test.risc1.pgmctr.m2.in1

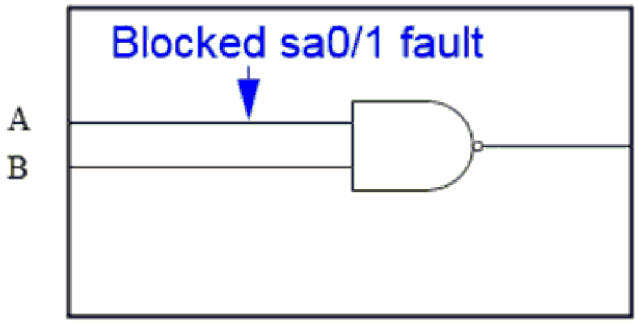
Feedback

Observability Analysis and Detection Analysis

Observability analysis indicates that the faults cannot be observed because the workload(s) blocked the activity of that fault location from propagating to an observation point. Such faults are created when a dominant signal to a gate never toggle, and stays at 0 or 1 throughout (depending on the gate).

Consider the example in Figure 227. If input B stays at 0 throughout, then the above sa0/1 fault is non-observable at the observation/detection points.

Figure 227: Observability Analysis and Detection Analysis

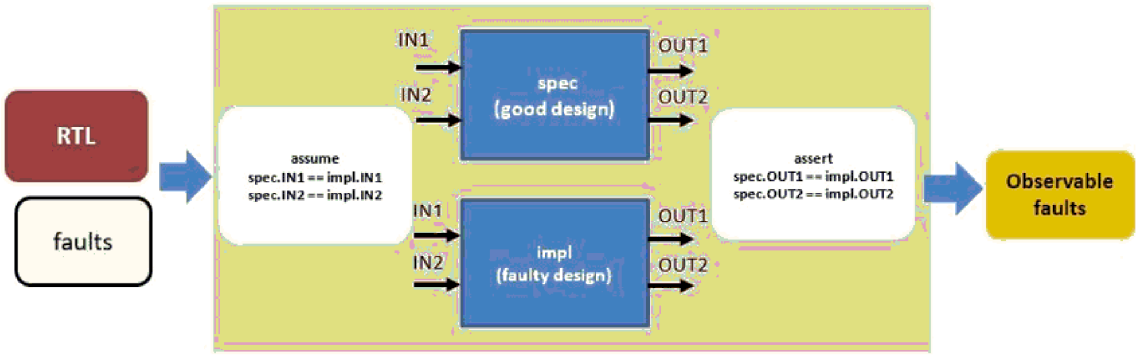


VC Formal FuSa performs comparison under the hood between the good and the faulty design to see if the faults can be propagated or observed at the observation or detection points. If the faults do not propagate, they are said to be safe and are marked in the database accordingly. But, if faults propagate to these points they can be marked with the relevant status based on individual customer policy.

Note:

When you use the same observation and detection points, the tool performs the observation and detection analysis in parallel at the same time.

Feedback



Faults from Z01X

CELL	NO	0	test.risc1.pgmctr.m5.in1
CELL	UT	0	test.risc1.pgmctr.m5.in0
CELL	UU	0	test.risc1.pgmctr.d5.d
CELL	NO	0	test.risc1.pgmctr.m4.in1
CELL	--	0	test.risc1.pgmctr.m1.in1
CELL	--	0	test.risc1.pgmctr.m5.sel
CELL	NC	0	test.risc1.mux2x5.b[4]
CELL	ND	0	test.risc1.pgmctr.cnt[5]
CELL	UT	0	test.risc1.pgmctr.m3.in1
CELL	--	0	test.risc1.pgmctr.m2.in1

VC Formal Filtered

CELL	ON	0	test.risc1.pgmctr.m5.in1
CELL	UT	0	test.risc1.pgmctr.m5.in0
CELL	UU	0	test.risc1.pgmctr.d5.d
CELL	NO	0	test.risc1.pgmctr.m4.in1
CELL	--	0	test.risc1.pgmctr.m1.in1
CELL	--	0	test.risc1.pgmctr.m5.sel
CELL	NC	0	test.risc1.mux2x5.b[4]
CELL	UB	0	test.risc1.pgmctr.cnt[5]
CELL	UT	0	test.risc1.pgmctr.m3.in1
CELL	--	0	test.risc1.pgmctr.m2.in1

23.2 Running FuSa with VC Formal

VC Formal Functional Safety Application (FuSa) app is designed to work seamlessly with different Functional Safety Solutions being offered by Synopsys.

Our recommendation is to run VC Formal FuSa for formal friendly blocks upfront before taking it to the simulation fault injection testing using Z01X. This will give an initial boost in diagnostic coverage by finding safe faults upfront while the simulation environment is still being developed.

For complex blocks or gate level designs, you can choose to run structural analysis upfront before taking it through simulation-based fault injection testing. This can help you in the following two ways:

1. In case you see a lot of faults marked to be structurally not observable, you should evaluate the observation points and see if something is missing
2. In case your observation points are correct, the faults marked structurally not observable can be excluded from simulation fault injection testing and save you significant effort.

Feedback

Once you are done with the simulation fault injection testing and have attained good diagnostic coverage, before coding additional patterns for the last mile coverage, use VC Formal and find the faults that are formally safe and does not need any analysis.

The following is the work flow for using FuSa application in VC Formal.

Note:

The filtered fault list can be saved after every step of VC Formal FuSa analysis to be consumed by Z01X.

23.2.1 Methodology

1. Prerequisite for FuSa application is to have a violation free basic formal setup with all the needed clocks, reset and constraints for the correct functioning of the design under consideration. To make sure the design is free from any violation, run the following command in FPV mode and clean any reported violation:

   a. Run the check

   ```
   check_fv_setup -block
   ```

   b. Report the violations

   ```
   report_fv_setup
   ```

   The `check_fv_setup` and `report_fv_setup` commands should only be used in the initial stages of the design being setup. They should be commented out during regular VC Formal FuSa usage as they have a non-trivial performance impact on the tool run.

2. Fault definition

   Fault database is the second requirement for setting up VC Formal FuSa flow. Fault definition in the form of Standard Fault Format (SFF) or Fault database (FDB) needs to be provided to VC Formal for fault instrumentation.

   Standard Fault Format or SFF file defines the fault type, fault location, and fault location type in the following format.

   ```
   FaultGenerate
   {
   NA [0,1] { VARI "chip_top.**" }
   NA [0,1] { PORT "chip_top.**" }
   }
   ```

   For more details on SFF, see the *Z01X user guide*.

   All the fault related data is managed in a Unified Fault Database (FDB). The FDB is organized by the FMEDA projects and each project holds fault campaigns, which are

Feedback

executed to measure the results for the Failure Modes. A fault campaign represents the full definition on what to execute and the results of the fault qualification. The FDB provides a common understanding of faults and fault statuses.

For more details on FDB, see the *VC Functional Safety Manager User Guide*.

Note:

The Unified Functional Safety Flow tools such as Synopsys VC FSM, ZOIX, VCS, vc_fcc, and VCF must have the same tool version to function correctly.

3. Constraints

   VC Formal retrieves the constraints available in the FDB. The constraints currently supported are of fixed value or constants, that is, “top.i1.a == 1’b0” or “top.i2.c == 2’b11”.

4. Observation and detection points

   In addition to design and fault database, you should provide the list of design nodes where these faults are expected to be observed or detected.

5. Tcl setup for FuSa

   Since FuSa requires a clean FPV Setup, one can start building on top of FPV setup with the following modifications:

   a. Defining FuSa appmode

   ```
   set_fml_appmode FUSA
   ```

   b. Importing fault list

   Use the `fusa_config` command to import the fault list (.sff) and unified fault database (FDB). This command must be used before reading the design.

   Syntax

   ```
   vcf> fusa_config -help
   Usage: fusa_config    # Configures VCST for the FuSa flow.
   [-sff <filename>]      (Name of the input SFF file)
   [-fdb <name-of-fault-db>] (Name of the fault database)
   [-fdb_path <path-to-fdb-project>] (Path to fault database)
   [-fdb_server <fault-db-server>] (Server for the fault database)
   [-fdb_campaign <name-of-fault-campaign>] (Name of the fault
    campaign)
   [-status <list-of-statuses>] (Status of faults which need to be
    explored using formal)
   [-dut_path <dutpath>]  (Path of dut in sff/fdb)
   [-save_fdb_incr <save_fdb_incr>] (Saves results incrementally in FDB
    when Formal analysis is ongoing. Units in minutes(M) or hours(H)
    (e.g. 10M, 12H). Minimum value should be 10M.)
   ```

Feedback

At least one of the options `-sff` or`-fdb` is mandatory for the `fusa_config` command. You cannot specify multiple `fusa_config` commands in the sff flow. You should use `fusa_clear` before using the fusa_config again in the sff flow.

Where,

- `-sff <sff_file>`: Specify the path of the SFF file/files to be read in. For example, `fusa_config -sff {promotion_table.sff in.sff}`
- `[-fdb <name-of-fault-db>]`: Specify the name of the saved fault database.
- `[-fdb_path <path-to-fdb-project>]`: Specify the path of the saved fault database. Specify this option when you are using the fdb flow. An error message is reported when `-sff/-fdb_server` is used along with the`-fdb_path` option.
- `-fdb_server <server_name:port>`: Specify the path of the fault database server.
- `-fdb_campaign <name-of-fault-campaign>`: Specify the name of the fault campaign being used in FDB.
- `-status <list-of-status>`: List of statuses to be used for formal analysis. You can to specify Fusa status groups as well. All inputs are validated in `fusa_generate` and if a status group was found in it will be replaced with relevant group's statuses.
- `-dut_path <dut-path>`: Map the name of dut path to that of SFF/FDB.
- `[-save_fdb_incr <save_fdb_incr>]`: Specify the time period (in minutes (M) or hours(H)) at which the results should be incrementally saved in the FDB when the Formal analysis is performed. The minimum value should be 10M.

Note:

The `fusa_config -save_fdb_incr`command is enabled by default, with the value of 1H. To disable the switch, use the `fusa_config -save_fdb_incr 0` command.

VC Formal FuSa does not support **.db* file in the SFF flow. the **.db*file is supported only in the FDB flow.

Table 43 shows the fault-flow support matrix.

Table 43: Fault-Flow Support Matrix

Fault Type	Structural Mode	Control Mode	Observe Mode	Detect Mode
Stuck-At 0	Supported	Supported	Supported	Supported
Stuck-At 0	Supported	Supported	Supported	Supported

Feedback

Fault Type	Structural Mode	Control Mode	Observe Mode	Detect Mode
Transient	Supported	Unsupported	Unsupported	Unsupported

Use Models:

Based on the source of fault injection points there can be following use models:

- Importing faults from SFF.

  Faults are defined in Standard Fault Format.

```
FaultGenerate FM1
{
    # Create faults on all ports in hierarchy
    NA [0,1] {PORT "top.**" }
    NA [0,1] {WIRE "top.**" }
    NA [0,1] {VARI "top.**" }
    NA [0,1] {ARRY "top.**" }
```

Fault definitions

  To import the faults from SFF, use the following `fusa_config` command:

  Syntax

```
fusa_config      -sff <sff-file-name>
[-status <list-of-status>]
```

  Even if the faults are imported from an SFF file, the `fusa_config` command generates an FDB internally.

  If you wish to use the FuSa Legacy SFF Flow, wherein an FDB is not generated internally, set the `fusa_legacy_sff_flow` formal variable to true.

```
set_fml_var fusa_legacy_sff_flow true
```

  This video explains how to selectively load faults from a Standard Fault Format (SFF) file in the VC Formal FuSa application. The process involves filtering faults based on fault types (stuck-at-0 or stuck-at-1) or design node types (for example, flops, ports, arrays, and nets). The demonstration uses a memory controller block, focusing on the address selection unit. Observation points include data and acknowledgment signals, while the error signal serves as the detection point.

- Importing faults from Unified Fault Database (FDB)

  Another possible way of specifying faults is through FDB. To import faults from FDB use the following `fusa_config` command:

  Syntax

  Using global FDB located on a server

Feedback

```
fusa_config -fdb <unified_fault_db>
                -fdb_server <server_name:port>
                -fdb_campaign <name-of-fault-campaign>
                [-status <list-of-status>]
                [-dut_path <dut-path>]
```

Using local FDB

```
fusa_config -fdb <unified_fault_db>
                -fdb_server <server_name:port>
                -fdb_campaign <name-of-fault-campaign>
                [-status <list-of-status>]
                [-dut_path <dut-path>]
```

Example

```
fusa_config -fdb_campaign <campaignName> -dut_path <top_module>
-fdb_path <path different to cwd>
```

An information message is reported when the SNPS_FDB_STORAGE_PATH environment variable is already specified:

[Info] FDB_STORAGE_PATH_ENV_USE: The switch -fdb_path is not specified with fusa_config command. Hence proceeding with the env var SNPS_FDB_STORAGE_PATH which has been set in the environment for the fdb path

Note:

If you are switching from the sff flow to the fdb flow to connect user created fdb to load new fault list, the -fdb_path must be specified in fusa_config, even if the fdb is present in the current working directory.

c. Mapping fault statuses

The *Internal Formal Status* as mentioned in Table 44 are mapped to a set of default Z01X statuses automatically for better usability. However, you can change the mapping of these internal statuses to suit your requirements using the `fusa_map_status` command.

By default, the default Z01X statuses are mapped to the new statuses (see *Status Codes Starting T-2022.06-1 Onwards* in Table 44) when you use the *fusa_map_status*command.

To map the default Z01X statuses to old statuses (see *Status Codes Prior to T-2022.06 Release* inTable 44), set the `fusa_disable_new_inconcl_statuses` formal variable to true. By default, the `fusa_disable_new_inconcl_statuses` formal variable is set to false.

Feedback

Table 44: FuSA Fault Statuses Mapping

Description	Internal Formal Status	Status Codes Prior to T-2022.06 Release	Status Codes Starting T-2022.06-1 Onwards
Formal no result	FNR	NA	NA
Structural Inconclusive	SIN		VV
Controllability Inconclusive	CIN		VV
Controllability Controllable	CCC		CC
Observability Inconclusive	OIN		XX
Detectability Inconclusive	DIN		XX
Observability observed	OOB	OO	OX
Detectability not detected	DND	DN	XF
Detectability detected	DDT	DD	XD
Observable and not-detected	OND	ON	OF
Structurally not observed not detected	SSF	UU	UU
Structurally not observed	SNO	UU	IV
Structurally not detected	SND	UU	VI
Controllability Non-controllable	CNC	UT	UT
Observability non-observable	ONO	UB	UB
Non-observable and detected	NOD	ND	UB
Observable and detected	ODT	OD	OD
Non-observable and not detected	NON	NN	UB

```
vcf> fusa_map_status -help
Usage: fusa_map_status    # Map an internal FuSa status to SFF
 status (Values:
        [-sff_status sffStatus]
                                (SFF status value)
        [internalStatus]        (Internal FuSa status to map:
                                 Values: FNR, SIN, SSF, SNO, SND,
 CNC,
```

Feedback

```
                                    CCC, CIN, ONO, OOB, OIN, DND, DDT,
DIN,
                                    NOD, NON, ODT, OND)
```

d. Reading design

   The RTL design is compiled using the `read_file` or `analyze` / `elaborate` commands. For more details, see section “Compiling Designs”.

   Safety properties in FuSa are not specified in SVA, therefore, the `-sva` option of `read_file` and `elaborate` command is generally not used. Use the `report_link` command after reading the design to ensure no modules are missing.

e. If modules or instances are structurally black boxed using the `set_blackbox` command, specify the `fusa_blackbox` command to avoid invalid proofs. The `fusa_blackbox`command should be specified after elaboration just before the `fusa_generate` command. It creates a safe black box abstracted model that allows data to propagate through the black box, thus avoiding false proofs.

```
vcf> fusa_blackbox -help
Usage: fusa_blackbox    # Create a blackbox for functional safety
 analysis in FuSa App.
       [-module <module-list>]
                                (Modules to be FuSa blackboxed)
       [-all]                   (Automatically generate paths from
 inputs to outputs for all blackboxed instances)
       [-all_auto_path]         (Automatically generate paths from
 the inputs to the outputs)
       [-no_auto_path]          (Do not automatically generate the
 path from the inputs to the outputs)
       [-seq_path]              (Enforce sequential propagation paths
 from the inputs to the outputs)
       [-comb_path]             (Enforce combinational propagation
 paths from the inputs to the outputs)
       [-ignore_unresolved]     (Ignore unresolved instances)
       [-data_path_only]        (Block the propagation through the
 mux select signals, but allow the propagation from the true and
 false branches of a mux)
       [-remove]                (remove the instance from FuSa
 blackboxing)
       [-task <task-name>]      (Name of the failure mode task to be
 FuSa blackboxed)
       [<instance-list>]        (List of FuSa blackbox instance
 names)
```

- `[-module list-of-modules]`: Specify the list of the module names
- `[-all]`: Create secure blackbox for all structural blackboxes
- `[-all_auto_path]`: Generate paths from the inputs to the outputs. Default

Feedback

- `[-no_auto_path]`: Do not automatically generate the path from the inputs to the outputs
- `[-seq_path]`: Enforce sequential propagation paths from the inputs to the outputs
- `[-comb_path]`: Enforce combinational propagation paths from the inputs to the outputs
- `[-ignore_unresolved]`: Do not create an `fsv_blackbox` for unresolved instances.
- `[-data_path_only]`: Block the propagation through the mux select signals, but allow the propagation from the true and false branches of a mux
- `[-remove]`: Specify this option to remove the instance from security blackboxing.
- `[-task <task-name>]`: Specify the failure mode task name in which you want to place the black box. This option is only supported when there are multiple failure modes.
- `[<instance-list>]`: Specify the instance name for which you want to create an abstract model.

  Note:
  The FuSa black boxes are not reported until the `fusa_generate` command has been executed.

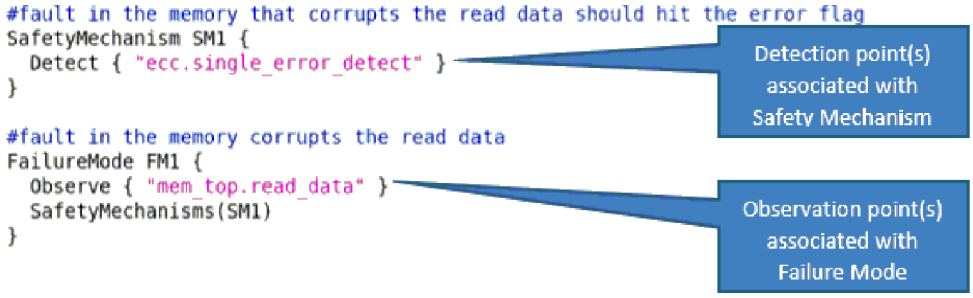
  For more information on black boxing after design read, see section “Black Boxing and Security Verification”.

f. Specifying Observation or Detection Points

You can specify the observation or detection points in the SFF or TCL.

Observation point specification in SFF:

Feedback



In the case of the FDB flow, the observation and detection points are automatically created if they are mentioned in the original SFF used to create the FDB.

VC Formal automatically extracts and performs FuSa checks on the specified observation/detection points. User also has flexibility to update the Observation and detection points using `fusa_observation` / `fusa_detection` TCL command.

Syntax

fusa_observation

```
fusa_observation        # Add/delete observation points
    [-add]               (Add the observation points)
    [-delete]            (Delete the observation points)
    [-file filename]     (File containing a list of signal names)
    [list-of-signals]    (List of signal names)
```

fusa_detection

```
fusa_detection           # Add/delete detection points
     [-add]               (Add the detection points)
     -delete]             (Delete the detection points)
     [-file filename]     (File containing a list of signal names)
     [list-of-signals]    (List of signal names)
```

g. Generating Properties for Running in VC Formal FuSa

Use the `fusa_generate` command to extract the faults and define the associated properties which you want to verify in VC Formal FuSa. There can be two different use models here:

- There can be multiple faults defined for same signal name but different object classes. By default, VC Formal FuSa honors each of these as different faults and honor each object class.

  To disable this support, set `fusa_multiple_fault_types_signal` to false.

  ```
  set_fml_var fusa_multiple_fault_types_signal false
  ```

Feedback

- When `fusa_multiple_fault_types_signal`is set to false, only the first fault with a given signal name is considered. Other faults with the same signal, but different object class are ignored.

  You run on the faults specified through the sff file or the unified fault database given with `fusa_config` command. In this case, you should not specify any options in the `fusa_generate` command.

6. When importing faults from the FDB, you can use multiple `fusa_config` and `fusa_generate` combinations as follows. Using these combinations enables you save time on `read_file`/`analyze` commands for reading a design.

```
fusa_config <options1>
fusa_generate <options2>
….
fusa_clear
fusa_config <options3>
fusa_generate <options4>
```

   The `fusa_clear` command enables you to clear all the faults and related properties. The `fusa_clear` command also enables you to rerun the `fusa_config` command before `fusa_generate`.

```
fusa_config <options1>
fusa_clear
fusa_config <options2>
fusa_generate <options3>
```

   When importing faults from an SFF file, the `fusa_clear` command allows you to change the SFF file again until reading the design.

```
fusa_config -sff file1.sff
fusa_clear
fusa_config -sff file2.sff
read_file <options1> / analyze <options2> + elaborate <options3>
fusa_generate <options4>
```

   Note:

   Specifying an SFF file with the `fusa_config` command after reading the design will result in an error.

   If the FuSa legacy SFF flow is used, issuing the `fusa_clear` command after reading the design will result in an error.

   Using any of fusa_config or fusa_generate again without a fusa_clear in between will result in an error.

Feedback

7. By default, FuSa supports faults over user clock and reset signals for structural analysis. The clock and reset faults are ignored in the controllability, observability and detectability analysis.

   To disable faults over user clock and reset signals for structural analysis, set the following variable to false.

   ```
   set_fml_var fusa_enable_clk_rst_structural false
   ```

   The following warning message is reported in the `fusa_generate` command if the `fusa_enable_clk_rst_structural`is set to false, and if clock or reset signal found in provided fault list.

   [Warning] FUSA_CLK_RST_ONLY_STRUCTURAL: Clock or reset faults present. They will only be considered during the structural analysis, and will be ignored in the controllability, observability and detectability analyses.

   Use the `fusa_get_ignored_signals` command is used to examine the set of signals that have been thus ignored.

   Syntax

   ```
   vcf> fusa_get_ignored_signals -help
   Usage: fusa_get_ignored_signals  # Get the set of clk/rst/skipped
    signals which have been ignored
   for inducing faults.
   [-clock]       (Outputs only the ignored clock signals)
   [-reset]       (Outputs only the ignored reset signals)
   [-skipped]     (Outputs only the skipped signals)
   ```

8. You can define faults in VC Formal using the switches in the `fusa_generate` command.

   Syntax

   ```
   fusa_generate -help
   Usage: fusa_generate  # Generate all the faults and related properties
    for the given scopes
   [-scopes list-of-scopes] (List of scopes to generate faults)
    [-excludes list-of-excludes] (List of scopes to exclude while
    generating faults)
   [-exact] (Match the patterns exactly. Do not consider partial matches)
   [-fault_type fault-type] (Type of fault. Values: st0, st1, both,
    transient, all: Values: st0, st1, both, transient, transient_0,
    transient_1, all)
   [-analysis list-of-analysis] (List of type of analysis to be
    performed. Values: structural, control, observe, detect: Values:
    control, detect, observe, structural)
   [-verbose] (Produce verbose outputs)
   [-types list-of-types] (List of types of design node type. Values:
    WIRE, PORT, VARI, ARRY: Values: WIRE, PORT, VARI, ARRY)
   ```

Feedback

You can select and filter faults using the `-scope` and `-exclude` option. These options inject the faults in the scope specified and ignores the faults specified in the -exclude option.

```
fusa_generate -scopes <list-of-scopes> -excludes <list-of-excludes>
```

Example

Let the list of faults generated by a plain `fusa_generate` command be the following.

```
NA |          0 |           7 |         st0 |          0 |    unknown |
   alureg.d1.qb
NA |          1 |           8 |         st1 |          0 |    unknown |
   alureg.d1.qb
NA |          2 |          17 |         st0 |          0 |    unknown |
   alureg.d2.qb
NA |          3 |          18 |         st1 |          0 |     unknown|
   alureg.d2.qb
NA |          4 |          27 |         st0 |          0 |    unknown |
   alureg.d3.qb
NA |          5 |          28 |         st1 |          0 |    unknown |
   alureg.d3.qb
NA |          6 |          37 |         st0 |          0 |    unknown |
   alureg.d4.qb
NA |          7 |          38 |         st1 |          0 |    unknown |
   alureg.d4.qb
NA |          8 |          47 |         st0 |          0 |    unknown |
   alureg.d5.qb
NA |          9 |          48 |         st1 |          0 |    unknown |
   alureg.d5.qb
NA |         16 |          87 |         st0 |          0 |    unknown |
  instreg.d1.qb
NA |         17 |          88 |         st1 |          0 |    unknown |
  instreg.d1.qb
NA |         18 |          97 |         st0 |          0 |    unknown |
  instreg.d2.qb
NA |         19 |          98 |         st1 |          0 |    unknown |
  instreg.d2.qb
NA |         20 |         107 |         st0 |          0 |    unknown |
 instreg.d3.qb
NA |         21 |         108 |         st1 |          0 |    unknown |
  instreg.d3.qb
NA |         22 |         117 |         st0 |          0 |    unknown |
  instreg.d4.qb
NA |         23 |         118 |         st1 |          0 |    unknown |
  instreg.d4.qb
NA |         24 |         127 |         st0 |          0 |    unknown |
  instreg.d5.qb
NA |         25 |         128 |         st1 |          0 |    unknown |
  instreg.d5.qb
NA |         32 |         177|         st0 |          0 |    unknown |
   pgmctr.d2.qb
```

Feedback

```
NA |         33 |         178 |         st1 |          0 |     unknown |
   pgmctr.d2.qb
NA |         34 |         187 |         st0 |          0 |     unknown |
   pgmctr.d3.qb
NA |         35 |         188 |         st1 |          0 |     unknown |
   pgmctr.d3.qb
NA |         36 |         197 |         st0 |          0 |     unknown |
   pgmctr.d4.qb
NA |         37 |         198 |         st1 |          0 |     unknown |
   pgmctr.d4.qb
NA |         38 |         207 |         st0 |          0 |     unknown |
   pgmctr.d5.qb
NA |         39 |         208 |         st1 |          0 |     unknown |
   pgmctr.d5.qb
```

When you use the following command, the list of faults generated are as follows:

```
fusa_generate -scopes {alureg pgmctr} -excludes {alureg.d4 pgmctr.d1
 pgmctr.d5}
NA | 0 | 7 | st0 | 0 | unknown | alureg.d1.qb
NA | 1 | 8 | st1 | 0 | unknown | alureg.d1.qb
NA | 4 | 27 | st0 | 0 | unknown | alureg.d3.qb
NA | 5 | 28 | st1 | 0 | unknown | alureg.d3.qb
NA | 8 | 47 | st0 | 0 | unknown | alureg.d5.qb
NA | 9 | 48 | st1 | 0 | unknown | alureg.d5.qb
NA |32 |177 | st0 | 0 | unknown |  pgmctr.d2.qb
NA | 33|178 |  t1 | 0 | unknown | pgmctr.d2.qb
NA | 34|187 | st0 | 0 | unknown | pgmctr.d3.qb
NA |35  188 | st1 | 0 | unknown | pgmctr.d3.qb
```

23.3 Running FuSA Tcl command mode

FuSa can be run in Tcl command mode, GUI Mode, batch mode, or regression mode.

1. Setting up FuSa mode

   FuSa app has 4 different modes of run Structural, Control, Observe and Detect.

   These modes can be set using the following fml var

   ```
   set_fml_var fusa_run_mode <structural/control/observe/detect>
   ```

2. Running checks

   After setting the relevant `fusa_run_mode`,use the `check_fv` command to run the corresponding FuSa checks.

3. Reporting

   Use the `fusa_report` command to get a report of the command. The status of the faults reported by this command are untestable, not observed, observed and unknown.

Feedback

```
Usage: fusa_report    # Reports information about Formal Fault DB.
        [-list]                 (Outputs one-line report per fault.)
        [-verbose]              (Outputs detailed report per fault.)
        [-no_summary]           (Do not print summary report)
        [-observation]          (Outputs only the Observation points.)
        [-detection]            (Outputs only the Detection points.)
        [-fault_id <id>]        (Id of the fault. Filled from unified
 DB or auto generated)
        [-blackbox]             (Outputs a report for all the
 fusa_blackbox)
        [-fdb_report]           (Report Fault Database)
        [-file <file>]          (Name of the sff file in which to save
 the fdb report)
        [-path]                 (Outputs the blackbox paths from the
 inputs to the outputs)
        [-path_config]          (Outputs the user configuartion of
 blackbox paths)
        [-col]                  (return the collection of instances
 specified with fusa_blackbox)
        [-with_top]             (reporting with the top as the prefix)
        [-instance <list-of-instance-names>]
                                (The list of instance names to be
 reported)
        [-fault_category]       (Reports all the faults in various
 categories)
        [-fdb_version]          (Return the FDB version used by
 VC-Formal.)
```

The following is an example output of the `fusa_report` command:

```
--------------------------------------------------------------------------
--------------
Summary of FuSa information
--------------------------------------------------------------------------
--------------
#Faults           :      20
Status >
   #SF            :       0
   #UR            :      10
   #NA            :      10
Analysis >
   #structural    :       0
   #control       :      10
   #observe       :       0
   #detect        :       0
   #unknown       :      10
#Excluded         :       0
#Observations     :       2
#Detections       :       0
```

4. Generating output fault list

Feedback

After performing FuSa analysis in VC Formal, fault list in the form of SFF or fault database can be exported to be consumed in Z01X for further analysis or into VCFSM for FMEDA calculations.

```
vcf> fusa_save -help
Usage: fusa_save    # Save the current results to file or fault DB
        [-sff filename]        (Name of the input SFF file)
        [-fault_db path-to-fault-db]
                               (Path to the fault database)
        [-test fault-db-test-name]
                               (Name of the test to save in fault
 database)
        [-status <list-of-status>]
                               (Status of faults which need to be
 saved)
        [-all_failure_modes]   (Save current results of all failure
 modes)
```

a. Saving results to sff

```
fusa_save -sff <filename> [-status <list-of-status>]
```

b. Saving results to FDB

- Local FDB

  ```
  fusa_save [-status <list-of-status>]
  ```

- Global FDB hosted on a server

  ```
  fusa_save -fault_db <path-to-fault-db> [-status <list-of-status>]
  ```

c. Save the results for all failure modes

```
fusa_save -all_failure_modes
```

d. Select the test file to save the Formal FuSa analysis results in the fault database:

```
fusa_save -fault_db <path-to-fault-db>[-test fault-db-test-name]
```

Note:

The output fault format should be same as input fault format. If SFF is imported, then only an SFF can be exported and if an FDB is imported, then only an FDB can be exported. Mixing both these flows is not possible.

23.4 Performing FuSa in VC Formal GUI

You can perform FuSa analysis in the VC Formal GUI.

Feedback

To perform FuSa checks:

1. Switch to the FuSa Appmode either from the drop-down list in the Verdi toolbar or from the RMB menu of TaskList.

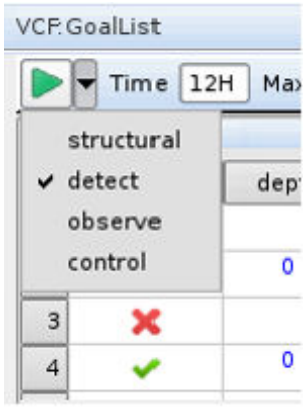
2. The FUSA taskList lists all the FuSa tasks.

   There are two progress bars for one FUSA task. The first line is summary for faults, and the second line is for summary of properties.

   In the example in the following figure, there are 100 faults and 300 properties in FUSA task, and none of them have been run.

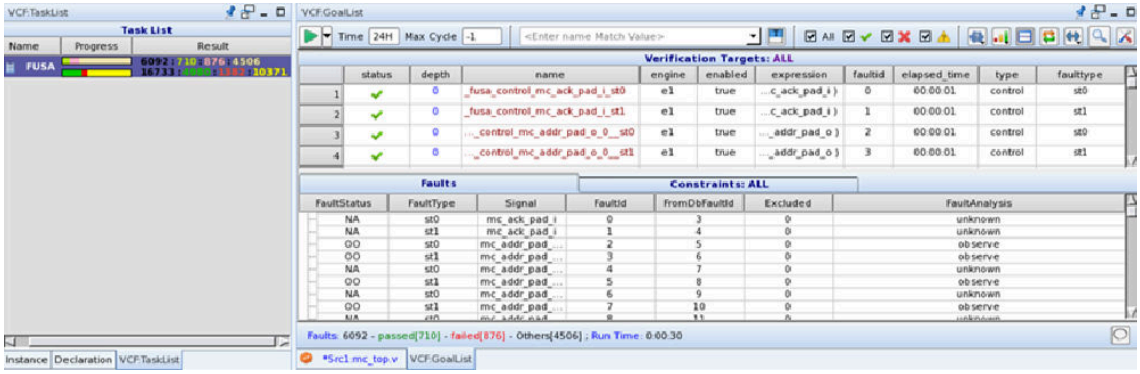
3. Click the drop-down menu of check_fv to select the type of analysis you want to perform on the faults.

Feedback



4. Click to run the FUSA checks.

Figure 228: VC Formal FuSa GUI



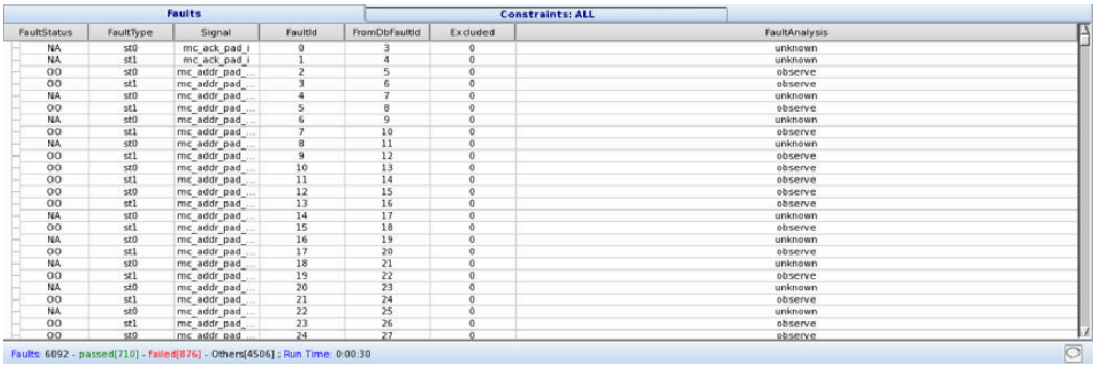
The GoalList lists all the properties in the specified FUSA task, and the *Faults* tab lists all the faults. Both the tables are updated as and when the checks are running.

- *Faults* tab

  The *Fault* tab contains all individual faults. The *Faults*tab displays all the faults imported and their details such as the FaultStatus, FaultType, Signal, and so on.

Feedback

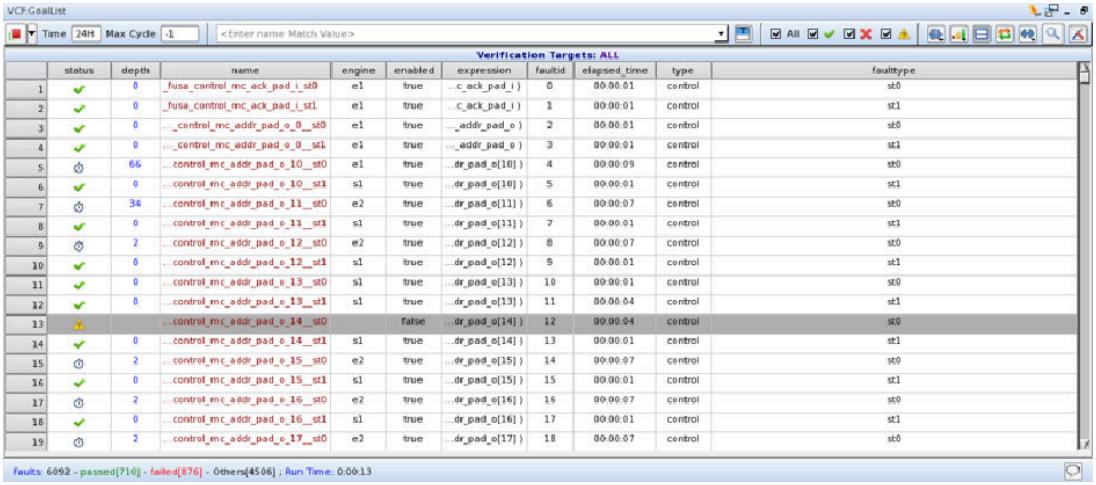
Figure 229: Faults Tab



- *Fault Properties*

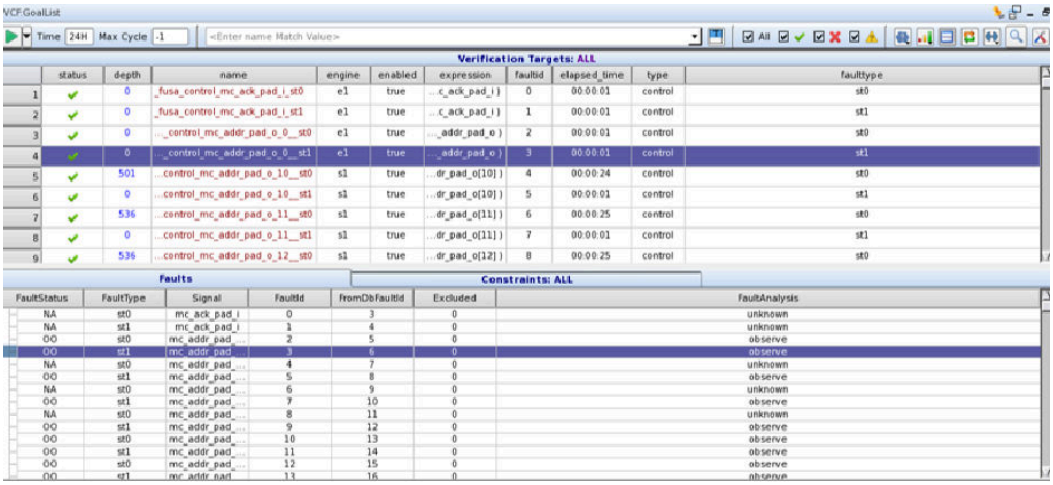
  For each fault, VC Formal automatically create properties in VC Formal property table, one for each controllability, observability and detectability.

Figure 230: Fault Properties Tab

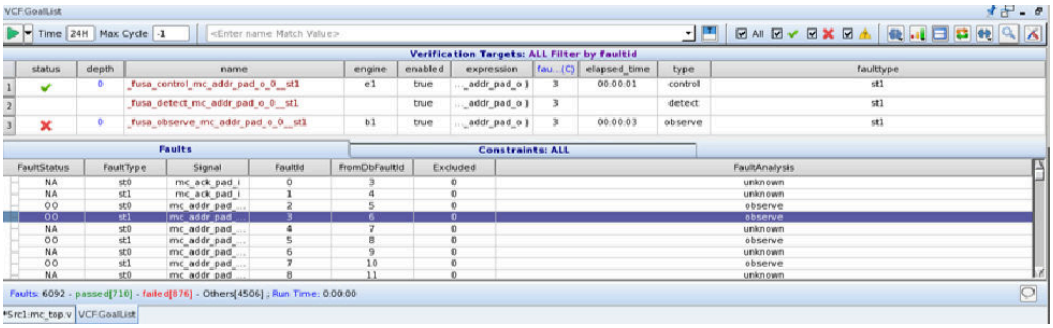


Feedback

- If you select one property in property table, the related fault is selected in the *Faults* table as shown in the following figure:



- Double-click the *Signal* column in *Faults* table, the property table is filtered, and shows the fault related properties in property table as shown in the following figure:



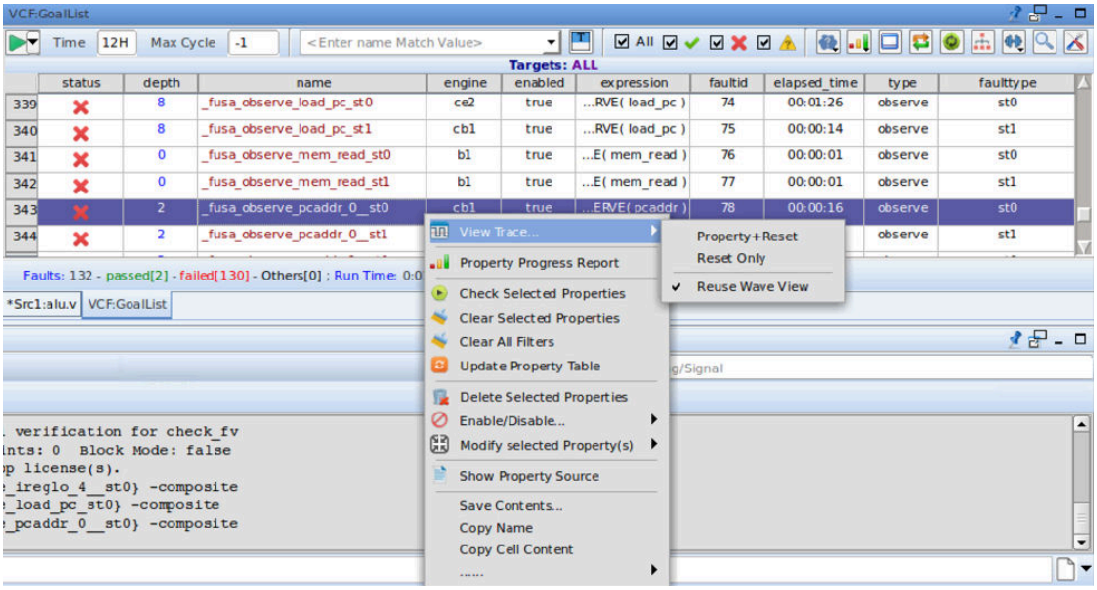
- Viewing the trace in GUI

  view_trace _fusa_detect_<signame>_st<0/1>

  The intended waveform should open with this command.

Feedback

Figure 231: Viewing Trace in GUI Mode



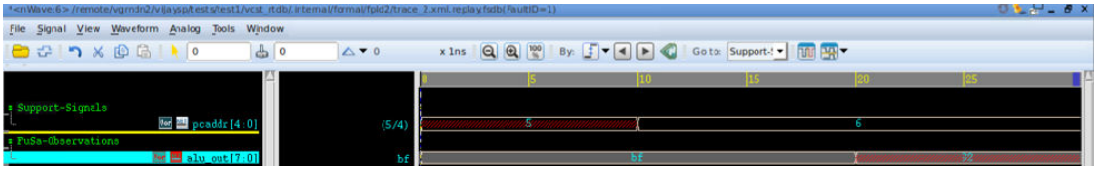
- The debug mechanism for VCF FuSa application is enhanced so that once tool gives falsification in the formal observability step, then in the CEX waveform, the debug tool shows the signal value in Good machine and highlight the parts where the signal value in faulty machine.

  In Formal detectability and observability analysis, the signals in the fan out of the faulty signal are checked for equivalence between good machine (GM) and faulty machine (FM). In Formal observability analysis, VC Formal compares the observation point values between GM and the FM. In either of these analysis, if there is a run where the values might differ, the analysis fails and produces a CEX.

  In the waveform, the values shown are as per the GM. The red shaded rectangles indicate the deviation are as per the FM. The values as per FM are not shown by default. Right-click on the signal and select *Add Fault Waveform* to show a separate signal with values as per FM.

  An example of the waveform in Formal FuSa is shown in Figure 232. The fault in this case is that the last bit of the signal *pcaddr* is stuck at 0 (see Figure 231). The value of *alu_out* starts differing in cycle 3.

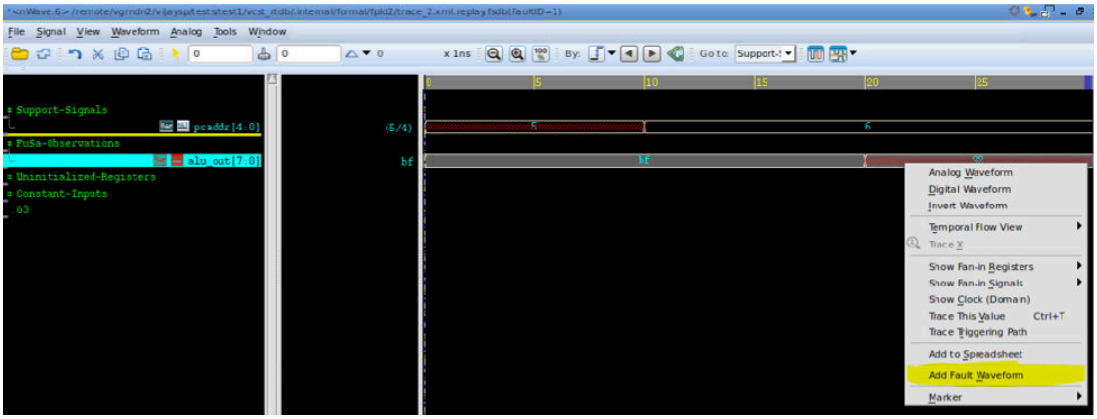
Figure 232: GM/FM Waveform in Formal FuSa



Feedback

As seen in Figure 233, the *alu_out* value as per GM is 0x92 in cycle 3. To look at the FM waveform, right-click on this waveform and select *Add Fault Waveform*, as shown in Figure 233.

Figure 233: Add Fault Waveform

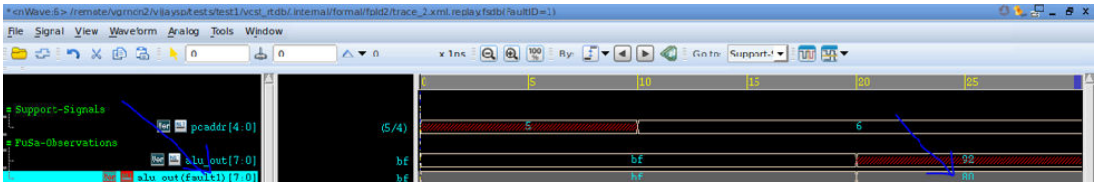


This results in showing the alu_out waveform as per the FM behavior, as shown in Figure 234. The signal name is suffixed with "(<fault_id>)" and as shown the value of the signal is 0x80 as per the FM behavior.

Note:

Double-click on signals of the Good machine to root-cause the issue in the source browser. This will contain values of both the good machine and bad machine.

Figure 234: FM Waveform



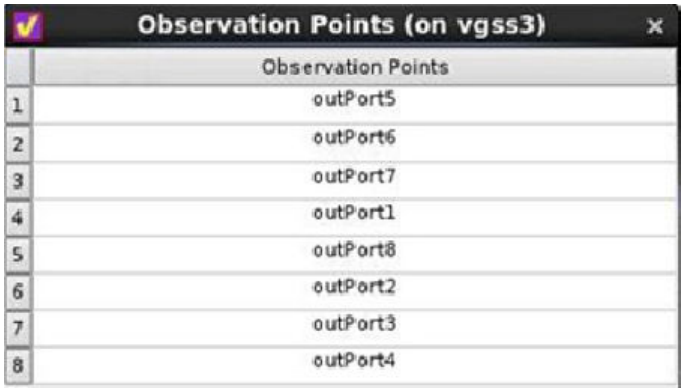
23.4.1 Viewing Observation Points and Performing Complexity Analysis

You can view all the observation points and perform complexity analysis for the observation points.

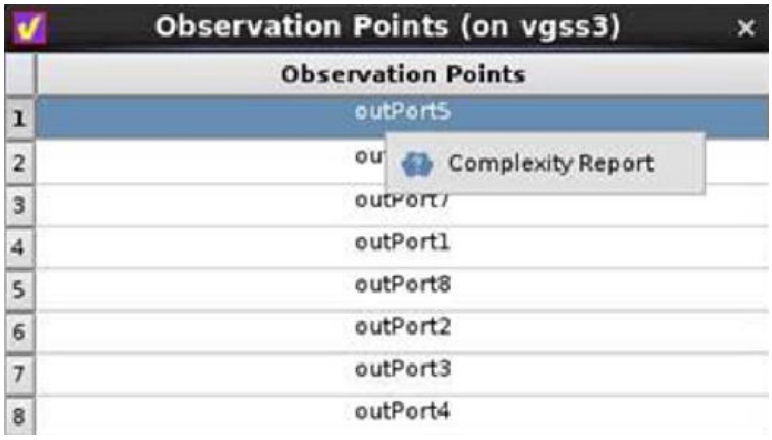
1. To view the observation points, Click in the GoalList tool bar.

   The *Observation Points* appears as shown in the following figure.

Feedback



2. To open the complexity report for the observation points, from the RMB menu of observation points, click *Complexity Report*.



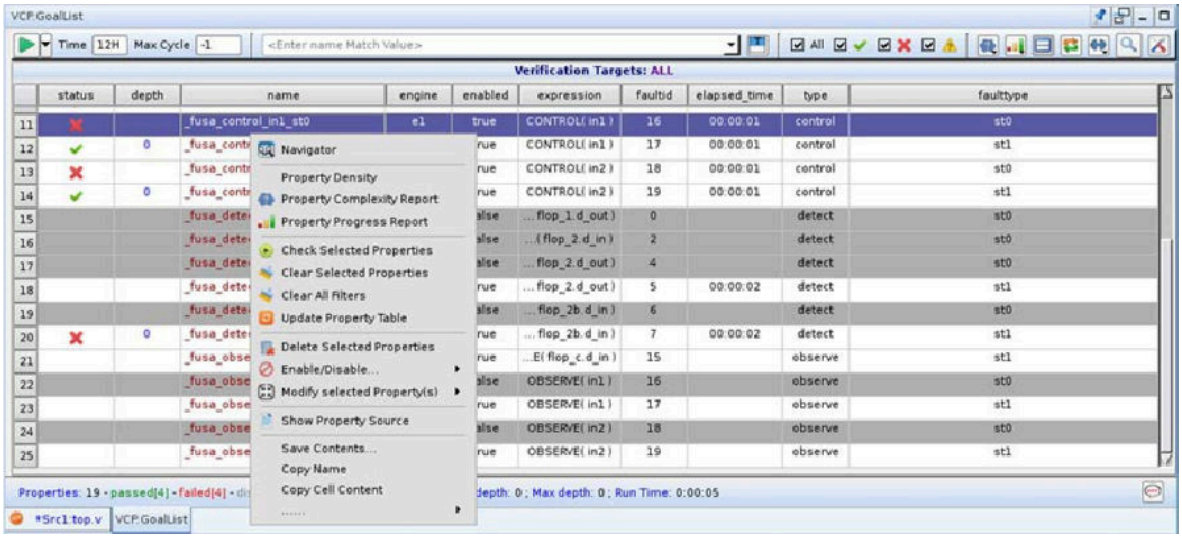
You can also select multiple observation points and select *Complexity Report*from RMB menu.

23.4.2 Performing RMB Menu Operations

You can perform all the RMB menu operations for the properties as shown in Figure 235. For more details on the RMB menu options, see section Debugging VC Formal Results in VC Formal GUI.

Feedback

Figure 235: RMB menu For Fault Properties



23.5 Running FuSA Analysis with VC FCM

VC Formal FuSA can also be invoked from VC FCM. VC FCM can be used a single platform to handle fault campaigns with different tools (VC Z01X, VC Formal FuSa). Using VC FCM as the interface for VC Formal FuSA and VC Z01X reduces the overhead of maintaining multiple SFFs, setups files, and run directories.

To run VC Formal FuSA analysis from VC FCM, start FCM with a Tcl command script file, and specify the path of the FDB.

```
vc_fcm -fdb_path <FDB dir> -tcl_script fcm.tcl
```

```
fcm::vcf    # Starts VC Formal in FCM shell
[-help]                    (Show this help)
[-f <filename>]            (TCL file name for 'vcf -f <tcl_file>')
[-lic_zoix]                (Use zoix license)
[-lic_elite]               (Use elite license)
[-max_time <string>]       (Maximum runtime for each requested stage, with
 time unit)
[-max_mem <string>]        (Maximum memory for each requested stage)
[-gen_model]               (Generate vcf model)
[-gen_checks]              (Generate checks)
[-map_status <filename>]   (File name for status mapping)
[-analysis <list>]         (List of analysis modes to generate checks)
[-verdi]                   (Open verdi)
[-status <list>]           (List of statuses)
[-top <string>]            (Top module name)
[-dut_path <string>]       (Hierarchical path to the DUT)
[-build_top <string>]      (Formal top module name if its different from
 testbench "top")
```

Feedback

```
[-vcs_cmd_line <string>] (VCS command-line arguments. Can be used to
 specify the VCS command line with -use_pre_compile_flow)
[-task <string>]         (Task name to set as current.)
[-regen_faults]          (Regenerate the faults)
[-check_structural]      (Run the 'structural' stage)
[-check_control]         (Run the 'control' stage)
[-check_observe]         (Run the 'observe' stage)
[-check_detect]          (Run the 'detect' stage)
[-fault_summary]         (Generate fault summary. Add '-summary' for
 summarized report)
[-report <string>]       (Options for fusa_report command. Add '-summary'
 for summarized report.)
[-save_fdb]              (Save to FDB)to invoke after task creation of cg
 in auto scale flow and before invoking check_fv.)
[-use_pre_compile_flow]  (Use pre-compile RTL flow.)
```

Sample fcm.tcl Script

The sample fcm.tcl script should contain the following details:

1. Connection to the FDB server.

   ```
   fdb_connect -fdb_path <FDB dir>
   ```

2. Commands to create fault database (FDB). You can also specify the path to an already existing FDB.

   ```
   create_campaign -args "-full64 -daidir simv.daidir -campaign <campaign
   name> -prune global -collapse 1 -sff <input SFF file> -dut <DUT top>"
   ```

3. Specify the VC Formal runtime configurations.

   ```
   vcf -max_mem 16GB
   vcf -max_time 12H
   ```

4. Specify the grid settings to perform the FuSA runs.

   ```
   set_submit_cmd -grid_type <> -cmd <grid control command> -task_type
   vcf
   ```

   Example

   ```
   set_submit_cmd -grid_type SGE -cmd "qsub -P bnormal -V -l
   arch=glinux,mem_free=4G" -task_type vcf
   ```

5. Compile and elaborate the design.

- For SV/VHDL designs:

  ```
  vcf -gen_model -top <DUT top> -build_top <Formal top> -dut_path <map formal top to DUT top>
  ```

  This would internally call `fusa_config` with FDB related option extracted from FCM, and `read_file` with vcs command line options extracted from the *simv.daidir*.

- SV/VHDL/mixed designs, pre-compile RTL flow can also be used.

  ```
  vcf -gen_model -top <DUT top> -build_top <Formal top> -dut_path <map formal top to DUT top> -use_pre_compile_flow -vcs_cmd_line <provide vcs options here>
  ```

Note:

You can only load faults with specified status at the fault regen stage in VC-FCM FuSA flow. For example, `vcf -regen_faults -status NN`

6. Generate FuSa related checks using the *-gen_check* option. The *-gen_check* option internally call the `fusa_generate` command and execute the Tcl file passed through -f

   ```
   vcf -gen_check -f ClockReset.tcl
   ```

7. Run COI Observability (structural analysis) and save the results back to FDB

   ```
   vcf -check_structural
   ```

8. Run VC Z01X.

   ```
   create_testcases -name {"test_result"} -exec "$::env(PWD)/simv" -args
    ""-fsim_args ""
   coats
   fcm_fsim
   ```

9. Generate reports after any step.

   ```
   report -campaign <campaign name> -report output.sff
   ```

10. Run VC Formal FuSa Control, Observe, Detect analysis, and save the results of these checks in the FDB

    ```
    vcf -check_control
    vcf -check_observe
    vcf -check_detect
    ```

11. Use VC Formal Verdi to debug the failures in the GUI.

    ```
    vcf -verdi
    ```

24

VC Formal Low Power Application

This section describes how you can use the Formal Low Power (FLP) application mode.

This section contains the following topics:

- About VC Formal Low Power Application
- VC Formal Low Power Connectivity Checking Application

24.1 About VC Formal Low Power Application

High demand for portable devices such as laptops, mobile phones, personal digital assistance, and smart cards including the Internet of Things (IoT) is driven by clock gating, multi-supply multi-voltage (MSMV), and power gating circuits with state retention. Therefore, performance-aware formal verification techniques are highly desirable where power intent is fully and correctly implemented.

IEEE 1801 standard was established by IEEE institute as Unified Power Format (UPF/IEEE 1801) to ensure the design operates correctly after the power management strategy is inserted. The UPF standard enables you to define and verify the power intent representing the power management architecture including the set of power domains, power switches, supply networks (and so on), and strategy specifications such as legal system power modes, power transitions (and so on).

Low power formal verification is one of the final stages in the verification cycle. Validating a complex power critical design in simulation (with more than 100 power islands) can be time consuming and tedious, as it requires developing additional power sequencing tests in the testbench to test all the power modes. Therefore, VC Formal FLP becomes extremely useful and productive to left shift the verification cycle and catch the bugs early in the flow.

During functional verification, VC Formal FLP with UPF/IEEE 1801 ensures that:

- The power controller with the UPF/IEEE 1801 is working correctly.
- The isolation clamp values are functionally correct.
- The retention signals are ordered correctly.

- The signal values going unknown are identified when an upstream power domain switches off.
- The power on reset can get back the design to a known state.

24.1.1 Supported Methodologies in VC Formal FLP

VC Formal FLP supports the following methodology:

- Formal Low Power Connectivity Checking (FLP CC)

24.2 VC Formal Low Power Connectivity Checking Application

The Formal Low Power Connectivity Checking (FLP CC) application proves conditional device wiring in the presence of a power intent. The FLP CC application checks if there is a structural connection between the source and the destination. The source or the destination can be from either the design or the UPF/IEEE 1801.

The FLP CC application takes the RTL design, power intent specification as UPF/IEEE 1801/IEEE1801, and the connection specifications as inputs, and verifies if the connection specifications hold good for the given design.

Note:

The FLP CC application does not support connectivity latency checks.

Connectivity checks are directional in nature which looks for value propagation from source to destination, proving a property of the form `(enable expressions) |-> (src == dest).`

A given connectivity check holds true if the following conditions exist:

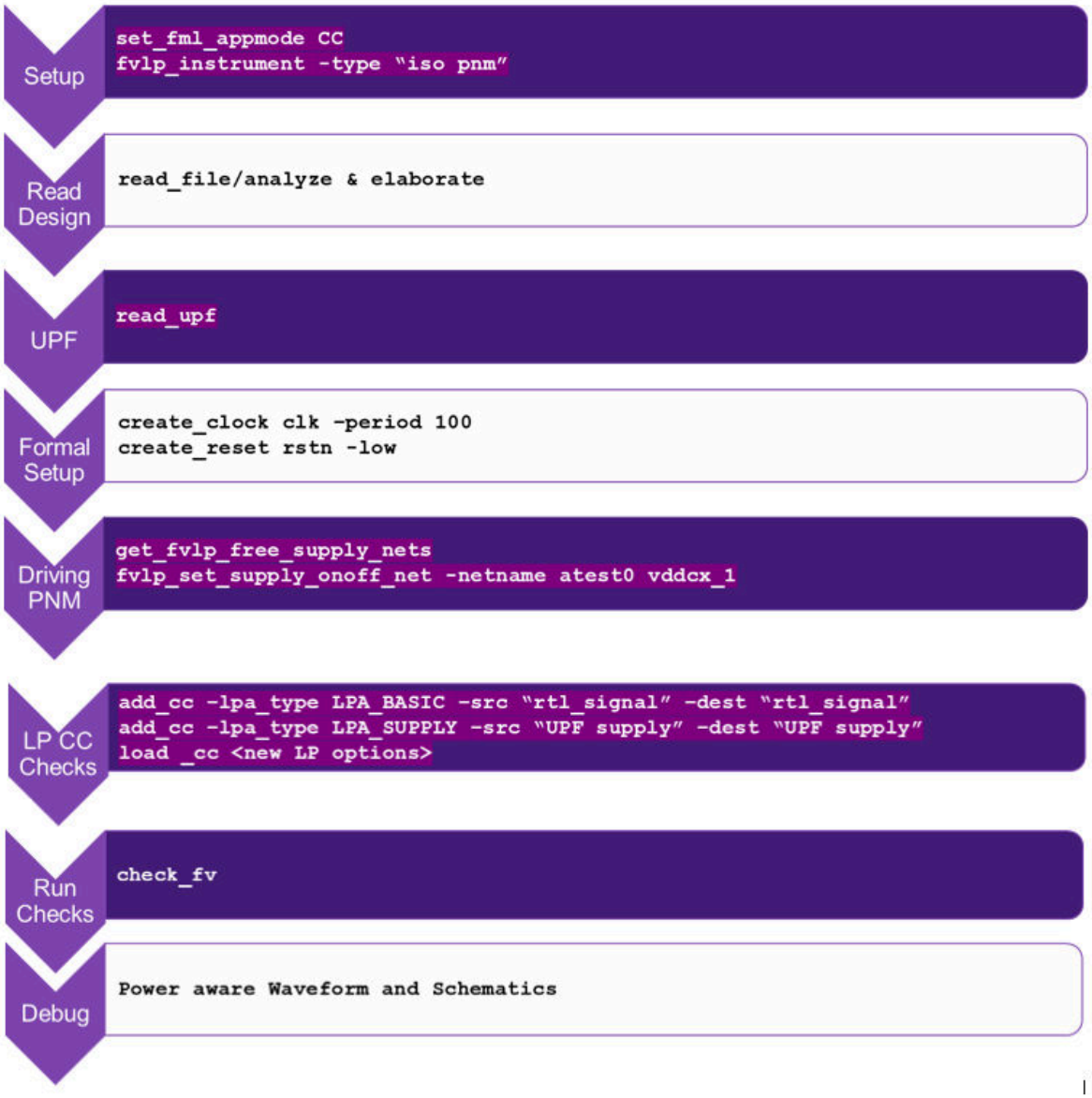
1. There is a structural path from the source to the destination. A structural path exists in the following cases:
   - When there is a physical directional path from the source to destination.
   - When the bit width of the source and destination is equal.
   - When the low power elements in the path of source and destination is not causing any issues.
2. The value of the source and the destination is always the same when the enable expression, if any, is true.

Feedback

24.2.1 FLP CC Methodology

Figure 236 illustrates the FLP CC methodology:

Figure 236: FLP CC Methodology



Setting up the FLP CC Flow

To enable the FLP CC flow:

- Set the `set_fml_appmode` variable to CC.

```
set_fml_appmode cc
```

- Use the `fvlp_instrument` command to specify the instrumentation required for the FLP CC flow.

  Syntax

```
fvlp_instrument [-type <iso, ret, pnm, bind_generics, ret1>]
```

  Where,

  - `[-type <iso, ret, pnm, bind_generics, ret1>]:` Specify the types of RTL annotations such as isolation policies, retention policies, and the power supply network model, that must be made from the UPF. Values: `iso, pnm, ret, bind_generics, ret1`.

  Example

```
fvlp_instrument -type "iso pnm ret"
```

Reading the UPF

Use the `read_upf` command to read in the UPF file and create an annotation of the UPF power network model, isolation, and retention policies as a part of the RTL design.

Example

```
read_upf "mypower.upf"
```

Specifying Clocks and Resets

Use the `create_clock` and `create_reset` commands to specify the clocks and resets in the design.

Setting up PMC

Power Management Controller (PMC) allows the user to define a formal testbench (constraints) setup for the UPF to compensate for the lack of a full PMC in the RTL.

It is only intended to capture the behavior of the PMC (absent) and not to define assumptions for the RTL Design Under Test (DUT). RTL DUT is not supported for connectivity checking in VC Formal as it makes engine convergence difficult on SOC designs.

Feedback

Adding Connectivity Properties

Use the `add_cc` and `load_cc` commands to perform FLP CC checks for low power designs.

For more details, see section Specifying Connectivity Properties.

Performing FLP CC Checks

Use the `check_fv` command to perform FLP CC checks.

Debugging FLP CC Results

To debug FLP CC results, use the following debug features:

- Debugging Failures Using Waveform
- Debugging Failures Using Hierarchical Power Browser Window
- Debugging Failures Using Schematics
- Debugging Failures Using Tcl Commands
- Using the Debug Reports

24.2.2 FLP CC Setup Example

This section provides an example setup of the FLP CC flow.

In the following example,

- The main UPF file (*main.upf*) is written for the DUT.
- For a RTL with dummy top module, create a dummy top domain in the UPF (*top_wrap.upf*) and include the dummy top with all its non-DUT sub-hierarchy. This ensures that the domain boundaries are defined correctly and the algorithms place the isolation cells in the proper locations.

The main execution flow is specified in the *vcf.tcl* file as follows:

- Use the `vcf -f vcf.tcl` command to run the main flow of the execution specified in the *vcf.tcl* file, in batch mode.
- Use the `vcf -f vcf.tcl -verdi` command to run the main flow of the execution specified in the *vcf.tcl* file, in Verdi GUI mode.

Contents of vcf.tcl file

```
set_fml_appmode CC
fvlp_instrument -type "iso pnm ret"
read_file -format sverilog -vcs "top.v gpu3.v pcm_ext.v -sverilog"
read_upf top_wrap.upf
```

```
create_clock clk -period 100
create_reset rstn -low
load_cc_set_param csv_lpa_type "%1%"
load_cc_set_param csv_prop_name "%2%"
load_cc_set_param csv_enable "%3%"
load_cc_set_param csv_src  "%4%"
load_cc_set_param csv_dest  "%5%"
load_cc_set_param csv_start_line  "2"
load_cc -format csv cc_props.csv
sim_run   #optional
check_fv
```

Contents of pcm_ext.v file

```
module pcm_monitor(clk, rstn, iso_en, …);
input clk, rstn, …;
power_state_simstate upf_simstate;
supply_net_type s;
initial begin
  $upf_miror_state("DUT/VDD", s);
end
assume_something1: assume property (
    @(posedge clk)
         upf_simstate != NORMAL || s.state != FULL_ON |->
                                   iso_en
);
…
endmodule
```

Contents of top.v file

```
module testbench(sig1, sig2);
reg  clk;//reg clock
reg  rstn;//reg active low async reset
input sig1;
input sig2;
…
…
//dut instance
GPU3 DUT(
.clk(clk),//reg clock
.rstn(rstn),//reg active low async reset
…
);
endmodule
module BIND_MODULE;
```

Contents of main.upf file

```
create_supply_set TOP_prim
…
create_supply_net VDD
…
```

Feedback

```
create_supply_port VDD_PORT
…
connect_supply_net VDD  -ports {VDD_PORT}
…
create_supply_set TOP_prim -function {power VDD} \
                           -function {ground VSS}  -update
…
create_power_domain TOP -supply {primary TOP_prim} \
                        -supply {default_isolation ISO_prim} \
                        -include_scope
…
set_isolation iso_pd2 -domain PD2 \
                      -elements {..} -applies_to outputs \
                      -clamp_value 1
set_isolation_control iso_pd2 -domain PD2 \
                       -isolation_signal iso_en \
                       -location parent  -isolation_sense high
add_port_state  VDD_PORT        -state {HV 1.0}
add_power_state
```

Contents of top_wrap.upf file

```
create_power_domain dummy -include_scope
set_scope DUT
load_upf   main.upf
```

Contents of cc_props.csv file

```
LPA_TYPE, LABEL, ENABLE, SOURCE, DESTINATION
LPA_BASIC, 3bit_debug, !test_active, source1_signame, dest1_signame
LPA_LATCH, mscr109_ctrl, !test_active && mscr109_reg[7:0]==8'b0000_0011,
 source2_signame, dest2_signame
```

Querying Port/Pin Clamp Properties

Use the `fvlp_check_clamp_value` command to know the destination signal and/or to verify if any of the port/pin is instrumented with a particular isolation.

Syntax

```
fvlp_check_clamp_value  <port/pin name>
```

Where,

- `-location <parent/self/default>:` Specify the location of the isolation policy.
- `-clamp_value <clamp value to check 0/1/LATCH/BASIC>:` Specify the clamp value.

  Note:

  When clamp value is set as `BASIC`, tool creates a LPA_BASIC property to check that no isolation is defined for the port.

Feedback

24.2.3 Specifying Connectivity Properties

A low power connectivity check contains the five elements: `name, src, dest, enable_expr,` and `lpa_type`, where:

- The `name` is an alphanumeric string starting with a letter.
- The `src` is a signal (net, port at bit level or word level) of the Design Under Test (DUT) or a constant value.
- The `dest` is a signal (net, port at bit level or word level) of the DUT.
- An `enable_expr` is a combinational expression that enables the connection. By default, the value of `enable_expr` is true.
- The `lpa_type` is a property category which specifies the kind of check to be performed.

The connection and their enabling conditions are specified in a spreadsheet either in *.csv* format or table format. This file can be loaded using the `load_cc` tcl command. For more details on specifying the connection conditions, see section Specifying CCs in CSV Format.

The `load_cc` command adds a new parameter in the input *csv* file by the name *csv_lpa_type*. Use the `load_cc_set_param` command to set the position of this field in the *csv* file as follows:

```
load_cc_set_param csv_lpa_type "%2%"
```

Connections can also be specified using the `add_cc` command as described in the section Specifying CCs in Tcl Format. Use the `load_cc` command multiple times with the `add_cc` commands to specify low power connectivity checks.

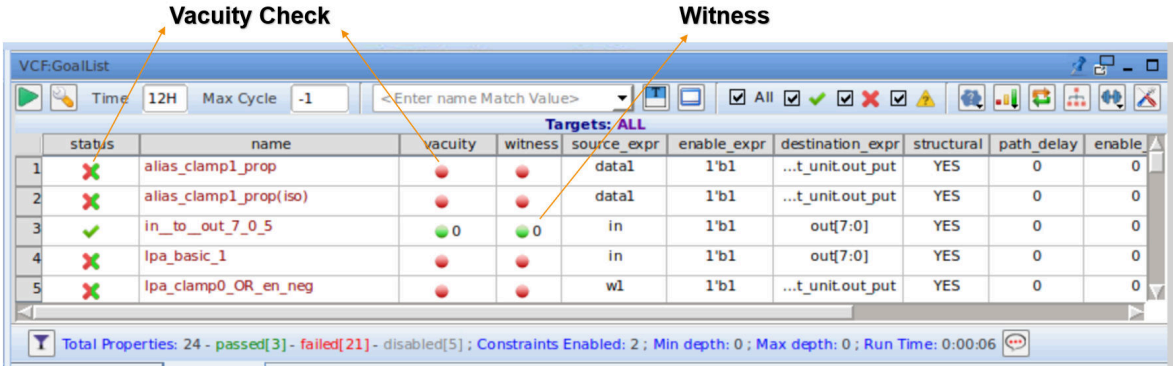
24.2.4 Enabling Vacuity and Witness in FLP CC

To enable vacuity and witness checks in FLP CC, set the following formal variables to true:

- `set_fml_var fvlp_enable_vacuity_creation true`
- `set_fml_var fvlp_enable_witness_creation true`

Feedback

Vacuity and witness status can be reviewed in the *vacuity* field and *witness* field respectively in the *GoalList*, as shown in the following figure:



Debugging Vacuous Property

The vacuous property in FLP CC might be because of the unreachability of the src/dest PD supply or isolation enable.

To debug vacuous property, run the following commands.

- `flp_cc_vacuity_gen_split <default for all vacuous property/property name>`

  `flp_cc_vacuity_gen_split` generates different cover properties like src,dest,iso,src_dest,src_iso,dest_iso … for each vacuous property and these properties are not seen show in property table. The tool creates 6 cover `src,dest, iso, src_dest, src_iso, dest_iso` for each property.

- `check_fv`

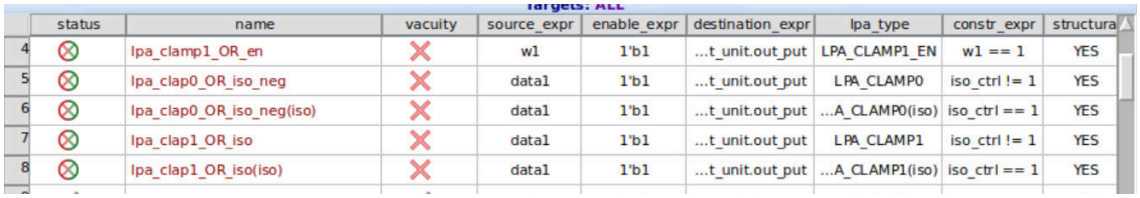
- To view the vacuous reasons for each property, use the following command:

```
flp_cc_vacuity_report < defaule for all vacuous properties/ property
 name>
flp_cc_vacuity_report lpa_clap0_OR_iso_neg(iso)
```

  - covered - lpa_clap0_OR_iso_neg(iso)_dest
  - uncoverable - lpa_clap0_OR_iso_neg(iso)_iso
  - covered - lpa_clap0_OR_iso_neg(iso)_src

Feedback

- uncoverable - lpa_clap0_OR_iso_neg(iso)_srciso
- uncoverable - lpa_clap0_OR_iso_neg(iso)_destiso



	status	name	vacuity	source_expr	enable_expr	destination_expr	lpa_type	constr_expr	structura
4	⊗	lpa_clamp1_OR_en	✗	w1	1'b1	...t_unit.out_put	LPA_CLAMP1_EN	w1 == 1	YES
5	⊗	lpa_clap0_OR_iso_neg	✗	data1	1'b1	...t_unit.out_put	LPA_CLAMP0	iso_ctrl != 1	YES
6	⊗	lpa_clap0_OR_iso_neg(iso)	✗	data1	1'b1	...t_unit.out_put	...A_CLAMP0(iso)	iso_ctrl == 1	YES
7	⊗	lpa_clap1_OR_iso	✗	data1	1'b1	...t_unit.out_put	LPA_CLAMP1	iso_ctrl != 1	YES
8	⊗	lpa_clap1_OR_iso(iso)	✗	data1	1'b1	...t_unit.out_put	...A_CLAMP1(iso)	iso_ctrl == 1	YES

24.2.5 Generating Constraints Using Power State Table (PST)

To create the constraints from PST, set the following formal variable to `true`:

```
set_fml_var fvlp_enable_pst_constraint true
```

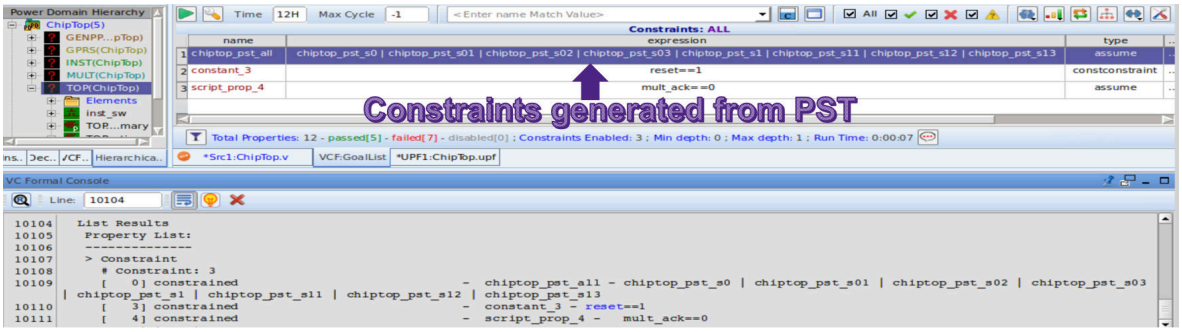
When the `fvlp_enable_pst_constraint` is set to true, VC Formal automatically creates the constraints automatically for all PST and PST states present in the design and adds it to the constraint table.

To create constraint based on user-provided PST and state information, set the following `flp_pst_config` variable as follows:

`flp_pst_config "chiptop_pst,s1"`.

Here `chiptop_pst` is one of the PST name present in the UPF and `s1` is state of this PST.

The following figure describes constraints generated from PST:



24.2.6 Supported LPA_TYPES

The FLP CC application supports the following categories of LPA_TYPES:

NON_LPA

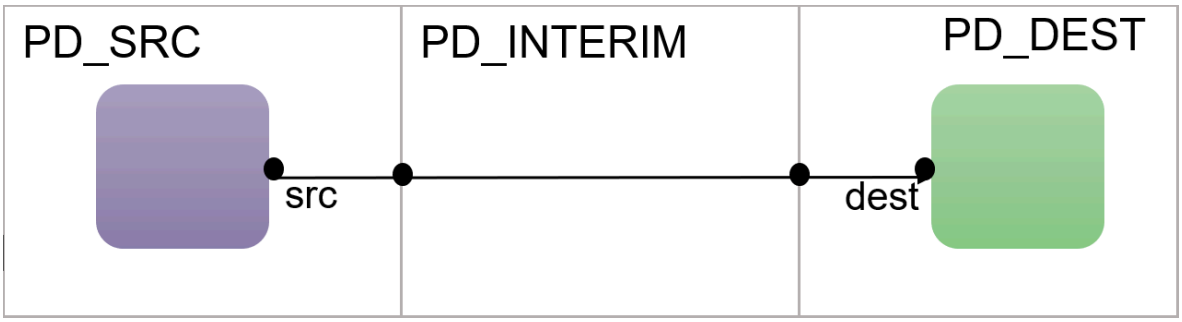
The connectivity check is treated as a non-LP aware connectivity check. This is a forced type for all the connectivity in the non-FLP CC flow, where UPF/IEEE 1801 is not read before `load_cc` or `add_cc` command.

If a user has not loaded the UPF/IEEE 1801, then the default type for all connectivity check in the FLP CC is NON_LPA. The NON_LPA type is similar to normal connectivity checking for RTL design which does not have any power intent.

The following figure describes structural connection for value propagation from source to destination, where,

- *PD_SRC*: Source power domain
- *PD_DEST*: Destination power domain
- *PD_INTERIM*: All the interim power domains that a signal crosses from PD_SRC to PD_DEST.

  In this type, PD_SRC, PD_INTERIM, and PD_DEST do not have any values and does not contribute to the property checking.



Feedback

LPA_BASIC

LPA_BASIC is the basic low power aware connectivity check. LPA_BASIC is the default value for the `csv_lpa_type` parameter in the FLP CC flow.

For the simplest case, where the path delay, enable hold, enable delay, and reset delay is zero, the LPA_BASIC check is defined as:

```
SD && En && DD |-> (dest == src)
```

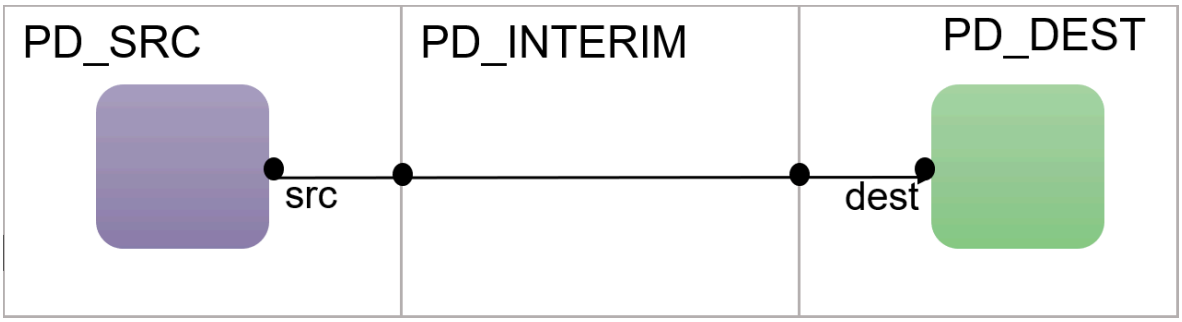
Where,

- *SD*: Source Domain
- *DD*: Destination Domain
- *En*: Enable condition
- *dest*: destination signal
- *src*: source signal

The `simstate == NORMAL` condition is added to the enable condition of the connectivity for both source and destination domains. If either of the source or destination belongs to a liberty cell, then `simstate` function of that lib cell is added to the enable condition. Check if the destination value matches with the source value and hence the connectivity, when SD and DD are both ON (NORMAL state) and when En is asserted.

The following figure describes structural connection for value propagation from source to destination, where,

- *PD_SRC*: Source power domain
- *PD_DEST*: Destination power domain
- *PD_INTERIM*: All the interim power domains that a signal crosses from PD_SRC to PD_DEST.



The effect of PD_INTERIM is checked on the connectivity, when both source and destination domains are at NORMAL state and PD_INTERIM is at CORRUPT state. The

Feedback

LPA_BASIC check enables you to verify all the feed-through signals or Always ON (AON) wires in the design.

Note:

Support for connectivity properties with non-zero delay/hold parameters are not supported, and an error is reported when such properties are used.

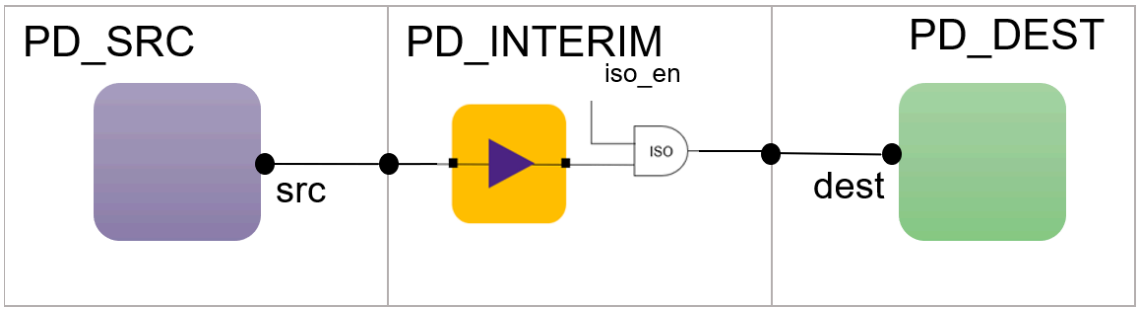
LPA_CLAMP0

LPA_CLAMP0 is similar to LPA_BASIC type, with the added condition that the destination should be clamped at 0 when the compound enable condition is met, but an isolation cell in the path is enabled.

LPA_CLAMP0 checks the connectivity between source and destination with a clamp 0 isolation inserted in the path. The check is reported as proven if both the connectivity and clamping component checks pass.

The following figure describes structural connection for value propagation from source to destination, where,

- *PD_SRC*: Source power domain
- *PD_DEST*: Destination power domain
- *PD_INTERIM*: All the interim power domains that a signal crosses from PD_SRC to PD_DEST.



For the simplest case with a single isolation cell on the path and where the path delay, enable hold, enable delay, and reset delay is zero, the LPA_CLAMP1 check is defined for connectivity and clamping component as follows:

- Connectivity component: `En && SD && DD && (iso_en != ISO_SENSE) |-> (src == dest)`
- Clamping component: `En && SD && DD && (iso_en == ISO_SENSE) |-> (dest == 'b0)`

Feedback

If there are multiple isolation cells found in the path such as back to back isolation schemes, all the isolation disable conditions is ANDed into the antecedent of the connectivity component and ORed in the clamping component as follows:

- Connectivity component: `En && SD && DD && (iso_en1 != ISO_SENSE1) && (iso_en2 != ISO_SENSE2) |-> (src == dest)`
- Clamping component: `En && SD && DD && ((iso_en1 == ISO_SENSE1) | | (iso_en2 == ISO_SENSE2)) |-> (dest == 1'b0)`

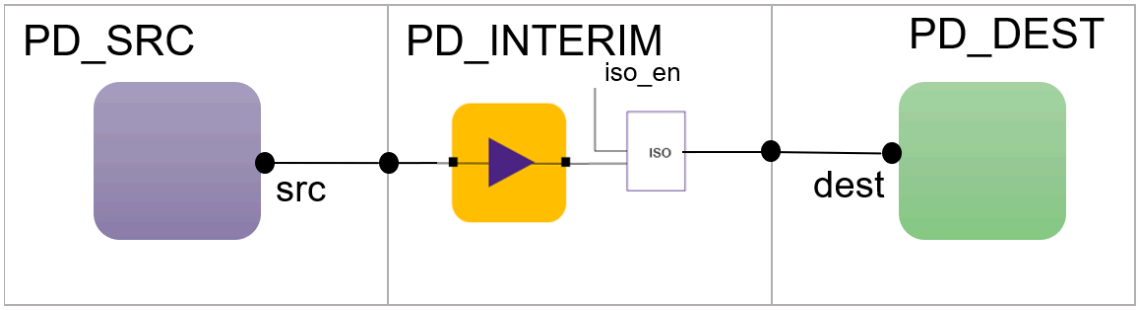
LPA_LATCH

LPA_LATCH is recommended for pad signal connectivity checks. This type is similar to LPA_BASIC type, with the added condition that the destination never has an edge when a clamp is enabled in the connectivity path.

LPA_LATCH is similar to LPA_CLAMP1 or LPA_CLAPM0 type, with the added condition that the clamped value must be latched such as the consequent of the clamping components must be `(not $fell(dest) and not $rose(dest))`.

The following figure describes structural connection for value propagation from source to destination, where,

- *PD_SRC*: Source power domain
- *PD_DEST*: Destination power domain
- *PD_INTERIM*: All the interim power domains that a signal crosses from PD_SRC to PD_DEST.



If the path from PD_SRC to PD_DEST shuts down and has an isolation clamping, then an edge should not be seen because that edge would be asynchronous and could cause some CDC violations.

Latch type isolation is used instead of AND or an OR gate isolation. Therefore, during isolation clamping the value that the destination will see is the last known value of the source instead of an edge on the destination, if there is any shut down on the path.

LPA_SUPPLY

LPA_SUPPLY is recommended for Power/Ground (PG) pin connectivity checking. The source and destination can be names of power supply objects from the UPF/IEEE 1801 or the design.

Each property of `-lpa_type` = LPA_SUPPLY has a source supply name and a destination supply name, each defined by two nets called as the state and the voltage net.

The LPA_SUPPLY property for the source supply nets `src_state` and `src_volt`, and the destination nets `dest_state` and `dest_volt` is defined as follows:

```
(enable_condition) |-> ((src_volt == dest_volt) && (src_state == dest_state))
```

Note:

The formation of `enable_condition` is same as that of any other non lpa property, where it is given in terms of RTL signals.

LPA_CLAMP0_EN, LPA_CLAMP1_EN, and LPA_LATCH_EN

LPA_CLAMP0_EN is recommended for checking the connectivity of the enable source to the enable pin of the instrumented isolation cell where the signal is clamped to 0. Here the physical path for any connectivity property is through the enable pin of an isolation cell instead of the data pin.

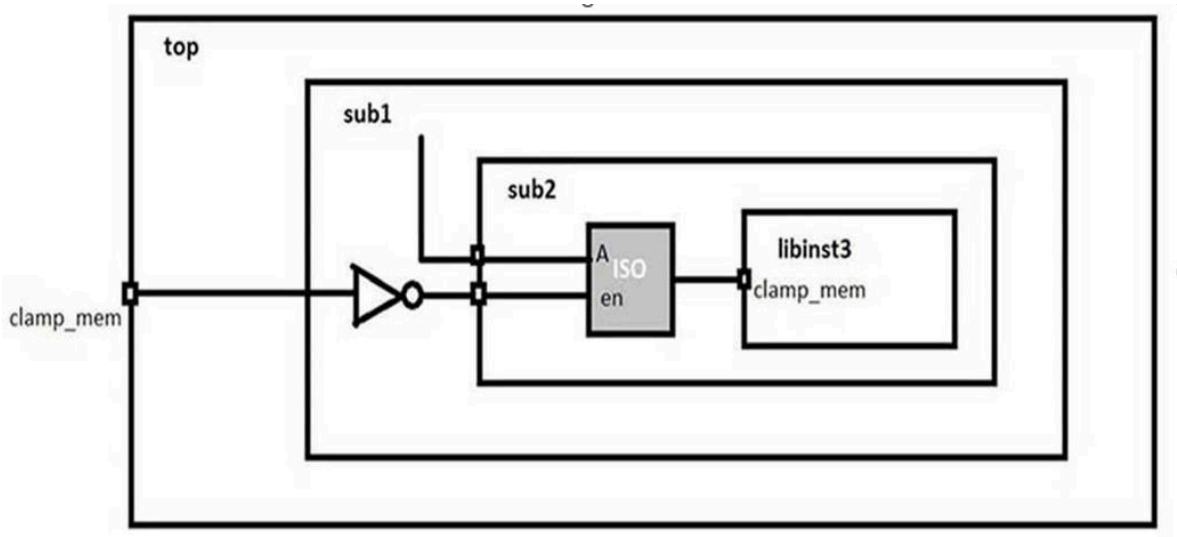
LPA_CLAMP1_EN is recommended for checking the connectivity of the enable source to the enable pin of instrumented isolation cell where the signal is clamped to 1. Here the physical path for any connectivity property is through the enable pin of an isolation cell instead of the data pin.

LPA_LATCH_EN is similar to LPA_CLAMP0_EN or LPA_CLAMP1_EN type, for checking the connectivity of the enable source to the enable pin of the instrumented latch type isolation cell.

When the physical path traverses through the enable pin of an isolation cell instead of the data pin, use the corresponding LPA_***_EN `lpa_type` for connectivity checking.

In the following figure, `top.clamp_mem` is the top level isolation enable signal which is connected to `top.sub1.sub2.libinst3.clamp_mem` through an isolation cell marked as ISO.

Feedback



Based on the isolation policy associated with the isolation cell, tool will extract the clamp value for the isolation cell and the `isolation_sense` for clamping the isolation cell. Accordingly the connectivity property checks if the destination gets the clamp value, when the source has the value of the `isolation_sense` specified in the property. Any other checking for `source != isolation_sense` is avoided.

LPA_***_EN performs the following structural checks:

- If the isolation cell is inferred from UPF/IEEE 1801 (not present in RTL),
  - Path from the source to destination is checked, through the data pin of the inferred isolation cell.
  - Source can be same as isolation enable (specified in the UPF/IEEE 1801 policy) or can exist in RTL on a path before isolation enable (specified in the UPF/IEEE 1801 policy). For inferred isolation cells, `iso_enable` is directly connected to the inferred isolation cell and structural errors are reported if the source exist on a path after the `iso_enable`.
- If the isolation cell is already inserted in the design,
  - An error is reported if you cannot associate any UPF/IEEE 1801 isolation policy to the isolation cell.
  - Structural errors are reported if the source is not on the path, connecting the isolation enable (mentioned in UPF/IEEE 1801 policy) to the enable pin of the isolation cell in the design.

In the above scenarios, the tool checks based on the `isolation_sense` specified in the UPF/IEEE 1801. A falsification is reported if any logic on the path causes sense to change before the isolation cell.

Feedback

Based on the type of isolation cell used the clamping components can be described as follows:

- For LPA_CLAMP1_EN, `En && SD && DD && (src == ISO_SENSE) |-> (dest == 'b1)`.
- For LPA_CLAMP0_EN, `En && SD && DD && (src == ISO_SENSE) |-> (dest == 'b0)`.
- For LPA_LATCH_EN, `En && SD && DD && (src == ISO_SENSE) |-> (not $fell(dest) and not $rose(dest))`.

24.2.7 Printing Combinational Logic for LPA_BASIC Property Path

For Always ON (AON) paths with properties having `lpa_type` as LPA_BASIC, by default, you check only if there are isolation cells in the path.

To check if there are other combinational cells like MUX, AND, OR, NOT, BUF, and so on, in the path along with isolation cell, enable the following formal variable:

```
set_fml_var fvlp_check_combo_logic true
```

An example for printing combinational logic is shown in the following figure:

```
vcf>add_cc -src data3 -dest dataout3 -lpa_type LPA_BASIC -name basic_check
[Warning] CC_LP_COMBO_LOGIC_PRESENT: Combination logic 'MUX' is present in path of LPA_BASIC property 'basic_check' for the hierarchy
'u2.nt.I_MUX_out2'.
        You may want to fix the path.
[Warning] CC_LP_COMBO_LOGIC_PRESENT: Combination logic 'INV' is present in path of LPA_BASIC property 'basic_check' for the hierarchy
'u2.nt.I_BNOT_N_3'.
        You may want to fix the path.
[Warning] CC_LP_ISOLATION_CELL_PRESENT: Isolation cell iso_ctrl present in path of LPA_BASIC property 'basic_check'.
        You may want to fix the path.
{"basic_check"}
```

24.2.8 UPF/IEEE 1801 Modeling: Isolation/Retention/Power Supply Network

VC Formal works with a model of the design which has a comprehensive functional representation of the power supply network (including power switches) inbuilt along with structural placement of isolation and retention policies.

- The isolation and retention policies are placed based on the structure of the design and UPF/IEEE 1801 policy directives.
- The power supply network model has a functional implementation. The supply values and domain simulation states seen in VC Formal traces are consistent with the power supply network connectivity and switching relationships outlined in UPF/IEEE 1801.

Feedback

Use the `-type` switch in the `fvlp_instrument` command, to switch off instrumentation for one or more types of UPF/IEEE 1801 elements as follows, where `iso` means isolation, `ret` means retention, and `pnm` means power network model:

```
fvlp_instrument -type "iso pnm ret"
```

Power Network Model

VC Formal uses Power Network Model (PNM) to instrument the power supply connectivity from the UPF/IEEE 1801. It dumps a Verilog module with all the connections specified in UPF/IEEE 1801 for different supplies.

The supply network is defined using a combination of all or some of the following supplies:

- Supply sets in UPF/IEEE 1801
- Supply nets in UPF/IEEE 1801
- Supply ports in UPF/IEEE 1801
- Power switches in UPF/IEEE 1801
- `set_related_supply_net` in UPF/IEEE 1801
- `set_port_attributes` in UPF/IEEE 1801
- `connect_supply_net` in UPF/IEEE 1801
- LDO (voltage regulator) instantiated in RTL
- Switch macros instantiated in RTL
- Partially routed supply wires/ports in RTL

Power Network Model resolves the root driver(s) for any supply such as supply net/supply set function. The root driver can be a primary undriven port, an unrefined supply set, an instantiated LDO, the control signals of a UPF/IEEE 1801 power switch policy, or an instantiated power switch. The support of `pg_function, switch_function`, and `pg_pin` attributes is required to infer the correct connectivity in the PNM.

The state of `supply_net` is determined by the resolution function specified in the UPF/IEEE 1801 modeling such as parallel, one-hot, or custom.

The supply (power/ground) powering of any instance is determined by the following precedence order:

- `set_port_attribute` or `set_related_supply_net` on any primary port of the design, including ports of black boxes.
- `connect_supply_net` to `pg_pin` of the instance, if it is an instantiated cell.

- For instantiated isolation cells the supply is determined from the isolation policy associated with the isolation cell.
- Supply of inferred isolation cells and backup supply for inferred retention cells is determined from the isolation and retention policy respectively.
- For all the other instances, the supply is the primary power and primary ground of the power domain the instance is portioned into.

VC Formal reads attributes from db (liberty) models for instantiated low power cells to infer low power functionality, pg connectivity, and low power corruption semantics as described below:

- `is_isolation, is_level_shifter, is_retention_cell, is_always_on, and switch_cell_type`
- `pg_pin, pg_type (primary, backup, and internal) direction (input, output, and internal), pg_function, and switch_function`
- `related_power_pin and related_ground_pin`

The primary ports are corrupted based on the supply in `set_related_supply_net` or `set_port_attribute`.

Constraining the Power Supplies in the UPF/IEEE 1801

To conditionally switch the free supplies available in the design UPF/IEEE 1801, use the following methods:

1. Using Tcl commands
   - Use the `get_fvlp_free_supply_nets` Tcl command to constrain the power supplies and return the undriven free supplies from the design UPF/IEEE 1801.

     Example

     To return the undriven free supplies, if the UPF/IEEE 1801 has 4 undriven supplies such as VDD1, VDD_ret, VDD2, and VSS and are freely available in the PNM, use:

     ```
     get_fvlp_free_supply_nets{"VDD1", "VDD_ret", "VDD2", "VSS"}
     ```

   - Use the `fvlp_set_supply_aon` Tcl command to drive the supplies mentioned as an argument with an AON supply.

     Example

     ```
     fvlp_set_supply_aon {"VDD1", "VDD_ret", "VDD2", "VSS"}
     [Info] FVLP_SET_INPUT_SUPPLY_VOLTAGE_I: Input supply 'VDD1' set to
      voltage 1v.
     [Info] FVLP_SET_INPUT_SUPPLY_VOLTAGE_I: Input supply 'VDD_ret' set
      to voltage 1v.
     ```

Feedback

```
[Info] FVLP_SET_INPUT_SUPPLY_VOLTAGE_I: Input supply 'VDD2' set to
 voltage 1v.
[Info] FVLP_SET_INPUT_SUPPLY_VOLTAGE_I: Input supply 'VSS' set to
 voltage 1v.
```

- Use the `fvlp_set_supply_onoff_net -netname <netname> {"supply"}` to switch ON or OFF the supply passed as an argument based on the driver from the RTL as specified with `-netname`.

  Example

```
fvlp_set_supply_onoff_net -netname ctrl  {"VDD"}
[Info] FVLP_SET_INPUT_ONOFF_I: Input supply 'VDD' connected to be
 driven on/off by RTL signal 'ctrl'.
```

24.2.9 Debugging Failures in FLP CC

Low power connectivity checks may fail for the following reasons:

- There exists no connectivity between the two points because of incorrect specification or broken RTL implementation.
- There exists broken path due to incorrect MUX selects or AND gate input tied to low signal.
- Wrong isolation cells being inserted in the UPF/IEEE 1801.
- Wrong `lpa_type` specified in the property.
- Missing isolation cells in the path.
- Intermediate domain supplies going to OFF state.
- Supply to isolation cells going to OFF state.

To debug open connection failures reported by the `report_fv` command, use the following debug features:

- *Hierarchical Power Browser* Window.
- Schematic view of the power domains with isolation, retention, and power rails.
- Back navigation to source code from all views.
- Waveform view of power rails, supply sets, domain simstates, and signal corruption.
- Additional columns in the *VCF.GoalList* window to show `lpa_type` and `fvlp_reason`.

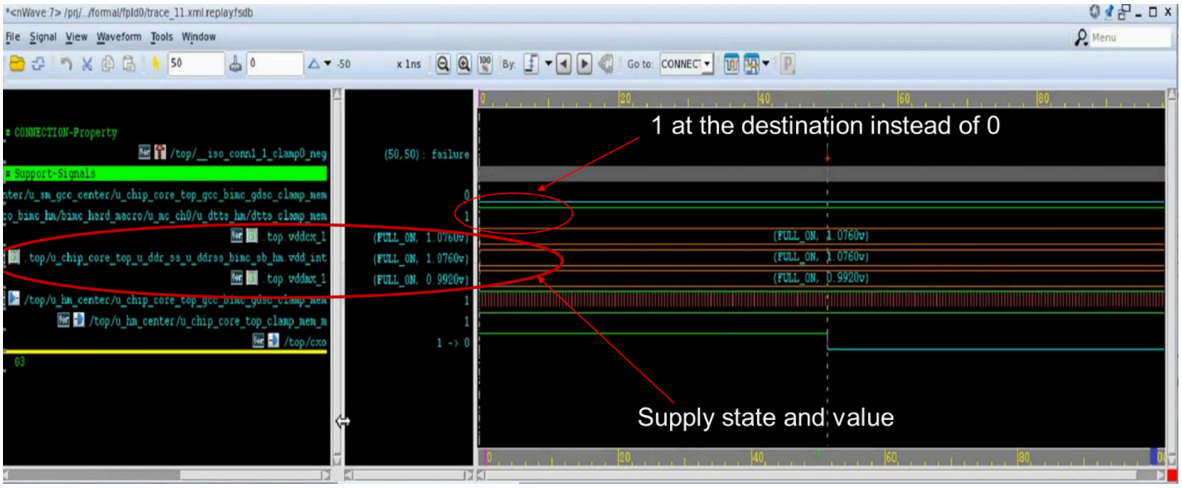
  Note:

  VC Formal FLP CC replays only the CC trace in the counter example waveform, by default.

Feedback

Debugging Failures Using Waveform

For failing properties, use the CEX trace to debug the cause for property failure in the waveform window as shown in the following figure:



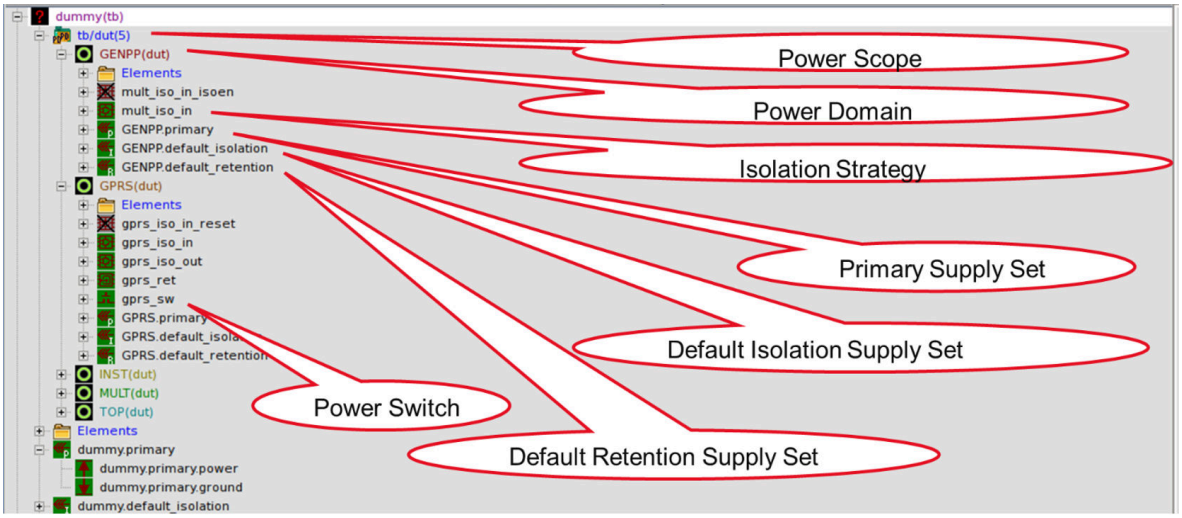
In the example, there exists a clamp 0 property, but at the wave you see that the clamped value is 1 instead of 0 and therefore the property failed:

```
add_cc -name conn_clamp0  -src ... -dest ... -lpa_type LPA_CLAMP0
```

Debugging Failures Using Hierarchical Power Browser Window

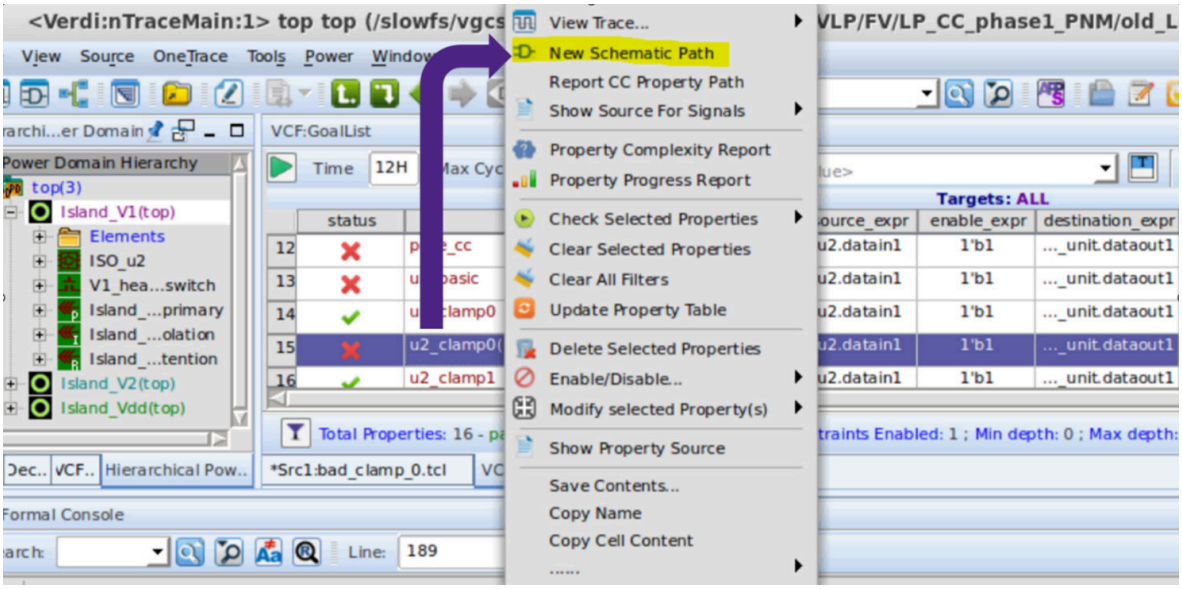
Use the *Hierarchical Power Browser* window as illustrated in the following figure, to view the power intent captured in the UPF. It provides information about different power domains, isolation, retention, supplies, power switches, and so on:

Feedback



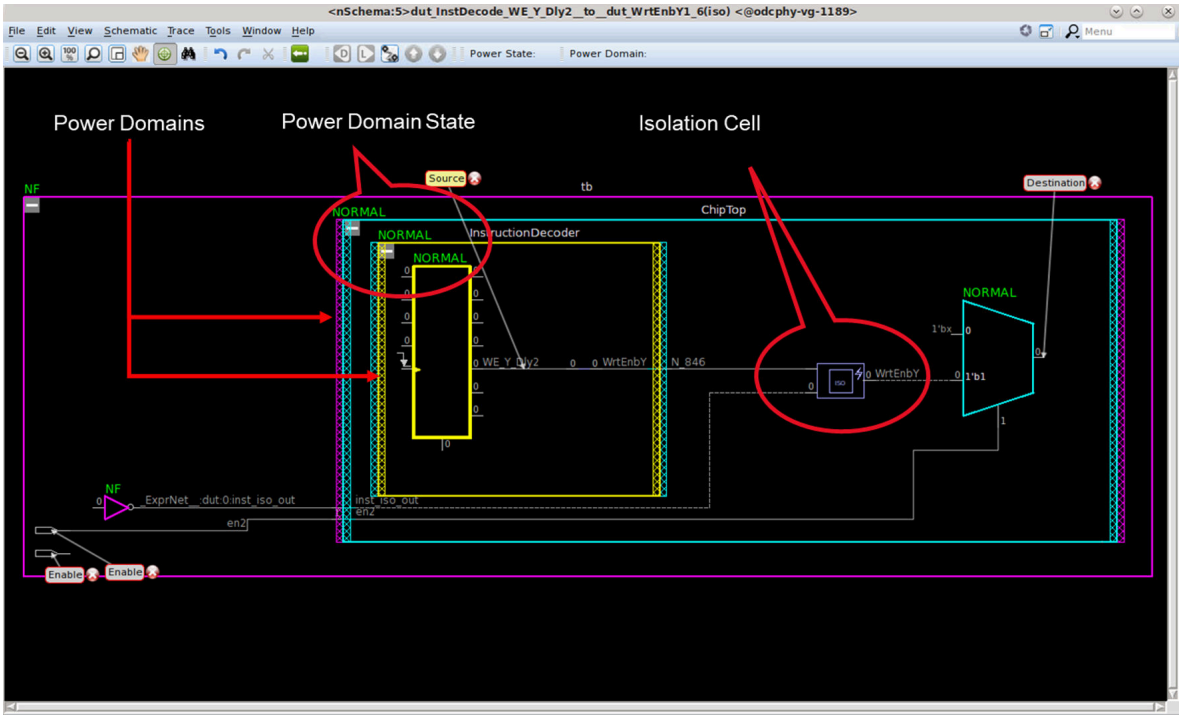
Debugging Failures Using Schematics

To open the schematic for debug, right-click on the specific property and select *New Schematic Path* to open the nSchema, as shown in the following figure:



Feedback

The nSchema window describes the power domain and the corresponding states, the source and destination signal, the enable condition, and isolation in the path, if any:

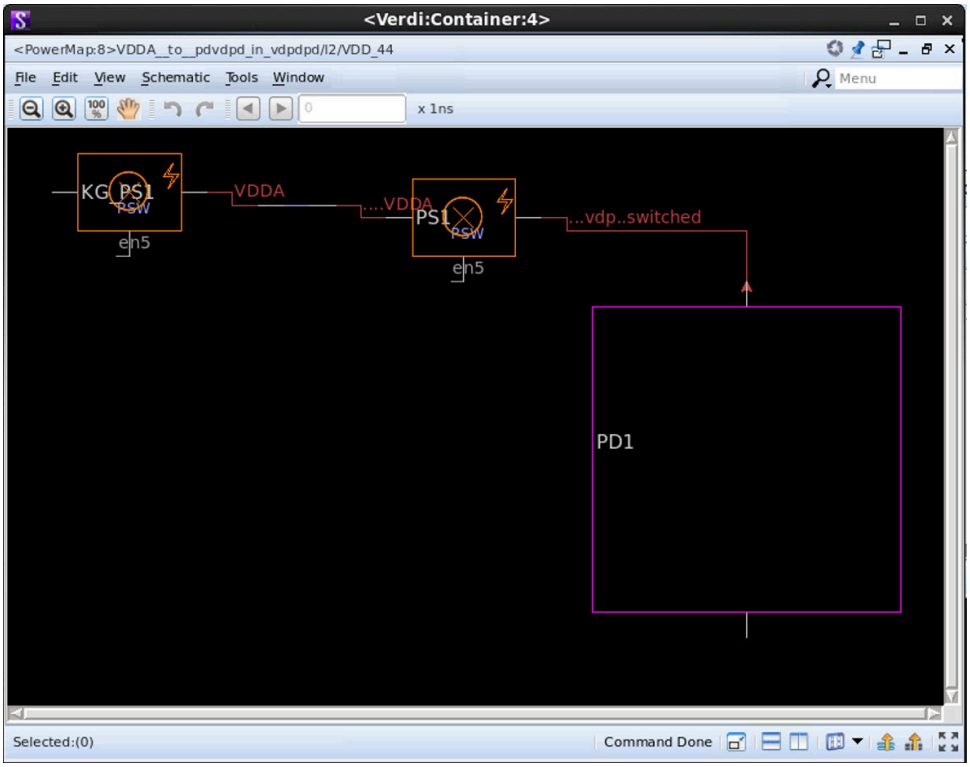


Using Schematics for Connectivity Properties with lpa_type=LPA_SUPPLY

Right-click on the specific property and select *New Schematic Path* to open the schematic for LPA_SUPPLY type connectivity properties. The schematic for LPA_SUPPLY type

Feedback

connectivity properties opens the Verdi partial power map, with the select elements to depict the connectivity path as shown in the following figure:



For more information on Verdi partial power map, see the *Verdi Power-Aware Debug User Guide*. The following elements are shown on the partial power map:

- For proper structurally connected LPA_SUPPLY properties, the UPF supply nets and power switches along the path of the connection are shown.
- For structurally disconnected properties, only the first UPF supply net along the path is shown. Double-click on the nets displayed on the map to expand the power map from the supply net.

  Note:

  The structurally disconnected properties appear as falsified properties and these falsifications have no trace to display.

The known limitations of partial power map are described as follows:

- The depiction of LIB cells and PG pins are not possible in the partial power map. This is a limitation imposed by Verdi. Therefore, the LPA_SUPPLY connections with lib cells and PG pins along the connectivity path or at the ends, may not show completely.
- Resolution functions have no representation in the partial power map.

Feedback

- Open the *View* menu and select *Net Name* and *Power Cell Name* to see net names and power switch names. By default, the net names and power switch names are not shown.
- Domains related to the depicted supply nets get shown currently, even though there is no need for it.

Debugging Failures Using Tcl Commands

- Use the `fvlp_cc_print_path <property name>` Tcl command, to see the elements that are present between the source and the destination of any property.
- Use the `report_trace_paths [get_trace_paths -from <src sig> -to <dest sig>]` Tcl command, to check elements between two signals.
- Use the `fvlp_cc_trace_fanin -prop <property Name> -dest <destination name>` Tcl command to get the fanin of a signal and debug PG pin related issues.
  - Use the `-incr` option with the `fvlp_cc_trace_fanin` command when there are parallel drivers, with the input of `-dest` as the first parallel driver.

    Examples

    ```
    vcf>fvlp_cc_trace_fanin -dest D1/D1_1/D1_2/D1_3/psw1/VDD -prop
     prop_h1
       PARALLEL DRIVERS {//
       SUPPLY_PORT pd1_1.upf:10 INPUT D1/D1_1/VDDA1_1
       SUPPLY_PORT pd1_1.upf:12 INPUT D1/D1_1/VDDA1_p
      }
    vcf> fvlp_cc_trace_fanin -dest D1/D1_1/VDDA1_1  -prop prop_h1 -incr
    PARALLEL DRIVERS {
        SUPPLY_PORT pd1.upf:9 INPUT D1/VDDA_1
        SUPPLY_PORT pd1.upf:17 OUTPUT(PSW) D1/PSW_1/VDDV
      }
    ```

    To trace again, use the following command:

    ```
    vcf>fvlp_cc_trace_fanin -dest D1/VDDA_1   -prop prop_h1 -incr
    PARALLEL DRIVERS {
    }
    ```

Using the Debug Reports

Use the following reports under the *vcst_rtdb/reports* directory to review and post-process results, for debugging:

- *instrument_summary.rpt*
- *isolation_association.rpt*
- *power_switch_association.rpt*

- *undefined_db_cell.rpt*
- *retention_policy_assoc.rpt*
- *lib_match.rpt*
- lib_mapping.rpt

Use the following log reports under the *vcst_rtdb/logs* directory:

- *db_link.log*
- *db_read.log*

Use the `-verbose` option with the `report_fv` command to additionally print the following:

- `source_root_supply:` Source root supply name
- `dest_root_supply:` Destination root supply name
- `path_supplies:` List of all the root supplies that directly affect the path from the source to destination.
- `iso_policies:` List of all isolation policies on the path from the source to destination.

25

Data Path Validation Application

VC Formal DPV leverages transactional equivalence to compare two versions of the design, one version representing the design functionality at architecture level (mostly untimed C or C++ model) and second version representing the pipeline implementation (mostly RTL). Integrated HECTOR™ technology within VC Formal help to verify data transformation blocks such as ALU, FPU, DSP, and so on.

Getting Started • *What is VC Formal DPV?* • *Accessing and Working with DPV Lab Example*	**Improving Convergence of Properties** • *Adding Cutpoints and Blackboxing* • *Enabling Case Splitting* • *Enabling Lemma partitioning* • *Enabling Assume Guarantee* • *Enabling HDPS for Multipliers/FMA Designs*
Setting Up and Running DPV • *Enabling DPV Application mode* • *Compiling Implementation Designs* • *Compiling Specification Designs* • *Adding Constraints and Mapping Designs* • *Running DPV in TCL*	**Debugging, Reviewing and Saving Results** • *Generating Verification Summary* • *Analyzing and Debugging Results (Proven, Falisified, Vacuous and Inconclusive Assertions)* • *Saving and Restoring Sessions* • *Replaying Counter Example Using GDB*
Accelerating Proofs • *Configuring Runtime Parameters* • *Configuring Effort Levels* • *Running on Compute Farm* • *Monitoring Progress* • *Enabling Regression Mode Accelerator*	**Getting Help on Commands** • *Setting DPV Specific Application Variables*

25.1 About DPV Application Mode

Data Path Validation (DPV) application uses HECTOR technology to verify data transformation blocks. Example of these data path blocks are floating point/integer adder, multiplier, divider, multiplication accumulate and so on. These design structures are very common in CPU, GPU, and AI/ML designs. VC Formal DPV leverages transactional equivalence to compare two versions of the design, one version representing the design functionality at architecture level (mostly untimed C or C++ model) and second version representing the pipeline implementation (mostly RTL).

Feedback

For more information on DPV, see the *VC Formal Data Path Validation User Guide*.

Note:

All the VC Formal applications require specific license keys. For more information on the license keys required for each of the VC Formal applications, see the VC Static Platform Installation Notes.

This video explains how to set up VC Formal DPV (Data Path Validation) for verifying an ALU design with both C and RTL models. The ALU performs operations like AND, OR, ADD, and SUBTRACT, with signals determining the operation type and bit size (16 or 32 bits). The setup involves creating a wrapper function for the C model using `Hector::registerInput` and `Hector::registerOutput` to define inputs and outputs, and encapsulating operations within `Hector::beginCapture` and `Hector::endCapture`.
The DPV setup requires a TCL file. First, compile the designs using `create_design` with `-name` and `-top` options, specifying clock and reset for RTL. Analyze the C model with cppan and the RTL with VCS. Map the inputs and outputs using `map_by_name` or explicit assume and lemma commands. Constraints and lemmas are defined in a TCL procedure, assigned to `user_assumes_lemmas_procedure`.

25.1.1 Video Links

Click the links to view videos on DPV:

- DPV Setup (6 mins)
- DPV Run and Debug (6 mins)
- DPV Debugging C/C++ model in DPV (5 mins)

25.2 Writing a DPV Wrapper for C/C++ Model

The following lines have been added to the C++ model for Hector.

```
#include <Hector.h>
void hector_wrapper()
{
int a, b, mul, madd;
Hector::registerInput("in_a", a);
Hector::registerInput("in_b", b);
Hector::registerOutput("out_mul", mul);
Hector::registerOutput("out_madd", madd);
Hector::beginCapture();
compute(a, b, mul, madd);
Hector::endCapture();
}
```

This wrapper function is required because the C++ language does not have a notion of input and output. For example the function,

```
Hector::registerInput("in_a", a);
```

indicates that the variable a should be treated as an input and it is to be referred to as in_a in all the subsequent steps of DPV.

The directives *Hector::beginCapture* and *Hector::endCapture* specify the start and end of the transaction respectively.

25.3 Compiling a design

Use the following steps to compile a design:

1. Create a design using the `create_design` command.
2. Analyze the design (This is a language specific step).
3. Elaborate the design (optional step).
4. Generate the DFG by calling the `compile_design` command.

25.3.1 Analyzing C++ Files

The command for analyzing C++ files is as follows:

```
cppan <options> <list of filenames>
```

25.3.2 Analyzing Verilog and VHDL files

For Verilog only designs, the command to analyze (and elaborate) the files is as follows:

```
vcs <options> <list of filenames>
```

In a mixed language design containing both Verilog and VHDL, the command for analyzing Verilog files is as follows:

```
vlogan <options> <list of filenames>
```

The command for analyzing VHDL files is as follows:

```
vhdlan <options> <list of filenames>
```

This step is used only for VHDL or mixed language designs where elaboration options are required. The command for elaborating the design is as follows:

```
vcs <options>
```

Feedback

25.3.3 Compiling C/C++ Specification Design

The following TCL procedure is added to the script to provide information about the specification design.

```
proc compile_spec {} {
create_design -name spec -top hector_wrapper
cppan foo.cc
compile_design spec

}
```

The `create_design` command creates a design with name spec and also specifies that the top-level function is `hector_wrapper`. Finally, the `compile_design` command compiles the design named spec into a DFG.

DPV supports compiling C++11 design. The original C++99 front-end continues to be supported. To use the newer C++11 front-end version, you require an additional license. By default, the original C++99 front-end is used to compile C/C++ models. To enable the C ++11 front-end, the following command must be placed in the DPV setup file.

```
set _hector_comp_use_new_flow true
```

25.3.4 Compiling RTL Implementation Design

The following TCL procedure in the command script file for the implementation design.

```
proc compile_impl {} {
create_design -name impl -top ima -clock clk -reset rst -negReset
vcs foo.v
compile_design impl
}
```

Here again, the `create_design` command creates a design with name impl and the top-level module as ima. The `-clock` option specifies the name of the clock input port. The -reset option identifies the reset input to the design and the `-negReset` option indicates that the reset is active low. If the reset were active high, then the option `-negReset` would be omitted (the default is active high). Next, the VCS command analyzes the Verilog program and the `compile_design` command compiles the impl design into a DFG.

25.3.5 Mapping the Designs

DPV maps the designs during the compose step using commands in a TCL procedure. Assume commands are used to specify that inputs of the two designs must be equal during specified phases of the transaction.

Feedback

assume commands can also be used to restrict the legal values of inputs. lemma commands indicate which output signals are to be checked for equivalence and at which phases. This data is passed to DPV by calling the following set TCL procedure:

```
set_user_assumes_lemmas_procedure "ual"
proc ual {} {
map_by_name -inputs -specphase 1 -implphase 1 # map inputs
map_by_name -outputs -specphase 1 -implphase 3 # map outputs
assume op = impl.in_opcode(1) == 1'b0 # constrain an input
}
```

The `map_by_name` command is a short-hand way to create a series of assume and lemma commands, one for each input or output in the two designs with matching names. The `-specphase` and `-implphase` switches indicate the timing correspondence between transactions in the two models. The first `map_by_name` command in the TCL procedure is equivalent to the following two commands:

```
assume _scv_assume_0 = (spec.in_a(1)[3:0]) == (impl.in_a(1)[3:0])
assume _scv_assume_0 = (spec.in_b(1)[3:0]) == (impl.in_b(1)[3:0])
```

The second `map_by_name` command is equivalent to the following two commands: lemma _scv_lemma_2 = (spec.out_mul(1) == impl.out_mul(3)) lemma _scv_lemma_3 = (spec.out_madd(1) == impl.out_madd(3)) For this small example, the `map_by_name` command does not save much typing. But for designs with many inputs and outputs, and matching naming schemes, it can make the scripts much more compact and readable.

25.3.6 Running the proofs

To start running the lemmas in a proof, `solveNB` command is used.

25.3.7 Starting DPV App in VC Formal

To start the DPV application in VC Formal, use the following option:

```
vcf -f <hector_tcl_file> <-verdi> -fmode DPV
```

You can also use the `set_fml_appmode` command in the top level script file to use the VC Formal in the DPV application mode:

```
set_fml_appmode DPV
```

Note:

You cannot switch to other application modes once you are in DPV application mode. You cannot switch to DPV once you are in other application modes.

Feedback

This video explains how to invoke VC Formal DPV, load a setup file, and debug lemmas. You can start by setting `VC_STATIC_HOME` and pointing to the correct license file. Use the `vcf` command with `-fml_token` and `-f` options to invoke VC Formal DPV in the GUI mode. Compile the implementation model using `compile_impl` and the specification model using `compile_spec`. Then, use the `compose` command to create a wrapper and instantiate both designs, visible in the hierarchy window.

To generate and run proofs, use the solveNB command with a proof name. Alternatively, generate proofs without running them using the gen_proof command. Debugging falsified lemmas involves analyzing counterexamples in nWave, where mismatched signals are marked red. To resolve issues, modify lemma preconditions, such as adding valid signals, and rerun the session using the restart button. This process ensures improved lemma convergence.

25.4 Performing DPV Analysis in VC Formal GUI

VC Formal provides the capability to perform DPV checks using the VC Formal GUI. When you run a DPV Tcl script in the `vcf_shell` in GUI mode:

- The DPV proofs/subproofs are listed as tasks/subtasks in the *Task List*.
- The Lemmas and Assume properties are listed in the *GoalList* tab.

The DPV GUI can be opened in the following ways:

- VC Formal GUI can be opened with `vcf -f … -fmode DPV -gui`
- The `set_fml_appmode DPV`is set in the top level script file

  Note:

  In the DPV application mode, the unproductive jobs are killed automatically. You must set the `set_vcf_auto_job_kill` formal variable to false to stop killing the unproductive jobs automatically. You cannot use the `proofwait -nokill` command to stop this.

To run the DPV checks:

1. Click to run the DPV task.

   The status of the lemmas appears in the *Status* column.

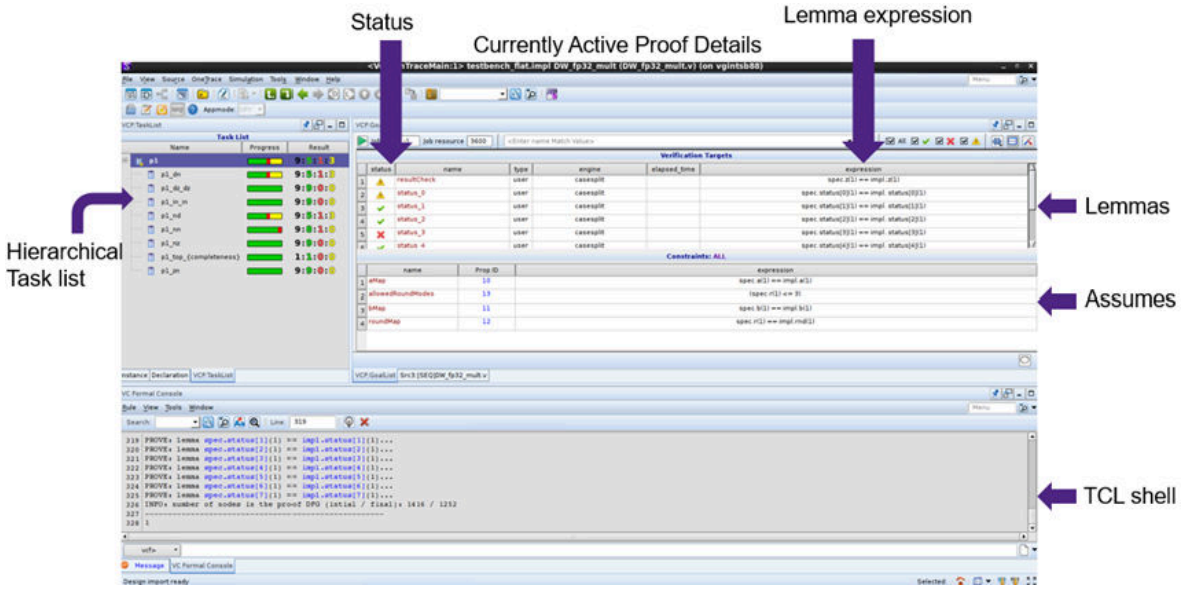
   Note:

   You can get the wallclock time consumed for solving a task using the `get_task_runtime <task_name>` command. The command reports the total runtime for a task in HOURS:MM:SS format.

Figure 237 shows the DPV application mode in VC Formal GUI

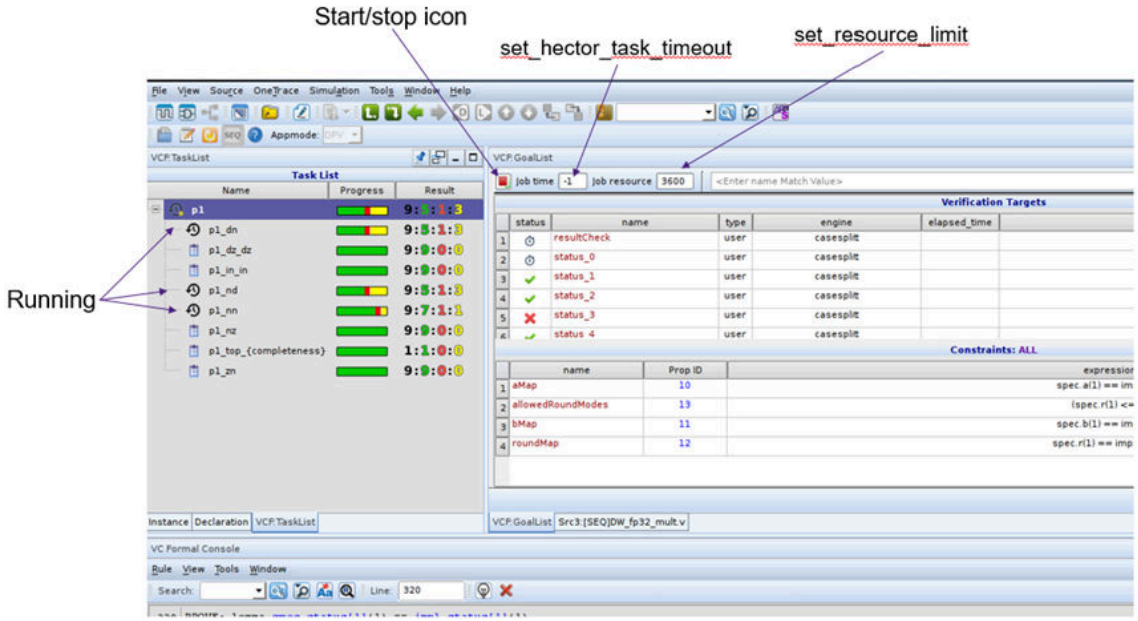
Feedback

Figure 237: DPV Application Mode in VC Formal GUI



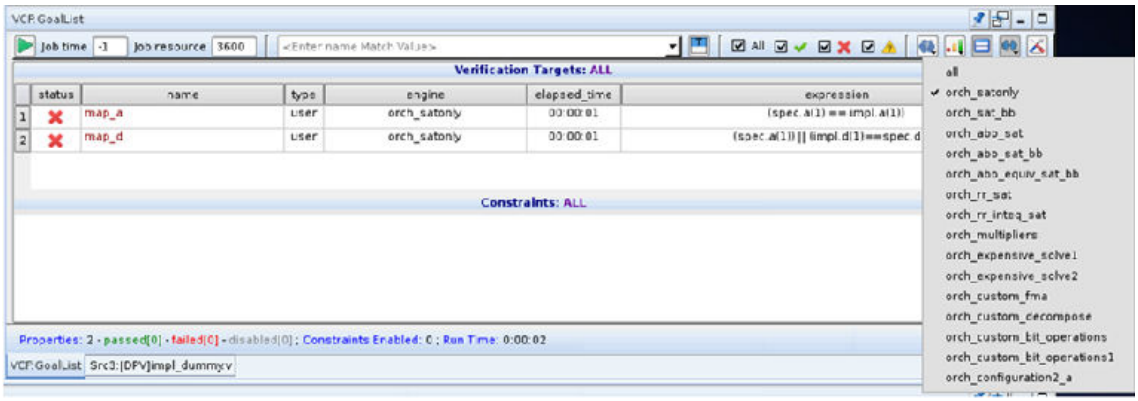
- Figure 238 shows the different limits that can be set before running the DPV task.

Figure 238: Different Settings of DPV mode



Feedback

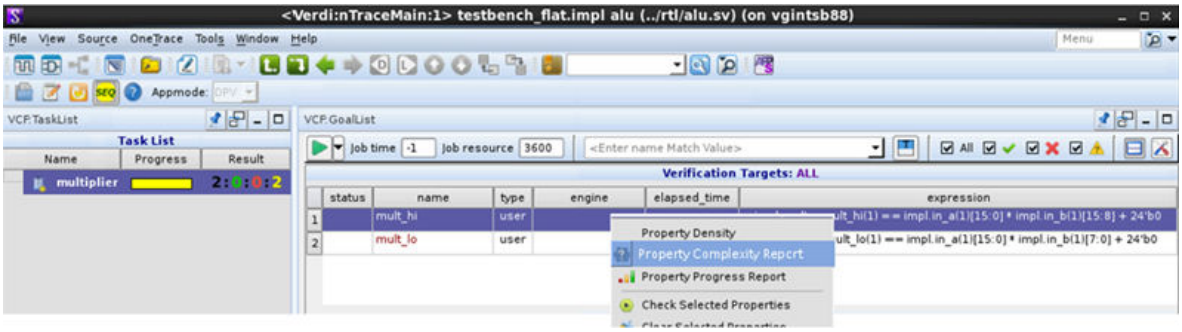
- You can select solve script for a proof. You can move from one proof to the other, and select a different solve script. VC Formal runs the lemmas in the proofs with the chosen solve script/scripts as shown in the following figure:



- You can add case splits on the fly for a proof using the caseSplitStrategy command, or can enable the case split (already defined inside a tcl procedure before in setup file) using `set_hector_case_splitting_procedure` as supported by tcl commands.
- You can view the Complexity Report in the DPV application mode.

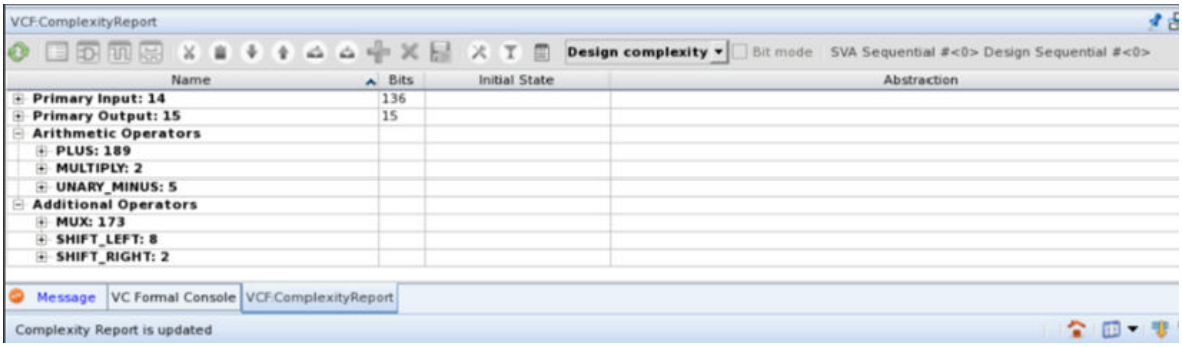
You can generate the complexity report for the designs from GUI using the icon and selecting *Show Complexity* from drop-down menu.

You can generate complexity report for properties by selecting the property, and from the RMB menu, select *Property Complexity Report*.

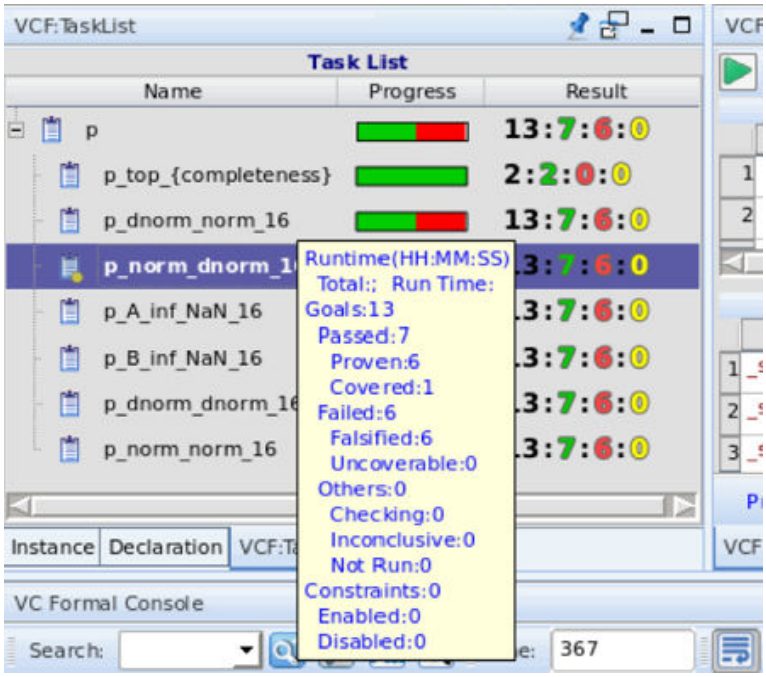


The following figure shows an examples of DPV complexity report:

Feedback

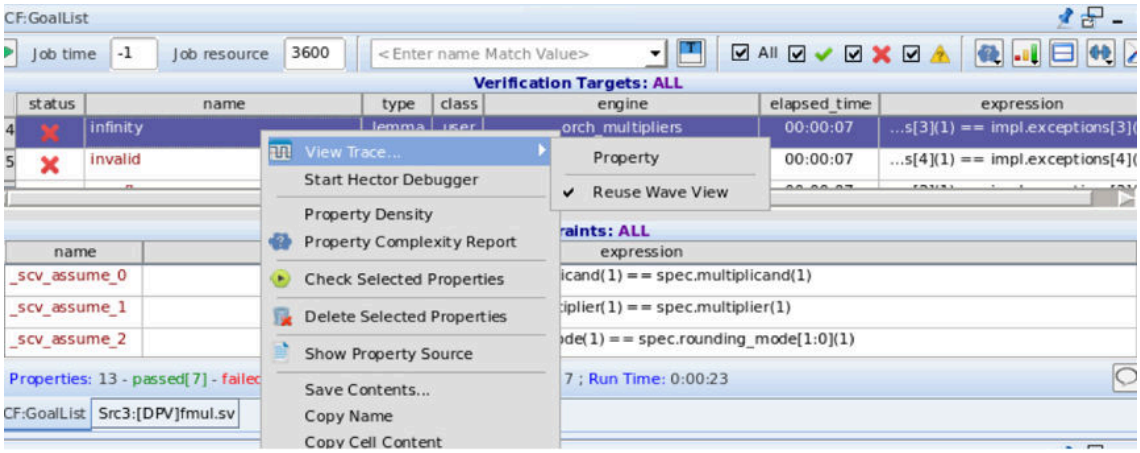


- To select a proof or subproof, double-click on the proofname in Task List. This is equivalent to running the `fvtask <proofname>` command from vcf console

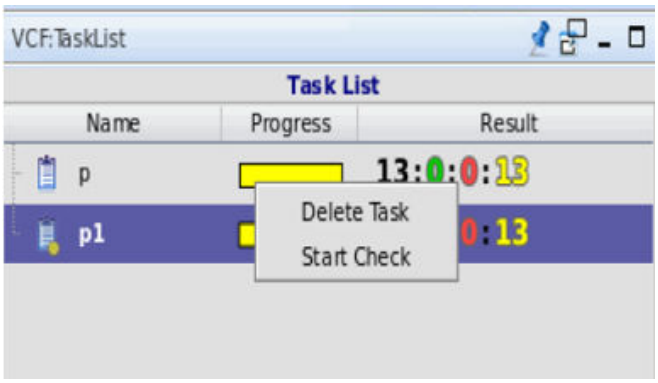


- The RMB menu for Lemma is available. The RMB menu options for a lemma is illustrated in the following figure.

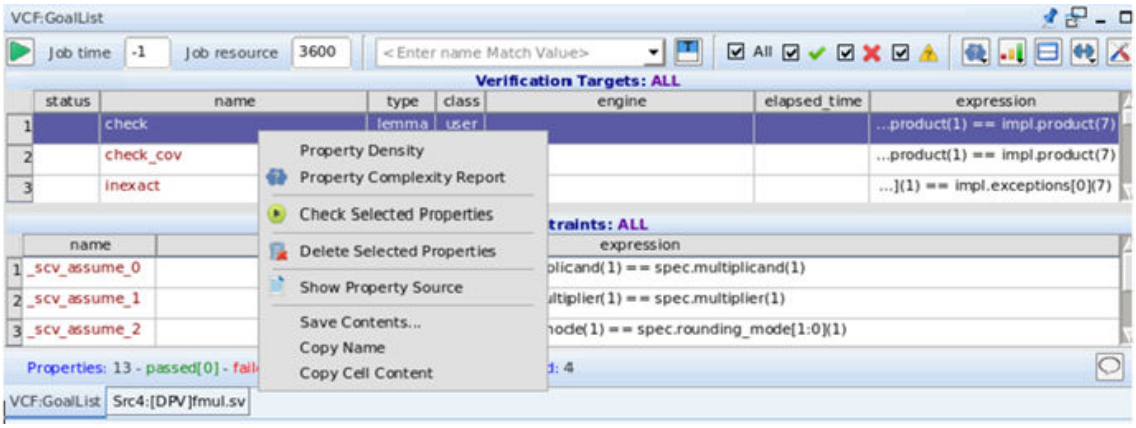
Feedback



- The RMB Menu option for proof/subproof in a tasklist view in Verdi GUI is illustrated in the following figure:



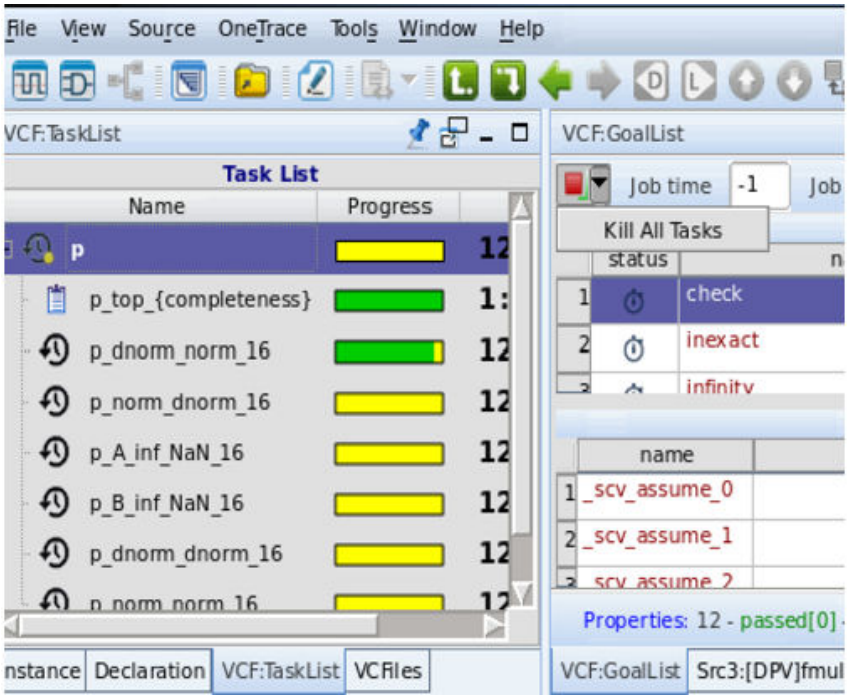
- To run selected properties in Verdi GUI, select the desired properties, and click *Check Selected Properties*from RMB menu.



- To view c/c++ source code in GUI, select the C/ C++ file in the *VC Files* pane.

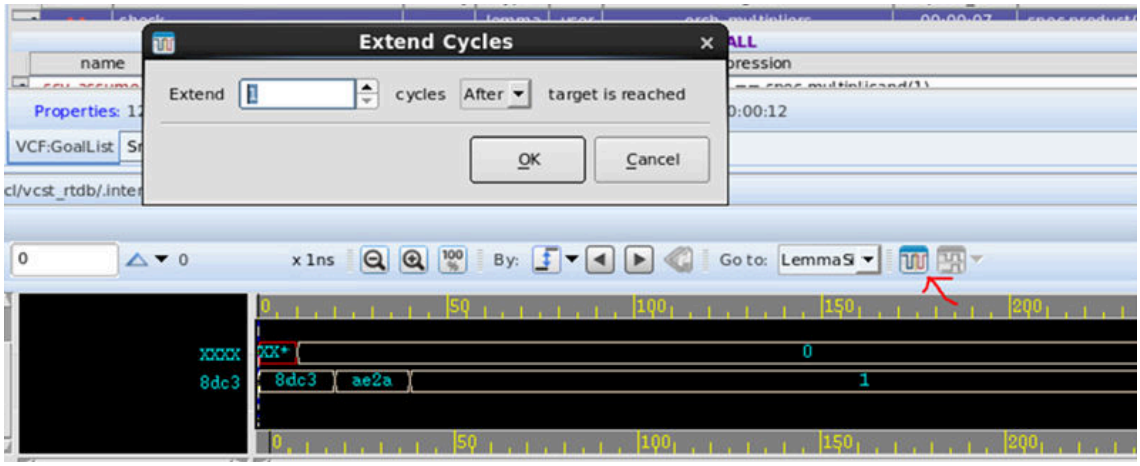
Feedback

- You can kill all the tasks from the GUI using the *Kill All Tasks* drop-down option present beside the "stop/start" icon to kill all tasks in GUI. It is equivalent to the `killTasks -all` command executed from the `vcf` shell.



- You can extend trace by n cycles by clicking the icon as shown in the following figure:

Feedback



- You can view the proof progress in the vcf console. To view the real time progress of proofs of different tasks, set the following variable in Tcl file:

```
set_fml_var set_vcf_display_lemma_status true
```

When you set the variable, the progress of the proofs is displayed as shown:

```
-----------------------------------------------------
[PROP_I_RESULT]: p_top_{completeness}/completeness success orch_satonly 00:00:00
[PROP_I_RESULT]: p_dnorm_norm_16/inexact cond-failed orch_multipliers 00:00:02
[PROP_I_RESULT]: p_dnorm_norm_16/underflow cond-failed orch_multipliers 00:00:02
[PROP_I_RESULT]: p/inexact cond-failed casesplit 00:00:02
[PROP_I_RESULT]: p/underflow cond-failed casesplit 00:00:02
[PROP_I_RESULT]: p/check cond-failed casesplit 00:00:00
[PROP_I_RESULT]: p/overflow cond-failed casesplit 00:00:00
[PROP_I_RESULT]: p/infinity cond-failed casesplit 00:00:00
[PROP_I_RESULT]: p/invalid cond-failed casesplit 00:00:00
[PROP_I_RESULT]: p_dnorm_norm_16/check cond-failed orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/overflow cond-failed orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/infinity cond-failed orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/invalid cond-failed orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_bd_0(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_bd_1(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_shft_0(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_shft_1(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_shft_2(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_dnorm_norm_16/spec._scv_shft_3(1) success orch_multipliers 00:00:10
[PROP_I_RESULT]: p_norm_dnorm_16/inexact cond-failed orch_multipliers 00:00:02
[PROP_I_RESULT]: p_norm_dnorm_16/underflow cond-failed orch_multipliers 00:00:02
```

- The source code location of AEP's in DPV is provided in the expression column of the property table.

25.5 Status of Lemmas in VC Formal DPV

The following table lists the different statuses of properties in VC Formal DPV.

Feedback

Table 45: Status of Lemmas in VC Formal DPV

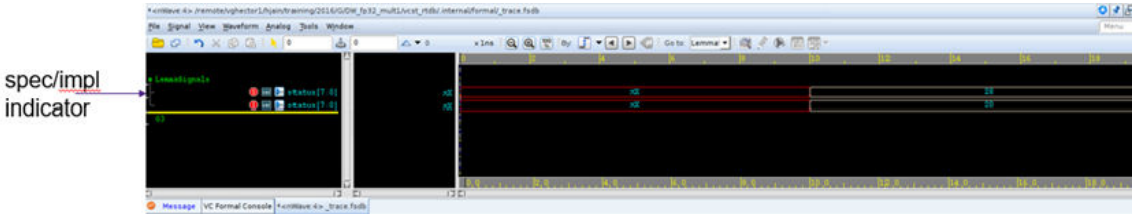
VCF status	GUI icon
success	
failed	
cond-success	
cond-failed	
running inconcl (top level proof)	
inconcl/killed/error/conflc	
nottried/scheduled	

25.6 Debugging Failures

- To debug failure in GUI: Right click on (unsuccessful lemma) and select *View Trace*. Or double-click to open the trace.

  The trace-failure with related signals in waveform appears.

Figure 239: Waveform Trace



Feedback

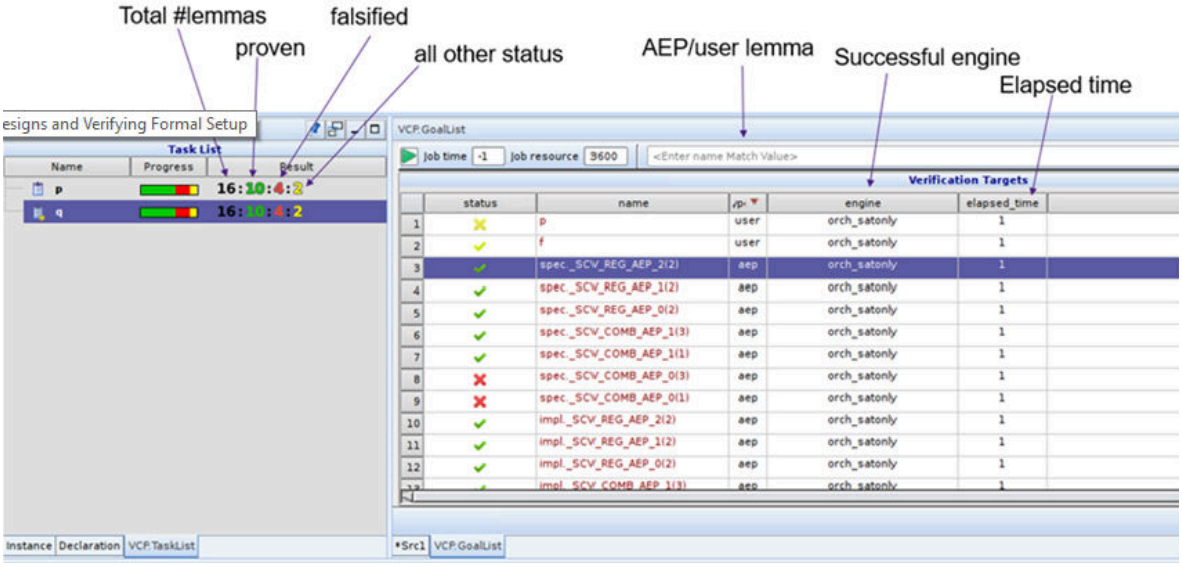
25.6.1 Correcting Errors

Properties can be falsified either due to incorrect lemma or incorrect/missing assumption or bug in source code. In all of these cases, perform the following steps to correct the falsified properties:

1. Edit the Tcl or RTL source code from DPV GUI (currently, you cannot edit C++ source code from DPV GUI).
2. Save the changes.
3. Click to restart VC Formal DPV.
4. Click to rerun the lemmas.

The status of all the lemmas can be viewed in the task list as shown in Figure 240.

Figure 240: Status of the Lemma for a Task



Feedback

This video explains how to debug a falsified lemma in VC Formal's Data Path Validation (DPV) application. When a lemma is falsified, double-click the falsification marker to replay the counter-example and view the waveform with related signals. To debug internal variables not automatically dumped, use the `Hector::show()` function in the C/C++ model, passing the desired variables. Recompile the model and rerun the proof to include these variables in the FSDB.

Alternatively, use the DDD debugger integrated with GDB. Ensure DPV generates the spec.exe executable. Invoke DDD from VC Formal using the command `simcex -gdb <lemma_name>`. This loads the counter-example file replay.cex into DDD for step debugging. Breakpoints can be added, and internal variables inspected. To debug externally, create the counter-example file using `simcex -extract <lemma_name>` and load it with spec.exe in DDD. Share spec.exe and replay.cex with designers for further analysis.

26

Appendix A - Design Query Commands

This chapter describes access commands, and attributes supported in the VC Static Platform for the various design objects. These commands operate using collections in a similar manner as Design Compiler (DC) and PrimeTime (PT). The chapter is organized into the following sections:

- Terminology Used in the VC Static Tools
- Design Query Commands
- Using Collections
- Using Attributes

26.1 Terminology Used in the VC Static Tools

This section describes the terminology used in the VC Static tools.

26.1.1 Designs

Designs are circuit descriptions that perform logical functions. They are described in various design formats, such as VHDL or Verilog HDL. Logic-level designs are represented as sets of boolean operators. Gate-level designs, such as netlists, are represented as interconnected library cells. Designs can be flat or hierarchical.

Flat Designs

Flat designs contain no sub-designs and have only one structural level. They contain library cells, nets and ports.

Hierarchical Designs

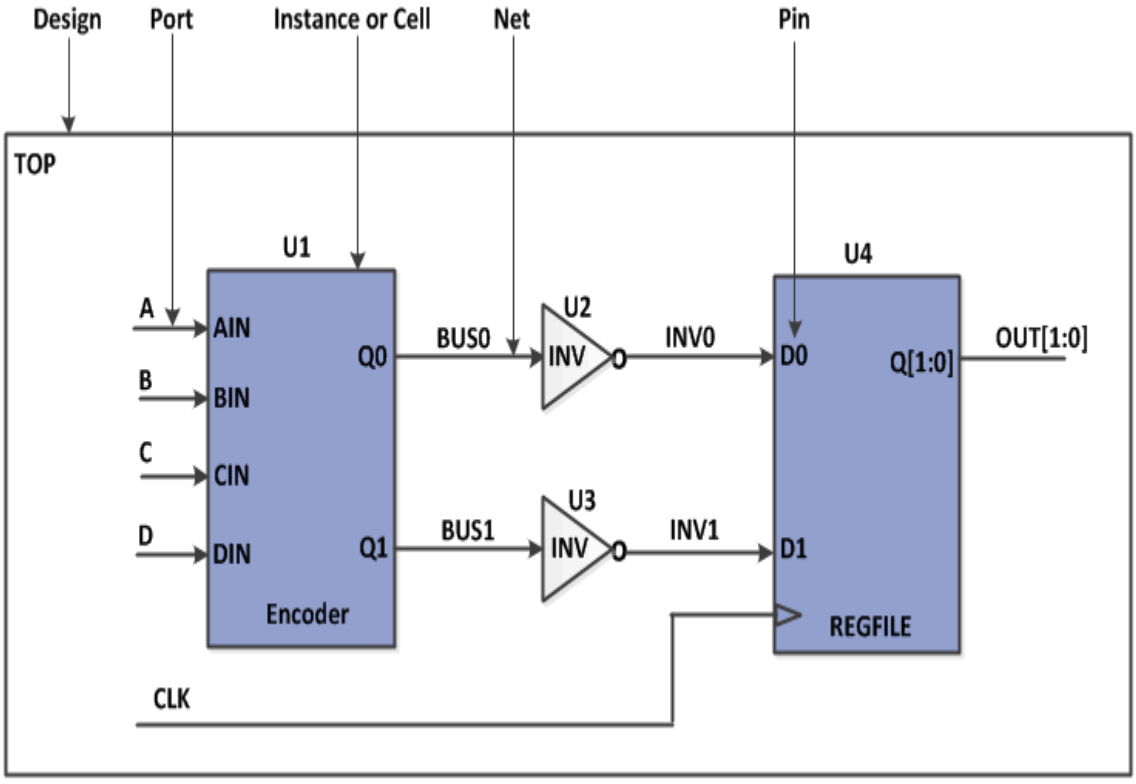
A hierarchical design contains one or more designs as sub-designs. Each sub-design can further contain other sub-designs, creating multiple levels of design hierarchy. Designs that contain sub designs are called parent designs.

Feedback

26.1.2 Design Objects

Figure 241 shows the design objects in a design called top. The VC Static Platform design-query commands and attributes are directed toward specific design objects.

Figure 241: Design Objects in a Design Called TOP



Design

A design consists of instances, nets, ports, and pins. It can contain sub-designs and library cells. In Figure 241, the designs are top, encoder, and regfile. The active design (the design being worked on) is called the current design.

Note:

Unlike DC, the VC Static Platform has no concept of current_design which is changeable. In this sense, the VC Static Platform current_design is always the design specified as the top design using the -top option in the read_file command.

Instance or Cell

An instance is an occurrence in a circuit of a reference (a library component or design) loaded in memory. Each instance has a unique name. A design can contain multiple instances. Each instance may point to the same or a different reference, but always has a unique name to distinguish it from other instances. An instance is also known as a cell.

A unique instance of a design within another design is called a hierarchical instance. A unique instance of a library cell within a design is called a leaf cell. Some commands work within the context of a hierarchical instance of the current design.

Ports

Ports are the inputs and outputs of a design. The port direction is designated as input, output, or inout.

Pins

Pins are the input and output of cells (such as gates and flip-flops) within a design. The ports of a sub design are pins within the parent design.

Nets

Nets are the wires that connect ports to pins and pins to each other.

26.2 Design Query Commands

This section lists the various Tcl commands that is available for you to access design objects after the design is successfully built.

- `all_connected`
- `all_designs`
- `all_fanin`
- `all_fanout`
- `all_inputs`
- `all_instances`
- `all_outputs`
- `all_registers`
- `get_blackbox`
- `get_cells`
- `get_designs`

- `get_lib_cells`
- `get_lib_pins`
- `get_nets`
- `get_sources`
- `get_trace_paths`
- `get_trace_path_elements`
- `get_object_name`
- `get_pins`
- `get_ports`
- `remove_blackbox`
- `report_trace_paths`
- `set_app_var`
- `set_blackbox`
- `set_message_severity`

Many commands supported by the VC Static Platform, construct collections of objects. In almost all cases, the commands allow various mechanisms for flexible specifications of what objects should be included in these collections. These mechanisms include:

- Wildcard name specification. These wild-cards can be glob-style, or regular expression style.
- Boolean expressions of object attributes and values.

26.2.1 Regular Expression Behavior

Many commands have a `-regexp` switch. When used, this switch has the following behavior:

`-regexp` - The command views the patterns (name) argument as a regular expression rather than a simple wildcard pattern.

This option also modifies the behavior of the =~ and !~ filter operators to use regular expressions rather than simple wildcard patterns.

Regular expression matching is similar to the Tcl regexp command. When using the `-regexp` option, take care in quoting the pattern argument and filter expression. Using rigid quoting with curly braces around regular expressions is recommended. Note that

regular expressions are always anchored; that is, the expression is assumed to begin matching at the beginning of an object name and end matching at the end of an object name. You can widen the search by adding ".*" to the beginning or end of the expressions, as needed.

For commands that have both switches, the -regexp and -exact options are mutually exclusive. You can specify only one of these options.

26.2.2 Filter Expression Behavior

Many commands have a `-filter` switch which allows the collection to be composed as a function of attributes of the objects being collected.

`-filter expression` - Filters the collection with the specified expression.

For each object which might be added to the collection, the expression is evaluated based on the objects's attributes. If the expression evaluates to true, the object is included in the result.

To see the list of object attributes that you can use in the expression, use the `list_attributes -application -class <class>` command.

26.2.3 Object Name Filtering

Almost all commands which return collections have a patterns field. This field is used as a convenience to filter the collection based on object name and wild-card value for the name.

`patterns` - Creates a collection of objects whose names match the specified patterns. Patterns can include the * (asterisk) and ? (question mark) wildcard characters. For more information on using and escaping wildcards, see the Using Wildcard Characters. Pattern matching is case sensitive unless you use the `-nocase`option. If `-filter` is also specified, then objects must evaluate to true for the filter expression, and the object name must pass the patterns field.

If you do not specify any of these arguments, the command uses * (asterisk) as the default pattern.

26.2.4 Getting Default or Current Values for Every Argument of the Command

Use the `get_command_option_values` command to query the default or current values for every argument of the command that has default and/or current-value-tracking enabled.

A "Tcl array set compatible" (possibly empty) list of argument names and values is returned as the Tcl result. The even-numbered entries in the list are the names of the

Feedback

arguments that were enabled for default-value- tracking or current-value-tracking and had at least one of these values set to a not-undefined value. Odd-numbered entry in the list is the default or current value of the argument name preceding it in the list.

Arguments that were not enabled for either default-value-tracking nor current-value-tracking are omitted from the output list. Similarly, arguments that were enabled for default-value-tracking or current-value- tracking, but for which no (not-undefined) default or current value is set, are also omitted from the result list.

If neither -current nor -default argument is specified, for each command argument that has either default-value-tracking or current-value-tracking (or both) enabled, the value returned is as follows:

- The current value is returned if current-value-tracking is enabled and a (not-undefined) current value is set.
- The default value is returned if default-value-tracking is enabled and a (not-undefined) default value is set.
- Otherwise the name and value pair for the argument is not included in the result list.

If `-current` is specified, the value returned for an argument is the current value if current-value-tracking is enabled, and a (not-undefined) current value has been set; otherwise the name and value pair for the argument is omitted from the result list.

If `-default` is specified, the value returned for an argument is the default value if default-value-tracking is enabled, and a (not-undefined) default value has been set; otherwise the name and value pair for the argument is omitted from the result list.

The result list from `get_command_option_values` includes argument values of both dash arguments and positional arguments (assuming that both kinds of argu ments of a command have been enabled for value-tracking). The command issues a Tcl error in a variety of situation as if an invalid command name was passed in with the -command argument.

Syntax

```
get_command_option_values [-default] [current] -command command_name
```

Where:

- `[-default]`: Gets the default or current option values, if available.
- `[-current]`: Gets the current option values, if available.
- `-command command_name`: Gets the option values for this command.

Example

```
vc_static_shell>test -opt1 10 -opt2 20
vc_static_shell>get_command_option_values -command test
-bar1 10 -bar2 20
```

26.3 Using Collections

VC Static Platform builds an internal database of the design structure and the attributes applied to the design. This database consists of several classes of objects, such as designs, libraries, ports, cells, nets, and pins. Many VC Static Platform commands operate on these objects.

A collection is a group of related objects exported to the Tcl user interface. Collections have an internal representation of the objects, and sometimes a string representation. The string representation is generally used only for error messages.

You can create collections of objects, and then apply a set of commands to interact with those collections. Collections can be homogeneous (contain objects of one type) or heterogeneous (contain objects of many types).

The collection based commands are divided into the following three categories:

- Commands that create collections of objects for use by another command.
- Commands that manipulate collections.
- Commands that query objects for you to view.

You can use wildcards and filtering criteria to narrow the focus of the collection or filter a collection. You can refer to collections in variables for reading (and in some cases writing) attributes, performing custom reporting or use in analysis scripts.

26.3.1 Creating Collections

In the VC Static Platform, you can pass a collection to a command in the following two ways:

- Use the result of a command that creates the collection.
- Use a variable that contains a collection.

For example, the following command creates a collection of nets and assigns that collection to a variable named mynets:

```
set mynets [get_nets]
```

Feedback

Primary Commands that Create Collections

The following commands are used to directly create collections of design objects:

- `get_cells`: Creates a collection of cells or instances
- `get_designs`: Creates a collection of designs
- `get_lib_cells`: Creates a collection of library cells
- `get_lib_pins`: Creates a collection of library cell pins
- `get_nets`: Creates a collection of nets
- `get_pins`: Creates a collection of pins
- `get_ports`: Creates a collection of ports

When used at the command prompt, any command that creates a collection will implicitly query the collection. You can also directly view the contents of a collection by using the `query_objects` command.

Note:

Do not use the echo, puts, or printvar commands as they will only return the string representation of the collection. This is the name arbitrarily assigned by the VC Static Platform to serve as an opaque handle to the collection.

Primary Collection Command Example

The `get_cells` command creates a collection of cells in the design. The syntax is as follows:

```
get_cells [-hierarchical] [-quiet][-regexp] [-nocase] [-exact] [-filter
 expression]
patterns | -of_objects objects
```

For example, enter the following command to query the cells with name beginning with `o` and referencing an FD2 library cell:

```
get_cells "o*" -filter {ref_name == FD2}
{"o_reg1", "o_reg2", "o_reg3", "o_reg4"}
```

Although the output looks like a list, it is not. The output is only a display of the `get_cells` result. The collection was not assigned to a variable nor passed to another command. So, it was deleted as soon as it was displayed.

Given a collection of pins, use the following commands to query the cells connected to those pins:

```
set pins [get_pins o*/CP]
{"o_reg1/CP", "o_reg2/CP"}
```

```
get_cells -of_objects $pins
{"o_reg1", "o_reg2"}
```

The following commands also create collections of design objects based on some sort of analysis or algorithm:

- `all_fanin`: Creates a collection of fanin of pins, ports, or nets
- `all_fanout`: Creates a collection of fanout of pins, ports, or nets
- `all_connected`: Creates a collection of objects connected to a net, pin, or port
- `all_inputs`: Creates a collection of input ports
- `all_outputs`: Creates a collection of output ports
- `all_registers`: Creates a collection of register cells or pins

Many other commands in the VC Static Platform return collections of objects other than design objects. The use of collections for those objects is exactly the same as described here.

Each collection created by a command such as `get_nets` has its own context. You cannot add to, remove from, or compare collections from different contexts such as different designs, different ports, or different cells. To compare objects, create one large collection containing all the objects of interest, and then sort, filter, or manipulate the objects in that collection.

For example, the following commands create two different sets of cells that cannot be compared:

```
set someCells [get_cells -filter …]
set moreCells [get_cells -filter …]
```

Even if the two collections represent the same cells, they are different objects, so comparing them will always report a mismatch:

```
if {$someCells != $moreCells} { echo "Mismatch!" }
```

The `compare_collections` command is used to compare two collections.

26.3.2 Saving Collections

To save a collection for later use, assign the result of the collection-creating command to a variable. For example:

```
set domA [get_power_domains -name domainA]{"domainA"}
set domB [get_power_domains -name domainB]{"domainB"}
set badXovers [get_crossovers -source $domA -dest $domB]
```

The result of the first two commands is two collections of one power domain object each. The third command then creates a collection of crossovers between the two power domains, saving the collection in the `badXover` variable.

In many cases, the collection consumes significant memory. Saving the collection in a variable helps the script to be more memory efficient. You can deallocate the collection by resetting the variable to which a collection was assigned (domA and domB). Alternatively, you can assign some other value to the variable which references the collection.

A collection is active only as long as it is referenced. Typically, a collection is referenced when a variable is set to the result of a command that creates it, or when it is passed as an argument to a command or a procedure. For example, if you save a collection of ports as follows:

```
set myports [get_ports *]
```

Either of the following two commands will delete the collection referenced by the myports variable:

```
unset myports
set myports "newvalue"
```

Collections can be implicitly deleted when they go out of scope, in other words, when the parent of the objects within the collection is deleted. For example, if your collection of ports is owned by a design, the collection is implicitly deleted when the design that owns the ports is deleted. When a collection is implicitly deleted, the variable that referenced the collection still holds a string representation of the collection. However, since the collection is gone, this string value is not useful.

26.3.3 Querying Objects in a Collection

To view the contents of a collection, use the `query_objects` command. This command searches for and displays objects in the VC Static Platform database. Do not use the `echo`, `puts`, or `printvar` commands, as doing so will only return the string representation of the collection, a name arbitrarily assigned by the VC Static Platform to serve as a handle to the collection.

The output of the `query_objects` command is similar to the DC find command. The `query_objects` command does not have a meaningful return value; it simply displays the objects found and returns the empty string.

Implicit Query

When commands that create collections are issued from the command prompt, they implicitly query the collection.

Feedback

Use the following command to retrieve ports that match the pattern in*:

```
get_ports in*
{"in0", "in1", "in2"}
```

You can also use the following commands to see the same ports.

```
query_objects [get_ports in*]
{"in0", "in1", "in2"}
query_objects -class port in*
{"in0", "in1", "in2"}
```

Using Wildcard Characters

26.3.4 Using Wildcard Characters

Most commands that create collections allow a list of patterns, which contain wildcard characters. The VC Static Platform uses some UNIX global-style matching operators, including:

- * – Matches 0 to n characters
- ? – Matches one character

Example:

Given ports i0, i1, in0, and in1, notice the queries in this sequence of commands:

```
get_ports i*
{"i0", "i1", "in0", "in1"}
query_objects [get_ports i?]
{"i0", "i1"}
```

Commands that create explicit collections, such as `get_cells`, allow you to search in the current instance hierarchically by using the `-hierarchical` option. The rules for different kinds of searches are as follows:

- Using a wildcard pattern alone matches leaf names in the current instance.

  Example - `get_cells i1*`

- Using a wildcard pattern with the `-hierarchical` option matches leaf names at each level of the hierarchy.

  For example, to find all cells in the hierarchy with the leaf name containing n1, enter:

  `get_cells *n1* -hierarchical`

- When you search for a wildcard pattern that contains the hierarchy separator, the VC Static Platform breaks up the pattern around the hierarchy separator and matches each piece at progressively deeper levels of the hierarchy.

For example, to find the cells in i1 that begin with i2, then return a collection of cells in each i1/i2* that has a leaf name of n1, enter:

```
get_cells i1/i2*/n1
```

- If an object you specify has a name that contains a backslash character (\) or any wildcard characters (* or ?), it is better to use the `-exact` option to find the object. Using `-exact` makes the primary collection creation command such as `get_cells` or `get_ports` consider the patterns argument to be a list of exact strings rather than a list of patterns with wildcards.

  For example:

  ```
  get_cells -exact [list {*cell*1}] ;# finds cell actually named *cell*1
  get_cells -exact [list {a\b1}] ;# finds cell actually named a\b1
  ```

- Wildcard matching uses the same logic as the Tcl string match command. Without the -exact option, each wildcard match character needs to be backslash-escaped in order to be considered exactly and not as a wildcard.

  For example:

  ```
  get_cells [list {\*cell\*1}]
  get_cells [list {a\\b1}]
  ```

Remember that the patterns argument to the collection creation command is a list. If you do not supply a well-formed list, additional backslashes will be necessary to communicate these special characters.

26.3.5 Filtering Collections

You can filter collections by using the `-filter` option with the primary commands that create collections.

Using the -filter Option

Many commands that create collections accept a `-filter` option that specifies a filter expression. A filter expression is a string composed of a series of logical expressions describing a set of constraints you want to place on a collection.

Each sub expression of a filter expression is a relation contrasting an attribute name such as name or direction with a value such as N23 or input, by means of an operator such as == or !=.

For example, the following command gets the cells in U1 that have an area no greater than 12 or reference a design (or library cell) named AN2P, AO2P, and so on. The command then assigns the collection to the Cells variable.

```
set Cells [get_cells "U1/*" -filter {area <= 12 || ref_name =~ "A*P"}]
```

For more information, see section Using Attributes.

Using Filter Operators

The filter language supports the following logical operators:

- AND or && – Logical AND (not case-sensitive)
- OR or || – Logical OR (not case-sensitive)

You can group logical expressions with parentheses to enforce precedence; otherwise, the VC Static Platform evaluates the expressions from left to right.

The filter language supports the following relational operators:

- == (Equal)
- != (Not equal)
- > (Greater than)
- < (Less than)
- >= (Greater than or equal to)
- <= (Less than or equal to)
- =~ (Matches pattern)
- !~ (Does not match pattern)

  For example, in the following filter expression,

  ```
  {area <= 12 || ref_name =~ "A*P"}
  ```

- Area is an attribute (or identifier)
- <= is a relational operator
- 12 is a value
- || is the logical OR operator

The filter language also supports the following existence operators:

- defined
- undefined

An existence operator determines whether an attribute is defined for an object.

For example:

```
sense == setup_clk_rise and defined(sdf_cond)
```

Feedback

The right side of a relation can consist of a string or a numeric literal. You do not need to enclose strings in quotation marks. This method is useful because a filter expression is usually the value for an argument, and the entire expression is enclosed in quotation marks.

As shown in the following command, you do not need to enclose the word in with quotation marks:

```
set iports [get_ports * -filter {direction == in}]
```

However, if an expression contains characters that are part of the filter language syntax, you must use curly braces {} to enclose the expression and double quotation marks “” to enclose string operands.

As shown in the following example, parentheses are part of the filter language, so they are double quoted, and the complete expression is grouped in curly braces:

```
set x [filter_collection $ports {full_name =~ "D(*)"}
```

Relation Rules

Parsing a filter expression can fail due to the following reasons:

- Syntax problems
- An invalid attribute name
- A type mismatch between an attribute and the value

The following are the basic relational rules:

- String attributes can be compared with any operator.
- Numeric attributes cannot be compared with patterns.
- Boolean attributes can be compared only with == and !=. The value can be only true or false.

Using Pattern Match Filter Operators

The pattern match filter operators, =~ and !~, behave differently in different circumstances. They do one of two types of pattern matching as follows:

- Simple wildcard matching (the default)
- Full regular expression matching

Commands that provide a `-filter` option also provide a `-regexp` option to enable you to change the filtering to full regular expressions. The `filter_collection` command also provides a `-regexp` option to enable full regular expressions.

Feedback

Using the filter_collection Command

The `filter_collection` command is useful for filtering an existing collection. However, it is usually less efficient than using the `-filter` option. For more information on the -filter option, see the Using the -filter Option section.

The `filter_collection` command takes a collection and a filter expression as arguments. The result is a new collection, or an empty string if no objects match the criteria. The following two commands are equivalent:

```
get_cells * -filter "ref_name =~ A*P"
filter_collection [get_cells *] "ref_name =~ A*P"
```

The `filter_collection` command is useful in translating DC scripts that use the filter command. In addition, if you derive several collections from one larger collection, using the `filter_collection` command might be more efficient than using the `-filter` option, as shown in the following series of commands:

```
set acells [get_cells "*"]
{"u1", "u2", "u3", "u4"}
set ands [filter_collection $acells "ref_name =~ AN*"]
{"u1", "u2"}
set ors [filter_collection $acells "ref_name =~ OR*"]
{"u3", "u4"}
```

Optionally, the `filter_collection` command accepts the `-regexp` option, which changes =~ and !~ to perform real regular expression matching.

26.3.6 Iterating Over the Elements of a Collection

The `foreach_in_collection` command iterates over a collection.

```
foreach_in_collection itr_var collections body
```

For more information on the `foreach_in_collection` command, see the *VC Static PLatform Command Reference Guide*. You can nest the `foreach_in_collection` command within other control structures, including another `foreach_in_collection` command.

During each iteration, the `itr_var` variable is set to a collection of exactly one object. Any command that accepts one of the collections also accepts the `itr_var` variable because they are of the same type (a string representation of a collection).

To generate a separate report for each cell, use the following commands:

```
foreach_in_collection itr [get_cells o*] {
redirect [format "%s%d" $fbase $ndx]report_cell
}
```

Feedback

Note:

You can use the foreach command to iterate over a list, but not a collection. Attempting to do so will cause the collection to be deleted.

26.3.7 Removing From a Collection and Adding to a Collection

The VC Static Platform allows you to remove objects from a collection or add objects to a collection by using the `remove_from_collection`, `add_to_collection`, and `filter_collection` commands.

The following is the syntax:

```
remove_from_collection collection object_spec collection
add_to_collection collection object_spec [-unique]
filter_collection collection expression [-regexp] [-nocase]
```

You can remove objects using the `filter_collection` command or by filtering objects when you initially create the collection. However, there are some constructs, such as removing the elements in one collection from another collection, that you cannot accomplish with filtering.

Use the `remove_from_collection` and `add_to_collection` commands for this purpose. The arguments to both commands are a collection and a specification of the objects that you want to add or remove. The specification can be a list of names, wildcard strings, or other collections.

The result of these commands is a new collection, or an empty string if the operation results in zero elements in the case of the `remove_from_collection` and `filter_collection` commands. The first argument (the base collection) is a read-only argument.

Examples:

This command sets the ports variable for collection of all ports except for clock:

```
set ports [remove_from_collection [get_ports "*"] CLOCK]
```

This command results in a collection of all cells in the design except those in the top level of hierarchy:

```
set lcells [remove_from_collection [get_cells * -hier] [get_cells *]]
```

You can add objects to a collection with the add_to_collection command:

```
set isos [add_to_collection [get_cells i*] [get_cells o*]]
```

Most collections are homogeneous since commands suchas `get_ports`, `get_cells`, create homogeneous collections. Some commands, such as `all_connected`, create

heterogeneous collections. You can also create heterogeneous collections using the `add_to_collection` command.

For example:

```
set a [get_ports PH*]
{"PH1", "PH2"}
set b [get_pins -regexp {reg(0|1)/CP}]
{"reg0/CP", "reg1/CP"}
query_objects -verbose [add_to_collection $a $b]
{"port:PH1", "port:PH2", "pin:reg0/CP", "pin:reg1/CP"}
```

For more information on the `remove_from_collection`, `add_to_collection`, and `filter_collection` commands, see the *VC Static PLatform Command Reference Guide*.

26.3.8 Sorting Collections

You can sort a collection using the `sort_collection` command. The following is the syntax:

```
sort_collection [-descending] collection criteria
```

The VC Static Platform sorts according to the list of attributes you specify and are ascending by default. You can reverse the order using the `-descending` option. In an ascending sort, the VC Static Platform first sorts boolean attributes with the objects that have the attribute set to false, then sorts the objects that have the attribute set to true. In the case of a sparse attribute, objects that do not have the attribute follow the objects that have the attribute.

For example, enter the following command to sort the ports by direction, then by full name:

```
sort_collection [get_ports *] {direction full_name}
{"in1", "in2", "out1", "out2"}
```

To get a collection of hierarchical cells decreasing alphabetically, followed by leaf cells decreasing alphabetically, sort the collection of cells using the `is_hierarchical` and `full_name` attributes. In the following example, i1 and i2 are hierarchical, and u1 and u2 are leaf cells:

```
set c [get_cells {u2 i2 u1 i1}]
{"u2", "i2", "u1", "i1"}
set cs [sort_collection -descending $c {is_hierarchical full_name}]
{"i2", "i1", "u2", "u1"}
```

The `is_hierarchical` attribute is boolean, so in a descending sort, cells that have the attribute set to true (i1 and i2) are first. To resolve the equality for the `is_hierarchical` attribute on cells i1 and i2, use the `full_name` attribute.

For more information on the `sort_collection` command, see the *VC Static PLatform Command Reference Guide*.

Feedback

26.3.9 Using Collection Utility Commands

The VC Static Platform provides the following additional commands for manipulating collections:

- `sizeof_collection`
- `compare_collections`
- `copy_collection`
- `index_collection`

Note:

Some collections cannot be indexed, copied, or compared.

For more information on these commands, see the *VC Static PLatform Command Reference Guide*.

Determining the Size of a Collection

The `sizeof_collection` command returns the number of elements in a collection. The following is the syntax:

```
sizeof_collection collection
```

You can use the empty string (the empty collection) as an argument. `sizeof_collection ""` is always 0.

Examples:

To gauge the size of a design, enter:

```
sizeof_collection [get_cells * -hier]
```

To determine whether a collection command yielded results, enter:

```
if {[sizeof_collection [get_cells U2/U2]] != 0}
```

26.3.10 Comparing Collections

The `compare_collections` command compares the contents of two collections, object for object. You can specify that the VC Static Platform compare the objects in order. The following is the syntax:

```
compare_collections collection1 collection2 [-order_dependent]
```

If the objects in both collections are the same, the result is 0 (like the result of string compare). If the objects are different, the result is non-zero.

26.3.11 Copying Collections

The `copy_collection` command duplicates a collection, resulting in a new collection. The base collection remains unchanged.

The `copy_collection` command is an efficient mechanism for duplicating an existing collection. However, copying a collection and having multiple references to the same collection are significantly different. Not all collections can be copied. The following is the syntax:

```
copy_collection collection
```

Examples:

If you create a collection and save a reference to it in variable c1, assigning the value of c1 to another variable c2 creates a second reference to the same collection:

```
set c1 [get_cells "U1*"]
{"U1", "U10"}
set c2 $c1
{"U1", "U10"}
echo $c1 $c2_sel3 _sel3
```

The previous commands, do not copy the collection; only `copy_collection` creates a new collection that is a duplicate of the original.

The following command sequence shows the result of copying a collection:

```
set collection1 [get_cells "U1*"]
{"U1", "U10"}
set collection2 [copy_collection $collection1]
{"U1", "U10"}
compare_collections $collection1 $collection20
```

26.3.12 Indexing Collections

The `index_collection` command creates a collection of one object that is the nth object in another collection. Following is the syntax:

```
index_collection collection index
```

Objects in a collection are numbered 0 through n-1. Not all collections can be indexed. Other collections which result from commands such as `get_cells` are not really ordered, but each has a predictable, repeatable order. The same command executed n times, for example, `get_cells *`, creates a collection of cells in the same order.

The index field of `index_collection` specifies the index into the collection.

Example:

To extract the first object in a collection, enter:

```
set c1 [get_cells {u1 u2}]
{"u1", "u2"}
index_collection $c1 0{"u1"}
```

26.4 Using Attributes

Attributes describe logical, electrical, physical, and other properties of objects in the design database. An attribute is attached to a design object and is saved with the design database. The VC Static Platform uses attributes on the following types of objects:

- Design objects, such as nets, pins, and ports.
- Cell instances within a design.
- Library components such as library cells and library cell pins.
- Objects related to low power.

An attribute has a name, a type, and a value. The following are the different types of attributes:

- String
- Numeric
- Logical (Boolean)

Some attributes are pre defined and are recognized by the VC Static Platform. These are termed *Application* attributes and are almost always read-only. Other attributes are user-defined and may be read or written by the user. Many attributes apply to one object type. For example, the `port_direction` attribute applies only to ports.

Some attributes apply to several object types. For example, the `full_name` attribute can apply to a net, cell, or port.

You can get detailed information about the pre defined attributes that apply to each object type by using the `list_attributes -application` command.

26.4.1 Attribute-Related Commands

The following are the VC Static Platform Tcl commands related to attributes:

- `define_user_attribute`
- `get_attribute`

Feedback

- `list_attributes`
- `remove_attribute`
- `set_attribute`

26.4.2 Predefined Application Attributes

This section describes the different pre-defined application attributes which are available on various design objects. The attribute names and expected values are described here.

- Application attributes are all read-only.
- Attributes of type boolean return true or false.
- Attribute names and values are case sensitive.

26.4.3 The object_class Attribute

Many collections are heterogeneous in nature. The `object_class` attribute is the mechanism used at the Tcl level to differentiate various object types. The `object_class` attribute is a string with the following values and meanings:

- cell - Cells/Instances
- port - Ports
- pin - Pins
- net - Net
- design - Modules
- lib_cell - Library Cell
- lib_pin - Library Pin

26.4.4 Cell Attributes

Table 46 lists the various cell attributes.

Table 46: Cell Attributes

Attribute Name	Type	Description
base_name	String	The leaf name of a cell. For example, the base_name of cell U1/U2/U3 is U3.

Feedback

Table 46: Cell Attributes (Continued)

Attribute Name	Type	Description
full_name	String	The complete name of a cell. For example, the full name of cell U3 within cell U2 within cell U1 is U1/ U2/U3.
is_three_state	Boolean	This attribute is true if a cell is a three-state (tri-state) device.
is_hierarchical	Boolean	This attribute is true for any hierarchical cell. It is false for library cells, also known as leaf cells.
is_sequential	Boolean	It is true only if a cell is a leaf-level cell and it is sequential.
is_combinational	Boolean	A cell is combinational if it is non-sequential or non-three-state and all of its outputs compute a combinational logic function. It is NOT black box.
is_black_box	Boolean	This attribute is true if the cell's reference is not linked to a library cell or design. This attribute is read-only. You cannot change the setting
is_mux	Boolean	This attribute is true if a cell is a multiplexer.
is_pad_cell	Boolean	This attribute is true if the library cell is a pad cell.
is_macro_switch	Boolean	This attribute is true if the reference library cell is defined as a fine-grain switch cell in the library.
number_of_pins	Integer	Number of pins on the cell.
ref_name	String	The name of the design or library cell of which the cell is (or will be) an instantiation. Also known as the reference name. The linker looks for a design or library cell by this name.
has_multi_ground_rails	Boolean	The attribute is true if a cell has multiple ground rails.
has_multi_power_rails	Boolean	The attribute is true if a cell has multiple power rails.
is_integrated_clock_gating_cell	Boolean	This attribute is true if a cell is defined in the library as an integrated clock-gating cell.
user_given_sam	Boolean	This attribute indicates if the instance/module are given in `set_abstract_model`.

Feedback

26.4.5 Pin Attributes

Table 47 lists the various pin attributes.

Table 47: Pin Attributes

Attribute Name	Type	Description
base_name	String	The leaf name of a pin. For example, the base name of pin i1/i1z1 is i1z1.
full_name	String	The complete name of a pin. For example, the full_name of pin i1z1 within cell i1 is i1/i1z1.
direction	String	The direction of a pin. Value can be in, out, inout, or internal.
pin_direction	string	Same as direction (for PT compatibility).
is_async_pin	Boolean	This attribute is true if a pin is an asynchronous preset/clear pin.
is_clock_pin	Boolean	This attribute is true if this is clock pin of a sequential cell instance.
is_clear_pin	Boolean	This attribute is true if a pin is an asynchronous clear pin.
is_clock_gating_pin	Boolean	This attribute is true if a pin for a clock-gating cell.
is_data_pin	Boolean	This attribute is true if a pin is a data pin of a sequential cell.
is_hierarchical	Boolean	This attribute is true for any pins that are instantiations of another design and false for pins that are instantiations of a library pin (also known as leaf pins).
is_mux_select_pin	Boolean	This attribute is true if a pin is the select pin of a multiplexer device.
is_three_state	Boolean	This attribute is true if a pin is a three-state (tri-state) driver.
is_three_state_enable_pin	Boolean	This attribute is true if a pin is an enable pin of a three-state device.
is_three_state_output_pin	Boolean	This attribute is true if a pin could output a three-state signal.
constant_value	String	The logic value of a pin tied to logic constant zero or one in the netlist.

Feedback

Table 47: Pin Attributes (Continued)

Attribute Name	Type	Description
is_enable_pin	Boolean	This attribute will return true for enable pin of a cell and will return false for other pins.
user_case_value	String	The user-specified logic value of a pin or port.
leaf_drivers	collection	Returns a collection of leaf driver pins of the pin.
leaf_loads	collection	Returns a collection of leaf load pins of the pin.

26.4.6 lib_pin Attributes

Table 48 lists the various `lib_pin` attributes.

Table 48: Lib Pin Atttributes

Attribute Name	Type	Description
base_name	String	The name of a library cell.For example, the base_name of library cell tech1/AN2 is AN2.
full_name	String	The fully qualified name of a library cell. This is the name of the library followed by the library cell name. For example, the full_name of library cell AN2 in library tech1 is tech1/AN2.
direction	String	The direction of a pin. Value can be in, out, inout, or internal.
pin_direction	String	Same as direction (PT compatibility).
is_async_pin	Boolean	This attribute is true if a pin is an asynchronous preset/clear pin.
is_clock_pin	Boolean	This attribute is true if at least one instance of that clock pin exists in the design that has the is_clock_pin attribute equal to true.
is_clear_pin	Boolean	This attribute is true if a pin is an asynchronous clear pin.
is_data_pin	Boolean	This attribute is true if at least one instance of that data pin exists in the design that has the is_data_pin attribute equal to true.

Feedback

Table 48: Lib Pin Atttributes (Continued)

Attribute Name	Type	Description
is_mux_select_pin	Boolean	This attribute is true if a library pin is a select pin of a multiplexer device.is_pad Boolean This attribute is true if the library pin is a pad.
is_three_state	Boolean	This attribute is true if the library pin is a three-state driver.
is_three_state_enable_pin	Boolean	This attribute is true if a pin is an enable pin of a three-state device.
is_three_state_output_pin	Boolean	This attribute is true if a pin could output a three-state signal.
when	String	This attribute will return the 'when' condition for the specified library timing arc.

26.4.7 Port Attributes

Table 49 lists the various port attributes.

Table 49: Port Attributes

Attribute Name	Type	Description
base_name	String	The leaf name of a pin. For example, the base name of pin i1/i1z1 is i1z1.
full_name	String	The complete name of a pin. For example, the full_name of pin i1z1 within cell i1 is i1/i1z1.
case_value	String	The user-specified logic value of a pin or port propagated from a case analysis or logic constant in the design.
constant_value	String	The logic value of a port tied to logic constant 0 or 1 in the netlist.
direction	String	The direction of a port. Value can be in, out, inout, or internal.
port_direction	String	Same as direction (PT compatibility).
user_case_value	String	The user-specified logic value of a pin or port.

Feedback

26.4.8 Net Attributes

Table 50 lists the various net attributes.

Table 50: Net Attributes

Attribute Name	Type	Description
base_name	String	The leaf name of a net. For example, the base name of net i1/i1z1 is i1z1.
full_name	String	The complete name of a net. For example, the full_name of net i1z1 within cell i1 is i1/i1z1.

26.4.9 Design Attributes

Table 51 lists the various design attributes.

Table 51: Design Attributes

Attribute Name	Type	Description
full_name	String	The name of a design. For example, the full_name of design TOP read in from /u/user/simple.db is TOP. This name can be ambiguous because several designs of the same name can be read in from different files.
extended_name	String	The complete, unambiguous name of a design. The extended_name of the design is the source_file_name attribute followed by a colon (:) followed by the full_name attribute. For example, the extended_name of design TOP read in from /u/user/ simple.db is /u/user/simple.db:TOP.
is_current	Boolean	This attribute is true for the current design. This attribute changes for all designs when you use the current_design command.
source_file_name	String	The name of the file from which the design was read.For example, the source_file_name of design TOP read in from /u/user/simple.v is /u/ user/simple.v.
user_given_sam	Boolean	This attribute indicates if the instance/module are given in `set_abstract_model`.

Feedback

26.4.10 lib_cell Attributes

Table 52 lists the various `lib_cell` attributes.

Table 52: lib_cell Attributes

Attribute Name	Type	Description
base_name	String	The name of a library cell. For example, the base_name of library cell tech1/AN2 is AN2.
full_name	String	The fully qualified name of a library cell. This is the name of the library followed by the library cell name. For example, the full_name of library cell AN2 in library tech1 is tech1/AN2.
is_black_box	Boolean	This attribute is true if the cell's reference is not linked to a library cell or design.
is_sequential	Boolean	This attribute is true if the library cell is sequential.
is_three_state	Boolean	This attribute is true if a library cell is a three-state device.
is_combinational	Boolean	A cell is combinational if it is non-sequential or non-three-state and all of its outputs compute a combinational logic function. It is NOT black box.
is_integrated_ clock_gating_cell	Boolean	This attribute is true if a cell is defined in the library as an integrated clock-gating cell.
has_multi_ground_rails	Boolean	The attribute is true if a cell has multiple ground rails.
has_multi_power_rails	Boolean	The attribute is true if a cell has multiple power rails.
is_isolation	Boolean	This attribute is true if a library cell is an isolation cell.
is_level_shifter	Boolean	This attribute is true if the library cell is a level-shifter cell.
is_macro_switch	Boolean	This attribute is true if the library cell is defined as a fine-grain switch cell in the library.
is_retention	Boolean	This attribute is true if the library cell is a retention cell.
is_mux	Boolean	This attribute is true if a library cell is a multiplexer.

Table 52: lib_cell Attributes (Continued)

Attribute Name	Type	Description
is_pad_cell	Boolean	This attribute is true if the library cell is a pad cell.
number_of_pins	Boolean	Number of pins on the cell.

27

Appendix B - SVA Coding Guidelines

When writing SVAs for formal analysis, remember that the assertions are also part of the formal model. You must be aware of the sequential elements that are inferred by the SVA code. In general, follow these principles:

- Keep it simple
  - Can be more easily applied to a specific problem
  - Compiles faster even if more in number
  - Executes faster, especially in formal tools
- Decompose the properties into simple temporal statements
  - Joint satisfaction implies the desired design behaviors
  - Use static variables.

The following are some coding guidelines:

- Put a label for every assertion, cover, assume.
- Avoid implications within implications (`a |-> b |-> c`) as this is difficult to understand. The `a |-> b |-> c` is equivalent to `a ##0 b |->c`.
- Do not use clock signal both in clocking event and inside the body of property, as clock signal in clocking event is not sampled, whereas the clock signal inside the body of property is sampled.
- Avoid $past with large delay values.
- Avoid $past, $rose, and so on as the first boolean, as this may lead to unexpected failure at time 0.
- Local variables add complexity. Local variables provide means for succinct description of data-transfer related properties. Succinctness hides run-time complexity.
  - A property with local variables cannot be used as an assumption if there is a non-constant or unbounded delay between the assignment and use of the local variable.

    For example

Feedback

```
property p;
int lv;
(en_i, lv = data_in) |=> ##[5:10] (en_o && (data_o==lv));
endproperty
```

- Linear properties with local variables satisfy the following conditions:
  - Singly clocked.
  - May have one of the following operators: constant delay/repetition operators (##n, [*n]), nexttime, suffix operators (|->, |=>, #-#, #=#).
  - Have none of the following operators: [->], [=], or, and, intersect, within; until, until_with, eventually and their strong versions; autonomous sequences.
  - **Examples**
    - Original form using $past

```
property p_past(M);
   a  |-> (b==1) ##1 (b==2) [*M] ##1 (b==4) && (x == $(past(x,
 2+M-1) + $past(x)));
endproperty
```

    - Equivalent linear property with local vars, possibly easier to understand

```
property p(a, b, int x,);
  //int n=x;
  (a, n=x) |-> (b==1 ##1 ((b==2)[*M], n=n+x) ##1 (b==4)  && (n ==
 x);
endproperty
```

    - If the addition of x to n is on each turn of the repetition

```
property p(a, b, int x,);
  //int n=x;
  (a, n=x) |-> (b==1 ##1 ((b==2), n=n+x) [*M] ##1 (b==4)  && (n
 == x);
endproperty
```

      Parameterization by M is not possible using $past

- Avoid “intersect” and “within” operators over complex sequences, as it can lead to state blow up.
- Sequence “and” may have similar effect as “intersect” unless it is the top-level operator.
  - a |=> s1 and s2 is OK
  - a |=> (s1 and s2) ## s3 is not, as it can lead to state blow up
- Unbounded goto operator in antecedent is preferable to the bounded one.

Feedback

`a ##1 b[->1] |-> c` better than `a ##1 (b[->1] intersect 1[*1:10]) |-> c`

- Avoid using “first_match” operator, it is either redundant or not supported in VC FV.
  - Redundant: `first_match(a ##2 b) |=> s - a ##2 b` has only one match anyway
  - Redundant: `a |=> first_match(a ##[1:4] b)` – first match is automatic on a property
  - Can be rewritten: `first_match(a ##[1:$] b) |=> s` to `a ##1 b[->1] |=> s`
  - Assertion is ignored: `first_match(a ##[1:4] b or c ##[0:5] ) |=> s`
- Do not use top-level “always” without “initial”. It is redundant (costly in simulation).
  - For example, `A: assert property(@posedge clk) always p)`; in SVA there is an implicit top-level always.
  - Equivalent to `initial A: assert property(@(posedge clk) always always p)`;
- Whenever possible do not use implication (|->, |=>, implies) in cover property. This leads to unnecessarily complex model.
  - Instead of

    ```
    C1: cover property(@(posedge clk) a ##[1:2] b |=> c ##[1:3]d)
    ```

    Write

    ```
    C1: cover property(@(posedge clk) a ##[1:2] b ##1 c ##[1:3]d)
    ```
  - Instead of

    ```
    C2: cover property(@(posedge clk) a ##[1:2] b |=> d until e);
    ```

    Write

    ```
    C2: cover property(@(posedge clk) a ##[1:2] b #=# d until e);
    ```

    or

    ```
    C2: cover property(@(posedge clk) a ##[1:2] b ##1 d [*0:$] ##1 e);
    ```
- Avoid multi-clocked assertions as they are expensive.

  ```
  Amclk: assert property(@(posedge clk1) s |=> @(posedge clk2) p);
  ```

  This requires an internal clock with frequency 2 x LCM (f(clk1), f(clk2)). This is costly at compile time and run time. Keep both s and p as simple as possible.
- Avoid single-clocked complex clocking expressions.

Feedback

```
Acmplx: assert property(@(edge clk1 or posedge clk2) p);
```

p is unclocked or has the same clock as assertion. These run on an internal clock with similar frequency requirements. This results in higher run time because of the faster clocks.

- When you use abort operators: `accept_on`, `reject_on`, `sync_accept_on`, `sync_reject_on`
  - It may simplify property formulation

    ```
    AAsync: assert property(@(posedge clk) start |=> accept_on (retry) check_transaction);
    ```

  - It declares success if a `retry` happens during the `check_transaction` property evaluation, asynchronously.
  - It requires fast internal clock to detect async occurrence. It imposes performance cost (in simulation too). But, an alternate formulation without async abort may be impossible.
  - If retry is synchronous on `posedge clk`, use synchronous abort

    ```
    ASync: assert property(@(posedge clk) start |=> sync_accept_on (retry) check_transaction);
    ```

  - This does not require fast internal clock as all runs on `posedge clk`. Alternate formulation without `sync_accept_on` difficult without knowing details of `check_transaction`.
- Avoid formal incompatible SVA constructs

  You can use SVAs in several environments with different verification tools. As a result, there are some elements of the language that are suitable for traditional testbench-driven testing but not for proving properties in formal analysis. This is due to the fact that in order to prove properties, a formal model of the synthesizable design and any assertions are required. For example, do not write assertions that contain non-synthesizable code.

  If you write your own assertions that are not formal-compatible, it is recommended that you use a macro compile directive to separate the formal compatible and non-formal compatible code.
- If you use the past operator, add a delay (##n) to avoid referencing uninitialized values (X) of the past operator.

  For example:

  ```
  property keep_stable;
  @(posedge clk) (!reset && !ack) |-> ##2(data==$past(data,2));
  ```

Feedback

```
endproperty
a1: assert property(keep_stable);
```

- Do not try to verify behavior during the reset sequence. The formal proof engines do not start until after the reset sequence completes.

  Example, do not use `reset ##1 !reset` in the antecedent of implication operators as follows:

  ```
  a1 : assert property(@(posedge clk) reset ##1 !reset |-> (a==8'd30));
  ```

  As the formal proof engines start after the reset sequence, the they do not see `reset ##1 !reset` and the result is a vacuous proof.

  Instead, you can use the following code to detect the end of the reset sequence:

  ```
  a2 : assert property(@(posedge clk) $past(reset) && !reset |->
  (a==8'd30));
  ```

  When reset de-asserts, `$past(reset)` still remembers the reset value 1; now, the formal proof engines can see `$past(reset) && !reset`.

- Avoid verifying signals from different directions in an assertion.
- Try to keep the consequent of an assertion simple. It is better to write several assertions each testing one part of a conjunction. Many simpler assertions can be proven in parallel. Split the assertions where the consequent is a conjunction of signals or properties.

  Instead of:

  ```
  A: assert property @(posedge clk) seq |=> (p1 and p2);
  ```

  Separate into two properties, each handling one of the consequent properties:

  ```
  A1: assert property @(posedge clk) seq |=> p1;
  A2: assert property @(posedge clk) seq |=> p2;
  ```

  The other problem with the assertion *A* is that it cannot be used as a constraint. To solve this problem, break the assertion into two separate properties; one for the req input to use as a constraint and the other for the ack output to use as a checker.

  ```
  // Constraint for req input
  property p1;
  @(posedge clk) start |-> ##[1:2] req;
  endproperty
  // Checker for ack output
  property p2;
  @(posedge clk) start |-> ##[1:2] ack;
  endproperty
  a1: assume property(p1);
  a2: assert property(p2);
  ```

Feedback

- Avoid referring only to registered signals or $past operators in assertions used as constraints. For example:

```
property constraint_1;
@(posedge clk) (stateVar1==2) |-> (stateVar2==1);
endproperty
a1: assert property(constraint_1);
```

  The problem here is that this assertion tries to constrain the values already generated in the past. To solve this problem, use primary inputs in the constraint so that the primary inputs can be controlled in order to satisfy the conditions of the assertion.

- Very large delay and repetition ranges in consequent can be deployed, but it must be avoided because the ranges might be unusable in assumptions; the run-time performance may suffer.

  Example

```
A: assert property(@(posedge clk) disable iff (rst) req |=> #[1:1000]
ack ##[1:200] done);
```

28

Appendix C - Formal Examples and Use Models

This section describes the Formal examples and use models.

- Running VC Formal Examples
- One-design Flow in VC SEQ

28.1 Running VC Formal Examples

Examples of Tcl command scripts to read a design with SVA assertions, configure the run, generate report on properties, run checks and debug falsifications for FPV app is available in the VC Formal installation. Similarly, examples of other VC Formal Apps as well as some debug related examples are also available. They can be found in the following VC Formal installation area:

```
$VC_STATIC_HOME/doc/vcst/examples/<app_name>/
```

The Tcl regression scripts and the related documents are available at the following locations:

```
$VC_STATIC_HOME/auxx/ctg/tcl/library/proc
$VC_STATIC_HOME/auxx/ctg/tcl/library/doc
```

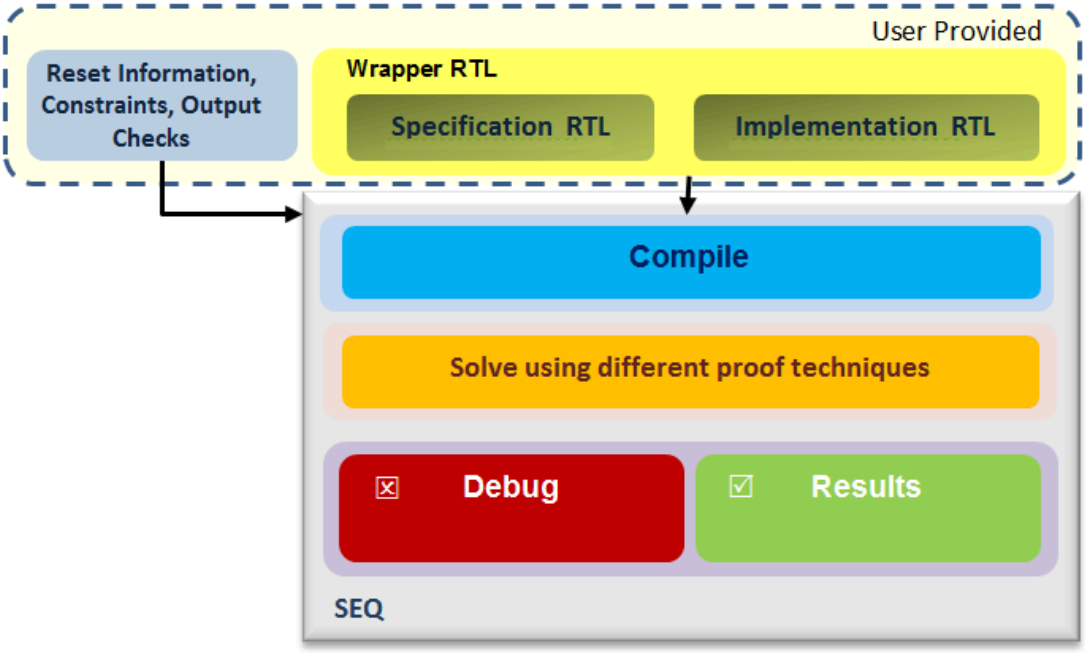
28.2 One-design Flow in VC SEQ

You can use the one-design flow, if you already have a setup that consists of a wrapper around the two designs (the file `wrapper.v`). In this wrapper, the inputs of the two designs may already be mapped as desired. Optionally, the output mappings can also be added.

The additional files (in the example directory: `$VC_STATIC_HOME/doc/vcst/examples/SEQ`) are as follows:

- `wrapper.v` – The wrapper RTL file inside design directory that instantiates the two designs to be compared, maps their inputs and sets up output checks.
- `run_1des.tcl`– The command script for the *one-design* flow.

Figure 242: Using VC SEQ One Design Method



To compare two designs in VC SEQ using the one-design flow, execute the following steps:

1. Compile the RTL wrapper as follows:

```
set_fml_appmode SEQ
seq_config -map_uninit -map_x zero -specprefix sp -implprefix im
read_file -sva -top wrapper -vcs
 "-sverilog ../design/rtl1.v ../design/rtl1cg_bad1.v ../
design/wrapper.v"
```

   You can also list the names of the files that need to be compiled in a file (for example, *file.list*) and use the following command:

```
read_file -sva -top wrapper -vcs "-sverilog -f file.list"
```

2. Map inputs, outputs, and black boxes of the two designs using the `map_by_name` command.

In the one-design flow, the inputs may already be mapped in the wrapper. However, you may also have written the wrapper using different signals going into the two designs. In the latter case, you need to map the inputs and outputs just like in the two-design flow.

The following command maps all the inputs and outputs of the two designs (by name) as assumptions or assertions. You can limit the mapping to only inputs or outputs using the `-input`t or the `-output`options respectively. For this example, the inputs are already mapped in the wrapper, so only the outputs are mapped.

```
map_by_name -output
```

3. Specify the clock and reset as follows:

```
create_clock -period 100 clk
create_reset rst -sense high
```

4. Provide additional assumptions and assertions

   Similar to the two design flow, you can provide additional assumptions and assertions using the `fvassume` and `fvassert` commands.

5. Run and save reset information

   VC SEQ needs an initial set of states with which it can start the two designs with. To identify these states, use the following commands:

```
sim_run -stable
sim_save_reset
```

6. Solve the equivalence problem

   To check all the assertions enabled in the setup using the assumptions defined and enabled in the setup, use the following commands:

```
check_fv
```

7. Analyze the result

   Use the following commands to analyze the results, similar to the two-design flow:

```
report_fv
```

   The principal difference between the two-design and the one-design flows is the compile step and the provision of explicit `-specprefix` and `-implprefix` options in the `seq_config` command.

29

Appendix D - Support for Formal Scoreboard and RAM/FIFO Verification Library

VC Formal has a set of abstraction models available to assist in verification. These abstract models have been optimized for formal and are comprised of the following:

- A formal scoreboard for data transport component verification.
- An abstract RAM and FIFO model for design replacement.

These models are available at the following locations:

- The Formal scoreboard verification library is present at $VC_STATIC_HOME/packages/formal_sva_lib/fml_scoreboard/

  For more information on how to use the formal scoreboard, see the $VC_STATIC_HOME/packages/formal_sva_lib/fml_scoreboard/doc/FormalScoreboard.pdf
- The Formal RAM and FIFO verification library is present at $VC_STATIC_HOME/packages/formal_sva_lib/fml_abstracted_models/ram_and_fifo/

  For more information on how to use the Formal RAM and FIFO verification library, see the $VC_STATIC_HOME/packages/formal_sva_lib/fml_abstracted_models/ram_and_fifo/doc/Formal_RAM_FIFO_Model.pdf.

30

Appendix E - Application Variables

This section contains the description for the application variables used in VC Formal. Use `set_app_var` and `get_app_var` to set and get the values of these variables. You can also get information on these application variables by using the`report_app_var -verbose` command. Even more detailed information can be obtained by man <app_var_name> command.

The application column shows which VC Formal App is affected by each of the application variables, the type of the variable, default value, possible values as well as description.

Variable Name	App	Type	Default Value	Possible Values	Description
allow_unconn_interface_port	Common	bool	false	true, false	When designs with unconnected interface ports are read using the -vcs command line options, VCS reports the following error message:"Error-[SV-UIP1] Unconnected interface port"However, when you set the allow_unconn_interface_port application variable to true, VCS reports a warning message and continues with the run."Warning-[SV-UIP-SIMON] Unconnected interface port". This app var is default 'true' for Formal mode.

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
analyze_skip_translate_body	Common	bool	true	true, false	When this application variable is set to true, VC Static honors pragmas in user HDL files (synopsys_translate_off and synopsys_translate_on) and skips the code in between the pragmas from its analysis. If set to false, VC Static ignores the pragmas and thereby tries to compile and elaborate the code inside the pragmas. If you want to analyze some files with honoring the pragmas and some other files by ignoring pragmas, you can set the Tcl variable accordingly before each file (or a set of files) that must to be compiled.
incr_compile_data_path	Common	String			This variable is used to specify the directory from which incremental compile data should be picked when `enable_incremental_compile` is set to true. This variable is useful when previous session's *vcst_rtdb* has been moved to a different location and you need to use that data in the incremental compile flow in the current session.

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
enable_verdi_kdb_libcell	Common	string	none	none	Enables rules to treat -v/-y files or `celldefine modules as lib cells in the Instance tree.By default, it is set to none, meaning no libcell. Verdi uses -ssv,-ssy,-ssz (design import options) to enable debugging into “-v lib.v”, “-y …”, and `celldefine.You can change it to “all” (same as “vlib,ylib,celldefine”), or choose the ones that you would want to turn into the corresponding types of libcells. For example, set_app_var enable_verdi_kdb_libcell “vlib,ylib”, here, -v/-y usage is treated as libcells. However, you can debug into `celldefine.
enable_auto_glassbox	Common	bool	true	true, false	When set to true, modules that throw an error during its processing are automatically glass boxed. When set to false, these modules are black boxed.
fml_auto_save	Common	string	default	default, true, false	Enables automatic checkpoints when check_fv is executed. 'default' disables automatic checkpoints, except in COV app mode. Set to 'false' to completely disable automatic checkpointing.
fml_cc_check_prefix	CC	string		""Any String	Enables user to prefix a string to the name for the connectivity checks
fml_cc_comb	CC	bool	false	true, false	Causes CC to optimize the check for combinational properties.
fml_cc_coverage_analyzer	CC	bool	false	true, false	Enables computing coverage analysis for the CC App

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
fml_cc_overabstract	CC	bool	false	true, false	Causes CC checks to perform to use an over abstracted model. Over abstraction only preserves a connected result. An unconnected result may be spurious.
fml_composite_trace	FPV	bool	true	true, false	By default, VC Formal shows the composite trace which consists of the reset trace and the formal trace for all apps except CC and FUSA. To disable this capability, use the following command: set_app_var fml_composite_trace false When composite trace is enabled, you can see a marker on the waveform indicating the end of the reset sequence. You can also see the complete trace including the reset phase.
fml_cov_enable_delay_stmt	COV	bool	true	true, false	Enables monitoring of delay stmt in line coverage
fml_cov_enable_int_unr	COV		false	true, false	When this application variable is set to true, VC Formal Coverage Analyzer can do intentional unreachability/unintentional reachability analysis. Definition of Intentional Uncoverable: A statement can be generalized to be intentionally uncoverable when the statement is found to be uncoverable and it is: (1) Non-FSM and default/branch assigning X value (2) FSM and default/branch

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
fml_cov_exclusion_crossfeed	COV	bool	false	true, false	When set to true, VC Formal FCA reports the number of assertions and covers that are excluded.The fml_cov_exclusion_crossfeed application variable has an effect only when the line coverage is enabled and the -sva switch is also passed at the time of reading the file (or elaboration for mixed designs)
fml_cov_int_unr_style	COV		all	x-assign, fsm,all	Specifies the RTL style for doing intentional analysis.
fml_cov_override_db_opt	COV	bool	false	true, false	In VC Formal Coverage Analyzer (FCA), you can import simulation coverage database, so that the FCA can work only on the unconverged items from the coverage database. However, for proper mapping of the goals between simulation coverage database and the ones extracted by FCA for formal coverage analysis, it is important that coverage sub-options (like cm_cond options) are same in simulation and formal. If the application variable fml_cov_override_db_opt is set to false, then tool automatically takes care of this mapping
fml_cov_tgl_input_port	COV	bool	false	true, false	This application variable needs to be true for VC Formal Coverage Analyzer to monitor coverage for input ports
fml_cov_tgl_only_seq	COV	bool	false	true, false	When this variable is set to true, VC Formal Coverage Analysis performs toggle coverage check only for the sequential elements in the design. It then skips monitoring toggle coverage for non-sequential items

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
fml_default_task_alias	Common	string		""	Set the alias name of the default application task. It can be set for the current default application (FPV, SEQ, COV, CC, AEP, etc.). The alias name can then be used in place of the default app name in the fvtask command.
fml_demote_save_exclusion_severity	COV	bool	false	true, false	Demotes the severity of save_cov_exclusion errors
fml_enable_fsm_report_complexity	Common	bool	false	true, false	Enables fsm information to be displayed in report_fv_complexity. This must be set prior to read_file/elaborate.
fml_enable_param_override_in_bind	Common	bool	true	true, false	Enables the tool to perform parameter overriding in bind file.
fml_enable_vhdl_aep	AEP	bool	false	true, false	Enables extraction of AEPs for VHDL designs
fml_enable_vhdl_cov	COV	bool	false	true, false	Enables instrumentation of coverage for VHDL blocks
fml_enable_vhdl_fta	FTA	bool	false	true, false	Enables fta vhdl methodology for analyze/elaborate flow
fml_jobs_alert_threshold	Common		90		This application variable controls the frequency of printing solver job alert messages. If the percentage of solver jobs that die due to memory-out, crash or signal-killed exceeds this threshold, then a message is printed to screen. The goal is to alert user to potential setup problems requesting machines with very little memory
fml_license_auto_release	Common	bool	true	true, false	Enable automatic license release at the end of check_fv

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
fml_mode_on	Common	bool	true	true, false	Enables formal mode defaults in vc-static. It is true by default when VC-Formal is executed from the vcf script, otherwise it is false when executed from the vcf script by non-Formal apps.
fml_multi_step_fta_flow	FTA	bool	false	true, false	Required when using -inject_fault with elaborate command
fml_orc_bmc_depth_profile	FPV	bool	false	true, false	
fml_quiet	Common	bool	false	true, false	Quiets down the messages coming during formal analysis.
fml_quiet_gui	Common	bool	true	true,false	Quiets down the messages coming during formal analysis when the gui is enabled.
fml_report_user_defined_net		bool	false	true, false	
fml_trace_auto_fsm_state_extraction	Common	bool	false	true, false	When set true, Verdi automatically extracts the FSM and FSM state names from the design or FSM signal. When add the signal to nWave, the FSM states appear in the waveform of the hex/decimal values. Right-click on the value column in nWave for change the alias or remove the alias
report_datetime_in_log	Common	bool	false	true, false	Set this variable to report the start date and time for each command in the session log file.

Feedback

Variable Name	App	Type	Default Value	Possible Values	Description
search_include_file_path	Common	bool	false	true, false	Use this application variable to prioritize searching within include Verilog paths over other file paths during the design read. When `search_include_file_path` is set to true, the tool searches for include files inside the directory where the analyzed design files are located instead of relying on the `include compiler directive and the`+incdir+<DIR_NAME>` option to find include files. The `search_include_file_path` application variable is similar to the `-search_incl_file_path` option in VCS.
stop_on_synth_error	Common	bool	false	true, false	Causes the application to exit when there is a synthesis error

31

Appendix F - Formal Variables

This section contains the description for the formal variables used in VC Formal. Use `set_fml_var` and `get_fml_var` to set and get the values of these variables. You can also get information on these application variables by using `report_fml_var` or `report_fml_var -verbose` to get help.

The application column shows which VC Formal application is affected by this variable.

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_abstract_arithop_with_const	Common	bool	false	true,false	Allow abstraction on arithmetic operators with constant operands.
fml_allow_reverse_cc_path	CC	bool	false	true, false	When true, allows reverse structural check, i.e., checks whether path exists from destination to source
fml_auto_abstraction	Common	bool	false	true, false	Enable automatic abstraction for design (set prior to loading design).
fml_auto_cleanup_run	Common	bool	false	true, false	Clean up run after it finishes. Improves disk usage.
fml_cc_autobbox	CC	bool	true	true, false	Enable Fully automatic blackboxing flow.
fml_cc_autobbox_func	CC	bool	false	true, false	Not recommended to use. Enable Fully automatic blackboxing flow without structural checking (only functional check). Used when interested only in functional result.
fml_cc_check_prefix	CC	string		""Any string	Enables user to prefix a string to the name for the connectivity checks

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_cc_composite_trace	CC	bool	false	true, false	Setting this variable to true will allow composite trace to be enabled but may impact build and trace generation performance for large designs.
fml_cc_enable_csv_based_naming	CC	bool	false	true, false	Set this variable to true, when you want to load connections with duplicate names from multiple csv files using the `load_cc` command. When the `fml_cc_enable_csv_based_naming` is set to true, the csv file name is appended to the property as a prefix, for example, `<csv_file_name>__<property_name>`. When set to false, if there are duplicate connection names in the csv files, an error message is reported.
fml_cc_instance_src_out_to_dest_in	CC	book	false	true, false	Used to extract connections only between out/inout ports of source instance to in/inout ports of destination instance.
fml_cc_refclk_on	CC				Enables reference clock in connectivity checks (default true). Different clock frequencies and polarities are not considered during structural checking and auto-bbox flow by default. User needs to turn off this optimization to consider different clock frequencies and polarities in the path.
fml_cc_structural_check	CC	string	path	path, none, full	Control the structural check in connectivity checks none - switch off the structural traversal full - structural traversal for enable-destination path as well as the source-destination path path - this is default. It will check for the source-destination path connectivity

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_cc_verify_rst_path	CC	bool	false	true, false	Enable reset path verification.
fml_compress_grid_log	Common	bool	true	true, false	Reduces verbosity of grid logs to save disk utilization during check_fv.
fml_coi_reduction	Common	bool	true	true, false	By default, VC Formal builds the formal model only for the cone-of-influence (COI) properties, including constraints. The COI reduction can ease resource usage by reducing the size of the formal model. When this variable is set to true, signals outside the COI is not visible in traces, and appears as 'NF' when viewing the values applied to the source code. To build formal model for all properties, set this variable to false.
fml_conflict_check_depth	FPV	int	2		Check for conflicting constraints up to a given depth.
fml_conflict_debug_minimal	FPV	bool	true	true, false	Report small (clock to minimal) set of conflicting constraints.
fml_conflict_debug_reset	FPV	bool	false	true, false	Report list of register names whose reset state which contribute to conflicting constraints.
fml_cov_enable_branch_cov	COV	bool	false	true, false	To turn on branch coverage when the -cov all switch has been used.
fml_cov_enable_strict_elfile_mode	COV	bool	false	true, false	Enables saving of exclusion file in strict mode.
fml_cov_enable_unconst_analysis	COV	bool	false	true, false	Enables identifying unreachables even under unconstrained analysis
fml_cov_gen_trace	COV	string	default	on/off/default	Generate traces for coverage app when check_fv is run

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_cov_ignore_case_default	COV	bool	false	true, false	Helps in pruning uncoverable default case items from coverage analysis.
fml_cov_tgl_handle_xinit_val	COV	bool	false	true, false	Enable a delay of 1 clock cycle while monitoring tgl coverage of sequentials with an 'x' value after reset state.
fml_cov_tgl_signal_legacy_lookup	COV	bool	false	true, false	Enable the legacy lookup method for coverage toggle signals.
fml_cov_tgl_undriven_as_unr	COV	bool	false	true, false	Used to classify undriven toggle signals as uncoverable.By default, undriven toggle signals are reported as covered by the tool.
fml_cov_tgl_x_driven_as_unr	COV	bool	false	true, false	Used to classify X driven toggle signals as uncoverable. By default, X driven toggle signals are reported as covered by the tool.
fml_deadend_check_depth	FPV	int	50		Control the max depth for a deadend, that is, do not report deadends of longer lengths.
fml_deadend_check_max_deadends	FPV	int	20		Max number of deadends to search for
fml_deadend_check_maxcycles	FPV	nit	500		Control the total number of cycles for which deadend engine is run.
fml_deadend_debug_minimal	FPV	bool	true	true, false	Report small (close to minimal) set of constraints causing deadend.
fml_deadend_debug_reset	FPV	bool	false	true, false	Report list of register names which participate in the deadend.
fml_defer_property_compile	Common	bool	false	true, false	Defer compilation of incremental properties until sim/formal model build.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_effort	Multiple Apps	string	default	default easy high bounded bug_hunting discovery bitlevel_hard no_decompose_expensive user orc	Specifies formal orchestration effort level. See section Orchestration in VC Formal for application specific details.
fml_enable_assume_proof	FPV	bool	false	true, false	Enables proofs as assumption for subsequent flow
fml_enable_covers_in_prop_density_cov	COV	bool	false	true, false	Enables mapping of property density to coverage database for SVA covers
fml_enable_noskipsimmemmodels	Elab	bool	TRUE	true, false	Enable support of simon option -noSkipSimMemModels in formal. Must be specified prior to read_file/elaborate.
fml_enable_prop_density_cov_map	COV	bool	false	true, false	Enables mapping of property density to coverage database
fml_enable_vhdl_bus_checks	AEP	bool	false	true, false	Enable bus check AEP
fml_export_eprediction_data	Common	string		""	File Location to save profiling information.
fml_formal_core_incremental	Common	book	true	true, false	By default, VC Formal computes the formal cores for both proven and inconclusive properties. To compute formal core either for the proven or inconclusive properties, set the variable to false.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_formal_core_liveness	Common	bool	true	true, false	Enables generating formal core for liveness properties
fml_formal_core_reduced_constraints	Common	bool	true	true, false	Enables reduced constraints computation during formal core generation.
fml_formal_core_verbose_level	Common	int	1	0, 1, 2	Set this formal variable to see progress messages when computing formal core either as part of `check_fv` (on-the-fly enabled) or `compute_formal_core`. 0: Do not print any messages. 1: Print messages when formal core is derived from an engine proof. Default value. 2: In addition to level 1 messages include bounded formal core update messages.
fml_fta_seq_debug	FTA	bool	false	true, false	Enable FTA SEQ debug mode. Debug non-detected faults by analyzing if those faults can propagate to any of the primary outputs.
fml_fca_enable_bad_code_check	COV	bool	false	true, false	Allow property filterting using "reason==simplication" which lists trivially uncoverable line and condition goals (found through simple logic rewrite).
fml_fta_alt_config_dir		String			Specify alternate directory path from where to pick up certitude_config.cer, certitude_hdl_files.cer files.
fml_fta_enable_cover	FTA	bool	false	true, false	Honors cover properties in FTA flow
fml_fta_par_app_high_effort	FTA	bool	false	true, false	Enable high effort level for parent app run of FTA
fml_inject_input_port_fault	FTA	bool	false	true, false	Enable injection of faults at input port.
fml_inject_stuck_at_x_fault	FTA	bool	false	true, false	Enable injection of StuckAtX faults

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_lp_aware	CC	bool	false	true, false	Enable LP aware connectivity checks.
fml_lp_constraint_auto	CC	bool	true	true, false	When fml_lp_aware is enabled in the CC app, automatically generate the LP constraints with read_upf
fml_lp_constraint_strategy	CC	string	disable	enable disable	LP constraints will try to neutralize the isolation and retention cells.
fml_lp_query_prefix_top	Common	String			String to be prefixed before the top in the output of lp related query commands.
fml_lp_query_replace_top	Common	String			String to replace top in the output of the lp related query commands.
fml_max_mem	Common	string	8GB		Limit formal engines to approximately the specified virtual memory usage. Units are GB or MB (12GB, 12000MB).
fml_max_proof_depth	Common	int	-1		Set the specified maximum bounded depth.
fml_max_time	Common	string	12H		Limit check_fv call to specified time. Negative time means no limit. Units in seconds(S), minutes(M) or hours(H) (e.g. 37M, 12H).
fml_max_time_per_fault	FTA	string	12H	Time units in seconds(S), minutes(M) or hours (H) (e.g. 37M, 12H)	Limit time of exploration of each fault in high effort to specified time. Negative time means no limit. Units in seconds(S), minutes(M) or hours(H) (e.g. 37M, 12H).

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_max_trace	Common	int	1		The number of unique counter-examples will be limited to this number for each falsified assertion.
fml_max_worker_per_property	FPV	int	-1		Specify max worker used per property. Use -1 to set unlimited workers.
fml_min_unk	Common	int	0	true, false	The call check_fv will terminate after this amount of unknowns being reached.
fml_navigator_conflict_warning	NAV	bool	true	true, false	Print constraint conflicts warning for extending properties or different input scenarios (default true).
fml_navigator_display_msgid	NAV	string		""	Display the specified message id while in the Navigator as a pop-up when encountered.
fml_navigator_ignore_msgid	NAV	string		""	Ignore the specified message id while in a Navigator task (does not show in pop-up).
fml_no_trace	Common	bool	false	true, false	Turns off trace generation.
fml_orc_tactic	FPV	string	prop_check		The check_fv call will use the specified orchestration tactic.
fml_progress_time_limit	Common	int	-1		Limit check_fv call to specified unproductive time. Negative time means no limit.
fml_prop_result_timestamp	Common	bool	false	true, false	Includes a timestamp in PROP_I_RESULT messages from check_fv. The following is an example of the PROP_I_RESULT message when this variable is set to true: *[Info] PROP_I_RESULT: [2024-11-13 14:31:41] FPV traffic.chk.cov_green_without_waiting_on_first b8 depth=71 property 00:00:02*

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_qual_fault_in_fcore	FTA	bool	false	true, false	Enables filtering of faults based on formal core results
fml_quiet_trace	Common	bool	true	true, false	Generated counter-examples will do extra analysis to minimize unneeded transitions on inputs.
fml_reset_auto_generation	Common	bool	true	true, false	Allow auto generation of the initial state at check_fv.
fml_reset_post_save_checking	Common	bool	false	true, false	Enable checking of commands that may impact the initial state if issued post sim_save_reset.
fml_reset_property_check	COV	bool	false	true, false	Enables property checking during reset state generation (sim_run)
fml_report_profile_data	Common	bool	false	true, false, global	false: Does not report the profiling information. By default, the fml_report_profile_data is set to false.true: Reports the profiling information.-global: Applies the specified value to all tasks.
fml_session_memory_warn_pct	Common	int	0	0 <= val <= 100	Specify this variable to get a warning message when the memory usage in local machine exceeds the percentage threshold specified in the `fml_session_memory_warn_pct` formal variable.
fml_show_loop_analysis	Elab	bool	false	true, false	Show combinational loop analysis during formal model creation.
fml_stop_on_failure	Common	int	0		The number of failures after which check_fv should be stopped.
fml_trace_extend_time_limit	Debug	int	30		Specify time limit (seconds) for extending trace on view_trace command. Default 30 seconds, minimum 10 seconds.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fml_trace_reeval_time_limit	Debug	int	30		Specify time limit (seconds) for reevaluating trace on view_trace command. Default 30 seconds, minimum 10 seconds.
fml_track_loc_const_select	CC	bool	false	true, false	Track source locations for constant selects.
fml_track_loc_reset	CC	bool	false	true, false	Track source locations for reset blocks.
fml_vacuity_on	FPV	bool	true	true, false	Enable vacuity checking for SVA assertions.
fml_verbose_grid_log	Common	bool	true	true, false	Reduces verbosity of grid logs to save disk utilization during check_fv.
fml_witness_on	FPV	bool	false	true, false	Enable witness trace generation for SVA assertions.
fml_x_detect_mode	Common	int	0	0 <= val <= 2	Specify x-detection modes. 0: no detection 1: detect all x's excluding inputs 2: detect all x's including inputs
fml_x_in_waveform	Common	bool	false	true, false	Additional option given to dfgsim to show X's in waveform.
fsv_verbose_level	FSV	int	2	0 <= val <= 2	The verbosity level of check_fv in FSV App.
fsv_bbox_conn_version	FSV	int	1	0 <= val <= 1	It controls the version of the connectivity analysis of `fsv_blackbox`. Its default value is 1. With `fsv_bbox_version` set to 2, the value of this fml_var will be changed to 2.
fsv_bbox_impl_version	FSV	int	1	0 <= val <= 2	It controls the version of the connectivity analysis of fsv_blackbox. Its default value is 1. With `fsv_bbox_version` set to 2, the value of this fml_var will be changed to 2.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fsv_compute_rootcause_auto	FSV	bool	false	true, false	When set to true, the root cause signals for falsified FSV properties are computing during the check_fv run.
fusa_disable_new_inconcl_statuses	FuSA	bool	false	true, false	By default, the default Z01X statuses are mapped to the new statuses (see Status Codes Starting T-2022.06-1 Onwards in Table 21-1) when you use the *fusa_map_status*command.To map the default Z01X statuses to old statuses (see Status Codes Prior to T-2022.06 Release in Table 21-1), set the `fusa_disable_new_inconcl_statuses formal variable to true. By default, the fusa_disable_new_inconcl_statuses` *formal variable is set to false.*
fusa_multiple_fault_types_signal	FuSa	bool	true	true, false	There can be multiple faults defined for same signal name but different object classes. When fusa_multiple_fault_types_signal is set to true, VC Formal FuSa honors each of these as different faults and honor each object class. When fusa_multiple_fault_types_signal is set to false, only the first fault with a given signal name is considered. Other faults with the same signal, but different object class are ignored.
fta_infer_cov_status	FTA	bool	false	true, false	Infers the coverage status of a line which has no fault injected from the status of the parent block.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
fv_create_vacuity_cover_properties	FPV	bool	false	true, false	For any FPV asserts/assumes with vacuity created, when this variable is set to true a second cover property named *${original_name}_vacuity* is created for the vacuity term. If there is a conflict in the property name, an integer is appended to the new property name to make it unique.
fv_property_check_clock	Common	bool	false	true, false	Enables/disables a check that will report a warning message when the clock of the clocked properties is dependent on a non-clock input or free signal. The properties clocked on signals impacted by free signals can trigger at an unexpected time, and have the incorrect version of the values sampled.
fxp_compute_rootcause_auto	FXP	bool	false	true, false	When set to true, the root cause signals for falsified FXP properties are computing during the check_fv run.
fxp_no_duplicate_asserts	FXP	bool	false	true, false	Do not generate duplicated asserts in FXP for signals or part selects which have been asserted.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
seq_bmc_depth	SEQ	int	100	2 <= val <= 100 000	Provide the upper bound for BMC in SEQ recipe. The SEQ orchestration uses this number as a hint on how to balance the orchestration early on. This value is the number of phases the orchestration tries for a bounded proof of the properties. For most designs, a cycle consists of two phases. Note that as the orchestration progresses it will try to proof higher bounded depths. The problem of setting this number too big is that it reduces the time spent by engines that tries to get a full proof for the properties. On the other hand, if you are trying to find deep falsification or increase the bounded depth of inconclusive properties then this number should be increased. For example, if you want to find if there are any falsification within 300 cycles, then you will set this variable to 600.
seq_root_cause_size	SEQ	int	6	0 <= val <= 100	Specify the number of registers that should be displayed in the waveform for the root cause analysis.The root cause analysis lists the registers that have different values compared to its mapped counterpart, and all the fan-in registers that have the same value compared to its mapped counterpart.
seq_split_multi_dim_array_port	SEQ	bool	false	true, false	Split higher dimensions of multi-dimensional primary output or blackbox input port into individual elements.
seq_split_struct_array_port	SEQ	bool	true	true, false	Split struct or array of struct port into individual elements.
seq_use_bg_for_critical_jobs	SEQ	bool	false	true, false	If this is set then the tool uses the backup grid for critical jobs.

Feedback

Formal Variable Name	App	Type	Default Value	Possible Values	Description
seq_use_decompose_partial_words	SEQ	bool	false	true, false	Default false. This can be useful if the implementation has registers that are only partially equivalent to the specification. For example, say there is a control register c which is 8 bit in the spec. In the implementation say the designer added some new features that are controlled by c. Now, c in the impl is 10 bits. The actual mapping is spec.c[7:4] == impl.c[9:6] and spec.c[3:0] == impl.c[3:0]. The tool automatically tries to find such mappings if it is set to true. Can have negative impact in scenarios where such mappings are not found.
seq_verbose_level	SEQ	int	2	0 <= val <= 2	The verbosity level of check_fv
fml_enable_verbose_depth	FPV	bool	false	true, false	When this application variable is set to true, VC Formal displays extra run-time messages related to the bounded proof depths. Each time a greater depth is reached for a given property, a message is printed in the shell transcript, and logged in the message database. The property name, its new depth, and the elapsed wallclock time are all in the message. An example might look like: *[Info] PROP_I_RESULT: FPV traffic.main.oh_state_out b8 depth=63 property 00:00:02* Note that the "traceon" and "traceoff" continues to work in cases where you want to pause the verbose screen reports during a run. See section Executing FPV Testbench for more details.

32

Appendix G - Engine Controls for User Controlled Orchestration

This section provides the list of engine controls for user controlled Orchestration.

- Engine Labeling
- Engine b1/b4/b6/b7/b10/b11/b13/b14/b15/b16/b17
- Engine b3
- Engines b5
- Engines b8/b9
- Engine b20
- Engine e1/e18/e23
- Engine e2
- Engine e3
- Engine e4
- Engine e7
- Engine e19
- Engines e8/e9/e15/e16/e24
- Engines e13/e14
- Engine e17
- Engine e25
- Engine s1
- Engine s7/s12
- Engine e12
- Engine I1/I11/I16/I13/I14/I18

- Engine I4
- Engine I5
- Engine I6
- Engine I8
- Engine I9
- Engine I10/I12/I17
- Engine I22
- Engine Usage

32.1 Engine Labeling

The following table provides the engine labeling for the FPV, FRV, FCA, and AEP applications:

Table 53: Engine Labeling (FPV, FRV, FCA, and AEP)

Engine Label	Description
e1, e2, e3, ….	Proof engines - best for proofs • Finds counter examples (cex). • Includes abstraction engines (creates a smaller problem for a goal).
b1, b3, ….	Bounded exploration engines - best for finding shortest length cex. Cycle skipping versions may not give shortest cex.
t1, t2, ….	Transformation engines - simplify/massage the problem (automatically created).Run as a part of orchestration.
s1, s2, ….	Semi-formal engines - memory efficient and can find cex (usually not the shortest).
l1, l2, …	Liveness engines for cex and proofs.
rp1, rf1, and rq1	Regression Mode Accelerator (RMA) engines.Run as a part of orchestration automatically (not through `fvorc`).
de	Dead-end analysis engine
cc	Conflicting constraint engineRun as a part of orchestration automatically (not through `fvorc`).

Feedback

32.2 Engine b1/b4/b6/b7/b10/b11/b13/b14/b15/b16/b17

The b1 engine is a key engine for exhaustive bounded exploration of the design. It is good at finding shortest counter examples (if timePerCycle option not used) and for increasing bounded depth of the properties. The b1 engine may consume more memory compared to other engines. The engines b4/b7 are often good at finding hard failures, but they do not guarantee to find shortest counterexamples. The engines b10/b11/b13/b14/b16/b17 are also key engines for exhaustive bounded exploration of the design that works on a simplified model.

There are two types of engines – single-goal engine and multi-goal engine. Single-goal engines work on a single goal in a single job whereas multi-goal engines work on multiple goals in a single job.

The engines b1/b4/b6/b7/b15 can work as a single goal or multi-goal engine. The other engines b10/b11/b13/b14/b16/b17 can work as a single goal engine only.

- -strategy <0/1>
  - Default value: 0
  - Model simplification is disabled when 1 is used.
- -initDepth <0-INT_MAX>
  - Default value: 0
  - Initial depth from which a counterexample is searched
- -endDepth <-1-INT_MAX>
  - Default value: 9999
  - End depth to which a counterexample is searched
- -timePerCycle <-1-INT_MAX>
  - Default value: -1
  - Amount of wall-clock time to spend in each cycle of the property
- -engBudget <-1-INT_MAX>
  - Default value: -1
  - Amount of engine budget to spend in each cycle of the property. This is a deterministic resource budget. This means that engine will follow the same trajectory when given same inputs irrespective of machine speed.
- -optimizeA <0-1>

    - Default value: 0
    - Try 1 for the design with hard failures.
- -oct <0-4>
    - Default value: 0 for b1/b4/b10/b13. 1 for b6/b7/b11/b14. 2 for b15/b16/b17.
    - Different ‘-oct’ value uses different internal solver algorithms.
    - Try 1-4 for the design with hard failures.
- -simpExp <0-3>
    - Default value: 0 for b10/b11/b16. 1 for b13/b14/b17.
    - Different ‘-simpExp’ values use different types of simplification.
    - Try 1-3 for the design with hard failures.

Example

```
> fvorc b1 -engOpts "-strategy 1 -initDepth 10 -endDepth 20 -timePerCycle
300 -oct 1"
```

32.3 Engine b3

The b3 engine is an effective engine that searches for a counterexample for a property. It sometimes finds a counterexample that b1 engine does not find. The engine can be useful for finding a counterexample, but not for increasing the bounded depth of the property.

This engine is a single goal engine.

- -maxDepth <-1-INT_MAX>
    - Max depth to which a counterexample is searched
    - Default value: 1000

Example

```
> fvorc b3 -engOpts "-maxDepth 2000"
```

32.4 Engines b5

The b5 engine is a useful engine that searches for a counter example engine in smaller chunks of designs that are relevant for property being checked. This allows it to be applied even on larger designs.

This engine is a single goal engine.

32.5 Engines b8/b9

The b8 engine uses a combination of formal and semi-formal search for finding failures with small resources.

We recommend the users to try with the `-all`option. The engine b9 that sets the engine option `-oct` to 1 by default is often good at finding hard failures.

This engine can work as a single goal or multi-goal engine.

- -strategy <0-2>
  - Controls exploration strategy in the engine. We recommend the users to try with strategy 0. If lot of workers are present, then all strategies can be tried.
  - Default value: 0
- -mode <0-3>
  - Different modes control different types of semi-formal search mode. We recommend the users to try with mode 0. If lot of workers are present, then all strategies can be tried.
  - Default value: 0
- -endDepth <-1-INT_MAX>
  - Default value: 9999
  - End depth to which a counterexample is searched
- -oct <0-4>
  - Default value: 0 for b8. 1 for b9.
  - Different ‘-oct’ value uses different internal solver algorithms.
  - Try 1-4 for the design with hard failures.
- -maxBudgetP <-1-INT_MAX>
  - Default value: -1
- -maxBudgetC <-1-INT_MAX>
  - Default value: -1
- -maxBudgetE <-1-INT_MAX>
  - Default value: -1

- -maxBudgetEC <-1-INT_MAX>
  - Default value: -1
- -maxBudgetR <-1-INT_MAX>
  - Default value: 500
- -timePerCycle <-1-INT_MAX>
  - Default value: -1
  - Amount of time to spend in each cycle

32.6 Engine b20

The engine b20 is often good at finding hard failures, but they do not guarantee to find shortest counterexamples.

This engine can work as a single goal engine.

- `-strategy <0-1>`
  - Controls exploration strategy in the engine.
  - Default value: 0
- `-multiplier <1.0-DBL_MAX>`
  - Exploration heuristic parameter in the algorithm
  - Default value: 1.5
- -optimizeA <0-1>
  -
    - Default value: 0
    - Try 1 for the design with hard failures.
- -oct <0-4>
  - Default value: 2
  - Different ‘-oct’ value uses different internal solver algorithms.
  - Try 1-4 for the design with hard failures.

32.7 Engine e1/e18/e23

The e1/e18/e23 engines are effective proof engine. They also memory efficient. They work well when design has few hundreds to few thousands (< 10k) registers. These engines

Feedback

may also scale for bigger designs depending on complexity of properties. In few cases these engines are also effective in finding deep counter examples.

These engines can work as a single goal or multi-goal engine.

- -strategy <0/1/2>
  - Default value: 0
  - Different strategies control different proof search strategies within the engine. In addition to default strategy we highly recommend trying strategy 1 and if more workers are available try strategy 2 as well.
- -lazy <0/1>
  - Default value: 0
  - Control whether to perform certain steps in eager or lazy fashion. For designs with more than few hundred registers we recommend trying value 1.
- -activity <0-2>
  - Default value: 0
  - Different ways of assigning weights to netlist elements
- -preserve <0/1>
  - Default value: 0
  - Internal heuristic in the algorithm
- -subBudget <-1-INT_MAX>
  - Default value: -1
  - Amount of the budget used for the base engine heuristic
- -subStep <0-1>
  - Default value: 1
  - When set to 0, one of the optimization steps in algorithm is skipped (without impacting functional result).
- -srStep <0-2>
  - Default value: 0
  - Enables sharing of information between different steps of the algorithm.
- -srStepSubA <0-1>

    - Default value: 0
    - Enables additional optimization in algorithm
- -srStepSubB <0-1>
    - Default value: 0
    - Enables additional optimization in algorithm
- -srStepBudget1 <-1-INT_MAX>
    - Default value: -1
    - Budget for controlling internal step in the engine
- -srStepBudget2 <-1-INT_MAX>
    - Default value: -1
    - Budget for controlling internal step in the engine

In addition to the above options, the e18 engine also supports following additional options.

- -joinStep <0-1>
    - Default value: 0
    - Enable additional join algorithm
- -joinStrategy <0-2>
    - Default value: 400
    - Control depth in the engine
- -joinDepth <0-INT_MAXs>
    - Default value: 400
    - Control depth in the engine
- -joinDepthAutoStrategy <0-3>
    - Default value: 0
    - Automatically figure out depth when this variable is > 0
- -budgetP <-1-INT_MAX>
    - Default value: 500
    - Budget control within join algorithm

- -budgetC <-1-INT_MAX>
  - Default value: 150000
  - Alternate budget control within join algorithm

32.8 Engine e2

The e2 engine is an effective proof engine. It can solve hundreds/thousands of easy complexity properties quickly. This engine also benefits from the knowledge of other proven properties. If the number of properties is huge, it may be better to partition property set and attempt to solve each partition with e2.

This engine can work as a single goal or multi-goal engine. It is recommended to use this engine in the thmulti-goal mode.

- -strategy <0-2>
  - Default value: 0
  - Different strategies control different proof search strategies within the engine. In addition to default strategy, we highly recommend trying strategy 1 and if more workers are available try strategy 2 as well.

32.9 Engine e3

The e3 engine is an important proof engine. It can sometimes solve properties that other engines cannot solve. However, this engine does need higher memory as compared to other engines like e1.

This engine can work as a single goal or multi-goal engine. We recommend using this in single goal mode.

32.10 Engine e4

The e4 engine works by identifying smaller chunks of designs relevant for verification of property. If the relevant portion of the design has small number of sequentials, then this engine can be effective in getting proofs.

This engine is a single goal engine.

32.11 Engine e7

The e7 engine is useful for designs with few sequentials (< 250). For such small designs this engine can be very effective.

This engine can work as a single goal or multi-goal engine. We recommend using it as a single-goal engine.

- -strategy <0/1>
  - Default value: 0
  - Default strategy is generally useful but depending on design/property alternative strategy can be useful as well.

32.12 Engine e19

The e19 engine can be a useful engine for designs with few sequentials (< 250). For such small designs this engine can be very effective.

This engine can work as a single goal or multi-goal engine. We recommend using it as a single-goal engine.

32.13 Engines e8/e9/e15/e16/e24

The e8/e9/e15/e16/e24 engines are effective proof engines. They allow engine to focus on smaller chunks of designs that are relevant for property being checked. This allows them to be applied even on larger designs.

These engines are single goal engines.

- -strategy <0/1/2>
  - Default value: 0
  - Different strategies control ways of identifying chunks of formal model relevant for checking the property. When lot of workers are available, we recommend trying all strategies.
- -compactionStartDepth <-1-INT_MAX>
  - Default value: -1
  - When this option is set to a value >= 0, then we ask the engine to do more work in identifying precisely parts of design that are impacting the property. This can sometimes help in getting convergence. Another utility of this option is when we

want to generate smaller formal cores. We recommend setting this option to 0 if the goal is to get smaller formal cores.

- In addition to above option the e8 engine also supports following additional options.

In addition to above option the e8 engine also supports following additional option.

- -mode <0/1/2>
  - Default value: 0
  - Different modes control different leaf level engines that are tried. When lot of workers are available we recommend trying all modes.

The e9/e15/e16/e24 engines also support following additional option.

- -simp <0/1>
  - Default value: 0
  - When this option is set to 1, the engine tries to simplify the identified chunks of the formal model.
- -level <-1-INT_MAX>
  - Default value: 2
  - Level of the identified chunks
- -optimize <0/1>
  - Default value: 0
  - Model optimization on the smaller chunks of the design
- -optimizeC <0/1>
  - Default value: 0
  - Another way of model optimization on the smaller chunks of the design
- -baseStrategy <0-3>
  - Default value: 0
  - Different proof search strategies for the base engine
- -baseLazy <0/1>
  - Default value: 0
  - Control whether to perform certain steps in the base engine in eager or lazy fashion
- -baseActivity <0-2>

    - Default value: 0
    - Different ways of assigning weights to netlist elements
- -basePreserve <0/1>
    - Default value: 0
    - Internal heuristic in the algorithm
- -baseSubBudget <-1-INT_MAX>
    - Default value: 1
    - Amount of the budget used for the base engine heuristic

The e24 engine also support following additional option.

- -simpExp <0/1>
    - Default value: 0
    - Model optimization on the original model

32.14 Engines e13/e14

Similar to e9/e15/e16 class of engines.

These engines are single goal engines.

- -strategy <0/1/2>
    - Default value: 0
    - Different strategies control different algorithms for identifying chunks of formal model relevant for checking the property. When lot of workers are available we recommend trying all strategies.
- -compactionStartDepth <-1-INT_MAX>
    - Default value: -1
    - When this option is set to a value >= 0, then we ask the engine to do more work in identifying more precisely parts of design that are impacting the property. This can sometimes help in getting convergence. Another utility of this option is when we want to generate smaller formal cores. We recommend setting this option to 0 if the goal is to get smaller formal cores.

32.15 Engine e17

The e17 engine can be a useful proof engine that simplifies the original model into a smaller model (which can be easier to converge).

This engine can work as a single goal or multi-goal engine.

32.16 Engine e25

The e25 engine is a new proof engine which tries to identify which parts of formal testbench are impacting the solver and leverages this information to focus the search. This engine is a single goal engine.

- -strategy <0-5>
  - Default value: 0
  - Different strategies control different proof search strategies with different base engines.
- -budgetC <-1-INT_MAX>
  - Default value: 3600
  - Amount of an engine budget to spend in an internal step
- -depth <0-INT_MAX>
  - Default value: 200
  - Depth to use in an internal step
- -compaction <0/1>
  - Default value: 0
  - When this option is set to 1, we ask the engine to do more work in identifying more precisely parts of design that are impacting the property.

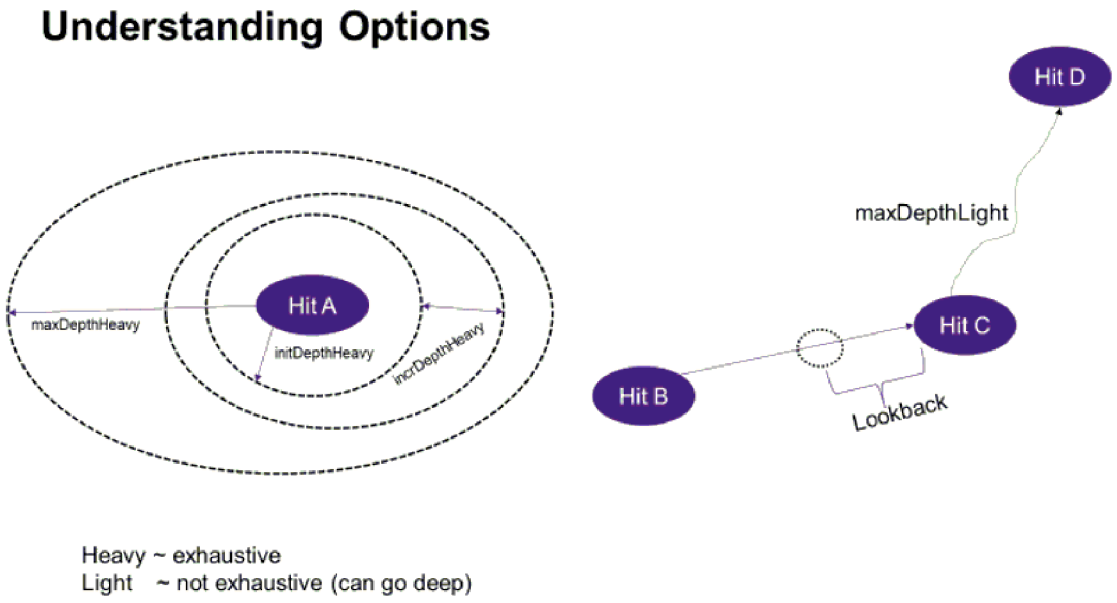
32.17 Engine s1

The s1 engine uses a combination of formal and semi-formal light-weight search for bug-hunting. This engine can be useful for finding easy failures.

Feedback

32.18 Engine s7/s12

The s7 engine uses a combination of formal and semi-formal search for bug-hunting. We refer to formal search in s7 as "heavy" search while we refer to semi-formal search in s7 as "light" search. As goals are hit (covered/falsified) by this engine they act as guideposts for subsequent search. The goal of this engine is to go deep into the design. Unlike b1 engine this engine is not designed to be exhaustive. This engine can be very effective in finding deep bugs provided good covers (guideposts) are given to help engine make progress. Many controls are available to fine-tune performance of this engine. These options are discussed in more detail below. Note that when a standard orchestration like bug hunting effort level is used then many of these options are automatically tried and varied. The engine s12 that sets the engine option '-oct' to 1 by default is often good at finding hard failures.



- -strategy <0-7>
  - Controls exploration strategy in the engine. Given a set of guideposts the strategy tells how to prioritize among different guideposts, that is, which guideposts gets examined first. We recommend trying at least strategies 0-3. If lot of workers are present, then all strategies can be tried. Meaning of different strategies
    - -strategy 0: Consider latest cover hits first
    - -strategy 1: Consider deepest cover hits first (depth first)
    - -strategy 2: Consider cover hits in the order they were generated (breadth first)
    - -strategy 3: Consider cover hits in an order derived from seed

Feedback

        - -strategy 4/5: Consider cover hits based on graph structure
        - -strategy 6/7: Consider cover hits based on dynamic structure
    - Default value: 0
    - -seed <num>
        - The seed is used to initialize random number generator in this engine. This generator is used in various places in the engine where a random choice needs to be made. It is recommended to vary the seed for each call of s7. This allows engine to explore different parts of state space. If seeds are changed with each run of the tool, then it increases chances of finding bugs at the cost of reproducibility (that is, results can change from run to run). We recommend saving traces after each run for debug purpose or have RMA enabled. If reproducibility is needed keep track of seeds used in a run.
        - Default value: 1234567
    - -initDepthHeavy <num>
        - The s7 engine does heavy searches in concentric circles around a guidepost. This option controls radius of the smallest such circle. Depending on complexity of goals on a design this option can be increased or decreased. For example, if all goals in a design have easily reached up to a depth of n, then may be okay to set this parameter to n. On the other hand, if some goals in a design are stuck at depth 5, then starting with a smaller value than default may be better.
        - Default value: 20
        - This depth option and the ones below are in terms of half clock-cycles or phases of system clock.
    - -maxDepthHeavy <num>
        - Controls maximum radius for a heavy search around a guidepost. Based on design this option can be varied.
        - Default value: 80
    - -incrDepthHeavy <num>
        - Controls increment for creating concentric circles of search around a guidepost.
        - Default value: 10
    - -maxDepthLight <num>

    - Controls depth for light-weight search. Light weight searches are generally cheaper, so we can set this parameter to a high value compared to options for heavy searches.
    - Default value: 500
- -lookback <num>
    - For every new guidepost created also look few specified numbers of clock phases back and use that for subsequent searches.
    - Default value: 10
- -multiplier <num>
    - The s7 engine is an iterative engine, this option allows how the resources need to be scaled when going to higher iterations.
    - Default value: 2
- -helpers <0|1>
    - Instructs the engine to generate more helper/cover properties based on formal model. This can be useful option to try if the user has not provided covers.
    - Default value: 0
    - Additional options when “-helpers 1”: 5 different categories of internal helpers
        - -helperTypeCPP <0|1>
            - Default value: 1
        - -helperTypeCV <0|1>
            - Default value: 0
        - -helperTypeCC <0|1>
            - Default value: 1
        - -helperTypeFP <0|1>
            - Default value: 1
        - -helperTypeIP <0|1>
            - Default value: 1
- -focusOnTop <num>

    - Tells the engine to focus on top <num> guideposts. Default is to use all guideposts.
    - Default value: -1
- -numRoundsLight <num>
    - For each guidepost how many light search attempts to do in each tick.
    - Default value: 5
- -skipHeavyOnProgress <0|1>
    - When set to 1, this flag allows skipping initial heavy search (which can be expensive in some cases) provided light search had made progress earlier.
    - Default value: 0
- -randomizeGoalHeavy <0|1>
    - Shuffle goals during heavy search. Default 0.
    - Default value: 0
- -oct <0-4>
    - Default value: 0 for s7. 1 for s12.
    - Different '-oct' value uses different internal solver algorithms.
    - Try 1-4 for the design with hard failures.

32.19 Engine e12

The e12 engine is an effective proof engine that spends more resources on simplifying the model. It sometimes works well on a large design. It is possible that the engine terminates early in the orchestration when there is no simplification in the model.

This engine is a single goal engine.

- -strategy <0-5>
    - Controls different strategies in the engine run.
    - Default value: 0
    - Try 0-4 together if there are enough number of workers.
    - Try 5 for the design with low sequential depth.
- -helperTypeBI <0/1>

- Default value: 0
- Try 1 to use helpers found in the engine run.

Example

```
> fvorc e12 -engine_opts "-strategy 3 -helperTypeBI 1"
```

32.20 Engine I1/I11/I16/I13/I14/I18

These are key engines for finding liveness counterexample. The engine I11 that sets the engine option '-oct' to 1 by default is often good at finding hard liveness failures. The engine I13/I14/I18 that sets the engine option '-engBudget' 2000 by default on I1/I11/I16 is often good at finding hard liveness failures that are not found by I1/I11/116.

This engine is a single goal engine.

- -oct <0-4>
  - Default value: 0 for I1/I13. 1 for I11/I14, 2 for I16/I18.
  - Different ‘-oct’ value uses different internal solver algorithms.
  - Try 1-4 for the design with hard failures.
- -engBudget <-1-INT_MAX>
  - Default value: -1. 2000 for I13/I14/I18.
  - Amount of engine budget to spend in the engine run

32.21 Engine I4

The I4 engine is an effective liveness proof engine that is mostly used for finding a proof. The engine finds a proof earlier than other liveness engines do in some cases.

This engine is a single goal engine.

- -strategy <0-10>
  - Controls different strategies in the engine run.
  - Default value: 0
  - Try 0-2 together if there are enough number of workers.
  - Try 3-4,9-10 for the design with low sequential depth.
- -optimize <0/1>

    - Default value: 0
- -optimizeA <0/1>
    - Default value: 0

Example

```
> fvorc l4 -engine_opts "-strategy 3 -optimize 1 optimizeA 1"
```

32.22 Engine I5

The I5 engine is an effective liveness proof engine that finds proofs for liveness properties in some cases that other liveness engines cannot find.

This engine is a single goal engine.

- -strategy <0-10>
    - Controls different strategies in the engine run.
    - Default value: 0
    - Try 0-2 together if there are enough number of workers.
    - Try 3-4,9-10 for the design with low sequential depth.
- -maxNum <1-INT_MAX>
    - Default value: 50
    - Try a bigger value (e.g. 100,200,300) if the engine terminates early in the orchestration.
- -maxDepthE <0-INT_MAX>
    - Default value: 200
- -maxBudgetB <-1-INT_MAX>
    - Default value: 60
- -optimize <0/1>
    - Default value: 0
- -optimizeA <0/1>
    - Default value: 0

Example

Feedback

```
> fvorc l5 -engine_opts "-strategy 2 -maxNum 200 -maxBudgetB 80"
```

32.23 Engine I6

The I6 engine is an effective liveness proof engine that spends more resources on simplifying the model. It is possible that the engine terminates early in the orchestration if there is no simplification in the model.

This engine is a single goal engine.

- -strategy <0-22>
  - Controls different strategies in the engine run.
  - Default value: 0
  - Try 0-2, 11-13 together if there are enough number of workers.
  - Try 3-4,9-10,14-16 for the design with low sequential depth.
- -maxNum <1-INT_MAX>
  - Default value: 50
  - Try a bigger value (e.g. 100,200,300) if the engine terminates early in the orchestration.
- -maxDepthE <0-INT_MAX>
  - Default value: 200
- -maxDepthC <0-INT_MAX>
  - Default value: 20
- -maxBudgetB <-1-INT_MAX>
  - Default value: 500
- -maxBudgetP <-1-INT_MAX>
  - Default value: -1
- -maxBudgetC <-1-INT_MAX>
  - Default value: -1
- -maxBudgetE <-1-INT_MAX>
  - Default value: -1
- -maxBudgetEC <-1-INT_MAX>

    - Default value: -1
- -maxBudgetR <-1-INT_MAX>
    - Default value: 500
- -order <0/1>
    - Default value: 0
    - Try 1 to try different simplification order. It sometimes provides a better simplification.
- -optimizeP <0/1>
    - Default value: 0
- -optimizePLevel <0-2>
    - Default value: 0
- -optimizePBudgetC <-1-INT_MAX>
    - o Default value: -1
- -optimizePBudgetP <-1-INT_MAX>
    - Default value: -1

Example

```
> fvorc l6 -engine_opts "-maxNum 300 -maxBudgetB 40"
```

32.24 Engine I8

The I8 engine is very effective liveness proof engine. The engine can sometimes find a liveness counterexample as well.

This engine is a single goal engine.

- -strategy <0-8>
    - Control different strategies in the engine run.
    - Default value: 0
    - Try 0-5 together if there are enough number of workers.
    - Try 6-8 for the design with low sequential depth.
- -level <-1-INT_MAX>

- Default value: 2

- -type <0/1>
  - Type of the simplification
  - Default value: 1
  - Trying 0 sometimes helps finding a proof.
- -simp <0/1>
  - Default value: 0
  - When this option is set to 1, the engine tries to simplify the model.
- -numIter <-1-INT_MAX>
  - Default value: -1
  - Try the option with '-strategy <6-8>'. Ignored with '-strategy <0-5>.

Example

```
> fvorc l8 -engine_opts "-strategy 6 -level 5 -type 0 -numIter 500"
```

32.25 Engine l9

The l9 engine is an effective liveness proof engine that sometimes finds a proof that other liveness engines do not find. The engine can sometimes find a liveness counterexample as well.

This engine is a single goal engine.

- -strategy <0-2>
  - Control different strategies in the engine run.
  - Default value: 0
- -level <-1-INT_MAX>
  - Default value: 2
- -type <0/1>
  - Type of the simplification
  - Default value: 1
  - Trying 0 sometimes helps finding a proof.

Feedback

Example

```
> fvorc l9 -engine_opts "-strategy 1 -level 4 -type 0"
```

32.26 Engine I10/I12/I17

The I10 engine uses a semi-formal search that is specialized for finding a liveness counterexample. Various model optimizations can be specified for the engine. If the engine stops working early in the orchestration, try to increase the endDepth or the maxBudgetR. The engine I12/I17 that sets the engine option '-oct' to 1/2 by default is often good at finding hard liveness failures.

This engine is a single goal engine.

- -strategy <0-1>
  - Control different strategies in the engine run.
  - Default value: 1
- -mode <1/2>
  - Default value: 1
  - Trying 2 sometimes finds a proof that is not found with 1.
- -endDepth <-1-INT_MAX>
  - Default value: 200
  - End depth to which a counterexample is searched.
  - Try a bigger value if the engine terminates early.
- -maxBudgetP <-1-INT_MAX>
  - Default value: -1
- -maxBudgetC <-1-INT_MAX>
  - Default value: -1
- -maxBudgetE <-1-INT_MAX>
  - Default value: -1
- -maxBudgetEC <-1-INT_MAX>
  - Default value: -1
- -maxBudgetR <-1-INT_MAX>

- Default value: 500
- Try a bigger value if the engine terminates early.

- -oct <0-4>
  - Default value: 0 for I10. 1 for I12, 2 for I17.
  - Different ‘-oct’ value uses different internal solver algorithms.
  - Try 1 or 2 for the design with hard failures.

Example

```
> fvorc l10 -engine_opts "-strategy 1 -mode 2 -oct 1"
```

32.27 Engine I22

The engine I22 is often good at finding hard liveness failures.

This engine is a single goal engine.

- -strategy <0-1>
  - Default value: 0
  - Controls exploration strategy in the engine.
- -multiplier <1.0-DBL_MAX>
  - Default value: 1.5
  - Exploration heuristic parameter in the algorithm.
- -optimizeA <0-1>
  - Default value: 0
  - Try 1 for the design with hard failures.
- -oct <0-4>
  - Default value: 2
  - Different ‘-oct’ value uses different internal solver algorithms.
  - Try 1-4 for the design with hard failures.

32.28 Engine Usage

The following table describes the engine usage:

Table 54: Engine Usage

Usage	Engine
Deep bug hunting	• s7 uses covers to help make progress and go deeper in state space; controls available to fine tune search, for example, breadth-first, depth-first, random, and so on. • Other s* engines like s1, s6, and s11 perform semi-formal search.
Single and multiple goals	• b1/b3-b11/b13-b17/e1-e3/e7/e10/e17-e19/e24/s1/s2/s7/s8/s11/s12 support either single or multiple goals. • Other engines work as single goal engines. • e2 is a very effective multi-goal proof engine for large number of properties.
Shortest CEX	b1/b6/b10/b11/b13-b17 is used when `timePerCycle` option is not specified.
CEX	• b1/b3 • b6/b7/b10/b11/b13-b17 is often good at finding hard failure that b1/b3 cannot find. • b8/b9 uses a combination of formal and semi-formal search for finding failures with small resources. Recommend to try with the `-all` option. • b5 focuses on CEX in smaller chunks of the design that are relevant to the property being checked. • e1/e18 can find deep CEX in rare cases.
Proof	• e1 (low memory profile), e18, and e12. • e2 can benefit from other proven properties. • e3 is complementary to other engines. • e9/e15/e16/e24/e8/e13/e14/e4 focus on smaller chunks of design that are relevant to the property being checked. • e17 simplifies the original model into a smaller model.
Liveness	• l1/l8/l9/l11/l13/l14/l16/l17 (liveness CEX). • l10/l12 (liveness CEX using semi formal search). • l4/l5/l8/l9 (proof) • l6 proof engine spends more resources in simplifying the model.

33

Verdi Debugging Tips for VC Formal

The following sections provide a quick reference for using the Verdi and Siloti systems with typical debug scenarios in VC Formal:

- Automatic Tracing of Value With Verdi's Temporal Flow View
- Automatic Tracing of X's With Verdi's Temporal Flow View
- Active Tracing of RTL
- Function Debugging
- Macro Debugging
- Count the Transitions of a Clock or Register
- Modify Existing Logical Operation Signals
- Collapse Source Code

33.1 Automatic Tracing of Value With Verdi's Temporal Flow View

1. In the *nWave* window, find a signal that you want to debug.
2. Select the signal.
3. Click the **Auto Trace** toolbar icon . or,
4. Right-click to invoke **Temporal Flow View -> New Temporal Flow View command**.
5. In the *Temporal Flow View* window, select a port signal, right-click to invoke **Trace This Value option**.

   If the value of multiple signals is traced, you may find that some paths are merged into the same path for a better high level view.

33.2 Automatic Tracing of X's With Verdi's Temporal Flow View

1. In the *nWave*window, select a signal that has transitioned to an X.
2. Click **Auto Trace** toolbar icon . or,
3. Right-click to invoke **Temporal Flow View -> New Temporal Flow View command**.
4. In the *Temporal Flow View* window, select a port signal, right-click to invoke **Trace X option**.
5. In the *Trace X Settings* form, set the tracing constraint and click **Trace**.

33.3 Active Tracing of RTL

1. In the *nWave* window, select a specific signal that you want to debug.
2. Double-click a transition point that you want to debug.
3. This takes you to the line in source code that is causing the transition.
4. In the main window, invoke **Source -> Active Annotation**.
5. Click on the signal, right-click to invoke **Active Trace**.

33.4 Function Debugging

1. In the *Source Code* pane, find a signal that is driven by a function, for example, "add", and so on.
2. Invoke **Source -> Function Annotation**.
3. Double-click the function, for example, "add".
4. Click on function, for example, "add" to jump back.

33.5 Macro Debugging

1. In the *Source Code* pane, find a macro, for example, MACRO1(CLOCK2).
2. Put your mouse cursor on MACRO1. The tip window shows the definition of MACRO1.
3. Invoke **Source -> Expand Macro**.

33.6 Count the Transitions of a Clock or Register

1. To turn on the grid count in *nWave*:
   a. In *nWave*, invoke **View -> Grid Options**.
   b. In the *Grid Options* form, enable the **Grid on**, **Rising Edge**, and **Grid Count with Start Number** options and click the **Apply**button at the bottom.
   c. Left-click in the *nWave* frame to move the start count on the signal that you want to count.
2. To jump to the 100th rising edge, for example:
3. Go back to the *Grid Options* form and click on the **Lock Grid Count** and **Jump Cursor to Grid Number**, type **100** in the text field and click the **Apply** button at the bottom.
4. From the same *Grid Options* form, disable the **Grid on** and **Rising Edge** options, then click the **Apply** button at the bottom to remove all grids.

33.7 Modify Existing Logical Operation Signals

1. In *nWave*, left-click a signal that you'd like to create a copy/modify.
2. In *nWave*, invoke **Signal -> Logical Operation**, and then select an existing logical operation signal in the *Logical Operation* form.
3. Modify the content in the **Expression** field, for example, if you have selected an 8 bit register, perform a logical right two bit shift operation like this: **"top/reg1[7:0]" >> 2** .
4. Click the **Create/Modify** button.

33.8 Collapse Source Code

1. In the main window, enable all three options in the **Automatic Source Code Folding** field in **Tools -> Preferences -> Source Code** page **-> Code Folding** page.
2. In the *Source Code* pane, you see process and while blocks with a [-] and [+] next to the line number. Click on the minus [-] icon to collapse code.
3. If you want to expand all the collapsed code, invoke **View -> Source Code Folder -> Expand All in Design** in the main window.
4. To collapse all the code in a specific module, invoke **View -> Source Code Folder -> Collect All in Design** in the main window.